

ABAQUS 2016

SCRIPTING REFERENCE GUIDE



3DEXPERIENCE®

Abaqus Scripting Reference Guide

Legal Notices

Abaqus, the 3DS logo, and SIMULIA are commercial trademarks or registered trademarks of Dassault Systèmes or its subsidiaries in the United States and/or other countries. Use of any Dassault Systèmes or its subsidiaries trademarks is subject to their express written approval.

Abaqus and this documentation may be used or reproduced only in accordance with the terms of the software license agreement signed by the customer, or, absent such an agreement, the then current software license agreement to which the documentation relates.

This documentation and the software described in this documentation are subject to change without prior notice.

Dassault Systèmes and its subsidiaries shall not be responsible for the consequences of any errors or omissions that may appear in this documentation.

© Dassault Systèmes, 2015

Other company, product, and service names may be trademarks or service marks of their respective owners. For additional information concerning trademarks, copyrights, and licenses, see the Legal Notices in the Abaqus 2016 Installation and Licensing Guide.

Preface

This section lists various resources that are available for help with using Abaqus Unified FEA software.

Support

Both technical software support (for problems with creating a model or performing an analysis) and systems support (for installation, licensing, and hardware-related problems) for Abaqus are offered through a global network of support offices, as well as through our online support system. Contact information for our regional offices is accessible from **SIMULIA→Locations** at www.3ds.com/simulia. The online support system is accessible by selecting the **SUBMIT A REQUEST** link at **Support - Dassault Systèmes** (<http://www.3ds.com/support>).

Online support

Dassault Systèmes provides a knowledge base of questions and answers, solutions to questions that we have answered, and guidelines on how to use Abaqus, Engineering Process Composer, Isight, Tosca, fe-safe, and other SIMULIA products. The knowledge base is available by using the **Search our Knowledge** option on www.3ds.com/support (<http://www.3ds.com/support>).

By using the online support system, you can also submit new requests for support. All support/service requests are tracked. If you contact us by means outside the system to discuss an existing support problem and you know the support request number, please mention it so that we can query the support system to see what the latest action has been.

Training

All SIMULIA regional offices offer regularly scheduled public training classes. The courses are offered in a traditional classroom form and via the Web. We also provide training seminars at customer sites. All training classes and seminars include workshops to provide as much practical experience with Abaqus as possible. For a schedule and descriptions of available classes, see the **Training** link at www.3ds.com/products-services/simulia (www.3ds.com/products-services/simulia) or call your support office.

Feedback

We welcome any suggestions for improvements to Abaqus software, the support tool, or documentation. We will ensure that any enhancement requests you make are considered for future releases. If you wish to make a suggestion about the service or products, refer to www.3ds.com/simulia. Complaints should be made by contacting your support office or by visiting **SIMULIA→Quality Assurance** at www.3ds.com/simulia (www.3ds.com/simulia).

Part I: Python commands

1. Abaqus/CAE Display Preferences commands

1.1 caePrefsAccess module

The Python module **caePrefsAccess** contains functions that enable you to edit the Abaqus/CAE preferences file, **abaqus_2016.gpr**.

Access

```
import caePrefsAccess
```

1.1.1 getGuiPrefsFileName(...)

This function enables you to retrieve the location of your **abaqus_2016.gpr** file.

Path

```
caePrefsAccess.getGuiPrefsFileName
```

Arguments

None.

Return value

A String with the default file name for the GUI preferences file.

1.1.2 getDisplayNamesInGuiPreferences(...)

The **abaqus_2016.gpr** file stores a separate **guiPreferences** record for each display that you use. This function returns a list of every **displayName** recorded in the preferences file.

Path

```
caePrefsAccess.getDisplayNamesInGuiPreferences
```

Required argument

fileName

A String that specifies the path to the preferences file.

Optional arguments

None.

Return value

A list of Strings of displayNames.

Exceptions

None.

1.1.3 printValuesList(...)

This function enables you to print all of the options and their values for a set of **guiPreferences** or **sessionOptions** settings derived from the **abaqus_2016.gpr** file.

Path

caePrefsAccess.printValuesList

Required argument

object

The **guiPreferences** object or **sessionOptions** object for which you want to print options and their values.

Optional arguments

maxRecursionDepth

An Int, or SymbolicConstant UNLIMITED, that specifies the depth of recursion when accessing the attributes of *object*.

asString

A Boolean specifying how the string representation of each option is printed. If *asString* is True, **printValuesList** prints the **str** of each option; otherwise **printValuesList** prints the **repr** of the options. The default value is False.

Return value

A String displaying the path, name, and value for all of the options in the object that you select.

Exceptions

None.

1.1.4 openGuiPreferences(...)

This function enables you to examine and change many default behaviors in the Abaqus/CAE graphical user interface. Abaqus stores preferences for each display you use in a separate **guiPreferences** section of the **abaqus_2016.gpr** file.

Path

caePrefsAccess.openGuiPreferences

Required argument

displayName

A String that specifies the display for which you want to investigate GUI preferences from the **abaqus_2016.gpr** file. You can retrieve the available display names in the file by using the **getDisplayNamesInGuiPreferences** method.

Optional argument

fileName

A String specifying the path to the preferences file. The **openGuiPreferences** method uses this argument if you are working with a preferences file that is not at the default location.

If this argument is omitted, the **abaqus_2016.gpr** file in your home directory is opened.

Return value

A CaeGuiPrefs object.

Exceptions

None.

1.1.5 openSessionOptions(...)

This function enables you to examine and change the default behavior for many session options Abaqus/CAE; that is, the settings that you can save in Abaqus/CAE from the **File→Save Display Options** menu option. Abaqus stores default session options in the **sessionOptions** section of the **abaqus_2016.gpr** file.

Path

caePrefsAccess.openSessionOptions

Required arguments

None.

Optional arguments*fileName*

A String specifying the path to the preferences file. The **openSessionOptions** method uses this argument if you are working with a preferences file that is not at the default location.

If this argument is omitted, the **abaqus_2016.gpr** file in your home directory is opened.

directory

A SymbolicConstant specifying the location of the preferences file. Possible values are:

- CURRENT to open the preferences file in the current directory (**caePrefsAccess.CURRENT**)
- HOME to open the preferences file in your home directory (**caePrefsAccess.HOME**)

The default value is HOME. Either *fileName* or *directory* must be supplied. The *fileName* or *directory* arguments are mutually exclusive.

Return value

A CaeKerPrefs object.

Exceptions

None.

1.2 CaeGuiPrefs object

The CaeGuiPrefs object contains the details of the graphical preferences in a **guiPreferences** section of the **abaqus_2016.gpr** file.

Access

```
import caePrefsAccess
caePrefsAccess.openGuiPreferences(...)
```

1.2.1 save(...)

This method saves the **guiPreferences** section specified in the current *fileName*.

Arguments**Required arguments**

None.

Optional argument*backupFile*

A Boolean specifying whether Abaqus should save a numbered backup copy of the preferences file, *fileName*. Default is True.

Return value

None

Exceptions

None.

1.2.2 saveAs(...)

This method saves the **guiPreferences** settings to the specified location.

Arguments**Required argument***fileName*

A String specifying the path to the preferences file.

Optional arguments

None.

Return value

None

Exceptions

None.

1.2.3 Members

The CaeGuiPrefs object has the following member:

fileName

A String specifying the path to the preferences file.

1.3 CaeKerPrefs object

The CaeKerPrefs object contains the details of the **sessionOptions**.

Access

```
import caePrefsAccess
caePrefsAccess.openSessionOptions(...)
```

1.3.1 save(...)

This method saves the **sessionOptions** in the current *fileName*.

Arguments

Required arguments

None.

Optional argument

backupFile

A Boolean specifying whether save a numbered backup copy of the preferences file, *fileName*.
Default is True.

Return value

None

Exceptions

None.

1.3.2 saveAs(...)

This method saves the **sessionOptions** to the specified location.

Arguments

Required arguments

fileName

A String specifying the path to the preferences file.

directory

A SymbolicConstant specifying the location of the preferences file. Possible values are:

- CURRENT to open the preferences file in the current directory (**caePrefsAccess.CURRENT**)
- HOME to open the preferences file in your home directory (**caePrefsAccess.HOME**)

The default value is HOME. Either *fileName* or *directory* must be supplied. The *fileName* or *directory* arguments are mutually exclusive.

Optional arguments

None.

Return value

None

Exceptions

None.

1.3.3 Members

The CaeKerPrefs object has the following member:

fileName

A String specifying the path to the preferences file that the CaeKerPrefs object represents.

2. **Adaptivity commands**

The Adaptivity commands are used to define objects, perform analyses, and calculate new meshes for the following:

- Arbitrary Lagrangian Eularian (ALE) adaptive smoothing (adaptive meshing)
- Varying topology adaptivity (adaptive remeshing)

2.1 **AdaptiveMeshConstraint object**

The AdaptiveMeshConstraint object is the abstract base type for other Arbitrary Lagrangian Eularian (ALE) style AdaptiveMeshConstraint objects. The AdaptiveMeshConstraint object has no explicit constructor. The methods and members of the AdaptiveMeshConstraint object are common to all objects derived from the AdaptiveMeshConstraint object.

Access

```
import step
mdb.models[name].adaptiveMeshConstraints[name]
```

2.1.1 **deactivate(...)**

This method deactivates the adaptive mesh constraint in the specified step and all subsequent steps.

Arguments

Required argument

stepName

A String specifying the name of the step in which the adaptive mesh constraint is deactivated.

Optional arguments

None.

Return value

None

Exceptions

TextError.

2.1.2 **move(...)**

This method moves the adaptive mesh constraint state from one step to a different step.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the adaptive mesh constraint state is moved.

toStepName

A String specifying the name of the step to which the adaptive mesh constraint state is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

2.1.3 **reset(...)**

This method resets the adaptive mesh constraint state of the specified step to the state of the previous analysis step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the adaptive mesh constraint state is reset.

Optional arguments

None.

Return value

None

Exceptions

TextError.

2.1.4 **resume()**

This method resumes the adaptive mesh constraint that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

2.1.5 **suppress()**

This method suppresses the adaptive mesh constraint.

Arguments

None.

Return value

None

Exceptions

None.

2.1.6 **delete(...)**

This method allows you to delete existing adaptive mesh constraints.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each adaptive mesh constraint to delete.

Optional arguments

None.

Return value**None****Exceptions**

None.

2.1.7 Members

The AdaptiveMeshConstraint object can have the following members:

name

A String specifying the adaptive mesh constraint repository key.

category

A SymbolicConstant specifying the category of the adaptive mesh constraint. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the adaptive mesh constraint is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the adaptive mesh constraint's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

2.2 AdaptiveMeshConstraintState object

The AdaptiveMeshConstraintState object is the abstract base type for other Arbitrary Lagrangian Eularian (ALE) style AdaptiveMeshConstraintState objects. The AdaptiveMeshConstraintState object has no explicit constructor or methods. The members of the AdaptiveMeshConstraintState object are common to all objects derived from the AdaptiveMeshConstraintState object.

Access**import step****mdb.models [name] .steps [name] .adaptiveMeshConstraintStates [name]****2.2.1 Members**

The AdaptiveMeshConstraintState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the AdaptiveMeshConstraintState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the adaptive mesh constraint has no amplitude reference.

2.3 AdaptiveMeshControl object

The AdaptiveMeshControl object is used to control various aspects of Arbitrary Lagrangian Eularian (ALE) style adaptive smoothing and advection algorithms applied to an ALE adaptive mesh domain.

Access

```
import step
mdb.models[name].adaptiveMeshControls[name]
```

2.3.1 AdaptiveMeshControl(...)

This method creates an AdaptiveMeshControl object.

Path

```
mdb.models[name].AdaptiveMeshControl
```

Required argument*name*

A String specifying the name of the object.

Optional arguments*remapping*

A SymbolicConstant specifying the remapping algorithm. Possible values are FIRST_ORDER_ADVECTION and SECOND_ORDER_ADVECTION. The default value is SECOND_ORDER_ADVECTION.

smoothingAlgorithm

A SymbolicConstant specifying the type of smoothing algorithm to use. Possible values are STANDARD and GEOMETRY_ENHANCED. The default value is GEOMETRY_ENHANCED.

smoothingPriority

A SymbolicConstant specifying the type of smoothing to perform. Possible values are UNIFORM and GRADED. The default value is UNIFORM.

initialFeatureAngle

A Float specifying the initial geometric feature angle, θ_I , in degrees. Possible values are $0^\circ \leq \theta_I \leq 180^\circ$. The default value is 30.0.

transitionFeatureAngle

A Float specifying the transitional feature angle, θ_T , in degrees. Possible values are $0^\circ \leq \theta_T \leq 180^\circ$. The default value is 30.0.

momentumAdvection

A SymbolicConstant specifying the type of momentum advection algorithm. Possible values are ELEMENT_CENTER_PROJECTION and HALF_INDEX_SHIFT. The default value is ELEMENT_CENTER_PROJECTION.

meshingPredictor

A SymbolicConstant specifying the nodal starting location to use for remeshing. Possible values are CURRENT and PREVIOUS. The default value is CURRENT.

curvatureRefinement

A Float specifying the solution dependence weight, α_C . Possible values are $0.0 \leq \alpha_C \leq 1.0$. The default value is 1.0.

volumetricSmoothingWeight

A Float specifying the weight used by Abaqus/Explicit for the volumetric smoothing method. The default value is 1.0.

laplacianSmoothingWeight

A Float specifying the weight for the Laplacian smoothing method. The default value is 0.0.

equipotentialSmoothingWeight

A Float specifying the weight for the equipotential smoothing method. The default value is 0.0.

meshConstraintAngle

A Float specifying the initial geometric feature angle, θ_C . Possible values are $0^\circ \leq \theta_C \leq 180^\circ$. The default value is 60.0.

originalConfigurationProjectionWeight

A Float specifying the weight for the original configuration projection method. The default value is 1.0.

standardVolumetricSmoothingWeight

A Float specifying the weight used by Abaqus/Standard for the volumetric smoothing method. The default value is 0.0.

Return value

An AdaptiveMeshControl object.

Exceptions

RangeError.

2.3.2 setValues(...)

This method modifies the AdaptiveMeshControl object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AdaptiveMeshControl** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

2.3.3 Members

The AdaptiveMeshControl object has members with the same names and descriptions as the arguments to the **AdaptiveMeshControl** method.

2.4 AdaptiveMeshDomain object

The AdaptiveMeshDomain object defines the region and controls that govern an Arbitrary Lagrangian Eularian (ALE) style adaptive smoothing mesh domain.

Access

```
import step
mdb.models[name].steps[name].adaptiveMeshDomains[name]
```

2.4.1 AdaptiveMeshDomain(...)

This method creates an AdaptiveMeshDomain object.

Path

```
mdb.models[name].steps[name].AdaptiveMeshDomain
```

Required argument

region

A Region object specifying the region to which the adaptive mesh domain is applied.

Optional arguments

controls

A String specifying the name of an AdaptiveMeshControl object.

frequency

An Int specifying the frequency in increments at which adaptive meshing will be performed. The default value is 10.

initialMeshSweeps

An Int specifying the number of mesh sweeps to be performed at the beginning of the first step in which this adaptive mesh definition is active. The default value is 5.

meshSweeps

An Int specifying the number of mesh sweeps to be performed in each adaptive mesh increment. The default value is 1.

Return value

An AdaptiveMeshDomain object.

Exceptions

RangeError.

2.4.2 setValues(...)

This method modifies the AdaptiveMeshDomain object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AdaptiveMeshDomain** method.

Return value

None

Exceptions

RangeError.

2.4.3 Members

The AdaptiveMeshDomain object has members with the same names and descriptions as the arguments to the **AdaptiveMeshDomain** method.

2.5 AdaptivityIteration object

The AdaptivityIteration object contains information about a given iteration of the varying topology adaptivity process (adaptive remeshing).

Access

```
import job
mdb.adaptivityProcesses[name].iterations[i]
```

2.5.1 AdaptivityIteration(...)

This method creates an AdaptivityIteration object.

Path

`mdb.adaptivityProcesses[name].AdaptivityIteration`

Required arguments

iteration

An Int specifying the sequence number for this iteration in the adaptivity process.

jobName

A String specifying the name of the job that was run for this iteration.

modelName

A String specifying the name of the model that was analyzed and remeshed in this iteration.

odbPath

A String specifying the path to the ODB file that was created for this iteration.

remeshingErrors

An Int specifying the number of part instances which generated errors while remeshing the model in this iteration of the adaptivity process.

Optional arguments

None.

Return value

An AdaptivityIteration object.

Exceptions

None.

2.5.2 Members

The AdaptivityIteration object has members with the same names and descriptions as the arguments to the **AdaptivityIteration** method.

In addition, the AdaptivityIteration object can have the following member:

ruleResults

A repository of RuleResult objects specifying the calculated results from sizing functions corresponding to the RemeshingRule objects for this iteration of an adaptivity process.

2.6 AdaptivityProcess object

The AdaptivityProcess object defines a series of jobs that will be submitted for analysis. Abaqus performs adaptive remeshing between each job.

Access

```
import job
mdb.adaptivityProcesses [name]
```

2.6.1 AdaptivityProcess(...)

This method creates an AdaptivityProcess object.

Path

```
mdb.AdaptivityProcess
```

Required arguments

name

A String specifying the name of the Adaptivity Process.

job

A ModelJob object specifying a job to be used as the prototype for all analysis jobs run by the adaptivity process.

Optional arguments

maxIterations

An Int specifying the maximum number of analysis jobs that will be run by the Adaptivity Process. Abaqus performs adaptive remeshing between each analysis. The default value is 3.

jobPrefix

A String specifying the prefix to use for jobs created by the adaptivity process. An empty string indicates that the name of the adaptivity process should be used. The default value is an empty string.

Return value

An AdaptivityProcess object.

Exceptions

AbaqusException.

2.6.2 submit(...)

This method begins the process of analysis and adaptive remeshing.

Arguments

Required arguments

None.

Optional arguments

waitForCompletion

A Boolean specifying whether to interrupt the execution of a script until the end of the adaptive remeshing process is reached.

datacheckJob

A Boolean specifying whether to run the adaptivity as a datacheck analysis. The default value is False. The *datacheckJob* and *continueJob* arguments cannot both be True.

continueJob

A Boolean specifying whether to run the adaptivity as a continuation analysis. The default value is False. The *datacheckJob* and *continueJob* arguments cannot both be True.

Return value

None

Exceptions

None.

2.6.3 **setValues(...)**

This method modifies the `AdaptivityProcess` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AdaptivityProcess** method, except for the *name* argument.

Return value

None

Exceptions

2.6.4 Members

The `AdaptivityProcess` object has members with the same names and descriptions as the arguments to the **`AdaptivityProcess`** method.

In addition, the `AdaptivityProcess` object can have the following members:

status

A `SymbolicConstant` specifying the status of the adaptivity process. Possible values are `SUBMITTED`, `RUNNING`, `ABORTED`, `TERMINATED`, and `COMPLETED`.

iterations

A repository of `AdaptivityIteration` objects specifying the `AdaptivityIteration` objects received during running the adaptivity process.

2.7 DisplacementAdaptiveMeshConstraint object

The `DisplacementAdaptiveMeshConstraint` object stores the data for an Arbitrary Lagrangian Eularian (ALE) style displacement/rotation adaptive mesh constraint.

The `DisplacementAdaptiveMeshConstraint` object is derived from the `AdaptiveMeshConstraint` object.

Access

```
import step
mdb.models[name].adaptiveMeshConstraints[name]
```

2.7.1 DisplacementAdaptiveMeshConstraint(...)

This method creates a `DisplacementAdaptiveMeshConstraint` object.

Path

```
mdb.models[name].DisplacementAdaptiveMeshConstraint
```

Required arguments

name

A String specifying the adaptive mesh constraint repository key.

createStepName

A String specifying the name of the step in which the adaptive mesh constraint is created.

region

A Region object specifying the region to which the adaptive mesh constraint is applied.

Optional arguments

u1

A Float or a SymbolicConstant specifying the displacement component in the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *u1*, *u2*, *u3*, *ur1*, *ur2*, and *ur3* are optional arguments, at least one of them must be specified.

u2

A Float or a SymbolicConstant specifying the displacement component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

u3

A Float or a SymbolicConstant specifying the displacement component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur1

A Float or a SymbolicConstant specifying the rotational displacement component about the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur2

A Float or a SymbolicConstant specifying the rotational displacement component about the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur3

A Float or a SymbolicConstant specifying the rotational displacement component about the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the adaptive mesh constraint has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

motionType

A SymbolicConstant specifying the mesh motion in relation to the underlying material. Possible values are INDEPENDENT, FOLLOW and USER_DEFINED. The default value is INDEPENDENT.

localCsys

None or a `DatumCsys` object specifying the local coordinate system of the adaptive mesh constraint's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A `DisplacementAdaptiveMeshConstraint` object.

Exceptions

None.

2.7.2 **setValues(...)**

This method modifies the data for an existing `DisplacementAdaptiveMeshConstraint` object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DisplacementAdaptiveMeshConstraint** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

2.7.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing `DisplacementAdaptiveMeshConstraint` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the adaptive mesh constraint is modified.

Optional arguments

u1

A Float or a SymbolicConstant specifying the displacement component in the 1-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

u2

A Float or a SymbolicConstant specifying the displacement component in the 2-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

u3

A Float or a SymbolicConstant specifying the displacement component in the 3-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur1

A Float or a SymbolicConstant specifying the rotational displacement component about the 1-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur2

A Float or a SymbolicConstant specifying the rotational displacement component about the 2-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur3

A Float or a SymbolicConstant specifying the rotational displacement component about the 3-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the adaptive mesh constraint is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

2.7.4 Members

The DisplacementAdaptiveMeshConstraint object can have the following members:

name

A String specifying the adaptive mesh constraint repository key.

category

A SymbolicConstant specifying the category of the adaptive mesh constraint. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the adaptive mesh constraint is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the adaptive mesh constraint's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

2.8 DisplacementAdaptiveMeshConstraintState object

The DisplacementAdaptiveMeshConstraintState object stores the propagating data for an Arbitrary Lagrangian Eularian (ALE) style displacement/rotation adaptive mesh constraint in a step. One instance of this object is created internally by the DisplacementAdaptiveMeshConstraint object for each step. The instance is also deleted internally by the DisplacementAdaptiveMeshConstraint object.

The DisplacementAdaptiveMeshConstraintState object has no constructor or methods.

The DisplacementAdaptiveMeshConstraintState object is derived from the AdaptiveMeshConstraintState object.

Access

```
import step
mdb.models[name].steps[name].adaptiveMeshConstraintStates[name]
```

2.8.1 Members

The DisplacementAdaptiveMeshConstraintState object has the following members:

u1

A Float or a Complex specifying the displacement component in the 1-direction.

u2

A Float or a Complex specifying the displacement component in the 2-direction.

u3

A Float or a Complex specifying the displacement component in the 3-direction.

ur1

A Float or a Complex specifying the rotational displacement component about the 1-direction.

ur2

A Float or a Complex specifying the rotational displacement component about the 2-direction.

ADAPTIVITY COMMANDS

ur3

A Float or a Complex specifying the rotational displacement component about the 3-direction.

u1State

A SymbolicConstant specifying the propagation state of the displacement component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u2State

A SymbolicConstant specifying the propagation state of the displacement component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u3State

A SymbolicConstant specifying the propagation state of the displacement component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur1State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur2State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur3State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the AdaptiveMeshConstraintState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE

- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the adaptive mesh constraint has no amplitude reference.

2.8.2 Corresponding analysis keywords

*ADAPTIVE MESH CONSTRAINT, TYPE=DISPLACEMENT (degree of freedom: 1, 2, 3, 4, 5, or 6)

2.9 ErrorIndicatorResult object

The ErrorIndicatorResult object contains result information corresponding to an error indicator variable in a RemeshingRule object for an adaptivity iteration.

Access

```
import job
mdb.adaptivityProcesses[name].iterations[i].ruleResults[name] \
.indicatorResults[name]
```

2.9.1 ErrorIndicatorResult(...)

This method creates an ErrorIndicatorResult with data for an error indicator variable in a RemeshingRule for a given adaptivity iteration.

Path

```
mdb.adaptivityProcesses[name].iterations[i].ruleResults[name] \
.ErrorIndicatorResult
```

Required arguments

name

A String specifying the name of the error indicator variable to which these results correspond.

results

A String-to-Float Dictionary specifying the calculated results from the sizing function corresponding to the error indicator variable represented by this `ErrorIndicatorResult`.

Optional arguments

None.

Return value

An `ErrorIndicatorResult` object.

Exceptions

`AbaqusException`.

2.9.2 Members

The `ErrorIndicatorResult` object has members with the same names and descriptions as the arguments to the `ErrorIndicatorResult` method.

2.10 Model object

The following commands operate on Model objects. For more information about the Model object, see “Model object,” Section 33.1.

Access

```
import mesh
```

2.10.1 adaptiveRemesh(...)

This method remeshes the model using the active remesh rules in the model and the error indicator results from a previous analysis.

Arguments

Required argument

odb

An Odb object containing error output field results.

Optional arguments

None.

Return value

An AdaptivityIteration Object.

Exceptions

None.

2.11 RemeshingRule object

The RemeshingRule object controls the adaptive remeshing resizing and the error indicators written to the output database for a specified region of the model.

Access

```
import mesh
mdb.models[name].remeshingRules[name]
```

2.11.1 RemeshingRule(...)

This method creates a RemeshingRule object.

Path

```
mdb.models[name].RemeshingRule
```

Required arguments

name

A String specifying the name of the object.

stepName

A String specifying the name of the step in which resizing should occur for this rule.

variables

A sequence of Strings specifying the output request variables that Abaqus will use as error indicators.

Optional arguments

description

A String specifying a descriptive string for this rule. The default value is an empty string.

region

The SymbolicConstant MODEL or a Region object specifying the region in which Abaqus will remesh and generate output. The SymbolicConstant MODEL represents the entire applicable model. The default value is MODEL.

sizingMethod

A SymbolicConstant specifying the method for calculating the new mesh sizes. The SymbolicConstant DEFAULT indicates that Abaqus will use the default calculation method for each individual variable. Possible values are DEFAULT, UNIFORM_ERROR, and MINIMUM_MAXIMUM. The default value is DEFAULT.

errorTarget

A Float specifying the target error percentage for each variable in the region. A value of 0.0 indicates that Abaqus will use automated error target reduction for the region. You use the *errorTarget* argument when *sizingMethod*=UNIFORM_ERROR. The default value is 0.0.

maxSolutionErrorTarget

A Float specifying the target error percentage at the location of the maximum solution value in the region. A value of 0.0 indicates that Abaqus will use automated error target reduction for the region. You use the *maxSolutionErrorTarget* argument when *sizingMethod*=MINIMUM_MAXIMUM. The default value is 0.0.

minSolutionErrorTarget

A Float specifying the target error percentage at the location of the minimum solution value in the region. A value of 0.0 indicates that Abaqus will use automated error target reduction for the region. You use the *minSolutionErrorTarget* argument when *sizingMethod*=MINIMUM_MAXIMUM. The default value is 0.0.

meshBias

An Int specifying an indication of how much Abaqus will bias the mesh toward the location of the maximum solution value in the region. The higher the value, the more the mesh will bias towards the location of the maximum solution value. You use the *meshBias* argument when *sizingMethod*=MINIMUM_MAXIMUM. The default value is 0.0.

minElementSize

A Float specifying the minimum size of any single element. The default value is 0.0.

maxElementSize

A Float specifying the maximum size of any single element. The default value is 0.0.

outputFrequency

A SymbolicConstant specifying the frequency with which the error indicators are saved to the output database file (.odb). Possible values are LAST_INCREMENT and ALL_INCREMENTS. The default value is LAST_INCREMENT.

specifyMinSize

A Boolean specifying an indication of whether to use a user-supplied minimum element size or to calculate a characteristic minimum element size. The default value is OFF.

specifyMaxSize

A Boolean specifying an indication of whether to use a user-supplied maximum element size or to calculate a characteristic maximum element size. The default value is ON.

coarseningFactor

A SymbolicConstant or an Int specifying an indication of the upper limit on the element growth from one remeshing iteration to the next. Possible values are DEFAULT_LIMIT and NOT_ALLOWED. The default value is DEFAULT_LIMIT.

refinementFactor

A SymbolicConstant or an Int specifying an indication of the upper limit on element shrinkage from one remeshing iteration to the next. Possible values are DEFAULT_LIMIT and NOT_ALLOWED. The default value is DEFAULT_LIMIT.

elementCountLimit

None or an Int specifying an approximate limit on the number of elements that will be created during remeshing. Use **None** to indicate there is not upper limit. The default value is **None**.

Return value

A RemeshingRule object.

Exceptions

AbaqusException.

2.11.2 resume()

This method resumes the remeshing rule that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

2.11.3 suppress()

This method suppresses the remeshing rule. Abaqus will not remesh regions where the rules are suppressed.

Arguments

None.

Return value

None

Exceptions

None.

2.11.4 **setValues(...)**

This method modifies the RemeshingRule object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RemeshingRule** method, except for the *name* argument.

Return value

None

Exceptions

2.11.5 **Members**

The RemeshingRule object has members with the same names and descriptions as the arguments to the **RemeshingRule** method.

In addition, the RemeshingRule object has the following member:

suppressed

A Boolean specifying whether the remeshing rule is suppressed. Remeshing of the remeshing rule's region will not occur if you suppress a rule. The default value is OFF.

2.12 RuleResult object

The RuleResult object contains result information corresponding to a RemeshingRule object for an adaptivity iteration.

Access

```
import job
mdb.adaptivityProcesses[name].iterations[i].ruleResults[name]
```

2.12.1 RuleResult(...)

This method creates a RuleResult with data for a RemeshingRule for a given adaptivity iteration.

Path

```
mdb.adaptivityProcesses[name].iterations[i].RuleResult
```

Required arguments

name

A String specifying the name of the Remeshing Rule to which these results correspond.

indicatorResults

A repository of ErrorIndicatorResult objects specifying the calculated results from the sizing function corresponding to the error indicator variables for the Remeshing Rule.

numElems

An Int specifying the number of elements before remeshing in the region of the Remeshing Rule.

minSizeElemCount

An Int specifying the number of elements that were constrained to the minimum element size by the Remeshing Rule.

Optional argument

satisfiedVars

A sequence of Strings specifying the error indicator variables that have satisfied the Remeshing Rule.

Return value

A RuleResult object.

Exceptions

AbaqusException.

2.12.2 Members

The `RuleResult` object has members with the same names and descriptions as the arguments to the `RuleResult` method.

2.13 VelocityAdaptiveMeshConstraint object

The `VelocityAdaptiveMeshConstraint` object stores the data for an Arbitrary Lagrangian Eularian (ALE) style velocity adaptive mesh constraint.

The `VelocityAdaptiveMeshConstraint` object is derived from the `AdaptiveMeshConstraint` object.

Access

```
import step
mdb.models[name].adaptiveMeshConstraints[name]
```

2.13.1 VelocityAdaptiveMeshConstraint(...)

This method creates a `VelocityAdaptiveMeshConstraint` object.

Path

```
mdb.models[name].VelocityAdaptiveMeshConstraint
```

Required arguments

name

A String specifying the adaptive mesh constraint repository key.

createStepName

A String specifying the name of the step in which the adaptive mesh constraint is created.

region

A Region object specifying the region to which the adaptive mesh constraint is applied.

Optional arguments

v1

A Float or a `SymbolicConstant` specifying the velocity component in the 1-direction. Possible values for the `SymbolicConstant` are `UNSET` and `SET`. The default value is `UNSET`.

Note: Although *v1*, *v2*, *v3*, *vr1*, *vr2*, and *vr3* are optional arguments, at least one of them must be specified.

v2

A Float or a SymbolicConstant specifying the velocity component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

v3

A Float or a SymbolicConstant specifying the velocity component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr1

A Float or a SymbolicConstant specifying the rotational velocity component about the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component about the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr3

A Float or a SymbolicConstant specifying the rotational velocity component about the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the adaptive mesh constraint has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

localCsys

None or a DatumCsys object specifying the local coordinate system of the adaptive mesh constraint's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

motionType

A SymbolicConstant specifying the mesh motion in relation to the underlying material. Possible values are INDEPENDENT, FOLLOW and USER_DEFINED. The default value is INDEPENDENT.

Return value

A VelocityAdaptiveMeshConstraint object.

Exceptions

None.

2.13.2 **setValues(...)**

This method modifies the data for an existing VelocityAdaptiveMeshConstraint object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VelocityAdaptiveMeshConstraint** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

2.13.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing **VelocityAdaptiveMeshConstraint** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the adaptive mesh constraint is modified.

Optional arguments

v1

A Float or a **SymbolicConstant** specifying the velocity component in the 1-direction. Possible values for the **SymbolicConstant** are SET and FREED.

v2

A Float or a **SymbolicConstant** specifying the velocity component in the 2-direction. Possible values for the **SymbolicConstant** are SET and FREED.

v3

A Float or a **SymbolicConstant** specifying the velocity component in the 3-direction. Possible values for the **SymbolicConstant** are SET and FREED.

vr1

A Float or a **SymbolicConstant** specifying the rotational velocity component about the 1-direction. Possible values for the **SymbolicConstant** are SET and FREED.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component about the 2-direction. Possible values for the SymbolicConstant are SET and FREED.

vr3

A Float or a SymbolicConstant specifying the rotational velocity component about the 3-direction. Possible values for the SymbolicConstant are SET and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the adaptive mesh constraint is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

2.13.4 Members

The VelocityAdaptiveMeshConstraint object can have the following members:

name

A String specifying the adaptive mesh constraint repository key.

category

A SymbolicConstant specifying the category of the adaptive mesh constraint. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the adaptive mesh constraint is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the adaptive mesh constraint's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

2.14 VelocityAdaptiveMeshConstraintState object

The VelocityAdaptiveMeshConstraintState object stores the propagating data for an Arbitrary Lagrangian Eularian (ALE) style velocity adaptive mesh constraint in a step. One instance of this object

is created internally by the `VelocityAdaptiveMeshConstraint` object for each step. The instance is also deleted internally by the `VelocityAdaptiveMeshConstraint` object.

The `VelocityAdaptiveMeshConstraintState` object has no constructor or methods.

The `VelocityAdaptiveMeshConstraintState` object is derived from the `AdaptiveMeshConstraintState` object.

Access

```
import step
mdb.models[name].steps[name].adaptiveMeshConstraintStates[name]
```

2.14.1 Members

The `VelocityAdaptiveMeshConstraintState` object has the following members:

v1

A Float specifying the velocity component in the 1-direction.

v2

A Float specifying the velocity component in the 2-direction.

v3

A Float specifying the velocity component in the 3-direction.

vr1

A Float specifying the rotational velocity component about the 1-direction.

vr2

A Float specifying the rotational velocity component about the 2-direction.

vr3

A Float specifying the rotational velocity component about the 3-direction.

v1State

A `SymbolicConstant` specifying the propagation state of the velocity component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v2State

A `SymbolicConstant` specifying the propagation state of the velocity component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v3State

A `SymbolicConstant` specifying the propagation state of the velocity component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr1State

A `SymbolicConstant` specifying the propagation state of the rotational velocity component about the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr2State

A SymbolicConstant specifying the propagation state of the rotational velocity component about the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr3State

A SymbolicConstant specifying the propagation state of the rotational velocity component about the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the AdaptiveMeshConstraintState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the adaptive mesh constraint has no amplitude reference.

2.14.2 Corresponding analysis keywords

*ADAPTIVE MESH CONSTRAINT, TYPE=VELOCITY (degree of freedom: 1, 2, 3, 4, 5, or 6)

3. Amplitude commands

Amplitude commands are used to create arbitrary time or frequency variations of load, displacement, and some interaction attributes throughout a step using step time or throughout an analysis using total time.

3.1 Amplitude object

The Amplitude object is the abstract base type for other Amplitude objects. The Amplitude object has no explicit constructor. The methods and members of the Amplitude object are common to all objects derived from the Amplitude.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.1.1 Members

The Amplitude object has the following members:

name

A String specifying the repository key.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

3.2 ActuatorAmplitude object

The ActuatorAmplitude object defines an actuator amplitude curve.

The ActuatorAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.2.1 **ActuatorAmplitude(...)**

This method creates a `ActuatorAmplitude` object.

Path

```
mdb.models[name].ActuatorAmplitude  
session.odbs[name].ActuatorAmplitude
```

Required argument

name

A String specifying the repository key.

Optional argument

timeSpan

A `SymbolicConstant` specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

An `ActuatorAmplitude` object.

Exceptions

`InvalidNameError` and `RangeError`.

3.2.2 **setValues(...)**

This method modifies the `ActuatorAmplitude` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ActuatorAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

`RangeError`.

3.2.3 Members

The `ActuatorAmplitude` object has members with the same names and descriptions as the arguments to the `ActuatorAmplitude` method.

3.2.4 Corresponding analysis keywords

*AMPLITUDE

3.3 BaselineCorrection object

The `BaselineCorrection` object modifies an acceleration history to minimize the overall drift of the displacement obtained from the time integration of the given acceleration.

Access

```
import amplitude
mdb.models[name].amplitudes[name].baselineCorrection
import odbAmplitude
session.odbs[name].amplitudes[name].baselineCorrection
```

3.3.1 BaselineCorrection(...)

This method creates a `BaselineCorrection` object.

Path

```
mdb.models[name].amplitudes[name].BaselineCorrection
session.odbs[name].amplitudes[name].BaselineCorrection
```

Required arguments

None.

Optional argument

intervals

A sequence of Floats specifying the correction time interval end points. Possible values are positive and monotonically increasing Floats. The default value is an empty sequence.

Return value

A `BaselineCorrection` object.

Exceptions

`RangeError`.

3.3.2 `setValues(...)`

This method modifies the `BaselineCorrection` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BaselineCorrection` method.

Return value

`None`

Exceptions

`RangeError`.

3.3.3 Members

The `BaselineCorrection` object has members with the same names and descriptions as the arguments to the `BaselineCorrection` method.

3.3.4 Corresponding analysis keywords

*BASELINE CORRECTION

3.4 Correlation object

A `Correlation` is an object used to define the cross-correlation as part of the definition of random loading.

The Correlation object is derived from the Amplitude object.

Access

```
import load
mdb.models[name].boundaryConditions[name].correlation[i]
```

3.4.1 Members

The Correlation object has the following members:

name

A String specifying the repository key.

approach

A SymbolicConstant specifying the approach used in the correlation data representation. Possible values are CORRELATED, MOVING_NOISE, UNCORRELATED, and USER. The default value is CORRELATED.

data

A tuple of tuples of Floats specifying the real and imaginary part of the scaling factor. If *approach*=MOVING_NOISE, then *data* represents the noise velocity components 1, 2, and 3.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

3.5 DecayAmplitude object

The DecayAmplitude object defines an amplitude curve using an exponential decay.

The DecayAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.5.1 DecayAmplitude(...)

This method creates a DecayAmplitude object.

Path

```
mdb.models[name].DecayAmplitude  
session.odbs[name].DecayAmplitude
```

Required arguments

name

A String specifying the repository key.

initial

A Float specifying the constant A_0 .

maximum

A Float specifying the coefficient A .

start

A Float specifying the starting time t_0 . Possible values are non-negative numbers.

decayTime

A Float specifying the decay time t_d . Possible values are non-negative numbers.

Optional argument

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A DecayAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.5.2 **setValues(...)**

This method modifies the DecayAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DecayAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.5.3 Members

The DecayAmplitude object has members with the same names and descriptions as the arguments to the `DecayAmplitude` method.

3.5.4 Corresponding analysis keywords

*AMPLITUDE

3.6 EquallySpacedAmplitude object

The EquallySpacedAmplitude object defines a list of amplitude values at fixed time intervals beginning at a specified value of time.

The EquallySpacedAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.6.1 EquallySpacedAmplitude(...)

This method creates an EquallySpacedAmplitude object.

Path

```
mdb.models[name].EquallySpacedAmplitude
session.odbs[name].EquallySpacedAmplitude
```

Required arguments

name

A String specifying the repository key.

fixedInterval

A Float specifying the fixed time interval at which the amplitude data are given. Possible values are positive numbers.

data

A sequence of Floats specifying the amplitude values.

Optional arguments

begin

A Float specifying the time at which the first amplitude data are given. Possible values are non-negative numbers. The default value is 0.0.

smooth

The SymbolicConstant SOLVER_DEFAULT or a Float specifying the degree of smoothing. Possible float values are $0 \leq \text{smoothing} \leq 0.5$. If *smooth*=SOLVER_DEFAULT, the default degree of smoothing will be determined by the solver. The default value is SOLVER_DEFAULT.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

An EquallySpacedAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.6.2 **setValues(...)**

This method modifies the EquallySpacedAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **EquallySpacedAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.6.3 Members

The `EquallySpacedAmplitude` object has members with the same names and descriptions as the arguments to the `EquallySpacedAmplitude` method.

In addition, the `EquallySpacedAmplitude` object can have the following member:

baselineCorrection

A `BaselineCorrection` object.

3.6.4 Corresponding analysis keywords

*AMPLITUDE

3.7 ModulatedAmplitude object

The `ModulatedAmplitude` object defines a modulated amplitude curve.

The `ModulatedAmplitude` object is derived from the `Amplitude` object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.7.1 ModulatedAmplitude(...)

This method creates a `ModulatedAmplitude` object.

Path

```
mdb.models[name].ModulatedAmplitude
```

`session.odbs[name].ModulatedAmplitude`

Required arguments

name

A String specifying the repository key.

initial

A Float specifying the constant A_0 .

magnitude

A Float specifying the coefficient A .

start

A Float specifying the starting time t_0 . Possible values are non-negative numbers.

frequency1

A Float specifying the circular frequency 1 (ω_1). Possible values are positive numbers.

frequency2

A Float specifying the circular frequency 2 (ω_2). Possible values are positive numbers.

Optional argument

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A ModulatedAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.7.2 **setValues(...)**

This method modifies the ModulatedAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ModulatedAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.7.3 Members

The ModulatedAmplitude object has members with the same names and descriptions as the arguments to the **ModulatedAmplitude** method.

3.7.4 Corresponding analysis keywords

*AMPLITUDE

3.8 PeriodicAmplitude object

The PeriodicAmplitude object defines an amplitude curve using a Fourier series.
The PeriodicAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.8.1 PeriodicAmplitude(...)

This method creates a PeriodicAmplitude object.

Path

```
mdb.models[name].PeriodicAmplitude
session.odbs[name].PeriodicAmplitude
```

AMPLITUDE COMMANDS

Required arguments

name

A String specifying the repository key.

frequency

A Float specifying the circular frequency ω . Possible values are positive numbers.

start

A Float specifying the starting time t_0 . Possible values are positive numbers.

a_0

A Float specifying the constant A_0 .

data

A sequence of pairs of Floats specifying A_i and B_i pairs.

Optional argument

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A PeriodicAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.8.2 **setValues(...)**

This method modifies the PeriodicAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PeriodicAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.8.3 Members

The PeriodicAmplitude object has members with the same names and descriptions as the arguments to the `PeriodicAmplitude` method.

3.8.4 Corresponding analysis keywords

*AMPLITUDE

3.9 PsdDefinition object

The PsdDefinition object defines the cross-spectral density frequency function for random response loading.

The PsdDefinition object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.9.1 PsdDefinition(...)

This method creates a PsdDefinition object.

Path

```
mdb.models[name].PsdDefinition
session.odbs[name].PsdDefinition
```

Required arguments

name

A String specifying the repository key.

AMPLITUDE COMMANDS

data

A sequence of sequences of Floats specifying the real part of the frequency function, the imaginary part of the frequency function, and the frequency or frequency band number values, depending on the value of *unitType*.

Optional arguments

unitType

A SymbolicConstant specifying the type of units for specifying the frequency function. FORCE implies power units. BASE implies gravity used to define base motion. DB implies decibel units. Possible values are FORCE, BASE, and DB. The default value is FORCE.

referenceGravityAcceleration

A Float specifying the reference gravity acceleration. This argument applies when *unitType* = BASE. The default value is 1.0.

referencePower

A Float specifying the reference power value, in load units squared. This argument applies when *unitType* = DB. The default value is 0.0.

user

A Boolean specifying whether the frequency function is defined in user subroutine UPSD. If specified, then *data* is not applicable, and the *unitType* value must not be DB. The default value is OFF.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

amplitude

A String specifying the name of the amplitude that describes the dynamic event used to define the cross-spectral density frequency function. The default value is an empty string.

Return value

A PsdDefinition object.

Exceptions

InvalidNameError and RangeError.

3.9.2 setValues(...)

This method modifies the PsdDefinition object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PsdDefinition** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.9.3 Members

The PsdDefinition object has members with the same names and descriptions as the arguments to the **PsdDefinition** method.

3.9.4 Corresponding analysis keywords

*PSD-DEFINITION

3.10 SmoothStepAmplitude object

The SmoothStepAmplitude object defines an amplitude that ramps up or down smoothly from one data point to another.

The SmoothStepAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.10.1 SmoothStepAmplitude(...)

This method creates a SmoothStepAmplitude object.

Path

```
mdb.models[name].SmoothStepAmplitude  
session.odbs[name].SmoothStepAmplitude
```

Required arguments

name

A String specifying the repository key.

data

A sequence of pairs of Floats specifying time/frequency and amplitude pairs. Possible values for time/frequency are positive numbers.

Optional argument

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A SmoothStepAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.10.2 setValues(...)

This method modifies the SmoothStepAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SmoothStepAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.10.3 Members

The SmoothStepAmplitude object has members with the same names and descriptions as the arguments to the **SmoothStepAmplitude** method.

3.10.4 Corresponding analysis keywords

*AMPLITUDE

3.11 SolutionDependentAmplitude object

The SolutionDependentAmplitude object defines a solution-dependent amplitude for superplastic forming analysis.

The SolutionDependentAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.11.1 SolutionDependentAmplitude(...)

This method creates a SolutionDependentAmplitude object.

Path

```
mdb.models[name].SolutionDependentAmplitude
session.odbs[name].SolutionDependentAmplitude
```

Required argument

name

A String specifying the repository key.

Optional arguments

initial

A Float specifying the initial amplitude value. Possible values are those between *minimum* and *maximum*. The default value is 1.0.

AMPLITUDE COMMANDS

minimum

A Float specifying the minimum amplitude value. Possible values are those smaller than *maximum* and *initial*. The default value is 0.1.

maximum

A Float specifying the maximum amplitude value. Possible values are those larger than *minimum* and *initial*. The default value is 1000.0.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A SolutionDependentAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.11.2 setValues(...)

This method modifies the SolutionDependentAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SolutionDependentAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.11.3 Members

The SolutionDependentAmplitude object has members with the same names and descriptions as the arguments to the **SolutionDependentAmplitude** method.

3.11.4 Corresponding analysis keywords

*AMPLITUDE

3.12 SpectrumAmplitude object

The SpectrumAmplitude object defines the spectrum of responses for displacement, velocity, or acceleration to be used in a response spectrum analysis.

The SpectrumAmplitude object is derived from the Amplitude object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.12.1 SpectrumAmplitude(...)

This method creates a SpectrumAmplitude object.

Path

```
mdb.models[name].SpectrumAmplitude
session.odbs[name].SpectrumAmplitude
```

Required arguments

name

A String specifying the repository key.

method

A SymbolicConstant specifying the method for specifying the spectrum. Possible values are DEFINE and CALCULATE.

data

A sequence of sequences of Floats specifying the magnitude, frequency, and damping values.

AMPLITUDE COMMANDS

Optional arguments

specificationUnits

A SymbolicConstant specifying the units used for specifying the spectrum. Possible values are DISPLACEMENT, VELOCITY, ACCELERATION, and GRAVITY. The default value is ACCELERATION.

eventUnits

A SymbolicConstant specifying the units used to describe the dynamic event in the amplitude used for the calculation. Possible values are EVENT_DISPLACEMENT, EVENT_VELOCITY, EVENT_ACCELERATION, and EVENT_GRAVITY. The default value is EVENT_ACCELERATION.

solution

A SymbolicConstant specifying the solution method for the dynamic equations. Possible values are ABSOLUTE_VALUE and RELATIVE_VALUE. The default value is ABSOLUTE_VALUE.

timeIncrement

A Float specifying the implicit time increment used to calculate the spectrum. This argument is required when the *method* = CALCULATE. The default value is 0.0.

gravity

A Float specifying the acceleration due to gravity. This argument applies only when *specificationUnits* = GRAVITY or *eventUnits* = GRAVITY. The default value is 1.0.

criticalDamping

A Boolean specifying whether to calculate the spectrum for only the specified range of critical damping values or a list of values. If *criticalDamping* = ON, the spectrum is calculated only for the specified range of critical damping values. If *criticalDamping* = OFF, the spectrum is calculated for a list of damping values. The default value is OFF.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

amplitude

A String specifying the name of the amplitude that describes the dynamic event used to calculate the spectrum. The default value is an empty string.

Return value

A SpectrumAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.12.2 **setValues(...)**

This method modifies the `SpectrumAmplitude` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SpectrumAmplitude` method, except for the *name* argument.

Return value

None

Exceptions

`RangeError`.

3.12.3 **Members**

The `SpectrumAmplitude` object has members with the same names and descriptions as the arguments to the `SpectrumAmplitude` method.

3.12.4 **Corresponding analysis keywords**

*SPECTRUM

3.13 **TabularAmplitude object**

The `TabularAmplitude` object defines an amplitude curve as a table of values at convenient points on the time scale.

The `TabularAmplitude` object is derived from the `Amplitude` object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
```

```
session.odbs[name].amplitudes[name]
```

3.13.1 TabularAmplitude(...)

This method creates a TabularAmplitude object.

Path

```
mdb.models[name].TabularAmplitude
session.odbs[name].TabularAmplitude
```

Required arguments

name

A String specifying the repository key.

data

A sequence of pairs of Floats specifying time/frequency and amplitude pairs. Possible values for time/frequency are positive numbers.

Optional arguments

smooth

The SymbolicConstant SOLVER_DEFAULT or a Float specifying the degree of smoothing. Possible float values are between 0 and 0.5. If *smooth*=SOLVER_DEFAULT, the default degree of smoothing will be determined by the solver. The default value is SOLVER_DEFAULT.

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A TabularAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.13.2 setValues(...)

This method modifies the TabularAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TabularAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.13.3 Members

The **TabularAmplitude** object has members with the same names and descriptions as the arguments to the **TabularAmplitude** method.

In addition, the **TabularAmplitude** object can have the following member:

baselineCorrection

A **BaselineCorrection** object.

3.13.4 Corresponding analysis keywords

*AMPLITUDE

3.14 UserAmplitude object

The **UserAmplitude** object defines an amplitude curve using the **UAMP** or **VUAMP** user subroutine.

The **UserAmplitude** object is derived from the **Amplitude** object.

Access

```
import amplitude
mdb.models[name].amplitudes[name]
import odbAmplitude
session.odbs[name].amplitudes[name]
```

3.14.1 UserAmplitude(...)

This method creates a **UserAmplitude** object.

Path

```
mdb.models[name].UserAmplitude  
session.odbs[name].UserAmplitude
```

Required arguments

name

A String specifying the repository key.

numVariables

An Int specifying the number of variables for the **UAMP** or **VUAMP** user subroutine.

Optional argument

timeSpan

A SymbolicConstant specifying the time span of the amplitude. Possible values are STEP and TOTAL. The default value is STEP.

Return value

A UserAmplitude object.

Exceptions

InvalidNameError and RangeError.

3.14.2 **setValues(...)**

This method modifies the UserAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **UserAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

3.14.3 Members

The UserAmplitude object has members with the same names and descriptions as the arguments to the **UserAmplitude** method.

3.14.4 Corresponding analysis keywords

*AMPLITUDE

4. Animation commands

4.1 AnimationController object

The AnimationController object controls all object-based animation to be displayed in the viewports. The AnimationController object has no constructor. Abaqus creates the *animationController* member when it creates the Session object.

Access

```
import animation
session.animationController
```

4.1.1 play(...)

This method begins the animation.

Arguments

Required arguments

None.

Optional argument

duration

The SymbolicConstant UNLIMITED or an Int specifying how many seconds to play the animation. The default value is UNLIMITED.

Return value

None

Exceptions

If *animationType*=NONE:

AnimationError: animationType not set

4.1.2 stop()

This method stops the animation.

Arguments

None.

Return value

None

Exceptions

None.

4.1.3 incrementFrame()

This method increments the animation frame.

Arguments

None.

Return value

None

Exceptions

None.

4.1.4 decrementFrame()

This method decrements the animation frame.

Arguments

None.

Return value

None

Exceptions

None.

4.1.5 **showFrame(...)**

This method renders the specified frame of the animation.

Arguments

Required arguments

None.

Optional arguments

frame

An Int specifying the frame number.

value

A Float specifying the frame: for *animationType*=TIME_HISTORY the frame with the time nearest to this value, for *animationType*=HARMONIC the frame with the angle nearest to this value, for *animationType*=SCALE_FACTOR the frame with the scale value nearest to this value.

Return value

None

Exceptions

None.

4.1.6 **showFirstFrame()**

This method renders the first frame of the animation.

Arguments

None.

Return value

None

Exceptions

None.

4.1.7 **showLastFrame()**

This method renders the last frame of the animation.

Arguments

None.

Return value

None

Exceptions

None.

4.1.8 **setValues(...)**

This method modifies the `AnimationController` object.

Arguments

Required arguments

None.

Optional arguments

animationType

A `SymbolicConstant` specifying the type of movie to play. Possible values are `SCALE_FACTOR`, `HARMONIC`, `TIME_HISTORY`, and `NONE`. The default value is `NONE`.

viewports

A sequence of pairs of `Strings` specifying the name of a viewport where the animation is active followed by a layer name, or the `SymbolicConstant` `ALL`. If a layer name is not supplied, the current layer is used. If the viewport is in single display mode, the layer name is ignored if it is specified. The default value is an empty sequence.

Return value

None

Exceptions

`RangeError`.

4.1.9 **Members**

The `AnimationController` object can have the following members:

animationType

A `SymbolicConstant` specifying the type of movie to play. Possible values are `SCALE_FACTOR`, `HARMONIC`, `TIME_HISTORY`, and `NONE`. The default value is `NONE`.

state

A SymbolicConstant specifying the state of the animation controller. Possible values are STOP and PLAY. The default value is STOP.

animationOptions

An AnimationOptions object.

viewports

A tuple of pairs of Strings specifying the name of a viewport where the animation is active followed by a layer name, or the SymbolicConstant ALL. If a layer name is not supplied, the current layer is used. If the viewport is in single display mode, the layer name is ignored if it is specified. The default value is an empty sequence.

4.2 AnimationOptions object

The AnimationOptions object is used to store values and attributes associated with an AnimationController object.

The AnimationOptions object has no constructor command. Abaqus creates the *animationOptions* member when it creates the AnimationController object.

Access

```
import animation
session.animationController.animationOptions
```

4.2.1 setValues(...)

This method modifies the AnimationOptions object.

Arguments

Required arguments

None.

Optional arguments

mode

A SymbolicConstant specifying the animation mode. Possible values are PLAY_ONCE, LOOP, LOOP_BACKWARD, and SWING. The default value is LOOP.

frameRate

An Int specifying the animation rate in frames/second. Possible values are $1 \leq \text{frameRate} \leq 100$. The default value is 50.

frameCounter

A Boolean specifying whether to show the frame counter. The default value is ON.

ANIMATION COMMANDS

relativeScaling

A SymbolicConstant specifying the relative scaling when the AnimationController object's *animationType*=SCALE_FACTOR or HARMONIC. Possible values are FULL_CYCLE and HALF_CYCLE. The default value is HALF_CYCLE.

numScaleFactorFrames

An Int specifying the number of frames to be used when the AnimationController object's *animationType*=SCALE_FACTOR or HARMONIC. The default value is 7.

timeHistoryMode

A SymbolicConstant specifying whether the time history animation is time based or frame based. Possible values are FRAME_BASED and TIME_BASED. The default value is FRAME_BASED.

maxTime

A Float specifying the maximum time for time based time history animation when *maxTimeAutoCompute* = False.

maxTimeAutoCompute

A Boolean specifying whether the animation maximum time value should be computed from the active frames when *timeHistoryMode* is set to TIME_BASED. The default value is ON.

minTime

A Float specifying the minimum time for time based time history animation when *minTimeAutoCompute* = False.

minTimeAutoCompute

A Boolean specifying whether the animation minimum time value should be computed from the active frames when *timeHistoryMode* is set to TIME_BASED. The default value is ON.

timeIncrement

A Float specifying the time increment for frame selection when *timeHistoryMode* is set to TIME_BASED.

xyUseHighlightMethod

A Boolean specifying whether to use the highlight method to draw the time tracker line and symbols. The default value is ON.

xyShowLine

A Boolean specifying whether to show the time tracker line. The default value is ON.

xyLineStyle

A SymbolicConstant specifying the X-Y time tracker line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

xyLineThickness

A SymbolicConstant specifying the X-Y time tracker line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is MEDIUM.

xyLineColor

A String specifying the color used to plot the X-Y time tracker line when *xyUseHighlightMethod* = False. The default value is "Yellow".

xyShowSymbol

A Boolean specifying whether to show the time tracker symbols. The default value is ON.

xySymbolMarker

A SymbolicConstant specifying the marker type to be used for all animation capable X–Y curve or the SymbolicConstant DEFAULT specifying that the system will take the marker associated to each curve Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER
- DEFAULT

The default value is DEFAULT.

xySymbolSize

A SymbolicConstant specifying the size of the markers. Possible values are SMALL, MEDIUM, and LARGE. The default value is MEDIUM.

xySymbolColor

A String specifying the color used to plot X–Y time tracker symbols when *xyUseHighlightMethod* = False. When setting the color to 'Default' the system will take the color associated to each curve. The default value is "Default".

Return value

None

Exceptions

None.

4.2.2 Members

The AnimationOptions object can have the following members:

mode

A SymbolicConstant specifying the animation mode. Possible values are PLAY_ONCE, LOOP, LOOP_BACKWARD, and SWING. The default value is LOOP.

ANIMATION COMMANDS

frameRate

An Int specifying the animation rate in frames/second. Possible values are $1 \leq \text{frameRate} \leq 100$. The default value is 50.

frameCounter

A Boolean specifying whether to show the frame counter. The default value is ON.

relativeScaling

A SymbolicConstant specifying the relative scaling when the AnimationController object's *animationType*=SCALE_FACTOR or HARMONIC. Possible values are FULL_CYCLE and HALF_CYCLE. The default value is HALF_CYCLE.

numScaleFactorFrames

An Int specifying the number of frames to be used when the AnimationController object's *animationType*=SCALE_FACTOR or HARMONIC. The default value is 7.

timeHistoryMode

A SymbolicConstant specifying whether the time history animation is time based or frame based. Possible values are FRAME_BASED and TIME_BASED. The default value is FRAME_BASED.

maxTime

A Float specifying the maximum time for time based time history animation when *maxTimeAutoCompute* = False.

maxTimeAutoCompute

A Boolean specifying whether the animation maximum time value should be computed from the active frames when *timeHistoryMode* is set to TIME_BASED. The default value is ON.

maxTimeAutoValue

A Float specifying the maximum time when *timeHistoryMode* is set to TIME_BASED and the *maxTimeAutoCompute* value is True. This value is computed as the maximum time of all active frames displayed in viewports where the animation is active.

minTime

A Float specifying the minimum time for time based time history animation when *minTimeAutoCompute* = False.

minTimeAutoCompute

A Boolean specifying whether the animation minimum time value should be computed from the active frames when *timeHistoryMode* is set to TIME_BASED. The default value is ON.

minTimeAutoValue

A Float specifying the minimum time when *timeHistoryMode* is set to TIME_BASED and the *minTimeAutoCompute* value is True. This value is computed as the minimum time of all active frames displayed in viewports where the animation is active.

timeIncrement

A Float specifying the time increment for frame selection when *timeHistoryMode* is set to TIME_BASED.

xyUseHighlightMethod

A Boolean specifying whether to use the highlight method to draw the time tracker line and symbols. The default value is ON.

xyShowLine

A Boolean specifying whether to show the time tracker line. The default value is ON.

xyLineStyle

A SymbolicConstant specifying the X-Y time tracker line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

xyLineThickness

A SymbolicConstant specifying the X-Y time tracker line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is MEDIUM.

xyShowSymbol

A Boolean specifying whether to show the time tracker symbols. The default value is ON.

xySymbolMarker

A SymbolicConstant specifying the marker type to be used for all animation capable X-Y curve or the SymbolicConstant DEFAULT specifying that the system will take the marker associated to each curve Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER
- DEFAULT

The default value is DEFAULT.

xySymbolSize

A SymbolicConstant specifying the size of the markers. Possible values are SMALL, MEDIUM, and LARGE. The default value is MEDIUM.

xyLineColor

A String specifying the color used to plot the X-Y time tracker line when *xyUseHighlightMethod* = False. The default value is "Yellow".

xySymbolColor

A String specifying the color used to plot *X–Y* time tracker symbols when *xyUseHighlightMethod* = False. When setting the color to 'Default' the system will take the color associated to each curve. The default value is "Default".

4.3 AVIOptions object

The AVIOptions object is used to store values and attributes to be used in generating AVI animation. The AVIOptions object has no constructor. Abaqus creates the *aviOptions* member when the animation module is imported.

Access

```
import animation
session.aviOptions
```

4.3.1 setValues(...)

This method modifies the AVIOptions object.

Arguments**Required arguments**

None.

Optional arguments*compressionMethod*

A SymbolicConstant specifying the compression method for the AVI format. Possible values are:

- RAW8, specifying an uncompressed format of 8 bits per pixel.
- RAW32, specifying an uncompressed format of 32 bits per pixel.
- RLE8, specifying a run length encoded format of 8 bits per pixel.
- CODEC, specifying a format defined by CODEC (encoder/decoder). The availability of the CODEC is system dependent

The default value is RLE8.

compressionQuality

An Int specifying the quality of the compression as a percentage when the *compressionMethod* is set to CODEC.

codecOptions

A String specifying the system specific options defining the CODEC when the *compressionMethod* is set to CODEC.

sizeDefinition

A SymbolicConstant specifying how the width and height of the image are specified. Possible values are SIZE_ON_SCREEN and USER_DEFINED. The default value is SIZE_ON_SCREEN.

imageSize

A pair of Ints specifying the width and height of the image in pixels when *sizeDefinition*=USER_DEFINED. Possible values are Ints in the range (*minWidth*, *minHeight*) ≤ (width, height) ≤ (*maxWidth*, *maxHeight*). The default value is the screen size.

Note: The values of the minimum width (*minWidth*) and height (*minHeight*) depend on the following:

- viewport font size,
- whether decorations are printed,
- decoration size, and
- screen resolution.

The minimum width and height are normally < 50 pixels. The values of the maximum width (*maxWidth*) and height (*maxHeight*) depend on the graphics capabilities of the system. The maximum width and height will be at least as large as the screen dimensions.

Return value

None

Exceptions

None.

4.3.2 Members

The AVIOptions object has members with the same names and descriptions as the arguments to the `setValues` method.

4.4 ImageAnimationOptions object

The ImageAnimationOptions object is used to store values and attributes associated with saving viewport animations. The ImageAnimationOptions object has no constructor. Abaqus creates the *imageAnimationOptions* member when the animation module is imported.

Access

`import animation`

`session.imageAnimationOptions`

4.4.1 **setValues(...)**

This method modifies the `ImageAnimationOptions` object.

Arguments

Required arguments

None.

Optional arguments

frameRate

An Int specifying the frame rate to record on the saved animation file. The effective frame rate in frames per second will be obtained by dividing the given frame rate by the time scale.

timeScale

An Int specifying the time scale to apply to the frame rate.

vpDecorations

A Boolean specifying whether to capture the viewport border and title. The default value is ON.

vpBackground

A Boolean specifying whether to capture viewport backgrounds. The default value is OFF.

compass

A Boolean specifying whether to capture the view compass. The default value is OFF.

Return value

None

Exceptions

None.

4.4.2 **Members**

The `ImageAnimationOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

4.5 **ImageAnimation object**

The `ImageAnimation` object is used to build frame by frame animation.

Access

```
import animation
session.imageAnimation
```

4.5.1 ImageAnimation(...)

This method creates an ImageAnimation object from the specified filename and format.

Path

```
session.ImageAnimation
```

Required arguments

fileName

A String specifying the name of the animation file to generate.

format

A SymbolicConstant specifying the format of the generated file. Possible values are AVI, QUICKTIME.

Optional arguments

None.

Return value

An ImageAnimation object.

Exceptions

None.

4.5.2 writeFrame(...)

This method adds a frame to the ImageAnimation object.

Arguments**Required arguments**

None.

Optional argument

canvasObjects

A sequence specifying the canvas objects to capture. The default is to capture all canvas objects.

Return value

None

Exceptions

None.

4.5.3 close()

This method closes the ImageAnimation object.

Arguments

None.

Return value

None

Exceptions

None.

4.5.4 closed()

This method indicates if the ImageAnimation is open or closed for writing animation frames.

Arguments

None.

Return value

None

Exceptions

None.

4.5.5 Members

The ImageAnimation object has the following member:

fileName

A String specifying the file to which the animation frames is to be written.

4.6 QuickTimeOptions object

The QuickTimeOptions object is used to store values and attributes to be used in generating QuickTime animation. The QuickTimeOptions object has no constructor. Abaqus creates the *quickTimeOptions* member when the animation module is imported.

Access

```
import animation
session.quickTimeOptions
```

4.6.1 setValues(...)

This method modifies the QuickTimeOptions object.

Arguments

Required arguments

None.

Optional arguments

compressionMethod

A SymbolicConstant specifying the compression method for the QuickTime format. Possible values are:

- RAW24, specifying an uncompressed format of 24 bits per pixel.
- RLE24, specifying a run length encoded format of 24 bits per pixel.

The default value is RLE24.

sizeDefinition

A SymbolicConstant specifying how the width and height of the image are specified. Possible values are SIZE_ON_SCREEN and USER_DEFINED. The default value is SIZE_ON_SCREEN.

imageSize

A pair of Ints specifying the width and height of the image in pixels when *sizeDefinition*=USER_DEFINED. Possible values are Ints in the range $(minWidth, minHeight) \leq (width, height) \leq (maxWidth, maxHeight)$. The default value is the screen size.

Note: The values of the minimum width (*minWidth*) and height (*minHeight*) depend on the following:

- viewport font size,
- whether decorations are printed,
- decoration size, and

- screen resolution.

The minimum width and height are normally < 50 pixels. The values of the maximum width (*maxWidth*) and height (*maxHeight*) depend on the graphics capabilities of the system. The maximum width and height will be at least as large as the screen dimensions.

Return value

None

Exceptions

None.

4.6.2 Members

The QuickTimeOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

4.7 Movie object

The Movie object is used to store values and attributes associated with a movie file. Upon creation, the Movie object is added to the **session.movies** repository.

Access

```
import animation
session.movies[name]
```

4.7.1 Movie(...)

This method creates a Movie object from the contents of the specified file.

Path

session.Movie

Required arguments

name

A String specifying the repository name for the movie.

fileName

A String specifying the file from which the movie is to be read. The file extension must be specified and indicates the movie format (**.avi**, **.mov**, **.mpeg**, or **.wmv**).

Optional arguments*startFrame*

An Int specifying the first frame to be displayed from this movie. The default value is 0.

endFrame

An Int specifying the last frame to be displayed from this movie. A negative number will indicate reverse numbering: -1 is the last frame of the movie. The default value is -1.

timelineStartFrame

An Int specifying the global timeline frame number that corresponds to *startFrame*. A value of 0 will indicate the first frame to be displayed in the viewport. The default value is 0.

timelineEndFrame

An Int specifying the global timeline frame number that corresponds to *endFrame*. A negative number will indicate reverse numbering: -1 indicates the last frame to be displayed in the viewport. The default value is -1.

timelineStartTime

A Float specifying the global timeline time that corresponds to the time of *startFrame*. The default value is 0.0.

timelineEndTime

The SymbolicConstant `END_FRAME_TIME` or a Float specifying the global timeline time that corresponds to the time of *endFrame*. The SymbolicConstant `END_FRAME_TIME` indicates the time in this movie corresponding to *endFrame*. The default value is `END_FRAME_TIME`.

Return value

A Movie object.

Exceptions

ValueError.

If *fileName* does not exist or can not be read:

ValueError: Unable to open movie file

If *fileName* references an unsupported movie file format:

ValueError: Unsupported movie format

If the contents of *fileName* are corrupt or can not be decoded:

ValueError: Unable to decode movie file

4.7.2 setValues(...)

This method modifies the Movie object.

Arguments

Required arguments

None.

Optional arguments

startFrame

An Int specifying the first frame to be displayed from this movie. The default value is 0.

endFrame

An Int specifying the last frame to be displayed from this movie. A negative number will indicate reverse numbering: -1 is the last frame of the movie. The default value is -1.

timelineStartFrame

An Int specifying the global timeline frame number that corresponds to *startFrame*. A value of 0 will indicate the first frame to be displayed in the viewport. The default value is 0.

timelineEndFrame

An Int specifying the global timeline frame number that corresponds to *endFrame*. A negative number will indicate reverse numbering: -1 indicates the last frame to be displayed in the viewport. The default value is -1.

timelineStartTime

A Float specifying the global timeline time that corresponds to the time of *startFrame*. The default value is 0.0.

timelineEndTime

The SymbolicConstant `END_FRAME_TIME` or a Float specifying the global timeline time that corresponds to the time of *endFrame*. The SymbolicConstant `END_FRAME_TIME` indicates the time in this movie corresponding to *endFrame*. The default value is `END_FRAME_TIME`.

Return value

None

Exceptions

`RangeError`.

If *startFrame*, *endFrame*, *timelineStartFrame*, *timelineEndFrame*, *timelineStartTime*, or *timelineEndTime* are outside their respective valid range.

`RangeError`: *startFrame* must be an Integer in the range: 0 to `numFrames-1`

4.7.3 Members

The `Movie` object has members with the same names and descriptions as the arguments to the **Movie** method.

In addition, the `Movie` object has the following members:

width

An Int specifying the width of the movie in pixels.

height

An Int specifying the height of the movie in pixels.

numFrames

An Int specifying the total number of frames on the movie file.

duration

A Float specifying the duration of the movie in seconds.

memory

An Int specifying the memory taken by the movie frames as selected.

4.8 Session object

The following commands operate on Session objects. For more information about the Session object, see “Session object,” Section 47.1.

Access

```
import animation
```

4.8.1 writeImageAnimation(...)

This method writes the animations present in the list of canvas objects to a file. It generates an animation file using the given file name and file format and uses the values in the appropriate options object.

Arguments

Required arguments

fileName

A String specifying the name of the animation file to generate.

format

A SymbolicConstant specifying the format of the generated file. Possible values are AVI, QUICKTIME, VRML, and COMPRESSED_VRML.

Optional argument

canvasObjects

A sequence specifying the canvas objects to capture. The default behavior is to capture all canvas objects.

ANIMATION COMMANDS

Return value

None

Exceptions

None.

5. Annotation commands

Annotation commands are used to create, position, and modify viewport annotations.

5.1 Annotation object

The Annotation object is the abstract base type for other Annotation objects. The Annotation object has no explicit constructor. The methods and members of the Annotation object are common to all objects derived from Annotation.

Access

```
import annotationToolset
mdb.annotations[name]
session.odbs[name].userData.annotations[name]
session.viewports[name].annotationsToPlot[i]
```

5.1.1 bringToFront()

This method brings the Annotation object to the top of the annotation stack.

Arguments

None.

Return value

None

Exceptions

None.

5.1.2 sendToBack()

This method sends the Annotation object to the bottom of the annotation stack.

Arguments

None.

Return value

None

Exceptions

None.

5.1.3 **bringForward()**

This method brings the Annotation object one position up in the annotation stack.

Arguments

None.

Return value

None

Exceptions

None.

5.1.4 **sendBackward()**

This method sends the Annotation object one position down in the annotation stack.

Arguments

None.

Return value

None

Exceptions

None.

5.1.5 **moveBefore(...)**

This method moves the Annotation object before another object in the same repository.

Arguments**Required argument***name*

A String specifying the name of the other Annotation object.

Optional arguments

None.

Return value

None

Exceptions

None.

5.1.6 moveAfter(...)

This method moves the Annotation object after another object in the same repository.

Arguments**Required argument***name*

A String specifying the name of the other Annotation object.

Optional arguments

None.

Return value

None

Exceptions

None.

5.1.7 translate(...)

This method translates the Annotation object on the viewport plane.

Arguments**Required arguments**

None.

Optional arguments

x

A Float specifying the *X* translation amount in millimeters.

y

A Float specifying the *Y* translation amount in millimeters.

Return value

None

Exceptions

None.

5.1.8 Members

The Annotation object has the following member:

name

A String specifying the annotation repository key.

5.2 AnnotationsToPlotArray object

The AnnotationsToPlotArray object is a sequence that stores references to plotted annotations. By adding annotations to and removing annotations from this sequence, you can control which annotations are displayed in a particular viewport.

Access

```
import annotationToolset
session.viewports[name].annotationsToPlot
```

5.2.1 bringForward(...)

This method brings the Annotation object one position forward in the AnnotationsToPlotArray sequence.

Arguments

Required argument

index

An Int specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.2 **bringToFront(...)**

This method brings the Annotation object to the beginning of the AnnotationsToPlotArray sequence.

Arguments**Required argument**

index

An Int specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.3 **moveAfter(...)**

This method moves the Annotation object after another object in the same AnnotationsToPlotArray sequence.

Arguments**Required arguments**

index

An Integer specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

other

An Integer specifying the index of the other Annotation object in the AnnotationsToPlotArray sequence after which this object will be moved.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.4 moveBefore(...)

This method moves the Annotation object before another object in the same AnnotationsToPlotArray sequence.

Arguments

Required arguments

index

An Int specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

other

An Int specifying the index of the other Annotation object in the AnnotationsToPlotArray sequence before which this object will be moved.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.5 sendBackward(...)

This method sends the Annotation object one position backward in the AnnotationsToPlotArray sequence.

Arguments

Required argument

index

An Int specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.6 sendToBack(...)

This method sends the Annotation object to the end of the AnnotationsToPlotArray sequence.

Arguments**Required argument**

index

An Int specifying the index of the Annotation object in the AnnotationsToPlotArray sequence.

Optional arguments

None.

Return value

None

Exceptions

None.

5.2.7 Members

The AnnotationsToPlotArray object has no members.

5.3 Arrow object

The Arrow object stores the visual settings and location of an arrow annotation.

The Arrow object is derived from the Annotation object.

Access

```
import annotationToolset
mdb.annotations [name]
```

```
session.odbs [name] .userData.annotations [name]
session.viewports [name] .annotationsToPlot [i]
```

5.3.1 Arrow(...)

This method creates an Arrow object.

Path

```
mdb.Arrow
session.odbs [name] .userData.Arrow
```

Required argument

name

A String specifying the annotation repository key.

Optional arguments

startPoint

A pair of Floats specifying the start point *X*- and *Y*-offsets in millimeters from *startAnchor*. The default value is (0, 0).

endPoint

A pair of Floats specifying the end point *X*- and *Y*-offsets in millimeters from *endAnchor*. The default value is (0, 0).

startAnchor

A SymbolicConstant or a sequence of Floats specifying a point. A sequence of two Floats specifies the *X*- and *Y*-coordinates as percentages of the viewport width and height. A sequence of three Floats specifies the *X*-, *Y*-, and *Z*-coordinates of a point in the model coordinate system. A SymbolicConstant indicates a relative position. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

endAnchor

A SymbolicConstant or a sequence of Floats specifying a point. A sequence of two Floats specifies the X- and Y-coordinates as percentages of the viewport width and height. A Sequence of three Floats specifies the X-, Y-, and Z-coordinates of a point in the model coordinate system. A SymbolicConstant indicates a relative position. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

startHeadStyle

A SymbolicConstant specifying the style of the start head. Possible values are:

- ARROW
- FILLED_ARROW
- HOLLOW_CIRCLE
- FILLED_CIRCLE
- HOLLOW_DIAMOND
- FILLED_DIAMOND
- HOLLOW_SQUARE
- FILLED_SQUARE
- NONE

The default value is NONE.

endHeadStyle

A SymbolicConstant specifying the style of the end head. Possible values are:

- ARROW
- FILLED_ARROW
- HOLLOW_CIRCLE
- FILLED_CIRCLE
- HOLLOW_DIAMOND
- FILLED_DIAMOND
- HOLLOW_SQUARE

ANNOTATION COMMANDS

- FILLED_SQUARE
- NONE

The default value is FILLED_ARROW.

startGap

A Float specifying the distance in millimeters between the arrow start point and the arrow start head. The default value is 0.0.

endGap

A Float specifying the distance in millimeters between the arrow end point and the arrow end head. The default value is 0.0.

color

A String specifying the color of the arrow. Possible string values are any valid color. The default value is "White".

lineStyle

A SymbolicConstant specifying the line style of the arrow. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

lineThickness

A SymbolicConstant specifying the line thickness of the arrow. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

Return value

An Arrow object.

Exceptions

InvalidNameError.

5.3.2 translateStartPoint(...)

This method translates the start point of the Arrow object on the viewport plane.

Arguments

Required arguments

None.

Optional arguments

x

A Float specifying the *X* translation amount in millimeters.

y

A Float specifying the *Y* translation amount in millimeters.

Return value

None

Exceptions

None.

5.3.3 **translateEndPoint(...)**

This method translates the end point of the Arrow object on the viewport plane.

Arguments**Required arguments**

None.

Optional arguments

x

A Float specifying the *X* translation amount in millimeters.

y

A Float specifying the *Y* translation amount in millimeters.

Return value

None

Exceptions

None.

5.3.4 **setValues(...)**

This method modifies the Arrow object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Arrow** method, except for the *name* argument.

Return value

None

Exceptions

None.

5.3.5 Members

The Arrow object has members with the same names and descriptions as the arguments to the **Arrow** method.

5.4 Text object

The Text object stores the text settings and location of a text annotation.

The Text object is derived from the Annotation object.

Access

```
import annotationToolset
mdb.annotations [name]
session.odbs [name] .userData.annotations [name]
session.viewports [name] .annotationsToPlot [i]
```

5.4.1 Text(...)

This method creates a Text object.

Path

```
mdb.Text
session.odbs [name] .userData.Text
```

Required argument

name

A String specifying the annotation repository key.

Optional arguments

text

A String specifying the text of the Text object. The default value is an empty string.

offset

A pair of Floats specifying the *X*- and *Y*-offsets in millimeters of the Text object from *anchor*. The default value is (0, 0).

anchor

A SymbolicConstant or a sequence of Floats specifying a point. A sequence of two Floats specifies the *X*- and *Y* coordinates as percentages of the viewport width and height. A Sequence of three Floats specifies the *X*-, *Y*-, and *Z*-coordinates of a point in the model coordinate system. A SymbolicConstant specifies a relative position. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

referencePoint

A SymbolicConstant or a sequence of Floats specifying a point. The sequence of two Floats specifies the *X*- and *Y*-coordinates of the reference point of the Text annotation given as percentages of its width and height. The SymbolicConstant indicates a relative position. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

rotationAngle

A Float specifying the amount of rotation in degrees about *referencePoint*. The default value is 0.0.

ANNOTATION COMMANDS

color

A String specifying the color of the Text object. Possible string values are any valid color. The default value is "White".

font

A String specifying the font of the Text object. Possible string values are any valid font specification. The default value is "-*-verdana-medium-r-normal--120-*".

backgroundStyle

A SymbolicConstant specifying the Text object background style. Possible values are MATCH, TRANSPARENT, and OTHER. The default value is TRANSPARENT.

backgroundColor

A String specifying the color of the Text object background. Possible string values are any valid color. The default value matches the viewport background.

box

A Boolean specifying whether the box around the text is shown. The default value is OFF.

justification

A SymbolicConstant specifying the Text object justification for multiline text. Possible values are JUSTIFY_LEFT, JUSTIFY_CENTER, and JUSTIFY_RIGHT. The default value is JUSTIFY_LEFT.

Return value

A Text object.

Exceptions

None.

5.4.2 setValues(...)

This method modifies the Text object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Text** method, except for the *name* argument.

Return value

None

Exceptions

None.

5.4.3 Members

The Text object has members with the same names and descriptions as the arguments to the **Text** method.

In addition, the Text object has the following members:

width

A Float specifying the width in millimeters of the Text object.

height

A Float specifying the height in millimeters of the Text object.

5.5 Viewport object

The following commands operate on Viewport objects. For more information about the Viewport object, see “Viewport object,” Section 11.4.

Access

```
import annotationToolset
```

5.5.1 plotAnnotation(...)

This method plots an Annotation object in a Viewport.

Arguments**Required argument**

annotation

An Annotation object to plot.

Optional argument

index

An Int specifying the index of the Annotation object in the sequence of annotations to plot. The default value is zero.

Return value

None

ANNOTATION COMMANDS

Exceptions

None.

6. Assembly commands

6.1 Assembly object

An Assembly object is a container for instances of parts. The Assembly object has no constructor command. Abaqus creates the *rootAssembly* member when a Model object is created.

Access

```
import assembly
mdb.models[name].rootAssembly
```

6.1.1 backup()

This method makes a backup copy of the features in the assembly. The **backup()** method is used in conjunction with the **restore()** method.

Arguments

None.

Return value

None

Exceptions

None.

6.1.2 clearGeometryCache()

This method deletes the geometry cache. Deleting the geometry cache reduces the amount of memory being used.

Arguments

None.

Return value

None

Exceptions

None.

6.1.3 deleteAllFeatures()

This method deletes all the features in the assembly.

Arguments

None.

Return value

None

Exceptions

None.

6.1.4 deleteFeatures(...)

This method deletes specified features from the assembly.

Arguments

Required argument

featureNames

A sequence of Strings specifying the feature names that will be deleted from the assembly.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.5 excludeFromSimulation(...)

This method excludes the specified part instances from the analysis.

Arguments**Required arguments***instances*

A sequence of PartInstance objects to be excluded from the analysis.

exclude

A Bool specifying whether to exclude the selected instances from the analysis or include them.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.6 featurelistInfo()

This method prints the name and status of all the features in the feature lists.

Arguments

None.

Return value

None

Exceptions

None.

6.1.7 getMassProperties(...)

This method returns the mass properties of the assembly, or instances or regions. Only beams, trusses, shells, solids, point, nonstructural mass, and rotary inertia elements are supported.

Arguments**Required arguments**

None.

Optional arguments

regions

A MeshElementArray, CellArray, FaceArray, EdgeArray, or list of PartInstance objects specifying the regions whose mass properties are to be queried. The whole assembly is queried by default.

relativeAccuracy

A SymbolicConstant specifying the relative accuracy for geometry computation. Possible values are LOW, MEDIUM, and HIGH. The default value is LOW.

useMesh

A Boolean specifying whether the mesh should be used in the computation if the geometry is meshed. The default value is False.

specifyDensity

A Boolean specifying whether a user-specified density should be used in regions with density errors such as undefined material density. The default value is False.

density

A double value specifying the user-specified density value to be used in regions with density errors. The user-specified density should be greater than 0.

specifyThickness

A Boolean specifying whether a user-specified thickness should be used in regions with thickness errors such as undefined thickness. The default value is False.

thickness

A double value specifying the user-specified thickness value to be used in regions with thickness errors. The user-specified thickness should be greater than 0.

miAboutCenterOfMass

A Boolean specifying if the moments of inertia should be evaluated about the center of mass. The default value is True.

miAboutPoint

A tuple of three floats specifying the coordinates of the point about which to evaluate the moment of inertia. By default if the moments of inertia are not being evaluated about the center of mass, they will be evaluated about the origin.

Return value

A Dictionary object with the following items:

area: None or a Float specifying the sum of the area of the specified faces. The area is computed only for one side for shells.

areaCentroid: None or a tuple of three Floats representing the coordinates of the area centroid.

volume: None or a Float specifying the volume of the specified regions.

volumeCentroid: None or a tuple of three Floats representing the coordinates of the volume centroid.

massFromMassPerUnitSurfaceArea: None or a Float specifying the mass due to mass per unit surface area.

mass: None or a Float specifying the mass of the specified regions. It is the total mass and includes mass from quantities such as mass per unit surface area.

centerOfMass: None or a tuple of three Floats representing the coordinates of the center of mass.

momentOfInertia: None or a tuple of six Floats representing the moments of inertia about the center of mass or about the point specified.

warnings: A tuple of SymbolicConstants representing the problems encountered while computing the mass properties. Possible SymbolicConstants are:

UNSUPPORTED_ENTITIES: Some unsupported entities exist in the specified regions. The mass properties are computed only for beams, trusses, shells, solids, point and non-structural mass elements, and rotary inertia elements. The mass properties are not computed for axisymmetric elements, springs, connectors, gaskets, or any other elements.

MISSING_THICKNESS: For some regions, the section definitions are missing thickness values.

ZERO_THICKNESS: For some regions, the section definitions have a zero thickness value.

VARIABLE_THICKNESS: The nodal thickness or field thickness specified for some regions has been ignored.

NON_APPLICABLE_THICKNESS: For some regions, the thickness value is not applicable to the corresponding sections specified on the regions.

MISSING_DENSITY: For some regions, the section definitions are missing material density values.

MISSING_MATERIAL_DEFINITION: For some regions, the material definition is missing.

ZERO_DENSITY: For some regions, the section definitions have a zero material density value.

UNSUPPORTED_DENSITY: For some regions, either a negative material density or a temperature dependent density has been specified, or the material value is missing for one or more plies in the composite section.

SHELL_OFFSETS: For shells, this method does not account for any offsets specified.

MISSING_SECTION_DEFINITION: For some regions, the section definition is missing.

UNSUPPORTED_SECTION_DEFINITION: The section definition provided for some regions is not supported.

REINFORCEMENTS: This method does not account for any reinforcements specified on the model.

SMEARED_PROPERTIES: For regions with composite section assignments, the density is smeared across the thickness. The volume centroid and center of mass computations for a composite shell use a lumped mass approach where the volume and mass is assumed to be lumped in the plane of the shell. As a result of these approximations the volume centroid, center of mass and moments of inertia may be slightly inaccurate for regions with composite section assignments.

UNSUPPORTED_NON_STRUCTURAL_MASS_ENTITIES: This method does not account for any non-structural mass on wires.

INCORRECT_MOMENT_OF_INERTIA: For geometry regions with non-structural mass per volume, the non-structural mass is assumed to be a point mass at the centroid of the regions. Thus, the

moments of inertia may be inaccurate as the distribution of the non-structural mass is not accounted for. Use the mesh for accurately computing the moments of inertia.

MISSING_BEAM_ORIENTATIONS: For some regions with beam section assignments, the beam section orientations are missing.

UNSUPPORTED_BEAM_PROFILES: This method supports the Box, Pipe, Circular, Rectangular, Hexagonal, Trapezoidal, I, L, T, Arbitrary, and Tapered beam profiles. Any other beam profile is not supported.

TAPERED_BEAM_MI: Moment of inertia calculations for tapered beams are not accurate.

SUBSTRUCTURE_INCORRECT_PROPERTIES: The user assigned density and thickness is not considered for substructures.

Exceptions

None.

6.1.8 getAngle(...)

This method returns the angle between the specified entities.

Arguments

Required arguments

plane1

A Face , MeshFace , or a Datum object specifying the first plane. The Datum object must represent a datum plane. The *plane1* and *line1* arguments are mutually exclusive. One of them must be specified.

plane2

A Face , MeshFace , or a Datum object specifying the second plane. The Datum object must represent a datum plane. The *plane2* and *line2* arguments are mutually exclusive. One of them must be specified.

line1

An Edge , MeshEdge , or a Datum object specifying the first curve. The Datum object must represent a datum axis. The *plane1* and *line1* arguments are mutually exclusive. One of them must be specified.

line2

An Edge , MeshEdge , or a Datum object specifying the second curve. The Datum object must represent a datum axis. The *plane2* and *line2* arguments are mutually exclusive. One of them must be specified.

Optional argument

commonVertex

If the two selected Edge objects have more than one vertex in common, this Vertex object specifies the vertex at which to evaluate the angle.

Return value

A Float specifying the angle between the specified entities. If you provide a plane as an argument, Abaqus/CAE computes the angle using the normal to the plane.

Exceptions

None.

6.1.9 getCoordinates(...)

This method returns the coordinates of a specified point.

Arguments**Required argument**

entity

A Vertex , Datum point, MeshNode , or ReferencePoint specifying the entity to query.

Optional arguments

None.

Return value

A tuple of three Floats representing the coordinates of the specified point.

Exceptions

None.

6.1.10 getDistance(...)

Depending on the arguments provided, this method returns one of the following:

- The distance between two points.
- The minimum distance between a point and an edge.
- The minimum distance between two edges.

Arguments**Required arguments**

entity1

A Vertex , Datum point, MeshNode , or Edge specifying the first entity from which to measure.

entity2

A Vertex , Datum point, MeshNode , or Edge specifying the second entity to which to measure.

Optional argument

printResults

A Boolean that determines whether a verbose output is to be printed. The default is True

Return value

A Float specifying the calculated distance.

Exceptions

None.

6.1.11 getFacesAndVerticesOfAttachmentLines(...)

Given an array of edge objects, this method returns a tuple of dictionary objects. Each object consists of five members including the attachment line and associated face and vertex objects.

Arguments

Required argument

edges

An EdgeArray object which is a sequence of Edge objects.

Optional arguments

None.

Return value

A tuple of dictionary objects. Each dictionary contains five items with the following keys:

edge: An Edge object specifying the attachment line.

startFace: A Face object specifying the face associated with one end of the attachment line.

endFace: A Face object specifying the face associated with the other end of the attachment line.

startVertex: A Vertex object specifying the vertex associated with one end of the attachment line.

This end is also associated with the startFace.

endVertex: A Vertex object specifying the vertex associated with the other end of the attachment line. This end is also associated with the endFace.

Exceptions

None.

6.1.12 **getSurfaceSections(...)**

This method returns a list of the sections assigned to the regions encompassed by the specified surface.

Arguments

Required argument

surface

A string specifying the Surface name.

Optional arguments

None.

Return value

A tuple of strings representing the section names. If no section names are found, the tuple will contain one empty string.

Exceptions

None.

6.1.13 **importEafFile(...)**

This method imports an assembly from an EAF file into the root assembly.

Arguments

Required argument

filename

A String specifying the path to the EAF file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.14 **importParasolidFile(...)**

This method imports an assembly from the Parasolid file into the root assembly.

Arguments

Required argument

filename

A String specifying the path to a Parasolid file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.15 **importCatiaV4File(...)**

This method imports an assembly from a CATIA V4 file into the root assembly.

Arguments

Required argument

filename

A String specifying the path to the CATIA V4 file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.16 importCatiaV5File(...)

This method imports an assembly from a CATIA V5 Elysium Neutral file into the root assembly.

Arguments**Required argument**

filename

A String specifying the path to the CATIA V5 Elysium Neutral file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.17 importEnfFile(...)

This method imports an assembly from an Elysium Neutral file created by Pro/ENGINEER, I-DEAS, or CATIA V5 into the root assembly.

Arguments**Required argument**

filename

A String specifying the path to the Elysium Neutral file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ASSEMBLY COMMANDS

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.18 importIdeasFile(...)

This method imports an assembly from an I-DEAS Elysium Neutral file into the root assembly.

Arguments

Required argument

filename

A String specifying the path to the I-DEAS Elysium Neutral file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value

None

Exceptions

None.

6.1.19 importProEFile(...)

This method imports an assembly from a Pro/ENGINEER Elysium Neutral file into the root assembly.

Arguments**Required argument***filename*

A String specifying the path to the Pro/ENGINEER Elysium Neutral file from which to import the assembly.

Optional argument

You use the following optional argument only to import specific instances and their associated parts.

ids

A sequence of Ints. Each Int in the sequence is a unique identifier of the occurrence in the assembly tree or component identifier associated with the part in the EAF file. If *ids* is an empty sequence, all occurrences or parts will be imported. The default value is an empty sequence.

Return value**None****Exceptions**

None.

6.1.20 makeDependent(...)

This method converts the specified part instances from independent to dependent part instances.

Arguments**Required argument***instances*

A sequence of PartInstance objects to convert to dependent part instances.

Optional arguments

None.

Return value**None****Exceptions**

None.

6.1.21 makeIndependent(...)

This method converts the specified part instances from dependent to independent part instances.

Arguments

Required argument

instances

A sequence of PartInstance objects to convert to independent part instances.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.22 printAssignedSections()

This method prints a summary of assigned connector sections.

Arguments

None.

Return value

None

Exceptions

None.

6.1.23 printConnectorOrientations()

This method prints a summary of connector orientations.

Arguments

None.

Return value

None

Exceptions

None.

6.1.24 projectReferencesOntoSketch(...)

This method projects the specified edges, vertices, and datum points from the assembly onto the specified ConstrainedSketch object. The edges, vertices, and datum points appear on the sketch as reference geometry.

Arguments**Required argument**

sketch

The ConstrainedSketch object on which the edges, vertices, and datum points are projected.

Optional arguments

filter

A SymbolicConstant specifying how to limit the amount of projection. Possible values are ALL_EDGES and COPLANAR_EDGES. If *filter*=COPLANAR_EDGES, edges that are coplanar to the sketching plane are the only candidates for projection. The default value is ALL_EDGES.

upToFeature

A Feature object specifying a marker in the feature-based history of the part. Abaqus/CAE projects onto the sketch only the part entities that were created before the feature specified by this marker. By default, all part entities are candidates for projection.

edges

A sequence of candidate edges to be projected onto the sketch. By default, all edges are candidates for projection.

vertices

A sequence of candidate vertices to be projected onto the sketch. By default, all vertices are candidates for projection.

Return value

None

Exceptions

None.

6.1.25 queryCachedStates()

This method displays the position of geometric states relative to the sequence of features in the assembly cache. The output is displayed in the message area.

Arguments

None.

Return value

None

Exceptions

None.

6.1.26 regenerate()

This method regenerates the assembly and brings it up to date with the latest values of the assembly parameters. When you modify features of an assembly, it may be convenient to postpone regeneration until you make all your changes, since regeneration can be time consuming. In contrast, when you modify features of a part that is included in the assembly, you should use this command to regenerate the assembly. When you regenerate the assembly, it will reflect the changes that you made to the part.

Arguments

None.

Return value

None

Exceptions

None.

6.1.27 regenerationWarnings()

This method prints any regeneration warnings associated with the features.

Arguments

None.

Return value

None

Exceptions

None.

6.1.28 restore()

This method restores the parameters of all features in the assembly to the value they had before a failed regeneration. Use the **restore** method after a failed regeneration, followed by a **regenerate** command.

Arguments

None.

Return value

None

Exceptions

None.

6.1.29 resumeAllFeatures()

This method resumes all the suppressed features in the part or assembly.

Arguments

None.

Return value

None

Exceptions

None.

6.1.30 resumeFeatures(...)

This method resumes the specified suppressed features in the assembly.

Arguments

Required argument

featureNames

A sequence of Strings specifying the names of features to resume.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.31 resumeLastSetFeatures()

This method resumes the last set of features to be suppressed in the assembly.

Arguments

None.

Return value

None

Exceptions

None.

6.1.32 rotate(...)

This method rotates given instances by the specified amount.

Arguments

Required arguments

instanceList

A sequence of Strings specifying the names of instances to rotate.

axisPoint

A sequence of three Floats specifying the coordinates of a point on the axis.

axisDirection

A sequence of three Floats specifying the direction of the axis.

angle

A Float specifying the rotation angle in degrees. Use the right-hand rule to determine the direction.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.33 **translate(...)**

This method translates given instances by the specified amount.

Arguments

Required arguments

instanceList

A sequence of Strings specifying the names of instances to translate.

vector

A sequence of three Floats specifying a translation vector.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.34 **saveGeometryCache()**

This method caches the current geometry, which improves regeneration performance.

Arguments

None.

Return value

None

Exceptions

None.

6.1.35 setValues(...)

This method modifies the behavior associated with the specified assembly.

Arguments

Required argument

regenerateConstraintsTogether

A Boolean specifying whether the positioning constraints in the assembly should be regenerated together before regenerating other assembly features. The default value is ON.

If the assembly has position constraint features and you modify the value of *regenerateConstraintsTogether*, Abaqus/CAE will regenerate the assembly features.

Optional arguments

None.

Return value

None

Exceptions

If one or more features in the assembly fails to regenerate:

FeatureError: Regeneration failed

6.1.36 suppressFeatures(...)

This method suppresses specified features.

Arguments

Required argument

featureNames

A sequence of Strings specifying the names of features to suppress in the assembly.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.37 unlinkInstances(...)

This method converts the specified PartInstance objects from linked child instances to regular instances. The parts associated with the selected instances will be converted to regular parts as well.

Arguments**Required argument**

instances

A sequence of PartInstance objects to be converted to regular part instances.

Optional arguments

None.

Return value

None

Exceptions

None.

6.1.38 writeAcisFile(...)

This method exports the assembly to a named file in ACIS format.

Arguments**Required argument**

fileName

A String specifying the name of the file to which to write.

Optional argument

version

A Float specifying the ACIS version. For example, the Float “12.0” corresponds to ACIS Version 12.0. The default value is the current version of ACIS.

Return value

None

Exceptions

None.

6.1.39 writeCADParameters(...)

This method writes the parameters that were imported from the CAD system to a parameter file.

Arguments

Required argument

paramFile

A String specifying the parameter file name.

Optional arguments

modifiedParams

A tuple of tuples each containing the part name, the parameter name and the modified parameter value. Default is an empty tuple.

updatePaths

A Bool specifying whether to update the path of the CAD model file specified in the *parameterFile* to the current directory, if the CAD model is present in the current directory.

Return value

None

Exceptions

None.

6.1.40 lock()

This method locks the assembly. Locking the assembly prevents any further changes to the assembly that can trigger regeneration of the assembly.

Arguments

None.

Return value

None

Exceptions

None.

6.1.41 unlock()

This method unlocks the assembly. Unlocking the assembly allows it to be regenerated after any modifications to the assembly.

Arguments

None.

Return value

None

Exceptions

None.

6.1.42 setMeshNumberingControl(...)

This method changes the start node and/or element labels on the specified independent part instances before or after Abaqus/CAE generates the meshes. For the meshed instances, Abaqus/CAE changes the node and/or element labels while preserving the original order and incrementation.

Arguments**Required argument***instances*

A sequence of PartInstance objects to change the start node and/or element labels.

Optional arguments*startNodeLabel*

A positive Integer specifying the new start node label.

startElemLabel

A positive Integer specifying the new start element label.

Return value

None

Exceptions

None.

6.1.43 **copyMeshPattern(...)**

This method copies a mesh pattern from a source region consisting of a set of shell elements or element faces onto a target face, mapping nodes and elements in a one-one correspondence between source and target.

Arguments

Required arguments

None.

Optional arguments

elements

A sequence of MeshElement objects or a Set object containing elements and specifying the source region.

faces

A sequence of Face objects that have associated with shell elements or element faces and specifying the source region.

elemFaces

A sequence of MeshFace objects specifying the source region.

targetFace

A MeshFace object specifying the target region. The target face can be of a different part instance.

nodes

A sequence of MeshNode objects or a Set object containing nodes on the boundary of source region which are to be positioned to the boundary of target face.

coordinates

A sequence of three-dimensional coordinate tuples specifying the coordinates for each of the given nodes. When specified, the number of coordinate tuples must match the number of given nodes, and be ordered to correspond to the given nodes in *ascending order* according to index. These coordinates are positions of the nodes of a mesh that will be the target face corresponding to nodes provided.

Return value

None

Exceptions

None.

6.1.44 smoothNodes(...)

This method smoothes the given nodes of a native mesh, moving them locally to a more optimal location that improves quality of the mesh

Arguments**Required arguments**

None.

Optional arguments

nodes

A sequence of MeshNode objects or a Set object containing nodes. Nodes that are not part of native mesh only will be smoothed

coordinates

A sequence of MeshNode objects or a Set object containing nodes.

Return value

None

Exceptions

None.

6.1.45 Members

The Assembly object can have the following members:

isOutOfDate

An Int specifying that feature parameters have been modified but that the assembly has not been regenerated. Possible values are 0 and 1.

timeStamp

A Float specifying which gives an indication when the assembly was last modified.

isLocked

An Int specifying whether the assembly is locked or not. Possible values are 0 and 1.

regenerateConstraintsTogether

A Boolean specifying whether the positioning constraints in the assembly should be regenerated together before regenerating other assembly features. The default value is ON.

ASSEMBLY COMMANDS

If the assembly has position constraint features and you modify the value of *regenerateConstraintsTogether*, Abaqus/CAE will regenerate the assembly features.

vertices

A `VertexArray` object specifying all the vertices existing at the assembly level. This member does not provide access to the vertices at the instance level.

edges

An `EdgeArray` object specifying all the edges existing at the assembly level. This member does not provide access to the edges at the instance level.

elements

A `MeshElementArray` object specifying all the elements existing at the assembly level. This member does not provide access to the elements at the instance level.

nodes

A `MeshNodeArray` object specifying all the nodes existing at the assembly level. This member does not provide access to the nodes at the instance level.

instances

A repository of `PartInstance` objects.

datums

A repository of `Datum` objects specifying all `Datum` objects in the assembly.

features

A repository of `Feature` objects specifying all `Feature` objects in the assembly.

featuresById

A repository of `Feature` objects specifying all `Feature` objects in the assembly.

The `Feature` objects in the **featuresById** repository are the same as the `Feature` objects in the **features** repository. However, the key to the objects in the **featuresById** repository is an integer specifying the *ID*, whereas the key to the objects in the **features** repository is a string specifying the *name*.

surfaces

A repository of `Surface` objects specifying for more information, see Chapter 45, “Region commands.”

allSurfaces

A repository of `Surface` objects specifying for more information, see Chapter 45, “Region commands.”

allInternalSurfaces

A repository of `Surface` objects specifying picked regions.

sets

A repository of `Set` objects.

allSets

A repository of `Set` objects specifying for more information, see Chapter 45, “Region commands.”

allInternalSets

A repository of Set objects specifying picked regions.

skins

A repository of Skin objects specifying the skins created on the assembly.

stringers

A repository of Stringer objects specifying the stringers created on the assembly.

referencePoints

A repository of ReferencePoint objects.

modelInstances

A repository of ModelInstance objects.

allinstances

A PartInstance object specifying the PartInstances and A ModelInstance object specifying the ModelInstances.

engineeringFeatures

An EngineeringFeature object.

modelName

A String specifying the name of the model to which the assembly belongs.

connectorOrientations

A ConnectorOrientationArray object.

sectionAssignments

A SectionAssignmentArray object.

6.2 ConnectorOrientation object

The ConnectorOrientation object is used to assign a connector orientation to a connector.

Access

```
import assembly
mdb.models[name].rootAssembly.connectorOrientations[i]
import odbAccess
session.odbs[name].rootAssembly.connectorOrientations[i]
```

6.2.1 ConnectorOrientation(...)

This method creates a ConnectorOrientation object.

Path

```
mdb.models[name].rootAssembly.ConnectorOrientation
```

`session.odbs[name].rootAssembly.ConnectorOrientation`

Required argument

region

A Set object specifying the region to which the orientation is assigned.

Optional arguments

localCsys1

A DatumCsys object specifying the local coordinate system of the first connector point. This value may be None, indicating the global coordinate system.

axis1

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle1

A Float specifying the angle of the additional rotation. The default value is 0.0.

orient2sameAs1

A Boolean specifying whether or not the second connector point is to use the same local coordinate system, axis, and angle as the first point. The default value is ON.

localCsys2

A DatumCsys object specifying the local coordinate system of the second connector point. This value may be None, indicating the global coordinate system.

axis2

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle2

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

A ConnectorOrientation object.

Exceptions

None.

6.2.2 **setValues(...)**

This method modifies the ConnectorOrientation object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConnectorOrientation` method.

Return value

None

Exceptions

None.

6.2.3 Members

The `ConnectorOrientation` object has members with the same names and descriptions as the arguments to the `ConnectorOrientation` method.

6.3 Feature object

The following commands operate on Feature objects. For more information about the Feature object, see “Feature object,” Section 20.1.

Access

```
import assembly
```

6.3.1 AttachmentLines(...)

This method creates a Feature object by creating attachment lines between the given set of source and target faces. The given points are first projected onto the source faces using the specified projection method. The points are then projected normal to the source faces onto the target faces. The user can specify the number of projections or the length of projection vector for projection onto the target faces. The lines are then created between the source face and the closest target face. Subsequent lines are created between the target faces.

Path

```
mdb.models[name].rootAssembly.AttachmentLines
```

Required arguments

name

A String specifying a unique Feature name.

points

A tuple of points. Each point can be a Vertex, Datum point, Reference point, an Attachment point, orphan mesh Node, or an Interesting point object.

sourceFaces

A sequence of Face objects specifying the geometry faces onto which the points are to be projected.

sourceElementFaces

A sequence of MeshFace objects specifying the orphan mesh element faces onto which the points are to be projected.

targetFaces

A sequence of Face objects specifying the geometry faces on which the attachment lines will terminate.

targetElementFaces

A sequence of MeshFace objects specifying the orphan mesh element faces on which the attachment lines will terminate.

Optional arguments

projectionMethod

A SymbolicConstant specifying the method to be used to project onto source faces. Possible values are PROJECT_BY_PROXIMITY and PROJECT_BY_DIRECTION. The default value is PROJECT_BY_PROXIMITY.

projectionDirStartPt

A point specifying the start point of the projection direction to project onto source faces. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting Point object, or a tuple of Floats representing the coordinates of a point.

projectionDirEndPt

A point specifying the end point of the projection direction to project onto source faces. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

sourceToTargetProjMethod

A SymbolicConstant specifying the method to be used to project onto target faces. Possible values are PROJECT_BY_NUMBER and PROJECT_BY_DISTANCE. The default value is PROJECT_BY_NUMBER.

numProjections

An integer specifying the maximum number of layers each point should be projected onto when the source to target projection method is PROJECT_BY_NUMBER.

projectionDistance

A float specifying the maximum distance of the projection vector when the source to target projection method is PROJECT_BY_DISTANCE.

flipSourceToTargetDirection

A Boolean specifying whether the computed projection direction from the source to the target faces should be flipped.

setName

A String specifying a unique set name.

Return value

A Feature object.

Exceptions

None.

6.3.2 Coaxial(...)

This method moves an instance so that its selected face is coaxial with the selected face of a fixed instance.

Path

`mdb.models[name].rootAssembly.Coaxial`

Required arguments*movableAxis*

A cylindrical or conical Face on the part instance to be moved.

fixedAxis

A cylindrical or conical Face on the part instance that remains fixed.

flip

A Boolean specifying whether the axes are forward aligned (OFF) or reverse aligned (ON).

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

6.3.3 CoincidentPoint(...)

This method moves an instance so that a specified point is coincident with a specified point of a fixed instance.

Path

```
mdb.models[name].rootAssembly.CoincidentPoint
```

Required arguments

movablePoint

A Vertex, a Datum point, or a ReferencePoint or a mesh node from an orphan mesh on the part instance to be moved.

fixedPoint

A Vertex, a Datum point, or a ReferencePoint or a mesh node from an orphan mesh on the part instance to remain fixed.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

6.3.4 EdgeToEdge(...)

This method moves an instance so that its edge is parallel to an edge of a fixed instance.

Path

```
mdb.models[name].rootAssembly.EdgeToEdge
```

Required arguments

movableAxis

A straight Edge, a Datum axis, or an element edge from an orphan mesh on the part instance to be moved.

fixedAxis

A straight Edge, a Datum axis, or an element edge from an orphan mesh on the part instance to remain fixed.

flip

A Boolean specifying whether the edges are forward aligned (OFF) or reverse aligned (ON).

clearance

A Float specifying the distance between the two edges (for two-dimensional and axisymmetric instances only).

Optional arguments

None.

Return value

A Feature Object.

Exceptions

AbaqusException.

6.3.5 FaceToFace(...)

This method moves an instance so that its face is coincident with a face of a fixed instance.

Path

`mdb.models [name] .rootAssembly.FaceToFace`

Required arguments

movablePlane

A planar face, a Datum plane, or a face from an orphan mesh on the part instance to be moved.

fixedPlane

A planar face, a Datum plane, or a face from an orphan mesh on the part instance to remain fixed.

flip

A Boolean specifying whether the normals to the faces are forward aligned (OFF) or reverse aligned (ON).

clearance

A Float specifying the distance between the two faces.

Optional arguments

None.

Return value

A Feature Object.

Exceptions

AbaqusException.

6.3.6 ParallelCsys(...)

This method moves an instance so that its Datum coordinate system is parallel to a Datum coordinate system of a fixed instance.

Path

```
mdb.models[name].rootAssembly.ParallelCsys
```

Required arguments

movableCsys

A Datum coordinate system on the part instance to be moved.

fixedCsys

A Datum coordinate system on the part instance to remain fixed.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

6.3.7 ParallelEdge(...)

This method moves an instance so that its edge is parallel to an edge of a fixed instance.

Path

```
mdb.models[name].rootAssembly.ParallelEdge
```

Required arguments

movableAxis

A straight Edge, a Datum axis, or an element edge from an orphan mesh on the part instance to be moved.

fixedAxis

A straight Edge, a Datum axis, or an element edge from an orphan mesh on the part instance to remain fixed.

flip

A Boolean specifying whether the edges are forward aligned (OFF) or reverse aligned (ON).

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

6.3.8 ParallelFace(...)

This method moves an instance so that its face is parallel to a face of a fixed instance.

Path

`mdb.models[name].rootAssembly.ParallelFace`

Required arguments*movablePlane*

A planar face, a Datum plane, or a face from an orphan mesh on the part instance to be moved.

fixedPlane

A planar face, a Datum plane, or a face from an orphan mesh on the part instance to remain fixed.

flip

A Boolean specifying whether the normals to the faces are forward aligned (OFF) or reverse aligned (ON).

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

6.4 PartInstance object

A PartInstance object is an instance of a Part object.

Access

```
import assembly
mdb.models[name].rootAssembly.allinstances
mdb.models[name].rootAssembly.instances[name]
```

6.4.1 Instance(...)

This method creates a PartInstance object and puts it into the instances repository.

Path

```
mdb.models[name].rootAssembly.Instance
```

Required arguments

name

A String specifying the repository key. The name must be a valid Abaqus object name.

part

A Part object to be instanced. If the part does not exist, no PartInstance object is created.

Optional arguments

autoOffset

A Boolean specifying whether to apply an auto offset to the new part instance that will offset it from existing part instances. The default value is OFF.

dependent

A Boolean specifying whether the part instance is dependent or independent. If *dependent*=OFF, the part instance is independent. The default value is OFF.

Return value

A PartInstance object.

Exceptions

None.

6.4.2 InstanceFromBooleanCut(...)

This method creates a PartInstance in the **instances** repository after subtracting or cutting the geometries of a group of part instances from that of a base part instance.

Path

```
mdb.models[name].rootAssembly.InstanceFromBooleanCut
```

Required arguments

name

A String specifying the repository key. The name must be a valid Abaqus object name.

instanceToBeCut

A PartInstance specifying the base instance from which to cut other instances.

cuttingInstances

A sequence of PartInstance objects specifying the instances with which to cut the base instance.

Optional argument

originalInstances

A SymbolicConstant specifying whether the original instances should be suppressed or deleted after the merge operation. Possible values are SUPPRESS or DELETE. The default value is SUPPRESS.

Return value

A PartInstance object.

Exceptions

None.

6.4.3 InstanceFromBooleanMerge(...)

This method creates a PartInstance in the **instances** repository after merging two or more part instances.

Path

```
mdb.models[name].rootAssembly.InstanceFromBooleanMerge
```

Required arguments

name

A String specifying the repository key. The name must be a valid Abaqus object name.

instances

A sequence of PartInstance objects specifying the part instances to merge.

Optional arguments

keepIntersections

A Boolean specifying whether the boundary intersections of Abaqus native part instances should be retained after the merge operation. The default value is False.

originalInstances

A SymbolicConstant specifying whether the original instances should be suppressed or deleted after the merge operation. Possible values are SUPPRESS or DELETE. The default value is SUPPRESS.

domain

A SymbolicConstant specifying whether geometry or mesh of the specified part instances is to be merged. Possible values are GEOMETRY, MESH or BOTH. The default value is GEOMETRY.

mergeNodes

A SymbolicConstant specifying which nodes of the specified part instances should be considered for merging. This argument is only applicable if *domain* is MESH. Possible values are BOUNDARY_ONLY, ALL, or NONE. The default value is BOUNDARY_ONLY.

nodeMergingTolerance

A Float specifying the maximum distance between nodes of the specified part instances that will be merged and replaced with a single node in the new part. The location of the new node is the average position of the deleted nodes. This argument is only applicable if *domain* is MESH. The default value is 10^{-6} .

removeDuplicateElements

A Boolean specifying whether elements with the same connectivity in the new part will be merged into a single element. This argument is only applicable if *domain* is MESH. The default value is True.

Return value

A PartInstance object.

Exceptions

None.

6.4.4 LinearInstancePattern(...)

This method creates multiple PartInstance objects in a linear pattern and puts them into the instances repository.

Path

`mdb.models[name].rootAssembly.LinearInstancePattern`

Required arguments

instanceList

A sequence of Strings specifying the names of instances to pattern.

number1

An Int specifying the total number of instances, including the original instances, that appear along the first direction in the pattern.

spacing1

A Float specifying the spacing between instances along the first direction in the pattern.

number2

An Int specifying the total number of instances, including the original instances, that appear along the second direction in the pattern.

spacing2

A Float specifying the spacing between instances along the second direction in the pattern.

Optional arguments

direction1

A sequence of three Floats specifying a vector along the first direction. The default value is (1.0, 0.0, 0.0).

direction2

A sequence of three Floats specifying a vector along the second direction. The default value is (0.0, 1.0, 0.0).

Return value

A sequence of PartInstance objects.

Exceptions

None.

6.4.5 RadialInstancePattern(...)

This method creates multiple PartInstance objects in a radial pattern and puts them into the instances repository.

Path

`mdb.models[name].rootAssembly.RadialInstancePattern`

Required arguments

instanceList

A sequence of Strings specifying the names of instances to pattern.

number

An Int specifying the total number of instances, including the original instances, that appear in the radial pattern.

totalAngle

A Float specifying the total angle in degrees between the first and last instance in the pattern. A positive angle corresponds to a counter-clockwise direction. The values 360° and -360° represent a special case where the pattern makes a full circle. In this case, because the copy would overlay the original, the copy is not placed at the last position. Possible values are $-360.0 \leq totalAngle \leq 360.0$.

Optional arguments

point

A sequence of three Floats specifying the center of the radial pattern. The default value is (0.0, 0.0, 0.0).

axis

A sequence of three Floats specifying the central axis of the radial pattern. The default value is (0.0, 0.0, 1.0).

Return value

A sequence of PartInstance objects.

Exceptions

None.

6.4.6 checkGeometry(...)

This method checks the validity of the geometry of the part instance and prints a count of all topological entities on the part instance (faces, edges, vertices, etc.).

Arguments

Required arguments

None.

Optional arguments

detailed

A Boolean specifying whether detailed output will be printed to the replay file. The default value is OFF.

level

An Int specifying which level of checking is performed. Values can range from 20 to 70, with higher values reporting less and less important errors. The default value is 20, which reports all critical errors. When the default value is used, the stored validity status is updated to agree with the result of this check.

Return value

None

Exceptions

None.

An exception is thrown if this is a dependent part instance and *level* was either not specified or was set to 20, because the validity status cannot be updated for a dependent part instance. In this case, this command should be called on the Part instead.

The geometry of dependent part instances cannot be changed.

6.4.7 Contact(...)

This method translates an instance along the specified direction until it is in contact with a fixed instance.

Arguments

Required arguments

movableList

A sequence of Face or Edge objects on the part instance to be moved.

fixedList

A sequence of Face or Edge objects on the part instance to remain fixed.

direction

A sequence of three Floats specifying the direction of contact.

clearance

A Float specifying the distance between the two faces along the direction of contact.

Optional argument

isFaceEdges

A Boolean specifying how Abaqus calculates the contact. If *isFaceEdges* is OFF, contact is computed from the movable face to the fixed face. If *isFaceEdges* is ON, contact is computed using only the edges of the movable face and not its interior. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

6.4.8 ConvertConstraints()

This method converts the position constraints of an instance to absolute positions. The method deletes the constraint features on the instance but preserves the position in space.

Arguments

None.

Return value

None

Exceptions

None.

6.4.9 getPosition()

This method prints the sum of the translations and rotations applied to the PartInstance object.

Arguments

None.

Return value

None

Exceptions

None.

6.4.10 getRotation()

This method returns a tuple including the point of rotation, axis of rotation, and rotation angle (in degrees).

Arguments

None.

Return value

A tuple including the point of rotation, axis of rotation, and rotation angle (in degrees).

Exceptions

None.

6.4.11 getTranslation()

This method returns a tuple of three Floats representing translation in the X-, Y-, and Z-directions.

Arguments

None.

Return value

A tuple of three Floats representing the translation.

Exceptions

None.

6.4.12 replace(...)

This method replaces one instance with an instance of another part.

Arguments**Required argument**

instanceOf

A Part object specifying which Part will be instanced in place of the original Part .

Optional argument

applyConstraints

A Boolean specifying whether to apply existing constraints on the new instance or to position the new instance in the same place as the original instance. The default value is True. A value of False indicates that constraints applies to the instance are deleted will be deleted from the feature list.

Return value

None

Exceptions

None.

6.4.13 rotateAboutAxis(...)

This method translates an instance by the specified amount.

Arguments

Required arguments

axisPoint

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of a point on the axis.

axisDirection

A sequence of three Floats specifying the direction vector of the axis.

angle

A Float specifying the rotation angle in degrees. Use the right-hand rule to determine the direction.

Optional arguments

None.

Return value

None

Exceptions

None.

6.4.14 translate(...)

This method translates an instance by the specified amount.

Arguments

Required argument

vector

A sequence of three Floats specifying a translation vector.

Optional arguments

None.

Return value

None

Exceptions

None.

6.4.15 translateTo(...)

This method translates an instance along the specified direction until it is in contact with a fixed instance.

Arguments**Required arguments**

movableList

A sequence of Face or Edge objects on the part instance to be moved.

fixedList

A sequence of Face or Edge objects on the part instances to remain fixed.

direction

A sequence of three Floats specifying the direction of contact.

clearance

A Float specifying the distance between the two faces along the direction of contact.

Optional argument

vector

A sequence of three Floats specifying a translation vector. If this argument is specified, the movable instance will be translated by the specified amount without solving for the actual contact.

Return value

A Feature object.

Exceptions

None.

6.4.16 Members

The PartInstance object can have the following members:

name

A String specifying the repository key. The name must be a valid Abaqus object name.

dependent

A Boolean specifying whether the part instance is dependent or independent. If *dependent*=OFF, the part instance is independent. The default value is OFF.

ASSEMBLY COMMANDS

excludedFromSimulation

A Boolean specifying whether the part instance is excluded from the simulation. If *excludedFromSimulation*=ON, the part instance is excluded from the simulation. The default value is OFF.

geometryValidity

A Boolean specifying the validity of the geometry of the instance. The value is computed, but it can be set to ON to perform feature and mesh operations on an invalid instance. There is no guarantee that such operations will work if the instance was originally invalid.

analysisType

A SymbolicConstant specifying the part type. Possible values are DEFORMABLE_BODY, EULERIAN, DISCRETE_RIGID_SURFACE, and ANALYTIC_RIGID_SURFACE.

referenceNode

An Int specifying the reference node number. This member is valid only if *analysisType*=DISCRETE_RIGID_SURFACE or ANALYTIC_RIGID_SURFACE.

part

A Part object specifying the instanced part.

sets

A repository of Set objects specifying the sets created on the part. For more information, see Chapter 45, “Region commands.”

surfaces

A repository of Surface objects specifying the surfaces created on the part. For more information, see Chapter 45, “Region commands.”

skins

A repository of Skin objects specifying the skins created on the part. For more information, see Chapter 45, “Region commands.”

stringers

A repository of Stringer objects specifying the stringers created on the part. For more information, see Chapter 45, “Region commands.”

vertices

A VertexArray object.

ignoredVertices

An IgnoredVertexArray object.

edges

An EdgeArray object.

ignoredEdges

An IgnoredEdgeArray object.

faces

A FaceArray object.

cells

A CellArray object.

datums

A repository of Datum objects.

elements

A MeshElementArray object.

nodes

A MeshNodeArray object.

elemFaces

A repository of MeshFace objects specifying all the element faces in the part instance. For a given element and a given face index within that element, the corresponding MeshFace object can be retrieved from the repository by using the key calculated as $(i*8 + j)$, where i and j are zero-based element and face indices, respectively.

elementFaces

A MeshFaceArray object.

elemEdges

A repository of MeshEdge objects specifying all the element edges in the part instance. For a given element and a given edge index on a given face within that element, the corresponding MeshEdge object can be retrieved from the repository by using the key calculated as $(i*32 + j*4 + k)$, where i , j , and k are zero-based element, face, and edge indices, respectively.

elementEdges

A MeshEdgeArray object.

referencePoints

A repository of ReferencePoint objects.

partName

A String specifying the name of the part from which the instance was created.

6.5 ModellInstance object

A ModellInstance object is an instance of a Model.

Access

```
import assembly
mdb.models[name].rootAssembly.modelInstances[i]
```

6.5.1 Instance(...)

This method creates a ModellInstance object and puts it into the instances repository.

Path

`mdb.models[name].rootAssembly.Instance`

Required arguments

name

The repository key. The name must be a valid Abaqus object name.

model

A Model object to be instantiated. If the model does not exist, no ModelInstance object is created.

Optional argument

autoOffset

A Boolean specifying whether to apply an auto offset to the new instance that will offset it from existing instances. The default value is OFF.

Return value

A ModelInstance object.

Exceptions

None.

6.5.2 ConvertConstraints()

This method converts the position constraints of an instance to absolute positions. The method deletes the constraint features on the instance but preserves the position in space.

Arguments

None.

Return value

None

Exceptions

None.

6.5.3 getPosition()

This method prints the sum of the translations and rotations applied to the ModelInstance object.

Arguments

None.

Return value

None

Exceptions

None.

6.5.4 translate(...)

This method translates an instance by the specified amount.

Arguments**Required argument**

vector

A sequence of three Floats specifying a translation vector.

Optional arguments

None.

Return value

None

Exceptions

None.

6.5.5 Members

The ModelInstance object can have the following members:

sets

A repository of Set objects specifying the sets created on the assembly. For more information, see Chapter 45, “Region commands.”

surfaces

A repository of Surface objects specifying the surfaces created on the assembly. For more information, see Chapter 45, “Region commands.”

vertices

A VertexArray object.

edges

An EdgeArray object.

elements

A MeshElementArray object.

nodes

A MeshNodeArray object.

datums

A repository of Datum objects.

referencePoints

A repository of ReferencePoint objects.

6.6 Model object

The following commands operate on Model objects. For more information about the Model object, see “Model object,” Section 33.1.

Access

```
import assembly
```

6.6.1 Instance(...)

This method copies a PartInstance object from the specified model and creates a new PartInstance object.

Path

```
mdb.models[name].Instance
```

Required arguments

name

A String specifying the repository key.

objectToCopy

A PartInstance object to be copied.

Optional arguments

None.

Return value

A Model object.

Exceptions

None.

6.6.2 convertAllSketches(...)

This method converts all sketches from Abaqus 6.5 or earlier to the equivalent ConstrainedSketch objects.

Arguments**Required arguments**

None.

Optional arguments*regenerate*

A Boolean specifying if all the features in assembly as well as in all the parts in the model should be regenerated after the conversion. The default value is True.

convertReversedSketches

A Boolean specifying whether sketches in analytic rigid parts should be converted even if they cause the orientation of surfaces defined on them to be flipped. The default value is True.

Return value

A list of strings describing any warnings or errors encountered during the conversion process.

Exceptions

None.

6.6.3 linkInstances(...)

This method links the selected PartInstance objects to the corresponding PartInstance objects from the specified models. If all instances of a Part are selected for linking, the Part will be linked as well. If not, a new linked child Part object will be created and added to the repository.

Arguments**Required argument***instancesMap*

A tuple of tuples containing the instance name to be linked and the corresponding PartInstance object to which it will be linked.

Optional arguments

None.

ASSEMBLY COMMANDS

Return value

A list of strings describing any warnings or errors encountered during the conversion process.

Exceptions

None.

7. Basic geometry commands

The basic geometry commands create geometric entities that are referenced by Part objects, Instance objects, and GeometrySet objects.

7.1 Cell object

Cells are volumetric regions of geometry.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].cells[i]
mdb.models[name].parts[name].allSets[name].cells[i]
mdb.models[name].parts[name].cells[i]
mdb.models[name].parts[name].sets[name].cells[i]
import assembly
mdb.models[name].rootAssembly.allinstances.cells[i]
mdb.models[name].rootAssembly.allinstances.sets[name].cells[i]
mdb.models[name].rootAssembly.allInternalSets[name].cells[i]
mdb.models[name].rootAssembly.allSets[name].cells[i]
mdb.models[name].rootAssembly.instances[name].cells[i]
mdb.models[name].rootAssembly.instances[name].sets[name].cells[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].cells[i]
mdb.models[name].rootAssembly.sets[name].cells[i]
```

7.1.1 getSize(...)

This method returns a Float indicating the volume of the cell.

Arguments

Required arguments

None.

Optional argument

printResults

A Boolean that determines whether a verbose output is to be printed. The default is True.

Return value

A Float.

Exceptions

None.

7.1.2 **getFaces()**

This method returns a sequence consisting of the face IDs of the faces which bound the cell.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.1.3 **getEdges()**

This method returns a sequence consisting of the edge IDs of the edges on the cell.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.1.4 **getVertices()**

This method returns a sequence consisting of the vertex IDs of the vertices on the cell.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.1.5 getAdjacentCells()

This method returns an array of cell objects that share at least one face of the cell.

Arguments

None.

Return value

A CellArray object which is a sequence of Cell objects.

Exceptions

None.

7.1.6 getNodes()

This method returns an array of node objects that are associated with the cell.

Arguments

None.

Return value

A MeshNodeArray object which is a sequence of MeshNode objects.

Exceptions

None.

7.1.7 **getElements()**

This method returns an array of element objects that are associated with the cell.

Arguments

None.

Return value

A MeshElementArray object which is a sequence of MeshElement objects.

Exceptions

None.

7.1.8 **Members**

The Cell object has the following members:

index

An Int specifying the index of the cell in the CellArray.

isReferenceRep

A Boolean specifying whether the cell belongs to the reference representation of the Part or Instance.

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the cell.

featureName

A tuple of Floats specifying the name of the feature that created this cell.

instanceName

A tuple of Floats specifying the name of the part instance for this cell (if applicable).

7.2 **CellArray object**

The CellArray is a sequence of Cell objects.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].cells
mdb.models[name].parts[name].allSets[name].cells
mdb.models[name].parts[name].cells
```

```

mdb.models[name].parts[name].sets[name].cells
import assembly
mdb.models[name].rootAssembly.allinstances.cells
mdb.models[name].rootAssembly.allinstances.sets[name].cells
mdb.models[name].rootAssembly.allInternalSets[name].cells
mdb.models[name].rootAssembly.allSets[name].cells
mdb.models[name].rootAssembly.instances[name].cells
mdb.models[name].rootAssembly.instances[name].sets[name].cells
mdb.models[name].rootAssembly.modelInstances[i].sets[name].cells
mdb.models[name].rootAssembly.sets[name].cells

```

7.2.1 CellArray(...)

This method creates a CellArray object.

Path

```
part.CellArray
```

Required argument

cells

A list of Cell objects.

Optional arguments

None.

Return value

A CellArray object.

Exceptions

None.

7.2.2 findAt(...)

This method returns the object or objects in the CellArray located at the given coordinates. **findAt** initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any entity that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities). The arbitrary point must not be shared by a second cell. If two cells intersect or coincide at the arbitrary point, **findAt** chooses the first cell that it encounters, and you should not rely on the return value being consistent.

BASIC GEOMETRY COMMANDS

findAt will always try to find objects among all the cells in the part or assembly instance and will not restrict itself to a subset even if the CellArray represents such subset.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either a Cell object or a sequence of Cell objects based on the type of input. If *coordinates* is a sequence of Floats, **findAt** returns the Cell object at that point. If *coordinates* is a sequence of sequence of Floats, **findAt** returns a sequence of Cell objects at the given locations. The sequence of sequence of Floats must be a sequence of sequence of point and normal data, not a sequence of point data. For example,

```
cells1 = myCrackedBlockInstance.cells.findAt(  
    ((5.5, -8.3, 294.2)),  
    ((12.1, -8.3, 287.8)),  
    ((14.4, -10.4, 285.5)),)
```

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

A Cell object.

Exceptions

None.

7.2.3 getSequenceFromMask(...)

This method returns the object or objects in the CellArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When large number of objects are involved, this method is highly efficient.

Arguments

Required argument

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A Cell object or a sequence of Cell objects.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.2.4 getMask()

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.2.5 getByBoundingBox(...)

This method returns an array of cell objects that lie within the specified bounding box.

Arguments**Required arguments**

None.

Optional arguments

xMin

A float specifying the minimum *X*-boundary of the bounding box.

yMin

A float specifying the minimum *Y*-boundary of the bounding box.

zMin

A float specifying the minimum *Z*-boundary of the bounding box.

xMax

A float specifying the maximum *X*-boundary of the bounding box.

BASIC GEOMETRY COMMANDS

yMax

A float specifying the maximum Y-boundary of the bounding box.

zMax

A float specifying the maximum Z-boundary of the bounding box.

Return value

A CellArray object, which is a sequence of Cell objects.

Exceptions

None.

7.2.6 getByBoundingCylinder(...)

This method returns an array of cell objects that lie within the specified bounding cylinder.

Arguments

Required arguments

center1

A tuple of the X-, Y-, and Z-coordinates of the center of the first end of the cylinder.

center2

A tuple of the X-, Y-, and Z-coordinates of the center of the second end of the cylinder.

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

A CellArray object, which is a sequence of Cell objects.

Exceptions

None.

7.2.7 getByBoundingSphere(...)

This method returns an array of cell objects that lie within the specified bounding sphere.

Arguments**Required arguments***center*

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

A CellArray object, which is a sequence of Cell objects.

Exceptions

None.

7.2.8 getBoundingBox()

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the cell sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum X-, Y-, and Z-boundary values of the bounding box.

high: a tuple of three floats representing the maximum X-, Y-, and Z-boundary values of the bounding box.

Exceptions

None.

7.2.9 Members

The CellArray object has no members.

7.3 Edge object

Edges are one-dimensional regions of geometry.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].edges[i]
mdb.models[name].parts[name].allInternalSurfaces[name].edges[i]
mdb.models[name].parts[name].allSets[name].edges[i]
mdb.models[name].parts[name].allSurfaces[name].edges[i]
mdb.models[name].parts[name].edges[i]
mdb.models[name].parts[name].sets[name].edges[i]
mdb.models[name].parts[name].surfaces[name].edges[i]
import assembly
mdb.models[name].rootAssembly.allinstances.edges[i]
mdb.models[name].rootAssembly.allinstances.sets[name].edges[i]
mdb.models[name].rootAssembly.allinstances.surfaces[name].edges[i]
mdb.models[name].rootAssembly.allInternalSets[name].edges[i]
mdb.models[name].rootAssembly.allInternalSurfaces[name].edges[i]
mdb.models[name].rootAssembly.allSets[name].edges[i]
mdb.models[name].rootAssembly.allSurfaces[name].edges[i]
mdb.models[name].rootAssembly.edges[i]
mdb.models[name].rootAssembly.instances[name].edges[i]
mdb.models[name].rootAssembly.instances[name].sets[name].edges[i]
mdb.models[name].rootAssembly.instances[name].surfaces[name].edges[i]
mdb.models[name].rootAssembly.modelInstances[i].edges[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].edges[i]
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name]\
.edges[i]
mdb.models[name].rootAssembly.sets[name].edges[i]
mdb.models[name].rootAssembly.surfaces[name].edges[i]
```

7.3.1 isTangentFlipped()

This method determines whether the tangent to the edge is flipped from its default direction by the use of the **flipTangent** method on a Part object.

Arguments

None.

Return value

A Boolean value of True if the tangent is flipped and False if not.

Exceptions

None.

7.3.2 getCurvature(...)

This method returns curvature information at a location on the edge.

Arguments**Required arguments**

parameter

A Float specifying the normalized parameter location on the edge where the curvature is to be computed. This argument is mutually exclusive with the argument *point*.

point

A tuple of X-, Y-, and Z-coordinates of a point at which the curvature is to be computed. If *point* does not lie on the edge an attempt is made to project it onto the edge and use the projected point.

Optional arguments

None.

Return value

A dictionary with keys 'evaluationPoint', 'curvature', 'radius', and 'tangent', where 'evaluationPoint' specifies the location at which the curvature was computed; 'curvature' specifies the curvature vector at that location; 'radius' is the radius of curvature; and 'tangent' specifies the tangent to the edge at that location.

Exceptions

The given edge is straight.

7.3.3 getFaces()

This method returns a sequence consisting of the face ids of the faces which share this edge.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.3.4 **getAdjacentEdges()**

This method returns an array of Edge objects that share at least one vertex of the edge.

Arguments

None.

Return value

An EdgeArray object, which is a sequence of Edge objects.

Exceptions

None.

7.3.5 **getEdgesByEdgeAngle(...)**

This method returns an array of Edge objects that are obtained by recursively finding adjacent edges that are at an angle of less than or equal to the specified face angle.

Arguments**Required argument**

angle

A float specifying the value of the face angle in degrees.

Optional arguments

None.

Return value

An EdgeArray object, which is a sequence of Edgeobjects.

Exceptions

None.

7.3.6 **getNodes()**

This method returns an array of node objects that are associated with the edge.

Arguments

None.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

7.3.7 **getElements()**

This method returns an array of element objects that are associated with the edge.

Arguments

None.

Return value

A MeshElementArray object which is a sequence of MeshElement objects.

Exceptions

None.

7.3.8 **getRadius()**

This method returns the radius of circular edges.

Arguments

None.

Return value

A Float specifying the radius.

Exceptions

The given edges is not circular.

7.3.9 **getSize(...)**

This method returns a Float indicating the length of the edge.

Arguments

Required arguments

None.

Optional argument

printResults

A Bool specifying whether verbose output is printed. The default is True.

Return value

A Float.

Exceptions

None.

7.3.10 **getVertices()**

This method returns a sequence of indices of the vertices that bound this edge. The first index refers to the vertex where the normalized curve parameter = 0.0, and the second index refers to the vertex where the normalized curve parameter = 1.0. If the edge is a closed curve, only one vertex index is returned.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.3.11 **Members**

The Edge object has the following members:

index

An Int specifying the index of the edge in the EdgeArray.

isReferenceRep

A Boolean specifying whether the edge belongs to the reference representation of the Part or Instance.

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the edge.

featureName

A tuple of Floats specifying the name of the feature that created this edge.

instanceName

A tuple of Floats specifying the name of the part instance for this edge (if applicable).

7.4 EdgeArray object

The EdgeArray is a sequence of Edge objects. If the part is modified, then EdgeArray must be updated for that part.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].edges
mdb.models[name].parts[name].allInternalSurfaces[name].edges
mdb.models[name].parts[name].allSets[name].edges
mdb.models[name].parts[name].allSurfaces[name].edges
mdb.models[name].parts[name].edges
mdb.models[name].parts[name].sets[name].edges
mdb.models[name].parts[name].surfaces[name].edges
import assembly
mdb.models[name].rootAssembly.allinstances.edges
mdb.models[name].rootAssembly.allinstances.sets[name].edges
mdb.models[name].rootAssembly.allinstances.surfaces[name].edges
mdb.models[name].rootAssembly.allInternalSets[name].edges
mdb.models[name].rootAssembly.allInternalSurfaces[name].edges
mdb.models[name].rootAssembly.allSets[name].edges
mdb.models[name].rootAssembly.allSurfaces[name].edges
mdb.models[name].rootAssembly.edges
mdb.models[name].rootAssembly.instances[name].edges
mdb.models[name].rootAssembly.instances[name].sets[name].edges
mdb.models[name].rootAssembly.instances[name].surfaces[name].edges
mdb.models[name].rootAssembly.modelInstances[i].edges
```

```
mdb.models[name].rootAssembly.modelInstances[i].sets[name].edges
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name].edges
mdb.models[name].rootAssembly.sets[name].edges
mdb.models[name].rootAssembly.surfaces[name].edges
```

7.4.1 EdgeArray(...)

This method creates an EdgeArray object.

Path

`part.EdgeArray`

Required argument

edges

A list of Edge objects.

Optional arguments

None.

Return value

A EdgeArray object.

Exceptions

None.

7.4.2 findAt(...)

This method returns the object or objects in the EdgeArray located at the given coordinates.

findAt initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any edge that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities). The arbitrary point must not be shared by a second edge. If two edges intersect or coincide at the arbitrary point, **findAt** chooses the first edge that it encounters, and you should not rely on the return value being consistent.

findAt will always try to find objects among all the edges in the part or assembly instance and will not restrict itself to a subset even if the EdgeArray represents such subset.

Arguments**Required argument***coordinates*

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either an Edge object or a sequence of Edge objects based on the type of input.

- If *coordinates* is a sequence of Floats, **findAt** returns the Edge object at that point.
- If you omit the *coordinates* keyword argument, **findAt** accepts as arguments a sequence of sequence of floats in the following format:

```
edges = e.findAt(((20.19686, -169.513997, 27.798593), ),
                 ((19.657627, -167.295749, 27.056402), ),
                 ((18.274129, -157.144741, 25.15218), ))
```

Optional argument*printWarning*

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

An Edge object or a sequence of Edge objects.

Exceptions

None.

7.4.3 getClosest(...)

This method returns an object or objects in the EdgeArray closest to the given set of points, where the given points need not lie on the edges in the EdgeArray.

Arguments**Required argument***coordinates*

A sequence of a sequence of floats, where each sequence of floats describes the X-, Y-, and Z-coordinates of a point.

```
r=e.getClosest(coordinates=((20.0,20.0,10.0),(-1.0, -15.0, 15),))

r.keys()
```

```
[0, 1]
```

```
r[0]
```

```
(mdb.models['Model-1'].parts['Part-1'].edges[3],  
    (15.7090625762939, 20.0, 10.0))
```

Optional argument

searchTolerance

A double specifying the distance within which the closest object must lie. The default value is half of the parent part/instance size.

Return value

This method returns a dictionary object. The key to the dictionary object is the position of the input point in the tuple specified in the *coordinates* starting at index 0. If a closest edge could be found then the value is a sequence consisting of two objects. The first object in the sequence is an Edge that is close to the input point referred to by the key. The second object in the sequence is a sequence of floats that specifies the X-, Y-, and Z-location of the closest point on the Edge to the given point. See program listing above.

Exceptions

None.

7.4.4 getSequenceFromMask(...)

This method returns the object or objects in the EdgeArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When a large number of objects are involved, this method is highly efficient.

Arguments

Required argument

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

An Edge object or a sequence of Edge objects.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.4.5 getMask()

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.4.6 getByBoundingBox(...)

This method returns an array of edge objects that lie within the specified bounding box.

Arguments**Required arguments**

None.

Optional arguments

xMin

A float specifying the minimum X-boundary of the bounding box.

yMin

A float specifying the minimum Y-boundary of the bounding box.

zMin

A float specifying the minimum Z-boundary of the bounding box.

xMax

A float specifying the maximum X-boundary of the bounding box.

yMax

A float specifying the maximum Y-boundary of the bounding box.

zMax

A float specifying the maximum Z-boundary of the bounding box.

Return value

An EdgeArray object, which is a sequence of Edge objects.

Exceptions

None.

7.4.7 getByBoundingCylinder(...)

This method returns an array of edge objects that lie within the specified bounding cylinder.

Arguments

Required arguments

center1

A tuple of the X-, Y-, and Z-coordinates of the center of the first end of the cylinder.

center2

A tuple of the X-, Y-, and Z-coordinates of the center of the second end of the cylinder.

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

An EdgeArray object, which is a sequence of Edge objects.

Exceptions

None.

7.4.8 getByBoundingSphere(...)

This method returns an array of edge objects that lie within the specified bounding sphere.

Arguments

Required arguments

center

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

An EdgeArray object, which is a sequence of Edge objects.

Exceptions

None.

7.4.9 getBoundingBox()

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the edge sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum X-, Y-, and Z-boundary values of the bounding box.

high: a tuple of three floats representing the maximum X-, Y-, and Z-boundary values of the bounding box.

Exceptions

None.

7.4.10 Members

The EdgeArray object has no members.

7.5 Face object

Faces are two-dimensional regions of geometry.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].faces[i]
```

```

mdb.models[name].parts[name].allInternalSurfaces[name].faces[i]
mdb.models[name].parts[name].allSets[name].faces[i]
mdb.models[name].parts[name].allSurfaces[name].faces[i]
mdb.models[name].parts[name].faces[i]
mdb.models[name].parts[name].sets[name].faces[i]
mdb.models[name].parts[name].surfaces[name].faces[i]
import assembly
mdb.models[name].rootAssembly.allinstances.faces[i]
mdb.models[name].rootAssembly.allinstances.sets[name].faces[i]
mdb.models[name].rootAssembly.allinstances.surfaces[name].faces[i]
mdb.models[name].rootAssembly.allInternalSets[name].faces[i]
mdb.models[name].rootAssembly.allInternalSurfaces[name].faces[i]
mdb.models[name].rootAssembly.allSets[name].faces[i]
mdb.models[name].rootAssembly.allSurfaces[name].faces[i]
mdb.models[name].rootAssembly.instances[name].faces[i]
mdb.models[name].rootAssembly.instances[name].sets[name].faces[i]
mdb.models[name].rootAssembly.instances[name].surfaces[name].faces[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].faces[i]
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name]\
    .faces[i]
mdb.models[name].rootAssembly.sets[name].faces[i]
mdb.models[name].rootAssembly.surfaces[name].faces[i]

```

7.5.1 getCentroid()

This method returns the centroid of a face.

Arguments

None.

Return value

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the centroid of the face.

Exceptions

None.

7.5.2 getCurvature(...)

This method returns information about the curvature at a location on the face.

Arguments**Required arguments***point*

A tuple specifying the X-, Y-, and Z coordinates of the point where the curvature is desired. If the *point* does not lie on the face it will be projected onto the face. This argument and *uParam* and *vParam* are mutually exclusive.

uParam

A Float specifying the normalized *U* parameter value at which the curvature is to be computed. This value must lie between (0,1). *vParam* must also be specified. This argument is mutually exclusive with *point*.

vParam

A Float specifying the normalized *V* parameter value at which the curvature is to be computed. This value must lie between (0,1).

Optional arguments

None.

Return value

A dictionary with keys 'evaluationPoint', 'principalAxis1', 'principalAxis2', 'curvature1', 'curvature2' and 'gaussianCurvature'. Where the evaluationPoint specifies the location at which the curvature was evaluated. 'principalAxis1' and 'principalAxis2' refer to the vectors specifying the two principal axes of the face. 'curvature1' and 'curvature2' specify the curvatures along the two principal axes.

Exceptions

None.

7.5.3 getElements()

This method returns an array of element objects that are associated with the face.

Arguments

None.

Return value

A MeshElementArray object which is a sequence of MeshElement objects.

Exceptions

None.

7.5.4 **getElementFaces(...)**

This method returns an array of mesh face objects. Each mesh face object contains the element label and the side of the element that lies on the geometric face.

Arguments

Required arguments

None.

Optional argument

faceSide

A symbolic constant specifying the side of the geometric face of a solid from which the element faces should be retrieved. Possible values are *SIDE1*, *SIDE2*, and *BOTH_SIDES*. The default value is *BOTH_SIDES*. For shell faces this option is ignored.

Return value

A MeshFaceArray object which is a sequence of MeshFace objects.

Exceptions

None.

7.5.5 **getNodes(...)**

This method returns an array of mesh node objects. Each mesh node object contains the label of the node that lies on the geometric face.

Arguments

Required arguments

None.

Optional argument

faceSide

A symbolic constant specifying the side of the geometric face of a solid from which the nodes should be retrieved. Possible values are *xoSIDE1*, *SIDE2*, and *BOTH_SIDES*. The default value is *BOTH_SIDES*. For shell faces and for faces with compatible meshes on either sides this option is ignored. Otherwise, the nodes on the specified side(s) of the face are output.

Return value

A MeshNodeArray object which is a sequence of MeshNode objects.

Exceptions

None.

7.5.6 getNormal(...)

This method returns the normal to a face at the location specified by the *pointOn* member. The normal at a different location on the face can be obtained by specifying the optional *point* argument.

Arguments**Required arguments**

None.

Optional argument

point

A tuple specifying the X-, Y-, and Z coordinates of the point where the normal to the face is desired. If the *point* does not lie on the face it will be projected onto the face.

Return value

A sequence of Floats specifying the X-, Y-, and Z-components of the normal to the face.

Exceptions

None.

An exception is raised if the optional argument *point* is provided but the point cannot be projected onto the face.

7.5.7 getSize(...)

This method returns a Float indicating the area of the face.

Arguments**Required arguments**

None.

Optional argument

printResults

A Bool specifying whether verbose output is printed. The default value is True.

Return value

A Float.

Exceptions

None.

7.5.8 getEdges()

This method returns a sequence consisting of the edge ids of the edges on the face.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.5.9 getVertices()

This method returns a sequence consisting of the vertex ids of the vertices of the face.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.5.10 getCells()

This method returns a sequence consisting of the cell ids of the cells to which this face belongs.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.5.11 getAdjacentFaces()

This method returns an array of face objects that share at least one edge of the face.

Arguments

None.

Return value

A FaceArray object which is a sequence of Face objects.

Exceptions

None.

7.5.12 getFacesByFaceAngle(...)

This method returns an array of Face objects that are obtained by recursively finding adjacent faces that are at an angle of less than or equal to the specified angle.

Arguments**Required argument**

angle

A float specifying the value of the face angle.

Optional arguments

None.

Return value

A FaceArray object, which is a sequence of Face objects.

Exceptions

None.

7.5.13 **getFacesByCurvature()**

This method returns an array of Face objects that are obtained by recursively finding adjacent faces that share the same curvature.

Arguments

None.

Return value

A FaceArray object, which is a sequence of Face objects.

Exceptions

None.

7.5.14 **isNormalFlipped()**

This method determines whether the normal to the face is flipped from its default direction by the use of the **flipNormal** method on a Part object.

Arguments

None.

Return value

A Boolean value of True if the normal is flipped and False if not.

Exceptions

None.

7.5.15 **Members**

The Face object has the following members:

index

An Int specifying the index of the face in the FaceArray.

isReferenceRep

A Boolean specifying whether the face belongs to the reference representation of the Part or Instance.

pointOn

A tuple of tuples of Floats specifying the coordinates. For a face of a shell *pointOn* specifies the X-, Y-, and Z-coordinates of a point located on the face and the X-, Y-, and Z-components of the normal to the face.

For a face of a solid *pointOn* specifies the X-, Y-, and Z-coordinates of a point located on the face.

featureName

A tuple of Floats specifying the name of the feature that created this face.

instanceName

A tuple of Floats specifying the name of the part instance for this face (if applicable).

7.6 FaceArray object

The FaceArray is a sequence of Face objects. If the part is modified, then FaceArray must be updated for that part.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].faces
mdb.models[name].parts[name].allInternalSurfaces[name].faces
mdb.models[name].parts[name].allSets[name].faces
mdb.models[name].parts[name].allSurfaces[name].faces
mdb.models[name].parts[name].faces
mdb.models[name].parts[name].sets[name].faces
mdb.models[name].parts[name].surfaces[name].faces
import assembly
mdb.models[name].rootAssembly.allinstances.faces
mdb.models[name].rootAssembly.allinstances.sets[name].faces
mdb.models[name].rootAssembly.allinstances.surfaces[name].faces
mdb.models[name].rootAssembly.allInternalSets[name].faces
mdb.models[name].rootAssembly.allInternalSurfaces[name].faces
mdb.models[name].rootAssembly.allSets[name].faces
mdb.models[name].rootAssembly.allSurfaces[name].faces
mdb.models[name].rootAssembly.instances[name].faces
mdb.models[name].rootAssembly.instances[name].sets[name].faces
mdb.models[name].rootAssembly.instances[name].surfaces[name].faces
mdb.models[name].rootAssembly.modelInstances[i].sets[name].faces
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name].faces
mdb.models[name].rootAssembly.sets[name].faces
```

```
mdb.models [name] .rootAssembly .surfaces [name] .faces
```

7.6.1 FaceArray(...)

This method creates a FaceArray object.

Path

```
part.FaceArray
```

Required argument

faces

A list of Face objects.

Optional arguments

None.

Return value

A FaceArray object.

Exceptions

None.

7.6.2 findAt(...)

This method returns the object or objects in the FaceArray located at the given coordinates.

findAt initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any face that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities). The arbitrary point must not be shared by a second face. If two faces intersect or coincide at the arbitrary point, **findAt** chooses the first face that it encounters, and you should not rely on the return value being consistent.

findAt will always try to find objects among all the faces in the part or assembly instance and will not restrict itself to a subset even if the FaceArray represents such subset.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either a Face object or a sequence of Face objects based on the type of input.

- If *coordinates* is a sequence of Floats, **findAt** returns the Face object at that point.

- If you omit the *coordinates* keyword argument, **findAt** accepts as arguments a sequence of pairs of sequences describing each face's coordinate and normal, and **findAt** returns a sequence of Face objects at the given locations. If you omit the *coordinates* keyword argument, you must also omit the *normal* argument.

```
faces = f.findAt((-16.438578, -41.835673, -24.19804), ),
              ((25.210364, -35.689868, 1.860314), ),
              ((26.727683, -38.207055, 4.164759), ))
```

Optional arguments

normal

A sequence of Floats specifying the X-, Y-, and Z-components of a vector indicating the face normal.

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

A Face object.

Exceptions

None.

7.6.3 getSequenceFromMask(...)

This method returns the object or objects in the FaceArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When a large number of objects are involved, this method is highly efficient.

Arguments

Required argument

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A Face object or a sequence of Face objects.

Exceptions

None.

7.6.4 **getMask()**

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.6.5 **getByBoundingBox(...)**

This method returns an array of face objects that lie within the specified bounding box.

Arguments

Required arguments

None.

Optional arguments

xMin

A float specifying the minimum *X*-boundary of the bounding box.

yMin

A float specifying the minimum *Y*-boundary of the bounding box.

zMin

A float specifying the minimum *Z*-boundary of the bounding box.

xMax

A float specifying the maximum *X*-boundary of the bounding box.

yMax

A float specifying the maximum *Y*-boundary of the bounding box.

zMax

A float specifying the maximum *Z*-boundary of the bounding box.

Return value

A FaceArray object, which is a sequence of Face objects.

Exceptions

None.

7.6.6 getByBoundingCylinder(...)

This method returns an array of face objects that lie within the specified bounding cylinder.

Arguments**Required arguments**

center1

A tuple of the X-, Y-, and Z-coordinates of the center of the first end of the cylinder.

center2

A tuple of the X-, Y-, and Z-coordinates of the center of the second end of the cylinder.

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

A FaceArray object, which is a sequence of Face objects.

Exceptions

None.

7.6.7 getByBoundingSphere(...)

This method returns an array of face objects that lie within the specified bounding sphere.

Arguments**Required arguments**

center

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

A FaceArray object, which is a sequence of Face objects.

Exceptions

None.

7.6.8 getBoundingBox()

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the face sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum X-, Y-, and Z-boundary values of the bounding box.

high: a tuple of three floats representing the maximum X-, Y-, and Z-boundary values of the bounding box.

Exceptions

None.

7.6.9 getClosest(...)

This method returns an object or objects in the FaceArray closest to the given set of points, where the given points need not lie on the faces in the FaceArray.

Arguments

Required argument

coordinates

A sequence of a sequence of floats, where each sequence of floats describes the X-, Y-, and Z-coordinates of a point.

```
r=f.getClosest(coordinates=((20.0,20.0,10.0),(-1.0, -15.0, 15),))
```



```

r.keys()

[0, 1]

r[0]

(mdb.models['Model-1'].parts['Part-1'].faces[0],
 (15.7090625762939, 20.0, 10.0))

```

Optional argument*searchTolerance*

A double specifying the distance within which the closest object must lie. The default value is half of the parent part/instance size.

Return value

This method returns a dictionary object. The key to the dictionary object is the position of the input point in the tuple specified in the *coordinates* starting at index 0. If a closest face could be found then the value is a sequence consisting of two objects. The first object in the sequence is a Face that is close to the input point referred to by the key. The second object in the sequence is a sequence of floats that specifies the X-, Y-, and Z-location of the closest point on the Face to the given point. See program listing above.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.6.10 Members

The FaceArray object has no members.

7.7 IgnoredEdge object

An IgnoredEdge object is a one-dimensional region of geometry that has been abstracted away by a virtual topology feature.

Access

```

import part
mdb.models[name].parts[name].ignoredEdges[i]
import assembly

```

```
mdb.models[name].rootAssembly.allinstances.ignoredEdges[i]  
mdb.models[name].rootAssembly.instances[name].ignoredEdges[i]
```

7.7.1 **getSize(...)**

This method returns a Float indicating the length of the edge.

Arguments

Required arguments

None.

Optional argument

printResults

A Bool specifying whether verbose output is printed. The default is True.

Return value

A Float.

Exceptions

None.

7.7.2 **getRadius()**

This method returns the radius of a circular IgnoredEdge object.

Arguments

None.

Return value

A Float specifying the radius.

Exceptions

The given IgnoredEdge object is not circular.

7.7.3 **getCurvature(...)**

This method returns curvature information at a location on the IgnoredEdge object.

Arguments**Required arguments***parameter*

A Float specifying the normalized parameter location on the IgnoredEdge where the curvature is to be computed. This argument is mutually exclusive with the argument *point*.

point

A tuple of X-, Y-, and Z-coordinates of a point at which the curvature is to be computed. If *point* does not lie on the IgnoredEdge an attempt is made to project it onto the IgnoredEdge and use the projected point.

Optional arguments

None.

Return value

A dictionary with keys 'evaluationPoint', 'curvature', 'radius', 'tangent'. Where 'evaluationPoint' specifies the location at which the curvature was computed. 'curvature' specifies the curvature vector at that location. 'radius' is the Radius of curvature and 'tangent' specifies the tangent to the IgnoredEdge at that location.

Exceptions

The given IgnoredEdge is straight.

7.7.4 Members

The IgnoredEdge object has the following members:

index

An Int specifying the index of the IgnoredEdge in the IgnoredEdgeArray .

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the edge.

7.8 IgnoredEdgeArray object

The IgnoredEdgeArray is a sequence of IgnoredEdge objects. If the part is modified, then IgnoredEdgeArray must be updated for that part.

Access

```
import part
```

```
mdb.models[name].parts[name].ignoredEdges
```

```
import assembly
mdb.models[name].rootAssembly.allinstances.ignoredEdges
mdb.models[name].rootAssembly.instances[name].ignoredEdges
```

7.8.1 findAt(...)

This method returns the object or objects in the IgnoredEdgeArray located at the given coordinates.

findAt initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any IgnoredEdge that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities). The arbitrary point must not be shared by a second IgnoredEdge. If two IgnoredEdge objects intersect or coincide at the arbitrary point, **findAt** chooses the first IgnoredEdge that it encounters, and you should not rely on the return value being consistent.

findAt will always try to find objects among all the ignored edges in the part or assembly instance and will not restrict itself to a subset even if the IgnoredEdgeArray represents such subset.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either an IgnoredEdge object or a sequence of IgnoredEdge objects based on the type of input.

- If *coordinates* is a sequence of Floats, **findAt** returns the IgnoredEdge object at that point.
- If you omit the *coordinates* keyword argument, **findAt** accepts as arguments a sequence of sequence of floats in the following format:

```
ignoredEdges = e.findAt(((20.19686, -169.513997, 27.798593), ),
                        ((19.657627, -167.295749, 27.056402), ),
                        ((18.274129, -157.144741, 25.15218), ))
```

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

An IgnoredEdge object or a sequence of IgnoredEdge objects.

Exceptions

None.

7.8.2 getSequenceFromMask(...)

This method returns the object or objects in the IgnoredEdgeArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When large number of objects are involved, this method is highly efficient.

Arguments**Required argument**

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

An IgnoredEdge object or a sequence of IgnoredEdge objects.

Exceptions

None.

7.8.3 getMask()

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.8.4 **getClosest(...)**

This method returns a object or objects in the IgnoredEdgeArray closest to the given set of points, where the given points need not lie on the edges in the IgnoredEdgeArray.

Arguments

Required argument

coordinates

A sequence of a sequence of floats, where each sequence of floats describes the X-, Y-, and Z-coordinates of a point.

```
r=e.getClosest(coordinates=((20.0,20.0,10.0),(-1.0, -15.0, 15),))

r.keys()

[0, 1]

r[0]

(mdb.models['Model-1'].parts['Part-1'].ignoredEdges[3],
 (15.7090625762939, 20.0, 10.0))
```

Optional argument

searchTolerance

A double specifying the distance within which the closest object must lie. The default value is half of the parent part/instance size.

Return value

This method returns a dictionary object. The key to the dictionary object is the position of the input point in the tuple specified in the *coordinates* starting at index 0. If a closest IgnoredEdge could be found then the value is a sequence consisting of two objects. The first object in the sequence is an IgnoredEdge that is close to the input point referred to by the key. The second object in the sequence, is a sequence of floats which specify the X-, Y-, and Z-location of the closest point on the IgnoredEdge to the given point. See program listing above.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.8.5 Members

The IgnoredEdgeArray object has no members.

7.9 IgnoredVertex object

An IgnoredVertex object is a point region of the geometry that was abstracted away by a virtual topology feature.

Access

```
import part
mdb.models[name].parts[name].ignoredVertices[i]
import assembly
mdb.models[name].rootAssembly.allinstances.ignoredVertices[i]
mdb.models[name].rootAssembly.instances[name].ignoredVertices[i]
```

7.9.1 Members

The IgnoredVertex object has the following members:

index

An Int specifying the index of the IgnoredVertex in the IgnoredVertexArray.

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of the vertex.

7.10 IgnoredVertexArray object

The IgnoredVertexArray is a sequence of IgnoredVertex objects. If the part is modified, then IgnoredVertexArray must be updated for that part.

Access

```
import part
mdb.models[name].parts[name].ignoredVertices
import assembly
mdb.models[name].rootAssembly.allinstances.ignoredVertices
mdb.models[name].rootAssembly.instances[name].ignoredVertices
```

7.10.1 **findAt(...)**

This method returns the object or objects in the IgnoredVertexArray located at the given coordinates.

findAt initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any IgnoredVertex object that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities).

findAt will always try to find objects among all the ignored vertices in the part or assembly instance and will not restrict itself to a subset even if the IgnoredVertexArray represents such subset.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either a IgnoredVertex object or a sequence of IgnoredVertex objects based on the type of input.

- If *coordinates* is a sequence of Floats, **findAt** returns the IgnoredVertex object at that point.
- If you omit the *coordinates* keyword argument, **findAt** accepts as arguments a sequence of sequence of floats in the following format:

```
verts = v.findAt(((20.19686, -169.513997, 27.798593), ),
                 ((19.657627, -167.295749, 27.056402), ),
                 ((18.274129, -157.144741, 25.15218), ))
```

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

An IgnoredVertex object or a sequence of IgnoredVertex objects.

Exceptions

None.

7.10.2 **getSequenceFromMask(...)**

This method returns the object or objects in the IgnoredVertexArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When large number of objects are involved, this method is highly efficient.

Arguments

Required argument

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

An IgnoredVertex object or a sequence of IgnoredVertex objects.

Exceptions

None.

7.10.3 **getMask()**

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.10.4 **getClosest(...)**

This method returns a object or objects in the IgnoredVertexArray closest to the given set of points, where the given points need not lie on the vertices in the IgnoredVertexArray.

Arguments

Required argument

coordinates

A sequence of a sequence of floats, where each sequence of floats describes the X-, Y-, and Z-coordinates of a point.

```
r=e.getClosest(coordinates=((20.0,20.0,10.0),(-1.0, -15.0, 15),))
```

```
r.keys()
```

```
[0, 1]
```

```
r[0]
```

```
(mdb.models['Model-1'].parts['Part-1'].ignoredVertices[3],  
 (15.7090625762939, 20.0, 10.0))
```

Optional argument

searchTolerance

A double specifying the distance within which the closest object must lie. The default value is half of the parent part/instance size.

Return value

This method returns a dictionary object. The key to the dictionary object is the position of the input point in the tuple specified in the *coordinates* starting at index 0. If a closest IgnoredVertex could be found then the value is a sequence consisting of two objects. The first object in the sequence is a IgnoredVertex that is close to the input point referred to by the key. The second object in the sequence, is a sequence of floats which specify the X-, Y-, and Z-location of the IgnoredVertex . See program listing above.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.10.5 Members

The IgnoredVertexArray object has no members.

7.11 InterestingPoint object

Interesting points can be located at the following:

- The middle of an edge.
- The middle of an arc.
- The center of an arc.

An InterestingPoint object is a temporary object and cannot be accessed from the Mdb object.

Access

```
import part
import assembly
```

7.11.1 InterestingPoint(...)

This method creates an interesting point along an edge. An InterestingPoint is a temporary object.

Path

```
mdb.models[name].parts[name].InterestingPoint
mdb.models[name].rootAssembly.instances[name].InterestingPoint
```

Required arguments

edge

An Edge object specifying the edge on which the interesting point is positioned.

rule

A SymbolicConstant specifying the position of the interesting point. Possible values are MIDDLE or CENTER.

Optional arguments

None.

Return value

An InterestingPoint object.

Exceptions

None.

7.11.2 Members

The InterestingPoint object has no members.

7.12 ModelDot object

The ModelDot object can be used to access an actual MeshNode , ReferencePoint , or Vertex object.

7.12.1 Members

The ModelDot object has no members.

7.13 ReferencePoint object

The ReferencePoint object has no direct constructor; it is created when a Feature object is created. The **ReferencePoint** method creates a Feature object that creates a ReferencePoint object.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].referencePoints[i]
mdb.models[name].parts[name].allSets[name].referencePoints[i]
mdb.models[name].parts[name].referencePoints[i]
mdb.models[name].parts[name].sets[name].referencePoints[i]
import assembly
mdb.models[name].rootAssembly.allinstances.referencePoints[i]
mdb.models[name].rootAssembly.allinstances.sets[name] \
.referencePoints[i]
mdb.models[name].rootAssembly.allInternalSets[name].referencePoints[i]
mdb.models[name].rootAssembly.allSets[name].referencePoints[i]
mdb.models[name].rootAssembly.instances[name].referencePoints[i]
mdb.models[name].rootAssembly.instances[name].sets[name] \
.referencePoints[i]
mdb.models[name].rootAssembly.modelInstances[i].referencePoints[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name] \
.referencePoints[i]
mdb.models[name].rootAssembly.referencePoints[i]
mdb.models[name].rootAssembly.sets[name].referencePoints[i]
```

7.13.1 Members

The `ReferencePoint` object has no members.

7.14 Transform object

The `MakeSketchTransform` method creates a Transform object. The Transform object has no direct constructor. A Transform object is a 4×3 matrix of Floats that represents the transformation from sketch coordinates to assembly coordinates or to part coordinates.

Access

```
import part
import assembly
```

7.14.1 matrix()

This method returns the transformation matrix as a tuple of 12 Floats.

Arguments

None.

Return value

A tuple of 12 Floats.

Exceptions

None.

7.14.2 Members

The Transform object has no members.

7.15 Vertex object

Vertices are point regions of geometry.

Access

```
import part
```

```
mdb.models[name].parts[name].allInternalSets[name].vertices[i]
mdb.models[name].parts[name].allSets[name].vertices[i]
mdb.models[name].parts[name].sets[name].vertices[i]
mdb.models[name].parts[name].vertices[i]
import assembly
mdb.models[name].rootAssembly.allinstances.sets[name].vertices[i]
mdb.models[name].rootAssembly.allinstances.vertices[i]
mdb.models[name].rootAssembly.allInternalSets[name].vertices[i]
mdb.models[name].rootAssembly.allSets[name].vertices[i]
mdb.models[name].rootAssembly.instances[name].sets[name].vertices[i]
mdb.models[name].rootAssembly.instances[name].vertices[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].vertices[i]
mdb.models[name].rootAssembly.modelInstances[i].vertices[i]
mdb.models[name].rootAssembly.sets[name].vertices[i]
mdb.models[name].rootAssembly.vertices[i]
```

7.15.1 getEdges()

This method returns a sequence consisting of the edge ids of the edges which share this vertex.

Arguments

None.

Return value

A tuple of integers.

Exceptions

None.

7.15.2 getNodes()

This method returns an array of node objects that are associated with the vertex.

Arguments

None.

Return value

A MeshNodeArray object which is a sequence of MeshNode objects.

Exceptions

None.

7.15.3 getElements()

This method returns an array of element objects that are associated with the vertex.

Arguments

None.

Return value

A MeshElementArray object which is a sequence of MeshElement objects.

Exceptions

None.

7.15.4 Members

The Vertex object has the following members:

index

An Int specifying the index of the Vertex in the VertexArray.

isReferenceRep

A Boolean specifying whether the vertex belongs to the reference representation of the Part or Instance.

pointOn

A tuple of Floats specifying the *X* -, *Y* -, and *Z* -coordinates of the vertex.

featureName

A tuple of Floats specifying the name of the feature that created this vertex.

instanceName

A tuple of Floats specifying the name of the part instance for this vertex (if applicable).

7.16 VertexArray object

The VertexArray is a sequence of Vertex objects. If the part is modified, then VertexArray must be updated for that part.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].vertices
mdb.models[name].parts[name].allSets[name].vertices
mdb.models[name].parts[name].sets[name].vertices
mdb.models[name].parts[name].vertices
import assembly
mdb.models[name].rootAssembly.allinstances.sets[name].vertices
mdb.models[name].rootAssembly.allinstances.vertices
mdb.models[name].rootAssembly.allInternalSets[name].vertices
mdb.models[name].rootAssembly.allSets[name].vertices
mdb.models[name].rootAssembly.instances[name].sets[name].vertices
mdb.models[name].rootAssembly.instances[name].vertices
mdb.models[name].rootAssembly.modelInstances[i].sets[name].vertices
mdb.models[name].rootAssembly.modelInstances[i].vertices
mdb.models[name].rootAssembly.sets[name].vertices
mdb.models[name].rootAssembly.vertices
```

7.16.1 VertexArray(...)

This method creates a VertexArray object.

Path

`part.VertexArray`

Required argument

vertices

A list of Vertex objects.

Optional arguments

None.

Return value

A VertexArray object.

Exceptions

None.

7.16.2 findAt(...)

This method returns the object or objects in the VertexArray located at the given coordinates.

findAt initially uses the ACIS tolerance of 1E-6. As a result, **findAt** returns any Vertex object that is at the arbitrary point specified or at a distance of less than 1E-6 from the arbitrary point. If nothing is found, **findAt** uses the tolerance for imprecise geometry (applicable only for imprecise geometric entities).

findAt will always try to find objects among all the vertices in the part or assembly instance and will not restrict itself to a subset even if the VertexArray represents such subset.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the object to find.

findAt returns either a Vertex object or a sequence of Vertex objects based on the type of input.

- If *coordinates* is a sequence of Floats, **findAt** returns the Vertex object at that point.
- If you omit the *coordinates* keyword argument, **findAt** accepts as arguments a sequence of sequence of floats in the following format:

```
verts = v.findAt(((20.19686, -169.513997, 27.798593), ),
                 ((19.657627, -167.295749, 27.056402), ),
                 ((18.274129, -157.144741, 25.15218), ))
```

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

A Vertex object or a sequence of Vertex objects.

Exceptions

None.

7.16.3 **getSequenceFromMask(...)**

This method returns the object or objects in the VertexArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When a large number of objects are involved, this method is highly efficient.

Arguments

Required argument

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A Vertex object or a sequence of Vertex objects.

Exceptions

None.

7.16.4 **getMask()**

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

7.16.5 **getByBoundingBox(...)**

This method returns an array of vertex objects that lie within the specified bounding box.

Arguments**Required arguments**

None.

Optional arguments

xMin

A float specifying the minimum *X*-boundary of the bounding box.

yMin

A float specifying the minimum *Y*-boundary of the bounding box.

zMin

A float specifying the minimum *Z*-boundary of the bounding box.

xMax

A float specifying the maximum *X*-boundary of the bounding box.

yMax

A float specifying the maximum *Y*-boundary of the bounding box.

zMax

A float specifying the maximum *Z*-boundary of the bounding box.

Return value

A `VertexArray` object, which is a sequence of `Vertex` objects.

Exceptions

None.

7.16.6 `getByBoundingCylinder(...)`

This method returns an array of vertex objects that lie within the specified bounding cylinder.

Arguments**Required arguments**

center1

A tuple of the *X*-, *Y*-, and *Z*-coordinates of the center of the first end of the cylinder.

center2

A tuple of the *X*-, *Y*-, and *Z*-coordinates of the center of the second end of the cylinder.

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

A VertexArray object, which is a sequence of Vertex objects.

Exceptions

None.

7.16.7 **getByBoundingSphere(...)**

This method returns an array of vertex objects that lie within the specified bounding sphere.

Arguments**Required arguments**

center

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

A VertexArray object, which is a sequence of Vertex objects.

Exceptions

None.

7.16.8 **getBoundingBox()**

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the vertex sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum X-, Y-, and Z-boundary values of the bounding box.

high: a tuple of three floats representing the maximum X-, Y-, and Z-boundary values of the bounding box.

Exceptions

None.

7.16.9 **getClosest(...)**

This method returns a object or objects in the VertexArray closest to the given set of points, where the given points need not lie on Vertex objects in the VertexArray.

Arguments

Required argument

coordinates

A sequence of a sequence of floats, where each sequence of floats describes the X-, Y-, and Z-coordinates of a point.

```
r=v.getClosest(coordinates=((20.0,20.0,10.0),(-1.0, -15.0, 15),))
```

```
r.keys()
```

```
[0, 1]
```

```
r[0]
```

```
(mdb.models['Model-1'].parts['Part-1'].vertices[0],  
  (15.7090625762939, 29.1666641235352, 20.0))
```

Optional argument

searchTolerance

A double specifying the distance within which the closest object must lie. The default value is half of the parent part/instance size.

Return value

This method returns a dictionary object. The key to the dictionary object is the position of the input point in the tuple specified in the *coordinates* starting at index 0. If a closest vertex could be found then the value is a sequence consisting of two objects. The first object in the sequence is a Vertex that is close to the input point referred to by the key. The second object in the sequence is a sequence of floats that specifies the X-, Y-, and Z-location of the Vertex . See program listing above.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

7.16.10 Members

The VertexArray object has no members.

8. Beam Section profile commands

The Beam Section profile commands are used to create profiles. A profile definition specifies the properties of a beam that are related to its cross-section geometry. When you define a beam section, you must include a reference to a profile in the section definition.

8.1 Profile object

The Profile object defines the geometrical properties of a beam cross-section. Profile is an abstract base type.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.1.1 beamProfilesFromOdb(...)

This method creates Profile objects by reading an output database. The new profiles are placed in the **profiles** repository.

Path

```
mdb.models[name].beamProfilesFromOdb
```

Required argument

fileName

A String specifying the name of the output database file (including the **.odb** extension) to be read. The String can also be the full path to the output database file if it is located in another directory.

Optional arguments

None.

Return value

A list of Profile objects.

Exceptions

None.

8.1.2 Members

The Profile object has the following member:

name

A String specifying the repository key.

8.2 ArbitraryProfile object

The ArbitraryProfile object defines the properties of an arbitrary profile.

The ArbitraryProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.2.1 ArbitraryProfile(...)

This method creates a ArbitraryProfile object.

Path

```
mdb.models[name].ArbitraryProfile
session.odbs[name].ArbitraryProfile
```

Required arguments

name

A String specifying the repository key.

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

The first sequence in the table specifies the following:

- 1-coordinate of the first point defining the profile.
- 2-coordinate of the first point defining the profile.

All other sequences in the table specify the following:

- 1-coordinate of the next point defining the profile.
- 2-coordinate of the next point defining the profile.
- The thickness of the segment ending at that point.

Return value

An `ArbitraryProfile` object.

Exceptions

`RangeError`.

8.2.2 `setValues(...)`

This method modifies the `ArbitraryProfile` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ArbitraryProfile` method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

8.2.3 Members

The `ArbitraryProfile` object has members with the same names and descriptions as the arguments to the `ArbitraryProfile` method.

8.2.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=ARBITRARY

8.3 BoxProfile object

The BoxProfile object defines the properties of a box profile.

The BoxProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.3.1 BoxProfile(...)

This method creates a BoxProfile object.

Path

```
mdb.models[name].BoxProfile
session.odbs[name].BoxProfile
```

Required arguments

name

A String specifying the repository key.

a

A Float specifying the *a* dimension of the box profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A Float specifying the *b* dimension of the box profile.

uniformThickness

A Boolean specifying whether the thickness is uniform.

t1

A Float specifying the uniform wall thickness if *uniformThickness*=ON, and the wall thickness of the first segment if *uniformThickness*=OFF.

Optional arguments*t2*

A Float specifying the wall thickness of the second segment. *t2* is required only when *uniformThickness*=OFF. The default value is 0.0.

t3

A Float specifying the wall thickness of the third segment. *t3* is required only when *uniformThickness*=OFF. The default value is 0.0.

t4

A Float specifying the wall thickness of the fourth segment. *t4* is required only when *uniformThickness*=OFF. The default value is 0.0.

Return value

A BoxProfile object.

Exceptions

RangeError.

8.3.2 setValues(...)

This method modifies the BoxProfile object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BoxProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.3.3 Members

The BoxProfile object has members with the same names and descriptions as the arguments to the **BoxProfile** method.

8.3.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=BOX

8.4 CircularProfile object

The CircularProfile object defines the properties of a solid circular profile.
The CircularProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.4.1 CircularProfile(...)

This method creates a CircularProfile object.

Path

```
mdb.models[name].CircularProfile
session.odbs[name].CircularProfile
```

Required arguments

name

A String specifying the repository key.

r

A positive Float specifying the *r* dimension (outer radius) of the circular profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

Optional arguments

None.

Return value

A CircularProfile object.

Exceptions

RangeError.

8.4.2 setValues(...)

This method modifies the CircularProfile object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CircularProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.4.3 Members

The CircularProfile object has members with the same names and descriptions as the arguments to the **CircularProfile** method.

8.4.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=CIRC

8.5 GeneralizedProfile object

The GeneralizedProfile object defines the properties of a profile via its area, moment of inertia, etc.
The GeneralizedProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.5.1 GeneralizedProfile(...)

This method creates a GeneralizedProfile object.

Path

```
mdb.models[name].GeneralizedProfile
session.odbs[name].GeneralizedProfile
```

Required arguments

name

A String specifying the repository key.

area

A Float specifying the cross-sectional area for the profile.

i11

A Float specifying the moment of inertia for bending about the 1-axis, I_{11} .

i12

A Float specifying the moment of inertia for cross bending, I_{12} .

i22

A Float specifying the moment of inertia for bending about the 2-axis, I_{22} .

j

A Float specifying the torsional constant, J .

gammaO

A Float specifying the sectorial moment, Γ_0 .

gammaW

A Float specifying the warping constant, Γ_W .

Optional arguments

None.

Return value

A GeneralizedProfile object.

Exceptions

RangeError.

8.5.2 setValues(...)

This method modifies the GeneralizedProfile object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GeneralizedProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.5.3 Members

The GeneralizedProfile object has members with the same names and descriptions as the arguments to the **GeneralizedProfile** method.

8.5.4 Corresponding analysis keywords

*BEAM GENERAL SECTION, SECTION=GENERAL or NONLINEAR GENERAL

8.6 HexagonalProfile object

The HexagonalProfile object defines the properties of a hexagonal profile.

The HexagonalProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.6.1 HexagonalProfile(...)

This method creates a HexagonalProfile object.

Path

```
mdb.models[name].HexagonalProfile
session.odbs[name].HexagonalProfile
```

Required arguments

name

A String specifying the repository key.

r

A positive Float specifying the *r* dimension (outer radius) of the hexagonal profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

t

A positive Float specifying the *t* dimension (wall thickness) of the hexagonal profile, $t < (\sqrt{3}/2)r$.

Optional arguments

None.

Return value

A HexagonalProfile object.

Exceptions

RangeError.

8.6.2 setValues(...)

This method modifies the HexagonalProfile object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **HexagonalProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.6.3 Members

The HexagonalProfile object has members with the same names and descriptions as the arguments to the **HexagonalProfile** method.

8.6.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=HEX

8.7 IProfile object

The IProfile object defines the properties of an I profile.

The IProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.7.1 IProfile(...)

This method creates an IProfile object.

Path

`mdb.models[name].IProfile`
`session.odbs[name].IProfile`

Required arguments

name

A String specifying the repository key.

l

A Float specifying the *l* dimension (offset of l-axis from the bottom flange surface) of the I profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

h

A Float specifying the *h* dimension (height) of the I profile.

b1

A Float specifying the *b1* dimension (bottom flange width) of the I profile.

b2

A Float specifying the *b2* dimension (top flange width) of the I profile.

t1

A Float specifying the *t1* dimension (bottom flange thickness) of the I profile.

t2

A Float specifying the *t2* dimension (top flange thickness) of the I profile.

t3

A Float specifying the *t3* dimension (web thickness) of the I profile.

Optional arguments

None.

Return value

An IProfile object.

Exceptions

RangeError.

8.7.2 **setValues(...)**

This method modifies the IProfile object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **IProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.7.3 Members

The IProfile object has members with the same names and descriptions as the arguments to the **IProfile** method.

8.7.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=I

8.8 LProfile object

The LProfile object defines the properties of a L profile.

The LProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.8.1 LProfile(...)

This method creates a LProfile object.

Path

`mdb.models[name].LProfile`
`session.odbs[name].LProfile`

Required arguments

name

A String specifying the repository key.

a

A positive Float specifying the *a* dimension (flange length) of the L profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A positive Float specifying the *b* dimension (flange length) of the L profile.

t1

A positive Float specifying the *t1* dimension (flange thickness) of the L profile ($t1 < b$).

t2

A positive Float specifying the *t2* dimension (flange thickness) of the L profile ($t2 < a$).

Optional arguments

None.

Return value

A LProfile object.

Exceptions

RangeError.

8.8.2 **setValues(...)**

This method modifies the LProfile object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.8.3 Members

The LProfile object has members with the same names and descriptions as the arguments to the `LProfile` method.

8.8.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=L

8.9 PipeProfile object

The PipeProfile object defines the properties of a circular pipe profile.
The PipeProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.9.1 PipeProfile(...)

This method creates a PipeProfile object.

Path

```
mdb.models[name].PipeProfile
session.odbs[name].PipeProfile
```

Required arguments

name

A String specifying the repository key.

r

A Float specifying the outer radius of the pipe. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

t

A Float specifying the wall thickness of the pipe.

Optional argument

formulation

A SymbolicConstant specifying the formulation. Possible values are THIN_WALL and THICK_WALL. The default value is THIN_WALL.

Return value

A PipeProfile object.

Exceptions

RangeError.

8.9.2 **setValues(...)**

This method modifies the PipeProfile object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PipeProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.9.3 Members

The PipeProfile object has members with the same names and descriptions as the arguments to the `PipeProfile` method.

8.9.4 Corresponding analysis keywords

*BEAM SECTION, SECTION=PIPE

*BEAM SECTION, SECTION=THICK PIPE

8.10 RectangularProfile object

The RectangularProfile object defines the properties of a solid rectangular profile.

The RectangularProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.10.1 RectangularProfile(...)

This method creates a RectangularProfile object.

Path

```
mdb.models[name].RectangularProfile
session.odbs[name].RectangularProfile
```

Required arguments

name

A String specifying the repository key.

a

A positive Float specifying the *a* dimension of the rectangular profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

BEAM SECTION PROFILE COMMANDS

b

A positive Float specifying the *b* dimension of the rectangular profile.

Optional arguments

None.

Return value

A RectangularProfile object.

Exceptions

RangeError.

8.10.2 **setValues(...)**

This method modifies the RectangularProfile object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RectangularProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.10.3 **Members**

The RectangularProfile object has members with the same names and descriptions as the arguments to the **RectangularProfile** method.

8.10.4 **Corresponding analysis keywords**

*BEAM SECTION, SECTION=RECT

8.11 TProfile object

The TProfile object defines the properties of a T profile.

The TProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.11.1 TProfile(...)

This method creates a TProfile object.

Path

```
mdb.models[name].TProfile
session.odbs[name].TProfile
```

Required arguments

name

A String specifying the repository key.

b

A positive Float specifying the *b* dimension (flange width) of the T profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

h

A positive Float specifying the *h* dimension (height) of the T profile.

l

A positive Float specifying the *l* dimension (offset of l–axis from the edge of web) of the T profile.

tf

A positive Float specifying the *tf* dimension (flange thickness) of the T profile ($tf < h$).

tw

A positive Float specifying the *tw* dimension (web thickness) of the T profile ($tw < b$).

Optional arguments

None.

Return value

A TProfile object.

Exceptions

RangeError.

8.11.2 **setValues(...)**

This method modifies the TProfile object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TProfile** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

8.11.3 **Members**

The TProfile object has members with the same names and descriptions as the arguments to the **TProfile** method.

8.11.4 **Corresponding analysis keywords**

*BEAM SECTION, SECTION=I

8.12 **TrapezoidalProfile object**

The TrapezoidalProfile object defines the properties of a trapezoidal profile.

The TrapezoidalProfile object is derived from the Profile object.

Access

```
import section
mdb.models[name].profiles[name]
import odbSection
session.odbs[name].profiles[name]
```

8.12.1 TrapezoidalProfile(...)

This method creates a TrapezoidalProfile object.

Path

```
mdb.models[name].TrapezoidalProfile
session.odbs[name].TrapezoidalProfile
```

Required arguments

name

A String specifying the repository key.

a

A positive Float specifying the *a* dimension of the Trapezoidal profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A positive Float specifying the *b* dimension of the Trapezoidal profile.

c

A positive Float specifying the *c* dimension of the Trapezoidal profile.

d

A Float specifying the *d* dimension of the Trapezoidal profile.

Optional arguments

None.

Return value

A TrapezoidalProfile object.

Exceptions

RangeError.

8.12.2 **setValues(...)**

This method modifies the `TrapezoidalProfile` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TrapezoidalProfile** method, except for the *name* argument.

Return value

None

Exceptions

`RangeError`.

8.12.3 **Members**

The `TrapezoidalProfile` object has members with the same names and descriptions as the arguments to the **TrapezoidalProfile** method.

8.12.4 **Corresponding analysis keywords**

*BEAM SECTION, SECTION=TRAPEZOID

9. Boundary Condition commands

A specific type of boundary condition object and a specific type of boundary condition state object are designed for each type of boundary condition. A `BoundaryCondition` object stores the non-propagating data of a boundary condition as well as a number of instances of the corresponding `BoundaryConditionState` object, each of which stores the propagating data of the boundary condition in a single step. Instances of the `BoundaryConditionState` object are created and deleted internally by its corresponding `BoundaryCondition` object.

9.1 BoundaryCondition object

The `BoundaryCondition` object is the abstract base type for other `BoundaryCondition` objects. The `BoundaryCondition` object has no explicit constructor. The methods and members of the `BoundaryCondition` object are common to all objects derived from the `BoundaryCondition`.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.1.1 deactivate(...)

This method deactivates the boundary condition in the specified step and all subsequent steps.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is deactivated.

Optional arguments

None.

Return value

None

Exceptions

`TextError`.

9.1.2 **move(...)**

This method moves the boundary condition state from one step to a different step.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the boundary condition state is moved.

toStepName

A String specifying the name of the step to which the boundary condition state is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

9.1.3 **reset(...)**

This method resets the boundary condition state of the specified step to the state of the previous analysis step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition state is reset.

Optional arguments

None.

Return value

None

Exceptions

TextError.

9.1.4 **resume()**

This method resumes the boundary condition that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

9.1.5 **suppress()**

This method suppresses the boundary condition.

Arguments

None.

Return value

None

Exceptions

None.

9.1.6 **delete(...)**

This method allows you to delete existing boundary conditions.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each boundary condition to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

9.1.7 Members

The BoundaryCondition object can have the following members:

name

A String specifying the boundary condition repository key.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.2 AccelerationBaseMotionBC object

The AccelerationBaseMotionBC object stores the data for an acceleration base motion boundary condition.

The AccelerationBaseMotionBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.2.1 AccelerationBaseMotionBC(...)

This method creates a AccelerationBaseMotionBC object.

Path

```
mdb.models[name].AccelerationBaseMotionBC
```


Required arguments*name*

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

dof

A SymbolicConstant specifying the constrained degree-of-freedom. Possible values for the SymbolicConstant are U1, U2, U3, UR1, UR2, UR3. The default value is U1.

Optional arguments*amplitudeScaleFactor*

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

*centerOfRotation*A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.*correlation*

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

*amplitude*A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.**Return value**

An AccelerationBaseMotionBC object.

Exceptions

None.

9.2.2 setValues(...)

This method modifies the data for an existing AccelerationBaseMotionBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AccelerationBaseMotionBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.2.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing **AccelerationBaseMotionBC** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional argument

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.2.4 Members

The AccelerationBaseMotionBC object can have the following members:

name

A String specifying the boundary condition repository key.

amplitudeScaleFactor

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

centerOfRotation

A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.

correlation

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.3 AccelerationBaseMotionBCState object

The AccelerationBaseMotionBCState object stores the propagating data for a velocity base motion boundary condition in a step. One instance of this object is created internally by the AccelerationBaseMotionBC object for each step. The instance is also deleted internally by the AccelerationBaseMotionBC object.

The AccelerationBaseMotionBCState object has no constructor or methods.

The AccelerationBaseMotionBCState object is derived from the BoundaryConditionState object.

BOUNDARY CONDITION COMMANDS

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.3.1 Members

The AccelerationBaseMotionBCState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.3.2 Corresponding analysis keywords

*BASE MOTION

9.4 AccelerationBC object

The AccelerationBC object stores the data for an acceleration boundary condition.

The AccelerationBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.4.1 AccelerationBC(...)

This method creates an AccelerationBC object.

Path

```
mdb.models[name].AccelerationBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

a1

A Float or a SymbolicConstant specifying the acceleration component in the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *a1*, *a2*, *a3*, *ar1*, *ar2*, and *ar3* are optional arguments, at least one of them must be specified.

a2

A Float or a SymbolicConstant specifying the acceleration component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

BOUNDARY CONDITION COMMANDS

a3

A Float or a SymbolicConstant specifying the acceleration component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar1

A Float or a SymbolicConstant specifying the rotational acceleration component about the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar2

A Float or a SymbolicConstant specifying the rotational acceleration component about the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar3

A Float or a SymbolicConstant specifying the rotational acceleration component about the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

Return value

An AccelerationBC object.

Exceptions

None.

9.4.2 setValues(...)

This method modifies the data for an existing AccelerationBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AccelerationBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.4.3 setValuesInStep(...)

This method modifies the propagating data for an existing **AccelerationBC** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

a1

A Float or a **SymbolicConstant** specifying the acceleration component in the 1-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

a2

A Float or a **SymbolicConstant** specifying the acceleration component in the 2-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

a3

A Float or a **SymbolicConstant** specifying the acceleration component in the 3-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

ar1

A Float or a **SymbolicConstant** specifying the rotational acceleration component about the 1-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

ar2

A Float or a **SymbolicConstant** specifying the rotational acceleration component about the 2-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

BOUNDARY CONDITION COMMANDS

ar3

A Float or a SymbolicConstant specifying the rotational acceleration component about the 3-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.4.4 Members

The AccelerationBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.5 AccelerationBCState object

The AccelerationBCState object stores the propagating data of an acceleration boundary condition in a step. One instance of this object is created internally by the AccelerationBC object for each step. The instance is also deleted internally by the AccelerationBC object.

The AccelerationBCState object has no constructor or methods.

The AccelerationBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.5.1 Members

The AccelerationBCState object has the following members:

a1

A Float specifying the acceleration component in the 1-direction.

a2

A Float specifying the acceleration component in the 2-direction.

a3

A Float specifying the acceleration component in the 3-direction.

ar1

A Float specifying the rotational acceleration component about the 1-direction.

ar2

A Float specifying the rotational acceleration component about the 2-direction.

ar3

A Float specifying the rotational acceleration component about the 3-direction.

a1State

A SymbolicConstant specifying the propagation state of the acceleration component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

a2State

A SymbolicConstant specifying the propagation state of the acceleration component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

a3State

A SymbolicConstant specifying the propagation state of the acceleration component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

ar1State

A SymbolicConstant specifying the propagation state of the rotational acceleration component about the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ar2State

A SymbolicConstant specifying the propagation state of the rotational acceleration component about the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ar3State

A SymbolicConstant specifying the propagation state of the rotational acceleration component about the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.5.2 Corresponding analysis keywords

*BOUNDARY, TYPE=ACCELERATION (degree of freedom: 1, 2, 3, 4, 5, or 6)

9.6 AcousticPressureBC object

The AcousticPressureBC object stores the data for an acoustic pressure boundary condition.

The AcousticPressureBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.6.1 AcousticPressureBC(...)

This method creates a AcousticPressureBC object.

Path

```
mdb.models[name].AcousticPressureBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the acoustic pressure magnitude. The default value is 0. The *magnitude* argument is optional if *distributionType*=USER_DEFINED.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

BOUNDARY CONDITION COMMANDS

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

An AcousticPressureBC object.

Exceptions

None.

9.6.2 **setValues(...)**

This method modifies the data for an existing AcousticPressureBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AcousticPressureBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.6.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing AcousticPressureBC object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments*magnitude*

A Float or the SymbolicConstant FREED specifying the acoustic pressure magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.6.4 Members

The AcousticPressureBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.7 AcousticPressureBCState object

The AcousticPressureBCState object stores the propagating data for an acoustic pressure boundary condition in a step. One instance of this object is created internally by the AcousticPressureBC object for each step. The instance is also deleted internally by the AcousticPressureBC object.

The AcousticPressureBCState object has no constructor or methods.

The AcousticPressureBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.7.1 Members

The AcousticPressureBCState object has the following members:

magnitude

A Float specifying the acoustic pressure magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the acoustic pressure magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.7.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 8)

9.8 BoundaryConditionState object

The BoundaryConditionState object is the abstract base type for other BoundaryConditionState objects. The BoundaryConditionState object has no explicit constructor or methods. The members of the BoundaryConditionState object are common to all objects derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.8.1 Members

The BoundaryConditionState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED

BOUNDARY CONDITION COMMANDS

- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.9 ConcentrationBC object

The ConcentrationBC object stores the data for a concentration boundary condition.

The ConcentrationBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.9.1 ConcentrationBC(...)

This method creates a ConcentrationBC object.

Path

```
mdb.models[name].ConcentrationBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments*fieldName*

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the concentration magnitude. The default value is 0. The *magnitude* argument is optional if *distributionType*=USER_DEFINED.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

A ConcentrationBC object.

Exceptions

None.

9.9.2 setValues(...)

This method modifies the data for an existing ConcentrationBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcentrationBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.9.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing ConcentrationBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

magnitude

A Float or the SymbolicConstant FREED specifying the concentration magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.9.4 **Members**

The ConcentrationBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.10 ConcentrationBCState object

The ConcentrationBCState object stores the propagating data for a concentration boundary condition in a step. One instance of this object is created internally by the ConcentrationBC object for each step. The instance is also deleted internally by the ConcentrationBC object.

The ConcentrationBCState object has no constructor or methods.

The ConcentrationBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.10.1 Members

The ConcentrationBCState object has the following members:

magnitude

A Float specifying the concentration magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the concentration magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.10.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 11)

9.11 ConnAccelerationBC object

The ConnAccelerationBC object stores the data for a connector acceleration boundary condition. The ConnAccelerationBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.11.1 ConnAccelerationBC(...)

This method creates an ConnAccelerationBC object on a wire region. Alternatively, the boundary condition may also be applied to a wire set referenced from an assembled fastener template model.

Path

`mdb.models [name] .ConnAccelerationBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

Optional arguments

region

The wire region to which the boundary condition is applied. This argument is not valid when *fastenerName* and *fastenerSetName* are specified.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

a1

A Float or a SymbolicConstant specifying the acceleration component in the connector's local 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *a1*, *a2*, *a3*, *ar1*, *ar2*, and *ar3* are optional arguments, at least one of them must be specified.

a2

A Float or a SymbolicConstant specifying the acceleration component in the connector's local 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

BOUNDARY CONDITION COMMANDS

a3

A Float or a SymbolicConstant specifying the acceleration component in the connector's local 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar1

A Float or a SymbolicConstant specifying the rotational acceleration component in the connector's local 4-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar2

A Float or a SymbolicConstant specifying the rotational acceleration component in the connector's local 5-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ar3

A Float or a SymbolicConstant specifying the rotational acceleration component in the connector's local 6-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

Return value

A ConnAccelerationBC object.

Exceptions

None.

9.11.2 **setValues(...)**

This method modifies the data for an existing ConnAccelerationBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnAccelerationBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.11.3 setValuesInStep(...)

This method modifies the propagating data for an existing **ConnAccelerationBC** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

a1

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 1-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

a2

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 2-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

a3

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 3-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

ar1

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 4-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

ar2

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 5-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

ar3

A Float or a **SymbolicConstant** specifying the connector acceleration component in the connector's local 6-direction. Possible values for the **SymbolicConstant** are **SET**, **UNCHANGED**, and **FREED**.

BOUNDARY CONDITION COMMANDS

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.11.4 Members

The ConnAccelerationBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.12 ConnAccelerationBCState object

The ConnAccelerationBCState object stores the propagating data of a connector acceleration boundary condition in a step. One instance of this object is created internally by the ConnAccelerationBC object for each step. The instance is also deleted internally by the ConnAccelerationBC object.

The ConnAccelerationBCState object has no constructor or methods.

The ConnAccelerationBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.12.1 Members

The ConnAccelerationBCState object has the following members:

a1

A Float specifying the connector acceleration component in the connector's local 1-direction.

a2

A Float specifying the connector acceleration component in the connector's local 2-direction.

a3

A Float specifying the connector acceleration component in the connector's local 3-direction.

ar1

A Float specifying the connector acceleration component in the connector's local 4-direction.

ar2

A Float specifying the connector acceleration component in the connector's local 5-direction.

ar3

A Float specifying the connector acceleration component in the connector's local 6-direction.

a1State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

a2State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

a3State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ar1State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 4-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ar2State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 5-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ar3State

A SymbolicConstant specifying the propagation state of the connector acceleration component in the connector's local 6-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.12.2 Corresponding analysis keywords

*CONNECTOR MOTION, TYPE=ACCELERATION (degree of freedom: 1, 2, 3, 4, 5, or 6)

9.13 ConnDisplacementBC object

The ConnDisplacementBC object stores the data for a connector displacement/rotation boundary condition.

The ConnDisplacementBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.13.1 ConnDisplacementBC(...)

This method creates a ConnDisplacementBC object on a wire region. Alternatively, the boundary condition may also be applied to a wire set referenced from an assembled fastener template model.

Path

```
mdb.models[name].ConnDisplacementBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

Optional arguments

region

The wire region to which the boundary condition is applied. This argument is not valid when *fastenerName* and *fastenerSetName* are specified.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

BOUNDARY CONDITION COMMANDS

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

u1

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *u1*, *u2*, *u3*, *ur1*, *ur2*, and *ur3* are optional arguments, at least one of them must be specified.

u2

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

u3

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur1

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 4-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur2

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 5-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur3

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 6-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

Return value

A ConnDisplacementBC object.

Exceptions

None.

9.13.2 setValues(...)

This method modifies the data for an existing ConnDisplacementBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnDisplacementBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.13.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConnDisplacementBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

u1

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 1-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

u2

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 2-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

u3

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the connector's local 3-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur1

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 4-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur2

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 5-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur3

A Float, a Complex, or a SymbolicConstant specifying the rotational component in the connector's local 6-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

Return value**None****Exceptions**

None.

9.13.4 Members

The ConnDisplacementBC object can have the following members:

name

A String specifying the boundary condition repository key.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.14 ConnDisplacementBCState object

The ConnDisplacementBCState object stores the propagating data for a connector displacement/rotation boundary condition in a step. One instance of this object is created internally by the ConnDisplacementBC object for each step. The instance is also deleted internally by the ConnDisplacementBC object.

The ConnDisplacementBCState object has no constructor or methods.

The ConnDisplacementBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.14.1 Members

The ConnDisplacementBCState object has the following members:

u1

A Float or a Complex specifying the displacement component in the connector's local 1-direction.

u2

A Float or a Complex specifying the displacement component in the connector's local 2-direction.

u3

A Float or a Complex specifying the displacement component in the connector's local 3-direction.

ur1

A Float or a Complex specifying the rotational component in the connector's local 4-direction.

ur2

A Float or a Complex specifying the rotational component in the connector's local 5-direction.

ur3

A Float or a Complex specifying the rotational component in the connector's local 6-direction.

u1State

A SymbolicConstant specifying the propagation state of the displacement component in the connector's local 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u2State

A SymbolicConstant specifying the propagation state of the displacement component in the connector's local 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u3State

A SymbolicConstant specifying the propagation state of the displacement component in the connector's local 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur1State

A SymbolicConstant specifying the propagation state of the rotational component in the connector's local 4-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur2State

A SymbolicConstant specifying the propagation state of the rotational component in the connector's local 5-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur3State

A SymbolicConstant specifying the propagation state of the rotational component in the connector's local 6-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.14.2 Corresponding analysis keywords

*CONNECTOR MOTION, TYPE=DISPLACEMENT (degree of freedom: 1, 2, 3, 4, 5, or 6)

9.15 ConnVelocityBC object

The ConnVelocityBC object stores the data for a connector velocity boundary condition.

The ConnVelocityBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.15.1 ConnVelocityBC(...)

This method creates a ConnVelocityBC object on a wire region. Alternatively, the boundary condition may also be applied to a wire set referenced from an assembled fastener template model.

Path

```
mdb.models[name].ConnVelocityBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

Optional arguments

region

The wire region to which the boundary condition is applied. This argument is not valid when *fastenerName* and *fastenerSetName* are specified.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

v1

A Float or a SymbolicConstant specifying the velocity component in the connector's local 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *v1*, *v2*, *v3*, *vr1*, *vr2*, and *vr3* are optional arguments, at least one of them must be specified.

v2

A Float or a SymbolicConstant specifying the velocity component in the connector's local 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

v3

A Float or a SymbolicConstant specifying the velocity component in the connector's local 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr1

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 4-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 5-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr3

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 6-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

Return value

A ConnVelocityBC object.

Exceptions

None.

9.15.2 **setValues(...)**

This method modifies the data for an existing ConnVelocityBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnVelocityBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.15.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing ConnVelocityBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

v1

A Float or a SymbolicConstant specifying the velocity component in the connector's local 1-direction. Possible values for the SymbolicConstant are SET and FREED.

v2

A Float or a SymbolicConstant specifying the velocity component in the connector's local 2-direction. Possible values for the SymbolicConstant are SET and FREED.

v3

A Float or a SymbolicConstant specifying the velocity component in the connector's local 3-direction. Possible values for the SymbolicConstant are SET and FREED.

vr1

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 4-direction. Possible values for the SymbolicConstant are SET and FREED.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 5-direction. Possible values for the SymbolicConstant are SET and FREED.

vr3

A Float or a SymbolicConstant specifying the rotational velocity component in the connector's local 6-direction. Possible values for the SymbolicConstant are SET and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.15.4 Members

The ConnVelocityBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

fastenerName

A String specifying the name of the assembled fastener to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

BOUNDARY CONDITION COMMANDS

fastenerSetName

A String specifying the assembled fastener template model set to which the boundary condition will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.16 ConnVelocityBCState object

The ConnVelocityBCState object stores the propagating data for a velocity boundary condition in a step. One instance of this object is created internally by the ConnVelocityBC object for each step. The instance is also deleted internally by the ConnVelocityBC object.

The ConnVelocityBCState object has no constructor or methods.

The ConnVelocityBCState object is derived from the BoundaryConditionState object.

Access

import load

mdb.models[name].steps[name].boundaryConditionStates[name]

9.16.1 Members

The ConnVelocityBCState object has the following members:

v1

A Float specifying the velocity component in the connector's local 1-direction.

v2

A Float specifying the velocity component in the connector's local 2-direction.

v3

A Float specifying the velocity component in the connector's local 3-direction.

vr1

A Float specifying the rotational velocity component in the connector's local 4-direction.

vr2

A Float specifying the rotational velocity component in the connector's local 5-direction.

vr3

A Float specifying the rotational velocity component in the connector's local 6-direction.

v1State

A SymbolicConstant specifying the propagation state of the velocity component in the connector's local 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v2State

A SymbolicConstant specifying the propagation state of the velocity component in the connector's local 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v3State

A SymbolicConstant specifying the propagation state of the velocity component in the connector's local 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr1State

A SymbolicConstant specifying the propagation state of the rotational velocity component in the connector's local 4-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr2State

A SymbolicConstant specifying the propagation state of the rotational velocity component in the connector's local 5-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr3State

A SymbolicConstant specifying the propagation state of the rotational velocity component in the connector's local 6-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

BOUNDARY CONDITION COMMANDS

- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.16.2 Corresponding analysis keywords

*CONNECTOR MOTION, TYPE=VELOCITY (degree of freedom: 1, 2, 3, 4, 5, or 6)

9.17 DisplacementBaseMotionBC object

The DisplacementBaseMotionBC object stores the data for a displacement base motion boundary condition.

The DisplacementBaseMotionBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.17.1 DisplacementBaseMotionBC(...)

This method creates a DisplacementBaseMotionBC object.

Path

```
mdb.models[name].DisplacementBaseMotionBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

dof

A SymbolicConstant specifying the constrained degree-of-freedom. Possible values for the SymbolicConstant are U1, U2, U3, UR1, UR2, UR3. The default value is U1.

Optional arguments

amplitudeScaleFactor

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

centerOfRotation

A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.

correlation

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A DisplacementBaseMotionBC object.

Exceptions

None.

9.17.2 setValues(...)

This method modifies the data for an existing DisplacementBaseMotionBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DisplacementBaseMotionBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.17.3 setValuesInStep(...)

This method modifies the propagating data for an existing **DisplacementBaseMotionBC** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional argument

amplitude

A String or a **SymbolicConstant** specifying the name of the amplitude reference. Possible values for the **SymbolicConstant** are **UNCHANGED** and **FREED**. **UNCHANGED** should be used if the amplitude is propagated from the previous analysis step. **FREED** should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.17.4 Members

The **DisplacementBaseMotionBC** object can have the following members:

name

A String specifying the boundary condition repository key.

amplitudeScaleFactor

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

centerOfRotation

A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.

correlation

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.18 DisplacementBaseMotionBCState object

The DisplacementBaseMotionBCState object stores the propagating data for a velocity base motion boundary condition in a step. One instance of this object is created internally by the DisplacementBaseMotionBC object for each step. The instance is also deleted internally by the DisplacementBaseMotionBC object.

The DisplacementBaseMotionBCState object has no constructor or methods.

The DisplacementBaseMotionBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.18.1 Members

The DisplacementBaseMotionBCState object has the following members:

BOUNDARY CONDITION COMMANDS

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.18.2 Corresponding analysis keywords

*BASE MOTION

9.19 DisplacementBC object

The DisplacementBC object stores the data for a displacement/rotation boundary condition.

The DisplacementBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.19.1 DisplacementBC(...)

This method creates a DisplacementBC object.

Path

```
mdb.models[name].DisplacementBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

u1

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

Note: Although *u1*, *u2*, *u3*, *ur1*, *ur2*, and *ur3* are optional arguments, at least one of them must be specified.

u2

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

u3

A Float, a Complex, or a SymbolicConstant specifying the displacement component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur1

A Float, a Complex, or a SymbolicConstant specifying the rotational displacement component about the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

BOUNDARY CONDITION COMMANDS

ur2

A Float, a Complex, or a SymbolicConstant specifying the rotational displacement component about the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

ur3

A Float, a Complex, or a SymbolicConstant specifying the rotational displacement component about the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

Return value

A DisplacementBC object.

Exceptions

None.

9.19.2 setValues(...)

This method modifies the data for an existing DisplacementBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DisplacementBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.19.3 setValuesInStep(...)

This method modifies the propagating data for an existing **DisplacementBC** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

u1

A Float, a Complex, or a **SymbolicConstant** specifying the displacement component in the 1-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

u2

A Float, a Complex, or a **SymbolicConstant** specifying the displacement component in the 2-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

u3

A Float, a Complex, or a **SymbolicConstant** specifying the displacement component in the 3-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

ur1

A Float, a Complex, or a **SymbolicConstant** specifying the rotational displacement component about the 1-direction. Possible values for the **SymbolicConstant** are SET, UNCHANGED, and FREED.

BOUNDARY CONDITION COMMANDS

ur2

A Float, a Complex, or a SymbolicConstant specifying the rotational displacement component about the 2-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

ur3

A Float, a Complex, or a SymbolicConstant specifying the rotational displacement component about the 3-direction. Possible values for the SymbolicConstant are SET, UNCHANGED, and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

Return value

None

Exceptions

None.

9.19.4 Members

The DisplacementBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

fieldName

A String specifying the name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.20 DisplacementBCState object

The DisplacementBCState object stores the propagating data for a displacement/rotation boundary condition in a step. One instance of this object is created internally by the DisplacementBC object for each step. The instance is also deleted internally by the DisplacementBC object.

The DisplacementBCState object has no constructor or methods.

The DisplacementBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.20.1 Members

The DisplacementBCState object has the following members:

u1

A Float or a Complex specifying the displacement component in the 1-direction.

u2

A Float or a Complex specifying the displacement component in the 2-direction.

u3

A Float or a Complex specifying the displacement component in the 3-direction.

BOUNDARY CONDITION COMMANDS

ur1

A Float or a Complex specifying the rotational displacement component about the 1-direction.

ur2

A Float or a Complex specifying the rotational displacement component about the 2-direction.

ur3

A Float or a Complex specifying the rotational displacement component about the 3-direction.

u1State

A SymbolicConstant specifying the propagation state of the displacement component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u2State

A SymbolicConstant specifying the propagation state of the displacement component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

u3State

A SymbolicConstant specifying the propagation state of the displacement component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur1State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur2State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

ur3State

A SymbolicConstant specifying the propagation state of the rotational displacement component about the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED

- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.20.2 Corresponding analysis keywords

*BOUNDARY, TYPE=DISPLACEMENT (degree of freedom: 1, 2, 3, 4, 5, or 6)

9.21 ElectricPotentialBC object

The ElectricPotentialBC object stores the data for an electrical potential boundary condition.

The ElectricPotentialBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.21.1 ElectricPotentialBC(...)

This method creates an ElectricPotentialBC object.

Path

```
mdb.models[name].ElectricPotentialBC
```

Required arguments

name

A String specifying the boundary condition repository key.

BOUNDARY CONDITION COMMANDS

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the electrical potential magnitude. The default value is 0. The *magnitude* argument is optional if *distributionType*=USER_DEFINED.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

An ElectricPotentialBC object.

Exceptions

None.

9.21.2 **setValues(...)**

This method modifies the data for an existing ElectricPotentialBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ElectricPotentialBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.21.3 setValuesInStep(...)

This method modifies the propagating data for an existing ElectricPotentialBC object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments*magnitude*

A Float or the SymbolicConstant FREED specifying the electrical potential magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.21.4 Members

The ElectricPotentialBC object can have the following members:

name

A String specifying the boundary condition repository key.

BOUNDARY CONDITION COMMANDS

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.22 ElectricPotentialBCState object

The ElectricPotentialBCState object stores the propagating data for a electrical potential boundary condition in a step. One instance of this object is created internally by the ElectricPotentialBC object for each step. The instance is also deleted internally by the ElectricPotentialBC object.

The ElectricPotentialBCState object has no constructor or methods.

The ElectricPotentialBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.22.1 Members

The ElectricPotentialBCState object has the following members:

magnitude

A Float specifying the electrical potential magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the electrical potential magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.22.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 9)

9.23 EulerianBC object

The EulerianBC object stores the data for an Eulerian boundary condition.

The EulerianBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.23.1 EulerianBC(...)

This method creates a EulerianBC object.

Path

`mdb.models[name].EulerianBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

definition

A SymbolicConstant specifying the flow conditions to be defined. Possible values are INFLOW, OUTFLOW, and BOTH. The default value is INFLOW.

inflowType

A SymbolicConstant specifying the control of material flow into the Eulerian domain. Possible values are FREE, NONE, and VOID. The default value is FREE.

outflowType

A SymbolicConstant specifying the control of flow of material out of the Eulerian domain. Possible values are ZERO_PRESSURE, FREE, NON_REFLECTING, and EQUILIBRIUM. The default value is ZERO_PRESSURE.

Return value

An EulerianBC object.

Exceptions

None.

9.23.2 setValues(...)

This method modifies the data for an existing EulerianBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

region

A Region object specifying the region to which the boundary condition is applied.

definition

A SymbolicConstant specifying the material flow conditions to be defined. Possible values are INFLOW, OUTFLOW, and BOTH. The default value is INFLOW.

inflowType

A SymbolicConstant specifying the control of material flow into the Eulerian domain. Possible values are FREE, NONE, and VOID. The default value is FREE.

outflowType

A SymbolicConstant specifying the control of material flow out of the Eulerian domain. Possible values are ZERO_PRESSURE, FREE, NON_REFLECTING, and EQUILIBRIUM. The default value is ZERO_PRESSURE.

Return value

None

Exceptions

None.

9.23.3 setValuesInStep(...)

This method modifies the propagating data for an existing EulerianBC object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

definition

A SymbolicConstant specifying the material flow conditions to be defined. Possible values are INFLOW, OUTFLOW, and BOTH. The default value is INFLOW.

inflowType

A SymbolicConstant specifying the control of material flow into the Eulerian domain. Possible values are FREE, NONE, and VOID. The default value is FREE.

BOUNDARY CONDITION COMMANDS

outflowType

A SymbolicConstant specifying the control of material flow out of the Eulerian domain. Possible values are ZERO_PRESSURE, FREE, NON_REFLECTING, and EQUILIBRIUM. The default value is ZERO_PRESSURE.

Return value

None

Exceptions

None.

9.23.4 Members

The EulerianBC object can have the following members:

name

A String specifying the boundary condition repository key.

definition

A SymbolicConstant specifying the flow conditions to be defined. Possible values are INFLOW, OUTFLOW, and BOTH. The default value is INFLOW.

inflowType

A SymbolicConstant specifying the control of material flow into the Eulerian domain. Possible values are FREE, NONE, and VOID. The default value is FREE.

outflowType

A SymbolicConstant specifying the control of flow of material out of the Eulerian domain. Possible values are ZERO_PRESSURE, FREE, NON_REFLECTING, and EQUILIBRIUM. The default value is ZERO_PRESSURE.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.24 EulerianBCState object

The EulerianBCState object stores the propagating data for an Eulerian boundary condition in a step. One instance of this object is created internally by the EulerianBC object for each step. The instance is also deleted internally by the EulerianBC object.

The EulerianBCState object has no constructor or methods.

The EulerianBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.24.1 Members

The EulerianBCState object has the following members:

definition

A SymbolicConstant specifying the material flow conditions to be defined. Possible values are INFLOW, OUTFLOW, and BOTH. The default value is INFLOW.

definitionState

A SymbolicConstant specifying the propagation state of the definition member. Possible values are UNSET, SET, and UNCHANGED.

inflowType

A SymbolicConstant specifying the material flow conditions to be defined. Possible values are FREE, NONE, and VOID. The default value is FREE.

inflowTypeState

A SymbolicConstant specifying the propagation state of the definition member. Possible values are UNSET, SET, and UNCHANGED.

outflowType

A SymbolicConstant specifying the material flow conditions to be defined. Possible values are ZERO_PRESSURE, FREE, NON_REFLECTING, and EQUILIBRIUM. The default value is ZERO_PRESSURE.

outflowTypeState

A SymbolicConstant specifying the propagation state of the definition member. Possible values are UNSET, SET, and UNCHANGED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.24.2 Corresponding analysis keywords

*EULERIAN BOUNDARY

9.25 EulerianMotionBC object

The EulerianMotionBC object stores the data for an Eulerian mesh motion boundary condition.

The EulerianMotionBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.25.1 EulerianMotionBC(...)

This method creates an EulerianMotionBC object.

Path

`mdb.models [name] .EulerianMotionBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

instanceName

A String specifying the name of the Eulerian part instance.

Optional arguments

followRegion

A Boolean specifying whether the mesh will follow a regular surface region or an Eulerian surface. The default value is ON.

region

A Region object specifying the region to which the boundary condition is applied.

materialName

A String specifying the name of the Eulerian surface to follow. This argument applies only when *followRegion*=False.

ctrPosition1

A SymbolicConstant specifying the 1-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition1

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition1

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio1

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 1 direction. If *expansionRatio1*=**None**, then there is no upper limit. The default value is **None**.

contractRatio1

A Float specifying the lower bounds on the allowable scaling of the mesh in the 1 direction. The default value is 0.0.

ctrPosition2

A SymbolicConstant specifying the 2-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

BOUNDARY CONDITION COMMANDS

posPosition2

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition2

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio2

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 2 direction. If *expansionRatio2*=**None**, then there is no upper limit. The default value is **None**.

contractRatio2

A Float specifying the lower bounds on the allowable scaling of the mesh in the 2 direction. The default value is 0.0.

ctrPosition3

A SymbolicConstant specifying the 3-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition3

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition3

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio3

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 3 direction. If *expansionRatio3*=**None**, then there is no upper limit. The default value is **None**.

contractRatio3

A Float specifying the lower bounds on the allowable scaling of the mesh in the 3 direction. The default value is 0.0.

allowContraction

A Boolean specifying whether the mesh is allowed to contract . The default value is ON.

aspectLimit

A Float specifying the maximum change in allowed aspect ratio (for any of the three mesh aspects, 1-2, 2-3, 3-1). The default value is 10.0.

vmaxFactor

A Float specifying the multiplier for the mesh nodal velocity limit. The default value is 1.01.

volThreshold

A Float specifying the lower bounds on the volume fraction when determining which nodes to include in the surface bounding box calculation for an Eulerian material surface. This argument applies only when *followRegion*=False. The default value is 0.5.

bufferSize

None or a Float specifying the buffer between the surface box and the Eulerian section mesh bounding box. The default value is 2.0.

Return value

An EulerianMotionBC object.

Exceptions

None.

9.25.2 setValues(...)

This method modifies the data for an existing EulerianMotionBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

instanceName

A String specifying the name of the Eulerian part instance.

followRegion

A Boolean specifying whether the mesh will follow a regular surface region or an Eulerian surface. The default value is ON.

region

A Region object specifying the region to which the boundary condition is applied.

materialName

A String specifying the name of the Eulerian surface to follow. This argument applies only when *followRegion*=False.

ctrPosition1

A SymbolicConstant specifying the 1-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition1

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition1

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

BOUNDARY CONDITION COMMANDS

expansionRatio1

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 1 direction. If *expansionRatio1*=**None**, then there is no upper limit. The default value is **None**.

contractRatio1

A Float specifying the lower bounds on the allowable scaling of the mesh in the 1 direction. The default value is 0.0.

ctrPosition2

A SymbolicConstant specifying the 2-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition2

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition2

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio2

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 2 direction. If *expansionRatio2*=**None**, then there is no upper limit. The default value is **None**.

contractRatio2

A Float specifying the lower bounds on the allowable scaling of the mesh in the 2 direction. The default value is 0.0.

ctrPosition3

A SymbolicConstant specifying the 3-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition3

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition3

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio3

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 3 direction. If *expansionRatio3*=**None**, then there is no upper limit. The default value is **None**.

contractRatio3

A Float specifying the lower bounds on the allowable scaling of the mesh in the 3 direction. The default value is 0.0.

allowContraction

A Boolean specifying whether the mesh is allowed to contract . The default value is ON.

aspectLimit

A Float specifying the maximum change in allowed aspect ratio (for any of the three mesh aspects, 1-2, 2-3, 3-1). The default value is 10.0.

vmaxFactor

A Float specifying the multiplier for the mesh nodal velocity limit. The default value is 1.01.

volThreshold

A Float specifying the lower bounds on the volume fraction when determining which nodes to include in the surface bounding box calculation for an Eulerian material surface. This argument applies only when *followRegion*=False. The default value is 0.5.

bufferSize

None or a Float specifying the buffer between the surface box and the Eulerian section mesh bounding box. The default value is 2.0.

Return value

None

Exceptions

None.

9.25.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing EulerianMotionBC object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments*ctrPosition1*

A SymbolicConstant specifying the 1-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition1

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition1

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

BOUNDARY CONDITION COMMANDS

expansionRatio1

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 1 direction. If *expansionRatio1*=**None**, then there is no upper limit. The default value is **None**.

contractRatio1

A Float specifying the lower bounds on the allowable scaling of the mesh in the 1 direction. The default value is 0.0.

ctrPosition2

A SymbolicConstant specifying the 2-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition2

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition2

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio2

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 2 direction. If *expansionRatio2*=**None**, then there is no upper limit. The default value is **None**.

contractRatio2

A Float specifying the lower bounds on the allowable scaling of the mesh in the 2 direction. The default value is 0.0.

ctrPosition3

A SymbolicConstant specifying the 3-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition3

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition3

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio3

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 3 direction. If *expansionRatio3*=**None**, then there is no upper limit. The default value is **None**.

contractRatio3

A Float specifying the lower bounds on the allowable scaling of the mesh in the 3 direction. The default value is 0.0.

allowContraction

A Boolean specifying whether the mesh is allowed to contract . The default value is ON.

aspectLimit

A Float specifying the maximum change in allowed aspect ratio (for any of the three mesh aspects, 1-2, 2-3, 3-1). The default value is 10.0.

vmaxFactor

A Float specifying the multiplier for the mesh nodal velocity limit. The default value is 1.01.

volThreshold

A Float specifying the lower bounds on the volume fraction when determining which nodes to include in the surface bounding box calculation for an Eulerian material surface. This argument applies only when *followRegion*=False. The default value is 0.5.

bufferSize

None or a Float specifying the buffer between the surface box and the Eulerian section mesh bounding box. The default value is 2.0.

Return value

None

Exceptions

None.

9.25.4 Members

The EulerianMotionBC object can have the following members:

name

A String specifying the boundary condition repository key.

followRegion

A Boolean specifying whether the mesh will follow a regular surface region or an Eulerian surface. The default value is ON.

ctrPosition1

A SymbolicConstant specifying the 1-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

ctrPosition2

A SymbolicConstant specifying the 2-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

ctrPosition3

A SymbolicConstant specifying the 3-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition1

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

BOUNDARY CONDITION COMMANDS

posPosition2

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

posPosition3

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition1

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition2

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition3

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio1

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 1 direction. If *expansionRatio1*=**None**, then there is no upper limit. The default value is **None**.

expansionRatio2

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 2 direction. If *expansionRatio2*=**None**, then there is no upper limit. The default value is **None**.

expansionRatio3

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 3 direction. If *expansionRatio3*=**None**, then there is no upper limit. The default value is **None**.

contractRatio1

A Float specifying the lower bounds on the allowable scaling of the mesh in the 1 direction. The default value is 0.0.

contractRatio2

A Float specifying the lower bounds on the allowable scaling of the mesh in the 2 direction. The default value is 0.0.

contractRatio3

A Float specifying the lower bounds on the allowable scaling of the mesh in the 3 direction. The default value is 0.0.

allowContraction

A Boolean specifying whether the mesh is allowed to contract . The default value is ON.

aspectLimit

A Float specifying the maximum change in allowed aspect ratio (for any of the three mesh aspects, 1-2, 2-3, 3-1). The default value is 10.0.

vmaxFactor

A Float specifying the multiplier for the mesh nodal velocity limit. The default value is 1.01.

volThreshold

A Float specifying the lower bounds on the volume fraction when determining which nodes to include in the surface bounding box calculation for an Eulerian material surface. This argument applies only when *followRegion=False*. The default value is 0.5.

bufferSize

None or a Float specifying the buffer between the surface box and the Eulerian section mesh bounding box. The default value is 2.0.

instanceName

A String specifying the name of the Eulerian part instance.

materialName

A String specifying the name of the Eulerian surface to follow. This argument applies only when *followRegion=False*.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys=None*, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.26 EulerianMotionBCState object

The EulerianMotionBCState object stores the propagating data for an Eulerian mesh motion boundary condition in a step. One instance of this object is created internally by the EulerianMotionBC object for each step. The instance is also deleted internally by the EulerianMotionBC object.

The EulerianMotionBCState object has no constructor or methods.

The EulerianMotionBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.26.1 Members

The EulerianMotionBCState object has the following members:

BOUNDARY CONDITION COMMANDS

ctrPosition1

A SymbolicConstant specifying the 1-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

ctrPosition2

A SymbolicConstant specifying the 2-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

ctrPosition3

A SymbolicConstant specifying the 3-direction translational constraint on the center of the Eulerian mesh. Possible values are FREE and FIXED. The default value is FREE.

posPosition1

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

posPosition2

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

posPosition3

A SymbolicConstant specifying the translational constraint on the positive (maximum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition1

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 1 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition2

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 2 direction. Possible values are FREE and FIXED. The default value is FREE.

negPosition3

A SymbolicConstant specifying the translational constraint on the negative (minimum) bounds of the mesh in the 3 direction. Possible values are FREE and FIXED. The default value is FREE.

expansionRatio1

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 1 direction. If *expansionRatio1*=**None**, then there is no upper limit. The default value is **None**.

expansionRatio2

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 2 direction. If *expansionRatio2*=**None**, then there is no upper limit. The default value is **None**.

expansionRatio3

None or a Float specifying the upper bounds on the allowable scaling of the mesh in the 3 direction. If *expansionRatio3*=**None**, then there is no upper limit. The default value is **None**.

contractRatio1

A Float specifying the lower bounds on the allowable scaling of the mesh in the 1 direction. The default value is 0.0.

contractRatio2

A Float specifying the lower bounds on the allowable scaling of the mesh in the 2 direction. The default value is 0.0.

contractRatio3

A Float specifying the lower bounds on the allowable scaling of the mesh in the 3 direction. The default value is 0.0.

allowContraction

A Boolean specifying whether the mesh is allowed to contract . The default value is ON.

aspectLimit

A Float specifying the maximum change in allowed aspect ratio (for any of the three mesh aspects, 1-2, 2-3, 3-1). The default value is 10.0.

vmaxFactor

A Float specifying the multiplier for the mesh nodal velocity limit. The default value is 1.01.

volThreshold

A Float specifying the lower bounds on the volume fraction when determining which nodes to include in the surface bounding box calculation for an Eulerian material surface. This argument applies only when *followRegion*=False. The default value is 0.5.

bufferSize

None or a Float specifying the buffer between the surface box and the Eulerian section mesh bounding box. The default value is 2.0.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.26.2 Corresponding analysis keywords

*EULERIAN MESH MOTION

9.27 FluidCavityPressureBC object

The FluidCavityPressureBC object stores the data for a fluid cavity pressure boundary condition. The FluidCavityPressureBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.27.1 FluidCavityPressureBC(...)

This method creates a FluidCavityPressureBC object.

Path

```
mdb.models[name].FluidCavityPressureBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

fluidCavity

A String specifying the name of a Fluid Cavity Interaction.

Optional arguments

magnitude

A Float specifying the fluid cavity pressure magnitude. The default value is 0.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

A FluidCavityPressureBC object.

Exceptions

None.

9.27.2 setValues(...)

This method modifies the data for an existing FluidCavityPressureBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidCavityPressureBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.27.3 setValuesInStep(...)

This method modifies the propagating data for an existing FluidCavityPressureBC object in the specified step.

BOUNDARY CONDITION COMMANDS

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

magnitude

A Float or the SymbolicConstant FREED specifying the fluid cavity pressure magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.27.4 Members

The FluidCavityPressureBC object can have the following members:

name

A String specifying the boundary condition repository key.

fluidCavity

A String specifying the name of a Fluid Cavity Interaction.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.28 FluidCavityPressureBCState object

The FluidCavityPressureBCState object stores the propagating data for a fluid cavity pressure boundary condition in a step. One instance of this object is created internally by the FluidCavityPressureBC object for each step. The instance is also deleted internally by the FluidCavityPressureBC object.

The FluidCavityPressureBCState object has no constructor or methods.

The FluidCavityPressureBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.28.1 Members

The FluidCavityPressureBCState object has the following members:

magnitude

A Float specifying the fluid cavity pressure magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the fluid cavity pressure magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE

BOUNDARY CONDITION COMMANDS

- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.28.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 8)

9.29 FluidInletOutletBC object

The FluidInletOutletBC object stores the data for a fluid inlet/outlet boundary condition.

The FluidInletOutletBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.29.1 FluidInletOutletBC(...)

This method creates a FluidInletOutletBC object.

Path

```
mdb.models[name].FluidInletOutletBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments*fieldName*

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

momentumType

A SymbolicConstant specifying the type of boundary condition defined on momentum equation. Possible values are PRESSURE, UNSET and VELOCITY. The default value is UNSET.

pressure

A Float or the SymbolicConstant UNSET specifying the pressure magnitude. Use a numeric value if *momentumType*=PRESSURE. The default value is UNSET.

v1

A Float or the SymbolicConstant UNSET specifying the velocity component in the 1-direction of the global coordinate system. The default value is UNSET.

Note: Although *v1*, *v2*, *v3* are optional arguments, at least one of them must be specified with numeric value when *momentumType*=VELOCITY.

v2

A Float or the SymbolicConstant UNSET specifying the velocity component in the 2-direction of the global coordinate system. The default value is UNSET.

v3

A Float or the SymbolicConstant UNSET specifying the velocity component in the 3-direction of the global coordinate system. The default value is UNSET.

momentumAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on momentum equation. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

temperature

A Float or the SymbolicConstant UNSET specifying the temperature magnitude. The default value is UNSET.

BOUNDARY CONDITION COMMANDS

temperatureAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the temperature with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

kineticEnergy

A Float or the SymbolicConstant UNSET specifying the kinetic energy magnitude. The default value is UNSET.

kineticEnergyAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

dissipationRate

A Float or the SymbolicConstant UNSET specifying the dissipation rate magnitude. The default value is UNSET.

dissipationRateAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

eddyViscosity

A Float or the SymbolicConstant UNSET specifying the eddy viscosity magnitude. The default value is UNSET.

eddyViscosityAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

Return value

A FluidInletOutletBC object.

Exceptions

None.

9.29.2 setValues(...)

This method modifies the data for an existing FluidInletOutletBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidInletOutletBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.29.3 setValuesInStep(...)

This method modifies the propagating data for an existing **FluidInletOutletBC** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

momentumType

A **SymbolicConstant** specifying the type of boundary condition defined on momentum equation. Possible values are **FREED**, **PRESSURE** and **VELOCITY**. Use **FREED** to remove the previously defined value.

pressure

A Float or the **SymbolicConstant** **FREED** specifying the pressure magnitude. Use a numeric value if *momentumType*=**PRESSURE**. Otherwise, use **FREED** to remove the previously defined value.

v1

A Float or the **SymbolicConstant** **FREED** specifying the velocity component in the 1-direction of the global coordinate system.

Note: Although *v1*, *v2*, *v3* are optional arguments, at least one of them must be specified with numeric value when *momentumType*=**VELOCITY**. Otherwise, use **FREED** to remove the previously defined value.

v2

A Float or the **SymbolicConstant** **FREED** specifying the velocity component in the 2-direction of the global coordinate system. Use **FREED** to remove the previously defined value.

BOUNDARY CONDITION COMMANDS

v3

A Float or the SymbolicConstant FREED specifying the velocity component in the 3-direction of the global coordinate system. Use FREED to remove the previously defined value.

momentumAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on momentum equation. FREED should be used if the boundary condition has no amplitude reference.

temperature

A Float or the SymbolicConstant FREED specifying the temperature magnitude. Use FREED to remove the previously defined value.

temperatureAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the temperature with time. FREED should be used if the boundary condition has no amplitude reference.

kineticEnergy

A Float or the SymbolicConstant FREED specifying the kinetic energy magnitude. Use FREED to remove the previously defined value.

kineticEnergyAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. FREED should be used if the boundary condition has no amplitude reference.

dissipationRate

A Float or the SymbolicConstant FREED specifying the dissipation rate magnitude. Use FREED to remove the previously defined value.

dissipationRateAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. FREED should be used if the boundary condition has no amplitude reference.

eddyViscosity

A Float or the SymbolicConstant FREED specifying the eddy viscosity magnitude. Use FREED to remove the previously defined value.

eddyViscosityAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. FREED should be used if the boundary condition has no amplitude reference.

Return value

None

Exceptions

None.

9.29.4 Members

The FluidInletOutletBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

fieldName

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.30 FluidInletOutletBCState object

The FluidInletOutletBCState object stores the propagating data of a fluid inlet/outlet boundary condition in a step. One instance of this object is created internally by the FluidInletOutletBC object for each step. The instance is also deleted internally by the FluidInletOutletBC object.

The FluidInletOutletBCState object has no constructor or methods.

The FluidInletOutletBCState object is derived from the BoundaryConditionState object.

Access

`import load`

`mdb.models[name].steps[name].boundaryConditionStates[name]`

9.30.1 Members

The FluidInletOutletBCState object has the following members:

momentumType

A SymbolicConstant specifying the type of boundary condition defined on momentum equation. Possible values are NONE, PRESSURE, and VELOCITY. The default value is NONE.

pressure

None or a Float specifying the pressure magnitude. The default value is **None**.

v1

None or a Float specifying the velocity component in the 1-direction of the global coordinate system. The default value is **None**.

v2

None or a Float specifying the velocity component in the 2-direction of the global coordinate system. The default value is **None**.

v3

None or a Float specifying the velocity component in the 3-direction of the global coordinate system. The default value is **None**.

momentumAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on momentum equation. The default value is NONE.

temperature

None or a Float specifying the temperature magnitude. The default value is **None**.

temperatureAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the temperature with time. The default value is NONE.

kineticEnergy

None or a Float specifying the kinetic energy magnitude. The default value is **None**.

kineticEnergyAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. The default value is NONE.

dissipationRate

None or a Float specifying the dissipation rate magnitude. The default value is **None**.

dissipationRateAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. The default value is NONE.

eddyViscosity

None or a Float specifying the eddy viscosity magnitude. The default value is **None**.

eddyViscosityAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. The default value is NONE.

momentumTypeState

A SymbolicConstant specifying the propagation state of the *momentumType* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

pressureState

A SymbolicConstant specifying the propagation state of the *pressure* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

v1State

A SymbolicConstant specifying the propagation state of the *v1* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

v2State

A SymbolicConstant specifying the propagation state of the *v2* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

v3State

A SymbolicConstant specifying the propagation state of the *v3* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

momentumAmplitudeState

A SymbolicConstant specifying the propagation state of the *momentumAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

temperatureState

A SymbolicConstant specifying the propagation state of the *temperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

temperatureAmplitudeState

A SymbolicConstant specifying the propagation state of the *temperatureAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

kineticEnergyState

A SymbolicConstant specifying the propagation state of the *kineticEnergy* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

kineticEnergyAmplitudeState

A SymbolicConstant specifying the propagation state of the *kineticEnergyAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

dissipationRateState

A SymbolicConstant specifying the propagation state of the *dissipationRate* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

dissipationRateAmplitudeState

A SymbolicConstant specifying the propagation state of the *dissipationRateAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

BOUNDARY CONDITION COMMANDS

eddyViscosityState

A SymbolicConstant specifying the propagation state of the *eddyViscosity* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

eddyViscosityAmplitudeState

A SymbolicConstant specifying the propagation state of the *eddyViscosityAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.30.2 Corresponding analysis keywords

*DSLOAD, TYPE=ACCELERATION (load type label: P, TEMP, TURBNU, VELX, VELY, or VELZ)

9.31 FluidWallConditionBC object

The FluidWallConditionBC object stores the data for a fluid wall condition boundary condition.
The FluidWallConditionBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.31.1 FluidWallConditionBC(...)

This method creates a FluidWallConditionBC object.

Path

```
mdb.models[name].FluidWallConditionBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

BOUNDARY CONDITION COMMANDS

type

A SymbolicConstant specifying the type of fluid wall condition boundary condition. Possible values are INFILTRATION, NO_SLIP and SHEAR. The default value is NO_SLIP.

v1

A Float or the SymbolicConstant UNSET specifying the velocity component in the 1-direction of the global coordinate system. The default value is UNSET.

v2

A Float or the SymbolicConstant UNSET specifying the velocity component in the 2-direction of the global coordinate system. The default value is UNSET.

v3

A Float or the SymbolicConstant UNSET specifying the velocity component in the 3-direction of the global coordinate system. The default value is UNSET.

velocityAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the velocities with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

thermalEnergyType

A SymbolicConstant specifying the type of boundary condition defined on thermal energy equation. Possible values are HEAT_FLUX, TEMPERATURE, and UNSET. The default value is UNSET.

temperature

A Float or the SymbolicConstant UNSET specifying the temperature magnitude. Use a numeric value if *thermalEnergyType*=TEMPERATURE. The default value is UNSET.

heatFlux

A Float or the SymbolicConstant UNSET specifying the heat flux magnitude. Use a numeric value if *thermalEnergyType*=HEAT_FLUX. The default value is UNSET.

thermalEnergyAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on thermal energy equation. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

kineticEnergy

A Float or the SymbolicConstant UNSET specifying the kinetic energy magnitude. The default value is UNSET.

kineticEnergyAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

dissipationRate

A Float or the SymbolicConstant UNSET specifying the dissipation rate magnitude. The default value is UNSET.

dissipationRateAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

eddyViscosity

A Float or the SymbolicConstant UNSET specifying the eddy viscosity magnitude. The default value is UNSET.

eddyViscosityAmplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET.

Return value

A FluidWallConditionBC object.

Exceptions

None.

9.31.2 setValues(...)

This method modifies the data for an existing FluidWallConditionBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidWallConditionBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.31.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing FluidWallConditionBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

type

A SymbolicConstant specifying the type of fluid wall condition boundary condition. Possible values are INFILTRATION, NO_SLIP, and SHEAR.

v1

A Float or the SymbolicConstant FREED specifying the velocity component in the 1-direction of the global coordinate system. Use FREED to remove the previously defined value.

v2

A Float or the SymbolicConstant FREED specifying the velocity component in the 2-direction of the global coordinate system. Use FREED to remove the previously defined value.

v3

A Float or the SymbolicConstant FREED specifying the velocity component in the 3-direction of the global coordinate system. Use FREED to remove the previously defined value.

velocityAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the velocities with time. FREED should be used if the boundary condition has no amplitude reference.

thermalEnergyType

A SymbolicConstant specifying the type of boundary condition defined on thermal energy equation. Possible values are FREED, HEAT_FLUX, and TEMPERATURE. Use FREED to remove the previously defined value.

temperature

A Float or the SymbolicConstant FREED specifying the temperature magnitude. Use a numeric value if *thermalEnergyType*=TEMPERATURE. Otherwise, use FREED to remove the previously defined value.

heatFlux

A Float or the SymbolicConstant FREED specifying the heat flux magnitude. Use a numeric value if *thermalEnergyType*=HEAT_FLUX. Otherwise, use FREED to remove the previously defined value.

thermalEnergyAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on thermal energy equation. FREED should be used if the boundary condition has no amplitude reference.

kineticEnergy

A Float or the SymbolicConstant FREED specifying the kinetic energy magnitude. Use FREED to remove the previously defined value.

kineticEnergyAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. FREED should be used if the boundary condition has no amplitude reference.

dissipationRate

A Float or the SymbolicConstant FREED specifying the dissipation rate magnitude. Use FREED to remove the previously defined value.

dissipationRateAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. FREED should be used if the boundary condition has no amplitude reference.

eddyViscosity

A Float or the SymbolicConstant FREED specifying the eddy viscosity magnitude. Use FREED to remove the previously defined value.

eddyViscosityAmplitude

A String or the SymbolicConstant FREED specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. FREED should be used if the boundary condition has no amplitude reference.

Return value

None

Exceptions

None.

9.31.4 Members

The FluidWallConditionBC object can have the following members:

name

A String specifying the boundary condition repository key.

BOUNDARY CONDITION COMMANDS

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

fieldName

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.32 FluidWallConditionBCState object

The FluidWallConditionBCState object stores the propagating data of a fluid wall condition boundary condition in a step. One instance of this object is created internally by the FluidWallConditionBC object for each step. The instance is also deleted internally by the FluidWallConditionBC object.

The FluidWallConditionBCState object has no constructor or methods.

The FluidWallConditionBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.32.1 Members

The FluidWallConditionBCState object has the following members:

type

A SymbolicConstant specifying the type of wall condition boundary condition. Possible values are INFILTRATION, NO_SLIP, and SHEAR. The default value is NO_SLIP.

v1

None or a Float specifying the velocity component in the 1-direction of the global coordinate system. The default value is **None**.

v2

None or a Float specifying the velocity component in the 2-direction of the global coordinate system. The default value is **None**.

v3

None or a Float specifying the velocity component in the 3-direction of the global coordinate system. The default value is **None**.

velocityAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the velocities with time. The default value is **NONE**.

thermalEnergyType

A SymbolicConstant specifying the type of boundary condition defined on thermal energy equation. Possible values are HEAT_FLUX, NONE, and TEMPERATURE. The default value is **NONE**.

temperature

None or a Float specifying the temperature magnitude. The default value is **None**.

heatFlux

None or a Float specifying the heat flux magnitude. The default value is **None**.

thermalEnergyAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the boundary condition values with time on thermal energy equation. The default value is **NONE**.

kineticEnergy

None or a Float specifying the kinetic energy magnitude. The default value is **None**.

kineticEnergyAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the kinetic energy with time. The default value is **NONE**.

dissipationRate

None or a Float specifying the dissipation rate magnitude. The default value is **None**.

dissipationRateAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the dissipation rate with time. The default value is **NONE**.

eddyViscosity

None or a Float specifying the eddy viscosity magnitude. The default value is **None**.

eddyViscosityAmplitude

None or a String specifying the name of the amplitude reference that gives the variation of the eddy viscosity with time. The default value is **NONE**.

typeState

A SymbolicConstant specifying the propagation state of the *type* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

BOUNDARY CONDITION COMMANDS

v1State

A SymbolicConstant specifying the propagation state of the *v1* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

v2State

A SymbolicConstant specifying the propagation state of the *v2* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

v3State

A SymbolicConstant specifying the propagation state of the *v3* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

velocityAmplitudeState

A SymbolicConstant specifying the propagation state of the *velocityAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

thermalEnergyTypeState

A SymbolicConstant specifying the propagation state of the *thermalEnergyType* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

temperatureState

A SymbolicConstant specifying the propagation state of the *temperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

heatFluxState

A SymbolicConstant specifying the propagation state of the *heatFlux* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

thermalEnergyAmplitudeState

A SymbolicConstant specifying the propagation state of the *thermalEnergyAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

kineticEnergyState

A SymbolicConstant specifying the propagation state of the *kineticEnergy* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

kineticEnergyAmplitudeState

A SymbolicConstant specifying the propagation state of the *kineticEnergyAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

dissipationRateState

A SymbolicConstant specifying the propagation state of the *dissipationRate* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

dissipationRateAmplitudeState

A SymbolicConstant specifying the propagation state of the *dissipationRateAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

eddyViscosityState

A SymbolicConstant specifying the propagation state of the *eddyViscosity* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

eddyViscosityAmplitudeState

A SymbolicConstant specifying the propagation state of the *eddyViscosityAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.32.2 Corresponding analysis keywords

*DSLOAD, TYPE=ACCELERATION (load type label: DIST, TEMP, TURBNU, VELX, VELY, or VELZ)

*DSFLUX (load type label: S)

9.33 MagneticVectorPotentialBC object

The MagneticVectorPotentialBC object stores the data for a magnetic vector potential boundary condition.

The MagneticVectorPotentialBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.33.1 MagneticVectorPotentialBC(...)

This method creates a MagneticVectorPotentialBC object.

Path

```
mdb.models[name].MagneticVectorPotentialBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

component1

A Complex, or a SymbolicConstant specifying the magnetic vector potential component in the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET

component2

A Complex, or a SymbolicConstant specifying the magnetic vector potential component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

component3

A Complex, or a SymbolicConstant specifying the magnetic vector potential component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A MagneticVectorPotentialBC object.

Exceptions

None.

9.33.2 setValues(...)

This method modifies the data for an existing MagneticVectorPotentialBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MagneticVectorPotentialBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.33.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing `MagneticVectorPotentialBC` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

component1

A Complex, or a `SymbolicConstant` specifying the magnetic vector potential component in the 1-direction. Possible values for the `SymbolicConstant` are `SET` and `UNCHANGED`.

component2

A Complex, or a `SymbolicConstant` specifying the magnetic vector potential component in the 2-direction. Possible values for the `SymbolicConstant` are `SET` and `UNCHANGED`.

component3

A Complex, or a `SymbolicConstant` specifying the magnetic vector potential component in the 3-direction. Possible values for the `SymbolicConstant` are `SET` and `UNCHANGED`.

amplitude

A String or a `SymbolicConstant` specifying the name of the amplitude reference. Possible values for the `SymbolicConstant` are `UNCHANGED` and `FREED`. `UNCHANGED` should be used if the amplitude is propagated from the previous analysis step. `FREED` should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

`None`

Exceptions

`None`.

9.33.4 **Members**

The `MagneticVectorPotentialBC` object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.34 MaterialFlowBC object

The MaterialFlowBC object stores the data for a connector material flow boundary condition.

The MaterialFlowBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.34.1 MaterialFlowBC(...)

This method creates a MaterialFlowBC object.

Path

```
mdb.models[name].MaterialFlowBC
```

Required arguments*name*

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

BOUNDARY CONDITION COMMANDS

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the material flow magnitude. The default value is 0. The *magnitude* argument is optional if *distributionType*=USER_DEFINED.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

A MaterialFlowBC object.

Exceptions

None.

9.34.2 setValues(...)

This method modifies the data for an existing MaterialFlowBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MaterialFlowBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.34.3 setValuesInStep(...)

This method modifies the propagating data for an existing MaterialFlowBC object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

magnitude

A Float or the SymbolicConstant FREED specifying the material flow magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.34.4 Members

The MaterialFlowBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

BOUNDARY CONDITION COMMANDS

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.35 MaterialFlowBCState object

The MaterialFlowBCState object stores the propagating data for a connector material flow boundary condition in a step. One instance of this object is created internally by the MaterialFlowBC object for each step. The instance is also deleted internally by the MaterialFlowBC object.

The MaterialFlowBCState object has no constructor or methods.

The MaterialFlowBCState object is derived from the BoundaryConditionState object.

Access

import load

mdb.models [name] .steps [name] .boundaryConditionStates [name]

9.35.1 Members

The MaterialFlowBCState object has the following members:

magnitude

A Float specifying the material flow magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the material flow magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.35.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 10)

9.36 PorePressureBC object

The PorePressureBC object stores the data for a pore pressure boundary condition.

The PorePressureBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.36.1 PorePressureBC(...)

This method creates a PorePressureBC object.

BOUNDARY CONDITION COMMANDS

Path

`mdb.models [name] .PorePressureBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the pore pressure magnitude. The default value is 0. The *magnitude* argument is optional if *distributionType*=USER_DEFINED.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

A PorePressureBC object.

Exceptions

None.

9.36.2 **setValues(...)**

This method modifies the data for an existing PorePressureBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PorePressureBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.36.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing PorePressureBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

magnitude

A Float or the SymbolicConstant FREED specifying the pore pressure magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.36.4 Members

The PorePressureBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.37 PorePressureBCState object

The PorePressureBCState object stores the propagating data for a pore pressure boundary condition in a step. One instance of this object is created internally by the PorePressureBC object for each step. The instance is also deleted internally by the PorePressureBC object.

The PorePressureBCState object has no constructor or methods.

The PorePressureBCState object is derived from the BoundaryConditionState object.

Access

`import load`

`mdb.models[name].steps[name].boundaryConditionStates[name]`

9.37.1 Members

The PorePressureBCState object has the following members:

magnitude

A Float specifying the pore pressure magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the pore pressure magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.37.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 8)

9.38 RetainedNodalDofsBC object

The RetainedNodalDofsBC object stores the data for a retained nodal dofs boundary condition.
The RetainedNodalDofsBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.38.1 RetainedNodalDofsBC(...)

This method creates a RetainedNodalDofsBC object.

Path

```
mdb.models[name].RetainedNodalDofsBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

u1

A Boolean specifying whether to retain the degree of freedom in the 1-direction. The default value is OFF indicating that the degree of freedom is not retained.

u2

A Boolean specifying whether to retain the degree of freedom in the 2-direction. The default value is OFF indicating that the degree of freedom is not retained.

u3

A Boolean specifying whether to retain the degree of freedom in the 3-direction. The default value is OFF indicating that the degree of freedom is not retained.

ur1

A Boolean specifying whether to retain the rotational degree of freedom about the 1-direction. The default value is OFF indicating that the degree of freedom is not retained.

ur2

A Boolean specifying whether to retain the rotational degree of freedom about the 2-direction. The default value is OFF indicating that the degree of freedom is not retained.

ur3

A Boolean specifying whether to retain the rotational degree of freedom about the 3-direction. The default value is OFF indicating that the degree of freedom is not retained.

Return value

A RetainedNodalDofsBC object.

Exceptions

None.

9.38.2 setValues(...)

This method modifies the data for an existing RetainedNodalDofsBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RetainedNodalDofsBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.38.3 setValuesInStep(...)

This method modifies the propagating data for an existing RetainedNodalDofsBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

BOUNDARY CONDITION COMMANDS

Optional arguments

u1

A Boolean specifying whether to retain the degree of freedom in the 1-direction.

u2

A Boolean specifying whether to retain the degree of freedom in the 2-direction.

u3

A Boolean specifying whether to retain the degree of freedom in the 3-direction.

ur1

A Boolean specifying whether to retain the rotational degree of freedom about the 1-direction.

ur2

A Boolean specifying whether to retain the rotational degree of freedom about the 2-direction.

ur3

A Boolean specifying whether to retain the rotational degree of freedom about the 3-direction.

Return value

None

Exceptions

None.

9.38.4 Members

The RetainedNodalDofsBC object can have the following members:

name

A String specifying the boundary condition repository key.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.39 SecondaryBaseBC object

The SecondaryBaseBC object stores the data for a secondary base boundary condition.

The SecondaryBaseBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.39.1 SecondaryBaseBC(...)

This method creates a SecondaryBaseBC object.

Path

```
mdb.models[name].SecondaryBaseBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

regions

A RegionArray object specifying the region to which the boundary condition is applied. Note that the usual *region* is ignored. The default value is MODEL.

dofs

A sequence of sequences of Ints specifying the constrained degrees-of-freedom.

Optional arguments

None.

Return value

A SecondaryBaseBC object.

Exceptions

None.

9.39.2 setValues(...)

This method modifies the data for an existing SecondaryBaseBC object in the step where it is created.

BOUNDARY CONDITION COMMANDS

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SecondaryBaseBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.39.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing **SecondaryBaseBC** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

None.

Return value

None

Exceptions

None.

9.39.4 **Members**

The **SecondaryBaseBC** object can have the following members:

name

A String specifying the boundary condition repository key.

dofs

A tuple of tuples of Ints specifying the constrained degrees-of-freedom.

regions

A RegionArray object specifying the region to which the boundary condition is applied. Note that the usual *region* is ignored. The default value is MODEL.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.40 SecondaryBaseBCState object

The SecondaryBaseBCState object stores the propagating data for a secondary base boundary condition in a step. One instance of this object is created internally by the SecondaryBaseBC object for each step. The instance is also deleted internally by the SecondaryBaseBC object.

The SecondaryBaseBCState object has no constructor or methods.

The SecondaryBaseBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.40.1 Members

The SecondaryBaseBCState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED

BOUNDARY CONDITION COMMANDS

- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.40.2 Corresponding analysis keywords

*BOUNDARY, BASE NAME

9.41 SubmodelBC object

The SubmodelBC object stores the data for a submodel boundary condition.

The SubmodelBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.41.1 SubmodelBC(...)

This method creates a SubmodelBC object.

Path

```
mdb.models[name].SubmodelBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

dof

A sequence of Ints specifying the degrees of freedom to which the boundary condition is applied.

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*='1' indicates the first step.

timeScale

A Boolean specifying whether to scale the time variable for the driven nodes' amplitude functions to match the submodel analysis step time. The default value is OFF.

shellThickness

A Float specifying the thickness of the shell in the global model. This argument is required for shell-to-solid submodeling and is not applicable to other submodels. The default value is 0.0.

Optional arguments*globalDrivingRegion*

A String specifying the element set in the global model that will be searched for elements whose responses will be used to drive the submodel. An empty string indicates that the entire global model will be searched. The default value is an empty string.

absoluteExteriorTolerance

None or a Float specifying the absolute value by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is **None**.

exteriorTolerance

None or a Float specifying the fraction of the average element size in the global model by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is 0.05.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

globalIncrement

An Int specifying the increment number in the global model step from which the solution will be used to specify the values of the driven variables. If *globalIncrement*=0, the solution from the last increment will be used. The *globalIncrement* argument is applicable only for linear perturbation steps. The default value is 0.

BOUNDARY CONDITION COMMANDS

centerZoneSize

A Float specifying the thickness of the center zone size around the shell midsurface. The default value is **None**.

Return value

A SubmodelBC object.

Exceptions

None.

9.41.2 **setValues(...)**

This method modifies the data for an existing SubmodelBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SubmodelBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.41.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing SubmodelBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments*fixed*

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is ON.

dof

A sequence of Ints specifying the degrees of freedom to which the boundary condition is applied. The *dof* argument is applicable only if *fixed*=OFF.

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*='1' indicates the first step. The *globalStep* argument is applicable only if *fixed*=OFF.

globalIncrement

An Int specifying the increment number in the global model step at which the solution will be used to specify the values of the driven variables. If *globalIncrement*=0, the solution from the last increment will be used. The *globalIncrement* argument is applicable only for linear perturbation steps and if *fixed*=OFF. The default value is 0.

centerZoneSize

A Float specifying the thickness of the center zone size around the shell midsurface. The default value is **None**.

The *centerZoneSize* argument is applicable only if *fixed*=OFF.

Return value

None

Exceptions

None.

9.41.4 Members

The SubmodelBC object can have the following members:

name

A String specifying the boundary condition repository key.

shellThickness

A Float specifying the thickness of the shell in the global model. This argument is required for shell-to-solid submodeling and is not applicable to other submodels. The default value is 0.0.

absoluteExteriorTolerance

None or a Float specifying the absolute value by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is **None**.

BOUNDARY CONDITION COMMANDS

exteriorTolerance

None or a Float specifying the fraction of the average element size in the global model by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is 0.05.

globalDrivingRegion

A String specifying the element set in the global model that will be searched for elements whose responses will be used to drive the submodel. An empty string indicates that the entire global model will be searched. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.42 SubmodelBCState object

The SubmodelBCState object stores the propagating data for a Submodel boundary condition in a step. One instance of this object is created internally by the SubmodelBC object for each step. The instance is also deleted internally by the SubmodelBC object.

The SubmodelBCState object has no constructor or methods.

The SubmodelBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.42.1 Members

The SubmodelBCState object has the following members:

dofState

A SymbolicConstant specifying the propagation state of the *dof* member. Possible values are SET and UNCHANGED.

globalStepState

A SymbolicConstant specifying the propagation state of the *globalStep* member. Possible values are SET and UNCHANGED.

globalIncrement

An Int specifying the increment number in the global model step at which the solution will be used to specify the values of the driven variables. This argument is applicable only for linear perturbation steps.

globalIncrementState

A SymbolicConstant specifying the propagation state of the *globalIncrement* member. Possible values are SET and UNCHANGED.

centerZoneSize

None or a Float specifying the thickness of the center zone size around the shell midsurface. The default value is **None**.

centerZoneSizeState

A SymbolicConstant specifying the propagation state of the *centerZoneSize* member. Possible values are SET and UNCHANGED.

scale

None or a Float specifying a scaling value applied to the applied displacements at the interface. The default value is 1.0.

scaleState

A SymbolicConstant specifying the propagation state of the *scale* member. Possible values are SET and UNCHANGED.

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*='1' indicates the first step.

dof

A tuple of Ints specifying the degrees of freedom to which the boundary condition is applied.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

BOUNDARY CONDITION COMMANDS

- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.42.2 Corresponding analysis keywords

*SUBMODEL
*BOUNDARY, SUBMODEL

9.43 TemperatureBC object

The TemperatureBC object stores the data for a temperature boundary condition.
The TemperatureBC object is derived from the BoundaryCondition object.

Access

```
import load  
mdb.models[name].boundaryConditions[name]
```

9.43.1 TemperatureBC(...)

This method creates a TemperatureBC object.

Path

```
mdb.models[name].TemperatureBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments*fieldName*

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

magnitude

A Float specifying the temperature magnitude. The default value is 0.

dof

A sequence of Ints specifying the degrees of freedom to which the boundary condition is applied. The default value is (11,).

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fixed

A Boolean specifying whether the boundary condition should remain fixed at the current values at the start of the step. The default value is OFF.

Return value

A TemperatureBC object.

Exceptions

None.

9.43.2 setValues(...)

This method modifies the data for an existing TemperatureBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TemperatureBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.43.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing TemperatureBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

magnitude

A Float or the SymbolicConstant FREED specifying the temperature magnitude.

dof

A sequence of Ints specifying the degrees of freedom to which the boundary condition is applied.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.43.4 **Members**

The TemperatureBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.44 TemperatureBCState object

The TemperatureBCState object stores the propagating data for a temperature boundary condition in a step. One instance of this object is created internally by the TemperatureBC object for each step. The instance is also deleted internally by the TemperatureBC object.

The TemperatureBCState object has no constructor or methods.

The TemperatureBCState object is derived from the BoundaryConditionState object.

Access

```
import load
```

```
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.44.1 Members

The TemperatureBCState object has the following members:

magnitude

A Float specifying the temperature magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the temperature magnitude. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

dofState

A SymbolicConstant specifying the propagation state of the *dof* member. Possible values are SET and UNCHANGED.

dof

A tuple of Ints specifying the degrees of freedom to which the boundary condition is applied.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.44.2 Corresponding analysis keywords

*BOUNDARY (degree of freedom: 11 for solids; 11, 12, etc. for shells)

9.45 TypeBC object

The TypeBC object stores the data for several types of predefined boundary conditions that are commonly used in stress/displacement analyses.

The TypeBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.45.1 EncastreBC(...)

This method creates an encastre TypeBC object.

Path

```
mdb.models[name].EncastreBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.2 PinnedBC(...)

This method creates a pinned TypeBC object.

Path

```
mdb.models[name].PinnedBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.3 XsymmBC(...)

This method creates a TypeBC object that specifies symmetry about the X-axis.

Path

```
mdb.models[name].XsymmBC
```

Required arguments*name*

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments*buckleCase*

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.4 YsymmBC(...)

This method creates a TypeBC object that specifies symmetry about the Y-axis.

Path`mdb.models [name] .YsymmBC`**Required arguments***name*

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

BOUNDARY CONDITION COMMANDS

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.5 ZsymmBC(...)

This method creates a TypeBC object that specifies symmetry about the Z-axis.

Path

```
mdb.models [name] .ZsymmBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.6 XasymmBC(...)

This method creates a TypeBC object that specifies antisymmetry about the *X*-axis.

Path

`mdb.models [name] .XasymmBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.7 YasymmBC(...)

This method creates a TypeBC object that specifies antisymmetry about the Y-axis.

Path

```
mdb.models [name] .YasymmBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.8 ZasymmBC(...)

This method creates a TypeBC object that specifies antisymmetry about the Z-axis.

Path

`mdb.models[name].ZasymmBC`

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys=None*, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

A TypeBC object.

Exceptions

None.

9.45.9 setValues(...)

This method modifies the data for an existing TypeBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

region

A Region object specifying the region to which the boundary condition is applied.

BOUNDARY CONDITION COMMANDS

typeName

A SymbolicConstant specifying the predefined boundary condition type. Possible values are XSYMM, YSYMM, ZSYMM, XASYMM, YASYMM, ZASYMM, PINNED, and ENCASTRE.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

Return value

None

Exceptions

None.

9.45.10 **setValuesInStep(...)**

This method always returns a value error for a TypeBC; it is inherited from the BoundaryCondition object.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional argument

typeName

A SymbolicConstant specifying the predefined boundary condition type. Possible values are XSYMM, YSYMM, ZSYMM, XASYMM, YASYMM, ZASYMM, PINNED, and ENCASTRE.

Return value

None

Exceptions

Value Error:

A Symmetry/Antisymmetry/Encastre BC cannot be edited in a propagated step.

9.45.11 Members

The TypeBC object can have the following members:

name

A String specifying the boundary condition repository key.

buckleCase

A SymbolicConstant specifying how the boundary condition is defined in a *BUCKLE analysis. Possible values are NOT_APPLICABLE, STRESS_PERTURBATION, BUCKLING_MODES, and PERTURBATION_AND_BUCKLING. The default value is NOT_APPLICABLE.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.46 TypeBCState object

The TypeBCState object stores the propagating data for a predefined boundary condition in a step. One instance of this object is created internally by the TypeBC object for each step. The instance is also deleted internally by the TypeBC object.

The TypeBCState object has no constructor or methods.

The TypeBCState object is derived from the BoundaryConditionState object.

Access

`import load`

`mdb.models[name].steps[name].boundaryConditionStates[name]`

9.46.1 Members

The TypeBCState object has the following members:

typeName

A SymbolicConstant specifying the predefined boundary condition type. Possible values are XSYMM, YSYMM, ZSYMM, XASYMM, YASYMM, ZASYMM, PINNED, and ENCASTRE.

BOUNDARY CONDITION COMMANDS

typeNameState

A SymbolicConstant specifying the propagation state of the predefined boundary condition type. The only possible value is UNCHANGED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.46.2 Corresponding analysis keywords

*BOUNDARY, TYPE=XSMM, YSYMM, ZSYMM, XASYMM, YASYMM, ZASYMM, PINNED, or ENCASTRE

9.47 VelocityBaseMotionBC object

The VelocityBaseMotionBC object stores the data for a velocity base motion boundary condition. The VelocityBaseMotionBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.47.1 VelocityBaseMotionBC(...)

This method creates a VelocityBaseMotionBC object.

Path

```
mdb.models[name].VelocityBaseMotionBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

dof

A SymbolicConstant specifying the constrained degree-of-freedom. Possible values for the SymbolicConstant are U1, U2, U3, UR1, UR2, UR3. The default value is U1.

Optional arguments

amplitudeScaleFactor

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

centerOfRotation

A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.

correlation

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A VelocityBaseMotionBC object.

Exceptions

None.

9.47.2 **setValues(...)**

This method modifies the data for an existing VelocityBaseMotionBC object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VelocityBaseMotionBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.47.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing VelocityBaseMotionBC object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional argument

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary

condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.47.4 Members

The VelocityBaseMotionBC object can have the following members:

name

A String specifying the boundary condition repository key.

amplitudeScaleFactor

A Float specifying the scale factor for the amplitude curve. The default value is 1.0.

useComplex

A Boolean specifying whether to define the imaginary (out-of-plane) portion of the base motion record given by amplitude definition. The default value is OFF.

centerOfRotation

A ModelDot object specifying a tuple containing one center of rotation. The default value is the global origin. This argument applies only when *dof*=UR1, UR2, or UR3.

correlation

A CorrelationArray object.

secondaryBase

A String specifying the name of the SecondaryBaseBC object associated with this boundary condition. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.48 VelocityBaseMotionBCState object

The VelocityBaseMotionBCState object stores the propagating data for a velocity base motion boundary condition in a step. One instance of this object is created internally by the VelocityBaseMotionBC object for each step. The instance is also deleted internally by the VelocityBaseMotionBC object.

The VelocityBaseMotionBCState object has no constructor or methods.

The VelocityBaseMotionBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.48.1 Members

The VelocityBaseMotionBCState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.48.2 Corresponding analysis keywords

*BASE MOTION

9.49 VelocityBC object

The VelocityBC object stores the data for a velocity boundary condition.

The VelocityBC object is derived from the BoundaryCondition object.

Access

```
import load
mdb.models[name].boundaryConditions[name]
```

9.49.1 VelocityBC(...)

This method creates a VelocityBC object.

Path

```
mdb.models[name].VelocityBC
```

Required arguments

name

A String specifying the boundary condition repository key.

createStepName

A String specifying the name of the step in which the boundary condition is created.

region

A Region object specifying the region to which the boundary condition is applied.

Optional arguments

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

v1

A Float or a SymbolicConstant specifying the velocity component in the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

BOUNDARY CONDITION COMMANDS

Note: Although *v1*, *v2*, *v3*, *vr1*, *vr2*, and *vr3* are optional arguments, at least one of them must be specified.

v2

A Float or a SymbolicConstant specifying the velocity component in the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

v3

A Float or a SymbolicConstant specifying the velocity component in the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr1

A Float or a SymbolicConstant specifying the rotational velocity component about the 1-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component about the 2-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

vr3

A Float or a SymbolicConstant specifying the rotational velocity component about the 3-direction. Possible values for the SymbolicConstant are UNSET and SET. The default value is UNSET.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the boundary condition has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

Return value

A VelocityBC object.

Exceptions

None.

9.49.2 setValues(...)

This method modifies the data for an existing VelocityBC object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VelocityBC** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

9.49.3 setValuesInStep(...)

This method modifies the propagating data for an existing VelocityBC object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the boundary condition is modified.

Optional arguments

v1

A Float or a SymbolicConstant specifying the velocity component in the 1-direction. Possible values for the SymbolicConstant are SET and FREED.

v2

A Float or a SymbolicConstant specifying the velocity component in the 2-direction. Possible values for the SymbolicConstant are SET and FREED.

v3

A Float or a SymbolicConstant specifying the velocity component in the 3-direction. Possible values for the SymbolicConstant are SET and FREED.

vr1

A Float or a SymbolicConstant specifying the rotational velocity component about the 1-direction. Possible values for the SymbolicConstant are SET and FREED.

vr2

A Float or a SymbolicConstant specifying the rotational velocity component about the 2-direction. Possible values for the SymbolicConstant are SET and FREED.

BOUNDARY CONDITION COMMANDS

vr3

A Float or a SymbolicConstant specifying the rotational velocity component about the 3-direction. Possible values for the SymbolicConstant are SET and FREED.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the boundary condition is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

9.49.4 Members

The VelocityBC object can have the following members:

name

A String specifying the boundary condition repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD. The default value is an empty string.

category

A SymbolicConstant specifying the category of the boundary condition. Possible values are MECHANICAL and THERMAL.

region

A Region object specifying the region to which the boundary condition is applied.

localCsys

None or a DatumCsys object specifying the local coordinate system of the boundary condition's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

9.50 VelocityBCState object

The VelocityBCState object stores the propagating data for a velocity boundary condition in a step. One instance of this object is created internally by the VelocityBC object for each step. The instance is also deleted internally by the VelocityBC object.

The VelocityBCState object has no constructor or methods.

The VelocityBCState object is derived from the BoundaryConditionState object.

Access

```
import load
mdb.models[name].steps[name].boundaryConditionStates[name]
```

9.50.1 Members

The VelocityBCState object has the following members:

v1

A Float specifying the velocity component in the 1-direction.

v2

A Float specifying the velocity component in the 2-direction.

v3

A Float specifying the velocity component in the 3-direction.

vr1

A Float specifying the rotational velocity component about the 1-direction.

vr2

A Float specifying the rotational velocity component about the 2-direction.

vr3

A Float specifying the rotational velocity component about the 3-direction.

v1State

A SymbolicConstant specifying the propagation state of the velocity component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v2State

A SymbolicConstant specifying the propagation state of the velocity component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

v3State

A SymbolicConstant specifying the propagation state of the velocity component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

BOUNDARY CONDITION COMMANDS

vr1State

A SymbolicConstant specifying the propagation state of the rotational velocity component about the 1-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr2State

A SymbolicConstant specifying the propagation state of the rotational velocity component about the 2-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

vr3State

A SymbolicConstant specifying the propagation state of the rotational velocity component about the 3-direction. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the amplitude reference. Possible values are UNSET, SET, UNCHANGED, FREED, and MODIFIED.

status

A SymbolicConstant specifying the propagation state of the BoundaryConditionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- PROPAGATED_FROM_BASE_STATE
- MODIFIED_FROM_BASE_STATE
- DEACTIVATED_FROM_BASE_STATE
- BUILT_INTO_MODES

amplitude

A String specifying the name of the amplitude reference. The String is empty if the boundary condition has no amplitude reference.

9.50.2 Corresponding analysis keywords

*BOUNDARY, TYPE=VELOCITY (degree of freedom: 1, 2, 3, 4, 5, or 6)

10. Calibration commands

The calibration commands are used for material calibration.

10.1 Calibration object

A Calibration object is the object used to specify a material calibration. The Calibration object stores the data that is used for specifying materials from test data.

Access

```
import calibration
mdb.models[name].calibrations[name]
```

10.1.1 Calibration(...)

This method creates a Calibration object.

Path

```
mdb.models[name].Calibration
```

Required argument

name

A String specifying the name of the new calibration.

Optional arguments

None.

Return value

A Calibration object.

Exceptions

InvalidNameError.

10.1.2 Members

The Calibration object has members with the same names and descriptions as the arguments to the **Calibration** method.

In addition, the Calibration object can have the following members:

dataSets

A DataSet object.

behaviors

A Behavior object.

10.2 Behavior object

The Behavior object specifies the method used for calibrating a material.

Access

```
import calibration
mdb.models[name].calibrations[name].behaviors[name]
```

10.2.1 Behavior(...)

This method creates a Behavior object.

Path

```
mdb.models[name].calibrations[name].Behavior
```

Required arguments

name

A String specifying the name of the new behavior.

typeName

A String specifying the type of the new Behavior. Values can be "ElasIsoBehavior", "ElasPlasIsoBehavior", "FeFpBehavior", or a user plug-in behavior type.

Optional arguments

None.

Return value

A Behavior object.

Exceptions

InvalidNameError.

10.2.2 setValues(...)

This method modifies the data for an existing behavior object.

Arguments**Required arguments**

None.

Optional arguments

E

Young's modulus. Only valid if the behavior type is ElasIsoBehavior.

nu

Poisson's ratio. Only valid if the behavior type is ElasIsoBehavior.

ds1Name

The name of the first data set. Only valid if the behavior type is ElasIsoBehavior or ElasPlasIsoBehavior

ds2Name

The name of the second data set. Only valid if the behavior type is ElasIsoBehavior or ElasPlasIsoBehavior

materialName

Material Name.

yieldPoint

Stress/strain value for the material yield point. Only valid if the behavior type is ElasPlasIsoBehavior

ultimatePoint

Stress/strain value for the material ultimate point. Only valid if the behavior type is ElasPlasIsoBehavior

plasticPoints

Stress/strain values for the plastic portion of material curve. Only valid if the behavior type is ElasPlasIsoBehavior

PoissonsRatio

Poisson's Ratio. Only valid if the behavior type is ElasPlasIsoBehavior

CALIBRATION COMMANDS

elasticModulus

Young's Modulus for the elastic portion of the material curve. Only valid if the behavior type is *ElasPlasIsoBehavior*

plasticPointsRange

Extent of the material plastic points. Only valid if the behavior type is *ElasPlasIsoBehavior*

name

Name of the behavior.

uniaxialName

Name of the uniaxial dataset. Only valid if the behavior type is *FeFpBehavior*

biaxialName

Name of the biaxial dataset. Only valid if the behavior type is *FeFpBehavior*

interpolation

'linear' specifies linear interpolation between data points, otherwise 'logarithmic'. Only valid if the behavior type is *FeFpBehavior*

uniWeight

Uniaxial weight factor, $\text{uniWeight} + \text{biWeight}$ should equal 1.0. Only valid if the behavior type is *FeFpBehavior*

biWeight

Biaxial weight factor, $\text{uniWeight} + \text{biWeight}$ should equal 1.0. Only valid if the behavior type is *FeFpBehavior*

uMullinsReload

A List of strings, specifying names of reloading *DataSet* objects obtained from uniaxial test data. Only valid if the behavior is of type *FeFpBehavior*

uMullinsUnload

A List of strings, specifying names of reloading *DataSet* objects obtained from uniaxial test data. Only valid if the behavior is of type *FeFpBehavior*

uPYieldPoint

A tuple specifying the coordinates of yield point of the permanent data set. Only valid if the behavior is of type *FeFpBehavior*

uPermSet

A List of strings, specifying names of permanent *DataSet* objects obtained from uniaxial test data. Only valid if the behavior is of type *FeFpBehavior*

uPrimary

A string specifying name of Primary *DataSet* object. Only valid if the behavior is of type *FeFpBehavior*

bMullinsReload

A List of strings, specifying names of reloading *DataSet* objects obtained from biaxial test data. Only valid if the behavior is of type *FeFpBehavior*

bMullinsUnload

A List of strings, specifying names of unloading DataSet objects obtained from biaxial test data. Only valid if the behavior is of type FeFpBehavior

bYieldPoint

A tuple specifying the coordinates of yield point of the permanent data set. Only valid if the behavior is of type FeFpBehavior

bPermSet

A List of strings, specifying names of permanent DataSet objects obtained from biaxial test data. Only valid if the behavior is of type FeFpBehavior

bPrimary

A string specifying name of Primary DataSet object. Only valid if the behavior is of type FeFpBehavior

Return value

None

Exceptions

None.

10.2.3 mapToMaterial(...)

This method appends the calibration data obtained from the DataSet object to an existing material object. In the case of ElasIsoBehavior, it appends the young's modulus and poisson's ratio. For ElasPlasIsoBehavior it appends the young's modulus, poisson's ratio and plastic points range and for FeFpBehavior it appends plastic points range and Mullins effect properties.

Arguments**Required arguments***materialName*

A String specifying the name of the existing material

Optional arguments

None.

Return value

None

Exceptions

None.

10.2.4 **compute_E(...)**

This method computes the value of young's modulus from the existing DataSet object. The method is only valid for ElasIsoBehavior type of behavior.

Arguments

Required arguments

dataSet

A DataSet object.

Optional arguments

None.

Return value

A tuple consisting of a and b values of the regression line($y = ax + b$), coefficient of determination(r -squared) value and the start and end-points of the line.

Exceptions

None.

10.2.5 **compute_nu(...)**

This method computes the value of Poisson's Ratio from the existing DataSet object. The method is only valid for ElasIsoBehavior and ElasPlasIsoBehavior type of behavior.

Arguments

Required arguments

dataSet

A DataSet object.

Optional arguments

None.

Return value

A tuple consisting of a and b values of the regression line($y = ax + b$), coefficient of determination(r -squared) value and the start and end-points of the line.

Exceptions

None.

10.2.6 **compute_ultimatePoint(...)**

This method computes the coordinates of the Ultimate point from the existing DataSet object. The method is only valid for ElasPlasIsoBehavior type of behavior.

Arguments

Required arguments

dataSet

A DataSet object.

Optional arguments

None.

Return value

Coordinates of the ultimate point.

Exceptions

None.

10.2.7 **compute_elasticModulus(...)**

This method computes the value of the elastic modulus from the yieldpoint value. The method is only valid for ElasPlasIsoBehavior type of behavior.

Arguments

Required arguments

yieldPoint

A tuple consisting of coordinates of the yieldpoint.

Optional arguments

None.

Return value

A float specifying the value of elastic modulus.

Exceptions

None.

10.2.8 **compute_plasticPoints(...)**

This method extracts the coordinates of the plastic Points. The method is only valid for ElasPlasIsoBehavior type of behavior.

Arguments

Required arguments

dataSet

A DataSet object.

slider_val

A float specifying the number of values to be taken.

start_index

A float specifying the lower limit of the range.

end_index

A float specifying the upper limit of the range.

Optional arguments

yp

Coordinates of the yieldpoint. The default value is (0,0).

Return value

A sequence of coordinates of the plastic points.

Exceptions

None.

10.2.9 **xyDataDissect(...)**

This method extracts primary, unload, reload and permanent DataSet objects from the existing DataSet object. The method is only valid for FeFpBehavior type of behavior.

Arguments

Required arguments

dsName

A string specifying the name of the uniaxial/biaxial test dataset.

modelName

A string specifying the name of the model to which the calibration behavior belongs.

calibrationName

A string specifying the name of the Calibration object to which the behavior belongs.

Optional arguments*biaxial*

A boolean specifying whether the test data is biaxial or uniaxial. The default value is True.

Return value

A sequence of strings specifying names of the DataSet objects containing loading, unloading, reloading and primary datasets.

Exceptions

None.

10.2.10 Members

The Behavior object has members with the same names and descriptions as the arguments to the **Behavior** method and **setValues** method.

In addition, the Behavior object can have the following members:

modelName

A string specifying the name of the model to which the behavior belongs.

calibrationName

A string specifying the name of calibration to which the behavior belongs.

biAxialAllName

A String specifying the name of the dataset containing all the raw data in the test data file. Only valid if the behavior is of type FeFpBehavior

uniAxialAllName

A String specifying the name of the dataset containing all the raw data in the test data file. Only valid if the behavior is of type FeFpBehavior

10.3 DataSet object

The DataSetobject specifies material test data.

Access

```
import calibration
mdb.models[name].calibrations[name].dataSets[name]
```

10.3.1 DataSet(...)

This method creates a DataSet object.

Path

```
mdb.models[name].calibrations[name].DataSet
```

Required arguments

name

A String specifying the name of the new dataset.

Optional arguments

data

A sequence of pairs of Floats specifying data set type pairs. Possible values are for stress/strain, force/displacement, or transverse strain/axial strain pairs.

type

A String specifying the type of the new dataset. Values can be "STRESS/STRAIN", "FORCE/DISPLACEMENT", or "AXIALSTRAIN/TRANSVERSESTRAIN". The default value is "STRESS/STRAIN".

form

A String specifying the form of the new dataset. Values can be "NOMINAL" or "TRUE". The default value is "NOMINAL".

Return value

A DataSet object.

Exceptions

InvalidNameError.

10.3.2 setValues(...)

This method modifies the data for an existing DataSet object.

Arguments

Required arguments

None.

Optional arguments*data*

A sequence of pairs of Floats specifying data set type pairs.

Return value

None

Exceptions

None.

10.3.3 Members

The DataSet object has members with the same names and descriptions as the arguments to the **DataSet** method.

11. Canvas commands

Canvas commands are used to create, position, and modify canvas objects. The Canvas commands need only the **abaqus** module to be imported; no additional modules need to be imported.

11.1 AttributeColorMap object

The AttributeColorMap object is used to store values and attributes associated with AttributeColorMap type objects. AttributeColorMap objects can be modified using the methods described below. The methods accessed via the Viewport object cause the AttributeColorMap object to be updated in the session.viewports[name].colorMappings repository.

Access

`session.viewports [name] .colorMappings [name]`

11.1.1 setDefaults()

This method resets the AttributeColorMap object to its default state.

Arguments

None.

Return value

None

Exceptions

None.

11.1.2 setValues(...)

This method modifies the AttributeColorMap object.

Arguments

Required arguments

None.

Optional arguments

At least one of the following must be provided:

overrides

A Dictionary object specifying a color mapping. Each key is of String type and specifies an attribute in the map; the corresponding values specify the color definition to apply to that attribute in the form (0|1, wire color, edge color, face color). The 0|1 defines the active status for the attribute. For example:

```
overrides={  
    'Part-1': (1, '#00FF00', '#00CCFF',  
               '#00FF00')}
```

defaultOverrides

A Dictionary object specifying a custom color mapping similar to overrides. For example:

```
defaultOverrides={  
    'Copper': (1, '#00FF00', '#00CCFF',  
               '#00FF00')}
```

The color mapping can contain keys that have not been created. When the key is created, it gets the appropriate values from this mapping.

Return value

None

Exceptions

None.

11.1.3 updateOverrides(...)

This method specifies additional overrides to be added to the current object definition.

Arguments

Required arguments

None.

Optional arguments

At least one of the following must be provided:

overrides

A Dictionary object specifying a color mapping. Each key is of String type and specifies an attribute in the map; the corresponding values specify the color definition to apply to that attribute in the form (0|1, wire color, edge color, face color). The 0|1 defines the active status for the attribute. For example:

```
overrides={
```

```
'Part-1': (1, '#00FF00', '#00CCFF',
           '#00FF00')}
```

defaultOverrides

A Dictionary object specifying a custom color mapping similar to overrides. For example:

```
defaultOverrides={
    'Copper': (1, '#00FF00', '#00CCFF',
              '#00FF00')}
```

The color mapping can contain keys that have not been created. When the key is created, it gets the appropriate values from this mapping.

Return value

None

Exceptions

None.

11.1.4 Members

The AttributeColorMap object has the following members:

mapType

A SymbolicConstant specifying the type of AttributeColorMap . Possible values are MATERIAL_MAP, SECTION_MAP, PART_MAP, ELSET_MAP, AVERAGING_REGION_MAP, and ELTYPE_MAP.

overrides

A Dictionary object specifying a color mapping. Each key is of String type and specifies an attribute in the map; the corresponding values specify the color definition to apply to that attribute in the form (0|1, wire color, edge color, face color). The 0|1 defines the active status for the attribute. For example:

```
overrides={
    'Part-1': (1, '#00FF00', '#00CCFF',
              '#00FF00')}
```

defaultOverrides

A Dictionary object specifying a custom color mapping similar to overrides. For example:

```
defaultOverrides={
    'Copper': (1, '#00FF00', '#00CCFF',
              '#00FF00')}
```

The color mapping can contain keys that have not been created. When the key is created, it gets the appropriate values from this mapping.

attributeColors

A Dictionary object specifying the color settings of each attribute as described in the **updateOverrides** method.

11.2 DrawingArea object

The DrawingArea object specifies the location and size of the drawing area used for placement of viewports.

Access

`session.drawingArea`

11.2.1 Members

The DrawingArea object has the following members:

width

A Float specifying the width in millimeters.

height

A Float specifying the height in millimeters.

origin

A pair of Floats specifying the coordinates of the bottom left hand corner in millimeters.

11.3 Layer object

Objects can be superimposed by displaying them in different layers of a viewport.

Access

`session.viewports [name] .layers [name]`

11.3.1 Layer(...)

This method creates a Layer object in the Layer repository.

Path

`session.viewports [name] .Layer`

Required argument*name*

A String specifying the repository key.

Optional argument*copyViewName*

A String specifying the name of the layer to copy.

Return value

A Layer object.

Exceptions

None.

11.3.2 **moveBefore(...)**

This method moves the layer object before another object in the layer repository.

Arguments**Required argument***name*

A String specifying the name of the other Layer object.

Optional arguments

None.

Return value

None

Exceptions

None.

11.3.3 **moveAfter(...)**

This method moves the layer object after another object in the layer repository.

Arguments

Required argument

name

A String specifying the name of the other Layer object.

Optional arguments

None.

Return value

None

Exceptions

None.

11.3.4 Members

The Layer object has members with the same names and descriptions as the arguments to the **Layer** method.

In addition, the Layer object can have the following members:

displayedObject

A Displayable object specifying the object to be displayed. The Displayable type is an abstract generalization. The concrete possible types are Part , Assembly , ConstrainedSketch , Odb , or XYPlot .

view

A View object specifying the object that controls viewing of the layer.

odbDisplay

An OdbDisplay object specifying the display options for the Odb object.

partDisplay

A PartDisplayOptions object specifying the display options for the Part object.

assemblyDisplay

An AssemblyDisplayOptions object specifying the display options for the Assembly object.

11.4 Viewport object

A viewport is the container for the graphics generated by the application. The Viewport object stores the various settings that determine how objects are displayed within that viewport.

Access`session.viewports [name]`**11.4.1 Viewport(...)**

This method creates a Viewport object with the specified origin and dimensions.

Path`session.Viewport`**Required argument***name*

A String specifying the repository key.

Optional arguments*origin*

A pair of Floats specifying the X- and Y-coordinates in millimeters in the canvas coordinate system of the lower left corner of the viewport. The default origin is (0, 0).

width

A Float specifying the width in millimeters of the viewport. Possible values are $30 \leq width \leq (maxWidth)$. The default value is 120.0.

Note: The maximum value of width (*maxWidth*) is the width of the screen in millimeters.

height

A Float specifying the height in millimeters of the viewport. This height includes the title bar. Possible values are $30 \leq height \leq (maxHeight)$. The default value is 80.0.

Note: The maximum value of height (*maxHeight*) is the height of the screen in millimeters.

border

A Boolean specifying whether the viewport border is visible in a printed image. The default value is ON.

titleBar

A Boolean specifying whether the viewport title should be displayed in a printed image. The default value is ON.

If *border* = OFF, the title will not be visible, even if *titleBar* = ON.

titleStyle

A SymbolicConstant specifying which title to use for the viewport title. Possible values are CUSTOM and SYSTEM. The default value is SYSTEM.

If *titleStyle* = CUSTOM, *customTitleString* will be used. If *titleStyle*= SYSTEM, a system-generated string will be used.

customTitleString

A String specifying the viewport title when *titleStyle* =CUSTOM. The default value is an empty string.

Return value

A Viewport object.

Exceptions

RangeError.

If the user attempts to delete the only viewport:

SystemError: the current viewport may not be deleted.

If *width* is out of range:

RangeError: width must be a Float in the range: $30 \leq \text{width} \leq \text{maxWidth}$

If *height* is out of range:

RangeError: height must be a Float in the range: $30 \leq \text{height} \leq \text{maxHeight}$

11.4.2 bringToFront()

This method moves the Viewport object to the front.

Arguments

None.

Return value

None

Exceptions

None.

11.4.3 disableMultipleColors()

This method disables applying multiple color mappings that was enabled using **enableMultipleColors**

Arguments

None.

Return value

None

Exceptions

None.

11.4.4 disableRefresh()

This method disables Viewport refresh. Some methods that require the Viewport to be up-to-date will override this setting. It is advisable to use this method sparingly.

Arguments

None.

Return value

None

Exceptions

None.

11.4.5 disableColorCodeUpdates()

This method disables Viewport updates and internal computations triggered because of color coding. Performance improvement will be significant when color coding is ON and repeating operations are performed using a script each of which requires color code updates. No benefit will be had when color coding is OFF.

Arguments

None.

Return value

None

Exceptions

None.

11.4.6 `enableMultipleColors()`

This method enables multiple color mappings to be applied at the same time. It also ensures that the Viewport is updated correctly when *initialColor* is set.

Arguments

None.

Return value

None

Exceptions

None.

11.4.7 `enableRefresh()`

This method enables Viewport refresh disabled using `disableRefresh`.

Arguments

None.

Return value

None

Exceptions

None.

11.4.8 `enableColorCodeUpdates()`

This method enables Viewport color code updates disabled using `disableColorCodeUpdates`.

Arguments

None.

Return value

None

Exceptions

None.

11.4.9 getActiveElementLabels(...)

This method returns the element labels currently active in the viewport based on the current display group and optionally based on any active cut if *useCut* is True. The labels will optionally be printed to the replay file if *printResults* is True. The **getActiveElementLabels** method has the following arguments:

Arguments**Required arguments**

None.

Optional arguments*useCut*

A Boolean flag to specify if any active cutting plane is to be considered in determining active elements.

printResults

A Boolean flag to specify if the active element labels are to be printed to the replay file.

Return value

A Dictionary object of labels for each active part instance.

Exceptions

None.

11.4.10 getActiveNodeLabels(...)

This method returns the node labels currently active in the viewport based on the current display group and optionally based on any active cut if *useCut* is True. The labels will optionally be printed to the replay file if *printResults* is True. The **getActiveNodeLabels** method has the following arguments:

Arguments**Required arguments**

None.

Optional arguments

useCut

A Boolean flag to specify if any active cutting plane is to be considered in determining active nodes.

printResults

A Boolean flag to specify if the active node labels are to be printed to the replay file.

Return value

A Dictionary object of labels for each active part instance.

Exceptions

None.

11.4.11 getPrimVarMinMaxLoc()

This method returns a dictionary containing the minimum, maximum and their location for the current primary variable. A contour plot should be displayed in the current viewport or else the method will return *None*.

Arguments

None.

Return value

A dictionary with keys 'minPartInstanceName', 'minElementLabel', 'minNodeLabel', 'minPosition', 'maxPartInstanceName', 'maxElementLabel', 'maxNodeLabel', 'maxPosition'

Exceptions

None.

11.4.12 makeCurrent()

This method makes the Viewport object the current viewport.

Arguments

None.

Return value

None

Exceptions

None.

11.4.13 maximize()

This method maximizes the Viewport object to fill the drawing area.

Arguments

None.

Return value

None

Exceptions

None.

11.4.14 minimize()

This method minimizes the Viewport object to appear as an abbreviated title bar.

Arguments

None.

Return value

None

Exceptions

None.

11.4.15 offset(...)

This method modifies the current *X-Y* location of the viewport by the specified distance.

Arguments**Required arguments**

None.

Optional arguments

deltaX

A Float specifying the offset in millimeters of the *X*-component of the viewport origin. The default value is 0.

deltaY

A Float specifying the offset in millimeters of the *Y*-component of the viewport origin. The default value is 0.

Return value

None

Exceptions

None.

11.4.16 `restore()`

This method restores a maximized or minimized Viewport object to its previous size and location.

Arguments

None.

Return value

None

Exceptions

None.

11.4.17 `sendToBack()`

This method moves the Viewport object to the back.

Arguments

None.

Return value

None

Exceptions

None.

11.4.18 setColor(...)

This method specifies the color assignment using *initialColor* and *translucency*. If *initialColor* has a value of 'As is', *translucency* has no effect. The **setColor** method has the following arguments:

Arguments**Required argument**

initialColor

A string specifying the initial color applied to the objects.

Optional argument

translucency

A float in the range of 0.0 to 1.0 specifying how translucent the objects drawn using *initialColor* needs to be.

Return value

None

Exceptions

None.

11.4.19 setColor(...)

This method specifies the color assignment using attributes specified by an *AttributeColorMap* object. The **setColor** method has the following arguments:

Arguments**Required argument**

colorMapping

An *AttributeColorMap* object. Possible values are any *AttributeColorMap* object.

Optional arguments

None.

Return value

None

Exceptions

None.

11.4.20 setColor(...)

This method specifies the color of a Leaf object.

Arguments

Required argument

leaf

A Leaf object. Possible values are any Leaf object.

Optional arguments

At least one of the following must be provided:

edgeColorWireHide

A String specifying the color to be used for drawing the edges of the elements contained in *leaf* when the render style is wireframe or hidden.

edgeColorFillShade

A String specifying the color to be used for drawing the edges of the elements contained in *leaf* when the render style is filled or shaded.

fillColor

A String specifying the color to be used for drawing the faces of the elements contained in *leaf* when the render style is filled or shaded.

nodeSymbolColor

A String specifying the color to be used for drawing the nodes contained in *leaf*.

nodeSymbolType

A SymbolicConstant specifying the node symbol types for the nodes contained in *leaf*. Possible values are FILLED_CIRCLE, FILLED_SQUARE, FILLED_DIAMOND, FILLED_TRI, HOLLOW_CIRCLE, HOLLOW_SQUARE, HOLLOW_DIAMOND, HOLLOW_TRI, CROSS, and XMARKER. The default value is HOLLOW_CIRCLE.

nodeSymbolSize

A SymbolicConstant specifying the node symbol size for the nodes contained in *leaf*. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

Return value

None

Exceptions

None.

11.4.21 forceRefresh()

This method causes the Viewport to refresh immediately. It is provided to allow scripts to refresh the Viewport before the script terminates. Normally, there would only be a single cumulative refresh that takes place immediately after the script completes.

Arguments

None.

Return value

None

Exceptions

None.

11.4.22 setValues(...)

This method modifies the Viewport object. The arguments to **setValues** are the same as the arguments to the **Viewport** method, except for the *name* argument. In addition, the **setValues** method has the following arguments:

Arguments**Required arguments**

None.

Optional arguments*displayedObject*

A Displayable object specifying the object to be displayed. The Displayable type is an abstract generalization. The concrete possible types are Part , Assembly , ConstrainedSketch , Odb , PlyStackPlot , or XYPlot . If *displayedObject*=**None**, Abaqus displays an empty viewport.

displayMode

A SymbolicConstant specifying the display mode of the viewport. Possible values are:

- SINGLE, specifying a single *displayedObject*.
- OVERLAY, specifying one or more layers to be displayed simultaneously—each layer contains one *displayedObject*.

CANVAS COMMANDS

visibleLayers

A sequence of Strings specifying the names of layers that will be displayed in the viewport when *displayMode* = OVERLAY.

viewManipLayers

A SymbolicConstant specifying which layer or layers will be controlled by the view manipulation tools when *displayMode*=OVERLAY. Possible values are ALL and CURRENT.

currentLayer

A String specifying which layer is affected by options settings when *displayMode* =OVERLAY. The current layer is also the only layer affected by view manipulations when *viewManipLayers* =CURRENT.

layerOffset

A Float specifying a factor to be used in offsetting layers in the screen Z direction. Possible values are -1 to 1. A negative value reverses the apparent order in which the layers are plotted.

Return value

None

Exceptions

RangeError.

11.4.23 addDrawings(...)

This method identifies the names of Drawing objects to be rendered in the Viewport.

Arguments

Required arguments

None.

Optional argument

names

A sequence of String values identifying keys in the session.drawings repository.

Return value

None

Exceptions

ValueError.

11.4.24 **removeDrawings(...)**

This method identifies the names of Drawing objects to no longer be rendered in the Viewport.

Arguments

Required arguments

None.

Optional argument

names

A sequence of String values identifying keys in the Viewport sequence.

Return value

None

Exceptions

ValueError.

11.4.25 **timeDisplay(...)**

This method refreshes the Viewport display *numFrames* times and then checks to see if *numSeconds* seconds have elapsed. If not, it will continue refreshing the Viewport until the time has elapsed. At completion, the actual number of refreshes (frames) rendered and elapsed time will be reported along with the calculated frames-per-second (fps).

Arguments

Required arguments

None.

Optional arguments

numFrames

An Int specifying the minimum number of times to refresh the Viewport. The default value is 0.

numSeconds

An Int specifying the minimum number of seconds to spend refreshing the Viewport. The default value is 10.

degreesPerFrame

A Float specifying the number of degrees to rotate the model view about its Z axis before each refresh. The default value is 0.0.

Return value

None

Exceptions

None.

11.4.26 Members

The Viewport object has members with the same names and descriptions as the arguments to the **Viewport** method.

In addition, the Viewport object can have the following members:

displayMode

A SymbolicConstant specifying the display mode of the viewport. Possible values are:

- SINGLE, specifying a single *displayedObject*.
- OVERLAY, specifying one or more layers to be displayed simultaneously—each layer contains one *displayedObject*.

viewManipLayers

A SymbolicConstant specifying which layer or layers will be controlled by the view manipulation tools when *displayMode*=OVERLAY. Possible values are ALL and CURRENT.

layerOffset

A Float specifying a factor to be used in offsetting layers in the screen Z direction. Possible values are -1 to 1. A negative value reverses the apparent order in which the layers are plotted.

windowState

A SymbolicConstant specifying the current state of a viewport. Possible values are NORMAL, MAXIMIZED, and MINIMIZED.

currentWidth

A Float specifying the width in millimeters of the current viewport, regardless of the value of *windowState*.

currentHeight

A Float specifying the height in millimeters of the current viewport, regardless of the value of *windowState*, and including the title bar.

applyLinkedCommands

A Boolean specifying whether the viewport is linked for synchronization. The default value is ON.

activeColorModes

A SymbolicConstant specifying the currently active color mappings. Possible values are:

- DEFAULT_COLORS
- PART_GEOM_MAP_COLORS

- ASSEMBLY_MAP_COLORS
- PART_MAP_COLORS
- INSTANCE_MAP_COLORS
- INSTANCE_TYPE_MAP_COLORS
- SECTION_MAP_COLORS
- MATERIAL_MAP_COLORS
- LOAD_MAP_COLORS
- BC_MAP_COLORS
- INTERACTION_MAP_COLORS
- CONSTRAINT_MAP_COLORS
- SET_MAP_COLORS
- SURFACE_MAP_COLORS
- INTERNAL_SET_MAP_COLORS
- INTERNAL_SURFACE_MAP_COLORS
- DISPLAY_GRP_MAP_COLORS
- SELECTION_GRP_MAP_COLORS
- ELTYPE_MAP_COLORS
- PLOT_MAP_COLORS
- MESH_MAP_COLORS

The default value is DEFAULT_COLORS.

colorMode

A SymbolicConstant specifying the last applied color mapping. Possible values are:

- DEFAULT_COLORS
- PART_GEOM_MAP_COLORS
- ASSEMBLY_MAP_COLORS
- PART_MAP_COLORS
- INSTANCE_MAP_COLORS
- INSTANCE_TYPE_MAP_COLORS
- SECTION_MAP_COLORS
- MATERIAL_MAP_COLORS
- LOAD_MAP_COLORS
- BC_MAP_COLORS
- INTERACTION_MAP_COLORS
- CONSTRAINT_MAP_COLORS
- SET_MAP_COLORS

CANVAS COMMANDS

- SURFACE_MAP_COLORS
- INTERNAL_SET_MAP_COLORS
- INTERNAL_SURFACE_MAP_COLORS
- DISPLAY_GRP_MAP_COLORS
- SELECTION_GRP_MAP_COLORS
- ELTYPE_MAP_COLORS
- PLOT_MAP_COLORS
- MESH_MAP_COLORS

The default value is DEFAULT_COLORS.

translucency

A Float specifying the translucency that will be applied to objects colored using *initialColor* and it needs to be set along with *initialColor*. If *initialColor* is set to 'As is' then translucency will have no effect.

animationConnect

A Boolean specifying whether an animation is connected to the viewport.

colorMappings

A repository of AttributeColorMap objects specifying the objects cannot be constructed but the following attribute maps are supported:

- "type"
- "Element set"
- "Material"
- "Section"
- "Default"
- "Part"
- "Part instance"
- "Element type"
- "Averaging region"
- "Assembly"
- "Property"
- "Set"
- "Surface"
- "Skin"
- "Profile"
- "Part shape"
- "Part status"
- "Part geometry"

- "Meshability"
- "Instance type"
- "Load"
- "Boundary condition"
- "Interaction"
- "Constraint"
- "Interaction type"
- "Constraint type"
- "Display group"
- "Selection group"
- "Interaction property"
- "Connector"
- "Connector type"
- "Connector property"
- "Internal set"
- "Internal surface"
- "mapColors"
- "autoColors"
- "overrides"
- "defaultAutoColors"
- "defaultOverrides"
- "objectToCopy"
- "colorMapping"
- "colorMappings"
- "colorMode"
- "attributeColors"
- "updateOverrides"
- "colorCodeOverride"
- "initialColor"
- "Layup"
- "Ply"

initialColor

A String specifying the color that will be applied to all objects in the viewport at the start of color coding. The possible values are 'As is', 'Default' or a string with a hexadecimal representation of a color.

CANVAS COMMANDS

currentLayer

A String specifying which layer is affected by options settings when *displayMode* =OVERLAY. The current layer is also the only layer affected by view manipulations when *viewManipLayers* =CURRENT.

displayedObject

A Displayable object specifying the object to be displayed. The Displayable type is an abstract generalization. The concrete possible types are Part , Assembly , ConstrainedSketch , Odb , PlyStackPlot , or XYPlot . If *displayedObject*=**None**, Abaqus displays an empty viewport.

layers

A repository of Layer objects specifying the key to the repository is a String with the name of the layer.

view

A View object specifying the object that controls viewing of the viewport content.

odbDisplay

An OdbDisplay object specifying the display options for the Odb object.

partDisplay

A PartDisplayOptions object specifying the display options for the Part object.

assemblyDisplay

An AssemblyDisplayOptions object specifying the display options for the Assembly object.

viewportAnnotationOptions

A ViewportAnnotationOptions object.

detailPlotOptions

A DetailPlotOptions object.

annotationsToPlot

An AnnotationsToPlotArray object.

visibleLayers

A tuple of Strings specifying the names of layers that will be displayed in the viewport when *displayMode* = OVERLAY.

currentOrigin

A pair of Floats specifying the X- and Y-coordinates in millimeters in the canvas coordinate system of the lower left corner of the current viewport, regardless of the value of *windowState*.

iconOrigin

A pair of Floats specifying the X- and Y-coordinates in millimeters of the lower left corner of the current viewport from a coordinate system having its origin in the lower left corner of the drawing area. This origin refers to the viewport location when *windowState* =MINIMIZED.

lightOptions

A LightOptions object.

imageOptions

An ImageOptions object.

movieOptions

A MovieOptions object.

drawings

A tuple of Strings specifying keys to the session.drawings repository. The default value is an empty sequence.

11.5 ImageOptions object

The ImageOptions object stores settings that control how an image is rendered in a particular viewport. ImageOptions objects are accessed from the image options associated with a particular viewport.

The ImageOptions object has no constructor; Abaqus creates the *imageOptions* member for a viewport when the viewport is created using the values in the *imageOptions* member of the current viewport.

Access

`session.viewports[name].imageOptions`

11.5.1 setValues(...)

This method modifies the ImageOptions object.

Arguments

Required arguments

None.

Optional arguments

imageName

A String specifying the name of the image. A list of valid image names is in the *images* repository in the *session* object.

showImage

A Boolean specifying whether an image should be displayed in the viewport background. The default value is OFF.

positionMethod

A SymbolicConstant specifying which positioning method is used to determine how the image will be scaled and positioned in the viewport. Possible values are:

- FIT_TO_VIEWPORT, specifying a display mode where the image is scaled to fit in the viewport using the specified *fitMethod*.
- AUTO_ALIGN, specifying a display mode where the image is scaled as specified by *xScale* and *yScale* and then positioned in the viewport using the specified *alignment*.

CANVAS COMMANDS

- **MANUAL**, specifying a display mode where the image is scaled as specified by *xScale* and *yScale* and then positioned in the viewport using the specified *origin*.

The default value is **FIT_TO_VIEWPORT**.

fitMethod

A SymbolicConstant specifying which type of fit is performed to scale and position the image in the viewport when *positionMethod* = **FIT_TO_VIEWPORT**. Possible values are:

- **BEST_FIT**, specifying a mode where the image is scaled to completely fit within the viewport.
- **FIT_WIDTH**, specifying a mode where the image width is scaled to match the viewport width.
- **FIT_HEIGHT**, specifying a mode where the image height is scaled to match the viewport height.

The default value is **BEST_FIT**.

alignment

A SymbolicConstant specifying the relative position of the image in the viewport when *positionMethod* = **AUTO_ALIGN**. Possible values are:

- **BOTTOM_LEFT**
- **BOTTOM_CENTER**
- **BOTTOM_RIGHT**
- **CENTER_LEFT**
- **CENTER**
- **CENTER_RIGHT**
- **TOP_LEFT**
- **TOP_CENTER**
- **TOP_RIGHT**

The default value is **CENTER**.

xScale

A Float specifying the scale applied to the image width. The *xScale* argument is ignored when *positionMethod* = **FIT_TO_VIEWPORT**. The default value is 1.0.

When *xScale* is negative, the image is mirrored about its y-axis but its position is not affected.

yScale

A Float specifying the scale applied to the image height. The *yScale* argument is ignored when *positionMethod* = **FIT_TO_VIEWPORT**. The default value is 1.0.

When *yScale* is negative, the image is mirrored about its x-axis but its position is not affected.

origin

A pair of Floats specifying the X- and Y-offsets in millimeters from the lower-left corner of the viewport. The *origin* argument is ignored unless *positionMethod* = **MANUAL**. The default value is (0, 0).

translucency

A Float specifying the translucency factor to use when displaying the image. Possible values are $0.0 \leq \text{translucency} \leq 1.0$ with 0.0 being invisible and 1.0 being opaque. The default value is 1.0.

options

None or an ImageOptions object specifying the object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in the *options* member. The default value is **None**.

Return value

None

Exceptions

RangeError.

11.5.2 Members

The ImageOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

11.6 MovieOptions object

The MovieOptions object stores settings that control how the movie background for an animation is rendered in a particular viewport. MovieOptions objects are accessed from the movie options associated with a particular viewport.

The MovieOptions object has no constructor; Abaqus creates the *movieOptions* member for a viewport when the viewport is created using the values in the *movieOptions* member of the current viewport.

Access

`session.viewports[name].movieOptions`

11.6.1 setValues(...)

This method modifies the MovieOptions object.

Arguments

Required arguments

None.

Optional arguments

movieName

A String specifying the name of the movie. A list of valid movie names is in the *movies* repository in the *session* object.

showMovie

A Boolean specifying whether a movie should be displayed in the viewport during an animation. The default value is OFF.

positionMethod

A SymbolicConstant specifying which positioning method is used to determine how the movie frames will be scaled and positioned in the viewport. Possible values are:

- FIT_TO_VIEWPORT, specifying a display mode where the movie frame is scaled to fit in the viewport using the specified *fitMethod*.
- AUTO_ALIGN, specifying a display mode where the movie frame is scaled as specified by *xScale* and *yScale* and then positioned in the viewport using the specified *alignment*.
- MANUAL, specifying a display mode where the movie frame is scaled as specified by *xScale* and *yScale* and then positioned in the viewport using the specified *origin*.

The default value is FIT_TO_VIEWPORT.

fitMethod

A SymbolicConstant specifying which type of fit is performed to scale and position the movie frame in the viewport when *positionMethod* = FIT_TO_VIEWPORT. Possible values are:

- BEST_FIT, specifying a mode where the movie frame is scaled to completely fit within the viewport.
- FIT_WIDTH, specifying a mode where the movie frame width is scaled to match the viewport width.
- FIT_HEIGHT, specifying a mode where the movie frame height is scaled to match the viewport height.

The default value is BEST_FIT.

alignment

A SymbolicConstant specifying the relative position of the movie frame in the viewport when *positionMethod* = AUTO_ALIGN. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT

- TOP_CENTER
- TOP_RIGHT

The default value is CENTER.

xScale

A Float specifying the scale applied to the movie frame width. The *xScale* argument is ignored when *positionMethod* = FIT_TO_VIEWPORT. The default value is 1.0.

When *xScale* is negative, the movie frame is mirrored about its y-axis but its position is not affected.

yScale

A Float specifying the scale applied to the movie frame height. The *yScale* argument is ignored when *positionMethod* = FIT_TO_VIEWPORT. The default value is 1.0.

When *yScale* is negative, the movie frame is mirrored about its x-axis but its position is not affected.

origin

A pair of Floats specifying the X- and Y-offsets in millimeters from the lower-left corner of the viewport. The *origin* argument is ignored unless *positionMethod* = MANUAL. The default value is (0, 0).

translucency

A Float specifying the translucency factor to use when displaying the movie frame. Possible values are $0.0 \leq \text{translucency} \leq 1.0$ with 0.0 being invisible and 1.0 being opaque. The default value is 1.0.

options

None or a MovieOptions object specifying the object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in the *options* member. The default value is **None**.

Return value

None

Exceptions

RangeError.

11.6.2 Members

The MovieOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

11.7 Highlight commands

The Highlight commands are used to highlight objects in the current viewport and to remove the highlighting.

11.7.1 highlight(...)

This method highlights an object in the current viewport.

Path

`highlight`

Arguments

Required argument

object

An object specifying the object in the current viewport to be highlighted. You can specify only a single object. The following objects are supported:

For the MDB

- Vertex
- Edge
- Face
- Surface
- Cell
- Node
- Element
- Element face
- Element edge
- Feature
- Datum
- Instance
- Set
- Load
- Boundary condition
- Predefined field
- Display group

For the ODB

- Node
- Element
- Display group

11.7.2 unhighlight(...)

This method removes highlighting from an object in the current viewport.

Path

`unhighlight`

Arguments**Required arguments**

object

An object specifying the object in the current viewport from which the highlighting will be removed. You can specify only a single object. See “highlight,” Section 11.7.1 for a list of supported objects.

12. Connector commands

A connector describes the relative motions between two points. A connector also describes the behavior associated with the relative motion.

12.1 ConnectorBehaviorOption object

The ConnectorBehaviorOption object is the abstract base type for other ConnectorBehaviorOption objects. The ConnectorBehaviorOption object has no explicit constructor. The members of the ConnectorBehaviorOption object are common to all objects derived from the ConnectorBehaviorOption.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.1.1 Members

The ConnectorBehaviorOption object has no members.

12.2 CDCTerm object

The CDCTerm object is used to create contributing terms for a DerivedComponent object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].derivedComponent.cdcTerms[i]
mdb.models[name].sections[name].behaviorOptions[i].derivedComponent \
.cdcTerms[i]
mdb.models[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i].derivedComponent.cdcTerms[i]
mdb.models[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i].derivedComponent.cdcTerms[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i] \
```

```

.connectorPotentials [i] .derivedComponent.cdcTerms [i]
session.odbs [name] .sections [name] .behaviorOptions [i] .derivedComponent\
.cdcTerms [i]
session.odbs [name] .sections [name] .behaviorOptions [i] \
.evolutionPotentials [i] .derivedComponent.cdcTerms [i]
session.odbs [name] .sections [name] .behaviorOptions [i] \
.initiationPotentials [i] .derivedComponent.cdcTerms [i]

```

12.2.1 CDCTerm(...)

This method creates a CDCTerm object.

Path

```

mdb.models [name] .sections [name] .behaviorOptions [i] \
.connectorPotentials [i] .derivedComponent.CDCTerm
mdb.models [name] .sections [name] .behaviorOptions [i] .derivedComponent\
.CDCTerm
mdb.models [name] .sections [name] .behaviorOptions [i] \
.evolutionPotentials [i] .derivedComponent.CDCTerm
mdb.models [name] .sections [name] .behaviorOptions [i] \
.initiationPotentials [i] .derivedComponent.CDCTerm
session.odbs [name] .sections [name] .behaviorOptions [i] \
.connectorPotentials [i] .derivedComponent.CDCTerm
session.odbs [name] .sections [name] .behaviorOptions [i] .derivedComponent\
.CDCTerm
session.odbs [name] .sections [name] .behaviorOptions [i] \
.evolutionPotentials [i] .derivedComponent.CDCTerm
session.odbs [name] .sections [name] .behaviorOptions [i] \
.initiationPotentials [i] .derivedComponent.CDCTerm

```

Required arguments

intrinsicComponents

A sequence of Ints specifying the components of relative motion for which the contributing term is defined. Possible values are $1 \leq \text{intrinsicComponents} \leq 6$. Only available components can be specified if the DerivedComponent object is being referenced by a Potential object. This is not the case if the DerivedComponent object is referenced by a ConnectorFriction object directly. The default value is an empty sequence.

table

A sequence of sequences of Floats specifying components numbers and temperature and field values. Each sequence of the table data specifies:

- The first intrinsic component number.
- If applicable, the second intrinsic component number.
- Etc.
- If applicable, the first independent component number.
- If applicable, the second independent component number.
- Etc.
- If applicable, the temperature value.
- If applicable, the value of the first field variable.
- If applicable, the value of the second field variable.
- Etc.

The default value is an empty sequence.

Optional arguments

termOperator

A SymbolicConstant specifying the method for combining contributing terms: square root of a sum of the squares, direct sum, or Macauley sum. Possible values are RSS, SUM, and MACAULEY. The default value is RSS.

termSign

A SymbolicConstant specifying the overall sign for the contributing term. Possible values are POSITIVE and NEGATIVE. The default value is POSITIVE.

localDependency

A Boolean specifying whether the table data depend on either components of relative position or components of constitutive relative motion. The default value is OFF.

indepCompType

A SymbolicConstant specifying whether localDependency refers to components of relative position or components of constitutive relative motion. Possible values are POSITION and MOTION. The default value is POSITION.

The *indepCompType* argument applies only if *localDependency*=ON.

indepComponents

A sequence of Ints specifying the independent components included in the derived component definition. Possible values are $1 \leq indepComponents \leq 6$. Only available components can be specified. The *indepComponents* argument applies only if *localDependency*=ON. The default value is an empty sequence.

tempDependency

A Boolean specifying whether the table data depend on temperature. The default value is OFF.

fieldDependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Return value

A CDCTerm object.

Exceptions

ValueError and TextError.

12.2.2 setValues(...)

This method modifies the CDCTerm object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CDCTerm** method.

Return value

None

Exceptions

ValueError.

12.2.3 Members

The CDCTerm object has members with the same names and descriptions as the arguments to the **CDCTerm** method.

In addition, the CDCTerm object can have the following member:

options

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for this ConnectorBehaviorOption .

12.3 ConnectorDamage object

The ConnectorDamage object defines damage behavior for one or more components of a connector's relative motion.

The ConnectorDamage object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.3.1 ConnectorDamage(...)

This method creates a connector damage behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorDamage
import odbConnectorBehavior
odbConnectorBehavior.ConnectorDamage
```

Required arguments

None.

Optional arguments*coupling*

A SymbolicConstant specifying whether or not the behavior is coupled. Possible values are UNCOUPLED and COUPLED. The default value is UNCOUPLED.

criterion

A SymbolicConstant specifying the damage initiation criterion to be used. Possible values are FORCE, MOTION, and PLASTIC_MOTION. The default value is FORCE.

initiationTemperature

A Boolean specifying whether the initiation data depend on temperature. The default value is OFF.

initiationPotentialOperator

A SymbolicConstant specifying the contribution operator for the initiation potential contributions. Possible values are SUM and MAXIMUM. The default value is SUM.

This argument is only if *coupling*=COUPLED and if *criterion*=FORCE or MOTION.

initiationPotentialExponent

A Float specifying the number equal to the inverse of the overall exponent in the initiation potential definition. The default value is 2.0.

This argument is applicable only if *coupling*=COUPLED, when *initiationPotentialOperator*=SUM, and when *criterion*=FORCE or MOTION.

CONNECTOR COMMANDS

initiationDependencies

An Int specifying the number of field variable dependencies for the initiation data. The default value is 0.

evolution

A Boolean specifying whether damage evolution data will be used. The default value is ON.

evolutionType

A SymbolicConstant specifying the type of damage evolution to be specified. Possible values are MOTION_TYPE and ENERGY_TYPE. The default value is MOTION_TYPE.

This argument is applicable only if *evolution*=ON.

softening

A SymbolicConstant specifying the damage evolution law to be specified. Possible values are LINEAR, EXPONENTIAL, and TABULAR. The default value is LINEAR.

This argument is applicable only if *evolution*=ON and when *evolutionType*=MOTION_TYPE.

useAffected

A Boolean specifying whether or not *affectedComponents* will be specified. If *useAffected*=OFF, then only the components of relative motion specified by *components* will undergo damage. The default value is OFF.

This argument is applicable only if *evolution*=ON.

degradation

A SymbolicConstant specifying the contribution of each damage mechanism when more than one damage mechanism is defined. Possible values are MAXIMUM and MULTIPLICATIVE. The default value is MAXIMUM.

This argument is applicable if *evolution*=ON.

evolutionTemperature

A Boolean specifying whether the evolution data depend on temperature. The default value is OFF.

This argument is applicable only if *evolution*=ON.

evolutionDependencies

An Int specifying the number of field variable dependencies for the evolution data. The default value is 0.

This argument is applicable only if *evolution*=ON.

evolutionPotentialOperator

A SymbolicConstant specifying the contribution operator for the evolution potential contributions. Possible values are SUM and MAXIMUM. The default value is SUM.

This argument is applicable only if *coupling*=COUPLED, when *evolution*=ON, when *evolutionType*=MOTION_TYPE, and when *criterion*=FORCE or MOTION.

evolutionPotentialExponent

A Float specifying the number equal to the inverse of the overall exponent in the evolution potential definition. The default value is 2.0.

This argument is applicable only if *coupling*=COUPLED, when *evolution*=ON, when *evolutionPotentialOperator*=SUM, when *evolutionType*=MOTION, and when *criterion*=FORCE or MOTION.

initiationPotentials

A ConnectorPotentialArray object specifying one ConnectorPotential object for each initiation potential contribution. This member can be specified only if *coupling*=COUPLED and if *criterion*=FORCE or MOTION.

evolutionPotentials

A ConnectorPotentialArray object specifying one ConnectorPotential object for each evolution potential contribution). This member can be specified only if *coupling*=COUPLED, if *evolution*=ON, if *evolutionType*=MOTION, and if *criterion*=FORCE or MOTION.

initiationTable

A sequence of sequences of Floats specifying the initiation properties. The default value is an empty sequence.

Items in the *initiationTable* data are described below.

evolutionTable

A sequence of sequences of Floats specifying the evolution properties. The default value is an empty sequence.

Items in the *evolutionTable* data are described below. This argument is only applicable if *evolution*=ON.

affectedComponents

A sequence of Ints specifying the components of relative motion that will be damaged. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. This argument is applicable only if *evolution*=ON and *useAffected*=ON. The default value is an empty sequence.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. This argument can be specified only if *coupling*=UNCOUPLD. The default value is an empty sequence.

Table data

Table data for *initiationTable*:

If *criterion*=FORCE, then each sequence of the table data specifies the following:

- Lower (compression) limiting force or moment. Use -1.0E+36 to indicate an unspecified lower limit.
- Upper (tension) limiting force or moment. Use 1.0E+36 to indicate an unspecified upper limit. At least one limit, lower or upper, must be specified.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

CONNECTOR COMMANDS

- Etc.

If *criterion*=MOTION, then each sequence of the table data specifies the following:

- Lower (compression) limiting connector constitutive relative displacement or rotation. Use -1.0E+36 to indicate an unspecified lower limit.
- Upper (tension) limiting connector constitutive relative displacement or rotation. Use 1.0E+36 to indicate an unspecified upper limit. At least one limit, lower or upper, must be specified.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *criterion*=PLASTIC_MOTION, then each sequence of the table data specifies the following:

- Relative equivalent plastic displacement/rotation at which damage will be initiated.
- Mode-mix ratio (only if *coupling*=COUPLED).
- Relative equivalent plastic displacement/rotation rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Table data for *evolutionTable*:

If *evolutionType*=MOTION and *softening*=LINEAR, then each sequence of the table data specifies the following:

- Post-initiation equivalent relative plastic motion at ultimate failure if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation) at ultimate failure.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=MOTION and *softening*=EXPONENTIAL, then each sequence of the table data specifies the following:

- Post-initiation equivalent relative plastic motion at ultimate failure if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation) at ultimate failure.
- Exponential law parameter.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=MOTION and *softening*=TABULAR, then each sequence of the table data specifies the following:

- Damage variable (cannot be less than 0 or greater than 1).
- Post-initiation equivalent relative plastic motion if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation).
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=ENERGY, then each sequence of the table data specifies the following:

- Total energy dissipated by damage at ultimate failure.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorDamage object.

Exceptions

ValueError and TextError.

12.3.2 setValues(...)

This method modifies the ConnectorDamage object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorDamage** method.

Return value

None

Exceptions

ValueError.

12.3.3 Members

The **ConnectorDamage** object has members with the same names and descriptions as the arguments to the **ConnectorDamage** method.

In addition, the **ConnectorDamage** object can have the following members:

initiationOptions

A **ConnectorOptions** object specifying the **ConnectorOptions** used to define tabular options for the damage initiation table.

evolutionOptions

A **ConnectorOptions** object specifying the **ConnectorOptions** used to define tabular options for the damage evolution table.

12.3.4 Corresponding analysis keywords

*CONNECTOR DAMAGE INITIATION, *CONNECTOR DAMAGE EVOLUTION,
*CONNECTOR POTENTIAL

12.4 ConnectorDamping object

The **ConnectorDamping** object defines damping behavior for one or more components of a connector's relative motion.

The **ConnectorDamping** object is derived from the **ConnectorBehaviorOption** object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
```

```
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.4.1 ConnectorDamping(...)

This method creates a connector damping behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorDamping
import odbConnectorBehavior
odbConnectorBehavior.ConnectorDamping
```

Required arguments

None.

Optional arguments

behavior

A SymbolicConstant specifying if the damping behavior is linear or nonlinear. Possible values are LINEAR and NONLINEAR. The default value is LINEAR.

coupling

A SymbolicConstant specifying whether the damping behavior is coupled between the connector's components of relative motion. If *behavior*=LINEAR, then possible values are UNCOUPLED and COUPLED. If *behavior*=NONLINEAR, then possible values are UNCOUPLED, COUPLED_POSITION, and COUPLED_MOTION. Possible values are UNCOUPLED, COUPLED, COUPLED_POSITION, and COUPLED_MOTION. The default value is UNCOUPLED.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

temperatureDependency

A Boolean specifying whether the behavior data depend on temperature. The default value is OFF.

frequencyDependency

A Boolean specifying whether the behavior data depend on frequency. This value is applicable only if *behavior*= LINEAR and *coupling*=UNCOUPLED. The default value is OFF.

table

A sequence of sequences of Floats specifying damping properties. Items in the table data are described below. The default value is an empty sequence.

CONNECTOR COMMANDS

independentComponents

A sequence of Ints specifying the list of independent components that are included in the definition of the connector damping data. This argument is applicable only if *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION. When this argument is applicable, at least one value must be specified. Only available components can be specified. The default value is an empty sequence.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq components \leq 6$. Only available components can be specified. The default value is an empty sequence.

Table data

If *behavior*=LINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Damping coefficient (force or moment per relative velocity).
- Frequency (cycles/time), if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Force or moment.
- Relative displacement or rotation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=LINEAR and *coupling*=COUPLED, the table data specify the symmetric portion of the damping matrix for the specified components followed by any temperature data and then any field data. For example, if components 2, 3, and 5 are specified, the table portion of the command is as follows:

table= ((C₂₂, C₂₃, C₂₅, C₃₃, C₃₅, C₅₅), ,)

The following items should then be specified as comma-separated data:

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION, each sequence of the table data specifies the following:

- Force or moment for the directions in the *components* list.
- Relative velocity for the directions in the *components* list.
- Relative position or angle for the first *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the first *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Relative position or angle for the second *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the second *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Etc. up to the Nth *independentComponents* direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorDamping object.

Exceptions

ValueError and TextError.

12.4.2 **setValues(...)**

This method modifies the ConnectorDamping object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorDamping** method.

Return value

None

Exceptions

ValueError.

12.4.3 Members

The ConnectorDamping object has members with the same names and descriptions as the arguments to the **ConnectorDamping** method.

In addition, the ConnectorDamping object can have the following member:

options

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for this ConnectorBehaviorOption .

12.4.4 Corresponding analysis keywords

*CONNECTOR DAMPING

12.5 ConnectorElasticity object

The ConnectorElasticity object defines elastic behavior for one or more components of a connector's relative motion.

The ConnectorElasticity object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.5.1 ConnectorElasticity(...)

This method creates a connector elasticity behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorElasticity
```



```
import odbConnectorBehavior
odbConnectorBehavior.ConnectorElasticity
```

Required arguments

None.

Optional arguments

behavior

A SymbolicConstant specifying whether the elastic behavior is linear, nonlinear, or rigid. Possible values are LINEAR, NONLINEAR, and RIGID. The default value is LINEAR.

coupling

A SymbolicConstant specifying whether the elastic behavior is coupled between the connector's components of relative motion. If *behavior*=LINEAR, then possible values are UNCOUPLED and COUPLED. If *behavior*=NONLINEAR, then possible values are UNCOUPLED, COUPLED_POSITION, and COUPLED_MOTION. Possible values are UNCOUPLED, COUPLED, COUPLED_POSITION, and COUPLED_MOTION. The default value is UNCOUPLED.

This argument is not applicable if *behavior*=RIGID.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

This argument is not applicable if *behavior*=RIGID.

temperatureDependency

A Boolean specifying whether the behavior data depend on temperature. The default value is OFF.

This argument is not applicable if *behavior*=RIGID.

frequencyDependency

A Boolean specifying whether the behavior data depend on frequency. This value is applicable only if *behavior*=LINEAR and *coupling*=UNCOUPLED. The default value is OFF.

This argument is not applicable if *behavior*=RIGID.

table

A sequence of sequences of Floats specifying elasticity properties. Items in the table data are described below. This argument is not applicable if *behavior*=RIGID. The default value is an empty sequence.

independentComponents

A sequence of Ints specifying the list of independent components that are included in the definition of the connector elasticity data. This argument is applicable only if *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION. If this argument is applicable, at least one value must be specified. Only available components can be specified. The default value is an empty sequence.

CONNECTOR COMMANDS

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Table data

If *behavior*=LINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Elastic stiffness (per each specified force/moment component).
- Frequency (cycles/time), if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Force or moment.
- Relative displacement or rotation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=LINEAR and *coupling*=COUPLED, the table data specify the symmetric portion of the elastic stiffness matrix for the specified components followed by any temperature data and then any field data. For example, if components 2, 3, and 5 are specified, the table portion of the command is as follows:

```
table= ( (D22, D23, D25, D33, D35, D55),  
)
```

The following items should then be specified as comma-separated data:

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION, each sequence of the table data specifies the following:

- Force or moment for the directions in the *components* list.

- Relative position or angle for the first *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the first *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Relative position or angle for the second *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the second *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Etc. up to the Nth *independentComponents* direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorElasticity object.

Exceptions

ValueError and TextError.

12.5.2 **setValues(...)**

This method modifies the ConnectorElasticity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorElasticity** method.

Return value

None

Exceptions

ValueError.

12.5.3 Members

The ConnectorElasticity object has members with the same names and descriptions as the arguments to the **ConnectorElasticity** method.

In addition, the ConnectorElasticity object can have the following member:

options

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for this ConnectorBehaviorOption .

12.5.4 Corresponding analysis keywords

*CONNECTOR ELASTICITY

12.6 ConnectorFailure object

The ConnectorFailure object defines failure criteria for one or more components of a connector's relative motion.

The ConnectorFailure object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.6.1 ConnectorFailure(...)

This method creates a connector failure behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorFailure
import odbConnectorBehavior
odbConnectorBehavior.ConnectorFailure
```

Required arguments

None.

Optional arguments*releaseComponent*

The SymbolicConstant ALL or an Int specifying the motion components that fail. If an Int is specified, only that motion component fails when the failure criteria are satisfied. If *releaseComponent*=ALL, all motion components fail. The default value is ALL.

minMotion

None or a Float specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is **None**.

maxMotion

None or a Float specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is **None**.

minForce

None or a Float specifying the lower bound of the force or moment in the directions of the specified components at which locking occurs, or no lower bound. The default value is **None**.

maxForce

None or a Float specifying the upper bound of the force or moment in the directions of the specified components at which locking occurs, or no upper bound. The default value is **None**.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorFailure object.

Exceptions

ValueError and TextError.

12.6.2 setValues(...)

This method modifies the ConnectorFailure object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorFailure** method.

Return value

None

Exceptions

ValueError.

12.6.3 Members

The **ConnectorFailure** object has members with the same names and descriptions as the arguments to the **ConnectorFailure** method.

12.6.4 Corresponding analysis keywords

*CONNECTOR FAILURE

12.7 ConnectorFriction object

The **ConnectorFriction** object defines Coulomb-like or hysteretic friction behavior for one or more components of a connector's relative motion.

The **ConnectorFriction** object is derived from the **ConnectorBehaviorOption** object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.7.1 ConnectorFriction(...)

This method creates a connector friction behavior option for a **ConnectorSection** object. Depending upon the arguments provided, the friction behavior can be Coulomb-like or hysteretic in nature.

Path

```
import connectorBehavior
connectorBehavior.ConnectorFriction
import odbConnectorBehavior
odbConnectorBehavior.ConnectorFriction
```

Required arguments

None.

Optional arguments*frictionModel*

A SymbolicConstant specifying the desired frictional response model. Possible values are PREDEFINED and USER_CUSTOMIZED. The default value is PREDEFINED.

slipStyle

A SymbolicConstant specifying the method of indicating the slip direction: either specified or computed based upon the force potential data. Possible values are SPECIFY and COMPUTE. The default value is SPECIFY.

This argument is applicable only if *frictionModel*=USER_CUSTOMIZED.

tangentDirection

None or an Int specifying the direction for which the frictional behavior is specified. Possible values are $1 \leq \textit{tangentDirection} \leq 6$, indicating an available component of relative motion. This argument applies only if *frictionModel*=USER_CUSTOMIZED and if *slipStyle*=SPECIFY. The default value is **None**.

stickStiffness

None or a Float specifying the stick stiffness associated with the frictional behavior in the direction specified by *tangentDirection*. If this argument is omitted, Abaqus computes an appropriate number for the stick stiffness. The default value is **None**.

componentType

A SymbolicConstant specifying the type of the *independentComponents*. Possible values are POSITION, MOTION, and NO_INDEPENDENT_COMPONENTS. The default value is NO_INDEPENDENT_COMPONENTS.

slipDependency

A Boolean specifying whether the table data depend on accumulated slip. The default value is OFF.

This argument applies only if *frictionModel*=USER_CUSTOMIZED.

temperatureDependency

A Boolean specifying whether the table data depend on temperature. The default value is OFF.

CONNECTOR COMMANDS

This argument applies only if *frictionModel*=USER_CUSTOMIZED.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

This argument applies only if *frictionModel*=USER_CUSTOMIZED.

useContactForceComponent

A Boolean specifying whether the contact force component will be defined. The default value is OFF.

This argument applies only if *frictionModel*=USER_CUSTOMIZED.

contactForceStyle

A SymbolicConstant specifying the method of indicating the contact force component direction: either specified or computed based on upon a DerivedComponent. Possible values are COMPONENT_NUMBER and DERIVED_COMPONENT. The default value is COMPONENT_NUMBER.

This argument is applicable only if *frictionModel*=USER_CUSTOMIZED and if *useContactForceComponent*=ON.

contactForceComponent

An Int specifying the contact force component direction. This argument applies only if *frictionModel*=USER_CUSTOMIZED, if *useContactForceComponent*=ON, and if *contactForceStyle*=COMPONENT_NUMBER. The default value is 0.

forcePotentialOperator

A SymbolicConstant specifying the contribution operator for the force potential contributions. Possible values are SUM and MAXIMUM. The default value is SUM.

This argument is applicable only if *frictionModel*=USER_CUSTOMIZED and if *slipStyle*=COMPUTE.

forcePotentialExponent

A Float specifying the number equal to the inverse of the overall exponent in the force potential definition. The default value is 2.0.

This argument is applicable only if *frictionModel*=USER_CUSTOMIZED, if *slipStyle*=COMPUTE, and if *forcePotentialOperator*=SUM.

connectorPotentials

A ConnectorPotentialArray object specifying one ConnectorPotential object for each force potential contribution. This member can be specified only if *frictionModel*=USER_CUSTOMIZED, and if *slipStyle*=COMPUTE.

table

A sequence of sequences of Floats specifying friction properties. The default value is an empty sequence.

If *frictionModel*=PREDEFINED, each sequence of the table data specifies:

- If applicable, the first geometric scaling constant relevant to frictional interactions.
- Etc., up to as many geometric scaling constants as are associated with this connection type.

- Internal contact force/moment generating friction in the first predefined slip direction.
- If applicable, internal contact force/moment generating friction in the second predefined slip direction.
- Connector constitutive relative motion in the direction specified by *independentComponent*.
- Accumulated slip in the first predefined slip direction, if the data depend on accumulated slip.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *frictionModel*=USER_CUSTOMIZED, each sequence of the table data specifies:

- Effective radius of the cylindrical or spherical surface over which frictional slip occurs in the connector associated with frictional effects in the direction specified by *tangentDirection*. This radius is relevant only if the connection type includes an available rotational component of relative motion and *tangentDirection*=SLIP_DIRECTION.
- Internal contact force/moment generating friction in the direction specified by *tangentDirection*.
- Connector constitutive relative motion in the direction specified by *independentComponent*.
- Accumulated slip in the direction specified by *tangentDirection*, if the data depend on accumulated slip.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

independentComponents

A sequence of Ints specifying the independent components. Possible values are $1 \leq \text{independentComponents} \leq 6$. In addition, each independent component value must be unique. The *independentComponents* argument applies only if *frictionModel*=USER_CUSTOMIZED. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorFriction object.

Exceptions

ValueError and TextError.

12.7.2 **setValues(...)**

This method modifies the `ConnectorFriction` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorFriction** method.

Return value

None

Exceptions

`ValueError`.

12.7.3 **Members**

The `ConnectorFriction` object has members with the same names and descriptions as the arguments to the **ConnectorFriction** method.

In addition, the `ConnectorFriction` object can have the following members:

tangentialBehavior

A `TangentialBehavior` object.

derivedComponent

A `DerivedComponent` object specifying the `DerivedComponent` used to compute the contact force component direction. This argument applies only if *frictionModel*=`USER_CUSTOMIZED`, if *useContactForceComponent*=`ON`, and if *contactForceStyle*=`DERIVED_COMPONENT`.

options

A `ConnectorOptions` object specifying the `ConnectorOptions` used to define tabular options for this `ConnectorBehaviorOption`.

12.7.4 **Corresponding analysis keywords**

`*CONNECTOR FRICTION`, `*FRICTION`, `*CONNECTOR POTENTIAL`, `*CONNECTOR DERIVED COMPONENT`

12.8 ConnectorLock object

The ConnectorLock object defines locking criteria for one or more available components of a connector's relative motion.

The ConnectorLock object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.8.1 ConnectorLock(...)

This method creates a connector lock behavior option for a ConnectorSection .

Path

```
import connectorBehavior
connectorBehavior.ConnectorLock
import odbConnectorBehavior
odbConnectorBehavior.ConnectorLock
```

Required arguments

None.

Optional arguments

lockingComponent

The SymbolicConstant ALL or an Int specifying the motion components that are locked. If an Int is specified, only that motion component is locked when the locking criteria are satisfied. If *lockingComponent*=ALL, all motion components are locked. The default value is ALL.

minMotion

None or a Float specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is **None**.

maxMotion

None or a Float specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is **None**.

CONNECTOR COMMANDS

minForce

None or a Float specifying the lower bound of the force or moment in the directions of the specified components at which locking occurs, or no lower bound. The default value is **None**.

maxForce

None or a Float specifying the upper bound of the force or moment in the directions of the specified components at which locking occurs, or no upper bound. The default value is **None**.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorLock object.

Exceptions

ValueError and TextError.

12.8.2 **setValues(...)**

This method modifies the ConnectorLock object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorLock** method.

Return value

None

Exceptions

ValueError.

12.8.3 **Members**

The ConnectorLock object has members with the same names and descriptions as the arguments to the **ConnectorLock** method.

12.8.4 Corresponding analysis keywords

*CONNECTOR LOCK

12.9 ConnectorOptions object

The ConnectorOptions object is used to define various options for connector behaviors. It can be used only in conjunction with CDCTerm , ConnectorDamage , ConnectorDamping , ConnectorElasticity , ConnectorFriction , and ConnectorPlasticity objects. Because the ConnectorDamage object contains two separate ConnectorOptions repositories (one for damage initiation and one for damage evolution), there are two ConnectorOptions constructors associated with that behavior—initiationOptions and evolutionOptions. The ConnectorPlasticity object also contains two separate ConnectorOptions repositories (one for isotropic hardening and one for kinematic hardening), so there are two ConnectorOptions constructors associated with that behavior— isotropicOptions and kinematicOptions.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].derivedComponent.cdcTerms[i].options
mdb.models[name].sections[name].behaviorOptions[i].derivedComponent \
.cdcTerms[i].options
mdb.models[name].sections[name].behaviorOptions[i].evolutionOptions
mdb.models[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i].derivedComponent.cdcTerms[i].options
mdb.models[name].sections[name].behaviorOptions[i].initiationOptions
mdb.models[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i].derivedComponent.cdcTerms[i].options
mdb.models[name].sections[name].behaviorOptions[i].isotropicOptions
mdb.models[name].sections[name].behaviorOptions[i].kinematicOptions
mdb.models[name].sections[name].behaviorOptions[i].options
import odbSection
session.odbs[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].derivedComponent.cdcTerms[i].options
session.odbs[name].sections[name].behaviorOptions[i].derivedComponent \
.cdcTerms[i].options
session.odbs[name].sections[name].behaviorOptions[i].evolutionOptions
session.odbs[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i].derivedComponent.cdcTerms[i].options
session.odbs[name].sections[name].behaviorOptions[i].initiationOptions
```

```

session.odbs [name] .sections [name] .behaviorOptions [i] \
    .initiationPotentials [i] .derivedComponent.cdcTerms [i] .options
session.odbs [name] .sections [name] .behaviorOptions [i] .isotropicOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .kinematicOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .options

```

12.9.1 ConnectorOptions(...)

This method creates a connector options object to be used in conjunction with an allowable connector behavior option, derived component term, or connector section.

Path

```

mdb.models [name] .sections [name] .behaviorOptions [i] \
    .connectorPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] .derivedComponent \
    .cdcTerms [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] \
    .evolutionPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] \
    .initiationPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
mdb.models [name] .sections [name] .behaviorOptions [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .connectorPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .derivedComponent \
    .cdcTerms [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .evolutionPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .initiationPotentials [i] .derivedComponent.cdcTerms [i] .ConnectorOptions
session.odbs [name] .sections [name] .behaviorOptions [i] .ConnectorOptions

```

Required arguments

None.

Optional arguments*useBehRegSettings*

A Boolean specifying whether or not to use the behavior-level settings for regularization options. This argument is applicable only for an Abaqus/Explicit analysis. The default value is ON.

regularize

A Boolean specifying whether or not the tabular data will be regularized. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=OFF. The default value is ON.

defaultTolerance

A Boolean specifying whether or not the analysis default regularization tolerance will be used. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=OFF and *regularize*=ON. The default value is ON.

regularization

A Float specifying the regularization increment to be used. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=OFF, *regularize*=ON, and *defaultTolerance*=OFF. The default value is 0.03.

defaultRateFactor

A Boolean specifying whether or not the analysis default rate filter factor will be used. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. The default value is ON.

rateFactor

A Float specifying the rate filter factor to be used. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. This argument is also applicable only if *defaultRateFactor*=OFF. The default value is 0.9.

interpolation

A SymbolicConstant specifying the type of interpolation increment to be used on rate-dependent tabular data. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. Possible values are LINEAR and LOGARITHMIC. The default value is LINEAR.

useBehExtSettings

A Boolean specifying whether or not to use the behavior-level settings for extrapolation options. The default value is ON.

extrapolation

A SymbolicConstant specifying the extrapolation technique to be used. This argument is applicable only if *useBehExtSettings*=OFF. Possible values are CONSTANT and LINEAR. The default value is CONSTANT.

Return value

A `ConnectorOptions` object.

Exceptions

`ValueError` and `TextError`.

12.9.2 `setValues(...)`

This method modifies the `ConnectorOptions` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConnectorOptions` method.

Return value

`None`

Exceptions

`ValueError`.

12.9.3 Members

The `ConnectorOptions` object has members with the same names and descriptions as the arguments to the `ConnectorOptions` method.

12.9.4 Corresponding analysis keywords

`*CONNECTOR BEHAVIOR`, `*CONNECTOR DAMAGE INITIATION`, `*CONNECTOR DAMAGE EVOLUTION`, `*CONNECTOR DAMPING`, `*CONNECTOR DERIVED COMPONENT`, `*CONNECTOR ELASTICITY`, `*CONNECTOR FRICTION`, and `*CONNECTOR PLASTICITY`

12.10 ConnectorPlasticity object

The ConnectorPlasticity object defines plastic behavior for one or more components of a connector's relative motion.

The ConnectorPlasticity object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.10.1 ConnectorPlasticity(...)

This method creates a connector plasticity behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorPlasticity
import odbConnectorBehavior
odbConnectorBehavior.ConnectorPlasticity
```

Required arguments

None.

Optional arguments

coupling

A SymbolicConstant specifying whether or not the behavior is coupled. Possible values are UNCOUPLED and COUPLED. The default value is UNCOUPLED.

isotropic

A Boolean specifying whether isotropic hardening data will be used. The default value is ON.

If *isotropic*=OFF, then *kinematic* must be specified as ON.

isotropicType

A SymbolicConstant specifying the type of isotropic hardening to be specified. Possible values are TABULAR and EXPONENTIAL_LAW. The default value is TABULAR.

This argument is applicable only if *isotropic*=ON.

CONNECTOR COMMANDS

isotropicTemperature

A Boolean specifying whether the isotropic data depend on temperature. The default value is OFF.
This argument is applicable only if *isotropic*=ON.

isotropicDependencies

An Int specifying the number of field variable dependencies for the isotropic data. The default value is 0.
This argument is applicable only if *isotropic*=ON.

kinematic

A Boolean specifying whether kinematic hardening data will be used. The default value is OFF.
If *kinematic*=OFF, then *isotropic* must be specified as ON.

kinematicType

A SymbolicConstant specifying the type of kinematic hardening to be specified. Possible values are HALF_CYCLE, STABILIZED, and PARAMETERS. The default value is HALF_CYCLE.
This argument is applicable only if *kinematic*=ON.

kinematicTemperature

A Boolean specifying whether the kinematic data depend on temperature. The default value is OFF.
This argument is applicable only if *kinematic*=ON.

kinematicDependencies

An Int specifying the number of field variable dependencies for the kinematic data. The default value is 0.
This argument is applicable only if *kinematic*=ON.

forcePotentialOperator

A SymbolicConstant specifying the contribution operator for the force potential contributions. Possible values are SUM and MAXIMUM. The default value is SUM.
This argument is applicable only if *coupling*=COUPLED.

forcePotentialExponent

A Float specifying the number equal to the inverse of the overall exponent in the force potential definition. The default value is 2.0.
This argument is applicable only if *coupling*=COUPLED and if *forcePotentialOperator*=SUM.

connectorPotentials

A ConnectorPotentialArray object specifying one ConnectorPotential object for each force potential contribution. This member can be specified only if *coupling*=COUPLED.

isotropicTable

A sequence of sequences of Floats specifying isotropic plasticity properties. Items in the *isotropicTable* data are described below. This argument is applicable only if *isotropic*=ON. The default value is an empty sequence.

kinematicTable

A sequence of sequences of Floats specifying kinematic plasticity properties. Items in the *kinematicTable* data are described below. This argument is applicable only if *kinematic*=ON. The default value is an empty sequence.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. This argument can be specified only if *coupling*=UNCOUPLED. The default value is an empty sequence.

Table data

Table data for *isotropicTable*:

If *isotropicType*=TABULAR, then each sequence of the table data specifies the following:

- Equivalent yield force or moment defining the size of the elastic range.
- Equivalent relative plastic motion.
- Equivalent relative plastic motion rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *isotropicType*=EXPONENTIAL_LAW, then each sequence of the table data specifies the following:

- Equivalent force or moment defining the size of the elastic range at zero plastic motion.
- Isotropic hardening parameter Q_{inf} .
- Isotropic hardening parameter b .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Table data for *kinematicTable*:

If *kinematicType*=HALF_CYCLE, then each sequence of the table data specifies the following:

- Yield force or moment.
- Connector relative plastic motion.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

CONNECTOR COMMANDS

If *kinematicType*=STABILIZED, then each sequence of the table data specifies the following:

- Yield force or moment.
- Connector relative plastic motion.
- Connector relative constitutive motion range.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *kinematicType*=PARAMETERS, then each sequence of the table data specifies the following:

- Yield force or moment at zero relative plastic motion.
- Kinematic hardening parameter C .
- Kinematic hardening parameter γ . Set $\gamma=0$ to specify linear Ziegler kinematic hardening.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorPlasticity object.

Exceptions

ValueError and TextError.

12.10.2 setValues(...)

This method modifies the ConnectorPlasticity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorPlasticity** method.

Return value

None

Exceptions

ValueError.

12.10.3 Members

The ConnectorPlasticity object has members with the same names and descriptions as the arguments to the **ConnectorPlasticity** method.

In addition, the ConnectorPlasticity object can have the following members:

isotropicOptions

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for the isotropic hardening table.

kinematicOptions

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for the kinematic hardening table.

12.10.4 Corresponding analysis keywords

*CONNECTOR PLASTICITY, *CONNECTOR HARDENING, *CONNECTOR POTENTIAL

12.11 ConnectorPotential object

The ConnectorPotential object is used to define a restricted set of mathematical functions to represent yield or limiting surfaces in the space spanned by connector available components. It can be used only in conjunction with ConnectorDamage , ConnectorFriction , and ConnectorPlasticity objects. Because the ConnectorDamage object contains two separate ConnectorPotential repositories (one for damage initiation and one for damage evolution), there are two ConnectorPotential constructors associated with that behavior—IniPotential and EvoPotential.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i]
mdb.models[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i]
mdb.models[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i]
```

```

import odbSection
session.odbs[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i]
session.odbs[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i]
session.odbs[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i]

```

12.11.1 ConnectorPotential(...)

This method creates a connector potential object to be used in conjunction with an allowable connector behavior option.

Path

```

mdb.models[name].sections[name].behaviorOptions[i].ConnectorPotential
session.odbs[name].sections[name].behaviorOptions[i] \
.ConnectorPotential

```

Required arguments

None.

Optional arguments

componentStyle

A SymbolicConstant specifying whether a component number or the name of the DerivedComponent object will be used in the contribution. Possible values are COMPONENT_NUMBER and DERIVED_COMPONENT. The default value is COMPONENT_NUMBER.

componentNumber

An Int specifying the component number used in the contribution. This argument is applicable only if *componentStyle*=COMPONENT_NUMBER. Possible values are $1 \leq \text{componentNumber} \leq 6$. Only available components can be specified. The default value is 0.

sign

A SymbolicConstant specifying the sign of the contribution. Possible values are POSITIVE and NEGATIVE. The default value is POSITIVE.

scaleFactor

A Float specifying the scaling factor for the contribution. The default value is 1.0.

positiveExponent

A Float specifying the positive exponent for the contribution. The default value is 2.0.

This argument is ignored if the potential operator of the invoking behavior option is set to MAXIMUM.

shiftFactor

A Float specifying the shift factor for the contribution. The default value is 0.0.

hFunction

A SymbolicConstant specifying the H function of the contribution: either absolute value, Macauley bracket, or the identity function. Possible values are ABS, MACAULEY, and IDENTITY. The default value is ABS.

The value of IDENTITY can be used only if *positiveExponent*=1.0 and the potential exponent of the invoking behavior option is also 1.0 (i.e., the potential operator of the invoking behavior option must be SUM).

Return value

A ConnectorPotential object.

Exceptions

ValueError and TextError.

12.11.2 setValues(...)

This method modifies the ConnectorPotential object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorPotential** method.

Return value

None

Exceptions

ValueError.

12.11.3 Members

The ConnectorPotential object has members with the same names and descriptions as the arguments to the **ConnectorPotential** method.

In addition, the ConnectorPotential object can have the following member:

derivedComponent

A DerivedComponent object specifying the DerivedComponent used in the contribution. This argument is applicable only if *componentStyle*=DERIVED_COMPONENT.

12.11.4 Corresponding analysis keywords

*CONNECTOR POTENTIAL

12.12 ConnectorStop object

The ConnectorStop object defines connector stops for one or more components of a connector's relative motion.

The ConnectorStop object is derived from the ConnectorBehaviorOption object.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i]
import odbSection
session.odbs[name].sections[name].behaviorOptions[i]
```

12.12.1 ConnectorStop(...)

This method creates a connector stop behavior option for a ConnectorSection object.

Path

```
import connectorBehavior
connectorBehavior.ConnectorStop
import odbConnectorBehavior
odbConnectorBehavior.ConnectorStop
```

Required arguments

None.

Optional arguments*minMotion*

None or a Float specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is **None**.

maxMotion

None or a Float specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is **None**.

components

A sequence of Ints specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorStop object.

Exceptions

ValueError and TextError.

12.12.2 setValues(...)

This method modifies the ConnectorStop object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorStop** method.

Return value

None

Exceptions

ValueError.

12.12.3 Members

The ConnectorStop object has members with the same names and descriptions as the arguments to the **ConnectorStop** method.

12.12.4 Corresponding analysis keywords

*CONNECTOR STOP

12.13 DerivedComponent object

A `DerivedComponent` object describes user-customized components for use in defining `ConnectorFriction` and `Potential` objects.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].derivedComponent
mdb.models[name].sections[name].behaviorOptions[i].derivedComponent
mdb.models[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i].derivedComponent
mdb.models[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i].derivedComponent
import odbSection
session.odbs[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].derivedComponent
session.odbs[name].sections[name].behaviorOptions[i].derivedComponent
session.odbs[name].sections[name].behaviorOptions[i] \
.evolutionPotentials[i].derivedComponent
session.odbs[name].sections[name].behaviorOptions[i] \
.initiationPotentials[i].derivedComponent
```

12.13.1 DerivedComponent()

This method creates a `DerivedComponent` object.

Path

```
mdb.models[name].sections[name].behaviorOptions[i] \
.connectorPotentials[i].DerivedComponent
mdb.models[name].sections[name].behaviorOptions[i].DerivedComponent
mdb.models[name].sections[name].behaviorOptions[i] \
```

```

    .evolutionPotentials [i] .DerivedComponent
mdb.models [name] .sections [name] .behaviorOptions [i] \
    .initiationPotentials [i] .DerivedComponent
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .connectorPotentials [i] .DerivedComponent
session.odbs [name] .sections [name] .behaviorOptions [i] .DerivedComponent
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .evolutionPotentials [i] .DerivedComponent
session.odbs [name] .sections [name] .behaviorOptions [i] \
    .initiationPotentials [i] .DerivedComponent

```

Arguments

None.

Return value

A `DerivedComponent` object.

Exceptions

`ValueError` and `TextError`.

12.13.2 setValues(...)

This method modifies the `DerivedComponent` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `DerivedComponent` method.

Return value

`None`

Exceptions

`ValueError`.

12.13.3 Members

The DerivedComponent object can have the following member:

cdcTerms

A CDCTermArray object.

12.13.4 Corresponding analysis keywords

*CONNECTOR DERIVED COMPONENT

12.14 TangentialBehavior object

The TangentialBehavior object specifies tangential behavior for a connector friction behavior option.

Access

```
import section
mdb.models[name].sections[name].behaviorOptions[i].tangentialBehavior
import odbSection
session.odbs[name].sections[name].behaviorOptions[i] \
.tangentialBehavior
```

12.14.1 TangentialBehavior(...)

This method creates a TangentialBehavior object.

Path

```
mdb.models[name].sections[name].behaviorOptions[i].TangentialBehavior
session.odbs[name].sections[name].behaviorOptions[i] \
.TangentialBehavior
```

Required arguments

None.

Optional arguments

formulation

A SymbolicConstant specifying the friction coefficient formulation. Possible values are PENALTY and EXPONENTIAL_DECAY. The default value is PENALTY.

slipRateDependency

A Boolean specifying whether the data depend on slip rate. The default value is OFF.

pressureDependency

A Boolean specifying whether the data depend on contact pressure. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variables for the data. The default value is 0.

exponentialDecayDefinition

A SymbolicConstant specifying the exponential decay definition for the data. Possible values are COEFFICIENTS and TEST_DATA. The default value is COEFFICIENTS.

shearStressLimit

None or a Float specifying no upper limit or the friction coefficient shear stress limit. The default value is **None**.

maximumElasticSlip

A SymbolicConstant specifying the method for modifying the allowable elastic slip. Possible values are FRACTION and ABSOLUTE_DISTANCE. The default value is FRACTION.

This argument applies only to Abaqus/Standard analyses.

fraction

A Float specifying the ratio of the allowable maximum elastic slip to a characteristic model dimension. The default value is 10^{-4} .

This argument applies only to Abaqus/Standard analyses.

absoluteDistance

None or a Float specifying the absolute magnitude of the allowable elastic slip. The default value is **None**.

This argument applies only to Abaqus/Standard analyses.

table

A sequence of sequences of Floats specifying the tangential properties. Items in the table data are described below. The default value is an empty sequence.

Table data

If *formulation*=PENALTY, the table data specify the following:

- Friction coefficient in the slip direction, μ .
- Slip rate, if the data depend on slip rate.
- Contact pressure, if the data depend on contact pressure.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

CONNECTOR COMMANDS

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=COEFFICIENTS, the table data specify the following:

- Static friction coefficient, μ_s .
- Kinetic friction coefficient, μ_k .
- Decay coefficient, d_c .

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=TEST_DATA, the table data specify the following:

- Static coefficient of friction.
- Dynamic friction coefficient measured at the reference slip rate, $\dot{\gamma}_2$.
- Reference slip rate, $\dot{\gamma}_2$, used to measure the dynamic friction coefficient.
- Kinetic friction coefficient, μ_∞ . This value corresponds to the asymptotic value of the friction coefficient at infinite slip rate, $\dot{\gamma}_\infty$.

Return value

A TangentialBehavior object.

Exceptions

None.

12.14.2 setValues(...)

This method modifies the TangentialBehavior object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TangentialBehavior** method.

Return value

None

Exceptions

None.

12.14.3 Members

The TangentialBehavior object has members with the same names and descriptions as the arguments to the **TangentialBehavior** method.

12.14.4 Corresponding analysis keywords

*FRICTION

13. Constraint commands

The Constraint commands define constraints between regions of the model.

13.1 Constraint object

The Constraint object is the abstract base type for other Constraint objects. The Constraint object has no explicit constructor. The members of the Constraint object are common to all objects derived from the Constraint.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.1.1 resume()

This method resumes the constraint that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

13.1.2 suppress()

This method suppresses the constraint.

Arguments

None.

Return value

None

Exceptions

None.

13.1.3 delete(...)

This method allows you to delete existing constraints.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each constraint to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

13.1.4 Members

The Constraint object has the following members:

name

A String specifying the constraint repository key.

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.2 AdjustPoints object

The AdjustPoints constraint object is used to adjust points (nodes) to a surface.

The AdjustPoints object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.2.1 AdjustPoints(...)

This method creates an `AdjustPoints` object.

Path

`mdb.models[name].AdjustPoints`

Required arguments

name

A String specifying the constraint repository key.

surface

A Region object specifying the surface to which the *controlPoints* are adjusted.

controlPoints

A Region object specifying the constraint control points.

Optional arguments

None.

Return value

An `AdjustPoints` object.

Exceptions

None.

13.2.2 setValues(...)

This method modifies the `AdjustPoints` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `AdjustPoints` method, except for the *name* argument.

Return value

None

Exceptions

None.

13.2.3 Members

The `AdjustPoints` object has members with the same names and descriptions as the arguments to the `AdjustPoints` method.

In addition, the `AdjustPoints` object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.2.4 Corresponding analysis keywords

*ADJUST

13.3 Coupling object

The Coupling object defines a constraint between a group of coupling nodes located on a region and a reference point.

The Coupling object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.3.1 Coupling(...)

This method creates a Coupling object.

Path

```
mdb.models[name].Coupling
```

Required arguments

name

A String specifying the constraint repository key.

surface

A Region object specifying the surface on which the coupling nodes are located.

controlPoint

A Region object specifying the constraint control point.

influenceRadius

The SymbolicConstant `WHOLE_SURFACE` or a Float specifying the influence radius.

couplingType

A SymbolicConstant specifying the coupling constraint type. Possible values are `KINEMATIC`, `DISTRIBUTING`, and `STRUCTURAL`.

Optional arguments

adjust

A Boolean specifying if the control point will be adjusted (moved) to the surface. The point will be adjusted in the direction normal to the specified surface. The default value is `OFF`.

localCsys

None or a DatumCsys object specifying the initial orientation of the local coordinate system for the coupling's degrees of freedom. If *localCsys*=**None**, the coupling is defined in the global coordinate system. The default value is **None**.

u1

A Boolean specifying if the displacement component in the 1-direction is constrained to the reference node for a kinematic coupling constraint. The default value is `ON`.

The *u1* argument applies only when *couplingType*=`KINEMATIC`.

u2

A Boolean specifying if the displacement component in the 2-direction is constrained to the reference node for a kinematic coupling constraint. The default value is `ON`.

The *u2* argument applies only when *couplingType*=`KINEMATIC`.

u3

A Boolean specifying if the displacement component in the 3-direction is constrained to the reference node for a kinematic coupling constraint. The default value is `ON`.

The *u3* argument applies only when *couplingType*=`KINEMATIC`.

ur1

A Boolean specifying if the rotational displacement component about the 1-direction is constrained to the reference node for a kinematic coupling constraint. The default value is `ON`.

The *ur1* argument applies only when *couplingType*=`KINEMATIC`.

ur2

A Boolean specifying if the rotational displacement component about the 2-direction is constrained to the reference node for a kinematic coupling constraint. The default value is `ON`.

The *ur2* argument applies only when *couplingType*=`KINEMATIC`.

CONSTRAINT COMMANDS

ur3

A Boolean specifying if the rotational displacement component about the 3-direction is constrained to the reference node for a kinematic coupling constraint. The default value is ON.

The *ur3* argument applies only when *couplingType*=KINEMATIC.

weightingMethod

A SymbolicConstant specifying an optional weighting method used for calculating the distributing weight factors. Possible values are UNIFORM, LINEAR, QUADRATIC, and CUBIC. The default value is UNIFORM.

The *weightingMethod* argument applies only when *couplingType*=DISTRIBUTING.

Return value

A Coupling object.

Exceptions

None.

13.3.2 setValues(...)

This method modifies the Coupling object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Coupling** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.3.3 Members

The Coupling object has members with the same names and descriptions as the arguments to the **Coupling** method.

In addition, the Coupling object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.3.4 Corresponding analysis keywords

*COUPLING

13.4 DisplayBody object

The DisplayBody object defines a constraint such that the specified instance is used for display only and does not take part in the analysis. However it will still be visible during postprocessing and its position at any frame will be defined by the translation and rotation of the specified control points.

The DisplayBody object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.4.1 DisplayBody(...)

This method creates a DisplayBody object.

Path

```
mdb.models[name].DisplayBody
```

Required arguments

name

A String specifying the constraint repository key.

instance

A PartInstance object specifying the part instance that is to be used for display only.

controlPoints

A ModelDotArray object specifying the motion of the PartInstance. The control points may be Vertex, ReferencePoint, or MeshNode objects. Their motion will control the motion of the PartInstance. If this argument is set to an empty sequence, the PartInstance will remain fixed in space during the analysis. The sequence can have either one object or three objects.

CONSTRAINT COMMANDS

Optional arguments

None.

Return value

A DisplayBody object.

Exceptions

None.

13.4.2 **setValues(...)**

This method modifies the DisplayBody object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DisplayBody** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.4.3 **Members**

The DisplayBody object has members with the same names and descriptions as the arguments to the **DisplayBody** method.

In addition, the DisplayBody object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.4.4 **Corresponding analysis keywords**

*DISPLAY BODY

13.5 EmbeddedRegion object

The EmbeddedRegion object allows you to embed a region of the model within a “host” region of the model or within the whole model.

The EmbeddedRegion object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.5.1 EmbeddedRegion(...)

This method creates a EmbeddedRegion object.

Path

```
mdb.models[name].EmbeddedRegion
```

Required arguments

name

A String specifying the constraint repository key.

embeddedRegion

A Region object specifying the body region to be embedded.

hostRegion

A Region object specifying the host region. A value of **None** indicates that the host region is the whole model.

Optional arguments

weightFactorTolerance

A Float specifying a small value below which the weighting factors will be zeroed out. The default value is 10^{-6} .

toleranceMethod

A SymbolicConstant specifying the method used to determine the embedded element tolerance. Possible values are ABSOLUTE, FRACTIONAL, and BOTH. The default value is BOTH.

absoluteTolerance

A Float specifying the absolute value by which a node on the embedded region may lie outside the host region. If *absoluteTolerance*=0.0, the *fractionalTolerance* value will be used. The default value is 0.0.

This argument applies only when *toleranceMethod*=ABSOLUTE or BOTH.

CONSTRAINT COMMANDS

fractionalTolerance

A Float specifying the fractional value by which a node on the embedded region may lie outside the host region. The fractional value is based on the average element size within the host region. The default value is 0.05.

If both tolerance arguments are specified, the smaller value will be used.

This argument applies only when *toleranceMethod*=FRACTIONAL or BOTH.

Return value

An EmbeddedRegion object.

Exceptions

None.

13.5.2 setValues(...)

This method modifies the EmbeddedRegion object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **EmbeddedRegion** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.5.3 Members

The EmbeddedRegion object has members with the same names and descriptions as the arguments to the **EmbeddedRegion** method.

In addition, the EmbeddedRegion object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.5.4 Corresponding analysis keywords

*EMBEDDED ELEMENT

13.6 Equation object

The Equation object defines a linear multi-point constraint between a set of degrees of freedom. The Equation object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.6.1 Equation(...)

This method creates an Equation object.

Path

```
mdb.models[name].Equation
```

Required arguments

name

A String specifying the constraint repository key.

terms

A sequence of (Float, String, Int, Int) sequences specifying a coefficient, Set name, degree of freedom, and coordinate system ID. The coordinate system ID is optional.

Optional arguments

None.

Return value

An Equation object.

Exceptions

If *terms* does not contain more than one entry:
Equation must have two or more terms.

13.6.2 **setValues(...)**

This method modifies the Equation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Equation** method, except for the *name* argument.

Return value

None

Exceptions

If *terms* does not contain more than one entry:
Equation must have two or more terms.

13.6.3 **Members**

The Equation object has members with the same names and descriptions as the arguments to the **Equation** method.

In addition, the Equation object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.6.4 **Corresponding analysis keywords**

*EQUATION

13.7 **MultipointConstraint object**

The MultipointConstraint object defines a constraint between a group of MultipointConstraint nodes located on a region and a reference point.

The MultipointConstraint object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.7.1 MultipointConstraint(...)

This method creates a MultipointConstraint object.

Path

```
mdb.models[name].MultipointConstraint
```

Required arguments

name

A String specifying the constraint repository key.

surface

A Region object specifying the surface on which the MultipointConstraint nodes are located.

controlPoint

A Region object specifying the constraint control point.

mpcType

A SymbolicConstant specifying the MPC type of the constraint. Possible values are BEAM_MPC, ELBOW_MPC, PIN_MPC, LINK_MPC, TIE_MPC, and USER_MPC.

Optional arguments

csys

None or a DatumCsys object specifying the initial orientation of the local coordinate system for the MultipointConstraint's degrees of freedom. If *localCsys=None*, the MultipointConstraint is defined in the global coordinate system. The default value is **None**.

userType

An Int specifying to differentiate between different constraint types in a user-defined MultipointConstraint. The default value is 0.

The *userType* argument applies only when *mpcType*=USER_MPC.

userMode

A SymbolicConstant specifying the mode of the constraint when it is user-defined. Possible values are DOF_MODE_MPC and NODE_MODE_MPC. The default value is DOF_MODE_MPC.

The *userMode* argument applies only when *mpcType*=USER_MPC.

Return value

A MultipointConstraint object.

Exceptions

None.

13.7.2 **setValues(...)**

This method modifies the MultipointConstraint object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MultipointConstraint** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.7.3 **Members**

The MultipointConstraint object has members with the same names and descriptions as the arguments to the **MultipointConstraint** method.

In addition, the MultipointConstraint object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.7.4 **Corresponding analysis keywords**

*MPC

13.8 **RigidBody object**

The RigidBody object constrains all the degrees of freedom on the specified regions to the degree of freedom of its associated reference point.

The RigidBody object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.8.1 RigidBody(...)

This method creates a RigidBody object.

Path

```
mdb.models[name].RigidBody
```

Required arguments

name

A String specifying the constraint repository key.

refPointRegion

A Region object specifying the reference point.

Optional arguments

bodyRegion

None or a Region object specifying the elements constrained to the movement of the reference point. The default value is **None**.

tieRegion

None or a Region object specifying the nodes tied to the movement of the reference point. The default value is **None**.

pinRegion

None or a Region object specifying the nodes pinned to the movement of the reference point. The default value is **None**.

surfaceRegion

None or a Region object specifying the analytic surface constrained to the movement of the reference point. The default value is **None**.

refPointAtCOM

A Boolean specifying whether the analysis product should recompute the reference point position to be at the center of mass. The default value is OFF.

isothermal

A Boolean specifying whether the temperature degree of freedom should be constrained. The default value is OFF.

CONSTRAINT COMMANDS

Return value

A RigidBody object.

Exceptions

None.

13.8.2 **setValues(...)**

This method modifies the RigidBody object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RigidBody** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.8.3 **Members**

The RigidBody object has members with the same names and descriptions as the arguments to the **RigidBody** method.

In addition, the RigidBody object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.8.4 **Corresponding analysis keywords**

*RIGID BODY

13.9 ShellSolidCoupling object

The ShellSolidCoupling object defines two surfaces to be tied together for the duration of a simulation. The ShellSolidCoupling object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.9.1 ShellSolidCoupling(...)

This method creates a ShellSolidCoupling object.

Path

```
mdb.models[name].ShellSolidCoupling
```

Required arguments

name

A String specifying the constraint repository key.

shellEdge

A Region object specifying the name of the shell edge surface.

solidFace

A Region object specifying the name of the solid surface.

Optional arguments

positionToleranceMethod

A SymbolicConstant specifying the method used to determine the position tolerance. Possible values are COMPUTED and SPECIFIED. The default value is COMPUTED.

positionTolerance

A Float specifying the position tolerance. The default value is 0.0.

The *positionTolerance* argument applies only when *positionToleranceMethod*=SPECIFIED.

Note: Abaqus will not constrain nodes on the solid face region outside the position tolerance.

influenceDistanceMethod

A SymbolicConstant specifying the method used to determine the influence distance. Possible values are DEFAULT and SPECIFIED. The default value is DEFAULT.

CONSTRAINT COMMANDS

influenceDistance

A Float specifying the influence distance. The *influenceDistance* argument applies only when *influenceDistanceMethod*=SPECIFIED. The default value is 0.0.

Return value

A ShellSolidCoupling object.

Exceptions

None.

13.9.2 setValues(...)

This method modifies the ShellSolidCoupling object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShellSolidCoupling** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.9.3 Members

The ShellSolidCoupling object has members with the same names and descriptions as the arguments to the **ShellSolidCoupling** method.

In addition, the ShellSolidCoupling object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.9.4 Corresponding analysis keywords

*SHELL TO SOLID COUPLING

13.10 Tie object

The Tie object defines two surfaces to be tied together for the duration of a simulation.

The Tie object is derived from the Constraint object.

Access

```
import interaction
mdb.models[name].constraints[name]
```

13.10.1 Tie(...)

This method creates a Tie object.

Path

```
mdb.models[name].Tie
```

Required arguments

name

A String specifying the constraint repository key.

master

A Region object specifying the name of the master surface.

slave

A Region object specifying the name of the slave surface.

Optional arguments

adjust

A Boolean specifying whether initial positions of tied slave nodes are adjusted to lie on the master surface. The default value is ON.

positionToleranceMethod

A SymbolicConstant specifying the method used to determine the position tolerance. Possible values are COMPUTED and SPECIFIED. The default value is COMPUTED.

positionTolerance

A Float specifying the position tolerance. The *positionTolerance* argument applies only when *positionToleranceMethod*=SPECIFIED. The default value is 0.0.

tieRotations

A Boolean specifying whether rotation degrees of freedom should be tied. The default value is ON.

CONSTRAINT COMMANDS

constraintRatioMethod

A SymbolicConstant specifying the method used to determine the constraint ratio. Possible values are DEFAULT and SPECIFIED. The default value is DEFAULT.

constraintRatio

A Float specifying the fractional distance between the master reference surface and the slave node at which the translational constraint should act. The *constraintRatio* argument applies only when *constraintRatioMethod*=SPECIFIED. The default value is 0.0.

constraintEnforcement

A SymbolicConstant specifying the discretization method. Possible values are SOLVER_DEFAULT, NODE_TO_SURFACE, and SURFACE_TO_SURFACE. The default value is SOLVER_DEFAULT.

thickness

A Boolean specifying whether shell element thickness is considered. The default value is ON.

Return value

A Tie object.

Exceptions

None.

13.10.2 swapSurfaces()

This method switches the master and slave surfaces of a tied constraint. This command is valid only during the step in which the interaction is created.

Arguments

None.

Return value

None

Exceptions

None.

13.10.3 setValues(...)

This method modifies the Tie object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Tie** method, except for the *name* argument.

Return value

None

Exceptions

None.

13.10.4 Members

The Tie object has members with the same names and descriptions as the arguments to the **Tie** method.

In addition, the Tie object has the following member:

suppressed

A Boolean specifying whether the constraint is suppressed or not. The default value is OFF.

13.10.5 Corresponding analysis keywords

*TIE

14. CustomKernel module

The `customKernel` module augments the `mdb`, `odb`, and `session` objects with an object called `customData`. The `customData` object can contain custom data that you created using standard Python objects; for example, dictionaries, strings, numbers, and classes. In addition, the `customData` object can contain objects that you created using the classes described in this section.

All objects that you create under the `mdb.customData` and `odb[odbName].customData` object will be stored in the Abaqus/CAE model and output database, respectively, when you save the database; all objects that you create under the `session.customData` object will be lost when you exit Abaqus/CAE.

The `customData` object is an instance of the `RepositorySupport` class; for more information, see `RepositorySupport`. The classes described in this section register with the GUI infrastructure when you create the class. As a result, if you write a custom GUI script, you can query these objects from the GUI, and you can also be notified when the contents of these objects change so you can update the GUI accordingly. For more information, see “Extending the Abaqus Scripting Interface,” Section 5.6 of the Abaqus Scripting User’s Guide and “Accessing kernel data from the GUI,” Section 6.7 of the Abaqus GUI Toolkit User’s Guide.

14.1 CommandRegister object

This class allows you to derive a general class that can be queried from the GUI and is capable of notifying the GUI when the contents of the class change.

Access

```
import customKernel
```

14.1.1 CommandRegister()

This class allows you to derive a general class that can be queried from the GUI and is capable of notifying the GUI when the contents of the class change.

Path

```
customKernel.CommandRegister
```

Arguments

None.

Return value

A CommandRegister object.

Exceptions

None.

14.1.2 Members

The CommandRegister object has no members.

14.2 RepositorySupport object

The **RepositorySupport** is a base class from which you can derive your own classes that are designed to contain custom repositories. Instances of this class can be queried from the GUI and are capable of notifying the GUI when the contents of the instance change.

The RepositorySupport object is derived from the CommandRegister object.

Access

```
import customKernel
mdb.customData
session.customData
session.odbs[name].customData
```

14.2.1 RepositorySupport()

This method creates a RepositorySupport object.

Path

```
customKernel.RepositorySupport
```

Arguments

None.

Return value

A RepositorySupport object.

Exceptions

None.

14.2.2 Repository(...)

This method installs a repository on the class. The repository is an instance of a **RegisteredDictionary** class. Refer to RegisteredDictionary for details on its methods.

The objects stored in the repository are assumed to have an attribute called *name* that stores the key used to access the object in the repository. The name attribute will be modified by the **changeKey** method.

Arguments**Required arguments**

name

A String specifying the name of the repository.

constructors

A constructor or sequence of constructors specifying which classes will store their instances in the repository.

Optional arguments

None.

Return value

None

Exceptions

None.

14.2.3 Members

The RepositorySupport object has no members.

14.3 RegisteredDictionary object

This class allows you to create a dictionary that can be queried from the GUI and is capable of notifying the GUI when the contents of the dictionary change. The keys to a RegisteredDictionary must be either strings or integers.

The RegisteredDictionary object is derived from the CommandRegister object.

Access

```
import customKernel
```

14.3.1 RegisteredDictionary()

This method creates a RegisteredDictionary object.

Path

```
customKernel.RegisteredDictionary
```

Arguments

None.

Return value

A RegisteredDictionary object.

Exceptions

None.

14.3.2 Methods()

The RegisteredDictionary object supports the same methods as a Python dictionary. In addition, the RegisteredDictionary object supports the **changeKey** method.

Arguments

None.

Return value

None

Exceptions

None.

14.3.3 changeKey(...)

This method changes the name of a key in the dictionary.

Arguments**Required arguments***fromName*

A String or an integer specifying the name of the key to be changed.

toName

A String or an integer specifying the new name for the key.

Optional arguments

None.

Return value

None

Exceptions

None.

14.3.4 Members

The RegisteredDictionary object has no members.

14.4 RegisteredList object

This class allows you to create a list that can be queried from the GUI and is capable of notifying the GUI when the contents of the list change.

The RegisteredList object is derived from the CommandRegister object.

Access

```
import customKernel
```

14.4.1 RegisteredList()

This method creates a RegisteredList object.

Path

```
customKernel.RegisteredList
```

Arguments

None.

Return value

A RegisteredList object.

Exceptions

None.

14.4.2 **Methods()**

The RegisteredList object supports the same methods as a standard Python list object.

Arguments

None.

Return value

None

Exceptions

None.

14.4.3 **Members**

The RegisteredList object has no members.

14.5 **RegisteredTuple object**

This class allows you to create a tuple that can be queried from the GUI and is capable of notifying the GUI when the contents of any of the tuple's members change.

The RegisteredTuple object is derived from the CommandRegister object.

Access

```
import customKernel
```

14.5.1 **RegisteredTuple(...)**

This method creates a RegisteredTuple object.

Path`customKernel.RegisteredTuple`**Required argument***tuple*

A tuple of objects. These objects must be derived from the CommandRegister class.

Optional arguments

None.

Return value

A RegisteredTuple object.

Exceptions

None.

14.5.2 Methods()

The RegisteredTuple object supports the same methods as a standard Python list object.

Arguments

None.

Return value

None

Exceptions

None.

14.5.3 Members

The RegisteredTuple object has no members.

15. Datum commands

Datum commands return Feature objects and inherit the methods of Feature objects; for more details, see Chapter 20, “Feature commands.” Datums can be created using methods on a Part or Assembly object.

Each command also creates a Datum object in the corresponding **datum** repository. The Datum object is used as an argument to other commands, such as Part and Partition commands.

15.1 Datum object

The Datum object is the abstract base type for other Datum objects. The Datum object has no explicit constructor. The methods and members of the Datum object are common to all objects derived from the Datum.

Access

```
import part
mdb.models[name].parts[name].datums[i]
import assembly
mdb.models[name].rootAssembly.allinstances.datums[i]
mdb.models[name].rootAssembly.datums[i]
mdb.models[name].rootAssembly.instances[name].datums[i]
mdb.models[name].rootAssembly.modelInstances[i].datums[i]
```

15.1.1 Members

The Datum object has no members.

15.2 DatumAxis object

The DatumAxis object has no direct constructor; it is created when a Feature object is created. For example, the **DatumAxisByCylFace** method creates a Feature object that creates a DatumAxis object.

The DatumAxis object is derived from the Datum object.

Access

```
import part
mdb.models[name].parts[name].datums[i]
import assembly
```

```

mdb.models[name].rootAssembly.allinstances.datums[i]
mdb.models[name].rootAssembly.datums[i]
mdb.models[name].rootAssembly.instances[name].datums[i]

```

15.2.1 Members

The DatumAxis object has the following members:

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the datum.

direction

A tuple of Floats specifying a sequence of three Floats specifying the direction of the axis.

15.3 DatumCsys object

The DatumCsys object has no direct constructor; it is created when a Feature object is created. For example, the **DatumCsysByOffset** method creates a Feature object that creates a DatumCsys object.

The DatumCsys object is derived from the Datum object.

Access

```

import part
mdb.models[name].parts[name].datums[i]
import assembly
mdb.models[name].rootAssembly.allinstances.datums[i]
mdb.models[name].rootAssembly.connectorOrientations[i].localCsys1
mdb.models[name].rootAssembly.connectorOrientations[i].localCsys2
mdb.models[name].rootAssembly.datums[i]
mdb.models[name].rootAssembly.instances[name].datums[i]
import odbAccess
session.odbs[name].rootAssembly.connectorOrientations[i].localCsys1
session.odbs[name].rootAssembly.connectorOrientations[i].localCsys2

```

15.3.1 Members

The DatumCsys object has the following members:

coordSysType

A SymbolicConstant specifying the type of the coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

A DatumPoint object specifying the origin of the coordinate system.

axis1

A DatumAxis object specifying the 1-direction of the coordinate system.

axis2

A DatumAxis object specifying the 2-direction of the coordinate system.

axis3

A DatumAxis object specifying the 3-direction of the coordinate system.

15.4 DatumPlane object

The DatumPlane object has no direct constructor; it is created when a Feature object is created. For example, the **DatumPlaneByPrincipalPlane** method creates a Feature object that creates a DatumPlane object.

The DatumPlane object is derived from the Datum object.

Access

```
import part
mdb.models[name].parts[name].datums[i]
import assembly
mdb.models[name].rootAssembly.allinstances.datums[i]
mdb.models[name].rootAssembly.datums[i]
mdb.models[name].rootAssembly.instances[name].datums[i]
```

15.4.1 Members

The DatumPlane object has the following members:

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the datum.

normal

A tuple of Floats specifying a sequence of three Floats specifying the normal.

15.5 DatumPoint object

The DatumPoint object has no direct constructor; it is created when a Feature object is created. For example, the **DatumPointByCoordinate** method creates a Feature object that creates a DatumPoint object.

The DatumPoint object is derived from the Datum object.

Access

```
import part
mdb.models[name].parts[name].datums[i]
import assembly
mdb.models[name].rootAssembly.allinstances.datums[i]
mdb.models[name].rootAssembly.datums[i]
mdb.models[name].rootAssembly.instances[name].datums[i]
```

15.5.1 Members

The DatumPoint object has the following member:

pointOn

A tuple of Floats specifying the X-, Y-, and Z-coordinates of a point located on the datum.

16. Display Group commands

16.1 DisplayGroup object

DisplayGroup objects are used to select a subset of the entities displayed in the viewport.

Access

```
session.displayGroups [name]
import assembly
session.viewports [name] .assemblyDisplay.displayGroup
session.viewports [name] .layers [name] .assemblyDisplay.displayGroup
import visualization
session.viewports [name] .layers [name] .odbDisplay.displayGroup
import part
session.viewports [name] .layers [name] .partDisplay.displayGroup
session.viewports [name] .odbDisplay.displayGroup
session.viewports [name] .partDisplay.displayGroup
```

16.1.1 DisplayGroup(...)

This method creates a DisplayGroup object.

Path

```
session.DisplayGroup
```

Required arguments

name

A String specifying the repository key.

leaf

A Leaf object specifying the items in the display group.

Optional arguments

None.

Return value

A DisplayGroup object.

Exceptions

InvalidNameError.

16.1.2 **add(...)**

This method adds the specified items to the display group.

Arguments**Required argument**

leaf

A Leaf object specifying the items to add to the display group.

Optional arguments

None.

Return value

None

Exceptions

None.

16.1.3 **either(...)**

This method redefines the display group to be only those items that are not shared by the *leaf* argument and by the display group.

Arguments**Required argument**

leaf

A Leaf object specifying the items to be excluded from the display group.

Optional arguments

None.

Return value

None

Exceptions

None.

16.1.4 intersect(...)

This method redefines the display group to be only those items that are shared by the *leaf* argument and the display group.

Arguments**Required argument**

leaf

A Leaf object specifying the items to be included in the display group.

Optional arguments

None.

Return value

None

Exceptions

None.

16.1.5 redoLast()

This method redoes the last undone operation on the display group.

Arguments

None.

Return value

None

Exceptions

None.

16.1.6 remove(...)

This method removes the specified items from the display group.

DISPLAY GROUP COMMANDS

Arguments

Required argument

leaf

A Leaf object specifying the items to remove from the display group.

Optional arguments

None.

Return value

None

Exceptions

None.

16.1.7 replace(...)

This method replaces the contents of the display group with the specified items.

Arguments

Required argument

leaf

A Leaf object specifying the items with which to replace the current display group contents.

Optional arguments

None.

Return value

None

Exceptions

None.

16.1.8 undoLast()

This method undoes the last operation performed on the display group.

Arguments

None.

Return value

None

Exceptions

None.

16.1.9 Members

The DisplayGroup object has the following members:

canUndo

A Boolean specifying whether Undo is possible or not.

canRedo

A Boolean specifying whether Redo is possible or not.

name

A String specifying the repository key.

module

A SymbolicConstant specifying the module in which the display group has been created. The possible values are PART, ASSEMBLY, PART_ASSEMBLY, ODB, and ALL.

modelName

A String specifying the name of the model to which the display group belongs when the module is part- or assembly-based.

partName

A String specifying the name of the part to which the display group belongs when the module is part-based.

16.2 DisplayGroupInstance object

A DisplayGroupInstance object stores the IDs of the entities displayed in a viewport. The DisplayGroupInstance object has no constructor. When you set a display group to be plotted in a viewport, Abaqus/CAE creates a DisplayGroupInstance object for each display group and places it in the DisplayGroupInstanceRepository object.

Access

```
import assembly
session.viewports[name].assemblyDisplay.displayGroupInstances[name]
session.viewports[name].layers[name].assemblyDisplay\
```

```
.displayGroupInstances [name]
import visualization
session.viewports [name] .layers [name] .odbDisplay\
.displayGroupInstances [name]
import part
session.viewports [name] .layers [name] .partDisplay\
.displayGroupInstances [name]
session.viewports [name] .odbDisplay.displayGroupInstances [name]
session.viewports [name] .partDisplay.displayGroupInstances [name]
```

16.2.1 nodes()

This method is used to obtain the list of nodes present in the DisplayGroupInstance object. It returns a Dictionary object keyed by part instance names, the value of which is a list of user node labels belonging to the part instance and contained in the DisplayGroupInstance object. This method is available only for DisplayGroupInstance objects that are members of the DisplayGroupInstance repository member of OdbDisplay object.

Arguments

None.

Return value

A Dictionary object.

Exceptions

None.

16.2.2 elements()

This method returns the list of elements present in the DisplayGroupInstance object. The **elements** method returns a Dictionary object that uses part instance names for the keys. The value of the items in the Dictionary object is a List of user element labels that belong to the part instance and are contained in the DisplayGroupInstance object. This method is available only for DisplayGroupInstance objects that are members of the DisplayGroupInstance repository member of the OdbDisplay object.

Arguments

None.

Return value

A Dictionary object.

Exceptions

None.

16.2.3 setValues(...)

This method modifies the DisplayGroupInstance object. The **setValues** method is available only for DisplayGroupInstance objects that are members of the DisplayGroupInstance repository member of the OdbDisplay object.

Arguments**Required arguments**

None.

Optional argument*lockOptions*

A Boolean specifying whether the display options stored on the DisplayGroupInstance object should be synchronized with changes to the viewport display options. This member is available only for DisplayGroupInstance objects that are members of the DisplayGroupInstance repository member of the OdbDisplay object. The default value is OFF.

Return value

None

Exceptions

None.

16.2.4 Members

The DisplayGroupInstance object can have the following members:

name

A String specifying the repository key.

lockOptions

A Boolean specifying whether the display options stored on the DisplayGroupInstance object should be synchronized with changes to the viewport display options. This member is available only for DisplayGroupInstance objects that are members of the DisplayGroupInstance repository member of the OdbDisplay object. The default value is OFF.

odbDisplayOptions

An *OdbDisplayOptions* object specifying this member is available only for *DisplayGroupInstance* objects that are members of the *DisplayGroupInstance* repository member of the *OdbDisplay* object.

16.3 DisplayGroupInstanceRepository object

The *DisplayGroupInstanceRepository* object stores *DisplayGroupInstance* objects. In addition to all the standard Python repository methods, the *DisplayGroupInstance* repository defines additional methods as described below.

Access

```
import visualization
session.viewports[name].layers[name].odbDisplay.displayGroupInstances
session.viewports[name].odbDisplay.displayGroupInstances
```

16.3.1 syncOptions(...)

This method synchronizes the display options stored on the *OdbDisplay* object with the display options stored on the *DisplayGroupInstance* object.

Arguments

Required argument

name

A String specifying the repository key.

Optional argument

updateInstances

A Boolean specifying whether to synchronize the display options on all the *DisplayGroupInstance* objects stored in the *DisplayGroupInstanceRepository* for which *lockOptions* is OFF. The default value of *updateInstances* is ON.

Return value

None

Exceptions

None.

16.3.2 Members

The DisplayGroupInstanceRepository object has no members.

16.4 Leaf object

Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

Leaf objects have similarities to Set objects; however, Leaf objects are evaluated when the DisplayGroup expression is evaluated, and they can have SymbolicConstant values (which are also evaluated when the DisplayGroup expression is evaluated).

Access

```
import displayGroupMdbToolset
import displayGroupOdbToolset
```

16.4.1 Leaf(...)

This method creates a Leaf object.

Path

Leaf

Required argument

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

Optional arguments

None.

Return value

A Leaf object.

Exceptions

None.

16.4.2 Members

The Leaf object has members with the same names and descriptions as the arguments to the **Leaf** method.

16.5 LeafFromDatums object

The LeafFromDatums object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromDatums object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.5.1 LeafFromDatums(...)

This method creates a Leaf object from a sequence of datum objects. Leaf objects specify the items in a display group.

Path

LeafFromDatums

Required argument

datumSeq

A sequence of datum objects.

Optional arguments

None.

Return value

A LeafFromDatums object.

Exceptions

None.

16.5.2 Members

The LeafFromDatums object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.6 LeafFromDisplayGroup object

The LeafFromDisplayGroup object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromDisplayGroup object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
import displayGroupOdbToolset
```

16.6.1 LeafFromDisplayGroup(...)

This method creates a Leaf object from a sequence of Display Group objects.

Path

LeafFromDisplayGroup

Required argument

displayGroup

A DisplayGroupArray object.

Optional arguments

None.

Return value

A LeafFromDisplayGroup object.

Exceptions

None.

16.6.2 Members

The `LeafFromDisplayGroup` object has members with the same names and descriptions as the arguments to the `LeafFromDisplayGroup` method.

In addition, the `LeafFromDisplayGroup` object has the following member:

leafType

A `SymbolicConstant` specifying the leaf type. Possible values are `EMPTY`, `DEFAULT_MODEL`, `ALL_ELEMENTS`, `ALL_NODES`, and `ALL_SURFACES`.

16.7 LeafFromElementLabels object

The `LeafFromElementLabels` object can be used whenever a `Leaf` object is expected as an argument. `Leaf` objects are used to specify the items in a display group. `Leaf` objects are constructed as temporary objects, which are then used as arguments to `DisplayGroup` commands.

The `LeafFromElementLabels` object is derived from the `Leaf` object.

Access

```
import displayGroupOdbToolset
```

16.7.1 LeafFromElementLabels(...)

This method creates a `Leaf` object from a sequence of element labels that belong to a single part instance.

Path

`LeafFromElementLabels`

Required arguments

partInstanceName

A `String` specifying the name of the part instance to which *elementLabels* refers.

elementLabels

A sequence of `Strings` specifying expressions that denote element labels. The expression can be any of the following:

- An `Int` specifying a single element label; for example, `1`.
- A `String` specifying a single element label; for example, `'7'`.
- A `String` specifying a sequence of element labels; for example, `'3:5'` and `'3:15:3'`.

Optional arguments

None.

Return value

A LeafFromElementLabels object.

Exceptions

None.

16.7.2 Members

The LeafFromElementLabels object has members with the same names and descriptions as the arguments to the **LeafFromElementLabels** method.

In addition, the LeafFromElementLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.8 LeafFromElementSets object

The LeafFromElementSets object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromElementSets object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.8.1 LeafFromElementSets(...)

This method creates a Leaf object from a sequence of element sets.

Path

LeafFromElementSets

Required argument*elementSets*

A sequence of Strings specifying element sets or a String specifying a single element set.

Optional arguments

None.

Return value

A `LeafFromElementSets` object.

Exceptions

None.

16.8.2 Members

The `LeafFromElementSets` object has members with the same names and descriptions as the arguments to the `LeafFromElementSets` method.

In addition, the `LeafFromElementSets` object has the following member:

leafType

A `SymbolicConstant` specifying the leaf type. Possible values are `EMPTY`, `DEFAULT_MODEL`, `ALL_ELEMENTS`, `ALL_NODES`, and `ALL_SURFACES`.

16.9 LeafFromElementVarRange object

The `LeafFromElementVarRange` object can be used whenever a `Leaf` object is expected as an argument. `Leaf` objects are used to specify the items in a display group. `Leaf` objects are constructed as temporary objects, which are then used as arguments to `DisplayGroup` commands.

The `LeafFromElementVarRange` object is derived from the `Leaf` object.

Access

```
import displayGroupOdbToolset
```

16.9.1 LeafFromElementVarRange(...)

This method creates a `Leaf` object from elements with values lying in a variable range.

Path

`LeafFromElementVarRange`

Required arguments

None.

Optional arguments*minimumRange*

A Float specifying the minimum value for the variable range. The default value is $-3.40282346639E38$.

maximumRange

A Float specifying the maximum value for the variable range. The default value is $3.40282346639e+038$.

insideRange

A Boolean specifying the method used to evaluate the range. If *insideRange*=ON, the range falls inside the specified minimum and maximum values. The default value is ON.

Return value

A LeafFromElementVarRange object.

Exceptions

None.

16.9.2 Members

The LeafFromElementVarRange object has members with the same names and descriptions as the arguments to the **LeafFromElementVarRange** method.

In addition, the LeafFromElementVarRange object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.10 LeafFromGeometry object

The LeafFromGeometry object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromGeometry object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.10.1 LeafFromGeometry(...)

This method creates a Leaf object from a sequence of edge, face and cell geometry objects. Any combination of edge, face or cell arguments is allowed however the arguments must specify at least one object--it is not permissible to create an empty leaf.

Path

LeafFromGeometry

Required arguments

None.

Optional arguments

edgeSeq

A sequence of geometry edges.

faceSeq

A sequence of geometry faces.

cellSeq

A sequence of geometry cells.

Return value

A LeafFromGeometry object.

Exceptions

If at least one of the sequences is not passed to this method:

Cannot define empty leaf.

16.10.2 Members

The LeafFromGeometry object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.11 LeafFromInstance object

The LeafFromInstance object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromInstance object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.11.1 LeafFromInstance(...)

This method creates a Leaf object from a sequence of part instance objects.

Path

LeafFromInstance

Required argument

instances

A PartInstance object or a Sequence of PartInstance objects.

Optional arguments

None.

Return value

A LeafFromInstance object.

Exceptions

If an invalid argument is passed to this method:

Cannot define empty leaf.

16.11.2 Members

The LeafFromInstance object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.12 LeafFromInstanceElementLabels object

The LeafFromInstanceElementLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromInstanceElementLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.12.1 LeafFromInstanceElementLabels(...)

This method creates a Leaf object from a sequence of Strings specifying the element labels. Leaf objects specify the items in a display group.

Path

```
LeafFromInstanceElementLabels
```

Required argument

elementLabels

A sequence of sequences specifying element labels. Each inner sequence consists of a PartInstance object followed by a sequence of Strings specifying element labels.

Optional arguments

None.

Return value

A LeafFromInstanceElementLabels object.

Exceptions

None.

16.12.2 Members

The LeafFromInstanceElementLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.13 LeafFromInstanceNodeLabels object

The LeafFromInstanceNodeLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromInstanceNodeLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.13.1 LeafFromInstanceNodeLabels(...)

This method creates a Leaf object from a sequence of Strings specifying the node labels. Leaf objects specify the items in a display group.

Path

```
LeafFromInstanceNodeLabels
```

Required argument

nodeLabels

A sequence of sequences specifying node labels. Each inner sequence consists of a PartInstance object followed by a sequence of Strings specifying node labels.

Optional arguments

None.

Return value

A LeafFromInstanceNodeLabels object.

Exceptions

None.

16.13.2 Members

The LeafFromInstanceNodeLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.14 LeafFromMeshElementLabels object

The LeafFromMeshElementLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromMeshElementLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.14.1 LeafFromMeshElementLabels(...)

This method creates a Leaf object from a sequence of mesh element objects. Leaf objects specify the items in a display group.

Path

LeafFromMeshElementLabels

Required argument

elementSeq

A sequence of MeshElement objects specifying elements.

Optional arguments

None.

Return value

A LeafFromMeshElementLabels object.

Exceptions

None.

16.14.2 Members

The LeafFromMeshElementLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.15 LeafFromMeshNodeLabels object

The LeafFromMeshNodeLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromMeshNodeLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.15.1 LeafFromMeshNodeLabels(...)

This method creates a Leaf object from a sequence of mesh node objects. Leaf objects specify the items in a display group.

Path

LeafFromMeshNodeLabels

Required argument

nodeSeq

A sequence of MeshNode objects specifying nodes.

Optional arguments

None.

Return value

A LeafFromMeshNodeLabels object.

Exceptions

None.

16.15.2 Members

The LeafFromMeshNodeLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.16 LeafFromMeshSurfaceSets object

The LeafFromMeshSurfaceSets object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromMeshSurfaceSets object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.16.1 LeafFromMeshSurfaceSets(...)

This method creates a Leaf object from a sequence of surface sets.

Path

LeafFromMeshSurfaceSets

Required argument

surfaceSets

A sequence of Surface objects.

Optional arguments

None.

Return value

A LeafFromMeshSurfaceSets object.

Exceptions

None.

16.16.2 Members

The LeafFromMeshSurfaceSets object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.17 LeafFromModelElemLabels object

The LeafFromModelElemLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromModelElemLabels object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.17.1 LeafFromModelElemLabels(...)

This method creates a Leaf object from a sequence of element labels spanning several part instances.

Path

```
LeafFromModelElemLabels
```

Required argument

elementLabels

A sequence of Strings specifying expressions that denote element labels per part instance in the model. Each part instance element expression is a sequence of a String specifying the part instance name and a sequence of element expressions; for example, `(('partInstance1', (1, '7', '3:15;3')), ('partInstance2', '8'))`.

The element expressions can be any of the following:

- An Int specifying a single element label; for example, `1`.
- A String specifying a single element label; for example, `'7'`.
- A String specifying a sequence of element labels; for example, `'3:5'` and `'3:15:3'`.

Optional arguments

None.

Return value

A LeafFromModelElemLabels object.

Exceptions

None.

16.17.2 Members

The `LeafFromModelElemLabels` object has members with the same names and descriptions as the arguments to the `LeafFromModelElemLabels` method.

In addition, the `LeafFromModelElemLabels` object has the following member:

leafType

A `SymbolicConstant` specifying the leaf type. Possible values are `EMPTY`, `DEFAULT_MODEL`, `ALL_ELEMENTS`, `ALL_NODES`, and `ALL_SURFACES`.

16.18 LeafFromModelNodeLabels object

The `LeafFromModelNodeLabels` object can be used whenever a `Leaf` object is expected as an argument. `Leaf` objects are used to specify the items in a display group. `Leaf` objects are constructed as temporary objects, which are then used as arguments to `DisplayGroup` commands.

The `LeafFromModelNodeLabels` object is derived from the `Leaf` object.

Access

```
import displayGroupOdbToolset
```

16.18.1 LeafFromModelNodeLabels(...)

This method creates a `Leaf` object from a sequence of node labels spanning several part instances.

Path

`LeafFromModelNodeLabels`

Required argument

nodeLabels

A sequence of `Strings` specifying expressions that denote node labels per part instance in the model. Each part instance node expression is a sequence of a `String` specifying the part instance name and a sequence of node expressions; for example, `(('partInstance1', (1, '7', '3:15;3')), ('partInstance2', '8'))`.

The node expressions can be any of the following:

- An `Int` specifying a single node label; for example, `1`.
- A `String` specifying a single node label; for example, `'7'`.
- A `String` specifying a sequence of node labels; for example, `'3:5'` and `'3:15:3'`.

Optional arguments

None.

Return value

A LeafFromModelNodeLabels object.

Exceptions

None.

16.18.2 Members

The LeafFromModelNodeLabels object has members with the same names and descriptions as the arguments to the **LeafFromModelNodeLabels** method.

In addition, the LeafFromModelNodeLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.19 LeafFromNodeLabels object

The LeafFromNodeLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromNodeLabels object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.19.1 LeafFromNodeLabels(...)

This method creates a Leaf object from a sequence of node labels that belong to a single part instance.

Path

LeafFromNodeLabels

Required arguments

partInstanceName

A String specifying the name of the part instance to which *nodeLabels* refers.

nodeLabels

A sequence of Strings specifying expressions that denote node labels. The expression can be any of the following:

- An Int specifying a single node label; for example, `1`.
- A String specifying a single node label; for example, `'7'`.
- A String specifying a sequence of node labels; for example, `'3:5'` and `'3:15:3'`.

Optional arguments

None.

Return value

A `LeafFromNodeLabels` object.

Exceptions

None.

16.19.2 Members

The `LeafFromNodeLabels` object has members with the same names and descriptions as the arguments to the **`LeafFromNodeLabels`** method.

In addition, the `LeafFromNodeLabels` object has the following member:

leafType

A `SymbolicConstant` specifying the leaf type. Possible values are `EMPTY`, `DEFAULT_MODEL`, `ALL_ELEMENTS`, `ALL_NODES`, and `ALL_SURFACES`.

16.20 LeafFromNodeSets object

The `LeafFromNodeSets` object can be used whenever a `Leaf` object is expected as an argument. `Leaf` objects are used to specify the items in a display group. `Leaf` objects are constructed as temporary objects, which are then used as arguments to `DisplayGroup` commands.

The `LeafFromNodeSets` object is derived from the `Leaf` object.

Access

```
import displayGroupOdbToolset
```

16.20.1 LeafFromNodeSets(...)

This method creates a `Leaf` object from a sequence of node sets.

Path**LeafFromNodeSets****Required argument***nodeSets*

A sequence of Strings specifying node sets or a String specifying a single node set.

Optional arguments

None.

Return value

A LeafFromNodeSets object.

Exceptions

None.

16.20.2 Members

The LeafFromNodeSets object has members with the same names and descriptions as the arguments to the **LeafFromNodeSets** method.

In addition, the LeafFromNodeSets object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.21 LeafFromNodeVarRange object

The LeafFromNodeVarRange object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromNodeVarRange object is derived from the Leaf object.

Access**import displayGroupOdbToolset****16.21.1 LeafFromNodeVarRange(...)**

This method creates a Leaf object from nodes with values lying in a variable range.

Path

LeafFromNodeVarRange

Required arguments

None.

Optional arguments

minimumRange

A Float specifying the minimum value for the variable range. The default value is $-3.40282346639E38$.

maximumRange

A Float specifying the maximum value for the variable range. The default value is $3.40282346639e+038$.

insideRange

A Boolean specifying the method used to evaluate the range. If *insideRange*=ON, the range falls inside the specified minimum and maximum values. The default value is ON.

Return value

A LeafFromNodeVarRange object.

Exceptions

None.

16.21.2 Members

The LeafFromNodeVarRange object has members with the same names and descriptions as the arguments to the **LeafFromNodeVarRange** method.

In addition, the LeafFromNodeVarRange object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.22 LeafFromOdbElementLayups object

The LeafFromOdbElementLayups object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementLayups object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.22.1 LeafFromOdbElementLayups(...)

This method creates a Leaf object from a sequence of Strings specifying layup names. Leaf objects specify the items in a display group.

Path

LeafFromOdbElementLayups

Required argument

elementLayups

A sequence of Strings specifying element layups.

Optional arguments

None.

Return value

A LeafFromOdbElementLayups object.

Exceptions

None.

16.22.2 Members

The LeafFromOdbElementLayups object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.23 LeafFromOdbElementMaterials object

The LeafFromOdbElementMaterials object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementMaterials object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.23.1 LeafFromOdbElementMaterials(...)

This method creates a Leaf object from a sequence of Strings specifying material names. Leaf objects specify the items in a display group.

Path

LeafFromOdbElementMaterials

Required argument

elementMaterials

A sequence of Strings specifying element materials.

Optional arguments

None.

Return value

A LeafFromOdbElementMaterials object.

Exceptions

None.

16.23.2 Members

The LeafFromOdbElementMaterials object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.24 LeafFromOdbElementPick object

The LeafFromOdbElementPick object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementPick object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.24.1 LeafFromOdbElementPick(...)

This method creates a Leaf object from a tuple containing machine readable, compact strings defining the elements picked for each part instance. Leaf objects specify the items in a display group.

Path

```
LeafFromOdbElementPick
```

Required argument

elementPick

A sequence of tuples of the form [part name, entity count, machine readable pick strings].

Optional arguments

None.

Return value

A LeafFromOdbElementPick object.

Exceptions

None.

16.24.2 Members

The LeafFromOdbElementPick object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.25 LeafFromOdbElementPlies object

The LeafFromOdbElementPlies object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementPlies object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.25.1 LeafFromOdbElementPlies(...)

This method creates a Leaf object from a sequence of Strings specifying ply names. Leaf objects specify the items in a display group.

Path

`LeafFromOdbElementPlies`

Required argument

elementPlies

A sequence of Strings specifying element plies.

Optional arguments

None.

Return value

A LeafFromOdbElementPlies object.

Exceptions

None.

16.25.2 Members

The LeafFromOdbElementPlies object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.26 LeafFromOdbElementSections object

The LeafFromOdbElementSections object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementSections object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.26.1 LeafFromOdbElementSections(...)

This method creates a Leaf object from a sequence of Strings specifying section names. Leaf objects specify the items in a display group.

Path

LeafFromOdbElementSections

Required argument

elementSections

A sequence of Strings specifying element Sections.

Optional arguments

None.

Return value

A LeafFromOdbElementSections object.

Exceptions

None.

16.26.2 Members

The LeafFromOdbElementSections object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.27 LeafFromOdbElementTypes object

The LeafFromOdbElementTypes object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbElementTypes object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.27.1 LeafFromOdbElementTypes(...)

This method creates a Leaf object from a sequence of Strings specifying element names. Leaf objects specify the items in a display group.

Path

LeafFromOdbElementTypes

Required argument

elementTypes

A sequence of Strings specifying element Types.

Optional arguments

None.

Return value

A LeafFromOdbElementTypes object.

Exceptions

None.

16.27.2 Members

The LeafFromOdbElementTypes object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.28 LeafFromOdbNodePick object

The LeafFromOdbNodePick object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbNodePick object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.28.1 LeafFromOdbNodePick(...)

This method creates a Leaf object from a tuple containing machine readable, compact strings defining the nodes picked for each part instance. Leaf objects specify the items in a display group.

Path

```
LeafFromOdbNodePick
```

Required argument

nodePick

A sequence of tuples of the form [part name, entity count, machine readable pick strings].

Optional arguments

None.

Return value

A LeafFromOdbNodePick object.

Exceptions

None.

16.28.2 Members

The LeafFromOdbNodePick object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.29 LeafFromOdbEdgePick object

The LeafFromOdbEdgePick object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromOdbEdgePick object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.29.1 LeafFromOdbEdgePick(...)

This method creates a Leaf object from a tuple containing machine readable, compact strings defining the edges picked for each part instance. Leaf objects specify the items in a display group.

Path

`LeafFromOdbEdgePick`

Required argument

edgePick

A sequence of tuples of the form [part name, entity count, machine readable pick strings].

Optional arguments

None.

Return value

A LeafFromOdbEdgePick object.

Exceptions

None.

16.29.2 Members

The LeafFromOdbEdgePick object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.30 LeafFromPartElementLabels object

The LeafFromPartElementLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromPartElementLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.30.1 LeafFromPartElementLabels(...)

This method creates a Leaf object from a sequence of Strings specifying element labels. Leaf objects specify the items in a display group.

Path

```
LeafFromPartElementLabels
```

Required arguments

part

A Part object.

elementLabels

A sequence of Strings specifying element labels.

Optional arguments

None.

Return value

A LeafFromPartElementLabels object.

Exceptions

None.

16.30.2 Members

The LeafFromPartElementLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.31 LeafFromPartInstance object

The LeafFromPartInstance object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromPartInstance object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.31.1 LeafFromPartInstance(...)

This method creates a Leaf object from a list of part instance names.

Path

LeafFromPartInstance

Required argument

partInstanceName

A sequence of Strings specifying the names of the part instances.

Optional arguments

None.

Return value

A LeafFromPartInstance object.

Exceptions

None.

16.31.2 Members

The LeafFromPartInstance object has members with the same names and descriptions as the arguments to the **LeafFromPartInstance** method.

In addition, the LeafFromPartInstance object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.32 LeafFromPartNodeLabels object

The LeafFromPartNodeLabels object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromPartNodeLabels object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.32.1 LeafFromPartNodeLabels(...)

This method creates a Leaf object from a sequence of Strings specifying node labels. Leaf objects specify the items in a display group.

Path

LeafFromPartNodeLabels

Required arguments

part

A Part object.

nodeLabels

A sequence of Strings specifying node labels.

Optional arguments

None.

Return value

A LeafFromPartNodeLabels object.

Exceptions

None.

16.32.2 Members

The LeafFromPartNodeLabels object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.33 LeafFromReferencePoint object

The LeafFromReferencePoint object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromReferencePoint object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.33.1 LeafFromReferencePoint(...)

This method creates a Leaf object from a sequence of ReferencePoint objects.

Path

```
LeafFromReferencePoint
```

Required argument

refPtSeq

A sequence of Reference Point objects.

Optional arguments

None.

Return value

A LeafFromReferencePoint object.

Exceptions

None.

16.33.2 Members

The LeafFromReferencePoint object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.34 LeafFromSets object

The LeafFromSets object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromSets object is derived from the Leaf object.

Access

```
import displayGroupMdbToolset
```

16.34.1 LeafFromSets(...)

This method creates a Leaf object from a sequence of Set objects.

Path

LeafFromSets

Required argument

sets

A sequence of Set objects.

Optional arguments

None.

Return value

A LeafFromSets object.

Exceptions

None.

16.34.2 Members

The LeafFromSets object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.35 LeafFromSurfaceSets object

The LeafFromSurfaceSets object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromSurfaceSets object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.35.1 LeafFromSurfaceSets(...)

This method creates a Leaf object from a sequence of surface sets.

Path

LeafFromSurfaceSets

Required argument

surfaceSets

A sequence of Strings specifying surface sets, or a String specifying a single surface set.

Optional arguments

None.

Return value

A LeafFromSurfaceSets object.

Exceptions

None.

16.35.2 Members

The LeafFromSurfaceSets object has members with the same names and descriptions as the arguments to the **LeafFromSurfaceSets** method.

In addition, the LeafFromSurfaceSets object has the following member:

leafType

A SymbolicConstant specifying the leaf type. Possible values are EMPTY, DEFAULT_MODEL, ALL_ELEMENTS, ALL_NODES, and ALL_SURFACES.

16.36 LeafFromSurfaceVarRange object

The LeafFromSurfaceVarRange object can be used whenever a Leaf object is expected as an argument. Leaf objects are used to specify the items in a display group. Leaf objects are constructed as temporary objects, which are then used as arguments to DisplayGroup commands.

The LeafFromSurfaceVarRange object is derived from the Leaf object.

Access

```
import displayGroupOdbToolset
```

16.36.1 LeafFromSurfaceVarRange(...)

This method creates a Leaf object from surfaces with values lying in a variable range.

Path

LeafFromSurfaceVarRange

Required arguments

None.

Optional arguments

minimumRange

A Float specifying the minimum value for the variable range. The default value is -3.40282346639E38.

maximumRange

A Float specifying the maximum value for the variable range. The default value is 3.40282346639e+038.

insideRange

A Boolean specifying the method used to evaluate the range. If *insideRange*=ON, the range falls inside the specified minimum and maximum values. The default value is ON.

Return value

A LeafFromSurfaceVarRange object.

Exceptions

None.

16.36.2 Members

The `LeafFromSurfaceVarRange` object has members with the same names and descriptions as the arguments to the **LeafFromSurfaceVarRange** method.

In addition, the `LeafFromSurfaceVarRange` object has the following member:

leafType

A `SymbolicConstant` specifying the leaf type. Possible values are `EMPTY`, `DEFAULT_MODEL`, `ALL_ELEMENTS`, `ALL_NODES`, and `ALL_SURFACES`.

17. Display Options commands

17.1 AssemblyDisplayOptions object

The AssemblyDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport. The AssemblyDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
import assembly
session.viewports[name].assemblyDisplay
session.viewports[name].layers[name].assemblyDisplay
```

17.1.1 setValues(...)

This method modifies the AssemblyDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

visibleInstances

A sequence of Strings specifying the names of the part instances that are visible in the viewport. The default value is an empty sequence.

step

A String specifying the step for which objects are to be displayed. Possible values are any valid step name. The default value is "Initial".

renderStyle

A SymbolicConstant specifying how the image in the viewport is rendered. Possible values are WIREFRAME, HIDDEN, SHADED, and FILLED. The default value is WIREFRAME.

mesh

A Boolean specifying whether the mesh is shown. The default value is OFF.

loads

A Boolean specifying whether loads are shown. The default value is OFF.

bcs

A Boolean specifying whether boundary conditions are shown. The default value is OFF.

DISPLAY OPTIONS COMMANDS

interactions

A Boolean specifying whether interactions are shown. The default value is OFF.

constraints

A Boolean specifying whether constraints are shown. The default value is OFF.

connectors

A Boolean specifying whether connectors are shown. The default value is OFF.

cnxEndPoints

A Boolean specifying whether the connector end points are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxLocalAxes

A Boolean specifying whether the connector local coordinate system axes are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxTypeLabels

A Boolean specifying whether the connector section type labels are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxTagDisplay

A Boolean specifying whether the tag information is displayed along with the connector section type labels. This member is applicable only if *connectors*=ON and if *cnxTypeLabels*=ON. The default value is OFF.

predefinedFields

A Boolean specifying whether fields and initial conditions are shown. The default value is OFF.

visibleDisplayGroups

A sequence of DisplayGroup objects specifying the DisplayGroups visible in the viewport. Currently the sequence can contain a maximum of one DisplayGroup object. The default value is an empty sequence.

engineeringFeatures

A Boolean specifying whether to display engineering features. The default value is OFF.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The *beamScaleFactor* must be greater than zero. The default value is 1.0.

optimizationTasks

A Boolean specifying whether optimization tasks are shown. The default value is OFF.

geometricRestrictions

A Boolean specifying whether geometric restrictions are shown. The default value is OFF.

stopConditions

A Boolean specifying whether stop conditions are shown. The default value is OFF.

Return value**None****Exceptions**

RangeError.

17.1.2 Members

The AssemblyDisplayOptions object can have the following members:

bcs

A Boolean specifying whether boundary conditions are shown. The default value is OFF.

connectors

A Boolean specifying whether connectors are shown. The default value is OFF.

*cnxEndPoint*s

A Boolean specifying whether the connector end points are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxLocalAxes

A Boolean specifying whether the connector local coordinate system axes are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxTypeLabels

A Boolean specifying whether the connector section type labels are shown. This member is applicable only if *connectors*=ON. The default value is ON.

cnxTagDisplay

A Boolean specifying whether the tag information is displayed along with the connector section type labels. This member is applicable only if *connectors*=ON and if *cnxTypeLabels*=ON. The default value is OFF.

constraints

A Boolean specifying whether constraints are shown. The default value is OFF.

engineeringFeatures

A Boolean specifying whether to display engineering features. The default value is OFF.

geometricRestrictions

A Boolean specifying whether geometric restrictions are shown. The default value is OFF.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The beamScaleFactor must be greater than zero. The default value is 1.0.

DISPLAY OPTIONS COMMANDS

predefinedFields

A Boolean specifying whether fields and initial conditions are shown. The default value is OFF.

interactions

A Boolean specifying whether interactions are shown. The default value is OFF.

loads

A Boolean specifying whether loads are shown. The default value is OFF.

mesh

A Boolean specifying whether the mesh is shown. The default value is OFF.

optimizationTasks

A Boolean specifying whether optimization tasks are shown. The default value is OFF.

stopConditions

A Boolean specifying whether stop conditions are shown. The default value is OFF.

renderStyle

A SymbolicConstant specifying how the image in the viewport is rendered. Possible values are WIREFRAME, HIDDEN, SHADED, and FILLED. The default value is WIREFRAME.

bcOptions

A BCDisplayOptions object.

constraintOptions

A ConstraintDisplayOptions object.

displayGroup

A DisplayGroup object specifying the current display group and referring to an object in the *displayGroups* member of Session .

displayGroupInstances

A repository of DisplayGroupInstance objects.

engineeringFeatureOptions

An EngineeringFeatureDisplayOptions object.

predefinedFieldOptions

A PredefinedFieldDisplayOptions object.

geometricRestrictionOptions

A GeometricRestrictionDisplayOptions object.

geometryOptions

A GeometryDisplayOptions object.

interactionOptions

An InteractionDisplayOptions object.

loadOptions

A LoadDisplayOptions object.

meshOptions

A MeshDisplayOptions object.

optimizationTaskOptions

An OptimizationTaskDisplayOptions object.

stopConditionOptions

A StopConditionDisplayOptions object.

symbolOptions

A SymbolDisplayOptions object.

visibleInstances

A tuple of Strings specifying the names of the part instances that are visible in the viewport. The default value is an empty sequence.

step

A String specifying the step for which objects are to be displayed. Possible values are any valid step name. The default value is "Initial".

17.2 BCDisplayOptions object

The BCDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports [name] .assemblyDisplay.bcs=ON
```

The BCDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports [name] .assemblyDisplay.bcOptions
```

```
session.viewports [name] .layers [name] .assemblyDisplay.bcOptions
```

17.2.1 setValues(...)

This method modifies the BCDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

displacement

A Boolean specifying whether displacement symbols are shown. The default value is ON.

velocity

A Boolean specifying whether velocity symbols are shown. The default value is ON.

DISPLAY OPTIONS COMMANDS

acceleration

A Boolean specifying whether acceleration symbols are shown. The default value is ON.

symmetry

A Boolean specifying whether symmetry symbols are shown. The default value is ON.

antiSymmetry

A Boolean specifying whether anti- symmetry symbols are shown. The default value is ON.

temperature

A Boolean specifying whether temperature symbols are shown. The default value is ON.

porePressure

A Boolean specifying whether pore pressure symbols are shown. The default value is ON.

fluidCavityPressure

A Boolean specifying whether fluid cavity pressure symbols are shown. The default value is ON.

acousticPressure

A Boolean specifying whether acoustic pressure symbols are shown. The default value is ON.

electricPotential

A Boolean specifying whether electric potential symbols are shown. The default value is ON.

concentration

A Boolean specifying whether concentration mass diffusion symbols are shown. The default value is ON.

encastre

A Boolean specifying whether encastre symbols are shown. The default value is ON.

pinned

A Boolean specifying whether pinned symbols are shown. The default value is ON.

Return value

None

Exceptions

None.

17.2.2 Members

The BCDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.3 ConstraintDisplayOptions object

The ConstraintDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports [name] .assemblyDisplay.constraints=ON
```

The ConstraintDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports [name] .assemblyDisplay.constraintOptions
```

```
session.viewports [name] .layers [name] .assemblyDisplay.constraintOptions
```

17.3.1 setValues(...)

This method modifies the ConstraintDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

constraintEquation

A Boolean specifying whether constraint equation symbols are shown. The default value is ON.

tieConstraint

A Boolean specifying whether tie constraint symbols are shown. The default value is ON.

rigidBodyConstraint

A Boolean specifying whether rigid body constraint symbols are shown. The default value is ON.

displayBodyConstraint

A Boolean specifying whether display body constraint symbols are shown. The default value is ON.

couplingConstraint

A Boolean specifying whether coupling constraint symbols are shown. The default value is ON.

Return value

None

Exceptions

None.

17.3.2 Members

The `ConstraintDisplayOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

17.4 EngineeringFeatureDisplayOptions object

The `EngineeringFeatureDisplayOptions` object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.engineeringFeatures=ON
```

The `EngineeringFeatureDisplayOptions` object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.engineeringFeatureOptions
session.viewports[name].layers[name].assemblyDisplay\
.engineeringFeatureOptions
import part
session.viewports[name].layers[name].partDisplay\
.engineeringFeatureOptions
session.viewports[name].partDisplay.engineeringFeatureOptions
```

17.4.1 setValues(...)

This method modifies the `EngineeringFeatureDisplayOptions` object.

Arguments

Required arguments

None.

Optional arguments

pointMassInertia

A Boolean specifying whether point mass inertia symbols are shown. The default value is ON.

nonstructuralMass

A Boolean specifying whether nonstructural mass symbols are shown. The default value is ON.

heatCapacitance

A Boolean specifying whether heat capacitance symbols are shown. The default value is ON.

contourIntegral

A Boolean specifying whether contour integral symbols are shown. The default value is ON.

springToGround

A Boolean specifying whether spring-to-ground symbols are shown. The default value is ON.

twoPointSpring

A Boolean specifying whether two-point spring symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.4.2 Members

The EngineeringFeatureDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.5 PredefinedFieldDisplayOptions object

The PredefinedFieldDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.predefinedFields=ON
```

The PredefinedFieldDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.predefinedFieldOptions  
session.viewports[name].layers[name].assemblyDisplay\  
.predefinedFieldOptions
```

17.5.1 setValues(...)

This method modifies the PredefinedFieldDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

temperatureField

A Boolean specifying whether temperature field symbols are shown. The default value is ON.

velocityField

A Boolean specifying whether translational velocity symbols are shown. The default value is ON.

generalField

A Boolean specifying whether general field symbols are shown. The default value is ON.

stressField

A Boolean specifying whether stress field symbols are shown. The default value is ON.

hardeningField

A Boolean specifying whether hardening field symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.5.2 Members

The `PredefinedFieldDisplayOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

17.6 GeometryDisplayOptions object

The `GeometryDisplayOptions` object stores settings that specify how assemblies are to be displayed in a particular viewport. The `GeometryDisplayOptions` object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.geometryOptions
session.viewports[name].layers[name].assemblyDisplay.geometryOptions
session.viewports[name].layers[name].partDisplay.geometryOptions
session.viewports[name].partDisplay.geometryOptions
```


17.6.1 **setValues(...)**

This method modifies the GeometryDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

geometryEdgesInShaded

A Boolean specifying whether geometry edges are displayed in shaded mode. The default value is ON.

geometryHiddenEdges

A Boolean specifying whether geometry hidden edges are displayed (dotted) in hidden line mode. The default value is OFF.

geometrySilhouetteEdges

A Boolean specifying whether geometry silhouette edges are displayed. The default value is ON.

datumAxes

A Boolean specifying whether datum axes are shown. The default value is ON.

datumCoordSystems

A Boolean specifying whether datum coordinate systems are shown. The default value is ON.

datumPlanes

A Boolean specifying whether datum planes are shown. The default value is ON.

referencePointLabels

A Boolean specifying whether referencePoint labels are shown. The default value is ON.

referencePointSymbols

A Boolean specifying whether referencePoint symbols are shown. The default value is ON.

referenceRepresentation

A Boolean specifying whether geometry that belongs to the reference representation of the Part or Instance is shown. The default value is OFF.

referenceRepTranslucency

A Boolean specifying whether to apply translucency to the geometry that belongs to the reference representation of the Part or Instance. The default value is ON.

Return value

None

Exceptions

RangeError.

17.6.2 Members

The GeometryDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.7 GeometricRestrictionDisplayOptions object

The GeometricRestrictionDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.geometricRestrictions=ON
```

The GeometricRestrictionDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.geometricRestrictionOptions  
session.viewports[name].layers[name].assemblyDisplay\  
.geometricRestrictionOptions
```

17.7.1 setValues(...)

This method modifies the GeometricRestrictionDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

drillControl

A Boolean specifying whether drill control geometric restriction symbols are shown. The default value is ON.

fixedRegion

A Boolean specifying whether fixed region geometric restriction symbols are shown. The default value is ON.

frozenArea

A Boolean specifying whether frozen area geometric restriction symbols are shown. The default value is ON.

growth

A Boolean specifying whether growth geometric restriction symbols are shown. The default value is ON.

penetrationCheck

A Boolean specifying whether penetration check geometric restriction symbols are shown. The default value is ON.

shapeDemoldControl

A Boolean specifying whether demold control (shape) geometric restriction symbols are shown. The default value is ON.

designDirection

A Boolean specifying whether design direction geometric restriction symbols are shown. The default value is ON.

shapeMemberSize

A Boolean specifying whether member size (shape) geometric restriction symbols are shown. The default value is ON.

shapePlanarSymmetry

A Boolean specifying whether planar symmetry (shape) geometric restriction symbols are shown. The default value is ON.

shapePointSymmetry

A Boolean specifying whether point symmetry (shape) geometric restriction symbols are shown. The default value is ON.

shapeRotationalSymmetry

A Boolean specifying whether rotational symmetry (shape) geometric restriction symbols are shown. The default value is ON.

stampControl

A Boolean specifying whether stamp control geometric restriction symbols are shown. The default value is ON.

slideRegionControl

A Boolean specifying whether slide region control geometric restriction symbols are shown. The default value is ON.

topologyCyclicSymmetry

A Boolean specifying whether cyclic symmetry geometric restriction symbols are shown. The default value is ON.

topologyDemoldControl

A Boolean specifying whether demold control (topology) geometric restriction symbols are shown. The default value is ON.

topologyMemberSize

A Boolean specifying whether member size (topology) geometric restriction symbols are shown. The default value is ON.

topologyPlanarSymmetry

A Boolean specifying whether planar symmetry (topology) geometric restriction symbols are shown. The default value is ON.

topologyPointSymmetry

A Boolean specifying whether point symmetry (topology) geometric restriction symbols are shown. The default value is ON.

topologyRotationalSymmetry

A Boolean specifying whether rotational symmetry (topology) geometric restriction symbols are shown. The default value is ON.

turnControl

A Boolean specifying whether turn control geometric restriction symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.7.2 Members

The GeometricRestrictionDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.8 GraphicsInfo object

The GraphicsInfo object stores information about the graphics adapter installed in your system. The GraphicsInfo object has no constructor or methods; Abaqus creates the *GraphicsInfo* member when a session is started.

If you execute Abaqus/CAE on a remote system and display the main window locally, the *glx server* is your local machine and the *glx client* is the remote machine. This definition of client and server follows the X11 naming convention. If you execute and display Abaqus/CAE on the same machine, you will typically see identical values for both the name of the server and the name of the client.

The members are all read-only.

Access

session.graphicsInfo

17.8.1 Members

The GraphicsInfo object has the following members:

glVersion

A sequence of the type (Int, Int, String) specifying the three components of the OpenGL version. The sequence consists of an Int with the OpenGL major version number, an Int with the OpenGL minor version number, and a String with any additional information.

glxServerVersion

A sequence of the type (Int, Int, String) specifying the three components of the glx version of the server. The sequence consists of an Int with the glx major version number, an Int with the glx minor version number, and a String with any additional information.

glxClientVersion

A sequence of the type (Int, Int, String) specifying the three components of the of glx version of the client. The sequence consists of an Int with the glx major version number, an Int with the glx minor version number, and String with any additional information.

glVendor

A String specifying the graphics adapter vendor. On hardware accelerated systems *glVendor* specifies the vendor that manufactured the adapter. On systems without hardware acceleration *glVendor* specifies the developer of the software graphics library.

glRenderer

A String specifying the name of the rendering device or the name of the software graphics library.

glxServerVendor

A String specifying the glx developer on the server side.

glxClientVendor

A String specifying the glx developer on the client side.

17.9 GraphicsOptions object

The GraphicsOptions object stores settings that control how objects are rendered in all viewports. GraphicsOptions objects are accessed in one of two ways:

- The default graphics options. These settings are used as defaults when you start a session and by the **Defaults** button on the **Graphics Options** dialog box.
- The current graphics options.

The GraphicsOptions object has no constructor; Abaqus creates both the *defaultGraphicsOptions* and the *graphicsOptions* members when a session is started.

When you start a session, Abaqus detects the graphics hardware installed on your system and uses the **setValues** method in the environment file (**abacus_v6.env**) to modify the members of the GraphicsOptions object. If your graphics hardware is not supported by Abaqus/CAE, or if you wish to override the default graphics options, you can modify settings in the environment file. For more information, see “Tuning graphics cards,” Section 5.3 of the Abaqus Installation and Licensing Guide.

Access

`session.defaultGraphicsOptions`
`session.graphicsOptions`

17.9.1 setValues(...)

This method modifies the GraphicsOptions object.

Arguments

Required arguments

None.

Optional arguments

graphicsDriver

A SymbolicConstant specifying the graphics driver to use. Abaqus/CAE currently uses OpenGL exclusively so the only possible value is OPEN_GL. OPEN_GL takes advantage of graphics adapter hardware acceleration.

doubleBuffering

A Boolean specifying whether double buffering is used. The default value is ON.

Double buffering controls where Abaqus/CAE draws its graphics. When *doubleBuffering*=OFF, everything is drawn directly to the screen and on many systems you can see the progress of the drawing operations. Most users find this distracting, especially in dynamic situations such as view manipulation or animation of results. When *doubleBuffering*=ON, the drawing occurs in a separate graphics buffer that is displayed when all the drawing operations are complete. This results in a much smoother display during view changes or animation. It is recommended that you set double buffering to ON.

displayLists

A Boolean specifying whether a display list will be used to accelerate graphics performance. The default value is ON.

When *displayLists*=ON, drawing operations are recorded in a list that can be quickly replayed. This results in faster drawing on most systems but requires extra memory to record the drawing operations. In the Visualization module, display lists are only used during view manipulations and then their use is subject to the setting of *viewManipDisplayListThreshold*.

highlightMethodHint

A sequence of SymbolicConstants specifying a hint used to modify the highlight method. Possible values are:

- **HARDWARE_OVERLAY**, specifying a hint of hardware overlay. The best graphics performance is achieved using hardware overlay, but not all systems and graphics adapters support hardware overlay.

- XOR, specifying a hint of XOR technique. The XOR technique uses a boolean pixel operation to simulate the drawing operations but can produce different colors depending on the color of the underlying pixels.
- SOFTWARE_OVERLAY, specifying a hint of software overlay. The software overlay method simulates the effect of hardware overlay.
- BLEND, specifying a hint of blend method. The blend method combines the color of the underlying pixel with the desired color producing an approximation of the transient graphics.

The default value is (HARDWARE_OVERLAY, XOR, SOFTWARE_OVERLAY, BLEND).

The values of this sequence are applied by Abaqus when you start a session in first to last order. The first successful value becomes the default highlight method. Not all graphics adapters support the HARDWARE_OVERLAY value and you must use the *highlightMethodHint* argument to provide an alternative.

You can use a single value to set the first element of the list, or you can use a tuple with one to four unique values. Abaqus sets any remaining elements of the tuple to unique values based on the default order.

dragMode

A SymbolicConstant specifying which rendering is used during dynamic rotations of the view. Possible values are:

- FAST, specifying a rendering mode where the image is rendered in wireframe.
- AS_IS, specifying a rendering mode where the image is rendered as is.

The default value is AS_IS.

When set to *dragMode*=FAST, a wireframe outline is drawn during view changes by rotation, pan, or zoom. When *dragMode*=AS_IS, everything displayed in the window will be drawn during view changes; however, the display may lag behind the mouse movement when the model is complex especially if you are using an older or slower system. For newer systems with graphics hardware acceleration the AS_IS setting can be accommodated without significant loss of performance.

antiAlias

A Boolean specifying whether lines will be smoothed to reduce the jagged effect of rasterization. The default value is ON.

autoFitAfterRotate

A Boolean specifying whether the model is automatically resized to fit the viewport after each view rotation. The default value is OFF.

polygonOffsetConstant

A Float specifying the offset added when drawing the faces of a polygon. The *polygonOffsetConstant* argument affects the behavior of only the OpenGL driver. Possible values are $0.0 \leq \text{polygonOffsetConstant} \leq 100.0$. The default value is platform dependent and is typically between 0.0 and 2.0.

DISPLAY OPTIONS COMMANDS

polygonOffsetSlope

A Float specifying the factor that multiplies the slope of each line before the line is added to the vertexes of a polygon face. The *polygonOffsetSlope* argument affects the behavior of only the OpenGL driver. Possible values are $0.0 \leq \text{polygonOffsetSlope} \leq 100.0$. The default value is platform dependent and is typically between 0.0 and 2.0.

printPolygonOffsetConstant

A Float specifying the offset added when drawing the faces of a polygon. *printPolygonOffsetConstant* is similar to *polygonOffsetConstant*; however, *printPolygonOffsetConstant* is used when printing and *polygonOffsetConstant* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different offset values for each driver.

printPolygonOffsetSlope

A Float specifying the factor that multiplies the slope of each line before the line is added to the vertexes of a polygon face. *printPolygonOffsetSlope* is similar to *polygonOffsetSlope*; however, *printPolygonOffsetSlope* is used when printing and *polygonOffsetSlope* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different offset values for each driver.

vertexArrays

A Boolean specifying how the three-dimensional vertices of the model are processed. When *vertexArrays*=OFF, each vertex of the model is processed separately. When *vertexArrays*=ON, the vertices are processed in large blocks resulting in faster display. Not all graphics adapters support this capability correctly. An indicator that the graphics adapters is not processing three-dimensional vertices correctly is the absence of graphics during “rubber banding” operations. For example, when dynamically dragging the radius of a circle in the Sketcher, the circle should be visible. The default value is ON.

vertexArraysInDisplayLists

A Boolean specifying whether the *vertexArrays* setting should temporarily be set to OFF when building a display list. The default value is ON.

Some graphics adapters do not properly support using vertex arrays inside a display list. Setting *vertexArraysInDisplayLists* to OFF has a smaller impact on graphics performance than setting *vertexArrays* or *displayLists* to OFF.

viewManipDisplayListThreshold

An Int specifying how large a display list may be created in order to accelerate view manipulation operations. Increasing this value when viewing large models will increase the delay before a view manipulation operation begins in order to obtain improved graphics performance during the view manipulation. If set high with a large model, the delay can be many seconds. In excessive cases, graphics memory can be exceeded and the result may be an empty display list (no visible model) for the view manipulation. This setting is treated as 0 if *displayLists*=OFF. Possible values are $0 \leq \text{viewManipDisplayListThreshold} \leq 20000$. The default value is 40.

directRendering

A Boolean specifying how Abaqus renders X11 graphics operations. When *directRendering*=OFF, the graphics are rendered through the X Server. When *directRendering*=ON, the graphics operations are sent directly to the graphics adapter producing faster displays. For maximum performance, the initial value is ON. This argument is used only when you first start Abaqus/CAE; you cannot configure *directRendering* during a session.

hardwareAcceleration

A Boolean specifying whether a hardware accelerated OpenGL graphics driver will be used on Windows platforms. The default value is ON.

When *hardwareAcceleration*=OFF, the graphics driver uses a software implementation of OpenGL that is included with the operating system. This results in slower drawing on most systems; however, you may have to use the software implementation of OpenGL if the hardware graphics driver is incompatible with Abaqus/CAE.

hardwareAcceleration is used only when you first start Abaqus/CAE on a Windows platform; you cannot configure *hardwareAcceleration* during a session. *hardwareAcceleration* is not used when you start Abaqus/CAE on an X-Windows platform and display to a Windows platform running Exceed or any other X-Windows server.

hardwareOverlay

A Boolean specifying whether hardware overlay planes will be used if available. The default value is the same value as the *hardwareOverlayAvailable* member.

When *hardwareOverlayAvailable*=OFF, it will not be possible to set *hardwareOverlay* to ON and the `HARDWARE_OVERLAY` highlight method will not be available. If viewports display a solid color and will not display the model, it will be necessary to inhibit hardware overlay completely by setting the `ABAQUS_EMULATE_OVERLAYS` environment variable (to any value) before starting Abaqus/CAE.

hardwareOverlay is used only when you first start Abaqus/CAE; you cannot configure *hardwareOverlay* during a session.

textureMapping

A Boolean specifying whether textures will be used to display contour plots. The default value is ON.

Turning off texture mapping is necessary only if viewports will not correctly display a contour plot of your model.

printTextureMapping

A Boolean specifying whether textures will be used to display contour plots. The default value is ON.

Turning off texture mapping for printing is necessary only if printed output does not correctly display a contour plot of your model. *printTextureMapping* is similar to *textureMapping*; however, *printTextureMapping* is used when printing and *textureMapping* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different settings for each driver.

DISPLAY OPTIONS COMMANDS

backgroundStyle

A SymbolicConstant specifying the background style to be used for all viewport windows. Possible values are SOLID and GRADIENT. The default value is SOLID.

If *backgroundStyle*=SOLID, the viewport background will appear as a solid color as specified by *backgroundColor*. If *backgroundStyle*=GRADIENT, the viewport background will be drawn as a gradient beginning with the *backgroundColor* at the top of the viewport and gradually blending to the *backgroundBottomColor* at the bottom of the viewport.

backgroundColor

A String specifying one of the two background colors for all viewport windows. The initial color is black. A list of valid color strings is in the *colors* map in the Session object.

backgroundBottomColor

A String specifying one of the two background colors for all viewport windows. This color is used only if *backgroundStyle* =GRADIENT. The initial color is black. A list of valid color strings is in the *colors* map in the Session object.

backgroundOverride

A Boolean specifying whether a viewport background style of GRADIENT can be overridden when displaying certain objects, such as sketches or XY plots. When overridden, the background will be the top color of the gradient background.

backfaceCulling

A Boolean specifying whether facets that are determined to be facing away from the viewer will be drawn. The default value is ON. *backfaceCulling* provides a performance enhancement when displaying solid elements where the front side of the element occludes the back side of the element. Set *backfaceCulling*=OFF if it appears that you are seeing the back side of an element and the front side is missing. You should also set *backfaceCulling*=OFF if you believe the display is not complete.

accelerateOffScreen

A Boolean specifying whether the hardware accelerated graphics driver will be used for off-screen rendering. The default value is ON if graphics hardware acceleration is available and has not been disabled via the hardwareAcceleration option, and the graphics driver supports the underlying technology. When set to OFF, an alternate (slower) technique will be used to create off-screen images. Off-screen rendering is used for Printing, Probe, and backing store (viewport refresh). Setting this value to OFF will force printed images to be rendered without hardware acceleration. This is useful when writing automated tests to produce raster images that you will want to compare across multiple machines that may have different graphics environments.

backingStore

A Boolean specifying whether a backing store will be used to refresh a viewport after a window occluding the viewport is moved or dismissed. The default value is ON.

shadersAvailable

A Boolean specifying if the graphics hardware supports the OpenGL Shading Language (GLSL).

translucencyMode

An Int specifying whether speed or accuracy is more important when drawing translucent objects. Lower values optimize for speed while higher values optimize for accuracy. The actual meaning of each setting will depend on the setting of *shadersAvailable* and the capabilities of the graphics hardware and driver. Possible values are $1 \leq \text{translucencyMode} \leq 6$. The default value is 4.

options

None or a GraphicsOptions object specifying the object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in the *options* member. The default value is **None**.

contourRangeTexturePrecision

A Float specifying a tolerance used when computing the appropriate scale for transforming result (contour) values to texture values. When set too low the 'out of range' colors may be incorrectly shown for values near the range limits. The default value is 0.5×10^{-5} .

Return value

None

Exceptions

RangeError.

17.9.2 Members

The GraphicsOptions object can have the following members:

backgroundOverride

A Boolean specifying whether a viewport background style of GRADIENT can be overridden when displaying certain objects, such as sketches or XY plots. When overridden, the background will be the top color of the gradient background.

backfaceCulling

A Boolean specifying whether facets that are determined to be facing away from the viewer will be drawn. The default value is ON. *backfaceCulling* provides a performance enhancement when displaying solid elements where the front side of the element occludes the back side of the element. Set *backfaceCulling*=OFF if it appears that you are seeing the back side of an element and the front side is missing. You should also set *backfaceCulling*=OFF if you believe the display is not complete.

graphicsDriver

A SymbolicConstant specifying the graphics driver to use. Abaqus/CAE currently uses OpenGL exclusively so the only possible value is OPEN_GL. OPEN_GL takes advantage of graphics adapter hardware acceleration.

doubleBuffering

A Boolean specifying whether double buffering is used. The default value is ON.

DISPLAY OPTIONS COMMANDS

Double buffering controls where Abaqus/CAE draws its graphics. When *doubleBuffering*=OFF, everything is drawn directly to the screen and on many systems you can see the progress of the drawing operations. Most users find this distracting, especially in dynamic situations such as view manipulation or animation of results. When *doubleBuffering*=ON, the drawing occurs in a separate graphics buffer that is displayed when all the drawing operations are complete. This results in a much smoother display during view changes or animation. It is recommended that you set double buffering to ON.

displayLists

A Boolean specifying whether a display list will be used to accelerate graphics performance. The default value is ON.

When *displayLists*=ON, drawing operations are recorded in a list that can be quickly replayed. This results in faster drawing on most systems but requires extra memory to record the drawing operations. In the Visualization module, display lists are only used during view manipulations and then their use is subject to the setting of *viewManipDisplayListThreshold*.

dragMode

A SymbolicConstant specifying which rendering is used during dynamic rotations of the view. Possible values are:

- FAST, specifying a rendering mode where the image is rendered in wireframe.
- AS_IS, specifying a rendering mode where the image is rendered as is.

The default value is AS_IS.

When set to *dragMode*=FAST, a wireframe outline is drawn during view changes by rotation, pan, or zoom. When *dragMode*=AS_IS, everything displayed in the window will be drawn during view changes; however, the display may lag behind the mouse movement when the model is complex especially if you are using an older or slower system. For newer systems with graphics hardware acceleration the AS_IS setting can be accommodated without significant loss of performance.

antiAlias

A Boolean specifying whether lines will be smoothed to reduce the jagged effect of rasterization. The default value is ON.

autoFitAfterRotate

A Boolean specifying whether the model is automatically resized to fit the viewport after each view rotation. The default value is OFF.

polygonOffsetConstant

A Float specifying the offset added when drawing the faces of a polygon. The *polygonOffsetConstant* argument affects the behavior of only the OpenGL driver. Possible values are $0.0 \leq \text{polygonOffsetConstant} \leq 100.0$. The default value is platform dependent and is typically between 0.0 and 2.0.

polygonOffsetSlope

A Float specifying the factor that multiplies the slope of each line before the line is added to the vertexes of a polygon face. The *polygonOffsetSlope* argument affects the behavior of only the OpenGL driver. Possible values are $0.0 \leq \text{polygonOffsetSlope} \leq 100.0$. The default value is platform dependent and is typically between 0.0 and 2.0.

printPolygonOffsetConstant

A Float specifying the offset added when drawing the faces of a polygon. *printPolygonOffsetConstant* is similar to *polygonOffsetConstant*; however, *printPolygonOffsetConstant* is used when printing and *polygonOffsetConstant* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different offset values for each driver.

printPolygonOffsetSlope

A Float specifying the factor that multiplies the slope of each line before the line is added to the vertexes of a polygon face. *printPolygonOffsetSlope* is similar to *polygonOffsetSlope*; however, *printPolygonOffsetSlope* is used when printing and *polygonOffsetSlope* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different offset values for each driver.

vertexArrays

A Boolean specifying how the three-dimensional vertices of the model are processed. When *vertexArrays*=OFF, each vertex of the model is processed separately. When *vertexArrays*=ON, the vertices are processed in large blocks resulting in faster display. Not all graphics adapters support this capability correctly. An indicator that the graphics adapters is not processing three-dimensional vertices correctly is the absence of graphics during “rubber banding” operations. For example, when dynamically dragging the radius of a circle in the Sketcher, the circle should be visible. The default value is ON.

vertexArraysInDisplayLists

A Boolean specifying whether the *vertexArrays* setting should temporarily be set to OFF when building a display list. The default value is ON.

Some graphics adapters do not properly support using vertex arrays inside a display list. Setting *vertexArraysInDisplayLists* to OFF has a smaller impact on graphics performance than setting *vertexArrays* or *displayLists* to OFF.

viewManipDisplayListThreshold

An Int specifying how large a display list may be created in order to accelerate view manipulation operations. Increasing this value when viewing large models will increase the delay before a view manipulation operation begins in order to obtain improved graphics performance during the view manipulation. If set high with a large model, the delay can be many seconds. In excessive cases, graphics memory can be exceeded and the result may be an empty display list (no visible model) for the view manipulation. This setting is treated as 0 if *displayLists*=OFF. Possible values are $0 \leq \text{viewManipDisplayListThreshold} \leq 20000$. The default value is 40.

directRendering

A Boolean specifying how Abaqus renders X11 graphics operations. When *directRendering*=OFF, the graphics are rendered through the X Server. When *directRendering*=ON, the graphics operations are sent directly to the graphics adapter producing faster displays. For maximum performance, the initial value is ON. This argument is used only when you first start Abaqus/CAE; you cannot configure *directRendering* during a session.

hardwareAcceleration

A Boolean specifying whether a hardware accelerated OpenGL graphics driver will be used on Windows platforms. The default value is ON.

When *hardwareAcceleration*=OFF, the graphics driver uses a software implementation of OpenGL that is included with the operating system. This results in slower drawing on most systems; however, you may have to use the software implementation of OpenGL if the hardware graphics driver is incompatible with Abaqus/CAE.

hardwareAcceleration is used only when you first start Abaqus/CAE on a Windows platform; you cannot configure *hardwareAcceleration* during a session. *hardwareAcceleration* is not used when you start Abaqus/CAE on an X-Windows platform and display to a Windows platform running Exceed or any other X-Windows server.

hardwareOverlay

A Boolean specifying whether hardware overlay planes will be used if available. The default value is the same value as the *hardwareOverlayAvailable* member.

When *hardwareOverlayAvailable*=OFF, it will not be possible to set *hardwareOverlay* to ON and the `HARDWARE_OVERLAY` highlight method will not be available. If viewports display a solid color and will not display the model, it will be necessary to inhibit hardware overlay completely by setting the `ABAQUS_EMULATE_OVERLAYS` environment variable (to any value) before starting Abaqus/CAE.

hardwareOverlay is used only when you first start Abaqus/CAE; you cannot configure *hardwareOverlay* during a session.

textureMapping

A Boolean specifying whether textures will be used to display contour plots. The default value is ON.

Turning off texture mapping is necessary only if viewports will not correctly display a contour plot of your model.

printTextureMapping

A Boolean specifying whether textures will be used to display contour plots. The default value is ON.

Turning off texture mapping for printing is necessary only if printed output does not correctly display a contour plot of your model. *printTextureMapping* is similar to *textureMapping*; however, *printTextureMapping* is used when printing and *textureMapping* is used for display. Some systems, especially Windows, use different OpenGL drivers for printing and display, and you may have to use different settings for each driver.

backgroundStyle

A SymbolicConstant specifying the background style to be used for all viewport windows. Possible values are SOLID and GRADIENT. The default value is SOLID.

If *backgroundStyle*=SOLID, the viewport background will appear as a solid color as specified by *backgroundColor*. If *backgroundStyle*=GRADIENT, the viewport background will be drawn as a gradient beginning with the *backgroundColor* at the top of the viewport and gradually blending to the *backgroundBottomColor* at the bottom of the viewport.

accelerateOffScreen

A Boolean specifying whether the hardware accelerated graphics driver will be used for off-screen rendering. The default value is ON if graphics hardware acceleration is available and has not been disabled via the hardwareAcceleration option, and the graphics driver supports the underlying technology. When set to OFF, an alternate (slower) technique will be used to create off-screen images. Off-screen rendering is used for Printing, Probe, and backing store (viewport refresh). Setting this value to OFF will force printed images to be rendered without hardware acceleration. This is useful when writing automated tests to produce raster images that you will want to compare across multiple machines that may have different graphics environments.

backingStore

A Boolean specifying whether a backing store will be used to refresh a viewport after a window occluding the viewport is moved or dismissed. The default value is ON.

highlightMethod

A SymbolicConstant specifying the highlight method. For the GraphicsOptions object, possible values of the member are HARDWARE_OVERLAY, XOR, SOFTWARE_OVERLAY, and BLEND.

hardwareOverlayAvailable

A Boolean specifying if the graphics hardware supports hardware overlay.

shadersAvailable

A Boolean specifying if the graphics hardware supports the OpenGL Shading Language (GLSL).

translucencyMode

An Int specifying whether speed or accuracy is more important when drawing translucent objects. Lower values optimize for speed while higher values optimize for accuracy. The actual meaning of each setting will depend on the setting of *shadersAvailable* and the capabilities of the graphics hardware and driver. Possible values are $1 \leq \text{translucencyMode} \leq 6$. The default value is 4.

contourRangeTexturePrecision

A Float specifying a tolerance used when computing the appropriate scale for transforming result (contour) values to texture values. When set too low the 'out of range' colors may be incorrectly shown for values near the range limits. The default value is 0.5×10^{-5} .

options

None or a GraphicsOptions object specifying the object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in the *options* member. The default value is **None**.

highlightMethodHint

A tuple of SymbolicConstants specifying a hint used to modify the highlight method. Possible values are:

- `HARDWARE_OVERLAY`, specifying a hint of hardware overlay. The best graphics performance is achieved using hardware overlay, but not all systems and graphics adapters support hardware overlay.
- `XOR`, specifying a hint of XOR technique. The XOR technique uses a boolean pixel operation to simulate the drawing operations but can produce different colors depending on the color of the underlying pixels.
- `SOFTWARE_OVERLAY`, specifying a hint of software overlay. The software overlay method simulates the effect of hardware overlay.
- `BLEND`, specifying a hint of blend method. The blend method combines the color of the underlying pixel with the desired color producing an approximation of the transient graphics.

The default value is (`HARDWARE_OVERLAY`, `XOR`, `SOFTWARE_OVERLAY`, `BLEND`).

The values of this sequence are applied by Abaqus when you start a session in first to last order. The first successful value becomes the default highlight method. Not all graphics adapters support the `HARDWARE_OVERLAY` value and you must use the *highlightMethodHint* argument to provide an alternative.

You can use a single value to set the first element of the list, or you can use a tuple with one to four unique values. Abaqus sets any remaining elements of the tuple to unique values based on the default order.

backgroundColor

A String specifying one of the two background colors for all viewport windows. The initial color is black. A list of valid color strings is in the *colors* map in the Session object.

backgroundBottomColor

A String specifying one of the two background colors for all viewport windows. This color is used only if *backgroundStyle* = `GRADIENT`. The initial color is black. A list of valid color strings is in the *colors* map in the Session object.

17.10 InteractionDisplayOptions object

The InteractionDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.interactions=ON
```

The InteractionDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.interactionOptions
session.viewports[name].layers[name].assemblyDisplay\
.interactionOptions
```

17.10.1 setValues(...)

This method modifies the InteractionDisplayOptions object.

Arguments**Required arguments**

None.

Optional arguments*surfaceContact*

A Boolean specifying whether surface contact symbols are shown. The default value is ON.

selfContact

A Boolean specifying whether self contact symbols are shown. The default value is ON.

elasticFoundation

A Boolean specifying whether elastic foundation symbols are shown. The default value is ON.

actuatorSensor

A Boolean specifying whether actuator/sensor symbols are shown. The default value is ON.

radiationAmbient

A Boolean specifying whether surface radiation-to-ambient symbols are shown. The default value is ON.

filmCondition

A Boolean specifying whether surface film condition symbols are shown. The default value is ON.

concentratedRadiationToAmbient

A Boolean specifying whether concentrated radiation-to-ambient symbols are shown. The default value is ON.

concentratedFilmCondition

A Boolean specifying whether concentrated film condition symbols are shown. The default value is ON.

Return value

None

Exceptions

`RangeError`.

17.10.2 Members

The `InteractionDisplayOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

17.11 Light object

The `Light` object stores settings that control how objects are lit when the `renderStyle` is set to `SHADED`.

The `Light` object has no constructor; Abaqus creates them as part of the *defaultLightOptions* and the *lightOptions* objects when a session is started.

Access

`session.defaultLightOptions.lights[i]`

`session.viewports[name].lightOptions.lights[i]`

17.11.1 setValues(...)

This method modifies the `Light` object.

Arguments

Required arguments

None.

Optional arguments

enabled

A Boolean specifying whether the light is on or off. The default value is `OFF`.

type

A `SymbolicConstant` specifying how the effect of the light should be calculated. Possible values are:

- `DIRECTIONAL`, specifying a constant vector should be used for the direction from the light to a vertex.
- `POINT`, specifying the vector from the light to each vertex should be calculated.

The default value is `DIRECTIONAL`.

When set to *type* =DIRECTIONAL, a constant vector is used for the direction from the light to a vertex. When *type* =POINT, the result is more realistic because the actual vector from the light to each vertex is calculated. However, overall performance is decreased.

latitude

A Float specifying the altitude of the light above or below the camera. Possible values are $-90.0 \leq \text{latitude} \leq 90.0$. The default value is 0.0.

longitude

A Float specifying the east/west position of the light to the left or right of the camera. Possible values are $-90.0 \leq \text{longitude} \leq 90.0$. The default value is 0.0.

diffuseColor

A String specifying the color of the light added to the scene by this light source. The initial value is 70% gray. A list of valid color strings is in the *colors* map in the *session* object.

specularColor

A String specifying the color of the specular highlights created by this light source. The initial value is 36% gray. A list of valid color strings is in the *colors* map in the *session* object.

Return value

None

Exceptions

RangeError.

17.11.2 Members

The Light object has members with the same names and descriptions as the arguments to the **setValues** method.

17.12 LightOptions object

The LightOptions object stores settings that control how objects are lit when the *renderStyle* is set to SHADED. LightOptions objects are accessed in one of two ways:

- The default light options. These settings are used as defaults when you start a session and by the **Defaults** button on the **Light Options** dialog box.
- The light options associated with a particular viewport.

The LightOptions object has no constructor; Abaqus creates the *defaultLightOptions* member when a session is started. When a new viewport is created, the *lightOptions* member is copied from the current viewport.

Access

```
session.defaultLightOptions
session.viewports[name].lightOptions
```

17.12.1 setValues(...)

This method modifies the LightOptions object.

Arguments

Required arguments

None.

Optional arguments

shading

A SymbolicConstant specifying the shading technique that should be used to fill facets for geometric bodies when *renderStyle* = SHADED. The lighting of facets for meshed geometry is not affected by this setting. Possible values are:

- GOURAUD, specifying that lighting values should be computed for each corner of a facet and interpolated to fill the remainder of the facet.
- PHONG, specifying that lighting values should be computed for each pixel of a facet.

The default value is GOURAUD.

When set to *shading* = GOURAUD, lighting values are computed for each corner of a facet and the lighting for the remainder of the facet is interpolated from the corner values. When *shading* = PHONG, lighting values are computed for each pixel of a facet.

Note: The shading technique can only be set to PHONG when the *shadersAvailable* member of GraphicsOptions is True.

Printed output will only contain Phong shading if the output format is raster and the *accelerateOffScreen* member of GraphicsOptions is ON.

The *renderStyle* setting is part of AssemblyDisplayOptions, PartDisplayOptions, and various other options objects not listed here since they are generally used for meshed geometry.

viewpoint

A SymbolicConstant specifying how specular highlights should be calculated. Possible values are:

- INFINITE, specifying that a constant vector should be used for the direction from the camera to a vertex when computing specular highlights.
- LOCAL, specifying that the vector from the camera to each vertex should be calculated when computing specular highlights.

The default value is INFINITE.

When set to *viewpoint* = INFINITE, a constant vector is used for the direction from the camera (viewpoint) to a vertex. When *viewpoint* = LOCAL, the result is more realistic because the actual vector from the camera to each vertex is calculated. However, overall performance is decreased.

ambientColor

A String specifying the light applied evenly to the entire scene independent of any individual light. The initial value is 20% gray. A list of valid color strings is in the *colors* map in the *session* object.

materialShininess

A Float specifying the degree to which specular reflection is focused. The higher the *materialShininess* argument, the more focused the specular highlight. Possible values are $0.0 \leq \text{polygonOffsetConstant} \leq 128.0$. The default value is 105.0.

Return value

None

Exceptions

RangeError.

17.12.2 Members

The LightOptions object has the following members:

shading

A SymbolicConstant specifying the shading technique that should be used to fill facets for geometric bodies when *renderStyle* = SHADED. The lighting of facets for meshed geometry is not affected by this setting. Possible values are:

- GOURAUD, specifying that lighting values should be computed for each corner of a facet and interpolated to fill the remainder of the facet.
- PHONG, specifying that lighting values should be computed for each pixel of a facet.

The default value is GOURAUD.

When set to *shading* = GOURAUD, lighting values are computed for each corner of a facet and the lighting for the remainder of the facet is interpolated from the corner values. When *shading* = PHONG, lighting values are computed for each pixel of a facet.

Note: The shading technique can only be set to PHONG when the *shadersAvailable* member of GraphicsOptions is True.

Printed output will only contain Phong shading if the output format is raster and the *accelerateOffScreen* member of GraphicsOptions is ON.

The *renderStyle* setting is part of AssemblyDisplayOptions, PartDisplayOptions, and various other options objects not listed here since they are generally used for meshed geometry.

viewpoint

A SymbolicConstant specifying how specular highlights should be calculated. Possible values are:

- INFINITE, specifying that a constant vector should be used for the direction from the camera to a vertex when computing specular highlights.
- LOCAL, specifying that the vector from the camera to each vertex should be calculated when computing specular highlights.

The default value is INFINITE.

When set to *viewpoint* = INFINITE, a constant vector is used for the direction from the camera (viewpoint) to a vertex. When *viewpoint* = LOCAL, the result is more realistic because the actual vector from the camera to each vertex is calculated. However, overall performance is decreased.

materialShininess

A Float specifying the degree to which specular reflection is focused. The higher the *materialShininess* argument, the more focused the specular highlight. Possible values are $0.0 \leq \textit{polygonOffsetConstant} \leq 128.0$. The default value is 105.0.

lights

A LightArray object of length 8.

ambientColor

A String specifying the light applied evenly to the entire scene independent of any individual light. The initial value is 20% gray. A list of valid color strings is in the *colors* map in the *session* object.

17.13 LoadDisplayOptions object

The LoadDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.loads=ON
```

The LoadDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.loadOptions  
session.viewports[name].layers[name].assemblyDisplay.loadOptions
```

17.13.1 setValues(...)

This method modifies the LoadDisplayOptions object.

Arguments**Required arguments**

None.

Optional arguments

concentratedForce

A Boolean specifying whether concentrated force symbols are shown. The default value is ON.

moment

A Boolean specifying whether moment symbols are shown. The default value is ON.

pressure

A Boolean specifying whether pressure symbols are shown. The default value is ON.

pipePressure

A Boolean specifying whether pipe pressure symbols are shown. The default value is ON.

bodyForce

A Boolean specifying whether body force symbols are shown. The default value is ON.

lineLoad

A Boolean specifying whether line load symbols are shown. The default value is ON.

gravity

A Boolean specifying whether gravity symbols are shown. The default value is ON.

boltLoad

A Boolean specifying whether bolt load symbols are shown. The default value is ON.

pegLoad

A Boolean specifying whether PEG load symbols are shown. The default value is ON.

connectorForce

A Boolean specifying whether connector force symbols are shown. The default value is ON.

connectorMoment

A Boolean specifying whether connector moment symbols are shown. The default value is ON.

inertiaRelief

A Boolean specifying whether inertia relief symbols are shown. The default value is ON.

rotationalInertiaLoad

A Boolean specifying whether rotational inertia load symbols are shown. The default value is ON.

coriolisForce

A Boolean specifying whether coriolis force symbols are shown. The default value is ON.

bodyHeatFlux

A Boolean specifying whether body heat flux symbols are shown. The default value is ON.

surfaceHeatFlux

A Boolean specifying whether surface heat flux symbols are shown. The default value is ON.

DISPLAY OPTIONS COMMANDS

concentratedHeatFlux

A Boolean specifying whether concentrated heat flux symbols are shown. The default value is ON.

concentratedPoreFluid

A Boolean specifying whether concentrated pore fluid symbols are shown. The default value is ON.

surfacePoreFluid

A Boolean specifying whether surface pore fluid symbols are shown. The default value is ON.

hydroFluidFlow

A Boolean specifying whether hydro fluid flow symbols are shown. The default value is ON.

concentratedCharge

A Boolean specifying whether concentrated charge symbols are shown. The default value is ON.

concentratedCurrent

A Boolean specifying whether concentrated current symbols are shown. The default value is ON.

surfaceCharge

A Boolean specifying whether surface charge symbols are shown. The default value is ON.

surfaceCurrent

A Boolean specifying whether surface current symbols are shown. The default value is ON.

bodyCharge

A Boolean specifying whether body charge symbols are shown. The default value is ON.

bodyCurrent

A Boolean specifying whether body current symbols are shown. The default value is ON.

inwardVolAccel

A Boolean specifying whether inward volume acceleration symbols are shown. The default value is ON.

bodyConcentrationFlux

A Boolean specifying whether body concentration flux symbols are shown. The default value is ON.

surfaceConcentrationFlux

A Boolean specifying whether surface concentration flux symbols are shown. The default value is ON.

concentratedConcentrationFlux

A Boolean specifying whether concentrated concentration flux symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.13.2 Members

The LoadDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.14 MeshDisplayOptions object

The MeshDisplayOptions object stores settings that specify how the assembly is displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.mesh=ON
```

The MeshDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.meshOptions  
session.viewports[name].layers[name].assemblyDisplay.meshOptions  
session.viewports[name].layers[name].partDisplay.meshOptions  
session.viewports[name].partDisplay.meshOptions
```

17.14.1 setValues(...)

This method modifies the MeshDisplayOptions object.

Arguments**Required arguments**

None.

Optional arguments

nodeLabels

A Boolean specifying whether node labels are shown. The default value is OFF.

elementLabels

A Boolean specifying whether element labels are shown. The default value is OFF.

meshVisibleEdges

A SymbolicConstant specifying how the mesh's edges are drawn. Possible values are:

- ALL, specifying that all edges are shown.

DISPLAY OPTIONS COMMANDS

- EXTERIOR, specifying that only exterior edges are shown.
- FEATURE, specifying that edges are shown based on *featureAngle*.
- FREE, specifying that only free edges are shown.
- NONE, specifying that no display edges are shown.

The default value is EXTERIOR.

featureAngle

A Float specifying the angle in degrees to be used in calculating a feature edge plot. Possible values are $0 \leq \text{featureAngle} \leq 90$. The default value is 20.0.

meshEdgesInShaded

A Boolean specifying whether mesh edges are displayed in shaded mode. The default value is ON.

meshTechnique

A Boolean specifying whether the regions of the assembly will be color coded based on the meshing technique assigned to the regions. This argument is ignored in partDisplay. The default value is OFF.

seeds

A Boolean specifying whether seeds are shown. This argument is ignored in partDisplay. The default value is OFF.

Return value

None

Exceptions

RangeError.

17.14.2 Members

The MeshDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.15 OptimizationTaskDisplayOptions object

The OptimizationTaskDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

session.viewports[name].assemblyDisplay.optimizationTasks=ON

The OptimizationTaskDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.optimizationTaskOptions
session.viewports[name].layers[name].assemblyDisplay\
.optimizationTaskOptions
```

17.15.1 setValues(...)

This method modifies the OptimizationTaskDisplayOptions object.

Arguments**Required arguments**

None.

Optional arguments

topologyTask

A Boolean specifying whether topology task symbols are shown. The default value is ON.

shapeTask

A Boolean specifying whether shape task symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.15.2 Members

The OptimizationTaskDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.16 PartDisplayOptions object

The PartDisplayOptions object stores settings that specify how parts are to be displayed in a particular viewport. The PartDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].layers[name].partDisplay
```

DISPLAY OPTIONS COMMANDS

```
import part
session.viewports[name].partDisplay
```

17.16.1 setValues(...)

This method modifies the PartDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments

renderStyle

A SymbolicConstant specifying how the image in the viewport is rendered. Possible values are WIREFRAME, HIDDEN, and SHADED. The default value is WIREFRAME.

visibleDisplayGroups

A sequence of DisplayGroup objects specifying the DisplayGroups visible in the viewport. Currently the sequence can contain a maximum of one DisplayGroup object. The default value is an empty sequence.

engineeringFeatures

A Boolean specifying whether engineering features are shown. The default value is OFF.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The beamScaleFactor must be greater than zero. The default value is 1.0.

Return value

None

Exceptions

RangeError.

17.16.2 Members

The PartDisplayOptions object can have the following members:

engineeringFeatures

A Boolean specifying whether engineering features are shown. The default value is OFF.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The beamScaleFactor must be greater than zero. The default value is 1.0.

mesh

A Boolean specifying whether the mesh should be displayed.

renderStyle

A SymbolicConstant specifying how the image in the viewport is rendered. Possible values are WIREFRAME, HIDDEN, and SHADED. The default value is WIREFRAME.

displayGroup

A DisplayGroup object specifying the current display group and referring to an object in the displayGroups member of Session .

displayGroupInstances

A repository of DisplayGroupInstance objects.

engineeringFeatureOptions

An EngineeringFeatureDisplayOptions object.

geometryOptions

A GeometryDisplayOptions object.

meshOptions

A MeshDisplayOptions object.

17.17 StopConditionDisplayOptions object

The StopConditionDisplayOptions object stores settings that specify how assemblies are to be displayed in a particular viewport when

```
session.viewports[name].assemblyDisplay.stopConditions=ON
```

The StopConditionDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.stopConditionOptions  
session.viewports[name].layers[name].assemblyDisplay\  
.stopConditionOptions
```

17.17.1 setValues(...)

This method modifies the StopConditionDisplayOptions object.

Arguments

Required arguments

None.

Optional argument

localStopCondition

A Boolean specifying whether local stop condition symbols are shown. The default value is ON.

Return value

None

Exceptions

RangeError.

17.17.2 Members

The StopConditionDisplayOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

17.18 SymbolDisplayOptions object

The SymbolDisplayOptions object stores settings that specify how the assembly is displayed in a particular viewport. The SymbolDisplayOptions object has no constructor. When you create a new viewport, the settings are copied from the current viewport.

Access

```
session.viewports[name].assemblyDisplay.symbolOptions  
session.viewports[name].layers[name].assemblyDisplay.symbolOptions
```

17.18.1 setValues(...)

This method modifies the SymbolDisplayOptions object.

Arguments

Required arguments

None.

Optional arguments*otherSymbolSize*

An Int specifying the size of the scalar attribute symbols. Possible values are $1 \leq \text{scalarSymbolSize} \leq 30$. The default value is 12.

arrowSymbolSize

An Int specifying the size of the vector and tensor attribute symbols. Possible values are $1 \leq \text{vectorSymbolSize} \leq 30$. The default value is 12.

faceSymbolDensity

An Int specifying the relative density of the attribute symbols drawn on geometric faces. Possible values are $1 \leq \text{faceSymbolDensity} \leq 10$. The default value is 5.

edgeSymbolDensity

An Int specifying the relative density of the attribute symbols drawn on geometric edges. Possible values are $1 \leq \text{edgeSymbolDensity} \leq 10$. The default value is 5.

meshSymbolFraction

A Float specifying the fraction of the attribute symbols drawn on orphan mesh regions. Possible values are $0.0 \leq \text{meshSymbolFraction} \leq 1.0$. The default value is 1.0.

showFields

A Boolean specifying whether symbols should be scaled based on analytical field value. The default value is ON.

Return value

None

Exceptions

None.

17.18.2 Members

The `SymbolDisplayOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

17.19 ViewportAnnotationOptions object

The `ViewportAnnotationOptions` object stores settings that control how annotations are rendered in a particular viewport. `ViewportAnnotationOptions` objects are accessed in one of two ways:

- The default viewport annotations. These settings are used as defaults when other `viewportAnnotationOptions` members are created and can be set to customize user preferences.
- The viewport annotations associated with a particular viewport.

The ViewportAnnotationOptions object has no constructor; Abaqus creates the *defaultViewportAnnotationOptions* member when a session is started. When a new viewport is created, the settings are copied from the current viewport.

Access

```
session.defaultViewportAnnotationOptions  
session.viewports[name].viewportAnnotationOptions
```

17.19.1 setValues(...)

This method modifies the ViewportAnnotationOptions object.

Arguments

Required arguments

None.

Optional arguments

triad

A Boolean specifying whether the view orientation triad is shown. The default value is ON.

triadPosition

A pair of Ints specifying the position of the view orientation triad as a percentage of the viewport size. Possible values are $(0, 0) \leq \text{triadPosition} \leq (100, 100)$. The default value is (6, 12).

triadColor

A String specifying the color of the view orientation triad. Possible values are any valid color. The default value is "White".

triadLabels

A SymbolicConstant specifying how the view orientation triad is labeled. Possible values are:

- NUMBERS, specifying the label axes 1, 2, 3.
- LETTERS, specifying the label axes X, Y, Z.

The default value is NUMBERS.

triadFont

A String specifying the font of the view orientation triad labels. Possible values are any valid font. The default value is "-*-verdana-bold-r-normal--120-*".

triadSize

An Int specifying the length of each triad axis as a percentage of the viewport size. Possible values are $1 \leq \text{legendDecimalPlaces} \leq 50$. The default value is 4.

legend

A Boolean specifying whether the legend is shown. The default value is ON.

legendMinMax

A Boolean specifying whether the minimum and maximum values for X - Y and contour plots are shown. The default value is OFF.

legendBox

A Boolean specifying whether the box around the legend is shown. The default value is ON.

legendDecimalPlaces

An Int specifying the number of decimal places to display in the legend. Possible values are $0 \leq \text{legendDecimalPlaces} \leq 9$. The default value is 3.

legendPosition

A pair of Ints specifying the position of the legend as a percentage of the viewport size. Possible values are $(0, 0) \leq \text{legendPosition} \leq (100, 100)$. The default value is (2, 98).

legendFont

A String specifying the font of the legend labels. Possible values are any valid font. The default value is "-*-verdana-medium-r-*-*-120-*-*-*-iso8859-1".

legendTextColor

A String specifying the color of the legend. Possible values are any valid color. The default value is "White".

legendBackgroundStyle

A SymbolicConstant specifying the legend background style. Possible values are MATCH, TRANSPARENT, and OTHER. The default value is TRANSPARENT.

The default color when OTHER is specified is the background color (black).

legendBackgroundColor

A String specifying the color of the legend background. Possible values are any valid color. The initial value matches the viewport background.

title

A Boolean specifying whether the title block is shown. The default value is ON.

titleBox

A Boolean specifying whether the box around the title block is shown. The default value is OFF.

titlePosition

A pair of Ints specifying the position of the title block as a percentage of the viewport size. Possible values are $(0, 0) \leq \text{titlePosition} \leq (100, 100)$. The default value is (13, 20).

titleFont

A String specifying the font of the title. Possible values are any valid font. The default value is "-*-verdana-medium-r-*-*-120-*-*-*-iso8859-1".

titleTextColor

A String specifying the color of the title. Possible values are any valid color. The default value is "White".

DISPLAY OPTIONS COMMANDS

titleBackgroundColor

A SymbolicConstant specifying the title block background style. Possible values are MATCH, TRANSPARENT, and OTHER. The default value is TRANSPARENT.

The default color when OTHER is specified is the background color (black).

titleBackgroundColor

A String specifying the color of the title block background. Possible values are any valid color. The initial value matches the viewport background.

state

A Boolean specifying whether the state block is shown. The default value is ON.

stateBox

A Boolean specifying whether the box around the state block is shown. The default value is OFF.

statePosition

A pair of Ints specifying the position of the state block as a percentage of the viewport size. Possible values are $(0, 0) \leq statePosition \leq (100, 100)$. The default value is (13, 12).

stateFont

A String specifying the font of the state label. Possible values are any valid font. The default value is "-*-verdana-medium-r-*-*120-*-*-*iso8859-1".

stateTextColor

A String specifying the color of the state block label. Possible values are any valid color. The default value is "White".

stateBackgroundColor

A SymbolicConstant specifying the state block background style. Possible values are MATCH, TRANSPARENT, and OTHER. The default value is TRANSPARENT.

The default color when OTHER is specified is the background color (black).

stateBackgroundColor

A String specifying the color of the state block background. Possible values are any valid color. The initial value matches the viewport background.

compass

A Boolean specifying whether the 3D Compass is shown. The default value is ON.

compassScale

A Float specifying the relative size of the 3D Compass in the viewport. Possible values are $0.5 \leq compassScale \leq 2.0$. The default value is 0.8.

compassPrivilegedPlane

A SymbolicConstant specifying the plane that will be used for the base of the 3D Compass. Possible values are XYPLANE, XZPLANE, and YZPLANE. The default value is XZPLANE.

Return value

None

Exceptions

RangeError.

17.19.2 Members

The ViewportAnnotationOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

18. Edit mesh commands

Edit mesh commands are used to edit an orphan mesh part or part instance or an Abaqus native mesh.

18.1 Assembly object

The following commands operate on Assembly objects. For more information about the Assembly object, see “Assembly object,” Section 6.1.

Access

```
import meshEdit
```

18.1.1 collapseMeshEdge(...)

This method collapses an edge of a quadrilateral or triangular element of a part instance.

Arguments

Required arguments

edge

A single MeshEdge object specifying the element edge to collapse.

collapseMethod

A SymbolicConstant specifying the method used to collapse the edge. Possible values are FORWARD, REVERSE, and AVERAGE.

Optional arguments

None.

Return value

None

Exceptions

None.

18.1.2 combineElement(...)

This method combines two triangular elements of a part instance.

Arguments

Required argument

elements

A sequence of triangular MeshElement objects specifying the elements to combine.

Optional arguments

None.

Return value

None

Exceptions

None.

18.1.3 deleteElement(...)

This method deletes the given elements from a part instance. The elements must have been generated using the bottom-up meshing technique.

Arguments

Required argument

elements

A sequence of MeshElement objects or a Set object containing elements.

Optional argument

deleteUnreferencedNodes

A Boolean specifying whether to delete all those associated nodes that become unreferenced after the given elements are deleted. The default value is OFF.

Return value

None

Exceptions

None.

18.1.4 projectNode(...)

This method projects the given nodes of a part instance onto a mesh entity, geometric entity, or a datum object.

Arguments**Required arguments***nodes*

A sequence of MeshNode objects to be projected.

projectionReference

An object specifying the target for the node projection operation. The *projectionReference* can be any one of the following objects: MeshNode , MeshEdge , MeshFace , Vertex , Edge , Face , DatumPoint , DatumAxis , or DatumPlane .

Optional arguments

None.

Return value

None

Exceptions

None.

18.1.5 editNode(...)

This method changes the coordinates of the given nodes on a part instance.

Arguments**Required argument***nodes*

A sequence of MeshNode objects or a Set object containing nodes.

Optional arguments*coordinate1*

A Float specifying the value of the first coordinate. If *coordinate1* and *offset1* are unspecified, the existing value does not change.

coordinate2

A Float specifying the value of the second coordinate. If *coordinate2* and *offset2* are unspecified, the existing value does not change.

coordinate3

A Float specifying the value of the third coordinate. If *coordinate3* and *offset3* are unspecified, the existing value does not change.

coordinates

A sequence of three-dimensional coordinate tuples specifying the coordinates for each of the given nodes. When specified, the number of coordinate tuples must match the number of given

nodes, and be ordered to correspond to the given nodes in *ascending order* according to index. Furthermore, *coordinate1*, *coordinate2*, *coordinate3*, *offset1*, *offset2*, or *offset3* may not be specified.

offset1

A Float specifying an offset to apply to the value of the first coordinate of the specified nodes.

offset2

A Float specifying an offset to apply to the value of the second coordinate of the specified nodes.

offset3

A Float specifying an offset to apply to the value of the third coordinate of the specified nodes.

localCsys

A DatumCsys object specifying the local coordinate system. If unspecified, the global coordinate system will be used.

projectToGeometry

A Boolean specifying whether to project nodes back to their original geometry. For example, if a node is on a face, this method first positions the node at the new location and then projects it back to the original face. The default value is ON.

Return value

None

Exceptions

A coordinate and an offset may not both be specified for the same coordinate component.

18.1.6 mergeNodes(...)

Merge the nodes of a part instance. The nodes must have been generated using the bottom-up meshing technique.

Arguments

Required argument

nodes

A sequence of MeshNode objects specifying the nodes to merge.

Optional arguments

tolerance

A Float specifying the maximum distance between nodes that will be merged to a single node. The location of the new node is the average position of the merged nodes. The default value is 10^{-6} .

removeDuplicateElements

A Boolean specifying whether elements with the same connectivity after the merge will merged into a single element. The default value is True.

Return value

None

Exceptions

None.

18.1.7 mergeNodes(...)

Merge two nodes of a part instance. At least one of the two nodes must have been generated using the bottom-up meshing technique.

Arguments**Required arguments**

node1

A MeshNode object specifying the first node to merge.

node2

A MeshNode object specifying the second node to merge.

Optional argument

removeDuplicateElements

A Boolean specifying whether elements with the same connectivity after the merge will merged into a single element. The default value is True.

Return value

None

Exceptions

None.

18.1.8 splitElement(...)

This method splits quadrilateral elements into triangular elements.

Arguments**Required argument**

elements

A sequence of quadrilateral MeshElement objects specifying the elements to split. Each quadrilateral element is split into two triangular elements by the shorter diagonal.

Optional arguments

None.

Return value

None

Exceptions

None.

18.1.9 splitMeshEdge(...)

This method splits an edge of a quadrilateral or triangular element of a part instance.

Arguments

Required argument

edge

A single MeshEdge object specifying the element edge to split.

Optional argument

parameter

A Float specifying the normalized distance along the *edge* at which to split. Possible values are $0.0 < \textit{parameter} < 1.0$. The default value is 0.5.

Return value

None

Exceptions

None.

18.1.10 swapMeshEdge(...)

This method swaps the diagonal of two adjacent triangular elements of a part instance.

Arguments

Required argument

edge

A single MeshEdge object specifying the element edge to swap.

Optional arguments

None.

Return value

None

Exceptions

None.

18.1.11 generateMeshByOffset(...)

This method generates a solid or shell mesh from an orphan mesh surface by generating layers of elements that propagate out normal to the surface boundary.

Arguments**Required arguments***region*

A Region object specifying the domain to be offset.

meshType

A Symbolic Constant specifying the type of mesh to be generated. Possible values are SOLID or SHELL.

totalThickness

A Float specifying the total thickness of the solid layers. This argument applies only when *meshType*=SOLID.

distanceBetweenLayers

A Float specifying the distance between shell layers. This argument applies only when *meshType*=SHELL.

numLayers

An Int specifying the number of element layers to be generated.

Optional arguments*offsetDirection*

A Symbolic Constant specifying the direction of the offset. This argument is required only when the given region relates to a shell mesh. Possible values are OUTWARD, INWARD, and BOTH. The default value is OUTWARD.

initialOffset

A Float specifying the magnitude of the initial offset. The default value is zero.

shareNodes

Boolean specifying whether the first layer of nodes should be shared with nodes on the base surface. The default value is False.

EDIT MESH COMMANDS

deleteBaseElements

A Boolean specifying whether to delete the shell elements after the offset layers are generated. The default value is False. This argument applies only when *meshType*=SHELL.

constantThicknessCorners

A Boolean specifying whether to use element-based thickness or nodal-based thickness. The default value is False.

extendElementSets

A Boolean specifying whether existing element sets that include base elements will be extended to include corresponding offset elements. The default value is False.

Return value

None

Exceptions

None.

18.1.12 redoMeshEdit()

This method executes the edit mesh or the bottom-up meshing operation most recently undone by the **undoMeshEdit** method on an assembly. A redo action must be currently available for the assembly. This implies that the user must have executed the **undoMeshEdit** method on the assembly and that the user has not subsequently executed any further edit mesh commands on the assembly. It also implies that the user provided a sufficient cache allowance to store the undo operation.

Arguments

None.

Return value

None

Exceptions

None.

18.1.13 undoMeshEdit()

This method undoes the most recent edit mesh or the bottom-up meshing operation on an assembly and restores the mesh on the affected part instance to its previous state. An edit mesh undo action must be available for the assembly. This implies that prior to executing an edit mesh command on the assembly, the user enabled edit mesh undo with a sufficient cache allowance to store the edit mesh operation.

Arguments

None.

Return value

None

Exceptions

None.

18.2 Part object

The following commands operate on Part objects. For more information about the Part object, see “Part object,” Section 37.1.

Access

```
import meshEdit
```

18.2.1 adjustMidsideNode(...)

This method is used to adjust the midside node of second-order elements of an orphan mesh part.

Arguments**Required arguments**

cornerNodes

A sequence of Node objects specifying the nodes towards which connected midside nodes will be biased.

parameter

A Float specifying the normalized distance along the edge of the midside nodes. Possible values are $0.0 \leq \text{parameter} \leq 1.0$, where 0.0 specifies the position of the corner node. The default value is 0.5.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.2 cleanMesh(...)

This method is used to collapse short element edges and delete collapsed elements, or grow short element edges, on an orphan mesh part composed of linear elements.

Arguments**Required argument**

mergeTolerance

A Float specifying the edge length tolerance. During the operation, edges shorter than the given tolerance will be collapsed, or grown to the specified length.

Optional arguments

growEdges

A Boolean specifying whether short element edges will be grown to the specified tolerance. Default is False, meaning short edges will be collapsed.

elements

The elements to consider as the domain for the operation. By default all elements on the part are considered. The elements may be given as a MeshElementArray , a list of MeshElement objects, a Set , or a list of Set objects.

refEdge

A MeshEdge specifying a reference edge to indicate a topological direction in a structured mesh that will limit which edges within the element domain are considered. That is, only edges that are found to be topologically parallel to the given reference edge will be considered by the operation. By default all edges of the element domain are considered, unless *thicknessDir* is specified, in which case the operation will attempt to determine the topological edges from the thickness direction.

thicknessDir

A tuple of two or three Floats indicating a vector along which element edge lengths will be measured.

moveLayers

A Boolean indicating whether element edges will be all grown in the direction of the thickness vector specified by *thicknessDir*. This argument is ignored unless *growEdges* is True and *thicknessDir* is provided. When this argument is True the growth of any given element edge will no longer be constrained by short edges on neighboring elements, but elements could move from their original positions in cases where there are multiple adjacent layers of thin elements. The default value is False.

Return value

None

Exceptions

None.

18.2.3 collapseMeshEdge(...)

This method collapses an edge of a quadrilateral or triangular element of an orphan mesh part or part instance.

Arguments**Required arguments**

edge

A single MeshEdge object specifying the element edge to collapse.

collapseMethod

A SymbolicConstant specifying the method used to collapse the edge. Possible values are FORWARD, REVERSE, and AVERAGE.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.4 combineElement(...)

This method combines two triangular elements of an orphan mesh part or an Abaqus native mesh.

Arguments**Required argument**

elements

A sequence of triangular MeshElement objects specifying the elements to combine.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.5 **convertSolidMeshToShell()**

This method removes all solid elements from an orphan mesh part and creates triangular or quadrilateral shell elements along their outer faces.

Arguments

None.

Return value

None

Exceptions

None.

18.2.6 **deleteElement(...)**

This method deletes the given elements from an orphan mesh part or an Abaqus native mesh. If the elements belong to an Abaqus native mesh then the elements must have been generated using the bottom-up meshing technique.

Arguments**Required argument**

elements

A sequence of MeshElement objects or a Set object containing elements.

Optional argument

deleteUnreferencedNodes

A Boolean specifying whether to delete all those associated nodes that become unreferenced after the given elements are deleted. The default value is OFF.

Return value

None

Exceptions

None.

18.2.7 deleteNode(...)

This method deletes the given nodes from an orphan mesh part.

Arguments**Required argument**

nodes

A sequence of MeshNode objects or a Set object containing nodes.

Optional argument

deleteUnreferencedNodes

A Boolean specifying whether to delete all those associated nodes that become unreferenced after the given nodes and the connected elements are deleted. The default value is OFF.

Return value

None

Exceptions

None.

18.2.8 editNode(...)

This method changes the coordinates of the given nodes on an orphan mesh part or on an Abaqus native mesh.

Arguments**Required argument**

nodes

A sequence of MeshNode objects or a Set object containing nodes.

Optional arguments

coordinate1

A Float specifying the value of the first coordinate. If *coordinate1* and *offset1* are unspecified, the existing value does not change.

EDIT MESH COMMANDS

coordinate2

A Float specifying the value of the second coordinate. If *coordinate2* and *offset2* are unspecified, the existing value does not change.

coordinate3

A Float specifying the value of the third coordinate. If *coordinate3* and *offset3* are unspecified, the existing value does not change.

coordinates

A sequence of three-dimensional coordinate tuples specifying the coordinates for each of the given nodes. When specified, the number of coordinate tuples must match the number of given nodes, and be ordered to correspond to the given nodes in *ascending order* according to index. Furthermore, *coordinate1*, *coordinate2*, *coordinate3*, *offset1*, *offset2*, or *offset3* may not be specified.

offset1

A Float specifying an offset to apply to the value of the first coordinate of the specified nodes.

offset2

A Float specifying an offset to apply to the value of the second coordinate of the specified nodes.

offset3

A Float specifying an offset to apply to the value of the third coordinate of the specified nodes.

localCsys

A DatumCsys object specifying the local coordinate system. If unspecified, the global coordinate system will be used.

projectToGeometry

A Boolean specifying whether to project nodes back to their original geometry. For example, if a node is on a face, this method first positions the node at the new location and then projects it back to the original face. The default value is ON.

Return value

None

Exceptions

A coordinate and an offset may not both be specified for the same coordinate component.

18.2.9 projectNode(...)

This method projects the given nodes onto a mesh entity, geometric entity, or a datum object. The nodes may belong to an orphan mesh part or to an Abaqus native mesh.

Arguments**Required arguments***nodes*

A sequence of MeshNode objects to be projected.

projectionReference

An object specifying the target for the node projection operation. The *projectionReference* can be any one of the following objects: MeshNode , MeshEdge , MeshFace , Vertex , Edge , Face , DatumPoint , DatumAxis , or DatumPlane .

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.10 generateMesh(...)

This method generates a new mesh on an orphan mesh part based on the original mesh.

Arguments**Required arguments**

None.

Optional argument*elemShape*

A SymbolicConstant specifying the element shape to be used for meshing. Possible values are:

TRI

Refine a planar triangular mesh and replace it with a new one. If no element sizes are attached, the new mesh will be governed by the sizes of the boundary edges in the old mesh.

TET

Create a tetrahedral mesh from a closed shell of triangular elements.

Return value

None

Exceptions

None.

18.2.11 generateMeshByOffset(...)

This method generates a solid or shell mesh from an orphan mesh surface by generating layers of elements that propagate out normal to the surface boundary.

Arguments**Required arguments**

region

A Region object specifying the domain to be offset.

meshType

A Symbolic Constant specifying the type of mesh to be generated. Possible values are SOLID or SHELL.

totalThickness

A Float specifying the total thickness of the solid layers. This argument applies only when *meshType*=SOLID.

distanceBetweenLayers

A Float specifying the distance between shell layers. This argument applies only when *meshType*=SHELL.

numLayers

An Int specifying the number of element layers to be generated.

Optional arguments

offsetDirection

A Symbolic Constant specifying the direction of the offset. This argument is required only when the given region relates to a shell mesh. Possible values are OUTWARD, INWARD, and BOTH. The default value is OUTWARD.

initialOffset

A Float specifying the magnitude of the initial offset. The default value is zero.

shareNodes

Boolean specifying whether the first layer of nodes should be shared with nodes on the base surface. The default value is False.

deleteBaseElements

A Boolean specifying whether to delete the shell elements after the offset layers are generated. The default value is False. This argument applies only when *meshType*=SHELL.

constantThicknessCorners

A Boolean specifying whether to use element-based thickness or nodal-based thickness. The default value is False.

extendElementSets

A Boolean specifying whether existing element sets that include base elements will be extended to include corresponding offset elements. The default value is False.

Return value

None

Exceptions

None.

18.2.12 mergeNodes(...)

Merge the nodes of an orphan mesh part, or nodes that were generated using the bottom-up meshing technique.

Arguments**Required argument***nodes*

A sequence of Node objects specifying the nodes to merge.

Optional arguments*tolerance*

A Float specifying the maximum distance between nodes that will be merged to a single node. The location of the new node is the average position of the merged nodes. The default value is 10^{-6} .

removeDuplicateElements

A Boolean specifying whether elements with the same connectivity after the merge will be merged into a single element. The default value is True.

keepHighLabels

A Boolean specifying which node labels will remain after nodes are merged. If True then the highest node labels are kept; when False the lowest node labels are kept. The default value is False. This argument applies only to merging of orphan mesh nodes.

Return value

None

Exceptions

None.

18.2.13 mergeElement(...)

Merge a selection of elements arranged in layers on an orphan mesh part into a single layer.

Arguments

Required arguments

edge

A MeshEdge of one of the specified elements that serves as a reference edge to indicate the topological direction for merging elements. All specified elements must be reachable by topological navigation from this element edge, and the topological direction must be unambiguous.

elements

A MeshElementArray , a list of MeshElement objects, a Set , or a list of Set objects containing the elements to be included in the merge operation.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.14 mergeNodes(...)

Merge two nodes of an orphan mesh part or an Abaqus native mesh. If the nodes belong to an Abaqus native mesh then at least one of the two nodes must have been generated using the bottom-up meshing technique.

Arguments

Required arguments

node1

A MeshNode object specifying the first node to merge.

node2

A MeshNode object specifying the second node to merge.

Optional arguments*removeDuplicateElements*

A Boolean specifying whether elements with the same connectivity after the merge will be merged into a single element. The default value is True.

keepHighLabels

A Boolean specifying which node label will remain after nodes are merged. If True then the higher node label is kept; when False the lower node label is kept. The default value is False. This argument applies only to merging of orphan mesh nodes.

Return value

None

Exceptions

None.

18.2.15 orientElements(...)

This method orients the stack direction of elements in a continuum shell or gasket mesh.

Arguments**Required arguments***pickedElements*

A sequence of MeshElement objects specifying the elements to orient.

referenceRegion

A MeshFace object specifying a reference top face that indicates the desired orientation.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.16 removeElementSize()

This method removes the global element size from an orphan mesh part.

Arguments

None.

Return value

None

Exceptions

None.

18.2.17 **renumberElement(...)**

This method assigns new labels to orphan mesh elements.

Arguments

Required arguments

None.

Optional arguments

Either *startLabel* and *increment*, or *offset*, or *labels* must be specified.

elements

A MeshElementArray or a tuple or list of MeshElement objects, or a Set containing elements to be renumbered. If unspecified, all elements in the part will be renumbered.

startLabel

A positive Int specifying the new label for the first element in *elements*.

increment

A positive Int specifying the increment used for computation of new labels for all consecutive elements in *elements*.

offset

An Int by which existing labels of the specified elements will be offset.

labels

A list of labels for the specified elements. The length of this list must match the number of specified elements.

Return value

None

Exceptions

Renumbering is attempted on a native part:

Error: Renumbering can be applied to orphan mesh parts only
 Renumbering data is specified incorrectly:
 Error: Either *startLabel* and *increment* or *offset* must be specified
 Renumbering will result in invalid labels:
 Error: Specified data will result in invalid labels
 Renumbering will result in conflicting labels:
 Error: Specified data will result in conflicting labels

18.2.18 **renumberNode(...)**

This method assigns new labels to orphan mesh nodes.

Arguments

Required arguments

None.

Optional arguments

Either *startLabel* and *increment*, or *offset*, or *labels* must be specified.

nodes

A `MeshNodeArray` or a tuple or list of `MeshNode` objects, or a `Set` containing nodes to be renumbered. If unspecified, all nodes in the part will be renumbered.

startLabel

A positive `Int` specifying the new label for the first node in *nodes*.

increment

A positive `Int` specifying the increment used for computation of new labels for all consecutive nodes in *nodes*.

offset

An `Int` by which existing labels of the specified nodes will be offset.

labels

A list of labels for the specified nodes. The length of this list must match the number of specified nodes.

Return value

None

Exceptions

Renumbering is attempted on a native part:
 Error: Renumbering can be applied to orphan mesh parts only
 Renumbering data is specified incorrectly:

Error: Either startLabel and increment or offset must be specified

Renumbering will result in invalid labels:

Error: Specified data will result in invalid labels

Renumbering will result in conflicting labels:

Error: Specified data will result in conflicting labels

18.2.19 **setElementSize(...)**

This method sets the global element size for an orphan mesh part.

Arguments

Required argument

size

A Float specifying the desired element size.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.20 **splitElement(...)**

This method splits quadrilateral elements of an orphan mesh part or a Abaqus native mesh into triangular elements.

Arguments

Required argument

elements

A sequence of quadrilateral MeshElement objects specifying the elements to split. Each quadrilateral element is split into two triangular elements by the shorter diagonal.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.21 splitMeshEdge(...)

This method splits an edge of a quadrilateral or triangular element of an orphan mesh part or an Abaqus native mesh.

Arguments**Required argument**

edge

A single MeshEdge object specifying the element edge to split.

Optional argument

parameter

A Float specifying the normalized distance along *edge* at which to split. Possible values are $0.0 < \textit{parameter} < 1.0$. The default value is 0.5.

Return value

None

Exceptions

None.

18.2.22 subdivideElement(...)

Subdivide a selection of elements on an orphan mesh part in one or more directions.

Arguments**Required arguments**

None.

Optional arguments

elements

A MeshElementArray , a list of MeshElement objects, a Set , or a list of Set objects containing the elements to be subdivided. By default all the elements of the part are subdivided.

divisionNumber

An Int specifying the number of resulting elements for each input element in each direction of the subdivision. If *face* or *edge* are not specified, elements will be subdivided according to this number in all possible directions. Must be greater than one. Default is 2.

face

A MeshFace object that serves as a reference for indicating two topological directions for the subdivision operation. Must be a face of one of the specified elements, and all specified elements must be reachable by topological navigation from this element face. May not be combined with *edge*.

edge

A MeshEdge object that serves as a reference for indicating a single topological direction for the subdivision operation. Must be an edge of one of the specified elements, and all specified elements must be reachable by topological navigation from this element edge. May not be combined with *face*.

Return value

None

Exceptions

None.

18.2.23 swapMeshEdge(...)

This method swaps the diagonal of two adjacent triangular elements of an orphan mesh part or an Abaqus native mesh.

Arguments

Required argument

edge

A single MeshEdge object specifying the element edge to swap.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.24 wrapMesh(...)

This method wraps a planar orphan mesh part about the Z-axis.

Arguments**Required argument***radius*

A Float specifying the radius of the cylinder about which the part is to be wrapped. The wrapping procedure will relocate a node at point (x, y) on the planar mesh to (x, θ, z) , where r is the specified radius, $\theta = \frac{x}{r}$, and $z=y$.

Optional arguments

None.

Return value

None

Exceptions

None.

18.2.25 redoMeshEdit()

This method executes the edit mesh or the bottom-up meshing operation most recently undone by the **undoMeshEdit** method on an part. A redo action must be currently available for the part. This implies that the user must have executed the **undoMeshEdit** method on the part and that the user has not subsequently executed any further edit mesh commands on the assembly. It also implies that the user provided a sufficient cache allowance to store the undo operation.

Arguments

None.

Return value

None

Exceptions

None.

18.2.26 undoMeshEdit()

This method undoes the most recent edit mesh or the bottom-up meshing operation on a part and restores the mesh to its previous state. An edit mesh undo action must be available for the part. This implies that prior to executing an edit mesh command on the part, the user enabled edit mesh undo with a sufficient cache allowance to store the edit mesh operation.

Arguments

None.

Return value

None

Exceptions

None.

18.3 MeshEditOptions object

The MeshEditOptions object stores settings that specify the behavior when editing meshes on parts or part instances.

The MeshEditOptions object has no constructor. Abaqus creates the *MeshEditOptions* member when a session is started.

Access

`mdb.meshEditOptions`

18.3.1 setValues(...)

This method modifies the MeshEditOptions object.

Arguments

Required arguments

None.

Optional arguments

maxUndoCacheElements

A Float specifying the maximum allowable mesh edit undo cache size in millions of elements. If this value is set to at least the number of elements on a given part or part instance, at least one level of undo/redo capability is assured for subsequent mesh edit operations on that part or part instance. The default value is 0.0.

enableUndo

A Boolean specifying whether undo/redo of mesh edit operations will be enabled. If *enableUndo* =OFF any existing cache for undo/redo operations will be cleared for all parts and assemblies in all models. The default value is OFF.

_suspendUndo

A Boolean specifying the suspension of undo/redo for mesh edit operations. When undo/redo is suspended, undo/redo will not be available after subsequent mesh edit operations on a given part or part instances. Any pre-existing cache for mesh edit operations on other parts or assemblies in any model will be unaffected. The default value is OFF.

If you change the value of *enableUndo* to True, Abaqus sets *_suspendUndo* to False.

Return value

None

Exceptions

None.

18.3.2 Members

The MeshEditOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

19. Engineering Feature commands

A specific type of engineering feature object is designed for each type of engineering feature.

19.1 EngineeringFeature object

The EngineeringFeature object is a container for various specific engineering feature repositories. The EngineeringFeature object has no explicit constructor or methods.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures
import assembly
mdb.models[name].rootAssembly.engineeringFeatures
```

19.1.1 assignSeam(...)

This method creates a seam crack along an edge or a face.

Arguments

Required argument

regions

A sequence of Region objects specifying the domain of the seam crack. The Region objects must be faces or edges.

Optional arguments

None.

Return value

None

Exceptions

None.

19.1.2 deleteSeam(...)

This method deletes a seam crack.

Arguments

Required argument

regions

A sequence of Region objects specifying the domain of the seam crack. The Region objects must be faces or edges.

Optional arguments

None.

Return value

None

Exceptions

None.

19.1.3 Members

The EngineeringFeature object can have the following members:

inertias

A repository of Inertia objects.

cracks

A repository of Crack objects.

fasteners

A repository of Fastener objects.

springDashpots

A repository of SpringDashpot objects.

19.2 AssembledFastener object

The AssembledFastener object defines an assembled fastener.

The AssembledFastener object is derived from the Fastener object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.fasteners[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.fasteners[name]
```

19.2.1 AssembledFastener(...)

This method creates an AssembledFastener object. Although the constructor is available both for parts and for the assembly, AssembledFastener objects are currently supported only under the assembly.

Path

```
mdb.models[name].parts[name].engineeringFeatures.AssembledFastener  
mdb.models[name].rootAssembly.engineeringFeatures.AssembledFastener
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region of attachment points to which assembled fasteners are applied.

templateModel

A String specifying the name of the template model.

controlSet

A Region object specifying the template model control point set. The set must contain a single node or vertex.

templateSurfaces

A sequence of Strings specifying the names of the template model surfaces that are referenced by tie or coupling constraints.

assignedSurfaces

A sequence of Strings specifying the names of the main model surfaces that will be substituted for the template model constraint surfaces.

propertyPrefix

A String specifying the name of the property prefix string. This string will be prepended to every property name as it is copied to the main model from the template model.

Optional arguments

orientMethod

A SymbolicConstant specifying the method used to orient the virtual instances of the template model at each attachment point. Possible values are NORMALS and CSYS. The default value is NORMALS.

localCsys

None or a DatumCsys object specifying the local coordinate system. If *localCsys=None*, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

This argument applies only when *orientMethod*=CSYS.

scriptName

A String specifying the name of the property generation script. The default value is an empty string.

Return value

An AssembledFastener object.

Exceptions

None.

19.2.2 **setValues(...)**

This method modifies the AssembledFastener object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AssembledFastener** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.2.3 **Members**

The AssembledFastener object has members with the same names and descriptions as the arguments to the **AssembledFastener** method.

In addition, the AssembledFastener object has the following member:

suppressed

A Boolean specifying whether the fastener is suppressed or not. The default value is OFF.

19.3 ContourIntegral object

The ContourIntegral object defines contour integral objects on an region. Currently only assembly regions are supported.

The ContourIntegral object is derived from the Crack object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.cracks[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.cracks[name]
```

19.3.1 ContourIntegral(...)

This method creates a ContourIntegral object. Although the constructor is available both for parts and for the assembly, ContourIntegral objects are currently supported only under the assembly.

Path

```
mdb.models[name].parts[name].engineeringFeatures.ContourIntegral
mdb.models[name].rootAssembly.engineeringFeatures.ContourIntegral
```

Required arguments

name

A String specifying the repository key.

crackFront

A RegionArray object specifying the crack-front region to which the contour integral is applied. If the crack-front consists of a single region, a Region object may be specified instead of a sequence with a single item in it.

crackTip

A RegionArray object specifying the crack-tip region to which the contour integral is applied. If the crack-tip consists of a single region, a Region object may be specified instead of a sequence with a single item in it.

extensionDirectionMethod

A SymbolicConstant specifying how the virtual crack extension direction vectors are defined. Possible values are CRACK_NORMAL and Q_VECTORS.

Optional arguments*symmetric*

A Boolean specifying whether the crack is defined on a half model (about a symmetry plane) or whether it is defined on the whole model. The default value is OFF.

listOfRegions

A Boolean specifying whether the regions specified by *crackFront* and *crackTip* are specified using a single region or tuples of region objects. The default value is OFF.

crackFrontName

A String specifying the name of the crack-front region generated from the tuple of regions specifying the crack-front region. This argument is valid only when *listOfRegions* is ON. The default value is *name*+“Front.”

crackTipName

A String specifying the name of the crack-tip region generated from the tuple of regions specifying the crack-tip region. This parameter is valid only when *listOfRegions*=ON. The default value is *name*+“Tip.”

crackNormal

A sequence of sequences of Floats specifying the two points of the vector that describes the crack normal direction. Each point is defined by a tuple of two or three coordinates indicating its position. This argument is required only when *extensionDirectionMethod*=CRACK_NORMAL. The default value is an empty sequence.

qVectors

A sequence of sequences of sequences of Floats specifying the vectors that indicate the set of crack extension directions. Each vector is described by a tuple of two points, and each point is described by a tuple of two or three coordinates indicating its position. This argument is required only when *extensionDirectionMethod*=Q_VECTORS. The default value is an empty sequence.

midNodePosition

A Float specifying the position of the midside node along the edges of the second-order elements that radiate from the crack tip. Possible values are $0.0 < \text{midNodeParameter} < 1.0$. The default value is 0.5.

collapsedElementAtTip

A SymbolicConstant specifying the crack-tip singularity. Possible values are NONE, SINGLE_NODE, and DUPLICATE_NODES. The default value is NONE.

Return value

A ContourIntegral object.

Exceptions

None.

19.3.2 **setValues(...)**

This method modifies the `ContourIntegral` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ContourIntegral** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.3.3 **Members**

The `ContourIntegral` object has members with the same names and descriptions as the arguments to the **ContourIntegral** method.

In addition, the `ContourIntegral` object has the following member:

suppressed

A Boolean specifying whether the crack is suppressed or not. The default value is OFF.

19.3.4 **Corresponding analysis keywords**

*CONTOUR INTEGRAL

19.4 **Crack object**

The Crack object is the abstract base type for `ContourIntegral` and future crack objects.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.cracks[name]
```

ENGINEERING FEATURE COMMANDS

```
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.cracks[name]
```

19.4.1 resume()

This method resumes the crack that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

19.4.2 suppress()

This method suppresses the crack.

Arguments

None.

Return value

None

Exceptions

None.

19.4.3 Members

The Crack object has the following members:

name

A String specifying the repository key.

suppressed

A Boolean specifying whether the crack is suppressed or not. The default value is OFF.

19.5 **DebondVCCT object**

The DebondVCCT object defines the parameters needed to activate crack propagation using VCCT.
The DebondVCCT object is derived from the Crack object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.cracks[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.cracks[name]
```

19.5.1 **DebondVCCT(...)**

This method creates a DebondVCCT object. Although the constructor is available both for parts and for the assembly, DebondVCCT objects are currently supported only under the assembly.

Path

```
mdb.models[name].parts[name].engineeringFeatures.DebondVCCT
mdb.models[name].rootAssembly.engineeringFeatures.DebondVCCT
```

Required arguments

name

A String specifying the repository key.

initiationStep

A String specifying the name of the step in which the DebondVCCT object is created.

surfToSurfInteraction

A String specifying the name of the SurfaceToSurfaceContactStd object that defines the surface to surface interaction for the crack surfaces.

Optional arguments

debondingForceAmplitude

A SymbolicConstant specifying whether the debond force between the two surfaces at the crack tip is to be released immediately or gradually during the following increment after debonding. Possible values are STEP and RAMP. The default value is STEP.

printToDATFrequency

An Int specifying the frequency at which output will be printed to DAT file. The default value is 1.

Return value

A DebondVCCT object.

Exceptions

None.

19.5.2 **setValues(...)**

This method modifies the DebondVCCT object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DebondVCCT** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.5.3 **Members**

The DebondVCCT object has members with the same names and descriptions as the arguments to the **DebondVCCT** method.

In addition, the DebondVCCT object has the following member:

suppressed

A Boolean specifying whether the crack is suppressed or not. The default value is OFF.

19.5.4 **Corresponding analysis keywords**

*DEBOND

19.6 DiscreteFastener object

The DiscreteFastener object defines a discrete fastener.

The DiscreteFastener object is derived from the Fastener object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.fasteners[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.fasteners[name]
```

19.6.1 DiscreteFastener(...)

This method creates a DiscreteFastener object. Although the constructor is available both for parts and for the assembly, DiscreteFastener objects are currently supported only under the assembly.

Path

```
mdb.models[name].parts[name].engineeringFeatures.DiscreteFastener
mdb.models[name].rootAssembly.engineeringFeatures.DiscreteFastener
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the fastener is applied.

influenceRadius

The SymbolicConstant WHOLE_SURFACE or a Float specifying the coupling influence radius.

Optional arguments

ur1

A Boolean specifying whether to constrain rotational displacement component about the 1-direction. The default value is ON.

ur2

A Boolean specifying whether to constrain rotational displacement component about the 2-direction. The default value is ON.

ur3

A Boolean specifying whether to constrain rotational displacement component about the 3-direction. The default value is ON.

coupling

A SymbolicConstant specifying the coupling method used to couple the displacement and rotation of each attachment point to the average motion of the surface nodes within the radius of influence from the fastening point. Possible values are CONTINUUM and STRUCTURAL. The default value is CONTINUUM.

weightingMethod

A SymbolicConstant specifying the weighting scheme to be used to weight the contribution of the displacements of the surface nodes within the radius of influence to the motion of the fastening point. UNIFORM, LINEAR, QUADRATIC, and CUBIC indicate uniform, linear decreasing, quadratic polynomial decreasing, and cubic polynomial monotonic decreasing weight distributions. Possible values are UNIFORM, LINEAR, QUADRATIC, and CUBIC. The default value is UNIFORM.

localCsys

None or a DatumCsys object specifying the local coordinate system of fastener couplings. If *localCsys=None*, couplings are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

Return value

A DiscreteFastener object.

Exceptions

None.

19.6.2 **setValues(...)**

This method modifies the DiscreteFastener object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DiscreteFastener** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.6.3 Members

The DiscreteFastener object has members with the same names and descriptions as the arguments to the **DiscreteFastener** method.

In addition, the DiscreteFastener object has the following member:

suppressed

A Boolean specifying whether the fastener is suppressed or not. The default value is OFF.

19.6.4 Corresponding analysis keywords

*COUPLING, *DISTRIBUTING

19.7 Fastener object

The Fastener object is the abstract base type for PointFastener , DiscreteFastener , and AssembledFastener .

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.fasteners[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.fasteners[name]
```

19.7.1 resume()

This method resumes the fastener that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

19.7.2 **suppress()**

This method suppresses the fastener.

Arguments

None.

Return value

None

Exceptions

None.

19.7.3 **Members**

The Fastener object has the following members:

name

A String specifying the repository key.

suppressed

A Boolean specifying whether the fastener is suppressed or not. The default value is OFF.

19.8 **HeatCapacitance object**

The HeatCapacitance object defines point heat capacitance on a part or an assembly region.

The HeatCapacitance object is derived from the Inertia object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.inertias[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.inertias[name]
```

19.8.1 **HeatCapacitance(...)**

This method creates a HeatCapacitance object.

Path

```

mdb.models[name].parts[name].engineeringFeatures.HeatCapacitance
mdb.models[name].rootAssembly.engineeringFeatures.HeatCapacitance

```

Required arguments*name*

A String specifying the repository key.

region

A Region object specifying the region to which the heat capacitance is applied.

table

A sequence of sequences of Floats specifying heat capacitance properties. The items in the table data are described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

The table data specify the following:

- Heat capacitance magnitude, ρcV (density $\times$ specific heat $\times$ volume).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A HeatCapacitance object.

Exceptions

None.

19.8.2 setValues(...)

This method modifies the HeatCapacitance object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **HeatCapacitance** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.8.3 Members

The **HeatCapacitance** object has members with the same names and descriptions as the arguments to the **HeatCapacitance** method.

In addition, the **HeatCapacitance** object has the following member:

suppressed

A Boolean specifying whether the inertia is suppressed or not. The default value is OFF.

19.8.4 Corresponding analysis keywords

*HEATCAP

19.9 Inertia object

The **Inertia** object is the abstract base type for **HeatCapacitance** , **NonstructuralMass** , and **PointMassInertia** .

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.inertias[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.inertias[name]
```


19.9.1 **resume()**

This method resumes the inertia that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

19.9.2 **suppress()**

This method suppresses the inertia.

Arguments

None.

Return value

None

Exceptions

None.

19.9.3 **Members**

The Inertia object has the following members:

name

A String specifying the repository key.

suppressed

A Boolean specifying whether the inertia is suppressed or not. The default value is OFF.

19.10 **NonstructuralMass object**

The NonstructuralMass object defines the mass contribution from nonstructural features into the model.

The NonstructuralMass object is derived from the Inertia object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.inertias[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.inertias[name]
```

19.10.1 NonstructuralMass(...)

This method creates a NonstructuralMass object.

Path

```
mdb.models[name].parts[name].engineeringFeatures.NonstructuralMass
mdb.models[name].rootAssembly.engineeringFeatures.NonstructuralMass
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the mass is applied.

units

A SymbolicConstant specifying the units used to specify the nonstructural mass. Possible values are TOTAL_MASS, MASS_PER_VOLUME, MASS_PER_AREA, and MASS_PER_LENGTH.

magnitude

A Float specifying the mass magnitude.

Optional argument

distribution

A SymbolicConstant specifying the distribution of the nonstructural mass. Possible values are MASS_PROPORTIONAL and VOLUME_PROPORTIONAL. The default value is MASS_PROPORTIONAL.

The *distribution* argument applies only when *units*=TOTAL_MASS.

Return value

A NonstructuralMass object.

Exceptions

None.

19.10.2 **setValues(...)**

This method modifies the `NonstructuralMass` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **NonstructuralMass** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.10.3 **Members**

The `NonstructuralMass` object has members with the same names and descriptions as the arguments to the **NonstructuralMass** method.

In addition, the `NonstructuralMass` object has the following member:

suppressed

A Boolean specifying whether the inertia is suppressed or not. The default value is OFF.

19.10.4 **Corresponding analysis keywords**

*NONSTRUCTURAL MASS

19.11 **PointFastener object**

The `PointFastener` object defines a point fastener.

The `PointFastener` object is derived from the `Fastener` object.

Access

`import part`

```

mdb.models[name].parts[name].engineeringFeatures.fasteners[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.fasteners[name]

```

19.11.1 PointFastener(...)

This method creates a PointFastener object. Although the constructor is available both for parts and for the assembly, PointFastener objects are currently supported only under the assembly.

Path

```

mdb.models[name].parts[name].engineeringFeatures.PointFastener
mdb.models[name].rootAssembly.engineeringFeatures.PointFastener

```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which fasteners are applied.

physicalRadius

A Float specifying the physical fastener radius.

Optional arguments

directionVector

A VertexArray object of length 2 specifying the direction of projection. Instead of through a Vertex, each point may be specified through a tuple of coordinates. The default value is None.

targetSurfaces

A RegionArray object specifying surfaces to be fastened. The default value is MODEL.

ur1

A Boolean specifying whether to constrain rotational displacement component about the 1-direction. The default value is ON.

ur2

A Boolean specifying whether to constrain rotational displacement component about the 2-direction. The default value is ON.

ur3

A Boolean specifying whether to constrain rotational displacement component about the 3-direction. The default value is ON.

attachmentMethod

A SymbolicConstant specifying the method used to locate points for attaching fasteners. Possible values are FACETOFACE, EDGETOFACE, FACETOEDGE, and EDGETOEDGE. The default value is FACETOFACE.

influenceRadius

The SymbolicConstant DEFAULT or a Float specifying the maximum distance from the projection point on a connected surface within which the nodes on that surface must lie to contribute to the motion of the projection point. If the value is DEFAULT, a radius is computed from the fastener diameter and the surface facet lengths. The default value is DEFAULT.

searchRadius

The SymbolicConstant DEFAULT or a Float specifying the distance from the positioning points within which the connected points must lie. The default value is DEFAULT.

maximumLayers

The SymbolicConstant ALL or an Int specifying the maximum number of layers for each fastener. If the value is ALL, the maximum possible number of layers within the searchRadius will be used for each fastener. The default value is ALL.

coupling

A SymbolicConstant specifying the coupling method used to couple the displacement and rotation of each attachment point to the average motion of the surface nodes within the radius of influence from the fastener projection point. Possible values are CONTINUUM and STRUCTURAL. The default value is CONTINUUM.

weightingMethod

A SymbolicConstant specifying the weighting scheme to be used to weight the contribution of the displacements of the surface nodes within the radius of influence to the motion of the fastener projection point. UNIFORM, LINEAR, QUADRATIC, and CUBIC indicate uniform, linear decreasing, quadratic polynomial decreasing, and cubic polynomial monotonic decreasing weight distributions. Possible values are UNIFORM, LINEAR, QUADRATIC, and CUBIC. The default value is UNIFORM.

additionalMass

A Float specifying the mass that will be distributed to fastener attachment points. The default value is 0.0.

adjustOrientation

A Boolean specifying whether to adjust localCsys such that the local z-axis for each fastener is normal to the surface that is closest to the reference node for that fastener. The default value is ON.

localCsys

None or a DatumCsys object specifying the local coordinate system. If *localCsys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

connectionType

A SymbolicConstant specifying the fastener connection type. Possible values are CONNECTOR and BEAM_MPC. The default value is CONNECTOR.

sectionName

A String specifying the connector section assigned to generated connectors. The default value is an empty string.

ENGINEERING FEATURE COMMANDS

connectorOrientationLocalCsys1

None or a DatumCsys object specifying the local coordinate system of the first connector point in generated connectors. If *connectorOrientationLocalCsys1=None*, the degrees of freedom are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

axis1

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied for the first point in generated connectors. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle1

A Float specifying the angle of the additional rotation for the first point in generated connectors. The default value is 0.0.

orient2SameAs1

A Boolean specifying whether or not the second connector point in generated connectors is to use the same local coordinate system, axis, and angle as the first point. The default value is ON.

connectorOrientationLocalCsys2

None or a DatumCsys object specifying the local coordinate system of the second connector point in generated connectors. If *connectorOrientationLocalCsys2=None*, the degrees of freedom are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

axis2

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied for the second point in generated connectors. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle2

A Float specifying the angle of the additional rotation for the second point in generated connectors. The default value is 0.0.

unsorted

A Boolean specifying whether the analysis product should leave targetSurfaces in the given unsorted order, or sort them by proximity to determine the connectivity of fastening points. The default value is OFF.

Return value

A PointFastener object.

Exceptions

None.

19.11.2 **setValues(...)**

This method modifies the PointFastener object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PointFastener** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.11.3 **Members**

The PointFastener object has members with the same names and descriptions as the arguments to the **PointFastener** method.

In addition, the PointFastener object has the following member:

suppressed

A Boolean specifying whether the fastener is suppressed or not. The default value is OFF.

19.11.4 **Corresponding analysis keywords**

*FASTENER

19.12 **PointMassInertia object**

The PointMassInertia object defines point masses and point rotary inertia on a part or an assembly region.

The PointMassInertia object is derived from the Inertia object.

Access

import part

```

mdb.models[name].parts[name].engineeringFeatures.inertias[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.inertias[name]

```

19.12.1 PointMassInertia(...)

This method creates a PointMassInertia object.

Path

```

mdb.models[name].parts[name].engineeringFeatures.PointMassInertia
mdb.models[name].rootAssembly.engineeringFeatures.PointMassInertia

```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the mass or rotary inertia is applied.

Optional arguments

mass

A Float specifying the mass magnitude for isotropic mass. This parameter cannot be specified when anisotropic mass terms are specified. The default value is 0.0.

mass1

A Float specifying the mass in the 1-direction for anisotropic mass. This parameter cannot be specified when isotropic mass is also specified. The default value is 0.0.

mass2

A Float specifying the mass in the 2-direction for anisotropic mass. This parameter cannot be specified when isotropic mass is also specified. The default value is 0.0.

mass3

A Float specifying the mass in the 3-direction for anisotropic mass. This parameter cannot be specified when isotropic mass is also specified. The default value is 0.0.

i11

A Float specifying the rotary inertia about the local 1-axis, I_{11} . The default value is 0.0.

i22

A Float specifying the rotary inertia about the local 2-axis, I_{22} . The default value is 0.0.

i33

A Float specifying the rotary inertia about the local 3-axis, I_{33} . The default value is 0.0.

i12

A Float specifying the product of inertia, I_{12} . The default value is 0.0.

i13

A Float specifying the product of inertia, I_{13} . The default value is 0.0.

i23

A Float specifying the product of inertia, I_{23} . The default value is 0.0.

localCsys

None or a DatumCsys object specifying the local coordinate system for the anisotropic mass terms (when specified), and the rotary inertia (when specified). If *localCsys*=**None**, the anisotropic mass and rotary inertia data are defined in the global coordinate system. The default value is **None**.

alpha

A Float specifying the alpha damping magnitude. The default value is 0.0.
This argument applies only to Abaqus/Standard analyses.

composite

A Float specifying the composite damping magnitude. The default value is 0.0.
This argument applies only to Abaqus/Standard analyses.

Return value

A PointMassInertia object.

Exceptions

None.

19.12.2 setValues(...)

This method modifies the PointMassInertia object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PointMassInertia** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.12.3 Members

The `PointMassInertia` object has members with the same names and descriptions as the arguments to the `PointMassInertia` method.

In addition, the `PointMassInertia` object has the following member:

suppressed

A Boolean specifying whether the inertia is suppressed or not. The default value is OFF.

19.12.4 Corresponding analysis keywords

*MASS, *ROTARY INERTIA

19.13 SpringDashpot object

The `SpringDashpot` object is the abstract base type for the `SpringDashpotToGround` and `TwoPointSpringDashpot` objects.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.springDashpots[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.springDashpots[name]
```

19.13.1 resume()

This method resumes the spring/dashpot that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

19.13.2 **suppress()**

This method suppresses the spring/dashpot.

Arguments

None.

Return value

None

Exceptions

None.

19.13.3 **Members**

The SpringDashpot object has the following members:

name

A String specifying the repository key.

suppressed

A Boolean specifying whether the spring/dashpot is suppressed or not. The default value is OFF.

19.14 **SpringDashpotToGround object**

The SpringDashpotToGround object defines springs and/or dashpots between points and ground on a part or an assembly region.

The SpringDashpotToGround object is derived from the SpringDashpot object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.springDashpots[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.springDashpots[name]
```

19.14.1 **SpringDashpotToGround(...)**

This method creates a SpringDashpotToGround object.

Path

```
mdb.models[name].parts[name].engineeringFeatures.SpringDashpotToGround  
mdb.models[name].rootAssembly.engineeringFeatures\  
.SpringDashpotToGround
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the springs and/or dashpots are applied.

dof

An Int specifying the degree of freedom associated with the spring and dashpot behaviors.

Optional arguments

orientation

None or a DatumCsys object specifying the local directions for the spring and/or dashpot. If *orientation=None*, the spring and/or dashpot data are defined in the global coordinate system. The default value is **None**.

springBehavior

A Boolean specifying whether to apply spring behavior to the selected points. The default value is OFF.

At least one of the arguments *springBehavior=ON* or *dashpotBehavior=ON* must be specified.

dashpotBehavior

A Boolean specifying whether to apply dashpot behavior to the selected points. The default value is OFF.

At least one of the arguments *springBehavior=ON* or *dashpotBehavior=ON* must be specified.

springStiffness

A Float specifying the force per relative displacement for the spring. The default value is 0.0.

dashpotCoefficient

A Float specifying the force per relative velocity for the dashpot. The default value is 0.0.

Return value

A SpringDashpotToGround object.

Exceptions

None.

19.14.2 **setValues(...)**

This method modifies the SpringDashpotToGround object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SpringDashpotToGround** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.14.3 **Members**

The SpringDashpotToGround object has members with the same names and descriptions as the arguments to the **SpringDashpotToGround** method.

In addition, the SpringDashpotToGround object has the following member:

suppressed

A Boolean specifying whether the spring/dashpot is suppressed or not. The default value is OFF.

19.14.4 **Corresponding analysis keywords**

*ELEMENT, TYPE=SPRING1; *ELEMENT, TYPE=DASHPOT1; *SPRING; *DASHPOT

19.15 **TwoPointSpringDashpot object**

The TwoPointSpringDashpot object defines springs and/or dashpots between two points on a part or an assembly.

The TwoPointSpringDashpot object is derived from the SpringDashpot object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.springDashpots[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.springDashpots[name]
```

19.15.1 TwoPointSpringDashpot(...)

This method creates a TwoPointSpringDashpot object.

Path

```
mdb.models[name].parts[name].engineeringFeatures.TwoPointSpringDashpot
mdb.models[name].rootAssembly.engineeringFeatures\
.TwoPointSpringDashpot
```

Required arguments

name

A String specifying the repository key.

regionPairs

A sequence of pairs of Region objects specifying the points between which the springs and/or dashpots are applied.

axis

A SymbolicConstant specifying whether the axis of the springs and/or dashpots follows the rotation of the nodes or is in a specified direction. Possible values are NODAL_LINE and FIXED_DOF.

Optional arguments

dof1

An Int specifying the degree of freedom with which the springs and/or dashpots are associated at their first points. The *dof1* argument applies only when *axis*=FIXED_DOFS. The default value is 0.

dof2

An Int specifying the degree of freedom with which the springs and/or dashpots are associated at their second points. The *dof2* argument applies only when *axis*=FIXED_DOFS. The default value is 0.

orientation

None or a DatumCsys object specifying the local directions for the spring and/or dashpot. If *orientation*=**None**, the spring and/or dashpot data are defined in the global coordinate system. The default value is **None**.

The *orientation* argument applies only when *axis*=FIXED_DOFS.

springBehavior

A Boolean specifying whether to apply spring behavior to the selected point pairs. The default value is OFF.

At least one of the arguments *springBehavior=ON* or *dashpotBehavior=ON* must be specified.

dashpotBehavior

A Boolean specifying whether to apply dashpot behavior to the selected point pairs. The default value is OFF.

At least one of the arguments *springBehavior=ON* or *dashpotBehavior=ON* must be specified.

springStiffness

A Float specifying the force per relative displacement for the springs. The default value is 0.0.

dashpotCoefficient

A Float specifying the force per relative velocity for the dashpots. The default value is 0.0.

Return value

A TwoPointSpringDashpot object.

Exceptions

None.

19.15.2 setValues(...)

This method modifies the TwoPointSpringDashpot object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TwoPointSpringDashpot** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.15.3 Members

The TwoPointSpringDashpot object has members with the same names and descriptions as the arguments to the **TwoPointSpringDashpot** method.

In addition, the TwoPointSpringDashpot object has the following member:

suppressed

A Boolean specifying whether the spring/dashpot is suppressed or not. The default value is OFF.

19.15.4 Corresponding analysis keywords

*ELEMENT, TYPE=SPRINGA; *ELEMENT, TYPE=SPRING2; *ELEMENT,
TYPE=DASHPOTA; *ELEMENT, TYPE=DASHPOT2; *SPRING; *DASHPOT

19.16 XFEMCrack object

The XFEMCrack object defines the parameters needed to model crack initiation or crack growth using XFEM technology. Currently only assembly regions are supported.

The XFEMCrack object is derived from the Crack object.

Access

```
import part
mdb.models[name].parts[name].engineeringFeatures.cracks[name]
import assembly
mdb.models[name].rootAssembly.engineeringFeatures.cracks[name]
```

19.16.1 XFEMCrack(...)

This method creates a XFEMCrack object. Although the constructor is available both for parts and for the assembly, XFEMCrack objects are currently supported only under the assembly.

Path

```
mdb.models[name].parts[name].engineeringFeatures.XFEMCrack
mdb.models[name].rootAssembly.engineeringFeatures.XFEMCrack
```


Required arguments*name*

A String specifying the repository key.

crackDomain

A Region object specifying the region that contains the crack or is likely to contain the crack.

Optional arguments*allowCrackGrowth*

A Boolean specifying whether the crack is allowed to propagate (grow). The default value is ON.

crackLocation

A Region object specifying the initial crack location. This parameter is required when *allowCrackGrowth*=OFF.

singularityCalcRadius

None or a Float specifying the radius from the crack tips within which the elements are used for crack singularity calculations. This argument applies only when *allowCrackGrowth*=OFF. The default value is **None**.

interactionProperty

A String specifying the name of the ContactProperty object that defines the contact properties for the crack surfaces. The default value is an empty string.

elemId

A sequence of Ints specifying the labels of the elements that are intersected by the initial crack location. This argument is used only by the input file reader.

nodeId

A sequence of Ints specifying the position of a node in the corresponding element connectivity. This argument is used only by the input file reader.

hasCrackFront

A sequence of Ints specifying the values indicating the inclusion/exclusion of the *crackFrontDist* values. A zero value indicates that *crackFrontDist* is not specified for the *ith* pair *elemId* and *nodeId*. This argument is used only by the input file reader.

crackPlaneDist

A sequence of Floats specifying the values of the first signed distance function. This argument is used by the input file reader.

crackFrontDist

A sequence of Floats specifying the values of the second signed distance function. This argument is used only by the input file reader.

Return value

A XFEMCrack object.

Exceptions

None.

19.16.2 **setValues(...)**

This method modifies the XFEMCrack object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **XFEMCrack** method, except for the *name* argument.

Return value

None

Exceptions

None.

19.16.3 **Members**

The XFEMCrack object has members with the same names and descriptions as the arguments to the **XFEMCrack** method.

In addition, the XFEMCrack object has the following member:

suppressed

A Boolean specifying whether the crack is suppressed or not. The default value is OFF.

19.16.4 **Corresponding analysis keywords**

*ENRICHMENT

*INITIAL CONDITIONS, TYPE=ENRICHMENT

20. Feature commands

Features in Abaqus/CAE include Parts, Datums, Partitions, and Assembly operations. The commands that create Feature objects on only the Part object are described in Chapter 37, “Part commands.” The commands that create Feature objects in only the rootAssembly object are described in Chapter 6, “Assembly commands.” The commands that create Feature objects on both the Part and the rootAssembly objects are described here.

20.1 Feature object

Abaqus/CAE is a feature-based modeling system, and features are stored in the Feature object. The user defines the parameters of the feature, and Abaqus/CAE modifies the model based on the value of the parameters. This evaluation of the parameters is called regeneration of the feature. Feature objects contain both the parameters and the resulting model modification.

Access

```
import part
mdb.models[name].parts[name].features[name]
mdb.models[name].parts[name].featuresById[i]
import assembly
mdb.models[name].rootAssembly.features[name]
mdb.models[name].rootAssembly.featuresById[i]
```

20.1.1 AttachmentPoints(...)

This method creates an attachment points Feature. Attachment points may be created using datum points, vertices, reference points, attachment points, interesting points, orphan mesh nodes or coordinates. Optionally, the attachment points can be projected on geometric faces or element faces.

Path

```
mdb.models[name].parts[name].AttachmentPoints
mdb.models[name].rootAssembly.AttachmentPoints
```

Required arguments

name

A String specifying a unique Feature name.

FEATURE COMMANDS

points

A tuple of points. Each point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

Optional arguments

projectionMethod

A SymbolicConstant specifying the projection method. Possible values are PROJECT_BY_PROXIMITY and PROJECT_BY_DIRECTION. The default value is PROJECT_BY_PROXIMITY.

projectOnFaces

A sequence of Face objects specifying the geometry faces onto which the points are to be projected.

projectOnElementFaces

A sequence of MeshFace objects specifying the orphan mesh element faces onto which the points are to be projected.

projectionDirStartPt

A point specifying the start point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

projectionDirEndPt

A point specifying the end point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

setName

A String specifying a unique set name.

Return value

A Feature object.

Exceptions

None.

20.1.2 AttachmentPointsAlongDirection(...)

This method creates a Feature object by creating attachment points along a direction or between two points. A Datum point, a Vertex, a Reference point, an Attachment point, an Interesting point, or an orphan mesh Node can be specified as the start or end point. The direction can be specified using a straight edge or a datum axis.

Path

`mdb.models[name].parts[name].AttachmentPointsAlongDirection`
`mdb.models[name].rootAssembly.AttachmentPointsAlongDirection`

Required arguments

name

A String specifying a unique Feature name.

startPoint

A point specifying the start point of the direction along which to create points. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

pointCreationMethod

A SymbolicConstant specifying the point creation method. Possible values are AUTO_FIT, NUM_PTS_ALONG_DIR, and NUM_PTS_BETWEEN_PTS.

Optional arguments

endPoint

A point specifying the end point if creating points between two points. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

direction

The direction can be specified by a straight edge or a datum axis.

spacing

A float specifying the spacing to be used between two points.

numPtsAlongDir

An integer specifying the number of points to be created along the specified direction.

numPtsBetweenPts

An integer specifying the number of points to be created between the start and end points.

createPtAtStartPt

A Boolean specifying whether to create an attachment point at the start point. The default value is True.

createPtAtEndPt

A Boolean specifying whether to create an attachment point at the end point. The default value is True.

projectionMethod

A SymbolicConstant specifying the projection method. Possible values are PROJECT_BY_PROXIMITY and PROJECT_BY_DIRECTION. The default value is PROJECT_BY_PROXIMITY.

FEATURE COMMANDS

projectOnFaces

A sequence of Face objects specifying the geometry faces onto which the points are to be projected.

projectOnElementFaces

A sequence of MeshFace objects specifying the orphan mesh element faces onto which the points are to be projected.

projectionDirStartPt

A point specifying the start point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

projectionDirEndPt

A point specifying the end point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

flipDirection

A Boolean specifying if the direction along which the attachment points are created should be reversed. This argument is valid only when *pointCreationMethod*=NUM_PTS_ALONG_DIR.

setName

A String specifying a unique set name.

Return value

A Feature object.

Exceptions

None.

20.1.3 AttachmentPointsOffsetFromEdges(...)

This method creates a Feature object by creating attachment points along or offset from one or more connected edges.

Path

```
mdb.models[name].parts[name].AttachmentPointsOffsetFromEdges  
mdb.models[name].rootAssembly.AttachmentPointsOffsetFromEdges
```

Required arguments

name

A String specifying a unique Feature name.

edges

A sequence of connected Edge objects specifying the geometry edges from which to offset the points.

Optional arguments*startPoint*

A Vertex of the selected edges that specifies the point from which to create points. This point can be one of the two end vertices of the connected edges. In case of edges forming a closed loop and having multiple vertices, this point can be any one of the vertices on the edges.

flipDirection

This parameter is required to indicate the direction in which to create the points. This parameter is required only in case of edges forming a closed loop.

pointCreationMethod

A SymbolicConstant specifying the point creation method. Possible values are BY_NUMBER or BY_SPACING.

numberOfPoints

An integer specifying the number of points to be created along the selected edges.

spacingBetweenPoints

A float specifying the spacing to be used between two points while creating the points between the start and end points of the edges.

offsetFromStartPoint

A float specifying the distance by which to offset the first point from the start vertex of the edge chain. The default value is 0.0.

offsetFromEndPoint

A float specifying the distance by which to offset the last point from the end vertex of the edge chain. This parameter should be specified only if the point creation method is BY_NUMBER. The default value is 0.0.

spacingMethod

A SymbolicConstant specifying the spacing method. Possible values are AUTO_FIT_PTS or SPECIFY_NUM_PTS. The default value is AUTO_FIT_PTS.

patterningMethod

A SymbolicConstant specifying the method to pattern of points. Possible values are PATTERN_ORTHOGONALLY or PATTERN_ALONG_DIRECTION.

referenceFace

A geometry Face object adjacent to one of the edges from which to offset the points to create a pattern of points when the PATTERN_ORTHOGONALLY method is chosen for patterning. The face is used to identify the patterning direction. If the number of rows is one and the initial offset is zero, the reference face may not be specified.

FEATURE COMMANDS

startPointForPatternDirection

A point specifying the start point of the direction along which to create a pattern of points when the PATTERN_ALONG_DIRECTION method is chosen for patterning. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

endPointForPatternDirection

A point specifying the end point of the direction along which to create a pattern of points when the PATTERN_ALONG_DIRECTION method is chosen for patterning. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

offsetFromEdges

A float specifying the distance by which to offset the first row of points from the edges.

numberOfRows

An integer specifying the number of rows of points to be created for the pattern. The default value is 1.

spacingBetweenRows

A float specifying the spacing to be used between two rows while creating a pattern of points.

projectionMethod

A SymbolicConstant specifying the projection method. Possible values are PROJECT_BY_PROXIMITY and PROJECT_BY_DIRECTION. The default value is PROJECT_BY_PROXIMITY.

projectOnFaces

A sequence of Face objects specifying the geometry faces onto which the points are to be projected.

projectOnElementFaces

A sequence of MeshFace objects specifying the orphan mesh element faces onto which the points are to be projected.

projectionDirStartPt

A point specifying the start point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

projectionDirEndPt

A point specifying the end point of the projection direction. The point can be a Vertex, Datum point, Reference point, Attachment point, orphan mesh Node, Interesting point object, or a tuple of Floats representing the coordinates of a point.

setName

A String specifying a unique set name.

Return value

A Feature object.

Exceptions

None.

20.1.4 DatumAxisByCylFace(...)

This method creates a Feature object and a DatumAxis object along the axis of a cylinder or cone.

Path

```
mdb.models[name].rootAssembly.DatumAxisByCylFace
mdb.models[name].parts[name].DatumAxisByCylFace
```

Required argument

face

A cylindrical or conical Face object.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.5 DatumAxisByNormalToPlane(...)

This method creates a Feature object and a DatumAxis object normal to the specified plane and passing through the specified point.

Path

```
mdb.models[name].rootAssembly.DatumAxisByNormalToPlane
mdb.models[name].parts[name].DatumAxisByNormalToPlane
```

Required arguments

plane

A planar Face, an ElementFace, or a Datum object representing a datum plane.

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

FEATURE COMMANDS

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.6 DatumAxisByParToEdge(...)

This method creates a Feature object and a DatumAxis object parallel to the specified edge and passing through the specified point.

Path

```
mdb.models[name].rootAssembly.DatumAxisByParToEdge  
mdb.models[name].parts[name].DatumAxisByParToEdge
```

Required arguments

edge

A straight Edge, an ElementEdge, or a Datum object representing a datum axis.

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.7 DatumAxisByPrincipalAxis(...)

This method creates a Feature object and a DatumAxis object along one of the three principal axes.

Path

```
mdb.models[name].rootAssembly.DatumAxisByPrincipalAxis
```

```
mdb.models [name] .parts [name] .DatumAxisByPrincipalAxis
```

Required argument

principalAxis

A SymbolicConstant specifying the principal axis. Possible values are XAXIS, YAXIS, and ZAXIS.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.8 DatumAxisByRotation(...)

This method creates a Feature object and a DatumAxis object in a three-dimensional model by rotating a line about the specified axis through the specified angle.

Path

```
mdb.models [name] .rootAssembly .DatumAxisByRotation  
mdb.models [name] .parts [name] .DatumAxisByRotation
```

Required arguments

line

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object specifying the line to rotate.

axis

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object specifying the axis about which to rotate the line.

angle

A Float specifying the angle in degrees to rotate the line.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.9 DatumAxisByRotation(...)

This method creates a Feature object and a DatumAxis object in a two-dimensional model by rotating a line about the specified point through the specified angle.

Path

```
mdb.models[name].rootAssembly.DatumAxisByRotation  
mdb.models[name].parts[name].DatumAxisByRotation
```

Required arguments

line

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object specifying the line to rotate.

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying the point about which to rotate the line.

angle

A Float specifying the angle in degrees to rotate the line.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.10 DatumAxisByThreePoint(...)

This method creates a Feature object and a DatumAxis object normal to the circle described by three points and through its center.

Path

```
mdb.models[name].rootAssembly.DatumAxisByThreePoint
```

```
mdb.models [name] .parts [name] .DatumAxisByThreePoint
```

Required arguments

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying the first point on the circle.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying the second point on the circle.

point3

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying the third point on the circle.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.11 DatumAxisByThruEdge(...)

This method creates a Feature object and a DatumAxis object along the specified edge.

Path

```
mdb.models [name] .rootAssembly .DatumAxisByThruEdge  
mdb.models [name] .parts [name] .DatumAxisByThruEdge
```

Required argument

edge

A straight Edge or an ElementEdge object.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.12 DatumAxisByTwoPlane(...)

This method creates a Feature object and a DatumAxis object at the intersection of two planes.

Path

```
mdb.models[name].rootAssembly.DatumAxisByTwoPlane  
mdb.models[name].parts[name].DatumAxisByTwoPlane
```

Required arguments

plane1

A planar Face, an ElementFace, or a Datum object representing a datum plane.

plane2

A planar Face, an ElementFace, or a Datum object representing a datum plane.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.13 DatumAxisByTwoPoint(...)

This method creates a Feature object and a DatumAxis object along the line joining two points.

Path

```
mdb.models[name].rootAssembly.DatumAxisByTwoPoint  
mdb.models[name].parts[name].DatumAxisByTwoPoint
```

Required arguments

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.14 DatumCsysByDefault(...)

This method creates a Feature object and a DatumCsys object from the specified default coordinate system at the origin.

Path

```
mdb.models [name] .rootAssembly.DatumCsysByDefault  
mdb.models [name] .parts [name] .DatumCsysByDefault
```

Required argument

coordSysType

A SymbolicConstant specifying the default coordinate system to be used. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

Optional argument

name

A String specifying the name of the DatumCsys.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.15 DatumCsysByOffset(...)

This method creates a Feature object and a DatumCsys object by offsetting the origin of an existing datum coordinate system to a specified point.

Path

```
mdb.models[name].rootAssembly.DatumCsysByOffset  
mdb.models[name].parts[name].DatumCsysByOffset
```

Required arguments

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

datumCoordSys

A Datum object representing a datum coordinate system from which to offset.

vector

A sequence of three Floats specifying the X-, Y-, and Z-offsets from *datumCoordSys*. The arguments *vector* and *point* are mutually exclusive, and one of them must be specified.

point

A Vertex, InterestingPoint, DatumPoint object or a sequence of three Floats specifying the X-, Y-, and Z-coordinates of a point in space. The point represents the origin of the new datum coordinate system. The arguments *vector* and *point* are mutually exclusive, and one of them must be specified.

Optional argument

name

A String specifying the name of the DatumCsys.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.16 DatumCsysByThreePoints(...)

This method creates a Feature object and a DatumCsys object from three points.

Path

```
mdb.models[name].parts[name].DatumCsysByThreePoints  
mdb.models[name].rootAssembly.DatumCsysByThreePoints
```


Required arguments*coordSysType*

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying the origin of the coordinate system.

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying a point on the X -axis or the r -axis. The *point1* and *line1* arguments are mutually exclusive. One of them must be specified.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point specifying a point in the X - Y plane or the r - θ plane. The *point2* and *line2* arguments are mutually exclusive. One of them must be specified.

line1

An Edge, an Element Edge, or a Datum object representing a datum axis specifying the X -axis or the r -axis. The *point1* and *line1* arguments are mutually exclusive. One of them must be specified.

line2

An Edge, an Element Edge, or a Datum object representing a datum axis specifying a vector in the X - Y plane or the r - θ plane. The *point2* and *line2* arguments are mutually exclusive. One of them must be specified.

Optional argument*name*

A String specifying the name of the DatumCsys.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.17 DatumCsysByTwoLines(...)

This method creates a Feature object and a DatumCsys object from two orthogonal lines. The origin of the new datum coordinate system is placed at the intersection of the two lines.

Path

```
mdb.models[name].rootAssembly.DatumCsysByTwoLines  
mdb.models[name].parts[name].DatumCsysByTwoLines
```

Required arguments

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

line1

A straight Edge, an ElementEdge, or a Datum object representing a datum axis specifying the *X*-axis or the *r*-axis.

line2

A straight Edge, an ElementEdge, or a Datum object representing a datum axis specifying a line in the *X*-*Y* plane or in the *r*- θ plane.

Optional argument

name

A String specifying the name of the DatumCsys.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.18 DatumPlaneByPrincipalPlane(...)

This method creates a Feature object and a DatumPlane object through the origin along one of the three principal planes.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByPrincipalPlane  
mdb.models[name].parts[name].DatumPlaneByPrincipalPlane
```

Required arguments

principalPlane

A SymbolicConstant specifying the principal plane. Possible values are XYPLANE, YZPLANE, and XZPLANE.

offset

A Float specifying the offset from the plane.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.19 DatumPlaneByOffset(...)

This method creates a Feature object and a DatumPlane object offset by a specified distance from an existing plane.

Path

`mdb.models[name].rootAssembly.DatumPlaneByOffset`

`mdb.models[name].parts[name].DatumPlaneByOffset`

Required arguments

plane

A planar Face, an ElementFace, or a Datum object representing a datum plane.

flip

A SymbolicConstant specifying whether the normal should be flipped. Possible values are SIDE1 and SIDE2.

offset

A Float specifying the offset from the plane.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.20 DatumPlaneByOffset(...)

This method creates a Feature object and a DatumPlane object offset from an existing plane and passing through the specified point.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByOffset
mdb.models[name].parts[name].DatumPlaneByOffset
```

Required arguments

plane

A planar Face, an ElementFace, or a Datum object representing a datum plane.

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.21 DatumPlaneByRotation(...)

This method creates a Feature object and a DatumPlane object by rotating a plane about the specified axis through the specified angle.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByRotation
mdb.models[name].parts[name].DatumPlaneByRotation
```

Required arguments

plane

A planar Face, an ElementFace, or a Datum object representing a datum plane.

axis

A straight Edge, an ElementEdge, or a Datum object representing a datum axis.

angle

A Float specifying the angle in degrees to rotate the plane.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.22 DatumPlaneByThreePoints(...)

This method creates a Feature object and a DatumPlane object defined by passing through three points.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByThreePoints
mdb.models[name].parts[name].DatumPlaneByThreePoints
```

Required arguments

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

point3

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.23 DatumPlaneByLinePoint(...)

This method creates a Feature object and a DatumPlane object that pass through the specified line and through the specified point that does not lie on the line.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByLinePoint
mdb.models[name].parts[name].DatumPlaneByLinePoint
```

Required arguments

line

A straight Edge, an ElementEdge, or a Datum object representing a datum axis.

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.24 DatumPlaneByPointNormal(...)

This method creates a Feature object and a DatumPlane object normal to the specified line and running through the specified point.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByPointNormal
mdb.models[name].parts[name].DatumPlaneByPointNormal
```

Required arguments

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

normal

A straight Edge, an ElementEdge, or a Datum object representing a datum axis.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.25 DatumPlaneByTwoPoint(...)

This method creates a Feature object and a DatumPlane object midway between two points and normal to the line connecting the points.

Path

```
mdb.models[name].rootAssembly.DatumPlaneByTwoPoint
mdb.models[name].parts[name].DatumPlaneByTwoPoint
```

Required arguments

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.26 DatumPointByCoordinate(...)

This method creates a Feature object and a DatumPoint object at the point defined by the specified coordinates.

Path

```
mdb.models[name].rootAssembly.DatumPointByCoordinate  
mdb.models[name].parts[name].DatumPointByCoordinate
```

Required argument

coords

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of the datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.27 DatumPointByOffset(...)

This method creates a Feature object and a DatumPoint object offset from an existing point by a vector.

Path

```
mdb.models[name].rootAssembly.DatumPointByOffset  
mdb.models[name].parts[name].DatumPointByOffset
```

Required arguments

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

vector

A sequence of three Floats specifying the X-, Y-, and Z-offsets from *point*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.28 DatumPointByMidPoint(...)

This method creates a Feature object and a DatumPoint object midway between two points.

Path

```
mdb.models[name].rootAssembly.DatumPointByMidPoint
mdb.models[name].parts[name].DatumPointByMidPoint
```

Required arguments

point1

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

point2

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.29 DatumPointByOnFace(...)

This method creates a Feature object and a DatumPoint object on the specified face, offset from two edges.

Path

```
mdb.models[name].rootAssembly.DatumPointByOnFace
mdb.models[name].parts[name].DatumPointByOnFace
```

Required arguments

face

A planar Face or a Datum object representing a datum plane.

FEATURE COMMANDS

edge1

A straight Edge or a Datum object representing a datum axis.

offset1

A Float specifying the offset from *edge1*.

edge2

A straight Edge or a Datum object representing a datum axis.

offset2

A Float specifying the offset from *edge2*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.30 DatumPointByEdgeParam(...)

This method creates a Feature object and a DatumPoint object along an edge at a selected distance from one end of the edge.

Path

`mdb.models[name].rootAssembly.DatumPointByEdgeParam`

`mdb.models[name].parts[name].DatumPointByEdgeParam`

Required arguments

edge

An Edge object.

parameter

A Float specifying the distance along *edge* to the DatumPoint object. Possible values are $0 < \text{parameter} < 1$.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException and RangeError.

20.1.31 DatumPointByProjOnEdge(...)

This method creates a Feature object and a DatumPoint object along an edge by projecting an existing point along the normal to the edge.

Path

```
mdb.models[name].rootAssembly.DatumPointByProjOnEdge
mdb.models[name].parts[name].DatumPointByProjOnEdge
```

Required arguments

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

edge

An Edge, an ElementEdge or a Datum object representing a datum axis.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.32 DatumPointByProjOnFace(...)

This method creates a Feature object and a DatumPoint object on a specified face by projecting an existing point onto the face.

Path

```
mdb.models[name].rootAssembly.DatumPointByProjOnFace
mdb.models[name].parts[name].DatumPointByProjOnFace
```

Required arguments

point

A Vertex, an InterestingPoint, a MeshNode, or a Datum object representing a datum point.

face

A Face object or a Datum object representing a datum plane.

Note: Any other types of planes are not supported.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.33 MakeSketchTransform(...)

This method creates a Transform object. A Transform object is a 4x3 matrix of Floats that represents the transformation from sketch coordinates to part coordinates.

Path

```
mdb.models[name].parts[name].MakeSketchTransform  
mdb.models[name].rootAssembly.MakeSketchTransform
```

Required argument

sketchPlane

A Datum plane object or a planar Face object specifying the sketch plane.

Optional arguments

origin

A sequence of Floats specifying the X-, Y-, and Z-coordinates that will be used as the origin of the sketch. The default value is computed as the centroid of the face.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

sketchPlaneSide

A SymbolicConstant specifying on which side of the *sketchPlane* the sketch is positioned. Possible values are SIDE1 and SIDE2. The default value is SIDE1.

sketchUpEdge

An Edge or DatumAxis object specifying the orientation of the sketch. If unspecified, the sketch is assumed to be oriented with the Y-direction pointing up.

Return value

A Transform object. A Transform is an object with one method that returns the transform matrix.

Exceptions

If the *sketchUpEdge* is parallel to the *sketchPlane*:

Up direction is parallel to plane normal

20.1.34 PartitionCellByDatumPlane(...)

This method partitions one or more cells using the given datum plane.

Path

```
mdb.models[name].parts[name].PartitionCellByDatumPlane
mdb.models[name].rootAssembly.PartitionCellByDatumPlane
```

Required arguments*cells*

A sequence of Cell objects specifying the cells to partition.

datumPlane

A DatumPlane object.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.35 PartitionCellByExtendFace(...)

This method partitions one or more cells by extending the underlying geometry of a given face to partition the target cells.

Path

```
mdb.models[name].parts[name].PartitionCellByExtendFace  
mdb.models[name].rootAssembly.PartitionCellByExtendFace
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

extendFace

A planar, cylindrical, conical, or spherical Face object.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.36 PartitionCellByExtrudeEdge(...)

This method partitions one or more cells by extruding selected edges in the given direction.

Path

```
mdb.models[name].parts[name].PartitionCellByExtrudeEdge  
mdb.models[name].rootAssembly.PartitionCellByExtrudeEdge
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

edges

The Edge objects to be extruded. The edges must be in the same plane. The edges must form a continuous chain, without branches. The edges must belong to the same PartInstance object.

line

A straight Edge or DatumAxis object specifying the extrude direction. *line* must be perpendicular to the plane formed by *edges*.

sense

A SymbolicConstant specifying the direction of the extrusion. Possible values are FORWARD and REVERSE. If *sense*=FORWARD, the extrusion is in the direction of *line*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.37 PartitionCellByPatchNCorners(...)

This method partitions a cell using an N-sided cutting patch defined by the given corner points.

Path

```
mdb.models[name].parts[name].PartitionCellByPatchNCorners
mdb.models[name].rootAssembly.PartitionCellByPatchNCorners
```

Required arguments

cell

A Cell object specifying the cell to partition.

cornerPoints

A sequence of Vertex, InterestingPoint, or DatumPoint objects. $3 \leq \text{len}(\text{cornerPoints}) \leq 5$. The corner points must not coincide.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.38 PartitionCellByPatchNEdges(...)

This method partitions a cell using an N-sided cutting patch defined by the given edges.

Path

```
mdb.models[name].parts[name].PartitionCellByPatchNEdges
mdb.models[name].rootAssembly.PartitionCellByPatchNEdges
```

Required arguments

cell

A Cell specifying the cell to partition.

edges

A sequence of Edge objects bounding the patch. The edges must form a closed loop. The Edge objects must belong to the same PartInstance object as *cell*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.39 PartitionCellByPlaneNormalToEdge(...)

This method partitions one or more cells using a plane normal to an edge at the given edge point.

Path

```
mdb.models[name].parts[name].PartitionCellByPlaneNormalToEdge
mdb.models[name].rootAssembly.PartitionCellByPlaneNormalToEdge
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

edge

An Edge object specifying the normal to the plane.

point

A Vertex, InterestingPoint, or DatumPoint object specifying a point on *edge*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.40 PartitionCellByPlanePointNormal(...)

This method partitions one or more cells using a plane defined by a point and a normal direction.

Path

```
mdb.models[name].parts[name].PartitionCellByPlanePointNormal
mdb.models[name].rootAssembly.PartitionCellByPlanePointNormal
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

point

A Vertex, InterestingPoint, or DatumPoint object specifying a point on the plane.

normal

A straight Edge or DatumAxis object specifying the normal to the plane.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.41 PartitionCellByPlaneThreePoints(...)

This method partitions one or more cells using a plane defined by three points.

Path

```
mdb.models[name].parts[name].PartitionCellByPlaneThreePoints  
mdb.models[name].rootAssembly.PartitionCellByPlaneThreePoints
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

point1

A Vertex, InterestingPoint, or DatumPoint object specifying a point on the plane.

point2

A Vertex, InterestingPoint, or DatumPoint object specifying a point on the plane.

point3

A Vertex, InterestingPoint, or DatumPoint object specifying a point on the plane.

Note: *point1*, *point2*, and *point3* must not be colinear and must not coincide.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.42 PartitionCellBySweepEdge(...)

This method partitions one or more cells by sweeping selected edges along the given sweep path.

Path

```
mdb.models[name].parts[name].PartitionCellBySweepEdge  
mdb.models[name].rootAssembly.PartitionCellBySweepEdge
```

Required arguments

cells

A sequence of Cell objects specifying the cells to partition.

edges

A sequence of Edge objects to be swept. The edges must be in the same plane. The edges must form a continuous chain without branches. The Edge objects must all belong to the same PartInstance object.

sweepPath

An Edge object specifying the sweep path. The start of *sweepPath* must be in the plane and perpendicular to the plane formed by *edges*. The sweep path must be planar.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.43 PartitionEdgeByDatumPlane(...)

This method partitions an edge where it intersects with a datum plane.

Path

```
mdb.models[name].parts[name].PartitionEdgeByDatumPlane
mdb.models[name].rootAssembly.PartitionEdgeByDatumPlane
```

Required arguments

edges

A sequence of Edge objects specifying the edges to partition.

datumPlane

A DatumPlane object specifying the location of the partition.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.44 PartitionEdgeByParam(...)

This method partitions one or more edges at the given normalized edge parameter.

Path

```
mdb.models[name].parts[name].PartitionEdgeByParam
mdb.models[name].rootAssembly.PartitionEdgeByParam
```

Required arguments

edges

A sequence of Edge objects specifying the edges to partition.

parameter

A Float specifying the normalized distance along *edge* at which to partition. Possible values are $0.0 < parameter < 1.0$.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.45 PartitionEdgeByPoint(...)

This method partitions an edge at the given point.

Path

```
mdb.models[name].parts[name].PartitionEdgeByPoint
mdb.models[name].rootAssembly.PartitionEdgeByPoint
```

Required arguments

edge

An Edge object specifying the edge to partition.

point

An InterestingPoint or DatumPoint object specifying a point on *edge*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.46 PartitionFaceByAuto(...)

This method automatically partitions a target face into simple regions that can be meshed using a structured meshing technique.

Path

```
mdb.models[name].parts[name].PartitionFaceByAuto
mdb.models[name].rootAssembly.PartitionFaceByAuto
```

Required argument

face

A Face object specifying the face to partition.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.47 PartitionFaceByCurvedPathEdgeParams(...)

This method partitions a face normal to two edges, using a curved path between the two given edge points defined by the normalized edge parameters.

Path

```
mdb.models[name].parts[name].PartitionFaceByCurvedPathEdgeParams
mdb.models[name].rootAssembly.PartitionFaceByCurvedPathEdgeParams
```

Required arguments

face

A Face object specifying the face to partition.

edge1

An Edge object specifying the start of the partition. The edge must belong to *face*.

parameter1

A Float specifying the distance along *edge1* at which to partition. Possible values are $0.0 \leq distance1 \leq 1.0$.

edge2

An Edge object specifying the end of the partition. The edge must belong to *face*.

parameter2

A Float specifying the distance along *edge2* at which to partition. Possible values are $0.0 \leq distance2 \leq 1.0$.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.48 PartitionFaceByCurvedPathEdgePoints(...)

This method partitions a face normal to two edges, using a curved path between the two given edge points.

Path

```
mdb.models[name].parts[name].PartitionFaceByCurvedPathEdgePoints  
mdb.models[name].rootAssembly.PartitionFaceByCurvedPathEdgePoints
```

Required arguments

face

A Face object specifying the face to partition.

edge1

An Edge object specifying the start of the partition. The edge must belong to *face*.

point1

A Vertex, InterestingPoint, or DatumPoint object specifying a point on *edge1*.

edge2

An Edge object specifying the end of the partition. The edge must belong to *face*.

point2

A Vertex, InterestingPoint, or DatumPoint object specifying a point on *edge2*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.49 PartitionFaceByDatumPlane(...)

This method partitions one or more faces using the given datum plane.

Path

```
mdb.models[name].parts[name].PartitionFaceByDatumPlane
mdb.models[name].rootAssembly.PartitionFaceByDatumPlane
```

Required arguments

faces

A sequence of Face objects specifying the faces to partition.

datumPlane

A DatumPlane object specifying the location of the partition.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.50 PartitionFaceByExtendFace(...)

This method partitions one or more faces by extending the underlying geometry of another given face to partition the target faces.

Path

```
mdb.models[name].parts[name].PartitionFaceByExtendFace
mdb.models[name].rootAssembly.PartitionFaceByExtendFace
```

Required arguments

faces

A sequence of Face objects specifying the faces to partition.

extendFace

A Face object that is to be extended to create the partition. The face to extend can be a planar, cylindrical, conical, or spherical face.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.51 PartitionFaceByIntersectFace(...)

This method partitions one or more faces using the given cutting faces to partition the target faces.

Path

```
mdb.models[name].parts[name].PartitionFaceByIntersectFace
mdb.models[name].rootAssembly.PartitionFaceByIntersectFace
```

Required arguments

faces

A sequence of Face objects specifying the faces to partition.

cuttingFaces

A sequence of Face objects that specify the cutting faces.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.52 PartitionFaceByProjectingEdges(...)

This method partitions one or more faces by projecting the given edges on the target faces.

Path

```
mdb.models[name].parts[name].PartitionFaceByProjectingEdges
mdb.models[name].rootAssembly.PartitionFaceByProjectingEdges
```

Required arguments*faces*

A sequence of Face objects specifying the faces to partition.

edges

A sequence of Edge objects specifying the edges that will be projected onto the target faces.

Optional argument*extendEdges*

A boolean specifying whether to extend the given edges at their free ends in the tangent direction before partitioning the target faces. The default value is False.

Return value

A Feature object.

Exceptions

None.

20.1.53 PartitionFaceByShortestPath(...)

This method partitions one or more faces using a minimum distance path between the two given points.

Path

```
mdb.models[name].parts[name].PartitionFaceByShortestPath  
mdb.models[name].rootAssembly.PartitionFaceByShortestPath
```

Required arguments

faces

A sequence of Face objects specifying the face to partition.

point1

A Vertex, InterestingPoint, or DatumPoint object.

point2

A Vertex, InterestingPoint, or DatumPoint object.

Note: *point1* and *point2* must not coincide, and they must both lie on the underlying surface geometry of at least one of the target faces.

Optional arguments

None.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.54 PartitionFaceBySketch(...)

This method partitions one or more planar faces by sketching on them.

Path

```
mdb.models[name].parts[name].PartitionFaceBySketch  
mdb.models[name].rootAssembly.PartitionFaceBySketch
```

Required arguments

faces

A sequence of Face objects specifying the faces to partition.

sketch

A ConstrainedSketch object specifying the partition.

Optional arguments*sketchUpEdge*

An Edge or DatumAxis object specifying the orientation of *sketch*. This edge or datum axis must not be orthogonal to the plane defined by *faces*. If unspecified, *sketch* is assumed to be oriented in with the *Y* direction pointing up.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.55 PartitionFaceBySketchDistance(...)

This method partitions one or more faces by sketching on a sketch plane and then projecting the sketch toward the target faces through the given distance.

Path

```
mdb.models[name].parts[name].PartitionFaceBySketchDistance
mdb.models[name].rootAssembly.PartitionFaceBySketchDistance
```

Required arguments*faces*

A sequence of Face objects specifying the faces to partition.

sketchPlane

A planar Face or DatumPlane object.

sketchPlaneSide

A SymbolicConstant specifying the side of the plane to be used for sketching. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object specifying the orientation of *sketch*. This edge must not be orthogonal to *sketchPlane*.

sketch

A ConstrainedSketch object specifying the partition.

distance

A Float specifying the projection distance. Possible values are *distance* > 0.0.

Optional argument

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.56 PartitionFaceBySketchRefPoint(...)

This method partitions one or more faces by sketching on a sketch plane and then projecting the sketch toward the target faces through a distance governed by the reference point.

Path

```
mdb.models[name].parts[name].PartitionFaceBySketchRefPoint  
mdb.models[name].rootAssembly.PartitionFaceBySketchRefPoint
```

Required arguments

faces

A sequence of Face objects specifying the faces to partition.

sketchPlane

A planar Face or DatumPlane object.

sketchUpEdge

An Edge object or a DatumAxis object specifying the orientation of *sketch*. This edge or datum axis must not be orthogonal to *sketchPlane*.

sketch

A ConstrainedSketch object specifying the partition.

point

A Vertex, InterestingPoint, or DatumPoint object specifying the distance to project *sketch*. The point must not lie on *sketchPlane*.

Optional argument

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.57 PartitionFaceBySketchThruAll(...)

This method partitions one or more faces by sketching on a sketch plane and then projecting toward the target faces through an infinite distance.

Path

```
mdb.models[name].parts[name].PartitionFaceBySketchThruAll
mdb.models[name].rootAssembly.PartitionFaceBySketchThruAll
```

Required arguments*faces*

A sequence of Face objects specifying the faces to partition.

sketchPlane

A planar Face or DatumPlane object.

sketchPlaneSide

A SymbolicConstant specifying the extrude direction of the sketch. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge or a DatumAxis object specifying the orientation of *sketch*. This edge or datum axis must not be orthogonal to *sketchPlane*.

sketch

A ConstrainedSketch object specifying the partition.

Optional argument*sketchOrientation*

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

AbaqusException.

20.1.58 ReferencePoint(...)

This method creates a Feature object and a ReferencePoint object at the specified location.

Path

```
mdb.models[name].rootAssembly.ReferencePoint  
mdb.models[name].parts[name].ReferencePoint
```

Required argument

point

A Vertex, InterestingPoint, a MeshNode, or a Datum object specifying a reference point. *point* can also be a sequence of three Floats representing the X-, Y-, and Z-coordinates of the point.

Optional arguments

instanceName

Used internally by the input file writer.

Return value

A Feature Object.

Exceptions

None.

20.1.59 RemoveWireEdges(...)

This method removes wire edges.

Path

```
mdb.models[name].parts[name].RemoveWireEdges  
mdb.models[name].rootAssembly.RemoveWireEdges
```

Required argument

wireEdgeList

A sequence of Edge objects specifying the edges to remove. Any specified edge that is not a wire edge will not be removed.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

20.1.60 WirePolyLine(...)

This method creates an additional Feature object by creating a series of wires joining points in pairs. When such a feature is created at the Part level, then each point can be either a datum point, a vertex, a reference point, an interesting point, an orphan mesh node, or the coordinates of a point. When such a feature is created at the Assembly level, then each point can only be a vertex, a reference point, or an orphan mesh node.

Path

```
mdb.models[name].parts[name].WirePolyLine  
mdb.models[name].rootAssembly.WirePolyLine
```

Required argument*points*

A tuple of point pairs, each pair being itself represented by a tuple. For part level features each point can be a Vertex, Datum point, Reference point, orphan mesh Node, or InterestingPoint object specifying the points through which the polyline wire will pass. Each point can also be a tuple of Floats representing the coordinates of a point. For assembly level features each point can only be a Vertex, Reference point, or orphan mesh Node specifying the points through which the polyline wire will pass (coordinates cannot be specified). In any of the pairs, the first or second point can be NONE. In that case, the point pair will create a zero-length wire, which is required for certain types of connectors. You must specify at least one pair.

Optional arguments*mergeType*

A SymbolicConstant specifying the merge behavior of the wire with existing geometry. If *mergeType* is MERGE, Abaqus merges the wire into solid regions of the part if the wire passes through them. If *mergeType* is IMPRINT, Abaqus imprints the wire on existing geometry as edges. If *mergeType* is SEPARATE, Abaqus neither merges nor imprints the spline wire with existing geometry. It creates the wire separately. The default value is IMPRINT.

FEATURE COMMANDS

meshable

A Boolean specifying whether the wire should be available for selection for meshing operations. If *meshable*=OFF, the wire can be used for connector section assignment. The default value is ON.

Return value

A Feature object.

Exceptions

None.

20.1.61 isSuppressed()

This method queries the suppressed state of the feature.

Arguments

None.

Return value

A Boolean value of True if the feature is suppressed and False if it is not suppressed.

Exceptions

None.

20.1.62 restore()

This method restores the parameters of a feature to the value they had when the **backup** method was invoked on the part or assembly. Use the **restore** method after the **backup** method.

Arguments

None.

Return value

None

Exceptions

None.

20.1.63 **resume()**

This method resumes suppressed features. Resuming a feature fully restores it to the part or assembly. You can resume the last feature you suppressed, all suppressed features, or just selected features. When you resume a child feature, Abaqus/CAE also resumes the parent features automatically.

Arguments

None.

Return value

None

Exceptions

None.

20.1.64 **setValues(...)**

This method modifies the Feature object.

Arguments

Required arguments

None.

Optional arguments

parameter

A Float specifying the normalized distance along *edge* at which to partition. Possible values are $0.0 < \textit{parameter} < 1.0$. You use this argument to modify a partition created with the created with the **PartitionEdgeByParam** method.

parameter1

A Float specifying the distance along *edge1* at which to partition. Possible values are $0.0 \leq \textit{parameter1} \leq 1.0$. You use this argument to modify a partition object created with the **PartitionFaceByCurvedPathEdgeParam** method.

parameter2

A Float specifying the distance along *edge2* at which to partition. Possible values are $0.0 \leq \textit{parameter2} \leq 1.0$. You use this argument to modify a partition object created with the **PartitionFaceByCurvedPathEdgeParam** method.

sketch

A ConstrainedSketch object specifying the partition. You use this argument to modify a partition object created with a sketch; for example, using the **PartitionFaceBySketch** method.

FEATURE COMMANDS

distance

A Float specifying the projection *distance*. Possible values are *distance* > 0.0. You use this argument to modify a partition object created with the **PartitionFaceBySketchDistance** method.

Return value

None

Exceptions

AbaqusException.

20.1.65 suppress()

This method suppresses features. Suppressing a feature is equivalent to temporarily removing the feature from the part or assembly. Suppressed features remain suppressed when you regenerate a part or assembly. You cannot suppress the base feature. In addition, if you suppress a parent feature, all of its child features are also suppressed automatically. Suppressed features can be restored with the **resume** command.

Arguments

None.

Return value

None

Exceptions

None.

20.1.66 Members

The Feature object has the following members:

name

A String specifying the repository key.

id

An Int specifying the ID of the feature.

20.2 FeatureOptions object

The FeatureOptions object stores the options that control the behavior of feature regeneration for all features in a model.

Access

```
import part
mdb.models[name].featureOptions
```

20.2.1 setValues(...)

This method modifies the FeatureOptions object for the specified model.

Arguments

Required arguments

None.

Optional arguments

checkSelfIntersection

A Boolean specifying whether Abaqus/CAE should perform self-intersection checks while regenerating features. The default value is ON.

autoCaching

A Boolean specifying whether geometric states should be automatically cached. The default value is ON.

maxCachedStates

An Int specifying the maximum number of caches to be stored with each part or with the assembly. The default value is 5.

Return value

None

Exceptions

None.

20.2.2 Members

The FeatureOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

21. Field commands

A Field object stores the non-propagating data of a field as well as a number of instances of the corresponding FieldState object. The FieldState object stores the propagating data of the field in a single step. A specific type of Field object and a specific type of FieldState object are designed for each type of predefined field. Instances of the FieldState object are created and deleted internally by its corresponding Field object.

21.1 Field object

The Field object is the abstract base type for other Field objects. The Field object has no explicit constructor. The methods and members of the Field object are common to all objects derived from the Field.

Access

```
import fields
```

21.1.1 Members

The Field object can have the following members:

name

A String specifying the repository key.

description

A String specifying the description of the field. The default value is an empty string.

21.2 AnalyticalField object

The AnalyticalField object is the abstract base type for other AnalyticalField objects. The AnalyticalField object has no explicit constructor. The methods and members of the AnalyticalField object are common to all objects derived from the AnalyticalField.

The AnalyticalField object is derived from the Field object.

Access

```
import fields
mdb.models[name].analyticalFields[name]
```

21.2.1 Members

The AnalyticalField object can have the following members:

name

A String specifying the repository key.

localCsys

None or a DatumCsys object specifying the local coordinate system of the field. If *localCsys=None*, the field is defined in the global coordinate system. The default value is **None**.

description

A String specifying the description of the field. The default value is an empty string.

21.3 DataTable object

A DataTable is an object used to define the domain and data for a DiscreteField .

Access

```
import field
mdb.models[name].discreteFields[name].data[i]
```

21.3.1 Members

The DataTable object has the following members:

dataWidth

An Int specifying the width of the data. Valid widths are 1, 6, 21, corresponding to scalar data, orientations and 4D tensors.

name

A String specifying the index.

instanceName

A String specifying the instance name.

domain

A tuple of Ints specifying the domain node, element or integration point identifiers.

table

A tuple of Floats specifying the data within the domain.

21.4 DiscreteField object

The DiscreteField object defines a varying field whose values correspond to distinct points within a domain.

The DiscreteField object is derived from the Field object.

Access

```
import fields
mdb.models[name].discreteFields[name]
```

21.4.1 DiscreteField(...)

This method creates a DiscreteField object.

Path

```
mdb.models[name].DiscreteField
```

Required arguments

name

A String specifying the repository key.

defaultValues

A sequence of Floats specifying a sequence of floats specifying the default values.

fieldType

A SymbolicConstant or an Int specifying the type of data represented by this discrete field. Possible values are SCALAR, ORIENTATION, and PRESCRIBEDCONDITION_DOF.

Optional arguments

location

A SymbolicConstant or an Int specifying the location of the domain data. Possible values are NODES and ELEMENTS. The default value is NODES.

dataWidth

An Int specifying the width of the supplied data. The default value is 1.

data

A DataTableArray object.

description

A String specifying the description of the field. The default value is an empty string.

orientationType

A SymbolicConstant specifying the type of the system being described by a discrete field used for an orientation. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL. The default value is CARTESIAN.

partLevelOrientation

A Boolean specifying whether or not the orientations are described in terms of part level coordinates. The default value is OFF.

Return value

A DiscreteField object.

Exceptions

AbaqusException.

21.4.2 DiscreteFieldByVolumeFraction(...)

This method creates a DiscreteField object that represents the volume fraction of each element of an Eulerian Instance that is occupied by a reference instance.

Path

```
mdb.models[name].rootAssembly.DiscreteFieldByVolumeFraction
```

Required arguments

name

A String specifying the repository key.

eulerianInstance

A PartInstance object specifying the elements for which volume fraction values will be computed.

referenceInstance

A PartInstance object specifying the region that either contains material or is empty of material.

Optional arguments

accuracy

A Symbolic Constant specifying the level of accuracy that will be used in computing volume fractions. Possible values are LOW, MEDIUM, or HIGH. The default value is MEDIUM.

materialLocation

A Symbolic Constant indicating whether the material is inside or outside the *referenceInstance*. Possible values are INSIDE or OUTSIDE. The default value is INSIDE.

description

A String specifying the description of the field. The default value is an empty string.

scaleFactor

A float specifying the fraction of the volume that is occupied by the *referenceInstance*. Valid values are between 0 and 1.

Return value

A DiscreteField object.

Exceptions

AbaqusException.

21.4.3 DiscreteFieldFromAnalytic(...)

This method creates a DiscreteField object from a AnalyticalField object.

Path

`mdb.models[name].DiscreteFieldFromAnalytic`

Required arguments*name*

A String specifying the repository key.

location

A SymbolicConstant or an Int specifying the location of the domain data. Possible values are NODES and ELEMENTS. The default value is NODES.

analyticFieldName

A String specifying the name of the AnalyticalField containing the source data.

region

A Region object for the field.

Optional arguments

None.

Return value

A DiscreteField object.

Exceptions

AbaqusException.

21.4.4 **setValues(...)**

This method modifies the `DiscreteField` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DiscreteField** method, except for the *name* argument.

Return value

None

Exceptions

21.4.5 **Members**

The `DiscreteField` object has members with the same names and descriptions as the arguments to the **DiscreteField** method.

21.5 **ExpressionField object**

The `ExpressionField` object defines a spatially varying field whose value is calculated from a user-supplied mathematical expression.

The `ExpressionField` object is derived from the `AnalyticalField` object.

Access

```
import fields
mdb.models[name].analyticalFields[name]
```

21.5.1 **ExpressionField(...)**

This method creates an `ExpressionField` object.

Path

```
mdb.models[name].ExpressionField
```

Required arguments*name*

A String specifying the repository key.

expression

A String specifying the Python expression to evaluate in space. Variables are X, Y, and Z; R, Th, and Z; or R, Th, and P based on the selected coordinate system.

Optional arguments*localCsys*

None or a DatumCsys object specifying the local coordinate system of the field. If *localCsys*=**None**, the field is defined in the global coordinate system. The default value is **None**.

description

A String specifying the description of the field. The default value is an empty string.

Return value

An ExpressionField object.

Exceptions

TextException.

21.5.2 setValues(...)

This method modifies the ExpressionField object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ExpressionField** method, except for the *name* argument.

Return value

None

Exceptions**21.5.3 Members**

The ExpressionField object has members with the same names and descriptions as the arguments to the **ExpressionField** method.

21.6 MappedField object

The MappedField object defines a spatially varying field whose value is calculated from an external source data.

The MappedField object is derived from the AnalyticalField object.

Access

```
import fields
mdb.models[name].analyticalFields[name]
```

21.6.1 MappedField(...)

This method creates an MappedField object.

Path

```
mdb.models[name].MappedField
```

Required argument

name

A String specifying the repository key.

Optional arguments

regionType

A SymbolicConstant specifying the data source region type. It can be either an ODB mesh or a cloud of points. Possible values are MESH and POINT. The default value is POINT.

partLevelData

A Boolean specifying whether or not the point cloud source data are described in terms of part level coordinates. If part level coordinates is employed, the local coordinate system defined in *localCsys* will be ignored. The default value is OFF.

pointDataFormat

A SymbolicConstant specifying point cloud source data format. Possible values are GRID and XYZ. The default value is XYZ.

gridPointPlane

A SymbolicConstant specifying the plane on which the point cloud source data of grid format are described. Possible values are XYPLANE, YZPLANE, and XZPLANE. The default value is XYPLANE.

defaultUnmappedValue

A Float specifying default parameter (field) value of target model region while its value cannot be calculated from the data source. The default value is 0.0.

mappingAlgorithm

A SymbolicConstant specifying the mapping algorithm for target surface, or on mesh target model when the parameter value are located at nodes, for example nodal temperatures. Possible values are SURFACE and VOLUMETRIC. The default value is SURFACE.

searchTolType

A SymbolicConstant specifying searching tolerance type in terms of absolute value or a fraction of the average of all element characteristic length in target model region. Possible values are ABSOLUTE and RELATIVE. The default value is RELATIVE.

boundarySearchTol

A Float specifying the search distance tolerance value on the exterior boundary of target model region. Source points within this distance will be included in computing the parameter value of target region. This tolerance applies to both surface and volumetric mapping. The default value is 0.01.

neighborhoodSearchTol

A Float specifying the search distance tolerance value used for distance weighting algorithm. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 1000000.0.

negativeNormalSearchTol

A Float specifying the search distance tolerance value in the negative normal of target surface region. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 0.15.

positiveNormalSearchTol

A Float specifying the search distance tolerance value in the positive normal of target surface region. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 0.05.

scaleCoordinates

A Boolean specifying whether or not to scale the user-supplied coordinate values from the point cloud or indicated ODB. The default value is OFF.

FIELD COMMANDS

gridPointData

A sequence of sequences of Floats specifying the point cloud source data of grid format. The default value is an empty sequence.

xyzPointData

A sequence of sequences of Floats specifying the point cloud source data of XYZ format. Each data item is defining the XYZ coordinates of a point and its field value. The default value is an empty sequence.

coordinateScalingFactors

A sequence of Floats specifying the scaling factors for the global 1, 2 and 3 directions. The default value is (1.0, 1.0, 1.0).

localCsys

None or a DatumCsys object specifying the local coordinate system of the field. If *localCsys*=**None**, the field is defined in the global coordinate system. The default value is **None**.

description

A String specifying the description of the field. The default value is an empty string.

Return value

A MappedField object.

Exceptions

AbaqusException.

21.6.2 setValues(...)

This method modifies the MappedField object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MappedField** method, except for the *name* argument.

Return value

None

Exceptions

21.6.3 Members

The MappedField object can have the following members:

name

A String specifying the repository key.

regionType

A SymbolicConstant specifying the data source region type. It can be either an ODB mesh or a cloud of points. Possible values are MESH and POINT. The default value is POINT.

partLevelData

A Boolean specifying whether or not the point cloud source data are described in terms of part level coordinates. If part level coordinates is employed, the local coordinate system defined in *localCsys* will be ignored. The default value is OFF.

pointDataFormat

A SymbolicConstant specifying point cloud source data format. Possible values are GRID and XYZ. The default value is XYZ.

gridPointPlane

A SymbolicConstant specifying the plane on which the point cloud source data of grid format are described. Possible values are XYPLANE, YZPLANE, and XZPLANE. The default value is XYPLANE.

defaultUnmappedValue

A Float specifying default parameter (field) value of target model region while its value cannot be calculated from the data source. The default value is 0.0.

mappingAlgorithm

A SymbolicConstant specifying the mapping algorithm for target surface, or on mesh target model when the parameter value are located at nodes, for example nodal temperatures. Possible values are SURFACE and VOLUMETRIC. The default value is SURFACE.

searchTolType

A SymbolicConstant specifying searching tolerance type in terms of absolute value or a fraction of the average of all element characteristic length in target model region. Possible values are ABSOLUTE and RELATIVE. The default value is RELATIVE.

boundarySearchTol

A Float specifying the search distance tolerance value on the exterior boundary of target model region. Source points within this distance will be included in computing the parameter value of target region. This tolerance applies to both surface and volumetric mapping. The default value is 0.01.

neighborhoodSearchTol

A Float specifying the search distance tolerance value used for distance weighting algorithm. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 1000000.0.

negativeNormalSearchTol

A Float specifying the search distance tolerance value in the negative normal of target surface region. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 0.15.

positiveNormalSearchTol

A Float specifying the search distance tolerance value in the positive normal of target surface region. Source points within this distance will be included in computing the parameter value of target region. This tolerance only applies to surface mapping. The default value is 0.05.

scaleCoordinates

A Boolean specifying whether or not to scale the user-supplied coordinate values from the point cloud or indicated ODB. The default value is OFF.

gridPointData

A tuple of tuples of Floats specifying the point cloud source data of grid format. The default value is an empty sequence.

xyzPointData

A tuple of tuples of Floats specifying the point cloud source data of XYZ format. Each data item is defining the XYZ coordinates of a point and its field value. The default value is an empty sequence.

odbMeshRegionData

An OdbMeshRegionData object specifying the external source data from ODB mesh region.

coordinateScalingFactors

A tuple of Floats specifying the scaling factors for the global 1, 2 and 3 directions. The default value is (1.0, 1.0, 1.0).

localCsys

None or a DatumCsys object specifying the local coordinate system of the field. If *localCsys*=**None**, the field is defined in the global coordinate system. The default value is **None**.

description

A String specifying the description of the field. The default value is an empty string.

21.7 OdbMeshRegionData object

The OdbMeshRegionData object defines the external source data of MappedField from an ODB file.

Access

`import field`


```
mdb.models[name].analyticalFields[name].odbMeshRegionData
```

21.7.1 OdbMeshRegionData(...)

This method creates an OdbMeshRegionData object.

Path

```
mdb.models[name].analyticalFields[name].OdbMeshRegionData
```

Required arguments

odbFileName

A String specifying the name of the output database file (including the .odb extension) to be read into as the source data. This String can also be the full path to the output database file if it is located in another directory.

variableLabel

A String specifying the field output variable.

Optional arguments

stepIndex

An Int specifying the step index. Possible values are $0 \leq \text{stepIndex} \leq (\text{numSteps} - 1)$. The default value is 0.

frameIndex

An Int specifying the frame in the specified step. Valid values are $0 \leq \text{frameIndex} \leq (\text{numFramesInStep} - 1)$. The default value is 0.

outputPosition

A SymbolicConstant specifying the position where the data is written in the output database. Data can be obtained only from the position at which it was written to the output database during the analysis. This position should be aligned with the field output variable. Possible values are:

- UNDEFINED_POSITION
- NODAL
- INTEGRATION_POINT
- ELEMENT_FACE
- ELEMENT_NODAL
- ELEMENT_CENTROID
- WHOLE_ELEMENT
- WHOLE_REGION
- WHOLE_PART_INSTANCE
- WHOLE_MODEL
- GENERAL_PARTICLE

FIELD COMMANDS

The default value is UNDEFINED_POSITION.

dataType

A SymbolicConstant specifying the data type of the field output variable which should be aligned with the variable. Currently only SCALAR is supported. Possible values are:

- ENUMERATION
- BOOLEAN
- INTEGER
- SCALAR
- VECTOR
- QUATERNION_2D
- QUATERNION_3D
- TENSOR
- TENSOR_3D_FULL
- TENSOR_3D_PLANAR
- TENSOR_3D_SURFACE
- TENSOR_2D_PLANAR
- TENSOR_2D_SURFACE

The default value is SCALAR.

storageType

A SymbolicConstant specifying the storage type of the field output variable which should be aligned with the variable. Possible values are FLOAT, DOUBLE, INTEGER, and BOOLEAN. The default value is FLOAT.

quantityToPlot

A SymbolicConstant specifying the quantity to plot. Currently only FIELD_OUTPUT is supported. Possible values are FIELD_OUTPUT and DISCONTINUITIES. The default value is FIELD_OUTPUT.

averageElementOutput

A Boolean specifying whether to average the element output. The default value is OFF.

useRegionBoundaries

A Boolean specifying whether to use region boundaries when averaging. The default value is OFF.

regionBoundaries

A SymbolicConstant specifying the type of averaging region boundaries. Currently only NONE and ODB_REGIONS are supported. Possible values are NONE, ODB_REGIONS, ELEMENT_SET, and DISPLAY_GROUPS. The default value is NONE.

includeFeatureBoundaries

A Boolean specifying whether to include additional averaging boundaries for shells and membranes based on feature edges. The default value is ON.

featureAngle

A Float specifying the feature angle to be used when *includeFeatureBoundaries*=ON. The default value is 20.0.

averageOnlyDisplayed

A Boolean specifying whether to average only values on displayed elements. The default value is OFF.

averagingThreshold

A Float specifying the nodal averaging threshold percentage. $0 \leq \text{averagingThreshold} \leq 100$. The default value is 75.0.

computeOrder

A SymbolicConstant specifying the order or the computations to be performed on the interested field output variable. Possible values are EXTRAPOLATE_AVERAGE_COMPUTE, EXTRAPOLATE_COMPUTE_AVERAGE, EXTRAPOLATE_COMPUTE, EXTRAPOLATE_COMPUTE_DISCONTINUITIES, and RAW_DATA. The default value is EXTRAPOLATE_COMPUTE_AVERAGE.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_MAG_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_MAG_AT_ANGLE=COMPLEX_MAG_AT_ANGLE. The default value is 0.0.

transformationType

A SymbolicConstant specifying the type of the transformation to apply to the output. Possible values are DEFAULT, NODAL, ANGULAR, and LAYUP_ORIENTATION. The default is DEFAULT.

sectionPoint

A Dictionary with String keys and String values. Each key specifies a region in the model; the corresponding value specifies a section point within that region. For example:

```
sectionPoint={'shell < MAT > < 7 section points >'  
(fraction = 1.0)', 'shell < MAT > < 5 section points >'  
'SPOS, (fraction = 1.0)', }
```

refinementType

A SymbolicConstant specifying the type of the FieldOutput object. Possible values for the SymbolicConstant are NO_REFINEMENT, INVARIANT and COMPONENT. Default argument

FIELD COMMANDS

is NO_REFINEMENT. *refinementType* is mandatory if *variableLabel* has an INVARIANT or a COMPONENT.

refinementLabel

A String specifying the Label of FieldOutput object. This is required only if the *refinementType* is INVARIANT or COMPONENT.

displayOutputPosition

A SymbolicConstant specifying the position from which to obtain the data. Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, ELEMENT_CENTROID, WHOLE_ELEMENT, WHOLE_REGION, WHOLE_PART_INSTANCE, WHOLE_MODEL, and GENERAL_PARTICLE.

Return value

An OdbMeshRegionData object.

Exceptions

TextException.

21.7.2 setValues(...)

This method modifies the OdbMeshRegionData object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **OdbMeshRegionData** method.

Return value

None

Exceptions

21.7.3 Members

The OdbMeshRegionData object has the following members:

stepIndex

An Int specifying the step index. Possible values are $0 \leq \text{stepIndex} \leq (\text{numSteps} - 1)$. The default value is 0.

frameIndex

An Int specifying the frame in the specified step. Valid values are $0 \leq \text{frameIndex} \leq (\text{numFramesInStep} - 1)$. The default value is 0.

outputPosition

A SymbolicConstant specifying the position from which to obtain data. Data can be obtained only from the position at which they were written to the output database during the analysis. This position should be aligned with the field output variable. Possible values are:

- UNDEFINED_POSITION
- NODAL
- INTEGRATION_POINT
- ELEMENT_FACE
- ELEMENT_NODAL
- ELEMENT_CENTROID
- WHOLE_ELEMENT
- WHOLE_REGION
- WHOLE_PART_INSTANCE
- WHOLE_MODEL
- GENERAL_PARTICLE

The default value is UNDEFINED_POSITION.

dataType

A SymbolicConstant specifying the data type of the field output variable which should be aligned with the variable. Currently only SCALAR is supported. Possible values are:

- ENUMERATION
- BOOLEAN
- INTEGER
- SCALAR
- VECTOR
- QUATERNION_2D
- QUATERNION_3D
- TENSOR
- TENSOR_3D_FULL
- TENSOR_3D_PLANAR
- TENSOR_3D_SURFACE
- TENSOR_2D_PLANAR
- TENSOR_2D_SURFACE

The default value is SCALAR.

FIELD COMMANDS

storageType

A SymbolicConstant specifying the storage type of the field output variable which should be aligned with the variable. Possible values are FLOAT, DOUBLE, INTEGER, and BOOLEAN. The default value is FLOAT.

quantityToPlot

A SymbolicConstant specifying the quantity to plot. Currently only FIELD_OUTPUT is supported. Possible values are FIELD_OUTPUT and DISCONTINUITIES. The default value is FIELD_OUTPUT.

averageElementOutput

A Boolean specifying whether to average the element output. The default value is OFF.

useRegionBoundaries

A Boolean specifying whether to use region boundaries when averaging. The default value is OFF.

regionBoundaries

A SymbolicConstant specifying the type of averaging region boundaries. Currently only NONE and ODB_REGIONS are supported. Possible values are NONE, ODB_REGIONS, ELEMENT_SET, and DISPLAY_GROUPS. The default value is NONE.

includeFeatureBoundaries

A Boolean specifying whether to include additional averaging boundaries for shells and membranes based on feature edges. The default value is ON.

featureAngle

A Float specifying the feature angle to be used when *includeFeatureBoundaries*=ON. The default value is 20.0.

averageOnlyDisplayed

A Boolean specifying whether to average only values on displayed elements. The default value is OFF.

averagingThreshold

A Float specifying the nodal averaging threshold percentage. $0 \leq \text{averagingThreshold} \leq 100$. The default value is 75.0.

computeOrder

A SymbolicConstant specifying the order or the computations to be performed on the interested field output variable. Possible values are EXTRAPOLATE_AVERAGE_COMPUTE, EXTRAPOLATE_COMPUTE_AVERAGE, EXTRAPOLATE_COMPUTE, EXTRAPOLATE_COMPUTE_DISCONTINUITIES, and RAW_DATA. The default value is EXTRAPOLATE_COMPUTE_AVERAGE.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_MAG_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=*COMPLEX_MAG_AT_ANGLE*=*COMPLEX_MAG_AT_ANGLE*. The default value is 0.0.

odbFileName

A String specifying the name of the output database file (including the .odb extension) to be read into as the source data. This String can also be the full path to the output database file if it is located in another directory.

variableLabel

A String specifying the field output variable.

transformationType

The default is Default(Respect element orientations but suppress nodal transformations)

displayOutputPosition

A SymbolicConstant specifying the position where the output is displayed in the viewport. Possible values are:

- UNDEFINED_POSITION
- NODAL
- INTEGRATION_POINT
- ELEMENT_FACE
- ELEMENT_NODAL
- ELEMENT_CENTROID
- WHOLE_ELEMENT
- WHOLE_REGION
- WHOLE_PART_INSTANCE
- WHOLE_MODEL
- GENERAL_PARTICLE

The default value is UNDEFINED_POSITION.

22. Filter commands

Filter commands are used to create real-time filters of output request data.

22.1 Filter object

The Filter object is the abstract base type for other Filter objects. The Filter object has no explicit constructor. The methods and members of the Filter object are common to all objects derived from the Filter.

Access

```
import filter
mdb.models[name].filters[name]
import odbFilter
session.odbs[name].filters[name]
```

22.1.1 Members

The Filter object has the following members:

name

A String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Float specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

A SymbolicConstant specifying the filter operator. Possible values are NONE, MIN, MAX, and ABS. The default value is NONE.

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is OFF.

limit

None or a Float specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is **None**.

invariant

A SymbolicConstant specifying the invariant to which filtering is applied. Possible values are NONE, FIRST, and SECOND. The default value is NONE.

22.2 ButterworthFilter object

The ButterworthFilter object defines a Butterworth type filter.

The ButterworthFilter object is derived from the Filter object.

Access

```
import filter
mdb.models[name].filters[name]
import odbFilter
session.odbs[name].filters[name]
```

22.2.1 ButterworthFilter(...)

This method creates a ButterworthFilter object.

Path

```
mdb.models[name].ButterworthFilter
session.odbs[name].ButterworthFilter
```

Required arguments

name

A String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Float specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

A SymbolicConstant specifying the filter operator. Possible values are NONE, MIN, MAX, and ABS. The default value is NONE.

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is OFF.

limit

None or a Float specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is **None**.

invariant

A SymbolicConstant specifying the invariant to which filtering is applied. Possible values are NONE, FIRST, and SECOND. The default value is NONE.

Return value

A ButterworthFilter object.

Exceptions

InvalidNameError and RangeError.

22.2.2 setValues(...)

This method modifies the ButterworthFilter object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ButterworthFilter** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

22.2.3 Members

The ButterworthFilter object has members with the same names and descriptions as the arguments to the **ButterworthFilter** method.

22.2.4 Corresponding analysis keywords

*FILTER

22.3 Chebyshev1Filter object

The Chebyshev1Filter object defines a Chebyshev type 1 filter.

The Chebyshev1Filter object is derived from the Filter object.

Access

```
import filter
mdb.models[name].filters[name]
import odbFilter
session.odbs[name].filters[name]
```

22.3.1 Chebyshev1Filter(...)

This method creates a Chebyshev1Filter object.

Path

```
mdb.models[name].Chebyshev1Filter
session.odbs[name].Chebyshev1Filter
```

Required arguments

name

A String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Float specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments

rippleFactor

A Float specifying the amount of allowable ripple in the filter. Possible values are non-negative numbers. The default value is 0.225.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for `OperatorFilter`. The default value is 2.

operation

A `SymbolicConstant` specifying the filter operator. Possible values are `NONE`, `MIN`, `MAX`, and `ABS`. The default value is `NONE`.

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is `OFF`.

limit

None or a Float specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when `Halt` is used. The default value is **None**.

invariant

A `SymbolicConstant` specifying the invariant to which filtering is applied. Possible values are `NONE`, `FIRST`, and `SECOND`. The default value is `NONE`.

Return value

A `Chebyshev1Filter` object.

Exceptions

`InvalidNameError` and `RangeError`.

22.3.2 `setValues(...)`

This method modifies the `Chebyshev1Filter` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **`setValues`** are the same as the arguments to the **`Chebyshev1Filter`** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

22.3.3 Members

The `Chebyshev1Filter` object has members with the same names and descriptions as the arguments to the `Chebyshev1Filter` method.

22.3.4 Corresponding analysis keywords

*FILTER

22.4 Chebyshev2Filter object

The `Chebyshev2Filter` object defines a Chebyshev type 2 filter.

The `Chebyshev2Filter` object is derived from the `Filter` object.

Access

```
import filter
mdb.models[name].filters[name]
import odbFilter
session.odbs[name].filters[name]
```

22.4.1 Chebyshev2Filter(...)

This method creates a `Chebyshev2Filter` object.

Path

```
mdb.models[name].Chebyshev2Filter
session.odbs[name].Chebyshev2Filter
```

Required arguments

name

A String specifying the repository key. This name `ANTIALIASING` is reserved for filters generated internally by the program.

cutoffFrequency

A Float specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments*rippleFactor*

A Float specifying the amount of allowable ripple in the filter. Possible values are non-negative numbers less than 1. The default value is 0.025.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

A SymbolicConstant specifying the filter operator. Possible values are NONE, MIN, MAX, and ABS. The default value is NONE.

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is OFF.

limit

None or a Float specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is **None**.

invariant

A SymbolicConstant specifying the invariant to which filtering is applied. Possible values are NONE, FIRST, and SECOND. The default value is NONE.

Return value

A Chebyshev2Filter object.

Exceptions

InvalidNameError and RangeError.

22.4.2 setValues(...)

This method modifies the Chebyshev2Filter object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Chebyshev2Filter** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

22.4.3 Members

The **Chebyshev2Filter** object has members with the same names and descriptions as the arguments to the **Chebyshev2Filter** method.

22.4.4 Corresponding analysis keywords

*FILTER

22.5 OperatorFilter object

The **OperatorFilter** object defines an operator filter.

The **OperatorFilter** object is derived from the **Filter** object.

Access

```
import filter
mdb.models[name].filters[name]
import odbFilter
session.odbs[name].filters[name]
```

22.5.1 OperatorFilter(...)

This method creates an **OperatorFilter** object.

Path

```
mdb.models[name].OperatorFilter
```


`session.odbs[name].OperatorFilter`

Required arguments

name

A String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Float specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

A SymbolicConstant specifying the filter operator. Possible values are NONE, MIN, MAX, and ABS. The default value is NONE.

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is OFF.

limit

None or a Float specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is **None**.

invariant

A SymbolicConstant specifying the invariant to which filtering is applied. Possible values are NONE, FIRST, and SECOND. The default value is NONE.

Return value

An OperatorFilter object.

Exceptions

InvalidNameError and RangeError.

22.5.2 setValues(...)

This method modifies the OperatorFilter object.

FILTER COMMANDS

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **OperatorFilter** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

22.5.3 Members

The **OperatorFilter** object has members with the same names and descriptions as the arguments to the **OperatorFilter** method.

22.5.4 Corresponding analysis keywords

*FILTER

23. History and Field Report commands

23.1 FieldReportOptions object

The `FieldReportOptions` object stores settings used by the `writeFieldReport` method when you write a `FieldOutput` object to an ASCII file. The `FieldReportOptions` object has no constructor. Abaqus creates the *fieldReportOptions* member when you import the Visualization module.

Access

```
import visualization
session.defaultFieldReportOptions
session.fieldReportOptions
```

23.1.1 setValues(...)

This method modifies the `FieldReportOptions` object.

Arguments

Required arguments

None.

Optional arguments

numColumns

An Int specifying the number of columns to display for the tabular report. The default value is 80.

numberFormat

A `NumberFormat` object specifying the format type, number of digits and precision used to print the numeric output.

printXYData

A Boolean specifying whether to include X–Y data values in the tabular report. The default value is ON.

printTotal

A Boolean specifying whether to include column totals in the tabular report. The default value is ON.

printMinMax

A Boolean specifying whether to include column summary minimum and maximum values in the tabular report. The default value is ON.

HISTORY AND FIELD REPORT COMMANDS

pageWidth

A SymbolicConstant specifying how the width of the tabular report is to be determined. Possible values are NO_LIMIT and SPECIFY. The default value is NO_LIMIT.

columnLayout

A SymbolicConstant specifying how values are to be presented in the tabular report. Possible values are SINGLE_TABLE and SEPARATE_TABLES. The default value is SINGLE_TABLE.

sort

A SymbolicConstant specifying the order in which values are to be sorted within a tabular report. Possible values are ASCENDING and DESCENDING. The default value is ASCENDING.

Return value

A FieldReportOptions object.

Exceptions

None.

23.1.2 Members

The FieldReportOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

23.2 FreeBodyReportOptions object

The FreeBodyReportOptions object stores settings used by the **writeFreeBodyReport** method when you write free body computational results to an ASCII file. The FreeBodyReportOptions object has no constructor. Abaqus creates the *freeBodyReportOptions* member when you import the Visualization module.

Access

```
import visualization
session.defaultFreeBodyReportOptions
session.freeBodyReportOptions
```

23.2.1 setValues(...)

This method modifies the FreeBodyReportOptions object.

Arguments**Required arguments**

None.

Optional arguments

numDigits

An Int specifying the number of decimal places. The default value is 3.

forceThreshold

A Float specifying the threshold value for force. The default value is 10^{-6} .

momentThreshold

A Float specifying the threshold value for moment. The default value is 10^{-6} .

numberFormat

A SymbolicConstant specifying the number format. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

reportFormat

A SymbolicConstant specifying the report format. Possible values are NORMAL_ANNOTATED and COMMA_SEPARATED_VALUES. The default value is NORMAL_ANNOTATED.

csysType

A SymbolicConstant specifying the coordinate system type. Possible values are GLOBAL and LOCAL. The default value is GLOBAL.

Return value

A FreeBodyReportOptions object.

Exceptions

None.

23.2.2 Members

The FreeBodyReportOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

23.3 OdbFieldVarList object

The read-only OdbFieldVarList object is a sequence listing all variables available for the current step and frame. Each item in the sequence is itself a sequence fully describing the given variable.

Access

```
import visualization
session.viewports[name].layers[name].odbDisplay.fieldVariables
session.viewports[name].odbDisplay.fieldVariables
```

23.3.1 Members

The OdbFieldVarList object has no members.

23.4 OdbModelFieldVarList object

The read-only OdbModelFieldVarList object lists all variables available for the model in the current OdbDisplay object.

Access

```
import visualization
session.viewports[name].layers[name].odbDisplay.modelVariableList
session.viewports[name].odbDisplay.modelVariableList
```

23.4.1 Members

The OdbModelFieldVarList object has no members.

23.5 writeFieldReport

This command writes a field output report to a file.

23.5.1 writeFieldReport(...)

This method writes a FieldOutput object to a user-defined ASCII file.

Path

```
session.writeFieldReport
```

Arguments

Required arguments

filename

A String specifying the name of the file to which field output will be written.

append

A Boolean specifying whether to append the field output to an existing file. The default value is ON.

sortItem

A String specifying the item by which to sort the tabular values.

odb

An Odb object from which to obtain field output values.

step

An Int (or *stepIndex*) specifying the step from which to obtain field output values. Possible values are $0 \leq \text{step} \leq (\text{numSteps} - 1)$.

frame

An Int (or *frameIndex*) specifying the frame from which to obtain field output values. Possible values are $0 \leq \text{frame} \leq (\text{numFramesInStep} - 1)$.

outputPosition

A SymbolicConstant specifying the position from which to obtain data. Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, ELEMENT_CENTROID, WHOLE_ELEMENT, WHOLE_REGION, WHOLE_PART_INSTANCE, WHOLE_MODEL, and GENERAL_PARTICLE.

displayGroup

A DisplayGroup object specifying the subset of the model for which to obtain data.

variable

A sequence of variable description sequences specifying one or more field output variables for which to obtain data. Each variable description sequence contains the following elements:

- *element0*: A String specifying the name of the variable.
- *element1*: A SymbolicConstant specifying the output position at which to report data. Possible values are ELEMENT_CENTROID, ELEMENT_FACE, ELEMENT_NODAL, GENERAL_PARTICLE, INTEGRATION_POINT, NODAL, WHOLE_ELEMENT, WHOLE_MODEL, WHOLE_PART_INSTANCE, and WHOLE_REGION.
- *element2*: A Sequence of tuples each consisting of a SymbolicConstant specifying the refinement (COMPONENT or INVARIANT), followed by a String specifying the name of a component or invariant for which to obtain values.
If this element is omitted, data are written for all components and invariants (if applicable). This element is required if *element3* (the following element in the tuple) is included.
- *element3 (if applicable)*: A Dictionary with a String key and a String value specifying a single section point at which to report data. The key specifies a region in the model; the corresponding value specifies a section point within that region. For example:

```
{'shell < MAT > < 7 section points >':'SPOS,
  (fraction = 1.0)'}
```

If this element is omitted, data are written for all section points (if applicable).

Optional arguments

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_MAG_AT_ANGLE. The initial value is COMPLEX_MAGNITUDE.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_MAG_AT_ANGLE. The initial value is 0.

Return value

None

Exceptions

None.

23.6 writeFreeBodyReport

This command writes a free body output report to a file.

23.6.1 writeFreeBodyReport(...)

This method writes a FreeBody object to a user-defined ASCII file.

Path

`session.writeFreeBodyReport`

Arguments

Required arguments

fileName

A String specifying the name of the file to which the free body output will be written.

append

A Boolean specifying whether to append the free body output to an existing file. The default value is ON.

Optional arguments

step

An Int identifying the step from which to obtain values. The default value is the current step.

frame

An Int identifying the frame from which to obtain values. The default value is the current frame.

stepFrame

A SymbolicConstant indicating whether to obtain the values from the specified frame or from all active frames. Possible values are SPECIFY and ALL. The default value is SPECIFY.

odb

An Odb object specifying the output database from which data will be read.

24. Input File Parser commands

The `inpParser` module provides functions that allow you to parse an Abaqus input file.

24.1 InputFile object

The `InputFile` object is used to store the definitions in an Abaqus input file. `InputFile` objects can be created using the methods described in this section.

Access

```
import inpParser
```

24.1.1 InputFile(...)

This method creates an `InputFile` object by reading an Abaqus input file.

Path

```
inpParser.InputFile
```

Required argument

file

A String specifying the path to the input file.

Optional argument

directory

A String specifying the path to the directory containing the input file.

Return value

An `InputFile` object.

Exceptions

None.

24.1.2 parse(...)

This method parses the input file associated with the `InputFile` object.

Optional arguments

organize

A Boolean specifying whether keywords should be organized into suboptions. The default is True.

verbose

A Boolean specifying whether verbose output is to be printed. If *verbose* is True, information about fatal errors is printed. If no fatal errors occur, there is no output. The default is False.

bulk

A Boolean specifying whether the input file includes bulk data that should be parsed. The default is True.

usePyArray

A Boolean specifying that **parse** method can return an AbaqusNDarray object for a keyword data value. In cases where large amounts of numerical data (i.e., large node arrays) are expected, it is recommended that you use the option **usePyArray=True**.

Return value

A KeywordSequence object.

Exceptions

If you parse an input file more than once, a ValueError is raised for each subsequent parsing.

24.1.3 Members

The InputFile object has the following members:

file

A String specifying the source file name of the Abaqus input file.

directory

A String specifying the directory where the input file is located.

includes

A sequence of Strings specifying any additional input files included in the specified input file.

missingIncludes

A sequence of Strings for input files included in the specified input file that could not be located.

24.2 Keyword object

The Keyword object is used to store a keyword definition from an Abaqus input file.

Keyword objects are returned via the **InputFile.parse()** method.

Access

```
import inpParser
```

24.2.1 Members

The InputFile object has the following members:

name

A String specifying the name of the keyword.

parameter

A Dictionary of Strings specifying the keyword parameters.

data

A sequence of sequences or an AbaqusNDarray object specifying the keyword data. The type of the leaf objects depends on the keyword. The AbaqusNDarray object is returned only if the data is suitable and if the **InputFile.parse()** method was called with the option **usePyArray=True**. In cases where large amounts of numerical data (i.e., large node arrays) are expected, it is recommended that you use the option **usePyArray=True**.

suboptions

A KeywordSequence specifying the suboptions of the keyword.

comments

A sequence of Strings specifying the comments.

24.3 AbaqusNDarray object

The AbaqusNDarray object is a sequence object derived from **numpy.ndarray** and is used to store numeric keyword data from an Abaqus input file. This object is similar to the **numpy.ndarray** object, but the numeric elements are returned as standard Python objects, not **numpy** numeric types.

The numeric elements can be:

- All ints.
- All floats.
- First column int, all other columns floats.

In the last of these cases, the member *colZeroIsInt* will be True; in the other two cases, it will be False.

25. Interaction commands

A specific type of interaction object and a specific type of interaction state object are designed for each type of interaction. An interaction object stores the non-propagating data of an interaction as well as a number of instances of the corresponding interaction state object, each of which stores the propagating data of the interaction in a single step. Instances of the interaction state object are created and deleted internally by its corresponding interaction object.

25.1 Interaction object

The Interaction object is the abstract base type for other Interaction objects. The Interaction object has no explicit constructor. Each of the Interaction objects has the following methods:

- `deactivate`
- `move`
- `reset`
- `resume`
- `suppress`
- `delete`

The methods are described below.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.1.1 deactivate(...)

This method deactivates the interaction in the specified step and all its subsequent steps.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is deactivated.

Optional arguments

None.

Return value

None

Exceptions

None.

25.1.2 move(...)

This method moves an interaction from one step to another.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which to move the interaction.

toStepName

A String specifying the name of the step to which to move the interaction.

Optional arguments

None.

Return value

None

Exceptions

None.

25.1.3 reset(...)

This method reactivates an interaction that was deactivated previously. The **reset** method is available during the step in which the interaction was deactivated originally.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is reactivated.

Optional arguments

None.

Return value

None

Exceptions

None.

25.1.4 resume()

This method resumes an interaction that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

25.1.5 suppress()

This method suppresses an interaction.

Arguments

None.

Return value

None

Exceptions

None.

25.1.6 delete(...)

This method allows you to delete existing interactions.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each interaction to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.1.7 Members

The Interaction object has the following member:

name

A String specifying the repository key.

25.2 AcousticImpedance object

The AcousticImpedance object defines surface impedance information or nonreflecting boundaries for acoustic and coupled acoustic-structural analyses.

The AcousticImpedance object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.2.1 AcousticImpedance(...)

This method creates an AcousticImpedance object.

Path

```
mdb.models[name].AcousticImpedance
```

Required arguments*name*

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the AcousticImpedance object is created.

surface

A Region object specifying the acoustic boundary surface.

Optional arguments*definition*

A SymbolicConstant specifying the type of acoustic impedance to be defined. Possible values are TABULAR and NONREFLECTING. The default value is TABULAR.

interactionProperty

A String specifying the AcousticImpedanceProp object associated with this interaction.

nonreflectingType

A SymbolicConstant specifying the type of nonreflecting geometry to be defined. Possible values are PLANE, IMPROVED, CIRCULAR, SPHERICAL, ELLIPTICAL, and PROLATE. The default value is PLANE.

This argument is valid only when *definition*=NONREFLECTING.

radius

A Float specifying the radius of the circle or sphere defining the boundary surface. The default value is 1.0.

This argument is valid only when *definition*=NONREFLECTING, and *nonreflectingType*=CIRCULAR or SPHERICAL.

semimajorAxis

A Float specifying the semimajor axis length of the ellipse or prolate spheroid defining the boundary surface. The default value is 1.0.

This argument is valid only when *definition*=NONREFLECTING, and *nonreflectingType*=ELLIPTICAL or PROLATE.

eccentricity

A Float specifying the eccentricity of the ellipse or prolate spheroid defining the boundary surface. The default value is 0.0.

This argument is valid only when *definition*=NONREFLECTING, and *nonreflectingType*=ELLIPTICAL or PROLATE.

centerCoordinates

A sequence of three Floats specifying the X, Y, and Z coordinates of the center of the ellipse or prolate spheroid defining the boundary surface. The default value is (0, 0, 0).

This argument is valid only when *definition*=NONREFLECTING, and *nonreflectingType*=ELLIPTICAL or PROLATE.

INTERACTION COMMANDS

directionCosine

A sequence of three Floats specifying the X, Y, and Z components of the direction cosine of the major axis of the ellipse or prolate spheroid defining the boundary surface. The default value is (0, 0, 1).

This argument is valid only when *definition*=NONREFLECTING, and *nonreflectingType*=ELLIPTICAL or PROLATE.

Return value

An AcousticImpedance object.

Exceptions

None.

25.2.2 setValues(...)

This method modifies the data for an existing AcousticImpedance object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AcousticImpedance** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.2.3 setValuesInStep(...)

This method modifies the propagating data for an existing AcousticImpedance object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional argument*interactionProperty*

A String specifying the AcousticImpedanceProp object associated with this interaction.

Return value

None

Exceptions

None.

25.2.4 Members

The AcousticImpedance object has members with the same names and descriptions as the arguments to the **AcousticImpedance** method except the optional arguments to the **setValuesInStep** method.

25.2.5 Corresponding analysis keywords

*SIMPEDANCE

25.3 AcousticImpedanceProp object

The AcousticImpedanceProp object is an interaction property that defines the properties referred to by an AcousticImpedance object.

The AcousticImpedanceProp object is derived from the InteractionProperty object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.3.1 AcousticImpedanceProp(...)

This method creates an AcousticImpedanceProp object.

Path

```
mdb.models[name].AcousticImpedanceProp
```

INTERACTION COMMANDS

Required arguments

name

A String specifying the interaction property repository key.

tableType

A SymbolicConstant specifying the type of tabular data to be defined. Possible values are IMPEDANCE and ADMITTANCE.

table

A sequence of sequences of Floats specifying acoustic impedance properties.

If *tableType*=IMPEDANCE, each sequence of the table data specifies:

- The real part of the complex impedance.
- The imaginary part of the complex impedance.
- Frequency, if the data depend on frequency.

If *tableType*=ADMITTANCE, each sequence of the table data specifies:

- The real part of the complex admittance.
- The imaginary part of the complex admittance.
- Frequency, if the data depend on frequency.

Optional argument

frequencyDependency

A Boolean specifying whether the *table* data depend on frequency. The default value is OFF.

Return value

An AcousticImpedanceProp object.

Exceptions

None.

25.3.2 setValues(...)

This method modifies the AcousticImpedanceProp object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `AcousticImpedanceProp` method, except for the *name* argument.

Return value

None

Exceptions

None.

25.3.3 Members

The `AcousticImpedanceProp` object has members with the same names and descriptions as the arguments to the `AcousticImpedanceProp` method.

25.3.4 Corresponding analysis keywords

*IMPEDANCE PROPERTY

25.4 AcousticImpedanceState object

The `AcousticImpedanceState` object stores the propagating data of an `AcousticImpedance` object in a step. One instance of this object is created internally by the `AcousticImpedance` object for each step. The instance is also deleted internally by the `AcousticImpedance` object.

The `AcousticImpedanceState` object has no constructor or methods.

The `AcousticImpedanceState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.4.1 Members

The `AcousticImpedanceState` object has the following members:

interactionPropertyState

A `SymbolicConstant` specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

INTERACTION COMMANDS

interactionProperty

A String specifying the name of the AcousticImpedanceProp object associated with this interaction.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.4.2 Corresponding analysis keywords

*SIMPEDANCE

25.5 ActuatorSensor object

The ActuatorSensor object defines a single point actuator where the actuation is determined by a user subroutine (**UEL**). The subroutine senses the data at the same point as the actuator.

The ActuatorSensor object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.5.1 ActuatorSensor(...)

This method creates an ActuatorSensor object.

Path

`mdb.models[name].ActuatorSensor`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the actuator/sensor interaction is created. *createStepName* must be set to 'Initial'.

point

A Region object specifying the point at which the constraint is applied.

interactionProperty

A String specifying the ActuatorSensorProp object associated with this interaction.

noCoordComponents

An Int specifying the number of coordinate components supplied to the user subroutine (**UEL**).

unsymm

A Boolean specifying whether the element matrices are symmetric (ON) or unsymmetric (OFF).
The default value is OFF.

noSolutionDepVar

An Int specifying the number of solution-dependent variables. The default value is 0.

userSubUel

A String specifying the name of the user subroutine (**UEL**) that defines the user element.

dof

A String specifying the degrees of freedom, separated by commas.

solutionDepVars

A sequence of Floats specifying the initial values of the solution-dependent variables.

Optional arguments

None.

Return value

An ActuatorSensor object.

Exceptions

None.

25.5.2 **setValues(...)**

This method modifies the `ActuatorSensor` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ActuatorSensor** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.5.3 **Members**

The `ActuatorSensor` object has members with the same names and descriptions as the arguments to the **ActuatorSensor** method.

25.5.4 **Corresponding analysis keywords**

- *ELEMENT
- *USER ELEMENT
- *INITIAL CONDITIONS, TYPE=SOLUTION

25.6 **ActuatorSensorProp object**

The `ActuatorSensorProp` object is an interaction property that defines the properties referred to by an `ActuatorSensor` object.

The `ActuatorSensorProp` object is derived from the `InteractionProperty` object.

Access

`import interaction`

```
mdb.models [name] .interactionProperties [name]
```

25.6.1 **ActuatorSensorProp(...)**

This method creates an **ActuatorSensorProp** object.

Path

```
mdb.models [name] .ActuatorSensorProp
```

Required argument

name

A String specifying the interaction property repository key.

Optional arguments

realProperties

A sequence of Floats specifying the **PROPS** array used by user subroutine **UEL**. The default value is an empty sequence.

integerProperties

A sequence of Ints specifying the **JPROPS** array used by user subroutine **UEL**. The default value is an empty sequence.

Return value

An **ActuatorSensorProp** object.

Exceptions

None.

25.6.2 **setValues(...)**

This method modifies the **ActuatorSensorProp** object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ActuatorSensorProp** method, except for the *name* argument.

Return value

None

Exceptions

None.

25.6.3 Members

The `ActuatorSensorProp` object has members with the same names and descriptions as the arguments to the `ActuatorSensorProp` method.

25.6.4 Corresponding analysis keywords

*UEL PROPERTY

25.7 ActuatorSensorState object

The `ActuatorSensorState` object stores the propagating data of an actuator sensor in a step. One instance of this object is created internally by the `ActuatorSensor` object for each step. The instance is also deleted internally by the `ActuatorSensor` object.

The `ActuatorSensorState` object has no constructor, methods, or members.

The `ActuatorSensorState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.7.1 Members

The `ActuatorSensorState` object has the following member:

status

A `SymbolicConstant` specifying the propagation state of the `InteractionState` object. Possible values are:

- `NOT_YET_ACTIVE`
- `CREATED`
- `PROPAGATED`

- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.8 CavityRadiation object

The CavityRadiation object defines cavities for thermal radiation heat transfer and controls the calculation of viewfactors.

The CavityRadiation object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.8.1 CavityRadiation(...)

This method creates a CavityRadiation object.

Path

```
mdb.models[name].CavityRadiation
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the cavity radiation interaction should be created.

surfaces

A RegionArray object specifying the surfaces for which radiation viewfactor control is being specified.

INTERACTION COMMANDS

Optional arguments

surfaceEmissivities

A sequence of Strings specifying the names of the Cavity Radiation properties containing the surface emissivity data. One name per specified surface. The emissivity data is ignored when *surfaceReflection*=OFF.

ambientTemp

None or a Float specifying the reference ambient temperature value, θ^0 . Specifying a value indicates an open cavity. The default value is **None**.

blocking

A SymbolicConstant specifying the blocking checks to be performed in the viewfactor calculations. Possible values are BLOCKING_ALL, NO_BLOCKING, and PARTIAL_BLOCKING. The default value is BLOCKING_ALL.

blockingSurfaces

A RegionArray object specifying the surfaces that provide blocking inside the cavity. This argument applies only when *blocking*=PARTIAL_BLOCKING.

rangeOfView

None or a Float specifying the maximum distance between surface facets at which viewfactors are calculated. More distant facets are deemed too far apart to exchange significant amounts of heat through radiation effects, and the viewfactors between these facets are assumed to be zero. If *rangeOfView*=**None**, there is no upper limit. The default value is **None**.

surfaceReflection

A Boolean specifying whether heat reflections are to be included in the cavity radiation calculations. The default value is ON.

viewfactorAccurTol

A Float specifying the acceptable tolerance for the viewfactor calculations. The default value is 0.05.

minInfinitesimalRatio

A Float specifying the facet area ratio above which the infinitesimal-to-finite area approximation is used for viewfactor calculations. The default value is 64.0.

numPointsPerEdge

An Int specifying the number of Gauss integration points to be used along each edge when the numerical integration of contour integrals is used for viewfactor calculations. One to five integration points are allowed. The default value is 3.

minLumpedAreaDS

A Float specifying the nondimensional distance-square value above which the lumped area approximation is used for viewfactor calculations. The default value is 5.0.

cyclicSymmetry

A Boolean specifying whether cyclic symmetry will be applied. This argument cannot be specified for axisymmetric models. The default value is OFF.

cyclicImages

An Int specifying the number of cyclically similar images that compose the cavity formed as a result of this symmetry. This argument applies only when *cyclicSymmetry*=ON. The default value is 2.

cyclicRotPt

A ModelDot object specifying the rotation axis point. This argument applies only when *cyclicSymmetry*=ON.

cyclicRotEndPt

A ModelDot object specifying the rotation axis end point. This argument applies only for three-dimensional models, and only when *cyclicSymmetry*=ON.

cyclicSymPt

A ModelDot object specifying the symmetry axis end point. This argument applies only when *cyclicSymmetry*=ON.

periodicSymmetries

An Int specifying the number of periodic symmetries that will be applied. The default value is 0.

periodicImages_1

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the first periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_1*. This argument applies only when *periodicSymmetries* is greater than zero. The default value is 2.

periodicImages_2

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the second periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_2*. This argument applies only when *periodicSymmetries* is greater than one. The default value is 2.

periodicImages_3

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the third periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_3*. This argument applies only when *periodicSymmetries* = 3. The default value is 2.

periodicSymAxis_1

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object indicating the first line of symmetry in two-dimensional models. This argument applies only for 2D models, and when *periodicSymmetries* is greater than zero.

periodicSymAxis_2

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object indicating the second line of symmetry in two-dimensional models. This argument applies only for two-dimensional models, and when *periodicSymmetries* = 2.

INTERACTION COMMANDS

periodicSymPlane_1

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the first plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *periodicSymmetries* is greater than zero.

periodicSymPlane_2

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the second plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *periodicSymmetries* is greater than one.

periodicSymPlane_3

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the third plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *periodicSymmetries* = 3.

periodicDistance_1

A sequence of sequences of Floats specifying the two points of the vector that describes the periodic distance for the first periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* is greater than zero. The default value is an empty sequence.

periodicDistance_2

A sequence of sequences of Floats specifying the two points of the vector that describes the periodic distance for the second periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* is greater than one. The default value is an empty sequence.

periodicDistance_3

A sequence of sequences of Floats specifying the two points of the vector that describes the periodic distance for the third periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* = 3. The default value is an empty sequence.

periodicSymZ

None or a Float specifying the Z value indicating the symmetry reference line in axisymmetric models. This argument applies only for axisymmetric models, and when *periodicSymmetries* = 1. The default value is **None**.

periodicDistZ

None or a Float specifying the Z value indicating the periodic distance in axisymmetric models. This argument applies only for axisymmetric models, and when *periodicSymmetries* = 1. The default value is **None**.

reflectionSymmetries

An Int specifying the number of reflection symmetries will be applied. The default value is 0.

reflectionSymAxis_1

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object indicating the first line of symmetry in two-dimensional models. This argument applies only for two-dimensional models, and when *reflectionSymmetries* is greater than zero.

reflectionSymAxis_2

A straight Edge, a Datum object representing a datum axis, or an ElementEdge object indicating the second line of symmetry in two-dimensional models. This argument applies only for two-dimensional models, and when *reflectionSymmetries* = 2.

reflectionSymPlane_1

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the first plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *reflectionSymmetries* is greater than zero.

reflectionSymPlane_2

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the second plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *reflectionSymmetries* is greater than one.

reflectionSymPlane_3

A planar Face, an ElementFace, or a Datum object representing a datum plane; indicating the third plane of symmetry in three-dimensional models. This argument applies only for three-dimensional models, and when *reflectionSymmetries* = 3.

reflectionSymZ

None or a Float specifying the Z value indicating the symmetry reference line in axisymmetric models. This argument applies only for axisymmetric models, and when *reflectionSymmetries* = 1. The default value is **None**.

Return value

A CavityRadiation object.

Exceptions

None.

25.8.2 setValues(...)

This method modifies the data for an existing CavityRadiation object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CavityRadiation** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.8.3 setValuesInStep(...)

This method modifies the propagating data of an existing **CavityRadiation** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

blocking

A **SymbolicConstant** specifying the blocking checks to be performed in the viewfactor calculations. Possible values are **BLOCKING_ALL**, **NO_BLOCKING**, and **PARTIAL_BLOCKING**. The default value is **BLOCKING_ALL**.

blockingSurfaces

A **RegionArray** object specifying the surfaces that provide blocking inside the cavity. This argument applies only when *blocking*=**PARTIAL_BLOCKING**.

rangeOfView

None or a **Float** specifying the maximum distance between surface facets at which viewfactors are calculated. More distant facets are deemed too far apart to exchange significant amounts of heat through radiation effects, and the viewfactors between these facets are assumed to be zero. If *rangeOfView*=**None**, there is no upper limit. The default value is **None**.

surfaceReflection

A **Boolean** specifying whether heat reflections are to be included in the cavity radiation calculations. The default value is **ON**.

viewfactorAccurTol

A **Float** specifying the acceptable tolerance for the viewfactor calculations. The default value is 0.05.

Return value**None****Exceptions**

None.

25.8.4 Members

The CavityRadiation object can have the following members:

name

A String specifying the repository key.

ambientTemp

None or a Float specifying the reference ambient temperature value, θ^0 . Specifying a value indicates an open cavity. The default value is **None**.

minInfinitesimalRatio

A Float specifying the facet area ratio above which the infinitesimal-to-finite area approximation is used for viewfactor calculations. The default value is 64.0.

numPointsPerEdge

An Int specifying the number of Gauss integration points to be used along each edge when the numerical integration of contour integrals is used for viewfactor calculations. One to five integration points are allowed. The default value is 3.

minLumpedAreaDS

A Float specifying the nondimensional distance-square value above which the lumped area approximation is used for viewfactor calculations. The default value is 5.0.

cyclicSymmetry

A Boolean specifying whether cyclic symmetry will be applied. This argument cannot be specified for axisymmetric models. The default value is OFF.

cyclicImages

An Int specifying the number of cyclically similar images that compose the cavity formed as a result of this symmetry. This argument applies only when *cyclicSymmetry*=ON. The default value is 2.

periodicSymmetries

An Int specifying the number of periodic symmetries that will be applied. The default value is 0.

periodicImages_1

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the first periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_1*. This argument applies only when *periodicSymmetries* is greater than zero. The default value is 2.

INTERACTION COMMANDS

periodicImages_2

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the second periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_2*. This argument applies only when *periodicSymmetries* is greater than one. The default value is 2.

periodicImages_3

An Int specifying the number of repetitions used in the numerical calculation of the cavity viewfactors resulting from the third periodic symmetry. The result of this symmetry is a cavity composed of the cavity surface defined in the model plus twice the value of *periodicImages_3*. This argument applies only when *periodicSymmetries* = 3. The default value is 2.

periodicSymZ

None or a Float specifying the Z value indicating the symmetry reference line in axisymmetric models. This argument applies only for axisymmetric models, and when *periodicSymmetries* = 1. The default value is **None**.

periodicDistZ

None or a Float specifying the Z value indicating the periodic distance in axisymmetric models. This argument applies only for axisymmetric models, and when *periodicSymmetries* = 1. The default value is **None**.

reflectionSymmetries

An Int specifying the number of reflection symmetries will be applied. The default value is 0.

reflectionSymZ

None or a Float specifying the Z value indicating the symmetry reference line in axisymmetric models. This argument applies only for axisymmetric models, and when *reflectionSymmetries* = 1. The default value is **None**.

createStepName

A String specifying the name of the step in which the cavity radiation interaction should be created.

surfaces

A RegionArray object specifying the surfaces for which radiation viewfactor control is being specified.

surfaceEmissivities

A tuple of Strings specifying the names of the Cavity Radiation properties containing the surface emissivity data. One name per specified surface. The emissivity data is ignored when *surfaceReflection*=OFF.

cyclicRotPt

A ModelDot object specifying the rotation axis point. This argument applies only when *cyclicSymmetry*=ON.

cyclicRotEndPt

A ModelDot object specifying the rotation axis end point. This argument applies only for three-dimensional models, and only when *cyclicSymmetry*=ON.

cyclicSymPt

A ModelDot object specifying the symmetry axis end point. This argument applies only when *cyclicSymmetry*=ON.

periodicDistance_1

A tuple of tuples of Floats specifying the two points of the vector that describes the periodic distance for the first periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* is greater than zero. The default value is an empty sequence.

periodicDistance_2

A tuple of tuples of Floats specifying the two points of the vector that describes the periodic distance for the second periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* is greater than one. The default value is an empty sequence.

periodicDistance_3

A tuple of tuples of Floats specifying the two points of the vector that describes the periodic distance for the third periodic symmetry. Each point is defined by a tuple of three coordinates indicating its position. This argument applies only when *periodicSymmetries* = 3. The default value is an empty sequence.

25.8.5 Corresponding analysis keywords

- *CAVITY DEFINITION, SET PROPERTY
- *CYCLIC
- *EMISSIVITY
- *PERIODIC
- *RADIATION SYMMETRY
- *RADIATION VIEWFACTOR
- *REFLECTION

25.9 CavityRadiationProp object

The CavityRadiationProp object is an interaction property that defines emissivity as a function of temperature and field variables.

The CavityRadiationProp object is derived from the InteractionProperty object.

Access

```
import interaction
```

```
mdb.models[name].interactionProperties[name]
```

25.9.1 CavityRadiationProp(...)

This method creates a CavityRadiationProp object.

Path

```
mdb.models[name].CavityRadiationProp
```

Required argument

name

A String specifying the interaction property repository key.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

property

A sequence of sequences of Floats specifying the following:

- The emissivity, ϵ .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CavityRadiationProp object.

Exceptions

None.

25.9.2 setValues(...)

This method modifies the CavityRadiationProp object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `CavityRadiationProp` method, except for the *name* argument.

Return value

None

Exceptions

None.

25.9.3 Members

The `CavityRadiationProp` object has members with the same names and descriptions as the arguments to the `CavityRadiationProp` method.

25.9.4 Corresponding analysis keywords

*EMISSION

25.10 CavityRadiationState object

The `CavityRadiationState` object stores the propagating data for a `CavityRadiation` object. One instance of this object is created internally by the `CavityRadiation` object for each step. The instance is also deleted internally by the `CavityRadiation` object.

The `CavityRadiationState` object has no constructor or methods.

The `CavityRadiationState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.10.1 Members

The CavityRadiationState object has the following members:

blocking

A SymbolicConstant specifying the blocking checks to be performed in the viewfactor calculations. Possible values are BLOCKING_ALL, NO_BLOCKING, and PARTIAL_BLOCKING.

blockingState

A SymbolicConstant specifying the propagation state of the blocking member. Possible values are UNSET, SET, UNCHANGED, and FREED.

blockingSurfacesState

A SymbolicConstant specifying the propagation state of the *blockingSurfaces* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

rangeOfView

A Float specifying the distance beyond which factors need not be calculated because surfaces are judged to be too far apart to “see” each other (due to blocking by other surfaces).

rangeOfViewState

A SymbolicConstant specifying the propagation state of the *rangeOfView* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

surfaceReflection

A Boolean specifying whether reflection must be included in the cavity radiation calculations. The default value is ON.

surfaceReflectionState

A SymbolicConstant specifying the propagation state of the *surfaceReflection* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

viewfactorAccuracyTol

A Float specifying the acceptable tolerance for the viewfactor calculations.

viewfactorAccuracyTolState

A SymbolicConstant specifying the propagation state of the *viewfactorAccuracyTol* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

blockingSurfaces

A tuple of Strings specifying the surfaces that provide blocking inside the cavity.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED

- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.10.2 Corresponding analysis keywords

*RADIATION VIEWFACTOR

25.11 CohesiveBehavior object

The CohesiveBehavior object specifies cohesive behavior for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].cohesiveBehavior
```

25.11.1 CohesiveBehavior(...)

This method creates a CohesiveBehavior object.

Path

```
mdb.models[name].interactionProperties[name].CohesiveBehavior
```

Required arguments

None.

Optional arguments

repeatedContacts

A Boolean specifying whether to enforce cohesive behavior for recurrent contacts at nodes on the slave surface subsequent to ultimate failure. The default value is OFF.

eligibility

A SymbolicConstant specifying the eligible slave nodes. Possible values are ALL_NODES, INITIAL_NODES, and SPECIFIED. The default value is ALL_NODES.

INTERACTION COMMANDS

defaultPenalties

A Boolean specifying whether to use the default contact penalties. The default value is ON.

coupling

A SymbolicConstant specifying whether the traction-separation coefficients are coupled or uncoupled. This argument is valid only for *defaultPenalties*=OFF. Possible values are UNCOUPLED and COUPLED. The default value is UNCOUPLED.

temperatureDependency

A Boolean specifying whether the coefficient data depend on temperature. This argument is valid only for *defaultPenalties*=OFF. The default value is OFF.

dependencies

An Int specifying the number of field variables. This argument is valid only for *defaultPenalties*=OFF. The default value is 0.

table

A sequence of sequences of Floats specifying the traction-separation coefficients. The items in the table data are described below. This argument is valid only for *defaultPenalties*=OFF.

Table data

If *coupling*=UNCOUPLED, the table data specify the following:

- Stiffness coefficient in the normal direction, K_{nn} .
- Stiffness coefficient in the first shear direction, K_{ss} .
- Stiffness coefficient in the second shear direction, K_{tt} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *coupling*=COUPLED, the table data specify the following:

- Stiffness coefficient in the normal direction, K_{nn} .
- Stiffness coefficient in the first shear direction, K_{ss} .
- Stiffness coefficient in the second shear direction, K_{tt} .
- Coupled stiffness coefficient, K_{ns} .
- Coupled stiffness coefficient, K_{nt} .
- Coupled stiffness coefficient, K_{st} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CohesiveBehavior object.

Exceptions

None.

25.11.2 setValues(...)

This method modifies the CohesiveBehavior object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CohesiveBehavior** method.

Return value

None

Exceptions

None.

25.11.3 Members

The CohesiveBehavior object has members with the same names and descriptions as the arguments to the **CohesiveBehavior** method.

25.11.4 Corresponding analysis keywords

*COHESIVE BEHAVIOR

25.12 ConcentratedFilmCondition object

The ConcentratedFilmCondition object defines concentrated film coefficients and associated sink temperatures.

The ConcentratedFilmCondition object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.12.1 ConcentratedFilmCondition(...)

This method creates a ConcentratedFilmCondition object.

Path

```
mdb.models[name].ConcentratedFilmCondition
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the ConcentratedFilmCondition object is created.

region

A Region object specifying the region to which the concentrated film condition interaction is applied. The interaction is applied to each node in the region.

definition

A SymbolicConstant specifying how the concentrated film condition is defined. Possible values are EMBEDDED_COEFF, PROPERTY_REF, USER_SUB, and FIELD.

Optional arguments

nodalArea

A Float specifying the area associated with the node where the concentrated film condition is applied. The default value is 1.0.

explicitRegionType

A SymbolicConstant specifying how the concentrated film condition is applied to the boundary of an adaptive mesh domain. Possible values are LAGRANGIAN, SLIDING, and EULERIAN. The default value is LAGRANGIAN.

This argument applies only during an Abaqus/Explicit analysis.

interactionProperty

A String specifying the name of the FilmConditionProp object associated with this interaction. The *interactionProperty* argument applies only when *definition*=PROPERTY_REF. The default value is an empty string.

field

A String specifying the name of the AnalyticalField object associated with this interaction. The *field* argument applies only when *definition*=FIELD. The default value is an empty string.

sinkTemperature

A Float specifying the reference sink temperature, θ^0 . The default value is 0.0.

sinkAmplitude

A String specifying the name of the Amplitude object that gives the variation of the sink temperature, θ^0 , with time. The default value is an empty string.

Note: Use **None** in an Abaqus/Standard analysis to specify that the reference sink temperature is applied immediately at the beginning of the step or linearly over the step. Use **None** in an Abaqus/Explicit analysis to specify that the reference sink temperature is applied throughout the step.

filmCoeff

A Float specifying the reference film coefficient value, h . The *filmCoeff* argument applies when *definition*=EMBEDDED_COEFF, *definition*=USER_SUB, or *definition*=FIELD. The default value is 0.0.

filmCoeffAmplitude

A String specifying the name of the Amplitude object that gives the variation of the film coefficient, h , with time. The default value is an empty string.

Note: Use **None** in an Abaqus/Standard analysis to specify that the reference film coefficient is applied immediately at the beginning of the step or linearly over the step. Use **None** in an Abaqus/Explicit analysis to specify that the reference film coefficient is applied throughout the step.

sinkFieldName

A String specifying the name of the AnalyticalField or DiscreteField object associated with the sink temperature. The *sinkFieldName* argument applies only when *sinkDistributionType*=ANALYTICAL_FIELD or *sinkDistributionType*=DISCRETE_FIELD. The default value is an empty string.

sinkDistributionType

A SymbolicConstant specifying how the sink temperature is distributed. Possible values are UNIFORM, ANALYTICAL_FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

Return value

A `ConcentratedFilmCondition` object.

Exceptions

None.

25.12.2 `setValues(...)`

This method modifies the data for an existing `ConcentratedFilmCondition` object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConcentratedFilmCondition` method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.12.3 `setValuesInStep(...)`

This method modifies the propagating data of an existing `ConcentratedFilmCondition` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

None.

The optional arguments to `setValuesInStep` are the same as the optional arguments to the `ConcentratedFilmConditionState` method.

Return value

None

Exceptions

None.

25.12.4 Members

The `ConcentratedFilmCondition` object has members with the same names and descriptions as the arguments to the `ConcentratedFilmCondition` method except the optional arguments to the `setValuesInStep` method.

25.13 ConcentratedFilmConditionState object

The `ConcentratedFilmConditionState` object stores the propagating data for a `ConcentratedFilmCondition` object. One instance of this object is created internally by the `ConcentratedFilmCondition` object for each step. The instance is also deleted internally by the `ConcentratedFilmCondition` object.

The `ConcentratedFilmConditionState` object has no constructor or methods.

The `ConcentratedFilmConditionState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.13.1 Members

The `ConcentratedFilmConditionState` object has the following members:

interactionPropertyState

A `SymbolicConstant` specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

sinkTemperature

A `Float` specifying the sink temperature.

sinkTemperatureState

A `SymbolicConstant` specifying the propagation state of the *sinkTemperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

INTERACTION COMMANDS

sinkAmplitudeState

A SymbolicConstant specifying the propagation state of the *sinkAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

filmCoeff

A Float specifying the film coefficient.

filmCoeffState

A SymbolicConstant specifying the propagation state of the *filmCoeff* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

filmCoeffAmplitudeState

A SymbolicConstant specifying the propagation state of the *filmCoeffAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

nodalArea

A Float specifying the area associated with the node where the concentrated film condition is applied.

nodalAreaState

A SymbolicConstant specifying the propagation state of the *nodalArea* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the FilmConditionProp object associated with this interaction.

sinkAmplitude

A String specifying the name of the Amplitude object that gives the variation of the sink temperature.

filmCoeffAmplitude

A String specifying the name of the Amplitude object that gives the variation of the film coefficient.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.13.2 Corresponding analysis keywords

*CFILM

25.14 ConcentratedRadiationToAmbient object

The ConcentratedRadiationToAmbient object defines radiant heat transfer between a point and its nonreflecting environment.

The ConcentratedRadiationToAmbient object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.14.1 ConcentratedRadiationToAmbient(...)

This method creates a ConcentratedRadiationToAmbient object.

Path

```
mdb.models[name].ConcentratedRadiationToAmbient
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the ConcentratedRadiationToAmbient object is created.

region

A Region object specifying the region to which the concentrated radiation interaction is applied. The interaction is applied to each node in the region.

ambientTemperature

A Float specifying the reference ambient temperature, θ^0 .

ambientTemperatureAmp

A String specifying the name of the Amplitude object that gives the variation of the ambient temperature with time.

Note: Use **None** in an Abaqus/Standard analysis to specify that the reference ambient temperature is applied immediately at the beginning of the step or linearly over the step. Use **None** in an Abaqus/Explicit analysis to specify that the reference ambient temperature is applied throughout the step.

emissivity

A Float specifying the emissivity, ϵ .

Optional arguments

nodalArea

A Float specifying the area associated with the node where the concentrated radiation interaction is applied. The default value is 1.0.

explicitRegionType

A SymbolicConstant specifying how the concentrated radiation is applied to the boundary of an adaptive mesh domain. Possible values are LAGRANGIAN, SLIDING, and EULERIAN. The default value is LAGRANGIAN.

Note: *explicitRegionType* applies only during an Abaqus/Explicit analysis.

field

A String specifying the name of the AnalyticalField object associated with this interaction. The *field* argument applies only when *distributionType*=ANALYTICAL_FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the radiation is defined. Possible values are UNIFORM and ANALYTICAL_FIELD. The default value is UNIFORM.

Return value

A ConcentratedRadiationToAmbient object.

Exceptions

None.

25.14.2 setValues(...)

This method modifies the data for an existing ConcentratedRadiationToAmbient object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcentratedRadiationToAmbient** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.14.3 setValuesInStep(...)

This method modifies the propagating data of an existing **ConcentratedRadiationToAmbient** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

None.

The optional arguments to **setValuesInStep** are the same as the arguments to the **ConcentratedRadiationToAmbient** method, except for the *name*, *createStepName*, *region*, and *explicitRegionType* arguments.

Return value

None

Exceptions

None.

25.14.4 Members

The **ConcentratedRadiationToAmbient** object has members with the same names and descriptions as the arguments to the **ConcentratedRadiationToAmbient** method.

25.15 ConcentratedRadiationToAmbientState object

The `ConcentratedRadiationToAmbientState` object stores the propagating data for a `ConcentratedRadiationToAmbient` object. One instance of this object is created internally by the `ConcentratedRadiationToAmbient` object for each step. The instance is also deleted internally by the `ConcentratedRadiationToAmbient` object.

The `ConcentratedRadiationToAmbientState` object has no constructor or methods.

The `ConcentratedRadiationToAmbientState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.15.1 Members

The `ConcentratedRadiationToAmbientState` object has the following members:

ambientTemperature

A Float specifying the ambient temperature.

ambientTemperatureState

A `SymbolicConstant` specifying the propagation state of the *ambientTemperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

ambientTemperatureAmpState

A `SymbolicConstant` specifying the propagation state of the *ambientTemperatureAmp* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

emissivity

A Float specifying the emissivity.

emissivityState

A `SymbolicConstant` specifying the propagation state of the *emissivity* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

nodalArea

A Float specifying the area associated with the node where the concentrated radiation is applied.

nodalAreaState

A `SymbolicConstant` specifying the propagation state of the *nodalArea* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

ambientTemperatureAmp

A String specifying the name of the Amplitude object that gives the variation of the ambient temperature with time.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.15.2 Corresponding analysis keywords

*CRADIATE

25.16 ContactControl object

The ContactControl object is the abstract base type for other ContactControl objects. The ContactControl object has no explicit constructor, members, or methods.

Access

```
import interaction
mdb.models[name].contactControls[name]
```

25.16.1 Members

The ContactControl object has no members.

25.17 ContactDamage object

The ContactDamage object specifies damage options for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].damage
```

25.17.1 Damage(...)

This method creates a ContactDamage object.

Path

```
mdb.models[name].interactionProperties[name].Damage
```

Required argument

initTable

A sequence of sequences of Floats specifying the values defining the damage initiation. The items in the table data are described below.

Optional arguments

criterion

A SymbolicConstant specifying the type of data used to define the initiation of damage. Possible values are MAX_STRESS, MAX_SEPARATION, QUAD_TRACTION, and QUAD_SEPARATION. The default value is MAX_STRESS.

initTempDep

A Boolean specifying whether the initiation data depend on temperature. The default value is OFF.

initDependencies

An Int specifying the number of initiation data field variables. The default value is 0.

useEvolution

A Boolean specifying whether evolution data will be defined. The default value is OFF.

evolutionType

A SymbolicConstant specifying the type of data used to define the evolution of damage. This argument is valid only when *useEvolution*=ON. Possible values are DISPLACEMENT and ENERGY. The default value is DISPLACEMENT.

softening

A SymbolicConstant specifying the type of data used to define the evolution softening response. This argument is valid only when *useEvolution*=ON. The TABULAR value can be used only when *evolutionType*=DISPLACEMENT. Possible values are LINEAR, EXPONENTIAL, and TABULAR. The default value is LINEAR.

useMixedMode

A Boolean specifying whether evolution data be defined using dependent behavior modes. This argument is valid only when *useEvolution*=ON. The default value is OFF.

mixedModeType

A SymbolicConstant specifying the mode mix fracture criterion. This argument is valid only when *useEvolution*=ON and when *useMixedMode*=ON. The POWER_LAW and BK values can be used only when *evolutionType*=ENERGY. Possible values are TABULAR, POWER_LAW, and BK. The default value is TABULAR.

modeMixRatio

A SymbolicConstant specifying the mode mix ratio type. This argument is valid only when *useEvolution*=ON and when *useMixedMode*=ON. The TRACTION value can be used only when *mixedModeType*=TABULAR. Possible values are ENERGY and TRACTION. The default value is ENERGY.

exponent

None or a Float specifying the exponent in the power-law or BK criterion that defines the variation of fracture energy with mode mix. This argument is valid only when *useEvolution*=ON and when *mixedModeType*=POWER_LAW or BK. The default value is **None**.

evolTempDep

A Boolean specifying whether the evolution data depend on temperature. This argument is valid only when *useEvolution*=ON. The default value is OFF.

evolDependencies

An Int specifying the number of evolution data field variables. This argument is valid only when *useEvolution*=ON. The default value is 0.

evolTable

A sequence of sequences of Floats specifying the values defining the damage evolution. The items in the table data are described below. This argument is valid only when *useEvolution*=ON.

useStabilization

A Boolean specifying whether stabilization data will be defined. This argument is valid only when *useEvolution*=ON. The default value is OFF.

viscosityCoef

None or a Float specifying the viscosity coefficient. This argument is valid only when *useStabilization*=ON. The default value is **None**.

Table data

Table data for *initTable*:

If *criterion*=MAX_STRESS or QUAD_TRACTION, the table data specify the following:

- Maximum nominal stress in the normal-only mode.
- Maximum nominal stress in the first shear direction (for a mode that involves separation only in this direction).
- Maximum nominal stress in the second shear direction (for a mode that involves separation only in this direction).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

INTERACTION COMMANDS

- Value of the second field variable.
- Etc.

If *criterion*=MAX_SEPARATION or QUAD_SEPARATION, the table data specify the following:

- Separation at damage initiation in a normal-only mode.
- Separation at damage initiation in a shear-only mode that involves separation only along the first shear direction.
- Separation at damage initiation in a shear-only mode that involves separation only along the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Table data for *evolTable*:

If *evolutionType*=DISPLACEMENT, *softening*=LINEAR, and *useMixedMode*=OFF, the table data specify the following:

- Effective total or plastic displacement at failure, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=ENERGY, *softening*=LINEAR or EXPONENTIAL, and *useMixedMode*=OFF, the table data specify the following:

- Fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=DISPLACEMENT, *softening*=LINEAR, *useMixedMode*=ON, *mixedModeType*=TABULAR, and *modeMixRatio*=ENERGY or TRACTION, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

- Value of the second field variable.
- Etc.

If *evolutionType*=ENERGY, *softening*=LINEAR or EXPONENTIAL, *useMixedMode*=ON, *mixedModeType*=TABULAR, and *modeMixRatio*=ENERGY or TRACTION, the table data specify the following:

- Fracture energy.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=DISPLACEMENT, *softening*=EXPONENTIAL, and *useMixedMode*=OFF, the table data specify the following:

- Effective total or plastic displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=DISPLACEMENT, *softening*=EXPONENTIAL, *useMixedMode*=ON, *mixedModeType*=TABULAR, and *modeMixRatio*=ENERGY or TRACTION, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=DISPLACEMENT, *softening*=TABULAR, and *useMixedMode*=OFF, the table data specify the following:

- Damage variable.

INTERACTION COMMANDS

- Effective total or plastic displacement at failure, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=DISPLACEMENT, *softening*=TABULAR, *useMixedMode*=ON, *mixedModeType*=TABULAR, and *modeMixRatio*=ENERGY or TRACTION, the table data specify the following:

- Damage variable.
- Effective total displacement, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=ENERGY, *softening*=LINEAR or EXPONENTIAL, *useMixedMode*=ON, *mixedModeType*=POWER_LAW or BK, and *modeMixRatio*=ENERGY, the table data specify the following:

- Normal mode fracture energy.
- Shear mode fracture energy for failure in the first shear direction.
- Shear mode fracture energy for failure in the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ContactDamage object.

Exceptions

None.

25.17.2 **setValues(...)**

This method modifies the `ContactDamage` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ContactDamage** method.

Return value

None

Exceptions

None.

25.17.3 **Members**

The `ContactDamage` object has the following members:

criterion

A `SymbolicConstant` specifying the type of data used to define the initiation of damage. Possible values are `MAX_STRESS`, `MAX_SEPARATION`, `QUAD_TRACTION`, and `QUAD_SEPARATION`. The default value is `MAX_STRESS`.

initTempDep

A Boolean specifying whether the initiation data depend on temperature. The default value is `OFF`.

initDependencies

An `Int` specifying the number of initiation data field variables. The default value is 0.

useEvolution

A Boolean specifying whether evolution data will be defined. The default value is `OFF`.

evolutionType

A `SymbolicConstant` specifying the type of data used to define the evolution of damage. This argument is valid only when *useEvolution*=`ON`. Possible values are `DISPLACEMENT` and `ENERGY`. The default value is `DISPLACEMENT`.

softening

A `SymbolicConstant` specifying the type of data used to define the evolution softening response. This argument is valid only when *useEvolution*=`ON`. The `TABULAR` value can be used only when *evolutionType*=`DISPLACEMENT`. Possible values are `LINEAR`, `EXPONENTIAL`, and `TABULAR`. The default value is `LINEAR`.

INTERACTION COMMANDS

useMixedMode

A Boolean specifying whether evolution data be defined using dependent behavior modes. This argument is valid only when *useEvolution*=ON. The default value is OFF.

mixedModeType

A SymbolicConstant specifying the mode mix fracture criterion. This argument is valid only when *useEvolution*=ON and when *useMixedMode*=ON. The POWER_LAW and BK values can be used only when *evolutionType*=ENERGY. Possible values are TABULAR, POWER_LAW, and BK. The default value is TABULAR.

modeMixRatio

A SymbolicConstant specifying the mode mix ratio type. This argument is valid only when *useEvolution*=ON and when *useMixedMode*=ON. The TRACTION value can be used only when *mixedModeType*=TABULAR. Possible values are ENERGY and TRACTION. The default value is ENERGY.

exponent

None or a Float specifying the exponent in the power-law or BK criterion that defines the variation of fracture energy with mode mix. This argument is valid only when *useEvolution*=ON and when *mixedModeType*=POWER_LAW or BK. The default value is **None**.

evolTempDep

A Boolean specifying whether the evolution data depend on temperature. This argument is valid only when *useEvolution*=ON. The default value is OFF.

evolDependencies

An Int specifying the number of evolution data field variables. This argument is valid only when *useEvolution*=ON. The default value is 0.

useStabilization

A Boolean specifying whether stabilization data will be defined. This argument is valid only when *useEvolution*=ON. The default value is OFF.

viscosityCoef

None or a Float specifying the viscosity coefficient. This argument is valid only when *useStabilization*=ON. The default value is **None**.

initTable

A tuple of tuples of Floats specifying the values defining the damage initiation. The items in the table data are described below.

evolTable

A tuple of tuples of Floats specifying the values defining the damage evolution. The items in the table data are described below. This argument is valid only when *useEvolution*=ON.

25.17.4 Corresponding analysis keywords

*DAMAGE INITIATION

*DAMAGE EVOLUTION
 *DAMAGE STABILIZATION

25.18 ContactDamping object

The ContactDamping object specifies damping for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].damping
```

25.18.1 Damping(...)

This method creates a ContactDamping object.

Path

```
mdb.models[name].interactionProperties[name].Damping
```

Required arguments

None.

Optional arguments

definition

A SymbolicConstant specifying the method used to define the damping. Possible values are DAMPING_COEFFICIENT and CRITICAL_DAMPING_FRACTION. The default value is DAMPING_COEFFICIENT.

tangentFraction

The SymbolicConstant DEFAULT or a Float specifying the tangential damping coefficient divided by the normal damping coefficient. The default value is DEFAULT.

clearanceDependence

A SymbolicConstant specifying the variation of the damping coefficient or fraction with respect to clearance. Possible values are STEP, LINEAR, and BILINEAR. The default value is STEP.

If *definition*=CRITICAL_DAMPING_FRACTION, the only possible value is STEP.

table

A sequence of pairs of Floats specifying the damping properties. The items in the table data are described below.

INTERACTION COMMANDS

Table data

If *definition*=DAMPING_COEFFICIENT and *clearanceDependence*=STEP, the table data specify the following:

- Damping coefficient.

If *definition*=DAMPING_COEFFICIENT and *clearanceDependence*=LINEAR or BILINEAR, the table data specify the following:

- Damping coefficient.
- Clearance.

Two pairs must be given for *clearanceDependence*=LINEAR and three pairs for *clearanceDependence*=BILINEAR. The first pair must have *clearance*=0.0, and the last pair must have *coefficient*=0.0.

If *definition*=CRITICAL_DAMPING_FRACTION, the table data specify the following:

- Critical damping fraction.

Return value

A ContactDamping object.

Exceptions

None.

25.18.2 setValues(...)

This method modifies the ContactDamping object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ContactDamping** method.

Return value

None

Exceptions

None.

25.18.3 Members

The ContactDamping object has the following members:

definition

A SymbolicConstant specifying the method used to define the damping. Possible values are DAMPING_COEFFICIENT and CRITICAL_DAMPING_FRACTION. The default value is DAMPING_COEFFICIENT.

tangentFraction

The SymbolicConstant DEFAULT or a Float specifying the tangential damping coefficient divided by the normal damping coefficient. The default value is DEFAULT.

clearanceDependence

A SymbolicConstant specifying the variation of the damping coefficient or fraction with respect to clearance. Possible values are STEP, LINEAR, and BILINEAR. The default value is STEP.

If *definition*=CRITICAL_DAMPING_FRACTION, the only possible value is STEP.

table

A tuple of pairs of Floats specifying the damping properties. The items in the table data are described below.

25.18.4 Corresponding analysis keywords

*CONTACT DAMPING

25.19 ContactExp object

The ContactExp object defines the contact domain and associated properties during contact in an Abaqus/Explicit analysis.

The ContactExp object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.19.1 ContactExp(...)

This method creates a ContactExp object.

INTERACTION COMMANDS

Path

`mdb.models [name] .ContactExp`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which this contact interaction is created.

Optional arguments

useAllstar

A Boolean specifying whether the contacting surface pair consists of all exterior faces, shell edges, beam segments, analytical rigid surfaces, and when applicable, Eulerian material surfaces.

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

includedPairs

A RegionPairs object specifying the domain pairs included in contact.

excludedPairs

A RegionPairs object specifying the domain pairs excluded from contact.

contactPropertyAssignments

A ContactPropertyAssignment object specifying the contact property assignments in the contact domain.

surfaceThicknessAssignments

A SurfaceThicknessAssignment object specifying the surface thickness assignments in the contact domain.

surfaceOffsetAssignments

A SurfaceOffsetAssignment object specifying the surface offset fraction assignments in the contact domain.

surfaceFeatureAssignments

A SurfaceFeatureAssignment object specifying the surface feature angle assignments in the contact domain.

smoothingAssignments

A SmoothingAssignment object specifying the surface smoothing assignments in the contact domain.

masterSlaveAssignments

A MasterSlaveAssignment object specifying the master-slave assignments in the contact domain.

Return value

A ContactExp object.

Exceptions

None.

25.19.2 ContactExp(...)

This method creates a ContactExp object.

Path

`mdb.models [name] .ContactExp`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which this contact interaction is created.

Optional arguments

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

useAllstar

A Boolean specifying whether the contacting surface pair consists of all exterior faces, shell edges, beam segments, analytical rigid surfaces, and when applicable, Eulerian material surfaces.

includedPairs

A sequence of pairs of Region objects or SymbolicConstants that specifies the surface pairs in contact. Possible values of the SymbolicConstants are ALLSTAR and SELF. This argument is valid only when *useAllstar*=OFF.

excludedPairs

A sequence of pairs of Region objects or SymbolicConstants that specifies the surface pairs excluded from contact. Possible values of the SymbolicConstants are ALLSTAR and SELF.

contactPropertyAssignments

A sequence of tuples specifying the properties assigned to each surface pair. Each tuple contains three entries:

- A Region object or the SymbolicConstant GLOBAL.
- A Region object or the SymbolicConstant SELF.

INTERACTION COMMANDS

- A String specifying an InteractionProperty object associated with this pair of regions.

surfaceThicknessAssignments

A sequence of tuples specifying the surface thickness assignments in the contact domain. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface thickness is assigned.
- A Float or a SymbolicConstant specifying the overriding thickness value to be used in the contact definition. Possible values of the SymbolicConstant are ORIGINAL or THINNING.
- A Float specifying a scale factor that multiplies the thickness value specified in the second entry.

surfaceOffsetAssignments

A sequence of tuples specifying the surface offset fraction assignments in the contact domain. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface offset fraction is assigned.
- A Float or a SymbolicConstant specifying the offset fraction value to be used in the contact definition. Possible values of the SymbolicConstant are ORIGINAL, SPOS, or SNEG.

surfaceFeatureAssignments

A sequence of tuples specifying the surface feature angle assignments in the contact domain. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface feature angle is assigned.
- A Float or a SymbolicConstant specifying the overriding feature angle value to be used in the contact definition. Possible values of the SymbolicConstant are PERIMETER, ALL, PICKED, or NONE.

smoothingAssignments

A sequence of tuples specifying the surface smoothing assignments in the contact domain. Each tuple contains two entries:

- A region object specifying the surface to which the smoothing option is assigned.
- A SymbolicConstant specifying the smoothing option to be used in the contact definition. Possible values of the SymbolicConstant are NONE, REVOLUTION, SPHERICAL, or TOROIDAL.

masterSlaveAssignments

A sequence of tuples specifying pure master-slave assignments in the contact domain. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL specifying the first surface that defines the master-slave assignment.

- A region object specifying the second surface in the master-slave assignment definition.
- A SymbolicConstant specifying the status of the first surface. Possible values are MASTER and SLAVE.

Return value

A ContactExp object.

Exceptions

None.

25.19.3 Members

The ContactExp object can have the following members:

name

A String specifying the repository key.

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

includedPairs

A RegionPairs object specifying the domain pairs included in contact.

excludedPairs

A RegionPairs object specifying the domain pairs excluded from contact.

contactPropertyAssignments

A ContactPropertyAssignment object specifying the contact property assignments in the contact domain.

surfaceThicknessAssignments

A SurfaceThicknessAssignment object specifying the surface thickness assignments in the contact domain.

surfaceOffsetAssignments

A SurfaceOffsetAssignment object specifying the surface offset fraction assignments in the contact domain.

surfaceFeatureAssignments

A SurfaceFeatureAssignment object specifying the surface feature angle assignments in the contact domain.

smoothingAssignments

A SmoothingAssignment object specifying the surface smoothing assignments in the contact domain.

masterSlaveAssignments

A MasterSlaveAssignment object specifying the master-slave assignments in the contact domain.

25.19.4 Corresponding analysis keywords

*CONTACT

25.20 ContactInitialization object

The ContactInitialization object is the abstract base type for other ContactInitialization objects. The ContactInitialization object has no explicit constructor, members, or methods.

Access

```
import interaction
mdb.models[name].contactInitializations[name]
```

25.20.1 Members

The ContactInitialization object has no members.

25.21 ContactProperty object

The ContactProperty object defines a contact interaction property.

The ContactProperty object is derived from the InteractionProperty object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.21.1 ContactProperty(...)

This method creates a ContactProperty object.

Path

```
mdb.models[name].ContactProperty
```

Required argument

name

A String specifying the interaction property repository key.

Optional arguments

None.

Return value

A ContactProperty object.

Exceptions

None.

25.21.2 Members

The ContactProperty object can have the following members:

tangentialBehavior

A ContactTangentialBehavior object.

normalBehavior

A NormalBehavior object.

damping

A ContactDamping object.

damage

A ContactDamage object.

fractureCriterion

A FractureCriterion object.

cohesiveBehavior

A CohesiveBehavior object.

thermalConductance

A ThermalConductance object.

heatGeneration

A GapHeatGeneration object.

radiation

A Radiation object.

geometricProperties

A GeometricProperties object.

electricalConductance

A GapElectricalConductance object.

25.21.3 Corresponding analysis keywords

*SURFACE INTERACTION

25.22 ContactPropertyAssignment object

The ContactPropertyAssignment object stores the contact property assignment definition for domain pairs in ContactExp and ContactStd objects. The ContactPropertyAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].contactPropertyAssignments
```

25.22.1 changeValuesInStep(...)

This method allows modification of contact property assignments to domain pairs already defined in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the contact property assignments are to be modified.

index

An Int specifying the position of the contact property assignment whose value is to be modified.

value

A String specifying the value of the contact property to be assigned to the domain pair whose index is referenced.

Optional arguments

None.

Return value

None

Exceptions

None.

25.22.2 appendInStep(...)

This method allows addition of contact property assignments to new domain pairs in a given step.

Arguments**Required arguments***stepName*

A String specifying the name of the step in which new contact property assignments are to be defined.

assignments

A sequence of tuples specifying the properties assigned to each surface pair. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL.
- A region object or the SymbolicConstant SELF. When used with a ContactExp object, this parameter can also be a string that references an Eulerian material surface.
- A String specifying a ContactProperty object associated with this pair of regions.

Optional arguments

None.

Return value

None

Exceptions

None.

25.22.3 delete(...)

The **delete** method allows you to delete existing contact property assignments.

Arguments

Required arguments

indices

A sequence of Ints specifying the index of each contact property assignment to delete. The *indices* and *surfPair* arguments are mutually exclusive.

surfPair

A sequence of tuples specifying the surface pair of each contact property assignment to delete. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL.
- A region object or the SymbolicConstant SELF. When used with a ContactExp object, this parameter can also be a string that references an Eulerian material surface.

surfPair and *indices* arguments are mutually exclusive.

Optional arguments

None.

Return value

None

Exceptions

None.

25.22.4 Members

The ContactPropertyAssignment object has no members.

25.22.5 Corresponding analysis keywords

*CONTACT PROPERTY ASSIGNMENT

25.23 ContactStabilization object

The ContactStabilization object is the abstract base type for other ContactStabilization objects. The ContactStabilization object has no explicit constructor, members, or methods.

Access

```
import interaction
mdb.models[name].contactStabilizations[name]
```

25.23.1 Members

The ContactStabilization object has no members.

25.24 ContactStd object

The ContactStd object defines the contact domain and associated properties during contact in an Abaqus/Standard analysis.

The ContactStd object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.24.1 ContactStd(...)

This method creates a ContactStd object.

Path

```
mdb.models[name].ContactStd
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which this contact interaction is created.

Optional arguments

useAllstar

A Boolean specifying whether the contacting surface pairs consist of all exterior faces in the model.

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

INTERACTION COMMANDS

includedPairs

A RegionPairs object specifying the domain pairs included in contact.

excludedPairs

A RegionPairs object specifying the domain pairs excluded from contact.

contactPropertyAssignments

A ContactPropertyAssignment object specifying the contact property assignments in the contact domain.

surfaceThicknessAssignments

A SurfaceThicknessAssignment object specifying the surface thickness assignments in the contact domain.

surfaceOffsetAssignments

A SurfaceOffsetAssignment object specifying the surface offset fraction assignments in the contact domain.

surfaceFeatureAssignments

A SurfaceFeatureAssignment object specifying the surface feature angle assignments in the contact domain.

masterSlaveAssignments

A MasterSlaveAssignment object specifying the master-slave assignments in the contact domain.

initializationAssignments

An InitializationAssignment object specifying the contact initialization assignments in the contact domain.

stabilizationAssignments

A StabilizationAssignment object specifying the contact stabilization assignments in the contact domain.

smoothingAssignments

A SmoothingAssignment object specifying the surface smoothing assignments in the contact domain.

Return value

A ContactStd object.

Exceptions

None.

25.24.2 ContactStd(...)

This method creates a ContactStd object.

Path

`mdb.models [name] .ContactStd`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which this contact interaction is created.

Optional arguments

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

useAllstar

A Boolean specifying whether the contacting surface pairs consist of all exterior faces in the model.

includedPairs

A sequence of pairs of Region objects or SymbolicConstants that specifies the surface pairs in contact. Possible values of the SymbolicConstants are ALLSTAR and SELF. This argument is valid only when *useAllstar*=OFF.

excludedPairs

A sequence of pairs of Region objects or SymbolicConstants that specifies the surface pairs excluded from contact. Possible values of the SymbolicConstants are ALLSTAR and SELF.

contactPropertyAssignments

A sequence of tuples specifying the properties assigned to each surface pair. Each tuple contains three entries:

- A Region object or the SymbolicConstant GLOBAL.
- A Region object or the SymbolicConstant SELF.
- A String specifying an InteractionProperty object associated with this pair of regions.

surfaceFeatureAssignments

A sequence of tuples specifying the surface feature angle assignments in the contact domain. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface feature angle is assigned.
- A Float or a SymbolicConstant specifying the overriding feature angle value to be used in the contact definition. Possible values of the SymbolicConstant are PERIMETER or NONE.

INTERACTION COMMANDS

surfaceThicknessAssignments

A sequence of tuples specifying the surface thickness assignments in the contact domain. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface thickness is assigned.
- A Float or a SymbolicConstant specifying the overriding thickness value to be used in the contact definition. The SymbolicConstant ORIGINAL may be specified instead of a floating point value.
- A Float specifying a scale factor that multiplies the thickness value specified in the second entry.

surfaceOffsetAssignments

A sequence of tuples specifying the surface offset fraction assignments in the contact domain. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the surface offset fraction is assigned.
- A Float or a SymbolicConstant specifying the offset fraction value to be used in the contact definition. Possible values of the SymbolicConstant are ORIGINAL, SPOS, or SNEG.

masterSlaveAssignments

A sequence of tuples specifying master-slave assignments in the contact domain. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL specifying the first surface that defines the master-slave assignment.
- A region object specifying the second surface in the master-slave assignment definition.
- A SymbolicConstant specifying the status of the first surface. Possible values are MASTER, SLAVE, and BALANCED.

initializationAssignments

A sequence of tuples specifying the contact initialization data assigned to each surface pair. Each tuple contains three entries:

- A Region object or the SymbolicConstant GLOBAL.
- A Region object or the SymbolicConstant SELF.
- A String specifying a StdInitialization object associated with this pair of regions.

stabilizationAssignments

A sequence of tuples specifying the contact stabilization assigned to each surface pair. Each tuple contains three entries:

- A Region object or the SymbolicConstant GLOBAL.
- A Region object or the SymbolicConstant SELF.

- A String specifying a StdStabilization object associated with this pair of regions.

smoothingAssignments

A sequence of tuples specifying the surface smoothing assignments in the contact domain. Each tuple contains two entries:

- A region object specifying the surface to which the smoothing option is assigned.
- A SymbolicConstant specifying the smoothing option to be used in the contact definition. Possible values of the SymbolicConstant are NONE, REVOLUTION, SPHERICAL, or TOROIDAL.

Return value

A ContactStd object.

Exceptions

None.

25.24.3 Members

The ContactStd object can have the following members:

name

A String specifying the repository key.

globalSmoothing

A Boolean specifying whether surface smoothing (geometric correction) is automatically applied to all eligible surfaces. The default value is ON.

includedPairs

A RegionPairs object specifying the domain pairs included in contact.

excludedPairs

A RegionPairs object specifying the domain pairs excluded from contact.

contactPropertyAssignments

A ContactPropertyAssignment object specifying the contact property assignments in the contact domain.

surfaceThicknessAssignments

A SurfaceThicknessAssignment object specifying the surface thickness assignments in the contact domain.

surfaceOffsetAssignments

A SurfaceOffsetAssignment object specifying the surface offset fraction assignments in the contact domain.

INTERACTION COMMANDS

masterSlaveAssignments

A MasterSlaveAssignment object specifying the master-slave assignments in the contact domain.

initializationAssignments

An InitializationAssignment object specifying the contact initialization assignments in the contact domain.

stabilizationAssignments

A StabilizationAssignment object specifying the contact stabilization assignments in the contact domain.

smoothingAssignments

A SmoothingAssignment object specifying the surface smoothing assignments in the contact domain.

surfaceFeatureAssignments

A SurfaceFeatureAssignment object specifying the surface feature angle assignments in the contact domain.

25.24.4 Corresponding analysis keywords

*CONTACT

25.25 ContactTangentialBehavior object

The ContactTangentialBehavior object specifies tangential behavior for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].tangentialBehavior
```

25.25.1 TangentialBehavior(...)

This method creates a ContactTangentialBehavior object.

Path

```
mdb.models[name].interactionProperties[name].TangentialBehavior
```

Required arguments

None.

Optional arguments*formulation*

A SymbolicConstant specifying the friction formulation. Possible values are FRICTIONLESS, PENALTY, EXPONENTIAL_DECAY, ROUGH, LAGRANGE, and USER_DEFINED. The default value is FRICTIONLESS.

directionality

A SymbolicConstant specifying the directionality of the friction. Possible values are ISOTROPIC and ANISOTROPIC. The default value is ISOTROPIC.

slipRateDependency

A Boolean specifying whether the data depend on slip rate. The default value is OFF.

pressureDependency

A Boolean specifying whether the data depend on contact pressure. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variables. The default value is 0.

exponentialDecayDefinition

A SymbolicConstant specifying the exponential decay definition. Possible values are COEFFICIENTS and TEST_DATA. The default value is COEFFICIENTS.

table

A sequence of sequences of Floats specifying tangential behavior. The items in the table data are described below.

shearStressLimit

None or a Float specifying the shear stress limit. If *shearStressLimit=None*, there is no upper limit. The default value is **None**.

maximumElasticSlip

A SymbolicConstant specifying what the maximum elastic slip will be. Possible values are FRACTION and ABSOLUTE_DISTANCE. The default value is FRACTION.

fraction

A Float specifying the fraction of a characteristic surface dimension. The default value is 0.0.

absoluteDistance

A Float specifying the absolute distance. The default value is 0.0.

elasticSlipStiffness

None or a Float specifying the elastic slip stiffness. If *elasticSlipStiffness=None*, there is no upper limit. The default value is **None**.

nStateDependentVars

An Int specifying the number of state-dependent variables. The default value is 0.

useProperties

A Boolean specifying whether property values will be used. The default value is OFF.

INTERACTION COMMANDS

Table data

If *formulation*=PENALTY or LAGRANGE, the table data specify the following:

- Friction coefficient in the first slip direction, μ_1 .
- Friction coefficient in the second slip direction, μ_2 (if *directionality*=ANISOTROPIC).
- Slip rate, if the data depend on slip rate.
- Contact pressure, if the data depend on contact pressure.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=COEFFICIENTS, the table data specify the following:

- Static friction coefficient.
- Kinetic friction coefficient.
- Decay coefficient.

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=TEST_DATA, the table data specify the following:

- Friction coefficient.
- Slip rate.

If *formulation*=USER_DEFINED, the table data specify the following:

- Friction property.

Return value

A ContactTangentialBehavior object.

Exceptions

None.

25.25.2 setValues(...)

This method modifies the ContactTangentialBehavior object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ContactTangentialBehavior** method.

Return value

None

Exceptions

None.

25.25.3 Members

The **ContactTangentialBehavior** object has the following members:

formulation

A **SymbolicConstant** specifying the friction formulation. Possible values are **FRICTIONLESS**, **PENALTY**, **EXPONENTIAL_DECAY**, **ROUGH**, **LAGRANGE**, and **USER_DEFINED**. The default value is **FRICTIONLESS**.

directionality

A **SymbolicConstant** specifying the directionality of the friction. Possible values are **ISOTROPIC** and **ANISOTROPIC**. The default value is **ISOTROPIC**.

slipRateDependency

A **Boolean** specifying whether the data depend on slip rate. The default value is **OFF**.

pressureDependency

A **Boolean** specifying whether the data depend on contact pressure. The default value is **OFF**.

temperatureDependency

A **Boolean** specifying whether the data depend on temperature. The default value is **OFF**.

dependencies

An **Int** specifying the number of field variables. The default value is 0.

exponentialDecayDefinition

A **SymbolicConstant** specifying the exponential decay definition. Possible values are **COEFFICIENTS** and **TEST_DATA**. The default value is **COEFFICIENTS**.

shearStressLimit

None or a **Float** specifying the shear stress limit. If *shearStressLimit*=**None**, there is no upper limit. The default value is **None**.

maximumElasticSlip

A **SymbolicConstant** specifying what the maximum elastic slip will be. Possible values are **FRACTION** and **ABSOLUTE_DISTANCE**. The default value is **FRACTION**.

fraction

A **Float** specifying the fraction of a characteristic surface dimension. The default value is 0.0.

INTERACTION COMMANDS

absoluteDistance

A Float specifying the absolute distance. The default value is 0.0.

elasticSlipStiffness

None or a Float specifying the elastic slip stiffness. If *elasticSlipStiffness*=**None**, there is no upper limit. The default value is **None**.

nStateDependentVars

An Int specifying the number of state-dependent variables. The default value is 0.

useProperties

A Boolean specifying whether property values will be used. The default value is OFF.

table

A tuple of tuples of Floats specifying tangential behavior. The items in the table data are described below.

25.25.4 Corresponding analysis keywords

*FRICTION

*CHANGE FRICTION

25.26 CyclicSymmetry object

The CyclicSymmetry object defines a cyclic symmetry analysis.

The CyclicSymmetry object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.26.1 CyclicSymmetry(...)

This method creates a CyclicSymmetry object.

Path

```
mdb.models[name].CyclicSymmetry
```

Required arguments*name*

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the cyclic symmetry interaction should be created.

master

A Region object specifying the master surface.

slave

A Region object specifying the slave surface.

repetitiveSectors

An Int specifying the total number of sectors in the cyclic symmetric model.

axisPoint1

A Region object specifying the first point of the axis of symmetry. The region should contain exactly one mesh node, vertex, interesting point, reference point or datum point. In a two-dimensional model *axisPoint1* is the only point used to define the axis of symmetry.

axisPoint2

A Region object specifying the second point of the axis of symmetry. The region should contain exactly one mesh node, vertex, interesting point, reference point or datum point. In a two-dimensional model this point is ignored.

Optional arguments*extractedNodalDiameter*

A SymbolicConstant specifying whether Abaqus should extract all possible nodal diameters or the nodal diameters between the user-specified values for *lowestNodalDiameter* and *highestNodalDiameter*. Possible values are ALL_NODAL_DIAMETER and SPECIFIED_NODAL_DIAMETER. The default value is ALL_NODAL_DIAMETER.

lowestNodalDiameter

An Int specifying the lowest nodal diameter to be used in the eigenfrequency analysis. The default value is 0.

highestNodalDiameter

An Int specifying the highest nodal diameter to be used in the eigenfrequency analysis. This argument value should be less than or equal to the half of the total number of sectors (as specified in the *repetitiveSectors* parameter). The default value is 0.

excitationNodalDiameter

An Int specifying the nodal diameter for which the modal-based steady-state dynamic analysis will be performed. This value should be greater than or equal to the lowest nodal diameter (specified in the *lowestNodalDiameter* parameter), and less than or equal to the highest nodal diameter (specified in the *highestNodalDiameter* parameter). The default value is 0.

INTERACTION COMMANDS

adjustTie

A Boolean specifying whether or not to adjust the slave surface of the cyclic symmetry to tie it to the master surface. The default value is ON.

positionTolerance

A Float specifying the position tolerance. The *positionTolerance* argument applies only when *positionToleranceMethod*=SPECIFY_TOLERANCE. The default value is 0.0.

positionToleranceMethod

A SymbolicConstant specifying the method used to determine the position tolerance. Possible values are COMPUTED_TOLERANCE and SPECIFY_TOLERANCE. The default value is COMPUTED_TOLERANCE.

Return value

A CyclicSymmetry object.

Exceptions

None.

25.26.2 swapSurfaces()

This method switches the master and slave surfaces of a cyclic symmetry interaction. This command is valid only during the step in which the interaction is created.

Arguments

None.

Return value

None

Exceptions

None.

25.26.3 setValues(...)

This method modifies the data for an existing CyclicSymmetry object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CyclicSymmetry** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.26.4 setValuesInStep(...)

This method modifies the propagating data of an existing **CyclicSymmetry** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

extractedNodalDiameter

A **SymbolicConstant** specifying whether Abaqus should extract all possible nodal diameters or the nodal diameters between the user-specified values for *lowestNodalDiameter* and *highestNodalDiameter*. Possible values are **ALL_NODAL_DIAMETER** and **SPECIFIED_NODAL_DIAMETER**. The default value is **ALL_NODAL_DIAMETER**.

lowestNodalDiameter

An Int specifying the lowest nodal diameter to be used in the eigenfrequency analysis. The default value is 0.

highestNodalDiameter

An Int specifying the highest nodal diameter to be used in the eigenfrequency analysis. This argument value should be less than or equal to the half of the total number of sectors (as specified in the *repetitiveSectors* parameter). The default value is 0.

excitationNodalDiameter

An Int specifying the nodal diameter for which the modal-based steady-state dynamic analysis will be performed. This value should be greater than or equal to the lowest nodal diameter (specified in the *lowestNodalDiameter* parameter), and less than or equal to the highest nodal diameter (specified in the *highestNodalDiameter* parameter). The default value is 0.

Return value

None

Exceptions

None.

25.26.5 Members

The `CyclicSymmetry` object has members with the same names and descriptions as the arguments to the `CyclicSymmetry` method except the optional arguments to the `setValuesInStep` method.

25.26.6 Corresponding analysis keywords

- *CLOAD, CYCLIC MODE
- *CYCLIC SYMMETRY MODEL
- *DLOAD, CYCLIC MODE
- *DSLOAD, CYCLIC MODE
- *SELECT CYCLIC SYMMETRY MODES
- *TIE, CYCLIC SYMMETRY

25.27 CyclicSymmetryState object

The `CyclicSymmetryState` object stores the propagating data for a `CyclicSymmetry` object. One instance of this object is created internally by the `CyclicSymmetry` object for each step. The instance is also deleted internally by the `CyclicSymmetry` object.

The `CyclicSymmetryState` object has no constructor or methods.

The `CyclicSymmetryState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.27.1 Members

The `CyclicSymmetryState` object has the following members:

extractedNodalDiameter

A SymbolicConstant specifying whether Abaqus should extract all possible nodal diameters or the nodal diameters between the user-specified values for *lowestNodalDiameter* and *highestNodalDiameter*. Possible values are ALL_NODAL_DIAMETER and SPECIFIED_NODAL_DIAMETER. The default value is ALL_NODAL_DIAMETER.

extractedNodalDiameterState

A SymbolicConstant specifying the propagation state of the *extractedNodalDiameter* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

lowestNodalDiameter

An Int specifying the lowest nodal diameter to be used in the eigenfrequency analysis. The default value is 0.

lowestNodalDiameterState

A SymbolicConstant specifying the propagation state of the *lowestNodalDiameter* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

highestNodalDiameter

An Int specifying the highest nodal diameter to be used in the eigenfrequency analysis. This argument value should be less than or equal to the half of the total number of sectors. The default value is 0.

highestNodalDiameterState

A SymbolicConstant specifying the propagation state of the *highestNodalDiameter* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

excitationNodalDiameter

An Int specifying the nodal diameter for which the modal-based steady-state dynamic analysis will be performed. This value should be greater than or equal to the lowest nodal diameter (specified in the *lowestNodalDiameter* parameter), and less than or equal to the highest nodal diameter (specified in the *highestNodalDiameter* parameter). The default value is 0.

excitationNodalDiameterState

A SymbolicConstant specifying the propagation state of the *excitationNodalDiameter* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.27.2 Corresponding analysis keywords

- *CLOAD, CYCLIC MODE
- *DLOAD, CYCLIC MODE
- *DSLOAD, CYCLIC MODE
- *SELECT CYCLIC SYMMETRY MODES

25.28 ElasticFoundation object

The ElasticFoundation object defines a mechanical foundation.

The ElasticFoundation object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.28.1 ElasticFoundation(...)

This method creates an ElasticFoundation object.

Path

```
mdb.models[name].ElasticFoundation
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the ElasticFoundation object is created.
createStepName must be set to 'Initial'.

surface

A Region object specifying the surface to which the foundation applies.

stiffness

A Float specifying the foundation stiffness per area (or per length for beams).

Optional arguments

None.

Return value

An ElasticFoundation object.

Exceptions

None.

25.28.2 setValues(...)

This method modifies the data for an existing ElasticFoundation object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ElasticFoundation** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.28.3 setValuesInStep(...)

This method modifies the propagating data of an existing ElasticFoundation object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the interaction is modified.

Optional argument

stiffness

A Float specifying the foundation stiffness per area (or per length for beams).

Return value

None

Exceptions

None.

25.28.4 Members

The ElasticFoundation object has members with the same names and descriptions as the arguments to the **ElasticFoundation** method except the optional arguments to the **setValuesInStep** method.

25.28.5 Corresponding analysis keywords

*FOUNDATION

25.29 ElasticFoundationState object

The ElasticFoundationState object stores the propagating data for an ElasticFoundation object. One instance of this object is created internally by the ElasticFoundation object for each step. The instance is also deleted internally by the ElasticFoundation object.

The ElasticFoundationState object has no constructor or methods.

The ElasticFoundationState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.29.1 Members

The ElasticFoundationState object has the following members:

stiffness

A Float specifying the foundation stiffness per area.

stiffnessState

A SymbolicConstant specifying the propagation state of the stiffness member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.30 ExpContactControl object

The ExpContactControl object is used in Abaqus/Explicit analyses to specify optional solution controls for problems involving contact between bodies.

The ExpContactControl object is derived from the ContactControl object.

Access

```
import interaction
mdb.models[name].contactControls[name]
```

25.30.1 ExpContactControl(...)

This method creates an ExpContactControl object.

Path

```
mdb.models[name].ExpContactControl
```

Required argument

name

A String specifying the contact controls repository key.

Optional arguments

globTrkChoice

A SymbolicConstant specifying whether or not the default value will be used for the maximum number of increments between global contact searches. Possible values are DEFAULT and SPECIFY. The default value is DEFAULT.

globTrkInc

An Int specifying the maximum number of increments between global contact searches. The *globTrkInc* argument applies only when *globTrkChoice*=SPECIFY. The default value is 100 for surface-to-surface contact and 4 for self-contact.

fastLocalTrk

A Boolean specifying whether to use the more computationally efficient local tracking method. The default value is ON.

scalePenalty

A Float specifying the factor by which Abaqus/Explicit will scale the default penalty stiffness to obtain the stiffnesses used for the penalty contact pairs. The default value is 1.0.

warpCheckPeriod

An Int specifying the number of increments between checks for highly warped facets on master surfaces. The default value is 20.

warpCutoff

A Float specifying the out-of-plane warping angle (in degrees), at which a facet will be considered to be highly warped. The default value is 20.0.

Return value

An ExpContactControl object.

Exceptions

RangeError.

25.30.2 **setValues(...)**

This method modifies the ExpContactControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ExpContactControl** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

25.30.3 Members

The ExpContactControl object has members with the same names and descriptions as the arguments to the `ExpContactControl` method.

25.30.4 Corresponding analysis keywords

*CONTACT CONTROLS

25.31 FilmCondition object

The FilmCondition object defines film coefficients and associated sink temperatures for coupled temperature-displacement analyses.

The FilmCondition object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.31.1 FilmCondition(...)

This method creates a FilmCondition object.

Path

```
mdb.models[name].FilmCondition
```

Required arguments

name

A String specifying the repository key.

INTERACTION COMMANDS

createStepName

A String specifying the name of the step in which the FilmCondition object is created.

surface

A Region object specifying the name of the surface to which the film condition interaction is applied.

definition

A SymbolicConstant specifying how the film condition is defined. Possible values are EMBEDDED_COEFF, PROPERTY_REF, USER_SUB, and FIELD.

Optional arguments

interactionProperty

A String specifying the name of the FilmConditionProp object associated with this interaction. The *interactionProperty* argument applies only when *definition*=PROPERTY_REF. The default value is an empty string.

sinkTemperature

A Float specifying the reference sink temperature, θ^0 . The default value is 0.0.

sinkAmplitude

A String specifying the name of the Amplitude object that gives the variation of the sink temperature, θ^0 , with time. The default value is an empty string.

Note: Use empty string in an Abaqus/Standard analysis to specify that the reference sink temperature is applied immediately at the beginning of the step or linearly over the step. Use empty string in an Abaqus/Explicit analysis to specify that the reference sink temperature is applied throughout the step.

filmCoeff

A Float specifying the reference film coefficient value, h . The *filmCoeff* argument applies when *definition*=EMBEDDED_COEFF, *definition*=USER_SUB, or *definition*=FIELD. The default value is 0.0.

filmCoeffAmplitude

A String specifying the name of the Amplitude object that gives the variation of the film coefficient, h , with time. The default value is an empty string.

Note: Use empty string in an Abaqus/Standard analysis to specify that the reference film coefficient is applied immediately at the beginning of the step or linearly over the step. Use empty string in an Abaqus/Explicit analysis to specify that the reference film coefficient is applied throughout the step.

field

A String specifying the name of the AnalyticalField object associated with this interaction. The *field* argument applies only when *definition*=FIELD. The default value is an empty string.

sinkFieldName

A String specifying the name of the AnalyticalField or DiscreteField object associated with the sink temperature. The *sinkFieldName* argument applies only when *sinkDistributionType*=ANALYTICAL_FIELD or *sinkDistributionType*=DISCRETE_FIELD. The default value is an empty string.

sinkDistributionType

A SymbolicConstant specifying how the sink temperature is distributed. Possible values are UNIFORM, ANALYTICAL_FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

Return value

A FilmCondition object.

Exceptions

None.

25.31.2 setValues(...)

This method modifies the data for an existing FilmCondition object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FilmCondition** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.31.3 setValuesInStep(...)

This method modifies the propagating data of an existing FilmCondition object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

None.

The optional arguments to **setValuesInStep** are the same as the optional arguments to the **FilmConditionState** method.

Return value

None

Exceptions

None.

25.31.4 Members

The **FilmCondition** object has members with the same names and descriptions as the arguments to the **FilmCondition** method except the optional arguments to the **setValuesInStep** method.

25.32 FilmConditionProp object

The **FilmConditionProp** object is an interaction property that defines a film coefficient as a function of temperature and field variables.

The **FilmConditionProp** object is derived from the **InteractionProperty** object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.32.1 FilmConditionProp(...)

This method creates a **FilmConditionProp** object.

Path

```
mdb.models[name].FilmConditionProp
```


Required argument*name*

A String specifying the interaction property repository key.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

property

A sequence of sequences of Floats specifying the following:

- The film coefficient, h .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FilmConditionProp object.

Exceptions

None.

25.32.2 setValues(...)

This method modifies the FilmConditionProp object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FilmConditionProp** method, except for the *name* argument.

Return value

None

Exceptions

None.

25.32.3 Members

The `FilmConditionProp` object has members with the same names and descriptions as the arguments to the `FilmConditionProp` method.

25.32.4 Corresponding analysis keywords

*FILM PROPERTY

25.33 FilmConditionState object

The `FilmConditionState` object stores the propagating data for a `FilmCondition` object. One instance of this object is created internally by the `FilmCondition` object for each step. The instance is also deleted internally by the `FilmCondition` object.

The `FilmConditionState` object has no constructor or methods.

The `FilmConditionState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.33.1 Members

The `FilmConditionState` object has the following members:

interactionPropertyState

A `SymbolicConstant` specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

sinkTemperature

A `Float` specifying the sink temperature.

sinkTemperatureState

A `SymbolicConstant` specifying the propagation state of the *sinkTemperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

sinkAmplitudeState

A SymbolicConstant specifying the propagation state of the *sinkAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

filmCoeff

A Float specifying the film coefficient.

filmCoeffState

A SymbolicConstant specifying the propagation state of the *filmCoeff* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

filmCoeffAmplitudeState

A SymbolicConstant specifying the propagation state of the *filmCoeffAmplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the FilmConditionProp object associated with this interaction.

sinkAmplitude

A String specifying the name of the Amplitude object that gives the variation of the sink temperature.

filmCoeffAmplitude

A String specifying the name of the Amplitude object that gives the variation of the film coefficient.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.33.2 Corresponding analysis keywords

*SFILM

*FILM

25.34 FluidCavity object

The FluidCavity object defines a surface-based cavity.

The FluidCavity object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.34.1 FluidCavity(...)

This method creates an FluidCavity object.

Path

```
mdb.models[name].FluidCavity
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the FluidCavity object is created.

cavityPoint

A Region object specifying the fluid cavity reference point.

cavitySurface

A Region object specifying the surface forming the boundary of the fluid cavity.

interactionProperty

A String specifying the FluidCavityProperty object associated with this interaction.

Optional arguments

ambientPressure

A Float specifying the magnitude of the ambient pressure. The default value is 0.0.

thickness

A Float specifying the out-of-plane thickness of the surface for two-dimensional models. This argument is valid only when using two-dimensional models. The default value is 1.0.

useAdiabatic

A Boolean specifying whether adiabatic behavior is assumed for the ideal gas. This argument is valid only when *interactionProperty* specifies a pneumatic definition. The default value is OFF.

checkNormals

A Boolean specifying whether the analysis will check the consistency of the surface normals. The default value is ON.

Return value

A FluidCavity object.

Exceptions

None.

25.34.2 setValues(...)

This method modifies the FluidCavity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidCavity** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.34.3 Members

The FluidCavity object has members with the same names and descriptions as the arguments to the **FluidCavity** method.

25.34.4 Corresponding analysis keywords

*FLUID CAVITY

25.35 FluidCavityProperty object

The FluidCavityProperty object is an interaction property that defines the fluid behavior for a surface-based fluid cavity.

The FluidCavityProperty object is derived from the InteractionProperty object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.35.1 FluidCavityProperty(...)

This method creates a FluidCavityProperty object.

Path

```
mdb.models[name].FluidCavityProperty
```

Required argument

name

A String specifying the interaction property repository key.

Optional arguments

definition

A SymbolicConstant specifying the type of fluid cavity property to be defined. Possible values are HYDRAULIC and PNEUMATIC. The default value is HYDRAULIC.

fluidDensity

None or a Float specifying the reference fluid density. This argument is applicable only when *definition*=HYDRAULIC, and is required in that case. The default value is **None**.

molecularWeight

None or a Float specifying the molecular weight of the ideal gas species. This argument is applicable only when *definition*=PNEUMATIC, and is required in that case. The default value is **None**.

useExpansion

A Boolean specifying whether thermal expansion coefficients will be defined. This argument is applicable only when *definition*=HYDRAULIC. The default value is OFF.

expansionTempDep

A Boolean specifying whether the thermal fluid expansion data will have temperature dependency. This argument is applicable only when *definition*=HYDRAULIC and when *useExpansion*=True. The default value is OFF.

expansionDependencies

An Int specifying the number of field variable dependencies in the thermal fluid expansion data. This argument is applicable only when *definition*=HYDRAULIC and when *useExpansion*=True. The default value is 0.

referenceTemperature

A Float specifying the reference temperature for the coefficient of thermal expansion. This argument is applicable only when *definition*=HYDRAULIC, when *useExpansion*=True, and when either *expansionTempDep*=True or when *expansionDependencies* is greater than 0. The default value is 0.0.

expansionTable

A sequence of sequences of Floats specifying the thermal expansion coefficients. This argument is applicable only when *definition*=HYDRAULIC and when *useExpansion*=True. Each sequence contains the following data:

- The mean coefficient of thermal expansion.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

useBulkModulus

A Boolean specifying whether fluid bulk modulus values will be defined. This argument is applicable only when *definition*=HYDRAULIC. The default value is OFF.

bulkModulusTempDep

A Boolean specifying whether the fluid bulk modulus data will have temperature dependency. This argument is applicable only when *definition*=HYDRAULIC and when *useBulkModulus*=True. The default value is OFF.

bulkModulusDependencies

An Int specifying the number of field variable dependencies in the fluid bulk modulus data. This argument is applicable only when *definition*=HYDRAULIC and when *useBulkModulus*=True. The default value is 0.

bulkModulusTable

A sequence of sequences of Floats specifying the fluid bulk modulus values. This argument is applicable only when *definition*=HYDRAULIC and when *useBulkModulus*=True. Each sequence contains the following data:

- The fluid bulk modulus.

INTERACTION COMMANDS

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

useCapacity

A Boolean specifying whether molar heat capacity values will be defined. This argument is applicable only when *definition*=PNEUMATIC. The default value is OFF.

capacityType

A SymbolicConstant specifying the method to define the molar heat capacity. Possible values are POLYNOMIAL and TABULAR. The default value is POLYNOMIAL.

capacityTempDep

A Boolean specifying whether the molar heat capacity data will have temperature dependency. This argument is applicable only when *definition*=PNEUMATIC, when *useCapacity*=True, and when *capacityType*=TABULAR. The default value is OFF.

capacityDependencies

An Int specifying the number of field variable dependencies in the molar heat capacity data. This argument is applicable only when *definition*=PNEUMATIC, when *useCapacity*=True, and when *capacityType*=TABULAR. The default value is 0.

capacityTable

A sequence of sequences of Floats specifying the molar heat capacity values in the form of a polynomial expression. This argument is applicable only when *definition*=PNEUMATIC, when *useCapacity*=True, and when *capacityType*=POLYNOMIAL. In this form, only one sequence is specified and that sequence contains the following data:

- The first molar heat capacity coefficient.
- The second molar heat capacity coefficient.
- The third molar heat capacity coefficient.
- The fourth molar heat capacity coefficient.
- The fifth molar heat capacity coefficient.

Alternatively, the sequence data may specify the molar heat capacity values at constant pressure for an ideal gas species. This argument is applicable only when *definition*=PNEUMATIC, when *useCapacity*=True, and when *capacityType*=TABULAR. Each sequence contains the following data:

- The molar heat capacity at constant pressure.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FluidCavityProperty object.

Exceptions

None.

25.35.2 setValues(...)

This method modifies the FluidCavityProperty object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidCavityProperty** method, except for the *name* argument.

Return value

None

Exceptions

None.

25.35.3 Members

The FluidCavityProperty object has members with the same names and descriptions as the arguments to the **FluidCavityProperty** method.

25.35.4 Corresponding analysis keywords

- *FLUID BEHAVIOR
- *CAPACITY
- *FLUID BULK MODULUS
- *FLUID DENSITY
- *FLUID EXPANSION
- *MOLECULAR WEIGHT

25.36 FluidCavityState object

The FluidCavityState object stores the propagating data for an FluidCavity object. One instance of this object is created internally by the FluidCavity object for each step. The instance is also deleted internally by the FluidCavity object.

The FluidCavityState object has no constructor or methods.

The FluidCavityState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.36.1 Members

The FluidCavityState object has the following member:

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.37 FluidExchange object

The FluidExchange object is used to define fluid exchange between two fluid cavities or between a fluid cavity and its environment.

The FluidExchange object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.37.1 FluidExchange(...)

This method creates an FluidExchange object.

Path

```
mdb.models[name].FluidExchange
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the FluidExchange object is created.

firstCavity

A String specifying the first FluidCavity object associated with this interaction. This will be the only cavity specified if *definition*=TO_ENVIRONMENT.

interactionProperty

A String specifying the FluidExchangeProperty object associated with this interaction.

Optional arguments

definition

A SymbolicConstant specifying the type of fluid exchange to be defined. Possible values are TO_ENVIRONMENT and BETWEEN_CAVITIES. The default value is TO_ENVIRONMENT.

secondCavity

A String specifying the second FluidCavity object associated with this interaction. This argument is applicable only when *definition*=BETWEEN_CAVITIES.

exchangeArea

A Float specifying the effective exchange area. The default value is 1.0.

Return value

A FluidExchange object.

Exceptions

None.

25.37.2 **setValues(...)**

This method modifies the `FluidExchange` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidExchange** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.37.3 **Members**

The `FluidExchange` object has members with the same names and descriptions as the arguments to the **FluidExchange** method.

25.37.4 **Corresponding analysis keywords**

*FLUID EXCHANGE

25.38 **FluidExchangeProperty object**

The `FluidExchangeProperty` object is an interaction property that defines the fluid exchange property for a flow between two fluid cavities or between a fluid cavity and its environment.

The `FluidExchangeProperty` object is derived from the `InteractionProperty` object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.38.1 FluidExchangeProperty(...)

This method creates a FluidExchangeProperty object.

Path

`mdb.models[name].FluidExchangeProperty`

Required arguments

name

A String specifying the interaction property repository key.

dataTable

A sequence of sequences of Floats specifying the viscous and hydrodynamic resistance coefficients when *definition*=BULK_VISCOSITY. Each sequence contains the following data:

- The viscous resistance coefficient.
- The hydrodynamic resistance coefficient.
- The average absolute pressure, if the data depend on pressure.
- The average temperature, if the data depend on temperature.
- The value of the first field variable, if the data depend on field variables.
- The value of the second field variable.
- Etc.

Alternatively, the sequence data may specify the mass flow rate. This is applicable only when *definition*=MASS_FLUX. In this form, only one sequence is specified and that sequence contains the following data:

- The mass flow rate per unit area.

Alternatively, the sequence data may specify the mass rate leakage. This is applicable only when *definition*=MASS_RATE_LEAK. Each sequence contains the following data:

- The absolute value of the mass flow rate per unit area. (The first tabular value entered must always be zero.)
- The absolute value of the pressure difference. (The first tabular value entered must always be zero.)
- The average absolute pressure, if the data depend on pressure.
- The average temperature, if the data depend on temperature.
- The value of the first field variable, if the data depend on field variables.
- The value of the second field variable.
- Etc.

INTERACTION COMMANDS

Alternatively, the sequence data may specify the volume flow rate. This is applicable only when *definition*=VOL_FLUX. In this form, only one sequence is specified and that sequence contains the following data:

- The volumetric flow rate per unit area.

Alternatively, the sequence data may specify the volume rate leakage. This is applicable only when *definition*=VOL_RATE_LEAK. Each sequence contains the following data:

- The absolute value of the volumetric flow rate per unit area. (The first tabular value entered must always be zero.)
- The absolute value of the pressure difference. (The first tabular value entered must always be zero.)
- The average absolute pressure, if the data depend on pressure.
- The average temperature, if the data depend on temperature.
- The value of the first field variable, if the data depend on field variables.
- The value of the second field variable.
- Etc.

Optional arguments

definition

A SymbolicConstant specifying the type of fluid exchange property to be defined. Possible values are BULK_VISCOSITY, MASS_FLUX, MASS_RATE_LEAK, VOL_FLUX, and VOL_RATE_LEAK. The default value is BULK_VISCOSITY.

pressureDependency

A Boolean specifying whether the data will have pressure dependency. This argument is applicable only when *definition*=BULK_VISCOSITY, or when *definition*=MASS_RATE_LEAK, or when *definition*=VOL_RATE_LEAK. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data will have temperature dependency. This argument is applicable only when *definition*=BULK_VISCOSITY, or when *definition*=MASS_RATE_LEAK, or when *definition*=VOL_RATE_LEAK. The default value is OFF.

fieldDependencies

An Int specifying the number of field variable dependencies in the data. This argument is applicable only when *definition*=BULK_VISCOSITY, or when *definition*=MASS_RATE_LEAK, or when *definition*=VOL_RATE_LEAK. The default value is 0.

Return value

A FluidExchangeProperty object.

Exceptions

None.

25.38.2 setValues(...)

This method modifies the `FluidExchangeProperty` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `FluidExchangeProperty` method, except for the *name* argument.

Return value

None

Exceptions

None.

25.38.3 Members

The `FluidExchangeProperty` object has members with the same names and descriptions as the arguments to the `FluidExchangeProperty` method.

25.38.4 Corresponding analysis keywords

*FLUID EXCHANGE PROPERTY

25.39 FluidExchangeState object

The `FluidExchangeState` object stores the propagating data for an `FluidExchange` object. One instance of this object is created internally by the `FluidExchange` object for each step. The instance is also deleted internally by the `FluidExchange` object.

The `FluidExchangeState` object has no constructor or methods.

The `FluidExchangeState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.39.1 Members

The FluidExchangeState object has the following member:

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.40 FractureCriterion object

The FractureCriterion object specifies fractureCriterion options for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].fractureCriterion
```

25.40.1 FractureCriterion(...)

This method creates a FractureCriterion object.

Path

```
mdb.models[name].interactionProperties[name].FractureCriterion
```


Required argument*initTable*

A sequence of sequences of Floats specifying the value defining the fracture criterion. The items in the table data are described below.

Optional arguments*type*

A SymbolicConstant specifying the type of data used to define the fracture criterion. Possible values are VCCT and ENHANCED VCCT. The default value is VCCT.

mixedModeBehavior

A SymbolicConstant specifying the mixed mode behavior type used to define fracture criterion. Possible values are BK, POWER, and REEDER. The default value is BK.

temperatureDependency

A Boolean specifying whether the fracture criterion data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of fracture criterion data field variables. The default value is 0.

tolerance

A Float specifying the tolerance for VCCT\Enhanced VCCT type. The default value is 0.2.

specifyUnstableCrackProp

A SymbolicConstant specifying whether to include unstable crack growth tolerance in fracture criterion. Possible values are ON and OFF. The default value is OFF.

unstableTolerance

The SymbolicConstant DEFAULT or a Float specifying the tolerance for unstable crack propagation. This parameter specified only if *specifyUnstableCrackProp*=ON. The default value is DEFAULT.

Table data

Table data for *initTable*:

If *type*=VCCT for *mixedModeBehavior*=BK or REEDER, the table data specify the following:

- Mode I critical energy release rate, G_{IC} .
- Mode II critical energy release rate, G_{IIC} .
- Mode III critical energy release rate, G_{IIIC} .
- Exponent, η .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=VCCT for *mixedModeBehavior*=POWER, the table data specify the following:

INTERACTION COMMANDS

- Mode I critical energy release rate, G_{IC} .
- Mode II critical energy release rate, G_{IIC} .
- Mode III critical energy release rate, G_{IIIC} .
- Exponent, am .
- Exponent, an .
- Exponent, ao .
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENHANCED VCCT for *mixedModeBehavior*=BK or REEDER, the table data specify the following:

- Mode I critical energy release rate for onset crack, G_{IC} .
- Mode II critical energy release rate for onset crack, G_{IIC} .
- Mode III critical energy release rate for onset crack, G_{IIIC} .
- Mode I critical energy release rate for crack propagation, G_{IC} .
- Mode II critical energy release rate for crack propagation, G_{IIC} .
- Mode III critical energy release rate for crack propagation, G_{IIIC} .
- Exponent, η .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENHANCED VCCT for *mixedModeBehavior*=POWER, the table data specify the following:

- Mode I critical energy release rate for onset crack, G_{IC} .
- Mode II critical energy release rate for onset crack, G_{IIC} .
- Mode III critical energy release rate for onset crack, G_{IIIC} .
- Mode I critical energy release rate for crack propagation, G_{IC} .
- Mode II critical energy release rate for crack propagation, G_{IIC} .
- Mode III critical energy release rate for crack propagation, G_{IIIC} .
- Exponent, am .
- Exponent, an .
- Exponent, ao .
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

- Etc.

Return value

A FractureCriterion object.

Exceptions

None.

25.40.2 setValues(...)

This method modifies the FractureCriterion object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FractureCriterion** method.

Return value

None

Exceptions

None.

25.40.3 Members

The FractureCriterion object has members with the same names and descriptions as the arguments to the **FractureCriterion** method.

25.40.4 Corresponding analysis keywords

*FRACTURE CRITERION

25.41 **GapElectricalConductance** object

The GapElectricalConductance object specifies electrical conductance for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].electricalConductance
```

25.41.1 **GapElectricalConductance(...)**

This method creates a GapElectricalConductance object.

Path

```
mdb.models[name].interactionProperties[name].GapElectricalConductance
```

Required arguments

None.

Optional arguments

definition

A SymbolicConstant specifying how the electrical conductance is defined. Possible values are TABULAR and USER_DEFINED. The default value is TABULAR.

clearanceDependency

A Boolean specifying whether to use clearance-dependent data. The default value is ON.

pressureDependency

A Boolean specifying whether to use pressure-dependent data. The default value is OFF.

temperatureDependencyC

A Boolean specifying whether to use temperature-dependent data with clearance dependency. The default value is OFF.

dependenciesC

An Int specifying the number of field variables to use with clearance dependency. The default value is 0.

clearanceDepTable

A sequence of sequences of Floats specifying clearance dependency data. The items in the table data are described below.

temperatureDependencyP

A Boolean specifying whether to use temperature-dependent data with pressure dependency. The default value is OFF.

dependenciesP

An Int specifying the number of field variables to use with pressure dependency. The default value is 0.

pressureDepTable

A sequence of sequences of Floats specifying pressure dependency data. The items in the table data are described below.

Table data

The *clearanceDepTable* data specify the following:

- Conductivity.
- Clearance.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The *pressureDepTable* data specify the following:

- Conductivity.
- Pressure.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GapElectricalConductance object.

Exceptions

None.

25.41.2 setValues(...)

This method modifies the GapElectricalConductance object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `GapElectricalConductance` method.

Return value

None

Exceptions

None.

25.41.3 Members

The `GapElectricalConductance` object has members with the same names and descriptions as the arguments to the `GapElectricalConductance` method.

25.41.4 Corresponding analysis keywords

*GAP ELECTRICAL CONDUCTANCE

25.42 GapHeatGeneration object

The `GapHeatGeneration` object specifies heat generation for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].heatGeneration
```

25.42.1 HeatGeneration(...)

This method creates a `GapHeatGeneration` object.

Path

```
mdb.models[name].interactionProperties[name].HeatGeneration
```

Required arguments

None.

Optional arguments*conversionFraction*

A Float specifying the fraction of dissipated energy caused by friction or electric currents that is converted to heat. The default value is 1.0.

slaveFraction

A Float specifying the fraction of converted heat distributed to the slave surface. The default value is 0.5.

Return value

A GapHeatGeneration object.

Exceptions

None.

25.42.2 setValues(...)

This method modifies the GapHeatGeneration object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GapHeatGeneration** method.

Return value

None

Exceptions

None.

25.42.3 Members

The GapHeatGeneration object has the following members:

conversionFraction

A Float specifying the fraction of dissipated energy caused by friction or electric currents that is converted to heat. The default value is 1.0.

slaveFraction

A Float specifying the fraction of converted heat distributed to the slave surface. The default value is 0.5.

25.42.4 Corresponding analysis keywords

*GAP HEAT GENERATION

25.43 GeometricProperties object

The GeometricProperties object specifies surface interaction properties.

Access

```
import interaction
mdb.models[name].interactionProperties[name].geometricProperties
```

25.43.1 GeometricProperties(...)

This method creates a GeometricProperties object.

Path

```
mdb.models[name].interactionProperties[name].GeometricProperties
```

Required arguments

None.

Optional arguments

contactArea

A Float specifying the out-of-plane thickness of the surface for a two-dimensional model or cross-sectional area for every node in the node-based surface. The default value is 1.0.

padThickness

None or a Float specifying the thickness of an interfacial layer between the contacting surfaces. If *padThickness*=**None**, there is no interfacial layer. The default value is **None**.

Return value

A GeometricProperties object.

Exceptions

None.

25.43.2 setValues(...)

This method modifies the `GeometricProperties` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `GeometricProperties` method.

Return value

None

Exceptions

None.

25.43.3 Members

The `GeometricProperties` object has members with the same names and descriptions as the arguments to the `GeometricProperties` method.

25.43.4 Corresponding analysis keywords

*SURFACE INTERACTION

25.44 IncidentWave object

The `IncidentWave` object defines incident wave interactions for acoustic and coupled acoustic-structural analyses.

The `IncidentWave` object is derived from the `Interaction` object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.44.1 IncidentWave(...)

This method creates an IncidentWave object.

Path

```
mdb.models[name].IncidentWave
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the IncidentWave object is created.

sourcePoint

A Region object specifying the incident wave source point.

standoffPoint

A Region object specifying the incident wave standoff point.

This argument is not valid when *definition*=CONWEP.

surface

A Region object specifying the surface defining the incident wave interaction. In problems involving fluid/surface boundaries, both the fluid surface and the solid surface comprising the boundary must have an incident wave interaction specified.

interactionProperty

A String specifying the IncidentWaveProperty object associated with this interaction.

Optional arguments

definition

A SymbolicConstant specifying the type of incident wave to be defined. The value must be PRESSURE for linear perturbation steps. An Explicit step is required when the value is set to CONWEP. Possible values are PRESSURE, ACCELERATION, UNDEX, and CONWEP. The default value is PRESSURE.

amplitude

A String specifying the name of the Amplitude object that defines the fluid pressure time history at the standoff point, if *definition*=PRESSURE. If *definition*=ACCELERATION, then this string specifies the name of the Amplitude object that defines the fluid particle acceleration time

history at the standoff point. This member can be specified only if *definition*=PRESSURE or ACCELERATION. The default value is an empty string.

imaginaryAmplitude

A String specifying the name of the Amplitude object that defines the imaginary component of the fluid pressure time history at the standoff point. This member is applicable only for linear perturbation steps and if *definition*=PRESSURE. The default value is an empty string.

surfaceNormal

A sequence of three Floats specifying the X, Y, and Z components of the direction cosine of the fluid surface normal.

This argument is valid only when *definition*=UNDEX.

initialDepth

None or a Float specifying the initial depth of the UNDEX bubble. The default value is **None**.

This argument is valid only when *definition*=UNDEX.

referenceMagnitude

A Float specifying the reference magnitude.

This argument is not valid when *definition*=CONWEP.

detonationTime

A Float specifying the time of detonation, given in total time.

This argument is valid only when *definition*=CONWEP.

magnitudeFactor

A Float specifying the magnitude scale factor. The default value is 1.0.

This argument is valid only when *definition*=CONWEP.

Return value

An IncidentWave object.

Exceptions

None.

25.44.2 setValues(...)

This method modifies the IncidentWave object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **IncidentWave** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.44.3 Members

The IncidentWave object has members with the same names and descriptions as the arguments to the `IncidentWave` method.

25.44.4 Corresponding analysis keywords

*INCIDENT WAVE INTERACTION

25.45 IncidentWaveProperty object

The IncidentWaveProperty object is an interaction property that defines the properties referred to by an IncidentWave object.

The IncidentWaveProperty object is derived from the InteractionProperty object.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.45.1 IncidentWaveProperty(...)

This method creates an IncidentWaveProperty object.

Path

```
mdb.models[name].IncidentWaveProperty
```

Required argument

name

A String specifying the interaction property repository key.

Optional arguments*definition*

A SymbolicConstant specifying the type of wave to be defined. Possible values are PLANAR, SPHERICAL, DIFFUSE, AIR_BLAST, and SURFACE_BLAST. The default value is PLANAR.

propagationModel

A SymbolicConstant specifying the spherical propagation model. Possible values are ACOUSTIC, UNDEX_CHARGE, and GENERALIZED_DECAY. The default value is ACOUSTIC.

This argument is valid only when *definition*=SPHERICAL.

soundSpeed

A Float specifying the speed of sound in the fluid.

This argument is not valid when *definition*=AIR_BLAST or when *definition*=SURFACE_BLAST.

fluidDensity

A Float specifying the fluid mass density.

This argument is not valid when *definition*=AIR_BLAST or when *definition*=SURFACE_BLAST.

specificHeatRatio

None or a Float specifying the ratio of specific heats for gas. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

gravity

None or a Float specifying the acceleration due to gravity. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

atmosphericPressure

None or a Float specifying the atmospheric pressure. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

dragCoefficient

None or a Float specifying the fluid drag coefficient. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

dragExponent

A Float specifying the fluid drag exponent. The default value is 2.0.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

waveEffects

A Boolean specifying whether or not to include wave effects in the fluid and gas. The default value is ON.

INTERACTION COMMANDS

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

chargeDensity

None or a Float specifying the density of the charge material. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

chargeMass

None or a Float specifying the mass of the charge material. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

constantK1

None or a Float specifying the charge material constant K. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

constantK2

None or a Float specifying the charge material constant k. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

constantA

None or a Float specifying the charge material constant A. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

constantB

None or a Float specifying the charge material constant B. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

constantKc

None or a Float specifying the charge material constant Kc. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

duration

None or a Float specifying the time duration for the bubble simulation. The default value is **None**.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

maximumSteps

An Int specifying the maximum number of time steps for the bubble simulation. The default value is 1500.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

relativeStepControl

A Float specifying the relative step size control parameter. The default value is 1×10^{-11} .

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

absoluteStepControl

A Float specifying the absolute step size control parameter. The default value is 1×10^{-11} .

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

stepControlExponent

A Float specifying the step size control exponent. The default value is 0.2.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=UNDEX_CHARGE.

genDecayA

A Float specifying the constant A associated with the generalized decay propagation model. The default value is 0.0.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=GENERALIZED_DECAY.

genDecayB

A Float specifying the constant B associated with the generalized decay propagation model. The default value is 0.0.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=GENERALIZED_DECAY.

genDecayC

A Float specifying the constant C associated with the generalized decay propagation model. The default value is 0.0.

This argument is valid only when *definition*=SPHERICAL and *propagationModel*=GENERALIZED_DECAY.

seedNumber

An Int specifying the seed number (N) for the diffuse source calculation. N^2 sources will be used in the simulation.

This argument is valid only when *definition*=DIFFUSE.

massTNT

A Float specifying the equivalent mass of TNT, in any preferred mass unit.

This argument is valid only when *definition*=AIR_BLAST or *definition*=SURFACE_BLAST.

massFactor

A Float specifying the multiplication factor to convert from the preferred mass unit to kilograms. The default value is 1.0.

This argument is valid only when *definition*=AIR_BLAST or *definition*=SURFACE_BLAST.

INTERACTION COMMANDS

lengthFactor

A Float specifying the multiplication factor to convert from the analysis length unit to meters. The default value is 1.0.

This argument is valid only when *definition*=AIR_BLAST or *definition*=SURFACE_BLAST.

timeFactor

A Float specifying the multiplication factor to convert from the analysis time unit to seconds. The default value is 1.0.

This argument is valid only when *definition*=AIR_BLAST or *definition*=SURFACE_BLAST.

pressureFactor

A Float specifying the multiplication factor to convert from the analysis pressure unit to pascals. The default value is 1.0.

This argument is valid only when *definition*=AIR_BLAST or *definition*=SURFACE_BLAST.

Return value

An IncidentWaveProperty object.

Exceptions

None.

25.45.2 setValues(...)

This method modifies the IncidentWaveProperty object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **IncidentWaveProperty** method, except for the *name* argument.

Return value

None

Exceptions

None.

25.45.3 Members

The IncidentWaveProperty object has members with the same names and descriptions as the arguments to the **IncidentWaveProperty** method.

25.45.4 Corresponding analysis keywords

- *INCIDENT WAVE INTERACTION PROPERTY
- *UNDEX CHARGE PROPERTY
- *CONWEP CHARGE PROPERTY

25.46 IncidentWaveState object

The IncidentWaveState object stores the propagating data of an IncidentWave object in a step. One instance of this object is created internally by the IncidentWave object for each step. The instance is also deleted internally by the IncidentWave object.

The IncidentWaveState object has no constructor or methods.

The IncidentWaveState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.46.1 Members

The IncidentWaveState object has the following member:

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.46.2 Corresponding analysis keywords

*INCIDENT WAVE INTERACTION

25.47 InitializationAssignment object

The InitializationAssignment object stores the contact initialization assignment definition for domain pairs in a ContactStd object. The InitializationAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].initializationAssignments
```

25.47.1 changeValuesInStep(...)

This method allows modification of contact initialization assignments to domain pairs already defined in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the contact initialization assignments are to be modified.

index

An Int specifying the position of the contact initialization assignment whose value is to be modified.

value

A String specifying the value of the contact initialization to be assigned to the domain pair whose index is referenced.

Optional arguments

None.

Return value

None

Exceptions

None.

25.47.2 appendInStep(...)

This method allows addition of contact initialization assignments to new domain pairs in a given step.

Arguments**Required arguments**

stepName

A String specifying the name of the step in which new contact initialization assignments are to be defined.

assignments

A sequence of tuples specifying the initializations assigned to each surface pair. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL.
- A region object or the SymbolicConstant SELF.
- A String specifying a StdInitialization object associated with this pair of regions.

Optional arguments

None.

Return value

None

Exceptions

None.

25.47.3 delete(...)

The **delete** method allows you to delete existing contact initialization assignments from a ContactStd object.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each contact initialization assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.47.4 Members

The InitializationAssignment object has no members.

25.47.5 Corresponding analysis keywords

*CONTACT INITIALIZATION ASSIGNMENT

25.48 InteractionProperty object

The InteractionProperty object is the abstract base type for other InteractionProperty objects. The InteractionProperty object has no explicit constructor, members, or methods.

Access

```
import interaction
mdb.models[name].interactionProperties[name]
```

25.48.1 Members

The InteractionProperty object has no members.

25.49 InteractionState object

The InteractionState object is the abstract base type for other InteractionState objects. The InteractionState object has no explicit constructor. The members of the InteractionState object are common to all objects derived from InteractionState.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.49.1 Members

The InteractionState object has the following member:

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.50 MasterSlaveAssignment object

The MasterSlaveAssignment object stores the master-slave assignment definition for surfaces in ContactExp and ContactStd objects. The MasterSlaveAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].masterSlaveAssignments
```

25.50.1 changeValuesInStep(...)

This method allows modification of master-slave assignments already defined on surface pairs in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the master-slave assignments are to be modified.

index

An Int specifying the position of the master-slave assignment whose value is to be modified.

value

A SymbolicConstant specifying the value of the master-slave role to be assigned to the surface whose index is referenced. Possible values are MASTER, SLAVE, and BALANCED. The SymbolicConstant BALANCED can be specified only in an Abaqus/Standard analysis.

Optional arguments

None.

Return value

None

Exceptions

None.

25.50.2 **appendInStep(...)**

This method allows addition of master-slave assignments to new surface pairs in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the master-slave assignments are to be defined.

assignments

A sequence of tuples specifying the master-slave assignments. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the master-slave attribute is assigned.
- A SymbolicConstant specifying the overriding master-slave value to be used for the first surface. Possible values of the SymbolicConstant are MASTER, SLAVE, and BALANCED. The SymbolicConstant BALANCED can be specified only in an Abaqus/Standard analysis.

Optional arguments

None.

Return value

None

Exceptions

None.

25.50.3 delete(...)

The `delete` method allows you to delete existing master-slave assignments.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each master-slave assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.50.4 Members

The `MasterSlaveAssignment` object has no members.

25.50.5 Corresponding analysis keywords

- *CONTACT FORMULATION, TYPE=PURE MASTER-SLAVE
- *CONTACT FORMULATION, TYPE=MASTER SLAVE ROLES

25.51 Model object

The following commands operate on Model objects. For more information about the Model object, see “Model object,” Section 33.1.

Access

```
import interaction
```

25.51.1 contactDetection(...)

This method uses contact detection to create SurfaceToSurfaceContactStd , SurfaceToSurfaceContactExp , and Tie objects.

Arguments

Required arguments

None.

Optional arguments

name

A String specifying the prefix used to generate repository keys. The default value is "CP-"

createStepName

A String specifying the name of the step in which the SurfaceToSurfaceContactStd , SurfaceToSurfaceContactExp , and Tie objects are created. The default value is "Initial."

searchDomain

A SymbolicConstant MODEL or a sequence of Strings specifying the names of instances to search. MODEL indicates the whole model is searched. The default value is MODEL.

defaultType

A SymbolicConstant specifying the default type of object to create. Possible values are CONTACT, CONTACT_STANDARD, CONTACT_EXPLICIT, and TIE. If CONTACT is used, the behavior is determined by the type of Step in the model. If an ExplicitDynamicsStep or TempDisplacementDynamicsStep exists, then SurfaceToSurfaceContactExp is created by default. Otherwise SurfaceToSurfaceContactStd is created by default. The default value is CONTACT.

interactionProperty

A String specifying the name of the ContactProperty object associated with any interactions created.

separationTolerance

A Float specifying the maximum separation for considering two surfaces to be candidates for contact, where separation is the maximum distance between the points of closest approach on the two surfaces. The default value is a function of the model.

extendByAngle

None or a Float specifying the angle for extending surface definitions to include adjacent faces. The default value is 20.

mergeWithinAngle

None or a Float specifying the angle for merging adjacent contact pairs which lie within the angle. The default value is 20.

searchSingleInstances

A Boolean specifying whether to include surface pairs within a single instance. The default value is OFF.

nameEachSurfaceFound

A Boolean specifying whether to assign a name to each surface found. The default value is ON.

createUnionOfMasterSurfaces

A Boolean specifying whether to create a surface that is the union of all master surfaces found. The default value is OFF.

createUnionOfSlaveSurfaces

A Boolean specifying whether to create a surface that is the union of all slave surfaces found. The default value is OFF.

createUnionOfMasterSlaveSurfaces

A Boolean specifying whether to create a surface that is the union of all master and slave surfaces found. The default value is OFF.

includePlanar

A Boolean specifying whether to include planar geometry. The default value is ON.

includeCylindricalSphericalToric

A Boolean specifying whether to include cylindrical, spherical and toric geometry. The default value is ON.

includeSplineBased

A Boolean specifying whether to include spline-based geometry. The default value is ON.

includeMeshSolid

A Boolean specifying whether to include solid mesh entities. The default value is ON.

includeMeshShell

A Boolean specifying whether to include shell mesh entities. The default value is ON.

includeMeshMembrane

A Boolean specifying whether to include mesh membrane entities. The default value is OFF.

includeOverclosed

A Boolean specifying whether to include overclosed pairs. The default value is ON.

INTERACTION COMMANDS

includeNonOverlapping

A Boolean specifying whether to include opposing geometry surfaces that do not overlap. The default value is OFF.

meshedGeometrySearchTechnique

A SymbolicConstant USE_GEOMETRY or USE_MESH specifying whether to locate pairs in meshed geometry using the geometric entities or mesh entities. The default value is USE_GEOMETRY.

useShellThickness

A Boolean specifying whether to account for shell thickness and offset during contact detection. The default value is ON.

surfaceSmoothing

A SymbolicConstant specifying whether to use surface smoothing for geometric surfaces in SurfaceToSurfaceContactStd interactions. Possible values are NONE and AUTOMATIC. The default value is AUTOMATIC.

Return value

None

Exceptions

None.

25.51.2 **getSurfaceSeparation(...)**

This method returns a list of all possible contacts that can be created using the ContactDetection method.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **getSurfaceSeparation** are the same as the arguments to the ContactDetection method, with the exception of the *createStepName*, *defaultType*, *interactionProperty*, and *nameEachSurfaceFound* arguments.

Return value

Tuple of tuples, where each tuple holds information, to be used in contact creation as follows:

A string specifying the name of the master surface used in contact.

A string specifying the name of the slave surface used in contact.

A float specifying the separation distance between the master surface and the slave surface.

A boolean specifying whether or not contact surfaces are overclosed.

Exceptions

None.

25.52 ModelChange object

The ModelChange object defines model change interactions for element removal and reactivation.
The ModelChange object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.52.1 ModelChange(...)

This method creates a ModelChange object.

Path

```
mdb.models[name].ModelChange
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the ModelChange object is created.

Optional arguments

isRestart

A Boolean specifying whether this interaction is being used solely to indicate that model change may be required in a subsequent restart analysis (either for elements or contact pairs). The default value is OFF.

regionType

A SymbolicConstant specifying the region selection type. This argument is valid only when *isRestart*=False. Possible values are GEOMETRY, SKINS, STRINGERS, and ELEMENTS. The default value is GEOMETRY.

region

A Region object specifying the elements to be removed or reactivated. This argument is valid only when *isRestart*=False.

INTERACTION COMMANDS

activeInStep

A Boolean specifying whether elements are being removed or reactivated. This argument is valid only when *isRestart*=False. The default value is OFF.

includeStrain

A Boolean specifying whether stress/displacement elements are reactivated with strain. This argument is valid only when *isRestart*=False and when *activeInStep*=True. The default value is OFF.

Return value

A ModelChange object.

Exceptions

None.

25.52.2 setValues(...)

This method modifies the data for an existing ModelChange object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ModelChange** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.52.3 setValuesInStep(...)

This method modifies the propagating data of an existing ModelChange object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments*activeInStep*

A Boolean specifying whether elements are being removed or reactivated. This argument is valid only when *isRestart*=False. The default value is OFF.

includeStrain

A Boolean specifying whether stress/displacement elements are reactivated with strain. This argument is valid only when *isRestart*=False and when *activeInStep*=True. The default value is OFF.

Return value

None

Exceptions

None.

25.52.4 Members

The `ModelChange` object has members with the same names and descriptions as the arguments to the `ModelChange` method.

25.52.5 Corresponding analysis keywords

*MODEL CHANGE, TYPE=ELEMENT

*MODEL CHANGE, ACTIVATE

25.53 NormalBehavior object

The `NormalBehavior` object specifies normal behavior for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].normalBehavior
```

25.53.1 NormalBehavior(...)

This method creates a `NormalBehavior` object.

Path

`mdb.models[name].interactionProperties[name].NormalBehavior`

Required arguments

None.

Optional arguments

contactStiffness

The SymbolicConstant **DEFAULT** or a Float specifying the contact stiffness. This argument is valid for *pressureOverclosure*=**LINEAR**. This argument is also valid for *pressureOverclosure*=**HARD** when *constraintEnforcementMethod*=**AUGMENTED_LAGRANGE** or **PENALTY**. A value of **DEFAULT** is valid only when the later conditions are met. A value of zero is equivalent to specifying **DEFAULT**. The default value is **DEFAULT**.

pressureOverclosure

A SymbolicConstant specifying the pressure-overclosure relationship to be used. Possible values are **HARD**, **EXPONENTIAL**, **LINEAR**, **TABULAR**, and **SCALE_FACTOR**. The default value is **HARD**.

allowSeparation

A Boolean specifying whether to allow separation after contact. The default value is **ON**.

maxStiffness

None or a Float specifying the maximum stiffness. If *maxStiffness*=**None**, there is no upper limit. The default value is **None**.

table

A sequence of sequences of Floats specifying the normal behavior properties. This argument is valid only for *pressureOverclosure*=**EXPONENTIAL** or **TABULAR**. The items in the table data are described below.

constraintEnforcementMethod

A SymbolicConstant specifying the method for enforcement of the contact constraint. Possible values are **DEFAULT**, **AUGMENTED_LAGRANGE**, **PENALTY**, and **DIRECT**. The default value is **DEFAULT**.

overclosureFactor

A Float specifying the overclosure measure (used to delineate the segments of the pressure-overclosure curve) as a percentage of the minimum element size in the contact region. The default value is 0.0.

overclosureMeasure

A Float specifying the overclosure measure (used to delineate the segments of the pressure-overclosure curve) directly. The default value is 0.0.

contactStiffnessScaleFactor

A Float specifying scale factor for the penalty stiffness or the geometric scaling of the "base" stiffness. The default value is 1.0.

initialStiffnessScaleFactor

A Float specifying an additional scale factor for the "base" default contact stiffness. The default value is 1.0.

clearanceAtZeroContactPressure

A Float specifying the clearance at which the contact pressure is zero. The default value is 0.0.

stiffnessBehavior

A SymbolicConstant specifying the type of penalty stiffness to be defined. This argument is valid only when *constraintEnforcementMethod*=PENALTY. Possible values are LINEAR and NONLINEAR. The default value is LINEAR.

stiffnessRatio

A Float specifying the ratio of the initial stiffness divided by the final stiffness. This argument is valid only when *stiffnessBehavior*=NONLINEAR. Possible values are $0 \leq \text{stiffnessRatio} < 1$. The default value is 0.01.

upperQuadraticFactor

A Float specifying the ratio of the overclosure at the maximum stiffness divided by the characteristic facet length. This argument is valid only when *stiffnessBehavior*=NONLINEAR. The default value is 0.03.

lowerQuadraticRatio

A Float specifying the ratio of the overclosure at the initial stiffness divided by the overclosure at the maximum stiffness, both relative to the clearance at which the contact pressure is zero. This argument is valid only when *stiffnessBehavior*=NONLINEAR. Possible values are $0 \leq \text{stiffnessRatio} < 1$. The default value is 0.33333.

Table data

If *pressureOverclosure*=EXPONENTIAL, the table data specify the following:

- Pressure at zero clearance, p_0 .
- Clearance at which the contact pressure is zero, c_0 .

If *pressureOverclosure*=TABULAR, the table data specify the following:

- Pressure.
- Overclosure.

Return value

A NormalBehavior object.

Exceptions

None.

25.53.2 **setValues(...)**

This method modifies the NormalBehavior object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **NormalBehavior** method.

Return value

None

Exceptions

None.

25.53.3 **Members**

The NormalBehavior object has members with the same names and descriptions as the arguments to the **NormalBehavior** method.

25.53.4 **Corresponding analysis keywords**

*SURFACE BEHAVIOR

25.54 **PressurePenetration object**

The PressurePenetration object defines pressure penetration loading simulated with surface-to-surface contact.

The PressurePenetration object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```


25.54.1 PressurePenetration(...)

This method creates a PressurePenetration object.

Path

`mdb.models[name].PressurePenetration`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the PressurePenetration object is created.

contactInteraction

A String specifying the name of the Surface-to-surface contact (Standard) interaction.

masterPoints

A RegionArray object specifying the points on the master surface that are exposed to the fluid.

slavePoints

A RegionArray object specifying the points on the slave surface that are exposed to the fluid.

penetrationPressure

A tuple of Floats specifying the fluid pressure magnitude. For steady state dynamic analyses, a tuple of Complexes specifying the fluid pressure magnitude.

criticalPressure

A tuple of Floats specifying the critical contact pressure below which fluid penetration starts to occur.

Optional arguments

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

penetrationTime

A Float specifying the fraction of the current step time over which the fluid pressure on newly penetrated contact surface segments is ramped up to the current magnitude. The default value is 0.001.

Return value

A PressurePenetration object.

Exceptions

None.

25.54.2 **setValues(...)**

This method modifies the data for an existing PressurePenetration object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PressurePenetration** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.54.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing PressurePenetration object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

penetrationPressure

A tuple of Floats specifying the fluid pressure magnitude. For steady state dynamic analyses, a tuple of Complexes specifying the fluid pressure magnitude.

criticalPressure

A tuple of Floats specifying the critical contact pressure below which fluid penetration starts to occur.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

penetrationTime

A Float specifying the fraction of the current step time over which the fluid pressure on newly penetrated contact surface segments is ramped up to the current magnitude. The default value is 0.001.

Return value

None

Exceptions

None.

25.54.4 Members

The PressurePenetration object has the following members:

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the PressurePenetration object is created.

contactInteraction

A String specifying the name of the Surface-to-surface contact (Standard) interaction.

masterPoints

A RegionArray object specifying the points on the master surface that are exposed to the fluid.

slavePoints

A RegionArray object specifying the points on the slave surface that are exposed to the fluid.

25.54.5 Corresponding analysis keywords

*PRESSURE PENETRATION

25.55 PressurePenetrationState object

The PressurePenetrationState object stores the propagating data of a PressurePenetration object in a step. One instance of this object is created internally by the PressurePenetration object for each step. The instance is also deleted internally by the PressurePenetration object.

The PressurePenetrationState object has no constructor or methods.

The PressurePenetrationState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.55.1 Members

The PressurePenetrationState object has the following members:

penetrationTime

A Float specifying the fraction of the current step time over which the fluid pressure on newly penetrated contact surface segments is ramped up to the current magnitude. The default value is 10^{-3} .

penetrationTimeState

A SymbolicConstant specifying the propagation state of the *penetrationTime* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

criticalPressureState

A SymbolicConstant specifying the propagation state of the *criticalPressure* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

penetrationPressureState

A SymbolicConstant specifying the propagation state of the *penetrationPressure* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

penetrationPressure

A tuple of Floats specifying the fluid pressure magnitude. For steady state dynamic analyses, a tuple of Complexes specifying the fluid pressure magnitude.

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

criticalPressure

A tuple of Floats specifying the critical contact pressure below which fluid penetration starts to occur.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.55.2 Corresponding analysis keywords

*PRESSURE PENETRATION

25.56 Radiation object

The Radiation object specifies radiation for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].radiation
```

25.56.1 Radiation(...)

This method creates a Radiation object.

Path

```
mdb.models[name].interactionProperties[name].Radiation
```

INTERACTION COMMANDS

Required arguments

masterEmissivity

A Float specifying the emissivity of the master surface.

slaveEmissivity

A Float specifying the emissivity of the slave surface.

table

A sequence of sequences of Floats specifying the following:

- Effective viewfactor, F .
- Gap clearance, d .

Optional arguments

None.

Return value

A Radiation object.

Exceptions

None.

25.56.2 **setValues(...)**

This method modifies the Radiation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Radiation** method.

Return value

None

Exceptions

None.

25.56.3 Members

The Radiation object has members with the same names and descriptions as the arguments to the `Radiation` method.

25.56.4 Corresponding analysis keywords

*GAP RADIATION

25.57 RadiationToAmbient object

The RadiationToAmbient object defines radiant heat transfer between a surface and its environment. The RadiationToAmbient object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.57.1 RadiationToAmbient(...)

This method creates a RadiationToAmbient object.

Path

```
mdb.models[name].RadiationToAmbient
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the RadiationToAmbient object is created.

surface

A Region object specifying the surface to which the radiation interaction is applied.

emissivity

A Float specifying the emissivity, ϵ .

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this interaction. The *field* argument applies only when *distributionType*=ANALYTICAL_FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the radiation is distributed. This argument applies only when *radiationType*=AMBIENT. Possible values are UNIFORM and ANALYTICAL_FIELD. The default value is UNIFORM.

radiationType

A SymbolicConstant specifying whether to use the default surface radiation behavior, or the cavity radiation approximation. Possible values are AMBIENT and CAVITY. The default value is AMBIENT.

ambientTemperature

A Float specifying the reference ambient temperature, θ^0 . This argument applies only when *radiationType*=AMBIENT. The default value is 0.0.

ambientTemperatureAmp

A String specifying the name of the Amplitude object that gives the variation of the ambient temperature with time.

Note: Use **None** in an Abaqus/Standard analysis to specify that the reference ambient temperature is applied immediately at the beginning of the step or linearly over the step. Use **None** in an Abaqus/Explicit analysis to specify that the reference ambient temperature is applied throughout the step. This argument applies only when *radiationType*=AMBIENT.

Return value

A RadiationToAmbient object.

Exceptions

None.

25.57.2 setValues(...)

This method modifies the data for an existing RadiationToAmbient object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RadiationToAmbient** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.57.3 setValuesInStep(...)

This method modifies the propagating data of an existing **RadiationToAmbient** object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

None.

The optional arguments to **setValuesInStep** are *emissivity*, *ambientTemperature*, and *ambientTemperatureAmp* arguments.

Return value

None

Exceptions

None.

25.57.4 Members

The **RadiationToAmbient** object has members with the same names and descriptions as the arguments to the **RadiationToAmbient** method except the optional arguments to the **setValuesInStep** method.

25.58 RadiationToAmbientState object

The RadiationToAmbientState object stores the propagating data for a RadiationToAmbient object. One instance of this object is created internally by the RadiationToAmbient object for each step. The instance is also deleted internally by the RadiationToAmbient object.

The RadiationToAmbientState object has no constructor or methods.

The RadiationToAmbientState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.58.1 Members

The RadiationToAmbientState object has the following members:

ambientTemperature

A Float specifying the ambient temperature.

ambientTemperatureState

A SymbolicConstant specifying the propagation state of the *ambientTemperature* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

ambientTemperatureAmpState

A SymbolicConstant specifying the propagation state of the *ambientTemperatureAmp* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

emissivity

A Float specifying the emissivity.

emissivityState

A SymbolicConstant specifying the propagation state of the *emissivity* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

ambientTemperatureAmp

A String specifying the name of the Amplitude object that gives the variation of the ambient temperature with time.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED

- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.58.2 Corresponding analysis keywords

*SRADIATE

25.59 RegionPairs object

The RegionPairs object stores the domain pair definition for ContactExp and ContactStd objects. The RegionPairs object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].excludedPairs
mdb.models[name].interactions[name].includedPairs
```

25.59.1 setValuesInStep(...)

This method allows addition and removal of domain pairs in a given step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the region pair assignments are to be modified.

Optional arguments

useAllstar

A Boolean specifying whether the contacting surface pair consists of all exterior faces and -- in an Abaqus/Explicit analysis -- analytical rigid surfaces, shell edges, and beam segments in the model.

INTERACTION COMMANDS

addPairs

A sequence of pairs of region objects or SymbolicConstants that specify the surface pairs to add to the included pairs of the ContactExp or ContactStd object in the given step. Possible values of the SymbolicConstants are GLOBAL and SELF. When used with a ContactExp object, the second parameter of each pair can also be a string that references an Eulerian material surface.

removePairs

A sequence of pairs of region objects or SymbolicConstants that specify the surface pairs to remove from the included pairs of the ContactExp or ContactStd object in the given step. Possible values of the SymbolicConstants are GLOBAL and SELF. When used with a ContactExp object, the second parameter of each pair can also be a string that references an Eulerian material surface.

Return value

None

Exceptions

None.

25.59.2 Members

The RegionPairs object has no members.

25.59.3 Corresponding analysis keywords

*CONTACT INCLUSIONS
*CONTACT EXCLUSIONS

25.60 SelfContactExp object

The SelfContactExp object defines self-contact during an Abaqus/Explicit analysis.

The SelfContactExp object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.60.1 SelfContactExp(...)

This method creates a SelfContactExp object.

Path

`mdb.models[name].SelfContactExp`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the SelfContactExp object is created.

surface

A Region object specifying the surface where self-contact is defined.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

Optional arguments

mechanicalConstraint

A SymbolicConstant specifying the mechanical constraint formulation. Possible values are KINEMATIC and PENALTY. The default value is KINEMATIC.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

A SelfContactExp object.

Exceptions

None.

25.60.2 setValues(...)

This method modifies the data for an existing SelfContactExp object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SelfContactExp** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.60.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing **SelfContactExp** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

interactionProperty

A String specifying the name of the **ContactProperty** object associated with this interaction.

contactControls

A String specifying the name of the **ContactControl** object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

None

Exceptions

None.

25.60.4 **Members**

The **SelfContactExp** object has members with the same names and descriptions as the arguments to the **SelfContactExp** method except the optional arguments to the **setValuesInStep** method.

25.61 SelfContactExpState object

The SelfContactExpState object stores the propagating data for a SelfContactExp object. One instance of this object is created internally by the SelfContactExp object for each step. The instance is also deleted internally by the SelfContactExp object.

The SelfContactExpState object has no constructor or methods.

The SelfContactExpState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.61.1 Members

The SelfContactExpState object has the following members:

interactionPropertyState

A SymbolicConstant specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

contactControlsState

A SymbolicConstant specifying the propagation state of the *contactControls* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

contactControls

A String specifying the name of the ContactControl object associated with this interaction.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.61.2 Corresponding analysis keywords

- *CONTACT CONTROLS
- *CONTACT PAIR
- *MODEL CHANGE, TYPE=CONTACT PAIR

25.62 SelfContactStd object

The SelfContactStd object defines self-contact during an Abaqus/Standard analysis.

The SelfContactStd object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.62.1 SelfContactStd(...)

This method creates a SelfContactStd object.

Path

```
mdb.models[name].SelfContactStd
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the SelfContactStd object is created.

surface

A Region object specifying the surface where self-contact is defined.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

Optional arguments*enforcement*

A SymbolicConstant specifying the discretization method. Possible values are `NODE_TO_SURFACE` and `SURFACE_TO_SURFACE`. The default value is `SURFACE_TO_SURFACE`.

thickness

A Boolean specifying whether shell/membrane element thickness is considered. The default value is ON.

This argument is valid only when *enforcement*=`SURFACE_TO_SURFACE`.

smooth

A Float specifying the degree of smoothing used for deformable or rigid master surfaces involved when *enforcement*=`NODE_TO_SURFACE`. The value given must lie between 0.0 and 0.5. The default value is 0.2.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

A SelfContactStd object.

Exceptions

None.

25.62.2 setValues(...)

This method modifies the data for an existing SelfContactStd object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SelfContactStd** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.62.3 **setValuesInStep(...)**

This method modifies the propagating data of an existing SelfContactStd object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

None

Exceptions

None.

25.62.4 **Members**

The SelfContactStd object has members with the same names and descriptions as the arguments to the **SelfContactStd** method except the optional arguments to the **setValuesInStep** method.

In addition, the SelfContactStd object has the following members:

contactTracking

A SymbolicConstant specifying the choice of contact tracking algorithm. The STATE tracking algorithm uses only normal projections and is specified by using ONE_CONFIG. The PATH tracking algorithm uses crossing and normal projections and is specified by using TWO_CONFIG. Possible values are ONE_CONFIG and TWO_CONFIG. The default value is TWO_CONFIG.

This argument is valid only when *enforcement*=SURFACE_TO_SURFACE.

supplementaryContact

A SymbolicConstant specifying the manner in which mid-face constraints are employed. Possible values are SELECTIVE, NEVER, and ALWAYS. The default value is SELECTIVE.

This argument is valid only when *enforcement*=NODE_TO_SURFACE.

25.63 SelfContactStdState object

The SelfContactStdState object stores the propagating data for a SelfContactStd object. One instance of this object is created internally by the SelfContactStd object for each step. The instance is also deleted internally by the SelfContactStd object.

The SelfContactStdState object has no constructor or methods.

The SelfContactStdState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.63.1 Members

The SelfContactStdState object has the following members:

interactionPropertyState

A SymbolicConstant specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

contactControlsState

A SymbolicConstant specifying the propagation state of the *contactControls* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the ContactProperty object associated with this interaction.

contactControls

A String specifying the name of the ContactControl object associated with this interaction.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED

INTERACTION COMMANDS

- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.63.2 Corresponding analysis keywords

*CONTACT CONTROLS
*CONTACT PAIR
*MODEL CHANGE, TYPE=CONTACT PAIR

25.64 SmoothingAssignment object

The SmoothingAssignment object stores the surface smoothing assignment definition for surfaces in ContactExp and ContactStd objects. The SmoothingAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].smoothingAssignments
```

25.64.1 changeValuesInStep(...)

This method allows modification of surface smoothing assignments already defined on surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the surface smoothing assignments are to be modified.

index

An Int specifying the position of the surface smoothing assignment whose value is to be modified.

value

A tuple specifying the value of the surface smoothing assignments for the surface whose index is referenced. Each tuple contains one entry:

- A SymbolicConstant specifying the surface smoothing value to be used for the surface. Possible values of the SymbolicConstant are NONE, REVOLUTION, SPHERICAL, and TOROIDAL.

Optional arguments

None.

Return value

None

Exceptions

None.

25.64.2 appendInStep(...)

This method allows addition of surface smoothing assignments to new surfaces in a given step.

Arguments**Required arguments**

stepName

A String specifying the name of the step in which new surface smoothing assignments are to be defined.

assignments

A sequence of tuples specifying the surface smoothing assignments. Each tuple contains two entries:

- A region object specifying the surface to which the smoothing is assigned.
- A SymbolicConstant specifying the surface smoothing value to be used for the surface. Possible values of the SymbolicConstant are NONE, REVOLUTION, SPHERICAL, and TOROIDAL.

Optional arguments

None.

Return value

None

Exceptions

None.

25.64.3 delete(...)

The **delete** method allows you to delete existing surface smoothing assignments from ContactExp and ContactStd objects.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each surface smoothing assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.64.4 Members

The SmoothingAssignment object has no members.

25.64.5 Corresponding analysis keywords

*SURFACE PROPERTY ASSIGNMENT, PROPERTY=GEOMETRIC CORRECTION

25.65 StabilizationAssignment object

The StabilizationAssignment object stores the contact stabilization assignment definition for domain pairs in a ContactStd object. The StabilizationAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].stabilizationAssignments
```

25.65.1 changeValuesInStep(...)

This method allows modification of contact stabilization assignments to domain pairs already defined in a given step.

Arguments**Required arguments**

stepName

A String specifying the name of the step in which the contact stabilization assignments are to be modified.

index

An Int specifying the position of the contact stabilization assignment whose value is to be modified.

value

A String specifying the value of the contact stabilization to be assigned to the domain pair whose index is referenced.

Optional arguments

None.

Return value

None

Exceptions

None.

25.65.2 appendInStep(...)

This method allows addition of contact stabilization assignments to new domain pairs in a given step.

Arguments**Required arguments**

stepName

A String specifying the name of the step in which new contact stabilization assignments are to be defined.

INTERACTION COMMANDS

assignments

A sequence of tuples specifying the stabilizations assigned to each surface pair. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL.
- A region object or the SymbolicConstant SELF.
- A String specifying a StdStabilization object associated with this pair of regions.

Optional arguments

None.

Return value

None

Exceptions

None.

25.65.3 delete(...)

The **delete** method allows you to delete existing contact stabilization assignments from a ContactStd object.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each contact stabilization assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.65.4 Members

The StabilizationAssignment object has no members.

25.65.5 Corresponding analysis keywords

*CONTACT STABILIZATION

25.66 StdContactControl object

The StdContactControl object is used in Abaqus/Standard analyses to specify optional solution controls for problems involving contact between bodies.

The StdContactControl object is derived from the ContactControl object.

Access

```
import interaction
mdb.models[name].contactControls[name]
```

25.66.1 StdContactControl(...)

This method creates an StdContactControl object.

Path

```
mdb.models[name].StdContactControl
```

Required argument

name

A String specifying the contact controls repository key.

Optional arguments

stiffnessScaleFactor

A Float specifying the factor by which Abaqus/Standard will scale the default penalty stiffness to obtain the stiffnesses used for the contact pairs. Only contact interactions defined with augmented Lagrangian surface behavior will be affected by this argument. The default value is 1.0.

penetrationTolChoice

A SymbolicConstant specifying whether the allowable penetration is an absolute value or a value relative to the characteristic contact surface face dimension. Only contact interactions defined with augmented Lagrangian surface behavior will be affected by this argument. Possible values are RELATIVE and ABSOLUTE. The default value is RELATIVE.

INTERACTION COMMANDS

relativePenetrationTolerance

A Float specifying the ratio of the allowable penetration to the characteristic contact surface face dimension. The float values represent percentages (e.g.: 0.001=0.1%). Only contact interactions defined with augmented Lagrangian surface behavior will be affected by this argument. The default value is 10^{-3} .

The *relativePenetrationTolerance* argument applies only when *penetrationTolChoice*=RELATIVE. The *relativePenetrationTolerance* and *absolutePenetrationTolerance* arguments are mutually exclusive.

absolutePenetrationTolerance

None or a Float specifying the allowable penetration. Only contact interactions defined with augmented Lagrangian surface behavior will be affected by this argument. The *absolutePenetrationTolerance* argument applies only when *penetrationTolChoice*=ABSOLUTE. The *relativePenetrationTolerance* and *absolutePenetrationTolerance* arguments are mutually exclusive. The default value is **None**.

frictionOnset

A SymbolicConstant specifying when the application of friction occurs. Possible values are:

- IMMEDIATE, specifying the friction is included in the increment when contact occurs.
- DELAYED, specifying the application of friction is delayed until the increment after contact occurs.

The default value is IMMEDIATE.

automaticTolerances

A Boolean specifying whether Abaqus/Standard should automatically compute an overclosure tolerance and a separation tolerance to prevent chattering in contact. The default value is OFF.

The *automaticTolerances* argument cannot be used with the *maxchp*, *perrmx*, and *uerrmx* arguments.

maxchp

An Int specifying the maximum number of points that are permitted to violate contact conditions in any increment. The default value is 0.

Either the *perrmx* or the *uerrmx* argument must be specified in conjunction with the *maxchp* argument.

perrmx

A Float specifying the maximum value of tensile stress (tensile force in GAP- or ITT-type contact elements) allowed to be transmitted at a contact point. The default value is 0.0.

The *perrmx* argument must be specified in conjunction with the *maxchp* argument.

uerrmx

A Float specifying the maximum overclosure distance allowed at a slave node that is considered to be open. The default value is 0.0.

The *uerrmx* argument must be specified in conjunction with the *maxchp* argument.

stabilizeChoice

A SymbolicConstant specifying whether or not viscous damping will be specified, and if so, how it will be specified. Possible values are NONE, AUTOMATIC, and COEFFICIENT. The default value is NONE.

dampFactor

A Float specifying the value of the damping factor. This value is multiplied by the calculated damping coefficient. The default value is 1.0.

This argument is only valid when *stabilizeChoice*=AUTOMATIC.

dampCoef

A Float specifying the damping coefficient. The default value is 0.0.

This argument is only valid when *stabilizeChoice*=COEFFICIENT.

tangFraction

A Float specifying the tangential stabilization as a fraction of the normal stabilization (damping). The default value is 1.0.

This argument is valid only if *stabilizeChoice* = AUTOMATIC or COEFFICIENT.

eosFraction

A Float specifying the fraction of the damping that remains at the end of the step. The default value is 0.0.

This argument is valid only if *stabilizeChoice* = AUTOMATIC or COEFFICIENT.

zeroDampingChoice

A SymbolicConstant specifying how the zero-damping clearance will be specified. Possible values are COMPUTE and SPECIFY. The default value is COMPUTE.

This argument is valid only if *stabilizeChoice* = AUTOMATIC or COEFFICIENT.

zeroDamping

None or a Float specifying the clearance at which damping becomes zero. This argument is valid only when *zeroDampingChoice*=SPECIFY. This argument is valid only if *stabilizeChoice* = AUTOMATIC or COEFFICIENT. The default value is **None**.

enforceWithLagrangeMultipliers

A SymbolicConstant specifying whether to enforce the contact constraints with Lagrange multipliers. Possible values are DEFAULT, ENFORCEMENT_OFF, and ENFORCEMENT_ON. The default value is DEFAULT.

Return value

A StdContactControl object.

Exceptions

RangeError.

25.66.2 **setValues(...)**

This method modifies the `StdContactControl` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StdContactControl** method, except for the *name* argument.

Return value

None

Exceptions

`RangeError`.

25.66.3 **Members**

The `StdContactControl` object has members with the same names and descriptions as the arguments to the **StdContactControl** method.

25.66.4 **Corresponding analysis keywords**

*CONTACT CONTROLS

25.67 **StdInitialization object**

The `StdInitialization` object is used in conjunction with `ContactStd` in Abaqus/Standard analyses to specify contact initialization data.

The `StdInitialization` object is derived from the `ContactInitialization` object.

Access

```
import interaction
mdb.models[name].contactInitializations[name]
```

25.67.1 StdInitialization(...)

This method creates a StdInitialization object.

Path

`mdb.models[name].StdInitialization`

Required argument

name

A String specifying the contact initialization repository key.

Optional arguments

overclosureType

A SymbolicConstant specifying the type of overclosure to be defined. Possible values are ADJUST, INTERFERENCE, and CLEARANCE. The default value is ADJUST.

interferenceDistance

None or a Float specifying the interference distance. This argument is valid only when *overclosureType*=INTERFERENCE. The default value is **None**.

clearanceDistance

None or a Float specifying the initial clearance distance. This argument is valid only when *overclosureType*=CLEARANCE, and must be specified in that case. The default value is **None**.

openingTolerance

None or a Float specifying the distance tolerance within which initial openings will undergo strain-free adjustments. This argument is not valid when *overclosureType*=INTERFERENCE unless a value has been specified for *interferenceDistance*. The default value is **None**.

overclosureTolerance

None or a Float specifying the distance tolerance within which initial overclosures will undergo strain-free adjustments.. The default value is **None**.

Return value

A StdInitialization object.

Exceptions

RangeError.

25.67.2 setValues(...)

This method modifies the StdInitialization object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StdInitialization** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

25.67.3 Members

The StdInitialization object has members with the same names and descriptions as the arguments to the **StdInitialization** method.

25.67.4 Corresponding analysis keywords

*CONTACT INITIALIZATION DATA

25.68 StdStabilization object

The StdStabilization object is used in conjunction with ContactStd in Abaqus/Standard analyses to specify contact stabilization.

The StdStabilization object is derived from the ContactStabilization object.

Access

```
import interaction
mdb.models[name].contactStabilizations[name]
```

25.68.1 StdStabilization(...)

This method creates a StdStabilization object.

Path

`mdb.models[name].StdStabilization`

Required argument

name

A String specifying the contact stabilization repository key.

Optional arguments

zeroDistance

None or a Float specifying the clearance distance at which the stabilization becomes zero. The default value is **None**.

reductionFactor

A Float specifying the factor by which the analysis will reduce the contact stabilization coefficient per increment. The default value is 0.1.

scaleFactor

A Float specifying the factor by which the analysis will scale the contact stabilization coefficient. The default value is 1.0.

tangentialFactor

A Float specifying the factor that scales the contact stabilization coefficient in the tangential direction. The default value is 0.0.

amplitude

A String specifying the name of the Amplitude object that defines a time-dependent scale factor for contact stabilization over the step. The default value is an empty string.

reset

A Boolean specifying whether to cancel carryover effects from contact stabilization specifications involving nondefault amplitudes that appeared in previous steps. The default value is OFF.

Return value

A StdStabilization object.

Exceptions

RangeError.

25.68.2 setValues(...)

This method modifies the StdStabilization object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StdStabilization** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

25.68.3 Members

The StdStabilization object has members with the same names and descriptions as the arguments to the **StdStabilization** method.

25.68.4 Corresponding analysis keywords

*CONTACT STABILIZATION

25.69 StdXplCosimulation object

The StdXplCosimulation object defines co-simulation behavior between Abaqus/Standard and Abaqus/Explicit.

The StdXplCosimulation object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.69.1 StdXplCosimulation(...)

This method creates a StdXplCosimulation object.

Path

`mdb.models[name].StdXplCosimulation`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the StdXplCosimulation object is created.

region

A Region object specifying the import and export region upon which the co-simulation exchanges data with the coupled analysis program.

Optional arguments

incrementation

A SymbolicConstant specifying whether the analysis programs use the same time increments or one is allowed to use more time increments than the other before exchanging data. Possible values are ALLOW_SUBCYCLING and LOCKSTEP. The default value is ALLOW_SUBCYCLING.

stepSize

A Float specifying the size of the increments to be used by Abaqus/Standard and Abaqus/Explicit. The default value is 0.0.

stepSizeDefinition

A SymbolicConstant specifying whether the increment size is the analysis default or a supplied variable. Possible values are DEFAULT and SPECIFIED. The default value is DEFAULT.

Return value

A StdXplCosimulation object.

Exceptions

None.

25.69.2 setValues(...)

This method modifies the StdXplCosimulation object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StdXplCosimulation** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.69.3 Members

The **StdXplCosimulation** object has members with the same names and descriptions as the arguments to the **StdXplCosimulation** method.

25.70 StdXplCosimulationState object

The **StdXplCosimulationState** object stores the propagating data for a **StdXplCosimulation** object. One instance of this object is created internally by the **StdXplCosimulation** object for each step. The instance is also deleted internally by the **StdXplCosimulation** object.

The **StdXplCosimulationState** object has no constructor or methods.

The **StdXplCosimulationState** object is derived from the **InteractionState** object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.70.1 Members

The **StdXplCosimulationState** object has the following member:

status

A **SymbolicConstant** specifying the propagation state of the **InteractionState** object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED

- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.70.2 Corresponding analysis keywords

*CO-SIMULATION,
 *CO-SIMULATION REGION,
 *CO-SIMULATION CONTROLS

25.71 SurfaceFeatureAssignment object

The SurfaceFeatureAssignment object stores the surface feature angle assignment definition for surfaces in ContactExp or ContactStd objects. The SurfaceFeatureAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].surfaceFeatureAssignments
```

25.71.1 changeValuesInStep(...)

This method allows modification of surface feature angle assignments already defined on surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the surface feature assignments are to be modified.

index

An Int specifying the position of the surface feature angle assignment whose value is to be modified.

value

A tuple specifying the value of the surface feature assignments for the surface whose index is referenced. Each tuple contains one entry:

- A Float or a SymbolicConstant specifying the overriding surface feature angle value to be used for the surface. Possible values of the SymbolicConstant are PERIMETER, ALL, PICKED, or NONE. The ALL and PICKED values cannot be specified with the GLOBAL region constant, and can be used only in the Explicit version of general contact.

Optional arguments

None.

Return value

None

Exceptions

None.

25.71.2 appendInStep(...)

This method allows addition of surface feature angle assignments to new surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which new surface feature angle assignments are to be defined.

assignments

A sequence of tuples specifying the surface feature angle assignments. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the feature angle is assigned.
- A Float or a SymbolicConstant specifying the overriding surface feature angle value to be used for the surface. Possible values of the SymbolicConstant are PERIMETER, ALL, PICKED, or NONE. The ALL and PICKED values cannot be specified with the GLOBAL region constant, and can be used only in the Explicit version of general contact.

Optional arguments

None.

Return value

None

Exceptions

None.

25.71.3 delete(...)

The **delete** method allows you to delete existing surface feature angle assignments from a ContactExp object.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each surface feature angle assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.71.4 Members

The SurfaceFeatureAssignment object has no members.

25.71.5 Corresponding analysis keywords

*SURFACE PROPERTY ASSIGNMENT, PROPERTY=FEATURE EDGE CRITERIA

25.72 SurfaceOffsetAssignment object

The SurfaceOffsetAssignment object stores the surface offset fraction assignment definition for surfaces in ContactExp and ContactStd objects. The SurfaceOffsetAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].surfaceOffsetAssignments
```

25.72.1 changeValuesInStep(...)

This method allows modification of surface offset fraction assignments already defined on surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the surface offset assignments are to be modified.

index

An Int specifying the position of the surface offset fraction assignment whose value is to be modified.

value

A tuple specifying the value of the surface offset assignments for the surface whose index is referenced. Each tuple contains one entry:

- A Float or a SymbolicConstant specifying the surface offset fraction value to be used for the surface. Possible values of the SymbolicConstant are ORIGINAL, SPOS, and SNEG.

Optional arguments

None.

Return value

None

Exceptions

None.

25.72.2 `appendInStep(...)`

This method allows addition of surface offset fraction assignments to new surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which new surface offset fraction assignments are to be defined.

assignments

A sequence of tuples specifying the surface offset fraction assignments. Each tuple contains two entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the offset fraction is assigned.
- A Float or a SymbolicConstant specifying the surface offset fraction value to be used for the surface. Possible values of the SymbolicConstant are ORIGINAL, SPOS, and SNEG.

Optional arguments

None.

Return value

None

Exceptions

None.

25.72.3 `delete(...)`

The `delete` method allows you to delete existing surface offset fraction assignments.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each surface offset fraction assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.72.4 Members

The SurfaceOffsetAssignment object has no members.

25.72.5 Corresponding analysis keywords

*SURFACE PROPERTY ASSIGNMENT, PROPERTY=OFFSET FRACTION

25.73 SurfaceThicknessAssignment object

The SurfaceThicknessAssignment object stores the surface thickness assignment definition for surfaces in ContactExp and ContactStd objects. The SurfaceThicknessAssignment object has no constructor or members.

Access

```
import interaction
mdb.models[name].interactions[name].surfaceThicknessAssignments
```

25.73.1 changeValuesInStep(...)

This method allows modification of surface thickness assignments already defined on surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which the surface thickness assignments are to be modified.

index

An Int specifying the position of the surface thickness assignment whose value is to be modified.

value

A tuple specifying the value of the surface thickness assignments for the surface whose index is referenced. Each tuple contains two entries:

- A Float or a SymbolicConstant specifying the overriding thickness value to be used in the contact definition. Possible values of the SymbolicConstant are ORIGINAL and THINNING. The SymbolicConstant THINNING can be specified only in an Abaqus/Explicit analysis.
- A Float specifying a scale factor that multiplies the thickness value specified in the second entry.

Optional arguments

None.

Return value

None

Exceptions

None.

25.73.2 appendInStep(...)

This method allows addition of surface thickness assignments to new surfaces in a given step.

Arguments

Required arguments

stepName

A String specifying the name of the step in which new surface thickness assignments are to be defined.

assignments

A sequence of tuples specifying the surface thickness assignments. Each tuple contains three entries:

- A region object or the SymbolicConstant GLOBAL specifying the surface to which the thickness is assigned.
- A Float or a SymbolicConstant specifying the overriding thickness value to be used in the contact definition. Possible values of the SymbolicConstant are ORIGINAL and THINNING. The SymbolicConstant THINNING can be specified only in an Abaqus/Explicit analysis.
- A Float specifying a scale factor that multiplies the thickness value specified in the second entry.

INTERACTION COMMANDS

Optional arguments

None.

Return value

None

Exceptions

None.

25.73.3 delete(...)

The **delete** method allows you to delete existing surface thickness assignments.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each surface thickness assignment to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

25.73.4 Members

The SurfaceThicknessAssignment object has no members.

25.73.5 Corresponding analysis keywords

*SURFACE PROPERTY ASSIGNMENT, PROPERTY=THICKNESS

25.74 SurfaceToSurfaceContactExp object

The SurfaceToSurfaceContactExp object defines surface-to-surface contact during an Abaqus/Explicit analysis.

The SurfaceToSurfaceContactExp object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.74.1 SurfaceToSurfaceContactExp(...)

This method creates a SurfaceToSurfaceContactExp object.

Path

```
mdb.models[name].SurfaceToSurfaceContactExp
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the SurfaceToSurfaceContactExp object is created.

master

A Region object specifying the master surface.

slave

A Region object specifying the slave surface.

sliding

A SymbolicConstant specifying the contact formulation. Possible values are FINITE and SMALL.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

Optional arguments

mechanicalConstraint

A SymbolicConstant specifying the mechanical constraint formulation. Possible values are KINEMATIC and PENALTY. The default value is KINEMATIC.

INTERACTION COMMANDS

weightingFactorType

A SymbolicConstant specifying the weighting for node-to-face contact. Possible values are DEFAULT and SPECIFIED. The default value is DEFAULT.

weightingFactor

A Float specifying the weighting factor for the contact surfaces when *weightingFactorType*=SPECIFIED. The default value is 0.0.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

initialClearance

A SymbolicConstant or a Float specifying the initial clearance at regions of contact. Possible values are OMIT and COMPUTED. The default value is OMIT.

halfThreadAngle

None or a sequence of Floats specifying the half thread angle used for bolt clearance. The default value is **None**.

pitch

None or a sequence of Floats specifying the pitch used for bolt clearance. The default value is **None**.

majorBoltDiameter

The SymbolicConstant COMPUTED or a Float specifying the major diameter of the bolt used for bolt clearance. The default value is COMPUTED.

meanBoltDiameter

The SymbolicConstant COMPUTED or a Float specifying the mean diameter of the bolt used for bolt clearance. The default value is COMPUTED.

datumAxis

A DatumAxis object specifying the orientation of the bolt hole when specifying bolt clearance.

useReverseDatumAxis

A Boolean specifying whether to reverse the bolt clearance direction given by the datum axis. The default value is OFF.

clearanceRegion

A Region object specifying the contact region for which clearance is specified.

Return value

A SurfaceToSurfaceContactExp object.

Exceptions

None.

25.74.2 **swapSurfaces()**

This method switches the master and slave surfaces of a surface-to-surface contact pair. This command is valid only during the step in which the interaction is created.

Arguments

None.

Return value

None

Exceptions

None.

25.74.3 **setValues(...)**

This method modifies the data for an existing `SurfaceToSurfaceContactExp` object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceToSurfaceContactExp** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.74.4 **setValuesInStep(...)**

This method modifies the propagating data for an existing `SurfaceToSurfaceContactExp` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional arguments

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. An empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

None

Exceptions

None.

25.74.5 Members

The SurfaceToSurfaceContactExp object has members with the same names and descriptions as the arguments to the **SurfaceToSurfaceContactExp** method except the optional arguments to the **setValuesInStep** method.

25.75 SurfaceToSurfaceContactStd object

The SurfaceToSurfaceContactStd object defines surface-to-surface contact during an Abaqus/Standard analysis.

The SurfaceToSurfaceContactStd object is derived from the Interaction object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.75.1 SurfaceToSurfaceContactStd(...)

This method creates a SurfaceToSurfaceContactStd object.

Path

`mdb.models [name] .SurfaceToSurfaceContactStd`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the SurfaceToSurfaceContactStd object is created.

master

A Region object specifying the master surface.

slave

A Region object specifying the slave surface.

sliding

A SymbolicConstant specifying the contact formulation. Possible values are FINITE and SMALL.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

Optional arguments

interferenceType

A SymbolicConstant specifying the type of time-dependent allowable interference for contact pairs and contact elements. Possible values are:

- NONE, specifying no allowable contact interference.
- SHRINK_FIT.
- UNIFORM.

The default value is NONE.

overclosure

A Float specifying the maximum overclosure distance allowed. This argument applies only when *interferenceType*=UNIFORM. The default value is 0.0.

interferenceDirectionType

A SymbolicConstant specifying the method used to determine the interference direction. Possible values are COMPUTED and DIRECTION_COSINE. The default value is COMPUTED.

direction

A sequence of three Floats specifying the following:

- *X*-direction cosine of the interference direction vector.
- *Y*-direction cosine of the interference direction vector.
- *Z*-direction cosine of the interference direction vector.

INTERACTION COMMANDS

This argument is required only when *interferenceDirectionType*=DIRECTION_COSINE.

amplitude

A String specifying the name of the amplitude curve that defines the magnitude of the prescribed interference during the step. Use **None** to specify that the prescribed interference is applied immediately at the beginning of the step and ramped down to zero linearly over the step.

smooth

A Float specifying the degree of smoothing used for deformable or rigid master surfaces involved when *enforcement*=NODE_TO_SURFACE. The value given must lie between 0.0 and 0.5. The default value is 0.2.

hcrit

A Float specifying the distance by which a slave node must penetrate the master surface before Abaqus/Standard abandons the current increment and tries again with a smaller increment. The default value is 0.0.

extensionZone

A Float specifying a fraction of the end segment or facet edge length by which the master surface is to be extended to avoid numerical round-off errors associated with contact modeling. The value given must lie between 0.0 and 0.2. The default value is 0.1.

adjustMethod

A SymbolicConstant specifying the adjust method. Possible values are NONE, OVERCLOSED, TOLERANCE, and SET. The default value is NONE.

adjustTolerance

A Float specifying the adjust tolerance. The default value is 0.0.

adjustSet

A Region object specifying the Set object to which the adjustment is to be applied.

enforcement

A SymbolicConstant specifying the discretization method. Possible values are NODE_TO_SURFACE and SURFACE_TO_SURFACE. The default value is SURFACE_TO_SURFACE.

thickness

A Boolean specifying whether shell/membrane element thickness is considered. The default value is ON.

This argument is not valid when *sliding*=FINITE and *enforcement*=NODE_TO_SURFACE.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. The empty string indicates that the default contact controls will be used. The default value is an empty string.

tied

A Boolean specifying whether the surfaces are to be "tied" together for the duration of the simulation. The default value is OFF.

initialClearance

A SymbolicConstant or a Float specifying the initial clearance at regions of contact. Possible values are OMIT and COMPUTED. The default value is OMIT.

halfThreadAngle

None or a sequence of Floats specifying the half thread angle used for bolt clearance. The default value is **None**.

pitch

None or a sequence of Floats specifying the pitch used for bolt clearance. The default value is **None**.

majorBoltDiameter

The SymbolicConstant COMPUTED or a Float specifying the major diameter of the bolt used for bolt clearance. The default value is COMPUTED.

meanBoltDiameter

The SymbolicConstant COMPUTED or a Float specifying the mean diameter of the bolt used for bolt clearance. The default value is COMPUTED.

datumAxis

A DatumAxis object specifying the orientation of the bolt hole when specifying bolt clearance.

useReverseDatumAxis

A Boolean specifying whether to reverse the bolt clearance direction given by the datum axis. The default value is OFF.

clearanceRegion

A Region object specifying the contact region for which clearance is specified.

surfaceSmoothing

A SymbolicConstant specifying whether to use surface smoothing for geometric surfaces in SurfaceToSurfaceContactStd interactions. Possible values are AUTOMATIC and NONE. The default value is NONE.

bondingSet

A Region object specifying the slave node sub-set for bonding, used only when the contact property CohesiveBehavior option specifies use.

Return value

A SurfaceToSurfaceContactStd object.

Exceptions

None.

25.75.2 swapSurfaces()

This method switches the master and slave surfaces of a surface-to-surface contact pair. This command is valid only for the step in which the interaction is created.

Arguments

None.

Return value

None

Exceptions

None.

25.75.3 setValues(...)

This method modifies the data for an existing SurfaceToSurfaceContactStd object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceToSurfaceContactStd** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.75.4 setValuesInStep(...)

This method modifies the propagating data for an existing SurfaceToSurfaceContactStd object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the interaction is modified.

Optional arguments*interactionProperty*

A String specifying the name of the ContactProperty object associated with this interaction.

interferenceType

A SymbolicConstant specifying the type of time-dependent allowable interference for contact pairs and contact elements. Possible values are:

- NONE, specifying no allowable contact interference.
- SHRINK_FIT.
- UNIFORM.

The default value is NONE.

overclosure

A Float specifying the maximum overclosure distance allowed. This argument applies only when *interferenceType*=UNIFORM. The default value is 0.0.

interferenceDirectionType

A SymbolicConstant specifying the method used to determine the interference direction. Possible values are COMPUTED and DIRECTION_COSINE. The default value is COMPUTED.

direction

A sequence of three Floats specifying the following:

- *X*-direction cosine of the interference direction vector.
- *Y*-direction cosine of the interference direction vector.
- *Z*-direction cosine of the interference direction vector.

This argument is required only when *interferenceDirectionType*=DIRECTION_COSINE.

amplitude

A String specifying the name of the amplitude curve that defines the magnitude of the prescribed interference during the step. Use **None** to specify that the prescribed interference is applied immediately at the beginning of the step and ramped down to zero linearly over the step.

contactControls

A String specifying the name of the ContactControl object associated with this interaction. The empty string indicates that the default contact controls will be used. The default value is an empty string.

Return value

None

Exceptions

None.

25.75.5 Members

The `SurfaceToSurfaceContactStd` object has members with the same names and descriptions as the arguments to the `SurfaceToSurfaceContactStd` method except the optional arguments to the `setValuesInStep` method.

In addition, the `SurfaceToSurfaceContactStd` object has the following members:

contactTracking

A `SymbolicConstant` specifying the choice of contact tracking algorithm. The `STATE` tracking algorithm uses only normal projections and is specified by using `ONE_CONFIG`. The `PATH` tracking algorithm uses crossing and normal projections and is specified by using `TWO_CONFIG`. Possible values are `ONE_CONFIG` and `TWO_CONFIG`. The default value is `TWO_CONFIG`.

This argument is valid only when `sliding=FINITE` and `enforcement=SURFACE_TO_SURFACE`.

supplementaryContact

A `SymbolicConstant` specifying the manner in which mid-face constraints are employed. Possible values are `SELECTIVE`, `NEVER`, and `ALWAYS`. The default value is `SELECTIVE`.

This argument is not valid when `sliding=FINITE` and `enforcement=SURFACE_TO_SURFACE`.

25.76 SurfaceToSurfaceExpState object

The `SurfaceToSurfaceExpState` object stores the propagating data for a `SurfaceToSurfaceContactExp` object. One instance of this object is created internally by the `SurfaceToSurfaceContactExp` object for each step. The instance is also deleted internally by the `SurfaceToSurfaceContactExp` object.

The `SurfaceToSurfaceExpState` object has no constructor or methods.

The `SurfaceToSurfaceExpState` object is derived from the `InteractionState` object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.76.1 Members

The SurfaceToSurfaceExpState object has the following members:

interactionPropertyState

A SymbolicConstant specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

contactControlsState

A SymbolicConstant specifying the propagation state of the *contactControls* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

contactControls

A String specifying the name of the ContactControl object associated with this interaction.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.76.2 Corresponding analysis keywords

*CONTACT CONTROLS

*CONTACT PAIR

25.77 SurfaceToSurfaceStdState object

The SurfaceToSurfaceStdState object stores the propagating data for a SurfaceToSurfaceContactStd object. One instance of this object is created internally by the SurfaceToSurfaceContactStd object for each step. The instance is also deleted internally by the SurfaceToSurfaceContactStd object.

The SurfaceToSurfaceStdState object has no constructor or methods.

The SurfaceToSurfaceStdState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.77.1 Members

The SurfaceToSurfaceStdState object has the following members:

interactionPropertyState

A SymbolicConstant specifying the propagation state of the *interactionProperty* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interferenceType

A SymbolicConstant specifying the interference type. Possible values are NONE, SHRINK_FIT, and UNIFORM.

interferenceTypeState

A SymbolicConstant specifying the propagation state of the *interferenceType* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

overclosure

A Float specifying the allowable overclosure.

overclosureState

A SymbolicConstant specifying the propagation state of the *overclosure* member. Possible values are COMPUTED and DIRECTION_COSINE.

interferenceDirectionType

A SymbolicConstant specifying the interference direction type. Possible values are COMPUTED and DIRECTION_COSINE.

interferenceDirectionTypeState

A SymbolicConstant specifying the propagation state of the *interferenceDirectionType* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

directionState

A SymbolicConstant specifying the propagation state of the *direction* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

contactControlsState

A SymbolicConstant specifying the propagation state of the *contactControls* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

interactionProperty

A String specifying the name of the ContactProperty object associated with this interaction.

amplitude

A String specifying the name of the Amplitude object that defines the magnitude of the prescribed interference during the step.

contactControls

A String specifying the name of the ContactControl object associated with this interaction.

direction

A tuple of three Floats specifying the following:

- *X*-direction cosine of the interference direction vector.
- *Y*-direction cosine of the interference direction vector.
- *Z*-direction cosine of the interference direction vector.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.77.2 Corresponding analysis keywords

- *CONTACT CONTROLS
- *CONTACT PAIR
- *CONTACT INTERFERENCE

25.78 ThermalConductance object

The ThermalConductance object specifies thermal conductance for a contact interaction property.

Access

```
import interaction
mdb.models[name].interactionProperties[name].thermalConductance
```

25.78.1 ThermalConductance(...)

This method creates a ThermalConductance object.

Path

```
mdb.models[name].interactionProperties[name].ThermalConductance
```

Required arguments

None.

Optional arguments

definition

A SymbolicConstant specifying how the thermal conductance is defined. Possible values are TABULAR and USER_DEFINED. The default value is TABULAR.

clearanceDependency

A Boolean specifying whether to use clearance-dependent data. The default value is ON.

pressureDependency

A Boolean specifying whether to use pressure-dependent data. The default value is OFF.

temperatureDependencyC

A Boolean specifying whether to use temperature-dependent data with clearance dependency. The default value is OFF.

massFlowRateDependencyC

A Boolean specifying whether to use mass-flow-rate-dependent data with clearance dependency. The default value is OFF.

dependenciesC

An Int specifying the number of field variables to use with clearance dependency. The default value is 0.

clearanceDepTable

A sequence of sequences of Floats specifying clearance dependency data. The items in the table data are described below.

temperatureDependencyP

A Boolean specifying whether to use temperature-dependent data with pressure dependency. The default value is OFF.

massFlowRateDependencyP

A Boolean specifying whether to use mass-flow-rate-dependent data with pressure dependency. The default value is OFF.

dependenciesP

An Int specifying the number of field variables to use with pressure dependency. The default value is 0.

pressureDepTable

A sequence of sequences of Floats specifying pressure dependency data. The items in the table data are described below.

Table data

The *clearanceDepTable* data specify the following:

- Conductivity.
- Clearance.
- Temperature, if the data depend on temperature.
- Mass flow rate, if the data depend on mass flow rate.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The *pressureDepTable* data specify the following:

- Conductivity.
- Pressure.
- Temperature, if the data depend on temperature.
- Mass flow rate, if the data depend on mass flow rate.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ThermalConductance object.

Exceptions

None.

25.78.2 **setValues(...)**

This method modifies the **ThermalConductance** object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ThermalConductance** method.

Return value

None

Exceptions

None.

25.78.3 **Members**

The **ThermalConductance** object has members with the same names and descriptions as the arguments to the **ThermalConductance** method.

25.78.4 **Corresponding analysis keywords**

*GAP CONDUCTANCE

25.79 **XFEMCrackGrowth object**

The **XFEMCrackGrowth** object defines the enrichment activation state for an **XFEMCrack** .

The **XFEMCrackGrowth** object is derived from the **Interaction** object.

Access

```
import interaction
mdb.models[name].interactions[name]
```

25.79.1 XFEMCrackGrowth(...)

This method creates an XFEMCrackGrowth object.

Path

```
mdb.models[name].XFEMCrackGrowth
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the XFEMCrackGrowth object is created.

crackName

A String specifying the XFEMCrack object associated with this interaction.

Optional argument

allowGrowth

A Boolean specifying whether the crack is allowed to grow (propagate) during this analysis step.
The default value is ON.

Return value

A XFEMCrackGrowth object.

Exceptions

None.

25.79.2 setValues(...)

This method modifies the data for an existing XFEMCrackGrowth object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **XFEMCrackGrowth** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

25.79.3 setValuesInStep(...)

This method modifies the propagating data for an existing XFEMCrackGrowth object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the interaction is modified.

Optional argument

allowGrowth

A Boolean specifying whether the crack is allowed to grow (propagate) during this analysis step.
The default value is ON.

Return value

None

Exceptions

None.

25.79.4 Members

The XFEMCrackGrowth object has members with the same names and descriptions as the arguments to the **XFEMCrackGrowth** method.

25.79.5 Corresponding analysis keywords

*ENRICHMENT ACTIVATION

25.80 XFEMCrackGrowthState object

The XFEMCrackGrowthState object stores the propagating data of an XFEMCrackGrowth object in a step. One instance of this object is created internally by the XFEMCrackGrowth object for each step. The instance is also deleted internally by the XFEMCrackGrowth object.

The XFEMCrackGrowthState object has no constructor or methods.

The XFEMCrackGrowthState object is derived from the InteractionState object.

Access

```
import interaction
mdb.models[name].steps[name].interactionStates[name]
```

25.80.1 Members

The XFEMCrackGrowthState object has the following members:

allowGrowth

A Boolean specifying whether the crack is allowed to grow (propagate) during this analysis step. The default value is ON.

allowGrowthState

A SymbolicConstant specifying the propagation state of the *allowGrowth* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the InteractionState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

25.80.2 Corresponding analysis keywords

*ENRICHMENT ACTIVATION

26. Job commands

The Job commands provide methods to create, modify, submit, and control jobs.

26.1 Job object

The Job object is the abstract base type for other Job objects. The Job object has no explicit constructor. The methods and members of the Job object are common to all objects derived from Job.

Access

```
import job
mdb.coexecutions[name].jobs[name]
mdb.jobs[name]
```

26.1.1 kill()

This method kills the analysis of a job.

Arguments

None.

Return value

None

Exceptions

None.

26.1.2 submit(...)

This method submits a job for analysis.

Arguments

Required arguments

None.

Optional arguments

consistencyChecking

A Boolean specifying whether to perform consistency checking for the job. The default value is ON. It is not recommended to turn the consistency checking off unless you are absolutely sure the model is consistent.

datacheckJob

A Boolean specifying whether to run the job as a datacheck analysis. The default value is False. The datacheckJob and continueJob arguments cannot both be True.

continueJob

A Boolean specifying whether to run the job as a continuation analysis. The default value is False. The datacheckJob and continueJob arguments cannot both be True.

Return value

None

Exceptions

None.

26.1.3 **waitForCompletion()**

This method interrupts the execution of the script until the end of the analysis. If you call the **waitForCompletion** method and the *status* member is neither SUBMITTED nor RUNNING, Abaqus assumes the analysis has either completed or aborted and returns immediately.

Arguments

None.

Return value

None

Exceptions

None.

26.1.4 **Members**

The Job object can have the following members:

name

A String specifying the name of the new job. The name must be a valid Abaqus/CAE object name.

type

A SymbolicConstant specifying the type of job. Possible values are ANALYSIS, SYNTAXCHECK, RECOVER, and RESTART. The default value is ANALYSIS.

If the object has the type JobFromInputFile , *type*=RESTART is not available.

waitHours

An Int specifying the number of hours to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitMinutes*. *waitHours* and *atTime* are mutually exclusive.

waitMinutes

An Int specifying the number of minutes to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitHours*. *waitMinutes* and *atTime* are mutually exclusive.

numCpus

An Int specifying the number of CPUs to use for this analysis if parallel processing is available. Possible values are *numCpus* > 0. The default value is 1.

memory

An Int specifying the amount of memory available to Abaqus analysis. The value should be expressed in the units supplied in *memoryUnits*. The default value is 90.

memoryUnits

A SymbolicConstant specifying the units for the amount of memory used in an Abaqus analysis. Possible values are PERCENTAGE, MEGA_BYTES, and GIGA_BYTES. The default value is PERCENTAGE.

getMemoryFromAnalysis

A Boolean specifying whether to retrieve the recommended memory settings from the last datacheck or analysis run and use those values in subsequent submissions. The default value is ON.

explicitPrecision

A SymbolicConstant specifying whether to use the double precision version of Abaqus/Explicit. Possible values are SINGLE, FORCE_SINGLE, DOUBLE, DOUBLE_CONSTRAINT_ONLY, and DOUBLE_PLUS_PACK. The default value is SINGLE.

nodalOutputPrecision

A SymbolicConstant specifying the precision of the nodal output written to the output database. Possible values are SINGLE and FULL. The default value is SINGLE.

parallelizationMethodExplicit

A SymbolicConstant specifying the parallelization method for Abaqus/Explicit. Possible values are LOOP and DOMAIN. The default value is LOOP.

JOB COMMANDS

numDomains

An Int specifying the number of domains for parallel execution in Abaqus/Explicit. When *parallelizationMethodExplicit*=DOMAIN, *numDomains* must be a multiple of *numCpus*. The default value is 1.

activateLoadBalancing

A Boolean specifying whether to activate dyanmic load balancing for jobs running on multiple processors with multiple domains in Abaqus/Explicit. The default value is OFF.

multiprocessingMode

A SymbolicConstant specifying whether an analysis is decomposed into threads or into multiple processes that communicate through a message passing interface (MPI). Possible values are DEFAULT, THREADS, and MPI. The default value is DEFAULT.

analysis

A SymbolicConstant specifying whether the job will be analyzed by Abaqus/Standard, Abaqus/Explicit, or Abaqus/CFD. Possible values are STANDARD, EXPLICIT, CFD, and UNKNOWN.

If the object has the type JobFromInputFile , *analysis*=UNKNOWN.

status

A SymbolicConstant specifying the status of the analysis. Possible values are SUBMITTED, RUNNING, ABORTED, TERMINATED, COMPLETED, CHECK_RUNNING, and CHECK_COMPLETED.

If the *message* member is empty, *status* is set to NONE.

queue

A String specifying the name of the queue to which to submit the job. The default value is an empty string.

Note: You can use the *queue* argument when creating a Job object on a Windows workstation; however, remote queues are available only on Linux platforms.

atTime

A String specifying the time at which to submit the job. If *queue* is empty, the string syntax must be valid for the Linux **at** command. If *queue* is set, the syntax must be valid according to the system administrator. The default value is an empty string.

Note: You can use the *atTime* argument when creating a Job object on a Windows workstation; however, the **at** command is available only on Linux platforms.

scratch

A String specifying the location of the scratch directory. The default value is an empty string.

userSubroutine

A String specifying the file containing the user's subroutine definitions. The default value is an empty string.

messages

A MessageArray object specifying the messages received during an analysis.

environment

A tuple of Strings specifying the environment variables and their values.

26.1.5 Corresponding analysis keywords

*HEADING

*PREPRINT

26.2 JobFromInputFile object

The JobFromInputFile object defines a Job object which analyzes a model contained in an input file.

The JobFromInputFile object is derived from the Job object.

Access

```
import job
mdb.jobs[name]
```

26.2.1 JobFromInputFile(...)

This method creates an analysis job using an input file for the model definition.

Path

```
mdb.JobFromInputFile
```

Required arguments

name

A String specifying the name of the new job. The name must be a valid Abaqus/CAE object name.

inputFileName

A String specifying the input file to read. Possible values are any valid file name. If the **.inp** extension is not included in the value of the argument, the system will append it for the user.

Optional arguments

type

A SymbolicConstant specifying the type of job. Possible values are ANALYSIS, SYNTAXCHECK, RECOVER, and RESTART. The default value is ANALYSIS.

If the object has the type JobFromInputFile, *type*=RESTART is not available.

queue

A String specifying the name of the queue to which to submit the job. The default value is an empty string.

Note: You can use the *queue* argument when creating a Job object on a Windows workstation; however, remote queues are available only on Linux platforms.

waitHours

An Int specifying the number of hours to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitMinutes*. *waitHours* and *atTime* are mutually exclusive.

waitMinutes

An Int specifying the number of minutes to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitHours*. *waitMinutes* and *atTime* are mutually exclusive.

atTime

A String specifying the time at which to submit the job. If *queue* is empty, the string syntax must be valid for the Linux **at** command. If *queue* is set, the syntax must be valid according to the system administrator. The default value is an empty string.

Note: You can use the *atTime* argument when creating a Job object on a Windows workstation; however, the **at** command is available only on Linux platforms.

scratch

A String specifying the location of the scratch directory. The default value is an empty string.

userSubroutine

A String specifying the file containing the user's subroutine definitions. The default value is an empty string.

numCpus

An Int specifying the number of CPUs to use for this analysis if parallel processing is available. Possible values are *numCpus* > 0. The default value is 1.

memory

An Int specifying the amount of memory available to Abaqus analysis. The value should be expressed in the units supplied in *memoryUnits*. The default value is 90.

memoryUnits

A SymbolicConstant specifying the units for the amount of memory used in an Abaqus analysis. Possible values are PERCENTAGE, MEGA_BYTES, and GIGA_BYTES. The default value is PERCENTAGE.

explicitPrecision

A SymbolicConstant specifying whether to use the double precision version of Abaqus/Explicit. Possible values are SINGLE, FORCE_SINGLE, DOUBLE, DOUBLE_CONSTRAINT_ONLY, and DOUBLE_PLUS_PACK. The default value is SINGLE.

nodalOutputPrecision

A SymbolicConstant specifying the precision of the nodal output written to the output database. Possible values are SINGLE and FULL. The default value is SINGLE.

parallelizationMethodExplicit

A SymbolicConstant specifying the parallelization method for Abaqus/Explicit. Possible values are LOOP and DOMAIN. The default value is LOOP.

numDomains

An Int specifying the number of domains for parallel execution in Abaqus/Explicit. When *parallelizationMethodExplicit*=DOMAIN, *numDomains* must be a multiple of *numCpus*. The default value is 1.

activateLoadBalancing

A Boolean specifying whether to activate dyanmic load balancing for jobs running on multiple processors with multiple domains in Abaqus/Explicit. The default value is OFF.

multiprocessingMode

A SymbolicConstant specifying whether an analysis is decomposed into threads or into multiple processes that communicate through a message passing interface (MPI). Possible values are DEFAULT, THREADS, and MPI. The default value is DEFAULT.

Return value

A JobFromInputFile object.

Exceptions

AbaqusException.

26.2.2 setValues(...)

This method modifies the JobFromInputFile object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **JobFromInputFile** method, except for the *name* argument.

Return value

None

Exceptions

26.2.3 Members

The JobFromInputFile object has members with the same names and descriptions as the arguments to the **JobFromInputFile** method.

In addition, the JobFromInputFile object can have the following members:

getMemoryFromAnalysis

A Boolean specifying whether to retrieve the recommended memory settings from the last datacheck or analysis run and use those values in subsequent submissions. The default value is ON.

analysis

A SymbolicConstant specifying whether the job will be analyzed by Abaqus/Standard, Abaqus/Explicit, or Abaqus/CFD. Possible values are STANDARD, EXPLICIT, CFD, and UNKNOWN.

If the object has the type JobFromInputFile, *analysis*=UNKNOWN.

status

A SymbolicConstant specifying the status of the analysis. Possible values are SUBMITTED, RUNNING, ABORTED, TERMINATED, COMPLETED, CHECK_RUNNING, and CHECK_COMPLETED.

If the *message* member is empty, *status* is set to NONE.

messages

A MessageArray object specifying the messages received during an analysis.

environment

A tuple of Strings specifying the environment variables and their values.

26.3 Message object

The Message object contains information about a given phase of the simulation. Job messages are not returned if a script is run without the Abaqus/CAE GUI (using the **noGUI** option).

Access

```
import job
mdb.coexecutions[name].jobs[name].messages[i]
mdb.jobs[name].messages[i]
```

26.3.1 Members

The Message object has the following members:

type

A SymbolicConstant specifying the type of message. Possible values are:

- ABORTED
- ANY_JOB
- ANY_MESSAGE_TYPE
- COMPLETED
- END_STEP
- ERROR
- HEADING
- HEALER_JOB
- HEALER_TYPE
- INTERRUPTED
- ITERATION
- JOB_ABORTED
- JOB_COMPLETED
- JOB_INTERRUPTED
- JOB_SUBMITTED
- MONITOR_DATA
- ODB_FILE
- ODB_FRAME
- STARTED
- STATE_FRAME
- STATUS

- STEP
- WARNING

data

A Dictionary object specifying the data returned by the analysis product. The value depends on the message returned. For a list of the possible entries, see the members of `DataObject` .

26.4 ModelJob object

The `ModelJob` object defines a `Job` object which analyzes a model on a model database (MDB).
The `ModelJob` object is derived from the `Job` object.

Access

```
import job
mdb.adaptivityProcesses[name].job
mdb.jobs[name]
```

26.4.1 Job(...)

This method creates an analysis job using a model on a model database (MDB) for the model definition.

Path

`mdb.Job`

Required arguments

name

A String specifying the name of the new job. The name must be a valid Abaqus/CAE object name.

model

A String specifying the name of the model to be analyzed or a `Model` object specifying the model to be analyzed.

Optional arguments

description

A String specifying a description of the job.

type

A `SymbolicConstant` specifying the type of job. Possible values are `ANALYSIS`, `SYNTAXCHECK`, `RECOVER`, and `RESTART`. The default value is `ANALYSIS`.

If the object has the type `JobFromInputFile`, `type=RESTART` is not available.

queue

A String specifying the name of the queue to which to submit the job. The default value is an empty string.

Note: You can use the *queue* argument when creating a Job object on a Windows workstation; however, remote queues are available only on Linux platforms.

waitHours

An Int specifying the number of hours to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitMinutes*. *waitHours* and *atTime* are mutually exclusive.

waitMinutes

An Int specifying the number of minutes to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitHours*. *waitMinutes* and *atTime* are mutually exclusive.

atTime

A String specifying the time at which to submit the job. If *queue* is empty, the string syntax must be valid for the Linux **at** command. If *queue* is set, the syntax must be valid according to the system administrator. The default value is an empty string.

Note: You can use the *atTime* argument when creating a Job object on a Windows workstation; however, the **at** command is available only on Linux platforms.

echoPrint

A Boolean specifying whether an echo of the input data is printed. The default value is OFF.

contactPrint

A Boolean specifying whether contact constraint data are printed. The default value is OFF.

modelPrint

A Boolean specifying whether model definition data are printed. The default value is OFF.

historyPrint

A Boolean specifying whether history data are printed. The default value is OFF.

scratch

A String specifying the location of the scratch directory. The default value is an empty string.

userSubroutine

A String specifying the file containing the user's subroutine definitions. The default value is an empty string.

JOB COMMANDS

numCpus

An Int specifying the number of CPUs to use for this analysis if parallel processing is available. Possible values are *numCpus* > 0. The default value is 1.

memory

An Int specifying the amount of memory available to Abaqus analysis. The value should be expressed in the units supplied in *memoryUnits*. The default value is 90.

memoryUnits

A SymbolicConstant specifying the units for the amount of memory used in an Abaqus analysis. Possible values are PERCENTAGE, MEGA_BYTES, and GIGA_BYTES. The default value is PERCENTAGE.

explicitPrecision

A SymbolicConstant specifying whether to use the double precision version of Abaqus/Explicit. Possible values are SINGLE, FORCE_SINGLE, DOUBLE, DOUBLE_CONSTRAINT_ONLY, and DOUBLE_PLUS_PACK. The default value is SINGLE.

nodalOutputPrecision

A SymbolicConstant specifying the precision of the nodal output written to the output database. Possible values are SINGLE and FULL. The default value is SINGLE.

parallelizationMethodExplicit

A SymbolicConstant specifying the parallelization method for Abaqus/Explicit. Possible values are LOOP and DOMAIN. The default value is LOOP.

numDomains

An Int specifying the number of domains for parallel execution in Abaqus/Explicit. When *parallelizationMethodExplicit*=DOMAIN, *numDomains* must be a multiple of *numCpus*. The default value is 1.

activateLoadBalancing

A Boolean specifying whether to activate dynamic load balancing for jobs running on multiple processors with multiple domains in Abaqus/Explicit. The default value is OFF.

multiprocessingMode

A SymbolicConstant specifying whether an analysis is decomposed into threads or into multiple processes that communicate through a message passing interface (MPI). Possible values are DEFAULT, THREADS, and MPI. The default value is DEFAULT.

Return value

A ModelJob object.

Exceptions

AbaqusException.

26.4.2 **writelnInput(...)**

This method writes an input file.

Arguments

Required arguments

None.

Optional argument

consistencyChecking

A Boolean specifying whether to perform consistency checking for the job. The default value is ON. It is not recommended to turn the consistency checking off unless you are absolutely sure the model is consistent.

Return value

None

Exceptions

None.

26.4.3 **setValues(...)**

This method modifies the `ModelJob` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ModelJob** method.

Return value

None

Exceptions

None.

26.4.4 **Members**

The `ModelJob` object can have the following members:

JOB COMMANDS

name

A String specifying the name of the new job. The name must be a valid Abaqus/CAE object name.

echoPrint

A Boolean specifying whether an echo of the input data is printed. The default value is OFF.

contactPrint

A Boolean specifying whether contact constraint data are printed. The default value is OFF.

modelPrint

A Boolean specifying whether model definition data are printed. The default value is OFF.

historyPrint

A Boolean specifying whether history data are printed. The default value is OFF.

model

A String specifying the name of the model to be analyzed or a Model object specifying the model to be analyzed.

description

A String specifying a description of the job.

type

A SymbolicConstant specifying the type of job. Possible values are ANALYSIS, SYNTAXCHECK, RECOVER, and RESTART. The default value is ANALYSIS.

If the object has the type JobFromInputFile, *type*=RESTART is not available.

waitHours

An Int specifying the number of hours to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitMinutes*. *waitHours* and *atTime* are mutually exclusive.

waitMinutes

An Int specifying the number of minutes to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitHours*. *waitMinutes* and *atTime* are mutually exclusive.

numCpus

An Int specifying the number of CPUs to use for this analysis if parallel processing is available. Possible values are *numCpus* > 0. The default value is 1.

memory

An Int specifying the amount of memory available to Abaqus analysis. The value should be expressed in the units supplied in *memoryUnits*. The default value is 90.

memoryUnits

A SymbolicConstant specifying the units for the amount of memory used in an Abaqus analysis. Possible values are PERCENTAGE, MEGA_BYTES, and GIGA_BYTES. The default value is PERCENTAGE.

getMemoryFromAnalysis

A Boolean specifying whether to retrieve the recommended memory settings from the last datacheck or analysis run and use those values in subsequent submissions. The default value is ON.

explicitPrecision

A SymbolicConstant specifying whether to use the double precision version of Abaqus/Explicit. Possible values are SINGLE, FORCE_SINGLE, DOUBLE, DOUBLE_CONSTRAINT_ONLY, and DOUBLE_PLUS_PACK. The default value is SINGLE.

nodalOutputPrecision

A SymbolicConstant specifying the precision of the nodal output written to the output database. Possible values are SINGLE and FULL. The default value is SINGLE.

parallelizationMethodExplicit

A SymbolicConstant specifying the parallelization method for Abaqus/Explicit. Possible values are LOOP and DOMAIN. The default value is LOOP.

numDomains

An Int specifying the number of domains for parallel execution in Abaqus/Explicit. When *parallelizationMethodExplicit*=DOMAIN, *numDomains* must be a multiple of *numCpus*. The default value is 1.

activateLoadBalancing

A Boolean specifying whether to activate dynamic load balancing for jobs running on multiple processors with multiple domains in Abaqus/Explicit. The default value is OFF.

multiprocessingMode

A SymbolicConstant specifying whether an analysis is decomposed into threads or into multiple processes that communicate through a message passing interface (MPI). Possible values are DEFAULT, THREADS, and MPI. The default value is DEFAULT.

analysis

A SymbolicConstant specifying whether the job will be analyzed by Abaqus/Standard, Abaqus/Explicit, or Abaqus/CFD. Possible values are STANDARD, EXPLICIT, CFD, and UNKNOWN.

If the object has the type JobFromInputFile, *analysis*=UNKNOWN.

status

A SymbolicConstant specifying the status of the analysis. Possible values are SUBMITTED, RUNNING, ABORTED, TERMINATED, COMPLETED, CHECK_RUNNING, and CHECK_COMPLETED.

If the *message* member is empty, *status* is set to NONE.

queue

A String specifying the name of the queue to which to submit the job. The default value is an empty string.

Note: You can use the *queue* argument when creating a Job object on a Windows workstation; however, remote queues are available only on Linux platforms.

atTime

A String specifying the time at which to submit the job. If *queue* is empty, the string syntax must be valid for the Linux **at** command. If *queue* is set, the syntax must be valid according to the system administrator. The default value is an empty string.

Note: You can use the *atTime* argument when creating a Job object on a Windows workstation; however, the **at** command is available only on Linux platforms.

scratch

A String specifying the location of the scratch directory. The default value is an empty string.

userSubroutine

A String specifying the file containing the user's subroutine definitions. The default value is an empty string.

messages

A MessageArray object specifying the messages received during an analysis.

environment

A tuple of Strings specifying the environment variables and their values.

26.5 Queue object

A Queue object tells the job where and how to submit a job remotely. A Queue object can be used as the *queue* argument to the **Job** method.

Access

```
import job
session.queues[name]
```

26.5.1 Queue(...)

This method creates a Queue object.

Note: Remote queues are available only on Linux platforms.

Path

```
session.Queue
```

Required arguments*name*

A String specifying the name of the new Queue object.

queueName

A String specifying the name of the remote analysis queue.

Optional arguments*hostName*

A String specifying the name of the remote host. The default value is an empty string.

fileCopy

A Boolean specifying if the results files are to be copied from the remote machine to the local machine. The default value is ON.

directory

A String specifying the remote location for the execution of the simulation. The default value is an empty string.

driver

A String specifying the designation of the remote driver. The default value is "abaqus".

remotePlatform

A SymbolicConstant specifying the type of operating system on the remote machine. The default value is Linux.

filesToCopy

A list of Strings specifying the files to be copied from the remote location to the local machine, or ALL. Strings specified in a list are the extensions of the job files that will be copied, such as ('log', 'dat', 'msg', 'sta', 'odb'). The default value is ALL.

deleteAfterCopy

A Boolean specifying whether remote files are to be deleted after they are copied to the local machine. The default value is OFF.

description

A String specifying a description of the queue. The default value is an empty string.

Return value

A Queue object.

ExceptionsIf *fileCopy*=ON and *hostName* is empty:

Remote queue host name is not set.

If *fileCopy*=ON and *directory* is empty:

Directory in which to run the job on the remote computer is not set.

26.5.2 Members

The Queue object has members with the same names and descriptions as the arguments to the `Queue` method.

26.6 Coexecution object

The Coexecution object contains a set of jobs as associated parameters to define a co-simulation analysis.

Access

```
import job
mdb.coexecutions[name]
```

26.6.1 kill()

This method kills the analysis of a co-execution.

Arguments

None.

Return value

None

Exceptions

None.

26.6.2 submit(...)

This method submits a co-execution for analysis.

Arguments

Required arguments

None.

Optional arguments

consistencyChecking

A Boolean specifying whether to perform consistency checking for the individual jobs. The default value is ON. It is not recommended to turn the consistency checking off unless you are absolutely sure the models are all consistent.

datacheckJob

A Boolean specifying whether to run the co-execution as a datacheck analysis. The default value is False. The *datacheckJob* and *continueJob* arguments cannot both be True.

continueJob

A Boolean specifying whether to run the co-execution as a continuation analysis. The default value is False. The *datacheckJob* and *continueJob* arguments cannot both be True.

Return value

None

Exceptions

None.

26.6.3 writeInput(...)

This method writes an input file for each analysis in the co-execution.

Arguments**Required arguments**

None.

Optional argument*consistencyChecking*

A Boolean specifying whether to perform consistency checking for the individual jobs. The default value is ON. It is not recommended to turn the consistency checking off unless you are absolutely sure the models are all consistent.

Return value

None

Exceptions

None.

26.6.4 waitForCompletion()

This method interrupts the execution of the script until the end of all the analyses. If you call the **waitForCompletion** method and the *status* member is neither SUBMITTED nor RUNNING, Abaqus assumes the analysis has either completed or aborted and returns immediately.

Arguments

None.

Return value

None

Exceptions

None.

26.6.5 Members

The Coexecution object can have the following members:

name

A String specifying the name of the new job. The name must be a valid Abaqus/CAE object name.

type

A SymbolicConstant specifying the type of analysis to execute for the co-execution. Possible values are ANALYSIS, SYNTAXCHECK, RECOVER, and RESTART. The default value is ANALYSIS.

masterAnalysisProduct

A SymbolicConstant specifying the analysis product type of the master model for the co-execution. The default value is ABAQUS.

waitHours

An Int specifying the number of hours to wait before submitting the co-execution. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitMinutes*. *waitHours* and *atTime* are mutually exclusive.

waitMinutes

An Int specifying the number of minutes to wait before submitting the job. This argument is ignored if *queue* is set. The default value is 0.

This argument works in conjunction with *waitHours*. *waitMinutes* and *atTime* are mutually exclusive.

status

A SymbolicConstant specifying the status of the co-execution. Possible values are SUBMITTED, RUNNING, ABORTED, TERMINATED, COMPLETED, CHECK_SUBMITTED, CHECK_RUNNING, and CHECK_COMPLETED.

If the *message* member of all the jobs are empty, *status* is set to NONE.

queue

A String specifying the name of the queue to which to submit the co-execution. The default value is an empty string.

Note: You can use the *queue* argument when creating a Coexecution object on a Windows workstation; however, remote queues are available only on Linux platforms.

atTime

A String specifying the time at which to submit the co-execution. If *queue* is empty, the string syntax must be valid for the Linux **at** command. If *queue* is set, the syntax must be valid according to the system administrator. The default value is an empty string.

Note: You can use the *atTime* argument when creating a Coexecution object on a Windows workstation; however, the **at** command is available only on Linux platforms.

jobs

A repository of Job objects specifying the jobs that comprise this co-execution.

slaveModels

A tuple of Strings specifying the names of the slave models for the co-execution.

slaveAnalysisProducts

A tuple of SymbolicConstants specifying the analysis product types of the slave models for the co-execution. The default value is an empty sequence.

masterModel

A String specifying the name of the master model for the co-execution.

26.6.6 Corresponding analysis keywords

*HEADING

*PREPRINT

26.7 OptimizationProcess object

The OptimizationProcess object defines a process to perform an optimization of a model defined using an optimization task.

Access

```
import job
mdb.optimizationProcesses [name]
```

26.7.1 OptimizationProcess(...)

This method creates an OptimizationProcess object.

Path

`mdb.OptimizationProcess`

Required arguments

name

A String specifying name of the optimization process.

model

A String specifying name of the model to be used for the optimization process.

task

A String specifying name of the optimization task to be used for the optimization process.

prototypeJob

A String specifying name of the job to be used as the prototype for all analysis jobs run by the optimization process.

Optional arguments

description

A String specifying a description of the optimization process.

maxDesignCycle

An Int specifying the maximum number of allowed design cycles for the optimization process. The default value is 15.

dataSaveFrequency

An Enum specifying whether Abaqus should save every iteration file in the optimization process or a selection of iteration files saved at a user-specified frequency. If you set *dataSaveFrequency*=OPT_DATASAVE_EVERY_CYCLE, Abaqus saves every iteration file; if you set *dataSaveFrequency*=OPT_DATASAVE_SPECIFY_CYCLE,). Abaqus saves iteration files according to the frequency defined by the *saveEvery* parameter. The default value is OPT_DATASAVE_SPECIFY_CYCLE.

saveInitial

A Boolean specifying whether the initial cycle should be saved when *dataSaveFrequency* is OPT_DATASAVE_SPECIFY_CYCLE. The default value is True.

saveFirst

A Boolean specifying whether the first cycle should be saved when *dataSaveFrequency* is OPT_DATASAVE_SPECIFY_CYCLE. The default value is True.

saveLast

A Boolean specifying whether the last cycle should be saved when *dataSaveFrequency* is `OPT_DATASAVE_SPECIFY_CYCLE`. The default value is `True`.

saveEvery

An Int specifying every *nth* cycle iterations to be saved when *dataSaveFrequency* is `OPT_DATASAVE_SPECIFY_CYCLE`. Abaqus saves file iterations for every *nth* iteration after iteration 1; if you set *saveEvery*=3, Abaqus saves file iterations for cycles 1, 4, 7, and so on. The default value is `None`.

Return value

An `OptimizationProcess` object.

Exceptions

`AbaqusException`.

26.7.2 writeParAndInputFiles()

This method allows you to write par and input files for an optimization task.

Arguments

`None`.

Return value

`None`

Exceptions

`None`.

26.7.3 submit(...)

This method submits an optimization process.

Arguments

Required arguments

`None`.

Optional argument

validate

A Boolean specifying whether Abaqus should perform the validation of the optimization process only. The default value is `False`.

Return value

None

Exceptions

None.

26.7.4 **waitForCompletion()**

This method interrupts the execution of the script until the end of all the analyses. If you call the **waitForCompletion** method and the *status* member is neither SUBMITTED nor RUNNING, Abaqus assumes the analysis has either completed or aborted and returns immediately.

Arguments

None.

Return value

None

Exceptions

None.

26.7.5 **extract(...)**

This method extracts a surface mesh from the optimized model.

Arguments

Required arguments

outputFileName

Name of the output file for the extracted surface mesh.

designCycle

The design cycle number for which the surface mesh should be extracted.

Optional arguments

isoValue

Value used to determine the positions on the element edges where the new nodes are created. Value between 0 and 1. Default: 0.3

smoothCycles

Number of smoothing cycles; if set to 0, no smoothing is performed. Default: 5

reductionPercent

Defines the percent of faces that should be removed during the data reduction. If set to 0, no data reduction occurs. If set to 100, the data reduction stops when no faces can be removed (that is checked using *reductionAngle* parameter). Value between 0 and 100. Default: 0

reductionAngle

Defines the maximal angle between adjacent faces at a node such that the node may be removed during the data reduction. Value in degrees between 0 and 90. Default: 15

targetVolume

Defines the target volume that is to be achieved iteratively by varying the isovalue. Value between 0 and 1. Default: 0

extractFormat

Tuple for the types of format of the output. Values are (OPT_EXTRACT_SMOOTH_ABAQUS_INPUT_FILE, OPT_EXTRACT_SMOOTH_STL). Default: OPT_EXTRACT_SMOOTH_ABAQUS_INPUT_FILE

resultFiltering

Possible string values are OFF or MODERATE or FULL. Defines if the element material values are to be filtered (averaged locally) before the isocut, and to what extent. Default: OFF

instances

Defines a list of names of part instances to be used for surface extraction. One file (according to *extractFormat*) is created for each part instance. If the argument is not specified, no part instance is selected and surface is generated for the whole model.

Return value

None

Exceptions

None.

26.7.6 **setValues(...)**

This method modifies the *OptimizationProcess* object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **OptimizationProcess** method, except for the *name* argument.

Return value

None

Exceptions

26.7.7 Members

The `OptimizationProcess` object has members with the same names and descriptions as the arguments to the `OptimizationProcess` method.

26.7.8 Corresponding analysis keywords

27. Load commands

A specific type of load object and a specific type of load state object are designed for each type of load. A load object stores the non-propagating data of a load as well as a number of instances of the corresponding load state object, each of which stores the propagating data of the load in a single step. Instances of the load state object are created and deleted internally by its corresponding load object.

27.1 Load object

The Load object is the abstract base type for other Load objects. The Load object has no explicit constructor. The methods and members of the Load object are common to all objects derived from Load.

Access

```
import load
mdb.models[name].loads[name]
```

27.1.1 deactivate(...)

This method deactivates the load in the specified step and all its subsequent steps.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is deactivated.

Optional arguments

None.

Return value

None

Exceptions

TextError.

27.1.2 move(...)

This method moves the load state object from one step to a different step.

LOAD COMMANDS

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the load state is moved.

toStepName

A String specifying the name of the step to which the load state is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

27.1.3 reset(...)

This method resets the load state of the specified step to the state of the previous general analysis step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load state is reset.

Optional arguments

None.

Return value

None

Exceptions

TextError.

27.1.4 resume()

This method resumes the load that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

27.1.5 suppress()

This method suppresses the load.

Arguments

None.

Return value

None

Exceptions

None.

27.1.6 delete(...)

This method allows you to delete existing loads.

Arguments**Required argument**

indices

A sequence of Ints specifying the index of each load to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

27.1.7 Members

The Load object can have the following members:

name

A String specifying the load repository key.

region

A Region object specifying the region to which the load is applied.

27.2 BodyCharge object

The BodyCharge object stores the data for a body charge.

The BodyCharge object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.2.1 BodyCharge(...)

This method creates a BodyCharge object.

Path

```
mdb.models[name].BodyCharge
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments*amplitude*

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

Return value

A BodyCharge object.

Exceptions

None.

27.2.2 setValues(...)

This method modifies the data for an existing BodyCharge object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyCharge** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.2.3 setValuesInStep(...)

This method modifies the propagating data for an existing BodyCharge object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.2.4 Members

The BodyCharge object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.3 BodyChargeState object

The BodyChargeState object stores the propagating data of a body charge in a step. One instance of this object is created internally by the BodyCharge object for each step. The instance is also deleted internally by the BodyCharge object.

The BodyChargeState object has no constructor or methods.

The BodyChargeState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.3.1 Members

The BodyChargeState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.3.2 Corresponding analysis keywords

*DECHARGE (load type label: EBF)

27.4 BodyConcentrationFlux object

The BodyConcentrationFlux object defines body concentration flux from a region or into a region.
The BodyConcentrationFlux object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.4.1 BodyConcentrationFlux(...)

This method creates a BodyConcentrationFlux object.

Path

```
mdb.models[name].BodyConcentrationFlux
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the body concentration flux magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the body concentration flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A BodyConcentrationFlux object.

Exceptions

None.

27.4.2 setValues(...)

This method modifies the data for an existing BodyConcentrationFlux object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyConcentrationFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.4.3 setValuesInStep(...)

This method modifies the propagating data for an existing BodyConcentrationFlux object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the Body heat flux is modified.

Optional arguments

magnitude

A Float specifying the Body concentration flux magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.4.4 Members

The BodyConcentrationFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the body concentration flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.5 BodyConcentrationFluxState object

The BodyConcentrationFluxState object stores the propagating data for a BodyConcentrationFlux object in a step. One instance of this object is created internally by the BodyConcentrationFlux object for each step. The instance is also deleted internally by the BodyConcentrationFlux object.

The BodyConcentrationFluxState object has no constructor or methods.

The BodyConcentrationFluxState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.5.1 Members

The BodyConcentrationFluxState object has the following members:

magnitude

A Float specifying the body concentration flux magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the body concentration flux magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.5.2 Corresponding analysis keywords

*CFLUX

27.6 BodyCurrent object

The BodyCurrent object stores the data for a body current.

The BodyCurrent object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.6.1 BodyCurrent(...)

This method creates a BodyCurrent object.

Path

```
mdb.models[name].BodyCurrent
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

Return value

A BodyCurrent object.

Exceptions

None.

27.6.2 **setValues(...)**

This method modifies the data for an existing BodyCurrent object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyCurrent** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.6.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing BodyCurrent object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.6.4 Members

The BodyCurrent object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.7 BodyCurrentDensity object

The BodyCurrentDensity object stores the data for a body current.

The BodyCurrentDensity object is derived from the Load object.

Access

```
import load
```

```
mdb.models[name].loads[name]
```

27.7.1 BodyCurrentDensity(...)

This method creates a BodyCurrentDensity object.

Path

`mdb.models[name].BodyCurrentDensity`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

comp1

A Complex specifying the first component of the load.

comp2

A Complex specifying the second component of the load.

comp3

A Complex specifying the third component of the load.

Optional arguments

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

Return value

A BodyCurrentDensity object.

Exceptions

None.

27.7.2 **setValues(...)**

This method modifies the data for an existing **BodyCurrentDensity** object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyCurrentDensity** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.7.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing **BodyCurrentDensity** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Complex specifying the first component of the load.

comp2

A Complex specifying the second component of the load.

comp3

A Complex specifying the third component of the load.

amplitude

A String or a **SymbolicConstant** specifying the name of the amplitude reference. Possible values for the **SymbolicConstant** are **UNCHANGED** and **FREED**. **UNCHANGED** should be used if the amplitude is propagated from the previous static analysis step. **FREED** should be used if the load

is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.7.4 Members

The BodyCurrentDensity object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

region

A Region object specifying the region to which the load is applied.

27.8 BodyCurrentState object

The BodyCurrentState object stores the propagating data of a body current in a step. One instance of this object is created internally by the BodyCurrent object for each step. The instance is also deleted internally by the BodyCurrent object.

The BodyCurrentState object has no constructor or methods.

The BodyCurrentState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.8.1 Members

The BodyCurrentState object has the following members:

magnitude

A Float specifying the load magnitude.

LOAD COMMANDS

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.8.2 Corresponding analysis keywords

*DECURRENT (load type label: CBF)

27.9 BodyForce object

The BodyForce object defines a distributed load.

The BodyForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.9.1 BodyForce(...)

This method creates a BodyForce object.

Path

`mdb.models[name].BodyForce`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

comp1

A Float or a Complex specifying the body force component in the 1-direction.

Note: Although *comp1*, *comp2*, and *comp3* are optional arguments, at least one of them must be nonzero unless *distributionType*=USER_DEFINED.

comp2

A Float or a Complex specifying the body force component in the 2-direction.

comp3

A Float or a Complex specifying the body force component in the 3-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A BodyForce object.

Exceptions

None.

27.9.2 **setValues(...)**

This method modifies the data for an existing BodyForce object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.9.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing BodyForce object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the body force component in the 1-direction. UNCHANGED should be used if the body force component is propagated from the previous analysis step.

comp2

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the body force component in the 2-direction. UNCHANGED should be used if the body force component is propagated from the previous analysis step.

comp3

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the body force component in the 3-direction. UNCHANGED should be used if the body force component is propagated from the previous analysis step.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.9.4 Members

The BodyForce object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.9.5 Corresponding analysis keywords

*DLOAD (load type label: BX, BY, BZ, BR, BXNU, BYNU, BZNU, or BRNU)

27.10 BodyForceState object

The BodyForceState object stores the propagating data of a body force in a step. One instance of this object is created internally by the BodyForce object for each step. The instance is also deleted internally by the BodyForce object.

The BodyForceState object has no constructor or methods.

The BodyForceState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.10.1 Members

The BodyForceState object has the following members:

comp1

A Float or a Complex specifying the body force component in the 1-direction.

comp2

A Float or a Complex specifying the body force component in the 2-direction.

comp3

A Float or a Complex specifying the body force component in the 3-direction.

comp1State

A SymbolicConstant specifying the propagation state of the body force component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

comp2State

A SymbolicConstant specifying the propagation state of the body force component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

comp3State

A SymbolicConstant specifying the propagation state of the body force component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED

- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.10.2 Corresponding analysis keywords

*DLOAD (load type label: BX, BY, BZ, BR, BXNU, BYNU, BZNU, or BRNU)

27.11 BodyHeatFlux object

The BodyHeatFlux object defines body heat flux from a region or into a region.

The BodyHeatFlux object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.11.1 BodyHeatFlux(...)

This method creates a BodyHeatFlux object.

Path

```
mdb.models[name].BodyHeatFlux
```

Required arguments

name

A String specifying the load repository key.

LOAD COMMANDS

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the body heat flux magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the body heat flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A BodyHeatFlux object.

Exceptions

None.

27.11.2 **setValues(...)**

This method modifies the data for an existing BodyHeatFlux object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BodyHeatFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.11.3 setValuesInStep(...)

This method modifies the propagating data for an existing BodyHeatFlux object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the body heat flux is modified.

Optional arguments

magnitude

A Float specifying the body heat flux magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.11.4 Members

The BodyHeatFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the body heat flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.12 BodyHeatFluxState object

The BodyHeatFluxState object stores the propagating data for a Body BodyHeatFlux object in a step. One instance of this object is created internally by the BodyHeatFlux object for each step. The instance is also deleted internally by the BodyHeatFlux object.

The BodyHeatFluxState object has no constructor or methods.

The BodyHeatFluxState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.12.1 Members

The BodyHeatFluxState object has the following members:

magnitude

A Float specifying the Body heat flux magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the Body heat flux magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.12.2 Corresponding analysis keywords

*DFLUX

27.13 BoltLoad object

The BoltLoad object defines a bolt load.

The BoltLoad object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.13.1 BoltLoad(...)

This method creates a BoltLoad object.

Path

```
mdb.models[name].BoltLoad
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the bolt load magnitude.

datumAxis

A DatumAxis object specifying the orientation of the pre-tension section normal.

Note: *datumAxis* is required only for Solid and Shell regions; it has no meaning for Wire regions.

Optional arguments

boltMethod

A SymbolicConstant specifying the method of applying the bolt load. Possible values are APPLY_FORCE and ADJUST_LENGTH. The default value is APPLY_FORCE.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A BoltLoad object.

Exceptions

TextError.

27.13.2 setValues(...)

This method modifies the data for an existing BoltLoad object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BoltLoad** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

27.13.3 setValuesInStep(...)

This method modifies the propagating data for an existing BoltLoad object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*boltMethod*

A SymbolicConstant specifying the type of bolt load. Possible values are APPLY_FORCE, ADJUST_LENGTH, and FIX_LENGTH. The default is APPLY_FORCE.

magnitude

A Float specifying the bolt load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value**None****Exceptions**

None.

27.13.4 Members

The BoltLoad object can have the following members:

name

A String specifying the load repository key.

datumAxis

A DatumAxis object specifying the orientation of the pre-tension section normal.

Note: *datumAxis* is required only for Solid and Shell regions; it has no meaning for Wire regions.

region

A Region object specifying the region to which the load is applied.

27.13.5 Corresponding analysis keywords

*PRE-TENSION SECTION

- *NODE (for the reference node)
- *NSET (for the reference node)

27.14 BoltLoadState object

The BoltLoadState object stores the propagating data of a bolt load in a step. One instance of this object is created internally by the BoltLoad object for each step. The instance is also deleted internally by the BoltLoad object.

The BoltLoadState object has no constructor or methods.

The BoltLoadState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.14.1 Members

The BoltLoadState object has the following members:

boltMethod

A SymbolicConstant specifying the type of bolt load. Possible values are APPLY_FORCE, ADJUST_LENGTH, and FIX_LENGTH.

boltMethodState

A SymbolicConstant specifying the propagation state of the bolt load type. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

magnitude

A Float specifying the bolt load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the bolt load magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED

- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.14.2 Corresponding analysis keywords

*CLOAD (when *boltMethod*=APPLY_FORCE)

*BOUNDARY (when *boltMethod*=ADJUST_LENGTH or FIX_LENGTH)

27.15 ConcCharge object

The ConcCharge object stores the data for a concentrated charge.

The ConcCharge object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.15.1 ConcCharge(...)

This method creates a ConcCharge object.

Path

```
mdb.models[name].ConcCharge
```

Required arguments

name

A String specifying the load repository key.

LOAD COMMANDS

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A ConcCharge object.

Exceptions

None.

27.15.2 **setValues(...)**

This method modifies the data for an existing ConcCharge object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcCharge** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.15.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConcCharge object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.15.4 Members

The ConcCharge object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.16 ConcConcFlux object

The ConcConcFlux object stores the data for a concentrated concentration flux.

The ConcConcFlux object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.16.1 ConcConcFlux(...)

This method creates a ConcConcFlux object.

Path

```
mdb.models[name].ConcConcFlux
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A ConcConcFlux object.

Exceptions

None.

27.16.2 setValues(...)

This method modifies the data for an existing ConcConcFlux object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcConcFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.16.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConcConcFlux object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.16.4 Members

The ConcConcFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.17 ConcCurrent object

The ConcCurrent object stores the data for a concentrated current.

The ConcCurrent object is derived from the Load object.

Access

```
import load
```

```
mdb.models[name].loads[name]
```

27.17.1 ConcCurrent(...)

This method creates a ConcCurrent object.

Path

`mdb.models[name].ConcCurrent`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A ConcCurrent object.

Exceptions

None.

27.17.2 setValues(...)

This method modifies the data for an existing ConcCurrent object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcCurrent** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.17.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConcCurrent object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.17.4 Members

The ConcCurrent object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.18 ConcCurrentState object

The ConcCurrentState object stores the propagating data of a concentrated current in a step. One instance of this object is created internally by the ConcCurrent object for each step. The instance is also deleted internally by the ConcCurrent object.

The ConcCurrentState object has no constructor or methods.

The ConcCurrentState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.18.1 Members

The ConcCurrentState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

LOAD COMMANDS

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.18.2 Corresponding analysis keywords

*CECURRENT

27.19 ConcentratedChargeState object

The ConcentratedChargeState object stores the propagating data of a concentrated charge in a step. One instance of this object is created internally by the ConcCharge object for each step. The instance is also deleted internally by the ConcCharge object.

The ConcentratedChargeState object has no constructor or methods.

The ConcentratedChargeState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.19.1 Members

The ConcentratedChargeState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.19.2 Corresponding analysis keywords

*CECHARGE

27.20 ConcentratedConcentrationFluxState object

The ConcentratedConcentrationFluxState object stores the propagating data of a concentrated concentration flux in a step. One instance of this object is created internally by the ConcConcFlux object for each step. The instance is also deleted internally by the ConcConcFlux object.

The ConcentratedConcentrationFluxState object has no constructor or methods.

The ConcentratedConcentrationFluxState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.20.1 Members

The ConcentratedConcentrationFluxState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.20.2 Corresponding analysis keywords

*CFLUX (degree of freedom: 11, which is assumed)

27.21 ConcentratedForce object

The ConcentratedForce object defines a concentrated force.

The ConcentratedForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.21.1 ConcentratedForce(...)

This method creates a ConcentratedForce object.

Path

```
mdb.models[name].ConcentratedForce
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

cf1

A Float or a Complex specifying the concentrated force component in the 1-direction. Although *cf1*, *cf2*, and *cf3* are optional arguments, at least one of them must be nonzero.

cf2

A Float or a Complex specifying the concentrated force component in the 2-direction.

cf3

A Float or a Complex specifying the concentrated force component in the 3-direction.

LOAD COMMANDS

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

follower

A Boolean specifying whether the direction of the force rotates with the rotation at each node of the region. You should provide the *follower* argument only if it is valid for the specified step. The default value is OFF.

localCsys

None or a DatumCsys object specifying the local coordinate system of the load's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

Return value

A ConcentratedForce object.

Exceptions

None.

27.21.2 setValues(...)

This method modifies the data for an existing ConcentratedForce object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcentratedForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.21.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConcentratedForce object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*cf1*

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the concentrated force component in the 1-direction. UNCHANGED should be used if the concentrated force component is propagated from the previous analysis step.

cf2

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the concentrated force component in the 2-direction. UNCHANGED should be used if the concentrated force component is propagated from the previous analysis step.

cf3

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the concentrated force component in the 3-direction. UNCHANGED should be used if the concentrated force component is propagated from the previous analysis step.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value**None****Exceptions**

None.

27.21.4 Members

The ConcentratedForce object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

follower

A Boolean specifying whether the direction of the force rotates with the rotation at each node of the region. You should provide the *follower* argument only if it is valid for the specified step. The default value is OFF.

localCsys

None or a DatumCsys object specifying the local coordinate system of the load's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.22 ConcentratedForceState object

The ConcentratedForceState object stores the propagating data for a concentrated force in a step. One instance of this object is created internally by the ConcentratedForce object for each step. The instance is also deleted internally by the ConcentratedForce object.

The ConcentratedForceState object has no constructor or methods.

The ConcentratedForceState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.22.1 Members

The ConcentratedForceState object has the following members:

cf1

A Float or a Complex specifying the concentrated force component in the 1-direction. Although *cf1*, *cf2*, and *cf3* are optional arguments, at least one of them must be nonzero.

cf2

A Float or a Complex specifying the concentrated force component in the 2-direction.

cf3

A Float or a Complex specifying the concentrated force component in the 3-direction.

cf1State

A SymbolicConstant specifying the propagation state of the concentrated force component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

cf2State

A SymbolicConstant specifying the propagation state of the concentrated force component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

cf3State

A SymbolicConstant specifying the propagation state of the concentrated force component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.22.2 Corresponding analysis keywords

*CLOAD (degree of freedom: 1, 2, or 3)

27.23 ConcentratedHeatFlux object

The ConcentratedHeatFlux object stores the data for a concentrated heat flux load.

The ConcentratedHeatFlux object is derived from the Load object.

Access

import load

```
mdb.models [name] .loads [name]
```

27.23.1 ConcentratedHeatFlux(...)

This method creates a ConcentratedHeatFlux object.

Path

```
mdb.models [name] .ConcentratedHeatFlux
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

dof

An Int specifying the degree of freedom of the node, to which the concentrated heat flux should be applied. The default value is 11.

Return value

A ConcentratedHeatFlux object.

Exceptions

None.

27.23.2 setValues(...)

This method modifies the data for an existing ConcentratedHeatFlux object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcentratedHeatFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.23.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConcentratedHeatFlux object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.23.4 Members

The ConcentratedHeatFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

dof

An Int specifying the degree of freedom of the node, to which the concentrated heat flux should be applied. The default value is 11.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.24 ConcentratedHeatFluxState object

The ConcentratedHeatFluxState object stores the propagating data of a concentrated heat flux load in a step. One instance of this object is created internally by the ConcentratedHeatFlux object for each step. The instance is also deleted internally by the ConcentratedHeatFlux object.

The ConcentratedHeatFluxState object has no constructor or methods.

The ConcentratedHeatFluxState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.24.1 Members

The ConcentratedHeatFluxState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.24.2 Corresponding analysis keywords

*CFLUX (degree of freedom: one out of integers 11 to 31)

27.25 ConcentratedPoreFluidState object

The ConcentratedPoreFluidState object stores the propagating data of a concentrated pore fluid flow load in a step. One instance of this object is created internally by the ConcPoreFluid object for each step. The instance is also deleted internally by the ConcPoreFluid object.

The ConcentratedPoreFluidState object has no constructor or methods.

The ConcentratedPoreFluidState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.25.1 Members

The ConcentratedPoreFluidState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.25.2 Corresponding analysis keywords

*CLOAD (degree of freedom: 8)

27.26 ConcPoreFluid object

The ConcPoreFluid object stores the data for a concentrated pore fluid flow load.

The ConcPoreFluid object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.26.1 ConcPoreFluid(...)

This method creates a ConcPoreFluid object.

Path

```
mdb.models[name].ConcPoreFluid
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A `ConcPoreFluid` object.

Exceptions

None.

27.26.2 `setValues(...)`

This method modifies the data for an existing `ConcPoreFluid` object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConcPoreFluid` method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.26.3 `setValuesInStep(...)`

This method modifies the propagating data for an existing `ConcPoreFluid` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a `SymbolicConstant` specifying the name of the amplitude reference. Possible values for the `SymbolicConstant` are `UNCHANGED` and `FREED`. `UNCHANGED` should be used if the amplitude is propagated from the previous static analysis step. `FREED` should be used if the load

is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.26.4 Members

The ConcPoreFluid object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.27 ConnectorForce object

The ConnectorForce object defines a connector force.

The ConnectorForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.27.1 ConnectorForce(...)

This method creates a ConnectorForce object on a wire region. Alternatively, the load may also be applied to a wire set referenced from an assembled fastener template model.

LOAD COMMANDS

Path

`mdb.models[name].ConnectorForce`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

Optional arguments

region

The wire region to which the load is applied. This argument is not valid when *fastenerName* and *fastenerSetName* are specified.

fastenerName

A String specifying the name of the assembled fastener to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

f1

A Float or a Complex specifying the connector force component in the connector's local 1-direction.

Note: Although *f1*, *f2*, and *f3* are optional arguments, at least one of them must be nonzero.

f2

A Float or a Complex specifying the connector force component in the connector's local 2-direction.

f3

A Float or a Complex specifying the connector force component in the connector's local 3-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A ConnectorForce object.

Exceptions

None.

27.27.2 setValues(...)

This method modifies the data for an existing ConnectorForce object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.27.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConnectorForce object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

f1

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the connector force component in the connector's local 1-direction. UNCHANGED should be used if the connector force component is propagated from the previous analysis step.

f2

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the connector force component in the connector's local 2-direction. UNCHANGED should be used if the connector force component is propagated from the previous analysis step.

LOAD COMMANDS

f3

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the connector force component in the connector's local 3-direction. UNCHANGED should be used if the connector force component is propagated from the previous analysis step.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.27.4 Members

The ConnectorForce object can have the following members:

name

A String specifying the load repository key.

fastenerName

A String specifying the name of the assembled fastener to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.28 ConnectorForceState object

The ConnectorForceState object stores the propagating data for a connector force in a step. One instance of this object is created internally by the ConnectorForce object for each step. The instance is also deleted internally by the ConnectorForce object.

The ConnectorForceState object has no constructor or methods.

The ConnectorForceState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.28.1 Members

The ConnectorForceState object has the following members:

f1

A Float or a Complex specifying the connector force component in the connector's local 1-direction.

f2

A Float or a Complex specifying the connector force component in the connector's local 2-direction.

f3

A Float or a Complex specifying the connector force component in the connector's local 3-direction.

f1State

A SymbolicConstant specifying the propagation state of the connector force component in the connector's local 1-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

f2State

A SymbolicConstant specifying the propagation state of the connector force component in the connector's local 2-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

f3State

A SymbolicConstant specifying the propagation state of the connector force component in the connector's local 3-direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

LOAD COMMANDS

- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.28.2 Corresponding analysis keywords

*CONNECTOR LOAD (degree of freedom: 1, 2, or 3)

27.29 ConnectorMoment object

The ConnectorMoment object stores the data for a connector moment.

The ConnectorMoment object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.29.1 ConnectorMoment(...)

This method creates a ConnectorMoment object on a wire region. Alternatively, the load may also be applied to a wire set referenced from an assembled fastener template model.

Path

```
mdb.models[name].ConnectorMoment
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

Optional arguments*region*

The wire region to which the load is applied. This argument is not valid when *fastenerName* and *fastenerSetName* are specified.

fastenerName

A String specifying the name of the assembled fastener to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

m1

A Float or a Complex specifying the moment component in the connector's local 4-direction.

m2

A Float or a Complex specifying the moment component in the connector's local 5-direction.

m3

A Float or a Complex specifying the moment component in the connector's local 6-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A ConnectorMoment object.

Exceptions

None.

27.29.2 setValues(...)

This method modifies the data for an existing ConnectorMoment object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorMoment** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.29.3 setValuesInStep(...)

This method modifies the propagating data for an existing ConnectorMoment object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

m1

A Float, a Complex, or a SymbolicConstant specifying the moment component in the connector's local 4-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

m2

A Float, a Complex, or a SymbolicConstant specifying the moment component in the connector's local 5-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

m3

A Float, a Complex, or a SymbolicConstant specifying the moment component in the connector's local 6-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.29.4 Members

The ConnectorMoment object can have the following members:

name

A String specifying the load repository key.

fastenerName

A String specifying the name of the assembled fastener to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerSetName* must also be specified. The default value is an empty string.

fastenerSetName

A String specifying the assembled fastener template model set to which the load will be applied. This argument is not valid when *region* is specified. When this argument is specified, *fastenerName* must also be specified. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.30 ConnectorMomentState object

The ConnectorMomentState object stores the propagating data for a connector moment in a step. One instance of this object is created internally by the ConnectorMoment object for each step. The instance is also deleted internally by the ConnectorMoment object.

The ConnectorMomentState object has no constructor or methods.

The ConnectorMomentState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.30.1 Members

The ConnectorMomentState object has the following members:

LOAD COMMANDS

m1

A Float or a Complex specifying the connector moment component in the connector's local 4-direction. Although *m1*, *m2*, and *m3* are optional arguments, at least one of them must be nonzero.

m2

A Float or a Complex specifying the connector moment component in the connector's local 5-direction.

m3

A Float or a Complex specifying the connector moment component in the connector's local 6-direction.

m1State

A SymbolicConstant specifying the propagation state of the load component in the connector's local 4-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

m2State

A SymbolicConstant specifying the propagation state of the load component in the connector's local 5-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

m3State

A SymbolicConstant specifying the propagation state of the load component in the connector's local 6-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.30.2 Corresponding analysis keywords

*CONNECTOR LOAD (degree of freedom: 4, 5, or 6)

27.31 CoriolisForce object

The CoriolisForce object stores the data for a coriolis force.
The CoriolisForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.31.1 CoriolisForce(...)

This method creates a CoriolisForce object.

Path

```
mdb.models[name].CoriolisForce
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

point1

A sequence of Floats specifying the first point on the axis of rotation for the load.

point2

A sequence of Floats specifying the second point on the axis of rotation for the load.

Optional arguments

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

Return value

A CoriolisForce object.

Exceptions

None.

27.31.2 **setValues(...)**

This method modifies the data for an existing CoriolisForce object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CoriolisForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.31.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing CoriolisForce object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*magnitude*

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value**None****Exceptions**

None.

27.31.4 Members

The CoriolisForce object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

point1

A tuple of Floats specifying the first point on the axis of rotation for the load.

point2

A tuple of Floats specifying the second point on the axis of rotation for the load.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.32 CoriolisForceState object

The CoriolisForceState object stores the propagating data of a coriolis force in a step. One instance of this object is created internally by the CoriolisForce object for each step. The instance is also deleted internally by the CoriolisForce object.

The CoriolisForceState object has no constructor or methods.

The CoriolisForceState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.32.1 Members

The CoriolisForceState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.32.2 Corresponding analysis keywords

*DLOAD (load type label: CORIO)

27.33 Gravity object

The Gravity object stores the data of a gravity load.

The Gravity object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.33.1 Gravity(...)

This method creates a Gravity object.

Path

```
mdb.models[name].Gravity
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

LOAD COMMANDS

comp1

A Float or a Complex specifying the component of the load in the 1-direction.

Note: Although *comp1*, *comp2*, and *comp3* are optional arguments, at least one of them must be nonzero.

comp2

A Float or a Complex specifying the component of the load in the 2-direction.

comp3

A Float or a Complex specifying the component of the load in the 3-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A Gravity object.

Exceptions

None.

27.33.2 setValues(...)

This method modifies the data for an existing Gravity object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Gravity** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.33.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing Gravity object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Float, a Complex, or a SymbolicConstant specifying the load component in the 1-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

comp2

A Float, a Complex, or a SymbolicConstant specifying the load component in the 2-direction. For details see *comp1*.

comp3

A Float, a Complex, or a SymbolicConstant specifying the load component in the 3-direction. For details see *comp1*.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.33.4 **Members**

The Gravity object can have the following members:

name

A String specifying the load repository key.

LOAD COMMANDS

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.34 GravityState object

The GravityState object stores the propagating data for a gravity load in a step. One instance of this object is created internally by the Gravity object for each step. The instance is also deleted internally by the Gravity object.

The GravityState object has no constructor or methods.

The GravityState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.34.1 Members

The GravityState object has the following members:

comp1

A Float or a Complex specifying the load component in the 1-direction.

comp2

A Float or a Complex specifying the load component in the 2-direction.

comp3

A Float or a Complex specifying the load component in the 3-direction.

comp1State

A SymbolicConstant specifying the propagation state of the load component in the 1-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp2State

A SymbolicConstant specifying the propagation state of the load component in the 2-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp3State

A SymbolicConstant specifying the propagation state of the load component in the 3-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.34.2 Corresponding analysis keywords

*DLOAD (load type label:GRAV)

27.35 HydrostaticFluidFlowState object

The HydrostaticFluidFlowState object stores the propagating data for a concentrated HydrostaticFluidFlow object in a step. One instance of this object is created internally by the HydrostaticFluidFlow object for each step. The instance is also deleted internally by the HydrostaticFluidFlow object.

The HydrostaticFluidFlowState object has no constructor or methods.

The HydrostaticFluidFlowState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.35.1 Members

The HydrostaticFluidFlowState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.35.2 Corresponding analysis keywords

*FLUID FLUX

27.36 InertiaRelief object

The InertiaRelief object defines an inertia relief load.

The InertiaRelief object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.36.1 InertiaRelief(...)

This method creates an InertiaRelief object.

Path

```
mdb.models[name].InertiaRelief
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

Optional arguments

u1

A Boolean specifying the 1-direction as a free direction.

Note: Although *u1*, *u2*, *u3*, *ur1*, *ur2*, and *ur3* are optional arguments, at least one of them must be specified. Further, any specified set of free directions cannot include only two rotational degrees of freedom.

u2

A Boolean specifying the 2-direction as a free direction.

u3

A Boolean specifying the 3-direction as a free direction.

ur1

A Boolean specifying the rotation about the 1–direction as a free direction.

ur2

A Boolean specifying the rotation about the 2–direction as a free direction.

LOAD COMMANDS

ur3

A Boolean specifying the rotation about the 3–direction as a free direction.

referencePoint

A sequence of Floats specifying the X, Y and Z-coordinates of a fixed rotation point or a point on the rotation axis or a point on the symmetry line, about which rotations are defined. Such a point must be specified only for certain combinations of free directions.

localCoordinates

None or a DatumCsys object specifying the local coordinate system of the rigid body degrees of freedom for the inertia relief load. If *localCoordinates*=**None**, the free directions are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

Return value

An InertiaRelief object.

Exceptions

None.

27.36.2 **setValues(...)**

This method modifies the data for an existing InertiaRelief object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **InertiaRelief** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.36.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing InertiaRelief object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*u1*

A Boolean specifying the 1-direction as a free direction.

u2

A Boolean specifying the 2-direction as a free direction.

u3

A Boolean specifying the 3-direction as a free direction.

ur1

A Boolean specifying the rotation about the 1–direction as a free direction.

ur2

A Boolean specifying the rotation about the 2–direction as a free direction.

ur3

A Boolean specifying the rotation about the 3–direction as a free direction.

referencePoint

A sequence of Floats specifying the point about which rotations are defined. The point can be specified only for certain combinations of free directions. The *referencePoint* argument can be one of the following:

- The *X*, *Y* and *Z*-coordinates of a fixed rotation point.
- A point on the rotation axis.
- A point on the symmetry line.

fixed

A Boolean specifying whether the inertia relief loading should remain fixed at the current loading at the start of the step. The default value is OFF.

Return value**None****Exceptions**

None.

27.36.4 Members

The InertiaRelief object can have the following members:

LOAD COMMANDS

name

A String specifying the load repository key.

localCoordinates

None or a DatumCsys object specifying the local coordinate system of the rigid body degrees of freedom for the inertia relief load. If *localCoordinates*=**None**, the free directions are defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

region

A Region object specifying the region to which the load is applied.

27.37 InertiaReliefState object

The InertiaReliefState object stores the propagating data for an inertia relief load in a step. One instance of this object is created internally by the InertiaRelief object for each step. The instance is also deleted internally by the InertiaRelief object.

The InertiaReliefState object has no constructor or methods.

The InertiaReliefState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.37.1 Members

The InertiaReliefState object has the following members:

u1

A Boolean specifying the 1-direction as a free direction.

u2

A Boolean specifying the 2-direction as a free direction.

u3

A Boolean specifying the 3-direction as a free direction.

ur1

A Boolean specifying the rotation about the 1-direction as a free direction.

ur2

A Boolean specifying the rotation about the 2-direction as a free direction.

ur3

A Boolean specifying the rotation about the 3-direction as a free direction.

u1State

A SymbolicConstant specifying the propagation state of the Boolean that identifies the local 1-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

u2State

A SymbolicConstant specifying the propagation state of the Boolean that identifies the local 2-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

u3State

A SymbolicConstant specifying the propagation state of the Boolean that identifies the local the 3-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

ur1State

A SymbolicConstant specifying the propagation state of the Boolean that identifies rotation about the local 1-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

ur2State

A SymbolicConstant specifying the propagation state of the Boolean that identifies the rotation about the local the 2-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

ur3State

A SymbolicConstant specifying the propagation state of the Boolean that identifies the rotation about the local the 3-direction as a free direction. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

fixed

A Boolean specifying whether the inertia relief loading should remain fixed at the current loading at the start of the step. The default value is OFF.

fixedState

A SymbolicConstant specifying the propagation state of the Boolean that identifies whether the inertia relief load should remain fixed at current level at the start of the step. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

referencePointState

A SymbolicConstant specifying the propagation state of the reference point of the inertia relief load. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

referencePoint

A tuple of Floats specifying the point about which rotations are defined. The point can be specified only for certain combinations of free directions. The *referencePoint* argument can be one of the following:

- The X, Y and Z-coordinates of a fixed rotation point.
- A point on the rotation axis.
- A point on the symmetry line.

LOAD COMMANDS

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.37.2 Corresponding analysis keywords

*INERTIA RELIEF

27.38 InwardVolAccel object

The InwardVolAccel object stores the data for an inward volume acceleration acoustic load.

The InwardVolAccel object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.38.1 InwardVolAccel(...)

This method creates a InwardVolAccel object.

Path

`mdb.models[name].InwardVolAccel`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the name of the first analysis step.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

An InwardVolAccel object.

Exceptions

None.

27.38.2 setValues(...)

This method modifies the data for an existing InwardVolAccel object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **InwardVolAccel** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.38.3 setValuesInStep(...)

This method modifies the propagating data for an existing **InwardVolAccel** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a **SymbolicConstant** specifying the name of the amplitude reference. Possible values for the **SymbolicConstant** are **UNCHANGED** and **FREED**. **UNCHANGED** should be used if the amplitude is propagated from the previous static analysis step. **FREED** should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.38.4 Members

The **InwardVolAccel** object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.39 InwardVolAccelState object

The InwardVolAccelState object stores the propagating data of an inward volume acceleration acoustic load in a step. One instance of this object is created internally by the InwardVolAccel object for each step. The instance is also deleted internally by the InwardVolAccel object.

The InwardVolAccelState object has no constructor or methods.

The InwardVolAccelState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.39.1 Members

The InwardVolAccelState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED

LOAD COMMANDS

- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.39.2 Corresponding analysis keywords

*CLOAD (degree of freedom: 8)

27.40 LineLoad object

The LineLoad object stores the data of an applied line load.

The LineLoad object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.40.1 LineLoad(...)

This method creates a LineLoad object.

Path

```
mdb.models[name].LineLoad
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

comp1

A Float or a Complex specifying the component of the load in the global or the beam local 1-direction.

Note: Although *comp1*, *comp2*, and *comp3* are optional arguments, at least one of them must be nonzero unless *distributionType*=USER_DEFINED.

comp2

A Float or a Complex specifying the component of the load in the global or the beam local 2-direction.

comp3

A Float or a Complex specifying the component of the load in the global 3-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

system

A SymbolicConstant specifying whether the load is applied in a global or the beam local frame of reference. Possible values are GLOBAL and LOCAL. The default value is GLOBAL.

Return value

A LineLoad object.

Exceptions

None.

27.40.2 setValues(...)

This method modifies the data for an existing LineLoad object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LineLoad** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.40.3 setValuesInStep(...)

This method modifies the propagating data for an existing LineLoad object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Float, a Complex, or a SymbolicConstant specifying the load component in the global or the beam local 1-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

comp2

A Float, a Complex, or a SymbolicConstant specifying the load component in the global or the beam local 2-direction. For details see *comp1*.

comp3

A Float, a Complex, or a SymbolicConstant specifying the load component in the global 3-direction. For details see *comp1*.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load

is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.40.4 Members

The LineLoad object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

system

A SymbolicConstant specifying whether the load is applied in a global or the beam local frame of reference. Possible values are GLOBAL and LOCAL. The default value is GLOBAL.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.41 LineLoadState object

The LineLoadState object stores the propagating data of a line load in a step. One instance of this object is created internally by the LineLoad object for each step. The instance is also deleted internally by the LineLoad object.

The LineLoadState object has no constructor or methods.

The LineLoadState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.41.1 Members

The LineLoadState object has the following members:

comp1

A Float or a Complex specifying the load component in the global or the beam local 1-direction.

comp2

A Float or a Complex specifying the load component in the global or the beam local 2-direction.

comp3

A Float or a Complex specifying the load component in the global 3-direction.

comp1State

A SymbolicConstant specifying the propagation state of the load component in the global or the beam local 1-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp2State

A SymbolicConstant specifying the propagation state of the load component in the global or the beam local 2-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp3State

A SymbolicConstant specifying the propagation state of the load component in the global 3-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.41.2 Corresponding analysis keywords

*DLOAD (load type label: PX, PY, PZ for a global reference frame, and P1, P2 for a local reference frame)

27.42 LoadState object

The LoadState object is the abstract base type for other LoadState objects. The LoadState object has no explicit constructor or methods. The members of the LoadState object are common to all objects derived from LoadState.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.42.1 Members

The LoadState object has the following members:

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.43 Moment object

The Moment object stores the data for a moment.

The Moment object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.43.1 Moment(...)

This method creates a Moment object.

Path

```
mdb.models[name].Moment
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

Optional arguments

cm1

A Float or a Complex specifying the load component in the 4-direction.

Note: Although *comp1*, *comp2*, and *comp3* are optional arguments, at least one of them must be nonzero.

cm2

A Float or a Complex specifying the load component in the 5- direction.

cm3

A Float or a Complex specifying the load component in the 6-direction.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

follower

A Boolean specifying whether the direction of the force rotates with the rotation of the node. You should provide the *follower* argument only if it is valid for the specified step. The default value is OFF.

localCsys

None or a DatumCsys object specifying the ID of the Datum coordinate system used as the local coordinate system of the load. If *localCsys*=**None**, the load is defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

Return value

A Moment object.

Exceptions

None.

27.43.2 setValues(...)

This method modifies the data for an existing Moment object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Moment** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.43.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing Moment object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Float, a Complex, or a SymbolicConstant specifying the load component in the 4-direction. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

comp2

A Float, a Complex, or a SymbolicConstant specifying the load component in the 5-direction. For details see *comp1*.

comp3

A Float, a Complex, or a SymbolicConstant specifying the load component in the 6-direction. For details see *comp1*.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.43.4 **Members**

The Moment object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

follower

A Boolean specifying whether the direction of the force rotates with the rotation of the node. You should provide the *follower* argument only if it is valid for the specified step. The default value is OFF.

localCsys

None or a DatumCsys object specifying the ID of the Datum coordinate system used as the local coordinate system of the load. If *localCsys*=**None**, the load is defined in the global coordinate system. When this member is queried, it returns an Int. The default value is **None**.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.44 MomentState object

The MomentState object stores the propagating data for a moment in a step. One instance of this object is created internally by the Moment object for each step. The instance is also deleted internally by the Moment object.

The MomentState object has no constructor or methods.

The MomentState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.44.1 Members

The MomentState object has the following members:

cm1

A Float or a Complex specifying the load component in the 4-direction.

cm2

A Float or a Complex specifying the load component in the 5-direction.

cm3

A Float or a Complex specifying the load component in the 6-direction.

LOAD COMMANDS

cm1State

A SymbolicConstant specifying the propagation state of the load component in the 4-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

cm2State

A SymbolicConstant specifying the propagation state of the load component in the 5-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

cm3State

A SymbolicConstant specifying the propagation state of the load component in the 6-direction. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.44.2 Corresponding analysis keywords

*CLOAD (degree of freedom: 4, 5, or 6)

27.45 PEGLoad object

The PEGLoad object stores the data for a PEG load.

The PEGLoad object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.45.1 PEGLoad(...)

This method creates a PEGLoad object.

Path

```
mdb.models[name].PEGLoad
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

comp1

A Float or a Complex specifying the load component at dof 1 of reference node 1.

Note: Although *comp1*, *comp2*, and *comp3* are optional arguments, at least one of them must be nonzero.

comp2

A Float or a Complex specifying the load component at dof 1 of reference node 2.

comp3

A Float or a Complex specifying the load component at dof 2 of reference node 2.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A PEGLoad object.

Exceptions

None.

27.45.2 **setValues(...)**

This method modifies the data for an existing PEGLoad object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PEGLoad** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.45.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing PEGLoad object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Float, a Complex, or a SymbolicConstant specifying the load component at dof 1 of reference node 1. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the load component is propagated from the previous static analysis step. Use FREED to remove a previously defined load component.

comp2

A Float, a Complex, or a SymbolicConstant specifying the load component at dof 1 of reference node 2. For details see *comp1*.

comp3

A Float, a Complex, or a SymbolicConstant specifying the load component at dof 2 of reference node 2. For details see *comp1*.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.45.4 Members

The PEGLoad object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.46 PEGLoadState object

The PEGLoadState object stores the propagating data for a concentrated force in a step. One instance of this object is created internally by the PEGLoad object for each step. The instance is also deleted internally by the PEGLoad object.

LOAD COMMANDS

The PEGLoadState object has no constructor or methods.

The PEGLoadState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.46.1 Members

The PEGLoadState object has the following members:

comp1

A Float or a Complex specifying the load component at dof 1 of reference node 1.

comp2

A Float or a Complex specifying the load component at dof 1 of reference node 2.

comp3

A Float or a Complex specifying the load component at dof 2 of reference node 2.

comp1State

A SymbolicConstant specifying the propagation state of the load component at dof 1 of reference node 1. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp2State

A SymbolicConstant specifying the propagation state of the load component at dof 1 of reference node 2. Possible values are UNSET, SET, UNCHANGED, and FREED.

comp3State

A SymbolicConstant specifying the propagation state of the load component at dof 2 of reference node 2. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE

- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.46.2 Corresponding analysis keywords

*CLOAD (degree of freedom: 1 or 2)

27.47 PipePressure object

The PipePressure object stores the data for a pressure applied to pipe or elbow elements.
The PipePressure object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.47.1 PipePressure(...)

This method creates a Pressure object.

Path

```
mdb.models[name].PipePressure
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the pressure is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the pressure magnitude.

Note: *magnitude* is optional if *distributionType*=USER_DEFINED.

LOAD COMMANDS

diameter

A Float specifying the effective inner or outer diameter.

hZero

A Float specifying the height of the zero pressure level when *distributionType*=HYDROSTATIC.

hReference

A Float specifying the height of the reference pressure level when *distributionType*=HYDROSTATIC.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are UNIFORM, HYDROSTATIC, USER_DEFINED, and FIELD. The default value is UNIFORM.

side

A SymbolicConstant specifying whether the pressure is applied internally or externally. Possible values are INTERNAL and EXTERNAL. The default value is INTERNAL.

Return value

A PipePressure object.

Exceptions

None.

27.47.2 setValues(...)

This method modifies the data for an existing PipePressure object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PipePressure** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.47.3 setValuesInStep(...)

This method modifies the propagating data for an existing PipePressure object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*magnitude*

A Float specifying the pressure magnitude.

hZero

A Float specifying the height of the zero pressure level when *distributionType*=HYDROSTATIC.

hReference

A Float specifying the height of the reference pressure level when *distributionType*=HYDROSTATIC.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.47.4 Members

The PipePressure object can have the following members:

LOAD COMMANDS

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are UNIFORM, HYDROSTATIC, USER_DEFINED, and FIELD. The default value is UNIFORM.

side

A SymbolicConstant specifying whether the pressure is applied internally or externally. Possible values are INTERNAL and EXTERNAL. The default value is INTERNAL.

diameter

A Float specifying the effective inner or outer diameter.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.48 PipePressureState object

The PipePressureState object stores the propagating data for a pipe pressure in a step. One instance of this object is created internally by the PipePressure object for each step. The instance is also deleted internally by the PipePressure object.

The PipePressureState object has no constructor or methods.

The PipePressureState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.48.1 Members

The PipePressureState object has the following members:

magnitude

A Float or a Complex specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

hZero

A Float specifying the height of the zero pressure level when the pipe pressure *distributionType*=HYDROSTATIC.

hZeroState

A SymbolicConstant specifying the propagation state of *hZero*. Possible values are UNSET, SET, UNCHANGED, and FREED.

hReference

A Float specifying the height of the reference pressure level when the pipe pressure *distributionType*=HYDROSTATIC.

hReferenceState

A SymbolicConstant specifying the propagation state of *hReference*. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.48.2 Corresponding analysis keywords

*DSLOAD
*DLOAD

27.49 PorDragBodyForce object

The PorDragBodyForce object stores the data for a fluid pore drag body force load.

The PorDragBodyForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.49.1 PorDragBodyForce(...)

This method creates a PorDragBodyForce object.

Path

```
mdb.models[name].PorDragBodyForce
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region specifying the point at which the pore drag body force value is applied.

fieldName

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

magnitude

A Float specifying the pore drag body force value.

Optional arguments

None.

Return value

A PorDragBodyForce object.

Exceptions

None.

27.49.2 setValues(...)

This method modifies the data for an existing PorDragBodyForce object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PorDragBodyForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.49.3 setValuesInStep(...)

This method modifies the propagating data for an existing PorDragBodyForce object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the load is modified.

Optional argument

magnitude

A Float specifying the pore drag body force value.

Return value

None

Exceptions

None.

27.49.4 Members

The PorDragBodyForce object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the boundary condition is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

fieldName

A String specifying The name of the AnalyticalField or DiscreteField object associated with this boundary condition. The *fieldName* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.50 PorDragBodyForceState object

The PorDragBodyForceState object stores the propagating data for a Body PorDragBodyForce object in a step. One instance of this object is created internally by the PorDragBodyForce object for each step. The instance is also deleted internally by the PorDragBodyForce object.

The PorDragBodyForceState object has no constructor or methods.

The PorDragBodyForceState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.50.1 Members

The PorDragBodyForceState object has the following members:

magnitude

A Float specifying the Pore drag body force magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the Pore drag body force magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.50.2 Corresponding analysis keywords

*DFLUX

27.51 Pressure object

The Pressure object defines a pressure load.

The Pressure object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.51.1 Pressure(...)

This method creates a Pressure object.

Path

`mdb.models[name].Pressure`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the pressure is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float or a Complex specifying the pressure magnitude.

Note: *magnitude* is optional if *distributionType*=USER_DEFINED.

hZero

A Float specifying the height of the zero pressure level when *distributionType*=HYDROSTATIC.

hReference

A Float specifying the height of the reference pressure level when *distributionType*=HYDROSTATIC.

Optional arguments

field

A String specifying the name of the AnalyticalField or DiscreteField object associated with this load. The *field* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

refPoint

A Region specifying the reference point from which the relative velocity is determined when *distributionType*=STAGNATION or VISCOUS.

distributionType

A SymbolicConstant specifying how the pressure is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, HYDROSTATIC, STAGNATION, VISCOUS, TOTAL_FORCE, and DISCRETE_FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A Pressure object.

Exceptions

None.

27.51.2 setValues(...)

This method modifies the data for an existing Pressure object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Pressure** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.51.3 setValuesInStep(...)

This method modifies the propagating data for an existing Pressure object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

LOAD COMMANDS

Optional arguments

magnitude

A Float or a Complex specifying the pressure magnitude.

hZero

A Float specifying the height of the zero pressure level when *distributionType*=HYDROSTATIC.

hReference

A Float specifying the height of the reference pressure level when *distributionType*=HYDROSTATIC.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.51.4 Members

The Pressure object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the pressure is distributed spatially. Possible values are UNIFORM, USER_DEFINED, FIELD, HYDROSTATIC, STAGNATION, VISCOUS, TOTAL_FORCE, and DISCRETE_FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField or DiscreteField object associated with this load. The *field* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.52 PressureState object

The PressureState object stores the propagating data for a pressure in a step. One instance of this object is created internally by the Pressure object for each step. The instance is also deleted internally by the Pressure object.

The PressureState object has no constructor or methods.

The PressureState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.52.1 Members

The PressureState object has the following members:

magnitude

A Float or a Complex specifying the pressure magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the pressure magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

hZero

A Float specifying the height of the zero pressure level when the pressure *distributionType*=HYDROSTATIC.

hZeroState

A SymbolicConstant specifying the propagation state of *hZero*. Possible values are UNSET, SET, UNCHANGED, and FREED.

hReference

A Float specifying the height of the reference pressure level when the pressure *distributionType*=HYDROSTATIC.

hReferenceState

A SymbolicConstant specifying the propagation state of *hReference*. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE

LOAD COMMANDS

- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.52.2 Corresponding analysis keywords

*DSLOAD
*DLOAD

27.53 ReferencePressure object

The ReferencePressure object stores the data for a fluid reference pressure load.
The ReferencePressure object is derived from the Load object.

Access

```
import load  
mdb.models[name].loads[name]
```

27.53.1 ReferencePressure(...)

This method creates a ReferencePressure object.

Path

```
mdb.models[name].ReferencePressure
```


Required arguments*name*

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region specifying the point at which the reference pressure value is applied.

magnitude

A Float specifying the reference pressure value.

Optional arguments

None.

Return value

A ReferencePressure object.

Exceptions

None.

27.53.2 setValues(...)

This method modifies the data for an existing ReferencePressure object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ReferencePressure** method, except for the *name* and *createStepName* arguments.

Return value**None****Exceptions**

None.

27.53.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing ReferencePressure object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional argument

magnitude

A Float specifying the reference pressure value.

Return value

None

Exceptions

None.

27.53.4 **Members**

The ReferencePressure object can have the following members:

name

A String specifying the load repository key.

region

A Region object specifying the region to which the load is applied.

27.54 **RotationalBodyForce object**

The RotationalBodyForce object stores the data for a rotational body force.

The RotationalBodyForce object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.54.1 RotationalBodyForce(...)

This method creates a RotationalBodyForce object.

Path

`mdb.models[name].RotationalBodyForce`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

point1

A sequence of Floats specifying the first point on the axis of rotation for the load.

point2

A sequence of Floats specifying the second point on the axis of rotation for the load.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

centrifugal

A Boolean specifying whether or not the effect of the load is centrifugal. The default value is OFF.

Note: At least one of *centrifugal* or *rotaryAcceleration* must be specified and only one must have the value ON.

rotaryAcceleration

A Boolean specifying whether or not the effect of the load is rotary acceleration. The default value is OFF.

LOAD COMMANDS

Note: At least one of *centrifugal* or *rotaryAcceleration* must be specified and only one must have the value ON.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A RotationalBodyForce object.

Exceptions

None.

27.54.2 setValues(...)

This method modifies the data for an existing RotationalBodyForce object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RotationalBodyForce** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.54.3 setValuesInStep(...)

This method modifies the propagating data for an existing RotationalBodyForce object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the load is modified.

Optional arguments*magnitude*

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value**None****Exceptions**

None.

27.54.4 Members

The RotationalBodyForce object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

centrifugal

A Boolean specifying whether or not the effect of the load is centrifugal. The default value is OFF.

Note: At least one of *centrifugal* or *rotaryAcceleration* must be specified and only one must have the value ON.

rotaryAcceleration

A Boolean specifying whether or not the effect of the load is rotary acceleration. The default value is OFF.

Note: At least one of *centrifugal* or *rotaryAcceleration* must be specified and only one must have the value ON.

point1

A tuple of Floats specifying the first point on the axis of rotation for the load.

point2

A tuple of Floats specifying the second point on the axis of rotation for the load.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.55 RotationalBodyForceState object

The RotationalBodyForceState object stores the propagating data of a rotational body force in a step. One instance of this object is created internally by the RotationalBodyForce object for each step. The instance is also deleted internally by the RotationalBodyForce object.

The RotationalBodyForceState object has no constructor or methods.

The RotationalBodyForceState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.55.1 Members

The RotationalBodyForceState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE

- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.55.2 Corresponding analysis keywords

*DLOAD (load type labels: CENTRIF, ROTA)

27.56 ShellEdgeLoad object

The ShellEdgeLoad object defines shell edge loads on a region.
The ShellEdgeLoad object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.56.1 ShellEdgeLoad(...)

This method creates a ShellEdgeLoad object.

Path

```
mdb.models[name].ShellEdgeLoad
```

Required arguments

name

A String specifying the load repository key.

LOAD COMMANDS

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float or Complex specifying the load magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED

Optional arguments

distributionType

A SymbolicConstant specifying how the shell edge load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

angle

A Float specifying an additional rotation of *directionVector* about an axis. The default value is 0. This parameter is available only if *traction* is GENERAL.

axis

A SymbolicConstant specifying the axis about which to apply an additional rotation of *directionVector*. Possible values are AXIS_1, AXIS_2, AXIS_3. The default value is AXIS_1.
This parameter is available only if *traction* is GENERAL.

localCsys

A DatumCsys object specifying the local coordinate system of the load's degrees of freedom. The default value is **None**, indicating that the degrees of freedom are defined in the global coordinate system or by the *userCsys* parameter if defined. This parameter is available only if *traction* is GENERAL. When this member is queried, it returns an Int.

userCsys

A String specifying a CSYS defined by a user-subroutine. The default value is **None**, indicating that the degrees of freedom are defined in the global coordinate system or by the *localCsys* parameter if defined. This parameter is available only if *traction* is GENERAL.

directionVector

A tuple of two points specifying the direction of the load. Each point is specified as a point region or a tuple of coordinates. If *traction* is SHEAR, then *directionVector* will be projected onto the region surface. This parameter is available only if *traction* is GENERAL.

follower

A Boolean specifying whether the direction of the force changes with rotation. The default value is ON. This parameter may be modified only if *traction* is GENERAL. You should provide the *follower* argument only if it is valid for the specified step.

resultant

A Boolean specifying whether to maintain a constant resultant force by defining traction per unit undeformed area. If *resultant* is OFF, traction is defined per unit deformed area. The default value is OFF. You should provide the *resultant* argument only if it is valid for the specified step.

traction

A SymbolicConstant specifying how to apply surface traction. Possible values are NORMAL, TRANSVERSE, SHEAR, MOMENT and GENERAL. The default value is NORMAL.

Return value

A ShellEdgeLoad object.

Exceptions

None.

27.56.2 setValues(...)

This method modifies the data for an existing ShellEdgeLoad object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShellEdgeLoad** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.56.3 setValuesInStep(...)

This method modifies the propagating data for an existing ShellEdgeLoad object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the surface pore fluid flow is modified.

Optional arguments

magnitude

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the load magnitude. UNCHANGED should be used if the magnitude is propagated from the previous analysis step.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.56.4 Members

The ShellEdgeLoad object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the shell edge load is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.57 ShellEdgeLoadState object

The ShellEdgeLoadState object stores the propagating data for a ShellEdgeLoad object in a step. One instance of this object is created internally by the ShellEdgeLoad object for each step. The instance is also deleted internally by the ShellEdgeLoad object.

The ShellEdgeLoadState object has no constructor or methods.

The ShellEdgeLoadState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.57.1 Members

The ShellEdgeLoadState object has the following members:

magnitude

A Float or a Complex specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.57.2 Corresponding analysis keywords

*DSLOAD
*DLOAD

27.58 SubmodelSB object

The SubmodelSB object stores the data for a submodel surface based load.
The SubmodelSB object is derived from the Load object.

Access

```
import load  
mdb.models[name].loads[name]
```

27.58.1 SubmodelSB(...)

This method creates a SubmodelSB object.

Path

```
mdb.models[name].SubmodelSB
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*='1' indicates the first step.

Optional arguments*globalDrivingRegion*

A String specifying the element set in the global model that will be searched for elements whose responses will be used to drive the submodel. An empty string indicates that the entire global model will be searched. The default value is an empty string.

absoluteExteriorTolerance

None or a Float specifying the absolute value by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is **None**.

exteriorTolerance

None or a Float specifying the fraction of the average element size in the global model by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is 0.05.

globalIncrement

An Int specifying the increment number in the global model step from which the solution will be used to specify the values of the driven variables. If *globalIncrement*=0, the solution from the last increment will be used. The *globalIncrement* argument is applicable only for linear perturbation steps. The default value is 0.

Return value

A SubmodelSB object.

Exceptions

None.

27.58.2 setValues(...)

This method modifies the data for an existing SubmodelSB object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SubmodelSB** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.58.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing SubmodelSB object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

fixed

A Boolean specifying whether the load should remain fixed at the current values at the start of the step. The default value is ON.

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*='1' indicates the first step. The *globalStep* argument is applicable only if *fixed*=OFF.

globalIncrement

An Int specifying the increment number in the global model step at which the solution will be used to specify the values of the driven variables. If *globalIncrement*=0, the solution from the last increment will be used. The *globalIncrement* argument is applicable only for linear perturbation steps and if *fixed*=OFF. The default value is 0.

Return value

None

Exceptions

None.

27.58.4 **Members**

The SubmodelSB object can have the following members:

name

A String specifying the load repository key.

absoluteExteriorTolerance

None or a Float specifying the absolute value by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is **None**.

exteriorTolerance

None or a Float specifying the fraction of the average element size in the global model by which a driven node of the submodel can lie outside the region of the elements of the global model. The default value is 0.05.

globalDrivingRegion

A String specifying the element set in the global model that will be searched for elements whose responses will be used to drive the submodel. An empty string indicates that the entire global model will be searched. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.59 SubmodelSBState object

The SubmodelSBState object stores the propagating data for a Submodel load in a step. One instance of this object is created internally by the SubmodelSB object for each step. The instance is also deleted internally by the SubmodelSB object.

The SubmodelSBState object has no constructor or methods.

The SubmodelSBState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.59.1 Members

The SubmodelSBState object has the following members:

globalStepState

A SymbolicConstant specifying the propagation state of the *globalStep* member. Possible values are SET and UNCHANGED.

globalIncrement

An Int specifying the increment number in the global model step at which the solution will be used to specify the values of the driven variables. This argument is applicable only for linear perturbation steps.

globalIncrementState

A SymbolicConstant specifying the propagation state of the *globalIncrement* member. Possible values are SET and UNCHANGED.

LOAD COMMANDS

globalStep

A String specifying the step in the global model from which Abaqus reads the values of the variables that will drive the submodel analysis. The String indicates the position of the step in the sequence of analysis steps. For example, *globalStep*=‘1’ indicates the first step.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.59.2 Corresponding analysis keywords

*SUBMODEL
*DSLOAD, SUBMODEL

27.60 SubstructureLoad object

The SubstructureLoad object defines a substructure load.

The SubstructureLoad object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```


27.60.1 SubstructureLoad(...)

This method creates a SubstructureLoad object.

Path

`mdb.models[name].SubstructureLoad`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the substructure load is created.

region

A Region object specifying the region to which the load is applied.

loadCaseNames

A list of names of the load cases that should be activated by this substructure load.

magnitude

A Float specifying the multiplier for the load case magnitude.

Optional argument

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SubstructureLoad object.

Exceptions

None.

27.60.2 setValues(...)

This method modifies the data for an existing SubstructureLoad object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SubstructureLoad** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.60.3 setValuesInStep(...)

This method modifies the propagating data for an existing **SubstructureLoad** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

loadCaseNames

A list of names of the load cases that should be activated by this substructure load.

magnitude

A Float specifying the multiplier for the load case magnitude.

amplitude

A String or a **SymbolicConstant** specifying the name of the amplitude reference. Possible values for the **SymbolicConstant** are **UNCHANGED** and **FREED**. **UNCHANGED** should be used if the amplitude is propagated from the previous analysis step. **FREED** should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.60.4 Members

The **SubstructureLoad** object can have the following members:

name

A String specifying the load repository key.

region

A Region object specifying the region to which the load is applied.

27.61 SubstructureLoadState object

The SubstructureLoadState object stores the propagating data for a substructure load in a step. One instance of this object is created internally by the SubstructureLoad object for each step. The instance is also deleted internally by the SubstructureLoad object.

The SubstructureLoadState object has no constructor or methods.

The SubstructureLoadState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.61.1 Members

The SubstructureLoadState object has the following members:

loadCaseNames

A tuple of strings specifying the names of the load cases to be activated.

magnitude

A Float or a Complex specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE

LOAD COMMANDS

- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.61.2 Corresponding analysis keywords

*SLOAD

27.62 SurfaceCharge object

The SurfaceCharge object stores the data for a surface charge.

The SurfaceCharge object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.62.1 SurfaceCharge(...)

This method creates a SurfaceCharge object.

Path

```
mdb.models[name].SurfaceCharge
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SurfaceCharge object.

Exceptions

None.

27.62.2 **setValues(...)**

This method modifies the data for an existing SurfaceCharge object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceCharge** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.62.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing SurfaceCharge object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.62.4 **Members**

The SurfaceCharge object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.63 SurfaceChargeState object

The SurfaceChargeState object stores the propagating data of a surface charge in a step. One instance of this object is created internally by the SurfaceCharge object for each step. The instance is also deleted internally by the SurfaceCharge object.

The SurfaceChargeState object has no constructor or methods.

The SurfaceChargeState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.63.1 Members

The SurfaceChargeState object has the following members:

magnitude

A Float specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.63.2 Corresponding analysis keywords

*DSECHARGE (load type label: ES)

27.64 SurfaceConcentrationFlux object

The SurfaceConcentrationFlux object defines surface concentration flux from a region or into a region.
The SurfaceConcentrationFlux object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.64.1 SurfaceConcentrationFlux(...)

This method creates a SurfaceConcentrationFlux object.

Path

```
mdb.models[name].SurfaceConcentrationFlux
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the surface concentration flux magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the surface concentration flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SurfaceConcentrationFlux object.

Exceptions

None.

27.64.2 setValues(...)

This method modifies the data for an existing SurfaceConcentrationFlux object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceConcentrationFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.64.3 setValuesInStep(...)

This method modifies the propagating data for an existing SurfaceConcentrationFlux object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the surface concentration flux is modified.

Optional arguments

magnitude

A Float specifying the surface concentration flux magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.64.4 Members

The SurfaceConcentrationFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the surface concentration flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.65 SurfaceConcentrationFluxState object

The SurfaceConcentrationFluxState object stores the propagating data for a SurfaceConcentrationFlux object in a step. One instance of this object is created internally by the SurfaceConcentrationFlux object for each step. The instance is also deleted internally by the SurfaceConcentrationFlux object.

The SurfaceConcentrationFluxState object has no constructor or methods.

The SurfaceConcentrationFluxState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.65.1 Members

The SurfaceConcentrationFluxState object has the following members:

magnitude

A Float specifying the surface concentration flux magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the surface concentration flux magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.65.2 Corresponding analysis keywords

*DSFLUX

27.66 SurfaceCurrent object

The SurfaceCurrent object stores the data for a surface current.
The SurfaceCurrent object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.66.1 SurfaceCurrent(...)

This method creates a SurfaceCurrent object.

Path

```
mdb.models[name].SurfaceCurrent
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the load magnitude.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SurfaceCurrent object.

Exceptions

None.

27.66.2 **setValues(...)**

This method modifies the data for an existing SurfaceCurrent object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceCurrent** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.66.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing SurfaceCurrent object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

magnitude

A Float specifying the load magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous static analysis step. FREED should be used if the load is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.66.4 Members

The SurfaceCurrent object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.67 SurfaceCurrentDensity object

The SurfaceCurrentDensity object stores the data for a surface current.

The SurfaceCurrentDensity object is derived from the Load object.

Access

```
import load
```

```
mdb.models[name].loads[name]
```

27.67.1 SurfaceCurrentDensity(...)

This method creates a SurfaceCurrentDensity object.

Path

`mdb.models[name].SurfaceCurrentDensity`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created. This must be the first analysis step name.

region

A Region object specifying the region to which the load is applied.

comp1

A Complex specifying the first component of the load.

comp2

A Complex specifying the second component of the load.

comp3

A Complex specifying the third component of the load.

Optional arguments

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SurfaceCurrentDensity object.

Exceptions

None.

27.67.2 **setValues(...)**

This method modifies the data for an existing `SurfaceCurrentDensity` object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the `SurfaceCurrentDensity` method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.67.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing `SurfaceCurrentDensity` object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments

comp1

A Complex specifying the first component of the load.

comp2

A Complex specifying the second component of the load.

comp3

A Complex specifying the third component of the load.

amplitude

A String or a `SymbolicConstant` specifying the name of the amplitude reference. Possible values for the `SymbolicConstant` are `UNCHANGED` and `FREED`. `UNCHANGED` should be used if the amplitude is propagated from the previous static analysis step. `FREED` should be used if the load

is changed to have no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.67.4 Members

The SurfaceCurrentDensity object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the load is distributed spatially. Possible values are UNIFORM and USER_DEFINED. The default value is UNIFORM.

region

A Region object specifying the region to which the load is applied.

27.68 SurfaceCurrentState object

The SurfaceCurrentState object stores the propagating data of a surface current in a step. One instance of this object is created internally by the SurfaceCurrent object for each step. The instance is also deleted internally by the SurfaceCurrent object.

The SurfaceCurrentState object has no constructor or methods.

The SurfaceCurrentState object is derived from the LoadState object.

Access

```
import load
mdb.models[name].steps[name].loadStates[name]
```

27.68.1 Members

The SurfaceCurrentState object has the following members:

magnitude

A Float specifying the load magnitude.

LOAD COMMANDS

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and FREED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.68.2 Corresponding analysis keywords

*DSECURRENT (load type label: CS)

27.69 SurfaceHeatFlux object

The SurfaceHeatFlux object defines surface heat flux from a region or into a region.

The SurfaceHeatFlux object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.69.1 SurfaceHeatFlux(...)

This method creates a SurfaceHeatFlux object.

Path

`mdb.models [name] .SurfaceHeatFlux`

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float specifying the surface heat flux magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

distributionType

A SymbolicConstant specifying how the surface heat flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

A SurfaceHeatFlux object.

Exceptions

None.

27.69.2 setValues(...)

This method modifies the data for an existing SurfaceHeatFlux object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceHeatFlux** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.69.3 setValuesInStep(...)

This method modifies the propagating data for an existing **SurfaceHeatFlux** object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the surface heat flux is modified.

Optional arguments

magnitude

A Float specifying the surface heat flux magnitude.

amplitude

A String or a **SymbolicConstant** specifying the name of the amplitude reference. Possible values for the **SymbolicConstant** are **UNCHANGED** and **FREED**. **UNCHANGED** should be used if the amplitude is propagated from the previous analysis step. **FREED** should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.69.4 Members

The SurfaceHeatFlux object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying how the surface heat flux is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.70 SurfaceHeatFluxState object

The SurfaceHeatFluxState object stores the propagating data for a surface SurfaceHeatFlux object in a step. One instance of this object is created internally by the SurfaceHeatFlux object for each step. The instance is also deleted internally by the SurfaceHeatFlux object.

The SurfaceHeatFluxState object has no constructor or methods.

The SurfaceHeatFluxState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.70.1 Members

The SurfaceHeatFluxState object has the following members:

magnitude

A Float specifying the surface heat flux magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the surface heat flux magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

LOAD COMMANDS

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.70.2 Corresponding analysis keywords

*DSFLUX

27.71 SurfacePoreFluid object

The SurfacePoreFluid object defines surface pore fluid flow from a region or into a region.

The SurfacePoreFluid object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.71.1 SurfacePoreFluid(...)

This method creates a SurfacePoreFluid object.

Path

```
mdb.models[name].SurfacePoreFluid
```

Required arguments*name*

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

*magnitude*A Float specifying the surface pore fluid flow magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.**Optional arguments***field*A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.*distributionType*

A SymbolicConstant specifying whether the load is uniform. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

*amplitude*A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.**Return value**

A SurfacePoreFluid object.

Exceptions

None.

27.71.2 setValues(...)

This method modifies the data for an existing SurfacePoreFluid object in the step where it is created.

Arguments**Required arguments**

None.

Optional argumentsThe optional arguments to **setValues** are the same as the arguments to the **SurfacePoreFluid** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.71.3 setValuesInStep(...)

This method modifies the propagating data for an existing SurfacePoreFluid object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the surface pore fluid flow is modified.

Optional arguments

magnitude

A Float specifying the surface pore fluid flow magnitude.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.71.4 Members

The SurfacePoreFluid object can have the following members:

name

A String specifying the load repository key.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

region

A Region object specifying the region to which the load is applied.

27.72 SurfacePoreFluidState object

The SurfacePoreFluidState object stores the propagating data for a SurfacePoreFluid object in a step. One instance of this object is created internally by the SurfacePoreFluid object for each step. The instance is also deleted internally by the SurfacePoreFluid object.

The SurfacePoreFluidState object has no constructor or methods.

The SurfacePoreFluidState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.72.1 Members

The SurfacePoreFluidState object has the following members:

magnitude

A Float specifying the surface pore fluid flow magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the surface pore fluid flow magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE

- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.72.2 Corresponding analysis keywords

*DSFLOW

27.73 SurfaceTraction object

The SurfaceTraction object defines surface traction on a region.

The SurfaceTraction object is derived from the Load object.

Access

```
import load
mdb.models[name].loads[name]
```

27.73.1 SurfaceTraction(...)

This method creates a SurfaceTraction object.

Path

```
mdb.models[name].SurfaceTraction
```

Required arguments

name

A String specifying the load repository key.

createStepName

A String specifying the name of the step in which the load is created.

region

A Region object specifying the region to which the load is applied.

magnitude

A Float or Complex specifying the load magnitude. *magnitude* is optional if *distributionType*=USER_DEFINED.

Optional arguments*distributionType*

A SymbolicConstant specifying how the surface traction is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the load has no amplitude reference. The default value is UNSET. You should provide the *amplitude* argument only if it is valid for the specified step.

angle

A Float specifying an additional rotation of *directionVector* about an axis. The default value is 0.0.

axis

A SymbolicConstant specifying the axis about which to apply an additional rotation of *directionVector*. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

localCsys

None or a DatumCsys object specifying the local coordinate system of the load's degrees of freedom. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system or by the *userCsys* parameter if defined. When this member is queried, it returns an Int. The default value is **None**.

userCsys

A String specifying a CSYS defined by a user-subroutine. If *userCsys*=**None**, the degrees of freedom are defined in the global coordinate system or by the *localCsys* parameter if defined. The default value is "None".

directionVector

A VertexArray object of length 2 specifying the direction of the load. Instead of through a Vertex, each point may be specified through a tuple of coordinates. If *traction* is SHEAR, then *directionVector* will be projected onto the region surface. This parameter is available only if *traction* is GENERAL or SHEAR.

follower

A Boolean specifying whether the direction of the force changes with rotation. The default value is ON.

This parameter may be modified only if *traction* is GENERAL. You should provide the *follower* argument only if it is valid for the specified step.

resultant

A Boolean specifying whether to maintain a constant resultant force by defining traction per unit undeformed area. If *resultant* is OFF, traction is defined per unit deformed area. The default value is OFF.

LOAD COMMANDS

You should provide the *resultant* argument only if it is valid for the specified step.

traction

A SymbolicConstant specifying how to apply surface traction. Possible values are SHEAR and GENERAL. The default value is SHEAR.

Return value

A SurfaceTraction object.

Exceptions

None.

27.73.2 **setValues(...)**

This method modifies the data for an existing SurfaceTraction object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SurfaceTraction** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

27.73.3 **setValuesInStep(...)**

This method modifies the propagating data for an existing SurfaceTraction object in the specified step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the load is modified.

Optional arguments*magnitude*

A Float, a Complex, or the SymbolicConstant UNCHANGED specifying the load magnitude. UNCHANGED should be used if the magnitude is propagated from the previous analysis step.

amplitude

A String or a SymbolicConstant specifying the name of the amplitude reference. Possible values for the SymbolicConstant are UNCHANGED and FREED. UNCHANGED should be used if the amplitude is propagated from the previous analysis step. FREED should be used if the load has no amplitude reference. You should provide the *amplitude* argument only if it is valid for the specified step.

Return value

None

Exceptions

None.

27.73.4 Members

The SurfaceTraction object can have the following members:

name

A String specifying the load repository key.

angle

A Float specifying an additional rotation of *directionVector* about an axis. The default value is 0.0.

axis

A SymbolicConstant specifying the axis about which to apply an additional rotation of *directionVector*. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

follower

A Boolean specifying whether the direction of the force changes with rotation. The default value is ON.

This parameter may be modified only if *traction* is GENERAL. You should provide the *follower* argument only if it is valid for the specified step.

resultant

A Boolean specifying whether to maintain a constant resultant force by defining traction per unit undeformed area. If *resultant* is OFF, traction is defined per unit deformed area. The default value is OFF.

You should provide the *resultant* argument only if it is valid for the specified step.

LOAD COMMANDS

traction

A SymbolicConstant specifying how to apply surface traction. Possible values are SHEAR and GENERAL. The default value is SHEAR.

distributionType

A SymbolicConstant specifying how the surface traction is distributed spatially. Possible values are UNIFORM, USER_DEFINED, and FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField object associated with this load. The *field* argument applies only when *distributionType*=FIELD. The default value is an empty string.

userCsys

A String specifying a CSYS defined by a user-subroutine. If *userCsys*=None, the degrees of freedom are defined in the global coordinate system or by the *localCsys* parameter if defined. The default value is "None".

localCsys

None or a DatumCsys object specifying the local coordinate system of the load's degrees of freedom. If *localCsys*=None, the degrees of freedom are defined in the global coordinate system or by the *userCsys* parameter if defined. When this member is queried, it returns an Int. The default value is None.

directionVector

A VertexArray object of length 2 specifying the direction of the load. Instead of through a Vertex , each point may be specified through a tuple of coordinates. If *traction* is SHEAR, then *directionVector* will be projected onto the region surface. This parameter is available only if *traction* is GENERAL or SHEAR.

region

A Region object specifying the region to which the load is applied.

27.74 SurfaceTractionState object

The SurfaceTractionState object stores the propagating data for a SurfaceTraction object in a step. One instance of this object is created internally by the SurfaceTraction object for each step. The instance is also deleted internally by the SurfaceTraction object.

The SurfaceTractionState object has no constructor or methods.

The SurfaceTractionState object is derived from the LoadState object.

Access

```
import load
```

```
mdb.models[name].steps[name].loadStates[name]
```

27.74.1 Members

The SurfaceTractionState object has the following members:

magnitude

A Float or a Complex specifying the load magnitude.

magnitudeState

A SymbolicConstant specifying the propagation state of the load magnitude. Possible values are UNSET, SET, UNCHANGED, and MODIFIED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitude* member. Possible values are UNSET, SET, UNCHANGED, and FREED.

status

A SymbolicConstant specifying the propagation state of the LoadState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- NO_LONGER_ACTIVE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE
- BUILT_INTO_BASE_STATE

amplitude

A String specifying the name of the amplitude reference. The String is empty if the load has no amplitude reference.

27.74.2 Corresponding analysis keywords

*DSLOAD

*DLOAD

28. Load Case commands

Load Case commands are used for configuring load cases in specific types of steps that may use them.

28.1 LoadCase object

The LoadCase object is used to define the loads and constraints comprising a particular loading condition and the linear response of a structure subjected to that loading condition.

Access

```
import step
mdb.models[name].steps[name].loadCases[name]
```

28.1.1 LoadCase(...)

This method creates a load case in a step.

Path

```
mdb.models[name].steps[name].LoadCase
```

Required argument

name

A String specifying the name of the object.

Optional arguments

boundaryConditions

A sequence of (String, Float) sequences specifying the name of a BoundaryCondition followed by a nonzero Float scaling factor. The default value is an empty sequence.

loads

A sequence of (String, Float) sequences specifying the name of a Load followed by a nonzero Float specifying a scale factor. The default value is an empty sequence.

includeActiveBaseStateBC

A Boolean specifying whether to include all active boundary conditions propagated or modified from the base state. The default value is ON.

Return value

A LoadCase object.

Exceptions

RangeError.

28.1.2 resume()

This method resumes the load case that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

28.1.3 suppress()

This method suppresses the load case.

Arguments

None.

Return value

None

Exceptions

None.

28.1.4 setValues(...)

This method modifies the LoadCase object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LoadCase** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

28.1.5 Members

The LoadCase object has members with the same names and descriptions as the arguments to the **LoadCase** method.

In addition, the LoadCase object has the following member:

suppressed

A Boolean specifying whether the load case is suppressed or not. The default value is OFF.

29. Material commands

The Material commands are used to define the materials in a model.

29.1 Material object

A Material object is the object used to specify a material. The Material object stores the various settings that determine how a material behaves.

A material is created by combining one or more individual material options and sub options. A particular material option is associated with the Material object through a member. For example: the *acousticMedium* member may contain an *AcousticMedium* object. The alternative of having a *MaterialOption* abstract base class and a container of *MaterialOptions* was rejected because it would make it more difficult to enforce the fact that one Material object cannot contain two *AcousticMedium* objects, for example.

Access

```
import material
mdb.models[name].materials[name]
import odbMaterial
session.odbs[name].materials[name]
```

29.1.1 Material(...)

This method creates a Material object.

Path

```
mdb.models[name].Material
session.odbs[name].Material
```

Required argument

name

A String specifying the name of the new material.

Optional arguments

description

A String specifying user description of the material. The default value is an empty string.

materialIdentifier

A String specifying material identifier for customer use. The default value is an empty string.

Return value

A Material object.

Exceptions

InvalidNameError.

29.1.2 materialsFromOdb(...)

This methods creates Material objects by reading an output database. The new materials are placed in the **materials** repository.

Path

`mdb.models[name].materialsFromOdb`

Required argument

fileName

A String specifying the name of the output database file (including the **.odb** extension) to be read.

This String can also be the full path to the output database file if it is located in another directory.

Optional arguments

None.

Return value

A list of Material objects.

Exceptions

None.

29.1.3 Members

The Material object has members with the same names and descriptions as the arguments to the **Material** method.

In addition, the Material object can have the following members:

acousticMedium

An AcousticMedium object.

brittleCracking

A BrittleCracking object.

capPlasticity

A CapPlasticity object.

castIronPlasticity
A CastIronPlasticity object.

clayPlasticity
A ClayPlasticity object.

concrete
A Concrete object.

concreteDamagedPlasticity
A ConcreteDamagedPlasticity object.

conductivity
A Conductivity object.

creep
A Creep object.

crushableFoam
A CrushableFoam object.

ductileDamageInitiation
A DamageInitiation object.

fldDamageInitiation
A DamageInitiation object.

flsdDamageInitiation
A DamageInitiation object.

johnsonCookDamageInitiation
A DamageInitiation object.

maxeDamageInitiation
A DamageInitiation object.

maxsDamageInitiation
A DamageInitiation object.

maxpeDamageInitiation
A DamageInitiation object.

maxpsDamageInitiation
A DamageInitiation object.

mkDamageInitiation
A DamageInitiation object.

msfldDamageInitiation
A DamageInitiation object.

quadeDamageInitiation
A DamageInitiation object.

quadsDamageInitiation
A DamageInitiation object.

MATERIAL COMMANDS

shearDamageInitiation

A DamageInitiation object.

hashinDamageInitiation

A DamageInitiation object.

damping

A Damping object.

deformationPlasticity

A DeformationPlasticity object.

density

A Density object.

depvar

A Depvar object.

dielectric

A Dielectric object.

diffusivity

A Diffusivity object.

druckerPrager

A DruckerPrager object.

elastic

An Elastic object.

electricalConductivity

An ElectricalConductivity object.

eos

An Eos object.

expansion

An Expansion object.

fluidLeakoff

A FluidLeakoff object.

gapFlow

A GapFlow object.

gasketThicknessBehavior

A GasketThicknessBehavior object.

gasketTransverseShearElastic

A GasketTransverseShearElastic object.

gasketMembraneElastic

A GasketMembraneElastic object.

gel

A Gel object.

heatGeneration
A HeatGeneration object.

hyperelastic
A Hyperelastic object.

hyperfoam
A Hyperfoam object.

hypoelastic
A Hypoelastic object.

inelasticHeatFraction
An InelasticHeatFraction object.

jouleHeatFraction
A JouleHeatFraction object.

latentHeat
A LatentHeat object.

lowDensityFoam
A LowDensityFoam object.

magneticPermeability
A MagneticPermeability object.

mohrCoulombPlasticity
A MohrCoulombPlasticity object.

moistureSwelling
A MoistureSwelling object.

mullinsEffect
A MullinsEffect object.

permeability
A Permeability object.

piezoelectric
A Piezoelectric object.

plastic
A Plastic object.

poreFluidExpansion
A PoreFluidExpansion object.

porousBulkModuli
A PorousBulkModuli object.

porousElastic
A PorousElastic object.

porousMetalPlasticity
A PorousMetalPlasticity object.

MATERIAL COMMANDS

regularization

A Regularization object.

solubility

A Solubility object.

sorption

A Sorption object.

specificHeat

A SpecificHeat object.

swelling

A Swelling object.

userDefinedField

A UserDefinedField object.

userMaterial

A UserMaterial object.

userOutputVariables

A UserOutputVariables object.

viscoelastic

A Viscoelastic object.

viscosity

A Viscosity object.

viscous

A Viscous object.

29.1.4 Corresponding analysis keywords

*MATERIAL

29.2 AcousticMedium object

The AcousticMedium object specifies the acoustic properties of a material.

Access

```
import material
mdb.models[name].materials[name].acousticMedium
import odbMaterial
session.odbs[name].materials[name].acousticMedium
```

29.2.1 AcousticMedium(...)

This method creates an `AcousticMedium` object.

Path

```
mdb.models[name].materials[name].AcousticMedium
session.odbs[name].materials[name].AcousticMedium
```

Required arguments

None.

Optional arguments

acousticVolumetricDrag

A Boolean specifying whether the *volumetricTable* data is specified. The default value is OFF.

temperatureDependencyB

A Boolean specifying whether the data in *bulkTable* depend on temperature. The default value is OFF.

temperatureDependencyV

A Boolean specifying whether the data in *volumetricTable* depend on temperature. The default value is OFF.

dependenciesB

An Int specifying the number of field variable dependencies for the data in *bulkTable*. The default value is 0.

dependenciesV

An Int specifying the number of field variable dependencies for the data in *volumetricTable*. The default value is 0.

bulkTable

A sequence of sequences of Floats specifying the following:

- Bulk modulus.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The default value is an empty sequence.

volumetricTable

A sequence of sequences of Floats specifying the following:

- Volumetric drag.
- Frequency.

MATERIAL COMMANDS

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The default value is an empty sequence.

Return value

An `AcousticMedium` object.

Exceptions

`RangeError`.

29.2.2 `setValues(...)`

This method modifies the `AcousticMedium` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `AcousticMedium` method.

Return value

`None`

Exceptions

`RangeError`.

29.2.3 Members

The `AcousticMedium` object has members with the same names and descriptions as the arguments to the `AcousticMedium` method.

29.2.4 Corresponding analysis keywords

*ACOUSTIC MEDIUM

29.3 AnnealTemperature object

The AnnealTemperature object specifies the material annealing temperature.

Access

```
import material
mdb.models[name].materials[name].plastic.annealTemperature
import odbMaterial
session.odbs[name].materials[name].plastic.annealTemperature
```

29.3.1 AnnealTemperature(...)

This method creates an AnnealTemperature object.

Path

```
mdb.models[name].materials[name].plastic.AnnealTemperature
session.odbs[name].materials[name].plastic.AnnealTemperature
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional argument

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The annealing temperature, θ .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

An AnnealTemperature object.

Exceptions

RangeError.

29.3.2 **setValues(...)**

This method modifies the `AnnealTemperature` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `AnnealTemperature` method.

Return value

None

Exceptions

RangeError.

29.3.3 **Members**

The `AnnealTemperature` object has members with the same names and descriptions as the arguments to the `AnnealTemperature` method.

29.3.4 **Corresponding analysis keywords**

*ANNEAL TEMPERATURE

29.4 **BiaxialTestData object**

The `BiaxialTestData` object provides equibiaxial test data (compression and/or tension).

Access

`import material`

```

mdb.models[name].materials[name].hyperelastic.biaxialTestData
mdb.models[name].materials[name].hyperfoam.biaxialTestData
mdb.models[name].materials[name].mullinsEffect.biaxialTests[i]
import odbMaterial
session.odbs[name].materials[name].hyperelastic.biaxialTestData
session.odbs[name].materials[name].hyperfoam.biaxialTestData
session.odbs[name].materials[name].mullinsEffect.biaxialTests[i]

```

29.4.1 BiaxialTestData(...)

This method creates a BiaxialTestData object.

Path

```

mdb.models[name].materials[name].hyperelastic.BiaxialTestData
mdb.models[name].materials[name].hyperfoam.BiaxialTestData
mdb.models[name].materials[name].mullinsEffect.BiaxialTestData
session.odbs[name].materials[name].hyperelastic.BiaxialTestData
session.odbs[name].materials[name].hyperfoam.BiaxialTestData
session.odbs[name].materials[name].mullinsEffect.BiaxialTestData

```

Required argument

table

A sequence of sequences of Floats specifying the following:

- Nominal stress, T_B .
- Nominal strain, ϵ_B .

Optional arguments

smoothing

None or an Int specifying the value for smoothing. If *smoothing*=**None**, no smoothing is employed. The default value is **None**.

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Return value

A `BiaxialTestData` object.

Exceptions

None.

29.4.2 `setValues(...)`

This method modifies the `BiaxialTestData` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BiaxialTestData` method.

Return value

None

Exceptions

None.

29.4.3 Members

The `BiaxialTestData` object has members with the same names and descriptions as the arguments to the `BiaxialTestData` method.

29.4.4 Corresponding analysis keywords

*BIAXIAL TEST DATA

29.5 BrittleCracking object

The BrittleCracking object specifies cracking and postcracking properties for the brittle cracking material model.

Access

```
import material
mdb.models[name].materials[name].brittleCracking
import odbMaterial
session.odbs[name].materials[name].brittleCracking
```

29.5.1 BrittleCracking(...)

This method creates a BrittleCracking object.

Path

```
mdb.models[name].materials[name].BrittleCracking
session.odbs[name].materials[name].BrittleCracking
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

A SymbolicConstant specifying the type of postcracking behavior. Possible values are STRAIN, DISPLACEMENT, and GFI. The default value is STRAIN.

Table data

If *type*=STRAIN the table data specify the following:

- Remaining direct stress after cracking.
- Direct cracking strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

MATERIAL COMMANDS

- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT the table data specify the following:

- Remaining direct stress after cracking.
- Direct cracking displacement.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=GFI the table data specify the following:

- Failure stress.
- Mode I fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleCracking object.

Exceptions

None.

29.5.2 **setValues(...)**

This method modifies the BrittleCracking object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BrittleCracking** method.

Return value

None

Exceptions

None.

29.5.3 Members

The BrittleCracking object has members with the same names and descriptions as the arguments to the **BrittleCracking** method.

In addition, the BrittleCracking object can have the following members:

brittleShear

A BrittleShear object.

brittleFailure

A BrittleFailure object.

29.5.4 Corresponding analysis keywords

*BRITTLE CRACKING

29.6 BrittleFailure object

The BrittleFailure object specifies the brittle failure of the material.

Access

```
import material
mdb.models[name].materials[name].brittleCracking.brittleFailure
import odbMaterial
session.odbs[name].materials[name].brittleCracking.brittleFailure
```

29.6.1 BrittleFailure(...)

This method creates a BrittleFailure object.

Path

```
mdb.models[name].materials[name].brittleCracking.BrittleFailure
session.odbs[name].materials[name].brittleCracking.BrittleFailure
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

failureCriteria

A SymbolicConstant specifying the failure criteria. Possible values are UNIDIRECTIONAL, BIDIRECTIONAL, and TRIDIRECTIONAL. The default value is UNIDIRECTIONAL.

Table data

If parent BrittleCracking member *type*=STRAIN the table data specify the following:

- Direct cracking failure strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If parent BrittleCracking member *type*=DISPLACEMENT or *type*=GFI the table data specify the following:

- Direct cracking failure displacement.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleFailure object.

Exceptions

RangeError.

29.6.2 **setValues(...)**

This method modifies the BrittleFailure object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BrittleFailure` method.

Return value

None

Exceptions

`RangeError`.

29.6.3 Members

The `BrittleFailure` object has members with the same names and descriptions as the arguments to the `BrittleFailure` method.

29.6.4 Corresponding analysis keywords

*BRITTLE FAILURE

29.7 BrittleShear object

The `BrittleShear` object specifies the postcracking shear behavior of a material used in a brittle cracking model.

Access

```
import material
mdb.models[name].materials[name].brittleCracking.brittleShear
import odbMaterial
session.odbs[name].materials[name].brittleCracking.brittleShear
```

29.7.1 BrittleShear(...)

This method creates a `BrittleShear` object.

MATERIAL COMMANDS

Path

`mdb.models[name].materials[name].brittleCracking.BrittleShear`
`session.odbs[name].materials[name].brittleCracking.BrittleShear`

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

A SymbolicConstant specifying the type of postcracking shear behavior. Possible values are RETENTION_FACTOR and POWER_LAW. The default value is RETENTION_FACTOR.

Table data

If *type*=RETENTION_FACTOR the table data specify the following:

- Shear retention factor.
- Crack opening strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=POWER_LAW the table data specify the following:

- e.
- p.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleShear object.

Exceptions

RangeError.

29.7.2 setValues(...)

This method modifies the BrittleShear object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BrittleShear** method.

Return value

None

Exceptions

RangeError.

29.7.3 Members

The BrittleShear object has members with the same names and descriptions as the arguments to the **BrittleShear** method.

29.7.4 Corresponding analysis keywords

*BRITTLE SHEAR

29.8 CapCreepCohesion object

The CapCreepCohesion object specifies a cap creep model and material properties.

Access

```
import material
mdb.models[name].materials[name].capPlasticity.capCreepCohesion
```

```
import odbMaterial
session.odbs[name].materials[name].capPlasticity.capCreepCohesion
```

29.8.1 CapCreepCohesion(...)

This method creates a CapCreepCohesion object.

Path

```
mdb.models[name].materials[name].capPlasticity.CapCreepCohesion
session.odbs[name].materials[name].capPlasticity.CapCreepCohesion
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the cap creep law. Possible values are STRAIN, TIME, SINGHM, and USER. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=SINGHM, the table data specify the following:

- *A*.
- α .
- *m*.
- t_1 .
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapCreepCohesion object.

Exceptions

None.

29.8.2 setValues(...)

This method modifies the CapCreepCohesion object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CapCreepCohesion** method.

Return value

None

Exceptions

None.

29.8.3 Members

The CapCreepCohesion object has members with the same names and descriptions as the arguments to the **CapCreepCohesion** method.

29.8.4 Corresponding analysis keywords

*CAP CREEP

29.9 CapCreepConsolidation object

The CapCreepConsolidation object specifies a cap creep model and material properties.

Access

```
import material
mdb.models[name].materials[name].capPlasticity.capCreepConsolidation
import odbMaterial
session.odbs[name].materials[name].capPlasticity.capCreepConsolidation
```

29.9.1 CapCreepConsolidation(...)

This method creates a CapCreepConsolidation object.

Path

```
mdb.models[name].materials[name].capPlasticity.CapCreepConsolidation
session.odbs[name].materials[name].capPlasticity.CapCreepConsolidation
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the cap creep law. Possible values are STRAIN, TIME, SINGHM, and USER. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

- Value of the second field variable.
- Etc.

If $law=SINGHM$, the table data specify the following:

- A .
- α .
- m .
- t_1 .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapCreepConsolidation object.

Exceptions

None.

29.9.2 **setValues(...)**

This method modifies the CapCreepConsolidation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CapCreepConsolidation** method.

Return value

None

Exceptions

None.

29.9.3 Members

The CapCreepConsolidation object has members with the same names and descriptions as the arguments to the `CapCreepConsolidation` method.

29.9.4 Corresponding analysis keywords

*CAP CREEP

29.10 CapHardening object

The CapHardening object specifies Drucker-Prager/Cap plasticity hardening.

Access

```
import material
mdb.models[name].materials[name].capPlasticity.capHardening
import odbMaterial
session.odbs[name].materials[name].capPlasticity.capHardening
```

29.10.1 CapHardening(...)

This method creates a CapHardening object.

Path

```
mdb.models[name].materials[name].capPlasticity.CapHardening
session.odbs[name].materials[name].capPlasticity.CapHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Hydrostatic pressure yield stress.
- Absolute value of the corresponding volumetric inelastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapHardening object.

Exceptions

RangeError.

29.10.2 setValues(...)

This method modifies the CapHardening object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CapHardening** method.

Return value

None

Exceptions

RangeError.

29.10.3 Members

The CapHardening object has members with the same names and descriptions as the arguments to the **CapHardening** method.

29.10.4 Corresponding analysis keywords

*CAP HARDENING

29.11 CapPlasticity object

The CapPlasticity object specifies the modified Drucker-Prager/Cap plasticity model.

Access

```
import material
mdb.models[name].materials[name].capPlasticity
import odbMaterial
session.odbs[name].materials[name].capPlasticity
```

29.11.1 CapPlasticity(...)

This method creates a CapPlasticity object.

Path

```
mdb.models[name].materials[name].CapPlasticity
session.odbs[name].materials[name].CapPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Material cohesion, d , in the p - t plane (Abaqus/Standard) or in the p - q plane (Abaqus/Explicit).
- Material angle of friction, β , in the p - t plane (Abaqus/Standard) or in the p - q plane (Abaqus/Explicit). Give the value in degrees.
- Cap eccentricity parameter, R . Its value must be greater than zero (typically $0.0 < R < 1.0$).
- Initial cap yield surface position, $\varepsilon_{vol}^{pl}|_0$.
- Transition surface radius parameter, α . The default value is 0.0 (i.e., no transition surface).

- (Not used in Abaqus/Explicit) K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. Possible values are $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, Abaqus/Standard assumes $K = 1.0$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapPlasticity object.

Exceptions

RangeError.

29.11.2 setValues(...)

This method modifies the CapPlasticity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CapPlasticity** method.

Return value

None

Exceptions

RangeError.

29.11.3 Members

The CapPlasticity object has members with the same names and descriptions as the arguments to the **CapPlasticity** method.

In addition, the CapPlasticity object can have the following members:

capCreepCohesion

A CapCreepCohesion object.

MATERIAL COMMANDS

capCreepConsolidation

A CapCreepConsolidation object.

capHardening

A CapHardening object.

29.11.4 Corresponding analysis keywords

*CAP PLASTICITY

29.12 CastIronCompressionHardening object

The CastIronCompressionHardening object specifies hardening for the Cast- Iron plasticity model.

Access

```
import material
mdb.models[name].materials[name].castIronPlasticity\
.castIronCompressionHardening
import odbMaterial
session.odbs[name].materials[name].castIronPlasticity\
.castIronCompressionHardening
```

29.12.1 CastIronCompressionHardening(...)

This method creates a CastIronCompressionHardening object.

Path

```
mdb.models[name].materials[name].castIronPlasticity\
.CastIronCompressionHardening
session.odbs[name].materials[name].castIronPlasticity\
.CastIronCompressionHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in compression, σ_c .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronCompressionHardening object.

Exceptions

RangeError.

29.12.2 setValues(...)

This method modifies the CastIronCompressionHardening object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CastIronCompressionHardening** method.

Return value

None

Exceptions

RangeError.

29.12.3 Members

The `CastIronCompressionHardening` object has members with the same names and descriptions as the arguments to the `CastIronCompressionHardening` method.

29.12.4 Corresponding analysis keywords

*CAST IRON COMPRESSION HARDENING

29.13 CastIronPlasticity object

The `CastIronPlasticity` object specifies the Cast Iron plasticity model.

Access

```
import material
mdb.models[name].materials[name].castIronPlasticity
import odbMaterial
session.odbs[name].materials[name].castIronPlasticity
```

29.13.1 CastIronPlasticity(...)

This method creates a `CastIronPlasticity` object.

Path

```
mdb.models[name].materials[name].CastIronPlasticity
session.odbs[name].materials[name].CastIronPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

The table data specify the following:

- Plastic Poisson's ratio, ν_{pl} (dimensionless).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronPlasticity object.

Exceptions

RangeError.

29.13.2 setValues(...)

This method modifies the CastIronPlasticity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CastIronPlasticity** method.

Return value

None

Exceptions

RangeError.

29.13.3 Members

The CastIronPlasticity object has members with the same names and descriptions as the arguments to the **CastIronPlasticity** method.

In addition, the CastIronPlasticity object can have the following members:

castIronTensionHardening

A CastIronTensionHardening object.

castIronCompressionHardening

A CastIronCompressionHardening object.

29.13.4 Corresponding analysis keywords

*CAST IRON PLASTICITY

29.14 CastIronTensionHardening object

The CastIronTensionHardening object specifies hardening for the Cast- Iron plasticity model.

Access

```
import material
mdb.models[name].materials[name].castIronPlasticity\
.castIronTensionHardening
import odbMaterial
session.odbs[name].materials[name].castIronPlasticity\
.castIronTensionHardening
```

29.14.1 CastIronTensionHardening(...)

This method creates a CastIronTensionHardening object.

Path

```
mdb.models[name].materials[name].castIronPlasticity\
.CastIronTensionHardening
session.odbs[name].materials[name].castIronPlasticity\
.CastIronTensionHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in uniaxial tension, σ_t .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronTensionHardening object.

Exceptions

RangeError.

29.14.2 setValues(...)

This method modifies the CastIronTensionHardening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CastIronTensionHardening** method.

Return value

None

Exceptions

RangeError.

29.14.3 Members

The `CastIronTensionHardening` object has members with the same names and descriptions as the arguments to the `CastIronTensionHardening` method.

29.14.4 Corresponding analysis keywords

*CAST IRON TENSION HARDENING

29.15 ClayHardening object

The `ClayHardening` object specifies hardening for the clay plasticity model.

Access

```
import material
mdb.models[name].materials[name].clayPlasticity.clayHardening
import odbMaterial
session.odbs[name].materials[name].clayPlasticity.clayHardening
```

29.15.1 ClayHardening(...)

This method creates a `ClayHardening` object.

Path

```
mdb.models[name].materials[name].clayPlasticity.ClayHardening
session.odbs[name].materials[name].clayPlasticity.ClayHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The hydrostatic pressure stress at yield, p_c .
- The absolute value of the corresponding volumetric plastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ClayHardening object.

Exceptions

RangeError.

29.15.2 setValues(...)

This method modifies the ClayHardening object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ClayHardening** method.

Return value

None

Exceptions

RangeError.

29.15.3 Members

The ClayHardening object has members with the same names and descriptions as the arguments to the **ClayHardening** method.

29.15.4 Corresponding analysis keywords

*CLAY HARDENING

29.16 ClayPlasticity object

The ClayPlasticity object specifies the extended Cam-clay plasticity model.

Access

```
import material
mdb.models[name].materials[name].clayPlasticity
import odbMaterial
session.odbs[name].materials[name].clayPlasticity
```

29.16.1 ClayPlasticity(...)

This method creates a ClayPlasticity object.

Path

```
mdb.models[name].materials[name].ClayPlasticity
session.odbs[name].materials[name].ClayPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

intercept

None or a Float specifying e_1 , the intercept of the virgin consolidation line with the void ratio axis in a plot of void ratio versus the logarithm of pressure stress. The default value is **None**.

This argument is valid only if *hardening*=EXPONENTIAL.

hardening

A SymbolicConstant specifying the type of hardening/softening definition. Possible values are EXPONENTIAL and TABULAR. The default value is EXPONENTIAL.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=EXPONENTIAL, the table data specify the following:

- Logarithmic plastic bulk modulus, λ (dimensionless).
- Stress ratio at critical state, M .
- The initial yield surface size, a_0 .
- β , the parameter defining the size of the yield surface on the “wet” side of critical state.
- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, a value of 1.0 is assumed.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=TABULAR, the table data specify the following:

- Stress ratio at critical state, M .
- The initial volumetric plastic strain, $\varepsilon_{vol}^{pl}|_0$, corresponding to $p_c|_0$ according to the ClayHardening definition.
- β , the parameter defining the size of the yield surface on the “wet” side of critical state.
- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ClayPlasticity object.

Exceptions

RangeError.

29.16.2 setValues(...)

This method modifies the ClayPlasticity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ClayPlasticity** method.

Return value

None

Exceptions

RangeError.

29.16.3 Members

The **ClayPlasticity** object has members with the same names and descriptions as the arguments to the **ClayPlasticity** method.

In addition, the **ClayPlasticity** object can have the following member:

clayHardening

A **ClayHardening** object.

29.16.4 Corresponding analysis keywords

*CLAY PLASTICITY

29.17 CombinedTestData object

The **CombinedTestData** object specifies simultaneously the normalized shear and bulk compliances or relaxation moduli as functions of time.

Access

```
import material
mdb.models[name].materials[name].viscoelastic.combinedTestData
import odbMaterial
session.odbs[name].materials[name].viscoelastic.combinedTestData
```

29.17.1 CombinedTestData(...)

This method creates a **CombinedTestData** object.

Path

```

mdb.models[name].materials[name].viscoelastic.CombinedTestData
session.odbs[name].materials[name].viscoelastic.CombinedTestData

```

Required argument*table*

A sequence of sequences of Floats specifying the items described below. The values of the table data depend on the value of the *time* member of the Viscoelastic object.

Optional arguments*volinf*

None or a Float specifying a normalized volume. The value of *volinf* depends on the value of the *time* member of the Viscoelastic object. The default value is **None**.

If *time*=RELAXATION_TEST_DATA, *volinf* specifies the value of the long-term normalized volumetric modulus, $k_R(\infty)$.

If *time*=CREEP_TEST_DATA, *volinf* specifies the value of the long-term normalized volumetric compliance, $j_K(\infty)$.

shrinf

None or a Float specifying a normalized shear. The value of *shrinf* depends on the value of the *time* member of the Viscoelastic object. The default value is **None**.

If *time*=RELAXATION_TEST_DATA, *shrinf* specifies the value of the long-term normalized shear modulus, $g_R(\infty)$.

If *time*=CREEP_TEST_DATA, *shrinf* specifies the value of the long-term normalized shear compliance, $j_S(\infty)$.

Table data

If *time*=RELAXATION_TEST_DATA, the table data specify the following:

- Normalized shear modulus, $g_R(t)$ ($0 \leq g_R(t) \leq 1$).
- Normalized volumetric (bulk) modulus, $k_R(t)$ ($0 \leq k_R(t) \leq 1$).
- Time t ($t > 0$).

If *time*=CREEP_TEST_DATA, the table data specify the following:

- Normalized shear compliance, $j_S(t)$ ($j_S(t) \geq 1$).
- Normalized volumetric (bulk) compliance, $j_K(t)$ ($j_K(t) \geq 1$).
- Time t ($t > 0$).

Return value

A CombinedTestData object.

Exceptions

None.

29.17.2 **setValues(...)**

This method modifies the `CombinedTestData` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `CombinedTestData` method.

Return value

None

Exceptions

None.

29.17.3 **Members**

The `CombinedTestData` object has members with the same names and descriptions as the arguments to the `CombinedTestData` method.

29.17.4 **Corresponding analysis keywords**

*COMBINED TEST DATA

29.18 **Concrete object**

The Concrete object defines concrete properties beyond the elastic range.

Access

`import material`

```

mdb.models[name].materials[name].concrete
import odbMaterial
session.odbs[name].materials[name].concrete

```

29.18.1 Concrete(...)

This method creates a Concrete object.

Path

```

mdb.models[name].materials[name].Concrete
session.odbs[name].materials[name].Concrete

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Absolute value of compressive stress.
- Absolute value of plastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Concrete object.

Exceptions

RangeError.

29.18.2 setValues(...)

This method modifies the Concrete object.

MATERIAL COMMANDS

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Concrete** method.

Return value

None

Exceptions

RangeError.

29.18.3 Members

The Concrete object has members with the same names and descriptions as the arguments to the **Concrete** method.

In addition, the Concrete object can have the following members:

failureRatios

A FailureRatios object.

shearRetention

A ShearRetention object.

tensionStiffening

A TensionStiffening object.

29.18.4 Corresponding analysis keywords

*CONCRETE

29.19 ConcreteCompressionDamage object

The ConcreteCompressionDamage object specifies hardening for the concrete damaged plasticity model.

Access

```
import material
mdb.models[name].materials[name].concreteDamagedPlasticity\
```

```
.concreteCompressionDamage
import odbMaterial
session.odbs[name].materials[name].concreteDamagedPlasticity\
.concreteCompressionDamage
```

29.19.1 ConcreteCompressionDamage(...)

This method creates a ConcreteCompressionDamage object.

Path

```
mdb.models[name].materials[name].concreteDamagedPlasticity\
.ConcreteCompressionDamage
session.odbs[name].materials[name].concreteDamagedPlasticity\
.ConcreteCompressionDamage
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

tensionRecovery

A Float specifying the value of the stiffness recovery factor, w_t , that determines the amount of tension stiffness that is recovered as loading changes from compression to tension. The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Compressive damage variable, d_c .
- Inelastic (crushing) strain, ϵ_c^{in} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteCompressionDamage object.

Exceptions

RangeError.

29.19.2 **setValues(...)**

This method modifies the ConcreteCompressionDamage object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcreteCompressionDamage** method.

Return value

None

Exceptions

RangeError.

29.19.3 **Members**

The ConcreteCompressionDamage object has members with the same names and descriptions as the arguments to the **ConcreteCompressionDamage** method.

29.19.4 **Corresponding analysis keywords**

*CONCRETE COMPRESSION DAMAGE

29.20 **ConcreteCompressionHardening object**

The ConcreteCompressionHardening object specifies hardening for the concrete damaged plasticity model.

Access

```
import material
mdb.models[name].materials[name].concreteDamagedPlasticity\
.concreteCompressionHardening
import odbMaterial
session.odbs[name].materials[name].concreteDamagedPlasticity\
.concreteCompressionHardening
```

29.20.1 ConcreteCompressionHardening(...)

This method creates a ConcreteCompressionHardening object.

Path

```
mdb.models[name].materials[name].concreteDamagedPlasticity\
.ConcreteCompressionHardening
session.odbs[name].materials[name].concreteDamagedPlasticity\
.ConcreteCompressionHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

rate

A Boolean specifying whether the data depend on rate. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in compression, σ_c .
- Inelastic (crushing) strain, ϵ_c^{in} .
- Inelastic (crushing) strain rate, $\dot{\epsilon}_c^{in}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteCompressionHardening object.

Exceptions

RangeError.

29.20.2 setValues(...)

This method modifies the ConcreteCompressionHardening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcreteCompressionHardening** method.

Return value

None

Exceptions

RangeError.

29.20.3 Members

The ConcreteCompressionHardening object has members with the same names and descriptions as the arguments to the **ConcreteCompressionHardening** method.

29.20.4 Corresponding analysis keywords

*CONCRETE COMPRESSION HARDENING

29.21 ConcreteDamagedPlasticity object

The ConcreteDamagedPlasticity object specifies the concrete damaged plasticity model.

Access

```
import material
mdb.models[name].materials[name].concreteDamagedPlasticity
import odbMaterial
session.odbs[name].materials[name].concreteDamagedPlasticity
```

29.21.1 ConcreteDamagedPlasticity(...)

This method creates a ConcreteDamagedPlasticity object.

Path

```
mdb.models[name].materials[name].ConcreteDamagedPlasticity
session.odbs[name].materials[name].ConcreteDamagedPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

The table data specify the following:

- Dilation angle, ψ (in degrees) in the p - q plane.
- Flow potential eccentricity, ϵ . The default value is 0.1.
- σ_{b0}/σ_{t0} , the ratio of initial equibiaxial compressive yield stress to initial uniaxial compressive yield stress. The default value is 1.16.
- K_c , the ratio of the second stress invariant on the tensile meridian, to that on the compressive meridian, at initial yield for any given value of the pressure invariant p such that the maximum principal stress is negative. The default value is 2/3.
- Viscosity parameter, μ , used for the viscoplastic regularization of the concrete constitutive equations in an Abaqus/Standard analysis. This parameter is ignored in an Abaqus/Explicit analysis. The default value is 0.0.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

MATERIAL COMMANDS

- Etc.

Return value

A `ConcreteDamagedPlasticity` object.

Exceptions

`RangeError`.

29.21.2 `setValues(...)`

This method modifies the `ConcreteDamagedPlasticity` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConcreteDamagedPlasticity` method.

Return value

`None`

Exceptions

`RangeError`.

29.21.3 Members

The `ConcreteDamagedPlasticity` object has members with the same names and descriptions as the arguments to the `ConcreteDamagedPlasticity` method.

In addition, the `ConcreteDamagedPlasticity` object can have the following members:

concreteCompressionHardening

A `ConcreteCompressionHardening` object.

concreteTensionStiffening

A `ConcreteTensionStiffening` object.

concreteCompressionDamage

A `ConcreteCompressionDamage` object.

concreteTensionDamage

A `ConcreteTensionDamage` object.

29.21.4 Corresponding analysis keywords

*CONCRETE DAMAGED PLASTICITY

29.22 ConcreteTensionDamage object

The ConcreteTensionDamage object specifies hardening for the concrete damaged plasticity model.

Access

```
import material
mdb.models[name].materials[name].concreteDamagedPlasticity\
.concreteTensionDamage
import odbMaterial
session.odbs[name].materials[name].concreteDamagedPlasticity\
.concreteTensionDamage
```

29.22.1 ConcreteTensionDamage(...)

This method creates a ConcreteTensionDamage object.

Path

```
mdb.models[name].materials[name].concreteDamagedPlasticity\
.ConcreteTensionDamage
session.odbs[name].materials[name].concreteDamagedPlasticity\
.ConcreteTensionDamage
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

compressionRecovery

A Float specifying the value of the stiffness recovery factor, w_c , that determines the amount of compression stiffness that is recovered as loading changes from tension to compression. The default value is 1.0.

MATERIAL COMMANDS

type

A SymbolicConstant specifying the type of tensile damage data. Set *type*=STRAIN to specify the tensile damage variable as a function of cracking strain. Set *type*=DISPLACEMENT to specify the tensile damage variable as a function of cracking displacement. Possible values are STRAIN and DISPLACEMENT. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Tensile damage variable, d_t .
- Direct cracking strain, ϵ_t^{ck} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Tensile damage variable, d_t .
- Direct cracking displacement, u_t^{ck} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteTensionDamage object.

Exceptions

RangeError.

29.22.2 setValues(...)

This method modifies the ConcreteTensionDamage object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConcreteTensionDamage` method.

Return value

None

Exceptions

RangeError.

29.22.3 Members

The `ConcreteTensionDamage` object has members with the same names and descriptions as the arguments to the `ConcreteTensionDamage` method.

29.22.4 Corresponding analysis keywords

*CONCRETE TENSION DAMAGE

29.23 ConcreteTensionStiffening object

The `ConcreteTensionStiffening` object specifies hardening for the concrete damaged plasticity model.

Access

```
import material
mdb.models[name].materials[name].concreteDamagedPlasticity\
.concreteTensionStiffening
import odbMaterial
session.odbs[name].materials[name].concreteDamagedPlasticity\
.concreteTensionStiffening
```

29.23.1 ConcreteTensionStiffening(...)

This method creates a ConcreteTensionStiffening object.

Path

```
mdb.models[name].materials[name].concreteDamagedPlasticity\  
.ConcreteTensionStiffening  
session.odbs[name].materials[name].concreteDamagedPlasticity\  
.ConcreteTensionStiffening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

rate

A Boolean specifying whether the data depend on rate. The default value is OFF.

type

A SymbolicConstant specifying the type of postcracking behavior data. Possible values are:

- STRAIN, specifying postfailure stress as a function of cracking strain.
- DISPLACEMENT, specifying postfailure stress as a function of cracking displacement.
- GFI, specifying failure stress as a function of the fracture energy.

The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Remaining direct stress after cracking, σ_t .
- Direct cracking strain, ϵ_t^{ck} .
- Direct cracking strain rate, $\dot{\epsilon}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Remaining direct stress after cracking, σ_t .
- Direct cracking displacement, u_t^{ck} .
- Direct cracking displacement rate, $\dot{u}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=GFI, the table data specify the following:

- Failure stress, σ_{t0} .
- Fracture energy, G_f .
- Direct cracking displacement rate, $\dot{u}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteTensionStiffening object.

Exceptions

RangeError.

29.23.2 setValues(...)

This method modifies the ConcreteTensionStiffening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConcreteTensionStiffening** method.

Return value

None

Exceptions

RangeError.

29.23.3 Members

The ConcreteTensionStiffening object has members with the same names and descriptions as the arguments to the **ConcreteTensionStiffening** method.

29.23.4 Corresponding analysis keywords

*CONCRETE TENSION STIFFENING

29.24 Conductivity object

The Conductivity object specifies thermal conductivity.

Access

```
import material
mdb.models[name].materials[name].conductivity
import odbMaterial
session.odbs[name].materials[name].conductivity
```

29.24.1 Conductivity(...)

This method creates a Conductivity object.

Path

```
mdb.models[name].materials[name].Conductivity
session.odbs[name].materials[name].Conductivity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments*type*

A SymbolicConstant specifying the type of conductivity. Possible values are ISOTROPIC, ORTHOTROPIC, and ANISOTROPIC. The default value is ISOTROPIC.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Conductivity, k .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- k_{11} .
- k_{22} .
- k_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- k_{11} .
- k_{12} .
- k_{22} .
- k_{13} .
- k_{23} .
- k_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Conductivity object.

Exceptions

RangeError.

29.24.2 setValues(...)

This method modifies the Conductivity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Conductivity** method.

Return value

None

Exceptions

RangeError.

29.24.3 Members

The Conductivity object has members with the same names and descriptions as the arguments to the **Conductivity** method.

29.24.4 Corresponding analysis keywords

*CONDUCTIVITY

29.25 ContactArea object

A ContactArea object specifies a suboption of gasket thickness behavior when *variableUnits*=FORCE on the GasketThicknessBehavior object. The ContactArea object defines the contact area or contact width versus closure curves to output an average pressure through variable CS11.

Access

```
import material
mdb.models[name].materials[name].gasketThicknessBehavior.contactArea
import odbMaterial
session.odbs[name].materials[name].gasketThicknessBehavior.contactArea
```

29.25.1 ContactArea(...)

This method creates a ContactArea object.

Path

```
mdb.models[name].materials[name].gasketThicknessBehavior.ContactArea
session.odbs[name].materials[name].gasketThicknessBehavior.ContactArea
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether contact area data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies included in the definition of the contact area data, in addition to temperature. The default value is 0.

Table data

- Contact area or width; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

MATERIAL COMMANDS

- Etc.

Return value

A **ContactArea** object.

Exceptions

None.

29.25.2 setValues(...)

This method modifies the **ContactArea** object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ContactArea** method.

Return value

None

Exceptions

None.

29.25.3 Members

The **ContactArea** object has members with the same names and descriptions as the arguments to the **ContactArea** method.

29.25.4 Corresponding analysis keywords

*GASKET CONTACT AREA

29.26 Creep object

The Creep object defines a creep law.

Access

```
import material
mdb.models[name].materials[name].creep
import odbMaterial
session.odbs[name].materials[name].creep
```

29.26.1 Creep(...)

This method creates a Creep object.

Path

```
mdb.models[name].materials[name].Creep
session.odbs[name].materials[name].Creep
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the strain-hardening law. Possible values are STRAIN, TIME, HYPERBOLIC_SINE, and USER. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=HYPERBOLIC_SINE, the table data specify the following:

- *A*.

MATERIAL COMMANDS

- B .
- n .
- ΔH , if the data depend on temperature.
- R .
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Creep object.

Exceptions

RangeError.

29.26.2 **setValues(...)**

This method modifies the Creep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Creep** method.

Return value

None

Exceptions

RangeError.

29.26.3 **Members**

The Creep object has members with the same names and descriptions as the arguments to the **Creep** method.

In addition, the Creep object can have the following members:

ornl

An Ornl object.

potential

A Potential object.

29.26.4 Corresponding analysis keywords

*CREEP

29.27 CrushableFoam object

The CrushableFoam object specifies the crushable foam plasticity model.

Access

```
import material
mdb.models[name].materials[name].crushableFoam
import odbMaterial
session.odbs[name].materials[name].crushableFoam
```

29.27.1 CrushableFoam(...)

This method creates a CrushableFoam object.

Path

```
mdb.models[name].materials[name].CrushableFoam
session.odbs[name].materials[name].CrushableFoam
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

hardening

A SymbolicConstant specifying the type of hardening/softening definition. Possible values are VOLUMETRIC and ISOTROPIC. The default value is VOLUMETRIC.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=VOLUMETRIC, the table data specify the following:

- Ratio, k , of initial yield stress in uniaxial compression, σ_c^0 , to initial yield stress in hydrostatic compression, p_c^0 ; $0.0 < k < 3.0$.
- Ratio, k_t , of yield stress in hydrostatic tension, p_t , to initial yield stress in hydrostatic compression, p_c^0 . The default value is 1.0.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=ISOTROPIC, the table data specify the following:

- Ratio, k , of initial yield stress in uniaxial compression, σ_c^0 , to initial yield stress in hydrostatic compression, p_c^0 ; $0.0 \leq k \leq 3.0$.
- Plastic Poisson's ratio, ν_p ; $-1 \leq \nu_p \leq 0.5$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `CrushableFoam` object.

Exceptions

`RangeError`.

29.27.2 `setValues(...)`

This method modifies the `CrushableFoam` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `CrushableFoam` method.

Return value

`None`

Exceptions

RangeError.

29.27.3 Members

The CrushableFoam object has members with the same names and descriptions as the arguments to the **CrushableFoam** method.

In addition, the CrushableFoam object can have the following members:

crushableFoamHardening

A CrushableFoamHardening object.

rateDependent

A RateDependent object.

29.27.4 Corresponding analysis keywords

*CRUSHABLE FOAM

29.28 CrushableFoamHardening object

The CrushableFoamHardening object specifies hardening for the crushable foam plasticity model.

Access

```
import material
mdb.models[name].materials[name].crushableFoam.crushableFoamHardening
import odbMaterial
session.odbs[name].materials[name].crushableFoam\
.crushableFoamHardening
```

29.28.1 CrushableFoamHardening(...)

This method creates a CrushableFoamHardening object.

Path

```
mdb.models[name].materials[name].crushableFoam.CrushableFoamHardening
session.odbs[name].materials[name].crushableFoam\
.CrushableFoamHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The yield stress in uniaxial compression, σ_c .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CrushableFoamHardening object.

Exceptions

RangeError.

29.28.2 **setValues(...)**

This method modifies the CrushableFoamHardening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CrushableFoamHardening** method.

Return value

None

Exceptions

RangeError.

29.28.3 Members

The `CrushableFoamHardening` object has members with the same names and descriptions as the arguments to the `CrushableFoamHardening` method.

29.28.4 Corresponding analysis keywords

*CRUSHABLE FOAM HARDENING

29.29 CycledPlastic object

The `CycledPlastic` object specifies cycled yield stress data for the ORNL constitutive model.

Access

```
import material
mdb.models[name].materials[name].plastic.cycledPlastic
import odbMaterial
session.odbs[name].materials[name].plastic.cycledPlastic
```

29.29.1 CycledPlastic(...)

This method creates a `CycledPlastic` object.

Path

```
mdb.models[name].materials[name].plastic.CycledPlastic
session.odbs[name].materials[name].plastic.CycledPlastic
```

Required argument*table*

A sequence of sequences of Floats specifying the items described below.

Optional argument

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

Table data

- Yield stress.
- Plastic strain.
- Temperature, if the data depend on temperature.

Return value

A CycledPlastic object.

Exceptions

None.

29.29.2 **setValues(...)**

This method modifies the CycledPlastic object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CycledPlastic** method.

Return value

None

Exceptions

None.

29.29.3 **Members**

The CycledPlastic object has members with the same names and descriptions as the arguments to the **CycledPlastic** method.

29.29.4 **Corresponding analysis keywords**

*CYCLED PLASTIC

29.30 **CyclicHardening object**

The CyclicHardening object defines the evolution of the elastic domain for the nonlinear isotropic/kinematic hardening model.

Access

```
import material
mdb.models[name].materials[name].plastic.cyclicHardening
import odbMaterial
session.odbs[name].materials[name].plastic.cyclicHardening
```

29.30.1 **CyclicHardening(...)**

This method creates a CyclicHardening object.

Path

```
mdb.models[name].materials[name].plastic.CyclicHardening
session.odbs[name].materials[name].plastic.CyclicHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

parameters

A Boolean specifying whether material parameters are to be input directly. The default value is OFF.

Table data

- Equivalent stress.
- Q_{∞} (only if *parameters*=ON).

MATERIAL COMMANDS

- Hardening parameter (only if *parameters=ON*).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CyclicHardening object.

Exceptions

None.

29.30.2 setValues(...)

This method modifies the CyclicHardening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CyclicHardening** method.

Return value

None

Exceptions

None.

29.30.3 Members

The CyclicHardening object has members with the same names and descriptions as the arguments to the **CyclicHardening** method.

29.30.4 Corresponding analysis keywords

*CYCLIC HARDENING

29.31 DamageEvolution object

The DamageEvolution object specifies material properties to define the evolution of damage.

Access

```
import material
mdb.models[name].materials[name].ductileDamageInitiation\
.damageEvolution
mdb.models[name].materials[name].fldDamageInitiation.damageEvolution
mdb.models[name].materials[name].flsdDamageInitiation.damageEvolution
mdb.models[name].materials[name].hashinDamageInitiation\
.damageEvolution
mdb.models[name].materials[name].johnsonCookDamageInitiation\
.damageEvolution
mdb.models[name].materials[name].maxeDamageInitiation.damageEvolution
mdb.models[name].materials[name].maxpeDamageInitiation.damageEvolution
mdb.models[name].materials[name].maxpsDamageInitiation.damageEvolution
mdb.models[name].materials[name].maxsDamageInitiation.damageEvolution
mdb.models[name].materials[name].mkDamageInitiation.damageEvolution
mdb.models[name].materials[name].msfldDamageInitiation.damageEvolution
mdb.models[name].materials[name].quadeDamageInitiation.damageEvolution
mdb.models[name].materials[name].quadsDamageInitiation.damageEvolution
mdb.models[name].materials[name].shearDamageInitiation.damageEvolution
import odbMaterial
session.odbs[name].materials[name].ductileDamageInitiation\
.damageEvolution
session.odbs[name].materials[name].fldDamageInitiation.damageEvolution
session.odbs[name].materials[name].flsdDamageInitiation\
.damageEvolution
session.odbs[name].materials[name].hashinDamageInitiation\
.damageEvolution
session.odbs[name].materials[name].johnsonCookDamageInitiation\
.damageEvolution
session.odbs[name].materials[name].maxeDamageInitiation\
.damageEvolution
session.odbs[name].materials[name].maxpeDamageInitiation\
.damageEvolution
```

```

session.odbs [name] .materials [name] .maxpsDamageInitiation\
.damageEvolution
session.odbs [name] .materials [name] .maxsDamageInitiation\
.damageEvolution
session.odbs [name] .materials [name] .mkDamageInitiation.damageEvolution
session.odbs [name] .materials [name] .msfldDamageInitiation\
.damageEvolution
session.odbs [name] .materials [name] .quadeDamageInitiation\
.damageEvolution
session.odbs [name] .materials [name] .quadsDamageInitiation\
.damageEvolution
session.odbs [name] .materials [name] .shearDamageInitiation\
.damageEvolution

```

29.31.1 DamageEvolution(...)

This method creates a DamageEvolution object.

Path

```

mdb.models [name] .materials [name] .ductileDamageInitiation\
.DamageEvolution
mdb.models [name] .materials [name] .fldDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .flsdDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .hashinDamageInitiation\
.DamageEvolution
mdb.models [name] .materials [name] .johnsonCookDamageInitiation\
.DamageEvolution
mdb.models [name] .materials [name] .maxeDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .maxpeDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .maxpsDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .maxsDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .mkDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .msfldDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .quadeDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .quadsDamageInitiation.DamageEvolution
mdb.models [name] .materials [name] .shearDamageInitiation.DamageEvolution
session.odbs [name] .materials [name] .ductileDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .fldDamageInitiation.DamageEvolution
session.odbs [name] .materials [name] .flsdDamageInitiation\
.DamageEvolution

```

```

session.odbs [name] .materials [name] .hashinDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .johnsonCookDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .maxeDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .maxpeDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .maxpsDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .maxsDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .mkDamageInitiation.DamageEvolution
session.odbs [name] .materials [name] .msfldDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .quadeDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .quadsDamageInitiation\
.DamageEvolution
session.odbs [name] .materials [name] .shearDamageInitiation\
.DamageEvolution

```

Required arguments

type

A SymbolicConstant specifying the type of damage evolution. Possible values are DISPLACEMENT and ENERGY.

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

degradation

A SymbolicConstant specifying the degradation. Possible values are MAXIMUM and MULTIPLICATIVE. The default value is MAXIMUM.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

mixedModeBehavior

A SymbolicConstant specifying the mixed mode behavior. Possible values are MODE_INDEPENDENT, TABULAR, POWER_LAW, and BK. The default value is MODE_INDEPENDENT.

MATERIAL COMMANDS

modeMixRatio

A SymbolicConstant specifying the mode mix ratio. Possible values are ENERGY and TRACTION. The default value is ENERGY.

power

None or a Float specifying the exponent in the power law or the Benzeggagh-Kenane criterion that defines the variation of fracture energy with mode mix for cohesive elements. The default value is **None**.

softening

A SymbolicConstant specifying the softening. Possible values are LINEAR, EXPONENTIAL, and TABULAR. The default value is LINEAR.

Table data

If *type*=DISPLACEMENT, and *softening*=LINEAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Equivalent total or plastic displacement at failure, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=LINEAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Fracture energy.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Equivalent total or plastic displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

MATERIAL COMMANDS

If *type*=ENERGY, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Fracture energy.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=TABULAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Damage variable.
- Equivalent total or plastic displacement, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=TABULAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Damage variable.
- Equivalent total or plastic displacement, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR or EXPONENTIAL, and *mixedModeBehavior*=POWER_LAW or BK, the table data specify the following:

- Normal mode fracture energy.
- Shear mode fracture energy for failure in the first shear direction.
- Shear mode fracture energy for failure in the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

- Value of the second field variable.
- Etc.

If *type*=ENERGY, *softening*=LINEAR and constructor for `DamageInitiation` = **HashinDamageInitiation** the table data specify the following:

- Fiber tensile fracture energy.
- Fiber compressive fracture energy.
- Matrix tensile fracture energy.
- Matrix compressive fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `DamageEvolution` object.

Exceptions

`RangeError`.

29.31.2 `setValues(...)`

This method modifies the `DamageEvolution` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **`setValues`** are the same as the arguments to the **`DamageEvolution`** method.

Return value

`None`

Exceptions

`RangeError`.

29.31.3 Members

The `DamageEvolution` object has members with the same names and descriptions as the arguments to the `DamageEvolution` method.

29.31.4 Corresponding analysis keywords

*DAMAGE EVOLUTION

29.32 DamageInitiation object

The `DamageInitiation` object specifies material properties to define the initiation of damage.

Access

```
import material
mdb.models[name].materials[name].ductileDamageInitiation
mdb.models[name].materials[name].fldDamageInitiation
mdb.models[name].materials[name].flsdDamageInitiation
mdb.models[name].materials[name].hashinDamageInitiation
mdb.models[name].materials[name].johnsonCookDamageInitiation
mdb.models[name].materials[name].maxeDamageInitiation
mdb.models[name].materials[name].maxpeDamageInitiation
mdb.models[name].materials[name].maxpsDamageInitiation
mdb.models[name].materials[name].maxsDamageInitiation
mdb.models[name].materials[name].mkDamageInitiation
mdb.models[name].materials[name].msfldDamageInitiation
mdb.models[name].materials[name].quadeDamageInitiation
mdb.models[name].materials[name].quadsDamageInitiation
mdb.models[name].materials[name].shearDamageInitiation
import odbMaterial
session.odbs[name].materials[name].ductileDamageInitiation
session.odbs[name].materials[name].fldDamageInitiation
session.odbs[name].materials[name].flsdDamageInitiation
session.odbs[name].materials[name].hashinDamageInitiation
session.odbs[name].materials[name].johnsonCookDamageInitiation
session.odbs[name].materials[name].maxeDamageInitiation
session.odbs[name].materials[name].maxpeDamageInitiation
```



```

session.odbs[name].materials[name].maxpsDamageInitiation
session.odbs[name].materials[name].maxsDamageInitiation
session.odbs[name].materials[name].mkDamageInitiation
session.odbs[name].materials[name].msfldDamageInitiation
session.odbs[name].materials[name].quadeDamageInitiation
session.odbs[name].materials[name].quadsDamageInitiation
session.odbs[name].materials[name].shearDamageInitiation

```

29.32.1 DuctileDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```

mdb.models[name].materials[name].DuctileDamageInitiation
session.odbs[name].materials[name].DuctileDamageInitiation

```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

MATERIAL COMMANDS

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Table data

If constructor is **DuctileDamageInitiation**, the table data specify the following:

- Equivalent fracture strain at damage initiation.
- Stress triaxiality.
- Strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **FldDamageInitiation**, the table data specify the following:

- Major principal strain at damage initiation.
- Minor principal strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor **FlsdDamageInitiation**, the table data specify the following:

- Major principal stress at damage initiation.
- Minor principal stress.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **JohnsonCookDamageInitiation**, the table data specify the following:

- Johnson-Cook failure parameter D1.
- Johnson-Cook failure parameter D2.
- Johnson-Cook failure parameter D3.
- Johnson-Cook failure parameter D4.
- Johnson-Cook failure parameter D5.
- Melting temperature.
- Transition temperature.
- Reference strain rate.

If constructor **MkDamageInitiation**, the table data specify the following:

- Flaw size relative to nominal thickness of the section.
- Angle (in degrees) with respect to the 1-direction of the local material orientation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MsfldDamageInitiation** and *definition*=MSFLD, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.
- Equivalent plastic strain at initiation of localized necking.
- Ratio of minor to major principal strains.
- Equivalent plastic strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MsfldDamageInitiation** and *definition*=FLD, the table data specify the following:

- Major principal strain at initiation of localized necking.
- Equivalent plastic strain at initiation of localized necking.

MATERIAL COMMANDS

- Ratio of minor to major principal strains.
- Equivalent plastic strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **QuadeDamageInitiation** or **MaxeDamageInitiation**, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the first shear direction.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **QuadsDamageInitiation** or **MaxsDamageInitiation**, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the first shear direction.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MaxpeDamageInitiation**, the table data specify the following:

- Maximum principal strain at damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MaxpsDamageInitiation**, the table data specify the following:

- Maximum principal stress at damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **ShearDamageInitiation**, the table data specify the following:

- Equivalent fracture strain at damage initiation.
- Shear stress ratio.
- Strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **HashinDamageInitiation**, the table data specify the following:

- Fiber tensile strength.
- Fiber compressive strength.
- Matrix tensile strength.
- Matrix compressive strength.
- Longitudinal shear strength.
- Transverse shear strength.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.2 FldDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`mdb.models [name] .materials [name] .FldDamageInitiation`
`session.odbs [name] .materials [name] .FldDamageInitiation`

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.3 FlsdDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models [name] .materials [name] .FlsdDamageInitiation  
session.odbs [name] .materials [name] .FlsdDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

MATERIAL COMMANDS

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.4 JohnsonCookDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`mdb.models [name] .materials [name] .JohnsonCookDamageInitiation`
`session.odbs [name] .materials [name] .JohnsonCookDamageInitiation`

Required argument*table*

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments*definition*

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

MATERIAL COMMANDS

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.5 MaxeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MaxeDamageInitiation  
session.odbs[name].materials[name].MaxeDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.6 MaxsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MaxsDamageInitiation
session.odbs[name].materials[name].MaxsDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.7 MkDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MkDamageInitiation
session.odbs[name].materials[name].MkDamageInitiation
```

Required argument*table*

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments*definition*

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

MATERIAL COMMANDS

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.8 MsfldDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MsfldDamageInitiation
session.odbs[name].materials[name].MsfldDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.9 QuadeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].QuadeDamageInitiation  
session.odbs[name].materials[name].QuadeDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.10 QuadsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].QuadsDamageInitiation  
session.odbs[name].materials[name].QuadsDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.11 MaxpeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MaxpeDamageInitiation
session.odbs[name].materials[name].MaxpeDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.12 MaxpsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models[name].materials[name].MaxpsDamageInitiation
session.odbs[name].materials[name].MaxpsDamageInitiation
```

Required argument*table*

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments*definition*

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

MATERIAL COMMANDS

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

position

An SymbolicConstant specifying the damage initiation position. Possible values are CENTROID, CRACKTIP and COMBINED. The default value is CENTROID.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.13 ShearDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`mdb.models[name].materials[name].ShearDamageInitiation`
`session.odbs[name].materials[name].ShearDamageInitiation`

Required argument*table*

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments*definition*

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

MATERIAL COMMANDS

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.14 HashinDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
mdb.models [name] .materials [name] .HashinDamageInitiation  
session.odbs [name] .materials [name] .HashinDamageInitiation
```

Required argument

table

A sequence of sequences of Floats specifying the items described in the "Table data" section.

Optional arguments

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

Return value

A DamageInitiation object.

Exceptions

RangeError.

29.32.15 setValues(...)

This method modifies the DamageInitiation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DamageInitiation** method.

Return value

None

Exceptions

None.

29.32.16 Members

The DamageInitiation object can have the following members:

definition

A SymbolicConstant specifying the damage initiation definition. Possible values are FLD and MSFLD. The default value is MSFLD.

feq

A Float specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Float specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Float specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Float specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Float specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Float specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Float specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

A SymbolicConstant specifying the damage initiation direction. Possible values are NMORI and TMORI. The default value is NMORI.

table

A tuple of tuples of Floats specifying the items described in the "Table data" section.

damageEvolution

A DamageEvolution object.

damageStabilization

A DamageStabilization object.

damageStabilizationCohesive

A DamageStabilizationCohesive object.

29.32.17 Corresponding analysis keywords

*DAMAGE INITIATION

29.33 DamageStabilization object

The DamageStabilization object specifies the viscosity coefficients for the damage model for fiber-reinforced materials.

Access

```
import material
```

MATERIAL COMMANDS

```
mdb.models[name].materials[name].ductileDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].fldDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].flsdDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].hashinDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].johnsonCookDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].maxeDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].maxpeDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].maxpsDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].maxsDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].mkDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].msfldDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].quadeDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].quadsDamageInitiation\  
.damageStabilization  
mdb.models[name].materials[name].shearDamageInitiation\  
.damageStabilization  
import odbMaterial  
session.odbs[name].materials[name].ductileDamageInitiation\  
.damageStabilization  
session.odbs[name].materials[name].fldDamageInitiation\  
.damageStabilization  
session.odbs[name].materials[name].flsdDamageInitiation\  
.damageStabilization  
session.odbs[name].materials[name].hashinDamageInitiation\  
.damageStabilization  
session.odbs[name].materials[name].johnsonCookDamageInitiation\  
.damageStabilization  
session.odbs[name].materials[name].maxeDamageInitiation\  
.damageStabilization
```

```

session.odbs[name].materials[name].maxpeDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].maxpsDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].maxsDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].mkDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].msfldDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].quadeDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].quadsDamageInitiation\
.damageStabilization
session.odbs[name].materials[name].shearDamageInitiation\
.damageStabilization

```

29.33.1 DamageStabilization(...)

This method creates a DamageStabilization object.

Path

```

mdb.models[name].materials[name].ductileDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].fldDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].flsdDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].hashinDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].johnsonCookDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].maxeDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].maxpeDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].maxpsDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].maxsDamageInitiation\
.DamageStabilization
mdb.models[name].materials[name].mkDamageInitiation\

```

MATERIAL COMMANDS

```
.DamageStabilization
mdb.models [name] .materials [name] .msfldDamageInitiation\
.DamageStabilization
mdb.models [name] .materials [name] .quadeDamageInitiation\
.DamageStabilization
mdb.models [name] .materials [name] .quadsDamageInitiation\
.DamageStabilization
mdb.models [name] .materials [name] .shearDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .ductileDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .fldDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .flsdDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .hashinDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .johnsonCookDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .maxeDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .maxpeDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .maxpsDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .maxsDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .mkDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .msfldDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .quadeDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .quadsDamageInitiation\
.DamageStabilization
session.odbs [name] .materials [name] .shearDamageInitiation\
.DamageStabilization
```

Required arguments

fiberTensileCoeff

A Float specifying the viscosity coefficient for the fiber tensile failure mode.

fiberCompressiveCoeff

A Float specifying the viscosity coefficient for the fiber compressive failure mode.

matrixTensileCoeff

A Float specifying the viscosity coefficient for the matrix tensile failure mode.

matrixCompressiveCoeff

A Float specifying the viscosity coefficient for the matrix compressive failure mode.

Optional arguments

None.

Return value

A DamageStabilization object.

Exceptions

RangeError.

29.33.2 setValues(...)

This method modifies the DamageStabilization object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DamageStabilization** method.

Return value

None

Exceptions

RangeError.

29.33.3 Members

The DamageStabilization object has members with the same names and descriptions as the arguments to the **DamageStabilization** method.

29.33.4 Corresponding analysis keywords

*DAMAGE STABILIZATION

29.34 DamageStabilizationCohesive object

The DamageStabilizationCohesive object specifies the viscosity coefficients for the damage model for surface-based cohesive behavior or enriched cohesive behavior.

Access

```
import material
mdb.models[name].materials[name].ductileDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].fldDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].flsdDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].hashinDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].johnsonCookDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].maxeDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].maxpeDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].maxpsDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].maxsDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].mkDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].msfldDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].quadeDamageInitiation\
.damageStabilizationCohesive
mdb.models[name].materials[name].quadsDamageInitiation\
.damageStabilizationCohesive
```



```

mdb.models[name].materials[name].shearDamageInitiation\
.damageStabilizationCohesive
import odbMaterial
session.odbs[name].materials[name].ductileDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].fldDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].flsdDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].hashinDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].johnsonCookDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].maxeDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].maxpeDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].maxpsDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].maxsDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].mkDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].msfldDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].quadeDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].quadsDamageInitiation\
.damageStabilizationCohesive
session.odbs[name].materials[name].shearDamageInitiation\
.damageStabilizationCohesive

```

29.34.1 DamageStabilizationCohesive(...)

This method creates a DamageStabilizationCohesive object.

Path

```

mdb.models[name].materials[name].ductileDamageInitiation\
.DamageStabilizationCohesive
mdb.models[name].materials[name].fldDamageInitiation\
.DamageStabilizationCohesive

```

MATERIAL COMMANDS

```
mdb.models [name] .materials [name] .flsdDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .hashinDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .johnsonCookDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .maxeDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .maxpeDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .maxpsDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .maxsDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .mkDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .msfldDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .quadeDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .quadsDamageInitiation\  
.DamageStabilizationCohesive  
mdb.models [name] .materials [name] .shearDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .ductileDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .fldDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .flsdDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .hashinDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .johnsonCookDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .maxeDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .maxpeDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .maxpsDamageInitiation\  
.DamageStabilizationCohesive  
session.odbs [name] .materials [name] .maxsDamageInitiation\  
.DamageStabilizationCohesive
```

```
.DamageStabilizationCohesive
session.odbs [name] .materials [name] .mkDamageInitiation\
.DamageStabilizationCohesive
session.odbs [name] .materials [name] .msfldDamageInitiation\
.DamageStabilizationCohesive
session.odbs [name] .materials [name] .quadeDamageInitiation\
.DamageStabilizationCohesive
session.odbs [name] .materials [name] .quadsDamageInitiation\
.DamageStabilizationCohesive
session.odbs [name] .materials [name] .shearDamageInitiation\
.DamageStabilizationCohesive
```

Required arguments

None.

Optional argument

cohesiveCoeff

None or a Float specifying the viscosity coefficient. The default value is **None**.

Return value

A DamageStabilizationCohesive object.

Exceptions

RangeError.

29.34.2 setValues(...)

This method modifies the DamageStabilizationCohesive object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DamageStabilizationCohesive** method.

Return value

None

Exceptions

RangeError.

29.34.3 Members

The `DamageStabilizationCohesive` object has members with the same names and descriptions as the arguments to the `DamageStabilizationCohesive` method.

29.34.4 Corresponding analysis keywords

*DAMAGE STABILIZATION

29.35 Damping object

The Damping object specifies material damping.

Access

```
import material
mdb.models[name].materials[name].damping
import odbMaterial
session.odbs[name].materials[name].damping
```

29.35.1 Damping(...)

This method creates a Damping object.

Path

```
mdb.models[name].materials[name].Damping
session.odbs[name].materials[name].Damping
```

Required arguments

None.

Optional arguments*alpha*

A Float specifying the α_R factor to create mass proportional damping in direct-integration and explicit dynamics. The default value is 0.0.

beta

A Float specifying the β_R factor to create stiffness proportional damping in direct-integration and explicit dynamics. The default value is 0.0.

composite

A Float specifying the fraction of critical damping to be used with this material in calculating composite damping factors for the modes (for use in modal dynamics). The default value is 0.0.

This argument applies only to Abaqus/Standard analyses.

structural

A Float specifying the structural factor to create material damping in direct-integration and explicit dynamics. The default value is 0.0.

Return value

A Damping object.

Exceptions

RangeError.

29.35.2 setValues(...)

This method modifies the Damping object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Damping** method.

Return value

None

Exceptions

RangeError.

29.35.3 Members

The Damping object has members with the same names and descriptions as the arguments to the **Damping** method.

29.35.4 Corresponding analysis keywords

*DAMPING

29.36 DeformationPlasticity object

The DeformationPlasticity object specifies the deformation plasticity model.

Access

```
import material
mdb.models[name].materials[name].deformationPlasticity
import odbMaterial
session.odbs[name].materials[name].deformationPlasticity
```

29.36.1 DeformationPlasticity(...)

This method creates a DeformationPlasticity object.

Path

```
mdb.models[name].materials[name].DeformationPlasticity
session.odbs[name].materials[name].DeformationPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional argument

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

Table data

- Young's modulus, E .

- Poisson's ratio, ν .
- Yield stress, σ^0 .
- Exponent, n .
- Yield offset, α .
- Temperature, if the data depend on temperature.

Return value

A DeformationPlasticity object.

Exceptions

RangeError.

29.36.2 setValues(...)

This method modifies the DeformationPlasticity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DeformationPlasticity** method.

Return value

None

Exceptions

RangeError.

29.36.3 Members

The DeformationPlasticity object has members with the same names and descriptions as the arguments to the **DeformationPlasticity** method.

29.36.4 Corresponding analysis keywords

*DEFORMATION PLASTICITY

29.37 Density object

The Density object specifies the material density.

Access

```
import material
mdb.models[name].materials[name].density
import odbMaterial
session.odbs[name].materials[name].density
```

29.37.1 Density(...)

This method creates a Density object.

Path

```
mdb.models[name].materials[name].Density
session.odbs[name].materials[name].Density
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

distributionType

A SymbolicConstant specifying how the density is distributed spatially. Possible values are UNIFORM, ANALYTICAL_FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

fieldName

A String specifying the name of the AnalyticalField or DiscreteField object associated with this material option. The *fieldName* argument applies only when *distributionType*=ANALYTICAL_FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

Table data

- The mass density.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Density object.

Exceptions

RangeError.

29.37.2 setValues(...)

This method modifies the Density object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Density** method.

Return value

None

Exceptions

RangeError.

29.37.3 Members

The Density object has members with the same names and descriptions as the arguments to the **Density** method.

29.37.4 Corresponding analysis keywords

*DENSITY

29.38 Depvar object

The Depvar object specifies solution-dependent state variables.

Access

```
import material
mdb.models[name].materials[name].depvar
import odbMaterial
session.odbs[name].materials[name].depvar
```

29.38.1 Depvar(...)

This method creates a Depvar object.

Path

```
mdb.models[name].materials[name].Depvar
session.odbs[name].materials[name].Depvar
```

Required arguments

None.

Optional arguments

deleteVar

An Int specifying the state variable number controlling the element deletion flag. The default value is 0.

This argument applies only to Abaqus/Explicit analyses.

n

An Int specifying the number of solution-dependent state variables required at each integration point. The default value is 0.

Return value

A Depvar object.

Exceptions

RangeError.

29.38.2 **setValues(...)**

This method modifies the Depvar object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Depvar** method.

Return value

None

Exceptions

RangeError.

29.38.3 **Members**

The Depvar object has members with the same names and descriptions as the arguments to the **Depvar** method.

29.38.4 **Corresponding analysis keywords**

*DEPVAR

29.39 **DetonationPoint object**

A DetonationPoint object specifies a suboption of the Eos object. The DetonationPoint object defines either isotropic linear elastic shear or linear viscous shear behavior for a hydrodynamic material.

Access

```
import material
mdb.models[name].materials[name].eos.detonationPoint
import odbMaterial
session.odbs[name].materials[name].eos.detonationPoint
```

29.39.1 DetonationPoint(...)

This method creates a DetonationPoint object.

Path

```
mdb.models[name].materials[name].eos.DetonationPoint  
session.odbs[name].materials[name].eos.DetonationPoint
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- X value for coordinate of detonation point.
- Y value for coordinate of detonation point.
- Z value for coordinate of detonation point.
- Detonation delay time.

Return value

A DetonationPoint object.

Exceptions

None.

29.39.2 setValues(...)

This method modifies the DetonationPoint object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DetonationPoint** method.

Return value

None

Exceptions

None.

29.39.3 Members

The DetonationPoint object has members with the same names and descriptions as the arguments to the `DetonationPoint` method.

29.39.4 Corresponding analysis keywords

*DETONATION POINT

29.40 Dielectric object

The Dielectric object specifies dielectric material properties.

Access

```
import material
mdb.models[name].materials[name].dielectric
import odbMaterial
session.odbs[name].materials[name].dielectric
```

29.40.1 Dielectric(...)

This method creates a Dielectric object.

Path

```
mdb.models[name].materials[name].Dielectric
session.odbs[name].materials[name].Dielectric
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the dielectric behavior. Possible values are ISOTROPIC, ORTHOTROPIC, and ANISOTROPIC. The default value is ISOTROPIC.

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Dielectric constant.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{11}^{φ} .
- D_{22}^{φ} .
- D_{33}^{φ} .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{11}^{φ} .
- D_{12}^{φ} .
- D_{22}^{φ} .
- D_{13}^{φ} .

- D_{23}^{φ} .
- D_{33}^{φ} .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Dielectric object.

Exceptions

None.

29.40.2 setValues(...)

This method modifies the Dielectric object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Dielectric** method.

Return value

None

Exceptions

None.

29.40.3 Members

The Dielectric object has members with the same names and descriptions as the arguments to the **Dielectric** method.

29.40.4 Corresponding analysis keywords

*DIELECTRIC

29.41 Diffusivity object

The Diffusivity object specifies mass diffusivity.

Access

```
import material
mdb.models[name].materials[name].diffusivity
import odbMaterial
session.odbs[name].materials[name].diffusivity
```

29.41.1 Diffusivity(...)

This method creates a Diffusivity object.

Path

```
mdb.models[name].materials[name].Diffusivity
session.odbs[name].materials[name].Diffusivity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of diffusivity. Possible values are ISOTROPIC, ORTHOTROPIC, and ANISOTROPIC. The default value is ISOTROPIC.

law

A SymbolicConstant specifying the diffusion behavior. Possible values are GENERAL and FICK. The default value is GENERAL.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Diffusivity, D .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{11} .
- D_{22} .
- D_{33} .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{11} .
- D_{12} .
- D_{22} .
- D_{13} .
- D_{23} .
- D_{33} .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Diffusivity object.

Exceptions

RangeError.

29.41.2 **setValues(...)**

This method modifies the Diffusivity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Diffusivity** method.

Return value

None

Exceptions

RangeError.

29.41.3 **Members**

The Diffusivity object has members with the same names and descriptions as the arguments to the **Diffusivity** method.

In addition, the Diffusivity object can have the following members:

pressureEffect

A PressureEffect object.

soretEffect

A SoretEffect object.

29.41.4 **Corresponding analysis keywords**

*DIFFUSIVITY

29.42 **DruckerPrager object**

The DruckerPrager object specifies the extended Drucker-Prager plasticity model.

Access

import material

```

mdb.models[name].materials[name].druckerPrager
import odbMaterial
session.odbs[name].materials[name].druckerPrager

```

29.42.1 DruckerPrager(...)

This method creates a DruckerPrager object.

Path

```

mdb.models[name].materials[name].DruckerPrager
session.odbs[name].materials[name].DruckerPrager

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

shearCriterion

A SymbolicConstant specifying the yield criterion. Possible values are LINEAR, HYPERBOLIC, and EXPONENTIAL. The default value is LINEAR.

This argument applies only to Abaqus/Standard analyses. Only the linear Drucker-Prager model is available in Abaqus/Explicit analyses.

eccentricity

A Float specifying the flow potential eccentricity, ϵ , a small positive number that defines the rate at which the hyperbolic flow potential approaches its asymptote. The default value is 0.1.

This argument applies only to Abaqus/Standard analyses.

testData

A Boolean specifying whether the material constants for the exponent model are to be computed by Abaqus/Standard from triaxial test data at different levels of confining pressure. The default value is OFF.

This argument is valid only if *shearCriterion*=EXPONENTIAL.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *shearCriterion*=LINEAR (the only option allowed in an Abaqus/Explicit analysis), the table data specify the following:

- Material angle of friction, β , in the p - t plane. Give the value in degrees.

MATERIAL COMMANDS

- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, a value of 1.0 is assumed.
- Dilation angle, ψ , in the p - t plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shearCriterion*=HYPERBOLIC, the table data specify the following:

- Material angle of friction, β , at high confining pressure in the p - q plane. Give the value in degrees.
- Initial hydrostatic tension strength, $p_t|_0$.
- Dilation angle, ψ , at high confining pressure in the p - q plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shearCriterion*=EXPONENTIAL, the table data specify the following:

- Dilation angle, ψ , at high confining pressure in the p - q plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPrager object.

Exceptions

RangeError.

29.42.2 **setValues(...)**

This method modifies the DruckerPrager object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DruckerPrager** method.

Return value

None

Exceptions

`RangeError`.

29.42.3 Members

The `DruckerPrager` object has members with the same names and descriptions as the arguments to the **DruckerPrager** method.

In addition, the `DruckerPrager` object can have the following members:

druckerPragerCreep

A `DruckerPragerCreep` object.

druckerPragerHardening

A `DruckerPragerHardening` object.

rateDependent

A `RateDependent` object.

triaxialTestData

A `TriaxialTestData` object.

29.42.4 Corresponding analysis keywords

*DRUCKER PRAGER

29.43 DruckerPragerCreep object

The `DruckerPragerCreep` object specifies creep for Drucker-Prager plasticity models.

Access

```
import material
mdb.models[name].materials[name].druckerPrager.druckerPragerCreep
import odbMaterial
session.odbs[name].materials[name].druckerPrager.druckerPragerCreep
```

29.43.1 DruckerPragerCreep(...)

This method creates a DruckerPragerCreep object.

Path

```
mdb.models [name] .materials [name] .druckerPrager.DruckerPragerCreep
session.odbs [name] .materials [name] .druckerPrager.DruckerPragerCreep
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the type of data defining the creep law. Possible values are:

- STRAIN, specifying a strain-hardening power law.
- TIME, specifying a time-hardening power law.
- SINGHM, specifying a Singh-Mitchell type law.
- USER, specifying the creep law is input from user subroutine **CREEP**.

The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=TIME or *law*=STRAIN, the table data specify the following:

- A . (Units of $F^{-n} L^{2n} T^{-1-m}$.)
- n .
- m .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=SINGHM, the table data specify the following:

- A . (Units of T^{-1} .)
- α . (Units of $F^{-1} L^2$.)
- m .

- t_1 . (Units of T.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPragerCreep object.

Exceptions

RangeError.

29.43.2 setValues(...)

This method modifies the DruckerPragerCreep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DruckerPragerCreep** method.

Return value

None

Exceptions

RangeError.

29.43.3 Members

The DruckerPragerCreep object has members with the same names and descriptions as the arguments to the **DruckerPragerCreep** method.

29.43.4 Corresponding analysis keywords

*DRUCKER PRAGER CREEP

29.44 DruckerPragerHardening object

The DruckerPragerHardening object specifies hardening for Drucker-Prager plasticity models.

Access

```
import material
mdb.models[name].materials[name].druckerPrager.druckerPragerHardening
import odbMaterial
session.odbs[name].materials[name].druckerPrager\
.druckerPragerHardening
```

29.44.1 DruckerPragerHardening(...)

This method creates a DruckerPragerHardening object.

Path

```
mdb.models[name].materials[name].druckerPrager.DruckerPragerHardening
session.odbs[name].materials[name].druckerPrager\
.DruckerPragerHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of data defining the hardening behavior. Possible values are COMPRESSION, TENSION, and SHEAR. The default value is COMPRESSION.

rate

A Boolean specifying whether the data depend on rate. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress.
- Absolute value of the corresponding plastic strain. (The first tabular value entered must always be zero.)
- Equivalent plastic strain rate, $\dot{\varepsilon}^{pl}$, for which this hardening curve applies.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPragerHardening object.

Exceptions

RangeError.

29.44.2 setValues(...)

This method modifies the DruckerPragerHardening object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DruckerPragerHardening** method.

Return value

None

Exceptions

RangeError.

29.44.3 Members

The DruckerPragerHardening object has members with the same names and descriptions as the arguments to the **DruckerPragerHardening** method.

29.44.4 Corresponding analysis keywords

*DRUCKER PRAGER HARDENING

29.45 Elastic object

The Elastic object specifies elastic material properties.

Access

```
import material
mdb.models[name].materials[name].elastic
import odbMaterial
session.odbs[name].materials[name].elastic
```

29.45.1 Elastic(...)

This method creates an Elastic object.

Path

```
mdb.models[name].materials[name].Elastic
session.odbs[name].materials[name].Elastic
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of elasticity data provided. Possible values are:

- ISOTROPIC
- ORTHOTROPIC
- ANISOTROPIC
- ENGINEERING_CONSTANTS
- LAMINA
- TRACTION

- COUPLED_TRACTION
- SHORT_FIBER
- SHEAR

The default value is ISOTROPIC.

noCompression

A Boolean specifying whether compressive stress is allowed. The default value is OFF.

noTension

A Boolean specifying whether tensile stress is allowed. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

moduli

A SymbolicConstant specifying the time-dependence of the elastic material constants. Possible values are INSTANTANEOUS and LONG_TERM. The default value is LONG_TERM.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- The Young's modulus, E .
- The Poisson's ratio, ν .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=SHEAR, the table data specify the following:

- The shear modulus, G .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENGINEERING_CONSTANTS, the table data specify the following:

- E_1 .
- E_2 .
- E_3 .
- ν_{12} .
- ν_{13} .
- ν_{23} .

MATERIAL COMMANDS

- G_{12} .
- G_{13} .
- G_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=LAMINA, the table data specify the following:

- E_1 .
- E_2 .
- ν_{12} .
- G_{12} .
- G_{13} . This shear modulus is needed to define transverse shear behavior in shells.
- G_{23} . This shear modulus is needed to define transverse shear behavior in shells.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{1111} .
- D_{1122} .
- D_{2222} .
- D_{1133} .
- D_{2233} .
- D_{3333} .
- D_{1212} .
- D_{1313} .
- D_{2323} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{1111} .
- D_{1122} .

- D_{2222} .
- D_{1133} .
- D_{2233} .
- D_{3333} .
- D_{1112} .
- D_{2212} .
- D_{3312} .
- D_{1212} .
- D_{1113} .
- D_{2213} .
- D_{3313} .
- D_{1213} .
- D_{1313} .
- D_{1123} .
- D_{2223} .
- D_{3323} .
- D_{1223} .
- D_{1323} .
- D_{2323} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $type=TRACTION$, the table data specify the following:

- E for warping elements; E_{nn} for cohesive elements.
- G_1 for warping elements; E_{ss} for cohesive elements.
- G_2 for warping elements; E_{tt} for cohesive elements.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $type=COUPLED_TRACTION$, the table data specify the following:

- E_{nn} .
- E_{ss} .
- E_{tt} .

MATERIAL COMMANDS

- E_{ns} .
- E_{nt} .
- E_{st} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=SHORT_FIBER, there is no table data.

Return value

An Elastic object.

Exceptions

RangeError.

29.45.2 setValues(...)

This method modifies the Elastic object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Elastic** method.

Return value

None

Exceptions

RangeError.

29.45.3 Members

The Elastic object has members with the same names and descriptions as the arguments to the **Elastic** method.

In addition, the Elastic object can have the following members:

failStress

A FailStress object.

failStrain

A FailStrain object.

29.45.4 Corresponding analysis keywords

*ELASTIC

29.46 ElectricalConductivity object

The ElectricalConductivity object specifies electrical conductivity.

Access

```
import material
mdb.models[name].materials[name].electricalConductivity
import odbMaterial
session.odbs[name].materials[name].electricalConductivity
```

29.46.1 ElectricalConductivity(...)

This method creates an ElectricalConductivity object.

Path

```
mdb.models[name].materials[name].ElectricalConductivity
session.odbs[name].materials[name].ElectricalConductivity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of electrical conductivity. Possible values are ISOTROPIC, ORTHOTROPIC, and ANISOTROPIC. The default value is ISOTROPIC.

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Electrical conductivity.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- σ_{11}^E .
- σ_{22}^E .
- σ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- σ_{11}^E .
- σ_{12}^E .
- σ_{22}^E .
- σ_{13}^E .
- σ_{23}^E .
- σ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

An ElectricalConductivity object.

Exceptions

RangeError.

29.46.2 setValues(...)

This method modifies the ElectricalConductivity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ElectricalConductivity** method.

Return value

None

Exceptions

RangeError.

29.46.3 Members

The ElectricalConductivity object has members with the same names and descriptions as the arguments to the **ElectricalConductivity** method.

29.46.4 Corresponding analysis keywords

*ELECTRICAL CONDUCTIVITY

29.47 Eos object

The Eos object specifies an equation of state model.

Access

```
import material
```

```

mdb.models[name].materials[name].eos
import odbMaterial
session.odbs[name].materials[name].eos

```

29.47.1 Eos(...)

This method creates an Eos object.

Path

```

mdb.models[name].materials[name].Eos
session.odbs[name].materials[name].Eos

```

Required arguments

None.

Optional arguments

type

A SymbolicConstant specifying the equation of state. Possible values are USUP, JWL, IDEALGAS, TABULAR, and IGNITIONANDGROWTH. The default value is IDEALGAS.

temperatureDependency

A Boolean specifying whether the data in *gasSpecificTable* depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies for the data in *gasSpecificTable*. The default value is 0.

detonationEnergy

A Float specifying the detonation energy text field. The default value is 0.0.

solidTable

A sequence of sequences of Floats specifying the following:

- $\$A_{\{s\}}$.
- $\$B_{\{s\}}$.
- $\$\{\omega\}_{\{s\}}$.
- $\$R_{\{1s\}}$.
- $\$R_{\{2s\}}$.

The default value is an empty sequence.

gasTable

A sequence of sequences of Floats specifying the following:

- $\$A_{\{g\}}$.
- $\$B_{\{g\}}$.

- ω_g .
- R_{1g} .
- R_{2g} .

The default value is an empty sequence.

reactionTable

A sequence of sequences of Floats specifying the following:

- Initial Pressure, P .
- Product co-volume, V_p .
- Exponent on the unreacted fraction (ignition term), α .
- First burn rate coefficient, G_1 .
- Exponent on the unreacted fraction (growth term), β .
- Exponent on the reacted fraction (growth term), δ .
- Pressure exponent (growth term), γ .
- Second burn rate coefficient, G_2 .
- Exponent on the unreacted fraction (completion term), ϵ .
- Exponent on the reacted fraction (completion term), ζ .
- Pressure exponent (completion term), η .
- Initial reacted fraction, $F_{ig}^{\max}$.
- Maximum reacted fraction for the growth term, $F_{G1}^{\max}$.
- Minimum reacted fraction, $F_{G2}^{\min}$.

The default value is an empty sequence.

gasSpecificTable

A sequence of sequences of Floats specifying the following:

- Specific Heat per unit mass.
- Temperature dependent data.
- Value of first field variable.
- Value of second field variable.
- Etc.

The default value is an empty sequence.

table

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

Table data

If *type*=IDEALGAS, the table data represents the following:

- Gas constant, R .

MATERIAL COMMANDS

- The ambient pressure, p_A . If this field is left blank, a default of 0.0 is used.

If *type*=JWL, the table data represents the following:

- Detonation wave speed, C_d .
- A .
- B .
- ω . (Dimensionless.)
- R_1 . (Dimensionless.)
- R_2 . (Dimensionless.)
- Pre-detonation bulk modulus, K_{pd} .
- Detonation energy density, E_0 .

If *type*=USUP, the table data represents the following:

- c_0 .
- s . (Dimensionless.)
- Γ_0 . (Dimensionless.)

If *type*=TABULAR, the table data represents the following:

- F_1 .
- F_2 .
- ε_{vol}^c . (Dimensionless.)

Return value

An Eos object.

Exceptions

None.

29.47.2 **setValues(...)**

This method modifies the Eos object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Eos** method.

Return value

None

Exceptions

None.

29.47.3 Members

The Eos object has members with the same names and descriptions as the arguments to the **Eos** method.

In addition, the Eos object can have the following members:

detonationPoint

A DetonationPoint object.

eosCompaction

An EosCompaction object.

29.47.4 Corresponding analysis keywords

*EOS

29.48 EosCompaction object

The EosCompaction object specifies material eos compaction.

Access

```
import material
mdb.models[name].materials[name].eos.eosCompaction
import odbMaterial
session.odbs[name].materials[name].eos.eosCompaction
```

29.48.1 EosCompaction(...)

This method creates an EosCompaction object.

Path

```
mdb.models[name].materials[name].eos.EosCompaction
```

MATERIAL COMMANDS

```
session.odbs[name].materials[name].eos.EosCompaction
```

Required arguments

soundSpeed

A Float specifying reference sound speed in the porous material.

porosity

A Float specifying value of the porosity of the unloaded material.

pressure

A Float specifying pressure required to initialize plastic behavior.

compactionPressure

A Float specifying compaction pressure at which all pores are crushed.

Optional arguments

None.

Return value

An EosCompaction object.

Exceptions

RangeError.

29.48.2 **setValues(...)**

This method modifies the EosCompaction object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **EosCompaction** method.

Return value

None

Exceptions

RangeError.

29.48.3 Members

The `EosCompaction` object has members with the same names and descriptions as the arguments to the `EosCompaction` method.

29.48.4 Corresponding analysis keywords

*EOS COMPACTION

29.49 Expansion object

The Expansion object specifies thermal expansion.

Access

```
import material
mdb.models[name].materials[name].expansion
import odbMaterial
session.odbs[name].materials[name].expansion
```

29.49.1 Expansion(...)

This method creates an Expansion object.

Path

```
mdb.models[name].materials[name].Expansion
session.odbs[name].materials[name].Expansion
```

Required arguments

None.

Optional arguments

type

A `SymbolicConstant` specifying the type of expansion. Possible values are `ISOTROPIC`, `ORTHOTROPIC`, `ANISOTROPIC`, and `SHORT_FIBER`. The default value is `ISOTROPIC`.

userSubroutine

A Boolean specifying whether a user subroutine is used to define the increments of thermal strain. The default value is `OFF`.

MATERIAL COMMANDS

zero

A Float specifying the value of θ^0 if the thermal expansion is temperature-dependent or field-variable-dependent. The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

table

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

This argument is required only if *type* is not USER.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- α in Abaqus/Standard or Abaqus/Explicit analysis, or β in Abaqus/CFD analysis.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- α_{11} .
- α_{22} .
- α_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- α_{11} .
- α_{22} .
- α_{33} . (Not used for plane stress case.)
- α_{12} .
- α_{13} .
- α_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

- Etc.

If `type=SHORT_FIBER`, there is no table data.

Return value

An Expansion object.

Exceptions

RangeError.

29.49.2 `setValues(...)`

This method modifies the Expansion object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `Expansion` method.

Return value

None

Exceptions

RangeError.

29.49.3 `Members`

The Expansion object has members with the same names and descriptions as the arguments to the `Expansion` method.

29.49.4 `Corresponding analysis keywords`

*EXPANSION

29.50 FailStrain object

The FailStrain object defines parameters for strain-based failure measures.

Access

```
import material
mdb.models[name].materials[name].elastic.failStrain
import odbMaterial
session.odbs[name].materials[name].elastic.failStrain
```

29.50.1 FailStrain(...)

This method creates a FailStrain object.

Path

```
mdb.models[name].materials[name].elastic.FailStrain
session.odbs[name].materials[name].elastic.FailStrain
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tensile strain limit in fiber direction, $X_{\varepsilon t}$.
- Compressive strain limit in fiber direction, $X_{\varepsilon c}$.
- Tensile strain limit in transverse direction, $Y_{\varepsilon t}$.
- Compressive strain limit in transverse direction, $Y_{\varepsilon c}$.
- Shear strain limit in the X – Y plane, S_{ε} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

- Etc.

Return value

A FailStrain object.

Exceptions

RangeError.

29.50.2 setValues(...)

This method modifies the FailStrain object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FailStrain** method.

Return value

None

Exceptions

RangeError.

29.50.3 Members

The FailStrain object has members with the same names and descriptions as the arguments to the **FailStrain** method.

29.50.4 Corresponding analysis keywords

*FAIL STRAIN

29.51 FailStress object

The FailStress object defines parameters for stress-based failure measures.

Access

```
import material
mdb.models[name].materials[name].elastic.failStress
import odbMaterial
session.odbs[name].materials[name].elastic.failStress
```

29.51.1 FailStress(...)

This method creates a FailStress object.

Path

```
mdb.models[name].materials[name].elastic.FailStress
session.odbs[name].materials[name].elastic.FailStress
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tensile stress limit in fiber direction, X_t .
- Compressive stress limit in fiber direction, X_c .
- Tensile stress limit in transverse direction, Y_t .
- Compressive stress limit in transverse direction, Y_c .
- Shear strength in the X – Y plane, S .
- Cross product term coefficient, f^* ($-1.0 \leq f^* \leq 1.0$). The default value is zero.
- Biaxial stress limit, σ_{biax} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FailStress object.

Exceptions

RangeError.

29.51.2 setValues(...)

This method modifies the FailStress object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FailStress** method.

Return value

None

Exceptions

RangeError.

29.51.3 Members

The FailStress object has members with the same names and descriptions as the arguments to the **FailStress** method.

29.51.4 Corresponding analysis keywords

*FAIL STRESS

29.52 FailureRatios object

The FailureRatios object specifies the shape of the failure surface for a Concrete model.

Access

```
import material
mdb.models[name].materials[name].concrete.failureRatios
import odbMaterial
session.odbs[name].materials[name].concrete.failureRatios
```

29.52.1 FailureRatios(...)

This method creates a FailureRatios object.

Path

```
mdb.models[name].materials[name].concrete.FailureRatios
session.odbs[name].materials[name].concrete.FailureRatios
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Ratio of the ultimate biaxial compressive stress to the uniaxial compressive ultimate stress. The default value is 1.16.
- Absolute value of the ratio of the uniaxial tensile stress at failure to the uniaxial compressive stress at failure. The default value is 0.09.
- Ratio of the magnitude of a principal component of plastic strain at ultimate stress in biaxial compression to the plastic strain at ultimate stress in uniaxial compression. The default value is 1.28.
- Ratio of the tensile principal stress value at shear in plane stress, when the other nonzero principal stress component is at the ultimate compressive stress value, to the tensile cracking stress under uniaxial tension. The default value is 1/3.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

- Etc.

Return value

A FailureRatios object.

Exceptions

RangeError.

29.52.2 setValues(...)

This method modifies the FailureRatios object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FailureRatios** method.

Return value

None

Exceptions

RangeError.

29.52.3 Members

The FailureRatios object has members with the same names and descriptions as the arguments to the **FailureRatios** method.

29.52.4 Corresponding analysis keywords

*FAILURE RATIOS

29.53 FluidLeakoff object

The FluidLeakoff object specifies leak-off coefficients for pore pressure cohesive elements.

Access

```
import material
mdb.models[name].materials[name].fluidLeakoff
import odbMaterial
session.odbs[name].materials[name].fluidLeakoff
```

29.53.1 FluidLeakoff(...)

This method creates a FluidLeakoff object.

Path

```
mdb.models[name].materials[name].FluidLeakoff
session.odbs[name].materials[name].FluidLeakoff
```

Required arguments

None.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

A SymbolicConstant specifying the type of fluid leak-off. Possible values are COEFFICIENTS and USER. The default value is COEFFICIENTS.

table

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

Table data

The table data specify the following:

- Fluid leak-off coefficient at top element surface.
- Fluid leak-off coefficient at bottom element surface.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FluidLeakoff object.

Exceptions

None.

29.53.2 setValues(...)

This method modifies the FluidLeakoff object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidLeakoff** method.

Return value

None

Exceptions

None.

29.53.3 Members

The FluidLeakoff object has members with the same names and descriptions as the arguments to the **FluidLeakoff** method.

29.53.4 Corresponding analysis keywords

*FLUID LEAKOFF

29.54 GapFlow object

The GapFlow object specifies tangential flow constitutive parameters for pore pressure cohesive elements.

Access

```
import material
mdb.models[name].materials[name].gapFlow
import odbMaterial
session.odbs[name].materials[name].gapFlow
```

29.54.1 GapFlow(...)

This method creates a GapFlow object.

Path

```
mdb.models[name].materials[name].GapFlow
session.odbs[name].materials[name].GapFlow
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

kmax

None or a Float specifying the maximum permeability value that should be used. If *kmax*=**None**, Abaqus assumes that the permeability is not bounded. This value is meaningful only when *type*=NEWTONIAN. The default value is **None**.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

A SymbolicConstant specifying the type of gap flow. Possible values are NEWTONIAN and POWER_LAW. The default value is NEWTONIAN.

Table data

If *type*=NEWTONIAN the table data specify the following:

- Pore viscosity.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=POWER_LAW the table data specify the following:

- Consistency.
- Exponent.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GapFlow object.

Exceptions

None.

29.54.2 setValues(...)

This method modifies the GapFlow object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GapFlow** method.

Return value

None

Exceptions

None.

29.54.3 Members

The GapFlow object has members with the same names and descriptions as the arguments to the **GapFlow** method.

29.54.4 Corresponding analysis keywords

*GAP FLOW

29.55 GasketMembraneElastic object

The GasketMembraneElastic object defines the elastic parameters for the membrane shear behavior of a gasket.

Access

```
import material
mdb.models[name].materials[name].GasketMembraneElastic
import odbMaterial
session.odbs[name].materials[name].GasketMembraneElastic
```

29.55.1 GasketMembraneElastic(...)

This method creates a GasketMembraneElastic object.

Path

```
mdb.models[name].materials[name].GasketMembraneElastic
session.odbs[name].materials[name].GasketMembraneElastic
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Young's modulus, E .
- Poisson's ratio, ν .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GasketMembraneElastic object.

Exceptions

RangeError.

29.55.2 setValues(...)

This method modifies the GasketMembraneElastic object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GasketMembraneElastic** method.

Return value

None

Exceptions

RangeError.

29.55.3 Members

The GasketMembraneElastic object has members with the same names and descriptions as the arguments to the **GasketMembraneElastic** method.

29.55.4 Corresponding analysis keywords

*GASKET ELASTICITY

29.56 GasketThicknessBehavior object

The GasketThicknessBehavior object defines the behavior in the thickness direction for a gasket.

Access

```
import material
mdb.models[name].materials[name].gasketThicknessBehavior
import odbMaterial
session.odbs[name].materials[name].gasketThicknessBehavior
```

29.56.1 GasketThicknessBehavior(...)

This method creates a GasketThicknessBehavior object.

Path

```
mdb.models[name].materials[name].GasketThicknessBehavior
session.odbs[name].materials[name].GasketThicknessBehavior
```

Required argument*table*

A sequence of sequences of Floats specifying loading data. The first sequence must contain only 0. At least two sequences must be specified if *type*=DAMAGE, and at least 3 sequences must be specified if *type*=ELASTIC_PLASTIC. The items in the table data are described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the loading data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies included in the definition of the loading data, in addition to temperature. The default value is 0.

tensileStiffnessFactor

A Float specifying the fraction of the initial compressive stiffness that defines the stiffness in tension. The default value is 10^{-3} .

type

A SymbolicConstant specifying a damage elasticity model or an elastic-plastic model for gasket thickness-direction behavior. Possible values are ELASTIC_PLASTIC and DAMAGE. The default value is ELASTIC_PLASTIC.

unloadingDependencies

An Int specifying the number of field variable dependencies included in the definition of the unloading data, in addition to temperature. The default value is 0.

unloadingTemperatureDependency

A Boolean specifying whether unloading data depends on temperature. The default value is OFF.

variableUnits

A SymbolicConstant specifying the behavior in terms of units of force (or force in unit length) versus closure or pressure versus closure. Possible values are STRESS and FORCE. The default value is STRESS.

yieldOnset

A Float specifying the closure value at which the onset of yield occurs or the relative drop in slope on the loading curve that defines the onset of plastic deformation (depending on the value of *yieldOnsetMethod*). The default value is 0.1.

yieldOnsetMethod

A SymbolicConstant specifying the method used to determine yield onset. Possible values are RELATIVE_SLOPE_DROP and CLOSURE_VALUE. The default value is RELATIVE_SLOPE_DROP.

unloadingTable

A sequence of sequences of Floats specifying unloading data. The items in the table data are described below. The default value is an empty sequence.

Table data

If *variableUnits*=STRESS, the loading table data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE, the loading table data specify the following:

- Force or force per unit length; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=STRESS and *type*=ELASTIC_PLASTIC, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Plastic closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

MATERIAL COMMANDS

- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE and *type*=ELASTIC_PLASTIC, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Plastic closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=STRESS and *type*=DAMAGE, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Maximum closure reached while loading the gasket; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE and *type*=DAMAGE, the *unloadingTable* data specify the following:

- Force or force per unit length; this value must be positive.
- Closure; this value must be positive.
- Maximum closure reached while loading the gasket; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GasketThicknessBehavior object.

Exceptions

RangeError.

29.56.2 **setValues(...)**

This method modifies the `GasketThicknessBehavior` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `GasketThicknessBehavior` method.

Return value

None

Exceptions

`RangeError`.

29.56.3 **Members**

The `GasketThicknessBehavior` object has members with the same names and descriptions as the arguments to the `GasketThicknessBehavior` method.

In addition, the `GasketThicknessBehavior` object can have the following member:

contactArea

A `ContactArea` object.

29.56.4 **Corresponding analysis keywords**

*GASKET THICKNESS BEHAVIOR

29.57 **GasketTransverseShearElastic object**

The `GasketTransverseShearElastic` object defines the elastic parameters for the transverse shear behavior of a gasket.

Access

```
import material
```

```

mdb.models [name] .materials [name] .gasketTransverseShearElastic
import odbMaterial
session.odbs [name] .materials [name] .gasketTransverseShearElastic

```

29.57.1 GasketTransverseShearElastic(...)

This method creates a GasketTransverseShearElastic object.

Path

```

mdb.models [name] .materials [name] .GasketTransverseShearElastic
session.odbs [name] .materials [name] .GasketTransverseShearElastic

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

variableUnits

A SymbolicConstant specifying the unit system in which the transverse shear behavior will be defined. Possible values are STRESS and FORCE. The default value is STRESS.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Shear stiffness. (This value cannot be negative.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GasketTransverseShearElastic object.

Exceptions

RangeError.

29.57.2 **setValues(...)**

This method modifies the `GasketTransverseShearElastic` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `GasketTransverseShearElastic` method.

Return value

None

Exceptions

`RangeError`.

29.57.3 **Members**

The `GasketTransverseShearElastic` object has members with the same names and descriptions as the arguments to the `GasketTransverseShearElastic` method.

29.57.4 **Corresponding analysis keywords**

*GASKET ELASTICITY

29.58 **Gel object**

The Gel object defines a swelling gel.

Access

```
import material
mdb.models[name].materials[name].gel
import odbMaterial
session.odbs[name].materials[name].gel
```

29.58.1 Gel(...)

This method creates a Gel object.

Path

```
mdb.models[name].materials[name].Gel  
session.odbs[name].materials[name].Gel
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- Radius of gel particles when completely dry, r_a^{dry} .
- Fully swollen radius of gel particles, r_a^f .
- Number of gel particles per unit volume, k_a .
- Relaxation time constant for long-term swelling of gel particles, τ_1 .

Return value

A Gel object.

Exceptions

None.

29.58.2 setValues(...)

This method modifies the Gel object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Gel** method.

Return value

None

Exceptions

None.

29.58.3 Members

The Gel object has members with the same names and descriptions as the arguments to the **Gel** method.

29.58.4 Corresponding analysis keywords

*GEL

29.59 HeatGeneration object

The HeatGeneration object includes volumetric heat generation in heat transfer analyses.

Access

```
import material
mdb.models[name].materials[name].heatGeneration
import odbMaterial
session.odbs[name].materials[name].heatGeneration
```

29.59.1 HeatGeneration()

This method creates a HeatGeneration object.

Path

```
mdb.models[name].materials[name].HeatGeneration
session.odbs[name].materials[name].HeatGeneration
```

Arguments

None.

Return value

A HeatGeneration object.

Exceptions

None.

29.59.2 Members

The HeatGeneration object has no members.

29.59.3 Corresponding analysis keywords

*HEAT GENERATION

29.60 Hyperelastic object

The Hyperelastic object specifies elastic properties for approximately incompressible elastomers.

Access

```
import material
mdb.models[name].materials[name].hyperelastic
import odbMaterial
session.odbs[name].materials[name].hyperelastic
```

29.60.1 Hyperelastic(...)

This method creates a Hyperelastic object.

Path

```
mdb.models[name].materials[name].Hyperelastic
session.odbs[name].materials[name].Hyperelastic
```

Required argument*table*

A sequence of sequences of Floats specifying the items described below. This argument is valid only if *testData*=OFF.

Optional arguments*type*

A SymbolicConstant specifying the type of strain energy potential. Possible values are:

- ARRUDA_BOYCE
- MARLOW
- MOONEY_RIVLIN
- NEO_HOOKE
- OGDEN
- POLYNOMIAL
- REDUCED_POLYNOMIAL
- USER
- VAN_DER_WAALS
- YEOH
- UNKNOWN

The default value is UNKNOWN.

moduliTimeScale

A SymbolicConstant specifying the nature of the time response. Possible values are INSTANTANEOUS and LONG_TERM. The default value is LONG_TERM.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

n

An Int specifying the order of the strain energy potential. The default value is 1.

If *testData*=ON and *type*=POLYNOMIAL, *n* can take only the values 1 or 2.

If *testData*=ON and *type*=OGDEN or if *testData*=OFF for either type, $1 \leq n \leq 6$.

If *type*=USER, this argument cannot be used.

beta

The SymbolicConstant FITTED_VALUE or a Float specifying the invariant mixture parameter. This argument is valid only if *testData*=ON and *type*=VAN_DER_WAALS. The default value is FITTED_VALUE.

testData

A Boolean specifying whether test data are supplied. The default value is ON.

MATERIAL COMMANDS

compressible

A Boolean specifying whether the hyperelastic material is compressible. This parameter is applicable only to user-defined hyperelastic materials. The default value is OFF.

properties

An Int specifying the number of property values needed as data for the user-defined hyperelastic material. The default value is 0.

deviatoricResponse

A SymbolicConstant specifying which test data to use. Possible values are UNIAXIAL, BIAXIAL, and PLANAR. The default value is UNIAXIAL.

volumetricResponse

A SymbolicConstant specifying the volumetric response. Possible values are DEFAULT, VOLUMETRIC_DATA, POISSON_RATIO, and LATERAL_NOMINAL_STRAIN. The default value is DEFAULT.

poissonRatio

A Float specifying the poisson ratio. This argument is valid only if *volumetricResponse*=POISSON_RATIO. The default value is 0.0.

materialType

A SymbolicConstant specifying the type of material. Possible values are ISOTROPIC and ANISOTROPIC. The default value is ISOTROPIC.

anisotropicType

A SymbolicConstant specifying the type of strain energy potential. Possible values are FUNG_ANISOTROPIC, FUNG_ORTHOTROPIC, HOLZAPFEL, and USER_DEFINED. The default value is FUNG_ANISOTROPIC.

formulation

A SymbolicConstant specifying the type of formulation. Possible values are STRAIN and INVARIANT. The default value is STRAIN.

behaviorType

A SymbolicConstant specifying the type of anisotropic hyperelastic material behavior. Possible values are INCOMPRESSIBLE and COMPRESSIBLE. The default value is INCOMPRESSIBLE.

dependencies

An Int specifying the number of field variable dependencies for the data in *volumetricTable*. The default value is 0.

localDirections

An Int specifying the number of local directions for the data in *volumetricTable*. The default value is 0.

Table data

If *type*=ARRUDA_BOYCE, the table data specify the following:

- μ .
- λ_m .

- D .
- Temperature, if the data depend on temperature.

If *type*=MOONEY_RIVLIN, the table data specify the following:

- C_{10} .
- C_{01} .
- D_1 .
- Temperature, if the data depend on temperature.

If *type*=NEO_HOOKE, the table data specify the following:

- C_{10} .
- D_1 .
- Temperature, if the data depend on temperature.

If *type*=OGDEN, the table data specify the following for values of n :

- μ_i and α_i for i from 1 to n .
- n coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=POLYNOMIAL, the table data specify the following for values of n :

- C_{ij} for each value of $(i + j)$ from 1 to n with i decreasing from $(i + j)$ to zero and j increasing from zero to $(i + j)$.
- n coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $3 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=REDUCED_POLYNOMIAL, the table data specify the following for values of n :

- C_{i0} for i from 1 to n .
- n coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=VAN_DER_WAALS, the table data specify the following:

- μ .
- λ_m .
- a .
- β .
- D .
- Temperature, if the data depend on temperature.

If *type*=YEOH, the table data specify the following:

MATERIAL COMMANDS

- C_{10} .
- C_{20} .
- C_{30} .
- D_1 .
- D_2 .
- D_3 .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed in an Abaqus/Explicit analysis.

The **None** object is the default value if *testData*=ON.

Return value

A Hyperelastic object.

Exceptions

InvalidNameError and RangeError.

29.60.2 **setValues(...)**

This method modifies the Hyperelastic object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Hyperelastic** method.

Return value

None

Exceptions

RangeError.

29.60.3 **Members**

The Hyperelastic object has members with the same names and descriptions as the arguments to the **Hyperelastic** method.

In addition, the Hyperelastic object can have the following members:

biaxialTestData

A BiaxialTestData object.

planarTestData

A PlanarTestData object.

uniaxialTestData

A UniaxialTestData object.

volumetricTestData

A VolumetricTestData object.

hysteresis

A Hysteresis object.

29.60.4 Corresponding analysis keywords

*HYPERELASTIC

29.61 Hyperfoam object

The Hyperfoam object specifies elastic properties for a hyperelastic foam.

Access

```
import material
mdb.models[name].materials[name].hyperfoam
import odbMaterial
session.odbs[name].materials[name].hyperfoam
```

29.61.1 Hyperfoam(...)

This method creates a Hyperfoam object.

Path

```
mdb.models[name].materials[name].Hyperfoam
session.odbs[name].materials[name].Hyperfoam
```

Required arguments

None.

Optional arguments

testData

A Boolean specifying whether test data are supplied. The default value is OFF.

poisson

None or a Float specifying the effective Poisson's ratio, ν , of the material. This argument is valid only when *testData*=ON. The default value is **None**.

n

An Int specifying the order of the strain energy potential. Possible values are $1 \leq n \leq 6$. The default value is 1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

moduli

A SymbolicConstant specifying the time-dependence of the material constants. Possible values are INSTANTANEOUS and LONG_TERM. The default value is LONG_TERM.

table

A sequence of sequences of Floats specifying the items described below. This argument is valid only when *testData*=OFF. The default value is an empty sequence.

Table data

The items in the table data specify the following for values of *n*:

- μ_i and α_i for *i* from 1 to *n*.
- ν_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

Return value

A Hyperfoam object.

Exceptions

RangeError.

29.61.2 setValues(...)

This method modifies the Hyperfoam object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Hyperfoam** method.

Return value

None

Exceptions

RangeError.

29.61.3 Members

The Hyperfoam object has members with the same names and descriptions as the arguments to the **Hyperfoam** method.

In addition, the Hyperfoam object can have the following members:

biaxialTestData

A BiaxialTestData object.

volumetricTestData

A VolumetricTestData object.

planarTestData

A PlanarTestData object.

simpleShearTestData

A SimpleShearTestData object.

uniaxialTestData

A UniaxialTestData object.

29.61.4 Corresponding analysis keywords

*HYPERFOAM

29.62 Hypoelastic object

The Hypoelastic object specifies hypoelastic material properties.

Access

```
import material
mdb.models[name].materials[name].hypoelastic
```

```
import odbMaterial
session.odbs[name].materials[name].hypoelastic
```

29.62.1 Hypoelastic(...)

This method creates a Hypoelastic object.

Path

```
mdb.models[name].materials[name].Hypoelastic
session.odbs[name].materials[name].Hypoelastic
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional argument

user

A Boolean specifying that hypoelasticity is defined by user subroutine **UHYPEL**. The default value is OFF.

Table data

- Instantaneous Young's modulus, E .
- Instantaneous Poisson's ratio, ν .
- First strain invariant, I_1 .
- Second strain invariant, I_2 .
- Third strain invariant, I_3 .

Return value

A Hypoelastic object.

Exceptions

None.

29.62.2 setValues(...)

This method modifies the Hypoelastic object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Hypoelastic** method.

Return value

None

Exceptions

None.

29.62.3 Members

The Hypoelastic object has members with the same names and descriptions as the arguments to the **Hypoelastic** method.

29.62.4 Corresponding analysis keywords

*HYPOELASTIC

29.63 Hysteresis object

The Hysteresis object specifies the creep part of the material model for the hysteretic behavior of elastomers.

Access

```
import material
mdb.models[name].materials[name].hyperelastic.hysteresis
import odbMaterial
session.odbs[name].materials[name].hyperelastic.hysteresis
```

29.63.1 Hysteresis(...)

This method creates a Hysteresis object.

Path

```
mdb.models[name].materials[name].hyperelastic.Hysteresis  
session.odbs[name].materials[name].hyperelastic.Hysteresis
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- Stress scaling factor.
- Creep parameter.
- Effective stress exponent.
- Creep strain exponent.

Return value

A Hysteresis object.

Exceptions

RangeError.

29.63.2 **setValues(...)**

This method modifies the Hysteresis object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Hysteresis** method.

Return value

None

Exceptions

RangeError.

29.63.3 Members

The Hysteresis object has members with the same names and descriptions as the arguments to the `Hysteresis` method.

29.63.4 Corresponding analysis keywords

*HYSTERESIS

29.64 InelasticHeatFraction object

The InelasticHeatFraction object defines the fraction of the rate of inelastic dissipation that appears as a heat source.

Access

```
import material
mdb.models[name].materials[name].inelasticHeatFraction
import odbMaterial
session.odbs[name].materials[name].inelasticHeatFraction
```

29.64.1 InelasticHeatFraction(...)

This method creates an InelasticHeatFraction object.

Path

```
mdb.models[name].materials[name].InelasticHeatFraction
session.odbs[name].materials[name].InelasticHeatFraction
```

Required arguments

None.

Optional argument

fraction

A Float specifying the fraction of inelastic dissipation rate that appears as a heat flux per unit volume. The fraction may include a unit conversion factor if required. Possible values are $0.0 \leq \text{fraction} \leq 1.0$. The default value is 0.9.

Return value

An InelasticHeatFraction object.

Exceptions

RangeError.

29.64.2 **setValues(...)**

This method modifies the InelasticHeatFraction object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **InelasticHeatFraction** method.

Return value

None

Exceptions

RangeError.

29.64.3 **Members**

The InelasticHeatFraction object has members with the same names and descriptions as the arguments to the **InelasticHeatFraction** method.

29.64.4 **Corresponding analysis keywords**

*INELASTIC HEAT FRACTION

29.65 JouleHeatFraction object

The JouleHeatFraction object defines the fraction of electric energy released as heat.

Access

```
import material
mdb.models[name].materials[name].jouleHeatFraction
import odbMaterial
session.odbs[name].materials[name].jouleHeatFraction
```

29.65.1 JouleHeatFraction(...)

This method creates a JouleHeatFraction object.

Path

```
mdb.models[name].materials[name].JouleHeatFraction
session.odbs[name].materials[name].JouleHeatFraction
```

Required arguments

None.

Optional argument

fraction

A Float specifying the fraction of electrical energy released as heat, including any unit conversion factor. Possible values are $0.0 \leq \textit{fraction} \leq 1.0$. The default value is 1.0.

Return value

A JouleHeatFraction object.

Exceptions

RangeError.

29.65.2 setValues(...)

This method modifies the JouleHeatFraction object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **JouleHeatFraction** method.

Return value

None

Exceptions

RangeError.

29.65.3 Members

The **JouleHeatFraction** object has members with the same names and descriptions as the arguments to the **JouleHeatFraction** method.

29.65.4 Corresponding analysis keywords

*JOULE HEAT FRACTION

29.66 LatentHeat object

The **LatentHeat** object specifies a material's latent heat.

Access

```
import material
mdb.models[name].materials[name].latentHeat
import odbMaterial
session.odbs[name].materials[name].latentHeat
```

29.66.1 LatentHeat(...)

This method creates a **LatentHeat** object.

Path

```

mdb.models[name].materials[name].LatentHeat
session.odbs[name].materials[name].LatentHeat

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- Latent heat per unit mass.
- Solidus temperature.
- Liquidus temperature.

Return value

A LatentHeat object.

Exceptions

RangeError.

29.66.2 setValues(...)

This method modifies the LatentHeat object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LatentHeat** method.

Return value

None

Exceptions

RangeError.

29.66.3 Members

The `LatentHeat` object has members with the same names and descriptions as the arguments to the `LatentHeat` method.

29.66.4 Corresponding analysis keywords

*LATENT HEAT

29.67 LowDensityFoam object

The `LowDensityFoam` object specifies properties for low-density foam.

Access

```
import material
mdb.models[name].materials[name].lowDensityFoam
import odbMaterial
session.odbs[name].materials[name].lowDensityFoam
```

29.67.1 LowDensityFoam(...)

This method creates a `LowDensityFoam` object.

Path

```
mdb.models[name].materials[name].LowDensityFoam
session.odbs[name].materials[name].LowDensityFoam
```

Required arguments

None.

Optional arguments

elementRemoval

A Boolean specifying whether elements are removed if exceeding maximum principal tensile stress. This argument is valid only when *maxAllowablePrincipalStress* is defined. The default value is OFF.

maxAllowablePrincipalStress

None or a Float specifying the maximum allowable principal tensile stress. The default value is **None**.

extrapolateStressStrainCurve

A Boolean specifying whether the stress-strain curve is extrapolated if exceeding maximum strain rate. The default value is OFF.

strainRateType

A SymbolicConstant specifying strain rate measure used for constitutive calculations. Possible values are PRINCIPAL and VOLUMETRIC. The default value is VOLUMETRIC.

mu0

A Float specifying the relaxation coefficient μ_0 . The default value is 10^{-4} .

mu1

A Float specifying the relaxation coefficient μ_1 . The default value is 0.5×10^{-2} .

alpha

A Float specifying the relaxation coefficient α . The default value is 2.0.

Return value

A LowDensityFoam object.

Exceptions

RangeError.

29.67.2 setValues(...)

This method modifies the LowDensityFoam object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LowDensityFoam** method.

Return value

None

Exceptions

RangeError.

29.67.3 Members

The LowDensityFoam object has members with the same names and descriptions as the arguments to the **LowDensityFoam** method.

In addition, the LowDensityFoam object can have the following members:

uniaxialTensionTestData

A UniaxialTestData object.

uniaxialCompressionTestData

A UniaxialTestData object.

29.67.4 Corresponding analysis keywords

*LOW DENSITY FOAM

29.68 MagneticPermeability object

The MagneticPermeability object specifies magnetic permeability.

Access

```
import material
mdb.models[name].materials[name].magneticPermeability
import odbMaterial
session.odbs[name].materials[name].magneticPermeability
```

29.68.1 MagneticPermeability(...)

This method creates a MagneticPermeability object.

Path

```
mdb.models[name].materials[name].MagneticPermeability
session.odbs[name].materials[name].MagneticPermeability
```


Required arguments*table*

A sequence of sequences of Floats specifying the items described below in “Table data.” If *type*=ORTHOTROPIC and *nonlinearBH*=ON, the data specified in the *table* is for the first direction and *table2* and *table3* must be specified.

table2

A sequence of sequences of Floats specifying the items described below in “Table data” in the second direction. *table2* must be specified only if *type*=ORTHOTROPIC and *nonlinearBH*=ON.

table3

A sequence of sequences of Floats specifying the items described below in “Table data” in the third direction. *table3* must be specified only if *type*=ORTHOTROPIC and *nonlinearBH*=ON.

Optional arguments*type*

A SymbolicConstant specifying the type of magnetic permeability. Possible values are ISOTROPIC, ORTHOTROPIC, and ANISOTROPIC. The default value is ISOTROPIC.

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

nonlinearBH

A Boolean specifying whether the magnetic behavior is nonlinear and available in tabular form of magnetic flux density versus magnetic field values. The default value is OFF.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Magnetic permeability.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ISOTROPIC, and *nonlinearBH*=TRUE, the table data specify the following:

- Magnitude of the magnetic flux density vector.
- Magnitude of the magnetic field vector.
- Temperature, if the data depend on temperature.

MATERIAL COMMANDS

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- μ_{11}^E .
- μ_{22}^E .
- μ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, and *nonlinearBH*=TRUE, the table data specify the following:

- Magnitude of the magnetic flux density vector in the first direction.
- Magnitude of the magnetic field vector in the second direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- μ_{11}^E .
- μ_{12}^E .
- μ_{22}^E .
- μ_{13}^E .
- μ_{23}^E .
- μ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MagneticPermeability object.

Exceptions

RangeError.

29.68.2 setValues(...)

This method modifies the MagneticPermeability object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MagneticPermeability** method.

Return value

None

Exceptions

RangeError.

29.68.3 Members

The MagneticPermeability object has members with the same names and descriptions as the arguments to the **MagneticPermeability** method.

29.68.4 Corresponding analysis keywords

*MAGNETIC PERMEABILITY

29.69 MohrCoulombHardening object

The MohrCoulombHardening object specifies hardening for the Mohr-Coulomb plasticity model.

Access

import material

```
mdb.models[name].materials[name].mohrCoulombPlasticity\  
.mohrCoulombHardening  
import odbMaterial  
session.odbs[name].materials[name].mohrCoulombPlasticity\  
.mohrCoulombHardening
```

29.69.1 MohrCoulombHardening(...)

This method creates a MohrCoulombHardening object.

Path

```
mdb.models[name].materials[name].mohrCoulombPlasticity\  
.MohrCoulombHardening  
session.odbs[name].materials[name].mohrCoulombPlasticity\  
.MohrCoulombHardening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Cohesion yield stress.
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MohrCoulombHardening object.

Exceptions

RangeError.

29.69.2 setValues(...)

This method modifies the MohrCoulombHardening object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MohrCoulombHardening** method.

Return value

None

Exceptions

RangeError.

29.69.3 Members

The MohrCoulombHardening object has members with the same names and descriptions as the arguments to the **MohrCoulombHardening** method.

29.69.4 Corresponding analysis keywords

*MOHR COULOMB HARDENING

29.70 MohrCoulombPlasticity object

The MohrCoulombPlasticity object specifies the extended Mohr-Coulomb plasticity model.

Access

`import material`

```
mdb.models[name].materials[name].mohrCoulombPlasticity
import odbMaterial
session.odbs[name].materials[name].mohrCoulombPlasticity
```

29.70.1 MohrCoulombPlasticity(...)

This method creates a MohrCoulombPlasticity object.

Path

```
mdb.models[name].materials[name].MohrCoulombPlasticity
session.odbs[name].materials[name].MohrCoulombPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

deviatoricEccentricity

None or a Float specifying the flow potential eccentricity in the deviatoric plane, e ; $1/2 \leq e \leq 1.0$. If *deviatoricEccentricity*=**None**, Abaqus calculates the value using the specified Mohr-Coulomb angle of friction. The default value is **None**.

meridionalEccentricity

A Float specifying the flow potential eccentricity in the meridional plane, ϵ . The default value is 0.1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

useTensionCutoff

A Boolean specifying whether tension cutoff specification is needed. The default value is OFF.

Table data

The table data specify the following:

- Friction angle (given in degrees), ϕ , at high confining pressure in the p - $R_{mc}q$ plane.
- Dilation angle, ψ , at high confining pressure in the p - $R_{mw}q$ plane.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MohrCoulombPlasticity object.

Exceptions

RangeError.

29.70.2 setValues(...)

This method modifies the MohrCoulombPlasticity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MohrCoulombPlasticity** method.

Return value

None

Exceptions

RangeError.

29.70.3 Members

The MohrCoulombPlasticity object has members with the same names and descriptions as the arguments to the **MohrCoulombPlasticity** method.

In addition, the MohrCoulombPlasticity object can have the following members:

mohrCoulombHardening

A MohrCoulombHardening object.

tensionCutOff

A TensionCutOff object.

29.70.4 Corresponding analysis keywords

*MOHR COULOMB

29.71 MoistureSwelling object

The MoistureSwelling object defines moisture-driven swelling.

Access

```
import material
mdb.models[name].materials[name].moistureSwelling
import odbMaterial
session.odbs[name].materials[name].moistureSwelling
```

29.71.1 MoistureSwelling(...)

This method creates a MoistureSwelling object.

Path

```
mdb.models[name].materials[name].MoistureSwelling
session.odbs[name].materials[name].MoistureSwelling
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- Volumetric moisture swelling strain, ε^{ms} .
- Saturation, s . This value must lie in the range $0.0 \leq s \leq 1.0$.

Return value

A MoistureSwelling object.

Exceptions

None.

29.71.2 **setValues(...)**

This method modifies the `MoistureSwelling` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MoistureSwelling** method.

Return value

None

Exceptions

None.

29.71.3 **Members**

The `MoistureSwelling` object has members with the same names and descriptions as the arguments to the **MoistureSwelling** method.

In addition, the `MoistureSwelling` object can have the following member:

ratios

A `Ratios` object.

29.71.4 **Corresponding analysis keywords**

*MOISTURE SWELLING

29.72 **MullinsEffect object**

The `MullinsEffect` specifies properties for mullins data.

Access

```
import material
mdb.models[name].materials[name].mullinsEffect
```

MATERIAL COMMANDS

```
import odbMaterial
session.odbs[name].materials[name].mullinsEffect
```

29.72.1 Members

The MullinsEffect object can have the following members:

definition

A SymbolicConstant specifying the method of specifying the data. Possible values are USER, CONSTANTS, and TEST_DATA. The default value is CONSTANTS.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

properties

An Int specifying the number of property values needed as data for the user-defined hyperelastic material. The default value is 0.

table

A tuple of tuples of Floats specifying the items described below. The default value is an empty sequence.

uniaxialTests

A UniaxialTestDataArray object.

biaxialTests

A BiaxialTestDataArray object.

planarTests

A PlanarTestDataArray object.

29.73 Ornl object

The Ornl object specifies the constitutive model developed by Oak Ridge National Laboratory.

Access

```
import material
mdb.models[name].materials[name].creep.ornl
mdb.models[name].materials[name].plastic.ornl
import odbMaterial
session.odbs[name].materials[name].creep.ornl
session.odbs[name].materials[name].plastic.ornl
```

29.73.1 Ornl(...)

This method creates an Ornl object.

Path

```

mdb.models[name].materials[name].creep.Ornl
mdb.models[name].materials[name].plastic.Ornl
session.odbs[name].materials[name].creep.Ornl
session.odbs[name].materials[name].plastic.Ornl

```

Required arguments

None.

Optional arguments

a

A Float specifying the saturation rates for kinematic shift caused by creep strain, as defined by Equation (15) of Section 4.3.3–3 of the Nuclear Standard. The default value corresponds to that section of the Standard. Set $a=0.0$ to use the 1986 revision of the Standard. The default value is 0.3.

h

None or a Float specifying the rate of kinematic shift with respect to creep strain [Equation (7) of Section 4.3.2–1 of the Nuclear Standard]. If $h=\mathbf{None}$, the value of h is determined according to Section 4.3.3–3 of the 1981 revision of the Standard. Set $h=0.0$ to use the 1986 revision of the Standard. The default value is **None**.

reset

A Boolean specifying whether to invoke the optional α reset procedure described in Section 4.3.5 of the Nuclear Standard. The default value is OFF.

Return value

An Ornl object.

Exceptions

RangeError.

29.73.2 setValues(...)

This method modifies the Ornl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Ornl** method.

Return value

None

Exceptions

RangeError.

29.73.3 Members

The Ornl object has members with the same names and descriptions as the arguments to the **Ornl** method.

29.73.4 Corresponding analysis keywords

*ORNL

29.74 Permeability object

The Permeability object defines permeability for pore fluid flow.

Access

```
import material
mdb.models[name].materials[name].permeability
import odbMaterial
session.odbs[name].materials[name].permeability
```

29.74.1 Permeability(...)

This method creates a Permeability object.

Path

`mdb.models[name].materials[name].Permeability`
`session.odbs[name].materials[name].Permeability`

Required arguments*specificWeight*

A Float specifying the specific weight of the wetting liquid, γ_w .

inertialDragCoefficient

A Float specifying The inertial drag coefficient of the wetting liquid, γ_w .

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments*type*

A SymbolicConstant specifying the type of permeability. Possible values are ISOTROPIC, ORTHOTROPIC, ANISOTROPIC, ISOTROPIC-CFD, and CARMAN_KOZENY. The default value is ISOTROPIC.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- k .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If *type*=ORTHOTROPIC, the table data specify the following:

- k_{11} .
- k_{22} .
- k_{33} .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If *type*=ANISOTROPIC, the table data specify the following:

- k_{11} .
- k_{12} .
- k_{22} .
- k_{13} .

MATERIAL COMMANDS

- k_{23} .
- k_{33} .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If `type=ISOTROPIC_CFD`, the table data specify the following:

- k .
- Porosity, e .

If `type=CARMAN_KOZENY`, the table data specify the following:

- Kozeny constant e .
- Pore particle radius, e .

Return value

A Permeability object.

Exceptions

RangeError.

29.74.2 `setValues(...)`

This method modifies the Permeability object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `Permeability` method.

Return value

None

Exceptions

RangeError.

29.74.3 `Members`

The Permeability object has members with the same names and descriptions as the arguments to the `Permeability` method.

In addition, the Permeability object can have the following members:

saturationDependence

A SaturationDependence object specifying the dependence of the permeability of a material on the saturation of the wetting liquid.

velocityDependence

A VelocityDependence object specifying the dependence of the permeability of a material on the velocity of fluid flow.

29.74.4 Corresponding analysis keywords

*PERMEABILITY

29.75 Piezoelectric object

The Piezoelectric object specifies piezoelectric material properties.

Access

```
import material
mdb.models[name].materials[name].piezoelectric
import odbMaterial
session.odbs[name].materials[name].piezoelectric
```

29.75.1 Piezoelectric(...)

This method creates a Piezoelectric object.

Path

```
mdb.models[name].materials[name].Piezoelectric
session.odbs[name].materials[name].Piezoelectric
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of material coefficients for the piezoelectric property. Possible values are STRAIN and STRESS. The default value is STRESS.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRESS, the table data specify the following:

- e_{111}^{φ} .
- e_{122}^{φ} .
- e_{133}^{φ} .
- e_{112}^{φ} .
- e_{113}^{φ} .
- e_{123}^{φ} .
- e_{211}^{φ} .
- e_{222}^{φ} .
- e_{233}^{φ} .
- e_{212}^{φ} .
- e_{213}^{φ} .
- e_{223}^{φ} .
- e_{311}^{φ} .
- e_{322}^{φ} .
- e_{333}^{φ} .
- e_{312}^{φ} .
- e_{313}^{φ} .
- e_{323}^{φ} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=STRAIN, the table data specify the following:

- d_{111}^{φ} .
- d_{122}^{φ} .
- d_{133}^{φ} .

- $d_{1\ 12}^{\varphi}$.
- $d_{1\ 13}^{\varphi}$.
- $d_{1\ 23}^{\varphi}$.
- $d_{2\ 11}^{\varphi}$.
- $d_{2\ 22}^{\varphi}$.
- $d_{2\ 33}^{\varphi}$.
- $d_{2\ 12}^{\varphi}$.
- $d_{2\ 13}^{\varphi}$.
- $d_{2\ 23}^{\varphi}$.
- $d_{3\ 11}^{\varphi}$.
- $d_{3\ 22}^{\varphi}$.
- $d_{3\ 33}^{\varphi}$.
- $d_{3\ 13}^{\varphi}$.
- $d_{3\ 23}^{\varphi}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Piezoelectric object.

Exceptions

None.

29.75.2 setValues(...)

This method modifies the Piezoelectric object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Piezoelectric** method.

Return value

None

Exceptions

None.

29.75.3 Members

The Piezoelectric object has members with the same names and descriptions as the arguments to the `Piezoelectric` method.

29.75.4 Corresponding analysis keywords

*PIEZOELECTRIC

29.76 PlanarTestData object

The `PlanarTestData` object specifies planar test (or pure shear) data (compression and/or tension).

Access

```
import material
mdb.models[name].materials[name].hyperelastic.planarTestData
mdb.models[name].materials[name].hyperfoam.planarTestData
mdb.models[name].materials[name].mullinsEffect.planarTests[i]
import odbMaterial
session.odbs[name].materials[name].hyperelastic.planarTestData
session.odbs[name].materials[name].hyperfoam.planarTestData
session.odbs[name].materials[name].mullinsEffect.planarTests[i]
```

29.76.1 PlanarTestData(...)

This method creates a `PlanarTestData` object.

Path

```
mdb.models[name].materials[name].hyperelastic.PlanarTestData
mdb.models[name].materials[name].hyperfoam.PlanarTestData
```

```

mdb.models [name] .materials [name] .mullinsEffect.PlanarTestData
session.odbs [name] .materials [name] .hyperelastic.PlanarTestData
session.odbs [name] .materials [name] .hyperfoam.PlanarTestData
session.odbs [name] .materials [name] .mullinsEffect.PlanarTestData

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

smoothing

None or an Int specifying the value for smoothing. If *smoothing=None*, no smoothing is employed. The default value is **None**.

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic material model, the table data specify the following:

- Nominal stress, T_S .
- Nominal strain in the direction of loading, ϵ_S .

For a hyperfoam material model, the table data specify the following:

- Nominal stress, T_L .
- Nominal strain in the direction of loading, ϵ_p .
- Nominal transverse strain, ϵ_3 . The default value is 0.

Return value

A PlanarTestData object.

Exceptions

None.

29.76.2 setValues(...)

This method modifies the PlanarTestData object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PlanarTestData** method.

Return value

None

Exceptions

None.

29.76.3 Members

The **PlanarTestData** object has members with the same names and descriptions as the arguments to the **PlanarTestData** method.

29.76.4 Corresponding analysis keywords

*PLANAR TEST DATA

29.77 Plastic object

The Plastic object specifies a metal plasticity model.

Access

```
import material
mdb.models[name].materials[name].plastic
import odbMaterial
session.odbs[name].materials[name].plastic
```

29.77.1 Plastic(...)

This method creates a Plastic object.

Path

```
mdb.models[name].materials[name].Plastic
session.odbs[name].materials[name].Plastic
```

Required argument*table*

A sequence of sequences of Floats specifying the items described below.

Optional arguments*hardening*

A SymbolicConstant specifying the type of hardening. Possible values are ISOTROPIC, KINEMATIC, COMBINED, JOHNSON_COOK, and USER. The default value is ISOTROPIC.

rate

A Boolean specifying whether the data depend on rate. The default value is OFF.

dataType

A SymbolicConstant specifying the type of combined hardening. This argument is only valid if *hardening*=COMBINED. Possible values are HALF_CYCLE, PARAMETERS, and STABILIZED. The default value is HALF_CYCLE.

strainRangeDependency

A Boolean specifying whether the data depend on strain range. This argument is only valid if *hardening*=COMBINED and *dataType*=STABILIZED. The default value is OFF.

numBackstresses

An Int specifying the number of backstresses. This argument is only valid if *hardening*=COMBINED. The default value is 1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=ISOTROPIC, or if *hardening*=COMBINED and *dataType*=HALF_CYCLE, the table data specify the following:

- Yield stress.
- Plastic strain.
- Equivalent plastic strain rate, $\dot{\epsilon}^{pl}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

MATERIAL COMMANDS

If *hardening*=COMBINED and *dataType*=STABILIZED, the table data specify the following:

- Yield stress.
- Plastic strain.
- Strain range, if the data depend on strain range.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=COMBINED and *dataType*=PARAMETERS, the table data specify the following:

- Yield stress at zero plastic strain.
- The first kinematic hardening parameter, C_1 .
- The first kinematic hardening parameter, γ_1 .
- If applicable, the second kinematic hardening parameter, C_2 .
- If applicable, the second kinematic hardening parameter, γ_2 .
- Etc.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=KINEMATIC, the table data specify the following:

- Yield stress.
- Plastic strain.
- Temperature, if the data depend on temperature.

If *hardening*=JOHNSON_COOK, the table data specify the following:

- A.
- B.
- n.
- m.
- Melting temperature.
- Transition temperature.

If *hardening*=USER, the table data specify the following:

- Hardening properties.

Return value

A Plastic object.

Exceptions

RangeError.

29.77.2 setValues(...)

This method modifies the Plastic object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Plastic** method.

Return value

None

Exceptions

RangeError.

29.77.3 Members

The Plastic object has members with the same names and descriptions as the arguments to the **Plastic** method.

In addition, the Plastic object can have the following members:

rateDependent

A RateDependent object.

potential

A Potential object.

cyclicHardening

A CyclicHardening object.

ornl

An Ornl object.

cycledPlastic

A CycledPlastic object.

annealTemperature

An AnnealTemperature object.

29.77.4 Corresponding analysis keywords

*PLASTIC

29.78 PoreFluidExpansion object

The PoreFluidExpansion object specifies the thermal expansion coefficient for a hydraulic fluid.

Access

```
import material
mdb.models[name].materials[name].poreFluidExpansion
import odbMaterial
session.odbs[name].materials[name].poreFluidExpansion
```

29.78.1 PoreFluidExpansion(...)

This method creates a PoreFluidExpansion object.

Path

```
mdb.models[name].materials[name].PoreFluidExpansion
session.odbs[name].materials[name].PoreFluidExpansion
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

zero

A Float specifying the value of θ^0 . The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Mean coefficient of thermal expansion, α .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PoreFluidExpansion object.

Exceptions

RangeError.

29.78.2 setValues(...)

This method modifies the PoreFluidExpansion object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PoreFluidExpansion** method.

Return value

None

Exceptions

RangeError.

29.78.3 Members

The PoreFluidExpansion object has members with the same names and descriptions as the arguments to the **PoreFluidExpansion** method.

29.78.4 Corresponding analysis keywords

*EXPANSION

29.79 PorousBulkModuli object

The PorousBulkModuli object defines bulk moduli for soils and rocks.

Access

```
import material
mdb.models[name].materials[name].porousBulkModuli
import odbMaterial
session.odbs[name].materials[name].porousBulkModuli
```

29.79.1 PorousBulkModuli(...)

This method creates a PorousBulkModuli object.

Path

```
mdb.models[name].materials[name].PorousBulkModuli
session.odbs[name].materials[name].PorousBulkModuli
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional argument

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

Table data

- Bulk modulus of solid grains.
- Bulk modulus of permeating fluid.
- Temperature, if the data depend on temperature.

Return value

A PorousBulkModuli object.

Exceptions

None.

29.79.2 setValues(...)

This method modifies the `PorousBulkModuli` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `PorousBulkModuli` method.

Return value

None

Exceptions

None.

29.79.3 Members

The `PorousBulkModuli` object has members with the same names and descriptions as the arguments to the `PorousBulkModuli` method.

29.79.4 Corresponding analysis keywords

*POROUS BULK MODULI

29.80 PorousElastic object

The `PorousElastic` object specifies elastic material properties for porous materials.

Access

`import material`

```
mdb.models[name].materials[name].porousElastic
import odbMaterial
session.odbs[name].materials[name].porousElastic
```

29.80.1 PorousElastic(...)

This method creates a PorousElastic object.

Path

```
mdb.models[name].materials[name].PorousElastic
session.odbs[name].materials[name].PorousElastic
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

shear

A SymbolicConstant specifying the shear definition form. Possible values are G and POISSON. The default value is POISSON.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *shear*=G, the table data specify the following:

- The logarithmic bulk modulus, κ . (Dimensionless.)
- The shear modulus, G .
- The elastic tensile limit, p_t^{el} . (This value cannot be negative.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shear*=POISSON, the table data specify the following:

- The logarithmic bulk modulus, κ . (Dimensionless.)
- The Poisson's ratio, ν .
- The elastic tensile limit, p_t^{el} . (This value cannot be negative.)
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PorousElastic object.

Exceptions

RangeError.

29.80.2 setValues(...)

This method modifies the PorousElastic object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PorousElastic** method.

Return value

None

Exceptions

RangeError.

29.80.3 Members

The PorousElastic object has members with the same names and descriptions as the arguments to the **PorousElastic** method.

29.80.4 Corresponding analysis keywords

*POROUS ELASTIC

29.81 PorousFailureCriteria object

The PorousFailureCriteria object specifies the material failure criteria for a porous metal.

Access

```
import material
mdb.models[name].materials[name].porousMetalPlasticity\
.porousFailureCriteria
import odbMaterial
session.odbs[name].materials[name].porousMetalPlasticity\
.porousFailureCriteria
```

29.81.1 PorousFailureCriteria(...)

This method creates a PorousFailureCriteria object.

Path

```
mdb.models[name].materials[name].porousMetalPlasticity\
.PorousFailureCriteria
session.odbs[name].materials[name].porousMetalPlasticity\
.PorousFailureCriteria
```

Required arguments

None.

Optional arguments

fraction

A Float specifying the void volume fraction at total failure, $f_F > 0$. The default value is 1.0.

criticalFraction

A Float specifying the critical void volume fraction, $f_c \geq 0$. The default value is 1.0.

Return value

A PorousFailureCriteria object.

Exceptions

RangeError.

29.81.2 **setValues(...)**

This method modifies the `PorousFailureCriteria` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `PorousFailureCriteria` method.

Return value

None

Exceptions

`RangeError`.

29.81.3 **Members**

The `PorousFailureCriteria` object has members with the same names and descriptions as the arguments to the `PorousFailureCriteria` method.

29.81.4 **Corresponding analysis keywords**

*POROUS FAILURE CRITERIA

29.82 **PorousMetalPlasticity object**

The `PorousMetalPlasticity` object specifies a porous metal plasticity model.

Access

```
import material
mdb.models[name].materials[name].porousMetalPlasticity
import odbMaterial
session.odbs[name].materials[name].porousMetalPlasticity
```

29.82.1 PorousMetalPlasticity(...)

This method creates a PorousMetalPlasticity object.

Path

```
mdb.models[name].materials[name].PorousMetalPlasticity
session.odbs[name].materials[name].PorousMetalPlasticity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

relativeDensity

None or a Float specifying the initial relative density of the material, r_0 . The default value is **None**.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- q_1 .
- q_2 .
- q_3 .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PorousMetalPlasticity object.

Exceptions

RangeError.

29.82.2 **setValues(...)**

This method modifies the `PorousMetalPlasticity` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the `PorousMetalPlasticity` method.

Return value

None

Exceptions

`RangeError`.

29.82.3 **Members**

The `PorousMetalPlasticity` object has members with the same names and descriptions as the arguments to the `PorousMetalPlasticity` method.

In addition, the `PorousMetalPlasticity` object can have the following members:

porousFailureCriteria

A `PorousFailureCriteria` object.

voidNucleation

A `VoidNucleation` object.

29.82.4 **Corresponding analysis keywords**

*POROUS METAL PLASTICITY

29.83 **Potential object**

The `Potential` object defines an anisotropic yield/creep model.

Access

```
import material
mdb.models[name].materials[name].creep.potential
mdb.models[name].materials[name].plastic.potential
mdb.models[name].materials[name].viscous.potential
import odbMaterial
session.odbs[name].materials[name].creep.potential
session.odbs[name].materials[name].plastic.potential
session.odbs[name].materials[name].viscous.potential
```

29.83.1 Potential(...)

This method creates a Potential object.

Path

```
mdb.models[name].materials[name].creep.Potential
mdb.models[name].materials[name].plastic.Potential
mdb.models[name].materials[name].viscous.Potential
session.odbs[name].materials[name].creep.Potential
session.odbs[name].materials[name].plastic.Potential
session.odbs[name].materials[name].viscous.Potential
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- R_{11} .
- R_{22} .
- R_{33} .
- R_{12} .
- R_{13} .

- R_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Potential object.

Exceptions

RangeError.

29.83.2 setValues(...)

This method modifies the Potential object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Potential** method.

Return value

None

Exceptions

RangeError.

29.83.3 Members

The Potential object has members with the same names and descriptions as the arguments to the **Potential** method.

29.83.4 Corresponding analysis keywords

*POTENTIAL

29.84 PressureEffect object

The PressureEffect object defines equivalent pressure stress driven mass diffusion.

Access

```
import material
mdb.models[name].materials[name].diffusivity.pressureEffect
import odbMaterial
session.odbs[name].materials[name].diffusivity.pressureEffect
```

29.84.1 PressureEffect(...)

This method creates a PressureEffect object.

Path

```
mdb.models[name].materials[name].diffusivity.PressureEffect
session.odbs[name].materials[name].diffusivity.PressureEffect
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Pressure stress factor, κ_p .
- Concentration.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PressureEffect object.

Exceptions

RangeError.

29.84.2 setValues(...)

This method modifies the PressureEffect object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PressureEffect** method.

Return value

None

Exceptions

RangeError.

29.84.3 Members

The PressureEffect object has members with the same names and descriptions as the arguments to the **PressureEffect** method.

29.84.4 Corresponding analysis keywords

*KAPPA

29.85 RateDependent object

The RateDependent object defines a rate-dependent viscoplastic model.

Access

```
import material
mdb.models[name].materials[name].crushableFoam.rateDependent
mdb.models[name].materials[name].druckerPrager.rateDependent
mdb.models[name].materials[name].plastic.rateDependent
import odbMaterial
session.odbs[name].materials[name].crushableFoam.rateDependent
session.odbs[name].materials[name].druckerPrager.rateDependent
session.odbs[name].materials[name].plastic.rateDependent
```

29.85.1 RateDependent(...)

This method creates a RateDependent object.

Path

```
mdb.models[name].materials[name].crushableFoam.RateDependent
mdb.models[name].materials[name].druckerPrager.RateDependent
mdb.models[name].materials[name].plastic.RateDependent
session.odbs[name].materials[name].crushableFoam.RateDependent
session.odbs[name].materials[name].druckerPrager.RateDependent
session.odbs[name].materials[name].plastic.RateDependent
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of rate-dependent data. Possible values are POWER_LAW, YIELD_RATIO, and JOHNSON_COOK. The default value is POWER_LAW.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=POWER_LAW, the table data specify the following:

- *D*.
- *n*.
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=YIELD_RATIO, the table data specify the following:

- Yield stress ratio, $R = \bar{\sigma}/\sigma^0$.
- Equivalent plastic strain rate, $\dot{\epsilon}^{pl}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=JOHNSON_COOK, the table data specify the following:

- C .
- $\dot{\epsilon}_0$.

Return value

A RateDependent object.

Exceptions

RangeError.

29.85.2 **setValues(...)**

This method modifies the RateDependent object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RateDependent** method.

Return value

None

Exceptions

RangeError.

29.85.3 Members

The RateDependent object has members with the same names and descriptions as the arguments to the `RateDependent` method.

29.85.4 Corresponding analysis keywords

*RATE DEPENDENT

29.86 Ratios object

The Ratios object specifies ratios that define anisotropic swelling.

Access

```
import material
mdb.models[name].materials[name].moistureSwelling.ratios
mdb.models[name].materials[name].swelling.ratios
import odbMaterial
session.odbs[name].materials[name].moistureSwelling.ratios
session.odbs[name].materials[name].swelling.ratios
```

29.86.1 Ratios(...)

This method creates a Ratios object.

Path

```
mdb.models[name].materials[name].moistureSwelling.Ratios
mdb.models[name].materials[name].swelling.Ratios
session.odbs[name].materials[name].moistureSwelling.Ratios
session.odbs[name].materials[name].swelling.Ratios
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- r_{11} .
- r_{22} .
- r_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Ratios object.

Exceptions

RangeError.

29.86.2 `setValues(...)`

This method modifies the Ratios object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Ratios** method.

Return value

None

Exceptions

RangeError.

29.86.3 Members

The Ratios object has members with the same names and descriptions as the arguments to the **Ratios** method.

29.86.4 Corresponding analysis keywords

*RATIOS

29.87 Regularization object

The Regularization object defines the tolerance to be used for regularizing material data.

Access

```
import material
mdb.models[name].materials[name].regularization
import odbMaterial
session.odbs[name].materials[name].regularization
```

29.87.1 Regularization(...)

This method creates a Regularization object.

Path

```
mdb.models[name].materials[name].Regularization
session.odbs[name].materials[name].Regularization
```

Required arguments

None.

Optional arguments

rtol

A Float specifying the tolerance to be used for regularizing material data. The default value is 0.03.

strainRateRegularization

A SymbolicConstant specifying the form of regularization of strain-rate-dependent material data. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

Return value

A Regularization object.

Exceptions

RangeError.

29.87.2 setValues(...)

This method modifies the Regularization object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Regularization** method.

Return value

None

Exceptions

RangeError.

29.87.3 Members

The Regularization object has members with the same names and descriptions as the arguments to the **Regularization** method.

29.87.4 Corresponding analysis keywords

*DASHPOT

29.88 SaturationDependence object

The SaturationDependence object specifies the dependence of the permeability of a material on the saturation of the wetting liquid.

Access

```
import material
mdb.models[name].materials[name].permeability.saturationDependence
import odbMaterial
session.odbs[name].materials[name].permeability.saturationDependence
```

29.88.1 SaturationDependence(...)

This method creates a SaturationDependence object.

Path

```
mdb.models[name].materials[name].permeability.SaturationDependence
session.odbs[name].materials[name].permeability.SaturationDependence
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- $k_{s,}$ (Dimensionless.)
- Saturation, s . (Dimensionless.)

Return value

A SaturationDependence object.

Exceptions

RangeError.

29.88.2 **setValues(...)**

This method modifies the `SaturationDependence` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SaturationDependence` method.

Return value

None

Exceptions

`RangeError`.

29.88.3 **Members**

The `SaturationDependence` object has members with the same names and descriptions as the arguments to the `SaturationDependence` method.

29.88.4 **Corresponding analysis keywords**

*PERMEABILITY

29.89 **ShearRetention object**

The `ShearRetention` object defines the reduction of the shear modulus associated with crack surfaces in a Concrete model as a function of the tensile strain across the crack.

Access

```
import material
mdb.models[name].materials[name].concrete.shearRetention
import odbMaterial
session.odbs[name].materials[name].concrete.shearRetention
```

29.89.1 ShearRetention(...)

This method creates a ShearRetention object.

Path

```
mdb.models[name].materials[name].concrete.ShearRetention
session.odbs[name].materials[name].concrete.ShearRetention
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- ρ^{close} for dry concrete. The default value is 1.0.
- ϵ^{max} for dry concrete. The default value is a very large number (full shear retention).
- ρ^{close} for wet concrete. The default value is 1.0.
- ϵ^{max} for wet concrete. The default value is a very large number (full shear retention).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ShearRetention object.

Exceptions

RangeError.

29.89.2 setValues(...)

This method modifies the ShearRetention object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShearRetention** method.

Return value

None

Exceptions

RangeError.

29.89.3 Members

The **ShearRetention** object has members with the same names and descriptions as the arguments to the **ShearRetention** method.

29.89.4 Corresponding analysis keywords

*SHEAR RETENTION

29.90 ShearTestData object

The **ShearTestData** object specifies the normalized shear creep compliance or relaxation modulus as a function of time.

Access

```
import material
mdb.models[name].materials[name].viscoelastic.shearTestData
import odbMaterial
session.odbs[name].materials[name].viscoelastic.shearTestData
```

29.90.1 ShearTestData(...)

This method creates a **ShearTestData** object.

Path

```
mdb.models[name].materials[name].viscoelastic.ShearTestData  
session.odbs[name].materials[name].viscoelastic.ShearTestData
```

Required argument

table

A sequence of sequences of Floats specifying values that depend on the *time* member of the Viscoelastic object.

If *time*=RELAXATION_TEST_DATA, the table data specify the following:

- Normalized shear relaxation modulus $g_R(t)$. ($0 \leq g_R(t) \leq 1$).
- Time t . ($t > 0$).

If *time*=CREEP_TEST_DATA, the table data specify the following:

- Normalized shear compliance $j_S(t)$. ($j_S(t) \geq 1$).
- Time t . ($t > 0$).

Optional argument

shrinf

None or a Float specifying a normalized shear. The value of *shrinf* depends on the value of the *time* member of the Viscoelastic object. The default value is **None**.

If *time*=RELAXATION_TEST_DATA, *shrinf* specifies the value of the long-term, normalized shear modulus $g_R(\infty)$.

If *time*=CREEP_TEST_DATA, *shrinf* specifies the value of the long-term, normalized shear compliance $j_S(\infty)$.

Return value

A ShearTestData object.

Exceptions

None.

29.90.2 setValues(...)

This method modifies the ShearTestData object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShearTestData** method.

Return value

None

Exceptions

None.

29.90.3 Members

The **ShearTestData** object has members with the same names and descriptions as the arguments to the **ShearTestData** method.

29.90.4 Corresponding analysis keywords

*SHEAR TEST DATA

29.91 SimpleShearTestData object

The **SimpleShearTestData** object provides simple shear test data.

Access

```
import material
mdb.models[name].materials[name].hyperfoam.simpleShearTestData
import odbMaterial
session.odbs[name].materials[name].hyperfoam.simpleShearTestData
```

29.91.1 SimpleShearTestData(...)

This method creates a **SimpleShearTestData** object.

Path

```
mdb.models[name].materials[name].hyperfoam.SimpleShearTestData
session.odbs[name].materials[name].hyperfoam.SimpleShearTestData
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- Nominal shear stress, T_S .
- Nominal shear strain, γ .
- Nominal transverse stress, T_T (normal to edge with shear stress). This stress value is optional.

Return value

A SimpleShearTestData object.

Exceptions

None.

29.91.2 **setValues(...)**

This method modifies the SimpleShearTestData object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SimpleShearTestData** method.

Return value

None

Exceptions

None.

29.91.3 Members

The SimpleShearTestData object has members with the same names and descriptions as the arguments to the `SimpleShearTestData` method.

29.91.4 Corresponding analysis keywords

*SIMPLE SHEAR TEST DATA

29.92 Solubility object

The Solubility object specifies solubility.

Access

```
import material
mdb.models[name].materials[name].solubility
import odbMaterial
session.odbs[name].materials[name].solubility
```

29.92.1 Solubility(...)

This method creates a Solubility object.

Path

```
mdb.models[name].materials[name].Solubility
session.odbs[name].materials[name].Solubility
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Solubility.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Solubility object.

Exceptions

RangeError.

29.92.2 **setValues(...)**

This method modifies the Solubility object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Solubility** method.

Return value

None

Exceptions

RangeError.

29.92.3 **Members**

The Solubility object has members with the same names and descriptions as the arguments to the **Solubility** method.

29.92.4 **Corresponding analysis keywords**

*SOLUBILITY

29.93 SoretEffect object

The SoretEffect object defines temperature gradient driven mass diffusion.

Access

```
import material
mdb.models[name].materials[name].diffusivity.soretEffect
import odbMaterial
session.odbs[name].materials[name].diffusivity.soretEffect
```

29.93.1 SoretEffect(...)

This method creates a SoretEffect object.

Path

```
mdb.models[name].materials[name].diffusivity.SoretEffect
session.odbs[name].materials[name].diffusivity.SoretEffect
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Soret effect factor, κ_s .
- Concentration.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A SoretEffect object.

Exceptions

RangeError.

29.93.2 **setValues(...)**

This method modifies the SoretEffect object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SoretEffect** method.

Return value

None

Exceptions

RangeError.

29.93.3 **Members**

The SoretEffect object has members with the same names and descriptions as the arguments to the **SoretEffect** method.

29.93.4 **Corresponding analysis keywords**

*KAPPA

29.94 **Sorption object**

The Sorption object defines absorption and exsorption behaviors of a partially saturated porous medium in the analysis of coupled wetting liquid flow and porous medium stress.

Access

```
import material
mdb.models[name].materials[name].sorption
import odbMaterial
session.odbs[name].materials[name].sorption
```

29.94.1 Sorption(...)

This method creates a Sorption object.

Path

```
mdb.models[name].materials[name].Sorption
session.odbs[name].materials[name].Sorption
```

Required argument

absorptionTable

A sequence of sequences of Floats specifying the items described below.

Optional arguments

lawAbsorption

A SymbolicConstant specifying absorption behavior. Possible values are LOG and TABULAR. The default value is TABULAR.

exsorption

A Boolean specifying whether the exsorption data is specified. The default value is OFF.

lawExsorption

A SymbolicConstant specifying exsorption behavior. Possible values are LOG and TABULAR. The default value is TABULAR.

scanning

A Float specifying the slope of the scanning line, $(du_w/ds)|_s$. This slope must be positive and larger than the slope of the absorption or exsorption behaviors. The default value is 0.0.

exsorptionTable

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

Table data

If *lawAbsorption*=TABULAR or *lawExsorption*=TABULAR, the *absorptionTable* and *exsorptionTable* data respectively specify the following:

- Pore pressure, u_w .
- Saturation, s .

MATERIAL COMMANDS

If *lawAbsorption*=LOG or *lawExsorption*=LOG, the *absorptionTable* and *exsorptionTable* data respectively specify the following:

- A.
- B.
- s_0 .
- s_1 .

Return value

A Sorption object.

Exceptions

RangeError.

29.94.2 **setValues(...)**

This method modifies the Sorption object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Sorption** method.

Return value

None

Exceptions

RangeError.

29.94.3 **Members**

The Sorption object has members with the same names and descriptions as the arguments to the **Sorption** method.

29.94.4 **Corresponding analysis keywords**

*SORPTION

29.95 **SpecificHeat object**

The SpecificHeat object specifies a material's specific heat.

Access

```
import material
mdb.models[name].materials[name].specificHeat
import odbMaterial
session.odbs[name].materials[name].specificHeat
```

29.95.1 **SpecificHeat(...)**

This method creates a SpecificHeat object.

Path

```
mdb.models[name].materials[name].SpecificHeat
session.odbs[name].materials[name].SpecificHeat
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the specific heat behavior. Possible values are CONSTANTVOLUME and CONSTANTPRESSURE. The default value is CONSTANTVOLUME.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Specific heat per unit mass.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

MATERIAL COMMANDS

- Value of the second field variable.
- Etc.

Return value

A `SpecificHeat` object.

Exceptions

`RangeError`.

29.95.2 `setValues(...)`

This method modifies the `SpecificHeat` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SpecificHeat` method.

Return value

`None`

Exceptions

`RangeError`.

29.95.3 Members

The `SpecificHeat` object has members with the same names and descriptions as the arguments to the `SpecificHeat` method.

29.95.4 Corresponding analysis keywords

*SPECIFIC HEAT

29.96 Swelling object

The Swelling object specifies time-dependent volumetric swelling for a material.

Access

```
import material
mdb.models[name].materials[name].swelling
import odbMaterial
session.odbs[name].materials[name].swelling
```

29.96.1 Swelling(...)

This method creates a Swelling object.

Path

```
mdb.models[name].materials[name].Swelling
session.odbs[name].materials[name].Swelling
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.
This argument is valid only when *law*=INPUT.

Optional arguments

law

A SymbolicConstant specifying the type of data defining the swelling behavior. Possible values are INPUT and USER. The default value is INPUT.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Volumetric swelling strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

MATERIAL COMMANDS

- Etc.

Return value

A Swelling object.

Exceptions

RangeError.

29.96.2 setValues(...)

This method modifies the Swelling object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Swelling** method.

Return value

None

Exceptions

RangeError.

29.96.3 Members

The Swelling object has members with the same names and descriptions as the arguments to the **Swelling** method.

In addition, the Swelling object can have the following member:

ratios

A Ratios object.

29.96.4 Corresponding analysis keywords

*SWELLING

29.97 **TensionCutOff object**

The TensionCutOff object specifies tension cutoff for different material models for example the Mohr-Coulomb plasticity model.

Access

```
import material
mdb.models[name].materials[name].mohrCoulombPlasticity.tensionCutOff
import odbMaterial
session.odbs[name].materials[name].mohrCoulombPlasticity.tensionCutOff
```

29.97.1 **TensionCutOff(...)**

This method creates a TensionCutOff object.

Path

```
mdb.models[name].materials[name].mohrCoulombPlasticity.TensionCutOff
session.odbs[name].materials[name].mohrCoulombPlasticity.TensionCutOff
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tension cutoff stress.
- The value of the corresponding tensile plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `TensionCutOff` object.

Exceptions

`RangeError`.

29.97.2 **setValues(...)**

This method modifies the `TensionCutOff` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TensionCutOff** method.

Return value

`None`

Exceptions

`RangeError`.

29.97.3 **Members**

The `TensionCutOff` object has members with the same names and descriptions as the arguments to the **TensionCutOff** method.

29.97.4 **Corresponding analysis keywords**

*TENSION CUTOFF

29.98 **TensionStiffening object**

The `TensionStiffening` object defines the retained tensile stress normal to a crack in a Concrete model.

Access

```
import material
mdb.models[name].materials[name].concrete.tensionStiffening
import odbMaterial
session.odbs[name].materials[name].concrete.tensionStiffening
```

29.98.1 TensionStiffening(...)

This method creates a TensionStiffening object.

Path

```
mdb.models[name].materials[name].concrete.TensionStiffening
session.odbs[name].materials[name].concrete.TensionStiffening
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying how the postcracking behavior is defined. Possible values are DISPLACEMENT and STRAIN. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Fraction of remaining stress to stress at cracking.
- Absolute value of the direct strain minus the direct strain at cracking.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Displacement, u_0 , at which a linear loss of strength after cracking gives zero stress.
- Temperature, if the data depend on temperature.

MATERIAL COMMANDS

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `TensionStiffening` object.

Exceptions

`RangeError`.

29.98.2 **setValues(...)**

This method modifies the `TensionStiffening` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TensionStiffening** method.

Return value

`None`

Exceptions

`RangeError`.

29.98.3 **Members**

The `TensionStiffening` object has members with the same names and descriptions as the arguments to the **TensionStiffening** method.

29.98.4 **Corresponding analysis keywords**

*TENSION STIFFENING

29.99 TriaxialTestData object

The TriaxialTestData object provides triaxial test data.

Access

```
import material
mdb.models[name].materials[name].druckerPrager.compressionTriaxialTestData
import odbMaterial
session.odbs[name].materials[name].druckerPrager.compressionTriaxialTestData
```

29.99.1 TriaxialTestData(...)

This method creates a TriaxialTestData object.

Path

```
mdb.models[name].materials[name].druckerPrager.compressionTriaxialTestData
session.odbs[name].materials[name].druckerPrager.compressionTriaxialTestData
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

a

None or a Float specifying the value of the material constant *a*. **None** is used when the value is unknown or it is not held fixed at the input value. The default value is **None**.

b

None or a Float specifying the value of the material constant *b*. **None** is used when the value is unknown or it is not held fixed at the input value. The default value is **None**.

pt

None or a Float specifying the value of the material constant *p_t*. **None** is used when the value is unknown or it is not held fixed at the input value. The default value is **None**.

Table data

- Sign and magnitude of confining stress, $\sigma_1 = \sigma_2$.
- Sign and magnitude of the stress in loading direction, σ_3 .

Return value

A `TriaxialTestData` object.

Exceptions

`RangeError`.

29.99.2 **setValues(...)**

This method modifies the `TriaxialTestData` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TriaxialTestData** method.

Return value

None

Exceptions

`RangeError`.

29.99.3 **Members**

The `TriaxialTestData` object has members with the same names and descriptions as the arguments to the **TriaxialTestData** method.

29.99.4 **Corresponding analysis keywords**

*TRIAXIAL TEST DATA

29.100 **Trs object**

The `Trs` object defines the temperature-time shift for time history viscoelastic analysis.

Access

```
import material
mdb.models[name].materials[name].viscoelastic.trs
mdb.models[name].materials[name].viscosity.trs
import odbMaterial
session.odbs[name].materials[name].viscoelastic.trs
session.odbs[name].materials[name].viscosity.trs
```

29.100.1 Trs(...)

This method creates a Trs object.

Path

```
mdb.models[name].materials[name].viscoelastic.Trs
mdb.models[name].materials[name].viscosity.Trs
session.odbs[name].materials[name].viscoelastic.Trs
session.odbs[name].materials[name].viscosity.Trs
```

Required arguments

None.

Optional arguments*definition*

A SymbolicConstant specifying the definition of the shift function. Possible values are WLF, ARRHENIUS, and USER. The default value is WLF.

table

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

This argument is valid only when *definition*=WLF.

Table data

- Reference temperature, θ_0 .
- Calibration constant, C_1 .
- Calibration constant, C_2 .

Return value

A Trs object.

Exceptions

None.

29.100.2 **setValues(...)**

This method modifies the **Trs** object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Trs** method.

Return value

None

Exceptions

None.

29.100.3 **Members**

The **Trs** object has members with the same names and descriptions as the arguments to the **Trs** method.

29.100.4 **Corresponding analysis keywords**

*TRS

29.101 **UniaxialTestData object**

The **UniaxialTestData** object provides uniaxial test data (compression and/or tension).

Access

```
import material
mdb.models[name].materials[name].hyperelastic.uniaxialTestData
mdb.models[name].materials[name].hyperfoam.uniaxialTestData
```

```

mdb.models[name].materials[name].lowDensityFoam\
.uniaxialCompressionTestData
mdb.models[name].materials[name].lowDensityFoam\
.uniaxialTensionTestData
mdb.models[name].materials[name].mullinsEffect.uniaxialTests[i]
import odbMaterial
session.odbs[name].materials[name].hyperelastic.uniaxialTestData
session.odbs[name].materials[name].hyperfoam.uniaxialTestData
session.odbs[name].materials[name].lowDensityFoam\
.uniaxialCompressionTestData
session.odbs[name].materials[name].lowDensityFoam\
.uniaxialTensionTestData
session.odbs[name].materials[name].mullinsEffect.uniaxialTests[i]

```

29.101.1 UniaxialTestData(...)

This method creates a UniaxialTestData object.

Path

```

mdb.models[name].materials[name].hyperelastic.UniaxialTestData
mdb.models[name].materials[name].hyperfoam.UniaxialTestData
mdb.models[name].materials[name].lowDensityFoam.UniaxialTestData
mdb.models[name].materials[name].mullinsEffect.UniaxialTestData
session.odbs[name].materials[name].hyperelastic.UniaxialTestData
session.odbs[name].materials[name].hyperfoam.UniaxialTestData
session.odbs[name].materials[name].lowDensityFoam.UniaxialTestData
session.odbs[name].materials[name].mullinsEffect.UniaxialTestData

```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

smoothing

None or an Int specifying the value for smoothing. If *smoothing=None*, no smoothing is employed. The default value is **None**.

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic material model, the table data specify the following:

- Nominal stress, T_U .
- Nominal strain, ϵ_U .

For a hyperfoam material model, the table data specify the following:

- Nominal stress, T_L .
- Nominal strain, ϵ_U .
- Nominal lateral strain, $\epsilon_2 = \epsilon_3$. The default value is 0.

For a low-density foam material model, the table data specify the following:

- Nominal stress, T_U .
- Nominal strain, ϵ_U .
- Nominal strain rate, $\dot{\epsilon}_U$.

Return value

A UniaxialTestData object.

Exceptions

None.

29.101.2 setValues(...)

This method modifies the UniaxialTestData object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **UniaxialTestData** method.

Return value

None

Exceptions

None.

29.101.3 Members

The `UniaxialTestData` object has members with the same names and descriptions as the arguments to the `UniaxialTestData` method.

29.101.4 Corresponding analysis keywords

*UNIAXIAL TEST DATA

29.102 UserDefinedField object

The `UserDefinedField` object redefines field variables at a material point.

Access

```
import material
mdb.models[name].materials[name].userDefinedField
import odbMaterial
session.odbs[name].materials[name].userDefinedField
```

29.102.1 UserDefinedField()

This method defines a `UserDefinedField` object.

Path

```
mdb.models[name].materials[name].UserDefinedField
session.odbs[name].materials[name].UserDefinedField
```

Arguments

None.

Return value

A `UserDefinedField` object.

Exceptions

None.

29.102.2 Members

The UserDefinedField object has no members.

29.102.3 Corresponding analysis keywords

*USER DEFINED FIELD

29.103 UserMaterial object

The UserMaterial object defines material constants for use in subroutines **UMAT**, **UMATHT**, or **VUMAT**.

Access

```
import material
mdb.models[name].materials[name].userMaterial
import odbMaterial
session.odbs[name].materials[name].userMaterial
```

29.103.1 UserMaterial(...)

This method creates a UserMaterial object.

Path

```
mdb.models[name].materials[name].UserMaterial
session.odbs[name].materials[name].UserMaterial
```

Required arguments

None.

Optional arguments

type

A SymbolicConstant specifying the type of material behavior defined by the command. Possible values are MECHANICAL, THERMAL, and THERMOMECHANICAL. The default value is MECHANICAL.

unsymm

A Boolean specifying if the material stiffness matrix, $\partial\Delta\sigma/\partial\Delta\varepsilon$, is not symmetric or, when a thermal constitutive model is used, if $\partial\mathbf{f}/\partial(\partial\theta/\partial\mathbf{x})$ is not symmetric. The default value is OFF.

This argument is valid only for an Abaqus/Standard analysis.

mechanicalConstants

A sequence of Floats specifying the mechanical constants of the material. This argument is valid only when *type*=MECHANICAL or THERMOMECHANICAL. The default value is an empty sequence.

thermalConstants

A sequence of Floats specifying the thermal constants of the material. This argument is valid only when *type*=THERMAL or THERMOMECHANICAL. The default value is an empty sequence.

Return value

A UserMaterial object.

Exceptions

RangeError.

29.103.2 setValues(...)

This method modifies the UserMaterial object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **UserMaterial** method.

Return value

None

Exceptions

RangeError.

29.103.3 Members

The UserMaterial object has members with the same names and descriptions as the arguments to the **UserMaterial** method.

29.103.4 Corresponding analysis keywords

*USER MATERIAL

29.104 UserOutputVariables object

The UserOutputVariables object specifies the number of user-defined output variables.

Access

```
import material
mdb.models[name].materials[name].userOutputVariables
import odbMaterial
session.odbs[name].materials[name].userOutputVariables
```

29.104.1 UserOutputVariables(...)

This method creates a UserOutputVariables object.

Path

```
mdb.models[name].materials[name].UserOutputVariables
session.odbs[name].materials[name].UserOutputVariables
```

Required arguments

None.

Optional argument

n

An Int specifying the number of user-defined variables required at each material point. The default value is 0.

Return value

A UserOutputVariables object.

Exceptions

RangeError.

29.104.2 **setValues(...)**

This method modifies the `UserOutputVariables` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `UserOutputVariables` method.

Return value

None

Exceptions

`RangeError`.

29.104.3 **Members**

The `UserOutputVariables` object has members with the same names and descriptions as the arguments to the `UserOutputVariables` method.

29.104.4 **Corresponding analysis keywords**

*USER OUTPUT VARIABLES

29.105 **VelocityDependence object**

The `VelocityDependence` object specifies the dependence of the permeability of a material on the velocity of fluid flow.

Access

```
import material
mdb.models[name].materials[name].permeability.velocityDependence
import odbMaterial
session.odbs[name].materials[name].permeability.velocityDependence
```

29.105.1 VelocityDependence(...)

This method creates a VelocityDependence object.

Path

```
mdb.models[name].materials[name].permeability.VelocityDependence  
session.odbs[name].materials[name].permeability.VelocityDependence
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

None.

Table data

- β . Only $\beta > 0.0$ is allowed.
- Void ratio, e .

Return value

A VelocityDependence object.

Exceptions

RangeError.

29.105.2 setValues(...)

This method modifies the VelocityDependence object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VelocityDependence** method.

Return value

None

Exceptions

RangeError.

29.105.3 Members

The VelocityDependence object has members with the same names and descriptions as the arguments to the `VelocityDependence` method.

29.105.4 Corresponding analysis keywords

*PERMEABILITY

29.106 Viscoelastic object

The Viscoelastic object specifies dissipative behavior for use with elasticity.

Access

```
import material
mdb.models[name].materials[name].viscoelastic
import odbMaterial
session.odbs[name].materials[name].viscoelastic
```

29.106.1 Viscoelastic(...)

This method creates a Viscoelastic object.

Path

```
mdb.models[name].materials[name].Viscoelastic
session.odbs[name].materials[name].Viscoelastic
```

Required arguments

domain

A SymbolicConstant specifying the domain definition. Possible values are:

- FREQUENCY, specifying a frequency domain. This domain is only available for an Abaqus/Standard analysis.

MATERIAL COMMANDS

- TIME, specifying a time domain.

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

frequency

A SymbolicConstant specifying the frequency domain definition. This argument is required only when *domain*=FREQUENCY. Possible values are FORMULA, TABULAR, PRONY, CREEP_TEST_DATA, and RELAXATION_TEST_DATA. The default value is FORMULA.

type

A SymbolicConstant specifying the type. This argument is required only when *domain*=FREQUENCY and *frequency*=TABULAR. Possible values are ISOTROPIC and TRACTION. The default value is ISOTROPIC.

preload

A SymbolicConstant specifying the preload. This argument is required only when *domain*=FREQUENCY and *frequency*=TABULAR. Possible values are NONE, UNIAXIAL, VOLUMETRIC, and UNIAXIAL_VOLUMETRIC. The default value is NONE.

time

A SymbolicConstant specifying the time domain definition. This argument is required only when *domain*=TIME. Possible values are PRONY, CREEP_TEST_DATA, RELAXATION_TEST_DATA, and FREQUENCY_DATA. The default value is PRONY.

errtol

A Float specifying the allowable average root-mean-square error of the data points in the least-squares fit. The Float values correspond to percentages; for example, 0.01 is 1%. The default value is 0.01.

This argument is valid only when *time*=CREEP_TEST_DATA, RELAXATION_TEST_DATA or FREQUENCY_DATA; or only when *frequency*=CREEP_TEST_DATA or RELAXATION_TEST_DATA.

nmax

An Int specifying the maximum number of terms N in the Prony series. The maximum value is 13. The default value is 13.

This argument is valid only when *time*=CREEP_TEST_DATA, RELAXATION_TEST_DATA or FREQUENCY_DATA; or only when *frequency*=CREEP_TEST_DATA or RELAXATION_TEST_DATA.

volumetricTable

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

Table data

If *frequency*=FORMULA, the table data for *table* specify the following:

- Real part of g_1^* ($g_1^*(\omega) = g_1^* f^{-a}$).

- Imaginary part of g_1^* .
- Value of a .
- Real part of k_1^* ($k_1^*(\omega) = k_1^* f^{-b}$). If the material is incompressible, this value is ignored.
- Imaginary part of k_1^* . If the material is incompressible, this value is ignored.
- Value of b . If the material is incompressible, this value is ignored.

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=NONE, or *time*=FREQUENCY_DATA the table data for *table* specify the following:

- Real part of ωg^* ($\omega \Re(g^*) = G_\ell / G_\infty$).
- Imaginary part of ωg^* ($\omega \Im(g^*) = 1 - G_s / G_\infty$).
- Real part of ωk^* ($\omega \Re(k^*) = K_\ell / K_\infty$). If the material is incompressible, this value is ignored.
- Imaginary part of ωk^* ($\omega \Im(k^*) = 1 - K_s / K_\infty$). If the material is incompressible, this value is ignored.
- Frequency f in cycles per time.

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=UNIAXIAL the table data for *table* specify the following:

- Loss modulus.
- Storage modulus.
- Frequency.
- Uniaxial strain.

If *frequency*=TABULAR and *type*=TRACTION and *preload*=NONE the table data for *table* specify the following:

- Normalized loss modulus.
- Normalized shear modulus.
- Frequency.

If *frequency*=TABULAR and *type*=TRACTION and *preload*=UNIAXIAL or *preload*=UNIAXIAL_VOLUMETRIC the table data for *table* specify the following:

- Loss modulus.
- Storage modulus.
- Frequency.
- Closure.

If *time*=PRONY or *frequency*=PRONY, the table data for *table* specify the following:

- $\bar{g}_1^P$, the modulus ratio in the first term in the Prony series expansion of the shear relaxation modulus.
- $\bar{k}_1^P$, the modulus ratio in the first term in the Prony series expansion of the bulk relaxation modulus.
- τ_1 , the relaxation time for the first term in the Prony series expansion.

MATERIAL COMMANDS

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=VOLUMETRIC or *preload*=UNIAXIAL_VOLUMETRIC the table data for *volumetricTable* specify the following:

- Loss modulus.
- Storage modulus.
- Frequency.
- Volume ratio.

Return value

A Viscoelastic object.

Exceptions

RangeError.

29.106.2 **setValues(...)**

This method modifies the Viscoelastic object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Viscoelastic** method.

Return value

None

Exceptions

RangeError.

29.106.3 **Members**

The Viscoelastic object has members with the same names and descriptions as the arguments to the **Viscoelastic** method.

In addition, the Viscoelastic object can have the following members:

combinedTestData

A CombinedTestData object.

shearTestData

A ShearTestData object.

trs

A Trs object.

volumetricTestData

A VolumetricTestData object.

29.106.4 Corresponding analysis keywords

*VISCOELASTIC

29.107 Viscosity object

The Viscosity object specifies mechanical viscosity.

Access

```
import material
mdb.models[name].materials[name].viscosity
import odbMaterial
session.odbs[name].materials[name].viscosity
```

29.107.1 Viscosity(...)

This method creates a Viscosity object.

Path

```
mdb.models[name].materials[name].Viscosity
session.odbs[name].materials[name].Viscosity
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

type

A SymbolicConstant specifying the type of viscosity. The default value is NEWTONIAN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=NEWTONIAN, the table data specify the following:

- Viscosity, k .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Viscosity object.

Exceptions

RangeError.

29.107.2 setValues(...)

This method modifies the Viscosity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Viscosity** method.

Return value

None

Exceptions

RangeError.

29.107.3 Members

The Viscosity object has members with the same names and descriptions as the arguments to the **Viscosity** method.

In addition, the Viscosity object can have the following member:

trs

A Trs object.

29.107.4 Corresponding analysis keywords

*VISCOSITY

29.108 Viscous object

The Viscous object specifies the viscous properties for a two-layer viscoplastic material model.

Access

```
import material
mdb.models[name].materials[name].viscous
import odbMaterial
session.odbs[name].materials[name].viscous
```

29.108.1 Viscous(...)

This method creates a Viscous object.

Path

```
mdb.models[name].materials[name].Viscous
session.odbs[name].materials[name].Viscous
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

law

A SymbolicConstant specifying the creep law. Possible values are STRAIN, TIME, and USER. The default value is STRAIN.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

MATERIAL COMMANDS

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- A .
- n .
- m .
- f .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=USER, the table data specify the following:

- f .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Viscous object.

Exceptions

None.

29.108.2 **setValues(...)**

This method modifies the Viscous object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Viscous** method.

Return value

None

Exceptions

None.

29.108.3 Members

The Viscous object has members with the same names and descriptions as the arguments to the **Viscous** method.

In addition, the Viscous object can have the following member:

potential

A Potential object.

29.108.4 Corresponding analysis keywords

*VISCOUS

29.109 VoidNucleation object

The VoidNucleation object defines the nucleation of voids in a porous material.

Access

```
import material
mdb.models[name].materials[name].porousMetalPlasticity.voidNucleation
import odbMaterial
session.odbs[name].materials[name].porousMetalPlasticity\
.voidNucleation
```

29.109.1 VoidNucleation(...)

This method creates a VoidNucleation object.

Path

```
mdb.models[name].materials[name].porousMetalPlasticity.VoidNucleation
session.odbs[name].materials[name].porousMetalPlasticity\
.VoidNucleation
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- ε_N , the mean value of the nucleation-strain normal distribution.
- s_N , the standard deviation of the nucleation-strain normal distribution.
- f_N , the volume fraction of nucleating voids.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A VoidNucleation object.

Exceptions

RangeError.

29.109.2 **setValues(...)**

This method modifies the VoidNucleation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VoidNucleation** method.

Return value

None

Exceptions

RangeError.

29.109.3 Members

The VoidNucleation object has members with the same names and descriptions as the arguments to the `VoidNucleation` method.

29.109.4 Corresponding analysis keywords

*VOID NUCLEATION

29.110 VolumetricTestData object

The VolumetricTestData object provides volumetric test data.

Access

```
import material
mdb.models[name].materials[name].hyperelastic.volumetricTestData
mdb.models[name].materials[name].hyperfoam.volumetricTestData
mdb.models[name].materials[name].viscoelastic.volumetricTestData
import odbMaterial
session.odbs[name].materials[name].hyperelastic.volumetricTestData
session.odbs[name].materials[name].hyperfoam.volumetricTestData
session.odbs[name].materials[name].viscoelastic.volumetricTestData
```

29.110.1 VolumetricTestData(...)

This method creates a VolumetricTestData object.

Path

```
mdb.models[name].materials[name].hyperelastic.VolumetricTestData
mdb.models[name].materials[name].hyperfoam.VolumetricTestData
```

MATERIAL COMMANDS

```
mdb.models [name] .materials [name] .viscoelastic.VolumetricTestData  
session.odbs [name] .materials [name] .hyperelastic.VolumetricTestData  
session.odbs [name] .materials [name] .hyperfoam.VolumetricTestData  
session.odbs [name] .materials [name] .viscoelastic.VolumetricTestData
```

Required argument

table

A sequence of sequences of Floats specifying the items described below.

Optional arguments

volinf

None or a Float specifying a normalized volumetric value that depends on the value of the *time* member of the Viscoelastic object. The default value is **None**.

If *time*=RELAXATION_TEST_DATA, *volinf* specifies the value of the long-term, normalized volumetric modulus, $k_R(\infty)$.

If *time*=CREEP_TEST_DATA, *volinf* specifies the value of the long-term, normalized volumetric compliance, $j_K(\infty)$.

This argument is valid only for a viscoelastic material model.

smoothing

None or an Int specifying the value for smoothing. If *smoothing*=**None**, no smoothing is employed. The default value is **None**.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic or hyperfoam material model, the table data specify the following:

- Pressure, p .
- Volume ratio, J (current volume/original volume).

For a viscoelastic material model, the values depend on the value of the *time* member of the Viscoelastic object.

If *time*=RELAXATION_TEST_DATA, the table data specify the following:

- Normalized volumetric (bulk) modulus $k_R(t)$, ($0 \leq k_R(t) \leq 1$).
- Time t ($t > 0$).

If *time*=CREEP_TEST_DATA, the table data specify the following:

- Normalized volumetric (bulk) compliance $j_K(t)$, ($j_K(t) \geq 1$).
- Time t ($t > 0$).

Return value

A VolumetricTestData object.

Exceptions

None.

29.110.2 setValues(...)

This method modifies the VolumetricTestData object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **VolumetricTestData** method.

Return value

None

Exceptions

None.

29.110.3 Members

The VolumetricTestData object has members with the same names and descriptions as the arguments to the **VolumetricTestData** method.

29.110.4 Corresponding analysis keywords

*VOLUMETRIC TEST DATA

29.111 evaluateMaterial

This command evaluates the behavior of a hyperelastic material under standard test conditions.

Access

```
from material import evaluateMaterial
```

29.111.1 evaluateMaterial(...)

This method evaluates the behavior of a hyperelastic material under standard test conditions.

Path

```
evaluateMaterial
```

Arguments

Required arguments

material

A Material object.

simulationName

A String specifying the name to be used for the material evaluation simulation.

Optional arguments

Note: Although *uniaxialStrainRange*, *biaxialStrainRange*, *planarStrainRange*, *volumeRatioRange*, and *simpleShearStrainRange* are optional arguments, at least one of them must be specified with a nonzero value.

dataSource

A SymbolicConstant specifying whether test data or coefficients should be used for the material definition in the unit element tests. Possible values are TEST_DATA or COEFFICIENTS.

strainEnergyPotentials

A sequence of SymbolicConstants specifying for which material models the material is to be evaluated. Possible values are POLY_N1, POLY_N2, POLY_N3, POLY_N4, POLY_N5, POLY_N6, OGDEN_N1, OGDEN_N2, OGDEN_N3, OGDEN_N4, OGDEN_N5, OGDEN_N6, REDUCED_POLY_N1, REDUCED_POLY_N2, REDUCED_POLY_N3, REDUCED_POLY_N4, REDUCED_POLY_N5, REDUCED_POLY_N6, ARRUDA_BOYCE, VAN_DER_WAALS, YEOH, MOONEY_RIVLIN, and NEO_HOOKE.

Note: The options POLY_N3, POLY_N4, POLY_N5, and POLY_N6 are valid only if the material was defined by providing coefficients of the strain energy potential.

marlowData

None or a sequence of SymbolicConstants specifying the types of test data to be included in the material definition of the Marlow material that is being evaluated. Possible values are UNIAXIAL, BIAXIAL, PLANAR, or VOLUMETRIC. The default value is **None**.

marlowDataType

None or a SymbolicConstant specifying the input data type for the Marlow material model. Possible values are TENSION, COMPRESSION, or BOTH.

testDataTypes

A sequence of SymbolicConstants specifying the types of test data to be included in the material definition of the material being evaluated. Possible values are UNIAXIAL, BIAXIAL, PLANAR, and VOLUMETRIC.

uniaxialStrainRange

A tuple of Floats specifying minimum and maximum nominal strains to be applied in the uniaxial tension test.

biaxialStrainRange

A tuple of Floats specifying the minimum and maximum nominal strains to be applied in the biaxial tension test.

planarStrainRange

A tuple of Floats specifying the minimum and maximum nominal strains to be applied in the planar test. The planar test is equivalent to a pure shear test.

volumeRatioRange

A tuple of Floats specifying the minimum and maximum compressive volume ratio.

simpleShearStrainRange

A tuple of Floats specifying the minimum and maximum nominal strains to be applied in the simple shear test.

viscoDataSource

None or a SymbolicConstant specifying whether test data or coefficients should be used for the viscoelastic material definition in the element tests. Possible values are TEST_DATA or COEFFICIENTS. The default value is **None**.

viscoTestDataTypes

None or a sequence of SymbolicConstants specifying the types of test data to be included in the material definition of the viscoelastic material being evaluated. Possible values are UNIAXIAL, BIAXIAL, PLANAR, or VOLUMETRIC. The default value is **None**.

relaxationTime

None or a Float specifying the time period for the stress relaxation response mode. The default value is **None**.

creepTime

None or a Float specifying the time period for the creep response mode. The default value is **None**.

Return value

None

Exceptions

If *dataSource*=TEST_DATA and *strainEnergyPotentials* contains POLY_N3, POLY_N4, POLY_N5, or POLY_N6:

MaterialEvaluationError: POLY_N3, POLY_N4, POLY_N5, or POLY_N6
not allowed for *dataSource*=TEST_DATA.

If the material evaluation failed:

MaterialEvaluationError: material evaluation failed, see
path to data file.

If the material type of the material to be evaluated is not hyperelastic:

MaterialEvaluationError: Material evaluation is currently
supported only for hyperelastic materials.

30. Mdb commands

30.1 Mdb object

The Mdb object is the high-level Abaqus model database. A model database stores models and analysis controls.

Access

`mdb`

30.1.1 Mdb(...)

This constructor creates an empty Mdb object.

Path

`Mdb`

Required arguments

None.

Optional argument

pathName

A String specifying the path to be used when the model database is saved to a file. If you do not provide a file extension, `.cae` is appended automatically to the path. The default value is an empty string.

Return value

A Mdb object.

Exceptions

None.

30.1.2 importDxf(...)

This method creates a ConstrainedSketch object from a file containing dxf-format (AutoCAD) geometry. Only a limited number of entities are supported. This format should be used only if no other formats are available.

Path

importDxf

Required argument

fileName

A String specifying the path to the dxf file to open.

Optional arguments

None.

Return value

A Mdb object.

Exceptions

None.

30.1.3 openMdb(...)

This method opens an existing model database file.

Path

openMdb

Required argument

pathName

A String specifying the path to the model database file to open. If you do not provide a file extension, Abaqus/CAE attempts to open the file with **.cae** appended to the path.

Optional arguments

None.

Return value

An Mdb object.

Exceptions

If the file is an invalid model database:
MdbError: invalid model database.

If the file contains a model database from an Abaqus release other than the Abaqus release you are currently running:

MdbError: incompatible release number, expected *<Abaqus release>*, got *<earlier or later Abaqus release>*

If the model database file is already opened in write mode:

MdbError: cannot open file: May be in use by another CAE session

If the command fails to open the model database file for reasons not mentioned above:

MdbError: cannot open file...

30.1.4 openAcis(...)

This method creates an `AcisFile` object from a file containing ACIS-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openAcis

Required argument

fileName

A String specifying the path to the ACIS file to open.

Optional argument

scaleFromFile

A Boolean specifying whether to scale, rotate, and translate the part using the transform read from the ACIS file. The default value is OFF.

Return value

An `AcisFile` object.

Exceptions

File is from a newer version of ACIS than the CAE kernel.

Texterror: ACIS File version exceeds Kernel.

The data in the ACIS file are corrupted.

Texterror: Failed to read ACIS file.

30.1.5 openCatia(...)

This method creates an `AcisFile` object from a file containing CATIA V4-format or V5-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openCatia

Required argument

fileName

A String specifying the path to the CATIA file to open.

Optional arguments

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID, SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. If a CATIA V4-format file contains different topologies, only the selected topology is imported from the file. The default value is SOLID.

convertUnits

A SymbolicConstant specifying whether the original units should be retained. Possible values are ON and OFF. The default value is OFF.

combineBodies

A Boolean specifying whether to combine the bodies in the CATPart file. If the bodies to be combined touch or overlap, invalid entities would result. For V4-format and CATProduct files, this option will be ignored.

Return value

An AcisFile object.

Exceptions

None.

30.1.6 openEnf(...)

This method creates an AcisFile object from a file containing Elysium Neutral File-format geometry that was created by CATIA V5, I-DEAS, or Pro/ENGINEER. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openEnf

Required arguments*fileName*

A String specifying the path to the Elysium Neutral File that was created by I-DEAS, Pro/ENGINEER, or CATIA V5.

fileType

A String specifying the type of CAD system that created the file. Possible values are “ideas”, “proe”, or “catiaV5”.

Optional arguments*topology*

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID, SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

convertUnits

A Boolean specifying if the dimensions of the part should be converted to millimeters. The default value is OFF.

Return value

An AcisFile object.

Exceptions

None.

30.1.7 openIges(...)

This method creates an AcisFile object from a file containing IGES-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path**openIges****Required argument***fileName*

A String specifying the path to the IGES file to open.

Optional arguments

trimCurve

A SymbolicConstant specifying the method used to define the trim curves that bound parametric surfaces. Possible values are:

- DEFAULT, use either of the following as specified by the contents of the IGES file.
- PARAMETRIC_DATA, use the parameter space of the surface being trimmed.
- THREED_DATA, use real space—the coordinate system of the part along with an indication that the trim curve lies on the parametric surface.

The default value is DEFAULT.

scaleFromFile

A SymbolicConstant specifying whether the imported geometry needs to be scaled using the units information available in the IGES file. Possible values are ON and OFF. The default value is OFF. When the argument is set to ON, the geometry is scaled to millimeters with respect to the unit system specified in the IGES file.

msbo

A Boolean specifying if the IGES file contains MSBO (Manifold Solid B-Rep Object) entities. The default value is False.

includedLayers

A sequence of Ints specifying the levels or layers of entities that will be translated from the IGES file to build the part. The default is to include all the layers.

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID, SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

uniteWires

A SymbolicConstant specifying whether the imported wires need to be united or not. Possible values are ON and OFF. The default value is ON. When importing a sketch, this value is set to OFF.

Return value

An AcisFile object.

Exceptions

The data in the IGES file are corrupted.

Texterror: Failed to read IGES file.

30.1.8 openParasolid(...)

This method creates an AcisFile object from a file containing Parasolid-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openParasolid

Required argument

fileName

A String specifying the path to the Parasolid file to open.

Optional argument

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID, SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

Return value

An AcisFile object.

Exceptions

None.

30.1.9 openStep(...)

This method creates an AcisFile object from a file containing STEP-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openStep

Required argument

fileName

A String specifying the path to the STEP file to open.

Optional argument

scale

A Float specifying the scaling factor to apply to the imported geometric entities. The default value is 1.0.

Return value

An AcisFile object.

Exceptions

The data in the STEP file are corrupted.

Texterror: Failed to read STEP file.

30.1.10 openVda(...)

This method creates an AcisFile object from a file containing VDA-FS-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

openVda

Required argument

fileName

A String specifying the path to the VDA-FS file to open.

Optional arguments

None.

Return value

An AcisFile object.

Exceptions

The data in the VDA-FS file are corrupted.

Texterror: Failed to read VDA file.

30.1.11 close()

This method closes an open Mdb object but does not save the Mdb object to disk. After closing the Mdb object, this method creates a new unnamed empty Mdb object.

Arguments

None.

Return value

None

Exceptions

None.

30.1.12 save()

This method saves an Mdb object to disk at the location specified by *pathName* (*pathName* is a member of the Mdb object).

Arguments

None.

Return value

None

Exceptions

If *pathName* is empty:

MdbError: cannot save file: pathname member is empty

If *pathName* is **abaqus.cae**:

MdbError: "abaqus.cae" is an invalid CAE filename.

If the command fails to save the Mdb object to disk for reasons not mentioned above:

MdbError: cannot save file...

30.1.13 saveAs(...)

This method saves an Mdb object to disk at the specified location.

Arguments**Required argument**

pathName

A String specifying the path to be used when the model database is saved to a file. If you do not provide a file extension, **.cae** is appended automatically to the path.

Optional arguments

None.

Return value

None

Exceptions

If *pathName* is **abaqus.cae**:

MdbError: “abaqus.cae” is an invalid CAE filename.

If the command fails to save the Mdb object to disk for reasons not mentioned above:

MdbError: cannot save file...

30.1.14 openAuxMdb(...)

This method opens an auxiliary Mdb object on the disk at the specified location. This enables models from the auxiliary Mdb object to be copied into the current Mdb.

Arguments

Required argument

pathName

A String specifying the path to the auxiliary Mdb which is to be opened. If you do not provide a file extension, **.cae** is appended automatically to the path.

Optional arguments

None.

Return value

None

Exceptions

If the file is an invalid model database:

MdbError: invalid model database.

If the file contains a model database from an Abaqus release other than the Abaqus release you are currently running:

MdbError: incompatible release number, expected *<Abaqus release>*, got *<earlier or later Abaqus release>*.

If the command fails to open the model database file for reasons not mentioned above:

MdbError: cannot open file...

30.1.15 **closeAuxMdb()**

This method closes the auxiliary Mdb which had been opened earlier using the **openAuxMdb** command.

Arguments

None.

Return value

None

Exceptions

If the auxiliary Mdb was not opened earlier.

MdbError: The auxiliary Mdb was not opened..

30.1.16 **getAuxMdbModelNames()**

This method returns a list of model names present in the auxiliary Mdb which had been opened earlier using the **openAuxMdb** command.

Arguments

None.

Return value

A list of model names present in the auxiliaryMdb

Exceptions

If the auxiliary Mdb was not opened earlier.

MdbError: The auxiliary Mdb was not opened..

30.1.17 **copyAuxMdbModel(...)**

This method copies a specified model from the auxiliary Mdb which had been opened earlier using the **openAuxMdb** command.

Arguments**Required argument**

fromName

A String specifying the model name in the auxiliary Mdb which is to be copied.

Optional argument

toName

A String specifying the name to be given to the model after it is copied into the Mdb. If this argument is not specified *toName* is assumed to be the same as *fromName*. If a model with name *toName* already exists in Mdb, it is overwritten.

Return value

None

Exceptions

If the auxiliary Mdb was not opened earlier.

MdbError: The auxiliary Mdb was not opened.

If the model *fromName* does not exist in the auxiliary Mdb.

KeyError: *fromName* does not exist.

30.1.18 Members

The Mdb object has members with the same names and descriptions as the arguments to the **Mdb** method.

In addition, the Mdb object can have the following members:

version

An Int specifying the release number of the Mdb object in memory.

lastChangedCount

A Float specifying the value of a counter associated with the Mdb object. The counter indicates when the Mdb object was last changed.

jobs

A repository of Job objects.

adaptivityProcesses

A repository of AdaptivityProcess objects.

coexecutions

A repository of Coexecution objects.

optimizationProcesses

A repository of OptimizationProcess objects.

meshEditOptions

A MeshEditOptions object specifying the undo/redo behavior when editing meshes on parts or part instances.

models

A repository of Model objects.

customData

A RepositorySupport object.

annotations

A repository of Annotation objects.

30.2 Mdb commands

The following command upgrades a model database (**.cae**) to the current release and writes the upgraded model database to a new file.

30.2.1 upgradeMdb(...)

This method upgrades an existing Mdb object to the current release and writes the upgraded version of the Mdb object to a file. In addition, Abaqus/CAE writes information about the status of the upgrade to the log file (*upgradedMdbPath.log*).

Path

upgradeMdb

Arguments

Required arguments

existingMdbPath

A String specifying the path to the file containing the model database to be upgraded.

upgradedMdbPath

A String specifying the path to the file that will contain the upgraded model database.

Optional arguments

None.

Return value

None

Exceptions

If the model database upgrade fails:

MdbError: cannot convert file

30.2.2 CombineOptResults(...)

This method combines the results from existing ODB files for each optimization cycle and writes a merged ODB file.

Path

CombineOptResults

Arguments

Required arguments

optResultLocation

A String specifying the path to the folder in which optimization results are present.

Optional arguments

optIter

A Symbolic Constant to specify the optimization cycles from which the results should be merged. The possible values are INITIAL_AND_LAST, NONE, ALL, LAST, EVERY_NCYCLES, SPECIFY.

The default value is INITIAL_AND_LAST.

nValues

An Int or a tuple of Ints specifying the optimization cycles from which the results should be merged. This argument is used only when EVERY_NCYCLES or SPECIFY is selected for *optIter*.

The default value is ALL.

models

A tuple of strings specifying the list of *models* for which the merging of results is performed.

The default value is ALL.

steps

A tuple of strings specifying the list of steps from the selected *models* to be included in the odb merge.

The default value is ALL.

analysisFieldVariables

A tuple of strings specifying the list of *analysisFieldVariables* to be included in the odb merge.

The default value is ALL.

includeResultsFrom

A Symbolic Constant to specify the target odb to which the results will be merged. The possible values are ORIGINAL_MODEL, FIRST or LAST.

The default value is FIRST.

originalModel

A String to specify the path of target odb if *includeResultsFrom* is set to ORIGINAL_MODEL.

31. Mesh commands

Mesh commands are used to mesh part instances and regions. Mesh commands are also used to assign element sizes, element types, and mesh control parameters.

31.1 Assembly object

The following commands operate on Assembly objects. For more information about the Assembly object, see “Assembly object,” Section 6.1.

Access

```
import mesh
```

31.1.1 assignStackDirection(...)

This method assigns a stack direction to geometric cells. The stack direction will be used to orient the elements during mesh generation.

Arguments

Required arguments

cells

A sequence of Cell objects specifying regions where to assign the stack direction.

referenceRegion

A Face object specifying the top side of the stack direction.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.2 associateMeshWithGeometry(...)

This method associates a geometric entity with mesh entities that are either orphan elements, bounds orphan elements, or were created using the bottom-up meshing technique.

Arguments

Required argument

geometricEntity

A Cell , a Face , an Edge , or a Vertex object specifying geometric entity to be associated with one or more mesh entities.

If the geometric entity is a Cell object then the argument *elements* must be specified.

If the geometric entity is a Face object then the argument *elemFaces* must be specified.

If the geometric entity is an Edge object then the argument *elemEdges* must be specified.

If the geometric entity is a Vertex object then the argument *node* must be specified.

Optional arguments

elements

A sequence of MeshElement objects specifying the elements to be associated with the geometric cell.

elemFaces

A sequence of MeshFace objects specifying the element faces to be associated with the geometric face.

elemEdges

A sequence of MeshEdge objects specifying the element edges to be associated with the geometric edge.

node

A MeshNode object specifying the mesh node to be associated with the geometric vertex.

Return value

None

Exceptions

None.

31.1.3 createVirtualTopology(...)

This method creates a virtual topology feature by automatically merging faces and edges based on a set of geometric parameters. The edges and vertices that are being merged will be ignored during mesh generation.

Arguments

Required argument

regions

A sequence of Face objects or PartInstance objects specifying the domain to search for geometric entities that need to be merged. Entities identified as candidates to be merged may be merged with entities from outside the specified region.

Optional arguments

mergeShortEdges

A Boolean specifying whether to merge short edges. The default value is False.

shortEdgeThreshold

A Float specifying a threshold that determines which edges are considered to be short. These edges are the candidate entities to be merged. This argument is a required argument if the argument *mergeShortEdges* equals True and it is ignored if the argument *mergeShortEdges* equals False.

mergeSmallFaces

A Boolean specifying whether to merge faces with small area. The default value is False.

smallFaceAreaThreshold

A Float specifying a threshold that determines which faces are considered to have a small area. These faces are the candidate entities to be merged. This argument is a required argument if the argument *mergeSmallFaces* equals True and it is ignored if the argument *mergeSmallFaces* equals False.

mergeSliverFaces

A Boolean specifying whether to merge faces with high aspect ratio. The default value is False.

faceAspectRatioThreshold

A Float specifying a threshold that determines which faces are considered to have high aspect ratio. These faces are candidate entities to be merged. This argument is a required argument if the argument *mergeSliverFaces* equals True and it is ignored if the argument *mergeSliverFaces* equals False.

mergeSmallAngleFaces

A Boolean specifying whether to merge faces that have a sharp corner angle. The default value is False.

smallFaceCornerAngleThreshold

A Float specifying a threshold that determines which face corner angles are considered to be small. These faces will be candidate entities to be merged. This argument is a required argument if the argument *mergeSmallAngleFaces* equals True and it is ignored if the argument *mergeSmallAngleFaces* equals False.

mergeThinStairFaces

A Boolean specifying whether to merge faces that represent a thin stair-like feature. The default value is False.

MESH COMMANDS

thinStairFaceThreshold

A Float specifying a threshold that determines which faces representing small stair-like features are considered thin. These faces will be candidate entities to be merged. This argument is required if the argument *mergeThinStairFaces* is True and it is ignored if *mergeThinStairFaces* is False.

ignoreRedundantEntities

A Boolean specifying whether to abstract away redundant edges and vertices. The default value is False.

cornerAngleTolerance

A Float specifying the angle deviation from 180 degrees at a vertex or at an edge such that the two edges radiating from the vertex or the two faces bounded by the edge can be merged. The default value is 30.0 degrees.

applyBlendControls

A Boolean specifying whether to verify that blend faces can be merged with neighboring faces. If *applyBlendControls* is True then all faces that have angle larger than *blendSubtendedAngleTolerance* and a radius smaller than *blendRadiusTolerance* will not be merged with neighboring faces unless the neighboring faces are also blend faces with similar geometric characteristics. The default value is False.

blendSubtendedAngleTolerance

A Float specifying the largest subtended angle of blend faces that can be merged with neighboring faces. This argument is a required argument if the argument *applyBlendControls* equals True and it is ignored if the argument *applyBlendControls* equals False.

blendRadiusTolerance

A Float specifying the smallest radius of curvature of blend faces that can be merged with neighboring faces. This argument is a required argument if the argument *applyBlendControls* equals True and it is ignored if the argument *applyBlendControls* equals False.

Return value

A Feature object.

Exceptions

None.

31.1.4 deleteBoundaryLayerControls(...)

This method deletes the control parameters for boundary layer mesh for all the specified regions.

Arguments**Required argument***regions*

A sequence of Cell objects specifying the regions for which to set the boundary layer mesh control parameters.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.5 deleteMesh(...)

This method deletes a subset of the mesh that contains the native elements from the given part instances or regions.

Arguments**Required argument***regions*

A sequence of PartInstance objects or Region objects specifying the part instances or regions from where the native mesh is to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.6 deleteMeshAssociationWithGeometry(...)

This method deletes the association of geometric entities with mesh entities.

Arguments

Required argument

geometricEntities

A sequence of Cell objects, Face objects, Edge objects, or Vertex objects specifying the geometric entities that will be disassociated from the mesh.

Optional argument

addBoundingEntities

A Boolean specifying whether the mesh will also be disassociated from the geometric entities that bounds the given *geometricEntities*. For example, if the argument *geometricEntities* contains a face, this boolean indicates whether the edges and vertices that bound the face will also be disassociated from the mesh. The default value is False.

Return value

None

Exceptions

None.

31.1.7 deletePreviewMesh()

This method deletes all boundary meshes in the assembly. See the *boundaryPreview* argument of **generateMesh** for information about generating boundary meshes.

Arguments

None.

Return value

None

Exceptions

None.

31.1.8 deleteSeeds(...)

This method deletes the global edge seeds from the given part instances or deletes the local edge seeds from the given edges.

Arguments**Required argument***regions*

A sequence of PartInstance objects or Edge objects specifying the part instances or edges from which the seeds are to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.9 generateBottomUpExtrudedMesh(...)

This method generates solid elements by extruding a 2D mesh along a vector, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments**Required arguments***cell*

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native part instances.

numberOfLayers

An Int specifying the number of layers to be generated along the extrusion vector.

extrudeVector

A sequence of sequences of Floats specifying the start point and end point of a vector. Each point is defined by a tuple of three coordinates indicating its position. The direction of the mesh extrusion operation is from the first point to the second point.

Optional arguments

At least one of the three optional "SourceSide" arguments must be specified.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the extrude meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the extrude meshing operation.

MESH COMMANDS

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the extrude meshing operation.

depth

A Float specifying the distance of the mesh extrusion. If unspecified, the vector length of the *extrudeVector* argument is assumed.

targetSide

A datum plane, a sequence of Face objects, a sequence of MeshFace objects, or a sequence of 2D MeshElement objects specifying the target of the extrude meshing operation. If specified, this argument overrides the *depth* argument, and all points on the source will be extruded in the direction of the extrusion vector until meeting the target.

biasRatio

A Float specifying a ratio of the element size in the extrusion direction between the source and the target sides of the extrusion. The default is 1.0, meaning no bias.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include extruded elements. This argument is ignored for native part instances. The default value is False.

Return value

None

Exceptions

None.

31.1.10 generateBottomUpSweptMesh(...)

This method generates solid elements by sweeping a 2D mesh, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments

Required argument

cell

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native part instances.

Optional arguments

At least one of the following three arguments must be specified: *geometrySourceSide*, *elemFacesSourceSide*, or *elemSourceSide*.

In addition one of the following two sets of arguments must also be specified:

- One of the following three arguments: *geometryConnectingSides*, *elemFacesConnectingSides*, or *elemConnectingSides*. In this case the *targetSide* is an optional argument.
- *targetSide* and *numberOfLayers*.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the sweep meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the sweep meshing operation.

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the sweep meshing operation.

geometryConnectingSides

A Region of Face objects specifying the connecting sides of the sweep meshing operation.

elemFacesConnectingSides

A sequence of MeshFace objects specifying connecting sides of the sweep meshing operation.

elemConnectingSides

A sequence of 2D MeshElement objects specifying connecting sides of the sweep meshing operation.

targetSide

A Face object specifying the target side of the sweep meshing operation.

numberOfLayers

An Int specifying the number of layers to be generated along the sweep direction.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include swept elements. This argument is ignored for native part instances. The default value is False.

Return value

None

Exceptions

None.

31.1.11 **generateBottomUpRevolvedMesh(...)**

This method generates solid elements by revolving a 2D mesh around an axis, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments

Required arguments

cell

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native part instances.

numberOfLayers

An Int specifying the number of layers of elements to be generated around the axis of revolution.

axisOfRevolution

A sequence of sequences of Floats specifying the two points of the vector that describes the axis of revolution. Each point is defined by a tuple of three coordinates indicating its position. The direction of the axis of revolution is from the first point to the second point. The orientation of the revolution operation follows the right-hand-rule about the axis of revolution.

angleOfRevolution

A Float specifying the angle of revolution.

Optional arguments

At least one of the three optional "SourceSide" arguments must be specified.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the revolve meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the revolve meshing operation.

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the revolve meshing operation.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include extruded elements. This argument is ignored for native part instances. The default value is False.

Return value

None

Exceptions

None.

31.1.12 generateMesh(...)

This method generates a mesh in the given part instances or regions.

Arguments

Required arguments

None.

Optional arguments

regions

A sequence of PartInstance objects or Region objects specifying the part instances or regions where the mesh is to be generated.

seedConstraintOverride

A Boolean specifying whether mesh generation is allowed to modify seed constraints. The default value is OFF.

meshTechniqueOverride

A Boolean specifying whether mesh generation is allowed to modify the existing mesh techniques so that a compatible mesh can be generated. The default value is OFF.

boundaryPreview

A Boolean specifying whether the generated mesh should be a boundary mesh. This option will only have an effect if any of the specified regions are to be meshed with tetrahedral elements or using the bottom-up technique with hexahedral or wedge elements. The default value is OFF.

boundaryMeshOverride

A Boolean specifying whether mesh generation is allowed to modify an existing boundary preview mesh. This option will only have an effect if any of the specified regions are to be meshed with tetrahedral elements and a boundary preview mesh already exists. The default value is OFF.

Return value

None

Exceptions

None.

31.1.13 getEdgeSeeds(...)

This method returns an edge seed parameter for a specified edge of an assembly.

Arguments

Required arguments

edge

An Edge object specifying the edge to be queried.

attribute

A SymbolicConstant specifying the type of edge seed attribute to return. Possible values are:

- EDGE_SEEDING_METHOD
- BIAS_METHOD
- NUMBER
- AVERAGE_SIZE
- DEVIATION_FACTOR
- MIN_SIZE_FACTOR
- BIAS_RATIO
- BIAS_MIN_SIZE
- BIAS_MAX_SIZE
- VERTEX_ADJ_TO_SMALLEST_ELEM
- SMALLEST_ELEM_LOCATION
- CONSTRAINT

The return value is dependent on the *attribute* argument.

- If *attribute*=EDGE_SEEDING_METHOD, the return value is a SymbolicConstant specifying the edge seeding method used to create the seeds along the edge. Possible values are:
 - UNIFORM_BY_NUMBER
 - UNIFORM_BY_SIZE
 - CURVATURE_BASED_BY_SIZE
 - BIASED
 - NONE
- If *attribute*=BIAS_METHOD, the return value is a SymbolicConstant specifying the bias type used to create the seeds along the edge. Possible values are:
 - SINGLE
 - DOUBLE
 - NONE

- If *attribute*=NUMBER, the return value is an Int specifying the number of element seeds along the edge.
- If *attribute*=AVERAGE_SIZE, the return value is a Float specifying the average element size along the edge.
- If *attribute*=DEVIATION_FACTOR, the return value is a Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length. If edge seeds are not defined, the return value is zero.
- If *attribute*=MIN_SIZE_FACTOR, the return value is a Float specifying the size of the smallest allowable element as a fraction of the specified global element size. If edge seeds are not defined, the return value is zero.
- If *attribute*=BIAS_RATIO, the return value is a Float specifying the length ratio of the largest element to the smallest element.
- If *attribute*=BIAS_MIN_SIZE, the return value is a Float specifying the length of the largest element; only applicable if the EDGE_SEEDING_METHOD is BIASED and seeds were specified by minimum and maximum sizes.
- If *attribute*=BIAS_MAX_SIZE, the return value is a Float specifying the length of the largest element; only applicable if the EDGE_SEEDING_METHOD is BIASED and seeds were specified by minimum and maximum sizes.
- If *attribute*=VERTEX_ADJ_TO_SMALLEST_ELEM, the return value is an Int specifying the ID of the vertex next to the smallest element; only applicable if the EDGE_SEEDING_METHOD is BIASED.
- If *attribute*=SMALLEST_ELEM_LOCATION, the return value is a SymbolicConstant specifying the location of smallest elements for double bias seeds; only applicable if the EDGE_SEEDING_METHOD is BIASED and BIAS_METHOD is DOUBLE. Possible values are:
 - SMALLEST_ELEM_AT_CENTER
 - SMALLEST_ELEM_AT_ENDS
 - NONE
- If *attribute*=CONSTRAINT, the return value is a SymbolicConstant specifying how close the seeds must be matched by the mesh. Possible values are:
 - FREE
 - FINER
 - FIXED
 - NONE

A value of NONE indicates that the edge is not seeded.

Optional arguments

None.

Return value

The return value is a Float, an Int, or a SymbolicConstant depending on the value of the *attribute* argument.

Exceptions

None.

31.1.14 getElementType(...)

This method returns the ElemType object of a given element shape assigned to a region of the assembly.

Arguments

Required arguments

region

A Cell , a Face , or an Edge object specifying the region to be queried.

elemShape

A SymbolicConstant specifying the shape of the element for which to return the element type.
Possible values are:

- LINE
- QUAD
- TRI
- HEX
- WEDGE
- TET

Optional arguments

None.

Return value

ElemType object.

Exceptions

TypeError

If the region cannot be associated with element types or if the *elemShape* is not consistent with the dimension of the *region*.

31.1.15 **getIncompatibleMeshInterfaces(...)**

This method returns a sequence of face objects that are meshed with incompatible elements.

Arguments

Required arguments

None.

Optional argument

cells

A sequence of cell objects which will be used to search the incompatible faces.

Return value

A sequence of Face objects.

Exceptions

None.

31.1.16 **getMeshControl(...)**

This method returns a mesh control parameter for the specified region of the assembly.

Arguments

Required arguments

region

A Cell , a Face , or an Edge object specifying the region to be queried.

attribute

A SymbolicConstant specifying the mesh control attribute to return. Possible values are:

- ELEM_SHAPE
- TECHNIQUE
- ALGORITHM
- MIN_TRANSITION

The return value is dependent on the *attribute* argument.

- If *attribute*=ELEM_SHAPE, the return value is a SymbolicConstant specifying the element shape used during meshing. Possible values are:
 - LINE

MESH COMMANDS

- QUAD
 - TRI
 - QUAD_DOMINATED
 - HEX
 - TET
 - WEDGE
 - HEX_DOMINATED
- If *attribute*=TECHNIQUE, the return value is a SymbolicConstant specifying the meshing technique to be used during meshing. Possible values are:
 - FREE
 - STRUCTURED
 - SWEEP
 - UNMESHABLE

Where UNMESHABLE indicates that no meshing technique is applicable with the currently assigned element shape.

- If *attribute*=ALGORITHM, the return value is a SymbolicConstant specifying the meshing algorithm to be used during meshing. Possible values are:
 - MEDIAL_AXIS
 - ADVANCING_FRONT
 - DEFAULT
 - NON_DEFAULT
 - NONE

Where NONE indicates that no algorithm is applicable.

- If *attribute*=MIN_TRANSITION, the return value is a Boolean indicating whether minimum transition will be used during meshing. This option is applicable only to the following:
 - Free quadrilateral meshing or sweep hexahedral meshing with *algorithm*=MEDIAL_AXIS.
 - Structured quadrilateral meshing.

Optional arguments

None.

Return value

The return value is a `SymbolicConstant` or a `Boolean` depending on the value of the *attribute* argument.

Exceptions

`TypeError`

The region cannot carry mesh controls.

31.1.17 getMeshStats(...)

This method returns the mesh statistics for the given part instances or regions.

Arguments**Required argument**

regions

A sequence or tuple of `PartInstance` objects or `Geometry` regions for which mesh statistics should be returned.

Optional arguments

None.

Return value

A `MeshStats` object.

Exceptions

None.

31.1.18 getPartSeeds(...)

This method returns a part seed parameter for the specified instance.

Arguments**Required arguments**

region

A `PartInstance` object specifying the part instance to be queried.

attribute

A `SymbolicConstant` specifying the type of part seed attribute to return. Possible values are:

- `SIZE`
- `DEFAULT_SIZE`

- `DEVIATION_FACTOR`
- `MIN_SIZE_FACTOR`

The return value is dependent on the value of the *attribute* argument.

- If *attribute*=`SIZE`, the return value is a Float specifying the assigned global element size. If part seeds are not defined, the return value is zero.
- If *attribute*=`DEFAULT_SIZE`, the return value is a Float specifying a suggested default global element size based upon the part geometry.
- If *attribute*=`DEVIATION_FACTOR`, the return value is a Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length. If part seeds are not defined, the return value is zero.
- If *attribute*=`MIN_SIZE_FACTOR`, the return value is a Float specifying the size of the smallest allowable element as a fraction of the specified global element size. If part seeds are not defined, the return value is zero.

Optional arguments

None.

Return value

The return value is a Float, and its value is dependent on the *attribute* argument.

Exceptions

An exception occurs if the part instance does not contain native geometry.

Error: Part instance does not contain native geometry

31.1.19 `getUnmeshedRegions()`

This method returns all geometric regions in the assembly that require a mesh for submitting an analysis but are either unmeshed or are meshed incompletely.

Arguments

None.

Return value

A Region object, or None.

Exceptions

None.

31.1.20 ignoreEntity(...)

This method creates a virtual topology feature. Virtual topology allows unimportant entities to be ignored during mesh generation. You can combine two adjacent faces by specifying a common edge to ignore. Similarly, you can combine two adjacent edges by specifying a common vertex to ignore.

Arguments

Required argument

entities

A sequence of vertices and edges specifying the entities to be ignored during meshing.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

31.1.21 restoreIgnoredEntity(...)

This method restores vertices and edges that have been merged using a virtual topology feature.

Arguments

Required argument

entities

A sequence of IgnoredVertex objects and IgnoredEdge objects specifying the entities to be restored.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

31.1.22 seedEdgeByBias(...)

This method seeds the given edges nonuniformly using the specified number of elements and bias ratio or the specified minimum and maximum element sizes.

Arguments

Required arguments

biasMethod

A SymbolicConstant specifying whether single- or double-biased seed distribution will be applied. If unspecified, single-biased seed distribution will be applied. Possible values are:

- SINGLE: Single-biased seed distribution will be applied.
- DOUBLE: Double-biased seed distribution will be applied.

end1Edges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near the end where the normalized curve parameter=0.0. You must provide either the *end1Edges* or the *end2Edges* argument or both when *biasMethod*=SINGLE and omit both of them when *biasMethod*=DOUBLE.

Note: You can determine which end is which by the order of the vertex indices returned by “getVertices,” Section 7.3.10.

end2Edges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near the end where the normalized curve parameter=1.0.

centerEdges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near edge center. You must provide either the *centerEdges* or the *endEdges* argument or both when *biasMethod*=DOUBLE and omit both of them when *biasMethod*=SINGLE.

endEdges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near edge ends.

ratio

A Float specifying the ratio of the largest element to the smallest element. Possible values are $1.0 \leq \text{ratio} \leq 10^6$.

number

An Int specifying the number of elements along each edge. Possible values are $1 \leq \text{number} \leq 10^4$.

minSize

A Float specifying the desired smallest element size.

maxSize

A Float specifying the desired largest element size.

Note: You must specify either the *ratio* and *number* or *minSize* and *maxSize* pair of arguments.

Optional argument

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.1.23 seedEdgeByNumber(...)

This method seeds the given edges uniformly based on the number of elements along the edges.

Arguments

Required arguments

edges

A sequence of Edge objects specifying the edges to seed.

number

An Int specifying the number of elements along each edge. Possible values are $1 \leq \textit{number} \leq 10^4$.

Optional argument

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.1.24 seedEdgeBySize(...)

This method seeds the given edges either uniformly or following edge curvature distribution, based on the desired element size.

Arguments

Required arguments

edges

A sequence of Edge objects specifying the edges to seed.

size

A Float specifying the desired element size.

Optional arguments

deviationFactor

A Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length.

minSizeFactor

A Float specifying the size of the smallest allowable element as a fraction of the specified global element size.

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.1.25 seedPartInstance(...)

This method assigns global edge seeds to the given part instances.

Arguments**Required arguments**

regions

A sequence of PartInstance objects specifying the part instances to seed.

size

A Float specifying the desired global element size for the edges.

Optional arguments

deviationFactor

A Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length.

minSizeFactor

A Float specifying the size of the smallest allowable element as a fraction of the specified global element size.

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.

Return value

None

Exceptions

None.

31.1.26 setBoundaryLayerControls(...)

This method sets the control parameters for boundary layer mesh for the specified regions.

Arguments

Required arguments

regions

A sequence of Cell objects specifying the regions for which to set the boundary layer mesh control parameters.

firstElemSize

A Float specifying the height of the first element layer off boundary. Possible values are $0.0 < firstElemSize \leq 10^6$.

growthFactor

A Float specifying the ratio of heights of any two consecutive element layers. Possible values are $1.0 \leq growthFactor \leq 10.0$.

numLayers

An Int specifying the number of element layers to be generated. Possible values are $1 \leq numLayers \leq 10^4$.

Optional arguments

inactiveFaces

A sequence of Face objects specifying the faces where boundary layer should not be generated. By default, boundary layer mesh will be generated on all faces of the selected regions.

setName

A String specifying a unique name for a set that will contain boundary layer elements.

Return value

None

Exceptions

None.

31.1.27 setElementType(...)

This method assigns element types to the specified regions.

Arguments

Required arguments

regions

A sequence of Geometry regions or MeshElement objects, or a Set object containing either geometry regions or elements, specifying the regions to which element types are to be assigned.

elemTypes

A sequence of ElemType objects, one for each element shape applicable to the regions.

Note: If an ElemType object has an UNKNOWN_xxx value for *elemCode*, its order will be deduced from the order of other valid ElemType objects within the same **setElementType** command. If no valid ElemType objects can be found, the order will remain unchanged.

Optional arguments

None.

Return value

None

Exceptions

As a result of the element assignment, a region must have the same library, family, and order for all its assigned element types. Otherwise, an exception will be thrown.

For example, suppose the Hex, Wedge, and Tet elements previously assigned to a cell are all linear. The user now constructs an ElemType object with a quadratic Hex element and includes only this object in the **setElementType** command. An exception will be thrown because the Wedge and Tet elements will remain linear (i.e., As Is) and become incompatible with the newly assigned quadratic Hex element.

31.1.28 setLogicalCorners(...)

This method sets the logical corners for a mappable face region.

Arguments

Required arguments

region

A Face region.

corners

Three, four, or five Vertex objects defining the logical corners for a given mappable face region.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.29 **setMeshControls(...)**

This method sets the mesh control parameters for the specified regions.

Arguments

Required argument

regions

A sequence of Face or Cell regions specifying the regions for which to set the mesh control parameters.

Optional arguments

elemShape

A SymbolicConstant specifying the element shape to be used for meshing. The default value is QUAD for Face regions and HEX for Cell regions. If unspecified, the existing element shape will remain unchanged. Possible values are:

- QUAD: Quadrilateral mesh.
- QUAD_DOMINATED: Quadrilateral-dominated mesh.
- TRI: Triangular mesh.
- HEX: Hexahedral mesh.
- HEX_DOMINATED: Hex-dominated mesh.
- TET: Tetrahedral mesh.
- WEDGE: Wedge mesh.

technique

A SymbolicConstant specifying the mesh technique to be used. The default value is FREE for Face regions. For Cell regions the initial value depends on the geometry of the regions and can be STRUCTURED, SWEEP, or “unmeshable.” If unspecified, the existing mesh technique(s) will remain unchanged. Possible values are:

- FREE: Free mesh technique.
- STRUCTURED: Structured mesh technique.
- SWEEP: Sweep mesh technique.
- BOTTOM_UP: Bottom-up mesh technique. Only applicable for cell regions.
- SYSTEM_ASSIGN: Allow the system to assign a suitable technique. The actual technique assigned can be STRUCTURED, SWEEP, or “unmeshable.”

algorithm

A SymbolicConstant specifying the algorithm used to generate the mesh for the specified regions. Possible values are MEDIAL_AXIS, ADVANCING_FRONT, and NON_DEFAULT. If unspecified, the existing value will remain unchanged. This option is applicable only to the following:

- Free quadrilateral or quadrilateral-dominated meshing. In this case the possible values are MEDIAL_AXIS and ADVANCING_FRONT.
- Sweep hexahedral or hexahedral-dominated meshing. In this case the possible values are MEDIAL_AXIS and ADVANCING_FRONT.
- Free tetrahedral meshing. In this case the only possible value is NON_DEFAULT, and it indicates that the free tetrahedral-meshing technique available in Abaqus 6.4 or earlier will be used. If algorithm is not specified, the default tetrahedral-meshing technique will be used.

minTransition

A Boolean specifying whether minimum transition is to be applied. The default value is ON. If unspecified, the existing value will remain unchanged. This option is applicable only in the following cases:

- Free quadrilateral meshing or hexahedral sweep meshing with *algorithm*=MEDIAL_AXIS.
- Structured quadrilateral meshing.

sizeGrowth

A SymbolicConstant specifying element size growth to be applied when generating the interior of a tetrahedral mesh. Possible values are MODERATE and MAXIMUM. If unspecified, the existing value will remain unchanged. This option only applies to the default tetrahedral mesher.

allowMapped

A Boolean specifying whether mapped meshing can be used to replace the selected mesh technique. The *allowMapped* argument is applicable only in the following cases:

- Free triangular meshing.
- Free quadrilateral or quadrilateral-dominated meshing with *algorithm*=ADVANCING_FRONT.
- Hexahedral or hexahedral-dominated sweep meshing with *algorithm*=ADVANCING_FRONT.
- Free tetrahedral meshing. *allowMapped*=True implies that mapped triangular meshing can be used on faces that bound three-dimensional *regions*.

Return value

None

Exceptions

None.

31.1.30 **setSweepPath(...)**

This method sets the sweep path for a sweepable region or the revolve path for a revolvable region.

Arguments

Required arguments

region

A sweepable region.

edge

An Edge object specifying the sweep or revolve path.

sense

A SymbolicConstant specifying the sweep sense. The sense will affect only how gasket elements will be created; it will have no effect if gasket elements are not used. Possible values are FORWARD or REVERSE.

If *sense*=FORWARD, the sense of the given edge's underlying curve will be used.

Optional arguments

None.

Return value

None

Exceptions

None.

31.1.31 **verifyMeshQuality(...)**

This method tests the quality of part instance meshes and returns poor-quality elements.

Arguments

Required argument

criterion

A SymbolicConstant specifying the criterion used for the quality check. Possible values are:

ANALYSIS_CHECKS

When this criterion is specified Abaqus/CAE will invoke the element quality checks included with the input file processor for Abaqus/Standard and Abaqus/Explicit.

ANGULAR_DEVIATION

The maximum amount (in degrees) that an element's face corner angles deviate from the ideal angle. The ideal angle is 90° for quadrilateral element faces and 60° for triangular element faces. Elements with an angular deviation larger than the specified threshold will fail this test.

ASPECT_RATIO

The ratio between the lengths of the longest and shortest edges of an element. Elements with an aspect ratio larger than the specified threshold will fail this test.

GEOM_DEVIATION_FACTOR

The largest geometric deviation factor evaluated along any of the element edges associated with geometric edges or faces. The geometric deviation factor along an element edge is calculated by dividing the maximum gap between the element edge and its associated geometry by the length of the element edge. Elements with a geometric deviation factor larger than the specified threshold will fail this test.

LARGE_ANGLE

The largest corner angle on any of an element's faces. Elements with face angles larger than the specified threshold (in degrees) will fail this test.

LONGEST_EDGE

The length of an element's longest edge. Elements with an edge longer than the specified threshold will fail this test.

MAX_FREQUENCY

An estimate of an element's contribution to the initial maximum allowable frequency for Abaqus/Standard analyses. This calculation requires appropriate section assignments and material definitions. Elements whose maximum allowable frequency is smaller than the given value will fail this test.

SHAPE_FACTOR

The shape factor for triangular and tetrahedral elements. This is the ratio between the element area or volume and the optimal element area or volume. Elements with a shape factor smaller than the specified threshold will fail this test.

SHORTEST_EDGE

The length of an element's shortest edge. Elements with an edge shorter than the specified threshold will fail this test.

SMALL_ANGLE

The smallest corner angle on any of an element's faces. Elements with face angles smaller than the given value (in degrees) will fail this test.

STABLE_TIME_INCREMENT

An estimate of an element's contribution to the initial maximum stable time increment for Abaqus/Explicit analyses. This calculation requires appropriate section assignments and material definitions. Elements that require a time increment smaller than the given value will fail this test.

Optional arguments

threshold

A Float value used to determine low quality elements according to the specified criterion. This argument is ignored when the ANALYSIS_CHECKS criterion is used. For other criterion, if this argument is unspecified then no list of failed elements will be returned.

elemShape

A SymbolicConstant specifying an element shape for limiting the query. Possible values are LINE, QUAD, TRI, HEX, WEDGE, and TET.

regions

A sequence of Region or MeshElement objects. If you do not specify the *regions* argument, all meshes in the assembly are considered.

Return value

A Dictionary object containing values for some number of the following keys: failedElements, warningElements, naElements (sequences of MeshElement objects); numElements (Int); average, worst (Float); worstElement (MeshElement object)

Exceptions

None.

31.2 Part object

The following commands operate on Part objects. For more information about the Part object, see “Part object,” Section 37.1.

Access

`import mesh`

31.2.1 assignStackDirection(...)

This method assigns a stack direction to geometric cells. The stack direction will be used to orient the elements during mesh generation.

Arguments

Required arguments

cells

A sequence of Cell objects specifying regions where to assign the stack direction.

referenceRegion

A Face object specifying the top side of the stack direction.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.2 associateMeshWithGeometry(...)

This method associates a geometric entity with mesh entities that are either orphan elements, bounds orphan elements, or were created using the bottom-up meshing technique.

Arguments

Required argument

geometricEntity

A Cell , a Face , an Edge , or a Vertex object specifying geometric entity to be associated with one or more mesh entities.

If the geometric entity is a Cell object then the argument *elements* must be specified.

If the geometric entity is a Face object then the argument *elemFaces* must be specified.

If the geometric entity is an Edge object then the argument *elemEdges* must be specified.

If the geometric entity is a Vertex object then the argument *node* must be specified.

Optional arguments

elements

A sequence of MeshElement objects specifying the elements to be associated with the geometric cell.

MESH COMMANDS

elemFaces

A sequence of MeshFace objects specifying the element faces to be associated with the geometric face.

elemEdges

A sequence of MeshEdge objects specifying the element edges to be associated with the geometric edge.

node

A MeshNode object specifying the mesh node to be associated with the geometric vertex.

Return value

None

Exceptions

None.

31.2.3 createVirtualTopology(...)

This method creates a virtual topology feature by automatically merging faces and edges based on a set of geometric parameters. The edges and vertices that are being merged will be ignored during mesh generation.

Arguments

Required arguments

None.

Optional arguments

regions

A sequence of Face objects specifying the domain to search for geometric entities that need to be merged. Entities identified as candidates to be merged may be merged with entities from outside the specified region. If *regions* is not specified then the entire part is the domain for searching geometric entities that need to be merged.

mergeShortEdges

A Boolean specifying whether to merge short edges. The default value is False.

shortEdgeThreshold

A Float specifying a threshold that determines which edges are considered to be short. These edges are the candidate entities to be merged. This argument is a required argument if the argument *mergeShortEdges* equals True and it is ignored if the argument *mergeShortEdges* equals False.

mergeSmallFaces

A Boolean specifying whether to merge faces with small area. The default value is False.

smallFaceAreaThreshold

A Float specifying a threshold that determines which faces are considered to have a small area. These faces are the candidate entities to be merged. This argument is a required argument if the argument *mergeSmallFaces* equals True and it is ignored if the argument *mergeSmallFaces* equals False.

mergeSliverFaces

A Boolean specifying whether to merge faces with high aspect ratio. The default value is False.

faceAspectRatioThreshold

A Float specifying a threshold that determines which faces are considered to have high aspect ratio. These faces are the candidate entities to be merged. This argument is a required argument if the argument *mergeSliverFaces* equals True and it is ignored if the argument *mergeSliverFaces* equals False.

mergeSmallAngleFaces

A Boolean specifying whether to merge faces that have a sharp corner angle. The default value is False.

smallFaceCornerAngleThreshold

A Float specifying a threshold that determines which face corner angles are considered to be small. These faces will be candidate entities to be merged. This argument is a required argument if the argument *mergeSmallAngleFaces* equals True and it is ignored if the argument *mergeSmallAngleFaces* equals False.

mergeThinStairFaces

A Boolean specifying whether to merge faces that represent a thin stair-like feature. The default value is False.

thinStairFaceThreshold

A Float specifying a threshold that determines which faces representing small stair-like features are considered thin. These faces will be candidate entities to be merged. This argument is required if the argument *mergeThinStairFaces* is True and it is ignored if *mergeThinStairFaces* is False.

ignoreRedundantEntities

A Boolean specifying whether to abstract away redundant edges and vertices. The default value is False.

cornerAngleTolerance

A Float specifying the angle deviation from 180 degrees at a vertex or at an edge such that the two edges radiating from the vertex or the two faces bounded by the edge can be merged. The default value is 30.0 degrees.

applyBlendControls

A Boolean specifying whether to verify that blend faces can be merged with neighboring faces. If *applyBlendControls* is true then all faces that have angle larger than *blendSubtendedAngleTolerance* and a radius smaller than *blendRadiusTolerance* will not be merged with neighboring faces unless the neighboring faces are also blend faces with similar geometric characteristics. The default value is False.

blendSubtendedAngleTolerance

A Float specifying the largest subtended angle of blend faces that can be merged with neighboring faces. This argument is a required argument if the argument *applyBlendControls* equals True and it is ignored if the argument *applyBlendControls* equals False.

blendRadiusTolerance

A Float specifying the smallest radius of curvature of blend faces that can be merged with neighboring faces. This argument is a required argument if the argument *applyBlendControls* equals True and it is ignored if the argument *applyBlendControls* equals False.

Return value

A Feature object.

Exceptions

None.

31.2.4 deleteBoundaryLayerControls(...)

This method deletes the control parameters for boundary layer mesh for all the specified regions.

Arguments

Required argument

regions

A sequence of Cell objects specifying the regions for which to set the boundary layer mesh control parameters.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.5 deleteMesh(...)

This method deletes a subset of the mesh that contains the native elements from the given parts or regions.

Arguments**Required argument***regions*

A sequence of Part objects or Region objects specifying the parts or regions from which the native mesh is to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.6 deleteMeshAssociationWithGeometry(...)

This method deletes the association of geometric entities with mesh entities.

Arguments**Required argument***geometricEntities*

A sequence of Cell objects, Face objects, Edge objects, or Vertex objects specifying the geometric entities that will be disassociated from the mesh.

Optional argument*addBoundingEntities*

A Boolean specifying whether the mesh will also be disassociated from the geometric entities that bounds the given *geometricEntities*. For example, if the argument *geometricEntities* contains a face, this boolean indicates whether the edges and vertices that bound the face will also be disassociated from the mesh. The default value is False.

Return value

None

Exceptions

None.

31.2.7 **deletePreviewMesh()**

This method deletes all boundary meshes in the parts. See the *boundaryPreview* argument of **generateMesh** for information about generating boundary meshes.

Arguments

None.

Return value

None

Exceptions

None.

31.2.8 **deleteSeeds(...)**

This method deletes the global edge seeds from the given parts or deletes the local edge seeds from the given edges.

Arguments

Required argument

regions

A sequence of Part objects or Edge objects specifying the parts or edges from which the seeds are to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.9 **generateMesh(...)**

This method generates a mesh in the given parts or regions.

Arguments**Required arguments**

None.

Optional arguments*regions*

A sequence of Part objects or Region objects specifying the parts or regions where the mesh is to be generated.

seedConstraintOverride

A Boolean specifying whether mesh generation is allowed to modify seed constraints. The default value is OFF.

meshTechniqueOverride

A Boolean specifying whether mesh generation is allowed to modify the existing mesh techniques so that a compatible mesh can be generated. The default value is OFF.

boundaryPreview

A Boolean specifying whether the generated mesh should be a boundary preview mesh. This option will only have an effect if any of the specified regions are to be meshed with tetrahedral elements or using the bottom-up technique with hexahedral or wedge elements. The default value is OFF.

boundaryMeshOverride

A Boolean specifying whether mesh generation is allowed to modify an existing boundary preview mesh. This option will only have an effect if any of the specified regions are to be meshed with tetrahedral elements and a boundary preview mesh already exists. The default value is OFF.

Return value

None

Exceptions

None.

31.2.10 generateBottomUpExtrudedMesh(...)

This method generates solid elements by extruding a 2D mesh along a vector, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments

Required arguments

cell

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native parts.

numberOfLayers

An Int specifying the number of layers to be generated along the extrusion vector.

extrudeVector

A sequence of sequences of Floats specifying the start point and end point of a vector. Each point is defined by a tuple of three coordinates indicating its position. The direction of the mesh extrusion operation is from the first point to the second point.

Optional arguments

At least one of the three optional "SourceSide" arguments must be specified.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the extrude meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the extrude meshing operation.

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the extrude meshing operation.

depth

A Float specifying the distance of the mesh extrusion. If unspecified, the vector length of the *extrudeVector* argument is assumed.

targetSide

A datum plane, a sequence of Face objects, a sequence of MeshFace objects, or a sequence of 2D MeshElement objects specifying the target of the extrude meshing operation. If specified, this argument overrides the *depth* argument, and all points on the source will be extruded in the direction of the extrusion vector until meeting the target.

biasRatio

A Float specifying a ratio of the element size in the extrusion direction between the source and the target sides of the extrusion. The default is 1.0, meaning no bias.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include extruded elements. This argument is ignored for native parts. The default value is False.

Return value

None

Exceptions

None.

31.2.11 generateBottomUpSweptMesh(...)

This method generates solid elements by sweeping a 2D mesh, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments**Required argument***cell*

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native parts.

Optional arguments

At least one of the following three arguments must be specified: *geometrySourceSide*, *elemFacesSourceSide*, or *elemSourceSide*.

In addition one of the following two sets of arguments must also be specified:

- One of the following three arguments: *geometryConnectingSides*, *elemFacesConnectingSides*, or *elemConnectingSides*. In this case the *targetSide* is an optional argument.
- *targetSide* and *numberOfLayers*.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the sweep meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the sweep meshing operation.

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the sweep meshing operation.

geometryConnectingSides

A Region of Face objects specifying connecting sides of the sweep meshing operation.

elemFacesConnectingSides

A sequence of MeshFace objects specifying connecting sides of the sweep meshing operation.

MESH COMMANDS

elemConnectingSides

A sequence of 2D MeshElement objects specifying connecting sides of the sweep meshing operation.

targetSide

A Face object specifying the target side of the sweep meshing operation.

numberOfLayers

An Int specifying the number of layers to be generated along the sweep direction.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include swept elements. This argument is ignored for native parts. The default value is False.

Return value

None

Exceptions

None.

31.2.12 generateBottomUpRevolvedMesh(...)

This method generates solid elements by revolving a 2D mesh around an axis, either on an orphan mesh or within a cell region using a bottom-up technique.

Arguments

Required arguments

cell

A Cell object specifying the geometric region where the mesh is to be generated. This argument is valid only for native parts.

numberOfLayers

An Int specifying the number of layers of elements to be generated around the axis of revolution.

axisOfRevolution

A sequence of sequences of Floats specifying the two points of the vector that describes the axis of revolution. Each point is defined by a tuple of three coordinates indicating its position. The direction of the axis of revolution is from the first point to the second point. The orientation of the revolution operation follows the right-hand-rule about the axis of revolution.

angleOfRevolution

A Float specifying the angle of revolution.

Optional arguments

At least one of the three optional "SourceSide" arguments must be specified.

geometrySourceSide

A Region of Face objects specifying the geometric domain to be used as the source for the revolve meshing operation.

elemFacesSourceSide

A sequence of MeshFace objects specifying the faces of 3D elements to be used as the source for the revolve meshing operation.

elemSourceSide

A sequence of 2D MeshElement objects specifying the elements to be used as the source for the revolve meshing operation.

extendElementSets

A Boolean specifying whether existing element sets that include source elements will be extended to also include extruded elements. This argument is ignored for native parts. The default value is False.

Return value

None

Exceptions

None.

31.2.13 getEdgeSeeds(...)

This method returns an edge seed parameter for a specified edge of a part.

Arguments**Required arguments***edge*

An Edge object specifying the edge to be queried.

attribute

A SymbolicConstant specifying the type of edge seed attribute to return. Possible values are:

- EDGE_SEEDING_METHOD
- BIAS_METHOD
- NUMBER
- AVERAGE_SIZE
- DEVIATION_FACTOR
- MIN_SIZE_FACTOR
- BIAS_RATIO
- BIAS_MIN_SIZE

MESH COMMANDS

- BIAS_MAX_SIZE
- VERTEX_ADJ_TO_SMALLEST_ELEM
- SMALLEST_ELEM_LOCATION
- CONSTRAINT

The return value is dependent on the *attribute* argument.

- If *attribute*=EDGE_SEEDING_METHOD, the return value is a SymbolicConstant specifying the edge seeding method used to create the seeds along the edge. Possible values are:
 - UNIFORM_BY_NUMBER
 - UNIFORM_BY_SIZE
 - CURVATURE_BASED_BY_SIZE
 - BIASED
 - NONE
- If *attribute*=BIAS_METHOD, the return value is a SymbolicConstant specifying the bias type used to create the seeds along the edge. Possible values are:
 - SINGLE
 - DOUBLE
 - NONE
- If *attribute*=NUMBER, the return value is an Int specifying the number of element seeds along the edge.
- If *attribute*=AVERAGE_SIZE, the return value is a Float specifying the average element size along the edge.
- If *attribute*=DEVIATION_FACTOR, the return value is a Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length. If edge seeds are not defined, the return value is zero.
- If *attribute*=MIN_SIZE_FACTOR, the return value is a Float specifying the size of the smallest allowable element as a fraction of the specified global element size. If edge seeds are not defined, the return value is zero.
- If *attribute*=BIAS_RATIO, the return value is a Float specifying the length ratio of the largest element to the smallest element.
- If *attribute*=BIAS_MIN_SIZE, the return value is a Float specifying the length of the largest element; only applicable if the EDGE_SEEDING_METHOD is BIASED and seeds were specified by minimum and maximum sizes.

- If *attribute*=BIAS_MAX_SIZE, the return value is a Float specifying the length of the largest element; only applicable if the EDGE_SEEDING_METHOD is BIASED and seeds were specified by minimum and maximum sizes.
- If *attribute*=VERTEX_ADJ_TO_SMALLEST_ELEM, the return value is an Int specifying the ID of the vertex next to the smallest element; only applicable if the EDGE_SEEDING_METHOD is BIASED.
- If *attribute*=SMALLEST_ELEM_LOCATION, the return value is a SymbolicConstant specifying the location of smallest elements for double bias seeds; only applicable if the EDGE_SEEDING_METHOD is BIASED and BIAS_METHOD is DOUBLE. Possible values are:
 - SMALLEST_ELEM_AT_CENTER
 - SMALLEST_ELEM_AT_ENDS
 - NONE
- If *attribute*=CONSTRAINT, the return value is a SymbolicConstant specifying how close the seeds must be matched by the mesh. Possible values are:
 - FREE
 - FINER
 - FIXED
 - NONE

A value of NONE indicates that the edge is not seeded.

Optional arguments

None.

Return value

The return value is a Float, an Int, or a SymbolicConstant depending on the value of the *attribute* argument.

Exceptions

None.

31.2.14 getElementType(...)

This method returns the ElemType object of a given element shape assigned to a region of a part.

Arguments

Required arguments

region

A Cell , a Face , or an Edge object specifying the region to be queried.

elemShape

A SymbolicConstant specifying the shape of the element for which to return the element type.

Possible values are:

- LINE
- QUAD
- TRI
- HEX
- WEDGE
- TET

Optional arguments

None.

Return value

ElemType object.

Exceptions

TypeError

The region cannot be associated with element types or the *elemShape* is not consistent with the dimension of the *region*.

31.2.15 getIncompatibleMeshInterfaces(...)

This method returns a sequence of face objects that are meshed with incompatible elements.

Arguments

Required arguments

None.

Optional argument

cells

A sequence of cell objects which will be used to search the incompatible faces.

Return value

A sequence of Face objects.

Exceptions

None.

31.2.16 getMeshControl(...)

This method returns a mesh control parameter for the specified region of a part.

Arguments**Required arguments**

region

A Cell , a Face , or an Edge object specifying the region to be queried.

attribute

A SymbolicConstant specifying the mesh control attribute to return. Possible values are:

- ELEM_SHAPE
- TECHNIQUE
- ALGORITHM
- MIN_TRANSITION

The return value depends on the value of the *attribute* argument.

- If *attribute*=ELEM_SHAPE, the return value is a SymbolicConstant specifying the element shape used during meshing. Possible values are:
 - LINE
 - QUAD
 - TRI
 - QUAD_DOMINATED
 - HEX
 - TET
 - WEDGE
 - HEX_DOMINATED
- If *attribute*=TECHNIQUE, the return value is a SymbolicConstant specifying the meshing technique to be used during meshing. Possible values are:
 - FREE

MESH COMMANDS

- STRUCTURED
- SWEEP
- UNMESHABLE

Where UNMESHABLE indicates that no meshing technique is applicable with the currently assigned element shape.

- If *attribute*=ALGORITHM, the return value is a SymbolicConstant specifying the meshing algorithm to be used during meshing. Possible values are:
 - MEDIAL_AXIS
 - ADVANCING_FRONT
 - DEFAULT
 - NON_DEFAULT
 - NONE

Where NONE indicates that no algorithm is applicable.

- If *attribute*=MIN_TRANSITION, the return value is a Boolean indicating whether minimum transition will be used during meshing. This option is applicable only to the following:
 - Free quadrilateral meshing or sweep hexahedral meshing with *algorithm*=MEDIAL_AXIS.
 - Structured quadrilateral meshing.

Optional arguments

None.

Return value

The return value is a SymbolicConstant or a Boolean depending on the value of the *attribute* argument.

Exceptions

TypeError

The region cannot carry mesh controls.

31.2.17 getMeshStats(...)

This method returns the mesh statistics for the given regions.

Arguments**Required argument***regions*

A sequence or tuple of Geometry regions for which mesh statistics should be returned.

Optional arguments

None.

Return value

A MeshStats object.

Exceptions

None.

31.2.18 getPartSeeds(...)

This method returns a part seed parameter for the part.

Arguments**Required argument***attribute*

A SymbolicConstant specifying the type of part seed attribute to return. Possible values are:

- SIZE
- DEFAULT_SIZE
- DEVIATION_FACTOR
- MIN_SIZE_FACTOR

The return value depends on the value of the *attribute* argument.

- If *attribute*=SIZE, the return value is a Float specifying the assigned global element size. If part seeds are not defined, the return value is zero.
- If *attribute*=DEFAULT_SIZE, the return value is a Float specifying a suggested default global element size based upon the part geometry.
- If *attribute*=DEVIATION_FACTOR, the return value is a Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length. If part seeds are not defined, the return value is zero.
- If *attribute*=MIN_SIZE_FACTOR, the return value is a Float specifying the size of the smallest allowable element as a fraction of the specified global element size. If part seeds are not defined, the return value is zero.

Optional arguments

None.

Return value

The return value is a Float that depends on the value of the *attribute* argument.

Exceptions

An exception occurs if the part does not contain native geometry.

Error: Part does not contain native geometry

31.2.19 getUnmeshedRegions()

This method returns all geometric regions in the part that require a mesh for submitting an analysis but are either unmeshed or are meshed incompletely.

Arguments

None.

Return value

A Region object, or None.

Exceptions

None.

31.2.20 ignoreEntity(...)

This method creates a virtual topology feature. Virtual topology allows unimportant entities to be ignored during mesh generation. You can combine two adjacent faces by specifying a common edge to ignore. Similarly, you can combine two adjacent edges by specifying a common vertex to ignore.

Arguments

Required argument

entities

A sequence of vertices and edges specifying the entities to be ignored during meshing.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

31.2.21 restoreIgnoredEntity(...)

This method restores vertices and edges that have been merged using a virtual topology feature.

Arguments**Required argument**

entities

A sequence of IgnoredVertex objects and IgnoredEdge objects specifying the entities to be restored.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

31.2.22 seedEdgeByBias(...)

This method seeds the given edges nonuniformly using the specified number of elements and bias ratio or the specified minimum and maximum element sizes.

Arguments**Required arguments**

biasMethod

A SymbolicConstant specifying whether single- or double-biased seed distribution will be applied. If unspecified, single-biased seed distribution will be applied. Possible values are:

- SINGLE: Single-biased seed distribution will be applied.
- DOUBLE: Double-biased seed distribution will be applied.

MESH COMMANDS

end1Edges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near the end where the normalized curve parameter=0.0. You must provide either the *end1Edges* or the *end2Edges* argument or both when *biasMethod*=SINGLE and omit both of them when *biasMethod*=DOUBLE.

Note: You can determine which end is which by the order of the vertex indices returned by “getVertices,” Section 7.3.10.

end2Edges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near the end where the normalized curve parameter=1.0.

centerEdges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near edge center. You must provide either the *centerEdges* or the *endEdges* argument or both when *biasMethod*=DOUBLE and omit both of them when *biasMethod*=SINGLE.

endEdges

A sequence of Edge objects specifying the edges to seed. The smallest elements will be positioned near edge ends.

ratio

A Float specifying the ratio of the largest element to the smallest element. Possible values are $1.0 \leq \text{ratio} \leq 10^6$.

number

An Int specifying the number of elements along each edge. Possible values are $1 \leq \text{number} \leq 10^4$.

minSize

A Float specifying the desired smallest element size.

maxSize

A Float specifying the desired largest element size.

Note: You must specify either the *ratio* and *number* or *minSize* and *maxSize* pair of arguments.

Optional argument

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.2.23 seedEdgeByNumber(...)

This method seeds the given edges uniformly based on the number of elements along the edges.

Arguments**Required arguments***edges*

A sequence of Edge objects specifying the edges to seed.

number

An Int specifying the number of elements along each edge. Possible values are $1 \leq \text{number} \leq 10^4$.

Optional argument*constraint*

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.2.24 seedEdgeBySize(...)

This method seeds the given edges either uniformly or following edge curvature distribution, based on the desired element size.

Arguments

Required arguments

edges

A sequence of Edge objects specifying the edges to seed.

size

A Float specifying the desired element size.

Optional arguments

deviationFactor

A Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length.

minSizeFactor

A Float specifying the size of the smallest allowable element as a fraction of the specified global element size.

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.
- FIXED: The seeds must be exactly matched by the mesh (only with respect to the number of elements, not to the nodal positioning).

Return value

None

Exceptions

None.

31.2.25 seedPart(...)

This method assigns global edge seeds to the given parts.

Arguments

Required argument

size

A Float specifying the desired global element size for the edges.

Optional arguments*deviationFactor*

A Float specifying the deviation factor h/L , where h is the chordal deviation and L is the element length.

minSizeFactor

A Float specifying the size of the smallest allowable element as a fraction of the specified global element size.

constraint

A SymbolicConstant specifying how closely the seeds must be matched by the mesh. The default value is FREE. If unspecified, the existing constraint will remain unchanged. Possible values are:

- FREE: The resulting mesh can be finer or coarser than the specified seeds.
- FINER: The resulting mesh can be finer than the specified seeds.

Return value

None

Exceptions

None.

31.2.26 setBoundaryLayerControls(...)

This method sets the control parameters for boundary layer mesh for the specified regions.

Arguments**Required arguments***regions*

A sequence of Cell objects specifying the regions for which to set the boundary layer mesh control parameters.

firstElemSize

A Float specifying the height of the first element layer off boundary. Possible values are $0.0 < firstElemSize \leq 10^6$.

growthFactor

A Float specifying the ratio of heights of any two consecutive element layers. Possible values are $1.0 \leq growthFactor \leq 10.0$.

numLayers

An Int specifying the number of element layers to be generated. Possible values are $1 \leq numLayers \leq 10^4$.

Optional arguments

inactiveFaces

A sequence of Face objects specifying the faces where boundary layer should not be generated.
By default, boundary layer mesh will be generated on all faces of the selected regions.

setName

A String specifying a unique name for a set that will contain boundary layer elements.

Return value

None

Exceptions

None.

31.2.27 **setElementType(...)**

This method assigns element types to the specified regions.

Arguments

Required arguments

regions

A sequence of Geometry regions or MeshElement objects, or a Set object containing either geometry regions or elements, specifying the regions to which element types are to be assigned.

elemTypes

A sequence of ElemType objects, one for each element shape applicable to the regions.

Note: If an ElemType object has an UNKNOWN_XXX value for *elemCode*, its order will be deduced from the order of other valid ElemType objects within the same **setElementType** command. If no valid ElemType objects can be found, the order will remain unchanged.

Optional arguments

None.

Return value

None

Exceptions

As a result of the element assignment, a region must have the same library, family, and order for all its assigned element types. Otherwise, an exception will be thrown.

For example, suppose the Hex, Wedge, and Tet elements previously assigned to a cell are all linear. The user now constructs an ElemType object with a quadratic Hex element and includes only this object in the **setElementType** command. An exception will be thrown because the Wedge and Tet elements will remain linear (i.e., As Is) and become incompatible with the newly assigned quadratic Hex element.

31.2.28 **setLogicalCorners(...)**

This method sets the logical corners for a mappable face region.

Arguments

Required arguments

region

A Face region.

corners

Three, four, or five Vertex objects defining the logical corners for a given mappable face region.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.29 **setMeshControls(...)**

This method sets the mesh control parameters for the specified regions.

Arguments

Required argument

regions

A sequence of Face or Cell regions specifying the regions for which to set the mesh control parameters.

Optional arguments

elemShape

A SymbolicConstant specifying the element shape to be used for meshing. The default value is QUAD for Face regions and HEX for Cell regions. If unspecified, the existing element shape will remain unchanged. Possible values are:

- QUAD: Quadrilateral mesh.

MESH COMMANDS

- QUAD_DOMINATED: Quadrilateral-dominated mesh.
- TRI: Triangular mesh.
- HEX: Hexahedral mesh.
- HEX_DOMINATED: Hex-dominated mesh.
- TET: Tetrahedral mesh.
- WEDGE: Wedge mesh.

technique

A SymbolicConstant specifying the mesh technique to be used. The default value is FREE for Face regions. For Cell regions the initial value depends on the geometry of the regions and can be STRUCTURED, SWEEP, or “unmeshable.” If unspecified, the existing mesh technique(s) will remain unchanged. Possible values are:

- FREE: Free mesh technique.
- STRUCTURED: Structured mesh technique.
- SWEEP: Sweep mesh technique.
- BOTTOM_UP: Bottom-up mesh technique. Only applicable for cell regions.
- SYSTEM_ASSIGN: Allow the system to assign a suitable technique. The actual technique assigned can be STRUCTURED, SWEEP, or “unmeshable.”

algorithm

A SymbolicConstant specifying the algorithm used to generate the mesh for the specified regions. Possible values are MEDIAL_AXIS, ADVANCING_FRONT, and NON_DEFAULT. If unspecified, the existing value will remain unchanged. This option is applicable only to the following:

- Free quadrilateral or quadrilateral-dominated meshing. In this case the possible values are MEDIAL_AXIS and ADVANCING_FRONT.
- Sweep hexahedral or hexahedral-dominated meshing. In this case the possible values are MEDIAL_AXIS and ADVANCING_FRONT.
- Free tetrahedral meshing. In this case the only possible value is NON_DEFAULT, and it indicates that the free tetrahedral-meshing technique available in Abaqus 6.4 or earlier will be used. If algorithm is not specified, the default tetrahedral-meshing technique will be used.

minTransition

A Boolean specifying whether minimum transition is to be applied. The default value is ON. If unspecified, the existing value will remain unchanged. This option is applicable only in the following cases:

- Free quadrilateral meshing or hexahedral sweep meshing with *algorithm*=MEDIAL_AXIS.
- Structured quadrilateral meshing.

sizeGrowth

A SymbolicConstant specifying element size growth to be applied when generating the interior of a tetrahedral mesh. Possible values are MODERATE and MAXIMUM. If unspecified, the existing value will remain unchanged. This option only applies to the default tetrahedral mesher.

allowMapped

A Boolean specifying whether mapped meshing can be used to replace the selected mesh technique. The *allowMapped* argument is applicable only in the following cases:

- Free triangular meshing.
- Free quadrilateral or quadrilateral-dominated meshing with *algorithm=ADVANCING_FRONT*.
- Hexahedral or hexahedral-dominated sweep meshing with *algorithm=ADVANCING_FRONT*.
- Free tetrahedral meshing. *allowMapped=True* implies that mapped triangular meshing can be used on faces that bound three-dimensional *regions*.

Return value

None

Exceptions

None.

31.2.30 setSweepPath(...)

This method sets the sweep path for a sweepable region or the revolve path for a revolvable region.

Arguments**Required arguments***region*

A sweepable region.

edge

An Edge object specifying the sweep or revolve path.

sense

A SymbolicConstant specifying the sweep sense. The sense will affect only how gasket elements will be created; it will have no effect if gasket elements are not used. Possible values are FORWARD or REVERSE.

If *sense=FORWARD*, the sense of the given edge's underlying curve will be used.

Optional arguments

None.

Return value

None

Exceptions

None.

31.2.31 **verifyMeshQuality(...)**

This method tests the mesh quality of a part and returns poor-quality elements.

Arguments

Required argument

criterion

A SymbolicConstant specifying the criterion used for the quality check. Possible values are:

ANALYSIS_CHECKS

When this criterion is specified Abaqus/CAE will invoke the element quality checks included with the input file processor for Abaqus/Standard and Abaqus/Explicit.

ANGULAR_DEVIATION

The maximum amount (in degrees) that an element's face corner angles deviate from the ideal angle. The ideal angle is 90° for quadrilateral element faces and 60° for triangular element faces. Elements with an angular deviation larger than the specified threshold will fail this test.

ASPECT_RATIO

The ratio between the lengths of the longest and shortest edges of an element. Elements with an aspect ratio larger than the specified threshold will fail this test.

GEOM_DEVIATION_FACTOR

The largest geometric deviation factor evaluated along any of the element edges associated with geometric edges or faces. The geometric deviation factor along an element edge is calculated by dividing the maximum gap between the element edge and its associated geometry by the length of the element edge. Elements with a geometric deviation factor larger than the specified threshold will fail this test.

LARGE_ANGLE

The largest corner angle on any of an element's faces. Elements with face angles larger than the specified threshold (in degrees) will fail this test.

LONGEST_EDGE

The length of an element's longest edge. Elements with an edge longer than the specified threshold will fail this test.

MAX_FREQUENCY

An estimate of an element's contribution to the initial maximum allowable frequency for Abaqus/Standard analyses. This calculation requires appropriate section assignments and material definitions. Elements whose maximum allowable frequency is smaller than the given value will fail this test.

SHAPE_FACTOR

The shape factor for triangular and tetrahedral elements. This is the ratio between the element area or volume and the optimal element area or volume. Elements with a shape factor smaller than the specified threshold will fail this test.

SHORTEST_EDGE

The length of an element's shortest edge. Elements with an edge shorter than the specified threshold will fail this test.

SMALL_ANGLE

The smallest corner angle on any of an element's faces. Elements with face angles smaller than the given value (in degrees) will fail this test.

STABLE_TIME_INCREMENT

An estimate of an element's contribution to the initial maximum stable time increment for Abaqus/Explicit analyses. This calculation requires appropriate section assignments and material definitions. Elements that require a time increment smaller than the given value will fail this test.

Optional arguments*threshold*

A Float value used to determine low quality elements according to the specified criterion. This argument is ignored when the ANALYSIS_CHECKS criterion is used. For other criterion, if this argument is unspecified then no list of failed elements will be returned.

elemShape

A SymbolicConstant specifying an element shape for limiting the query. Possible values are LINE, QUAD, TRI, HEX, WEDGE, and TET.

regions

A sequence of Region or MeshElement objects. If you do not specify the *regions* argument, the entire part mesh is considered.

Return value

A Dictionary object containing values for some number of the following keys: `failedElements`, `warningElements`, `naElements` (sequences of `MeshElement` objects); `numElements` (Int); `average`, `worst` (Float); `worstElement` (`MeshElement` object)

Exceptions

None.

31.3 ElemType object

The `ElemType` object is an argument object used as an argument in the `setElementType` command.

Access

```
import mesh
```

31.3.1 ElemType(...)

This method creates an `ElemType` object.

Path

ElemType

Required argument

elemCode

A `SymbolicConstant` specifying the Abaqus element code or just the element shape. Possible values are:

- `C3D8R`, specifying a 8-node linear brick, reduced integration with hourglass control.
- `CODE`, specifying add more codes.
- `UNKNOWN_TRI`, specifying an unknown element type associated with a triangular shape.
- `UNKNOWN_QUAD`, specifying an unknown element type associated with a quadrilateral shape.
- `UNKNOWN_HEX`, specifying an unknown element type associated with a hexahedral shape.
- `UNKNOWN_WEDGE`, specifying an unknown element type associated with a wedge shape.
- `UNKNOWN_TET`, specifying an unknown element type associated with a tetrahedral shape.

Optional arguments*elemLibrary*

A SymbolicConstant specifying the Abaqus element library to use. Possible values are STANDARD and EXPLICIT. The default value is STANDARD.

hourglassStiffness

A Float specifying the hourglass stiffness. (For shell elements this is the membrane hourglass stiffness.) A value of zero indicates the default value should be used. The default value will be used where appropriate. The default value is 0.0.

This argument is applicable only to some Abaqus/Standard elements.

bendingHourglass

A Float specifying the bending hourglass stiffness. A value of zero indicates the default value should be used. The default value will be used where appropriate. The default value is 0.0.

This argument is applicable only to some Abaqus/Standard elements.

drillingHourglass

A Float specifying the drilling hourglass scaling factor. A value of zero indicates the default value should be used. The default value will be used where appropriate. The default value is 0.0.

This argument is applicable only to some Abaqus/Standard elements.

kinematicSplit

A SymbolicConstant specifying the kinematic split control. Possible values are AVERAGE_STRAIN, ORTHOGONAL, and CENTROID. The default value is AVERAGE_STRAIN.

This argument is applicable only to some Abaqus/Explicit elements.

distortionControl

A Boolean specifying whether to prevent negative element volumes or other excessive distortions in crushable materials. The default value is OFF.

This argument is applicable only to some Abaqus/Explicit elements.

lengthRatio

A Float specifying the length ratio for distortion control in crushable materials. Possible values are $0.0 \leq \text{lengthRatio} \leq 1.0$. The default value is *lengthRatio*=0.1

This argument is applicable only when *distortionControl* is ON.

secondOrderAccuracy

A Boolean specifying the second-order accuracy option. The default value is OFF.

This argument is applicable only to some Abaqus/Explicit elements.

hourglassControl

A SymbolicConstant specifying the hourglass control. Possible values are RELAX_STIFFNESS, STIFFNESS, VISCOUS, ENHANCED, and COMBINED. The default value is ENHANCED.

This argument is applicable only to some Abaqus/Explicit elements.

weightFactor

A Float specifying a weight factor when *hourglassControl*=COMBINED. The default value is 0.5.

This argument is applicable only to some Abaqus/Explicit elements.

MESH COMMANDS

displacementHourglass

A Float specifying the displacement hourglass scaling factor. The default value will be used where appropriate. The default value is 1.0.

This argument is applicable only to some Abaqus/Explicit elements.

rotationalHourglass

A Float specifying the rotational hourglass scaling factor. The default value will be used where appropriate. The default value is 1.0.

This argument is applicable only to some Abaqus/Explicit elements.

outOfPlaneDisplacementHourglass

A Float specifying the out-of-plane displacement hourglass scaling factor. The default value will be used where appropriate. The default value is 1.0.

This argument is applicable only to some Abaqus/Explicit elements.

elemDeletion

A SymbolicConstant specifying the element deletion option. Possible values are DEFAULT, ON, and OFF. The default value is DEFAULT.

particleConversion

A SymbolicConstant specifying the particle conversion option for smoothed particle hydrodynamics. When not OFF or DEFAULT this argument refers to the criterion used for conversion of elements to particles. Possible values are DEFAULT, OFF, TIME, STRAIN, and STRESS. The default value is DEFAULT.

This argument is applicable only to some Abaqus/Explicit elements.

particleConversionThreshold

A Float specifying the threshold value for the particle conversion criterion specified by *particleConversion*. The default value is 0.0.

This argument is applicable only to some Abaqus/Explicit elements.

particleConversionPPD

An Int specifying the number of particles per direction for element conversion when *particleConversion* is specified. The default value is 1.

This argument is applicable only to some Abaqus/Explicit elements.

particleConversionKernel

A SymbolicConstant specifying the interpolation function for particle conversion when *particleConversion* is specified. Possible values are CUBIC, QUADRATIC, and QUINTIC. The default value is CUBIC.

This argument is applicable only to some Abaqus/Explicit elements.

maxDegradation

A Float specifying the maximum degradation option for damage control. The default value is -1.0.

viscosity

A Float specifying the viscosity option. The default value is 0.0.

This argument is applicable only to some Abaqus/Standard elements.

linearBulkViscosity

A Float specifying the linear bulk viscosity scaling factor option for Abaqus/Explicit. The default value is 1.0.

quadraticBulkViscosity

A Float specifying the quadratic bulk viscosity scaling factor option for Abaqus/Explicit. The default value is 1.0.

Return value

An ElemType object.

Exceptions

None.

31.3.2 Members

The ElemType object has members with the same names and descriptions as the arguments to the **ElemType** method.

31.4 MeshEdge object

The MeshEdge object refers to an element edge. It has no constructor or members. A MeshEdge object can be accessed via a MeshEdgeArray or a repository on a part or part instance.

Access

```
import part
mdb.models[name].parts[name].elemEdges[i]
mdb.models[name].parts[name].elementEdges[i]
import assembly
mdb.models[name].rootAssembly.allinstances.elemEdges[i]
mdb.models[name].rootAssembly.allinstances.elementEdges[i]
mdb.models[name].rootAssembly.instances[name].elemEdges[i]
mdb.models[name].rootAssembly.instances[name].elementEdges[i]
```

31.4.1 getElements()

This method returns a tuple of elements that share the element edge.

Arguments

None.

Return value

A tuple of MeshElement objects.

Exceptions

None.

31.4.2 getElementsViaTopology()

This method returns an array of MeshElement objects that are obtained by recursively finding adjacent elements via topology.

Arguments

Optional arguments

domain

A MeshElementArray object specifying the domain to include in the search. By default, all elements in the mesh are included.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.4.3 getNodesViaTopology()

This method returns an array of MeshNode objects that lie along element edges topologically in line with the element edge.

Arguments

Optional arguments

domain

A MeshElementArray object specifying the domain to include in the search. By default, all elements in the mesh are included.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.4.4 getElemFaces()

This method returns a tuple of unique MeshFace objects that share the element edge.

Arguments

None.

Return value

A tuple of MeshFace objects.

Exceptions

None.

31.4.5 getNodes()

This method returns a tuple of nodes on the element edge.

Arguments

None.

Return value

A tuple of MeshNode objects.

Exceptions

None.

31.4.6 Members

The MeshEdge object has no members.

31.5 MeshElement object

The MeshElement object refers to an element of a native mesh or an orphan mesh. A MeshElement object can be accessed via a part or part instance using an index that refers to the internal numbering of the **element** repository. The index does not refer to the element label.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].elements[i]
mdb.models[name].parts[name].allInternalSurfaces[name].elements[i]
mdb.models[name].parts[name].allSets[name].elements[i]
mdb.models[name].parts[name].allSurfaces[name].elements[i]
mdb.models[name].parts[name].elements[i]
mdb.models[name].parts[name].sets[name].elements[i]
mdb.models[name].parts[name].surfaces[name].elements[i]
import assembly
mdb.models[name].rootAssembly.allinstances.elements[i]
mdb.models[name].rootAssembly.allinstances.sets[name].elements[i]
mdb.models[name].rootAssembly.allinstances.surfaces[name].elements[i]
mdb.models[name].rootAssembly.allInternalSets[name].elements[i]
mdb.models[name].rootAssembly.allInternalSurfaces[name].elements[i]
mdb.models[name].rootAssembly.allSets[name].elements[i]
mdb.models[name].rootAssembly.allSurfaces[name].elements[i]
mdb.models[name].rootAssembly.elements[i]
mdb.models[name].rootAssembly.instances[name].elements[i]
mdb.models[name].rootAssembly.instances[name].sets[name].elements[i]
mdb.models[name].rootAssembly.instances[name].surfaces[name] \
.elements[i]
mdb.models[name].rootAssembly.modelInstances[i].elements[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].elements[i]
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name] \
.elements[i]
mdb.models[name].rootAssembly.sets[name].elements[i]
mdb.models[name].rootAssembly.surfaces[name].elements[i]
```

31.5.1 Element(...)

This method creates an element on an orphan mesh part from a sequence of nodes.

Path

`mdb.models[name].parts[name].Element`

Required arguments

nodes

A sequence of MeshNode objects.

elemShape

A SymbolicConstant specifying the shape of the new element. Possible values are LINE2, LINE3, TRI3, TRI6, QUAD4, QUAD8, TET4, TET10, WEDGE6, WEDGE15, HEX8, and HEX20.

Optional argument

label

An Int specifying the element label.

Return value

A MeshElement object.

Exceptions

None.

31.5.2 getNodes()

This method returns a tuple of node objects of the element.

Arguments

None.

Return value

A tuple of MeshNode objects.

Exceptions

None.

31.5.3 getElemEdges()

This method returns a tuple of unique element edge objects on the element.

Arguments

None.

Return value

A tuple of MeshEdge objects.

Exceptions

None.

31.5.4 getElemFaces()

This method returns a tuple of unique element face objects on the element.

Arguments

None.

Return value

A tuple of MeshFace objects.

Exceptions

None.

31.5.5 getAdjacentElements()

This method returns an array of element objects adjacent to the mesh element.

Arguments

None.

Return value

A MeshElementArray object which is a sequence of MeshElement objects.

Exceptions

None.

31.5.6 **getElementsByFeatureEdge(...)**

This method returns an array of mesh element objects that are obtained by recursively finding adjacent elements along a feature edge with a face angle of less than or equal to the specified angle.

Arguments

Required argument

angle

A float specifying the value of the face angle in degrees.

Optional arguments

None.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.5.7 **setValues(...)**

This method modifies the MeshElement object.

Arguments

Required arguments

None.

Optional argument

label

An Int specifying the element label. This member may only be edited if the element belongs to an orphan mesh part. The specified label must be non-negative and must not be in use by any other element of the same part.

Return value

None

Exceptions

None.

31.5.8 Members

The MeshElement object has the following members:

label

An Int specifying the element label.

type

A SymbolicConstant specifying the Abaqus element code.

instanceName

A String specifying the name of the part instance that owns this element.

connectivity

A tuple of Ints specifying the internal node indices that define the nodal connectivity. It is important to note the difference with OdbMeshElement object of ODB where the connectivity is node labels instead of node indices.

31.6 MeshElementArray object

The MeshElementArray is a sequence of MeshElement objects.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].elements
mdb.models[name].parts[name].allInternalSurfaces[name].elements
mdb.models[name].parts[name].allSets[name].elements
mdb.models[name].parts[name].allSurfaces[name].elements
mdb.models[name].parts[name].elements
mdb.models[name].parts[name].sets[name].elements
mdb.models[name].parts[name].surfaces[name].elements
import assembly
mdb.models[name].rootAssembly.allinstances.elements
mdb.models[name].rootAssembly.allinstances.sets[name].elements
mdb.models[name].rootAssembly.allinstances.surfaces[name].elements
mdb.models[name].rootAssembly.allInternalSets[name].elements
mdb.models[name].rootAssembly.allInternalSurfaces[name].elements
mdb.models[name].rootAssembly.allSets[name].elements
mdb.models[name].rootAssembly.allSurfaces[name].elements
mdb.models[name].rootAssembly.elements
mdb.models[name].rootAssembly.instances[name].elements
mdb.models[name].rootAssembly.instances[name].sets[name].elements
```



```

mdb.models[name].rootAssembly.instances[name].surfaces[name].elements
mdb.models[name].rootAssembly.modelInstances[i].elements
mdb.models[name].rootAssembly.modelInstances[i].sets[name].elements
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name] \
.elements
mdb.models[name].rootAssembly.sets[name].elements
mdb.models[name].rootAssembly.surfaces[name].elements

```

31.6.1 MeshElementArray(...)

This method creates a MeshElementArray object.

Path

```
mesh.MeshElementArray
```

Required argument

elements

A list of MeshElement objects.

Optional arguments

None.

Return value

A MeshElementArray object.

Exceptions

None.

31.6.2 getFromLabel(...)

This method returns the object in the MeshElementArray with the given label.

Arguments

Required argument

label

An Int specifying the label of the object.

Optional arguments

None.

Return value

A MeshElement object.

Exceptions

None.

31.6.3 **getSequenceFromMask(...)**

This method returns the objects in the MeshElementArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When a large number of objects are involved, this method is highly efficient.

Arguments**Required argument**

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A MeshElementArray object.

Exceptions

None.

31.6.4 **getMask()**

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

31.6.5 **getByBoundingBox(...)**

This method returns an array of element objects that lie within the specified bounding box.

Arguments

Required arguments

None.

Optional arguments

xMin

A float specifying the minimum X boundary of the bounding box.

yMin

A float specifying the minimum Y boundary of the bounding box.

zMin

A float specifying the minimum Z boundary of the bounding box.

xMax

A float specifying the maximum X boundary of the bounding box.

yMax

A float specifying the maximum Y boundary of the bounding box.

zMax

A float specifying the maximum Z boundary of the bounding box.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.6.6 **getByBoundingCylinder(...)**

This method returns an array of element objects that lie within the specified bounding cylinder.

Arguments

Required arguments

center1

A tuple of the X-, Y-, and Z-coordinates of the center of the first end of the cylinder.

center2

A tuple of the X-, Y-, and Z-coordinates of the center of the second end of the cylinder.

MESH COMMANDS

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.6.7 getByBoundingSphere(...)

This method returns an array of element objects that lie within the specified bounding sphere.

Arguments

Required arguments

center

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.6.8 getBoundingBox()

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the element sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum x, y, and z boundary values of the bounding box.

high: a tuple of three floats representing the maximum x, y, and z boundary values of the bounding box.

Exceptions

None.

31.6.9 sequenceFromLabels(...)

This method returns the objects in the MeshElementArray identified using the specified labels.

Arguments**Required argument**

labels

A sequence of Ints specifying the labels.

Optional arguments

None.

Return value

A MeshElementArray object.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

31.6.10 Members

The MeshElementArray object has no members.

31.7 MesherOptions object

The MesherOptions object controls the default settings that Abaqus uses for all meshing methods. The MesherOptions object has no constructor. Abaqus creates the *MesherOptions* member when a session is started.

MesherOptions commands are intended for use at the beginning of scripts and in the `abaqus_env` file only; they should not be used during an Abaqus/CAE session.

Access

`session.defaultMesherOptions`

31.7.1 `setValues(...)`

This method modifies the MesherOptions object.

Arguments

Required arguments

None.

Optional arguments

elemShape2D

A SymbolicConstant specifying the default element shape for meshing two-dimensional objects. Possible values are QUAD, QUAD_DOMINATED, and TRI. The default value is QUAD_DOMINATED.

elemShape3D

A SymbolicConstant specifying the default element shape for meshing three-dimensional objects. Possible values are HEX, HEX_DOMINATED, WEDGE, and TET. The default value is HEX.

quadAlgorithm

A SymbolicConstant specifying the default algorithm for meshing an object with quad- or quad-dominated elements. Possible values are ADVANCING_FRONT and MEDIAL_AXIS. The default value is ADVANCING_FRONT.

allowMapped

A Boolean specifying whether Abaqus/CAE should allow mapped meshing, where appropriate. The default value is OFF.

minTransition

A Boolean specifying whether Abaqus/CAE should attempt to minimize the mesh transition when it moves from a coarse mesh to a fine mesh. The default value is ON.

Return value

None

Exceptions

None.

31.7.2 Members

The `MeshOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

31.8 MeshFace object

The `MeshFace` object refers to an element face. It has no constructor or members. A `MeshFace` object can be accessed via a `MeshFaceArray` or a repository on a part or part instance.

Access

```
import part
mdb.models[name].parts[name].elementFaces[i]
mdb.models[name].parts[name].elemFaces[i]
import assembly
mdb.models[name].rootAssembly.allinstances.elementFaces[i]
mdb.models[name].rootAssembly.allinstances.elemFaces[i]
mdb.models[name].rootAssembly.instances[name].elementFaces[i]
mdb.models[name].rootAssembly.instances[name].elemFaces[i]
```

31.8.1 getElemEdges()

This method returns a tuple of unique element edges on the element face.

Arguments

None.

Return value

A tuple of `MeshEdge` objects.

Exceptions

None.

31.8.2 getElements()

This method returns a tuple of elements that share the element face.

Arguments

None.

Return value

A tuple of MeshElement objects.

Exceptions

None.

31.8.3 getNodes()

This method returns a tuple of nodes on the element face.

Arguments

None.

Return value

A tuple of MeshNode objects.

Exceptions

None.

31.8.4 getNodesByFaceAngle(...)

This method returns an array of mesh node objects that are obtained by recursively finding adjacent element faces that are at an angle of less than or equal to the specified angle.

Arguments

Required argument

angle

A float specifying the value of the face angle.

Optional arguments

None.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.8.5 getNormal(...)

This method returns the normal direction for the element face.

Arguments**Required arguments**

None.

Optional arguments

None.

Return value

A tuple of 3 floats representing the unit normal vector. If the element face is collapsed such that a normal cannot be computed, a zero-length vector is returned.

Exceptions

None.

31.8.6 getElemFacesByFaceAngle(...)

This method returns an array of element face objects that are obtained by recursively finding adjacent element faces that are at an angle of less than or equal to the specified angle.

Arguments**Required argument**

angle

A float specifying the value of the face angle.

Optional arguments

None.

Return value

A MeshFaceArray object, which is a sequence of MeshFace objects.

Exceptions

None.

31.8.7 **getElemEdgesByFaceAngle(...)**

This method returns an array of element edge objects that are obtained by recursively finding adjacent element edges that are at an angle of less than or equal to the specified face angle.

Arguments

Required argument

angle

A float specifying the value of the face angle in degrees.

Optional arguments

None.

Return value

A MeshEdgeArray object, which is a sequence of MeshEdge objects.

Exceptions

None.

31.8.8 **getElementsByFaceAngle(...)**

This method returns an array of mesh Element objects that are obtained by recursively finding adjacent element faces that are at an angle of less than or equal to the specified angle.

Arguments

Required argument

angle

A float specifying the value of the face angle.

Optional arguments

None.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.8.9 **getElemFacesByLimitingAngle(...)**

This method returns an array of element edge objects that are obtained by recursively finding adjacent element faces that are at an angle of less than or equal to the specified face angle with the seed face.

Arguments

Required argument

angle

A float specifying the value of the face angle in degrees.

Optional arguments

None.

Return value

A MeshFaceArray object, which is a sequence of MeshFace objects.

Exceptions

None.

31.8.10 **getElementsViaTopology()**

This method returns an array of mesh Element objects that are obtained by recursively finding adjacent elements via topology.

Arguments

None.

Return value

A MeshElementArray object, which is a sequence of MeshElement objects.

Exceptions

None.

31.8.11 **getElemFacesByLayer()**

This method returns an array of element face objects, obtained by traversing shell elements or the exterior of a solid mesh, and recursively finding adjacent element faces by layer.

Arguments

Required argument

numLayers

A int specifying the value of the number of layers.

Optional arguments

None.

Return value

A MeshFaceArray object, which is a sequence of MeshFace objects.

Exceptions

None.

31.8.12 Members

The MeshFace object has the following members:

label

An Int specifying an Int specifying the element label.

face

An Int specifying a symbolic constant specifying the side of the element.

31.9 MeshNode object

The MeshNode object refers to a node of a native mesh or an orphan mesh. A MeshNode object can be accessed via a part or part instance using an index that refers to the internal numbering of the **node** repository. The index does not refer to the node label.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].nodes[i]
mdb.models[name].parts[name].allInternalSurfaces[name].nodes[i]
mdb.models[name].parts[name].allSets[name].nodes[i]
mdb.models[name].parts[name].allSurfaces[name].nodes[i]
mdb.models[name].parts[name].nodes[i]
mdb.models[name].parts[name].retainedNodes[i]
mdb.models[name].parts[name].sets[name].nodes[i]
mdb.models[name].parts[name].surfaces[name].nodes[i]
```

```

import assembly
mdb.models[name].rootAssembly.allinstances.nodes[i]
mdb.models[name].rootAssembly.allinstances.sets[name].nodes[i]
mdb.models[name].rootAssembly.allinstances-surfaces[name].nodes[i]
mdb.models[name].rootAssembly.allInternalSets[name].nodes[i]
mdb.models[name].rootAssembly.allInternalSurfaces[name].nodes[i]
mdb.models[name].rootAssembly.allSets[name].nodes[i]
mdb.models[name].rootAssembly.allSurfaces[name].nodes[i]
mdb.models[name].rootAssembly.instances[name].sets[name].nodes[i]
mdb.models[name].rootAssembly.instances[name].surfaces[name].nodes[i]
mdb.models[name].rootAssembly.modelInstances[i].nodes[i]
mdb.models[name].rootAssembly.modelInstances[i].sets[name].nodes[i]
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name] \
.nodes[i]
mdb.models[name].rootAssembly.nodes[i]
mdb.models[name].rootAssembly.sets[name].nodes[i]
mdb.models[name].rootAssembly.surfaces[name].nodes[i]

```

31.9.1 Node(...)

This method creates a node on an orphan mesh part.

Path

```
mdb.models[name].parts[name].Node
```

Required argument

coordinates

A sequence of three Floats specifying the coordinates of the new node.

Optional arguments

localCsys

A DatumCsys object specifying the local coordinate system. If unspecified, the global coordinate system will be used.

label

An Int specifying the node label.

Return value

A MeshNode object.

Exceptions

None.

31.9.2 getElemEdges()

This method returns a tuple of element edge objects that share the node.

Arguments

None.

Return value

A tuple of MeshEdge objects.

Exceptions

None.

31.9.3 getElemFaces()

This method returns a tuple of element face objects that share the node.

Arguments

None.

Return value

A tuple of MeshFace objects.

Exceptions

None.

31.9.4 getElements()

This method returns a tuple of element objects that share the node.

Arguments

None.

Return value

A tuple of MeshElement objects.

Exceptions

None.

31.9.5 getNodesByFeatureEdge(...)

This method returns an array of mesh node objects that are obtained by recursively finding adjacent nodes along a feature edge that are at an angle of less than or equal to the specified face angle.

Arguments**Required argument**

angle

A float specifying the value of the face angle in degrees.

Optional arguments

None.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.9.6 setValues(...)

This method modifies the MeshNode object.

Arguments**Required arguments**

None.

Optional argument

label

An Int specifying the node label. This member may only be edited if the node belongs to an orphan mesh part. The specified label must be non-negative and must not be in use by any other node of the same part.

Return value

None

Exceptions

None.

31.9.7 Members

The MeshNode object has the following members:

label

An Int specifying the node label.

instanceName

A String specifying the name of the part instance that owns this node.

coordinates

A tuple of three Floats specifying the coordinates of the new node.

31.10 MeshNodeArray object

The MeshNodeArray is a sequence of MeshNode objects.

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name].nodes
mdb.models[name].parts[name].allInternalSurfaces[name].nodes
mdb.models[name].parts[name].allSets[name].nodes
mdb.models[name].parts[name].allSurfaces[name].nodes
mdb.models[name].parts[name].nodes
mdb.models[name].parts[name].retainedNodes
mdb.models[name].parts[name].sets[name].nodes
mdb.models[name].parts[name].surfaces[name].nodes
import assembly
mdb.models[name].rootAssembly.allinstances.nodes
mdb.models[name].rootAssembly.allinstances.sets[name].nodes
mdb.models[name].rootAssembly.allinstances.surfaces[name].nodes
mdb.models[name].rootAssembly.allInternalSets[name].nodes
mdb.models[name].rootAssembly.allInternalSurfaces[name].nodes
mdb.models[name].rootAssembly.allSets[name].nodes
mdb.models[name].rootAssembly.allSurfaces[name].nodes
```



```

mdb.models[name].rootAssembly.instances[name].nodes
mdb.models[name].rootAssembly.instances[name].sets[name].nodes
mdb.models[name].rootAssembly.instances[name].surfaces[name].nodes
mdb.models[name].rootAssembly.modelInstances[i].nodes
mdb.models[name].rootAssembly.modelInstances[i].sets[name].nodes
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name].nodes
mdb.models[name].rootAssembly.nodes
mdb.models[name].rootAssembly.sets[name].nodes
mdb.models[name].rootAssembly.surfaces[name].nodes

```

31.10.1 MeshNodeArray(...)

This method creates a MeshNodeArray object.

Path

```
mesh.MeshNodeArray
```

Required argument

nodes

A list of MeshNode objects.

Optional arguments

None.

Return value

A MeshNodeArray object.

Exceptions

None.

31.10.2 getFromLabel(...)

This method returns the object in the MeshNodeArray with the given label.

Arguments

Required argument

label

An Int specifying the label of the object.

Optional arguments

None.

Return value

A MeshNode object.

Exceptions

None.

31.10.3 **getSequenceFromMask(...)**

This method returns the objects in the MeshNodeArray identified using the specified *mask*. This command is generated when the JournalOptions are set to COMPRESSEDINDEX. When a large number of objects are involved, this method is highly efficient.

Arguments**Required argument**

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A MeshNodeArray object.

Exceptions

None.

31.10.4 **getMask()**

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

31.10.5 **getByBoundingBox(...)**

This method returns an array of nodes that lie within the specified bounding box.

Arguments

Required arguments

None.

Optional arguments

xMin

A float specifying the minimum X boundary of the bounding box.

yMin

A float specifying the minimum Y boundary of the bounding box.

zMin

A float specifying the minimum Z boundary of the bounding box.

xMax

A float specifying the maximum X boundary of the bounding box.

yMax

A float specifying the maximum Y boundary of the bounding box.

zMax

A float specifying the maximum Z boundary of the bounding box.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.10.6 **getByBoundingCylinder(...)**

This method returns an array of node objects that lie within the specified bounding cylinder.

Arguments

Required arguments

center1

A tuple of the X-, Y-, and Z-coordinates of the center of the first end of the cylinder.

center2

A tuple of the X-, Y-, and Z-coordinates of the center of the second end of the cylinder.

MESH COMMANDS

radius

A float specifying the radius of the cylinder.

Optional arguments

None.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.10.7 getByBoundingSphere(...)

This method returns an array of node objects that lie within the specified bounding sphere.

Arguments

Required arguments

center

A tuple of the X-, Y-, and Z-coordinates of the center of the sphere.

radius

A float specifying the radius of the sphere.

Optional arguments

None.

Return value

A MeshNodeArray object, which is a sequence of MeshNode objects.

Exceptions

None.

31.10.8 getBoundingBox()

This method returns a dictionary of two tuples representing minimum and maximum boundary values of the bounding box of the minimum size containing the node sequence.

Arguments

None.

Return value

A Dictionary object with the following items:

low: a tuple of three floats representing the minimum x, y and z boundary values of the bounding box.

high: a tuple of three floats representing the maximum x, y and z boundary values of the bounding box.

31.10.9 getClosest()

This method returns the node or nodes closest to the given point or set of points.

Arguments**Required arguments**

coordinates

A point defined by x, y, and z values or a list of such points.

Optional arguments

numToFind

The number of nodes to find for each given point. For example, if *numToFind* is 2, then the 2 closest points, if available and within *searchTolerance*, will be returned in order of proximity for each input point. The default is 1.

searchTolerance

A float specifying a search radius for each point. By default, no search radius is defined, and all nodes in the sequence will be searched.

Return value

A MeshNode, or a list of MeshNode objects, or a list of lists of MeshNode objects, depending on the number of points given and the number of nodes requested.

Exceptions

None.

31.10.10 sequenceFromLabels(...)

This method returns the objects in the MeshNodeArray identified using the specified labels.

Arguments

Required argument

labels

A sequence of Ints specifying the labels.

Optional arguments

None.

Return value

A MeshNodeArray object.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

31.10.11 Members

The MeshNodeArray object has no members.

31.11 MeshFaceArray object

The MeshFaceArray is a sequence of MeshFace objects.

Access

```
import part
mdb.models[name].parts[name].elementFaces
import assembly
mdb.models[name].rootAssembly.allinstances.elementFaces
mdb.models[name].rootAssembly.instances[name].elementFaces
```

31.11.1 MeshFaceArray(...)

This method creates a MeshFaceArray object.

Path

```
mesh.MeshFaceArray
```

Required argument*faces*

A list of MeshFace objects.

Optional arguments

None.

Return value

A MeshFaceArray object.

Exceptions

None.

31.11.2 getSequenceFromMask(...)

This method returns the objects in the MeshFaceArray identified using the specified *mask*. When large number of objects are involved, this method is highly efficient.

Arguments**Required argument***mask*

A String specifying the object or objects.

Optional arguments

None.

Return value

A MeshFaceArray object.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

31.11.3 getMask()

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

31.11.4 Members

The MeshFaceArray object has no members.

31.12 MeshEdgeArray object

The MeshEdgeArray is a sequence of MeshEdge objects.

Access

```
import part
mdb.models[name].parts[name].elementEdges
import assembly
mdb.models[name].rootAssembly.allinstances.elementEdges
mdb.models[name].rootAssembly.instances[name].elementEdges
```

31.12.1 MeshEdgeArray(...)

This method creates a MeshEdgeArray object.

Path

`mesh.MeshEdgeArray`

Required argument

edges

A list of MeshEdge objects.

Optional arguments

None.

Return value

A MeshEdgeArray object.

Exceptions

None.

31.12.2 getSequenceFromMask(...)

This method returns the objects in the MeshEdgeArray identified using the specified *mask*. When large number of objects are involved, this method is highly efficient.

Arguments**Required argument**

mask

A String specifying the object or objects.

Optional arguments

None.

Return value

A MeshEdgeArray object.

Exceptions

An exception occurs if the resulting sequence is empty.

Error: The mask results in an empty sequence

31.12.3 getMask()

This method returns a string specifying the object or objects.

Arguments

None.

Return value

A String specifying the object or objects.

Exceptions

None.

31.12.4 Members

The MeshEdgeArray object has no members.

31.13 MeshStats object

The MeshStats object is a query object for holding mesh statistics and is returned by the `getMeshStats` command. The object does not have any methods.

Access

```
import mesh
```

31.13.1 Members

The MeshStats object has the following members:

numPointElems

An Int specifying the number of point elements.

numLineElems

An Int specifying the number of line elements.

numQuadElems

An Int specifying the number of quadrilateral elements.

numTriElems

An Int specifying the number of triangular elements.

numHexElems

An Int specifying the number of hexahedral elements.

numWedgeElems

An Int specifying the number of wedge elements.

numTetElems

An Int specifying the number of tetrahedral elements.

numPyramidElems

An Int specifying the number of pyramid elements.

numNodes

An Int specifying the number of nodes.

numMeshedRegions

An Int specifying the number of regions that contain a mesh.

32. Messaging commands

Messaging commands are available only if Abaqus/CAE is run interactively using the GUI.

32.1 DataObject object

An instance of the DataObject object is passed to each callback. The DataObject object has no methods. The members of a DataObject object depend on the type of the object. All DataObject instances have the following members, regardless of type:

- *clientHost*
- *clientName*
- *phase*
- *processId*
- *threadId*
- *timeStamp*

The possible DataObject types and the additional members for each type are as follows:

ABORTED

- *message*

COMPLETED

- *message*

END_STEP

- *stepId*

ERROR

- *message*

MESSAGING COMMANDS

HEADING

- *heading*

MONITOR_DATA

- *dof*
- *node*
- *nset*
- *procedure*
- *time*
- *value*

ODB_FILE

- *file*

STARTED

- No additional members.

STATUS

- *attempts*
- *equilibrium*
- *increment*
- *iterations*
- *severe*
- *step*
- *stepTime*
- *timeIncrement*
- *totalTime*

STEP

- *stepId*
- *stepName*

WARNING

- *message*

32.1.1 Members

The DataObject object has the following members:

phase

A SymbolicConstant specifying the phase of the analysis. Possible values are BATCHPRE_PHASE, PACKAGER_PHASE, STANDARD_PHASE, EXPLICIT_PHASE, CALCULATOR_PHASE, and UNKNOWN_PHASE.

processId

An Int specifying the process ID of the analysis product.

threadId

An Int specifying the thread ID of the analysis product. Threads are used for parallel or multiprocessing; in most cases *threadId* is set to zero.

timeStamp

An Int specifying the time the message was sent in seconds since 00:00:00 UTC, January 1, 1970.

attempts

An Int specifying the number of attempts made to reach equilibrium during this step.

dof

An Int specifying the degree of freedom requested for monitoring the output.

equilibrium

An Int specifying the number of equilibrium iterations made during this increment.

increment

An Int specifying the increment of the analysis.

iterations

An Int specifying the number of iterations in the step.

MESSAGING COMMANDS

node

An Int specifying the node number requested for monitoring output.

severe

An Int specifying the number of severe discontinuity iterations completed during this increment.

step

An Int specifying the current step number. Step number 1 corresponds to the first step.

stepId

An Int specifying the ID of the step.

stepTime

A Float specifying the step time corresponding to the current increment.

time

A Float specifying the total time corresponding to the monitor data.

timeIncrement

A Float specifying the time increment used in the current step.

totalTime

A Float specifying the total time completed in the analysis.

value

A Float specifying the current value of the degree of freedom requested for monitoring.

clientHost

A String specifying the host name of the machine that is running the analysis.

clientName

A String specifying the name of the client that responded to the callback function. Possible values are “BatchPre”, “Packager”, “Standard”, “Explicit”, and “Calculator”.

file

A String specifying the full path of the output database.

heading

A String specifying the job heading.

message

A String specifying the job heading.

nset

A String specifying the node set specified for monitoring output.

stepName

A String specifying the name of the step.

32.2 MonitorMgr object

An instance of the MonitorMgr object is created when you import the abaqus module. No other instance of the MonitorMgr object is required. (This MonitorMgr object is not to be confused with the degree of freedom (DOF) monitor that is constructed from the Step object.)

Access

`monitorManager`

32.2.1 addMessageCallback(...)

This method specifies a callback function that will be called when the specified message is received from the analysis product.

For more information, see “An example of a callback function,” Section 6.8.3 of the Abaqus Scripting User’s Guide.

Arguments

Required arguments

jobName

A String specifying the name of the job to be monitored or the SymbolicConstant ANY_JOB.

messageType

A SymbolicConstant specifying which message type will call this callback. Possible values are:

- ABORTED
- ANY_JOB
- ANY_MESSAGE_TYPE
- COMPLETED
- END_STEP
- ERROR
- HEADING
- HEALER_JOB
- HEALER_TYPE
- INTERRUPTED
- ITERATION
- JOB_ABORTED
- JOB_COMPLETED
- JOB_INTERRUPTED

MESSAGING COMMANDS

- JOB_SUBMITTED
- MONITOR_DATA
- ODB_FILE
- ODB_FRAME
- SIMULATION_ABORTED
- SIMULATION_COMPLETED
- SIMULATION_INTERRUPTED
- SIMULATION_SUBMITTED
- STARTED
- STATUS
- STEP
- WARNING

callback

A Python function to be called. The interface definition of the callback function is :

```
def onMessage(jobName, messageType, data, userData)
```

- *jobName* is a String.
- *messageType* is a SymbolicConstant with possible values as listed previously for the **addMessageCallback** method.
- *data* is a DataObject object.
- *userData* is the object passed as the *userData* argument to the **addMessageCallback** method.

Optional argument

userData

Any Python object or **None**. This object is passed to the callback function.

Return value

None

Exceptions

None.

32.2.2 **removeMessageCallback(...)**

This method removes a callback function. You specify the callback function to remove using the same arguments you used to add the callback.

Arguments

Required arguments

jobName

A String specifying the name of the job to be monitored or the SymbolicConstant ANY_JOB.

messageType

A SymbolicConstant specifying which message type will call this callback. Possible values are:

- ABORTED
- ANY_JOB
- ANY_MESSAGE_TYPE
- COMPLETED
- END_STEP
- ERROR
- HEADING
- HEALER_JOB
- HEALER_TYPE
- INTERRUPTED
- ITERATION
- JOB_ABORTED
- JOB_COMPLETED
- JOB_INTERRUPTED
- JOB_SUBMITTED
- MONITOR_DATA
- ODB_FILE
- ODB_FRAME
- SIMULATION_ABORTED
- SIMULATION_COMPLETED
- SIMULATION_INTERRUPTED
- SIMULATION_SUBMITTED
- STARTED
- STATUS
- STEP

MESSAGING COMMANDS

- WARNING

callback

A Python function to be called; it must be the same as the *callback* argument specified in the original call to **addMessageCallback**.

userData

Any Python object or **None**; it must be the same as the *userData* argument specified in the original call to **addMessageCallback**.

Optional arguments

None.

Return value

None

Exceptions

None.

32.2.3 checkMonitorStatus()

This method raises a **MonitorError** exception if the monitoring status is not **ENABLED**.

Arguments

None.

Return value

None

Exceptions

MonitorError:
Status is not **ENABLED**

32.2.4 Members

The **MonitorMgr** object has no members.

33. Model commands

Model commands are used to create Abaqus/CAE models. A finished model contains all the data that Abaqus/CAE needs to create and submit an analysis to Abaqus/Standard or Abaqus/Explicit. Models are stored in a model database.

33.1 Model object

Abaqus creates a Model object named **Model-1** when a session is started.

Access

`mdb.models [name]`

33.1.1 Model(...)

This method creates a Model object.

Path

`mdb.Model`

Required argument

name

A String specifying the repository key.

Optional arguments

description

A String specifying the purpose and contents of the Model object. The default value is an empty string.

stefanBoltzmann

None or a Float specifying the Stefan-Boltzmann constant. The default value is **None**.

absoluteZero

None or a Float specifying the absolute zero constant. The default value is **None**.

waveFormulation

A SymbolicConstant specifying the type of incident wave formulation to be used in acoustic problems. Possible values are NOT_SET, SCATTERED, and TOTAL. The default value is NOT_SET.

MODEL COMMANDS

modelType

A SymbolicConstant specifying the analysis model type. Possible values are STANDARD_EXPLICIT CFD and ELECTROMAGNETIC. The default is STANDARD_EXPLICIT.

universalGas

None or a Float specifying the universal gas constant. The default value is **None**.

copyConstraints

A boolean specifying whether to copy the constraints created in the model to the model that instances this model. The default value is ON.

copyConnectors

A boolean specifying whether to copy the connectors created in the model to the model that instances this model. The default value is ON.

copyInteractions

A boolean specifying whether to copy the interactions created in the model to the model that instances this model. The default value is ON.

Return value

A Model object.

Exceptions

None.

33.1.2 ModelFromInputFile(...)

This method creates a Model object by reading the keywords in an input file and creating the corresponding Abaqus/CAE objects.

Path

mdb.ModelFromInputFile

Required arguments

name

A String specifying the repository key.

inputFileName

A String specifying the name of the input file (including the **.inp** extension) to be parsed into the new model. This String can also be the full path to the input file if it is located in another directory.

Optional arguments

None.

Return value

A Model object.

Exceptions

None.

33.1.3 ModelFromOdbFile(...)

This method creates a Model object by reading an output database and creating any corresponding Abaqus/CAE objects.

Path

`mdb.ModelFromOdbFile`

Required arguments

name

A String specifying the repository key.

odbFileName

A String specifying the name of the output database file (including the `.odb` extension) to be read into the new model. This String can also be the full path to the output database file if it is located in another directory.

Optional arguments

None.

Return value

A Model object.

Exceptions

None.

33.1.4 ModelFromNastranFile(...)

This method creates a Model object by reading the keywords in a Nastran bulk data file or Nastran input file and creating any corresponding Abaqus/CAE objects. The default values is discussed in following and can be defined alternatively in the Abaqus environment file as the one used for the translator from Nastran to Abaqus. For more information, see “Translating Nastran bulk data files to Abaqus input files,” Section 3.2.30 of the Abaqus Analysis User’s Guide.

Path

mdb.ModelFromNastranFile

Required arguments

modelName

A String specifying the repository key.

inputFileName

A String specifying the name of the Nastran input file (including the **.bdf**, **.dat**, **.nas**, **.nastran**, **.blk**, **.bulk** extension) to be read into the new model. This String can also be the full path to the Nastran input file if it is located in another directory.

Optional arguments

sectionConsolidation

A SymbolicConstant specifying the method used to create shell section. Possible values are PRESERVE_SECTION, GROUP_BY_MATERIAL, and NONE. If PRESERVE_SECTION is used, an Abaqus section is created corresponding to each shell property ID. If GROUP_BY_MATERIAL is used, a single Abaqus section is created for all homogeneous elements referencing the same material. In both cases, material orientations and offsets are created using discrete fields. If NONE is used, a separate shell section is created for each combination of orientation, material offset, and/or thickness. The default is PRESERVE_SECTION.

preIntegratedShell

A Boolean specifying whether the pre-integrated shell section is created in default for shell element. The default value is OFF.

weightMassScaling

A Boolean specifying whether the value on the Nastran data line PARAM, WTMASS is used as a multiplier for all density, mass, and rotary inertia values created in the Abaqus input file. The default value is ON.

loadCases

A Boolean specifying whether each SUBCASE for linear static analyses is translated to a *LOAD CASE option, and all such *LOAD CASE options are grouped in a single *STEP option. The default value is ON.

coupleBeamOffsets

A Boolean specifying whether to translate the beam element connectivity to newly created nodes at the offset location and rigidly coupling the new and original nodes. If not, beam element offsets are translated to the *CENTROID and *SHEAR CENTER options, which are suboptions of the *BEAM GENERAL SECTION option. The default value is ON. When the beam element references a PBARL or PBEAML property or if the beam offset has a significant component in the direction of the beam axis, the setting for this argument is always ON.

cbar

A String specifying the 2-node beam that is created from CBAR and CBEAM elements. Possible values are B31 and B33. The default is B31.

cquad4

A String specifying the 4-node shell that is created from CQUAD4 elements. Possible values are S4 and S4R. The default is S4. If a reduced-integration element is chosen, the enhanced hourglass formulation is applied automatically.

chexa

A String specifying the 8-node brick that is created from CHEXA elements. Possible values are C3D8I, C3D8 and C3D8R. The default is C3D8I. If a reduced-integration element is chosen, the enhanced hourglass formulation is applied automatically.

ctetra

A String specifying the 10-node tetrahedron that is created from CTETRA elements. Possible values are C3D10 and C3D10M. The default is C3D10.

keepTranslatedFiles

A Boolean specifying whether to keep the generated Abaqus input file after the model is created from the Nastran input file. The default value is ON.

Return value

A Model object.

Exceptions

None.

33.1.5 setValues(...)

This method modifies the Model object.

Arguments

Required arguments

None.

Optional arguments

description

A String specifying the purpose and contents of the Model object. The default value is an empty string.

noPartsInputFile

A Boolean specifying whether an input file should be written without parts and assemblies. The default value is OFF.

MODEL COMMANDS

absoluteZero

None or a Float specifying the absolute zero constant. The default value is **None**.

stefanBoltzmann

None or a Float specifying the Stefan-Boltzmann constant. The default value is **None**.

waveFormulation

A SymbolicConstant specifying the type of incident wave formulation to be used in acoustic problems. Possible values are NOT_SET, SCATTERED, and TOTAL. The default value is NOT_SET.

universalGas

None or a Float specifying the universal gas constant. The default value is **None**.

restartJob

A String specifying the name of the job that generated the restart data.

restartStep

A String specifying the name of the step where the restart analysis will start.

restartIncrement

An Int specifying the increment, interval, iteration or cycle where the restart analysis will start. To select the end of the step use the SymbolicConstant STEP_END.

endRestartStep

A Boolean specifying that the step specified by *restartStep* should be terminated at the increment specified by *restartIncrement*.

globalJob

A String specifying the name of the job that generated the results for the global model.

shellToSolid

A Boolean specifying that a shell global model drives a solid submodel.

copyConstraints

A Boolean specifying whether to copy the constraints created in the model to the model that instances this model.

copyConnectors

A Boolean specifying whether to copy the connectors created in the model to the model that instances this model

copyInteractions

A Boolean specifying whether to copy the interactions created in the model to the model that instances this model.

Return value

None

Exceptions

None.

33.1.6 Members

The Model object can have the following members:

name

A String specifying the repository key.

stefanBoltzmann

None or a Float specifying the Stefan-Boltzmann constant. The default value is **None**.

absoluteZero

None or a Float specifying the absolute zero constant. The default value is **None**.

waveFormulation

A SymbolicConstant specifying the type of incident wave formulation to be used in acoustic problems. Possible values are NOT_SET, SCATTERED, and TOTAL. The default value is NOT_SET.

universalGas

None or a Float specifying the universal gas constant. The default value is **None**.

noPartsInputFile

A Boolean specifying whether an input file should be written without parts and assemblies. The default value is OFF.

restartIncrement

An Int specifying the increment, interval, iteration or cycle where the restart analysis will start. To select the end of the step use the SymbolicConstant STEP_END.

endRestartStep

A Boolean specifying that the step specified by *restartStep* should be terminated at the increment specified by *restartIncrement*.

shellToSolid

A Boolean specifying that a shell global model drives a solid submodel.

lastChangedCount

A Float specifying the time stamp that indicates when the model was last changed.

description

A String specifying the purpose and contents of the Model object. The default value is an empty string.

restartJob

A String specifying the name of the job that generated the restart data.

restartStep

A String specifying the name of the step where the restart analysis will start.

MODEL COMMANDS

globalJob

A String specifying the name of the job that generated the results for the global model.

copyConstraints

A boolean specifying the status of constraints created in a model, in the model which instances this model.

copyConnectors

A boolean specifying the status of connectors created in a model, in the model which instances this model.

copyInteractions

A boolean specifying the status of interactions created in a model, in the model which instances this model.

keywordBlock

A KeywordBlock object.

rootAssembly

An Assembly object.

amplitudes

A repository of Amplitude objects.

profiles

A repository of Profile objects.

boundaryConditions

A repository of BoundaryCondition objects.

constraints

A repository of Constraint objects.

analyticalFields

A repository of AnalyticalField objects.

discreteFields

A repository of DiscreteField objects.

predefinedFields

A repository of PredefinedField objects.

interactions

A repository of Interaction objects.

interactionProperties

A repository of InteractionProperty objects.

contactControls

A repository of ContactControl objects.

contactInitializations

A repository of ContactInitialization objects.

contactStabilizations

A repository of ContactStabilization objects.

linkedInstances

A tuple of tuples of Strings specifying the linked child PartInstance name in the current model to the corresponding parent PartInstance name in a different model.

linkedParts

A tuple of tuples of Strings specifying the linked child Part name in the current model to the corresponding parent Part name in a different model.

loads

A repository of Load objects.

materials

A repository of Material objects.

calibrations

A repository of Calibration objects.

sections

A repository of Section objects.

remeshingRules

A repository of RemeshingRule objects.

sketches

A repository of ConstrainedSketch objects.

parts

A repository of Part objects.

steps

A repository of Step objects.

featureOptions

A FeatureOptions object.

adaptiveMeshConstraints

A repository of AdaptiveMeshConstraint objects.

adaptiveMeshControls

A repository of AdaptiveMeshControl objects.

timePoints

A repository of TimePoint objects.

filters

A repository of Filter objects.

integratedOutputSections

A repository of IntegratedOutputSection objects.

fieldOutputRequests

A repository of FieldOutputRequest objects.

MODEL COMMANDS

historyOutputRequests

A repository of HistoryOutputRequest objects.

optimizationTasks

A repository of OptimizationTask objects.

33.1.7 Corresponding analysis keywords

*PHYSICAL CONSTANTS

33.2 KeywordBlock object

The KeywordBlock object contains a representation of its model in the Abaqus input file format. You may edit the contents of the KeywordBlock to add solver functionality that is not supported by Abaqus/CAE. As a general rule, edits to the KeywordBlock object should be made as the last step prior to writing the actual Abaqus input file, thus avoiding possible conflicts with changes made using other MDB commands. The KeywordBlock object has no constructor. A KeywordBlock object is created when you create a model object. A model object contains only one KeywordBlock object.

Access

`mdb.models[name].keywordBlock`

33.2.1 setValues(...)

This method modifies the KeywordBlock object.

Arguments

Required arguments

None.

Optional argument

edited

A Boolean specifying whether this objects *sieBlocks* member has been edited. Setting *edited=False* will set the *sieBlocks* member to an empty tuple, thereby discarding all previous edits.

Return value

None

Exceptions

None.

33.2.2 insert(...)

This method inserts a String at a specified position in the *sieBlocks* member.

Arguments**Required arguments**

position

An Int specifying the position in the *sieBlocks* member after which the new string will be inserted.

text

A String specifying the text to be inserted. The text represents an Abaqus input file keyword and its associated data

Optional arguments

None.

Return value

None

Exceptions

IndexError.

33.2.3 replace(...)

This method replaces a String at a specified position in the *sieBlocks* member.

Arguments**Required arguments**

position

An Int specifying the position of the String to be replaced in the *sieBlocks* member.

text

A String specifying the text to be replaced. The text represents an Abaqus input file keyword and its associated data.

Optional arguments

None.

Return value

None

Exceptions

IndexError.

33.2.4 synchVersions(...)

This method synchronizes, or merges, the edits made in this object with those made in the model using other scripting commands or the user interface. The **synchVersions** method updates the *sieBlocks* member. The *sieBlocks* member is empty prior to the first call to **synchVersions**. As a side effect, **synchVersions** sets *lastSynchCount* to the current value of the counter associated with the Mdb object, which is used to determine if synchronization is necessary.

Arguments**Required argument***storeNodesAndElements*

A Boolean specifying whether the nodal coordinates and element connectivities (i.e. the data lines for the *NODE and *ELEMENT keyword blocks) are to be stored in the *sieBlocks* member. All other keywords and their data lines are always stored. The default value is True. If *storeNodesAndElements* is True, the size of the keywordBlock data will be similar to that of the input file. Since the KeywordBlock is stored in the Abaqus/CAE database, this will result in a larger database. It will also result in a slower execution of the **synchVersions** command. If *storeNodesAndElements* is False, the data lines are not stored in *sieBlocks*. Consequently, only set *storeNodesAndElements*=True if you wish to make changes to the *NODE or *ELEMENT data lines themselves. If your task is limited to reading nodal coordinates and element connectivities (i.e. not editing this information) then it is generally better to access this information from other parts of the Mdb .

Optional arguments

None.

Return value

None

Exceptions

IndexError.

33.2.5 Members

The KeywordBlock object has the following members:

edited

A Boolean specifying whether the Keywords Editor has been used to change the model.

lastSynchCount

A Float specifying the value of the counter associated with the Mdb object at the most recent synchronization.

sieBlocks

A tuple of Strings specifying a sequence of Strings that is identical to the information written to the Abaqus input file. Each String in the sequence represents an Abaqus input file keyword along with the parameters and data lines associated with the keyword. A String can also be a comment in the input file. You initialize this data member by calling **synchVersions**. After you initialize the data member, you use calls to **replace** and **insert** to record your edits in the correct location. If the last call to **synchVersions** used the argument *storeNodesAndElements*=False, the entry for the keywords *NODE and *ELEMENT will contain only the keyword and its parameters, not the data lines.

34. Odb commands

The Python ODB API commands are used to read and write data from an output database (**.odb**) file. The path to the Odb object can be via the **session.odbs** repository or via a variable. In this chapter the **Access** and **Path** statements refer to a variable called **odb** that represents an existing Odb object.

34.1 Odb object

The Odb object is the in-memory representation of an output database (ODB) file.

Access

```
import odbAccess
session.odbs[name]
```

34.1.1 Odb(...)

This method creates a new Odb object.

Path

```
session.Odb
```

Required argument

name

A String specifying the repository key.

Optional arguments

analysisTitle

A String specifying the title of the output database. The default value is an empty string.

description

A String specifying the description of the output database. The default value is an empty string.

path

A String specifying the path to the file where the new output database (**.odb**) file will be written. The default value is an empty string.

Return value

An Odb object.

Exceptions

None.

34.1.2 close()

This method closes an output database.

Arguments

None.

Return value

None

Exceptions

None.

34.1.3 getFrame(...)

This method returns the frame at the specified time, frequency, or mode. It will not interpolate values between frames. The method is not applicable to an Odb object containing steps with different domains or to an Odb object containing a step with load case specific data.

Arguments

Required argument

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the total time or frequency.

Optional argument

match

A SymbolicConstant specifying which frame to return if there is no frame at the exact frame value. Possible values are CLOSEST, BEFORE, AFTER, and EXACT. The default value is CLOSEST.

When *match*=CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the exact frame is not found:

OdbError: Frame not found.

34.1.4 save()

This method saves output to an output database (**.odb**) file.

Arguments

None.

Return value

None

Exceptions

OdbError

Database save failed. The database was opened as read-only. Modification of data is not permitted.

34.1.5 update()

This method is used to update an Odb object in memory while an Abaqus analysis writes data to the associated output database. **update** checks if additional steps have been written to the output database since it was opened or last updated. If additional steps have been written to the output database, **update** adds them to the Odb object.

Arguments

None.

Return value

A Boolean specifying whether additional steps or frames were added to the Odb object.

Exceptions

None.

34.1.6 Members

The Odb object has members with the same names and descriptions as the arguments to the **Odb** method.

ODB COMMANDS

In addition, the Odb object can have the following members:

isReadOnly

A Boolean specifying whether the output database was opened with read-only access.

amplitudes

A repository of Amplitude objects.

filters

A repository of Filter objects.

rootAssembly

An OdbAssembly object.

jobData

A JobData object.

parts

A repository of OdbPart objects.

materials

A repository of Material objects.

steps

A repository of OdbStep objects.

sections

A repository of Section objects.

sectionCategories

A repository of SectionCategory objects.

sectorDefinition

A SectorDefinition object.

userData

A UserData object.

customData

A RepositorySupport object.

profiles

A repository of Profile objects.

34.2 AnalyticSurface object

The AnalyticSurface object is a geometric surface that can be described with straight and/or curved line segments.

Access

```
import odbAccess
```

```
session.odbs[name].parts[name].analyticSurface
session.odbs[name].rootAssembly.instances[name].analyticSurface
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.analyticSurface
```

34.2.1 Members

The AnalyticSurface object has the following members:

name

A String specifying the name of the analytic surface.

type

A SymbolicConstant specifying the type of AnalyticSurface object. Possible values are SEGMENTS, CYLINDER, and REVOLUTION.

filletRadius

A Float specifying radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

segments

An OdbSequenceAnalyticSurfaceSegment object specifying the profile associated with the surface.

localCoordData

A tuple of tuples of Floats specifying the global coordinates of points representing the local coordinate system, if used.

34.3 AnalyticSurfaceSegment object

An individual segment of the analytic surface.

Access

```
import odbAccess
session.odbs[name].parts[name].analyticSurface.segments[i]
session.odbs[name].rootAssembly.instances[name].analyticSurface \
.segments[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.analyticSurface.segments[i]
```

34.3.1 AnalyticSurfaceSegment(...)

This method creates an AnalyticSurfaceSegment object.

Path

`odbAccess.AnalyticSurfaceSegment`

Required arguments

type

A SymbolicConstant specifying the type of AnalyticSurfaceSegment. Possible values are START, LINE, CIRCLE, and PARABOLA.

data

A sequence of sequences of Floats specifying the coordinates of point/s representing the segment of the AnalyticSurface object. If *type*=CIRCLE, the first row contains coordinates of the end point and the second row contains coordinates of the center point. If *type*=PARABOLA, the first row contains coordinates of the middle point and the second row contains coordinates of the end point. If *type*=START or *type*=LINE, a single row contains coordinates of the start/end point.

Optional arguments

None.

Return value

An AnalyticSurfaceSegment object.

Exceptions

None.

34.3.2 Members

The AnalyticSurfaceSegment object has members with the same names and descriptions as the arguments to the `AnalyticSurfaceSegment` method.

34.4 BeamOrientation object

The BeamOrientation object represents the direction of the first beam section axis $\mathbf{n}_1$. Specifying the beam orientation using an additional node in the element connectivity list is not supported.

Access

```
import odbAccess
session.odbs[name].parts[name].beamOrientations[i]
session.odbs[name].rootAssembly.instances[name].beamOrientations[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
```

```
.instance.beamOrientations [i]
```

34.4.1 Members

The BeamOrientation object can have the following members:

method

A SymbolicConstant specifying the orientation assignment method. Possible values are N1_COSINES, CSYS, and VECT.

region

An OdbSet object specifying a region for which the beam orientation is defined.

vector

A tuple of Floats specifying direction cosines of the n_1 -direction of the beam cross-section.

34.5 FieldBulkData object

The FieldBulkData object represents the entire field data for a class of elements or nodes. All elements in a class correspond to the same element type and material.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i].fieldOutputs[name] \
.bulkDataBlocks [i]
```

34.5.1 Members

The FieldBulkData object can have the following members:

position

A SymbolicConstant specifying the position of the output in the element. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- ELEMENT_FACE.
- CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

type

A SymbolicConstant specifying the output type. Possible values are SCALAR, VECTOR, TENSOR_3D_FULL, TENSOR_3D_PLANAR, TENSOR_3D_SURFACE, TENSOR_2D_PLANAR, and TENSOR_2D_SURFACE.

instance

An OdbInstance object specifying the part to which the labels belong.

sectionPoint

A SectionPoint object specifying the section point number of the current block of data.

elementLabels

A sequence of Ints specifying the element labels of the elements in the block. *elementLabels* is valid only if *position*=INTEGRATION_POINT, CENTROID, ELEMENT_NODAL, or ELEMENT_FACE.

nodeLabels

A sequence of Ints specifying the node labels of the nodes in the block. *nodeLabels* is valid only if *position*=ELEMENT_NODAL or NODAL.

componentLabels

A sequence of Strings specifying the component labels.

integrationPoints

A sequence of Ints specifying the integration points in the elements in the block. *integrationPoints* is available only if *position*=INTEGRATION_POINT.

data

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *data* is a sequence containing the components for each element or node in the block. If the underlying data are in double precision, an exception will be thrown.

conjugateData

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *conjugateData* is a sequence containing the imaginary part of the components for each element or node in the block. If the underlying data are in double precision, an exception will be thrown.

mises

A sequence of Floats specifying the calculated von Mises stress at each output location in the block of element data, or NULL. The value is valid only when the *validInvariants* member includes MISES; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

localCoordSystem

A pointer to an array of Floats specifying the quaternion representing the local coordinate system at each output location. The quaternion is returned in the form

$$\mathbf{q} = (\mathbf{q}, q_0),$$

which is the reverse of that shown in “Rotation variables,” Section 1.3.1 of the Abaqus Theory Guide. *localCoordSystem* is available only for TENSOR data written in a local coordinate system. If the underlying data are in double precision, an exception will be thrown.

34.6 FieldLocation object

The FieldLocation object specifies locations for which data are available in the field. For example, a displacement field will have a FieldLocation object with a *position* member value of NODAL. The FieldLocation object has no constructor; it is created automatically as an element of the *location* member of a FieldOutput object by the **addData** method of a FieldOutput object.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i].fieldOutputs[name] \
    .locations[i]
```

34.6.1 Members

The FieldLocation object can have the following members:

position

A SymbolicConstant specifying the position of the output in the element. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- ELEMENT_FACE.
- CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

sectionPoints

A SectionPointArray object.

34.7 FieldOutput object

A FieldOutput object contains field data for a specific output variable.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i].fieldOutputs[name]
```

34.7.1 FieldOutput(...)

This method creates a FieldOutput object.

Path

```
session.odbs[name].steps[name].frames[i].FieldOutput
```

Required arguments

name

A String specifying the output variable name.

description

A String specifying the output variable. Colon (:) should not be used as a part of the field output description.

type

A SymbolicConstant specifying the output type. Possible values are SCALAR, VECTOR, TENSOR_3D_FULL, TENSOR_3D_PLANAR, TENSOR_3D_SURFACE, TENSOR_2D_PLANAR, and TENSOR_2D_SURFACE.

Optional arguments

componentLabels

A sequence of Strings specifying the labels for each component of the value. The length of the sequence must match the type. If *type*=TENSOR, the default value is *name* with the suffixes ('11', '22', '33', '12', '13', '23'). If *type*=VECTOR, the default value is *name* with the suffixes ('1', '2', '3'). If *type*=SCALAR, the default value is an empty sequence.

validInvariants

A sequence of SymbolicConstants specifying which invariants should be calculated for this field. An empty sequence indicates that no invariants are valid for this field. Possible values are:

- MAGNITUDE
- MISES
- TRESCA
- PRESS
- INV3
- MAX_PRINCIPAL
- MID_PRINCIPAL

- MIN_PRINCIPAL
- MAX_INPLANE_PRINCIPAL
- MIN_INPLANE_PRINCIPAL
- OUTOFPLANE_PRINCIPAL

The default value is an empty sequence.

isEngineeringTensor

A Boolean specifying whether the field is an engineering tensor or not. Setting *isEngineeringTensor* to true makes a tensor field behave as a strain like quantity where the off-diagonal components of tensor are halved for invariants computation. This parameter applies only to tensor field outputs. The default value is OFF.

Return value

A FieldOutput object.

Exceptions

None.

34.7.2 FieldOutput(...)

This method creates a FieldOutput object from an existing FieldOutput object.

Path

`session.odbs[name].steps[name].frames[i].FieldOutput`

Required argument

field

A FieldOutput object.

Optional arguments

name

A String specifying the name of the FieldOutput object.

description

A String specifying the output variable. Colon (:) should not be used as a part of the field output description.

Return value

A FieldOutput object.

Exceptions

None.

34.7.3 **addData(...)**

This method adds data to a FieldOutput object.

Arguments

Required arguments

position

A SymbolicConstant specifying the position of the output. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

instance

An OdbInstance object specifying the namespace for labels.

labels

A sequence of Ints specifying the labels of the nodes or elements where the values in *data* are located. The node or element labels must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

A sequence of sequences of Floats specifying the data values for the specified *position*, *instance*, and *labels*. The values must be given in the correct order. Element nodal data follow the order of nodal connectivity defined in the Abaqus documentation. Integration point data follow the order defined in the Abaqus documentation. Section point data for beams and shells follow the convention given in the Abaqus documentation. For more information, see Part VI, “Elements,” of the Abaqus Analysis User’s Guide. These data create FieldValue objects internally.

Optional arguments

sectionPoint

A SectionPoint object specifying the location in the section. Although *sectionPoint* is an optional argument to the **addData** method, omitting the argument does have consequences for visualization. If you omit the argument when you are writing field output data for a shell or a beam, you cannot subsequently select the section point to display when you are displaying the field output data using the Visualization module.

localCoordSystem

The *localCoordSystem* parameter can be specified using either of the following:

- A sequence of sequences of Floats specifying the 3×3 matrix of direction cosines of the local coordinate system. This argument is available only for fields with type=TENSOR or VECTOR.
- A sequence of matrices of floats specifying the direction cosines of the local coordinates systems, where the sequence is the same length as *data*. If *localCoordSystem* is a matrix, a different local coordinate system applies to each data value.

User supplied values of *localCoordSystem* are transposed before storing in the database.

Return value

None

Exceptions

The **addData** method throws many exceptions of type `odbException`. For example, if the local coordinate system is specified for scalar data:

`odbException: Transformation not allowed for scalar data.`

34.7.4 addData(...)

This method adds the data from a field created using the **getSubset** method and mathematical operators to the database. The user must create a field to contain the new data and then use the **addData** method to assign the data from the fields.

Arguments

Required argument

field

A FieldOutput object specifying the data to add.

Optional arguments

None.

Return value

None

Exceptions

The **addData** method throws many exceptions of type `odbException`. For example, if the local coordinate system is specified for scalar data:

`odbException: Transformation not allowed for scalar data.`

34.7.5 **addData(...)**

This method adds data to a FieldOutput object.

Arguments

Required arguments

position

A SymbolicConstant specifying the position of the output. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.
- ELEMENT_FACE_INTEGRATION_POINT, specifying the values calculated at the element face integration points.
- SURFACE_INTEGRATION_POINT, specifying the values calculated at the surface integration points. Selecting this value prompts the Visualization module to calculate the sum of the values at the ELEMENT_FACE_INTEGRATION_POINT position from multiple surfaces.

set

An OdbSet object specifying the instance-level set defining the region for **addData**. The set must be defined in the same output database as the output database into which the new field output data is being written. The node or element labels in the set must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

A sequence of sequences of Floats specifying the data values for the specified position and labels in the set. Each row of data provides the value at one unique position. The width of each row should match the number of required components for the data. The values must be given in the order that matches the ordering of labels in the set.

The order of the element nodal data, integration point data, and section point data for beams and shells follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.”

Optional arguments

sectionPoint

A SectionPoint object specifying the location in the section. Although *sectionPoint* is an optional argument to the **addData** method, omitting the argument does have consequences for visualization. If you omit the argument when you are writing field output data for a shell or a

beam, you cannot subsequently select the section point to display when you are displaying the field output data using the Visualization module.

conjugateData

An odb_SequenceSequenceFloat object specifying the imaginary data values for the specified position, instance, and labels. You must provide this data when you add complex fields to the output database. The order of the values follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.

Return value

None

Exceptions

If you specify an odbSet containing entities from multiple instances:

odbException: Entities from multiple instances present in set.

The **addData** method throws many exceptions of type odbException. For example, if the local coordinate system is specified for scalar data:

odbException: Transformation not allowed for scalar data.

34.7.6 getScalarField(...)

This method generates a scalar field containing the extracted component or calculated invariant values. The new field will hold values for the same nodes or elements as the parent field. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Arguments

Required argument

invariant

A SymbolicConstant specifying the invariant. Possible values areMAGNITUDE, MISES, TRESCA, PRESS, INV3, MAX_PRINCIPAL, MID_PRINCIPAL, MIN_PRINCIPAL, MAX_INPLANE_PRINCIPAL, MIN_INPLANE_PRINCIPAL, and OUTOFPLANE_PRINCIPAL.

Optional arguments

None.

Return value

AFieldOutput object.

Exceptions

None.

34.7.7 getScalarField(...)

This method generates a scalar field containing the extracted component or calculated invariant values. The new field will hold values for the same nodes or elements as the parent field. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Arguments

Required argument

componentLabel

A String specifying the component label, such as “S11”.

Optional arguments

None.

Return value

A FieldOutput object.

Exceptions

None.

34.7.8 getSubset(...)

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional arguments

position

A SymbolicConstant specifying the position of the output in the element. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.

- **CENTROID**, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

If the requested field values are not found in the output database at the specified **ELEMENT_NODAL** or **CENTROID** positions, they are extrapolated from the field data at the **INTEGRATION_POINT** position.

readOnly

A Boolean specifying whether the extrapolated data returned by this call is written to the output database. The default value is OFF.

Return value

A FieldOutput object.

Exceptions

None.

34.7.9 getSubset(...)

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

region

An OdbSet specifying the region for which to extract values. Nodes or elements in the sets must be sorted and arranged in ascending order. Use of unsorted sets is not supported. This is especially true for sets in Abaqus output databases obtained through running the analysis in multiple domains. In such cases, a new sorted set must be created before using it in the getSubset method.

Return value

A FieldOutput object.

Exceptions

None.

34.7.10 getSubset(...)

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

localCoordSystem

A sequence of sequences of Floats specifying the 3×3 matrix of direction cosines. Field values associated with the supplied coordinate system will be extracted.

Return value

A FieldOutput object.

Exceptions

None.

34.7.11 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

sectionPoint

A SectionPoint object.

Return value

A FieldOutput object.

Exceptions

None.

34.7.12 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

location

A FieldLocation object.

Return value

A FieldOutput object.

Exceptions

None.

34.7.13 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

region

An OdbMeshElement specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

34.7.14 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

region

An OdbMeshNode specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

34.7.15 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

region

An OdbInstance specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

34.7.16 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Arguments

Required arguments

None.

Optional argument

elementType

A String specifying the element type for which to extract values. The string must correspond to a valid Abaqus element type.

Return value

A FieldOutput object.

Exceptions

None.

34.7.17 **getTransformedField(...)**

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

34.7.18 **getTransformedField(...)**

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will

perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

34.7.19 getTransformedField(...)

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

rotationField

A FieldOutput object specifying the nodal rotational displacement vectors required by moving coordinate systems that follow a 6-dof node, to determine instantaneous configurations.

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

34.7.20 getConnectorFieldXformedToNodeA(...)

This method generates a new vector field containing the transformed component values of the parent connector field to the node A coordinate system. The new field will hold values for the same connector elements as the parent field. Some connection types such as Axial, Link, Slip Ring, and Radial Thrust require that the deformationField be specified.

Arguments

Required arguments

None.

Optional argument

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

Return value

A FieldOutput object.

Exceptions

The **getConnectorFieldXformedToNodeA** method throws an exception if the field requires a deformationField and the argument is not supplied.

odbException: Deformation field is required for transforming this connector field.

34.7.21 **setComponentLabels(...)**

This method sets the component labels for the FieldOutput object.

Arguments

Required argument

componentLabels

A sequence of Strings specifying the labels for each component of the value. The length of the sequence must match the type. If *type*=TENSOR, the default value is *name* with the suffixes ('11', '22', '33', '12', '13', '23'). If *type*=VECTOR, the default value is *name* with the suffixes ('1', '2', '3'). If *type*=SCALAR, the default value is an empty sequence.

Return value

None

Exceptions

None.

34.7.22 **setDataType(...)**

This method sets the data type of a FieldOutput object.

Arguments

Required argument

type

A SymbolicConstant specifying the output type. Possible values are SCALAR, VECTOR, TENSOR_3D_FULL, TENSOR_3D_PLANAR, TENSOR_3D_SURFACE, TENSOR_2D_PLANAR, and TENSOR_2D_SURFACE.

Return value

None

Exceptions

None.

34.7.23 setValidInvariants(...)

This method sets the invariants valid for the FieldOutput object.

Arguments

Required argument

validInvariants

A sequence of SymbolicConstants specifying which invariants should be calculated for this field. An empty sequence indicates that no invariants are valid for this field. Possible values are:

- MAGNITUDE
- MISES
- TRESCA
- PRESS
- INV3
- MAX_PRINCIPAL
- MID_PRINCIPAL
- MIN_PRINCIPAL
- MAX_INPLANE_PRINCIPAL
- MIN_INPLANE_PRINCIPAL
- OUTOFPLANE_PRINCIPAL

The default value is an empty sequence.

Return value

None

Exceptions

None.

34.7.24 Members

The FieldOutput object has members with the same names and descriptions as the arguments to the **FieldOutput** method.

In addition, the FieldOutput object can have the following members:

dim

An Int specifying the dimension of vector or the first dimension (number of rows) of matrix.

dim2

An Int specifying the second dimension (number of columns) of matrix.

isComplex

A Boolean specifying whether the data are complex.

locations

A FieldLocationArray object.

values

A FieldValueArray object specifying the order of the objects in the array is determined by the Abaqus Scripting Interface; see the *data* argument to the **addData** method for a description of the order.

34.8 FieldValue object

The FieldValue object represents the field data at a point. The FieldValue object has no constructor; it is created by the Odb object when data are added to the FieldOutput object using the **addData** method. For faster, bulk-data access, see “Using bulk data access to an output database,” Section 10.10.7 of the Abaqus Scripting User’s Guide.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i]
```

34.8.1 Members

The FieldValue object has the following members:

position

A SymbolicConstant specifying the position of the output in the element. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- ELEMENT_FACE, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element.
- CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

precision

A SymbolicConstant specifying the precision of the output in the element. Possible values are:

- SINGLE_PRECISION, specifying that the output values are in single precision.
- DOUBLE_PRECISION, specifying that the output values are in double precision.

elementLabel

An Int specifying the element label of the element containing the location. *elementLabel* is available only if *position*=INTEGRATION_POINT, CENTROID, ELEMENT_NODAL, or ELEMENT_FACE.

nodeLabel

An Int specifying the node label of the node containing the location. *nodeLabel* is available only if *position*=ELEMENT_NODAL or NODAL.

integrationPoint

An Int specifying the integration point in the element. *integrationPoint* is available only if *position*=INTEGRATION_POINT.

face

A SymbolicConstant specifying the face of the element. *face* is available only if *position*=ELEMENT_FACE.

type

A SymbolicConstant specifying the output type. Possible values are SCALAR, VECTOR, TENSOR_3D_FULL, TENSOR_3D_PLANAR, TENSOR_3D_SURFACE, TENSOR_2D_PLANAR, and TENSOR_2D_SURFACE.

magnitude

A Float specifying the length or magnitude of the vector. *magnitude* is valid only when *type*=VECTOR.

mises

A Float specifying the calculated von Mises stress. The value is valid only when the *validInvariants* member includes MISES; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

ODB COMMANDS

tresca

A Float specifying the calculated Tresca stress. The value is valid only when the *validInvariants* member includes TRESCA; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

press

A Float specifying the calculated pressure stress. The value is valid only when the *validInvariants* member includes PRESS; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

inv3

A Float specifying the calculated third stress invariant. The value is valid only when the *validInvariants* member includes INV3; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

maxPrincipal

A Float specifying the calculated maximum principal stress. The value is valid only when the *validInvariants* member includes MAX_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

midPrincipal

A Float specifying the calculated intermediate principal stress. The value is valid only when the *validInvariants* member includes MID_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

minPrincipal

A Float specifying the minimum principal stress. The value is valid only when the *validInvariants* member includes MIN_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

maxInPlanePrincipal

A Float specifying the maximum principal in-plane stress. The value is valid only when the *validInvariants* member includes MAX_INPLANE_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

minInPlanePrincipal

A Float specifying the calculated minimum principal in-plane stress. The value is valid only when the *validInvariants* member includes MIN_INPLANE_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

outOfPlanePrincipal

A Float specifying the calculated principal out-of-plane stress. The value is valid only when the *validInvariants* member includes OUTOFPLANE_PRINCIPAL; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

instance

An OdbInstance object specifying the part to which the labels belong.

sectionPoint

A SectionPoint object.

localCoordSystem

A tuple of tuples of Floats specifying the 3 x 3 matrix of Floats specifying the direction cosines of the local coordinate system. Each sequence represents a row in the direction cosine matrix. *localCoordSystem* is available only for TENSOR data written in a local coordinate system. If the underlying data are in double precision, an exception will be thrown.

localCoordSystemDouble

A tuple of tuples of Floats specifying the 3 x 3 matrix of Doubles specifying the direction cosines of the local coordinate system. Each sequence represents a row in the direction cosine matrix. *localCoordSystemDouble* is available only for TENSOR data written in a local coordinate system. If the underlying data are in single precision, an exception will be thrown.

data

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *data* is a sequence containing the components. If the underlying data are in double precision an exception will be thrown.

dataDouble

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *data* is a sequence containing the components. If the underlying data are in single precision, an exception will be thrown.

conjugateData

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *conjugateData* is a sequence containing the components. If the underlying data are in double precision, an exception will be thrown.

conjugateDataDouble

A tuple of Floats specifying data in the form described by *type*. If *type*=TENSOR or VECTOR, *conjugateData* is a sequence containing the components. If the underlying data are in single precision, an exception will be thrown.

34.9 HistoryOutput object

The HistoryOutput object contains the history output at a point for the specified variable.

Access

```
import odbAccess
session.odbs[name].steps[name].historyRegions[name] \
.historyOutputs[name]
```

34.9.1 HistoryOutput(...)

This method creates a HistoryOutput object.

Path

`session.odbs[name].steps[name].historyRegions[name].HistoryOutput`

Required arguments

name

A String specifying the output variable name.

description

A String specifying the output variable.

type

A SymbolicConstant specifying the output type. Only SCALAR is currently supported.

Optional argument

validInvariants

A sequence of SymbolicConstants specifying which invariants should be calculated for this field. Possible values are MAGNITUDE, MISES, TRESCA, PRESS, INV3, MAX_PRINCIPAL, MID_PRINCIPAL, and MIN_PRINCIPAL. The default value is an empty sequence.

Return value

A HistoryOutput object.

Exceptions

None.

34.9.2 **addData(...)**

This method adds data to the *data* member of the HistoryOutput object.

Arguments

Required arguments

frame

A Double specifying the frame value. *frame* can be specified in step time, frequency, or mode number.

value

A Double specifying the value of the variable at the frame value specified in *frame*.

Optional arguments

None.

Return value

None

Exceptions

None.

34.9.3 **addData(...)**

This method adds data to the *data* member of the HistoryOutput object.

Arguments

Required arguments

frame

A sequence of Floats specifying the frame values. *frame* can be specified in step time, frequency, or mode number.

value

A sequence of Floats specifying the value of the variable at the frame values specified in *frame*.

Optional arguments

None.

Return value

None

Exceptions

If the length of *frame* is not the same as the length of *value* a ValueError is raised.

34.9.4 **addData(...)**

This method adds data to the *data* member of the HistoryOutput object.

Arguments

Required argument

data

A sequence of pairs of Floats specifying the pairs (*frameValue*, *value*) where *frameValue* is either time, frequency, or mode and *value* is the value of the specified variable at *frameValue*. (This value depends on the type of the variable.)

Optional arguments

None.

Return value

None

Exceptions

None.

34.9.5 Members

The HistoryOutput object has members with the same names and descriptions as the arguments to the **HistoryOutput** method.

In addition, the HistoryOutput object has the following members:

data

A tuple of pairs of Floats specifying the pairs (*frameValue*, *value*) where *frameValue* is either time, frequency, or mode and *value* is the value of the specified variable at *frameValue*. (This value depends on the type of the variable.)

conjugateData

A tuple of pairs of Floats specifying the imaginary portion of a specified complex variable at each frame value (time, frequency, or mode). The pairs have the form (*frameValue*, *value*).

34.10 HistoryPoint object

The HistoryPoint object specifies the point at which history data will be collected. The HistoryPoint object is a temporary object used as an argument to the **HistoryRegion** method.

Access

```
import odbAccess
session.odbs[name].steps[name].historyRegions[name].point
```

34.10.1 HistoryPoint(...)

This method creates a HistoryPoint object for a node.

Path

```
odbAccess.HistoryPoint
```


Required argument

node

An OdbMeshNode object specifying the node for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

34.10.2 HistoryPoint(...)

This method creates a HistoryPoint object for an element.

Path

`session.odbs[name].steps[name].historyRegions[name].HistoryPoint`

Required argument

element

An OdbMeshElement object specifying the element for which the data are to be collected.

Optional arguments

ipNumber

An Int specifying the integration point. This argument is used to define a history output position of INTEGRATION_POINT or ELEMENT_FACE_INTEGRATION_POINT. The default value is 0.

sectionPoint

A SectionPoint object.

face

A SymbolicConstant specifying the element face. This argument is used to define a history output position of ELEMENT_FACE or ELEMENT_FACE_INTEGRATION_POINT. Possible values are:

- FACE_UNKOWN, specifying this value indicates that no value has been specified.
- FACE1, specifying this value indicates that element face 1 has been specified.
- FACE2, specifying this value indicates that element face 2 has been specified.
- FACE3, specifying this value indicates that element face 3 has been specified.
- FACE4, specifying this value indicates that element face 4 has been specified.

- FACE5, specifying this value indicates that element face 5 has been specified.
- FACE6, specifying this value indicates that element face 6 has been specified.
- SIDE1, specifying this value indicates that element side 1 has been specified.
- SIDE2, specifying this value indicates element side 2 has been specified.
- END1, specifying this value indicates that element end 1 has been specified.
- END2, specifying this value indicates that element end 2 has been specified.
- END3, specifying this value indicates that element end 3 has been specified.

The default value is FACE_UNKNOWN.

node

An OdbMeshNode object specifying the node for which the data are to be collected.

Return value

A HistoryPoint object.

Exceptions

None.

34.10.3 HistoryPoint(...)

This method creates a HistoryPoint object for a region.

Path

`session.odbs[name].steps[name].historyRegions[name].HistoryPoint`

Required argument

region

An OdbSet object specifying the region for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

34.10.4 HistoryPoint(...)

This method creates a HistoryPoint object for the OdbAssembly object.

Path

`session.odbs[name].steps[name].historyRegions[name].HistoryPoint`

Required argument

assembly

An OdbAssembly object specifying the assembly for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

34.10.5 HistoryPoint(...)

This method creates a HistoryPoint object for the OdbInstance object.

Path

`session.odbs[name].steps[name].historyRegions[name].HistoryPoint`

Required argument

instance

An OdbInstance object specifying the instance for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

34.10.6 Members

The HistoryPoint object has members with the same names and descriptions as the arguments to the **HistoryPoint** method.

In addition, the HistoryPoint object can have the following members:

ipNumber

An Int specifying the integration point. This argument is used to define a history output position of INTEGRATION_POINT or ELEMENT_FACE_INTEGRATION_POINT. The default value is 0.

face

A SymbolicConstant specifying the element face. This argument is used to define a history output position of ELEMENT_FACE or ELEMENT_FACE_INTEGRATION_POINT. Possible values are:

- FACE_UNKOWN, specifying this value indicates that no value has been specified.
- FACE1, specifying this value indicates that element face 1 has been specified.
- FACE2, specifying this value indicates that element face 2 has been specified.
- FACE3, specifying this value indicates that element face 3 has been specified.
- FACE4, specifying this value indicates that element face 4 has been specified.
- FACE5, specifying this value indicates that element face 5 has been specified.
- FACE6, specifying this value indicates that element face 6 has been specified.
- SIDE1, specifying this value indicates that element side 1 has been specified.
- SIDE2, specifying this value indicates element side 2 has been specified.
- END1, specifying this value indicates that element end 1 has been specified.
- END2, specifying this value indicates that element end 2 has been specified.
- END3, specifying this value indicates that element end 3 has been specified.

The default value is FACE_UNKNOWN.

position

A SymbolicConstant specifying the result position of the history point. Possible values are:

- NODAL, specifying the values calculated at the nodes.
- ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- INTEGRATION_POINT, specifying the values calculated at the integration points.
- ELEMENT_FACE, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element.
- ELEMENT_FACE_INTEGRATION_POINT, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element when the surface facets have integration points.
- WHOLE_ELEMENT, specifying the results obtained for whole element variables.

- `WHOLE_REGION`, specifying the results for an entire region of the model.
- `WHOLE_PART_INSTANCE`, specifying the results for an entire part instance of the model.
- `WHOLE_MODEL`, specifying the results for the entire model.

element

An `OdbMeshElement` object specifying the element for which the data are to be collected.

sectionPoint

A `SectionPoint` object.

region

An `OdbSet` object specifying the region for which the data are to be collected.

assembly

An `OdbAssembly` object specifying the assembly for which the data are to be collected.

instance

An `OdbInstance` object specifying the instance for which the data are to be collected.

34.11 HistoryRegion object

The `HistoryRegion` object contains history data for a single location in the model.

Access

```
import odbAccess
session.odbs[name].steps[name].historyRegions[name]
```

34.11.1 HistoryRegion(...)

This method creates a `HistoryRegion` object.

Path

```
session.odbs[name].steps[name].HistoryRegion
```

Required arguments

name

A String specifying the name of the `HistoryRegion` object.

description

A String specifying the description of the `HistoryRegion` object.

point

A `HistoryPoint` object specifying the point to which the history data refer.

Optional argument

loadCase

None or an OdbLoadCase object specifying the load case associated with the HistoryRegion object. The default value is **None**.

Return value

A HistoryRegion object.

Exceptions

None.

34.11.2 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Arguments

Required argument

variableName

A String specifying the name of the output variable to return.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

34.11.3 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Arguments

Required argument

start

A Float specifying the start of the subset. This is the same as the first item in the data array member of the HistoryOutput object.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

34.11.4 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Arguments

Required arguments

start

A Float specifying the start of the subset. This is the same as the first item in the data array member of the HistoryOutput object.

end

A Float specifying the end of the subset.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

34.11.5 **Members**

The HistoryRegion object has members with the same names and descriptions as the arguments to the **HistoryRegion** method.

In addition, the HistoryRegion object can have the following members:

position

A SymbolicConstant specifying the position of the history output. Possible values are NODAL, INTEGRATION_POINT, WHOLE_ELEMENT, WHOLE_REGION, and WHOLE_MODEL.

historyOutputs

A repository of HistoryOutput objects.

34.12 JobData object

The JobData object describes the context in which the analysis was run.

Access

```
import odbAccess
session.odbs[name].jobData
```

34.12.1 Members

The JobData object has the following members:

name

A String specifying the name of the job.

analysisCode

A SymbolicConstant specifying the analysis code. Possible values are ABAQUS_STANDARD, ABAQUS_EXPLICIT, and UNKNOWN_ANALYSIS_CODE.

precision

A SymbolicConstant specifying the precision. Only SINGLE_PRECISION is currently supported. Possible values are DOUBLE_PRECISION and SINGLE_PRECISION.

version

A String specifying the release of the analysis code.

creationTime

A String specifying the date and time at which the analysis was run.

modificationTime

A String specifying the date and time at which the database was last modified.

machineName

A String specifying the name of the machine on which the analysis was run.

productAddOns

A String specifying an odb_Sequence of productAddOns. Possible values are: Possible values are AQUA, DESIGN, BIORID, CEL, SOLITER, and CAVPARALLEL.

34.13 OdbAssembly object

The OdbAssembly object has no constructor; it is created automatically when an Odb object is created. Abaqus creates the *rootAssembly* member when an Odb object is created.

Access

```
import odbAccess
session.odbs[name].rootAssembly
```

34.13.1 ConnectorOrientation(...)

This method assigns a connector orientation to a connector region.

Arguments

Required argument

region

An OdbSet specifying a region.

Optional arguments

localCsys1

An OdbDatumCsys object specifying the first connector node local coordinate system or **None**, indicating the global coordinate system.

axis1

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation of the first connector node is applied. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle1

A Float specifying the angle of the additional rotation about the first connector node axis. The default value is 0.0.

orient2sameAs1

A Boolean specifying whether the same orientation settings should be used for the second node of the connector. The default value is OFF.

localCsys2

An OdbDatumCsys object specifying the second connector node local coordinate system or **None**, indicating the global coordinate system.

axis2

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation of the second connector node is applied. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle2

A Float specifying the angle of the additional rotation about the second connector node axis. The default value is 0.0.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Connector orientation assignment requires element set.

34.13.2 SectionAssignment(...)

This method is used to assign a section to a region on an instance. Only connector sections can be assigned at the assembly level.

Arguments

Required arguments

region

An OdbSet specifying a region.

section

A Section object.

Optional arguments

None.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Section assignment requires element set.

34.13.3 addElements(...)

This method is used to define elements using nodes defined at the OdbAssembly and/or OdbInstance level. For connector elements connected to ground, specify the lone node in the connectivity. The position of the ground node cannot be specified. This is a limitation.

WARNING: Adding elements not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Arguments

Required arguments

labels

A sequence of Ints specifying the element labels.

connectivity

A sequence of sequences of Ints specifying the nodal connectivity.

instanceNames

A sequence of Strings specifying the instanceNames of each node in the nodal connectivity array.
If the node is defined at the assembly level, the instance name should be an empty string

type

A String specifying the element type.

Optional arguments

elementSetName

A String specifying a name for this element set. The default value is the empty string.

sectionCategory

A SectionCategory object for this element set.

Return value

None

Exceptions

Only certain element types are permitted at the assembly level. e.g., connector elements.

OdbError: Addition of this element type is not permitted at the assembly level

If length of label array does not match connectivity data length:

OdbError: Connectivity array must be provided for all element

34.13.4 addNodes(...)

This method adds nodes to the OdbAssembly object using node labels and coordinates.

WARNING: Adding nodes not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Arguments

Required arguments

labels

A sequence of Ints specifying the node labels.

coordinates

A sequence of sequences of Floats specifying the nodal coordinates.

Optional argument

nodeSetName

A String specifying a name for this node set. The default value is None.

Return value

None

Exceptions

If length of labels does not match length of coordinates:

OdbError: Number of node labels and coordinates does not match

If width of coordinate array does not match assembly dimension:

OdbError: Node location specification does not correspond to part dimensions

34.13.5 RigidBody(...)

This method defines an OdbRigidBody on the assembly.

Arguments

Required argument

referenceNode

An OdbSet specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are INPUT and CENTER_OF_MASS. The default value is INPUT.

isothermal

A Boolean specifying an isothermal rigid body. The default value is OFF. This parameter is used only for a fully coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

34.13.6 Members

The OdbAssembly object can have the following members:

instances

A repository of OdbInstance objects.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

nodes

An OdbMeshNodeArray object.

elements

An OdbMeshElementArray object.

datumCsyses

A repository of OdbDatumCsys objects.

sectionAssignments

A SectionAssignmentArray object.

rigidBodies

An OdbRigidBodyArray object.

pretensionSections

An OdbPretensionSectionArray object.

connectorOrientations

A ConnectorOrientationArray object.

34.14 OdbDatumCsys object

The OdbDatumCsys object contains a coordinate system that can be stored in an output database. You can create the datum coordinate system in the Visualization module during an Abaqus/CAE session and save the datum coordinate system to the output database before you exit Abaqus/CAE. Alternatively, the analysis code can write the datum coordinate system to the output database.

Access

```
import odbAccess
session.odbs[name].rootAssembly.datumCsyses[name]
```

34.14.1 DatumCsysByThreePoints(...)

This method creates an OdbDatumCsys object using three points. A datum coordinate system created with this method results in a fixed system.

Path

```
session.odbs[name].rootAssembly.DatumCsysByThreePoints
```

Required arguments

name

A String specifying the repository key.

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

A sequence of Floats specifying the coordinates of the origin of the datum coordinate system.

point1

A sequence of Floats specifying the coordinates of a point on the local 1- or r -axis.

point2

A sequence of Floats specifying the coordinates of a point in the 1–2 or r – θ plane.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

34.14.2 DatumCsysByThreeNodes(...)

This method creates an OdbDatumCsys object using the coordinates of three OdbMeshNode objects. A datum coordinate system created with this method results in a system that follows the position of the three nodes. Results, such as those for displacement, are resolved into the orientation of the datum coordinate system without regard to the position of its origin. The last three arguments are given in the form of an OdbMeshNode object.

Path

`session.odbs[name].rootAssembly.DatumCsysByThreeNodes`

Required arguments

name

A String specifying the repository key.

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

An OdbMeshNode object specifying a node at the origin of the datum coordinate system.

point1

An OdbMeshNode object specifying a node on the local 1- or r -axis.

point2

An OdbMeshNode object specifying a node in the 1–2 or r – θ plane.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

34.14.3 DatumCsysByThreeCircNodes(...)

This method is convenient to use where there are no nodes along the axis of a hollow cylinder or at the center of a hollow sphere. The three nodes that you provide as arguments determine a circle in space. The center of the circle is the origin of the datum coordinate system. The normal to the circle is parallel to the z -axis of a cylindrical coordinate system or to the ϕ -axis of a spherical coordinate system. The line from the origin to the first node defines the r -axis.

Path

`session.odbs[name].rootAssembly.DatumCsysByThreeCircNodes`

Required arguments

name

A String specifying the repository key.

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

node1Arc

An OdbMeshNode object that lies on the circular arc.

node2Arc

An OdbMeshNode object that lies on the circular arc.

node3Arc

An OdbMeshNode object that lies on the circular arc.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

34.14.4 DatumCsysBy6dofNode(...)

A datum coordinate system created with this method results in a system that follows the position of a node. The node location defines the origin of the datum coordinate system. The rotational displacement (UR1, UR2, UR3) of the node defines the orientation of the coordinate system axes. Results, such as

those for displacement, are resolved into the orientation of the datum coordinate system without regard to the position of its origin. The last argument is given in the form of an OdbMeshNode object.

Path

`session.odbs[name].rootAssembly.DatumCsysBy6dofNode`

Required arguments

name

A String specifying the repository key.

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

An OdbMeshNode object specifying the origin of the datum coordinate system.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

34.14.5 DatumCsys(...)

This method copies oneOdbDatumCsys object to a new OdbDatumCsys object.

Path

`session.odbs[name].rootAssembly.DatumCsys`

Required arguments

name

A String specifying the repository key.

datumCsys

An OdbDatumCsys object specifying the object to be copied.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

34.14.6 Members

The OdbDatumCsys object has the following members:

name

A String specifying the repository key.

coordSysType

A SymbolicConstant specifying the type of coordinate system. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

origin

A tuple of Floats specifying the coordinates of the origin of the datum coordinate system.

xAxis

A tuple of Floats specifying a point on the X-axis.

yAxis

A tuple of Floats specifying a point on the Y-axis.

zAxis

A tuple of Floats specifying a point on the Z-axis.

34.15 OdbFrame object

The domain of the OdbFrame object is taken from the parent step.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i]
```

34.15.1 Frame(...)

This method creates an OdbFrame object and appends it to the frame sequence.

Path

```
session.odbs[name].steps[name].Frame
```

Required arguments

incrementNumber

An Int specifying the frame increment number within the step. The base frame has normally increment number 0, and the results run from 1. In case of multiple load cases, the same increment number is duplicated for each loadcase.

frameValue

A Float specifying the value in units determined by the *domain* member of the Step object. The equivalent in the time domain is *stepTime*; in the frequency domain the equivalent is *frequency*; and in the modal domain the equivalent is *mode*.

Optional argument

description

A String specifying the contents of the frame. The default value is an empty string.

Return value

An OdbFrame object.

Exceptions

None.

34.15.2 Frame(...)

This constructor creates an OdbFrame object in the frequency domain and appends it to the frame sequence. The arguments to the constructor are valid only when *domain*=FREQUENCY or *domain*=MODAL.

Path

`session.odbs[name].steps[name].Frame`

Required arguments

mode

An Int specifying the eigenmode. This member is valid only if *domain*=MODAL.

frequency

A Float specifying the frequency. This member is valid only if *domain*=FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

Optional argument

description

A String specifying the contents of the frame. The default value is an empty string.

Return value

An OdbFrame object.

Exceptions

None.

34.15.3 Frame(...)

This constructor creates an OdbFrame object for a specific load case and appends it to the frame sequence.

Path

`session.odbs[name].steps[name].Frame`

Required argument

loadCase

An OdbLoadCase object specifying the load case for the frame.

Optional arguments

description

A String specifying the contents of the frame. The default value is an empty string.

frequency

A Float specifying the frequency. This member is valid only if *domain*=FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

Return value

An OdbFrame object.

Exceptions

None.

34.15.4 Members

The OdbFrame object has members with the same names and descriptions as the arguments to the **Frame** method.

In addition, the OdbFrame object can have the following members:

cyclicModeNumber

An Int specifying the cyclic mode number associated with the data stored on this frame. Only frequency analyses of cyclic symmetry models possess cyclic mode numbers.

domain

A SymbolicConstant specifying the domain of the step of which the frame is a member. Possible values are TIME, FREQUENCY, and MODAL.

frequency

A Float specifying the frequency. This member is valid only if *domain*=FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

mode

An Int specifying the eigenmode. This member is valid only if *domain*=MODAL.

associatedFrame

An OdbFrame object specifying the real or imaginary portion of the data corresponding to this cyclic symmetry mode.

fieldOutputs

A repository of FieldOutput objects specifying the key to the *fieldOutputs* repository is a String representing an output variable.

loadCase

An OdbLoadCase object specifying the load case for the frame.

34.16 OdbInstance object

A part instance is the usage of a part within an assembly.

Access

```
import odbAccess
session.odbs[name].rootAssembly.instances[name]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance
```

34.16.1 Instance(...)

This method creates an OdbInstance object from an OdbPart object.

Path

```
session.odbs[name].rootAssembly.Instance
```

Required arguments

name

A String specifying the instance name.

ODB COMMANDS

object

An OdbPart object.

Optional argument

localCoordSystem

A sequence of sequences of three Floats specifying the rotation and translation of the part instance in the global Cartesian coordinate system. The first three sequences specify the new local coordinate system with its center at the origin.

- The first sequence specifies a point on the 1-axis.
- The second sequence specifies a point on the 2-axis.
- The third sequence specifies a point on the 3-axis.

The fourth sequence specifies the translation of the local coordinate system from the origin to its intended location.

For example, the following sequence moves a part 10 units in the *X*-direction with no rotation:

```
localCoordSystem = ((1, 0, 0), (0, 1, 0),  
                    (0, 0, 1), (10, 0, 0))
```

The following sequence moves a part 5 units in the *X*-direction with rotation:

```
localCoordSystem = ((0, 1, 0), (1, 0, 0),  
                    (0, 0, 1), (5, 0, 0))
```

transforms a part containing the two points

```
Pt1= (1,0,0)  
Pt2= (2,0,0)
```

to

```
Pt1 = (0, 6, 0)  
Pt2 = (0, 7, 0)
```

Return value

An OdbInstance object.

Exceptions

InvalidNameError.

34.16.2 assignBeamOrientation(...)

This method assigns a beam section orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

method

A SymbolicConstant specifying the assignment method. Only a value of N1_COSINES is currently supported.

vector

A sequence of three Floats specifying the approximate local $\mathbf{n}_1$ -direction of the beam cross-section.

Optional arguments

None.

Return value

None

Exceptions

None.

34.16.3 assignMaterialOrientation(...)

This method assigns a material orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

stackDirection

A SymbolicConstant specifying the stack or thickness direction of the material. Possible values are STACK_1, STACK_2, STACK_3, and STACK_ORIENTATION. The default value is STACK_3.

Return value

None

Exceptions

None.

34.16.4 assignRebarOrientation(...)

This method assigns a rebar reference orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

34.16.5 getElementFromLabel(...)

This method is used to retrieved an element with a specific label from an instance object.

Arguments

Required argument

label

An Int specifying the element label.

Optional arguments

None.

Return value

An OdbMeshElement object.

Exceptions

If no element with the specified label exists:

OdbError: Invalid element label

34.16.6 getNodeFromLabel(...)

This method is used to retrieved a node with a specific label from an instance object.

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

34.16.7 assignSection(...)

This method is used to assign a section to a region on an instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

section

A Section object.

Optional arguments

None.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Section assignment requires element set.

If the element set is not from the current instance:

OdbError: Section assignment requires element set from this part instance.

34.16.8 AnalyticRigidSurf2DPlanar(...)

This method is used to define a two-dimensional AnalyticSurface object on the instance.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments.
The default value is 0.0.

Return value

None

Exceptions

If OdbPart associated with the part instance is of type `THREE_D`:

OdbError: 2D-Planar Analytic Rigid Surface can be defined only if the instance is of type `TWO_D_PLANAR` or `AXISYMMETRIC`.

34.16.9 AnalyticRigidSurfExtrude(...)

This method is used to define a three-dimensional cylindrical AnalyticSurface on the instance.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional arguments

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

localCoordData

A sequence of sequences of Floats specifying the global coordinates of points used to define the local coordinate system.

Return value

None

Exceptions

If OdbPart associated with the part instance is not of type `THREE_D`:

OdbError: Analytic Rigid Surface of type `CYLINDER` can be defined only if the instance is of type `THREE_D`.

34.16.10 AnalyticRigidSurfRevolve(...)

This method is used to define a three-dimensional AnalyticSurface of revolution on the instance.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of `AnalyticSurfaceSegment` objects or an `OdbSequenceAnalyticSurfaceSegment` object.

Optional arguments

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

localCoordData

A sequence of sequences of Floats specifying the global coordinates of points used to define the local coordinate system.

Return value

None

Exceptions

If `OdbPart` associated with the part instance is not of type `THREE_D`:

`OdbError`: Analytic Rigid Surface of type `REVOLUTION` can be defined only if the instance is of type `THREE_D`.

34.16.11 RigidBody(...)

This method defines an `OdbRigidBody` on the instance.

Arguments

Required argument

referenceNode

An `OdbSet` specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are `INPUT`, and `CENTER_OF_MASS`. The default value is `INPUT`.

isothermal

A Boolean specifying an isothermal rigid body. The default value is `OFF`. This parameter is used only for a fully-coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

34.16.12 getNodeFromLabel(...)

This method is used to retrieved a node with a specific label from an instance object.

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

34.16.13 getNodeFromLabel(...)

This method is used to retrieved a node with a specific label from an instance object.

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

34.16.14 Members

The OdbInstance object can have the following members:

name

A String specifying the instance name.

type

A SymbolicConstant specifying the type of the Part object. Only a value of DEFORMABLE_BODY is currently supported.

embeddedSpace

A SymbolicConstant specifying the dimensionality of the Part object. Possible values are THREE_D, TWO_D_PLANAR, AXISYMMETRIC, and UNKNOWN_DIMENSION.

resultState

A SymbolicConstant specifying the state of the Instance as modified by the analysis. This member is only present if the Instance is part of the RootAssemblyState tree. Possible values are:

- PROPAGATED, specifying that the value is the same as the previous frame or the original rootAssembly.
- MODIFIED, specifying that the geometry of the instance has been changed at this frame.

The default value is PROPAGATED.

nodes

An OdbMeshNodeArray object.

elements

An OdbMeshElementArray object.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

sectionAssignments

A SectionAssignmentArray object.

rigidBodies

An OdbRigidBodyArray object.

beamOrientations

A BeamOrientationArray object.

materialOrientations

A MaterialOrientationArray object.

rebarOrientations

A RebarOrientationArray object.

analyticSurface

An AnalyticSurface object specifying analytic Surface defined on the instance.

34.17 OdbLoadCase object

The OdbLoadCase object describes a load case.

Access

```
import odbAccess
session.odbs[name].steps[name].frames[i].loadCase
session.odbs[name].steps[name].historyRegions[name].loadCase
session.odbs[name].steps[name].loadCases[name]
```

34.17.1 LoadCase(...)

This method creates an OdbLoadCase object.

Path

```
session.odbs[name].steps[name].LoadCase
```

Required argument

name

A String specifying the name of the OdbLoadCase object.

Optional arguments

None.

Return value

An OdbLoadCase object.

Exceptions

None.

34.17.2 Members

The OdbLoadCase object has members with the same names and descriptions as the arguments to the `LoadCase` method.

34.18 OdbMeshElement object

OdbMeshElement objects are created with the `part.addElements` or `rootAssembly.addElements` methods.

Access

```
import odbAccess
session.odbs[name].parts[name].elements[i]
session.odbs[name].parts[name].elementSets[name].elements[i]
session.odbs[name].parts[name].nodeSets[name].elements[i]
session.odbs[name].parts[name].surfaces[name].elements[i]
session.odbs[name].rootAssembly.elements[i]
session.odbs[name].rootAssembly.elementSets[name].elements[i]
session.odbs[name].rootAssembly.instances[name].elements[i]
session.odbs[name].rootAssembly.instances[name].elementSets[name] \
.elements[i]
session.odbs[name].rootAssembly.instances[name].nodeSets[name] \
.elements[i]
session.odbs[name].rootAssembly.instances[name].surfaces[name] \
.elements[i]
session.odbs[name].rootAssembly.nodeSets[name].elements[i]
```



```

session.odbs[name].rootAssembly-surfaces[name].elements[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elements[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elementSets[name].elements[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.nodeSets[name].elements[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance-surfaces[name].elements[i]

```

34.18.1 Members

The OdbMeshElement object can have the following members:

label

An Int specifying the element label.

type

A String specifying the element type.

sectionCategory

A SectionCategory object specifying the element section properties.

connectivity

A tuple of Ints specifying the element connectivity. For connector elements connected to ground, the other node is repeated in the connectivity data. The position of the ground node cannot be ascertained. This is a limitation. It is important to note the difference with MeshElement object of MDB where the connectivity is node indices instead of node labels.

instanceNames

A tuple of Strings specifying the instance names for nodes in the element connectivity.

instanceName

A String specifying the instance name.

34.19 OdbMeshNode object

OdbMeshNode objects are created with the `part.addNodes` method.

Access

```

import odbAccess
session.odbs[name].parts[name].elementSets[name].nodes[i]
session.odbs[name].parts[name].nodes[i]
session.odbs[name].parts[name].nodeSets[name].nodes[i]
session.odbs[name].parts[name].surfaces[name].nodes[i]

```

```

session.odbs[name].rootAssembly.elementSets[name].nodes[i]
session.odbs[name].rootAssembly.instances[name].elementSets[name] \
.nodes[i]
session.odbs[name].rootAssembly.instances[name].nodes[i]
session.odbs[name].rootAssembly.instances[name].nodeSets[name] \
.nodes[i]
session.odbs[name].rootAssembly.instances[name].surfaces[name] \
.nodes[i]
session.odbs[name].rootAssembly.nodes[i]
session.odbs[name].rootAssembly.nodeSets[name].nodes[i]
session.odbs[name].rootAssembly.surfaces[name].nodes[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elementSets[name].nodes[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.nodes[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.nodeSets[name].nodes[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.surfaces[name].nodes[i]

```

34.19.1 Members

The OdbMeshNode object has the following members:

label

An Int specifying the node label.

coordinates

A tuple of Floats specifying the nodal coordinates in the global Cartesian coordinate system.

34.20 OdbPart object

The OdbPart object is similar to the kernel Part object and contains nodes and elements, but not geometry.

Access

```

import odbAccess
session.odbs[name].parts[name]

```

34.20.1 Part(...)

This method creates an OdbPart object. Nodes and elements are added to this object at a later stage.

Path

`session.odbs[name].Part`

Required arguments

name

A String specifying the part name.

embeddedSpace

A SymbolicConstant specifying the dimensionality of the Part object. Possible values are THREE_D, TWO_D_PLANAR, and AXISYMMETRIC.

type

A SymbolicConstant specifying the type of the Part object. Possible values are DEFORMABLE_BODY and ANALYTIC_RIGID_SURFACE.

Optional arguments

None.

Return value

An OdbPart object.

Exceptions

None.

34.20.2 addElements(...)

This method adds elements to an OdbPart object using element labels and nodal connectivity.

WARNING: Adding elements not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Arguments

Required arguments

labels

A sequence of Ints specifying the element labels.

connectivity

A sequence of sequences of Ints specifying the nodal connectivity.

type

A String specifying the element type.

Optional arguments

elementSetName

A String specifying a name for this element set. The default value is the empty string.

sectionCategory

A SectionCategory object for this element set.

Return value

None

Exceptions

None.

34.20.3 addElements(...)

This method adds elements to an OdbPart object using a sequence of element labels and nodal connectivity.

WARNING: Adding elements not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Arguments

Required arguments

elementData

A sequence of sequences of Ints specifying the element labels and nodal connectivity, in the form ((*label*, *c1*, *c2*, *c3*, *c4*), (*label*, *c1*, *c2*, *c3*, *c4*), ...).

type

A String specifying the element type. The value can be user defined.

Optional arguments

elementSetName

A String specifying a name for this element set. The default value is None.

sectionCategory

A SectionCategory object for this element set.

Return value

None

Exceptions

None.

34.20.4 addNodes(...)

This method adds nodes to an OdbPart object using node labels and coordinates.

WARNING: *Adding nodes not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.*

Arguments

Required arguments

labels

A sequence of Ints specifying the node labels.

coordinates

A sequence of sequences of Floats specifying the nodal coordinates.

Optional argument

nodeSetName

A String specifying a name for this node set. The default value is None.

Return value

None

Exceptions

None.

34.20.5 addNodes(...)

This method adds nodes to an OdbPart object using a sequence of node labels and coordinates.

WARNING: *Adding nodes not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.*

Arguments

Required argument

nodeData

A sequence of tuples specifying the node labels and coordinates, in the form $((label, x, y, z), (label, x, y, z), \dots)$.

Optional argument

nodeSetName

A String specifying a name for this node set. The default value is None.

Return value

None

Exceptions

None.

34.20.6 assignBeamOrientation(...)

This method assigns a beam section orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

method

A SymbolicConstant specifying the assignment method. Only a value of N1_COSINES is currently supported.

vector

A sequence of three Floats specifying the approximate local $\mathbf{n}_1$ -direction of the beam cross-section.

Optional arguments

None.

Return value

None

Exceptions

None.

34.20.7 assignMaterialOrientation(...)

This method assigns a material orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCSys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

stackDirection

A SymbolicConstant specifying the stack or thickness direction of the material. Possible values are STACK_1, STACK_2, STACK_3, and STACK_ORIENTATION. The default value is STACK_3.

Return value

None

Exceptions

None.

34.20.8 assignRebarOrientation(...)

This method assigns a rebar reference orientation to a region of a part instance.

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

34.20.9 getElementFromLabel(...)

This method is used to retrieve an element with a specific label from a part object.

Arguments

Required argument

label

An Int specifying the element label.

Optional arguments

None.

Return value

An OdbMeshElement object.

Exceptions

If no element with the specified label exists:

OdbError: Invalid element label

34.20.10 getNodeFromLabel(...)

This method is used to retrieve a node with a specific label from a part object.

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

34.20.11 AnalyticRigidSurf2DPlanar(...)

This method is used to define a two-dimensional AnalyticSurface object on the part object.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments.
The default value is 0.0.

Return value

None

Exceptions

If OdbPart is of type THREE_D:

OdbError: 2D-Planar Analytic Rigid Surface can be defined only if the part is of type TWO_D_PLANAR or AXISYMMETRIC.

34.20.12 AnalyticRigidSurfExtrude(...)

This method is used to define a three-dimensional cylindrical AnalyticSurface on the part object.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If OdbPart is not of type THREE_D:

OdbError: Analytic Rigid Surface of type CYLINDER can be defined only if the part is of type THREE_D.

34.20.13 AnalyticRigidSurfRevolve(...)

This method is used to define a three-dimensional AnalyticSurface of revolution on the part object.

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If OdbPart is not of type THREE_D:

OdbError: Analytic Rigid Surface of type REVOLUTION can be defined only if the part is of type THREE_D.

34.20.14 RigidBody(...)

This method defines an OdbRigidBody on the part object.

Arguments

Required argument

referenceNode

An OdbSet specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are INPUT and CENTER_OF_MASS. The default value is INPUT.

isothermal

A Boolean specifying an isothermal rigid body. The default value is OFF. This parameter is used only for a fully-coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

34.20.15 Members

The OdbPart object has members with the same names and descriptions as the arguments to the **Part** method.

In addition, the OdbPart object can have the following members:

nodes

An OdbMeshNodeArray object.

elements

An OdbMeshElementArray object.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

sectionAssignments

A SectionAssignmentArray object.

beamOrientations

A BeamOrientationArray object.

materialOrientations

A MaterialOrientationArray object.

rebarOrientations

A RebarOrientationArray object.

rigidBodies

An OdbRigidBodyArray object.

analyticSurface

An AnalyticSurface object specifying analytic Surface defined on the instance.

34.21 OdbPretensionSection object

The pretension section object is used to define an assembly load. It associates a pretension node with a pretension section.

Access

```
import odbAccess
session.odbs[name].rootAssembly.pretensionSections[i]
```

34.21.1 Members

The OdbPretensionSection object can have the following members:

node

An OdbSet object specifying the node set containing the pretension node.

element

An OdbSet object specifying the element set that defines the pretension section.

surface

An OdbSet object specifying the surface set that defines the pretension section.

normal

A tuple of Floats specifying the components of the normal to the pretension section.

34.22 OdbRigidBody object

The Rigid body object is used to bind a set of elements and/or a set of nodes and/or an analytical surface with a reference node.

Access

```
import odbAccess
session.odbs[name].parts[name].rigidBodies[i]
session.odbs[name].rootAssembly.instances[name].rigidBodies[i]
session.odbs[name].rootAssembly.rigidBodies[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
    .instance.rigidBodies[i]
```

34.22.1 RigidBody(...)

This method creates a OdbRigidBody object.

Path

```
session.odbs[name].rootAssembly.instances[name].RigidBody
session.odbs[name].rootAssembly.RigidBody
```

Required argument

referenceNode

An OdbSet object specifying the reference node set associated with the rigid body.

Optional arguments

position

A SymbolicConstant specifying the specific location of the OdbRigidBody reference node relative to the rest of the rigid body. Possible values are INPUT and CENTER_OF_MASS. The default value is INPUT.

isothermal

A Boolean specifying whether the OdbRigidBody can have temperature gradients or be isothermal. This is used only for fully coupled thermal-stress analysis. The default value is ON.

elements

An OdbSet object specifying the element set whose motion is governed by the motion of rigid body reference node.

tieNodes

An OdbSet object specifying the node set which have both translational and rotational degrees of freedom associated with the rigid body.

pinNodes

An OdbSet object specifying the node set which have only translational degrees of freedom associated with the rigid body.

analyticSurface

An AnalyticSurface object specifying the analytic surface whose motion is governed by the motion of rigid body reference node.

Return value

An OdbRigidBody object.

Exceptions

None.

34.22.2 Members

The OdbRigidBody object has members with the same names and descriptions as the arguments to the **RigidBody** method.

34.23 OdbSequenceAnalyticSurfaceSegment object

A sequence of AnalyticSurfaceSegment describing an analytic surface profile.

Access

```
import odbAccess
session.odbs[name].parts[name].analyticSurface.segments
session.odbs[name].rootAssembly.instances[name].analyticSurface\
.segments
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.analyticSurface.segments
```

34.23.1 AnalyticSurfaceProfile()

This method creates a OdbSequenceAnalyticSurfaceSegment object.

Path

`odbAccess.AnalyticSurfaceProfile`

Arguments

None.

Return value

An OdbSequenceAnalyticSurfaceSegment object.

Exceptions

None.

34.23.2 Start(...)

This method adds a AnalyticSurfaceSegment describing the first segment of the surface profile.

Arguments

Required argument

origin

A sequence of Floats specifying the coordinates of start point.

Optional arguments

None.

Return value

None

Exceptions

None.

34.23.3 Line(...)

This method adds a `AnalyticSurfaceSegment` describing the line segment of the surface profile.

Arguments

Required argument

endPoint

A sequence of Floats specifying the coordinates of end point.

Optional arguments

None.

Return value

None

Exceptions

None.

34.23.4 Circle(...)

This method adds a `AnalyticSurfaceSegment` describing a circular segment of the surface profile.

Arguments

Required arguments

center

A sequence of Floats specifying the coordinates of center of the circular segment.

endPoint

A sequence of Floats specifying the coordinates of end point of the circular segment.

Optional arguments

None.

Return value

None

Exceptions

None.

34.23.5 Parabola(...)

This method adds a `AnalyticSurfaceSegment` describing a parabolic segment of the surface profile.

Arguments

Required arguments

middlePoint

A sequence of Floats specifying the coordinates of middle point of the parabolic segment.

endPoint

A sequence of Floats specifying the coordinates of end point of the parabolic segment.

Optional arguments

None.

Return value

None

Exceptions

None.

34.23.6 Members

The `OdbSequenceAnalyticSurfaceSegment` object has no members.

34.24 OdbSet object

The set objects are used to identify regions of a model.

Access

```
import odbAccess
session.odbs[name].parts[name].elementSets[name]
session.odbs[name].parts[name].nodeSets[name]
```

```

session.odbs[name].parts[name].surfaces[name]
session.odbs[name].rootAssembly.elementSets[name]
session.odbs[name].rootAssembly.instances[name].elementSets[name]
session.odbs[name].rootAssembly.instances[name].nodeSets[name]
session.odbs[name].rootAssembly.instances[name].surfaces[name]
session.odbs[name].rootAssembly.nodeSets[name]
session.odbs[name].rootAssembly.surfaces[name]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elementSets[name]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.nodeSets[name]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.surfaces[name]

```

34.24.1 NodeSet(...)

This method creates a node set from an array of OdbMeshNode objects (for part instance-level sets) or from a sequence of arrays of OdbMeshNode objects (for assembly-level sets).

Path

```

session.odbs[name].parts[name].NodeSet
session.odbs[name].rootAssembly.instances[name].NodeSet
session.odbs[name].rootAssembly.NodeSet

```

Required arguments

name

A String specifying the name of the set and the repository key.

nodes

A sequence of OdbMeshNode objects. For example, for a part:

```
nodes=part1.nodes[1:5]
```

For an assembly:

```
nodes=(instance1.nodes[6:7], instance2.nodes[1:5])
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.2 NodeSetFromNodeLabels(...)

This method creates a node set from a sequence of node labels.

Path

```
session.odbs[name].parts[name].NodeSetFromNodeLabels
session.odbs[name].rootAssembly.instances[name].NodeSetFromNodeLabels
session.odbs[name].rootAssembly.NodeSetFromNodeLabels
```

Required arguments

name

A String specifying the name of the set and the repository key.

nodeLabels

A sequence of node labels. A node label is a sequence of Int node identifiers. For example, for a part:

```
nodeLabels=(2,3,5,7)
```

For an assembly:

```
nodeLabels=(('Instance-1', (2,3,5,7)), ('Instance-2', (1,2,3)))
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.3 ElementSet(...)

This method creates an element set from an array of OdbMeshElement objects (for part instance-level sets) or from a sequence of arrays of OdbMeshElement objects (for assembly-level sets).

Path

```
session.odbs[name].parts[name].ElementSet  
session.odbs[name].rootAssembly.instances[name].ElementSet
```

Required arguments

name

A String specifying the name of the set and the repository key.

elements

A sequence of OdbMeshElement objects. For example, for a part:

```
elements=instance1.elements[1:5]
```

For an assembly:

```
elements=(instance1.elements[1:5], instance2.elements[1:5])
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.4 ElementSetFromElementLabels(...)

This method creates an element set from a sequence of element labels.

Path

```
session.odbs[name].parts[name].ElementSetFromElementLabels  
session.odbs[name].rootAssembly.instances[name].ElementSetFromElementLabels  
session.odbs[name].rootAssembly.ElementSetFromElementLabels
```

Required arguments

name

A String specifying the name of the set and the repository key.

elementLabels

A sequence of element labels. An element label is a sequence of Int element identifiers. For example, for a part:

```
elementLabels=(2,3,5,7)
```

For an assembly:

```
elementLabels=(('Instance-1', (2,3,5,7)), ('Instance-2', (1,2,3)))
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.5 MeshSurface(...)

This method creates a surface from the element and side identifiers for the assembly.

Path

```
session.odbs[name].parts[name].MeshSurface
session.odbs[name].rootAssembly.instances[name].MeshSurface
session.odbs[name].rootAssembly.MeshSurface
```

Required arguments

name

A String specifying the name of the set and the repository key.

meshSurfaces

A sequence of sequences. Each sequence consists of an element sequence and a side identifier. The possible side identifiers depend on the type of element, as described in the following table:

Sequence of elements	Side identifiers
Solid elements	FACE1, FACE2, FACE3, FACE4, FACE5, FACE6
Three-dimensional shell elements	SIDE1, SIDE2

Two-dimensional elements	FACE1, FACE2, FACE3, FACE4
Wire elements	END, END2

For example:

```
side1Elements=instance1.elements[217:218]
side2Elements=instance2.elements[100:105]
assembly.MeshSurface(name='Surf-1',
meshSurfaces=((side1Elems,SIDE1),
(side2Elems,SIDE2)))
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.6 MeshSurfaceFromElsets(...)

This method creates a mesh surface from a sequence of element sets.

Path

```
session.odbs[name].parts[name].MeshSurfaceFromElsets
session.odbs[name].rootAssembly.instances[name].MeshSurfaceFromElsets
session.odbs[name].rootAssembly.MeshSurfaceFromElsets
```

Required arguments

name

A String specifying the name of the set and the repository key.

elementSetSeq

A sequence of element sets. For example,

```
elementSetSeq=((elset1,SIDE1),(elset2,SIDE2))
```

where `elset1=session.odbs[name].rootAssembly.elementSets['Clutch']`
and `SIDE1` and `SIDE2` indicate the side of the element set.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.7 MeshSurfaceFromLabels(...)

This method creates a mesh surface from a sequence of surface labels.

Path

```
session.odbs[name].parts[name].MeshSurfaceFromLabels
session.odbs[name].rootAssembly.instances[name].MeshSurfaceFromLabels
session.odbs[name].rootAssembly.MeshSurfaceFromLabels
```

Required arguments

name

A String specifying the name of the set and the repository key.

surfaceLabels

A sequence of surface labels. For example,

```
surfaceLabels= (('Instance-1', ((10, FACE1), (11, FACE2))),
                ('Instance-2', ((10, FACE3), (12, FACE4))))
```

where **10** is an element number and **FACE1** indicates the side of the element.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

34.24.8 Members

The OdbSet object can have the following members:

name

A String specifying the name of the set and the repository key.

instanceNames

A tuple of Strings specifying the namespaces for the nodes, elements, and faces; **None** if the set is on a Part or an OdbInstance object.

nodes

An OdbMeshNodeArray object specifying the nodes of an OdbSet. If a set spans more than one part instance, this member is a sequence of sequences for each part instance.

elements

An OdbMeshElementArray object specifying the elements of an OdbSet. If a set spans more than one part instance, this member is a sequence of sequences for each part instance.

faces

A tuple of SymbolicConstants specifying the element face. If a set spans more than one part instance, this member is a sequence of sequences for each part instance.

34.25 OdbStep object

An output database contains the same steps of the model database that originated it.

Access

```
import odbAccess
session.odbs[name].steps[name]
```

34.25.1 Step(...)

This method creates an OdbStep object.

Path

```
session.odbs[name].Step
```

Required arguments

name

A String specifying the repository key.

description

A String specifying the step description.

domain

A SymbolicConstant specifying the domain of the step. Possible values are TIME, FREQUENCY, ARC_LENGTH, and MODAL.

The type of OdbFrame object that can be created for this step is based on the value of the *domain* argument.

Optional arguments

timePeriod

A Float specifying the time period of the step. *timePeriod* is required if *domain*=TIME; otherwise, this argument is not applicable. The default value is 0.0.

previousStepName

A String specifying the preceding step. If *previousStepName* is the empty string, the last step in the repository is used. If *previousStepName* is not the last step, this will result in a change to the *previousStepName* member of the step that was in that position. A special value 'Initial' refers to the internal initial model step and may be used exclusively for inserting a new step at the first position before any other existing steps. The default value is an empty string.

procedure

A String specifying the step procedure. The default value is an empty string.

totalTime

A Float specifying the analysis time spend in all the steps previous to this step. The default value is -1.0.

Return value

An OdbStep object.

Exceptions

If *previousStepName* is invalid:

ValueError: previousStepName is invalid

34.25.2 getFrame(...)

This method retrieves an OdbFrame object associated with a given frame value.

Arguments

Required argument

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the step time or frequency.

Optional argument

match

A SymbolicConstant specifying which frame to return if there is no frame at the exact frame value. Possible values are CLOSEST, BEFORE, AFTER, and EXACT. The default value is CLOSEST.

When *match*=CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:

OdbError: Frame not found.

34.25.3 getFrame(...)

This method retrieves an OdbFrame object associated with a given load case.

Arguments

Required argument

loadCase

An OdbLoadCase object specifying a load case in the step.

Optional arguments

None.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:

OdbError: Frame not found.

34.25.4 getFrame(...)

This method retrieves an OdbFrame object associated with a given load case and frame value.

Arguments

Required arguments

loadCase

An OdbLoadCase object specifying a load case in the step.

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the step time or frequency.

Optional argument

match

A SymbolicConstant specifying which frame to return if there is no frame at the exact frame value. Possible values are CLOSEST, BEFORE, AFTER, and EXACT. The default value is CLOSEST.

When *match*=CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:

OdbError: Frame not found.

34.25.5 getHistoryRegion(...)

This method retrieves a HistoryRegion object associated with a HistoryPoint in the model.

Arguments

Required argument

point

A HistoryPoint object specifying the point in the model.

Optional argument

loadCase

An OdbLoadCase object specifying a load case in the step.

Return value

A HistoryRegion object.

Exceptions

If a HistoryRegion object is not found:

OdbError: HistoryRegion not found.

34.25.6 **setDefaultDeformedField(...)**

This method sets the default deformed field variable in a step.

Arguments

Required argument

field

A FieldOutput object specifying the default deformed field variable for visualization.

Optional arguments

None.

Return value

None

Exceptions

None.

34.25.7 **setDefaultField(...)**

This method sets the default field variable in a step.

Arguments

Required argument

field

A FieldOutput object specifying the default field variable for visualization.

Optional arguments

None.

Return value

None

Exceptions

None.

34.25.8 **Members**

The OdbStep object has members with the same names and descriptions as the arguments to the **Step** method.

In addition, the OdbStep object can have the following members:

number

An Int specifying the step number.

nlgeom

A Boolean specifying whether geometric nonlinearity can occur in this step.

mass

A Float specifying the current value of the mass of the model. This does not include the mass of the acoustic media if any present.

acousticMass

A Float specifying the current value of the mass of the acoustic media of the model.

frames

An OdbFrameArray object.

historyRegions

A repository of HistoryRegion objects.

loadCases

A repository of OdbLoadCase objects.

massCenter

A tuple of Floats specifying the coordinates of the center of mass.

inertiaAboutCenter

A tuple of Floats specifying the moments and products of inertia about the center of mass. For 3-D models inertia quantities are written in the following order: I(XX), I(YY), I(ZZ), I(XY), I(XZ), and I(YZ). For 2-D models only I(ZZ) and I(XY) are outputted.

inertiaAboutOrigin

A tuple of Floats specifying the moments and products of inertia about the origin of the global coordinate system. For 3-D models inertia quantities are written in the following order: I(XX), I(YY), I(ZZ), I(XY), I(XZ), and I(YZ). For 2-D models only I(ZZ) and I(XY) are outputted.

acousticMassCenter

A tuple of Floats specifying the coordinates of the center of mass of the acoustic media.

34.26 RebarOrientation object

The RebarOrientation object represents the orientation of the rebar reference.

Access

```
import odbAccess
```

```
session.odbs[name].parts[name].rebarOrientations[i]
```

```
session.odbs[name].rootAssembly.instances[name].rebarOrientations[i]
```

```
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
```

```
.instance.rebarOrientations[i]
```

34.26.1 Members

The RebarOrientation object can have the following members:

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. Possible values are AXIS_1, AXIS_2, and AXIS_3.

angle

A Float specifying the angle of the additional rotation.

region

An OdbSet object specifying a region for which the rebar orientation is defined.

csys

An OdbDatumCsys object specifying a datum coordinates system.

34.27 ScratchOdb object

A scratch output database is associated with an open output database and is used to store session-related, non-persistent objects, such as Step, Frame and FieldOutput objects. Abaqus creates a scratch output database when needed for these non-persistent objects during an Abaqus/CAE session. Abaqus deletes the scratch output database when the associated output database is closed.

Access

```
import odbAccess
session.scratchOdb[sname]
```

34.27.1 ScratchOdb(...)

This method creates a new ScratchOdb object.

Path

```
session.ScratchOdb
```

Required argument

odb

An Odb object specifying the output database with which to associate.

Optional arguments

None.

Return value

A ScratchOdb object.

Exceptions

None.

34.27.2 Members

The ScratchOdb object has no members.

34.28 SectionCategory object

The SectionCategory object is used to group regions of the model with like sections. Section definitions that contain the same number of section points or integration points are grouped together.

To access data for a particular section definition, use the individual Section objects in the output database. For more information, see Chapter 8, “Beam Section profile commands,” and Chapter 46, “Section commands.”

Access

```
import odbAccess
session.odbs[name].parts[name].elements[i].sectionCategory
session.odbs[name].parts[name].elementSets[name].elements[i] \
.sectionCategory
session.odbs[name].parts[name].nodeSets[name].elements[i] \
.sectionCategory
session.odbs[name].parts[name].surfaces[name].elements[i] \
.sectionCategory
session.odbs[name].rootAssembly.elements[i].sectionCategory
session.odbs[name].rootAssembly.elementSets[name].elements[i] \
.sectionCategory
session.odbs[name].rootAssembly.instances[name].elements[i] \
.sectionCategory
session.odbs[name].rootAssembly.instances[name].elementSets[name] \
.elements[i].sectionCategory
session.odbs[name].rootAssembly.instances[name].nodeSets[name] \
.elements[i].sectionCategory
session.odbs[name].rootAssembly.instances[name].surfaces[name] \
.elements[i].sectionCategory
```

```

session.odbs[name].rootAssembly.nodeSets[name].elements[i] \
.sectionCategory
session.odbs[name].rootAssembly-surfaces[name].elements[i] \
.sectionCategory
session.odbs[name].sectionCategories[name]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elements[i].sectionCategory
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elementSets[name].elements[i].sectionCategory
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.nodeSets[name].elements[i].sectionCategory
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance-surfaces[name].elements[i].sectionCategory

```

34.28.1 SectionCategory(...)

This method creates a SectionCategory object.

Path

```
session.odbs[name].SectionCategory
```

Required arguments

name

A String specifying the name of the category.

description

A String specifying the description of the category.

Optional arguments

None.

Return value

A SectionCategory object.

Exceptions

None.

34.28.2 Members

The SectionCategory object has members with the same names and descriptions as the arguments to the **SectionCategory** method.

In addition, the SectionCategory object can have the following member:

sectionPoints

A SectionPointArray object.

34.29 SectionPoint object

The SectionPoint object describes the location of a section point within a section category.

Access

```
import odbAccess
session.odbs[name].parts[name].elements[i].sectionCategory\
.sectionPoints[i]
session.odbs[name].parts[name].elementSets[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].parts[name].nodeSets[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].parts[name].surfaces[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.elements[i].sectionCategory\
.sectionPoints[i]
session.odbs[name].rootAssembly.elementSets[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.instances[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.instances[name].elementSets[name] \
.elements[i].sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.instances[name].nodeSets[name] \
.elements[i].sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.instances[name].surfaces[name] \
.elements[i].sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.nodeSets[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].rootAssembly.surfaces[name].elements[i] \
.sectionCategory.sectionPoints[i]
session.odbs[name].sectionCategories[name].sectionPoints[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name] \
.locations[i].sectionPoints[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
.instance.elements[i].sectionCategory.sectionPoints[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
```

```
.instance.elementSets[name].elements[i].sectionCategory\  
.sectionPoints[i]  
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \  
.instance.nodeSets[name].elements[i].sectionCategory.sectionPoints[i]  
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \  
.instance-surfaces[name].elements[i].sectionCategory.sectionPoints[i]  
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \  
.sectionPoint
```

34.29.1 SectionPoint(...)

This method creates a `SectionPoint` object.

Path

```
session.odbs[name].sectionCategories[name].SectionPoint
```

Required arguments

number

An Int specifying the number of the section point. See “Beam elements,” Section 29.3 of the Abaqus Analysis User’s Guide, and “Shell elements,” Section 29.6 of the Abaqus Analysis User’s Guide, for the numbering convention.

description

A String specifying the description of the section point.

Optional arguments

None.

Return value

A `SectionPoint` object.

Exceptions

None.

34.29.2 Members

The `SectionPoint` object has members with the same names and descriptions as the arguments to the `SectionPoint` method.

34.30 SectorDefinition object

The SectorDefinition object describes the number of symmetry sectors and axis of symmetry for a cyclic symmetry model.

Access

```
import odbAccess
session.odbs[name].sectorDefinition
```

34.30.1 Members

The SectorDefinition object has the following members:

numSectors

An Int specifying the number of sectors in the cyclic symmetry model.

symmetryAxis

A tuple of tuples of Floats specifying the coordinates of two points on the axis of symmetry.

34.31 UserData object

The UserData object contains user-defined XY data. The UserData object has no constructor; it is created automatically when an Odb object is created.

Access

```
import odbAccess
session.odbs[name].userData
```

34.31.1 XYData(...)

This method creates an XYData object from a sequence of X–Y data pairs.

Path

```
session.odbs[name].userData.XYData
```

Required arguments

name

A String specifying the repository key.

data

A sequence of pairs of Floats specifying the X–Y data pairs.

Optional arguments

sourceDescription

A String specifying the source of the X–Y data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

A String specifying the content of the X–Y data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

A String specifying additional information about the X–Y data (e.g., “for whole model”). The default value is an empty string.

legendLabel

A String specifying the label to be used in the legend. The default value is the name of the XYData object.

xValuesLabel

A String specifying the label for the X-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

yValuesLabel

A String specifying the label for the Y-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

axis1QuantityType

A QuantityType object specifying the QuantityType object associated to the X -axis1- values.

axis2QuantityType

A QuantityType object specifying the QuantityType object associated to the Y -axis2- values.

Return value

An XYData object.

Exceptions

InvalidNameError.

34.31.2 Members

The UserData object can have the following members:

name

A String specifying the repository key.

sourceDescription

A String specifying the source of the *X–Y* data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

A String specifying the content of the *X–Y* data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

A String specifying additional information about the *X–Y* data (e.g., “for whole model”). The default value is an empty string.

xValuesLabel

A String specifying the label for the *X*-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

yValuesLabel

A String specifying the label for the *Y*-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

axis1QuantityType

A *QuantityType* object specifying the *QuantityType* object associated to the *X -axis1-* values.

axis2QuantityType

A *QuantityType* object specifying the *QuantityType* object associated to the *Y -axis2-* values.

legendLabel

A String specifying the label to be used in the legend. The default value is the name of the *XYData* object.

xyDataObjects

A repository of *XYData* objects.

annotations

A repository of *Annotation* objects.

data

A tuple of pairs of Floats specifying the *X–Y* data pairs.

34.32 Odb commands

The Odb commands do the following:

- Determine if an output database (*.odb*) file needs to be upgraded to the current release.
- Determine the extreme value for an output variable over a number of fields; for example, over a number of load cases.
- Open an existing output database file and create a new Odb object.
- Upgrade an output database file to the current release and write the upgraded output database to a new file.

34.32.1 isUpgradeRequiredForOdb(...)

This method determines if an output database file needs to be upgraded to the current release.

You can access this method using either of the following techniques:

- From a script running outside Abaqus/CAE. For example,

```
import odbAccess
needsUpgrade = odbAccess.isUpgradeRequiredForOdb(
    upgradeRequiredOdbPath='myOdb.odb')
```

- From the Visualization module in Abaqus/CAE. For example,

```
import visualization
needsUpgrade = session.isUpgradeRequiredForOdb(
    upgradeRequiredOdbPath='myOdb.odb')
```

Arguments

Required argument

upgradeRequiredOdbPath

An String specifying the path to an output database file to test. The test determines if the output database needs to be upgraded to the current release.

Optional arguments

None.

Return value

A Boolean indicating the result of the test. A value of True indicates that the output database needs to be upgraded to the current release.

Exceptions

None.

34.32.2 maxEnvelope(...)

Retrieve the maximum value of an output variable over a number of fields.

Arguments

Required arguments

The **maxEnvelope** method does not use keyword arguments.

Argument

A list of similar fieldOutput objects from which the maximum value will be computed.

A SymbolicConstant specifying the invariant or component label to be used when comparing vectors or tensors. Possible values are:

- MAGNITUDE
- MISES
- TRESCA
- PRESS
- INV3
- MAX_PRINCIPAL
- MID_PRINCIPAL
- MIN_PRINCIPAL

You must provide either this argument or the following argument if the field is a vector or tensor.

A String specifying the component of the tensor or vector to be used for selecting the maximum value.

Optional arguments

None.

Return value

A sequence of two fieldOutput objects. The first fieldOutput object contains the maximum value. The second fieldOutput object contains the index of the field containing the maximum value. The index follows the order in which fields are positioned in the list of fieldOutput objects provided as the argument to the function.

Exceptions

OdbError

TypeError

This function takes no keyword arguments.

34.32.3 minEnvelope(...)

Retrieve the minimum value of an output variable over a number of fields.

Arguments

Required arguments

The **minEnvelope** method does not use keyword arguments.

Argument

A list of similar fieldOutput objects from which the maximum value will be computed.

A SymbolicConstant specifying the invariant or component label to be used when comparing vectors or tensors. Possible values are:

- MAGNITUDE
- MISES
- TRESCA
- PRESS
- INV3
- MAX_PRINCIPAL
- MID_PRINCIPAL
- MIN_PRINCIPAL

You must provide either this argument or the following argument if the field is a vector or tensor.

A String specifying the component of the tensor or vector to be used for selecting the minimum value.

Optional arguments

None.

Return value

A sequence of two fieldOutput objects. The first fieldOutput object contains the minimum value. The second fieldOutput object contains the index of the field containing the minimum value. The index follows the order in which fields are positioned in the list of fieldOutput objects provided as the argument to the function.

Exceptions

OdbError

TypeError

This function takes no keyword arguments.

34.32.4 openOdb(...)

This method opens an existing output database (`.odb`) file and creates a new Odb object. You typically execute this method outside of Abaqus/CAE when, in most cases, only one output database is open at any time. For example,

```
import odbAccess
shockLoadOdb = odbAccess.openOdb(path='myOdb.odb')
```


Arguments

Required argument

path

A String specifying the path to an existing output database (**.odb**) file.

Optional argument

readOnly

A Boolean specifying whether the file will permit only read access or both read and write access. The initial value is False, indicating that both read and write access will be permitted.

readInternalSets

A Boolean specifying whether the file will permit access to sets specified as Internal on the database. The initial value is False, indicating that internal sets will not be read.

Return value

An Odb object.

Exceptions

If the output database was generated by a previous release of Abaqus and needs upgrading:

OdbError: The database is from a previous release of Abaqus. Run **abaqus upgrade -job <newFilename> -odb <oldFileName>** to upgrade it.

If the output database was generated by a newer release of Abaqus, and the installation of Abaqus needs upgrading:

OdbError: Abaqus installation must be upgraded before this output database can be opened.

34.32.5 openOdb(...)

This method opens an existing output database (**.odb**) file and creates a new Odb object. This method is accessed only via the session object inside Abaqus/CAE and adds the new Odb object to the **session.odbs** repository. This method allows you to open multiple output databases at the same time and to use the repository key to specify a particular output database. For example,

```
import visualization
session.openOdb(name='myOdb', path='stress.odb', readOnly=True)
```

Arguments

Required argument

name

A String specifying the repository key. If the *name* is not the same as the *path* to the output database (**.odb**) file, the *path* must be specified as well. Additionally, to support backwards compatibility

of the interface, if the *name* parameter is omitted, the *path* and *name* will be presumed to be the same.

Optional arguments

path

A String specifying the path to an existing output database (**.odb**) file.

readOnly

A Boolean specifying whether the file will permit only read access or both read and write access. The initial value is TRUE when the output database file is opened from Abaqus/CAE, indicating that only read access will be permitted.

Return value

An Odb object.

Exceptions

If the output database was generated by a previous release of Abaqus and needs upgrading:

OdbError: The database is from a previous release of Abaqus.

Run **abaqus upgrade -job <newFileName> -odb <oldFileName>** to upgrade it.

If the output database was generated by a newer release of Abaqus, and the installation of Abaqus needs upgrading:

OdbError: Abaqus installation must be upgraded before this output database can be opened.

If the file is not a valid database:

AbaqusError: Cannot open file <filename>.

34.32.6 upgradeOdb(...)

This method upgrades an existing Odb object to the current release and writes the upgraded version of the Odb object to a file. In addition, Abaqus/CAE writes information about the status of the upgrade to a log (***.log**) file.

You can access this method using either of the following techniques:

- From a script running outside Abaqus/CAE. For example,

```
import odbAccess
odbAccess.upgradeOdb(existingOdbPath='oldOdb',
                     upgradedOdbPath='upgradedOdb')
```

- From the session object in Abaqus/CAE. For example,

```
import visualization
session.upgradeOdb(existingOdbPath='oldOdb',
                  upgradedOdbPath='upgradedOdb')
```

Arguments

Required arguments

existingOdbPath

An String specifying the path to the file containing the output database to be upgraded.

upgradedOdbPath

An String specifying the path to the file that will contain the upgraded output database.

Optional arguments

None.

Return value

None

Exceptions

If the output database upgrade fails:

OdbError: cannot convert database

35. Odb Display commands

35.1 OdbDisplay object

The OdbDisplay object stores the context of an output database for a viewport. The OdbDisplay object has no constructor. Abaqus creates the *defaultOdbDisplay* member when you import the Visualization module. Abaqus creates the *odbDisplay* member when a viewport is created, using the attributes from the previous active viewport. The previous active viewport contains the attributes from the *defaultOdbDisplay* object for the session. The attributes from the *defaultOdbDisplay* object are copied from the previous active viewport to create the new viewport.

OdbDisplay objects are accessed in one of two ways:

- The default output database display options. These settings are used as defaults when other *odbDisplay* members are created and can be set to customize user preferences.
- The output database display options associated with a particular viewport.

Access

```
session.viewports[name].layers[name].odbDisplay
import visualization
session.viewports[name].odbDisplay
```

35.1.1 moveCameraToCsys()

This method specifies a new position for the camera. It is available only when *movieMode*=ON (in the View object). The new camera position is the origin of the coordinate system specified by the *cameraCsysName* member of the BasicOptions object.

Arguments

None.

Return value

None

Exceptions

None.

35.1.2 setDeformedVariable(...)

This method specifies the field output variable or FieldOutput object to be used when displaying the deformed shape of the model.

Arguments

Required arguments

One of the following two mutually exclusive arguments must be provided:

variableLabel

A String specifying the field output variable.

field

A String specifying the FieldOutput object.

Optional arguments

None.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.3 setFrame(...)

This method specifies the step and frame for the OdbDisplay object.

Arguments

Required arguments

step

An Int specifying the step index. Possible values are $0 \leq \text{step} \leq (\text{numSteps} - 1)$.

frame

An Int specifying the frame in the specified step. Valid values are $0 \leq \text{frame} \leq (\text{numFramesInStep} - 1)$. If $\text{frame} \geq (\text{numFramesInStep} - 1)$ the last frame will be displayed.

Optional arguments

None.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

If the Odb object does not contain valid step data:

There are no valid step data on the odb. Requested operation cancelled.

If an invalid step index is passed in as argument:

Invalid step index:step. Available step indices: 0 - n

35.1.4 setFrame(...)

This method specifies the frame for the OdbDisplay object.

Arguments

Required argument

frame

An OdbFrame object.

Optional arguments

None.

Return value

None

Exceptions

None.

35.1.5 setPrimaryVariable(...)

This method specifies the field output variable for which to obtain results.

Arguments

Required arguments

variableLabel

A String specifying the field output variable. The *variableLabel* and *field* arguments are mutually exclusive.

field

A String specifying the FieldOutput object. The *variableLabel* and *field* arguments are mutually exclusive.

outputPosition

A SymbolicConstant specifying the position from which to obtain data. Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, ELEMENT_CENTROID, WHOLE_ELEMENT, WHOLE_REGION, WHOLE_PART_INSTANCE, WHOLE_MODEL, and GENERAL_PARTICLE. Data can be obtained only from the position at which they were written to the output database during the analysis.

Optional arguments

refinement

A sequence of a SymbolicConstant and a String. Possible values for the SymbolicConstant are INVARIANT and COMPONENT. The String specifies an available component or invariant for the specified *variableLabel*. This argument is required only if a refinement is available for the specified *variableLabel*.

sectionPoint

A Dictionary with String keys and String values. Each key specifies a region in the model; the corresponding value specifies a section point within that region. For example:

```
sectionPoint={'shell < MAT > < 7 section points >':'SPOS,
              (fraction = 1.0)', 'shell < MAT > < 5 section points >':'
              'SPOS, (fraction = 1.0)', }
```

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.6 setPrimarySectionPoint(...)

This method specifies the section point for the current primary, symbol and status variables.

Arguments

Required arguments

sectionPoint

A Dictionary with String keys and String values. Each key specifies a region in the model; the corresponding value specifies a section point within that region. For example:


```
sectionPoint={'shell < MAT > < 7 section points >':'SPOS,
(fraction = 1.0)', 'shell < MAT > < 5 section points >':
'SPOS, (fraction = 1.0)', }
```

activePly

A String specifying the active ply name.

Optional arguments

None.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.7 setStatusVariable(...)

This method specifies the field output variable for filtering element display based on a status criteria.

Arguments

Required arguments

variableLabel

A String specifying the field output variable. The *variableLabel* and *field* arguments are mutually exclusive.

field

A String specifying the FieldOutput object. The *variableLabel* and *field* arguments are mutually exclusive.

outputPosition

A SymbolicConstant specifying the position from which to obtain data. Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, ELEMENT_CENTROID, WHOLE_ELEMENT, WHOLE_REGION, WHOLE_PART_INSTANCE, WHOLE_MODEL, and GENERAL_PARTICLE. Data can be obtained only from the position at which they were written to the output database during the analysis.

Optional arguments

refinement

A sequence of a SymbolicConstant and a String. Possible values for the SymbolicConstant are INVARIANT and COMPONENT. The String specifies an available component or invariant for the specified *variableLabel*. This argument is required only if a refinement is available for the specified *variableLabel*.

sectionPoint

A Dictionary with String keys and String values. Each key specifies a region in the model; the corresponding value specifies a section point within that region. For example:

```
sectionPoint={'shell < MAT > < 7 section points >':'SPOS,
              (fraction = 1.0)', 'shell < MAT > < 5 section points >':
              'SPOS, (fraction = 1.0)', }
```

statusMinimum

A Float specifying the minimum result value to be considered for element removal.

statusMaximum

A Float specifying the maximum result value to be considered for element removal.

statusInsideRange

A Boolean utilized when both *statusMinimum* and *statusMaximum* are given. Elements will be removed when they contain values between the minimum and maximum, inclusive, when true. Elements will be removed when they contain values outside of the minimum and maximum, exclusive, when false.

useStatus

A Boolean to specify if the status variable is to be active.

applyStatusToUndeformed

A Boolean to specify if the active status variable is to remove elements from undeformed plots. The default value is False.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.8 setSymbolVariable(...)

This method specifies the field output variable for which to obtain results used for symbol plots. This variable must be in the form of vector or tensor data. The output quantity can also be specified with this command to control the display of resultants, or components.

Arguments

Required arguments

variableLabel

A String specifying the field output variable. The *variableLabel* and *field* arguments are mutually exclusive.

field

A String specifying the FieldOutput object. The *variableLabel* and *field* arguments are mutually exclusive.

outputPosition

A SymbolicConstant specifying the position from which to obtain data. Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, ELEMENT_CENTROID, WHOLE_ELEMENT, WHOLE_REGION, WHOLE_PART_INSTANCE, WHOLE_MODEL, and GENERAL_PARTICLE. Data can be obtained only from the position at which they were written to the output database during the analysis.

Optional arguments

refinement

A sequence of a SymbolicConstant and a String. Possible values for the SymbolicConstant are INVARIANT and COMPONENT. The String specifies an available component or invariant for the specified *variableLabel*. This argument is required only if a refinement is available for the specified *variableLabel*.

sectionPoint

A Dictionary with String keys and String values. Each key specifies a region in the model; the corresponding value specifies a section point within that region. For example:

```
sectionPoint={'shell < MAT > < 7 section points >':'SPOS,
              (fraction = 1.0)', 'shell < MAT > < 5 section points >':'
              'SPOS, (fraction = 1.0)', }
```

tensorQuantity

A SymbolicConstant specifying the tensor quantity to be displayed with the symbol. This value is set in the SymbolOptions object.

vectorQuantity

A SymbolicConstant specifying the vector quantity to be displayed with the symbol. This value is set in the SymbolOptions object.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.9 setStreamVariable(...)

This method specifies the field output variable for which to obtain results used for stream plots. This variable must be in the form of nodal vector data.

Arguments

Required argument

variableLabel

A String specifying the field output variable.

Optional arguments

None.

Return value

None

Exceptions

If the viewport is not associated with any Odb object:

The current viewport is not associated with an ODB file. Requested operation cancelled.

35.1.10 setValues(...)

This method specifies member values for the OdbDisplay object.

Arguments

Required arguments

None.

Optional arguments

visibleDisplayGroups

A List of DisplayGroup objects.

viewCut

A Boolean specifying whether to display the cut. The default value is OFF.

viewCutNames

A List of ViewCut objects.

Return value

None

Exceptions

None.

35.1.11 Members

The OdbDisplay object can have the following members:

name

A String specifying the name of the output database associated with the OdbDisplay object.

fieldSteps

A tuple of Strings specifying field steps.

Each item in the sequence consists of a tuple that contains the following step information:

- *element0*: A String specifying the step name.
- *element1*: A String specifying the step description.
- *element2*: A Float specifying the time value at the start of the step.
- *element3*: A Float specifying the time period of the step.
- *element4*: A Float specifying the user modified time period of the step.
- *element5*: An Int specifying the domain type of the step. Possible values are:
 - 0: Time domain
 - 1: Frequency domain
 - 2: Modal domain
 - 3: Arc Length (Riks) domain
- *element6*: A String specifying the default frame label.
- *element7*: A sequence of strings specifying the frame labels for all available frames in the step.
- *element8*: A sequence of floats specifying the frame values for all available frames in the step.

ODB DISPLAY COMMANDS

- *element9*: A Int specifying whether the step is user defined. Possible values are 0 indicating the step is defined in the analysis and 1 indicating the step is user defined.
- *element10*: A sequence of machine readable strings encoding the currently active frame numbers.

fieldVariables

An OdbFieldVarList object.

modelVariableList

An OdbModelFieldVarList object.

nodeSet

A repository of OdbSet objects specifying the set label. The repository is read-only.

elementSet

A repository of OdbSet objects specifying the set label. The repository is read-only.

surfaceSet

A repository of OdbSet objects specifying the set label. The repository is read-only.

display

A DisplayOptions object.

contourOptions

A ContourOptions object.

commonOptions

A CommonOptions object.

symbolOptions

A SymbolOptions object.

superimposeOptions

A SuperimposeOptions object.

displayBodyOptions

A DisplayBodyOptions object.

freeBodyOptions

A FreeBodyOptions object.

streamOptions

A StreamOptions object.

viewCutOptions

A ViewCutOptions object.

viewCuts

A repository of ViewCut objects.

displayGroup

A DisplayGroup object specifying the current display group and referring to an object in the *displayGroups* member of Session .

displayGroupInstances

A DisplayGroupInstanceRepository object.

basicOptions

A BasicOptions object.

materialOrientationOptions

An OrientationOptions object.

fieldFrame

A tuple of Strings specifying the step label and the frame label when the current step is user defined. Alternatively, *fieldFrame* maybe specified as a pair of Ints with the step index and the frame index, when the current step is defined in the analysis.

primaryVariable

A tuple specifying variables.

Each item in the sequence consists of a tuple containing the following elements:

- Element 0: A String specifying the variable label.
- Element 1: An Int specifying the output position. Possible integer values are:
 - 0: UNDEFINED_POSITION
 - 1: NODAL
 - 2: INTEGRATION_POINT
 - 3: ELEMENT_FACE
 - 4: ELEMENT_NODAL
 - 5: WHOLE_ELEMENT
 - 6: ELEMENT_CENTROID
 - 7: WHOLE_REGION
 - 8: WHOLE_PART_INSTANCE
 - 9: WHOLE_MODEL
 - 10: GENERAL_PARTICLE
- Element 2: An Int specifying the data type. Possible values are:
 - 0: ENUMERATION
 - 1: BOOLEAN
 - 2: INTEGER
 - 3: SCALAR
 - 4: VECTOR
 - 5: QUATERNION_2D
 - 6: QUATERNION_3D
 - 7: TENSOR
 - 8: TENSOR_3D_FULL

- 9: TENSOR_3D_PLANAR
 - 10: TENSOR_3D_SURFACE
 - 11: TENSOR_2D_PLANAR
 - 12: TENSOR_2D_SURFACE
- Element 3: An Int specifying the storage type. Possible values are:
 - 0: FLOAT
 - 1: DOUBLE
 - 2: INTEGER
 - 3: BOOLEAN
- Element 4: An Int specifying the refinement type. Possible values are:
 - 0: NO_REFINEMENT
 - 1: INVARIANT
 - 2: COMPONENT
- Element 5: A String specifying the refinement label.
- Element 6: An Int specifying the refinement index.
- Element 7: An Int specifying whether section point information is available. Possible values are 1 when section point information is available; 0, when this information is unavailable.
- Element 8: A sequence of a String specifying the name of the ply and category selection tuples (see below) specifying the section point information.
 - A category selection tuple consists of the following elements:
 - Element 0: A String specifying the category label.
 - Element 1: An Int specifying whether to use both top and bottom section points to obtain results. Possible values are 1 to use both section points and 0 to use only the top section point.
 - Element 2: An Int specifying the top section point index.
 - Element 3: A String specifying the top section label.
 - Element 4: An Int specifying the bottom section point index.
 - Element 5: A String specifying the bottom section label.
 - Element 6: An Int specifying the category id.
- Element 9: An Int specifying whether the data are complex. Possible values are 1 when the data are complex; 0, when the data is not complex.
- Element 10: A Float specifying the minimum possible value for the data.
- Element 11: A Float specifying the maximum possible value for the data.
- Element 12: An Int specifying whether the data is derived. Possible values are 1 when the data is derived; 0, when the data is not derived.

deformedVariable

A tuple specifying variables.

For information on the sequence, see the member *primaryVariable*.

statusVariable

A tuple of SymbolicConstants specifying variables.

For information on the sequence, see the member *primaryVariable*.

symbolVariable

A tuple of SymbolicConstants specifying variables.

For information on the sequence, see the member *primaryVariable*.

applyStatusToUndeformed

A tuple of SymbolicConstants specifying a Boolean to specify if elements are to be removed in undeformed states based on an active status variable

statusInsideRange

A tuple of SymbolicConstants specifying a Boolean to specify if the status range should be inside a specified minimum and maximum. The range will be outside when false.

statusMinimum

A tuple of Floats specifying a Float value for the minimum status range value.

statusMaximum

A tuple of Floats specifying a Float value for the maximum status range value.

useStatus

A tuple of SymbolicConstants specifying a Boolean to specify if elements are to be removed based on the status variable

firstFrame

A pair of Ints specifying the step index and the frame index of the first available frame. This sequence is read-only.

prevFrame

A pair of Ints specifying the step index and the frame index of the frame previous to the current frame. This sequence is read-only.

nextFrame

A pair of Ints specifying the step index and the frame index of the frame following the current frame. This sequence is read-only.

lastFrame

A pair of Ints specifying the step index and the frame index of the last available frame. This sequence is read-only.

35.2 CommonOptions object

The CommonOptions object stores values and attributes that are common to all plot states. The CommonOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.commonOptions* member when you import the Visualization module. Abaqus creates a *commonOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.commonOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

CommonOptions objects are accessed in one of two ways:

- The default common options. These settings are used as defaults when other *commonOptions* members are created. These settings can be set to customize user preferences.
- The common options associated with a particular viewport.

The CommonOptions object is derived from the DGCommonOptions object.

Access

```
import visualization
session.defaultOdbDisplay.commonOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions
session.viewports[name].layers[name].assemblyDisplay \
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].layers[name].odbDisplay.commonOptions
session.viewports[name].layers[name].odbDisplay \
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].layers[name].partDisplay \
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].odbDisplay.commonOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions
```

35.2.1 setValues(...)

This method modifies the CommonOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A CommonOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is SHADED.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

deformationScaling

A SymbolicConstant specifying the deformation scale factor mode. Possible values are AUTO, UNIFORM, and NONUNIFORM. The default value is AUTO.

uniformScaleFactor

A Float specifying the uniform deformation scaling constant when *deformationScaling*=UNIFORM. The default value is *autoDeformationScaleValue*.

nonuniformScaleFactor

A sequence of three Floats specifying the deformation scaling in each of the three coordinate directions when *deformationScaling*=NONUNIFORM. The default value is (*autoDeformationScaleValue*, *autoDeformationScaleValue*, *autoDeformationScaleValue*).

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

ODB DISPLAY COMMANDS

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-*-120-*-*-m-*-*-*".

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

coordinateScaleFactors

A sequence of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals or surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

Return value

None

Exceptions

RangeError.

35.2.2 Members

The CommonOptions object can have the following members:

deformationScaling

A SymbolicConstant specifying the deformation scale factor mode. Possible values are AUTO, UNIFORM, and NONUNIFORM. The default value is AUTO.

uniformScaleFactor

A Float specifying the uniform deformation scaling constant when *deformationScaling*=UNIFORM. The default value is *autoDeformationScaleValue*.

autoDeformationScaleValue

A Float specifying the deformation scale factor value when *deformationScaling*=AUTO. This value is read-only.

nonuniformScaleFactor

A tuple of three Floats specifying the deformation scaling in each of the three coordinate directions when *deformationScaling*=NONUNIFORM. The default value is (*autoDeformationScaleValue*, *autoDeformationScaleValue*, *autoDeformationScaleValue*).

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is SHADED.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

ODB DISPLAY COMMANDS

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals or surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-*120-*-*m-*-*-*".

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

35.3 ContourOptions object

The *ContourOptions* object stores values and attributes associated with a contour plot. The *ContourOptions* object has no constructor command. Abaqus creates a *defaultOdbDisplay.contourOptions* member when you import the Visualization module. Abaqus creates a *contourOptions* member when it creates the *OdbDisplay* object, using the values from *defaultOdbDisplay.contourOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

ContourOptions objects are accessed in one of two ways:

- The default contour options. These settings are used as defaults when other *contourOptions* members are created. These settings can be set to customize user preferences.

- The contour options associated with a particular viewport.

The ContourOptions object is derived from the DGContourOptions object.

Access

```
import visualization
session.defaultOdbDisplay.contourOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].layers[name].odbDisplay.contourOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].odbDisplay.contourOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
```

35.3.1 setValues(...)

This method modifies the ContourOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A ContourOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

contourType

A SymbolicConstant specifying the contour type. Possible values are LINE, BANDED, QUILT, and ISOSURFACE. The default value is BANDED.

contourMethod

A SymbolicConstant specifying the contour rendering method. Possible values are TEXTURE_MAPPED and TESSELLATED. The default value is TEXTURE_MAPPED.

tickmarkPlots

A Boolean specifying whether tick mark plots should be displayed on line-type elements. If *tickmarkPlots*=ON, Abaqus displays a tick mark plot. If *tickmarkPlots*=OFF, Abaqus displays contours on the element faces. The default value is OFF.

contourStyle

A SymbolicConstant specifying the interval style of the contour plot. Possible values are CONTINUOUS and UNIFORM. The default value is UNIFORM.

numIntervals

An Int specifying the number of intervals when *contourStyle*=UNIFORM. Possible values are $2 \leq \text{numIntervals} \leq 24$. The default value is 12.

intervalType

A SymbolicConstant specifying the interval type of the contour plot. Possible values are UNIFORM, LOG, and USER_DEFINED. The default value is UNIFORM.

intervalValues

A sequence of Floats specifying the interval values when *intervalType*=USER_DEFINED.

maxAutoCompute

A Boolean specifying whether the contour range maximum value should be computed from the output data to be contoured. The default value is ON.

maxValue

A Float specifying the contour range maximum value to be used in the plot when *maxAutoCompute*=ON. The default value is *autoMaxValue*.

minAutoCompute

A Boolean specifying whether the contour range minimum value should be computed from the output data to be contoured. The default value is ON.

minValue

A Float specifying the contour range minimum value to be used in the plot when *minAutoCompute*=ON. The default value is *autoMinValue*.

animationAutoLimits

A SymbolicConstant specifying the method to be used when contour limits are automatically computed for animation. *animationAutoLimits* will only effect the minimum limit and/or maximum limit when *minAutoCompute* and/or *maxAutoCompute*=True. Possible values are FIRST_AND_LAST, CURRENT_FRAME, RECOMPUTE_EACH_FRAME, and ALL_FRAMES. The default value is ALL_FRAMES.

edgeColorLine

A String specifying the edge color to be used when *contourType*=LINE. The default value is "White".

edgeColorBandedQuilt

A String specifying the edge color to be used when *contourType*=BANDED or QUILT. The default value is "Black".

spectrum

A String specifying the name of the color spectrum to be used in the contour plot. The default value is "Rainbow".

outsideLimitsMode

A SymbolicConstant specifying the color of contour values that exceed the limits of the plot. Possible values are SPECTRUM and SPECIFY.

When *outsideLimitsMode*=SPECTRUM, the maximum and minimum contour spectrum colors are used for values that exceed the limits of the plot. When *outsideLimitsMode*=SPECIFY, the values of *outsideLimitsAboveColor* and *outsideLimitsBelowColor* are used for values that exceed the limits of the plot.

outsideLimitsAboveColor

A String specifying the color to be used for values that exceed the limits of the plot when *outsideLimitsMode*=SPECIFY. The default value is "Grey80".

outsideLimitsBelowColor

A String specifying the color to be used for values that exceed the limits of the plot when *outsideLimitsMode*=SPECIFY. The default value is "Grey20".

intervalLineAttributes

A sequence of sequences of SymbolicConstants specifying the line style and line thickness for each interval in the plot when *contourType*=LINE. The size of the outer sequence must be equal to *numIntervals*-1. The inner sequence consists of two SymbolicConstants specifying the line style and line thickness. For possible values, refer to the *edgeLineStyle* and *edgeLineThickness* members of the DGCommonOptions object. The default is ((SOLID, VERY_THIN),).

contourEdges

A Boolean specifying whether to plot the edges of each contour interval when *contourType*=BANDED or ISOSURFACE. The default value is OFF.

contourEdgeColor

A String specifying the color to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. The default value is "Grey60".

contourEdgeStyle

A SymbolicConstant specifying the edge line style to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

contourEdgeThickness

A SymbolicConstant specifying the edge line thickness to be used to plot the edge of the contour intervals when *contourType*=BANDED or ISOSURFACE. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

tickmarkAxisLength

A SymbolicConstant specifying the length of the tick mark plot axes. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

tickmarkBaseValue

A Float specifying the base contour value defining the tick mark axis contour value that intersects the elements. Possible values are $autoMinValue \leq tickmarkBaseValue \leq autoMaxValue$. The default value is 0.0.

tickmarkOrientation

A SymbolicConstant specifying the orientation of the tick mark plots. Possible values are N1 and N2. The default value is N2.

tickmarkCurveColor

A String specifying the color to be used to plot the tick mark curve. The default value is "Cyan".

averagedOrientationDisplay

A Boolean specifying the display of the nodal averaged coordinate systems used when averaging element vector or tensor data. The default value is OFF.

extrapolatedAveraging

A Boolean specifying whether to auto-compute contour limits using extrapolated values alone or extrapolated values that are averaged. The default value is OFF.

showMaxLocation

A Boolean specifying whether to display location of maximum value. The default value is OFF.

showMinLocation

A Boolean specifying whether to display location of minimum value. The default value is OFF.

Return value

None

Exceptions

RangeError.

35.3.2 Members

The ContourOptions object can have the following members:

contourType

A SymbolicConstant specifying the contour type. Possible values are LINE, BANDED, QUILT, and ISOSURFACE. The default value is BANDED.

numIntervals

An Int specifying the number of intervals when *contourStyle*=UNIFORM. Possible values are $2 \leq numIntervals \leq 24$. The default value is 12.

intervalType

A SymbolicConstant specifying the interval type of the contour plot. Possible values are UNIFORM, LOG, and USER_DEFINED. The default value is UNIFORM.

ODB DISPLAY COMMANDS

maxAutoCompute

A Boolean specifying whether the contour range maximum value should be computed from the output data to be contoured. The default value is ON.

maxValue

A Float specifying the contour range maximum value to be used in the plot when *maxAutoCompute*=ON. The default value is *autoMaxValue*.

minAutoCompute

A Boolean specifying whether the contour range minimum value should be computed from the output data to be contoured. The default value is ON.

minValue

A Float specifying the contour range minimum value to be used in the plot when *minAutoCompute*=ON. The default value is *autoMinValue*.

animationAutoLimits

A SymbolicConstant specifying the method to be used when contour limits are automatically computed for animation. *animationAutoLimits* will only effect the minimum limit and/or maximum limit when *minAutoCompute* and/or *maxAutoCompute*=True. Possible values are FIRST_AND_LAST, CURRENT_FRAME, RECOMPUTE_EACH_FRAME, and ALL_FRAMES. The default value is ALL_FRAMES.

outsideLimitsMode

A SymbolicConstant specifying the color of contour values that exceed the limits of the plot. Possible values are SPECTRUM and SPECIFY.

When *outsideLimitsMode*=SPECTRUM, the maximum and minimum contour spectrum colors are used for values that exceed the limits of the plot. When *outsideLimitsMode*=SPECIFY, the values of *outsideLimitsAboveColor* and *outsideLimitsBelowColor* are used for values that exceed the limits of the plot.

extrapolatedAveraging

A Boolean specifying whether to auto-compute contour limits using extrapolated values alone or extrapolated values that are averaged. The default value is OFF.

showMaxLocation

A Boolean specifying whether to display location of maximum value. The default value is OFF.

showMinLocation

A Boolean specifying whether to display location of minimum value. The default value is OFF.

autoMaxValue

A Float specifying the maximum value to be used in the plot. The value is computed from the output data to be contoured. This value is read-only.

autoMinValue

A Float specifying the minimum value to be used in the plot. The value is computed from the output data to be contoured. This value is read-only.

outsideLimitsAboveColor

A String specifying the color to be used for values that exceed the limits of the plot when *outsideLimitsMode*=SPECIFY. The default value is "Grey80".

outsideLimitsBelowColor

A String specifying the color to be used for values that exceed the limits of the plot when *outsideLimitsMode*=SPECIFY. The default value is "Grey20".

spectrum

A String specifying the name of the color spectrum to be used in the contour plot. The default value is "Rainbow".

intervalValues

A tuple of Floats specifying the interval values when *intervalType*=USER_DEFINED.

contourMethod

A SymbolicConstant specifying the contour rendering method. Possible values are TEXTURE_MAPPED and TESSELLATED. The default value is TEXTURE_MAPPED.

tickmarkPlots

A Boolean specifying whether tick mark plots should be displayed on line-type elements. If *tickmarkPlots*=ON, Abaqus displays a tick mark plot. If *tickmarkPlots*=OFF, Abaqus displays contours on the element faces. The default value is OFF.

contourStyle

A SymbolicConstant specifying the interval style of the contour plot. Possible values are CONTINUOUS and UNIFORM. The default value is UNIFORM.

contourEdges

A Boolean specifying whether to plot the edges of each contour interval when *contourType*=BANDED or ISOSURFACE. The default value is OFF.

contourEdgeStyle

A SymbolicConstant specifying the edge line style to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

contourEdgeThickness

A SymbolicConstant specifying the edge line thickness to be used to plot the edge of the contour intervals when *contourType*=BANDED or ISOSURFACE. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

averagedOrientationDisplay

A Boolean specifying the display of the nodal averaged coordinate systems used when averaging element vector or tensor data. The default value is OFF.

matchingPlyLabels

A Boolean specifying whether the label for the matching ply shows up in the viewport. The default value is OFF.

colorByMatchingPlies

A Boolean specifying whether the contour color is driven by the matching ply. The default value is OFF.

tickmarkAxisLength

A SymbolicConstant specifying the length of the tick mark plot axes. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

tickmarkBaseValue

A Float specifying the base contour value defining the tick mark axis contour value that intersects the elements. Possible values are $autoMinValue \leq tickmarkBaseValue \leq autoMaxValue$. The default value is 0.0.

tickmarkOrientation

A SymbolicConstant specifying the orientation of the tick mark plots. Possible values are N1 and N2. The default value is N2.

edgeColorLine

A String specifying the edge color to be used when *contourType*=LINE. The default value is "White".

edgeColorBandedQuilt

A String specifying the edge color to be used when *contourType*=BANDED or QUILT. The default value is "Black".

contourEdgeColor

A String specifying the color to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. The default value is "Grey60".

tickmarkCurveColor

A String specifying the color to be used to plot the tick mark curve. The default value is "Cyan".

intervalLineAttributes

A tuple of tuples of SymbolicConstants specifying the line style and line thickness for each interval in the plot when *contourType*=LINE. The size of the outer sequence must be equal to *numIntervals*-1. The inner sequence consists of two SymbolicConstants specifying the line style and line thickness. For possible values, refer to the *edgeLineStyle* and *edgeLineThickness* members of the DGCommonOptions object. The default is ((SOLID, VERY_THIN),).

35.4 DefaultOdbDisplay object

The DefaultOdbDisplay object is a limited-functionality version of the OdbDisplay object.

Access

```
import visualization
session.defaultOdbDisplay
```


35.4.1 Members

The DefaultOdbDisplay object has the following members:

basicOptions

A BasicOptions object.

commonOptions

A CommonOptions object.

contourOptions

A ContourOptions object.

displayBodyOptions

A DisplayBodyOptions object.

freeBodyOptions

A FreeBodyOptions object.

streamOptions

A StreamOptions object.

materialOrientationOptions

An OrientationOptions object.

superimposeOptions

A SuperimposeOptions object.

symbolOptions

A SymbolOptions object.

viewCutOptions

A ViewCutOptions object.

35.5 DisplayBodyOptions object

The DisplayBodyOptions object stores values and attributes that are common to all plot states. The DisplayBodyOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.displayBodyOptions* member when you import the Visualization module. Abaqus creates a *displayBodyOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.displayBodyOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

DisplayBodyOptions objects are accessed in one of two ways:

- The default display body options. These settings are used as defaults when other *displayBodyOptions* members are created. These settings can be set to customize user preferences.
- The display body options associated with a particular viewport.

The DisplayBodyOptions object is derived from the DGDisplayBodyOptions object.

Access

```
import visualization
session.defaultOdbDisplay.displayBodyOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].odbDisplay.displayBodyOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].odbDisplay.displayBodyOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
```

35.5.1 setValues(...)

This method modifies the DisplayBodyOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A DisplayBodyOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

coordinateScaleFactors

A sequence of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

Return value

None

Exceptions

RangeError.

35.5.2 Members

The DisplayBodyOptions object can have the following members:

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

35.6 OrientationOptions object

The OrientationOptions object stores values and attributes associated with a material orientation plot. The OrientationOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.materialOrientationOptions* member when you import the Visualization module. Abaqus creates a *materialOrientationOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.materialOrientationOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

OrientationOptions objects are accessed in one of two ways:

- The default material orientation options. These settings are used as defaults when other *materialOrientationOptions* members are created. These settings can be set to customize user preferences.
- The material orientation options associated with a particular viewport.

The OrientationOptions object is derived from the DGOrientationOptions object.

Access

```
import visualization
session.defaultOdbDisplay.materialOrientationOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].odbDisplay\
.materialOrientationOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
session.viewports[name].odbDisplay.materialOrientationOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
```

35.6.1 setValues(...)

This method modifies the OrientationOptions object.

Arguments

Required arguments

None.

Optional arguments

options

An OrientationOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

axis1Color

A String specifying the color of axis 1 of the material orientation triad. The default value is "Cyan".

showAxis1

A Boolean specifying whether axis 1 of the material orientation triad should be displayed. The default value is ON.

axis2Color

A String specifying the color of axis 2 of the material orientation triad. The default value is "Yellow".

showAxis2

A Boolean specifying whether axis 2 of the material orientation triad should be displayed. The default value is ON.

axis3Color

A String specifying the color of axis 3 of the material orientation triad. The default value is "Red".

showAxis3

A Boolean specifying whether axis 3 of the material orientation triad should be displayed. The default value is ON.

symbolSize

A Float specifying the size of the material orientation triad. The default value is 12.0.

lineThickness

A SymbolicConstant specifying the thickness of the material orientation triad. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

orientation

A SymbolicConstant specifying the orientation used for composites. Possible values are PLY and LAYUP. The default value is PLY.

arrowheadStyle

A SymbolicConstant specifying the arrowhead style for the material orientation triad. Possible values are NONE, FILLED, and WIRE. The default value is NONE.

scaleMode

A SymbolicConstant specifying the scaling basis for the material orientation triad. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

Return value

None

Exceptions

RangeError.

35.6.2 Members

The OrientationOptions object can have the following members:

showAxis1

A Boolean specifying whether axis 1 of the material orientation triad should be displayed. The default value is ON.

showAxis2

A Boolean specifying whether axis 2 of the material orientation triad should be displayed. The default value is ON.

showAxis3

A Boolean specifying whether axis 3 of the material orientation triad should be displayed. The default value is ON.

symbolSize

A Float specifying the size of the material orientation triad. The default value is 12.0.

lineThickness

A SymbolicConstant specifying the thickness of the material orientation triad. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

orientation

A SymbolicConstant specifying the orientation used for composites. Possible values are PLY and LAYUP. The default value is PLY.

arrowheadStyle

A SymbolicConstant specifying the arrowhead style for the material orientation triad. Possible values are NONE, FILLED, and WIRE. The default value is NONE.

scaleMode

A SymbolicConstant specifying the scaling basis for the material orientation triad. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

axis1Color

A String specifying the color of axis 1 of the material orientation triad. The default value is "Cyan".

axis2Color

A String specifying the color of axis 2 of the material orientation triad. The default value is "Yellow".

axis3Color

A String specifying the color of axis 3 of the material orientation triad. The default value is "Red".

35.7 SuperimposeOptions object

The SuperimposeOptions object stores values and attributes associated with the undeformed shape when the display contains both the deformed shape and the undeformed shape. The SuperimposeOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.superimposeOptions* member when you import the Visualization module. Abaqus creates a *superimposeOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.superimposeOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

SuperimposeOptions objects are accessed in one of two ways:

- The default superimpose options. These settings are used as defaults when other *superimposeOptions* members are created. These settings can be set to customize user preferences.
- The superimpose options associated with a particular viewport.

The SuperimposeOptions object is derived from the DGSuperimposeOptions object.

Access

```
import visualization
session.defaultOdbDisplay.superimposeOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].odbDisplay.superimposeOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
session.viewports[name].odbDisplay.superimposeOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
```

35.7.1 setValues(...)

This method modifies the SuperimposeOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A SuperimposeOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is WIREFRAME.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is FEATURE.

NONE can be used only when *renderStyle*=SHADED.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "Green".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "Green".

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-*-120-*-m-*-*-".

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

ODB DISPLAY COMMANDS

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

coordinateScaleFactors

A sequence of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals and surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

deformedOffsetMode

A SymbolicConstant specifying the offset of the deformed shape from the undeformed shape. Possible values are NONE, UNIFORM, and NONUNIFORM. The default value is NONE.

NONE specifies that there should be no offset of the deformed shape from the undeformed shape.

uniformOffset

A Float specifying the uniform offset value when *deformedOffsetMode*=UNIFORM. The default value is 0.0.

nonuniformOffset

A sequence of three Floats specifying the offset value in each of the three coordinate directions when *deformedOffsetMode*=NONUNIFORM. The default value is (0, 0, 0).

Return value

None

Exceptions

RangeError.

35.7.2 Members

The SuperimposeOptions object can have the following members:

deformedOffsetMode

A SymbolicConstant specifying the offset of the deformed shape from the undeformed shape. Possible values are NONE, UNIFORM, and NONUNIFORM. The default value is NONE.

NONE specifies that there should be no offset of the deformed shape from the undeformed shape.

uniformOffset

A Float specifying the uniform offset value when *deformedOffsetMode*=UNIFORM. The default value is 0.0.

nonuniformOffset

A tuple of three Floats specifying the offset value in each of the three coordinate directions when *deformedOffsetMode*=NONUNIFORM. The default value is (0, 0, 0).

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is WIREFRAME.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is FEATURE.

NONE can be used only when *renderStyle*=SHADED.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

ODB DISPLAY COMMANDS

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals and surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "Green".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "Green".

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-*-120-*-*-m-*-*-*".

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

35.8 SymbolOptions object

The *SymbolOptions* object stores values and attributes associated with a symbol plot. The *SymbolOptions* object has no constructor command. Abaqus creates a *defaultOdbDisplay.symbolOptions* member when you import the Visualization module. Abaqus creates a *symbolOptions* member when it creates the *OdbDisplay* object, using the values from *defaultOdbDisplay.symbolOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

SymbolOptions objects are accessed in one of two ways:

- The default symbol options. These settings are used as defaults when other *symbolOptions* members are created. These settings can be set to customize user preferences.
- The symbol options associated with a particular viewport.

The *SymbolOptions* object is derived from the *DGSymbolOptions* object.

Access

```
import visualization
```

```

session.defaultOdbDisplay.symbolOptions
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].odbDisplay.symbolOptions
session.viewports[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions
session.viewports[name].odbDisplay.symbolOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions

```

35.8.1 setValues(...)

This method modifies the SymbolOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A SymbolOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

vectorQuantity

A SymbolicConstant specifying the vector quantity to display. Possible values are RESULTANT and VECTOR_COMPONENT. The default value is RESULTANT.

vectorLineThickness

A SymbolicConstant specifying the vector line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

vectorArrowheadStyle

A SymbolicConstant specifying the vector arrowhead style. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

vectorColor

A String specifying the vector color. The default value is "Red".

vectorColorMethod

A SymbolicConstant specifying the vector color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

vectorColorSpectrum

A String specifying the vector color spectrum name. The default value is "Rainbow".

vectorIntervalNumber

An Int specifying the number of color intervals for vector symbols. The default value is 12.

symbolDensity

A Float specifying the factor for randomized sampling. The default value is 1.0.

constantLengthArrows

A Boolean specifying whether to use constant-length arrows for vector symbols. The default value is OFF.

tensorColorMethod

A SymbolicConstant specifying the tensor color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

tensorColorSpectrum

A String specifying the tensor color spectrum name. The default value is "Rainbow".

tensorIntervalNumber

An Int specifying the number of color intervals for tensor symbols. The default value is 12.

vectorMaxValueAutoCompute

A Boolean specifying whether the maximum vector value is to be computed automatically. The default value is ON.

vectorMaxValue

A Float specifying the user-specified maximum vector value. The default value is *autoVectorMaxValue*.

vectorMinValueAutoCompute

A Boolean specifying whether the minimum vector value is to be computed automatically. The default value is ON.

vectorMinValue

A Float specifying the user-specified minimum vector value. The default value is *autoVectorMinValue*.

tensorQuantity

A SymbolicConstant specifying the tensor quantity to display. Possible values are ALL_PRINCIPAL_COMPONENTS, PRINCIPAL_COMPONENT, ALL_DIRECT_COMPONENTS, and DIRECT_COMPONENT. The default value is ALL_PRINCIPAL_COMPONENTS.

arrowSymbolSize

An Int specifying the length of vector and tensor symbols. The default value is 6.

tensorMaxPrinColor

A String specifying the color of the maximum principal tensor symbols. The default value is "Red".

tensorMinPrinColor

A String specifying the color of the minimum principal tensor symbols. The default value is "Cyan".

tensorMidPrinColor

A String specifying the color of the intermediate principal tensor symbols. The default value is "Yellow".

tensorSelectedPrinColor

A String specifying the color of the selected principal tensor symbols. The default value is "Red".

tensorLineThickness

A SymbolicConstant specifying the line thickness of the tensor symbols. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

tensorArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the tensor symbols. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

tensorMaxValueAutoCompute

A Boolean specifying whether the maximum tensor value is to be computed automatically. The default value is ON.

tensorMaxValue

A Float specifying the user-specified maximum tensor value. The default value is *autoTensorMaxValue*.

tensorMinValueAutoCompute

A Boolean specifying whether the minimum tensor value is to be computed automatically. The default value is ON.

tensorMinValue

A Float specifying the user-specified minimum tensor value. The default value is *autoTensorMinValue*.

Return value

None

Exceptions

RangeError.

35.8.2 Members

The SymbolOptions object can have the following members:

vectorQuantity

A SymbolicConstant specifying the vector quantity to display. Possible values are RESULTANT and VECTOR_COMPONENT. The default value is RESULTANT.

vectorMaxValueAutoCompute

A Boolean specifying whether the maximum vector value is to be computed automatically. The default value is ON.

vectorMaxValue

A Float specifying the user-specified maximum vector value. The default value is *autoVectorMaxValue*.

vectorMinValueAutoCompute

A Boolean specifying whether the minimum vector value is to be computed automatically. The default value is ON.

vectorMinValue

A Float specifying the user-specified minimum vector value. The default value is *autoVectorMinValue*.

tensorQuantity

A SymbolicConstant specifying the tensor quantity to display. Possible values are ALL_PRINCIPAL_COMPONENTS, PRINCIPAL_COMPONENT, ALL_DIRECT_COMPONENTS, and DIRECT_COMPONENT. The default value is ALL_PRINCIPAL_COMPONENTS.

tensorMaxValueAutoCompute

A Boolean specifying whether the maximum tensor value is to be computed automatically. The default value is ON.

tensorMaxValue

A Float specifying the user-specified maximum tensor value. The default value is *autoTensorMaxValue*.

tensorMinValueAutoCompute

A Boolean specifying whether the minimum tensor value is to be computed automatically. The default value is ON.

tensorMinValue

A Float specifying the user-specified minimum tensor value. The default value is *autoTensorMinValue*.

autoVectorMaxValue

The SymbolicConstant NOT_SET or a Float specifying the vector maximum value when *vectorMaxValueAutoCompute*=ON. This value is read-only. The default value is NOT_SET.

autoVectorMinValue

The SymbolicConstant NOT_SET or a Float specifying the vector minimum value when *vectorMinValueAutoCompute*=ON. This value is read-only. The default value is NOT_SET.

ODB DISPLAY COMMANDS

autoTensorMaxValue

The SymbolicConstant NOT_SET or a Float specifying the tensor maximum value when *tensorMaxValueAutoCompute*=ON. This value is read-only. The default value is NOT_SET.

autoTensorMinValue

The SymbolicConstant NOT_SET or a Float specifying the tensor minimum value when *tensorMinValueAutoCompute*=ON. This value is read-only. The default value is NOT_SET.

vectorLineThickness

A SymbolicConstant specifying the vector line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

vectorColorMethod

A SymbolicConstant specifying the vector color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

tensorColorMethod

A SymbolicConstant specifying the tensor color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

vectorArrowheadStyle

A SymbolicConstant specifying the vector arrowhead style. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

arrowSymbolSize

An Int specifying the length of vector and tensor symbols. The default value is 6.

vectorIntervalNumber

An Int specifying the number of color intervals for vector symbols. The default value is 12.

symbolDensity

A Float specifying the factor for randomized sampling. The default value is 1.0.

constantLengthArrows

A Boolean specifying whether to use constant-length arrows for vector symbols. The default value is OFF.

tensorIntervalNumber

An Int specifying the number of color intervals for tensor symbols. The default value is 12.

tensorLineThickness

A SymbolicConstant specifying the line thickness of the tensor symbols. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

tensorArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the tensor symbols. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

numberFormatT

A SymbolicConstant specifying the number format for tensor. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

numberFormatV

A SymbolicConstant specifying the number format for vector. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

arrowScaleMode

A SymbolicConstant specifying the arrow scaling mode. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

drawLabelT

A Boolean specifying whether to draw tensor labels. The default value is OFF.

drawLabelV

A Boolean specifying whether to draw vector labels. The default value is OFF.

numDigitsT

An Int specifying the number of digits in the tensor label. The default value is 2.

numDigitsV

An Int specifying the number of digits in the vector label. The default value is 2.

vectorColor

A String specifying the vector color. The default value is "Red".

vectorColorSpectrum

A String specifying the vector color spectrum name. The default value is "Rainbow".

tensorColorSpectrum

A String specifying the tensor color spectrum name. The default value is "Rainbow".

textColorT

A String specifying the text color for tensor. The default value is "Yellow".

textColorV

A String specifying the text color for vector. The default value is "Yellow".

textFontT

A String specifying the text font for tensor. The default value is "verdana".

textFontV

A String specifying the text font for vector. The default value is "verdana".

tensorMaxPrinColor

A String specifying the color of the maximum principal tensor symbols. The default value is "Red".

tensorMinPrinColor

A String specifying the color of the minimum principal tensor symbols. The default value is "Cyan".

tensorMidPrinColor

A String specifying the color of the intermediate principal tensor symbols. The default value is "Yellow".

tensorSelectedPrinColor

A String specifying the color of the selected principal tensor symbols. The default value is "Red".

35.9 ViewCut object

The ViewCut object is used to store values and attributes associate with ViewCut type objects. ViewCut objects can be created using the methods described below. The methods accessed via the OdbDisplay object cause the ViewCut object to be added to the session.viewports[name].odbDisplay.viewCuts repository.

Access

```
import visualization
session.viewports[name].layers[name].odbDisplay.viewCuts[name]
session.viewports[name].odbDisplay.viewCuts[name]
```

35.9.1 ViewCut(...)

This method creates a ViewCut object.

Path

```
session.viewports[name].layers[name].odbDisplay.ViewCut
session.viewports[name].odbDisplay.ViewCut
```

Required arguments

name

A String specifying the repository key.

shape

A SymbolicConstant specifying the shape of the ViewCut object. Possible values are PLANE, CYLINDER, SPHERE, and ISOSURFACE.

origin

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of the origin of the plane, cylinder or sphere cut. This origin is not required if the cut shape is ISOSURFACE or if the cut is defined by the *csysName* argument.

normal

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the normal axis to the plane defining the cut, when the plane is defined using the *origin* argument or a SymbolicConstant defining this normal axis, when the cut is defined by the *csysName* argument. Possible values are AXIS_1, AXIS_2, AXIS_3. This axis is not required if the cut *shape* is CYLINDER, SPHERE or ISOSURFACE.

axis2

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of the second axis of the plane defining the cut, when the plane is defined using the *origin* argument or a SymbolicConstant defining this second axis, when the cut is defined by the *csysName* argument. Possible values are AXIS_1, AXIS_2, AXIS_3. This axis is used to rotate the plane cut. It is not required if the cut *shape* is CYLINDER, SPHERE or ISOSURFACE.

csysName

A String specifying the name of the DatumCsys object to be used to define the cut. This name is not required if the cut *shape* is ISOSURFACE or if the cut is defined by the *origin* argument.

cylinderAxis

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the cylinder axis defining the cut, when the cut is defined using the *origin* argument or a SymbolicConstant defining this cylinder axis, when the cut is defined by the *csysName* argument. Possible values are AXIS_1, AXIS_2, AXIS_3. This axis is not required if the cut *shape* is PLANE, SPHERE, or ISOSURFACE.

Optional arguments

followDeformation

A Boolean specifying whether the cut will follow the deformation or be static. The default value is OFF.

overrideAveraging

A Boolean specifying averaging for element based fields associated with isosurface cuts will be set to compute-average with a threshold of 100% when true. The current field options will be used when false. The default value is ON.

referenceFrame

A SymbolicConstant specifying which reference frame will be used when the cut follows the deformation. Possible values are FIRST_FRAME, LAST_FRAME, and CURRENT_FRAME. The default value is FIRST_FRAME.

Return value

A ViewCut object.

Exceptions

None.

35.9.2 **setValues(...)**

This method modifies the ViewCut object.

Arguments

Required arguments

None.

Optional arguments

At least one of the following must be provided:

angle

A Float specifying the rotation angle of the cut defined with a *shape* set to PLANE.

motion

A SymbolicConstant specifying the type of motion for the cut defined with a *shape* set to PLANE. Possible values are TRANSLATE and ROTATE. The default value is TRANSLATE.

position

A Float specifying the position of the cut defined with a *shape* set to PLANE.

radius

A Float specifying the radius of the cut defined with a *shape* set to CYLINDER or SPHERE.

rotationAxis

A SymbolicConstant specifying the rotation axis for the cut defined with a *shape* set to PLANE. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_2.

value

A Float specifying the value of the cut defined with a *shape* set to ISOSURFACE.

showModelAboveCut

A Boolean specifying whether to display the model above the cut. The default value is OFF.

showModelOnCut

A Boolean specifying whether to display the model on the cut. The default value is ON.

showModelBelowCut

A Boolean specifying whether to display the model below the cut. The default value is ON.

showFreeBodyCut

A Boolean specifying whether to display the free body cut. The default value is OFF.

csysName

A String specifying the name of the DatumCsys object to be used to define the cut. This name is not required if the cut *shape* is ISOSURFACE or if the cut is defined by the *origin* argument.

origin

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of the origin of the plane, cylinder or sphere cut. This origin is not required if the cut shape is ISOSURFACE or if the cut is defined by the *csysName* argument.

normal

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the normal axis to the plane defining the cut, when the plane is defined using the *origin* argument or a SymbolicConstant defining this normal axis, when the cut is defined by the *csysName* argument. Possible values are

AXIS_1, AXIS_2, AXIS_3. This axis is not required if the cut *shape* is CYLINDER, SPHERE or ISOSURFACE.

axis2

A sequence of three Floats specifying the X-, Y-, and Z-coordinates of the second axis of the plane defining the cut, when the plane is defined using the *origin* argument or a SymbolicConstant defining this second axis, when the cut is defined by the *csysName* argument. Possible values are AXIS_1, AXIS_2, AXIS_3. This axis is used to rotate the plane cut. It is not required if the cut *shape* is CYLINDER, SPHERE or ISOSURFACE.

cylinderAxis

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the cylinder axis defining the cut, when the cut is defined using the *origin* argument or a SymbolicConstant defining this cylinder axis, when the cut is defined by the *csysName* argument. Possible values are AXIS_1, AXIS_2, AXIS_3. This axis is not required if the cut *shape* is PLANE, SPHERE, or ISOSURFACE.

followDeformation

A Boolean specifying whether the cut will follow the deformation or be static. The default value is OFF.

overrideAveraging

A Boolean specifying averaging for element based fields associated with isosurface cuts will be set to compute-average with a threshold of 100% when true. The current field options will be used when false. The default value is ON.

referenceFrame

A SymbolicConstant specifying which reference frame will be used when the cut follows the deformation. Possible values are FIRST_FRAME, LAST_FRAME, and CURRENT_FRAME. The default value is FIRST_FRAME.

Return value

None

Exceptions

None.

35.9.3 updateVariable()

This method updates the field associated with an isosurface cut to be consistent with the current primary variable.

Arguments

None.

Return value

None

Exceptions

None.

35.9.4 Members

The ViewCut object has members with the same names and descriptions as the arguments to the **ViewCut** method.

In addition, the ViewCut object has the following members:

angle

A Float specifying the rotation angle of the cut defined with a *shape* set to PLANE.

motion

A SymbolicConstant specifying the type of motion for the cut defined with a *shape* set to PLANE. Possible values are TRANSLATE and ROTATE. The default value is TRANSLATE.

position

A Float specifying the position of the cut defined with a *shape* set to PLANE.

radius

A Float specifying the radius of the cut defined with a *shape* set to CYLINDER or SPHERE.

rotationAxis

A SymbolicConstant specifying the rotation axis for the cut defined with a *shape* set to PLANE. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_2.

value

A Float specifying the value of the cut defined with a *shape* set to ISOSURFACE.

showModelAboveCut

A Boolean specifying whether to display the model above the cut. The default value is OFF.

showModelOnCut

A Boolean specifying whether to display the model on the cut. The default value is ON.

showModelBelowCut

A Boolean specifying whether to display the model below the cut. The default value is ON.

showFreeBodyCut

A Boolean specifying whether to display the free body cut. The default value is OFF.

active

A Boolean specifying whether the cut is displayed.

cutRange

A pair of Floats specifying the acceptable range for positioning the cut.

35.10 ViewerOptions object

The ViewerOptions object specifies options to set the result caching parameters. The ViewerOptions object has no constructor. Abaqus creates the *viewerOptions* member when a session is started.

Access

```
import visualization
session.viewerOptions
```

35.10.1 setValues(...)

This method modifies the ViewerOptions object.

Arguments

Required arguments

None.

Optional arguments

primaryVariableCaching

A Boolean specifying whether results are currently cached. Caching improves the performance of subsequent access. The default value is ON.

deformedVariableCaching

A Boolean specifying whether deformation vectors are currently cached. Caching improves the performance of subsequent access. The default value is ON.

cutVariableCaching

A Boolean specifying whether the values used for displaying cut models are currently cached. Caching improves the performance of subsequent access. The default value is ON.

odbUpdateChecking

A Boolean specifying whether the current **.odb** file should be checked for updates. Setting *odbUpdateChecking* to OFF can improve Viewer performance when accessing data from a remote file. The default value is ON.

odbUpdateCheckInterval

An Int specifying the minimum time between status checks (in seconds). The default value is 0.

Return value

None

Exceptions

None.

35.10.2 Members

The ViewerOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

36. Optimization commands

Optimization commands are used to perform topology, shape, or sizing optimization of your model given a set of objectives and a set of restrictions.

36.1 OptimizationTask object

The OptimizationTask object is the abstract base type for other OptimizationTask objects. The OptimizationTask object has no explicit constructor. The methods and members of the OptimizationTask object are common to all objects derived from OptimizationTask.

Access

```
import optimization
mdb.models[name].optimizationTasks[name]
```

36.1.1 Members

The OptimizationTask object can have the following members:

name

A String specifying the optimization task repository key.

region

The SymbolicConstant MODEL or a Region object specifying the region to which the optimization task is applied. The default value is MODEL.

designResponses

A repository of DesignResponse objects.

objectiveFunctions

A repository of ObjectiveFunction objects.

optimizationConstraints

A repository of OptimizationConstraint objects.

geometricRestrictions

A repository of GeometricRestriction objects.

stopConditions

A repository of StopCondition objects.

36.2 BeadTask object

The BeadTask object defines a bead task.

The BeadTask object is derived from the OptimizationTask object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name]
```

36.2.1 BeadTask(...)

This method creates a BeadTask object.

Path

```
mdb.models[name].BeadTask
```

Required argument

name

A String specifying the optimization task repository key.

Optional arguments

algorithm

A SymbolicConstant specifying the optimization task algorithm. Possible values are GENERAL_OPTIMIZATION and CONDITION_BASED_OPTIMIZATION. The default value is GENERAL_OPTIMIZATION.

areBCRegionsFrozen

A Boolean specifying whether to exclude elements with boundary conditions from the optimization. The default value is OFF.

beadIter

An int specifying step size of the optimization. The default value is 1.

beadMaxMembraneStress

A float specifying maximum membrane/bending stress. The default value is 0.1.

beadMinStress

A float specifying minimum stress. The default value is 0.001.

beadPerturbation

A Sets perturbation size for finite differences. The default value is 0.0001.

beadWidth

A SymbolicConstant specifying the Optimization product default or a float specifying the bead width. The default value is DEFAULT.

curveSmooth

A float specifying relative value to the middle element edge length such that normals in this area do not cross each other. The default value is 5.

filterRadius

A float specifying the filter radius. The default value is 4.

filterRadiusBy

A SymbolicConstant specifying the method used to define filter radius. Possible values are VALUE and REFERENCE. The default is VALUE.

flipNormalDir

A Boolean specifying whether the growth direction is along the normal direction of elements or opposite to the normal direction. The default value is OFF

frozenBoundaryConditionRegion

When nodes with boundary conditions are excluded from the optimization (*frozenBoundaryConditionRegions* = ON), you can specify that this exclusion apply to nodes throughout the model or only to those nodes from a specific region. Set this parameter to the SymbolicConstant MODEL to apply the freeze to the entire model, or set this parameter to a Region object to specify an individual region over which nodes with boundary conditions should be frozen. The default value is MODEL.

isSensCalcOnlyOnDesignNodes

A Boolean specifying whether to calculate the sensitivities only on design nodes or the whole model. The default value is ON

modeTrackingRegion

The SymbolicConstant MODEL or a Region object specifying the region to use for mode tracking. The default value is MODEL.

nodalMoveLimit

A Float specifying the maximum change in nodal displacement per design cycle. The default value is 0.1.

nodeSmooth

A SymbolicConstant specifying the Optimization product default or a float specifying the node smooth. The default value is DEFAULT.

nodeUpdateStrategy

A SymbolicConstant specifying the strategy for how the nodal displacements are updated in the method of moving asymptotes. Possible values are NORMAL, CONSERVATIVE, and AGGRESSIVE. The default value is CONSERVATIVE.

OPTIMIZATION COMMANDS

numTrackedModes

An Int specifying the number of modes included in mode tracking. The default value is 5.

updateShapeBasisVectors

A SymbolicConstant specifying whether to update shape basis vectors in the first design cycle or every design cycle. Possible values are EVERY_CYCLE and FIRST_CYCLE. The default value is EVERY_CYCLE.

Return value

A BeadTask object.

Exceptions

None.

36.2.2 setValues(...)

This method modifies the BeadTask object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BeadTask** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.2.3 Members

The BeadTask object has members with the same names and descriptions as the arguments to the **BeadTask** method.

In addition, the BeadTask object can have the following members:

designResponses

A repository of DesignResponse objects.

objectiveFunctions

A repository of ObjectiveFunction objects.

optimizationConstraints

A repository of OptimizationConstraint objects.

geometricRestrictions

A repository of GeometricRestriction objects.

36.3 CombinedTermDesignResponse object

The CombinedTermDesignResponse object defines a combined-term design response.

The CombinedTermDesignResponse object is derived from the DesignResponse object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].designResponses[name]
```

36.3.1 CombinedTermDesignResponse(...)

This method creates a CombinedTermDesignResponse object.

Path

```
mdb.models[name].optimizationTasks[name].CombinedTermDesignResponse
```

Required arguments

name

A String specifying the design response repository key.

terms

A sequence of Strings specifying the names of the design responses to combine.

Optional arguments

filterMaxRadius

None or a sequence of Floats specifying the maximum radius of influence used when *method* is FILTER. The default value is **None**.

filterExponent

A Float specifying the exponent used when *method* is FILTER. The default value is 1.0.

OPTIMIZATION COMMANDS

filterRadiusReduction

A Float specifying the reduction of the radius depending on surface bending, used when *method* is FILTER. The default value is 0.2.

highCutOff

None or a sequence of Floats specifying the upper bound of the vector value used when *method* is CUT_OFF. All values greater than the *highCutOff* are set to the *highCutOff* value. The default value is **None**.

lowCutOff

A Float specifying the lower bound of the vector value used when *method* is CUT_OFF. All values less than the *lowCutOff* are treated as 0. The default value is 0.0.

method

A SymbolicConstant specifying the method used to combine selected design responses. Possible values are:

- ABSOLUTE_DIFFERENCE
- ABSOLUTE_VALUE
- ADD
- COSINE
- CUT_OFF
- DELTA_OVER_1_ITERATION
- DELTA_OVER_2_ITERATIONS
- DELTA_OVER_3_ITERATIONS
- DELTA_OVER_4_ITERATIONS
- DELTA_OVER_5_ITERATIONS
- DELTA_OVER_6_ITERATIONS
- DIVIDE
- EXPONENTIAL
- FILTER
- INTEGER
- LOG
- MAXIMUM
- MINIMUM
- MULTIPLY
- NATURAL_LOG
- NEAREST_INTEGER
- NORM
- NORM_FIRST
- NTH_POWER

- NTH_ROOT
- SIGN
- SINE
- SQUARE_ROOT
- SUBTRACT
- TANGENT
- WEIGHTED_ADD

The default value is ADD.

weights

A sequence of Floats specifying the weights to apply to the list of design responses used when *method* is WEIGHTED_ADD. The default value is an empty sequence.

Return value

A CombinedTermDesignResponse object.

Exceptions

None.

36.3.2 setValues(...)

This method modifies the CombinedTermDesignResponse object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CombinedTermDesignResponse** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.3.3 Members

The CombinedTermDesignResponse object has members with the same names and descriptions as the arguments to the **CombinedTermDesignResponse** method.

36.4 DesignDirection object

The DesignDirection object defines a design direction geometric restriction.

The DesignDirection object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.4.1 DesignDirection(...)

This method creates a DesignDirection object.

Path

```
mdb.models[name].optimizationTasks[name].DesignDirection
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

masterPoint

None or a Region object specifying the master point used when *masterPointDetermination* is SPECIFY. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for assigning point priority. Possible values are MAXIMUM, MINIMUM, and SPECIFY. The default value is MAXIMUM.

movementRestriction

A SymbolicConstant specifying whether movement in the region should follow only the direction of the *masterPoint*, only the magnitude, or both the magnitude of the *masterPoint* and the directions specified by *u1*, *u2* and *u3*. Possible values are DIRECTION, MAGNITUDE, and VECTOR. The default value is VECTOR.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

u1

A Boolean specifying whether movement in the region should follow the *masterPoint* in the 1-direction. This is used when *movementRestriction* is VECTOR. The default value is ON.

u2

A Boolean specifying whether movement in the region should follow the *masterPoint* in the 2-direction. This is used when *movementRestriction* is VECTOR. The default value is ON.

u3

A Boolean specifying whether movement in the region should follow the *masterPoint* in the 3-direction. This is used when *movementRestriction* is VECTOR. The default value is ON.

Return value

A DesignDirection object.

Exceptions

None.

36.4.2 setValues(...)

This method modifies the DesignDirection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `DesignDirection` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.4.3 Members

The `DesignDirection` object has members with the same names and descriptions as the arguments to the `DesignDirection` method.

36.5 DesignResponse object

The `DesignResponse` object is the abstract base type for other `DesignResponse` objects. The `DesignResponse` object has no explicit constructor. The methods and members of the `DesignResponse` object are common to all objects derived from `DesignResponse`.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].designResponses[name]
```

36.5.1 Members

The `DesignResponse` object has the following member:

name

A String specifying the design response repository key.

36.6 DrillControl object

The `DrillControl` object defines a drill control geometric restriction.

The `DrillControl` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.6.1 DrillControl(...)

This method creates a DrillControl object.

Path

```
mdb.models[name].optimizationTasks[name].DrillControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

clientDirection

A VertexArray object of length 2 specifying the direction of the drill axis positioned at the csys origin. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If csys=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

drawAngle

A Float specifying the draw angle. The default value is 0.0.

masterPoint

None or a Region object specifying the master point used when *masterPointDetermination* is SPECIFY. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for assigning point priority. Possible values are MAXIMUM, MINIMUM, and SPECIFY. The default value is MAXIMUM.

OPTIMIZATION COMMANDS

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

undercutTolerance

A Float specifying the undercut tolerance. The default value is 0.0.

Return value

A DrillControl object.

Exceptions

None.

36.6.2 setValues(...)

This method modifies the DrillControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DrillControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.6.3 Members

The DrillControl object has members with the same names and descriptions as the arguments to the **DrillControl** method.

36.7 FixedRegion object

The FixedRegion object defines a fixed region geometric restriction.

The FixedRegion object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.7.1 FixedRegion(...)

This method creates a FixedRegion object.

Path

```
mdb.models[name].optimizationTasks[name].FixedRegion
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

u1

A Boolean specifying whether to fix the region in the 1-direction. The default value is OFF.

OPTIMIZATION COMMANDS

u2

A Boolean specifying whether to fix the region in the 2-direction. The default value is OFF.

u3

A Boolean specifying whether to fix the region in the 3-direction. The default value is OFF.

Return value

A FixedRegion object.

Exceptions

None.

36.7.2 setValues(...)

This method modifies the FixedRegion object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FixedRegion** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.7.3 Members

The FixedRegion object has members with the same names and descriptions as the arguments to the **FixedRegion** method.

36.8 FrozenArea object

The FrozenArea object defines a frozen area geometric restriction.

The FrozenArea object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.8.1 FrozenArea(...)

This method creates a FrozenArea object.

Path

```
mdb.models[name].optimizationTasks[name].FrozenArea
```

Required argument

name

A String specifying the geometric restriction repository key.

Optional argument

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Return value

A FrozenArea object.

Exceptions

None.

36.8.2 setValues(...)

This method modifies the FrozenArea object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FrozenArea** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.8.3 Members

The **FrozenArea** object has members with the same names and descriptions as the arguments to the **FrozenArea** method.

36.9 GeometricRestriction object

The **GeometricRestriction** object is the abstract base type for other **GeometricRestriction** objects. The **GeometricRestriction** object has no explicit constructor. The methods and members of the **GeometricRestriction** object are common to all objects derived from **GeometricRestriction**.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.9.1 Members

The **GeometricRestriction** object can have the following members:

name

A String specifying the geometric restriction repository key.

region

A **Region** object specifying the region to which the geometric restriction is applied. When used with a **TopologyTask**, there is no default value. When used with a **ShapeTask**, the default value is **MODEL**.

36.10 Growth object

The Growth object defines a growth geometric restriction.

The Growth object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.10.1 Growth(...)

This method creates a Growth object.

Path

```
mdb.models[name].optimizationTasks[name].Growth
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

growth

A Float specifying the maximum optimization displacement in the growth direction. Either *growth* or *shrink* or both must be specified. The default value is 0.0.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

shrink

A Float specifying the maximum optimization displacement in the shrink direction. Either *growth* or *shrink* or both must be specified. The default value is 0.0.

Return value

A Growth object.

Exceptions

None.

36.10.2 **setValues(...)**

This method modifies the Growth object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Growth** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.10.3 **Members**

The Growth object has members with the same names and descriptions as the arguments to the **Growth** method.

36.11 **LocalStopCondition object**

The LocalStopCondition object defines a local stop condition.

The LocalStopCondition object is derived from the StopCondition object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].stopConditions[name]
```

36.11.1 LocalStopCondition(...)

This method creates a LocalStopCondition object.

Path

```
mdb.models[name].optimizationTasks[name].LocalStopCondition
```

Required arguments

name

A String specifying the stop condition repository key.

referenceFactor

A Float specifying the factor used to modify the reference value.

Optional arguments

comparisonOperation

A SymbolicConstant specifying the operation used to compare the selected value to the reference value. Possible values are LESS_THAN, LESS_THAN_EQUAL, EQUAL, GREATER_THAN_EQUAL, and GREATER_THAN. The default value is LESS_THAN.

identifier

A SymbolicConstant specifying the variable identifier of the compared value. Possible values are:

- ABSOLUTE_GROWTH_MOVEMENT
- ABSOLUTE_SHRINK_MOVEMENT
- GROWTH_MOVEMENT
- SHRINK_MOVEMENT
- MOVEMENT
- TOTAL_ABSOLUTE_MOVEMENT
- EQUIV_STRESS
- FREE_TASK_REGION_EQUIV_STRESS
- RESTRICTED_TASK_REGION_EQUIV_STRESS
- SURFACE_POINT_EQUIV_STRESS
- TASK_REGION_EQUIV_STRESS

The default value is MOVEMENT.

OPTIMIZATION COMMANDS

identifierOperation

A SymbolicConstant specifying the operation used to evaluate values in the region. Possible values are MAXIMUM, MINIMUM, and SUM. The default value is MAXIMUM.

referenceDesignCycle

A SymbolicConstant specifying the iteration from which a value is compared to the reference value. Possible values are FIRST and PREVIOUS. The default value is PREVIOUS.

referenceOperation

A SymbolicConstant specifying the operation used to modify the reference value by the reference factor. Possible values are ADD, DIVIDE, MULTIPLY, and SUBTRACT. The default value is ADD.

region

The SymbolicConstant MODEL or a Region object specifying the region to which the stop condition is applied. The default value is MODEL.

Return value

A LocalStopCondition object.

Exceptions

None.

36.11.2 setValues(...)

This method modifies the LocalStopCondition object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **LocalStopCondition** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.11.3 Members

The `LocalStopCondition` object has members with the same names and descriptions as the arguments to the `LocalStopCondition` method.

36.12 ObjectiveFunction object

The `ObjectiveFunction` object defines the objective of the optimization.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].objectiveFunctions[name]
```

36.12.1 ObjectiveFunction(...)

This method creates an `ObjectiveFunction` object.

Path

```
mdb.models[name].optimizationTasks[name].ObjectiveFunction
```

Required arguments

name

A String specifying the objective function repository key.

objectives

An `OptimizationObjectiveArray` object.

Optional argument

target

A `SymbolicConstant` specifying the target of the objective function. Possible values are `MINIMIZE`, `MAXIMIZE`, and `MINIMIZE_MAXIMUM`. The default value is `MINIMIZE`.

Return value

An `ObjectiveFunction` object.

Exceptions

InvalidNameError and RangeError.

36.12.2 `setValues(...)`

This method modifies the `ObjectiveFunction` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ObjectiveFunction` method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

36.12.3 Members

The `ObjectiveFunction` object has members with the same names and descriptions as the arguments to the `ObjectiveFunction` method.

36.13 OptimizationConstraint object

The `OptimizationConstraint` object constrains an optimization from making changes to the topology of the model.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].optimizationConstraints[name]
```

36.13.1 OptimizationConstraint(...)

This method creates an OptimizationConstraint object.

Path

```
mdb.models[name].optimizationTasks[name].OptimizationConstraint
```

Required arguments

name

A String specifying the optimization constraint repository key.

designResponse

A String specifying the name of the design response to constrain.

restrictionValue

A Float specifying the value to which the design response should be constrained.

Optional argument

restrictionMethod

A SymbolicConstant specifying the method used to constrain the design response. Possible values are ABSOLUTE_EQUAL, ABSOLUTE_GREATER_THAN_EQUAL, ABSOLUTE_LESS_THAN_EQUAL, RELATIVE_EQUAL, RELATIVE_GREATER_THAN_EQUAL, and RELATIVE_LESS_THAN_EQUAL. The default value is ABSOLUTE_EQUAL.

Return value

An OptimizationConstraint object.

Exceptions

InvalidNameError and RangeError.

36.13.2 setValues(...)

This method modifies the OptimizationConstraint object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **OptimizationConstraint** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

36.13.3 Members

The OptimizationConstraint object has members with the same names and descriptions as the arguments to the **OptimizationConstraint** method.

36.14 OptimizationObjective object

An OptimizationObjective is an object used to define objectives in an objective function.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].objectiveFunctions[name] \
    .objectives[i]
```

36.14.1 Members

The OptimizationObjective object has the following members:

suppress

A Boolean specifying whether the objective is suppressed or not. The default value is OFF.

weight

A Float specifying the weight applied to the design response value. The default value is 1.0.

referenceValue

The SymbolicConstant DEFAULT or a Float specifying the reference value used in evaluating a design response. For topology optimization, DEFAULT> indicates the reference value is 0. For shape optimization, DEFAULT indicates the reference value is the nodal average. The default value is DEFAULT.

designResponse

A String specifying the name of the design response.

36.15 PenetrationCheck object

The PenetrationCheck object defines a penetration check geometric restriction.

The PenetrationCheck object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.15.1 PenetrationCheck(...)

This method creates a PenetrationCheck object.

Path

```
mdb.models[name].optimizationTasks[name].PenetrationCheck
```

Required arguments

name

A String specifying the geometric restriction repository key.

penetrationCheckRegion

A Region object specifying the penetration check region.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional argument

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

Return value

A `PenetrationCheck` object.

Exceptions

None.

36.15.2 `setValues(...)`

This method modifies the `PenetrationCheck` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `PenetrationCheck` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.15.3 Members

The `PenetrationCheck` object has members with the same names and descriptions as the arguments to the `PenetrationCheck` method.

36.16 ShapeDemoldControl object

The `ShapeDemoldControl` object defines a shape demold control geometric restriction.

The `ShapeDemoldControl` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.16.1 ShapeDemoldControl(...)

This method creates a ShapeDemoldControl object.

Path

```
mdb.models[name].optimizationTasks[name].ShapeDemoldControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

pullDirection

A VertexArray object of length 2 specifying the demold pull direction. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

collisionCheckRegion

The SymbolicConstant DEMOLD_REGION or a Region object specifying the collision check region. If the value is DEMOLD_REGION, then the value of *region* is used as both the demold region and the collision check region. The default value is DEMOLD_REGION.

csys

None or a DatumCsys object specifying the local coordinate system of the *pullDirection*. If *csys=None*, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

drawAngle

A Float specifying the draw angle. The default value is 0.0.

masterPointDetermination

A SymbolicConstant specifying the rule for assigning point priority. Possible values are MAXIMUM and MINIMUM. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

OPTIMIZATION COMMANDS

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

undercutTolerance

A Float specifying the undercut tolerance. The default value is 0.0.

Return value

A ShapeDemoldControl object.

Exceptions

None.

36.16.2 setValues(...)

This method modifies the ShapeDemoldControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShapeDemoldControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.16.3 Members

The ShapeDemoldControl object has members with the same names and descriptions as the arguments to the **ShapeDemoldControl** method.

36.17 ShapeMemberSize object

The ShapeMemberSize object defines a shape member size geometric restriction.

The ShapeMemberSize object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.17.1 ShapeMemberSize(...)

This method creates a ShapeMemberSize object.

Path

```
mdb.models[name].optimizationTasks[name].ShapeMemberSize
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

maxThickness

A Float specifying the maximum thickness. The default value is 0.0.

minThickness

A Float specifying the minimum thickness. The default value is 0.0.

sizeRestriction

A SymbolicConstant specifying whether to restrict the minimum or maximum thickness. Possible values are MAXIMUM and MINIMUM. The default value is MINIMUM.

Return value

A `ShapeMemberSize` object.

Exceptions

None.

36.17.2 `setValues(...)`

This method modifies the `ShapeMemberSize` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ShapeMemberSize` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.17.3 Members

The `ShapeMemberSize` object has members with the same names and descriptions as the arguments to the `ShapeMemberSize` method.

36.18 `ShapePlanarSymmetry` object

The `ShapePlanarSymmetry` object defines a shape planar symmetry geometric restriction.

The `ShapePlanarSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.18.1 ShapePlanarSymmetry(...)

This method creates a ShapePlanarSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].ShapePlanarSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

clientDirection

A VertexArray object of length 2 specifying the vector positioned at the csys origin which is normal to the symmetry plane. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for determining the master node. Possible values are MAXIMUM and MINIMUM. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

Return value

A ShapePlanarSymmetry object.

Exceptions

None.

36.18.2 setValues(...)

This method modifies the ShapePlanarSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShapePlanarSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.18.3 Members

The ShapePlanarSymmetry object has members with the same names and descriptions as the arguments to the **ShapePlanarSymmetry** method.

36.19 ShapePointSymmetry object

The ShapePointSymmetry object defines a shape point symmetry geometric restriction.

The ShapePointSymmetry object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.19.1 ShapePointSymmetry(...)

This method creates a ShapePointSymmetry object.

Path

```
mdb.models [name] .optimizationTasks [name] .ShapePointSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the symmetry point represented as the origin of a local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for determining the master node. Possible values are MAXIMUM and MINIMUM. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

Return value

A ShapePointSymmetry object.

Exceptions

None.

36.19.2 **setValues(...)**

This method modifies the `ShapePointSymmetry` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShapePointSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.19.3 **Members**

The `ShapePointSymmetry` object has members with the same names and descriptions as the arguments to the **ShapePointSymmetry** method.

36.20 **ShapeRotationalSymmetry object**

The `ShapeRotationalSymmetry` object defines a shape rotational symmetry geometric restriction.

The `ShapeRotationalSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.20.1 ShapeRotationalSymmetry(...)

This method creates a ShapeRotationalSymmetry object.

Path

```
mdb.models [name] .optimizationTasks [name] .ShapeRotationalSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

clientDirection

A VertexArray object of length 2 specifying the vector positioned at the csys origin, used as the axis of symmetry. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

angle

A Float specifying the segment size of the repeating pattern in degrees. If the *angle* value is 0, no repeating pattern is created. The default value is 0.0.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

masterPoint

None or a Region object specifying the master point used when *masterPointDetermination* is SPECIFY. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for determining the master node. Possible values are MAXIMUM, MINIMUM, and SPECIFY. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

OPTIMIZATION COMMANDS

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

Return value

A ShapeRotationalSymmetry object.

Exceptions

None.

36.20.2 setValues(...)

This method modifies the ShapeRotationalSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShapeRotationalSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.20.3 Members

The ShapeRotationalSymmetry object has members with the same names and descriptions as the arguments to the **ShapeRotationalSymmetry** method.

36.21 ShapeTask object

The ShapeTask object defines a shape task.

The ShapeTask object is derived from the OptimizationTask object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name]
```

36.21.1 ShapeTask(...)

This method creates a ShapeTask object.

Path

```
mdb.models[name].ShapeTask
```

Required arguments

name

A String specifying the optimization task repository key.

Optional arguments

absoluteStepSizeControl

A SymbolicConstant specifying whether to control the permitted absolute step size by the average optimization displacement or minimum optimization displacement. Possible values are MINIMUM and AVERAGE. The default value is MINIMUM.

activateDurability

A boolean specifying whether or not the durability approach of optimization is turned on. The default value is ON.

additionalDurabilityFiles

A String specifying the path of additional files pertaining to durability optimization. Only valid if the *activateDurability* argument is ON.

constrainedLaplacianConvergenceLevel

A SymbolicConstant specifying the constrained Laplacian convergence level. Possible values are NORMAL, CONSERVATIVE, and AGGRESSIVE. The default value is NORMAL.

curvatureSmoothingEdgeLength

A Float specifying the edge length for the movement vector. The default value is 5.0.

durabilityInputfile

A string specifying the path of the input file. Only valid if the *activateDurability* argument is ON and is a required argument in that case.

OPTIMIZATION COMMANDS

durabilitySolver

A String specifying the type of solver for durability optimization. Possible values are: FE_SAFE, FEMFAT, FALANCS, MSC_FATIGUE, FE_FATIGUE, DESIGN_LIFE, CUSTOM, FEMSITE. The default value is FE_SAFE. Only valid if the *activateDurability* argument is ON.

equalityConstraintTolerance

A Float specifying the equality constraint tolerance. The default value is 10^{-3} .

featureRecognitionAngle

A Float specifying the mesh smoothing feature recognition angle for edges and corners. The default value is 30.0.

filterExponent

A Float specifying the weight depending on the radius, used when *filterMaxRadius* is specified. The default value is 1.0.

filterMaxRadius

None or a Float specifying the maximum influence radius for equivalent stress. The default value is **None**.

filterRadiusReduction

None or a Float specifying the reduction of the radius depending on surface bending, used when *filterMaxRadius* is specified. The default value is **None**.

firstCycleDeletedVolumeTechnique

A SymbolicConstant specifying the method of specifying volume that can be removed immediately in the first design cycle. Possible values are OFF, PERCENTAGE, and ABSOLUTE. The default value is OFF.

freezeBoundaryConditionRegions

A Boolean specifying whether to exclude nodes with boundary conditions from the optimization. The default value is OFF.

frozenBoundaryConditionRegion

The SymbolicConstant MODEL or a Region object specifying the region in which to freeze boundary condition regions, or the SymbolicConstant MODEL, used with *freezeBoundaryConditionRegions*. The default value is MODEL.

geometricRestrictionEvaluationFrequency

A SymbolicConstant specifying the frequency of evaluating geometric restrictions during mesh smoothing. Possible values are LOW, MEDIUM, and HIGH. The default value is LOW.

growthScaleFactor

A Float specifying the the scale factor to apply to optimization displacements for nodes with growth. The default value is 1.0.

haltUponViolation

A Boolean specifying whether to halt the optimization if quality criteria are not satisfied. The default value is OFF.

layerReferenceRegion

None or a Region object specifying the region specifying the first node layer for mesh smoothing, used when *meshSmoothingRegionMethod* is TASK_REGION_LAYERS. The default value is **None**.

meshSmoothingRegionMethod

A SymbolicConstant specifying the method used to determine the mesh smoothing region. The REGION value uses the *smoothingRegion*. The NUMBER_OF_LAYERS value uses the *layerReferenceRegion*. The TASK_REGION_LAYERS value will smooth six layers using the task region. Possible values are TASK_REGION_LAYERS, REGION, and NUMBER_OF_LAYERS. The default value is TASK_REGION_LAYERS.

meshSmoothingStrategy

A SymbolicConstant specifying the method smoothing strategy. Possible values are CONSTRAINED_LAPLACIAN and LOCAL_GRADIENT. The default value is CONSTRAINED_LAPLACIAN.

midsideInterpolation

A SymbolicConstant specifying the approach used when treating midside node positions during optimization. POSITIONS indicates midside node positions are interpolated linearly by position. OPTIMIZATION_DISPLACEMENT indicates they are interpolated by optimization displacement of corner nodes. Possible values are POSITIONS and OPTIMIZATION_DISPLACEMENT. The default value is POSITIONS.

numFreeNodeLayers

The SymbolicConstant FIX_NONE or an Int specifying the number of node layers adjoining the task region to remain free during mesh smoothing. A value of 0 indicates that no layers are free and all layers are fixed. The default value is 0.

numSmoothedElementLayers

None or an Int specifying the number of layers for mesh smoothing when *meshSmoothingRegionMethod* is NUMBER_OF_LAYERS. The default value is **None**.

presumeFeasibleBCRegionAtStart

A Boolean specifying whether to ignore automatically frozen boundary condition regions in the first design cycle. This is used with *freezeBoundaryConditionRegions*. The default value is ON.

quadMaxAngle

A Float specifying the maximum angle for quad elements during mesh smoothing. The default value is 160.0.

quadMinAngle

A Float specifying the minimum angle for quad elements during mesh smoothing. The default value is 20.0.

quadSkew

A Float specifying the skew angle for quad elements during mesh smoothing, used with *reportQualityViolation*. The default value is 30.0.

OPTIMIZATION COMMANDS

quadTaper

A Float specifying the taper for quad elements during mesh smoothing, used with *reportQualityViolation*. The default value is 0.5.

region

The SymbolicConstant MODEL or a Region object specifying the region to which the optimization task is applied. The default value is MODEL.

reportPoorQualityElements

A Boolean specifying whether to report poor quality elements during mesh smoothing. The default value is OFF.

reportQualityViolation

A Boolean specifying whether to report a quality criteria violation during mesh smoothing. The default value is OFF.

shrinkScaleFactor

A Float specifying the scale factor to apply to optimization displacements for nodes with shrinkage. The default value is 1.0.

smoothingRegion

None or a Region object specifying the mesh smoothing region, used when *meshSmoothingRegionMethod* is REGION. The default value is **None**.

targetMeshQuality

A SymbolicConstant specifying the target mesh quality for mesh smoothing. Possible values are NONE, LOW, MEDIUM, and HIGH. The default value is LOW.

tetAspectRatio

A Float specifying the tet element aspect ratio during mesh smoothing. The default value is 100.0.

tetMaxAspect

A Float specifying the maximum tet element aspect ratio during mesh smoothing. The default value is 8.0.

tetMinAspect

A Float specifying the minimum tet element aspect ratio during mesh smoothing. The default value is 0.222.

tetSkew

A Float specifying the tet element skew value during mesh smoothing. The default value is 100.0.

triMaxAngle

A Float specifying the tri element maximum angle during mesh smoothing. The default value is 140.0.

triMinAngle

A Float specifying the tri element maximum angle during mesh smoothing. The default value is 20.0.

updateShapeBasisVectors

A SymbolicConstant specifying whether to update shape basis vectors in the first design cycle or every design cycle. Possible values are EVERY_CYCLE and FIRST_CYCLE. The default value is EVERY_CYCLE.

Return value

A ShapeTask object.

Exceptions

None.

36.21.2 setValues(...)

This method modifies the ShapeTask object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ShapeTask** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.21.3 Members

The ShapeTask object has members with the same names and descriptions as the arguments to the **ShapeTask** method.

In addition, the ShapeTask object can have the following members:

designResponses

A repository of DesignResponse objects.

objectiveFunctions

A repository of ObjectiveFunction objects.

optimizationConstraints

A repository of OptimizationConstraint objects.

geometricRestrictions

A repository of GeometricRestriction objects.

stopConditions

A repository of StopCondition objects.

36.22 SingleTermDesignResponse object

The SingleTermDesignResponse object defines a single-term design response.

The SingleTermDesignResponse object is derived from the DesignResponse object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].designResponses[name]
```

36.22.1 SingleTermDesignResponse(...)

This method creates a SingleTermDesignResponse object.

Path

```
mdb.models[name].optimizationTasks[name].SingleTermDesignResponse
```

Required arguments

name

A String specifying the design response repository key.

identifier

A String specifying the name of the variable identifier.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

drivingRegion

None or a sequence of Floats specifying the driving region used when *identifier* is an internal nodal variable. The default value is **None**.

operation

A SymbolicConstant specifying the operation used on values in the region. Possible values are MAXIMUM, MINIMUM, and SUM. The default value is SUM.

region

The SymbolicConstant MODEL or a Region object specifying the region of the design response variable. The default value is MODEL.

shellLayer

A SymbolicConstant specifying the location used for shell layer values. Possible values are BOTTOM, MAXIMUM, MIDDLE, MINIMUM, and TOP. The default value is MAXIMUM.

stepOperation

A SymbolicConstant specifying the operation used on values across steps and load cases. Possible values are MAXIMUM, MINIMUM, and SUM. The default value is SUM.

stepOptions

A StepOptionArray object.

Return value

A SingleTermDesignResponse object.

Exceptions

None.

36.22.2 setValues(...)

This method modifies the SingleTermDesignResponse object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SingleTermDesignResponse** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.22.3 Members

The `SingleTermDesignResponse` object has members with the same names and descriptions as the arguments to the `SingleTermDesignResponse` method.

36.23 SizingClusterAreas object

The `SizingClusterAreas` object defines a sizing cluster areas geometric restriction.

The `SizingClusterAreas` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.23.1 SizingClusterAreas(...)

This method creates a `SizingClusterAreas` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingClusterAreas
```

Required argument

name

A String specifying the geometric restriction repository key.

regions

Tuple of `Region` objects specifying the regions to which the geometric restriction is applied.

Return value

A `SizingClusterAreas` object.

Exceptions

None.

36.23.2 **setValues(...)**

This method modifies the `SizingClusterAreas` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SizingClusterAreas** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.23.3 **Members**

The `SizingClusterAreas` object has members with the same names and descriptions as the arguments to the **SizingClusterAreas** method.

36.24 **SizingCyclicSymmetry object**

The `SizingCyclicSymmetry` object defines a sizing cyclic symmetry geometric restriction.

The `SizingCyclicSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.24.1 **SizingCyclicSymmetry(...)**

This method creates a `SizingCyclicSymmetry` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingCyclicSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

translation

A Float specifying the translation distance.

Optional arguments

axis

A SymbolicConstant specifying the translation direction defined along an axis positioned at the csys origin. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A SizingCyclicSymmetry object.

Exceptions

None.

36.24.2 setValues(...)

This method modifies the SizingCyclicSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SizingCyclicSymmetry` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.24.3 Members

The `SizingCyclicSymmetry` object has members with the same names and descriptions as the arguments to the `SizingCyclicSymmetry` method.

36.25 SizingFrozenArea object

The `SizingFrozenArea` object defines a sizing frozen area geometric restriction.

The `SizingFrozenArea` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.25.1 SizingFrozenArea (...)

This method creates a `SizingFrozenArea` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingFrozenArea
```

Required argument

name

A String specifying the geometric restriction repository key.

OPTIMIZATION COMMANDS

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

None.

Return value

A SizingFrozenArea object.

Exceptions

None.

36.25.2 setValues(...)

This method modifies the SizingFrozenArea object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SizingFrozenArea** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.25.3 Members

The SizingFrozenArea object has members with the same names and descriptions as the arguments to the **SizingFrozenArea** method.

36.26 SizingMemberSize object

The SizingMemberSize object defines a sizing member size geometric restriction.

The SizingMemberSize object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.26.1 SizingMemberSize(...)

This method creates a SizingMemberSize object.

Path

```
mdb.models[name].optimizationTasks[name].SizingMemberSize
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

minWidth

A Float specifying the min width.

36.26.2 setValues(...)

This method modifies the sizingMemberSize object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **sizingMemberSize** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.26.3 Members

The `sizingMemberSize` object has members with the same names and descriptions as the arguments to the `sizingMemberSize` method.

36.27 SizingPlanarSymmetry object

The `SizingPlanarSymmetry` object defines a sizing planar symmetry geometric restriction. The `SizingPlanarSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.27.1 SizingPlanarSymmetry(...)

This method creates a `SizingPlanarSymmetry` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingPlanarSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are `AXIS_1`, `AXIS_2`, and `AXIS_3`. The default value is `AXIS_1`.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A SizingPlanarSymmetry object.

Exceptions

None.

36.27.2 setValues(...)

This method modifies the sizingPlanarSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **sizingPlanarSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.27.3 Members

The sizingPlanarSymmetry object has members with the same names and descriptions as the arguments to the **sizingPlanarSymmetry** method.

36.28 SizingPointSymmetry object

The SizingPointSymmetry object defines a sizing point symmetry geometric restriction.

The `SizingPointSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.28.1 SizingPointSymmetry(...)

This method creates a `SizingPointSymmetry` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingPointSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

csys

None or a `DatumCsys` object specifying the position of the symmetry point defined as the origin of a local coordinate system. If `csys=None`, the global coordinate system is used. When this member is queried, it returns an `Int`. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A `SizingPointSymmetry` object.

Exceptions

None.

36.28.2 setValues(...)

This method modifies the `SizingPointSymmetry` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SizingPointSymmetry` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.28.3 Members

The `SizingPointSymmetry` object has members with the same names and descriptions as the arguments to the `SizingPointSymmetry` method.

36.29 SizingRotationalSymmetry object

The `SizingRotationalSymmetry` object defines a sizing rotational symmetry geometric restriction. The `SizingRotationalSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.29.1 SizingRotationalSymmetry(...)

This method creates a `SizingRotationalSymmetry` object.

Path

```
mdb.models[name].optimizationTasks[name].SizingRotationalSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

angle

A Float specifying the repeating segment size, an angle in degrees.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are `AXIS_1`, `AXIS_2`, and `AXIS_3`. The default value is `AXIS_1`.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A SizingRotationalSymmetry object.

Exceptions

None.

36.29.2 setValues(...)

This method modifies the SizingRotationalSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SizingRotationalSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.29.3 Members

The `SizingRotationalSymmetry` object has members with the same names and descriptions as the arguments to the `SizingRotationalSymmetry` method.

36.30 SizingTask object

The `SizingTask` object defines a Sizing task.

The `SizingTask` object is derived from the `OptimizationTask` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name]
```

36.30.1 SizingTask(...)

This method creates a `SizingTask` object.

Path

```
mdb.models[name].SizingTask
```

Required argument

name

A String specifying the optimization task repository key.

Optional arguments

elementthicknessDeltaStopCriteria

A Float specifying the stop criteria based upon the change in element thickness. The default value is 0.5×10^{-2} .

OPTIMIZATION COMMANDS

freezeBoundaryConditionRegions

A Boolean specifying whether to exclude elements with boundary conditions from the optimization. The default value is OFF.

freezeLoadRegions

A Boolean specifying whether to exclude elements with loads and elements with loaded nodes from the optimization. The default value is ON.

modeTrackingRegion

The SymbolicConstant MODEL or a Region object specifying the region to use for mode tracking. The default value is MODEL.

numFulfilledStopCriteria

An Int specifying the number of stop criteria. The default value is 2.

numTrackedModes

An Int specifying the number of modes included in mode tracking. The default value is 5.

objectiveFunctionDeltaStopCriteria

A Float specifying the stop criteria based upon the change in objective function. The default value is 0.001.

stopCriteriaDesignCycle

An Int specifying the first design cycle used to evaluate convergence criteria. The default value is 4.

thicknessMoveLimit

A Float specifying the maximum change in thickness per design cycle. The default value is 0.25.

thicknessUpdateStrategy

A SymbolicConstant specifying the strategy for how the thickness is updated in the method of moving asymptotes. Possible values are NORMAL, CONSERVATIVE, and AGGRESSIVE. The default value is NORMAL.

Return value

A SizingTask object.

Exceptions

None.

36.30.2 setValues(...)

This method modifies the SizingTask object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SizingTask** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.30.3 Members

The SizingTask object has members with the same names and descriptions as the arguments to the **SizingTask** method.

In addition, the SizingTask object can have the following members:

designResponses

A repository of DesignResponse objects.

objectiveFunctions

A repository of ObjectiveFunction objects.

optimizationConstraints

A repository of OptimizationConstraint objects.

geometricRestrictions

A repository of GeometricRestriction objects.

stopConditions

A repository of StopCondition objects.

36.31 SizingThicknessControl object

The SizingThicknessControl object defines a sizing thickness control geometric restriction.

The SizingThicknessControl object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.31.1 SizingThicknessControl(...)

This method creates a SizingThicknessControl object.

Path

```
mdb.models[name].optimizationTasks[name].SizingThicknessControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

thicknessRestrictionBy

A SymbolicConstant specifying whether to define thickness restriction by value or fraction.
Possible values are VALUE and FRACTION.

Optional arguments

Note: You must specify either *maxThickness* or *minThickness*.

maxThickness

A Float specifying the maximum thickness.

minThickness

A Float specifying the minimum thickness.

Return value

A SizingThicknessControl object.

Exceptions

None.

36.31.2 setValues(...)

This method modifies the SizingThicknessControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SizingThicknessControl` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.31.3 Members

The `SizingThicknessControl` object has members with the same names and descriptions as the arguments to the `SizingThicknessControl` method.

36.32 SlideRegionControl object

The `SlideRegionControl` object defines a slide region control geometric restriction.

The `SlideRegionControl` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.32.1 SlideRegionControl(...)

This method creates a `SlideRegionControl` object.

Path

```
mdb.models[name].optimizationTasks[name].SlideRegionControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

OPTIMIZATION COMMANDS

clientDirection

A VertexArray object of length 2 specifying the axis of revolution. Instead of through a Vertex, each point may be specified through a tuple of coordinates. This is used when *approach* is TURN.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

approach

A SymbolicConstant specifying the restriction approach. The SymbolicConstant FREE_FORM indicates a free-form slide region, and the SymbolicConstant TURN indicates that the restriction should conserve a turnable surface. Possible values are FREE_FORM and TURN. The default value is FREE_FORM.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. This is used when *approach* is TURN. The default value is **None**.

freeFormRegion

None or a Region object specifying the free-form region. This is used when *approach* is FREE_FORM. The default value is **None**.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

revolvedRegion

None or a Region object specifying the region to revolve into a slide region. This is used when *approach* is TURN. The default value is **None**.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. This is used when *approach* is TURN. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. This is used when *approach* is TURN. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. This is used when *approach* is TURN. The default value is 0.01.

Return value

A SlideRegionControl object.

Exceptions

None.

36.32.2 setValues(...)

This method modifies the SlideRegionControl object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SlideRegionControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.32.3 Members

The SlideRegionControl object has members with the same names and descriptions as the arguments to the **SlideRegionControl** method.

36.33 StampControl object

The StampControl object defines a stamp control geometric restriction.

The StampControl object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.33.1 StampControl(...)

This method creates a StampControl object.

Path

```
mdb.models[name].optimizationTasks[name].StampControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

clientDirection

A VertexArray object of length 2 specifying the stamping direction. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

drawAngle

A Float specifying the draw angle. The default value is 0.0.

masterPoint

None or a Region object specifying the master point used when *masterPointDetermination* is SPECIFY. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for assigning point priority. Possible values are MAXIMUM, MINIMUM, and SPECIFY. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

undercutTolerance

A Float specifying the undercut tolerance. The default value is 0.0.

Return value

A StampControl object.

Exceptions

None.

36.33.2 setValues(...)

This method modifies the StampControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StampControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.33.3 Members

The StampControl object has members with the same names and descriptions as the arguments to the **StampControl** method.

36.34 StepOption object

A StepOption is an object used to define step options in a design response.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].designResponses[name] \
    .stepOptions[i]
```

36.34.1 Members

The StepOption object can have the following members:

lowerMode

The SymbolicConstant ALL or an Int specifying the lower mode in the range of modes to consider in the step. *lowerMode* is ignored for steps without modes. The default value is ALL.

upperMode

The SymbolicConstant ALL or an Int specifying the upper mode in the range of modes to consider in the step. *upperMode* is ignored for steps without modes. The default value is ALL.

loadCase

The SymbolicConstant ALL or a String specifying the name of the load case. *loadCase* is ignored when the specified *step* does not contain a load case. The default value is ALL.

step

The SymbolicConstant ALL or a String specifying the name of the step. The default value is ALL.

model

A string specifying the name of the model from which the steps are supposed to be used in the design response. Specify only if the steps are not from the current model.

36.35 StopCondition object

The StopCondition object is the abstract base type for other StopCondition objects. The StopCondition object has no explicit constructor. The methods and members of the StopCondition object are common to all objects derived from StopCondition.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].stopConditions[name]
```

36.35.1 Members

The StopCondition object can have the following members:

name

A String specifying the stop condition repository key.

region

The SymbolicConstant MODEL or a Region object specifying the region to which the stop condition is applied. The default value is MODEL.

36.36 TopologyCyclicSymmetry object

The TopologyCyclicSymmetry object defines a topology cyclic symmetry geometric restriction.

The TopologyCyclicSymmetry object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.36.1 TopologyCyclicSymmetry(...)

This method creates a TopologyCyclicSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyCyclicSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

translation

A Float specifying the translation distance.

Optional arguments

axis

A SymbolicConstant specifying the translation direction defined along an axis positioned at the csys origin. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A TopologyCyclicSymmetry object.

Exceptions

None.

36.36.2 setValues(...)

This method modifies the TopologyCyclicSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TopologyCyclicSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.36.3 Members

The TopologyCyclicSymmetry object has members with the same names and descriptions as the arguments to the **TopologyCyclicSymmetry** method.

36.37 TopologyDemoldControl object

The TopologyDemoldControl object defines a topology demold control geometric restriction.

The TopologyDemoldControl object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.37.1 TopologyDemoldControl(...)

This method creates a TopologyDemoldControl object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyDemoldControl
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system of the *pullDirection*. If *csys=None*, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

draftAngle

A Float specifying the draft angle. The default value is 0.0.

OPTIMIZATION COMMANDS

collisionCheckRegion

The SymbolicConstant DEMOLD_REGION or a Region object specifying the collision check region. If the value is DEMOLD_REGION, then the value of *region* is used as both the demold region and the collision check region. The default value is DEMOLD_REGION.

pointRegion

A Region object specifying the point on a plane perpendicular to the pull direction, used to specify the central plane when *technique* is POINT.

pullDirection

A VertexArray object of length 2 specifying the demold pull direction. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

technique

A SymbolicConstant specifying the demold technique. Possible values are AUTO, AUTO_TIGHT, POINT, SURFACE, and STAMP. The default value is AUTO.

Return value

A TopologyDemoldControl object.

Exceptions

None.

36.37.2 setValues(...)

This method modifies the TopologyDemoldControl object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TopologyDemoldControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.37.3 Members

The `TopologyDemoldControl` object has members with the same names and descriptions as the arguments to the `TopologyDemoldControl` method.

36.38 TopologyMemberSize object

The `TopologyMemberSize` object defines a topology member size geometric restriction.

The `TopologyMemberSize` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.38.1 TopologyMemberSize(...)

This method creates a `TopologyMemberSize` object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyMemberSize
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A `Region` object specifying the region to which the geometric restriction is applied. When used with a `TopologyTask`, there is no default value. When used with a `ShapeTask`, the default value is `MODEL`.

Optional arguments

maxThickness

A Float specifying the maximum thickness. The default value is 0.0.

minThickness

A Float specifying the minimum thickness. The default value is 0.0.

separation

A Float specifying the minimum gap. The default value is 0.0.

sizeRestriction

A SymbolicConstant specifying whether to restrict the minimum or maximum thickness or an envelope of both. Possible values are ENVELOPE, MAXIMUM, and MINIMUM. The default value is MINIMUM.

Return value

A TopologyMemberSize object.

Exceptions

None.

36.38.2 setValues(...)

This method modifies the TopologyMemberSize object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TopologyMemberSize** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.38.3 Members

The TopologyMemberSize object has members with the same names and descriptions as the arguments to the **TopologyMemberSize** method.

36.39 TopologyPlanarSymmetry object

The TopologyPlanarSymmetry object defines a topology planar symmetry geometric restriction.

The TopologyPlanarSymmetry object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.39.1 TopologyPlanarSymmetry(...)

This method creates a TopologyPlanarSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyPlanarSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If csys=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A TopologyPlanarSymmetry object.

Exceptions

None.

36.39.2 `setValues(...)`

This method modifies the `TopologyPlanarSymmetry` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `TopologyPlanarSymmetry` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.39.3 Members

The `TopologyPlanarSymmetry` object has members with the same names and descriptions as the arguments to the `TopologyPlanarSymmetry` method.

36.40 `TopologyPointSymmetry` object

The `TopologyPointSymmetry` object defines a topology point symmetry geometric restriction.

The `TopologyPointSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.40.1 TopologyPointSymmetry(...)

This method creates a TopologyPointSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyPointSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

csys

None or a DatumCsys object specifying the position of the symmetry point defined as the origin of a local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A TopologyPointSymmetry object.

Exceptions

None.

36.40.2 setValues(...)

This method modifies the TopologyPointSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `TopologyPointSymmetry` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.40.3 Members

The `TopologyPointSymmetry` object has members with the same names and descriptions as the arguments to the `TopologyPointSymmetry` method.

36.41 TopologyRotationalSymmetry object

The `TopologyRotationalSymmetry` object defines a topology rotational symmetry geometric restriction. The `TopologyRotationalSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.41.1 TopologyRotationalSymmetry(...)

This method creates a `TopologyRotationalSymmetry` object.

Path

```
mdb.models[name].optimizationTasks[name].TopologyRotationalSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

angle

A Float specifying the repeating segment size, an angle in degrees.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

ignoreFrozenArea

A Boolean specifying whether to ignore frozen areas. The default value is OFF.

Return value

A TopologyRotationalSymmetry object.

Exceptions

None.

36.41.2 setValues(...)

This method modifies the TopologyRotationalSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TopologyRotationalSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.41.3 Members

The `TopologyRotationalSymmetry` object has members with the same names and descriptions as the arguments to the `TopologyRotationalSymmetry` method.

36.42 TopologyTask object

The `TopologyTask` object defines a topology task.

The `TopologyTask` object is derived from the `OptimizationTask` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name]
```

36.42.1 TopologyTask(...)

This method creates a `TopologyTask` object.

Path

```
mdb.models[name].TopologyTask
```

Required argument

name

A String specifying the optimization task repository key.

Optional arguments

algorithm

A `SymbolicConstant` specifying the optimization task algorithm. Possible values are `GENERAL_OPTIMIZATION` and `CONDITION_BASED_OPTIMIZATION`. The default value is `GENERAL_OPTIMIZATION`.

densityMoveLimit

A Float specifying the maximum density change per design cycle. The default value is 0.25.

densityUpdateStrategy

A SymbolicConstant specifying the strategy for how the densities are updated in the method of moving asymptotes. Possible values are NORMAL, CONSERVATIVE, and AGGRESSIVE. The default value is NORMAL.

elementDensityDeltaStopCriteria

A Float specifying the stop criteria based upon the change in element densities. The default value is 0.5×10^{-2} .

filterRadius

None or a Float specifying the mesh filter radius for mesh independence and minimum size. The default value is **None**.

firstCycleDeletedVolume

A Float specifying the volume that can be removed immediately in the first design cycle. The default value is 5.0.

firstCycleDeletedVolumeTechnique

A SymbolicConstant specifying the method of quantifying volume that can be removed immediately in the first design cycle. Possible values are OFF, PERCENTAGE, and ABSOLUTE. The default value is OFF.

freezeBoundaryConditionRegions

A Boolean specifying whether to exclude elements with boundary conditions from the optimization. The default value is OFF.

freezeLoadRegions

A Boolean specifying whether to exclude elements with loads and elements with loaded nodes from the optimization. The default value is ON.

frequencySpectrumWeight

A Float specifying the weighting factor for frequency spectrum peaks. The default value is 6.0.

initialDensity

A SymbolicConstant specifying the Optimization product default or a float specifying the initial density. The default value is DEFAULT.

materialInterpolationPenalty

A Float specifying the penalty factor for the material interpolation technique. The default value is 3.0.

materialInterpolationTechnique

A SymbolicConstant specifying the material interpolation technique: optimization product default, solid isotropic material with penalization, or rational approximation of material properties. Possible values are DEFAULT, SIMP, and RAMP. The default value is DEFAULT.

maxDensity

A Float specifying the maximum density in the density update. The default value is 1.0.

OPTIMIZATION COMMANDS

minDensity

A Float specifying the minimum density in the density update. The default value is 10^{-3} .

modeTrackingRegion

The SymbolicConstant MODEL or a Region object specifying the region to use for mode tracking. The default value is MODEL.

numDesignCycles

An Int specifying the number of design cycles permitted when *stepSize* is DYNAMIC. The default value is 15.

numFulfilledStopCriteria

An Int specifying the number of stop criteria. The default value is 2.

numTrackedModes

An Int specifying the number of modes included in mode tracking. The default value is 5.

objectiveFunctionDeltaStopCriteria

A Float specifying the stop criteria based upon the change in objective function. The default value is 10^{-3} .

region

The SymbolicConstant MODEL or a Region object specifying the region to which the optimization task is applied. The default value is MODEL.

softDeletionMethod

A SymbolicConstant specifying the method used when *softDeletionRegion* is specified. The STANDARD method avoids creating disconnected regions. The AGGRESSIVE method only considers the *softDeletionThreshold*. The MAX_SHEAR_STRAIN, MAX_ELASTOPLASTIC_STRAIN and VOLUME_COMPRESSION methods do not need the *softDeletionRadius*. Possible values are STANDARD, AGGRESSIVE, MAX_SHEAR_STRAIN, MIN_PRINCIPAL_STRAIN, MAX_ELASTOPLASTIC_STRAIN and VOLUME_COMPRESSION. The default value is STANDARD.

softDeletionRadius

A Float specifying the radius to use when considering neighboring soft elements to delete. The default value is 0.0.

softDeletionRegion

None or a Region object specifying the region in which the soft elements should be deleted during optimization. The default value is **None**.

softDeletionThreshold

A Float specifying the relative material density value used to identify soft elements. Those with values below the threshold are considered for removal. For STANDARD and AGGRESSIVE methods positive values are accepted and the default value is 0.05. For MAX_SHEAR_STRAIN and MAX_ELASTOPLASTIC_STRAIN methods positive values are accepted whereas for MIN_PRINCIPAL_STRAIN and VOLUME_COMPRESSION methods negative values are accepted.

stepSize

A SymbolicConstant specifying the size of the increment for volume modification. Possible values are DYNAMIC, VERY_SMALL, SMALL, MODERATE, MEDIUM, and LARGE. The default value is MEDIUM.

stiffnessMassDamping

The SymbolicConstant AVERAGE_EDGE_LENGTH or a Float specifying the stiffness mass damping for the task region. The default value is AVERAGE_EDGE_LENGTH.

stopCriteriaDesignCycle

An Int specifying the first design cycle used to evaluate convergence criteria. The default value is 4.

structuralMassDamping

None or a Float specifying the structural mass damping for the task region. The default value is **None**.

viscousMassDamping

None or a Float specifying the viscous mass damping for the task region. The default value is **None**.

viscousStiffnessDamping

None or a Float specifying the viscous stiffness damping for the task region. The default value is **None**.

Return value

A TopologyTask object.

Exceptions

None.

36.42.2 setValues(...)

This method modifies the TopologyTask object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TopologyTask** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.42.3 Members

The `TopologyTask` object has members with the same names and descriptions as the arguments to the **`TopologyTask`** method.

In addition, the `TopologyTask` object can have the following members:

designResponses

A repository of `DesignResponse` objects.

objectiveFunctions

A repository of `ObjectiveFunction` objects.

optimizationConstraints

A repository of `OptimizationConstraint` objects.

geometricRestrictions

A repository of `GeometricRestriction` objects.

stopConditions

A repository of `StopCondition` objects.

36.43 TurnControl object

The `TurnControl` object defines a turn control geometric restriction.

The `TurnControl` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.43.1 TurnControl(...)

This method creates a `TurnControl` object.

Path

```
mdb.models[name].optimizationTasks[name].TurnControl
```

Required arguments*name*

A String specifying the geometric restriction repository key.

clientDirection

A VertexArray object of length 2 specifying the direction of the rotation axis as a vector positioned at the *csys* origin. Instead of through a Vertex, each point may be specified through a tuple of coordinates.

region

A Region object specifying the region to which the geometric restriction is applied. When used with a TopologyTask, there is no default value. When used with a ShapeTask, the default value is MODEL.

Optional arguments*csys*

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

masterPoint

None or a Region object specifying the master point used when *masterPointDetermination* is SPECIFY. The default value is **None**.

masterPointDetermination

A SymbolicConstant specifying the rule for assigning point priority. Possible values are MAXIMUM, MINIMUM, and SPECIFY. The default value is MAXIMUM.

presumeFeasibleRegionAtStart

A Boolean specifying whether to ignore the geometric restriction in the first design cycle. The default value is ON.

tolerance1

A Float specifying the geometric tolerance in the 1-direction. The default value is 0.01.

tolerance2

A Float specifying the geometric tolerance in the 2-direction. The default value is 0.01.

tolerance3

A Float specifying the geometric tolerance in the 3-direction. The default value is 0.01.

Return value

A TurnControl object.

Exceptions

None.

36.43.2 **setValues(...)**

This method modifies the `TurnControl` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TurnControl** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.43.3 **Members**

The `TurnControl` object has members with the same names and descriptions as the arguments to the **TurnControl** method.

36.44 **BeadFixedRegion object**

The `BeadFixedRegion` object defines a fixed region geometric restriction.

The `BeadFixedRegion` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.44.1 **BeadFixedRegion(...)**

This method creates a BeadFixedRegion object.

Path

```
mdb.models [name] .optimizationTasks [name] .BeadFixedRegion
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

u1

A Boolean specifying whether to fix the region in the 1-direction. The default value is OFF.

u2

A Boolean specifying whether to fix the region in the 2-direction. The default value is OFF.

u3

A Boolean specifying whether to fix the region in the 3-direction. The default value is OFF.

Return value

A BeadFixedRegion object.

Exceptions

None.

36.44.2 **setValues(...)**

This method modifies the BeadFixedRegion object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BeadFixedRegion` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.44.3 Members

The `BeadFixedRegion` object has members with the same names and descriptions as the arguments to the `BeadFixedRegion` method.

36.45 **BeadGrowth object**

The `BeadGrowth` object defines a growth geometric restriction.

The `BeadGrowth` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.45.1 **BeadGrowth(...)**

This method creates a `BeadGrowth` object.

Path

```
mdb.models[name].optimizationTasks[name].BeadGrowth
```


Required arguments*name*

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments*beadGrowth*

A Float specifying the maximum optimization displacement in the growth direction. Either *beadGrowth* or *shrink* or both must be specified. The default value is 0.0.

shrink

A Float specifying the maximum optimization displacement in the shrink direction. Either *beadGrowth* or *shrink* or both must be specified. The default value is 0.0.

Return value

A BeadGrowth object.

Exceptions

None.

36.45.2 setValues(...)

This method modifies the BeadGrowth object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BeadGrowth** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.45.3 Members

The `BeadGrowth` object has members with the same names and descriptions as the arguments to the `BeadGrowth` method.

36.46 **BeadPenetrationCheck object**

The `BeadPenetrationCheck` object defines a penetration check geometric restriction.

The `BeadPenetrationCheck` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.46.1 **BeadPenetrationCheck(...)**

This method creates a `BeadPenetrationCheck` object.

Path

```
mdb.models[name].optimizationTasks[name].BeadPenetrationCheck
```

Required arguments

name

A String specifying the geometric restriction repository key.

beadPenetrationCheckRegion

A Region object specifying the penetration check region.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional argument

Return value

A `BeadPenetrationCheck` object.

Exceptions

None.

36.46.2 setValues(...)

This method modifies the `BeadPenetrationCheck` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BeadPenetrationCheck` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.46.3 Members

The `BeadPenetrationCheck` object has members with the same names and descriptions as the arguments to the `BeadPenetrationCheck` method.

36.47 BeadPlanarSymmetry object

The `BeadPlanarSymmetry` object defines a bead planar symmetry geometric restriction.

The `BeadPlanarSymmetry` object is derived from the `GeometricRestriction` object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.47.1 **BeadPlanarSymmetry(...)**

This method creates a BeadPlanarSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].BeadPlanarSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

Return value

A BeadPlanarSymmetry object.

Exceptions

None.

36.47.2 **setValues(...)**

This method modifies the BeadPlanarSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BeadPlanarSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.47.3 Members

The **BeadPlanarSymmetry** object has members with the same names and descriptions as the arguments to the **BeadPlanarSymmetry** method.

36.48 BeadPointSymmetry object

The **BeadPointSymmetry** object defines a point symmetry geometric restriction.

The **BeadPointSymmetry** object is derived from the **GeometricRestriction** object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.48.1 BeadPointSymmetry(...)

This method creates a **BeadPointSymmetry** object.

Path

```
mdb.models[name].optimizationTasks[name].BeadPointSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

csys

None or a DatumCsys object specifying the position of the symmetry point defined as the origin of a local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

Return value

A BeadPointSymmetry object.

Exceptions

None.

36.48.2 setValues(...)

This method modifies the BeadPointSymmetry object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BeadPointSymmetry** method, except for the *name* argument.

Return value

None

Exceptions

None.

36.48.3 Members

The BeadPointSymmetry object has members with the same names and descriptions as the arguments to the **BeadPointSymmetry** method.

36.49 **BeadRotationalSymmetry object**

The BeadRotationalSymmetry object defines a bead rotational symmetry geometric restriction.

The BeadRotationalSymmetry object is derived from the GeometricRestriction object.

Access

```
import optimization
mdb.models[name].optimizationTasks[name].geometricRestrictions[name]
```

36.49.1 **BeadRotationalSymmetry(...)**

This method creates a BeadRotationalSymmetry object.

Path

```
mdb.models[name].optimizationTasks[name].BeadRotationalSymmetry
```

Required arguments

name

A String specifying the geometric restriction repository key.

angle

A Float specifying the repeating segment size, an angle in degrees.

region

A Region object specifying the region to which the geometric restriction is applied.

Optional arguments

axis

A SymbolicConstant specifying the axis of symmetry. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

csys

None or a DatumCsys object specifying the local coordinate system. If *csys*=**None**, the global coordinate system is used. When this member is queried, it returns an Int. The default value is **None**.

Return value

A `BeadRotationalSymmetry` object.

Exceptions

None.

36.49.2 `setValues(...)`

This method modifies the `BeadRotationalSymmetry` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `BeadRotationalSymmetry` method, except for the *name* argument.

Return value

None

Exceptions

None.

36.49.3 Members

The `BeadRotationalSymmetry` object has members with the same names and descriptions as the arguments to the `BeadRotationalSymmetry` method.

37. Part commands

37.1 Part object

The Part object defines the physical attributes of a structure. Parts are instantiated into the assembly and positioned before an analysis.

Access

```
import part
mdb.models[name].parts[name]
```

37.1.1 Part(...)

This method creates a Part object and places it in the parts repository.

Path

```
mdb.models[name].Part
```

Required arguments

name

A String specifying the repository key.

dimensionality

A SymbolicConstant specifying the dimensionality of the part. Possible values are THREE_D, TWO_D_PLANAR, and AXISYMMETRIC.

type

A SymbolicConstant specifying the type of the part. Possible values are DEFORMABLE_BODY, EULERIAN, DISCRETE_RIGID_SURFACE, and ANALYTIC_RIGID_SURFACE.

Optional argument

twist

A Boolean specifying whether to include a twist DEGREE OF FREEDOM in the part (only available when *dimensionality*=AXISYMMETRIC and *type*=DEFORMABLE_BODY). The default value is OFF.

Return value

A Part object.

Exceptions

`InvalidNameError`.

37.1.2 Part(...)

This method copies a Part object and places the copy in the parts repository.

Path

`mdb.models[name].Part`

Required arguments

name

A String specifying the repository key.

objectToCopy

A Part Object to be copied.

Optional arguments

scale

A Float specifying the scaling factor to apply to the imported geometric entities during copy. If a scale is specified, *compressFeatureList* will be set to ON, regardless of whether it is specified in the command. The default value is 1.

mirrorPlane

A SymbolicConstant specifying how the part is to be mirrored during copy. Possible values are XYPLANE, XZPLANE, YZPLANE. If a mirror plane is specified, *compressFeatureList* will be set to ON, regardless of whether it is specified in the command. The default value is NONE.

compressFeatureList

A Boolean specifying whether to compress the feature list when copying a Part object. The default value is OFF. If *mirrorPlane* or *scale* is specified, this argument is ignored.

When you compress the feature, list the resulting part will have a single feature. Any datums or sets in the original part will be lost.

separate

A Boolean specifying whether to separate disconnected regions into parts. The default value is OFF.

Return value

A Part object.

Exceptions

InvalidNameError.

37.1.3 PartFromBooleanCut(...)

This method creates a Part in the **parts** repository after subtracting or cutting the geometries of a group of part instances from that of a base part instance.

Path

```
mdb.models[name].rootAssembly.PartFromBooleanCut
```

Required arguments

name

A String specifying the repository key.

instanceToBeCut

A PartInstance specifying the base instance from which to cut other instances.

cuttingInstances

A sequence of PartInstance objects specifying the instances with which to cut the base instance.

Optional arguments

None.

Return value

A Part object.

Exceptions

InvalidNameError.

37.1.4 PartFromBooleanMerge(...)

This method creates a Part in the **parts** repository after merging two or more part instances. The part instances can be either Abaqus native parts or orphan mesh parts, but they cannot be a combination of both.

Path

```
mdb.models[name].rootAssembly.PartFromBooleanMerge
```

Required arguments

name

A String specifying the repository key.

instances

A sequence of PartInstance objects specifying the part instances to merge.

Optional arguments

keepIntersections

A Boolean specifying whether the boundary intersections of Abaqus native part instances should be retained after the merge operation. The default value is False.

mergeNodes

A SymbolicConstant specifying whether the nodes of orphan mesh part instances should be retained after the merge operation. Possible values are BOUNDARY_ONLY, ALL, or NONE. The default value is BOUNDARY_ONLY.

nodeMergingTolerance

A Float specifying the maximum distance between nodes of orphan mesh part instances that will be merged and replaced with a single new node. The location of the new node is the average position of the deleted nodes. The default value is 10^{-6} .

removeDuplicateElements

A Boolean specifying whether elements with the same connectivity after the merge will merged into a single element. The default value is ON.

domain

A SymbolicConstant specifying whether the part instances being merged are geometric instances or mesh instances. Possible values are GEOMETRY, MESH or BOTH. The default value is GEOMETRY.

Return value

A Part object.

Exceptions

InvalidNameError.

37.1.5 PartFromExtrude2DMesh(...)

This method creates a Part object by extruding an existing two-dimensional orphan mesh Part object in the positive Z-direction and places it in the parts repository.

Path

```
mdb.models[modelName].PartFromExtrude2DMesh
```

Required arguments*name*

A String specifying the repository key.

part

A Part object specifying an existing two-dimensional orphan mesh Part object.

depth

A Float specifying the total extrusion distance.

elementSize

A Float specifying an approximate element length in the extruded direction.

Optional arguments

None.

Return value

A Part object.

Exceptions

InvalidNameError.

If the specified part is not an orphan mesh part:

Cannot extrude a geometric part.

If the specified part is not two-dimensional:

Cannot extrude a 3D part.

If the specified part is a rigid body:

Cannot change dimension of a rigid body.

37.1.6 PartFromGeometryFile(...)This method creates a Part object and places it in the **parts** repository.**Path**`mdb.models[modelName].PartFromGeometryFile`**Required arguments***name*

A String specifying the repository key.

geometryFile

An AcisFile object specifying a file containing geometry.

PART COMMANDS

dimensionality

A SymbolicConstant specifying the dimensionality of the part. Possible values are THREE_D, TWO_D_PLANAR, and AXISYMMETRIC.

type

A SymbolicConstant specifying the type of the part. Possible values are DEFORMABLE_BODY, EULERIAN, DISCRETE_RIGID_SURFACE, and ANALYTIC_RIGID_SURFACE.

Optional arguments

bodyNum

An Int specifying the desired body to be selected from an ACIS object containing a list of N ACIS bodies. Possible values are $1 \leq \text{bodyNum} \leq N$. The default value is 1.

combine

A Boolean specifying whether to create a single part by combining all the bodies in the ACIS object. This argument is ignored if *bodyNum* is specified. The default value is False.

booleanSolids

A Boolean specifying whether the solids should be boolean while combining all the bodies. The default value is FALSE.

retainBoundary

A Boolean specifying whether the intersecting boundaries should be retained while boolean the solids. The default value is FALSE.

usePartNameFromFile

A Boolean specifying whether the part names specified in a STEP file should be used as the names in the Abaqus model database. If this option is TRUE, the part names in the STEP file will be used; if FALSE, each imported part will be named using the text of the *name* argument followed by a number. This functionality is available only for import from STEP files; for import from all other types of files this option should be FALSE.

stitchTolerance

A Float indicating the maximum gap to be stitched. The value should be smaller than the minimum feature size and bigger than the maximum gap expected to be stitched in the model. Otherwise this command may remove small (sliver) edges that are smaller than the tolerance. The default value is 1.0

twist

A Boolean specifying whether to include a twist DEGREE OF FREEDOM in the part (only available when *dimensionality*=AXISYMMETRIC and *type*=DEFORMABLE_BODY). The default value is OFF.

scale

A Float specifying the scaling factor to apply to the imported geometric entities. The default value is 1.0.

convertToAnalytical

An Int specifying whether to convert to analytical entities. Possible values are 0 or 1. The default value is 0. If *convertToAnalytical*=1, all the numerical entities, such as splines, are converted to analytical entities, such as arcs and lines, during the repair phase of the command.

convertToPrecise

An Int specifying whether to convert to precise geometry. Possible value are 0 or 1. The default value is 0. If *convertToPrecise*=1, the application will attempt to re-evaluate the tolerant entities to be more precise.

Return value

A Part object.

Exceptions

InvalidNameError.

If the ACIS file is corrupt:

PartError: the file is corrupt

If the dimensionality does not correspond to what is found in the ACIS file:

PartError: dimensionality does not match the contents of the file

If the type does not correspond to what is found in the ACIS file:

PartError: type does not match the contents of the file

37.1.7 PartFromInstanceMesh(...)

This method creates a Part object containing the mesh found in the supplied PartInstance objects and places the new Part object in the parts repository.

Path

`mdb.models[name].rootAssembly.PartFromInstanceMesh`

Required argument

name

A String specifying the repository key.

Optional arguments

partInstances

A sequence of PartInstance objects to be used in the creation of the new mesh part. If the *partInstances* argument is omitted, the new Part object contains the mesh of all the part instances in the assembly.

PART COMMANDS

copyPartSets

A Boolean specifying whether to copy sets, surfaces, and attributes from the base part or parts of the specified part instances to the new part. The default is False.

copyAssemblySets

A Boolean specifying whether to copy assembly-level sets that reference entities of the specified part instances to the new part. The default is False.

Return value

A Part object.

Exceptions

InvalidNameError.

If the analysis type (deformable or rigid) is not consistent among the supplied part instances:

The selected part instances do not have a consistent analysis type.

If the assembly does not contain a mesh:

The current assembly does not contain a mesh for a mesh part.

If the specified part instances do not contain a mesh:

The selected part instances do not have a mesh for a mesh part.

37.1.8 PartFromMesh(...)

This method creates a Part object containing the mesh found in the part and places the new Part object in the parts repository.

Path

`mdb.models [name] .parts [name] .PartFromMesh`

Required argument

name

A String specifying the repository key.

Optional argument

copySets

A Boolean specifying whether to copy sets, surfaces, and attributes to the new part. The default is False.

Return value

A Part object.

Exceptions

InvalidNameError.

If the part does not contain a mesh:

The current part does not contain a mesh for a mesh part.

37.1.9 PartFromMeshMirror(...)

This method creates a Part object by mirroring an existing orphan mesh Part object about a specified plane and places it in the parts repository. The result is a union of the original and the mirrored copy. Contrast the **PartFromMeshMirror** method with the *mirrorPlane* argument of the **Part** copy constructor. The *mirrorPlane* argument creates only the second half of the part but does not unite the two halves.

Path

```
mdb.models[modelName].PartFromMeshMirror
```

Required arguments

name

A String specifying the repository key.

part

A Part object specifying an existing orphan mesh part.

point1

A sequence of three Floats specifying a point on the mirror plane. This point is the local origin in the local system of the plane.

point2

A sequence of three Floats specifying a point in the direction of the normal to the mirror plane. This point must not be coincident with *point1*.

Optional arguments

None.

Return value

A Part object.

Exceptions

InvalidNameError.

If the specified part is not an orphan mesh part:

Cannot mirror a geometric part.

If the specified part is a rigid body:

PART COMMANDS

Cannot mirror a rigid body.

If *point1* and *point2* are coincident:

Mirror plane director has zero length.

If the specified part is two-dimensional and the plane is not parallel to the Z-axis:

Mirror plane must be parallel to Z axis for 2D parts

37.1.10 PartFromNodesAndElements(...)

This method creates a Part object from nodes and elements and places it in the **parts** repository.

Path

```
mdb.models[modelName].PartFromNodesAndElements
```

Required arguments

name

A String specifying the repository key.

dimensionality

A SymbolicConstant specifying the dimensionality of the part. Possible values are THREE_D, TWO_D_PLANAR, and AXISYMMETRIC.

type

A SymbolicConstant specifying the type of the part. Possible values are DEFORMABLE_BODY, EULERIAN, DISCRETE_RIGID_SURFACE, and ANALYTIC_RIGID_SURFACE.

nodes

A sequence of (*nodeLabels*, *nodeCoords*) specifying the nodes of the mesh. *nodeLabels* is a sequence of Ints specifying the node labels, and *nodeCoords* is a sequence of sequences of three Floats specifying the nodal coordinates.

elements

A sequence of sequences of(*meshType*, *elementLabels*, *elementConns*) specifying the elements of the mesh. *meshType* is a String specifying the element type. *elementLabels* is a sequence of Ints specifying the element labels. *elementConns* is a sequence of sequences of node labels specifying the element connectivity.

Optional argument

twist

A boolean specifying whether the part is defined with twist. This option has meaning only when *dimensionality*=AXISYMMETRIC. Possible values are ON and OFF. The default value is OFF.

Return value

A Part object.

Exceptions

None.

37.1.11 PartFromOdb(...)

This method creates an orphan mesh Part object by reading an output database. The new part is placed in the **parts** repository.

Path

`mdb.models[modelName].PartFromOdb`

Required arguments

name

A String specifying the repository key.

odb

An output database object.

Optional arguments

fileName

A String specifying the name of the output database file from which to create the part. The default value is an empty string.

instance

A String specifying the part instance in the output database from which to create the part. If no instance name is specified, Abaqus creates an orphan mesh part from the first part instance in the output database.

elementSet

A String specifying an element set defined on the output database. Only elements from this set will be imported. The default is to import all element sets.

shape

A SymbolicConstant specifying the configuration state. Possible values are UNDEFORMED and DEFORMED. The default value is UNDEFORMED.

step

An Int specifying the step number for reading deformed coordinates. $0 \leq \text{step} \leq N - 1$ where N is the number of available steps. The default value is the last available step. You should specify the *step* argument only when *shape*=DEFORMED.

frame

An Int specifying the frame number for reading deformed coordinates. $0 \leq \text{frame} \leq N - 1$ where N is the number of available frames. The default value is the last available frame. You should specify the *frame* argument only when *shape*=DEFORMED.

PART COMMANDS

twist

A Boolean specifying whether to include a twist DEGREE OF FREEDOM in the part (only available when *dimensionality*=AXISYMMETRIC and *type*=DEFORMABLE_BODY). The default value is OFF.

Return value

A Part object.

Exceptions

InvalidNameError.

If the output database contains elements of more than one dimensionality or type:

File contains both axisymmetric and nonaxisymmetric elements.

File contains both 2D and 3D elements.

File contains both rigid and deformable elements.

If more than one part is found on the output database:

PartError: importing of more than one part is not currently supported

If the output database does not contain any valid results for the specified step:

Error. File does not contain any valid frames.

If the specified step and frame do not contain any displacements:

Error. Specified frame does not contain nodal displacements.

If the specified element set is not found on the output database:

Error. Specified element set is not defined in the ODB.

If the step number is invalid:

OdiError: Invalid step index: i. Available step indices: 0 - j.

If the frame number is invalid:

OdiError: Invalid frame index: i. Available frame indices: 0 - j.

37.1.12 PartFromSection3DMeshByPlane(...)

This method creates a Part object by cutting an existing three-dimensional orphan mesh Part object by a plane and places it in the **parts** repository. This method is valid only for orphan mesh parts composed of 8-node brick elements.

Path

`mdb.models[modelName].PartFromSection3DMeshByPlane`

Required arguments

name

A String specifying the repository key.

part

A Part object specifying an existing three-dimensional orphan mesh part.

point1

A Sequence of three Floats specifying a point on the cutting plane. This point is the local origin in the local system of the plane.

point2

A Sequence of three Floats specifying a point in the direction of the normal to the cutting plane. This point must not be coincident with *point1*.

point3

A sequence of three Floats specifying the direction of the local 1-axis in the local system of the plane. This point must not project onto *point1*.

Optional arguments

None.

Return value

A Part object.

Exceptions

InvalidNameError.

If the specified part is not an orphan mesh part:

Cannot reduce dimension of a geometric part.

If the specified part is not three-dimensional:

Cannot reduce dimension of a 2D part.

If the specified part is a rigid body:

Cannot change dimension of a rigid body.

If *point1* and *point2* are coincident:

Cutting plane director has zero length.

If *point3* projects onto *point1*:

Local axis point projects to origin.

If no elements are cut by the specified plane:

Cannot reduce part dimension.

37.1.13 PartFromSubstructure(...)

This method creates a substructure Part object by reading a substructure sim file and places it in the **parts** repository.

Path

`mdb.models [name] .PartFromSubstructure`

Required arguments

name

A String specifying the repository key.

substructureFile

A substructure sim file.

odbFile

The output database file corresponding to the substructure sim file.

Optional arguments

None.

Return value

A Part object.

Exceptions

InvalidNameError.

If the specified part is not a substructure:

File specified does not contain a substructure.

If the specified part already exists:

A part with the same name already exists.

If the substructure cannot be imported:

The output database is missing nodes and elements.

Nested substructures are not supported.

The substructure sim file was generated using a version that is different from the current version.

37.1.14 Part2DGeomFrom2DMesh(...)

This method creates a geometric Part object from the outline of an existing two-dimensional orphan mesh Part object and places it in the **parts** repository. If the **Part2DGeomFrom2DMesh** method cannot create a valid two-dimensional shell section from the two-dimensional mesh, the method fails and creates an empty geometry part with a failed base shell feature.

Path

`mdb.models [modelName] .Part2DGeomFrom2DMesh`

Required arguments*name*

A String specifying the repository key.

part

A Part object specifying an existing two-dimensional orphan mesh Part object.

featureAngle

A Float specifying the angle (in degrees) between line segments that triggers a break in the geometry.

Optional arguments*splineCurvatureLimit*

A Float specifying the traversal angle in degrees of the spline that triggers a break in the geometry. The default value is 90.

*twist*A Boolean specifying whether to include a twist DEGREE OF FREEDOM in the part (only available when *dimensionality*=*AXISYMMETRIC* and *type*=*DEFORMABLE_BODY*). The default value is OFF.**Return value**

A Part object.

Exceptions

InvalidNameError.

If the specified part is not an orphan mesh part:

Specified part must be an orphan mesh.

If the **Part2DGeomFrom2DMesh** method cannot create a valid two-dimensional shell section from the two-dimensional mesh:

Planar shell feature failed

If the specified part is not two-dimensional:

Cannot create a geometry from a 3D part.

If the specified part is a rigid body:

Cannot create a geometry from a rigid body.

37.1.15 setValues(...)

This method modifies the Part object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Part** method. In addition, **setValues** has the following optional argument:

geometryRefinement

A SymbolicConstant specifying how the part's surface is being refined when faceted for display. Possible values are EXTRA_COARSE, COARSE, MEDIUM, FINE, and EXTRA_FINE.

startNodeLabel

A positive Integer specifying the new start node label for the part mesh that currently exists or will be generated. If the part is meshed, Abaqus/CAE changes the node labels while preserving the original order and incrementation.

startElemLabel

A positive Integer specifying the new start element label for the part mesh that currently exists or will be generated. If the part is meshed, Abaqus/CAE changes the element labels while preserving the original order and incrementation.

Return value

None

Exceptions

RangeError.

37.1.16 addGeomToSketch(...)

This method converts a part into a sketch by projecting all of the edges of the part onto the X-Y plane of the sketch. You can use **addGeomToSketch** with a part of any modeling space.

Arguments

Required argument

sketch

A ConstrainedSketch object.

Optional arguments

None.

Return value

None

Exceptions

None.

37.1.17 assignThickness(...)

This method assigns thickness data to shell faces. The thickness can be used while assigning shell and membrane sections to faces.

Arguments**Required argument**

faces

A sequence of Face objects specifying the regions where thickness will be applied.

Optional arguments

thickness

A Float specifying the thickness along the given *faces*. Either *thickness*, *topFaces*, or *bottomFaces* must be specified.

topFaces

A sequence of Face objects whose distance to *faces* argument is used to calculate the thickness along the *faces*. The combination of *topFaces* and *bottomFaces* determines the thickness and the offset of the elements. If *bottomFaces* is not specified then the thickness is twice the distance to the *topFaces*. This argument will be ignored if *thickness* is specified. Either *thickness*, *topFaces*, or *bottomFaces* must be specified.

bottomFaces

A sequence of Face objects whose distance to *faces* is used to calculate the thickness along the *faces*. The combination of *topFaces* and *bottomFaces* determines the thickness and the offset of the elements. If *topFaces* is not specified then the thickness is twice the distance to the *bottomFaces*. This argument will be ignored if *thickness* is specified. Either *thickness*, *topFaces*, or *bottomFaces* must be specified.

Return value

None

Exceptions

None.

37.1.18 backup()

This method makes a backup copy of the features in the part. Use the restore method to retrieve the part's features from the backup.

Arguments

None.

Return value

None

Exceptions

None.

37.1.19 checkGeometry(...)

This method checks the validity of the geometry of the part and prints a count of all topological entities on the part (faces, edges, vertices, etc.).

Arguments

Required arguments

None.

Optional arguments

detailed

A Boolean specifying whether detailed output will be printed to the replay file. The default value is OFF.

reportFacetErrors

A Boolean specifying whether faces are checked for proper facetting. The default value is OFF.

level

An Int specifying which level of checking is performed. Values can range from 20 to 70, with higher values reporting less and less important errors. The default value is 20, which reports all critical errors. When the default value is used, the stored validity status is updated to agree with the result of this check.

Return value

None

Exceptions

None.

37.1.20 clearGeometryCache()

This method clears the geometry cache. Clearing the geometry cache reduces the amount of memory being used to cache part features.

Arguments

None.

Return value

None

Exceptions

None.

37.1.21 deleteAllFeatures()

This method deletes all the features in the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.22 deleteFeatures(...)

This method deletes the given features.

Arguments**Required argument**

featureNames

A sequence of Strings specifying the feature names that will be deleted from the part.

Optional arguments

None.

Return value

None

Exceptions

None.

37.1.23 **getAngle(...)**

This method returns the angle between the specified entities.

Arguments

Required arguments

plane1

A Face , MeshFace , or a Datum object specifying the first plane. The Datum object must represent a datum plane. The *plane1* and *line1* arguments are mutually exclusive. One of them must be specified.

plane2

A Face , MeshFace , or a Datum object specifying the second plane. The Datum object must represent a datum plane. The *plane2* and *line2* arguments are mutually exclusive. One of them must be specified.

line1

An Edge , MeshEdge , or a Datum object specifying the first curve. The Datum object must represent a datum axis. The *plane1* and *line1* arguments are mutually exclusive. One of them must be specified.

line2

An Edge , MeshEdge , or a Datum object specifying the second curve. The Datum object must represent a datum axis. The *plane2* and *line2* arguments are mutually exclusive. One of them must be specified.

Optional argument

commonVertex

If the two selected Edge objects have more than one vertex in common, this Vertex object specifies the vertex at which to evaluate the angle.

Return value

A Float specifying the angle between the specified entities. If you provide a plane as an argument, Abaqus/CAE computes the angle using the normal to the plane.

Exceptions

None.

37.1.24 getArea(...)

This method returns the total surface area of a given face or group of faces.

Arguments**Required argument**

faces

A sequence of face objects whose area the method will calculate.

Optional argument

relativeAccuracy

A Float specifying that the area computation should stop when the specified relative accuracy has been achieved. The default value is 0.000001 (0.0001%).

Return value

A Float specifying the sum of the calculated areas of the given faces.

Exceptions

None.

37.1.25 getAssociatedCADPaths()

This method returns the paths to the associated CAD part and root file. These are only available if the part was imported from one of the supported CAD softwares using the Associative Import capability. The root file can be the assembly file or the part file, depending on what which one was imported.

Arguments

None.

Return value

A sequence containing the path to the associated CAD part and assembly file.

Exceptions

None.

37.1.26 **getCADParameters()**

This method returns the names and values of the CAD parameters associated with the part. These are only available if the part was imported from one of the supported CAD softwares using the Associative Import capability, and if the parameter names defined in that CAD software are prefixed with the string ABQ.

Arguments

None.

Return value

A dictionary object representing a map of the name of the parameter and its associated value.

Exceptions

None.

37.1.27 **getCentroid(...)**

Depending on the arguments provided, this method returns the following:

- The location of the centroid of a given face or group of faces.
- The location of the centroid of a given cell or group of cells.

Arguments

Required arguments

faces

A sequence of face objects whose centroid the method will calculate. The arguments *faces* and *cells* are mutually exclusive.

cells

A sequence of face objects whose centroid the method will calculate. The arguments *faces* and *cells* are mutually exclusive.

Optional argument

relativeAccuracy

A Float specifying that the centroid computation should stop when the specified relative accuracy has been achieved. The default value is 0.000001 (0.0001%).

Return value

A sequence of Floats specifying the X-, Y-, and Z-coordinates of the centroid.

Exceptions

None.

37.1.28 getCoordinates(...)

This method returns the coordinates of specified point.

Arguments**Required argument**

entity

A Vertex , Datum point, MeshNode , or ReferencePoint specifying the entity to query.

Optional arguments

None.

Return value

A tuple of 3 Floats representing the coordinates of the specified point.

Exceptions

None.

37.1.29 getCurvature(...)

This method returns the maximum curvature of a given edge or group of edges. For an arc, the curvature is constant over the entire edge, and equal to the inverse of the radius. For a straight line, the curvature is constant and equal to 0. For a spline edge, the curvature varies over a range, and this methods computes the maximum.

Arguments**Required argument**

edges

A sequence of edge objects whose curvature the method will calculate.

Optional argument

samplePoints

An Int specifying the number of points along each edge at which the curvature will be computed. The higher the number of sample points, the better the accuracy of the computation. The default value is 100.

Return value

A Float specifying the maximum curvature.

Exceptions

None.

37.1.30 **getDistance(...)**

Depending on the arguments provided, this method returns one of the following:

- The distance between two points.
- The minimum distance between a point and an edge.
- The minimum distance between two edges.

Arguments

Required arguments

entity1

A Vertex , Datum point, MeshNode , or Edge specifying the first entity from which to measure.

entity2

A Vertex , Datum point, MeshNode , or Edge specifying the second entity to which to measure.

Optional arguments

None.

Return value

A Float specifying the distance between *entity1* and *entity2*.

Exceptions

None.

37.1.31 **getLength(...)**

This method returns the length of a given edge or group of edges.

Arguments**Required argument***edges*

A sequence of edge objects whose total length the method will calculate.

Optional arguments

None.

Return value

A Float specifying the total length.

Exceptions

None.

37.1.32 getPerimeter(...)

This method returns the total perimeter of a given face or group of faces. All faces need to be on the same part. If the specified faces have shared edges, these edges are excluded from the computation, thus providing the length of the outer perimeter of the specified faces.

Arguments**Required argument***faces*

A sequence of face objects whose perimeter the method will calculate.

Optional arguments

None.

Return value

A Float specifying the perimeter.

Exceptions

None.

37.1.33 getVolume(...)

This method returns the volume area of a given cell or group of cells.

Arguments

Required argument

cells

A sequence of cell objects whose volume the method will calculate.

Optional argument

relativeAccuracy

A Float specifying the relative accuracy of the computation. The default value is 0.000001 (0.0001%).

Return value

A Float specifying the sum of the areas of the given faces.

Exceptions

None.

37.1.34 **getMassProperties(...)**

This method returns the mass properties of a part or region. Only beams, trusses, shells, solids, point, nonstructural mass, and rotary inertia elements are supported.

Arguments

Required arguments

None.

Optional arguments

regions

A MeshElementArray, CellArray, FaceArray, or EdgeArray specifying the regions whose mass properties are to be queried. The whole part is queried by default.

relativeAccuracy

A SymbolicConstant specifying the relative accuracy for geometry computation. Possible values are LOW, MEDIUM and HIGH. The default value is LOW.

useMesh

A Boolean specifying whether the mesh should be used in the computation if the geometry is meshed. The default value is False.

specifyDensity

A Boolean specifying whether a user-specified density should be used in regions with density errors such as undefined material density. The default value is False.

density

A double value specifying the user-specified density value to be used in regions with density errors. The user-specified density should be greater than 0.

specifyThickness

A Boolean specifying whether a user-specified thickness should be used in regions with thickness errors such as undefined thickness. The default value is False.

thickness

A double value specifying the user-specified thickness value to be used in regions with thickness errors. The user-specified thickness should be greater than 0.

miAboutCenterOfMass

A Boolean specifying if the moments of inertia should be evaluated about the center of mass. The default value is True.

miAboutPoint

A tuple of three floats specifying the coordinates of the point about which to evaluate the moment of inertia. By default if the moments of inertia are not being evaluated about the center of mass, they will be evaluated about the origin.

Return value

A Dictionary object with the following items:

area: None or a Float specifying the sum of the area of the specified faces. The area is computed only for one side for shells.

areaCentroid: None or a tuple of three Floats representing the coordinates of the area centroid.

volume: None or a Float specifying the volume of the specified regions.

volumeCentroid: None or a tuple of three Floats representing the coordinates of the volume centroid.

massFromMassPerUnitSurfaceArea: None or a Float specifying the mass due to mass per unit surface area.

mass: None or a Float specifying the mass of the specified regions. It is the total mass and includes mass from quantities such as mass per unit surface area.

centerOfMass: None or a tuple of three Floats representing the coordinates of the center of mass.

momentOfInertia: None or a tuple of six Floats representing the moments of inertia about the center of mass or about the point specified.

warnings: A tuple of SymbolicConstants representing the problems encountered while computing the mass properties. Possible SymbolicConstants are:

UNSUPPORTED_ENTITIES: Some unsupported entities exist in the specified region. The mass properties are computed only for beams, trusses, shells, solids, point and non-structural mass elements and rotary inertia elements. The mass properties are not computed for axisymmetric elements, springs, connectors, gaskets or any other elements.

MISSING_THICKNESS: For some regions, the section definitions are missing thickness values.

ZERO_THICKNESS: For some regions, the section definitions have a zero thickness value.

PART COMMANDS

VARIABLE_THICKNESS: The nodal thickness or field thickness specified for some regions has been ignored.

NON_APPLICABLE_THICKNESS: For some regions, the thickness value is not applicable to the corresponding sections specified on the regions.

MISSING_DENSITY: For some regions, the section definitions are missing material density values.

MISSING_MATERIAL_DEFINITION: For some regions, the material definition is missing.

ZERO_DENSITY: For some regions, the section definitions have a zero material density value.

UNSUPPORTED_DENSITY: For some regions, either a negative material density or a temperature dependent density has been specified, or the material value is missing for one or more plies in the composite section.

SHELL_OFFSETS: For shells, this method does not account for any offsets specified.

MISSING_SECTION_DEFINITION: For some regions, the section definition is missing.

UNSUPPORTED_SECTION_DEFINITION: The section definition provided for some regions is not supported.

REINFORCEMENTS: This method does not account for any reinforcements specified on the model.

SMEARED_PROPERTIES: For regions with composite section assignments, the density is smeared across the thickness. The volume centroid and center of mass computations for a composite shell use a lumped mass approach where the volume and mass is assumed to be lumped in the plane of the shell. As a result of these approximations the volume centroid, center of mass and moments of inertia may be slightly inaccurate for regions with composite section assignments.

UNSUPPORTED_NON_STRUCTURAL_MASS_ENTITIES: This method does not account for any non-structural mass on wires.

INCORRECT_MOMENT_OF_INERTIA: For geometry regions with non-structural mass per volume, the non-structural mass is assumed to be a point mass at the centroid of the regions. Thus, the moments of inertia may be inaccurate as the distribution of the non-structural mass is not accounted for. Use the mesh for accurately computing the moments of inertia.

MISSING_BEAM_ORIENTATIONS: For some regions with beam section assignments, the beam section orientations are missing.

UNSUPPORTED_BEAM_PROFILES: This method supports the Box, Pipe, Circular, Rectangular, Hexagonal, Trapezoidal, I, L, T, Arbitrary, and Tapered beam profiles. Any other beam profile is not supported.

TAPERED_BEAM_MI: Moment of inertia calculations for tapered beams are not accurate.

SUBSTRUCTURE_INCORRECT_PROPERTIES: The user assigned density and thickness is not considered for substructures.

UNSUPPORTED_NON_STRUCTURAL_MASS_PROPORTIONAL: Non-structural mass with Mass Proportional distribution is not supported. Results are computed using Volume Proportional distribution.

Exceptions

None.

37.1.35 getFeatureFaces(...)

This method returns a sequence of Face objects that are created by the given feature.

Arguments**Required argument**

name

A string specifying the feature name.

Optional arguments

None.

Return value

Sequence of Face objects.

Exceptions

An exception occurs if a feature with the given name does not exist.

Error : Incorrect feature name.

37.1.36 getFeatureEdges(...)

This method returns a sequence of Edge objects that are created by the given feature.

Arguments**Required argument**

name

A string specifying the feature name.

Optional arguments

None.

Return value

Sequence of Edge objects.

Exceptions

An exception occurs if a feature with the given name does not exist.

Error : Incorrect feature name.

37.1.37 **getFeatureCells(...)**

This method returns a sequence of Cell objects that are created by the given feature.

Arguments

Required argument

name

A string specifying the feature name.

Optional arguments

None.

Return value

Sequence of Cell objects.

Exceptions

An exception occurs if a feature with the given name does not exist.

Error : Incorrect feature name.

37.1.38 **getFeatureVertices(...)**

This method returns a sequence of Vertex objects that are created by the given feature.

Arguments

Required argument

name

A string specifying the feature name.

Optional arguments

None.

Return value

Sequence of Vertex objects.

Exceptions

An exception occurs if a feature with the given name does not exist.

Error : Incorrect feature name.

37.1.39 **isAlignedWithSketch()**

This method checks if the normal of an analytical rigid surface part is aligned with that of its sketch.

Arguments

None.

Return value

A Boolean value of True if the part is aligned with the sketch and False if it is not aligned.

Exceptions

If the part is not an analytical rigid part:

`AbaqusException`: Can only be used with analytical rigid parts.

37.1.40 **printAssignedSections()**

This method prints information on each section that has been assigned to a region of the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.41 **projectEdgesOntoSketch(...)**

This method projects the selected edges of a part onto the specified `ConstrainedSketch` object. The edges appear as sketch geometry after projection. If the plane of projection is not parallel to the specified edge, the resultant sketch geometry may be of a different type. For example, a circular edge can be projected as an ellipse or a line depending on the angle of the plane of projection. By default, the projected edge will be constrained to the background geometry. You can remove this constraint by setting *constrainToBackground* to False.

Arguments

Required arguments

sketch

The ConstrainedSketch object on which the edges are projected.

edges

A sequence of candidate edges to be projected onto the sketch.

Optional argument

constrainToBackground

A Boolean that determines whether the projected edges need to be constrained to the background geometry. The default is True.

Return value

None

Exceptions

None.

37.1.42 projectReferencesOntoSketch(...)

This method projects the vertices of specified edges, and datum points from the part onto the specified ConstrainedSketch object. The vertices and datum points appear on the sketch as reference geometry.

Arguments

Required argument

sketch

The ConstrainedSketch object on which the edges, vertices, and datum points are projected.

Optional arguments

filter

A SymbolicConstant specifying how to limit the amount of projection. Possible values are ALL_EDGES and COPLANAR_EDGES. If *filter*=COPLANAR_EDGES, edges that are coplanar to the sketching plane are the only candidates for projection. The default value is ALL_EDGES.

upToFeature

A Feature object specifying a marker in the feature-based history of the part. Abaqus/CAE projects onto the sketch only the part entities that were created before the feature specified by this marker. By default, part entities in features created before the sketch you are editing are candidates for projection.

edges

A sequence of candidate edges whose vertices need to be projected onto the sketch. By default, all edges specified by the *filter* argument are candidates for projection.

vertices

A sequence of candidate vertices to be projected onto the sketch. By default, all vertices are candidates for projection.

Return value

None

Exceptions

None.

37.1.43 queryAttributes(...)

This method prints the following information about a part:

- the name, modeling space, and analysis type; and
- whether twist is included (only available when the modeling space is axisymmetric and the analysis type is deformable); and
- the number of vertices, edges, faces and cells if applicable.

Arguments**Required arguments**

None.

Optional argument*printResults*

A Boolean which specifies whether the above information is to be printed. The default value is True

Return value

A Dictionary object with string keys and integer values which returns the above information with the keys being numVertices, numEdges, numFaces, numCells, numWiredEdges, numShellFaces and numSolidFaces.

Exceptions

None.

37.1.44 **queryCachedStates()**

This method displays the position of geometric states relative to the sequence of features in the part cache. The output is displayed in the message area.

Arguments

None.

Return value

None

Exceptions

None.

37.1.45 **queryGeometry(...)**

This method prints the following information about a part:

- the name, modeling space, and analysis type;
- whether twist is included (only available when the modeling space is axisymmetric and the analysis type is deformable);
- a 3D point representing the minimum of the part's bounding box;
- a 3D point representing the maximum of the part's bounding box;
- a 3D point representing the part's centroid (only on 3D solid parts); and
- the volume (only on 3D solid parts).

Arguments

Required arguments

None.

Optional arguments

relativeAccuracy

A Float specifying that the property computations should stop when the specified relative accuracy has been achieved. The default value is 0.000001 (0.0001%).

printResults

A Boolean which specifies whether the above information is to be printed. The default value is True.

Return value

A Dictionary object with string keys, which returns the above information with the keys being name, space, type, volume, centroid, category and boundingBox.

Exceptions

None.

37.1.46 queryRegionsMissingSections()

This method returns all regions in the part that do not have a section assignment but require one for analysis.

Arguments

None.

Return value

A Region object, or None.

Exceptions

None.

37.1.47 queryDisjointPlyRegions()

This method provides a list of all composite plies in the current part which have disjoint regions.

Arguments

None.

Return value

None

Exceptions

None.

37.1.48 **regenerate()**

This method regenerates a part. When you modify features, it may be convenient to postpone regeneration until you make all your changes, since regeneration can be time consuming.

Arguments

None.

Return value

None

Exceptions

None.

37.1.49 **regenerationWarnings()**

This method prints any regeneration warnings associated with the features.

Arguments

None.

Return value

None

Exceptions

None.

37.1.50 **removeInvalidGeometry()**

Removes all invalid entities from the part, leaving a valid part. This is not recorded as a feature in the feature list, therefore it should be used on parts that have a single feature (such as an imported part).

Note: This may remove valid entities that are connected to invalid ones. You can identify invalid entities using the query toolset before using this command.

Arguments

None.

Return value

None

Exceptions

None.

37.1.51 restore()

This method restores the parameters of all features in the assembly to the value they had before a failed regeneration. Use the **restore** method after a failed regeneration, followed by a **regenerate** command.

Arguments

None.

Return value

None

Exceptions

None.

37.1.52 resumeAllFeatures()

This method resumes all the suppressed features in the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.53 resumeFeatures(...)

This method resumes the specified suppressed features in the part.

PART COMMANDS

Arguments

Required argument

featureNames

A tuple of names of features which are to be resumed.

Optional arguments

None.

Return value

None

Exceptions

None.

37.1.54 resumeLastSetFeatures()

This method resumes the last set of features to be suppressed in the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.55 saveGeometryCache()

This method caches the current geometry. Caching the current geometry improves regeneration performance.

Arguments

None.

Return value

None

Exceptions

None.

37.1.56 setAssociatedCADPaths(...)

This method sets the paths to the associated CAD part and root file. This method is only available if the part was imported from one of the supported CAD softwares using the Associative Import capability. The root file can be the assembly file or the part file, depending on the one that was imported. This method can be used to specify the new paths when the CAD data is moved to a different directory.

Arguments**Required arguments**

None.

Optional arguments

partFile

A String specifying the name of the associated CAD part file.

rootFile

A String specifying the name of the root associated CAD file. This can be the same as the part file or can be the assembly file, depending on the one that was imported.

Return value

None

Exceptions

None.

37.1.57 suppressFeatures(...)

This method suppresses the given features.

Arguments**Required argument**

featureNames

A tuple of names of features which are to be suppressed in the part.

Optional arguments

None.

Return value

None

Exceptions

None.

37.1.58 writeAcisFile(...)

This method exports the geometry of the part to a named file in ACIS format.

Arguments

Required argument

fileName

A String specifying the name of the file to which to write.

Optional argument

version

A Float specifying the ACIS version. For example, the Float “12.0” corresponds to ACIS Version 12.0. The default value is the current version of ACIS.

Return value

None

Exceptions

If the part is an orphan mesh part:

Cannot export orphan mesh parts to ACIS.

37.1.59 writeCADParameters(...)

This method writes the parameters that were imported from the CAD system to a parameter file.

Arguments

Required argument

paramFile

A String specifying the parameter file name.

Optional arguments*modifiedParams*

A tuple of tuples each containing the part name, the parameter name, and the modified parameter value. Default is an empty tuple.

updatePaths

A Bool specifying whether to update the path of the CAD model file specified in the *parameterFile* to the current directory, if the CAD model is present in the current directory.

Return value

None

Exceptions

None.

37.1.60 writeIgesFile(...)

This method exports the geometry of the part to a named file in IGES format.

Arguments**Required arguments***fileName*

A String specifying the name of the file to which to write.

flavor

A SymbolicConstant specifying a particular flavor of IGES. Possible values are STANDARD, AUTOCAD, SOLIDWORKS, JAMA, and MSBO.

Optional arguments

None.

Return value

None

Exceptions

If the part is an orphan mesh part:

Cannot export orphan mesh parts to IGES.

37.1.61 writeStepFile(...)

This method exports the geometry of the part to a named file in STEP format.

Arguments

Required argument

fileName

A String specifying the name of the file to which to write.

Optional arguments

None.

Return value

None

Exceptions

If the part contains no geometry:

Parterror: Cannot export orphan mesh parts to STEP.

37.1.62 writeVdaFile(...)

This method exports the geometry of the part to a named file in VDA-FS format.

Arguments

Required argument

fileName

A String specifying the name of the file to which to write.

Optional arguments

None.

Return value

None

Exceptions

If the part is an orphan mesh part:

Cannot export orphan mesh parts to VDA-FS.

37.1.63 copyMeshPattern(...)

This method copies a mesh pattern from a source region consisting of a set of shell elements or element faces onto a target face, mapping nodes and elements in a one-one correspondence between source and target.

Arguments**Required arguments**

None.

Optional arguments*elements*

A sequence of MeshElement objects or a Set object containing elements and specifying the source region.

faces

A sequence of Face objects that have associated with shell elements or element faces and specifying the source region.

elemFaces

A sequence of MeshFace objects specifying the source region.

targetFace

A MeshFace object specifying the target region.

nodes

A sequence of MeshNode objects or a Set object containing nodes on the boundary of source region which are to be positioned to the boundary of target face.

coordinates

A sequence of three-dimensional coordinate tuples specifying the coordinates for each of the given nodes. When specified, the number of coordinate tuples must match the number of given nodes, and be ordered to correspond to the given nodes in *ascending order* according to index. These coordinates are positions of the nodes of a mesh that will be the target face corresponding to nodes provided.

Return value

None

Exceptions

None.

37.1.64 smoothNodes(...)

This method smoothes the given nodes of a native mesh, moving them locally to a more optimal location that improves quality of the mesh

Arguments**Required arguments**

None.

PART COMMANDS

Optional arguments

nodes

A sequence of MeshNode objects or a Set object containing nodes. Nodes that are not part of native mesh only will be smoothed

coordinates

A sequence of MeshNode objects or a Set object containing nodes.

Return value

None

Exceptions

None.

37.1.65 Lock()

This method locks the part. Locking the part prevents any further changes to the part that can trigger regeneration of the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.66 Unlock()

This method unlocks the part. Unlocking the part allows it to be regenerated after any modifications to the part.

Arguments

None.

Return value

None

Exceptions

None.

37.1.67 LockForUpgrade()

This method locks the part for upgrade. Locking the part prevents any further changes to the part that can trigger regeneration of the part. When the part is unlocked, all the parts are upgraded and regenerated.

Arguments

None.

Return value

None

Exceptions

None.

37.1.68 Members

The Part object has members with the same names and descriptions as the arguments to the **Part** method.

In addition, the Part object can have the following members:

geometryValidity

A Boolean specifying the validity of the geometry of the part. The value is computed, but it can be set to ON to perform feature and mesh operations on an invalid part. There is no guarantee that such operations will work if the part was originally invalid.

isOutOfDate

An Int specifying that feature parameters have been modified but that the part has not been regenerated. Possible values are 0 and 1.

timeStamp

A Float specifying when the part was last modified.

vertices

A VertexArray object specifying all the vertices in the part.

ignoredVertices

An IgnoredVertexArray object specifying all the ignored vertices in the part.

edges

An EdgeArray object specifying all the edges in the part.

ignoredEdges

An IgnoredEdgeArray object specifying all the ignored edges in the part.

PART COMMANDS

faces

A FaceArray object specifying all the faces in the part.

cells

A CellArray object specifying all the cells in the part.

features

A repository of Feature objects specifying all the features in the part.

featuresById

A repository of Feature objects specifying all Feature objects in the part. The Feature objects in the **featuresById** repository are the same as the Feature objects in the **features** repository. However, the key to the objects in the **featuresById** repository is an integer specifying the *ID*, whereas the key to the objects in the **features** repository is a string specifying the *name*.

datums

A repository of Datum objects specifying all the datums in the part.

elements

A MeshElementArray object specifying all the elements in the part.

elemFaces

A repository of MeshFace objects specifying all the element faces in the part. For a given element and a given face index within that element, the corresponding MeshFace object can be retrieved from the repository by using the key calculated as $(i*8 + j)$, where *i* and *j* are zero-based element and face indices, respectively.

elementFaces

A MeshFaceArray object specifying all the unique element faces in the part.

nodes

A MeshNodeArray object specifying all the nodes in the part.

retainedNodes

A MeshNodeArray object specifying all the retained nodes in the substructure part.

sets

A repository of Set objects specifying for more information, see Set .

allSets

A repository of Set objects specifying the contents of the *allSets* repository is the same as the contents of the *sets* repository.

allInternalSets

A repository of Set objects specifying picked regions.

surfaces

A repository of Surface objects specifying for more information, see Surface .

allSurfaces

A repository of Surface objects specifying the contents of the *allSurfaces* repository is the same as the contents of the *surfaces* repository.

allInternalSurfaces

A repository of Surface objects specifying picked regions.

skins

A repository of Skin objects specifying the skins created on the part.

stringers

A repository of Stringer objects specifying the stringers created on the part.

referencePoints

A repository of ReferencePoint objects.

engineeringFeatures

An EngineeringFeature object.

sectionAssignments

A SectionAssignmentArray object.

materialOrientations

A MaterialOrientationArray object.

compositeLayups

A repository of CompositeLayup objects.

elemEdges

A repository of MeshEdge objects specifying all the element edges in the part. For a given element and a given edge index on a given face within that element, the corresponding MeshEdge object can be retrieved from the repository by using the key calculated as $(i*32 + j*4 + k)$, where i , j , and k are zero-based element, face, and edge indices, respectively.

elementEdges

A MeshEdgeArray object specifying all the unique element edges in the part.

37.2 Feature object

The following commands operate on Feature objects. For more information about the Feature object, see “Feature object,” Section 20.1.

Access

```
import part
```

37.2.1 AutoRepair()

This method carries out a sequence of geometry repair operations if it contains invalid entities. It is expected to improve the geometry, but it does not guarantee that the number of invalid entities will decrease. In some cases, it can also increase the number of invalid entities. Since a number of geometry

repair operations and validity checks are performed, it could be a slow operation depending on the complexity of the geometry.

Path

```
mdb.models[name].parts[name].AutoRepair
```

Arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.2 AddCells(...)

This method tries to convert a shell entity to a solid entity. The conversion is not always successful.

Path

```
mdb.models[name].parts[name].AddCells
```

Required argument

faceList

A sequence of Face objects specifying the faces bounding the cell to add.

Optional argument

flipped

A Boolean specifying the direction of feature creation. The possible values are True and False. The default is True indicating that the direction is opposite to the face normal. When multiple faces are selected, Abaqus attempts to create cells on both sides of the selected faces and ignores the *flipped* argument.

Return value

A Feature object.

Exceptions

None.

37.2.3 AnalyticRigidSurf2DPlanar(...)

This method creates a first Feature object for an analytical rigid surface by creating a planar wire from the given ConstrainedSketch object.

Path

```
mdb.models [name] .parts [name] .AnalyticRigidSurf2DPlanar
```

Required argument

sketch

A ConstrainedSketch object specifying the planar wire.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.4 AnalyticRigidSurfExtrude(...)

This method creates a first Feature object for an analytical rigid surface by extruding the given ConstrainedSketch object by the given depth, creating a surface.

Path

```
mdb.models [name] .parts [name] .AnalyticRigidSurfExtrude
```

Required argument

sketch

A ConstrainedSketch object specifying the planar wire.

Optional argument

depth

A Float specifying the extrusion depth. The default value is 1.0.

Return value

A Feature object.

Exceptions

None.

37.2.5 **AnalyticRigidSurfRevolve(...)**

This method creates a first Feature object for an analytical rigid surface by revolving the given ConstrainedSketch object by 360° about the Y-axis.

Path

```
mdb.models[name].parts[name].AnalyticRigidSurfRevolve
```

Required argument

sketch

A ConstrainedSketch object specifying the surface to be revolved.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.6 **AssignMidsurfaceRegion(...)**

This method assign a mid-surface property to sequence of Cell objects. If a reference representation of the part does not exist, it creates one. It also copies the *cells* to the reference representation and deletes the *cells* from the active representation of the part.

Path

```
mdb.models[name].parts[name].AssignMidsurfaceRegion
```

Required argument

cellList

A sequence of Cell objects specifying the regions that will be used for mid-surface construction. These regions will be copied to the reference representation of the part.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.7 BaseSolidExtrude(...)

This method creates a first Feature object by extruding the given ConstrainedSketch object by the given depth, creating a solid. The ConstrainedSketch object must define a closed profile.

Path

```
mdb.models[name].parts[name].BaseSolidExtrude
```

Required arguments

sketch

A ConstrainedSketch object specifying the plane shape to be extruded.

depth

A Float specifying the extrusion depth. Possible values are $10^{-5} \leq \text{depth} \leq 10^5$.

Optional arguments

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

Return value

A Feature object.

Exceptions

RangeError.

37.2.8 BaseSolidRevolve(...)

This method creates a first Feature object by revolving the given ConstrainedSketch object by the given angle, creating a solid. The ConstrainedSketch object must define a closed profile and an axis of revolution. The axis is defined by a single construction line.

Path

`mdb.models[name].parts[name].BaseSolidRevolve`

Required arguments

sketch

A ConstrainedSketch object specifying the shape to be revolved.

angle

A Float specifying the revolve angle in degrees. Possible values are $10^{-4} \leq \text{angle} \leq 360$.

Note: If *pitch* > 0, there is no upper limit for *angle*.

Optional arguments

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction, measured between corresponding points on the sketch when it has completed one full revolution about the axis of revolution. Possible values are $0 \leq \text{pitch} \leq 10^5$. The default value, 0, implies a normal revolve.

flipRevolveDirection

A Boolean specifying whether to override the direction of feature creation. If *flipRevolveDirection*=OFF, the default direction of revolution is used. If *flipRevolveDirection*=ON, the revolve direction is reversed. The default value is OFF.

flipPitchDirection

A Boolean specifying whether to override the direction of translation. If *flipPitchDirection*=OFF, the direction of translation is given by the direction of the revolve axis. If *flipPitchDirection*=ON, the translation direction is reversed. The default value is OFF.

moveSketchNormalToPath

A Boolean specifying whether to rotate the sketch so that it is normal to the path of revolution when using the *pitch* option. If *moveSketchNormalToPath*=OFF, the sketch plane remains parallel

to the revolve axis. If *moveSketchNormalToPath*=ON, the sketch is moved to match the angle created by the *pitch* before being revolved. The default value is OFF.

Return value

A Feature object.

Exceptions

RangeError.

37.2.9 BaseSolidSweep(...)

This method creates a first Feature object by sweeping the given profile ConstrainedSketch object along the path defined by the path ConstrainedSketch object, creating a solid. The profile ConstrainedSketch object must define a closed profile. The origin of the profile sketch is positioned at the start of the sweep path and swept perpendicular to the path. No checks are made for self-intersection.

Path

`mdb.models[name].parts[name].BaseSolidSweep`

Required arguments

sketch

A ConstrainedSketch object specifying the profile to be swept.

path

A ConstrainedSketch object specifying the path of the sweep.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.10 BaseShell(...)

This method creates a first Feature object by creating a planar shell from the given ConstrainedSketch object. The ConstrainedSketch object must define a closed profile.

Path

`mdb.models[name].parts[name].BaseShell`

Required argument

sketch

A ConstrainedSketch object specifying the planar shell.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.11 BaseShellExtrude(...)

This method creates a first Feature object by extruding the given ConstrainedSketch object by the given depth, creating a shell. The ConstrainedSketch object can define either an open or closed profile.

Path

`mdb.models[name].parts[name].BaseShellExtrude`

Required arguments

sketch

A ConstrainedSketch object specifying the shape to be extruded.

depth

A Float specifying the extrusion depth. Possible values are Floats > 0 .

Optional arguments

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be

specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

Return value

A Feature object.

Exceptions

RangeError.

37.2.12 BaseShellRevolve(...)

This method creates a first Feature object by revolving the given ConstrainedSketch object by the given angle, creating a shell. The ConstrainedSketch object can define either an open or closed profile and an axis of revolution. The axis is defined by a single construction line.

Path

`mdb.models[name].parts[name].BaseShellRevolve`

Required arguments

sketch

A ConstrainedSketch object specifying the shape to be revolved.

angle

A Float specifying the revolve angle in degrees. Possible values are $0 \leq \text{angle} \leq 360$.

Note: If *pitch* > 0, there is no upper limit for *angle*.

Optional arguments

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction, measured between corresponding points on the sketch when it has completed one full revolution about the axis of revolution. Possible values are $0 \leq \text{pitch} \leq 10^5$. The default value, 0, implies a normal revolve.

flipRevolveDirection

A Boolean specifying whether to override the direction of feature creation. If *flipRevolveDirection*=OFF, the default direction of revolution is used. If *flipRevolveDirection*=ON, the revolve direction is reversed. The default value is OFF.

flipPitchDirection

A Boolean specifying whether to override the direction of translation. If *flipPitchDirection*=OFF, the direction of translation is given by the direction of the revolve axis. If *flipPitchDirection*=ON, the translation direction is reversed. The default value is OFF.

moveSketchNormalToPath

A Boolean specifying whether to rotate the sketch so that it is normal to the path of revolution when using the *pitch* option. If *moveSketchNormalToPath*=OFF, the sketch plane remains parallel to the revolve axis. If *moveSketchNormalToPath*=ON, the sketch is moved to match the angle created by the *pitch* before being revolved. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.13 BaseShellSweep(...)

This method creates a first Feature object by sweeping the given section ConstrainedSketch object along the path defined by the path ConstrainedSketch object, creating a shell. The ConstrainedSketch object can define either an open or closed profile. The origin of the profile sketch is positioned at the start of the sweep path and swept perpendicular to the path. No checks are made for self-intersection.

Path

```
mdb.models [name] .parts [name] .BaseShellSweep
```

Required arguments

sketch

A ConstrainedSketch object specifying the section to be swept.

path

A ConstrainedSketch object specifying the path of the sweep.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.14 BaseWire(...)

This method creates a first Feature object by creating a planar wire from the given ConstrainedSketch object.

Path

```
mdb.models [name] .parts [name] .BaseWire
```

Required argument

sketch

A ConstrainedSketch object specifying the planar wire.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.15 BlendFaces(...)

This method creates a Feature object by creating new faces that blends two sets of faces.

Path

```
mdb.models [name] .parts [name] .BlendFaces
```

Required arguments

side1

A sequence of Edge objects specifying one side of the blend. The edges must form a continuous chain without branches.

side2

A sequence of Edge or Face objects specifying the second side of the blend. If *side2* contains Edge objects then they must form a continuous chain without branches.

Optional arguments

method

A SymbolicConstant indicating a method for creating blends. This argument is a required argument if *side2* contains Edge object and it is ignored if *side2* contains Faceobjects. It can have one of the following values:

- TANGENT: The blend is tangent to the sides.
- SHORTEST_PATH: The blend connects the two sides based on linear interpolation between the two sides.
- SPECIFY_PATH: The blend connects the two sides along a specified path.

path

An Edge object that connects *side1* to *side2* and specifies the path for creating the blend. This argument is required if *method*=SPECIFY_PATH; otherwise, it is ignored.

Return value

A Feature object.

Exceptions

None.

37.2.16 Chamfer(...)

This method creates an additional Feature object by chamfering the given list of edges with a given length.

Path

```
mdb.models[name].parts[name].Chamfer
```

Required arguments

length

A Float specifying the length of the chamfer.

edgeList

A sequence of Edge objects specifying the edges to chamfer.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.17 Mirror(...)

This method mirrors existing part geometry across a plane to create new geometry.

Path

```
mdb.models [name] .parts [name] .Mirror
```

Required argument

mirrorPlane

A Datum plane object or a planar Face object.

keepOriginal

A boolean specifying whether or not the original part geometry should be retained.

Optional arguments

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.18 ConvertToAnalytical()

This method attempts to change entities into a simpler form that will speed up processing and make entities available during feature operations.

Path

```
mdb.models [name] .parts [name] .ConvertToAnalytical
```

Arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.19 **ConvertToPrecise(...)**

This method attempts to change imprecise entities so that the geometry becomes precise.

Path

`mdb.models[name].parts[name].ConvertToPrecise`

Required arguments

None.

Optional argument

method

A SymbolicConstant specifying the method to be used to convert the part to precise. Possible values are RECOMPUTE_GEOMETRY and TIGHTEN_GAPS. The default value is RECOMPUTE_GEOMETRY.

Return value

A Feature object.

Exceptions

None.

37.2.20 **CoverEdges(...)**

This method generates a face using the given edges as the face's boundaries. The **CoverEdges** method generates a face by creating the geometry consisting of the underlying surface, associated edges, and vertices.

Path

`mdb.models[name].parts[name].CoverEdges`

Required argument*edgeList*

A sequence of Edge objects specifying the edges that bound the new face.

Optional argument*tryAnalytical*

A Boolean specifying whether the newly created face should be analytical or not. The default is False.

Return value

A Feature object.

Exceptions

If the given boundary is not a closed loop:

Parterror: Cannot find a closed loop

If the given boundary contains a zero length component:

Parterror: Cannot find a closed loop

If the underlying surface is too difficult to fit:

Parterror: Cannot construct face geometry

37.2.21 Cut(...)

This method creates an additional Feature object by cutting a hole using the given ConstrainedSketch object.

Path

```
mdb.models [name] .parts [name] .Cut
```

Required arguments*sketchPlane*

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar cut.

Optional argument

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM.

Return value

A Feature object.

Exceptions

None.

37.2.22 CutExtrude(...)

This method creates an additional Feature object by extruding the given ConstrainedSketch object by the given depth and cutting away material in the solid and shell regions of the part. The ConstrainedSketch object must define a closed profile. The **CutExtrude** method creates a blind cut (using *depth*), an up-to-face cut (using *upToFace*), or a through-all cut (if *depth* and *upToFace* are not specified).

Path

`mdb.models [name] .parts [name] .CutExtrude`

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM.

sketch

A ConstrainedSketch object specifying the planar sketch to be extruded.

Optional arguments

depth

A Float specifying the extrusion depth. If *depth* is specified, the cut will be a blind cut. The default is to not specify a depth.

upToFace

A Face specifying the face up to which to cut. If *upToFace* is specified, the cut will be an up-to-face cut. The default is to not specify a face.

Note: If neither *depth* nor *upToFace* is specified, the cut will be a through-all cut.

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

flipExtrudeDirection

A Boolean specifying whether to override the direction of feature creation. If the value is OFF, it means use the direction defined by the *sketchPlaneSide*; if the value is ON, it means use the opposite direction to the one defined by *sketchPlaneSide*. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.23 CutLoft(...)

This method creates an additional Feature object by lofting between the given sections and cutting away material from the part. You define the sections using a sequence of edges from the part or an EdgeArray .

Path

```
mdb.models [name] .parts [name] .CutLoft
```

Required argument*loftsections*

A sequence of sequences of edges specifying the cross-sections to be lofted. Each outer sequence specifies a section through which the method will pass the loft feature. Each outer sequence can

be defined as a sequence of edges or as an *EdgeArray*. The edges specifying a section must form a simple closed profile and must not contain multiple loops.

Optional arguments

startCondition

A *SymbolicConstant* specifying the tangent direction at the start section of the loft feature. Possible values are NONE, NORMAL, RADIAL, and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *startCondition* argument in conjunction with the *endCondition* argument.

endCondition

A *SymbolicConstant* specifying the tangent direction at the end section of the loft feature. Possible values are NONE, NORMAL, RADIAL, and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *endCondition* argument in conjunction with the *startCondition* argument.

startTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the start section lies. You must specify the *startTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \textit{startTangent} \leq 180.0$.

startMagnitude

A Float specifying the magnitude of the *startTangent*. You must specify the *startMagnitude* argument if *startCondition*=SPECIFIED. Possible values are $0.0 < \textit{startMagnitude} < 100.0$.

endTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the end section lies. You must specify the *endTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \textit{endTangent} \leq 180.0$.

endMagnitude

A Float specifying the magnitude of the *endTangent*. This argument is to be used when the *endCondition* argument has the value SPECIFIED. Possible values are $0.0 < \textit{endMagnitude} < 100.0$.

globalSmoothing

A Boolean specifying whether each path defined in the *paths* argument is applied locally or globally.

- If the path is applied locally, its effect is felt only on faces created from the edges on the *loftSections* through which the *paths* pass through.
- If the path is applied globally, an averaging algorithm is applied over all the paths defined and is distributed over all the faces created.

The default value is ON (globally).

Return value

A Feature object.

Exceptions

None.

37.2.24 CutRevolve(...)

This method creates an additional Feature object by revolving the given ConstrainedSketch object by the given angle and cutting away material from the part. The ConstrainedSketch object must define a closed profile and an axis of revolution.

Path

`mdb.models[name].parts[name].CutRevolve`

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM.

sketch

A ConstrainedSketch object specifying the planar sketch to be revolved.

angle

A Float specifying the angle in degrees to be revolved.

Optional arguments

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction, measured between corresponding points on the sketch when it has completed one full revolution about the axis of revolution. Possible values are $0 \leq \text{pitch} \leq 10^5$. The default value, 0, implies a normal revolve.

flipRevolveDirection

A Boolean specifying whether to override the direction of feature creation. If *flipRevolveDirection*=OFF, the default direction of revolution is used. If *flipRevolveDirection*=ON, the revolve direction is reversed. The default value is OFF.

flipPitchDirection

A Boolean specifying whether to override the direction of translation. If *flipPitchDirection*=OFF, the direction of translation is given by the direction of the revolve axis. If *flipPitchDirection*=ON, the translation direction is reversed. The default value is OFF.

moveSketchNormalToPath

A Boolean specifying whether to rotate the sketch so that it is normal to the path of revolution when using the *pitch* option. If *moveSketchNormalToPath*=OFF, the sketch plane remains parallel to the revolve axis. If *moveSketchNormalToPath*=ON, the sketch is moved to match the angle created by the *pitch* before being revolved. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.25 CutSweep(...)

This method creates an additional Feature object by sweeping the given ConstrainedSketch object along a path which may be a ConstrainedSketch or a sequence of Edge objects and cutting away material from the part. If the profile section is a ConstrainedSketch object, it must define a closed profile. The section sketch can be created at the normal plane at the start of the sweep path or it may be created on a Datum plane or a planar Face . No checks are made for self-intersection.

Path

`mdb.models [name] .parts [name] .CutSweep`

Required arguments

path

Path may either be a ConstrainedSketch object or a sequence of Edge objects specifying the path of the sweep.

profile

Profile may either be a ConstrainedSketch object or a Face object specifying the section to be swept.

Optional arguments*pathPlane*

A Datum plane object or a planar Face object. Only required when path is a ConstrainedSketch object.

pathUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the path sketch. Only required when path is a ConstrainedSketch object.

pathOrientation

A SymbolicConstant specifying the orientation of *pathUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. Default value is RIGHT. Only required when path is a ConstrainedSketch object.

sketchPlane

A Datum plane object or a planar Face object specifying the plane on which to sketch the profile. Not required when profile is a Face object. When profile is chosen as a ConstrainedSketch object, user may or may not give this as input. If user does not give this as input, the normal plane at the start of the path will be the sketchPlane.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the profile sketch. Only required when profile is a ConstrainedSketch object.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. Default value is RIGHT. Only required when profile is a ConstrainedSketch object.

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

profileNormal

A Boolean specifying whether to keep the profile normal same as original or varying through out the sweep path. When *profileNormal*=OFF, the profile normal will vary through out the sweep path. When *profileNormal*=ON, the profile normal will be same as original through out the sweep path. The default value is OFF.

flipSweepDirection

A Boolean specifying whether to flip the direction in which sweep operation will be performed. When *flipSweepDirection*=OFF, sweep operation will be performed in the direction of path direction. When *flipSweepDirection*=ON, sweep operation will be performed in the direction opposite to the path direction. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.26 ExtendFaces(...)

This method extends faces along its free edges by offsetting the external edges along the surfaces. One of *distance*, *upToReferenceRep*, or *upToFaces* must be used to specify how far the faces need to be extended.

Path

```
mdb.models [name] .parts [name] .ExtendFaces
```

Required arguments

None.

Optional arguments

faces

A sequence of Face objects specifying the faces to be extended. The faces cannot belong to the reference representation. The *faces* and *extendAlong* arguments are mutually exclusive. One of them must be specified.

extendAlong

A sequence of Edge objects specifying the edges where to extend the faces. Only free edges are considered. The interior edges will be ignored. The *faces* and *extendAlong* arguments are mutually exclusive. One of them must be specified.

distance

A Float indicating the distance to extend the faces along the edges. Either *distance*, *upToReferenceRep*, or *upToFaces* must be specified.

upToFaces

A sequence of Face objects specifying the faces that the selected faces should be extended up to.

trimToExtendedTargetSurfaces

A Boolean indicating that the surfaces of up to target faces should be extended before extending and trimming the selected faces. The default value is True.

upToReferenceRep

A Boolean indicating that the selected faces should be extended along the selected edges and be trimmed along their intersection with the reference representation.

Return value

A Feature object.

Exceptions

None.

37.2.27 FaceFromElementFaces(...)

This method creates a geometry face from a collection of orphan element faces.

Path

`mdb.models[name].parts[name].FaceFromElementFaces`

Required argument*elementFaces*

A Region object specifying the collection of orphan element faces.

Optional arguments*stitch*

A Boolean specifying whether the created geometry face should be stitched with existing geometry faces. Default value is TRUE.

stitchTolerance

A Float indicating the maximum gap to be stitched. The value should be smaller than the minimum feature size and bigger than the maximum gap expected to be stitched in the model. Otherwise this command may remove small (sliver) edges that are smaller than the tolerance. If stitch tolerance is not provided then default value of 0.001 will be used for stitching.

analyticFitTolerance

A Float indicating the analytical surface fitting tolerance. If analytical tolerance is not provided then default value of 0.015 will be used for analytical surface fitting.

associateFace

A Boolean specifying whether the created geometry face should be associated with the mesh. Default value is TRUE.

Return value

A Feature object.

Exceptions

None.

37.2.28 HoleBlindFromEdges(...)

This method creates an additional Feature object by creating a circular blind hole of the given diameter and depth and cutting away material in the solid and shell regions of the part. The center of the hole is offset from two non-parallel straight edges by the given distances.

Path

`mdb.models[name].parts[name].HoleBlindFromEdges`

Required arguments

plane

A Datum plane object or a planar Face object.

planeSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

diameter

A Float specifying the diameter of the hole.

edge1

An Edge object specifying the edge from which *distance1* is measured.

distance1

A Float specifying the offset from *edge1*.

edge2

An Edge object specifying the edge from which *distance2* is measured.

distance2

A Float specifying the offset from *edge2*.

depth

A Float specifying the depth of the hole.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.29 HoleFromEdges(...)

This method creates an additional Feature object by creating a circular hole of the given diameter in a 2D planar part and cutting away material in the shell and wire regions of the part. The center of the hole is offset from two non-parallel straight edges by the given distances.

Path

```
mdb.models[name].parts[name].HoleFromEdges
```

Required arguments

diameter

A Float specifying the diameter of the hole.

edge1

An Edge object specifying the edge from which *distance1* is measured.

distance1

A Float specifying the offset from *edge1*.

edge2

An Edge object specifying the edge from which *distance2* is measured.

distance2

A Float specifying the offset from *edge2*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.30 HoleThruAllFromEdges(...)

This method creates an additional Feature object by creating a circular through hole of the given diameter and cutting away material in the solid and shell regions of the part. The center of the hole is offset from two non-parallel straight edges by the given distances.

Path

`mdb.models[name].parts[name].HoleThruAllFromEdges`

Required arguments

plane

A Datum plane object or a planar Face object.

planeSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

diameter

A Float specifying the diameter of the hole.

edge1

An Edge object specifying the edge from which *distance1* is measured.

distance1

A Float specifying the offset from *edge1*.

edge2

An Edge object specifying the edge from which *distance2* is measured.

distance2

A Float specifying the offset from *edge2*.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.31 MergeEdges(...)

This method merges edges either by extending the user selection or using only the selected edges.

Path

`mdb.models[name].parts[name].MergeEdges`

Required arguments

None.

Optional arguments*edgeList*

A sequence of Edge objects specifying the edges to be merged.

extendSelection

A Boolean specifying whether the user selection needs to be extended to include edges till branching occurs. Branching is said to occur when the vertex of an edge is shared by more than two edges.

Return value

A Feature object.

Exceptions

None.

37.2.32 OffsetFaces(...)

This method creates new faces by offsetting existing faces.

Path

`mdb.models[name].parts[name].OffsetFaces`

Required argument*faceList*

A sequence of Face objects specifying the faces that will be offset. The faces may belong to the part or to the reference representation associated with the part.

Optional arguments*distance*

A Float indicating the distance to offset the faces. Either *distance* or *targetFaces* must be specified.

targetFaces

A sequence of Face objects whose distance to the faces argument together with the *targetFacesMethod* determines the distance to offset the faces. Either *distance* or *targetFaces* must be specified.

targetFacesMethod

A SymbolicConstant indicating how to calculate the distance to offset. It can have one of the following values:

- `HALF_OF_AVERAGE`: Offset the faces by a distance equals to half the average distance to target faces.

PART COMMANDS

- **CLOSEST_POINT_FRACTION**: Offset the faces by a distance equals to the fraction of the distance to the approximate closest point on the selected target faces.
- **FARTHEST_POINT_FRACTION**: Offset the faces by a distance equals to the fraction of the distance to the approximate farthest point on the selected target faces.

fractionDistance

A Float indicating the fraction of the distance to the closest or the farthest point on the target faces. It's default value is 0.5.

trimToReferenceRep

A Boolean indicating whether to extend the offset faces and trim them along their intersection with the reference representation.

Return value

A Feature object.

Exceptions

None.

37.2.33 RemoveCells(...)

This method converts a solid entity to a shell entity.

Path

```
mdb.models[name].parts[name].RemoveCells
```

Required argument

cellList

A sequence of Cell objects specifying the cells to remove.

Optional arguments

None.

Return value

A Boolean value.

Exceptions

If the intended volume to be turned into a shell entity is not three-dimensional.
Parterror: Geometry that is not 3-dimensional does not contain cells.

37.2.34 RemoveFaces(...)

This method removes faces from a solid entity or from a shell entity.

Path

```
mdb.models [name] .parts [name] .RemoveFaces
```

Required argument

faceList

A sequence of Face objects specifying the faces to remove.

Optional argument

deleteCells

A Boolean specifying whether all cells are to be deleted when the faces are removed. The default value is False.

Return value

A Feature object.

Exceptions

None.

37.2.35 RemoveFacesAndStitch(...)

This method removes faces from a solid entity and attempts to close the resulting gap by extending the neighboring faces of the solid.

Path

```
mdb.models [name] .parts [name] .RemoveFacesAndStitch
```

Required argument

faceList

A sequence of Face objects specifying the faces to remove.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.36 RemoveRedundantEntities(...)

This method removes redundant edges and vertices from a solid or a shell entity. One of the two arguments is required.

Path

```
mdb.models [name] .parts [name] .RemoveRedundantEntities
```

Required arguments

None.

Optional arguments

vertexList

A sequence of Vertex objects specifying the vertices to be removed.

edgeList

A sequence of Edge objects specifying the edges to be removed.

removeEdgeVertices

A Boolean specifying whether the vertices of the redundant edges need to be removed. The default is True.

Return value

A Feature object.

Exceptions

If the selected entity is not a redundant entity.

Parterror: None of the selected entities are redundant.

37.2.37 RepairFaceNormals(...)

This method works on the entire part or a sequence of shell faces. When the entire part is selected, it aligns all the shell face normals, and inverts all of the solid faces' normals if the solid was originally inside out. When a few shell faces are selected, it inverts the normals of the selected faces.

Path

```
mdb.models [name] .parts [name] .RepairFaceNormals
```

Required arguments

None.

Optional argument

faceList

A sequence of Face objects.

Return value

A Feature object.

Exceptions

None.

37.2.38 RepairInvalidEdges(...)

This method repairs invalid edges. It will always attempt to improve edges even if none of selected edges are initially invalid and may leave behind invalid edges that could not be repaired.

Path

```
mdb.models[name].parts[name].RepairInvalidEdges
```

Required argument

edgeList

A sequence of Edge objects.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.39 RepairSliver(...)

This method repairs the selected sliver from the selected face. The sliver area is specified using two points. A face partition is carried out at the specified points and the smaller of the two faces is removed.

Path

`mdb.models[name].parts[name].RepairSliver`

Required arguments

face

A Face object specifying the face on which the sliver is located.

point1

A point specifying the location for partition creation. It can be a Vertex object, an Interesting Point or three coordinates specifying the point on an edge of the *face*.

point2

A point specifying the location for partition creation. It can be a Vertex object, an Interesting Point or three coordinates specifying the point on an edge of the *face*.

Optional argument

toleranceChecks

A Boolean specifying whether to use internal tolerance checks to restrict the size of the sliver face being removed. The default is True.

Return value

A Feature object.

Exceptions

None.

37.2.40 RepairSmallEdges(...)

This method repairs small edges. This method will attempt to replace selected small edges with vertices and extend the adjacent faces and edges. This method might leave behind some small edges that cannot be removed.

Path

`mdb.models[name].parts[name].RepairSmallEdges`

Required argument

edgeList

A sequence of Edge objects.

Optional argument*toleranceChecks*

A Boolean specifying whether to use internal tolerance checks to restrict the size of the edges being removed. The default is True.

Return value

A Feature object.

Exceptions

None.

37.2.41 RepairSmallFaces(...)

This method repairs small faces. It will attempt to replace the selected small faces with edges or vertices and extend the adjacent faces. This method might leave behind some small faces that cannot be removed.

Path

```
mdb.models[name].parts[name].RepairSmallFaces
```

Required argument*faceList*

A sequence of Face objects.

Optional argument*toleranceChecks*

A Boolean specifying whether to use internal tolerance checks to restrict the size of the faces being removed. The default is True.

Return value

A Feature object.

Exceptions

None.

37.2.42 ReplaceFaces(...)

This method replaces the selected faces with a single face. If one single face is selected, that alone is replaced with a new face.

Path

`mdb.models[name].parts[name].ReplaceFaces`

Required argument

faceList

A sequence of Face objects to be replaced.

Optional argument

stitch

A Boolean specifying whether the newly created face needs to be stitched to the existing geometry. The default is True.

Return value

A Feature object.

Exceptions

None.

37.2.43 Round(...)

This method creates an additional Feature object by rounding (filleting) the given list of entities with the given radius.

Path

`mdb.models[name].parts[name].Round`

Required arguments

radius

A Float specifying the radius of the fillets.

edgeList

A sequence of Edge objects. Solid and Shell edges of a part can be rounded. The operation will fail for non-manifold edges. The *edgeList* and *vertexList* arguments are mutually exclusive. One of them must be specified.

vertexList

A sequence of Vertex objects. Vertices that are connected to two wire edges can be rounded. The operation will fail for a vertex connected to a face. The *edgeList* and *vertexList* arguments are mutually exclusive. One of them must be specified.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.2.44 Shell(...)

This method creates an additional Feature object by creating a planar shell from the given ConstrainedSketch object. The ConstrainedSketch object must define a closed profile.

Path

```
mdb.models[name].parts[name].Shell
```

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar shell.

Optional argument

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

None.

37.2.45 ShellExtrude(...)

This method creates an additional Feature object by extruding the given ConstrainedSketch object by the given depth, creating a shell protrusion. The ConstrainedSketch object can define either an open or closed profile.

Path

`mdb.models[name].parts[name].ShellExtrude`

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar sketch to be extruded.

Optional arguments

depth

A Float specifying the extrusion depth. The default is to not specify a depth. Either *depth* or *upToFace* must be used to define the extrusion depth.

upToFace

A Face specifying the face up to which to extrude. If *upToFace* is specified, the extrusion will be an up-to-face extrusion. The default is to not specify a face. Either *depth* or *upToFace* must be used to define the extrusion depth.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be

specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

flipExtrudeDirection

A Boolean specifying whether to override the direction of feature creation. If the value is OFF, it means use the direction defined by the *sketchPlaneSide*; if the value is ON, it means use the opposite direction to the one defined by *sketchPlaneSide*. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.46 ShellLoft(...)

This method creates an additional Feature object by lofting between the given sections and adding shell faces to the part. You define the sections using a sequence of edges from the part or an EdgeArray .

Path

```
mdb.models [name] .parts [name] .ShellLoft
```

Required argument

loftsections

A sequence of sequences of edges specifying the cross-sections to be lofted. Each outer sequence specifies a section through which the method will pass the loft feature. Each outer sequence can be defined as a sequence of edges or as an EdgeArray. The edges specifying a section must form a simple closed profile and must not contain multiple loops.

Optional arguments

startCondition

A SymbolicConstant specifying the tangent direction at the start section of the loft feature. Possible values are NONE, NORMAL, RADIAL and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *startCondition* argument in conjunction with the *endCondition* argument.

PART COMMANDS

endCondition

A SymbolicConstant specifying the tangent direction at the end section of the loft feature. Possible values are NONE, NORMAL, RADIAL and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *endCondition* argument in conjunction with the *startCondition* argument.

startTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the start section lies. You must specify the *startTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \textit{startTangent} \leq 180.0$.

startMagnitude

A Float specifying the magnitude of the *startTangent*. You must specify the *startMagnitude* argument if *startCondition*=SPECIFIED. Possible values are $0.0 < \textit{startMagnitude} < 100.0$.

endTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the end section lies. You must specify the *endTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \textit{endTangent} \leq 180.0$.

endMagnitude

A Float specifying the magnitude of the *endTangent*. This argument is to be used when the *endCondition* argument has the value SPECIFIED. Possible values are $0.0 < \textit{endMagnitude} < 100.0$.

paths

A sequence of sequences of edges that pass through each section in the loft feature. Each sequence specifies a path followed by the face or an edge created by a loft feature. Each path must start at the first section, end at the last section, and pass through each section. In addition, the order of the sequences must be the same as the order of the sections in the *loftsections* argument. Each path must not self-intersect and must be tangent continuous. In addition, the paths must not intersect each other. You cannot use the *paths* argument in conjunction with the *startCondition* and *endCondition* arguments.

globalSmoothing

A Boolean specifying whether each path defined in the *paths* argument is applied locally or globally.

- If the path is applied locally, its effect is felt only on faces created from the edges on the *loftsections* through which the *paths* pass through.
- If the path is applied globally, an averaging algorithm is applied over all the paths defined and is distributed over all the faces created.

The default value is ON (globally).

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.47 ShellRevolve(...)

This method creates an additional Feature object by revolving the given ConstrainedSketch object by the given angle, creating a shell protrusion. The ConstrainedSketch object can define either an open or closed profile and an axis of revolution. The axis is defined by a single construction line. For a description of the plane positioning arguments, see **SolidExtrude**.

Path

`mdb.models[name].parts[name].ShellRevolve`

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar sketch to be revolved.

angle

A Float specifying the angle in degrees to be revolved.

Optional arguments

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction, measured between corresponding points on the sketch when it has completed one full revolution about the axis of revolution. Possible values are $0 \leq pitch \leq 10^5$. The default value, 0, implies a normal revolve.

PART COMMANDS

flipRevolveDirection

A Boolean specifying whether to override the direction of feature creation. If *flipRevolveDirection*=OFF, the default direction of revolution is used. If *flipRevolveDirection*=ON, the revolve direction is reversed. The default value is OFF.

flipPitchDirection

A Boolean specifying whether to override the direction of translation. If *flipPitchDirection*=OFF, the direction of translation is given by the direction of the revolve axis. If *flipPitchDirection*=ON, the translation direction is reversed. The default value is OFF.

moveSketchNormalToPath

A Boolean specifying whether to rotate the sketch so that it is normal to the path of revolution when using the *pitch* option. If *moveSketchNormalToPath*=OFF, the sketch plane remains parallel to the revolve axis. If *moveSketchNormalToPath*=ON, the sketch is moved to match the angle created by the *pitch* before being revolved. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.48 ShellSweep(...)

This method creates an additional Feature object by sweeping the given ConstrainedSketch object or a sequence of Edge objects along a path which may be a ConstrainedSketch or a sequence of Edge objects, creating a shell swept protrusion. The section can be an open or a closed profile. The section sketch can be created at the normal plane at the start of the sweep path or it may be created on a Datum plane or a planar Face . No checks are made for self-intersection.

Path

```
mdb.models[name].parts[name].ShellSweep
```

Required arguments

path

Path may either be a ConstrainedSketch object or a sequence of Edge objects specifying the path of the sweep.

profile

Profile may either be a *ConstrainedSketch* object or a sequence of *Edge* objects specifying the section to be swept.

Optional arguments*pathPlane*

A Datum plane object or a planar *Face* object. Only required when *path* is a *ConstrainedSketch* object.

pathUpEdge

An *Edge* object or a Datum axis object specifying the vertical (*Y*) direction of the path sketch. Only required when *path* is a *ConstrainedSketch* object.

pathOrientation

A *SymbolicConstant* specifying the orientation of *pathUpEdge* on the sketch. Possible values are *RIGHT*, *LEFT*, *TOP*, and *BOTTOM*. Default value is *RIGHT*. Only required when *path* is a *ConstrainedSketch* object.

sketchPlane

A Datum plane object or a planar *Face* object specifying the plane on which to sketch the profile. Not required when *profile* is a *Face* object. When *profile* is chosen as *ConstrainedSketch* object, user may or may not give this as input. If user does not give this as input, the normal plane at the start of the path will be the *sketchPlane*.

sketchUpEdge

An *Edge* object or a Datum axis object specifying the vertical (*Y*) direction of the profile sketch. Only required when *profile* is a *ConstrainedSketch* object.

sketchOrientation

A *SymbolicConstant* specifying the orientation of *sketchUpEdge* on the sketch. Possible values are *RIGHT*, *LEFT*, *TOP*, and *BOTTOM*. Default value is *RIGHT*. Only required when *profile* is a *ConstrainedSketch* object.

draftAngle

A *Float* specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A *Float* specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

profileNormal

A Boolean specifying whether to keep the profile normal same as original or varying through out the sweep path. When *profileNormal*=OFF, the profile normal will vary through out the sweep path. When *profileNormal*=ON, the profile normal will be same as original through out the sweep path. The default value is OFF.

flipSweepDirection

A Boolean specifying whether to flip the direction in which sweep operation will be performed. When *flipSweepDirection*=OFF, sweep operation will be performed in the direction of path direction. When *flipSweepDirection*=ON, sweep operation will be performed in the direction opposite to the path direction. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.49 SolidExtrude(...)

This method creates an additional Feature object by extruding the given ConstrainedSketch object by the given depth, creating a solid protrusion. The ConstrainedSketch object must define a closed profile.

Path

`mdb.models[name].parts[name].SolidExtrude`

Required arguments

sketchPlane

A Datum plane object or a planar Face object.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar sketch to be extruded.

Optional arguments*depth*

A Float specifying the extrusion depth. The default is to not specify a depth. Either *depth* or *upToFace* must be used to define the extrusion depth.

upToFace

A Face specifying the face up to which to extrude. If *upToFace* is specified, the extrusion will be an up-to-face extrusion. The default is to not specify a face. Either *depth* or *upToFace* must be used to define the extrusion depth.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

flipExtrudeDirection

A Boolean specifying whether to override the direction of feature creation. If the value is OFF, it means use the direction defined by the *sketchPlaneSide*; if the value is ON, it means use the opposite direction to the one defined by *sketchPlaneSide*. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.50 SolidLoft(...)

This method creates an additional Feature object by lofting between the given sections and adding material to the part. You define the sections using a sequence of edges from the part or an EdgeArray .

Path

```
mdb.models [name] .parts [name] .SolidLoft
```

Required argument*loftsections*

A sequence of sequences of edges specifying the cross-sections to be lofted. Each outer sequence specifies a section through which Abaqus will pass the loft feature. Each outer sequence can be defined as a sequence of edges or as an EdgeArray. The edges specifying a section must form a simple closed profile and must not contain multiple loops.

Optional arguments*startCondition*

A SymbolicConstant specifying the tangent direction at the start section of the loft feature. Possible values are NONE, NORMAL, RADIAL and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *startCondition* argument in conjunction with the *endCondition* argument.

endCondition

A SymbolicConstant specifying the tangent direction at the end section of the loft feature. Possible values are NONE, NORMAL, RADIAL and SPECIFIED. You can specify this argument only if the start and end sections are planar. You cannot use this argument in conjunction with the *path* argument. You must use the *endCondition* argument in conjunction with the *startCondition* argument.

startTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the start section lies. You must specify the *startTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \text{startTangent} \leq 180.0$.

startMagnitude

A Float specifying the magnitude of the *startTangent*. You must specify the *startMagnitude* argument if *startCondition*=SPECIFIED. Possible values are $0.0 < \text{startMagnitude} < 100.0$.

endTangent

A Float specifying the angle in degrees of the tangent with respect to the plane in which the end section lies. You must specify the *endTangent* argument if *startCondition*=SPECIFIED. Possible values are $0.0 \leq \text{endTangent} \leq 180.0$.

endMagnitude

A Float specifying the magnitude of the *endTangent*. This argument is to be used when the *endCondition* argument has the value SPECIFIED. Possible values are $0.0 < \text{endMagnitude} < 100.0$.

paths

A sequence of sequences of edges that pass through each section in the loft feature. Each sequence specifies a path followed by the face or an edge created by a loft feature. Each path must start at the first section, end at the last section, and pass through each section. In addition, the order of the sequences must be the same as the order of the sections in the *loftsections* argument. Each path must not self-intersect and must be tangent continuous. In addition, the paths must not intersect each other. You cannot use the *paths* argument in conjunction with the *startCondition* and *endCondition* arguments.

globalSmoothing

A Boolean specifying whether each path defined in the *paths* argument is applied locally or globally.

- If the path is applied locally, its effect is felt only on faces created from the edges on the *loftsections* through which the *paths* pass through.
- If the path is applied globally, an averaging algorithm is applied over all the paths defined and is distributed over all the faces created.

The default value is ON (globally).

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.51 SolidRevolve(...)

This method creates an additional Feature object by revolving the given ConstrainedSketch object by the given angle, creating a solid protrusion. The ConstrainedSketch object must define a closed profile and an axis of revolution. The axis is defined by a single construction line.

Path

```
mdb.models[name].parts[name].SolidRevolve
```

Required arguments*sketchPlane*

A Datum plane object or a planar Face object.

PART COMMANDS

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar sketch to be revolved.

angle

A Float specifying the angle in degrees to be revolved.

Optional arguments

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction, measured between corresponding points on the sketch when it has completed one full revolution about the axis of revolution. Possible values are $0 \leq \text{pitch} \leq 10^5$. The default value, 0, implies a normal revolve.

flipRevolveDirection

A Boolean specifying whether to override the direction of feature creation. If *flipRevolveDirection*=OFF, the default direction of revolution is used. If *flipRevolveDirection*=ON, the revolve direction is reversed. The default value is OFF.

flipPitchDirection

A Boolean specifying whether to override the direction of translation. If *flipPitchDirection*=OFF, the direction of translation is given by the direction of the revolve axis. If *flipPitchDirection*=ON, the translation direction is reversed. The default value is OFF.

moveSketchNormalToPath

A Boolean specifying whether to rotate the sketch so that it is normal to the path of revolution when using the *pitch* option. If *moveSketchNormalToPath*=OFF, the sketch plane remains parallel to the revolve axis. If *moveSketchNormalToPath*=ON, the sketch is moved to match the angle created by the *pitch* before being revolved. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.52 SolidSweep(...)

This method creates an additional Feature object by sweeping the given ConstrainedSketch object or a Face object along a path which may be a ConstrainedSketch or a sequence of Edge objects, creating a solid swept protrusion. If the profile section is a ConstrainedSketch object, it must define a closed profile. The section sketch can be created at the normal plane at the start of the sweep path or it may be created on a Datum plane or a planar Face . No checks are made for self-intersection.

Path

`mdb.models[name].parts[name].SolidSweep`

Required arguments*path*

Path may either be a ConstrainedSketch object or a sequence of Edge objects specifying the path of the sweep.

profile

Profile may either be a ConstrainedSketch object or a Face object specifying the section to be swept.

Optional arguments*pathPlane*

A Datum plane object or a planar Face object. Only required when path is a ConstrainedSketch object.

pathUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the path sketch. Only required when path is a ConstrainedSketch object.

pathOrientation

A SymbolicConstant specifying the orientation of *pathUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. Default value is RIGHT. Only required when path is a ConstrainedSketch object.

sketchPlane

A Datum plane object or a planar Face object specifying the plane on which to sketch the profile. Not required when profile is a Face object. When profile is chosen as ConstrainedSketch object, user may or may not give this as input. If user does not give this as input, the normal plane at the start of the path will be the sketchPlane.

PART COMMANDS

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the profile sketch. Only required when profile is a ConstrainedSketch object.

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. Default value is RIGHT. Only required when profile is a ConstrainedSketch object.

draftAngle

A Float specifying the draft angle in degrees. Possible values are $-90.0 \leq \text{draftAngle} \leq 90.0$. By convention, for a positive draft angle an outer loop will draft outward and an inner loop will draft inward. The opposite is true for a negative draft angle. The default value, 0, implies a normal extrude. The arguments *draftAngle* and *pitch* are mutually exclusive.

pitch

A Float specifying the pitch. The pitch is the distance traveled along the axial direction by the sketch when the sketch has completed one full revolution about the twist axis. Pitch can be specified as positive or negative to achieve right-handed or left-handed twist about the twist axis, respectively. The default value, 0, implies a normal extrude. Possible values are $-10^5 \leq \text{pitch} \leq 10^5$. The arguments *draftAngle* and *pitch* are mutually exclusive.

profileNormal

A Boolean specifying whether to keep the profile normal same as original or varying through out the sweep path. When *profileNormal*=OFF, the profile normal will vary through out the sweep path. When *profileNormal*=ON, the profile normal will be same as original through out the sweep path. The default value is OFF.

flipSweepDirection

A Boolean specifying whether to flip the direction in which sweep operation will be performed. When *flipSweepDirection*=OFF, sweep operation will be performed in the direction of path direction. When *flipSweepDirection*=ON, sweep operation will be performed in the direction opposite to the path direction. The default value is OFF.

keepInternalBoundaries

A Boolean specifying whether internal boundaries will be retained. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.53 **Stitch(...)**

This method attempts to create a valid part by binding together free and imprecise edges of all the faces of a part. If *edgeList* is not given, a global stitch will be performed. If *stitchTolerance* is not specified, a value of 1.0 will be used.

Path

```
mdb.models [name] .parts [name] .Stitch
```

Required arguments

None.

Optional arguments

edgeList

A sequence of Edge objects specifying the edges that need to be stitched.

stitchTolerance

A Float indicating the maximum gap to be stitched. The value should be smaller than the minimum feature size and bigger than the maximum gap expected to be stitched in the model. Otherwise this command may remove small (sliver) edges that are smaller than the tolerance.

Return value

A Feature object.

Exceptions

None.

37.2.54 **Wire(...)**

This method creates an additional Feature object by creating a planar wire from the given ConstrainedSketch object. The ConstrainedSketch object must define a closed profile.

Path

```
mdb.models [name] .parts [name] .Wire
```

Required arguments

sketchPlane

A Datum plane object or a planar Face object specifying the plane on which to sketch.

sketchPlaneSide

A SymbolicConstant specifying the direction of feature creation. Possible values are SIDE1 and SIDE2.

sketchUpEdge

An Edge object or a Datum axis object specifying the vertical (Y) direction of the sketch.

sketch

A ConstrainedSketch object specifying the planar sketch to be revolved.

Optional argument

sketchOrientation

A SymbolicConstant specifying the orientation of *sketchUpEdge* on the sketch. Possible values are RIGHT, LEFT, TOP, and BOTTOM. The default value is RIGHT.

Return value

A Feature object.

Exceptions

None.

37.2.55 WireSpline(...)

This method creates an additional Feature object by creating a spline wire that passes through a sequence of given points. Each point can be a datum point, a vertex, an interesting point, or a tuple.

Path

`mdb.models[name].parts[name].WireSpline`

Required argument

points

A sequence of Vertex, Datum point, or InterestingPoint objects specifying the points through which the spline wire will pass. *points* can also be a sequence of tuples of Floats. You must specify at least two values in the sequence.

Optional arguments

mergeType

A SymbolicConstant specifying the merge behavior of the wire with existing geometry. If *mergeType* is MERGE, Abaqus merges the wire into solid regions of the part if the wire passes through them. If *mergeType* is IMPRINT, Abaqus imprints the spline wire on existing geometry as edges. If *mergeType* is SEPARATE, Abaqus neither merges nor imprints the spline wire with existing geometry. It creates the wire separately. The default value is IMPRINT.

smoothClosedSpline

A Boolean specifying the behavior of Abaqus when the points defining a spline wire form a closed loop (the start and end points are the same). If *smoothClosedSpline*=ON, Abaqus creates a smooth spline wire where the tangencies at the end point meet smoothly. If *smoothClosedSpline*=OFF, Abaqus does not automatically create a smooth end condition. The default value is OFF.

Return value

A Feature object.

Exceptions

None.

37.2.56 WirePolyLine(...)

This method creates an additional Feature object by creating a polyline wire that passes through a sequence of given points. Each point can be a datum point, a vertex, an interesting point, or a tuple.

Path

`mdb.models[name].parts[name].WirePolyLine`

Required argument*points*

A sequence of Vertex, Datum point, or InterestingPoint objects specifying the points through which the polyline wire will pass. *points* can also be a sequence of tuples of Floats. You must specify at least two values in the sequence.

Optional arguments*mergeType*

A SymbolicConstant specifying the merge behavior of the wire with existing geometry. If *mergeType* is MERGE, Abaqus merges the wire into solid regions of the part if the wire passes through them. If *mergeType* is IMPRINT, Abaqus imprints the wire on existing geometry as edges. If *mergeType* is SEPARATE, Abaqus neither merges nor imprints the spline wire with existing geometry. It creates the wire separately. The default value is IMPRINT.

meshable

A Boolean specifying whether the wire should be available for selection in meshing operations. If *meshable*=OFF, the wire can be used for connector section assignment. The default value is ON.

Return value

A Feature object.

Exceptions

None.

37.2.57 WireFromEdge(...)

This method creates an additional Feature object by creating a Wire by selecting one or more Edge objects of a Solid or Shell part.

Path

```
mdb.models[name].parts[name].WireFromEdge
```

Required argument

edgeList

A list of Edge objects specifying the edges from which the wire is to be created.

Optional arguments

None.

Return value

A Feature object.

Exceptions

None.

37.3 AcisFile object

The AcisFile object is a file object used to open ACIS-, STEP-, and IGES-format files.

Access

```
import part
```

37.3.1 openAcis(...)

This method creates an AcisFile object from a file containing ACIS-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path**mdb.openAcis****Required argument***fileName*

A String specifying the path to the ACIS file to open.

Optional argument*scaleFromFile*

A Boolean specifying whether to scale, rotate, and translate the part using the transform read from the ACIS file. The default value is OFF.

Return value

An AcisFile object.

Exceptions

File is from a newer version of ACIS than the CAE kernel.

Texterror: ACIS File version exceeds Kernel.

The data in the ACIS file are corrupted.

Texterror: Failed to read ACIS file.

37.3.2 openCatia(...)

This method creates an AcisFile object from a file containing CATIA V4-format or V5-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path**mdb.openCatia****Required argument***fileName*

A String specifying the path to the CATIA file to open.

Optional arguments*topology*

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID, SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a

PART COMMANDS

shell entity and not as a solid entity. If a CATIA V4-format file contains different topologies, only the selected topology is imported from the file. The default value is SOLID .

convertUnits

A SymbolicConstant specifying whether the original units should be retained. Possible values are ON and OFF. The default value is OFF.

combineBodies

A Boolean specifying whether to combine the bodies in the CATPart file. If the bodies to be combined touch or overlap, invalid entities would result. For V4-format and CATProduct files, this option will be ignored.

Return value

An AcisFile object.

Exceptions

None.

37.3.3 openEnf(...)

This method creates an AcisFile object from a file containing Elysium Neutral File-format geometry that was created by CATIA V5, I-DEAS, or Pro/ENGINEER. This object is subsequently used by the **PartFromGeometryFile** method.

Path

mdb.openEnf

Required arguments

fileName

A String specifying the path to the Elysium Neutral File that was created by I-DEAS, Pro/ENGINEER, or CATIA V5.

fileType

A String specifying the type of CAD system that created the file. Possible values are “ideas”, “proe”, or “catiaV5” or a combination similar to “proe/ideas/catiaV5” if the type is unknown.

Optional arguments

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID , SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

convertUnits

A Boolean specifying if the dimensions of the part should be converted to millimeters. The default value is OFF.

Return value

An AcisFile object.

Exceptions

None.

37.3.4 openIges(...)

This method creates an AcisFile object from a file containing IGES-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

mdb.openIges

Required argument

fileName

A String specifying the path to the IGES file to open.

Optional arguments

trimCurve

A SymbolicConstant specifying the method used to define the trim curves that bound parametric surfaces. Possible values are:

- **DEFAULT**, use either of the following as specified by the contents of the IGES file.
- **PARAMETRIC_DATA**, use the parameter space of the surface being trimmed.
- **THREED_DATA**, use real space—the coordinate system of the part along with an indication that the trim curve lies on the parametric surface.

The default value is **DEFAULT**.

scaleFromFile

A SymbolicConstant specifying whether the imported geometry needs to be scaled using the units information available in the IGES file. Possible values are **ON** and **OFF**. The default value is **OFF**. When the argument is set to **ON**, the geometry is scaled to millimeters with respect to the unit system specified in the IGES file.

msbo

A Boolean specifying if the IGES file contains MSBO (Manifold Solid B-Rep Object) entities. The default value is **False**.

PART COMMANDS

includedLayers

A sequence of Ints specifying the levels or layers of entities that will be translated from the IGES file to build the part. The default is to include all the layers.

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID , SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

uniteWires

A SymbolicConstant specifying whether the imported wires need to be united or not. Possible values are ON and OFF. The default value is ON. When importing a sketch, this value is set to OFF.

Return value

An AcisFile object.

Exceptions

The data in the IGES file are corrupted.

Texterror: Failed to read IGES file.

37.3.5 openParasolid(...)

This method creates an AcisFile object from a file containing Parasolid-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

mdb.openParasolid

Required argument

fileName

A String specifying the path to the Parasolid file to open.

Optional argument

topology

A SymbolicConstant specifying the topology of the data to be read from the file and of the part to be created. Possible values are SOLID , SHELL, and WIRE. If *topology*=SOLID, Abaqus/CAE attempts to attach cells to create a solid. If *topology*=SHELL, Abaqus/CAE builds the body as a shell entity and not as a solid entity. The default value is SOLID.

Return value

An AcisFile object.

Exceptions

None.

37.3.6 openStep(...)

This method creates an AcisFile object from a file containing STEP-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

`mdb.openStep`

Required argument

fileName

A String specifying the path to the STEP file to open.

Optional argument

scale

A Float specifying the scaling factor to apply to the imported geometric entities. The default value is 1.0.

Return value

An AcisFile object.

Exceptions

The data in the STEP file are corrupted.

Texterror: Failed to read STEP file.

37.3.7 openVda(...)

This method creates an AcisFile object from a file containing VDA-FS-format geometry. This object is subsequently used by the **PartFromGeometryFile** method.

Path

`mdb.openVda`

Required argument

fileName

A String specifying the path to the VDA-FS file to open.

Optional arguments

None.

Return value

An AcisFile object.

Exceptions

The data in the VDA-FS file are corrupted.

Texterror: Failed to read VDA file.

37.3.8 writeAcisFile(...)

This method exports the assembly to a named file in ACIS format.

Arguments

Required argument

fileName

A String specifying the name of the file to which to write.

Optional argument

version

A Float specifying the ACIS version. For example, the Float “12.0” corresponds to ACIS Version 12.0. The default value is the current version of ACIS.

Return value

None

Exceptions

None.

37.3.9 Members

The AcisFile object has the following member:

numberOfParts

An Int specifying the number of parts in the object.

38. Partition commands

The partition commands are used to partition edges, faces, and cells into new regions. A partition command can be invoked for a Part object or for an Assembly object. The partition commands create Feature objects and are described in “Feature object,” Section 20.1.

39. Path and Probe commands

Path commands are used to define a line through your model by specifying a series of points. You can view results along the path in the form of an X - Y plot. Probe commands are used to determine model data and analysis results at selected locations.

39.1 Path object

The Path object defines a line through your model by specifying a series of nodes or points.

Access

```
import visualization
session.paths[name]
```

39.1.1 Path(...)

This method creates a Path object.

Path

```
session.Path
```

Required arguments

name

A String specifying the repository key.

type

A SymbolicConstant specifying the type of path being created. Possible values are NODE_LIST, POINT_LIST, EDGE_LIST, CIRCUMFERENTIAL, and RADIAL.

expression

A sequence specifying the nodes or points that make up the path. The definition of the path expression depends on the *type* argument.

- If *type*=NODE_LIST, *expression* must be a sequence of sequences. Each inner sequence contains two items, the first item is a String specifying the name of a part instance, and the second item can be either a sequence of Ints or a sequence of Strings, each specifying a range of Ints.
- If *type*=POINT_LIST, *expression* must be a sequence of tuples of three Floats, specifying the coordinates of each point.

- If *type*=EDGE_LIST, *expression* must be a sequence of sequences. Each inner sequence contains two items, the first item is a String specifying the name of the part instance, and the second item is a sequence of tuples of four Ints that uniquely identify an element edge. The four Ints are:
 1. The element label.
 2. The element face index (one-based).
 3. The face edge index (one-based).
 4. The edge direction. A positive number specifies that the edge direction runs from the edge start node to the edge end node. A negative number specifies the opposite.
- When *type*=CIRCUMFERENTIAL or RADIAL, *expression* must be a sequence of three tuples of three Floats, specifying the coordinates of the points used to define a coordinate system.

circleDefinition

A SymbolicConstant specifying the method in which the circle is being defined. This argument is valid only when *type*=CIRCUMFERENTIAL or RADIAL. Possible values are ORIGIN_AXIS and POINT_ARC.

When the value is ORIGIN_AXIS, the first two points in *expression* are points on the rotational axis and the third point lies on the x-axis. When the value is POINT_ARC, the three points in *expression* are points lying on the arc of the circle.

numSegments

An Int specifying the number of equal segments in the path. This argument is valid only when *type*=CIRCUMFERENTIAL or RADIAL.

startAngle

A Float specifying the start angle of the circumferential path. This argument is valid only when *type*=CIRCUMFERENTIAL.

endAngle

A Float specifying the end angle of the circumferential path. This argument is valid only when *type*=CIRCUMFERENTIAL.

radius

The SymbolicConstant CIRCLE_RADIUS or a Float specifying the radius of the circumferential path. This argument is valid only when *type*=CIRCUMFERENTIAL.

radialAngle

A Float specifying the angle between the radial path and the X-axis of the specified coordinate system. This argument is valid only when *type*=RADIAL.

startRadius

The SymbolicConstant CIRCLE_RADIUS or a Float specifying the start radius of the radial path. This argument is valid only when *type*=RADIAL.

endRadius

The SymbolicConstant `CIRCLE_RADIUS` or a Float specifying the end radius of the radial path. This argument is valid only when *type*=`RADIAL`.

Optional arguments

None.

Return value

A Path object.

Exceptions

`ModelError`, `ErrorUnsupportedNodeData`, `ErrorUnsupportedPointData`, `ErrorIncorrectPathData`, `KeyError`, `ErrorEmptyPathName`, `ErrorPathNotFound`, and `ErrorNoOdbPathDisplay`.

`ValueError`: When *type*=`CIRCUMFERENTIAL` or `RADIAL`, the three points specified in *expression* are collinear.

39.1.2 Members

The Path object has members with the same names and descriptions as the arguments to the `Path` method.

39.2 CurrentProbeValues object

The CurrentProbeValues object has no constructor. The CurrentProbeValues object is created when you import the Visualization module.

Access

```
import visualization
session.currentProbeValues
```

39.2.1 Members

The CurrentProbeValues object has the following member:

values

A tuple of Floats specifying the values obtained while probing. These values are updated constantly as the user moves the mouse over the object being probed.

39.3 FreeBody object

The FreeBody object defines a section across which resultant forces and moments are computed.

Access

```
import visualization
session.freeBodies[name]
```

39.3.1 FreeBodyFromEdges(...)

This method creates a FreeBody object and places it in the freeBodies repository.

Path

```
session.FreeBodyFromEdges
```

Required arguments

name

A string name for the free body.

edges

A DisplayGroup leaf object that specifies the physical constituents of the free body.

Optional arguments

summationLoc

A SymbolicConstant specifying the location of the summation point. Possible values are CENTROID, NODAL_AVERAGE and SPECIFY. The default value is CENTROID.

summationPoint

A tuple of 3 floats specifying the summation point.

componentResolution

A SymbolicConstant specifying the component resolution. Possible values are NORMAL_TANGENTIAL and CSYS. The default value is NORMAL_TANGENTIAL.

csysName

A string specifying the name of the coordinate system.

Return value

A FreeBody object.

Exceptions

None.

39.3.2 FreeBodyFromFaces(...)

This method creates a FreeBody object and places it in the freeBodies repository.

Path**`session.FreeBodyFromFaces`****Required arguments***name*

A string name for the free body.

faces

A DisplayGroup leaf object that specifies the physical constituents of the free body.

Optional arguments*summationLoc*

A SymbolicConstant specifying the location of the summation point. Possible values are CENTROID, NODAL_AVERAGE and SPECIFY. The default value is CENTROID.

summationPoint

A tuple of 3 floats specifying the summation point.

componentResolution

A SymbolicConstant specifying the component resolution. Possible values are NORMAL_TANGENTIAL and CSYS. The default value is NORMAL_TANGENTIAL.

csysName

A string specifying the name of the coordinate system.

Return value

A FreeBody object.

Exceptions

None.

39.3.3 FreeBodyFromNodesElements(...)

This method creates a FreeBody object and places it in the freeBodies repository.

Path**`session.FreeBodyFromNodesElements`****Required arguments***name*

A string name for the free body.

elements

A DisplayGroup leaf object that specifies the physical constituents of the free body.

nodes

A DisplayGroup leaf object that specifies the physical constituents of the free body.

Optional arguments

summationLoc

A SymbolicConstant specifying the location of the summation point. Possible values are CENTROID, NODAL_AVERAGE and SPECIFY. The default value is CENTROID.

summationPoint

A tuple of 3 floats specifying the summation point.

componentResolution

A SymbolicConstant specifying the component resolution. Possible values are NORMAL_TANGENTIAL and CSYS. The default value is NORMAL_TANGENTIAL.

csysName

A string specifying the name of the coordinate system.

Return value

A FreeBody object.

Exceptions

None.

39.3.4 Members

The FreeBody object has no members.

39.4 NodeQuery object

The NodeQuery object specifies nodes and their coordinates in a path. The NodeQuery object has no constructor or methods. Abaqus creates the *nodeQuery* member when you import the **visualization** module.

Access

```
import visualization
session.nodeQuery
```


39.4.1 Members

The NodeQuery object has the following members:

nodeId

An Int specifying the ID of the most recently queried node. If the last query was unsuccessful, *nodeId*=-1.

nodePos

A tuple of Floats specifying the X-, Y-, and Z-coordinates of the most recently queried node.

39.5 ProbeOptions object

The ProbeOptions object is used to store settings associated with probing a model or an X-Y plot. The ProbeOptions object has no constructor. Abaqus creates the *defaultProbeOptions* and the *probeOptions* members when you import the Visualization module. When probing is initiated for the first time, the values in the *probeOptions* member are initialized using the values from the *defaultProbeOptions* member.

Access

```
import visualization
session.defaultProbeOptions
session.probeOptions
```

39.5.1 setValues(...)

This method modifies the settings on the ProbeOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A ProbeOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

probeEntity

A SymbolicConstant specifying the entity being probed when *probeObject*="ODB". Possible values are NODE and ELEMENT. The default value is ELEMENT.

probeOutputPosition

A SymbolicConstant specifying the output position to probe for field output results when *probeObject*="ODB". Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, and ELEMENT_CENTROID.

When *probeEntity*=NODE, the only possible value is NODAL. When *probeEntity*=ELEMENT, the only possible values are INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, and ELEMENT_CENTROID, and the default is INTEGRATION_POINT.

partInstance

A Boolean specifying whether to display the part instance information. This member is valid when *probeObject*="ODB." The default value is ON.

elementID

A Boolean specifying whether to display the element ID information. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementType

A Boolean specifying whether to display the element type information. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementConnectivity

A Boolean specifying whether to display the element connectivity. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementFieldResults

A Boolean specifying whether to display the element field output results. This member is valid when *probeObject*="ODB", *probeEntity*=ELEMENT, and *isFieldOutputAvailable*=ON. The default value is ON.

nodeId

A Boolean specifying whether to display the node ID when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

baseCoordinates

A Boolean specifying whether to display the base coordinates of a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

deformedCoordinates

A Boolean specifying whether to display the deformed coordinates of a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

attachedElements

A Boolean specifying whether to display the elements attached to a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

nodeFieldResults

A Boolean specifying whether to display the node field output results. This member is valid when *probeObject*="ODB", *probeEntity*=NODE, and *isFieldOutputAvailable*=ON. The default value is ON.

legend

A Boolean specifying whether to display the legend for a curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

xValue

A Boolean specifying whether to display the x-coordinate value of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

yValue

A Boolean specifying whether to display the y-coordinate value of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

sequenceID

A Boolean specifying whether to display the sequence ID of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

interpolateXy

A Boolean specifying whether to interpolate values within a line segment when *probeObject*="XYPlot." When *interpolateXy*=OFF, probing returns the nearest X-Y data point on the curve. When *interpolateXy*=ON, probing interpolates data to return a value at the nearest point on the curve. The default value is OFF.

Return value

None

Exceptions

None.

39.5.2 Members

The ProbeOptions object has the following members:

probeEntity

A SymbolicConstant specifying the entity being probed when *probeObject*="ODB". Possible values are NODE and ELEMENT. The default value is ELEMENT.

probeOutputPosition

A SymbolicConstant specifying the output position to probe for field output results when *probeObject*="ODB". Possible values are NODAL, INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, and ELEMENT_CENTROID.

When *probeEntity*=NODE, the only possible value is NODAL. When *probeEntity*=ELEMENT, the only possible values are INTEGRATION_POINT, ELEMENT_FACE, ELEMENT_NODAL, and ELEMENT_CENTROID, and the default is INTEGRATION_POINT.

PATH AND PROBE COMMANDS

partInstance

A Boolean specifying whether to display the part instance information. This member is valid when *probeObject*="ODB." The default value is ON.

elementID

A Boolean specifying whether to display the element ID information. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementType

A Boolean specifying whether to display the element type information. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementConnectivity

A Boolean specifying whether to display the element connectivity. This member is valid when *probeObject*="ODB" and *probeEntity*=ELEMENT. The default value is ON.

elementFieldResults

A Boolean specifying whether to display the element field output results. This member is valid when *probeObject*="ODB", *probeEntity*=ELEMENT, and *isFieldOutputAvailable*=ON. The default value is ON.

nodeId

A Boolean specifying whether to display the node ID when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

baseCoordinates

A Boolean specifying whether to display the base coordinates of a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

deformedCoordinates

A Boolean specifying whether to display the deformed coordinates of a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

attachedElements

A Boolean specifying whether to display the elements attached to a node when *probeObject*="ODB" and *probeEntity*=NODE. The default value is ON.

nodeFieldResults

A Boolean specifying whether to display the node field output results. This member is valid when *probeObject*="ODB", *probeEntity*=NODE, and *isFieldOutputAvailable*=ON. The default value is ON.

legend

A Boolean specifying whether to display the legend for a curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

xValue

A Boolean specifying whether to display the x-coordinate value of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

yValue

A Boolean specifying whether to display the y-coordinate value of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

sequenceID

A Boolean specifying whether to display the sequence ID of the point on the curve being probed. This member is valid when *probeObject*="XYPlot." The default value is ON.

interpolateXy

A Boolean specifying whether to interpolate values within a line segment when *probeObject*="XYPlot." When *interpolateXy*=OFF, probing returns the nearest X-Y data point on the curve. When *interpolateXy*=ON, probing interpolates data to return a value at the nearest point on the curve. The default value is OFF.

isFieldOutputAvailable

A Boolean specifying whether field output is available for probing when *probeObject*="XYPlot." This member is read-only."

probeObject

A String specifying the type of the displayed object being probed. Possible values are "ODB" and "XYPlot". This member is read-only.

39.6 ProbeReport object

The ProbeReport object is used to store settings associated with tabular reports of probe data. The ProbeReport object has no constructor. Abaqus creates the *defaultProbeReport* and the *probeReport* members when you import the Visualization module. When probing is initiated for the first time, the values in the *probeReport* member are initialized using the values from the *defaultProbeReport* member.

Access

```
import visualization
session.defaultProbeReport
session.probeReport
```

39.6.1 setValues(...)

This method modifies the ProbeReport object.

Arguments**Required arguments**

None.

Optional arguments

options

None or a ProbeReport object specifying values to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

numColumns

An Int specifying the number of columns in the report file. This argument is valid only when *pageWidth*=SPECIFY. The default value is 80.

numDigits

An Int specifying the number of significant digits to be written for decimal values. The default value is 6.

numFormat

A SymbolicConstant specifying the number format to be used when formatting decimal values. Possible values are AUTOMATIC, ENGINEERING, and SCIENTIFIC. The default value is ENGINEERING.

pageWidth

A SymbolicConstant specifying the page width format. Possible values are NO_LIMIT and SPECIFY. The default value is NO_LIMIT.

printTotal

A Boolean specifying whether to print the total value of either the field output result (when *probeObject*="ODB") or the *x*- and *y*-coordinates (when *probeObject*="XYPlot"). The default value is OFF.

printMinMax

A Boolean specifying whether to print the minimum and maximum values of either the field output result (when *probeObject*="ODB") or the *x*- and *y*-coordinates (when *probeObject*="XYPlot"). The default value is OFF.

Return value

None

Exceptions

None.

39.6.2 Members

The ProbeReport object has members with the same names and descriptions as the arguments to the **setValues** method.

39.7 SelectedProbeValues object

The SelectedProbeValues object has no constructor. The SelectedProbeValues object is created when you import the Visualization module.

Access

```
import visualization
session.selectedProbeValues
```

39.7.1 Members

The SelectedProbeValues object has the following members:

length

An Int specifying the length of the *values* member.

fieldOutputAvailable

A Boolean specifying whether any probe values have been selected (as is necessary prior to writing to a file).

values

A tuple of tuples of Floats specifying the selected probe values.

lastValues

A tuple of Floats specifying the last sequence of the *values* member.

39.8 Spectrum object

The Spectrum object defines a color spectrum for the contour display.

Access

```
import visualization
session.spectrums [name]
```

39.8.1 Spectrum(...)

This method creates a Spectrum object and places it in the spectrums repository.

Path

```
session.Spectrum
```

Required arguments

name

A string name for the spectrum.

colors

A sequence of strings indicating the colors of the spectrum.

Optional arguments

None.

Return value

A Spectrum object.

Exceptions

None.

39.8.2 Members

The Spectrum object has no members.

39.9 Stream object

TheStream object defines a set of streamlines in fluid mechanics.

Access

```
import visualization
session.streams [name]
```

39.9.1 Stream(...)

This method creates aStream object and places it in the streams repository.

Path

```
session.Stream
```

Required arguments

name

A string name for the stream.

numPointsOnRake

An integer specifying the number of points along the rake.

Optional arguments

pointA

A tuple of 3 floats specifying the starting point of the rake. Alternatively, a string representation of the node selected in the viewport.

pointB

A tuple of 3 floats specifying the end point of the rake. Alternatively, a string representation of the node selected in the viewport.

path

APath object that specifies the rake.

Return value

A Stream object.

Exceptions

None.

39.9.2 Members

The Stream object has no members.

40. Plot Options commands

Plot options commands are used to control the appearance of plots in the Visualization module. Plots can be undeformed, deformed, contour, symbol, or material orientation.

40.1 BasicOptions object

The BasicOptions object stores values and attributes associated with an OdbDisplay object. The BasicOptions object has no constructor command. Abaqus creates the *defaultOdbDisplay.basicOptions* member when you import the Visualization module. Abaqus creates a *basicOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.basicOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the attributes from the previous active viewport. The previous active viewport contains the attributes from the *defaultOdbDisplay* object for the session. The attributes from the *defaultOdbDisplay* object are copied from the previous active viewport to create the new viewport.

BasicOptions objects are accessed in one of two ways:

- The default basic options. These settings are used as defaults when other *basicOptions* members are created. These settings can be set to customize user preferences.
- The basic options associated with a particular viewport.

Access

```
import visualization
session.defaultOdbDisplay.basicOptions
session.viewports[name].layers[name].odbDisplay.basicOptions
session.viewports[name].odbDisplay.basicOptions
```

40.1.1 setValues(...)

This method modifies the BasicOptions object.

Arguments

Required arguments

None.

Optional arguments

options

A BasicOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

PLOT OPTIONS COMMANDS

cameraCsysName

A String specifying the name of the coordinate system driving the moving camera.

cameraMovesWithCsys

A Boolean specifying whether the camera moves with the coordinate system. The default value is OFF.

cameraFollowsRotation

A Boolean specifying whether the camera, when it moves, follows the rotation of the coordinate system. The default value is OFF.

averagingThreshold

A Float specifying the nodal averaging threshold percentage. Possible values are $0 < averagingThreshold < 100$. The default value is 75.0.

quantityToPlot

A SymbolicConstant specifying the quantity to plot. Possible values are FIELD_OUTPUT and DISCONTINUITIES. The default value is FIELD_OUTPUT.

curveRefinementLevel

A SymbolicConstant specifying the refinement level for drawing curves. Possible values are EXTRA_COARSE, COARSE, MEDIUM, FINE, and EXTRA_FINE. The default value is COARSE.

noResultsColor

A String specifying the color of elements that do not have any results. The default value is "White".

featureAngle

A Float specifying the feature angle to be used when *visibleEdges*=FEATURE. The default value is 30.0.

otherSymbolSize

An Int specifying the size of various glyph symbols (boundary conditions, coupling constraints, etc.). The default value is 6.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The beamScaleFactor must be greater than zero. The default value is 1.0.

renderShellThickness

A Boolean specifying whether to render the shell thickness. The default value is OFF.

shellScaleFactor

A Float specifying the shell thickness scale factor. The shellScaleFactor must be greater than zero. The default value is 1.0.

accountForDeactivatedElems

A Boolean specifying whether to account for deactivated elements. The default value is ON.

bcDisplay

A Boolean specifying whether to display boundary conditions. The default value is OFF.

connectorDisplay

A Boolean specifying whether to display connectors. The default value is OFF.

highlightConnectorPts

A Boolean specifying whether to highlight connector points. The default value is ON.

showConnectorAxes

A Boolean specifying whether to display connector orientations or coordinate systems. The default value is ON.

showConnectorType

A Boolean specifying whether to display the text that describes the connector type. The default value is ON.

pointElements

A Boolean specifying whether to display point type elements. The default value is ON.

referencePoints

A Boolean specifying whether to display reference points. *referencePoints* is valid only when *pointElements*=ON. The default value is ON.

massElements

A Boolean specifying whether to display mass, heat capacity and inertia elements. *massElements* is valid only when *pointElements*=ON. The default value is OFF.

springElements

A Boolean specifying whether to display spring and dashpot elements. *springElements* is valid only when *pointElements*=ON. The default value is OFF.

spotWelds

A Boolean specifying whether to display spot weld and distributed coupling elements. *spotWelds* is valid only when *pointElements*=ON. The default value is OFF.

tracerParticles

A Boolean specifying whether to display tracer particles. *tracerParticles* is valid only when *pointElements*=ON. The default value is OFF.

pc3dElements

A Boolean specifying whether to display PC3D elements. *pc3dElements* is valid only when *pointElements*=ON. The default value is ON.

pd3dElements

A Boolean specifying whether to display PD3D elements. *pd3dElements* is valid only when *pointElements*=ON. The default value is ON.

sweepArs

A Boolean specifying whether to sweep the analytical surfaces. The default value is ON or OFF, depending on the characteristics of your model.

PLOT OPTIONS COMMANDS

sweepElem

A Boolean specifying whether to sweep the deformable elements. The default value is ON or OFF, depending on the characteristics of your model.

sweepStartAngleArs

A Float specifying the starting angle (in degrees) from which to sweep the analytical surfaces when *sweepArs*=ON. The default value is 0.0.

sweepStartAngleElem

A Float specifying the starting angle (in degrees) from which to sweep the model when *sweepElem*=ON. The default value is 0.0.

sweepEndAngleArs

A Float specifying the angle (in degrees) through which to sweep the analytical surfaces when *sweepArs*=ON. The default value is 360.0.

sweepEndAngleElem

A Float specifying the angle (in degrees) through which to sweep the model when *sweepElem*=ON. The default value is 180.0.

numSweepSegmentsArs

An Int specifying the number of segments to display when *sweepArs*=ON. The default value is 10 or 20, depending on characteristics of your model.

numSweepSegmentsElem

An Int specifying the number of segments to display when *sweepElem*=ON. The default value is 10 or 20, depending on characteristics of your model.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_MAG_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_MAG_AT_ANGLE. The default value is 0.0.

sectionResults

A SymbolicConstant specifying which of the section points to use. Possible values are USE_BOTTOM, USE_TOP, USE_BOTTOM_AND_TOP, and USE_ENVELOPE. The default value is USE_BOTTOM.

envelopeCriteria

A SymbolicConstant specifying the envelope criterion. Possible values are MAX_VALUE, MIN_VALUE, and MAX_ABS_VALUE. The default value is MAX_ABS_VALUE.

envelopeDataPosition

A SymbolicConstant specifying the output position for envelope calculations. Possible values are CENTROID, ELEMENT_NODAL, and INTEGRATION_POINT. The default value is INTEGRATION_POINT.

plyResultLocation

A SymbolicConstant specifying the ply result location. Possible values are BOTTOM, MIDDLE, TOP, and TOP_AND_BOTTOM. The default value is MIDDLE.

sectionPointScheme

A SymbolicConstant specifying the section point scheme. Possible values are CATEGORY_BASED and PLY_BASED. The default value is CATEGORY_BASED.

sweepSectors

A Boolean specifying whether to sweep the cyclic symmetry sectors. The default value is OFF.

sectorSelectionType

A SymbolicConstant specifying how sectors will be selected for sweeping. Possible values are SELECT_BY_NUMBER, SELECT_BY_ANGLE, and SELECT_ALL. The default value is SELECT_BY_NUMBER.

selectedSectorNumbers

A sequence of Ints specifying which sectors to display when *sectorSelectionType*=SELECT_BY_NUMBER. Possible values are $1 \leq \text{selectedSectorNumbers} \leq$ the number of sectors. The default value is (1).

sweepSectorStartAngle

A Float specifying the angle (in degrees) from which to sweep cyclic symmetry sectors when *sweepSectors*=ON. Possible values are multiples of the sector angle such that $0 \leq \text{sweepSectorStartAngle} \leq 360$. The default value is 0.0.

sweepSectorEndAngle

A Float specifying the angle (in degrees) through which to sweep cyclic symmetry sectors when *sweepSectors*=ON. Possible values are multiples of the sector angle such that $0 < \text{sweepSectorEndAngle} \leq 360$. The default value is 360.0.

extrudeArs

A Boolean specifying whether to extrude analytical surfaces. The default value is ON or OFF depending on characteristics of your model.

extrudeArsDepthAutoCompute

A Boolean specifying whether to use automatic depth determination when extruding analytical surfaces. The default value is ON. The default value is ON.

extrudeElem

A Boolean specifying whether to extrude deformable elements. The default value is ON or OFF depending on characteristics of your model.

extrudeArsDepth

A Float specifying the depth (in model units) by which the analytical surfaces are to be extruded when *extrudeArs*=ON. The default value is 1.0.

extrudeElemDepth

A Float specifying the depth (in model units) by which the deformable elements are to be extruded when *extrudeElem*=ON. The default value is 1.0.

PLOT OPTIONS COMMANDS

mirrorPatternOrder

A SymbolicConstant specifying the order of operations to create the mirror pattern. Possible values are MIRROR_RECT_CIRC, RECT_MIRROR_CIRC, MIRROR_CIRC_RECT, RECT_CIRC_MIRROR, CIRC_MIRROR_RECT, and CIRC_RECT_MIRROR. The default value is MIRROR_RECT_CIRC.

mirrorCsysName

The SymbolicConstant GLOBAL or a String specifying the name of the mirror's coordinate system. The default value is GLOBAL.

mirrorAboutXyPlane

A Boolean specifying whether to mirror about the XY plane. The default value is OFF.

mirrorAboutXzPlane

A Boolean specifying whether to mirror about the XZ plane. The default value is OFF.

mirrorAboutYzPlane

A Boolean specifying whether to mirror about the YZ plane. The default value is OFF.

mirrorDisplayBodies

A Boolean specifying whether to mirror display bodies. The default value is OFF.

patternCsysName

The SymbolicConstant GLOBAL or a String specifying the name of the pattern's coordinate system. The default value is GLOBAL.

patternNumX

An Int specifying the number of patterns to create in the X-direction for a rectangular pattern. The default value is 1.

patternNumY

An Int specifying the number of patterns to create in the Y-direction for a rectangular pattern. The default value is 1.

patternNumZ

An Int specifying the number of patterns to create in the Z-direction for a rectangular pattern. The default value is 1.

patternOffsetX

A Float specifying the offset of the pattern along the X-axis for a rectangular pattern. The default value is 0.0.

patternOffsetY

A Float specifying the offset of the pattern along the Y-axis for a rectangular pattern. The default value is 0.0.

patternOffsetZ

A Float specifying the offset of the pattern along the Z-axis for a rectangular pattern. The default value is 0.0.

patternRotationAxis

A SymbolicConstant specifying the axis of rotation for a circular. Possible values are XAXIS, YAXIS, and ZAXIS. The default value is ZAXIS.

patternTotalAngle

A Float specifying the total angle of a circular pattern. The default value is 360.0.

patternNumCircular

An Int specifying the number of patterns to create in a circular pattern. The default value is 1.

couplingDisplay

A Boolean specifying whether to display coupling constraints. The default value is ON.

coordSystemDisplay

A Boolean specifying whether to display coordinate systems. The default value is OFF.

scratchCoordSystemDisplay

A Boolean specifying whether to display coordinate systems that represent user-defined orientations. The default value is OFF.

transformationType

A SymbolicConstant specifying the transformation to apply to the PrimaryVariable. Possible values are DEFAULT, NODAL, USER_SPECIFIED, ANGULAR, and LAYUP_ORIENTATION. The default value is DEFAULT.

If *transformationType*=NODAL, Abaqus will transform nodal vector fields into any orientation defined in the analysis with the *TRANSFORM option. Setting *transformationType*=NODAL has no effect on element-based results.

If *transformationType*=USER_SPECIFIED, Abaqus will transform tensor and vector fields into the coordinate system specified by *datumCsys*.

If *transformationType*=LAYUP_ORIENTATION, Abaqus will transform tensor and vector fields into the layup orientation defined in the composite section. The odb should contain the field SORIENT in order to use this option.

datumCsys

A DatumCsys object specifying the coordinate system to use for results transformation when *transformationType*=USER_SPECIFIED.

rigidTransformPrimary

A Boolean specifying whether to perform a rigid transformation of nodal vector datasets based on the active user specific system. The default value is OFF.

rigidTransformDeformed

A Boolean specifying whether to perform a rigid transformation of current deformedVariable based on the active user specific system. The default value is OFF.

transformOnDeformed

A Boolean specifying whether to include the effects of deformation on the transformation calculations. The default value is ON.

PLOT OPTIONS COMMANDS

averageElementOutput

A Boolean specifying whether to average the element output. The default value is ON.

averageOnlyDisplayed

A Boolean specifying whether to average only values on displayed elements. The default value is ON.

computeOutput

A SymbolicConstant specifying the order or the computations to be performed. Possible values are EXTRAPOLATE_AVERAGE_COMPUTE, EXTRAPOLATE_COMPUTE_AVERAGE, EXTRAPOLATE_COMPUTE, EXTRAPOLATE_COMPUTE_DISCONTINUITIES, and RAW_DATA. The default value is EXTRAPOLATE_AVERAGE_COMPUTE.

regionBoundaries

A SymbolicConstant specifying the type of averaging region boundaries. Possible values are NONE, ODB_REGIONS, ELEMENT_SET, and DISPLAY_GROUPS. The default value is ODB_REGIONS.

useRegionBoundaries

A Boolean specifying whether to use region boundaries when averaging. The default value is ON.

userRegions

A sequence of Strings specifying either element set or display group names (depending on the value of *regionBoundaries*) defining the averaging region boundaries. The default value is an empty sequence.

includeFeatureBoundaries

A Boolean specifying whether to include additional averaging boundaries for shells and membranes based on feature edges. The default value is ON.

Return value

None

Exceptions

If *featureAngle* is not in the valid range:

RangeError: *featureAngle* must be a float in the range 0-90, inclusive

40.1.2 Members

The BasicOptions object can have the following members:

regionAveraging

A Boolean specifying whether to ignore region boundaries when computing values. The default value is ON.

cameraMovesWithCsys

A Boolean specifying whether the camera moves with the coordinate system. The default value is OFF.

cameraFollowsRotation

A Boolean specifying whether the camera, when it moves, follows the rotation of the coordinate system. The default value is OFF.

averagingThreshold

A Float specifying the nodal averaging threshold percentage. Possible values are $0 < \text{averagingThreshold} < 100$. The default value is 75.0.

quantityToPlot

A SymbolicConstant specifying the quantity to plot. Possible values are FIELD_OUTPUT and DISCONTINUITIES. The default value is FIELD_OUTPUT.

extrapAlgorithm

A SymbolicConstant specifying the extrapolation algorithm. This member is for internal use only. The only possible value is EXTRAP_COMPUTE_AVERAGE.

curveRefinementLevel

A SymbolicConstant specifying the refinement level for drawing curves. Possible values are EXTRA_COARSE, COARSE, MEDIUM, FINE, and EXTRA_FINE. The default value is COARSE.

featureAngle

A Float specifying the feature angle to be used when *visibleEdges*=FEATURE. The default value is 30.0.

otherSymbolSize

An Int specifying the size of various glyph symbols (boundary conditions, coupling constraints, etc.). The default value is 6.

renderBeamProfiles

A Boolean specifying whether to render the beam profiles. The default value is OFF.

beamScaleFactor

A Float specifying the beam profile scale factor. The beamScaleFactor must be greater than zero. The default value is 1.0.

renderShellThickness

A Boolean specifying whether to render the shell thickness. The default value is OFF.

shellScaleFactor

A Float specifying the shell thickness scale factor. The shellScaleFactor must be greater than zero. The default value is 1.0.

accountForDeactivatedElements

A Boolean specifying whether to account for deactivated elements. The default value is ON.

bcDisplay

A Boolean specifying whether to display boundary conditions. The default value is OFF.

PLOT OPTIONS COMMANDS

connectorDisplay

A Boolean specifying whether to display connectors. The default value is OFF.

highlightConnectorPts

A Boolean specifying whether to highlight connector points. The default value is ON.

showConnectorAxes

A Boolean specifying whether to display connector orientations or coordinate systems. The default value is ON.

showConnectorType

A Boolean specifying whether to display the text that describes the connector type. The default value is ON.

pointElements

A Boolean specifying whether to display point type elements. The default value is ON.

referencePoints

A Boolean specifying whether to display reference points. *referencePoints* is valid only when *pointElements*=ON. The default value is ON.

massElements

A Boolean specifying whether to display mass, heat capacity and inertia elements. *massElements* is valid only when *pointElements*=ON. The default value is OFF.

springElements

A Boolean specifying whether to display spring and dashpot elements. *springElements* is valid only when *pointElements*=ON. The default value is OFF.

spotWelds

A Boolean specifying whether to display spot weld and distributed coupling elements. *spotWelds* is valid only when *pointElements*=ON. The default value is OFF.

tracerParticles

A Boolean specifying whether to display tracer particles. *tracerParticles* is valid only when *pointElements*=ON. The default value is OFF.

pc3dElements

A Boolean specifying whether to display PC3D elements. *pc3dElements* is valid only when *pointElements*=ON. The default value is ON.

pd3dElements

A Boolean specifying whether to display PD3D elements. *pd3dElements* is valid only when *pointElements*=ON. The default value is ON.

sweepArs

A Boolean specifying whether to sweep the analytical surfaces. The default value is ON or OFF, depending on the characteristics of your model.

sweepElem

A Boolean specifying whether to sweep the deformable elements. The default value is ON or OFF, depending on the characteristics of your model.

sweepStartAngleArs

A Float specifying the starting angle (in degrees) from which to sweep the analytical surfaces when *sweepArs*=ON. The default value is 0.0.

sweepStartAngleElem

A Float specifying the starting angle (in degrees) from which to sweep the model when *sweepElem*=ON. The default value is 0.0.

sweepEndAngleArs

A Float specifying the angle (in degrees) through which to sweep the analytical surfaces when *sweepArs*=ON. The default value is 360.0.

sweepEndAngleElem

A Float specifying the angle (in degrees) through which to sweep the model when *sweepElem*=ON. The default value is 180.0.

numSweepSegmentsArs

An Int specifying the number of segments to display when *sweepArs*=ON. The default value is 10 or 20, depending on characteristics of your model.

numSweepSegmentsElem

An Int specifying the number of segments to display when *sweepElem*=ON. The default value is 10 or 20, depending on characteristics of your model.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_MAG_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_MAG_AT_ANGLE. The default value is 0.0.

sectionResults

A SymbolicConstant specifying which of the section points to use. Possible values are USE_BOTTOM, USE_TOP, USE_BOTTOM_AND_TOP, and USE_ENVELOPE. The default value is USE_BOTTOM.

envelopeCriteria

A SymbolicConstant specifying the envelope criterion. Possible values are MAX_VALUE, MIN_VALUE, and MAX_ABS_VALUE. The default value is MAX_ABS_VALUE.

envelopeDataPosition

A SymbolicConstant specifying the output position for envelope calculations. Possible values are CENTROID, ELEMENT_NODAL, and INTEGRATION_POINT. The default value is INTEGRATION_POINT.

plyResultLocation

A SymbolicConstant specifying the ply result location. Possible values are BOTTOM, MIDDLE, TOP, and TOP_AND_BOTTOM. The default value is MIDDLE.

PLOT OPTIONS COMMANDS

sectionPointScheme

A SymbolicConstant specifying the section point scheme. Possible values are CATEGORY_BASED and PLY_BASED. The default value is CATEGORY_BASED.

sweepSectors

A Boolean specifying whether to sweep the cyclic symmetry sectors. The default value is OFF.

sectorSelectionType

A SymbolicConstant specifying how sectors will be selected for sweeping. Possible values are SELECT_BY_NUMBER, SELECT_BY_ANGLE, and SELECT_ALL. The default value is SELECT_BY_NUMBER.

sweepSectorStartAngle

A Float specifying the angle (in degrees) from which to sweep cyclic symmetry sectors when *sweepSectors*=ON. Possible values are multiples of the sector angle such that $0 \leq \text{sweepSectorStartAngle} \leq 360$. The default value is 0.0.

sweepSectorEndAngle

A Float specifying the angle (in degrees) through which to sweep cyclic symmetry sectors when *sweepSectors*=ON. Possible values are multiples of the sector angle such that $0 < \text{sweepSectorEndAngle} \leq 360$. The default value is 360.0.

extrudeArs

A Boolean specifying whether to extrude analytical surfaces. The default value is ON or OFF depending on characteristics of your model.

extrudeArsDepthAutoCompute

A Boolean specifying whether to use automatic depth determination when extruding analytical surfaces. The default value is ON. The default value is ON.

extrudeElem

A Boolean specifying whether to extrude deformable elements. The default value is ON or OFF depending on characteristics of your model.

extrudeArsDepth

A Float specifying the depth (in model units) by which the analytical surfaces are to be extruded when *extrudeArs*=ON. The default value is 1.0.

extrudeElemDepth

A Float specifying the depth (in model units) by which the deformable elements are to be extruded when *extrudeElem*=ON. The default value is 1.0.

mirrorPatternOrder

A SymbolicConstant specifying the order of operations to create the mirror pattern. Possible values are MIRROR_RECT_CIRC, RECT_MIRROR_CIRC, MIRROR_CIRC_RECT, RECT_CIRC_MIRROR, CIRC_MIRROR_RECT, and CIRC_RECT_MIRROR. The default value is MIRROR_RECT_CIRC.

mirrorAboutXyPlane

A Boolean specifying whether to mirror about the XY plane. The default value is OFF.

mirrorAboutXzPlane

A Boolean specifying whether to mirror about the XZ plane. The default value is OFF.

mirrorAboutYzPlane

A Boolean specifying whether to mirror about the YZ plane. The default value is OFF.

mirrorDisplayBodies

A Boolean specifying whether to mirror display bodies. The default value is OFF.

patternNumX

An Int specifying the number of patterns to create in the X-direction for a rectangular pattern. The default value is 1.

patternNumY

An Int specifying the number of patterns to create in the Y-direction for a rectangular pattern. The default value is 1.

patternNumZ

An Int specifying the number of patterns to create in the Z-direction for a rectangular pattern. The default value is 1.

patternOffsetX

A Float specifying the offset of the pattern along the X-axis for a rectangular pattern. The default value is 0.0.

patternOffsetY

A Float specifying the offset of the pattern along the Y-axis for a rectangular pattern. The default value is 0.0.

patternOffsetZ

A Float specifying the offset of the pattern along the Z-axis for a rectangular pattern. The default value is 0.0.

patternRotationAxis

A SymbolicConstant specifying the axis of rotation for a circular. Possible values are XAXIS, YAXIS, and ZAXIS. The default value is ZAXIS.

patternTotalAngle

A Float specifying the total angle of a circular pattern. The default value is 360.0.

patternNumCircular

An Int specifying the number of patterns to create in a circular pattern. The default value is 1.

couplingDisplay

A Boolean specifying whether to display coupling constraints. The default value is ON.

coordSystemDisplay

A Boolean specifying whether to display coordinate systems. The default value is OFF.

scratchCoordSystemDisplay

A Boolean specifying whether to display coordinate systems that represent user-defined orientations. The default value is OFF.

PLOT OPTIONS COMMANDS

transformationType

A SymbolicConstant specifying the transformation to apply to the PrimaryVariable. Possible values are DEFAULT, NODAL, USER_SPECIFIED, ANGULAR, and LAYUP_ORIENTATION. The default value is DEFAULT.

If *transformationType*=NODAL, Abaqus will transform nodal vector fields into any orientation defined in the analysis with the *TRANSFORM option. Setting *transformationType*=NODAL has no effect on element-based results.

If *transformationType*=USER_SPECIFIED, Abaqus will transform tensor and vector fields into the coordinate system specified by *datumCsys*.

If *transformationType*=LAYUP_ORIENTATION, Abaqus will transform tensor and vector fields into the layup orientation defined in the composite section. The odb should contain the field SORIENT in order to use this option.

rigidTransformPrimary

A Boolean specifying whether to perform a rigid transformation of nodal vector datasets based on the active user specific system The default value is OFF.

rigidTransformDeformed

A Boolean specifying whether to perform a rigid transformation of current deformedVariable based on the active user specific system The default value is OFF.

transformOnDeformed

A Boolean specifying whether to include the effects of deformation on the transformation calculations The default value is ON.

modelCanExtrude

A Boolean specifying whether the model contains any elements or surfaces that can be extruded.

sweepModelType

An Int specifying the types of sweepable elements and surfaces contained in the model, if any.

averageElementOutput

A Boolean specifying whether to average the element output. The default value is ON.

averageOnlyDisplayed

A Boolean specifying whether to average only values on displayed elements. The default value is ON.

regionBoundaries

A SymbolicConstant specifying the type of averaging region boundaries. Possible values are NONE, ODB_REGIONS, ELEMENT_SET, and DISPLAY_GROUPS. The default value is ODB_REGIONS.

useRegionBoundaries

A Boolean specifying whether to use region boundaries when averaging. The default value is ON.

includeFeatureBoundaries

A Boolean specifying whether to include additional averaging boundaries for shells and membranes based on feature edges. The default value is ON.

numSectors

An Int specifying the number of sectors of a cyclic symmetric model. The value is automatically computed from the cyclic symmetric model. This value is read-only.

sectorAngle

A Float specifying the sector angle of a cyclic symmetric model. The value is automatically computed from the cyclic symmetric model. This value is read-only.

automaticExtrudeDepth

A Float specifying the automatic extrude depth used to extrude analytical rigid surfaces in the default setting. This value is read-only.

cameraCsysName

A String specifying the name of the coordinate system driving the moving camera.

noResultsColor

A String specifying the color of elements that do not have any results. The default value is "White".

mirrorCsysName

The SymbolicConstant GLOBAL or a String specifying the name of the mirror's coordinate system. The default value is GLOBAL.

patternCsysName

The SymbolicConstant GLOBAL or a String specifying the name of the pattern's coordinate system. The default value is GLOBAL.

datumCsys

A DatumCsys object specifying the coordinate system to use for results transformation when *transformationType*=USER_SPECIFIED.

selectedSectorNumbers

A tuple of Ints specifying which sectors to display when *sectorSelectionType*=SELECT_BY_NUMBER. Possible values are $1 \leq \text{selectedSectorNumbers} \leq$ the number of sectors. The default value is (1).

userRegions

A tuple of Strings specifying either element set or display group names (depending on the value of *regionBoundaries*) defining the averaging region boundaries. The default value is an empty sequence.

40.2 **DGCommonOptions object**

The DGCommonOptions object stores values and attributes that are common to all plot states. The DGCommonOptions object has no constructor command. Abaqus creates an *odbDisplayOptions.commonOptions* member when a display group instance is created, using values from *odbDisplay.commonOptions*.

Access

```

session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.commonOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.commonOptions

```

40.2.1 Members

The DGCommonOptions object can have the following members:

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is SHADED.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals or surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

PLOT OPTIONS COMMANDS

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-120-*-m-*-*".

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

40.3 DGContourOptions object

The DGContourOptions object stores values and attributes associated with a contour plot. The DGContourOptions object has no constructor command. Abaqus creates an *odbDisplayOptions.contourOptions* member when a display group instance is created, using values from *odbDisplay.contourOptions*.

Access

```
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.contourOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.contourOptions
```

40.3.1 Members

The DGContourOptions object can have the following members:

contourMethod

A SymbolicConstant specifying the contour rendering method. Possible values are TEXTURE_MAPPED and TESSELLATED. The default value is TEXTURE_MAPPED.

tickmarkPlots

A Boolean specifying whether tick mark plots should be displayed on line-type elements. If *tickmarkPlots*=ON, Abaqus displays a tick mark plot. If *tickmarkPlots*=OFF, Abaqus displays contours on the element faces. The default value is OFF.

PLOT OPTIONS COMMANDS

contourStyle

A SymbolicConstant specifying the interval style of the contour plot. Possible values are CONTINUOUS and UNIFORM. The default value is UNIFORM.

contourEdges

A Boolean specifying whether to plot the edges of each contour interval when *contourType*=BANDED or ISOSURFACE. The default value is OFF.

contourEdgeStyle

A SymbolicConstant specifying the edge line style to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

contourEdgeThickness

A SymbolicConstant specifying the edge line thickness to be used to plot the edge of the contour intervals when *contourType*=BANDED or ISOSURFACE. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

averagedOrientationDisplay

A Boolean specifying the display of the nodal averaged coordinate systems used when averaging element vector or tensor data. The default value is OFF.

matchingPlyLabels

A Boolean specifying whether the label for the matching ply shows up in the viewport. The default value is OFF.

colorByMatchingPlies

A Boolean specifying whether the contour color is driven by the matching ply. The default value is OFF.

tickmarkAxisLength

A SymbolicConstant specifying the length of the tick mark plot axes. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

tickmarkBaseValue

A Float specifying the base contour value defining the tick mark axis contour value that intersects the elements. Possible values are $autoMinValue \leq tickmarkBaseValue \leq autoMaxValue$. The default value is 0.0.

tickmarkOrientation

A SymbolicConstant specifying the orientation of the tick mark plots. Possible values are N1 and N2. The default value is N2.

edgeColorLine

A String specifying the edge color to be used when *contourType*=LINE. The default value is "White".

edgeColorBandedQuilt

A String specifying the edge color to be used when *contourType*=BANDED or QUILT. The default value is "Black".

contourEdgeColor

A String specifying the color to be used to plot the contour edges when *contourType*=BANDED or ISOSURFACE. The default value is "Grey60".

tickmarkCurveColor

A String specifying the color to be used to plot the tick mark curve. The default value is "Cyan".

intervalLineAttributes

A tuple of tuples of SymbolicConstants specifying the line style and line thickness for each interval in the plot when *contourType*=LINE. The size of the outer sequence must be equal to *numIntervals*-1. The inner sequence consists of two SymbolicConstants specifying the line style and line thickness. For possible values, refer to the *edgeLineStyle* and *edgeLineThickness* members of the DGCommonOptions object. The default is ((SOLID, VERY_THIN),).

40.4 DisplayOptions object

The DisplayOptions object stores a plot state.

Access

```
import visualization
session.viewports[name].layers[name].odbDisplay.display
session.viewports[name].odbDisplay.display
```

40.4.1 setValues(...)

This method modifies the DisplayOptions object.

Arguments**Required arguments**

None.

Optional arguments*options*

A DisplayOptions object from which values are to be copied. If other arguments are also supplied to **setValues**, they will override the values in *options*. The default value is **None**.

plotState

A sequence of SymbolicConstants specifying the plot state of the display. Possible values are UNDEFORMED, DEFORMED, CONTOURS_ON_UNDEF, CONTOURS_ON_DEF, SYMBOLS_ON_UNDEF, SYMBOLS_ON_DEF, ORIENT_ON_UNDEF, and ORIENT_ON_DEF. The default value is (UNDEFORMED).

Return value

None

Exceptions

None.

40.4.2 Members

The DisplayOptions object can have the following member:

plotState

A tuple of SymbolicConstants specifying the plot state of the display. Possible values are UNDEFORMED, DEFORMED, CONTOURS_ON_UNDEF, CONTOURS_ON_DEF, SYMBOLS_ON_UNDEF, SYMBOLS_ON_DEF, ORIENT_ON_UNDEF, and ORIENT_ON_DEF. The default value is (UNDEFORMED).

40.5 DGDDisplayBodyOptions object

The DGDDisplayBodyOptions object stores values and attributes applied to display bodies. The DGDDisplayBodyOptions object has no constructor command. Abaqus creates an *odbDisplayOptions.displayBodyOptions* member when a display group instance is created, using values from *odbDisplay.displayBodyOptions*.

Access

```
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.displayBodyOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.displayBodyOptions
```


40.5.1 Members

The DGDDisplayBodyOptions object can have the following members:

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is EXTERIOR.

NONE can be used only when *renderStyle*=SHADED.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "White".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale=ON*. The default value is (1, 1, 1).

40.6 FreeBodyOptions object

The FreeBodyOptions object stores values and attributes associated with a free body plot. The FreeBodyOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.freeBodyOptions* member when you import the Visualization module. Abaqus creates a *FreeBodyOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.freeBodyOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

FreeBodyOptions objects are accessed in one of two ways:

- The default free body options. These settings are used as defaults when other *freeBodyOptions* members are created. These settings can be set to customize user preferences.
- The free body options associated with a particular viewport.

Access

```
import visualization
session.defaultOdbDisplay.freeBodyOptions
session.viewports[name].layers[name].odbDisplay.freeBodyOptions
session.viewports[name].odbDisplay.freeBodyOptions
```

40.6.1 setValues(...)

This method modifies the FreeBodyOptions object.

Arguments

Required arguments

None.

Optional arguments

comp1ColorF

A String specifying color of the first force component. The default value is "#FF1932".

comp1ColorM

A String specifying color of the first moment component. The default value is "#0000FF".

comp2ColorF

A String specifying color of the second force component. The default value is "#FF1932".

comp2ColorM

A String specifying color of the second moment component. The default value is "#0000FF".

comp3ColorF

A String specifying color of the third force component. The default value is "#FF1932".

comp3ColorM

A String specifying color of the third moment component. The default value is "#0000FF".

resultantColorF

A String specifying color of the resultant force. The default value is "#FF1932".

resultantColorM

A String specifying color of the resultant moment. The default value is "#0000FF".

textColorF

A String specifying text color for force. The default value is "Yellow".

textColorM

A String specifying text color for moment. The default value is "Yellow".

textFontF

A String specifying text font for force. The default value is "verdana".

textFontM

A String specifying text font for moment. The default value is "verdana".

numberFormatF

A SymbolicConstant specifying the number format for force. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

numberFormatM

A SymbolicConstant specifying the number format for moment. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

scaleModeF

A SymbolicConstant specifying the size scaling mode for force. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

scaleModeM

A SymbolicConstant specifying the size scaling mode for moment. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

vectorDisplay

A SymbolicConstant specifying the vector display mode. Possible values are RESULTANT and COMPONENT. The default value is RESULTANT.

numDigitsF

An Int specifying the number of digits in the force label. The default value is 3.

numDigitsM

An Int specifying the number of digits in the moment label. The default value is 3.

PLOT OPTIONS COMMANDS

sizePercentageF

A Float specifying the size of the force symbol as a percentage of the screen or model. The default value is 10.0.

sizePercentageM

A Float specifying the size of the moment symbol as a percentage of the screen or model. The default value is 10.0.

thresholdF

A Float specifying the force threshold value. The default value is 10^{-6} .

thresholdM

A Float specifying the moment threshold value. The default value is 10^{-6} .

drawLabelF

A Boolean specifying whether to draw force labels. The default value is ON.

drawLabelM

A Boolean specifying whether to draw moment labels. The default value is ON.

showComp1F

A Boolean specifying whether to show the first force component. The default value is ON.

showComp1M

A Boolean specifying whether to show the first moment component. The default value is ON.

showComp2F

A Boolean specifying whether to show the second force component. The default value is ON.

showComp2M

A Boolean specifying whether to show the second moment component. The default value is ON.

showComp3F

A Boolean specifying whether to show the third force component. The default value is ON.

showComp3M

A Boolean specifying whether to show the third moment component. The default value is ON.

showForce

A Boolean specifying whether to show forces. The default value is ON.

showMoment

A Boolean specifying whether to show moments. The default value is ON.

constantLengthArrow

A Boolean specifying whether to use a constant length for all arrows. The default value is OFF.

Return value

None

Exceptions

None.

40.6.2 Members

The FreeBodyOptions object has members with the same names and descriptions as the arguments to the `setValues` method.

40.7 HistoryVariable object

The HistoryVariable object stores history data.

Access

```
import visualization
session.odbData[name].historyVariables[i]
```

40.7.1 Members

The HistoryVariable object has the following members:

name

A String specifying the history request label. This String is read-only.

legendLabel

A String specifying the legend text. This String is read-only.

steps

A tuple of (String, Int, SymbolicConstant) tuples specifying the steps. This sequence is read-only. Each inner sequence contains the following elements:

- *stepLabel*: A String specifying the step label.
- *stepNumber*: An Int specifying the step number.
- *procedureDomain*: A SymbolicConstant specifying the analysis type of the step, which can have these values: “TIME,” “FREQUENCY,” or “MODAL.”

40.8 OdbData object

The OdbData object stores non persistent values and attributes associated with an open odb for the given session. The OdbData object has no constructor. Abaqus creates the *odbData* repository when you import the Visualization module. Abaqus creates a OdbData object when an odb is opened.

Access

```
import visualization
session.odbdData[name]
```

40.8.1 setValues(...)

This method modifies the OdbData object.

Arguments

Required arguments

None.

Optional arguments

activeFrames

A sequence specifying the active frames, or the SymbolicConstant ALL. Each item in the sequence is a tuple defining the stepName followed by a sequence of expressions specifying frame numbers. The expression can be any of the following:

- An Int specifying a single frame number; for example, `1`.
- A String specifying a single frame number ; for example, `'7'`.
- A String specifying a sequence of frame numbers; for example, `'3:5'` and `'3:15:3'`.

For these expressions a negative number will indicate reverse numbering: -1 is the last frame of the step, -2 is the one before the last frame. Frame numbering starts at 0.

stepPeriods

A sequence of (String, Float) sequences specifying the stepName and the stepPeriod. Alternatively, this member may take the value ODB_VALUES.

Return value

None

Exceptions

None.

40.8.2 Members

The OdbData object can have the following members:

activeFrames

A tuple specifying the active frames, or the SymbolicConstant ALL. Each item in the sequence is a tuple defining the stepName followed by a sequence of expressions specifying frame numbers. The expression can be any of the following:

- An Int specifying a single frame number; for example, **1**.
- A String specifying a single frame number ; for example, **'7'**.
- A String specifying a sequence of frame numbers; for example, **'3:5'** and **'3:15:3'**.

For these expressions a negative number will indicate reverse numbering: -1 is the last frame of the step, -2 is the one before the last frame. Frame numbering starts at 0.

stepPeriods

A tuple of (String, Float) tuples specifying the stepName and the stepPeriod. Alternatively, this member may take the value ODB_VALUES.

historyVariables

A repository of HistoryVariable objects specifying the history request label. The repository is read-only.

steps

A repository of OdbDataStep objects specifying the list of steps. The repository is read-only.

instances

A repository of OdbDataInstance objects specifying the list of instances. The repository is read-only.

materials

A repository of OdbDataMaterial objects specifying the list of materials. The repository is read-only.

sections

A repository of OdbDataSection objects specifying the list of sections. The repository is read-only.

elementSets

A repository of OdbDataElementSet objects specifying the list of element sets. The repository is read-only.

nodeSets

A repository of OdbDataNodeSet objects specifying the list of node sets. The repository is read-only.

surfaceSets

A repository of OdbDataSurfaceSet objects specifying the list of surface sets. The repository is read-only.

datumCsyses

A repository of OdbDataDatumCsys objects specifying the list of coordinate systems defined in the model. The repository is read-only.

kinematicCouplings

A repository of CouplingConstraint objects specifying the list of kinematic couplings. The repository is read-only.

distributingCouplings

A repository of CouplingConstraint objects specifying the list of distributing couplings. The repository is read-only.

shellSolidCouplings

A repository of CouplingConstraint objects specifying the list of shellsolid couplings. The repository is read-only.

rigidbodies

A repository of RigidBodyConstraint objects specifying the list of rigid body constraints. The repository is read-only.

multiPointConstraints

A repository of MpcConstraint objects specifying the list of multipoint constraints. The repository is read-only.

ties

A repository of TieConstraint objects specifying the list of Tie constraints. The repository is read-only.

diagnosticData

An OdbDiagnosticData object.

40.9 OdbDataDatumCsys object

The OdbDataDatumCsys object stores coordinate system data.

Access

```
import visualization
session.odbData[name].datumCsyses[i]
```

40.9.1 Members

The OdbDataDatumCsys object can have the following members:

name

A String specifying the coordinate system name. This String is read-only.

type

A SymbolicConstant specifying the coordinate system type. This String is read-only. Possible values are CARTESIAN, CYLINDRICAL, and SPHERICAL.

xAxis

A tuple of three Floats specifying a sequence of three floats specifying the x-Axis vector. The default value is (1, 0, 0).

yAxis

A tuple of three Floats specifying a sequence of three floats specifying the y-Axis vector. The default value is (0, 1, 0).

zAxis

A tuple of three Floats specifying a sequence of three floats specifying the z-Axis vector. The default value is (0, 0, 1).

origin

A tuple of three Floats specifying a sequence of three floats specifying the origin. The default value is (0, 0, 0).

40.10 OdbDataElementSet object

The OdbDataElementSet object stores element set data.

Access

```
import visualization
session.odbData[name].elementSets[i]
```

40.10.1 Members

The OdbDataElementSet object has the following members:

name

A String specifying the set name. This attribute is read-only.

elements

A String-to-tuple-of-Ints Dictionary specifying the elements in the set. This attribute is read-only.

40.11 OdbDataFrame object

The OdbDataFrame object.

Access

```
import visualization
session.odbData[name].steps[i].frames[i]
```

40.11.1 setValues(...)

This method modifies the OdbDataFrame object.

Arguments

Required argument

activateFrame

A Boolean specifying whether to activate the frame.

Optional argument

update

A Boolean specifying whether to update the model. The default value is ON

Return value

None

Exceptions

None.

40.11.2 Members

The OdbDataFrame object has no members.

40.12 OdbDataInstance object

The OdbDataInstance object instance data.

Access

```
import visualization
session.odbData[name].instances[i]
```

40.12.1 Members

The OdbDataInstance object has the following member:

name

A String specifying the instance name. This attribute is read-only.

40.13 OdbDataMaterial object

The OdbDataMaterial object stores material data.

Access

```
import visualization
session.odbData[name].materials[i]
```

40.13.1 Members

The OdbDataMaterial object has the following members:

name

A String specifying the set name. This attribute is read-only.

elements

A String-to-tuple-of-Ints Dictionary specifying the elements in the set. This attribute is read-only.

40.14 OdbDataNodeSet object

The OdbDataNodeSet object stores node set data.

Access

```
import visualization
session.odbData[name].nodeSets[i]
```

40.14.1 Members

The OdbDataNodeSet object has the following members:

name

A String specifying the set name. This attribute is read-only.

nodes

A String-to-tuple-of-Ints Dictionary specifying the nodes in the set. This attribute is read-only.

40.15 OdbDataSection object

The OdbDataSection object stores section data.

Access

```
import visualization
session.odbData[name].sections[i]
```

40.15.1 Members

The OdbDataSection object has the following members:

name

A String specifying the set name. This attribute is read-only.

elements

A String-to-tuple-of-Ints Dictionary specifying the elements in the set. This attribute is read-only.

40.16 OdbDataStep object

The OdbDataStep object.

Access

```
import visualization
session.odbData[name].steps[i]
```

40.16.1 setValues(...)

This method modifies the OdbDataStep object.

Arguments

Required argument

activateFrames

A Boolean specifying whether to activate all the frames in the step.

Optional argument

update

A Boolean specifying whether to update the model. The default value is ON

Return value

None

Exceptions

None.

40.16.2 Members

The OdbDataStep object has the following member:

frames

An OdbDataFrameArray object specifying the list of frames. The list is read-only.

40.17 OdbDataSurfaceSet object

The OdbDataSurfaceSet object stores surface set data.

Access

```
import visualization
session.odbData[name].surfaceSets[i]
```

40.17.1 Members

The OdbDataSurfaceSet object has the following members:

name

A String specifying the set name. This attribute is read-only.

elements

A String-to-tuple-of-Ints Dictionary specifying the elements in the set. This attribute is read-only.

facets

A String-to-tuple-of-Ints Dictionary specifying the facets corresponding to the *elements*. This attribute is read-only.

40.18 CouplingConstraint object

The CouplingConstraint object.

Access

```
import visualization
session.odbData[name].kinematicCouplings[i]
session.odbData[name].distributingCouplings[i]
session.odbData[name].shellSolidCouplings[i]
```

40.18.1 constraintData(...)

This method returns node numbers of the surface being controlled by the control point.

Arguments

Required arguments

None.

Optional arguments

None.

Return value

Tuple-of-Ints Dictionary specifying the node numbers on the controlled surface.

Exceptions

None.

40.18.2 Members

The CouplingConstraint object has the following members:

name

A String specifying the coupling name. This attribute is read-only.

type

A String specifying the type of coupling. This attribute is read-only.

40.19 RigidBodyConstraint object

The RigidBodyConstraint object.

Access

```
import visualization
session.odbData[name].rigidbodies[i]
```

40.19.1 constraintData(...)

This method returns constraint data if any are associated with the object.

Arguments

Required arguments

None.

Optional arguments

None.

Return value

A String Value: NONE in the absence of constraint data.

Exceptions

None.

40.19.2 Members

The RigidBodyConstraint object has the following members:

name

A String specifying the rigidbody constraint name. This attribute is read-only.

40.20 TieConstraint object

The TieConstraint object.

Access

```
import visualization
session.odbData[name].ties[i]
```

40.20.1 constraintData(...)

This method returns constraint data if any are associated with the object

Arguments**Required arguments**

None.

Optional arguments

None.

Return value

A tuple containing coordinates of the nodes pertaining to the constraint.

Exceptions

None.

40.20.2 Members

The TieConstraint object has the following members:

name

A String specifying the tie constraint name. This attribute is read-only.

40.21 MpcConstraint object

The MpcConstraint object.

Access

```
import visualization
session.odbData[name].multiPointConstraints[i]
```

40.21.1 constraintData(...)

This method returns constraint data if any are associated with the object.

Arguments

Required arguments

None.

Optional arguments

None.

Return value

A tuple containing coordinates of the nodes pertaining to the constraint.

Exceptions

None.

40.21.2 Members

The MpcConstraint object has the following members:

name

A String specifying the multipoint constraint name. This attribute is read-only.

40.22 OdbDiagnosticData object

The OdbDiagnosticData object.

Access

```
import visualization
session.odbData[name].diagnosticData
```

40.22.1 Members

The OdbDiagnosticData object has the following members:

analysisErrors

A repository of OdbAnalysisError objects.

analysisWarnings

A repository of OdbAnalysisWarning objects.

steps

A repository of OdbDiagnosticStep objects.

jobTime

An OdbJobTime object.

numericalProblemSummary

An OdbNumericalProblemSummary object.

isXplDoublePrecision

A boolean specifying whether or not double precision is used for the analysis. This attribute is read-only.

jobStatus

A String specifying the job status after the analysis. This attribute is read-only.

numDomains

An int specifying the number of domains. This attribute is read-only.

numberOfAnalysisErrors

An int specifying the number of analysis errors encountered. This attribute is read-only.

numberOfAnalysisWarnings

An int specifying the number of analysis warnings encountered. This attribute is read-only.

numberOfSteps

An int specifying the number of steps present in the analysis. This attribute is read-only.

40.23 OdbAnalysisError object

The OdbAnalysisError object stores the description of different errors encountered during the analysis.

Access

```
import visualization
session.odbData[name].diagnosticData.analysisErrors[i]
```

40.23.1 Members

The OdbAnalysisError object has the following members:

incrementNumber

An int specifying the increment number where the analysis was aborted. This attribute is read-only.

iterationNumber

An int specifying the iteration number where the analysis was aborted. This attribute is read-only.

attemptNumber

An int specifying the attempt number on which the analysis was aborted. This attribute is read-only.

category

String specifying the category of error. This attribute is read-only.

data

An OdbAuxiliaryData object.

description

String specifying the cause of the error. This attribute is read-only.

detailStrings

String specifying the exact nature of the problem. This attribute is read-only.

knowledgeItem

String specifying the exact reason for the error encountered. This attribute is read-only.

numberOfVariations

An int specifying the number of variations. This attribute is read-only.

stepNumber

An int specifying the step number on which the error was encountered. This attribute is read-only.

40.24 OdbAnalysisWarning object

The OdbAnalysisWarning object stores the description of different warnings encountered during the analysis.

Access

```
import visualization
session.odbData[name].diagnosticData.analysisWarnings[i]
```

40.24.1 Members

The OdbAnalysisWarning object has members that are similar to the OdbAnalysisError object.

40.25 OdbJobTime object

The OdbJobTime object stores the analysis time of a job.

Access

```
import visualization
session.odbData[name].diagnosticData.jobTime
```

40.25.1 Members

The OdbJobTime object has the following members:

systemTime

A float specifying the systemtime for the analysis. This attribute is read-only.

userTime

A float specifying the usertime for the analysis. This attribute is read-only.

wallclockTime

A float specifying the wallclocktime for the analysis. This attribute is read-only.

40.26 OdbNumericalProblemSummary object

The OdbNumericalProblemSummary object stores the numerical problem summary of a job.

Access

```
import visualization
session.odbData[name].diagnosticData.numericalProblemSummary
```

40.26.1 Members

The OdbNumericalProblemSummary object has the following members:

convergedNegativeEigenValues

A boolean specifying whether negative eigenvalues converged during the analysis. This attribute is read-only.

convergedNumericalSingularities

A boolean specifying whether numerical singularities converged during the analysis. This attribute is read-only.

convergedZeroPivots

A boolean specifying whether pivot points converged during the analysis. This attribute is read-only.

numberOfZeroPivots

An int specifying the number of zero pivots. This attribute is read-only.

numberOfNumericalSingularities

An int specifying the number of numerical singularities. This attribute is read-only.

numberOfNegativeEigenValues

An int specifying the number of negative eigenvalues. This attribute is read-only.

40.27 OdbDiagnosticStep object

The OdbDiagnosticStep object stores step data.

Access

```
import visualization
session.odbData[name].diagnosticData.steps[i]
```

40.27.1 **extractData(...)**

This method creates a temporary XYData object, with increments on the x-axis and requested output on the y-axis.

Arguments

Required argument

incrementStatistics

An enum specifying the requested output variable for the data table. Possible enum values are NUM_ATTEMPTS (the number of attempts), NUM_SDI (the number of severe discontinuity iterations), NUM_EQI (the number of equivalent iterations), NUM_ITERS (the number of iterations), STEP_TIME (the cumulative step time until that increment) or INC_SIZE (the step time for each increment).

Optional arguments

None.

Return value

None

Exceptions

None.

40.27.2 **Members**

The OdbDiagnosticStep object has the following members:

activeXplStatus

A Tuple of the status values. This attribute is read-only.

characteristicElementLength

A float specifying the characteristic element length for the step. This attribute is read-only.

contactDiagnostics

A repository of OdbContactDiagnostic objects.

explicitIncrementStatus

A sequence of string specifying the explicit increment status. This attribute is read-only.

extrapolation

A String specifying the method (Linear or logarithmic) used for extrapolation. This attribute is read-only.

PLOT OPTIONS COMMANDS

incrementationScheme

A String specifying the method of incrementation (Auto or fixed). This attribute is read-only.

incrementsCompleted

An int specifying the number of completed increments. This attribute is read-only.

increments

A repository of OdbDiagnosticIncrement objects.

initialTimeIncrement

A float specifying the initial increment size for the step. This attribute is read-only.

isNlgeom

A boolean specifying whether or not the effects of geometric nonlinearities are considered. This attribute is read-only.

isPerturbation

A boolean specifying whether or not the step is a perturbation step. This attribute is read-only.

isStabilized

A boolean specifying whether or not stabilization for the system in any form is considered. This attribute is read-only.

isRiks

A boolean specifying whether the step is static riks. This attribute is read-only.

isUnsymm

A boolean specifying whether the matrix storage is unsymmetric. This attribute is read-only.

matrixSolver

A string specifying the method of solving (Direct or Iterative). This attribute is read-only.

maximumNumberOfIncrements

An int specifying the maximum number of allowed increments in the step. This attribute is read-only.

maximumTimeIncrement

A float specifying the size of the allowed maximum time increment in the step. This attribute is read-only.

minimumTimeIncrement

A float specifying the size of the allowed minimum time increment in the step. This attribute is read-only.

name

A string specifying the name of the step. This attribute is read-only.

number

An int specifying the step number. This attribute is read-only.

numberOfContactDiagnostics

An int specifying the number of contact diagnostics encountered. This attribute is read-only.

numberOfIncrements

An int specifying the number of increments taken in the step to complete the solution. This attribute is read-only.

numberOfXplStatus

An int specifying the number of the explicit status. This attribute is read-only.

stabilizeFactor

A float specifying the stabilize factor. This attribute is read-only.

stepTimeCompleted

A float specifying the time taken for the completion of the step. This attribute is read-only.

timePeriod

A float specifying the duration for the step. This attribute is read-only.

40.28 OdbAuxiliaryData object

The OdbAuxiliaryData object stores auxiliary data related to the steps in the analysis.

Access

```
import visualization
session.odbData[name].diagnosticData.analysisErrors[i].data[i]
session.odbData[name].diagnosticData.analysisWarnings[i].data[i]
session.odbData[name].diagnosticData.steps[i].contactDiagnostics[i].da
```

40.28.1 Members

The OdbAuxiliaryData object has the following members:

elementInstanceNames

A tuple consisting of element instance names. This attribute is read-only.

elementLabels

A tuple consisting of element label names. This attribute is read-only.

hasValues

A boolean specifying whether the OdbAuxiliaryData object has values in it. This attribute is read-only.

nodeInstanceNames

A tuple consisting of node instance names. This attribute is read-only.

nodeLabels

A tuple consisting of node label names. This attribute is read-only.

40.29 OdbDiagnosticIncrement object

The OdbDiagnosticIncrement object.

Access

```
import visualization
session.odbData[name].diagnosticData.steps[i].increments[i]
```

40.29.1 Members

The OdbDiagnosticIncrement object has the following members:

attempts

A repository of OdbDiagnosticAttempt objects.

initialSize

A float specifying the size of the initial increment. This attribute is read-only.

isConverged

A boolean specifying whether the solution converged for the particular increment. This attribute is read-only.

numberOfAttempts

An int specifying the number of attempts for the particular increment. This attribute is read-only.

numberOfElementDiagnostics

An int specifying the number of element diagnostics encountered for the particular increment. This attribute is read-only.

size

A float specifying the size of the particular increment. This attribute is read-only.

stepTimeCompleted

A float specifying the amount of step time completed in the particular increment. This attribute is read-only.

40.30 OdbDiagnosticAttempt object

The OdbDiagnosticAttempt object.

Access

```
import visualization
session.odbData[name].diagnosticData.steps[i].increments[i].attempts[i]
```

40.30.1 Members

The OdbDiagnosticAttempt object has the following members:

autoStabilize

A boolean specifying the state of Auto-stabilization. This attribute is read-only.

isConverged

A boolean specifying the state of convergence for the attempt. This attribute is read-only.

isCutBack

A boolean specifying the state of cutback. This attribute is read-only.

needsReordering

A boolean specifying whether or not reordering is needed. This attribute is read-only.

numberOfCutbackDiagnostics

An int specifying the number of cutback diagnostics. This attribute is read-only.

numberOfIterations

An int specifying the number of iterations for the particular attempt. This attribute is read-only.

numberOfSevereDiscontinuityIterations

An int specifying the number of iterations with severe discontinuities This attribute is read-only.

size

A float specifying the size of the increment of the particular attempt. This attribute is read-only.

40.31 OdbContactDiagnostics object

The OdbDiagnosticContact object.

Access

```
import visualization
session.odbData[name].diagnosticData.steps[i].contactDiagnostics[i]
```

40.31.1 Members

The OdbContactDiagnostic object has the following members:

data

A repository of OdbAuxiliaryData objects. This attribute is read-only.

description

A string specifying the opening/overclosure status of the contact. This attribute is read-only.

detailStrings

A sequence of strings specifying the nature of each of the contact pair. This attribute is read-only.

type

A string specifying the type of contact initialization. This attribute is read-only.

defaultFormats

A string specifying the default format value. This attribute is read-only.

elementDescriptions

A string specifying the element description. This attribute is read-only.

nodeDescriptions

A string specifying the node description. This attribute is read-only.

40.32 OdbDisplayOptions object

The OdbDisplayOptions object stores the display options associated with an OdbInstance object. This object does not have a constructor. Abaqus creates the OdbDisplayOptions object when an OdbInstance object is created using the display options associated with the current viewport at the time of creation.

Access

```
import assembly
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions
import visualization
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions
import part
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions
```

```
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions
```

40.32.1 Members

The OdbDisplayOptions object has the following members:

commonOptions

A DGCommonOptions object.

superimposeOptions

A DGSuperimposeOptions object.

contourOptions

A DGContourOptions object.

symbolOptions

A DGSymbolOptions object.

materialOrientationOptions

A DGOrientationOptions object.

displayBodyOptions

A DGDisplayBodyOptions object.

40.33 OptionArg object

The OptionArg object is used to store values and attributes as a temporary object to be associated with a viewCutOptions object. The OptionArg object has only a constructor command.

Access

```
import visualization
session.defaultOdbDisplay.viewCutOptions.aboveOptions
session.defaultOdbDisplay.viewCutOptions.belowOptions
session.defaultOdbDisplay.viewCutOptions.onOptions
session.viewports[name].layers[name].odbDisplay.viewCutOptions\
.aboveOptions
session.viewports[name].layers[name].odbDisplay.viewCutOptions\
.belowOptions
session.viewports[name].layers[name].odbDisplay.viewCutOptions\
.onOptions
session.viewports[name].odbDisplay.viewCutOptions.aboveOptions
session.viewports[name].odbDisplay.viewCutOptions.belowOptions
session.viewports[name].odbDisplay.viewCutOptions.onOptions
```

40.33.1 OptionArg(...)

This method creates an OptionArg object.

Path

`visualization.OptionArg`

Required arguments

None.

Optional arguments

renderStyle

A SymbolicConstant specifying the render style of the plot. Possible values are WIREFRAME, FILLED, HIDDEN, and SHADED. The default value is WIREFRAME.

visibleEdges

A SymbolicConstant specifying which edges to plot. Possible values are ALL, EXTERIOR, FEATURE, FREE, and NONE. The default value is FEATURE.

NONE can be used only when *renderStyle*=SHADED.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the undeformed plot when *renderStyle*=WIREFRAME or HIDDEN. The default value is "Green".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the undeformed plot when *renderStyle*=FILLED or SHADED. The default value is "Black".

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "Green".

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency* = ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

Return value

An OptionArg object.

Exceptions

RangeError.

40.33.2 Members

The OptionArg object has members with the same names and descriptions as the arguments to the `OptionArg` method.

40.34 DGOrientationOptions object

The DGOrientationOptions object stores values and attributes associated with a material orientation plot. The DGOrientationOptions object has no constructor command. Abaqus creates an `odbDisplayOptions.materialOrientationOptions` member when a display group instance is created, using values from `odbDisplay.materialOrientationOptions`.

Access

```
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.materialOrientationOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.materialOrientationOptions
```

40.34.1 Members

The DGOrientationOptions object can have the following members:

PLOT OPTIONS COMMANDS

showAxis1

A Boolean specifying whether axis 1 of the material orientation triad should be displayed. The default value is ON.

showAxis2

A Boolean specifying whether axis 2 of the material orientation triad should be displayed. The default value is ON.

showAxis3

A Boolean specifying whether axis 3 of the material orientation triad should be displayed. The default value is ON.

symbolSize

A Float specifying the size of the material orientation triad. The default value is 6.0.

lineThickness

A SymbolicConstant specifying the thickness of the material orientation triad. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

orientation

A SymbolicConstant specifying the orientation used for composites. Possible values are PLY and LAYUP. The default value is PLY.

arrowheadStyle

A SymbolicConstant specifying the arrowhead style for the material orientation triad. Possible values are NONE, FILLED, and WIRE. The default value is NONE.

scaleMode

A SymbolicConstant specifying the scaling basis for the material orientation triad. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

axis1Color

A String specifying the color of axis 1 of the material orientation triad. The default value is "Cyan".

axis2Color

A String specifying the color of axis 2 of the material orientation triad. The default value is "Yellow".

axis3Color

A String specifying the color of axis 3 of the material orientation triad. The default value is "Red".

40.35 StreamOptions object

The StreamOptions object stores values and attributes associated with a stream plot. The StreamOptions object has no constructor command. Abaqus creates a *defaultOdbDisplay.streamOptions* member when you import the Visualization module. Abaqus creates a *StreamOptions* member when it creates the OdbDisplay object, using the values from *defaultOdbDisplay.streamOptions*. Abaqus creates the *odbDisplay* member when a viewport is created, using the values from *defaultOdbDisplay*.

StreamOptions objects are accessed in one of two ways:

- The default stream options. These settings are used as defaults when other *streamOptions* members are created. These settings can be set to customize user preferences.
- The stream options associated with a particular viewport.

Access

```
import visualization
session.defaultOdbDisplay.streamOptions
session.viewports[name].layers[name].odbDisplay.streamOptions
session.viewports[name].odbDisplay.streamOptions
```

40.35.1 setValues(...)

This method modifies the StreamOptions object.

Arguments

Required arguments

None.

Optional arguments

colorMethod

A SymbolicConstant specifying the method for coloring stream lines. Possible values are UNIFORM, SPECTRUM, and CONTINUOUS. The default value is UNIFORM.

uniformColor

A String specifying uniform color of the stream lines. The default value is "#FFFF00".

lineThickness

A Float specifying the stream line thickness. The default value is 6.0.

showArrow

A Boolean specifying whether to show arrows. The default value is OFF.

numArrows

An Int specifying the number of arrows on each stream line. The default value is 10.

Return value

None

Exceptions

None.

40.35.2 Members

The `StreamOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

40.36 DGSuperimposeOptions object

The `DGSuperimposeOptions` object stores values and attributes associated with the undeformed shape when the display contains both the deformed shape and the undeformed shape. The `DGSuperimposeOptions` object has no constructor command. Abaqus creates an `odbDisplayOptions.superimposeOptions` member when a display group instance is created, using values from `odbDisplay.superimposeOptions`.

Access

```
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.superimposeOptions
session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.superimposeOptions
```

40.36.1 Members

The `DGSuperimposeOptions` object can have the following members:

renderStyle

A `SymbolicConstant` specifying the render style of the plot. Possible values are `WIREFRAME`, `FILLED`, `HIDDEN`, and `SHADED`. The default value is `WIREFRAME`.

visibleEdges

A `SymbolicConstant` specifying which edges to plot. Possible values are `ALL`, `EXTERIOR`, `FEATURE`, `FREE`, and `NONE`. The default value is `FEATURE`.

`NONE` can be used only when *renderStyle*=`SHADED`.

edgeLineStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeLineThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

colorCodeOverride

A Boolean specifying whether to allow color coded items in the output database to override the edge and fill color settings. The default value is ON.

elemLabels

A Boolean specifying whether to plot the element labels. The default value is OFF.

faceLabels

A Boolean specifying whether to plot the face labels. The default value is OFF.

nodeLabels

A Boolean specifying whether to plot the node labels. The default value is OFF.

nodeSymbols

A Boolean specifying whether to plot the node symbols. The default value is OFF.

nodeSymbolType

A SymbolicConstant specifying the node symbol types. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER

The default value is HOLLOW_CIRCLE.

nodeSymbolSize

A SymbolicConstant specifying the node symbol size. Possible values are SMALL, MEDIUM, and LARGE. The default value is SMALL.

elementShrink

A Boolean specifying whether elements are displayed in a shrunk format. The default value is OFF.

PLOT OPTIONS COMMANDS

elementShrinkFactor

An Int specifying the percentage to shrink the elements when *elementShrink*=ON. Possible values are $0 \leq \text{elementShrinkPercentage} \leq 90$. The default value is 5.

coordinateScale

A Boolean specifying whether to scale coordinates. The default value is OFF.

normals

A Boolean specifying whether to draw arrows that indicate the directions of element and surface normals. The default value is OFF.

normalDisplay

A SymbolicConstant specifying whether to draw element normals and surface normals. Possible values are ELEMENT and SURFACE. The default value is ELEMENT.

normalArrowLength

A SymbolicConstant specifying the length of the normal arrows. Possible values are SHORT, MEDIUM, and LONG. The default value is MEDIUM.

normalLineThickness

A SymbolicConstant specifying the thickness of the normal arrows. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

normalArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the normal arrows. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

edgeColorWireHide

A String specifying the color to be used to plot the edges of the model when *renderStyle*=WIREFRAME or HIDDEN. The default value is "Green".

edgeColorFillShade

A String specifying the color to be used to plot the edges of the model when *renderStyle*=FILLED or SHADED. The default value is "Black".

fillColor

A String specifying the color to be used to fill elements when *renderStyle*=FILLED or SHADED. The default value is "Green".

labelFont

A String specifying the label font to be used for all model labels. The default value is "-*-courier-medium-r-normal-*-*-120-*-*-m-*-*-".

elemLabelColor

A String specifying the color to be used to plot the element labels. The default value is "Cyan".

faceLabelColor

A String specifying the color to be used to plot the face labels. The default value is "Red".

nodeLabelColor

A String specifying the color to be used to plot the node labels. The default value is "Yellow".

nodeSymbolColor

A String specifying the color to be used to plot the node symbols. The default value is "Yellow".

faceNormalColor

A String specifying the color to be used to plot the normal to a nonbeam element or to a surface. The default value is "Red".

beamN1Color

A String specifying the color to be used to plot an arrow along the beam n_1 -direction. The default value is "Blue".

beamN2Color

A String specifying the color to be used to plot an arrow along the beam n_2 -direction. The default value is "Red".

beamTangentColor

A String specifying the color to be used to plot an arrow along the tangent to a beam. The default value is "White".

coordinateScaleFactors

A tuple of three Floats specifying the coordinate scaling in each of the three coordinate directions when *coordinateScale*=ON. The default value is (1, 1, 1).

40.37 DGSymbolOptions object

The DGSymbolOptions object stores values and attributes associated with a symbol plot. The DGSymbolOptions object has no constructor command. Abaqus creates an *odbDisplayOptions.symbolOptions* member when a display group instance is created, using values from *odbDisplay.symbolOptions*.

Access

```
session.viewports[name].assemblyDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].assemblyDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].odbDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
session.viewports[name].layers[name].partDisplay\
.displayGroupInstances[name].odbDisplayOptions.symbolOptions
```

```

session.viewports[name].odbDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions
session.viewports[name].partDisplay.displayGroupInstances[name] \
.odbDisplayOptions.symbolOptions

```

40.37.1 Members

The DGSymbolOptions object can have the following members:

vectorLineThickness

A SymbolicConstant specifying the vector line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

vectorColorMethod

A SymbolicConstant specifying the vector color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

tensorColorMethod

A SymbolicConstant specifying the tensor color method. Possible values are UNIFORM and SPECTRUM. The default value is SPECTRUM.

vectorArrowheadStyle

A SymbolicConstant specifying the vector arrowhead style. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

arrowSymbolSize

An Int specifying the length of vector and tensor symbols. The default value is 6.

vectorIntervalNumber

An Int specifying the number of color intervals for vector symbols. The default value is 12.

symbolDensity

A Float specifying the factor for randomized sampling. The default value is 1.0.

constantLengthArrows

A Boolean specifying whether to use constant-length arrows for vector symbols. The default value is OFF.

tensorIntervalNumber

An Int specifying the number of color intervals for tensor symbols. The default value is 12.

tensorLineThickness

A SymbolicConstant specifying the line thickness of the tensor symbols. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

tensorArrowheadStyle

A SymbolicConstant specifying the arrowhead style of the tensor symbols. Possible values are NONE, FILLED, and WIRE. The default value is WIRE.

numberFormatT

A SymbolicConstant specifying the number format for tensor. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

numberFormatV

A SymbolicConstant specifying the number format for vector. Possible values are SCIENTIFIC, FIXED, and ENGINEERING. The default value is SCIENTIFIC.

arrowScaleMode

A SymbolicConstant specifying the arrow scaling mode. Possible values are MODEL_SIZE and SCREEN_SIZE. The default value is MODEL_SIZE.

drawLabelT

A Boolean specifying whether to draw tensor labels. The default value is OFF.

drawLabelV

A Boolean specifying whether to draw vector labels. The default value is OFF.

numDigitsT

An Int specifying the number of digits in the tensor label. The default value is 2.

numDigitsV

An Int specifying the number of digits in the vector label. The default value is 2.

vectorColor

A String specifying the vector color. The default value is "Red".

vectorColorSpectrum

A String specifying the vector color spectrum name. The default value is "Rainbow".

tensorColorSpectrum

A String specifying the tensor color spectrum name. The default value is "Rainbow".

textColorT

A String specifying the text color for tensor. The default value is "Yellow".

textColorV

A String specifying the text color for vector. The default value is "Yellow".

textFontT

A String specifying the text font for tensor. The default value is "verdana".

textFontV

A String specifying the text font for vector. The default value is "verdana".

tensorMaxPrinColor

A String specifying the color of the maximum principal tensor symbols. The default value is "Red".

tensorMinPrinColor

A String specifying the color of the minimum principal tensor symbols. The default value is "Cyan".

tensorMidPrinColor

A String specifying the color of the intermediate principal tensor symbols. The default value is "Yellow".

tensorSelectedPrinColor

A String specifying the color of the selected principal tensor symbols. The default value is "Red".

40.38 ViewCutOptions object

The `ViewCutOptions` object stores values and attributes associated with a view cut plot. The `ViewCutOptions` object has no constructor command. Abaqus creates a `defaultOdbDisplay.viewCutOptions` member when you import the Visualization module. Abaqus creates an `viewCutOptions` member when it creates the `OdbDisplay` object, using the values from `defaultOdbDisplay.viewCutOptions`. Abaqus creates the `odbDisplay` member when a viewport is created, using the values from `defaultOdbDisplay`.

`ViewCutOptions` objects are accessed in one of two ways:

- The default view cut options. These settings are used as defaults when other `viewCutOptions` members are created. These settings can be set to customize user preferences.
- The view cut options associated with a particular viewport.

Access

```
import visualization
session.defaultOdbDisplay.viewCutOptions
session.viewports[name].layers[name].odbDisplay.viewCutOptions
session.viewports[name].odbDisplay.viewCutOptions
```

40.38.1 setValues(...)

This method modifies the `ViewCutOptions` object.

Arguments

Required arguments

None.

Optional arguments

options

A `ViewCutOptions` object from which values are to be copied. If other arguments are also supplied to `setValues`, they will override the values in *options*. The default value is **None**.

belowOptions

None or an `OptionArg` object specifying values to be used for defining the options applicable on the model below the cut. The default value is **None**.

useBelowOptions

A Boolean specifying whether to use the options defined for displaying the model below the cut. The default value is OFF.

onOptions

None or an OptionArg object specifying values to be used for defining the options applicable on the model on the cut. The default value is **None**.

useOnOptions

A Boolean specifying whether to use the options defined for displaying the model on the cut. The default value is OFF.

aboveOptions

None or an OptionArg object specifying values to be used for defining the options applicable on the model above the cut. The default value is **None**.

useAboveOptions

A Boolean specifying whether to use the options defined for displaying the model above the cut. The default value is OFF.

freeBodyCutThru

A SymbolicConstant specifying the domain through which the free body cuts. Possible values are CURRENT_DISPLAY_GROUP and WHOLE_MODEL. The default value is CURRENT_DISPLAY_GROUP.

freeBodyStepThru

A SymbolicConstant specifying the domain through which the free body steps. Possible values are ACTIVE_CUT_RANGE and PREDEFINED_PATH. The default value is ACTIVE_CUT_RANGE.

numCutFreeBody

An Int specifying number of free bodies per view cut. The default value is 1.

displaySlicing

A Boolean specifying whether to display slicing. The default value is OFF.

slicingAtPathNodes

A Boolean specifying whether to slice at path nodes. The default value is OFF.

freeBodySumOnPath

A Boolean specifying whether to put the summation point at the path. The default value is ON.

cutFreeBodyMin

A Float specifying free body minimum value. The default value is 0.0.

cutFreeBodyMax

A Float specifying free body maximum value. The default value is 0.0.

showHeatFlowRate

A SymbolicConstant specifying whether to show the heat flow rate when available. Possible values are ON and OFF. The default value is ON.

summationLoc

A SymbolicConstant specifying the summation location for the free body cut. Possible values are CENTROID and SPECIFY. The default value is CENTROID.

PLOT OPTIONS COMMANDS

componentResolution

A SymbolicConstant specifying the component resolution choice for the free body cut. Possible values are NORMAL_TANGENTIAL and CSYS. The default value is NORMAL_TANGENTIAL.

csysName

The SymbolicConstant GLOBAL or a String specifying the coordinate system name for the free body cut's component resolution. The default value is GLOBAL.

pathName

A String specifying the name of the path along which slicing occurs. The default value is an empty string.

summationPoint

A sequence of three Floats specifying the summation point for the free body cut. The default value is (0, 0, 0).

yAxis

A sequence of three Floats specifying the Y axis for free body component resolution. The default value is (0, 1, 0).

Return value

None

Exceptions

RangeError.

40.38.2 Members

The ViewCutOptions object can have the following members:

useBelowOptions

A Boolean specifying whether to use the options defined for displaying the model below the cut. The default value is OFF.

useOnOptions

A Boolean specifying whether to use the options defined for displaying the model on the cut. The default value is OFF.

useAboveOptions

A Boolean specifying whether to use the options defined for displaying the model above the cut. The default value is OFF.

numCutFreeBody

An Int specifying number of free bodies per view cut. The default value is 1.

displaySlicing

A Boolean specifying whether to display slicing. The default value is OFF.

slicingAtPathNodes

A Boolean specifying whether to slice at path nodes. The default value is OFF.

freeBodySumOnPath

A Boolean specifying whether to put the summation point at the path. The default value is ON.

cutFreeBodyMin

A Float specifying free body minimum value. The default value is 0.0.

cutFreeBodyMax

A Float specifying free body maximum value. The default value is 0.0.

freeBodyCutThru

A SymbolicConstant specifying the domain through which the free body cuts. Possible values are CURRENT_DISPLAY_GROUP and WHOLE_MODEL. The default value is CURRENT_DISPLAY_GROUP.

freeBodyStepThru

A SymbolicConstant specifying the domain through which the free body steps. Possible values are ACTIVE_CUT_RANGE and PREDEFINED_PATH. The default value is ACTIVE_CUT_RANGE.

showHeatFlowRate

A SymbolicConstant specifying whether to show the heat flow rate when available. Possible values are ON and OFF. The default value is ON.

summationLoc

A SymbolicConstant specifying the summation location for the free body cut. Possible values are CENTROID and SPECIFY. The default value is CENTROID.

componentResolution

A SymbolicConstant specifying the component resolution choice for the free body cut. Possible values are NORMAL_TANGENTIAL and CSYS. The default value is NORMAL_TANGENTIAL.

belowOptions

None or an OptionArg object specifying values to be used for defining the options applicable on the model below the cut. The default value is **None**.

onOptions

None or an OptionArg object specifying values to be used for defining the options applicable on the model on the cut. The default value is **None**.

aboveOptions

None or an OptionArg object specifying values to be used for defining the options applicable on the model above the cut. The default value is **None**.

csysName

The SymbolicConstant GLOBAL or a String specifying the coordinate system name for the free body cut's component resolution. The default value is GLOBAL.

pathName

A String specifying the name of the path along which slicing occurs. The default value is an empty string.

summationPoint

A tuple of three Floats specifying the summation point for the free body cut. The default value is (0, 0, 0).

yAxis

A tuple of three Floats specifying the Y axis for free body component resolution. The default value is (0, 1, 0).

40.39 DetailPlotOptions object

The DetailPlotOptions object stores values and attributes associated with a Viewport object. The DetailPlotOptions object has no constructor command. Abaqus creates the *detailPlotOptions* member whenever a Viewport is created.

Access

`session.viewports[name].detailPlotOptions`

40.39.1 Members

The DetailPlotOptions object has the following member:

plyStackPlotOptions

A PlyStackPlotOptions object.

40.40 PlyStackPlotOptions object

The PlyStackPlotOptions object stores values and attributes associated with a Viewport object. The PlyStackPlotOptions object has no constructor command. Abaqus creates the *detailPlotOptions.plyStackPlotOptions* member whenever a Viewport is created.

Access

`session.viewports[name].detailPlotOptions.plyStackPlotOptions`

40.40.1 setValues(...)

This method modifies the PlyStackPlotOptions object.

Arguments**Required arguments**

None.

Optional arguments*renderStyle*

A SymbolicConstant specifying how the plies in the viewport are rendered. Possible values are WIRE_FRAME, FILLED, and SHADED. The default value is SHADED.

showEdges

A Boolean specifying whether to draw the edges for the plies. The default value is OFF.

edgeStyle

A SymbolicConstant specifying the edge line style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

edgeThickness

A SymbolicConstant specifying the edge line thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

plyDisplay

A SymbolicConstant specifying whether to display the plies in an unsymmetric staircase fashion or symmetric staircase fashion. Possible values are UNSYMMETRIC and SYMMETRIC. The default value is UNSYMMETRIC.

startLayer

An Int specifying the start ply. The default value is 1.

numLayers

An Int specifying the number of plies to show. The default value is 30.

evenNumPlyColor

A String specifying the ply color for the even numbered plies. The numbering is based on the displayed plies and not on the actual ply number in the layup. The default value is "#A2B3FF".

oddNumPlyColor

A String specifying the ply color for the odd numbered plies. The numbering is based on the displayed plies and not on the actual ply number in the layup. The default value is "#ECC9AD".

showFibers

A Boolean specifying whether to show the fibers. The default value is ON.

fiberColor

A String specifying the fiber color. The default value is "Red".

fiberStyle

A SymbolicConstant specifying the fiber style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

PLOT OPTIONS COMMANDS

fiberThickness

A SymbolicConstant specifying the fiber thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

fiberSpacing

A Float specifying the spacing between the fibers. The default value is 0.1.

showReferencePlane

A Boolean specifying whether to show the reference surface. The default value is OFF.

referenceSurfaceColor

A String specifying the reference surface color. The default value is "White".

translucency

A Boolean specifying whether to set translucency. The default value is OFF.

translucencyFactor

A Float specifying the translucency factor when *translucency*=ON. Possible values are $0.0 \leq \text{translucencyFactor} \leq 1.0$. The default value is 0.3.

translucencySort

A Boolean specifying whether to use depth sorting. The default value is OFF.

showReferenceOutline

A Boolean specifying whether to show the reference outline. The default value is OFF.

referenceOutlineColor

A String specifying the reference outline color. The default value is "Red".

referenceOutlineStyle

A SymbolicConstant specifying the reference outline style. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

referenceOutlineThickness

A SymbolicConstant specifying the reference outline thickness. Possible values are VERY_THIN, THIN, MEDIUM, and THICK. The default value is VERY_THIN.

allLabelsFont

A String specifying the font for all the labels. The default value is “*-verdana-medium-r-normal-*-80-*-p-*-” on Windows and “*-verdana-medium-r-normal-*-90-*-p-*-” on Linux platforms. The default value is an empty string.

showMaterialNames

A Boolean specifying whether to show the material names. The default value is OFF.

materialNamesColor

A String specifying the material names text color. The default value is "Green".

showOrientationNames

A Boolean specifying whether to show the orientation names or the fiber angles. The default value is OFF.

orientationNamesColor

A String specifying the orientation names fiber angles text color. The default value is "Orange".

showStateBlock

A Boolean specifying whether to show the state block. The default value is ON.

stateBlockColor

A String specifying the state block text color. The default value is "White".

showPlyNames

A Boolean specifying whether to show the ply names. The default value is ON.

plyNamesColor

A String specifying the ply names text color. The default value is "Yellow".

showThicknessLabels

A Boolean specifying whether to show the thickness labels. The default value is ON.

thicknessLabelsColor

A String specifying the thickness labels text color. The default value is "Red".

showIntPoints

A Boolean specifying whether to show the through thickness integration points. The default value is OFF.

intPointsColor

A String specifying the through thickness integration points color. The default value is "Blue".

sizeX

A Float specifying the size of the ply in the X-direction. The default value is 1.5.

sizeY

A Float specifying the size of the ply in the Y-direction. The default value is 1.0.

sizeZ

A Float specifying the size of the ply in the Z-direction. The default value is 0.8.

Return value

None

Exceptions

None.

40.40.2 Members

The `PlyStackPlotOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

40.41 MdbData object

The MdbData object has no constructor. Abaqus creates an MdbData object when a cae file is opened or a new model is created. There is one MdbData for each model in session. MdbData is updated when it is displayed in a viewport.

Access

```
import visualization
session.mdbData [name]
```

40.41.1 Members

The MdbData object has the following members:

stepPeriods

A tuple of (String, Float) tuples specifying the stepName and the stepPeriod.

steps

A repository of MdbDataStep objects specifying the list of steps. The repository is read-only.

instances

A repository of MdbDataInstance objects specifying the list of instances. The repository is read-only.

40.42 MdbDataFrame object

The MdbDataFrame object. There is one and only MdbDataFrame in a MdbDataStep .

Access

```
import visualization
session.mdbData [name] .steps [i] .frames [i]
```

40.42.1 Members

The MdbDataFrame object has no members.

40.43 MdbDataInstance object

The MdbDataInstance object instance data. It corresponds to same named part instance with a mesh in the cae model.

Access

```
import visualization
session.mdbData[name].instances[i]
```

40.43.1 Members

The MdbDataInstance object has the following member:

name

A String specifying the instance name. This attribute is read-only.

40.44 MdbDataStep object

The MdbDataStep object. It corresponds to same named step in the cae model.

Access

```
import visualization
session.mdbData[name].steps[i]
```

40.44.1 Members

The MdbDataStep object has the following member:

frames

A MdbDataFrameArray object specifying the list of frames. The list is read-only. There is only one frame in a step.

41. Plug-in registration commands

Plug-in commands register kernel and GUI plug-ins in either the **Plug-ins** menu or in a toolbox. Plug-in registration commands for accessing both kernel and GUI plug-ins are located in Chapter 2, “Plug-in registration commands” of the Abaqus GUI Toolkit Reference Guide.

42. Predefined field commands

A `PredefinedField` object stores the non-propagating data of a predefined field as well as a number of instances of the corresponding `PredefinedFieldState` object. The `PredefinedFieldState` object stores the propagating data of the predefined field in a single step. A specific type of `PredefinedField` object and a specific type of `PredefinedFieldState` object are designed for each type of predefined field. Instances of the `PredefinedFieldState` object are created and deleted internally by its corresponding `PredefinedField` object.

42.1 PredefinedField object

The `PredefinedField` object is the base object for the objects in the **predefined field** repository. The methods and members of the `PredefinedField` object are common to all objects derived from `PredefinedField`.

An instance of any `PredefinedField` object can be obtained through the **predefined field** repository of the `Model` object. An instance of any `PredefinedFieldState` object can be obtained through the **predefined field** repository of the `Step` object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.1.1 move(...)

This method moves a specific `PredefinedFieldState` object from one step to a different step.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the `PredefinedFieldState` object is moved.

toStepName

A String specifying the name of the step to which the `PredefinedFieldState` object is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

42.1.2 resume()

This method resumes the predefined field that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

42.1.3 suppress()

This method suppresses the predefined field.

Arguments

None.

Return value

None

Exceptions

None.

42.1.4 delete(...)

This method allows you to delete existing fields.

Arguments

Required argument

indices

A sequence of Ints specifying the index of each field to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

42.1.5 Members

The `PredefinedField` object can have the following members:

name

A String specifying the repository key.

region

A `Region` object specifying the region to which the predefined field is applied. *Region* is ignored if the predefined field has an *instances* member available. *Region* is also ignored if the predefined field has a *distributionType* member available, and *distributionType*=`FROM_FILE` or `FROM_FILE_AND_USER_DEFINED`.

42.2 FluidCavityPressure object

The `FluidCavityPressure` object stores the data for initial fluid cavity pressures. The base class *region* argument can not be specified with this object.

The `FluidCavityPressure` object is derived from the `PredefinedField` object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.2.1 FluidCavityPressure(...)

This method creates a `FluidCavityPressure` object.

Path

```
mdb.models[name].FluidCavityPressure
```

PREDEFINED FIELD COMMANDS

Required arguments

name

A String specifying the repository key.

fluidCavity

A String specifying the name of a Fluid Cavity Interaction.

fluidPressure

A Float specifying the initial fluid pressure.

Optional arguments

None.

Return value

A FluidCavityPressure object.

Exceptions

None.

42.2.2 **setValues(...)**

This method modifies the FluidCavityPressure object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidCavityPressure** method, except for the *name* argument.

Return value

None

Exceptions

None.

42.2.3 **Members**

The FluidCavityPressure object has members with the same names and descriptions as the arguments to the **FluidCavityPressure** method.

In addition, the FluidCavityPressure object can have the following member:

region

A Region object on which the *fluidCavity* interaction is specified.

42.2.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=FLUID PRESSURE

42.3 FluidDensity object

The FluidDensity object stores the data for fluid density predefined fields.

The FluidDensity object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.3.1 FluidDensity(...)

This method creates a FluidDensity object.

Path

```
mdb.models[name].FluidDensity
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the predefined field is created.

region

A Region specifying the domain of the fluid density. Use 'None' to specify the whole model.

Optional argument

density

A Float specifying the fluid density. The default value is 0.0.

Return value

A FluidDensity object.

Exceptions

None.

42.3.2 **setValues(...)**

This method modifies the FluidDensity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidDensity** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

42.3.3 **Members**

The FluidDensity object can have the following members:

name

A String specifying the repository key.

density

A Float specifying the fluid density. The default value is 0.0.

region

A Region object specifying the region to which the predefined field is applied.

42.3.4 **Corresponding analysis keywords**

*INITIAL CONDITIONS, TYPE=DENSITY

42.4 FluidThermalEnergy object

The FluidThermalEnergy object stores the data for fluid thermal energy predefined fields. This predefined field is applicable only when an energy equation has been selected in a flow step.

The FluidThermalEnergy object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.4.1 FluidThermalEnergy(...)

This method creates a FluidThermalEnergy object.

Path

```
mdb.models[name].FluidThermalEnergy
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the predefined field is created.

region

A Region specifying the domain of the fluid thermal energy. Use 'None' to specify the whole model.

Optional arguments

temperature

A Float specifying the fluid temperature. The default value is 0.0.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are MAGNITUDE, ANALYTICAL_FIELD, and DISCRETE_FIELD. The default value is MAGNITUDE.

Return value

A FluidThermalEnergy object.

Exceptions

None.

42.4.2 **setValues(...)**

This method modifies the `FluidThermalEnergy` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidThermalEnergy** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

42.4.3 **Members**

The `FluidThermalEnergy` object can have the following members:

name

A String specifying the repository key.

temperature

A Float specifying the fluid temperature. The default value is 0.0.

distributionType

A `SymbolicConstant` specifying whether the load is uniform. Possible values are `MAGNITUDE`, `ANALYTICAL_FIELD`, and `DISCRETE_FIELD`. The default value is `MAGNITUDE`.

field

A String specifying the name of the `AnalyticalField` object associated with this predefined field. The *field* argument applies only when *distributionType*=`ANALYTICAL_FIELD`. The default value is an empty string.

region

A `Region` object specifying the region to which the predefined field is applied.

42.4.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=TEMPERATURE, ELEMENT AVERAGE

42.5 FluidTurbulence object

The FluidTurbulence object stores the data for fluid turbulence predefined fields. This predefined field is applicable only when a turbulence model has been selected in a flow step.

The FluidTurbulence object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.5.1 FluidTurbulence(...)

This method creates a FluidTurbulence object.

Path

```
mdb.models[name].FluidTurbulence
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the predefined field is created.

region

A Region specifying the domain of the fluid turbulence. Use 'None' to specify the whole model.

Optional arguments

eddyViscosity

A Float specifying the eddy viscosity for the fluid turbulence. This parameter is applicable only when the Spalart-Allmaras turbulence model has been selected in a flow step. The default value is 0.0.

PREDEFINED FIELD COMMANDS

dissipationRate

A Float specifying the dissipation rate for the fluid turbulence. This parameter is applicable only when the k-epsilon renormalization group (RNG) turbulence model has been selected in a flow step. The default value is 0.0.

turbulentKE

A Float specifying the kinetic energy for the fluid turbulence. This parameter is applicable only when the k-epsilon renormalization group (RNG) turbulence model has been selected in a flow step. The default value is 0.0.

Return value

A FluidTurbulence object.

Exceptions

None.

42.5.2 setValues(...)

This method modifies the FluidTurbulence object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidTurbulence** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

42.5.3 Members

The FluidTurbulence object can have the following members:

name

A String specifying the repository key.

eddyViscosity

A Float specifying the eddy viscosity for the fluid turbulence. This parameter is applicable only when the Spalart-Allmaras turbulence model has been selected in a flow step. The default value is 0.0.

dissipationRate

A Float specifying the dissipation rate for the fluid turbulence. This parameter is applicable only when the k-epsilon renormalization group (RNG) turbulence model has been selected in a flow step. The default value is 0.0.

turbulentKE

A Float specifying the kinetic energy for the fluid turbulence. This parameter is applicable only when the k-epsilon renormalization group (RNG) turbulence model has been selected in a flow step. The default value is 0.0.

region

A Region object specifying the region to which the predefined field is applied.

42.5.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=TURBNU, TYPE=TURBKE, TYPE= TURBEPS, ELEMENT AVERAGE

42.6 FluidVelocity object

The FluidVelocity object stores the data for fluid velocity Predefined fields.

The FluidVelocity object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.6.1 FluidVelocity(...)

This method creates a FluidVelocity object.

Path

```
mdb.models[name].FluidVelocity
```

PREDEFINED FIELD COMMANDS

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the predefined field is created.

region

A Region specifying the domain of the fluid velocity. Use 'None' to specify the whole model.

Optional arguments

velocity1

A Float specifying the fluid velocity in the 1 direction. The default value is 0.0.

velocity2

A Float specifying the fluid velocity in the 2 direction. The default value is 0.0.

velocity3

A Float specifying the fluid velocity in the 3 direction. The default value is 0.0.

Return value

A FluidVelocity object.

Exceptions

None.

42.6.2 **setValues(...)**

This method modifies the FluidVelocity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FluidVelocity** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

42.6.3 Members

The FluidVelocity object can have the following members:

name

A String specifying the repository key.

velocity1

A Float specifying the fluid velocity in the 1 direction. The default value is 0.0.

velocity2

A Float specifying the fluid velocity in the 2 direction. The default value is 0.0.

velocity3

A Float specifying the fluid velocity in the 3 direction. The default value is 0.0.

region

A Region object specifying the region to which the predefined field is applied.

42.6.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=VELOCITY, ELEMENT AVERAGE

42.7 IMAField object

A IMAField is an object used to define material instance name volume fractions for the MaterialAssignment predefined field.

Access

```
import load
mdb.models[name].predefinedFields[name].fieldList
```

42.7.1 Members

The IMAField object can have the following members:

region

A Region object specifying the sub-region of the selected part instance to which the volume fractions will be applied.

discFieldList

A tuple of Strings specifying the name of the discrete fields that contain the volume fraction data. The length of the tuple corresponds to the number of material instance names, as established by the assigned Eulerian section.

42.8 IMARegion object

A IMARegion is an object used to define material instance name volume fractions for the MaterialAssignment predefined field.

Access

```
import load
mdb.models[name].predefinedFields[name].assignmentList
```

42.8.1 Members

The IMARegion object can have the following members:

region

A Region object specifying the sub-region of the selected part instance to which the volume fractions will be applied.

fractionList

A tuple of Floats specifying the volume fractions, per material instance name. The length of the tuple corresponds to the number of material instance names, as established by the assigned Eulerian section.

42.9 InitialState object

The InitialState object stores the data for an initial state predefined field.

The InitialState object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.9.1 InitialState(...)

This method creates an InitialState predefined field object.

Path

`mdb.models[name].InitialState`

Required arguments

name

A String specifying the repository key.

instances

A PartInstanceArray object specifying the instances to which the predefined field is applied.

fileName

A String specifying the name of the job that generated the initial state data.

Optional arguments

endStep

The SymbolicConstant LAST_STEP or an Int specifying the step from which the initial state values are to be read or the SymbolicConstant LAST_STEP. The default value is LAST_STEP.

endIncrement

The SymbolicConstant STEP_END or an Int specifying the increment, interval or iteration of the step set in *endStep* or the SymbolicConstant STEP_END. The default value is STEP_END.

updateReferenceConfiguration

A Boolean specifying whether to update the reference configuration based on the import data. The default value is OFF.

Return value

An InitialState object.

Exceptions

None.

42.9.2 setValues(...)

This method modifies the InitialState object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **InitialState** method, except for the *name* argument.

Return value

None

Exceptions

None.

42.9.3 Members

The `InitialState` object has members with the same names and descriptions as the arguments to the `InitialState` method.

42.9.4 Corresponding analysis keywords

*INSTANCE

42.10 KinematicHardening object

The `KinematicHardening` object stores the data for initial equivalent plastic strains and, if relevant, the initial backstress tensor.

The `KinematicHardening` object is derived from the `PredefinedField` object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.10.1 KinematicHardening(...)

This method creates a `KinematicHardening` object.

Path

```
mdb.models[name].KinematicHardening
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the predefined field is applied.

Optional arguments

numBackStress

An Int specifying the number of backstresses. The default value is 1.

equivPlasticStrain

A sequence of Floats specifying the initial equivalent plastic strain.

backStress

A sequence of sequences of Floats specifying the initial backstress tensor for kinematic hardening models. The default value is an empty sequence.

sectPtNum

A sequence of Ints specifying section point numbers. This argument is valid only when *definition*=SECTION_PTS.

definition

A SymbolicConstant specifying different types of kinematic hardening. Possible values are KINEMATIC_HARDENING, CRUSHABLE_FOAM, REBAR, SECTION_PTS, and USER_DEFINED. The default value is KINEMATIC_HARDENING.

rebarLayerNames

A sequence of Strings specifying rebar layer names. This argument is valid only when *definition*=REBAR.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are MAGNITUDE and ANALYTICAL_FIELD. The default value is MAGNITUDE.

Return value

A KinematicHardening object.

Exceptions

None.

42.10.2 setValues(...)

This method modifies the KinematicHardening object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **KinematicHardening** method, except for the *name* argument.

Return value

None

Exceptions

None.

42.10.3 Members

The KinematicHardening object has members with the same names and descriptions as the arguments to the **KinematicHardening** method.

In addition, the KinematicHardening object can have the following member:

field

A String specifying the name of the AnalyticalField object associated with this predefined field. The *field* argument applies only when *distributionType*=ANALYTICAL_FIELD. The default value is an empty string.

42.10.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=HARDENING

42.11 MaterialAssignment object

The MaterialAssignment object stores the data for an initial material assignment predefined field, for use with an Eulerian analysis.

The MaterialAssignment object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.11.1 MaterialAssignment(...)

This method creates a MaterialAssignment predefined field object.

Path

`mdb.models[name].MaterialAssignment`

Required arguments

name

A String specifying the repository key.

instanceList

A PartInstanceArray object specifying the part instances to which the predefined field is applied. All instances must be assigned the same Eulerian section.

Optional arguments

useFields

A Boolean specifying whether the volume fraction data will be uniform or defined by discrete fields. The default value is OFF.

assignmentList

A sequence of tuples specifying the uniform volume fractions to be assigned. This argument is valid only when *useFields*=FALSE. Each tuple contains two entries:

- A Region object.
- A tuple of Floats specifying the uniform volume fraction values. The length of the tuple must match the number of material instance names specified in the Eulerian section assigned to part instances specified by *instanceList*.

fieldList

A sequence of tuples specifying the discrete volume fractions to be assigned. This argument is valid only when *useFields*=TRUE. Each tuple contains two entries:

- A Region object.
- A tuple of Strings specifying Discrete Field names. The length of the tuple must match the number of material instance names specified in the Eulerian section assigned to part instances specified by *instanceList*.

colorList

A sequence of three Ints specifying colors used to display the material instance assignments. This is a sequence of R,G,B colors, where the values are represented by integers between 0 and 255. The default value is an empty sequence.

Return value

A MaterialAssignment object.

Exceptions

None.

42.11.2 setValues(...)

This method modifies the MaterialAssignment object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MaterialAssignment** method, except for the *name* argument.

Return value

None

Exceptions

None.

42.11.3 Members

The MaterialAssignment object has members with the same names and descriptions as the arguments to the **MaterialAssignment** method.

In addition, the MaterialAssignment object can have the following member:

region

A Region object specifying the region to which the predefined field is applied.

42.11.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=EULERIAN MATERIAL

42.12 **PredefinedFieldState object**

The PredefinedFieldState object is the base object for the objects in the **predefinedFieldState** repository of the Step object. The members of the PredefinedFieldState object are common to all objects derived from PredefinedFieldState.

The PredefinedFieldState object has no constructor or methods.

Access

```
import load
mdb.models[name].steps[name].predefinedFieldStates[name]
```

42.12.1 **Members**

The PredefinedFieldState object has the following member:

status

A SymbolicConstant specifying the propagation state of the PredefinedFieldState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- DEACTIVATED_TO_INITIAL
- NO_LONGER_ACTIVE
- RESET_TO_INITIAL
- TO_BE_COMPUTED
- PROPAGATED_FROM_COMPUTED
- BUILT_INTO_BASE_STATE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE

This member exists in all PredefinedFieldState objects, but different predefined fields use different subsets of the entire list of possible values depending on propagation rules.

42.13 Temperature object

The Temperature object stores the data for temperature predefined fields.

The Temperature object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.13.1 Temperature(...)

This method creates a Temperature object.

Path

```
mdb.models[name].Temperature
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the predefined field is created.

region

A Region object specifying the region to which the predefined field is applied. *Region* is ignored if the predefined field has a *distributionType* member available, and *distributionType*=FROM_FILE

Optional arguments

distributionType

A SymbolicConstant specifying how the predefined field varies spatially. Possible values are UNIFORM, USER_DEFINED, FROM_FILE, FIELD, FROM_FILE_AND_USER_DEFINED, and DISCRETE_FIELD. The default value is UNIFORM.

crossSectionDistribution

A SymbolicConstant specifying how the predefined field is distributed over the cross section of the region. Possible values are

- CONSTANT_THROUGH_THICKNESS
- GRADIENTS_THROUGH_SHELL_CS
- GRADIENTS_THROUGH_BEAM_CS
- POINTS_THROUGH_SECTION

The default value is `CONSTANT_THROUGH_THICKNESS`.

field

A String specifying the name of the AnalyticalField or DiscreteField object associated with this predefined field. The *field* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

amplitude

A String or the SymbolicConstant UNSET specifying the name of the amplitude reference. UNSET should be used if the predefined field has no amplitude reference. The default value is UNSET.

Note: *amplitude* should be given only if it is valid for the specified step.

fileName

A String specifying the name of the file from which the temperature values are to be read when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED.

beginStep

An Int specifying the first step from which temperature values are to be read or the SymbolicConstant FIRST_STEP or LAST_STEP. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

beginIncrement

An Int specifying the first increment of the step set in *beginStep* or the SymbolicConstants STEP_START or STEP_END. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

endStep

An Int specifying the last step from which temperature values are to be read or the SymbolicConstants FIRST_STEP and LAST_STEP. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

endIncrement

An Int specifying the last increment of the step set in *endStep* or the SymbolicConstants STEP_START and STEP_END. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

interpolate

A SymbolicConstant specifying whether to interpolate a field read from an output database or results file. Possible values are OFF, ON or MIDSIDE_ONLY. The default value is OFF.

magnitudes

A Sequence of Doubles specifying the temperature values when *distributionType*=UNIFORM or FIELD. The value of the *magnitudes* argument is a function of the *crossSectionDistribution* argument, as shown in the following list:

PREDEFINED FIELD COMMANDS

- If *crossSectionDistribution*=CONSTANT_THROUGH_THICKNESS then *magnitudes* is a Double specifying the temperature.
- If *crossSectionDistribution*=GRADIENTS_THROUGH_SHELL_CS then *magnitudes* is a sequence of Doubles specifying the mean value and the gradient in the thickness direction.
- If *crossSectionDistribution*=GRADIENTS_THROUGH_BEAM_CS then *magnitudes* is a sequence of Doubles specifying the mean value, the gradient in the N1 direction, and the gradient in the N2 direction.
- If *crossSectionDistribution*=POINTS_THROUGH_SECTION then *magnitudes* is a sequence of Doubles specifying the temperature at each point.

absoluteExteriorTolerance

A Float specifying the absolute value by which a driven node of the field can lie outside the region of the elements of the global model. The default value is 0.0. This argument cannot be used with *midside*.

exteriorTolerance

A Float specifying the fraction of the average element size in the global model by which a driven node of the field can lie outside the region of the elements of the global model. The default value is 0.0. This argument cannot be used with *midside*.

Return value

A Temperature object.

Exceptions

None.

42.13.2 move(...)

This method moves the TemperatureState object from one step to a different step.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the PredefinedFieldState is moved.

toStepName

A String specifying the name of the step to which the PredefinedFieldState is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

42.13.3 setValues(...)

This method modifies the data for an existing Temperature object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Temperature** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

42.13.4 setValuesInStep(...)

This method modifies the propagating data for an existing Temperature object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the predefined field is modified.

Optional arguments

None.

The optional arguments to **setValuesInStep** are the same as the optional arguments to the **Temperature** method, except for the *distributionType* and *crossSectionDistribution* arguments.

Return value

None

Exceptions

None.

42.13.5 Members

The Temperature object can have the following members:

name

A String specifying the repository key.

distributionType

A SymbolicConstant specifying how the predefined field varies spatially. Possible values are UNIFORM, USER_DEFINED, FROM_FILE, FIELD, FROM_FILE_AND_USER_DEFINED, and DISCRETE_FIELD. The default value is UNIFORM.

field

A String specifying the name of the AnalyticalField or DiscreteField object associated with this predefined field. The *field* argument applies only when *distributionType*=FIELD or *distributionType*=DISCRETE_FIELD. The default value is an empty string.

region

A Region object specifying the region to which the predefined field is applied. *Region* is ignored if the predefined field has an *instances* member available. *Region* is also ignored if the predefined field has a *distributionType* member available, and *distributionType*=FROM_FILE or FROM_FILE_AND_USER_DEFINED.

42.13.6 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=TEMPERATURE

*TEMPERATURE

42.14 TemperatureState object

The TemperatureState object stores the propagating data of a temperature in a step. One instance of this object is created internally by the Temperature object for each step.

The TemperatureState object has no constructor or methods.

The TemperatureState object is derived from the PredefinedFieldState object.

Access

```
import load
mdb.models[name].steps[name].predefinedFieldStates[name]
```

42.14.1 Members

The TemperatureState object can have the following members:

fileNameState

A SymbolicConstant specifying the propagation state of the *fileName* member. Possible values are UNSET, SET, and UNCHANGED.

beginStep

A SymbolicConstant or an Int specifying the first step from which temperature values are to be read. This argument is valid only when *distribution*=FROM_FILE or *distribution*=FROM_FILE_AND_USER_DEFINED. Possible values are FIRST_STEP, LAST_STEP, and NONE. The default value is NONE.

beginStepState

A SymbolicConstant specifying the propagation state of the *beginStep* member. Possible values are UNSET, SET, and UNCHANGED.

beginIncrement

None or an Int specifying the first increment of the step set in *beginStep* or the SymbolicConstants STEP_START or STEP_END. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

beginIncrementState

A SymbolicConstant specifying the propagation state of the *beginIncrement* member. Possible values are UNSET, SET, and UNCHANGED.

endStep

None or an Int specifying the last step from which temperature values are to be read or the SymbolicConstants FIRST_STEP and LAST_STEP. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

endStepState

A SymbolicConstant specifying the propagation state of the *endStep* member. Possible values are UNSET, SET, and UNCHANGED.

endIncrement

None or an Int specifying the last increment of the step set in *endStep* or the SymbolicConstants STEP_START and STEP_END. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED. The default value is **None**.

PREDEFINED FIELD COMMANDS

endIncrementState

A SymbolicConstant specifying the propagation state of the *endIncrement* member. Possible values are UNSET, SET, and UNCHANGED.

midside

A Boolean specifying that temperatures in second-order elements are to be interpolated from corner node temperatures. This argument is valid only when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED.

midsideState

A SymbolicConstant specifying the propagation state of the *midside* member. Possible values are UNSET, SET, and UNCHANGED.

amplitudeState

A SymbolicConstant specifying the propagation state of the *amplitudeState* member. Possible values are UNSET, SET, and UNCHANGED.

fileName

A String specifying the name of the file from which the temperature values are to be read when *distributionType*=FROM_FILE or *distributionType*=FROM_FILE_AND_USER_DEFINED.

amplitude

The SymbolicConstant UNSET or a String specifying the name of the amplitude reference. The SymbolicConstant UNSET should be used if the predefined field has no amplitude reference. The default value is UNSET.

Note: *amplitude* should be given only if it is valid for the specified step.

magnitudesState

A tuple of SymbolicConstants specifying the propagation state of each item of the *magnitudes* member. Possible values are UNSET, SET, and UNCHANGED.

magnitudes

A tuple of Floats specifying the temperature values when *distributionType*=UNIFORM or *distributionType*=FIELD. The value of the *magnitudes* argument is a function of the *crossSectionDistribution* argument, as shown in the following list:

- If *crossSectionDistribution*=CONSTANT_THROUGH_THICKNESS then *magnitudes* is a Double specifying the temperature.
- If *crossSectionDistribution*=GRADIENTS_THROUGH_SHELL_CS then *magnitudes* is a sequence of Doubles specifying the mean value and the gradient in the thickness direction.
- If *crossSectionDistribution*=GRADIENTS_THROUGH_BEAM_CS then *magnitudes* is a sequence of Doubles specifying the mean value, the gradient in the N1 direction, and the gradient in the N2 direction.
- If *crossSectionDistribution*=POINTS_THROUGH_SECTION then *magnitudes* is a sequence of Doubles specifying the temperature at each point.

status

A SymbolicConstant specifying the propagation state of the PredefinedFieldState object. Possible values are:

- NOT_YET_ACTIVE
- CREATED
- PROPAGATED
- MODIFIED
- DEACTIVATED
- DEACTIVATED_TO_INITIAL
- NO_LONGER_ACTIVE
- RESET_TO_INITIAL
- TO_BE_COMPUTED
- PROPAGATED_FROM_COMPUTED
- BUILT_INTO_BASE_STATE
- TYPE_NOT_APPLICABLE
- INSTANCE_NOT_APPLICABLE

This member exists in all PredefinedFieldState objects, but different predefined fields use different subsets of the entire list of possible values depending on propagation rules.

42.15 Velocity object

The Velocity object stores the data for an initial velocity predefined field.

The Velocity object is derived from the PredefinedField object.

Access

```
import load
mdb.models[name].predefinedFields[name]
```

42.15.1 Velocity(...)

This method creates a Velocity predefined field object.

Path

```
mdb.models[name].Velocity
```

Required arguments

name

A String specifying the repository key.

region

A Region object specifying the region to which the predefined field is applied.

velocity1

A Float specifying the first component of the velocity.

velocity2

A Float specifying the second component of the velocity.

velocity3

A Float specifying the third component of the velocity.

omega

A Float specifying the angular velocity.

axisBegin

A sequence of Floats specifying the X-, Y-, and Z- coordinates of the starting point of the axis about which *omega* is defined.

axisEnd

A sequence of Floats specifying the X-, Y-, and Z- coordinates of the end point of the axis about which *omega* is defined.

Optional arguments

field

A String specifying the name of the AnalyticalField object associated with this predefined field. The *field* argument applies only when *distributionType*=FIELD_ANALYTICAL. The default value is an empty string.

distributionType

A SymbolicConstant specifying whether the load is uniform. Possible values are MAGNITUDE and FIELD_ANALYTICAL. The default value is MAGNITUDE.

Return value

A Velocity object.

Exceptions

None.

42.15.2 setValues(...)

This method modifies the Velocity object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Velocity** method, except for the *name* argument.

Return value

None

Exceptions

None.

42.15.3 Members

The Velocity object has members with the same names and descriptions as the arguments to the **Velocity** method.

42.15.4 Corresponding analysis keywords

*INITIAL CONDITIONS, TYPE=VELOCITY

*INITIAL CONDITIONS, TYPE=ROTATING VELOCITY

43. Print commands

The print commands are used to print selected canvas objects to a file or to a Windows or PostScript printer and to control the format of the printed output.

43.1 PrintOptions object

The PrintOptions object stores the common settings that Abaqus uses for all print methods. The PrintOptions object has no constructor. Abaqus creates the *printOptions* member when a session is started.

Access

`session.printOptions`

43.1.1 setValues(...)

This method modifies the PrintOptions object.

Arguments

Required arguments

None.

Optional arguments

rendition

A SymbolicConstant specifying how the image is rendered. Possible values are BLACK_AND_WHITE, GREYSCALE, and COLOR. The default value is COLOR.

vpDecorations

A Boolean specifying whether the output includes the viewport border and title. The default value is ON.

vpBackground

A Boolean specifying whether the output includes viewport backgrounds. The default value is OFF.

compass

A Boolean specifying whether the output includes the view compass. The default value is OFF.

printCommand

A String specifying the default print command that will be used by the `printToPrinter` method if no *printCommand* argument is provided. The default value is "lpr".

PRINT COMMANDS

deleteTemporaryFiles

A Boolean specifying whether to delete the temporary files created when an image is printed. The default value is ON.

You should set the *deleteTemporaryFiles* argument to False if your printer does not support print spooling.

reduceColors

A Boolean specifying whether the raster image printed is reduced from true color to 256 colors to reduce file size. The quality of the image will be compromised. The default value is OFF.

Return value

None

Exceptions

None.

43.1.2 Members

The PrintOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

43.2 EpsOptions object

The EpsOptions object stores the settings that Abaqus uses when printing using Encapsulated PostScript format. The EpsOptions object has no constructor. Abaqus creates the *epsOptions* member when a session is started.

Access

session.epsOptions

43.2.1 setValues(...)

This method modifies the EpsOptions object.

Arguments

Required arguments

None.

Optional arguments*imageSize*

The SymbolicConstant SIZE_ON_SCREEN or a pair of Floats specifying the width and height of the image in the units specified by *units*. Possible numeric values are *imageSize* $\geq$ (*minWidth*, *minHeight*). The default value is SIZE_ON_SCREEN.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is 10 mm (approximately 0.4 inches).

units

A SymbolicConstant specifying the units of the *imageSize* argument. This argument is ignored if *imageSize* is SIZE_ON_SCREEN. Possible values are INCHES and MM. The default value is INCHES.

resolution

A SymbolicConstant specifying the resolution of the image in dots per inch (dpi). The *resolution* can be DPI_1200 only if *imageFormat*=VECTOR. Possible values are DPI_75, DPI_150, DPI_300, DPI_450, DPI_600, and DPI_1200. The default value is DPI_150.

fontType

A SymbolicConstant specifying the PostScript font substitution rules to be applied. Possible values are PS_ALWAYS, PS_IF_AVAILABLE, and AS_DISPLAYED. The default value is PS_IF_AVAILABLE.

imageFormat

A SymbolicConstant specifying how the viewport display will be represented. Possible values are VECTOR and RASTER. The default value is VECTOR.

shadingQuality

A SymbolicConstant specifying how fine the shading of curved surfaces will be for vector images. Possible values are EXTRA_COARSE, COARSE, MEDIUM, FINE, and EXTRA_FINE. The default value is MEDIUM.

Return value

None

Exceptions

RangeError.

If either element of *imageSize* is out of range:

RangeError: *imageSize* must be SIZE_ON_SCREEN or a sequence of 2 Floats $\geq$ (*minWidth*, *minHeight*)

43.2.2 Members

The EpsOptions object has members with the same names and descriptions as the arguments to the `setValues` method.

43.3 PageSetupOptions object

The PageSetupOptions object stores the settings that Abaqus uses when printing using a Windows printer. The PageSetupOptions object has no constructor. Abaqus creates the *pageSetupOptions* member when a session is started.

Access

`session.pageSetupOptions`

43.3.1 setValues(...)

This method modifies the PageSetupOptions object.

Arguments

Required arguments

None.

Optional arguments

imageSize

A SymbolicConstant or a pair of Floats specifying the size of the printed image in the currently selected units (inches or millimeters). Possible values are FIT_TO_PAGE and SIZE_ON_SCREEN. The default value is FIT_TO_PAGE.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is 10 mm (approximately 0.4 inches).

units

A SymbolicConstant specifying the units to use for the margins and image size. Possible values are INCHES and MM. The default value is INCHES.

quality

A SymbolicConstant specifying the quality of the image. Possible values are COARSE, MEDIUM, and FINE. The default value is MEDIUM.

topMargin

A Float specifying the top margin of the paper in the currently selected units (inches or millimeters). Possible values are *topMargin* ≥ 0 . The default value is 0.5.

bottomMargin

A Float specifying the bottom margin of the paper in the currently selected units (inches or millimeters). Possible values are *bottomMargin* ≥ 0 . The default value is 0.5.

leftMargin

A Float specifying the left margin of the paper in the currently selected units (inches or millimeters). Possible values are *leftMargin* ≥ 0 . The default value is 0.5.

rightMargin

A Float specifying the right margin of the paper in the currently selected units (inches or millimeters). Possible values are *rightMargin* ≥ 0 . The default value is 0.5.

orientation

A SymbolicConstant specifying the orientation of the image. Possible values are PORTRAIT and LANDSCAPE. The default value is PORTRAIT.

logo

A Boolean specifying whether the output includes the Abaqus logo. The default value is ON.

date

A Boolean specifying whether the output includes the date. The default value is ON.

Return value

None

Exceptions

RangeError.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is 10 mm (approximately 0.4 inches).

If *leftMargin* + *rightMargin* is out of range:

RangeError: leftMargin and rightMargin must produce image width $\geq$ minWidth

If *topMargin* + *bottomMargin* is out of range:

RangeError: topMargin and bottomMargin must produce image height $\geq$ minHeight

43.3.2 Members

The PageSetupOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

43.4 PngOptions object

The PngOptions object stores the settings that Abaqus uses when printing in PNG format. The PngOptions object has no constructor. Abaqus creates the *pngOptions* member when a session is started.

Access

`session.pngOptions`

43.4.1 setValues(...)

This method modifies the PngOptions object.

Arguments

Required arguments

None.

Optional argument

imageSize

The SymbolicConstant `SIZE_ON_SCREEN` or a pair of Ints specifying the width and height of the image in pixels. Possible values are $(minWidth, minHeight) \leq imageSize \leq (maxWidth, maxHeight)$. The default value is `SIZE_ON_SCREEN`.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is the number of pixels that occupy 10 mm at the current screen resolution. The value is typically around 50 pixels and may be different for width and height.

The maximum value of width and height (*maxWidth* and *maxHeight*) is the largest number of pixels supported by the system graphics and will be at least as large as the screen dimensions.

Return value

None

Exceptions

RangeError.

If either the width or height arguments of *imageSize* are out of range (where *minWidth* and *minHeight* are the number of pixels corresponding to approximately 10 mm for a given display and *maxWidth* and *maxHeight* are the largest allowable number of pixels supported by the system graphics):

RangeError: imageSize must be SIZE_ON_SCREEN or a sequence of 2 ints in the range (minWidth, minHeight) <= (width, height) <= (maxWidth, maxHeight).

43.4.2 Members

The PngOptions object has members with the same names and descriptions as the arguments to the `setValues` method.

43.5 PsOptions object

The PsOptions object stores the settings that Abaqus uses when printing using PostScript format. The PsOptions object has no constructor. Abaqus creates the *psOptions* member when a session is started.

Access

`session.psOptions`

43.5.1 setValues(...)

This method modifies the PsOptions object.

Arguments

Required arguments

None.

Optional arguments

paperSize

A SymbolicConstant specifying the paper size. Possible values are:

- LETTER
- LEGAL
- LEDGER
- A0
- A1
- A2
- A3
- A4
- A5

The default value is LETTER.

PRINT COMMANDS

topMargin

A Float specifying the top margin of the paper in inches. Possible values are *topMargin* ≥ 0 . The default value is 0.5.

bottomMargin

A Float specifying the bottom margin of the paper in inches. Possible values are *bottomMargin* ≥ 0 . The default value is 0.5.

leftMargin

A Float specifying the left margin of the paper in inches. Possible values are *leftMargin* ≥ 0 . The default value is 0.5.

rightMargin

A Float specifying the right margin of the paper in inches. Possible values are *rightMargin* ≥ 0 . The default value is 0.5.

orientation

A SymbolicConstant specifying the orientation of the image. Possible values are PORTRAIT and LANDSCAPE. The default value is PORTRAIT.

logo

A Boolean specifying whether the output includes the Abaqus logo. The default value is ON.

date

A Boolean specifying whether the output includes the date. The default value is ON.

resolution

A SymbolicConstant specifying the resolution of the image in dots per inch (dpi). The *resolution* can be DPI_1200 only if *imageFormat* = VECTOR. Possible values are DPI_75, DPI_150, DPI_300, DPI_450, DPI_600, and DPI_1200. The default value is DPI_150.

fontType

A SymbolicConstant specifying the PostScript font substitution rules to be applied. Possible values are PS_ALWAYS, PS_IF_AVAILABLE, and AS_DISPLAYED. The default value is PS_IF_AVAILABLE.

imageFormat

A SymbolicConstant specifying how the viewport display will be represented. Possible values are VECTOR and RASTER. The default value is VECTOR.

shadingQuality

A SymbolicConstant specifying how fine the shading of curved surfaces will be for vector images. Possible values are EXTRA_COARSE, COARSE, MEDIUM, FINE, and EXTRA_FINE. The default value is MEDIUM.

Return value

None

Exceptions

RangeError.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is 10 mm (approximately 0.4 inches).

If *leftMargin* + *rightMargin* is out of range:

RangeError: *leftMargin* and *rightMargin* must produce image width $\geq$ *minWidth*

If *topMargin* + *bottomMargin* is out of range:

RangeError: *topMargin* and *bottomMargin* must produce image height $\geq$ *minHeight*

43.5.2 Members

The PsOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

43.6 SvgOptions object

The SvgOptions object stores the settings that Abaqus uses when printing in SVG format. The SvgOptions object has no constructor. Abaqus creates the *svgOptions* member when a session is started.

Access

`session.svgOptions`

43.6.1 setValues(...)

This method modifies the SvgOptions object.

Arguments**Required arguments**

None.

Optional argument

imageSize

The SymbolicConstant SIZE_ON_SCREEN or a pair of Ints specifying the width and height of the image in pixels. Possible values are $(minWidth, minHeight) \leq imageSize \leq (maxWidth, maxHeight)$. The default value is SIZE_ON_SCREEN.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is the number of pixels that occupy 10 mm at the current screen resolution. The value is typically around 50 pixels and may be different for width and height.

The maximum value of width and height (*maxWidth* and *maxHeight*) is the largest number of pixels supported by the system graphics and will be at least as large as the screen dimensions.

Return value

None

Exceptions

RangeError.

If either the width or height arguments of *imageSize* are out of range (where *minWidth* and *minHeight* are the number of pixels corresponding to approximately 10 mm for a given display and *maxWidth* and *maxHeight* are the largest allowable number of pixels supported by the system graphics):

RangeError: *imageSize* must be `SIZE_ON_SCREEN` or a sequence of 2 ints in the range $(minWidth, minHeight) \leq (width, height) \leq (maxWidth, maxHeight)$.

43.6.2 Members

The `SvgOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

43.7 TiffOptions object

The `TiffOptions` object stores the settings that Abaqus uses when printing in TIFF format. The `TiffOptions` object has no constructor. Abaqus creates the *tiffOptions* member when a session is started.

Access

`session.tiffOptions`

43.7.1 setValues(...)

This method modifies the `TiffOptions` object.

Arguments

Required arguments

None.

Optional argument*imageSize*

The SymbolicConstant `SIZE_ON_SCREEN` or a pair of Ints specifying the width and height of the image in pixels. Possible values are $(minWidth, minHeight) \leq imageSize \leq (maxWidth, maxHeight)$. The default value is `SIZE_ON_SCREEN`.

Note: The minimum value of width and height (*minWidth* and *minHeight*) is the number of pixels that occupy 10 mm at the current screen resolution. The value is typically around 50 pixels and may be different for width and height.

The maximum value of width and height (*maxWidth* and *maxHeight*) is the largest number of pixels supported by the system graphics and will be at least as large as the screen dimensions.

Return value**None****Exceptions**

RangeError.

If either the width or height arguments of *imageSize* are out of range (where *minWidth* and *minHeight* are the number of pixels corresponding to approximately 10 mm for a given display and *maxWidth* and *maxHeight* are the largest allowable number of pixels supported by the system graphics):

RangeError: *imageSize* must be `SIZE_ON_SCREEN` or a sequence of 2 Ints in the range $(minWidth, minHeight) \leq (width, height) \leq (maxWidth, maxHeight)$.

43.7.2 Members

The `TiffOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

44. Property commands

The Property commands are used to create and manage reinforcements and to assign properties to a part (see also Chapter 44, “Property commands,” and Chapter 46, “Section commands”). The Property commands are methods of a Part object.

44.1 SectionAssignment object

The SectionAssignment object is used to specify a section assignment on an assembly, part or part instance. Section assignments on the assembly are limited to connector elements only.

Access

```
import section
mdb.models[name].parts[name].sectionAssignments[i]
import assembly
mdb.models[name].rootAssembly.sectionAssignments[i]
import odbAccess
session.odbs[name].parts[name].sectionAssignments[i]
session.odbs[name].rootAssembly.instances[name].sectionAssignments[i]
session.odbs[name].rootAssembly.sectionAssignments[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
    .instance.sectionAssignments[i]
```

44.1.1 SectionAssignment(...)

This method creates a SectionAssignment object.

Path

```
mdb.models[name].parts[name].SectionAssignment
mdb.models[name].rootAssembly.SectionAssignment
```

Required arguments

region

A Set object specifying the region to which the section is assigned.

sectionName

A String specifying the name of the section.

PROPERTY COMMANDS

Optional arguments

thicknessAssignment

A SymbolicConstant specifying section thickness assignment method. Possible values are FROM_SECTION and FROM_GEOMETRY. The default value is FROM_SECTION.

offset

A Float specifying the offset of the shell section. The default value is 0.0.

offsetType

A SymbolicConstant specifying the method used to define the shell offset. If *offsetType* is set to OFFSET_FIELD the *offsetField* must have a value. Possible values are SINGLE_VALUE, MIDDLE_SURFACE, TOP_SURFACE, BOTTOM_SURFACE, FROM_GEOMETRY, and OFFSET_FIELD. The default value is SINGLE_VALUE.

offsetField

A String specifying the name of the field specifying the offset. The default value is "".

Return value

A SectionAssignment object.

Exceptions

None.

44.1.2 resume()

This method resumes the section assignment that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

44.1.3 suppress()

This method suppresses the section assignment.

Arguments

None.

Return value

None

Exceptions

None.

44.1.4 getVertices(...)

This method is only valid for connector section assignments. This method returns a sequence consisting of tuples of coordinates of the connector's endpoints.

Arguments

None.

Return value

A sequence of tuples of floats.

Exceptions

None.

An exception is thrown if `getVertices()` is called on any section assignment except connector section assignment.

This method is valid only for connector section assignments.

44.1.5 setValues(...)

This method modifies the `SectionAssignment` object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `SectionAssignment` method.

Return value

None

Exceptions

None.

44.1.6 Members

The SectionAssignment object has members with the same names and descriptions as the arguments to the **SectionAssignment** method.

In addition, the SectionAssignment object has the following member:

suppressed

A Boolean specifying whether the section assignment is suppressed or not. The default value is OFF.

44.2 CompositeLayup object

The CompositeLayup object is used to specify a composite layup on a part.

Access

```
import part
mdb.models[name].parts[name].compositeLayups[i]
```

44.2.1 CompositeLayup(...)

This method creates a CompositeLayup object.

Path

```
mdb.models[name].parts[name].CompositeLayup
```

Required argument

name

A String specifying the repository key.

Optional arguments

description

A String specifying a description of the composite layup.

offsetType

A SymbolicConstant specifying the method used to define the shell offset. If *offsetType*=OFFSET_FIELD the *offsetField* argument is required. This member is valid only if *elementType*=SHELL. Possible values are SINGLE_VALUE, MIDDLE_SURFACE, TOP_SURFACE, BOTTOM_SURFACE, OFFSET_FIELD, and GLOBAL. The default value is GLOBAL.

offsetField

A String specifying The name of the field specifying the offset. This member is valid only if *elementType*=SHELL. The default value is an empty string.

offsetValues

A Float specifying The offset of the shell section. This member is valid only if *elementType*=SHELL. The default value is 0.0.

elementType

A SymbolicConstant specifying the type of element in the composite layup. Possible values are SHELL, CONTINUUM_SHELL, and SOLID. The default value is SHELL.

symmetric

A Boolean specifying whether or not the layup should be made symmetric by the analysis. The default value is OFF.

Return value

A CompositeLayup object.

Exceptions

AbaqusException.

44.2.2 suppress()

This method suppresses a composite layup.

Arguments

None.

Return value

None

Exceptions

None.

44.2.3 **resume()**

This method resumes a composite layup that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

44.2.4 **deletePlies()**

This method deletes all of the plies from a composite layup.

Arguments

None.

Return value

None

Exceptions

None.

44.2.5 **setValues(...)**

This method modifies the `CompositeLayup` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CompositeLayup** method, except for the *name* argument.

Return value

None

Exceptions

None.

44.2.6 Members

The CompositeLayup object has members with the same names and descriptions as the arguments to the **CompositeLayup** method.

In addition, the CompositeLayup object has the following members:

section

A GeometryShellSection object.

orientation

A MaterialOrientation object.

plies

A CompositePlyArray object specifying the plies that make up this composite layup.

44.2.7 Corresponding analysis keywords

*SHELL SECTION

*SHELL GENERAL SECTION

*SOLID SECTION

44.3 CompositePly object

The CompositePly object defines the material layers in a composite layup.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].plies[i]
```

44.3.1 CompositePly(...)

This method creates a CompositePly object.

Path

`mdb.models[name].parts[name].compositeLayups[name].CompositePly`

Required arguments

thickness

A Float specifying the thickness of the section layer.

region

A Region object specifying the region to which the composite ply applies.

material

A String specifying the name of the material for the ply.

plyName

A String specifying the ply identifier for this section layer. The default value is an empty string.

orientationType

A SymbolicConstant specifying the method used to define the relative orientation. If *orientationType*=SPECIFY_ORIENT the *orientationValue* argument is required. If *orientationType*=CSYS the *orientation* argument is required. Possible values are CSYS, SPECIFY_ORIENT, ANGLE_0, ANGLE_45, ANGLE_90, and ANGLE_NEG45. The default value is ANGLE_0.

thicknessType

A SymbolicConstant specifying the method used to define the thickness. If *thicknessType*=SPECIFY_THICKNESS, the *thickness* argument is required. Possible values are SPECIFY_THICKNESS and FIELD_THICKNESS. The default value is SPECIFY_THICKNESS.

Optional arguments

orientationValue

A Float specifying the relative orientation of the section layer. The default value is 0.0.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=ANALYTICAL_FIELD or *thicknessType*=DISCRETE_FIELD. The default value is an empty string.

numIntPts

An Int specifying the number of integration points to be used through the section layer. This argument is valid only if *preIntegrate*=OFF. The default value is 3.

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. The

axis argument applies only if a valid reference is provided for the *orientation*. Possible values are `AXIS_1`, `AXIS_2`, and `AXIS_3`. The default value is `AXIS_1`.

angle

A Float specifying the angle of the additional rotation. The *angle* argument applies only if a valid reference is provided for the *orientation*. The default value is 0.0.

additionalRotationType

A SymbolicConstant specifying the method used to describe the additional rotation when a valid orientation is specified. Use *orientationType*=`ANGLE_0` and *additionalRotationType*=`ROTATION_FIELD` to specify a discrete field of rotations for this CompositePly. Possible values are `ROTATION_NONE`, `ROTATION_ANGLE`, and `ROTATION_FIELD`. The default value is `ROTATION_NONE`.

orientation

The SymbolicConstant None or a DatumCsys object specifying a coordinate system reference for the relative orientation of this layer. The default value is None.

additionalRotationField

A String specifying the name of the field specifying the additional rotation. The default value is an empty string.

Return value

A CompositePly object.

Exceptions

AbaqusException.

44.3.2 Members

The CompositePly object has members with the same names and descriptions as the arguments to the `CompositePly` method.

44.4 MaterialOrientation object

The MaterialOrientation object represents the orientation of the material properties and composite layups.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].orientation
mdb.models[name].parts[name].materialOrientations[i]
import odbAccess
```

```
session.odbs[name].parts[name].materialOrientations[i]
session.odbs[name].rootAssembly.instances[name] \
    .materialOrientations[i]
session.odbs[name].steps[name].frames[i].fieldOutputs[name].values[i] \
    .instance.materialOrientations[i]
```

44.4.1 MaterialOrientation(...)

This method creates a MaterialOrientation object.

Path

```
mdb.models[name].parts[name].MaterialOrientation
```

Required arguments

None.

Optional arguments

region

A Set object specifying a region for which the material orientation is defined.

localCsys

A DatumCsys object specifying the local coordinate system or **None**, describing the material orientation for the given region. In the ODB, this member was previously accessible using "csys," but support has now been added for localCsys and the csys member will be deprecated.

axis

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation (if accessed from the ODB instead of the MDB, it will be a string instead of a float). The default value is 0.0.

stackDirection

A SymbolicConstant specifying the stack or thickness direction. Possible values are STACK_1, STACK_2, STACK_3, and STACK_ORIENTATION. The default value is STACK_3.

fieldName

A String specifying the name of the DiscreteField object specifying the orientation. The default value is an empty string.

orientationType

A SymbolicConstant specifying the method used to define the material orientation. If *orientationType*=SYSTEM, the *region* and *localCsys* arguments are required. If

orientationType=FIELD, the *fieldName* argument is required. Possible values are GLOBAL, SYSTEM, FIELD, DISCRETE, and USER. The default value is GLOBAL.

normalAxisDirection

A SymbolicConstant specifying the axis that is defined by the normal axis direction for a discrete orientation. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_3.

normalAxisDefinition

A SymbolicConstant specifying the method used to define the normal axis direction for a discrete orientation. Possible values are SURFACE, NORMAL_DATUM, and NORMAL_VECTOR. The default value is NORMAL_VECTOR.

normalAxisRegion

A Surface object specifying a region whose geometric normals define the normal axis for the discrete orientation.

normalAxisDatum

A DatumAxis object specifying the Datum Axis or **None**, describing the normal axis direction for the discrete orientation.

flipNormalDirection

A Boolean specifying the flag to reverse the direction of the defined normal axis direction. The default value is OFF.

normalAxisVector

A sequence of Floats specifying the vector that defines the direction of the normal axis of the discrete orientation.

primaryAxisDirection

A SymbolicConstant specifying the axis that is defined by the primary axis direction for a discrete orientation. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

primaryAxisDefinition

A SymbolicConstant specifying the method used to define the primary axis direction for a discrete orientation. Possible values are SURFACE, PRIMARY_DATUM, and PRIMARY_VECTOR. The default value is PRIMARY_VECTOR.

primaryAxisRegion

A Set object specifying a region whose geometric tangents define the primary axis for the discrete orientation.

primaryAxisDatum

A DatumAxis object specifying the Datum Axis or **None**, describing the primary axis direction for the discrete orientation.

flipPrimaryDirection

A Boolean specifying the flag to reverse the direction of the defined primary axis direction. The default value is OFF.

primaryAxisVector

A sequence of Floats specifying the vector that defines the direction of the primary axis of the discrete orientation.

Return value

A MaterialOrientation object.

Exceptions

None.

44.4.2 ReferenceOrientation(...)

This method creates a MaterialOrientation object.

Path

`mdb.models[name].parts[name].compositeLayups[name].ReferenceOrientation`

Required arguments

None.

Optional arguments

localCsys

A DatumCsys object specifying the local coordinate system or **None**, describing the material orientation for the given region. In the ODB, this member was previously accessible using "csys," but support has now been added for localCsys and the csys member will be deprecated.

axis

A SymbolicConstant specifying the axis of a datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation (if accessed from the ODB instead of the MDB, it will be a string instead of a float). The default value is 0.0.

stackDirection

A SymbolicConstant specifying the stack or thickness direction. Possible values are STACK_1, STACK_2, STACK_3, and STACK_ORIENTATION. The default value is STACK_3.

fieldName

A String specifying the name of the DiscreteField object specifying the orientation. The default value is an empty string.

orientationType

A SymbolicConstant specifying the method used to define the material orientation. If *orientationType*=SYSTEM, the *region* and *localCsys* arguments are required. If *orientationType*=FIELD, the *fieldName* argument is required. Possible values are GLOBAL, SYSTEM, FIELD, DISCRETE, and USER. The default value is GLOBAL.

additionalRotationField

A String specifying the name of the DiscreteField object specifying the additional rotation. The default value is an empty string.

additionalRotationType

A SymbolicConstant specifying the method used to describe the additional rotation when a valid orientation is specified. Possible values are ROTATION_NONE, ROTATION_ANGLE, and ROTATION_FIELD. The default value is ROTATION_NONE.

normalAxisDirection

A SymbolicConstant specifying the axis that is defined by the normal axis direction for a discrete orientation. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_3.

normalAxisDefinition

A SymbolicConstant specifying the method used to define the normal axis direction for a discrete orientation. Possible values are SURFACE, DATUM, and VECTOR. The default value is VECTOR.

normalAxisRegion

A Surface object specifying a region whose geometric normals define the normal axis for the discrete orientation.

normalAxisDatum

A DatumAxis object specifying the Datum Axis or **None**, describing the normal axis direction for the discrete orientation.

flipNormalDirection

A Boolean specifying the flag to reverse the direction of the defined normal axis direction. The default value is OFF.

normalAxisVector

A sequence of Floats specifying the vector that defines the direction of the normal axis of the discrete orientation.

primaryAxisDirection

A SymbolicConstant specifying the axis that is defined by the primary axis direction for a discrete orientation. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

primaryAxisDefinition

A SymbolicConstant specifying the method used to define the primary axis direction for a discrete orientation. Possible values are EDGE, DATUM, and VECTOR. The default value is VECTOR.

PROPERTY COMMANDS

primaryAxisRegion

A Set object specifying a region whose geometric tangents define the primary axis for the discrete orientation.

primaryAxisDatum

A DatumAxis object specifying the Datum Axis or **None**, describing the primary axis direction for the discrete orientation.

flipPrimaryDirection

A Boolean specifying the flag to reverse the direction of the defined primary axis direction. The default value is OFF.

primaryAxisVector

A sequence of Floats specifying the vector that defines the direction of the primary axis of the discrete orientation.

Return value

A MaterialOrientation object.

Exceptions

None.

44.4.3 setValues(...)

This method modifies the MaterialOrientation object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MaterialOrientation** method.

Return value

None

Exceptions

None.

44.4.4 Members

The MaterialOrientation object has members with the same names and descriptions as the arguments to the **MaterialOrientation** method.

In addition, the MaterialOrientation object can have the following members:

additionalRotationType

A SymbolicConstant specifying the method used to describe the additional rotation when a valid orientation is specified. Possible values are ROTATION_NONE, ROTATION_ANGLE, and ROTATION_FIELD. The default value is ROTATION_NONE.

additionalRotationField

A String specifying the name of the DiscreteField object specifying the additional rotation. The default value is an empty string.

44.5 PlyStackPlot object

The PlyStackPlot object is used to plot the stacking of plies in a composite layup or in a composite shell section.

Access

```
import section
import visualization
```

44.5.1 MdbPlyStackPlot(...)

This method creates a PlyStackPlot object from a region of a part that contains a composite shell layup.

Path

```
section.MdbPlyStackPlot
```

Required arguments

part

A Part object.

region

A Region object which contains a composite shell layup.

Optional arguments

None.

Return value

A PlyStackPlot object.

Exceptions

None.

44.5.2 OdbPlyStackPlot(...)

This method creates a PlyStackPlot object from a composite shell section of an Odb object.

Path

`visualization.OdbPlyStackPlot`

Required arguments

odb

An Odb object.

sectionName

A String specifying the section name that contains a composite shell section.

Optional argument

offset

A Float specifying the shell offset. The default value is 0.0.

Return value

A PlyStackPlot object.

Exceptions

None.

44.5.3 Members

The PlyStackPlot object has no members.

44.6 Property assignment commands

The Property assignment commands are used to assign and unassign properties to parts. The **part** and **section** modules must be imported to give access to the Property assignment commands.

Access

```
import part
import section
```

44.6.1 assignBeamSectionOrientation(...)

This method assigns a beam section orientation to a region of a part.

Path

```
mdb.models[name].parts[name].assignBeamSectionOrientation
```

Arguments**Required arguments***region*

A sequence of geomSequences of Edge objects or a sequence of sequences of one-dimensional elements.

method

A SymbolicConstant specifying the assignment method. Only a value of N1_COSINES is currently supported.

n1

A sequence of three Floats specifying the approximate local $\mathbf{n}_1$ -direction of the beam cross-section.

Optional arguments

None.

Return value

None

Exceptions

None.

44.6.2 assignMaterialOrientation(...)

This method assigns a material orientation to a region.

Path

```
mdb.models[name].parts[name].assignMaterialOrientation
```

Arguments

Required argument

region

A sequence of geomSequences of Vertex, Edge, Face, and Cell objects or a sequence of sequences of elements.

localCsys

A Datum object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

44.6.3 assignRebarOrientation(...)

This method assigns a rebar reference orientation to a region.

Path

`mdb.models [name] .parts [name] .assignRebarOrientation`

Arguments

Required argument

region

A sequence of geomSequences of Vertex, Edge, Face, and Cell objects or a sequence of sequences of elements.

localCsys

A Datum object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments*axis*

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

44.6.4 flipNormal(...)

This method flips the normals of shell or membrane elements of an orphan mesh or of two-dimensional geometric regions.

Path

```
mdb.models[name].parts[name].flipNormal
```

Arguments**Required argument***regions*

A Region object specifying the region on which normals are flipped. For 3D parts, the region contains Face objects or two-dimensional triangle or quadrilateral Element objects. For axisymmetric parts, the region contains Edge objects or line Elements objects.

Optional argument*referenceRegion*

A two-dimensional element object whose normal is to be matched. If unspecified, all the normals associated with the given regions will be flipped. The *referenceRegion* argument is applicable only if the argument regions contain a sequence of quadrilateral or triangular elements.

Return value

None

Exceptions

None.

44.6.5 flipTangent(...)

This method flips the tangents of beam or truss elements of an orphan mesh or of one-dimensional geometric regions.

Path

```
mdb.models[name].parts[name].flipTangent
```

Arguments

Required argument

regions

A Region object specifying the region on which normals are flipped. The region contains Edge objects or one-dimensional Element objects.

Optional arguments

None.

Return value

None

Exceptions

None.

44.6.6 unassignBeamSectionOrientation(...)

This method deletes a beam section orientation assignment.

Path

```
mdb.models[name].parts[name].unassignBeamSectionOrientation
```

Arguments

Required argument

index

An Int specifying the number of the beam section orientation assignment to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

44.6.7 unassignMaterialOrientation(...)

This method deletes a material orientation assignment.

Path

```
mdb.models[name].parts[name].unassignMaterialOrientation
```

Arguments**Required argument**

index

An Int specifying the number of the material assignment to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

44.6.8 unassignRebarOrientation(...)

This method deletes a rebar orientation assignment.

Path

```
mdb.models[name].parts[name].unassignRebarOrientation
```

PROPERTY COMMANDS

Arguments

Required argument

index

An Int specifying the number of the rebar reference orientation assignment to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

45. Region commands

Region commands are used to create part and assembly sets or surfaces from elements, nodes, and geometry. For more information, see “Specifying a region,” Section 6.6 of the Abaqus Scripting User’s Guide. Part and assembly objects have the following member, a repository of Set objects:

- `sets`

In turn, a Set object can contain any one of the following types:

- `elements`
- `nodes`
- `geometry`

A Set object can contain a number of entities of a single type (nodes, elements, or geometry) or a combination of node and element types. However, except for nodes and elements, a Set object cannot contain a combination of types.

The following are members of the Set object:

- `nodes`
- `elements`
- `cells`
- `faces`
- `edges`
- `vertices`
- `referencePoints`

Region commands are also used to create surfaces on the assembly. Surfaces are sets with additional “sidedness” information.

Part sets contain regions of a part. You can assign section definitions to a set created by selecting a region of a part. The part sets can be accessed from the instance; however, the section definition you assigned to a region is copied automatically to all instances of that part in the assembly.

Assembly sets contain regions of an assembly and are used by many commands that operate on the assembly. For example, you can apply a load or boundary condition to a set created by selecting a region of the assembly. Sets can include regions from multiple part instances.

45.1 Assembly object

The following commands operate on Assembly objects. For more information about the Assembly object, see “Assembly object,” Section 6.1.

Access

```
import regionToolset
```

45.1.1 clashSets(...)

This command prints a message describing the relationship between the contents of two sets. Possible outcomes are:

- Both sets are the same.
- Set 2 is a subset of set 1.
- Set 2 is a superset of set 1.
- Set 2 intersects set 1.
- Set 2 touches set 1 (their boundaries intersect).
- Set 2 and set 1 are disjoint.

This command accepts only positional arguments and has no keywords.

Arguments

Required arguments

arg1

A Set or Surface object specifying set 1.

arg2

A Set or Surface object specifying set 2.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.2 deleteSets(...)

This command deletes the given sets from the assembly.

Arguments**Required argument***setNames*

A sequence of Strings specifying the set names that will be deleted from the assembly.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.3 markSetInternal(...)

This command marks the given Set as internal or external.

Arguments**Required arguments***setName*

A string specifying the Set name.

internalSet

A Boolean specifying whether the Set should be marked as internal.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.4 markSurfaceInternal(...)

This command marks the given Surface as internal or external.

REGION COMMANDS

Arguments

Required arguments

setName

A string specifying the Surface name.

internalSurface

A Boolean specifying whether the Surface should be marked as internal.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.5 isSetInternal(...)

This command returns a flag indicating whether the Set is Internal.

Arguments

Required argument

setName

A string specifying the Set name.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.6 isSurfaceInternal(...)

This command returns a flag indicating whether the Surface is Internal.

Arguments**Required argument***surfaceName*

A string specifying the Surface name.

Optional arguments

None.

Return value

None

Exceptions

None.

45.1.7 deleteSurfaces(...)

This command deletes the given surfaces from the assembly.

Arguments**Required argument***surfaceNames*

A sequence of Strings specifying the surface names that will be deleted from the assembly.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2 Part object

The following commands operate on Part objects. For more information about the Part object, see “Part object,” Section 37.1.

Access

```
import regionToolset
```

45.2.1 clashSets(...)

This command prints a message describing the relationship between the contents of two sets. Possible outcomes are:

- Both sets are the same.
- Set 2 is a subset of set 1.
- Set 2 is a superset of set 1.
- Set 2 intersects set 1.
- Set 2 touches set 1 (their boundaries intersect).
- Set 2 and set 1 are disjoint.

This command accepts only positional arguments and has no keywords.

Arguments

Required arguments

arg1

A Set or Surface object specifying set 1.

arg2

A Set or Surface object specifying set 2.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.2 markSetInternal(...)

This command marks the given Set as internal or external.

Arguments**Required arguments**

setName

A string specifying the Set name.

internalSet

A Boolean specifying whether the Set should be marked as internal.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.3 markSurfaceInternal(...)

This command marks the given Surface as internal or external.

Arguments**Required arguments**

setName

A string specifying the Surface name.

internalSurface

A Boolean specifying whether the Surface should be marked as internal.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.4 isSetInternal(...)

This command returns a flag indicating whether the Set is Internal.

REGION COMMANDS

Arguments

Required argument

setName

A string specifying the Set name.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.5 isSurfaceInternal(...)

This command returns a flag indicating whether the Surface is Internal.

Arguments

Required argument

surfaceName

A string specifying the Surface name.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.6 deleteSets(...)

This command deletes the given sets from the part.

Arguments**Required argument***setNames*

A sequence of Strings specifying the set names that will be deleted from the part.

Optional arguments

None.

Return value

None

Exceptions

None.

45.2.7 deleteSurfaces(...)

This command deletes the given surfaces from the part.

Arguments**Required argument***surfaceNames*

A sequence of Strings specifying the surface names that will be deleted from the part.

Optional arguments

None.

Return value

None

Exceptions

None.

45.3 Region object

The purpose of the Region object is to provide a link between an attribute and the geometric or discrete entities to which the attribute is applied. An attribute (Load, BC, IC, etc.) is applied to one or more Region objects; each Region object in turn is associated with a picked Set or Surface or with a named Set

REGION COMMANDS

or Surface. The Region object provides a unified interface (or "façade") to data and functionality located at the Set or Surface.

A script that applies an attribute to a picked Set or Surface requires the explicit creation of a Region object and will implicitly create a matching internal Set or Surface. Conversely, a script that applies an attribute to a named Set or Surface requires the explicit creation of that Set or Surface (see 39.4) and will implicitly create a region object.

The reference between Region and Set/Surface is by name (user-provided or internal.) If the Set/Surface is suppressed, deleted, or renamed, the Region object will not be able to find the associated Set/Surface, and will communicate that to the attribute, but will not be affected otherwise. If a Set/Surface with the same name becomes available (only possible with user-provided names) the Region object will re-establish the connection. Another way of correcting this problem is to re-apply the attribute.

Wherever a particular Load, BC, IC, etc. command accepts a named set or a named surface, that command will also accept a Region object. For example,

```
myRegion = regionToolset.Region(edges=edges1)
mdb.models['Model-1'].DisplacementBC(name='BC-1',
    createStepName='Initial', region=myRegion, u1=SET,
    u2=SET)
myRegion = regionToolset.Region(elements=e[1:100])
p = mdb.models['mirror'].parts['COLLAR_MIRROR-1']
p.SectionAssignment(region=myRegion, sectionName='Section-1')
```

Abaqus does not provide a **regions** repository; as an alternative, you should assign a variable to a Region object and refer to the variable. The lifecycle of a Region object is similar to the lifecycle of a Leaf object used by display groups; as a result, you should use a Region object immediately after you create it. The contents of a Region object are not intended to survive regeneration. If you use an out-of-date Region object, the script is unlikely to function correctly.

Querying an attribute's Region will return a Region tuple. The contents of the tuple are the set name followed by the owners of the set and three flags expressed as integers. The flags indicate the region space, the type of region and the whether the region is an internal set.

Access

```
import regionToolset
```

45.3.1 Region(...)

This command creates a set-like region. For example,

```
myRegion = regionToolset.Region(vertices=v[2:4],
    edges=e[4:5]+e[6:9])
```

The arguments are the same as the arguments to the **Set** method, except for the *name* argument.

In most cases, the constructor will be called with only one argument of sequences. The arguments *xVertices*, *xEdges*, and *xFaces* are used to exclude lower-dimension entities and to provide finer control on the content of the region. For example, the following statement defines a region enclosing a square face but without two of its edges:

```
region = regionToolset.Region(faces=f[3:4], xEdges=e[1:3])
```

Path

Region

Required arguments

None.

Optional arguments

elements

A sequence of Element objects. The default value is **None**.

nodes

A sequence of Node objects. The default value is **None**.

vertices

A sequence of Vertex objects. The default value is **None**.

edges

A sequence of Edge objects. The default value is **None**.

faces

A sequence of Face objects. The default value is **None**.

cells

A sequence of Cell objects. The default value is **None**.

referencePoints

A sequence of ReferencePoint objects. The default value is an empty sequence.

xVertices

A sequence of Vertex objects that excludes specific vertices from the region. The default value is **None**.

xEdges

A sequence of Vertex objects that excludes specific edges from the region. The default value is **None**.

xFaces

A sequence of Vertex objects that excludes specific faces from the region. The default value is **None**.

skinFaces

A tuple of tuples specifying a skin name and the sequence of faces associated with this skin. Valid only for geometric regions on 3D and 2D parts.

REGION COMMANDS

skinEdges

A tuple of tuples specifying a skin name and the sequence of edges associated with this skin. Valid only for geometric regions on Axisymmetric parts.

stringerEdges

A tuple of tuples specifying a stringer name and the sequence of edges associated with this stringer. Valid only for geometric regions on 3D and 2D parts.

Return value

A Region object.

Exceptions

None.

45.3.2 **Region(...)**

This command creates a surface-like region. For example,

```
myRegion = regionToolset.Region(side1Faces=f[12:14])
```

The arguments are the same as the arguments to the **Surface** method, except for the *name* argument.

Path

Region

Required arguments

None.

Optional arguments

In most cases, you will provide only one argument of sequences.

side1Faces

A sequence of Face objects (surface applies to SIDE1 of face). The default value is **None**.

side2Faces

A sequence of Face objects (surface applies to SIDE2 of face). The default value is **None**.

side12Faces

A sequence of Face objects (surface applies to both SIDE1 and SIDE2 of face). The default value is **None**.

side1Edges

A sequence of Edge objects (surface applies to SIDE1 of edge). The default value is **None**.

side2Edges

A sequence of Edge objects (surface applies to SIDE2 of edge). The default value is **None**.

end1Edges

A sequence of Edge objects (surface applies to END1 of edge). The default value is **None**.

end2Edges

A sequence of Edge objects (surface applies to END2 of edge). The default value is **None**.

circumEdges

A sequence of Edge objects (surface applies circumferentially to edge). The default value is **None**.

face1Elements

A sequence of Element objects (surface applies to FACE1 of element). The default value is **None**.

face2Elements

A sequence of Element objects (surface applies to FACE2 of element). The default value is **None**.

face3Elements

A sequence of Element objects (surface applies to FACE3 of element). The default value is **None**.

face4Elements

A sequence of Element objects (surface applies to FACE4 of element). The default value is **None**.

face5Elements

A sequence of Element objects (surface applies to FACE5 of element). The default value is **None**.

face6Elements

A sequence of Element objects (surface applies to FACE6 of element). The default value is **None**.

side1Elements

A sequence of Element objects (surface applies to SIDE1 of element). The default value is **None**.

side2Elements

A sequence of Element objects (surface applies to SIDE2 of element). The default value is **None**.

side12Elements

A sequence of Element objects (surface applies to both SIDE1 and SIDE2 of element). The default value is **None**.

end1Elements

A sequence of Element objects (surface applies to END1 of element). The default value is **None**.

end2Elements

A sequence of Element objects (surface applies to END2 of element). The default value is **None**.

circumElements

A sequence of Element objects (surface applies to circumference of element). The default value is **None**.

Return value

A Region object.

Exceptions

None.

45.3.3 Members

The Region object has no members.

45.4 Set object

If a set spans more than one part instance, the members *vertices*, *edges*, *faces*, *cells*, *elements*, or *nodes* return a sequence of all the queried vertices/edges/faces/cells/elements/nodes respectively for each part instance contained in the set. For example:

```
assembly=mdb.models['Transmission'].rootAssembly
clutchElements=assembly.instances['Clutch'].elements
clutchSet=clutchElements[16:18]+clutchElements[78:80]
housingElements=assembly.instances['Housing'].elements
housingSet=housingElements[45:48]
transmissionSet=assembly.Set(name='TransmissionSet', \
                             elements=(clutchSet, housingSet))
len(transmissionSet.elements)=7
transmissionSet.elements[0]=mdb.models['Transmission'].rootAssembly.in
transmissionSet.elements[6]=mdb.models['Transmission'].rootAssembly.in
```

Access

```
import part
mdb.models[name].parts[name].allInternalSets[name]
mdb.models[name].parts[name].allSets[name]
mdb.models[name].parts[name].sets[name]
import assembly
mdb.models[name].rootAssembly.allinstances.sets[name]
mdb.models[name].rootAssembly.allInternalSets[name]
mdb.models[name].rootAssembly.allSets[name]
mdb.models[name].rootAssembly.instances[name].sets[name]
mdb.models[name].rootAssembly.modelInstances[i].sets[name]
mdb.models[name].rootAssembly.sets[name]
```

45.4.1 Set(...)

This method creates a set from a sequence of objects in a model database.

Path

```
mdb.models[name].parts[name].Set
mdb.models[name].rootAssembly.Set
```

Required argument*name*

A String specifying the repository key.

Optional arguments

At least one sequence argument must be provided—*elements*, *nodes*, *vertices*, *edges*, *faces*, *cells*, or *referencePoints*. The arguments *xVertices*, *xEdges*, and *xFaces* are used to exclude lower-dimension entities and to provide finer control on the content of the set. For example, the following statement defines a region enclosing a square face but without two of its edges:

```
set = mdb.models['Model-1'].rootAssembly.Set(name='mySet', \
        faces=f[3:4], \
                                           xEdges=e[1:3])
```

nodes

A sequence of MeshNode objects. The default value is **None**.

elements

A sequence of MeshElement objects. The default value is **None**.

region

A Region object specifying other objects to be included in the set. The default value is **None**.

vertices

A sequence of Vertex objects. The default value is **None**.

edges

A sequence of Edge objects. The default value is **None**.

faces

A sequence of Face objects. The default value is **None**.

cells

A sequence of Cell objects. The default value is **None**.

xVertices

A sequence of Vertex objects that excludes specific vertices from the set. The default value is **None**.

xEdges

A sequence of Edge objects that excludes specific edges from the set. The default value is **None**.

xFaces

A sequence of Face objects that excludes specific faces from the set. The default value is **None**.

REGION COMMANDS

referencePoints

A sequence of ReferencePoint objects. The default value is an empty sequence.

skinFaces

A tuple of tuples specifying a skin name and the sequence of faces associated with this skin. Valid only for geometric regions on 3D and 2D parts.

skinEdges

A tuple of tuples specifying a skin name and the sequence of edges associated with this skin. Valid only for geometric regions on Axisymmetric parts.

stringerEdges

A tuple of tuples specifying a stringer name and the sequence of edges associated with this stringer. Valid only for geometric regions on 3D and 2D parts.

Return value

A Set object.

Exceptions

InvalidNameError.

45.4.2 Set(...)

This method copies a set from an existing set.

Path

```
mdb.models[name].parts[name].Set  
mdb.models[name].rootAssembly.Set
```

Required arguments

name

A String specifying the name of the set.

objectToCopy

A Set object to be copied.

Optional arguments

None.

Return value

A Set object.

Exceptions

InvalidNameError.

45.4.3 SetByBoolean(...)

This method creates a set by performing a boolean operation on two or more input sets.

Path

```
mdb.models[name].parts[name].SetByBoolean
mdb.models[name].rootAssembly.SetByBoolean
```

Required arguments

name

A String specifying the repository key.

sets

A sequence of Set objects.

Optional argument

operation

A SymbolicConstant specifying the boolean operation to perform. Possible values are UNION, INTERSECTION, and DIFFERENCE. The default value is UNION. Note that if DIFFERENCE is specified, the order of the given input sets is important; All sets specified after the first one are subtracted from the first one.

Return value

A Set object.

Exceptions

InvalidNameError.

45.4.4 SetFromColor(...)

This method creates a set containing faces of the part marked with a specified color attribute. Third-party applications can assign color attributes to faces, and the color attribute can be imported into Abaqus from an ACIS file. You can use this method to create sets only on parts; however, you can access the sets from instances of the parts in the assembly.

Path

```
mdb.models[name].parts[name].SetFromColor
```

Required arguments

name

A String specifying the repository key.

color

A sequence of three Ints specifying the RGB color. Values can range from 0 to 255. The first integer is for red, the second for green, and the third for blue. For example, a face colored in yellow is identified by:

```
color=(255,255,0)
```

Optional arguments

None.

Return value

A Set object.

Exceptions

InvalidNameError.

45.4.5 SetFromElementLabels(...)

This method creates a set from a sequence of element labels in a model database.

Path

```
mdb.models[name].parts[name].SetFromElementLabels  
mdb.models[name].rootAssembly.SetFromElementLabels
```

Required arguments

name

A String specifying the repository key.

elementLabels

A sequence of element labels. An element label is a sequence of Int element identifiers. For example, for a part:

```
elementLabels=(2,3,5,7)
```

For an assembly:

```
elementLabels=(('Instance-1', (2,3,5,7)),
               ('Instance-2', (1,2,3)))
```

Optional arguments

None.

Return value

A Set object.

Exceptions

InvalidNameError.

45.4.6 SetFromNodeLabels(...)

This method creates a set from a sequence of node labels in a model database.

Path

```
mdb.models[name].parts[name].SetFromNodeLabels
mdb.models[name].rootAssembly.SetFromNodeLabels
```

Required arguments

name

A String specifying the repository key.

nodeLabels

A sequence of node labels. A node label is a sequence of Int node identifiers. For example, for a part:

```
nodeLabels=(2,3,5,7)
```

For an assembly:

```
nodeLabels=(('Instance-1', (2,3,5,7)), ('Instance-2', (1,2,3)))
```

Optional arguments

None.

Return value

A Set object.

Exceptions

`InvalidNameError`.

45.4.7 MapSetsFromOdb(...)

This method creates sets based on mapping sets from element centroid locations in an Odb.

Path

```
mdb.models[name].parts[name].MapSetsFromOdb
```

Required arguments

odbPath

A String specifying the path to the ODB containing the source sets.

odbSets

A list of names of sets on the ODB to map.

Optional arguments

partSets

A list of names of sets to construct or use corresponding to the ODB sets.

method

An enum to specify `OVERWRITE`, `APPEND`, `INTERSECT`, or `REMOVE`. The default is `OVERWRITE`.

Return value

A Set object or a tuple of Set objects.

Exceptions

None.

45.4.8 Members

The Set object has the following members:

elements

A `MeshElementArray` object.

nodes
A MeshNodeArray object.

vertices
A VertexArray object.

edges
An EdgeArray object.

faces
A FaceArray object.

cells
A CellArray object.

referencePoints
A ReferencePointArray object.

45.5 Surface object

The Surface object stores surfaces selected from the assembly. A surface is comprised of geometric or discrete entities but not both. An instance of a Surface object is available from the *surface* member of the Assembly object.

Access

```
import part
mdb.models[name].parts[name].allInternalSurfaces[name]
mdb.models[name].parts[name].allSurfaces[name]
mdb.models[name].parts[name].surfaces[name]
import assembly
mdb.models[name].rootAssembly.allInstances.surfaces[name]
mdb.models[name].rootAssembly.allInternalSurfaces[name]
mdb.models[name].rootAssembly.allSurfaces[name]
mdb.models[name].rootAssembly.instances[name].surfaces[name]
mdb.models[name].rootAssembly.modelInstances[i].surfaces[name]
mdb.models[name].rootAssembly.surfaces[name]
```

45.5.1 Surface(...)

This method creates a surface from a sequence of objects in a model database. The surface will apply to the sides specified by the arguments. For example,

```
surface=mdb.models['Model-1'].parts['Part-1'].Surface(sidelFaces=sidelFace
```

Path

```
mdb.models[name].parts[name].Surface  
mdb.models[name].rootAssembly.Surface
```

Required argument

On three-dimensional solid faces, you can use the following arguments:

- *side1Faces*
- *side2Faces*

On three-dimensional shell faces, you can use the following arguments:

- *side1Faces*
- *side2Faces*
- *side12Faces*

On three-dimensional wire edges, you can use the following arguments:

- *end1Edges*
- *end2Edges*
- *circumEdges*

On three-dimensional or two-dimensional or axisymmetric edges, you can use the following arguments:

- *side1Edges*
- *side2Edges*

On two-dimensional or axisymmetric shell elements, you can use the following arguments:

- *face1Elements*
- *face2Elements*
- *face3Elements*
- *face14Elements*

On solid elements, you can use the following arguments:

- *face1Elements*
- *face2Elements*
- *face3Elements*
- *face4Elements*
- *face5Elements*
- *face6Elements*

On three-dimensional shell elements, you can use the following arguments:

- *side1Elements*
- *side2Elements*

- *side12Elements*

On three-dimensional wire elements, you can use the following arguments:

- *end1Elements*
- *end2Elements*
- *circumElements*

On two-dimensional or axisymmetric wire elements, you can use the following arguments:

- *side1Elements*
- *side2Elements*

name

A String specifying the repository key. The default value is an empty string.

Optional arguments

side1Elements

A sequence of MeshElement objects (surface applies to SIDE1 of element). The default value is **None**.

side2Elements

A sequence of MeshElement objects (surface applies to SIDE2 of element). The default value is **None**.

side12Elements

A sequence of MeshElement objects (surface applies to both SIDE1 and SIDE2 of element). The default value is **None**.

end1Elements

A sequence of MeshElement objects (surface applies to END1 of element). The default value is **None**.

end2Elements

A sequence of MeshElement objects (surface applies to END2 of element). The default value is **None**.

circumElements

A sequence of MeshElement objects (surface applies to circumference of element). The default value is **None**.

face1Elements

A sequence of MeshElement objects (surface applies to FACE1 of element). The default value is **None**.

face2Elements

A sequence of MeshElement objects (surface applies to FACE2 of element). The default value is **None**.

REGION COMMANDS

face3Elements

A sequence of MeshElement objects (surface applies to FACE3 of element). The default value is **None**.

face4Elements

A sequence of MeshElement objects (surface applies to FACE4 of element). The default value is **None**.

face5Elements

A sequence of MeshElement objects (surface applies to FACE5 of element). The default value is **None**.

face6Elements

A sequence of MeshElement objects (surface applies to FACE6 of element). The default value is **None**.

side1Faces

A sequence of Face objects (surface applies to SIDE1 of face). The default value is **None**.

side2Faces

A sequence of Face objects (surface applies to SIDE2 of face). The default value is **None**.

side12Faces

A sequence of Face objects (surface applies to both SIDE1 and SIDE2 of face). The default value is **None**.

side1Edges

A sequence of Edge objects (surface applies to SIDE1 of edge). The default value is **None**.

side2Edges

A sequence of Edge objects (surface applies to SIDE2 of edge). The default value is **None**.

end1Edges

A sequence of Edge objects (surface applies to END1 of edge). The default value is **None**.

end2Edges

A sequence of Edge objects (surface applies to END2 of edge). The default value is **None**.

circumEdges

A sequence of Edge objects (surface applies circumferentially to edge). The default value is **None**.

Return value

A Surface object.

Exceptions

InvalidNameError.

45.5.2 Surface(...)

This method copies a surface from an existing surface.

Path

```
mdb.models[name].parts[name].Surface
mdb.models[name].rootAssembly.Surface
```

Required arguments*name*

A String specifying the name of the surface.

objectToCopy

A Surface object to be copied.

Optional arguments

None.

Return value

A Surface object.

Exceptions

InvalidNameError.

45.5.3 SurfaceByBoolean(...)

This method creates a surface by performing a boolean operation on two or more input surfaces.

Path

```
mdb.models[name].parts[name].SurfaceByBoolean
mdb.models[name].rootAssembly.SurfaceByBoolean
```

Required arguments*name*

A String specifying the repository key.

surfaces

A sequence of Surface objects.

Optional argument*operation*

A SymbolicConstant specifying the boolean operation to perform. Possible values are UNION, INTERSECTION, and DIFFERENCE. The default value is UNION. Note that if DIFFERENCE is specified, the order of the given input surfaces is important; All surfaces specified after the first one are subtracted from the first one.

Return value

A Surface object.

Exceptions

InvalidNameError.

45.5.4 SurfaceFromElsets(...)

This method creates a surface from a sequence of element sets in a model database.

Path

```
mdb.models[name].rootAssembly.SurfaceFromElsets
```

Required arguments

name

A String specifying the repository key.

elementSetSeq

A sequence of element sets. For example,

```
elementSetSeq=((elset1, S1),(elset2, S2))
```

where `elset1=mdb.models[name].rootAssembly.sets['Clutch']` and `S1` and `S2` indicate the side of the element set.

Optional arguments

None.

Return value

A Surface object.

Exceptions

InvalidNameError.

45.5.5 Members

The Surface object can have the following members:

edges

An EdgeArray object.

faces

A FaceArray object.

elements

A MeshElementArray object.

nodes

A MeshNodeArray object.

sides

A tuple of SymbolicConstants specifying the sides; for example, (SIDE1, SIDE2).

instances

A tuple of Ints specifying the instances. This member is not applicable for a Surface object on an output database.

45.6 Skin object

The Skin object stores information on skin reinforcements created on entities.

Access

```

import part
mdb.models[name].parts[name].skins[name]
import assembly
mdb.models[name].rootAssembly.allinstances.skins[name]
mdb.models[name].rootAssembly.instances[name].skins[name]
mdb.models[name].rootAssembly.skins[name]

```

45.6.1 Skin(...)

This method creates a skin from a sequence of objects in a model database. At least one of the optional arguments needs to be specified.

Path

```

mdb.models[name].parts[name].Skin

```

Required argument

name

A String specifying the repository key. The default value is an empty string.

Optional arguments

faces

A sequence of Face objects specifying the faces on which skins should be created. Applicable to three and two dimensional parts.

edges

A sequence of Edge objects specifying the edges on which skins should be created. Applicable to axisymmetric parts.

elementFaces

A sequence of MeshFace objects specifying the mesh faces on which skins should be created. Applicable to three and two dimensional parts.

elementEdges

A sequence of MeshEdge objects specifying the mesh edges on which skins should be created. Applicable to axisymmetric parts.

Return value

A Skin object.

Exceptions

InvalidNameError.

45.6.2 EditSkin(...)

This method modifies underlying entities of the selected skin. At least one of the optional arguments needs to be specified.

Path

```
mdb.models[name].parts[name].EditSkin
```

Required argument

name

A String specifying the repository key. The default value is an empty string.

Optional arguments

faces

A sequence of Face objects specifying the faces on which skins should be created. Applicable to three and two dimensional parts.

edges

A sequence of Edge objects specifying the edges on which skins should be created. Applicable to axisymmetric parts.

elementFaces

A sequence of MeshFace objects specifying the mesh faces on which skins should be created.
Applicable to three and two dimensional parts.

elementEdges

A sequence of MeshEdge objects specifying the mesh edges on which skins should be created.
Applicable to axisymmetric parts.

Return value

A Skin object.

Exceptions

InvalidNameError.

45.6.3 Members

The Skin object has the following members:

elements

A MeshElementArray object.

edges

An EdgeArray object.

faces

A FaceArray object.

45.7 Stringer object

The Stringer object stores information on stringer reinforcements created on entities.

Access

```
import part
mdb.models[name].parts[name].stringers[name]
import assembly
mdb.models[name].rootAssembly.allinstances.stringers[name]
mdb.models[name].rootAssembly.instances[name].stringers[name]
mdb.models[name].rootAssembly.stringers[name]
```

45.7.1 Stringer(...)

This method creates a stringer from a sequence of objects in a model database. At least one of the optional arguments needs to be specified.

Path

`mdb.models[name].parts[name].Stringer`

Required argument

name

A String specifying the repository key. The default value is an empty string.

Optional arguments

edges

A sequence of Edge objects specifying the edges on which stringers should be created. Applicable to three and two dimensional parts.

elementEdges

A sequence of MeshEdge objects specifying the mesh edges on which stringers should be created. Applicable to three and two dimensional parts.

Return value

A Stringer object.

Exceptions

InvalidNameError.

45.7.2 EditStringer(...)

This method modifies underlying entities of the selected stringer. At least one of the optional arguments needs to be specified.

Path

`mdb.models[name].parts[name].EditStringer`

Required argument

name

A String specifying the repository key. The default value is an empty string.

Optional arguments*edges*

A sequence of Edge objects specifying the edges on which stringers should be created. Applicable to three and two dimensional parts.

elementEdges

A sequence of MeshEdge objects specifying the mesh edges on which stringers should be created. Applicable to three and two dimensional parts.

Return value

A Stringer object.

Exceptions

InvalidNameError.

45.7.3 Members

The Stringer object has the following members:

elements

A MeshElementArray object.

edges

An EdgeArray object.

46. Section commands

The Section commands are used to create sections and profiles with their associated properties and behavior. The various section objects are all derived from the Section object. The various profile objects are all derived from the Profile object. See Chapter 44, “Property commands,” for the property assignment commands.

46.1 Section object

The Section object defines the properties of a section. The Section object is the abstract base type for other Section objects. The Section object has no explicit constructor. The methods and members of the Section object are common to all objects derived from the Section.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.1.1 sectionsFromOdb(...)

This method creates Section objects by reading an output database. The new sections are placed in the `sections` repository.

Path

```
mdb.models[name].sectionsFromOdb
```

Required argument

fileName

A String specifying the name of the output database file (including the `.odb` extension) to be read. This String can also be the full path to the output database file if it is located in another directory.

Optional arguments

None.

Return value

A list of Section objects.

Exceptions

None.

46.1.2 Members

The Section object has the following member:

name

A String specifying the repository key.

46.2 AcousticInfiniteSection object

The AcousticInfiniteSection object defines the properties of an acoustic section.

The AcousticInfiniteSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.2.1 AcousticInfiniteSection(...)

This method creates an AcousticInfiniteSection object.

Path

```
mdb.models[name].AcousticInfiniteSection
session.odbs[name].AcousticInfiniteSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material.

Optional arguments

thickness

A Float specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

order

An Int specifying the number of ninth-order polynomials that will be used to resolve the variation of the acoustic field in the infinite direction. Possible values are $0 < order \leq 10$. The default value is 10.

Return value

An `AcousticInfiniteSection` object.

Exceptions

`InvalidNameError` and `RangeError`.

46.2.2 `setValues(...)`

This method modifies the `AcousticInfiniteSection` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `AcousticInfiniteSection` method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

46.2.3 Members

The `AcousticInfiniteSection` object has members with the same names and descriptions as the arguments to the `AcousticInfiniteSection` method.

46.2.4 Corresponding analysis keywords

*SOLID SECTION

46.3 AcousticInterfaceSection object

The AcousticInterfaceSection object defines the properties of an acoustic section.

The AcousticInterfaceSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.3.1 AcousticInterfaceSection(...)

This method creates an AcousticInterfaceSection object.

Path

```
mdb.models[name].AcousticInterfaceSection
session.odbs[name].AcousticInterfaceSection
```

Required argument

name

A String specifying the repository key.

Optional argument

thickness

A Float specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

Return value

An AcousticInterfaceSection object.

Exceptions

InvalidNameError and RangeError.

46.3.2 setValues(...)

This method modifies the AcousticInterfaceSection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AcousticInterfaceSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.3.3 Members

The **AcousticInterfaceSection** object has members with the same names and descriptions as the arguments to the **AcousticInterfaceSection** method.

46.3.4 Corresponding analysis keywords

*INTERFACE

46.4 BeamSection object

The **BeamSection** object defines the properties of a beam section.

The **BeamSection** object is derived from the **Section** object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.4.1 BeamSection(...)

This method creates a **BeamSection** object.

SECTION COMMANDS

Path

`mdb.models[name].BeamSection`
`session.odbs[name].BeamSection`

Required arguments

name

A String specifying the repository key.

integration

A SymbolicConstant specifying the integration method for the section. Possible values are BEFORE_ANALYSIS and DURING_ANALYSIS.

profile

A String specifying the name of the profile. This argument represents the start profile in case of *beamShape*=TAPERED.

Optional arguments

poissonRatio

A Float specifying the Poisson's ratio of the section. The default value is 0.0.

thermalExpansion

A Boolean specifying whether to use thermal expansion data. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

density

None or a Float specifying the density of the section. The default value is **None**.

referenceTemperature

None or a Float specifying the reference temperature of the section. The default value is **None**.

temperatureVar

A SymbolicConstant specifying the temperature variation for the section. Possible values are LINEAR and INTERPOLATED. The default value is LINEAR.

alphaDamping

A Float specifying the α_R factor to create mass proportional damping in direct-integration dynamics. The default value is 0.0.

betaDamping

A Float specifying the β_R factor to create stiffness proportional damping in direct-integration dynamics. The default value is 0.0.

compositeDamping

A Float specifying the fraction of critical damping to be used in calculating composite damping factors for the modes (for use in modal dynamics). The default value is 0.0.

useFluidInertia

A Boolean specifying whether added mass effects will be simulated. The default value is OFF.

submerged

A SymbolicConstant specifying whether the section is either full submerged or half submerged. This argument applies only when *useFluidInertia* = True. Possible values are FULLY and HALF. The default value is FULLY.

fluidMassDensity

None or a Float specifying the mass density of the fluid. This argument applies only when *useFluidInertia* = True and must be specified in that case. The default value is **None**.

crossSectionRadius

None or a Float specifying the radius of the cylindrical cross-section. This argument applies only when *useFluidInertia* = True and must be specified in that case. The default value is **None**.

lateralMassCoef

A Float specifying the added mass coefficient, C_A , for lateral motions of the beam. This argument applies only when *useFluidInertia* = True. The default value is 1.0.

axialMassCoef

A Float specifying the added mass coefficient, $C(A - E)$, for motions along the axis of the beam. This argument affects only the term added to the free end(s) of the beam, and applies only when *useFluidInertia* = True. The default value is 0.0.

massOffsetX

A Float specifying the local 1-coordinate of the center of the cylindrical cross-section with respect to the beam cross-section. This argument applies only when *useFluidInertia* = True. The default value is 0.0.

massOffsetY

A Float specifying the local 2-coordinate of the center of the cylindrical cross-section with respect to the beam cross-section. This argument applies only when *useFluidInertia* = True. The default value is 0.0.

beamShape

A SymbolicConstant specifying the change in cross-section of the beam along length. Possible values are CONSTANT and TAPERED. The default value is CONSTANT. This parameter is available for manipulating the model database but not for the ODB API.

material

A String specifying the name of the material. The default value is an empty string. The material is required when *integration* is "DURING_ANALYSIS".

SECTION COMMANDS

table

A sequence of sequences of Floats specifying the items described below. The default value is an empty sequence.

outputPts

A sequence of pairs of Floats specifying the positions at which output is requested. The default value is an empty sequence.

centroid

A pair of Floats specifying the X – Y coordinates of the centroid. The default value is (0.0, 0.0).

shearCenter

A pair of Floats specifying the X – Y coordinates of the shear center. The default value is (0.0, 0.0).

profileEnd

A String specifying the name of the end profile. The type of the end profile must be same as that of the start profile. This argument is valid only when *beamShape*=TAPERED. The default value is an empty string. This parameter is available for manipulating the model database but not for the ODB API.

Table data

The table data specify the following:

- E, the Young's modulus of the section.
- G, the torsional shear modulus of the section.
- Thermal expansion coefficient, if using thermal expansion.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BeamSection object.

Exceptions

None.

46.4.2 setValues(...)

This method modifies the BeamSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BeamSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.4.3 Members

The BeamSection object has members with the same names and descriptions as the arguments to the **BeamSection** method.

In addition, the BeamSection object can have the following member:

beamTransverseShear

A TransverseShearBeam object specifying the transverse shear stiffness properties.

46.4.4 Corresponding analysis keywords

- *BEAM GENERAL SECTION
- *BEAM SECTION
- *BEAM FLUID INERTIA
- *CENTROID
- *DAMPING
- *SHEAR CENTER
- *SECTION POINTS

46.5 CohesiveSection object

The CohesiveSection object defines the properties of a cohesive section.

The CohesiveSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
```

```
import odbSection
session.odbs[name].sections[name]
```

46.5.1 CohesiveSection(...)

This method creates a CohesiveSection object.

Path

```
mdb.models[name].CohesiveSection
session.odbs[name].CohesiveSection
```

Required arguments

name

A String specifying the repository key.

response

A SymbolicConstant specifying the geometric assumption that defines the constitutive behavior of the cohesive elements. Possible values are TRACTION_SEPARATION, CONTINUUM, and GASKET.

material

A String specifying the name of the material.

Optional arguments

initialThicknessType

A SymbolicConstant specifying the method used to compute the initial thickness. Possible values are:

- SOLVER_DEFAULT, specifying that Abaqus will use the analysis product default
- GEOMETRY, specifying that Abaqus will compute the thickness from the nodal coordinates of the elements.
- SPECIFY, specifying that Abaqus will use the value given for *initialThickness*

The default value is SOLVER_DEFAULT.

initialThickness

A Float specifying the initial thickness for the section. The *initialThickness* argument applies only when *initialThicknessType*=SPECIFY. The default value is 1.0.

outOfPlaneThickness

None or a Float specifying the out-of-plane thickness for the section. The default value is **None**.

Return value

A CohesiveSection object.

Exceptions

RangeError and InvalidNameError.

46.5.2 setValues(...)

This method modifies the CohesiveSection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CohesiveSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.5.3 Members

The CohesiveSection object has members with the same names and descriptions as the arguments to the **CohesiveSection** method.

46.5.4 Corresponding analysis keywords

*COHESIVE SECTION

46.6 CompositeShellSection object

The CompositeShellSection object defines the properties of a composite shell section.

The CompositeShellSection object is derived from the GeometryShellSection object.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.6.1 CompositeShellSection(...)

This method creates a CompositeShellSection object.

Path

```
mdb.models[name].parts[name].compositeLayups[i].CompositeShellSection
mdb.models[name].CompositeShellSection
session.odbs[name].CompositeShellSection
```

Required arguments

name

A String specifying the repository key.

layup

A SectionLayerArray object specifying the shell cross-section.

Optional arguments

symmetric

A Boolean specifying whether or not the layup should be made symmetric by the analysis. The default value is OFF.

thicknessType

A SymbolicConstant specifying the distribution used for defining the thickness of the elements. Possible values are UNIFORM, ANALYTICAL_FIELD, DISCRETE_FIELD, NODAL_ANALYTICAL_FIELD, and NODAL_DISCRETE_FIELD. The default value is UNIFORM.

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (ON) or integrated during the analysis (OFF). The default value is OFF.

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- **DEFAULT**, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- **VALUE**, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is **DEFAULT**.

poisson

A Float specifying the Poisson's ratio. Possible values are $-1.0 \leq \textit{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=**VALUE**. The default value is 0.5.

integrationRule

A SymbolicConstant specifying the shell section integration rule. Possible values are **SIMPSON** and **GAUSS**. The default value is **SIMPSON**.

temperature

A SymbolicConstant specifying the mode used for temperature and field variable input across the section thickness. Possible values are **GRADIENT** and **POINTWISE**. The default value is **GRADIENT**.

idealization

A SymbolicConstant specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to **ON**. Possible values are **NO_IDEALIZATION**, **SMEAR_ALL_LAYERS**, **MEMBRANE**, and **BENDING**. The default value is **NO_IDEALIZATION**.

nTemp

None or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*=**POINTWISE**. The default value is **None**.

thicknessModulus

None or a Float specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is **None**.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is **OFF**.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

layoutName

A String specifying the layout name for this section. The default value is an empty string.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=**ANALYTICAL_FIELD** or *thicknessType*=**DISCRETE_FIELD**. The default value is an empty string.

SECTION COMMANDS

nodalThicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*=NODAL_ANALYTICAL_FIELD or *thicknessType*=NODAL_DISCRETE_FIELD. The default value is an empty string.

Return value

A CompositeShellSection object.

Exceptions

None.

46.6.2 setValues(...)

This method modifies the CompositeShellSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CompositeShellSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.6.3 Members

The CompositeShellSection object has members with the same names and descriptions as the arguments to the **CompositeShellSection** method.

In addition, the CompositeShellSection object can have the following members:

rebarLayers

A RebarLayers object specifying reinforcement properties.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

46.6.4 Corresponding analysis keywords

*SHELL SECTION
*SHELL GENERAL SECTION

46.7 CompositeSolidSection object

The CompositeSolidSection object defines the properties of a composite solid section.
The CompositeSolidSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.7.1 CompositeSolidSection(...)

This method creates a CompositeSolidSection object.

Path

```
mdb.models[name].CompositeSolidSection
session.odbs[name].CompositeSolidSection
```

Required arguments

name

A String specifying the repository key.

layup

A SectionLayerArray object specifying the solid cross-section.

Optional arguments

symmetric

A Boolean specifying whether or not the layup should be made symmetric by the analysis. The default value is OFF.

layupName

A String specifying the layup name for this section. The default value is an empty string.

SECTION COMMANDS

Return value

A CompositeSolidSection object.

Exceptions

None.

46.7.2 setValues(...)

This method modifies the CompositeSolidSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CompositeSolidSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.7.3 Members

The CompositeSolidSection object has members with the same names and descriptions as the arguments to the **CompositeSolidSection** method.

46.7.4 Corresponding analysis keywords

*SOLID SECTION

46.8 ConnectorSection object

A ConnectorSection object describes the connection type and the behavior of a connector.

The ConnectorSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.8.1 ConnectorSection(...)

This method creates a ConnectorSection object.

Path

```
mdb.models[name].ConnectorSection
session.odbs[name].ConnectorSection
```

Required argument

name

A String specifying the repository key.

Optional arguments

assembledType

A SymbolicConstant specifying the assembled connection type. Possible values are:

- NONE
- BEAM
- BUSHING
- CVJOINT
- CYLINDRICAL
- HINGE
- PLANAR
- RETRACTOR
- SLIPRING
- TRANSLATOR
- UJOINT
- WELD

The default value is NONE.

SECTION COMMANDS

You cannot include the *assembledType* argument if *translationalType* or *rotationalType* are given a value other than NONE. At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given a value other than NONE.

rotationalType

A SymbolicConstant specifying the basic rotational connection type. Possible values are:

- NONE
- ALIGN
- CARDAN
- CONSTANT_VELOCITY
- EULER
- FLEXION_TORSION
- FLOW_CONVERTER
- PROJECTION_FLEXION_TORSION
- REVOLUTE
- ROTATION
- ROTATION_ACCELEROMETER
- UNIVERSAL

The default value is NONE.

You cannot include the *rotationalType* argument if *assembledType* is given a value other than NONE. At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given a value other than NONE.

translationalType

A SymbolicConstant specifying the basic translational connection type. Possible values are:

- NONE
- ACCELEROMETER
- AXIAL
- CARTESIAN
- JOIN
- LINK
- PROJECTION_CARTESIAN
- RADIAL_THRUST
- SLIDE_PLANE
- SLOT

The default value is NONE.

You cannot include the *translationalType* argument if *assembledType* is given a value other than NONE. At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given a value other than NONE.

integration

A SymbolicConstant specifying the time integration scheme to use for analysis. This argument is applicable only to an Abaqus/Explicit analysis. Possible values are UNSPECIFIED, IMPLICIT, and EXPLICIT. The default value is UNSPECIFIED.

u1ReferenceLength

None or a Float specifying the reference length associated with constitutive response for the first component of relative motion. The default value is **None**.

u2ReferenceLength

None or a Float specifying the reference length associated with constitutive response for the second component of relative motion. The default value is **None**.

u3ReferenceLength

None or a Float specifying the reference length associated with constitutive response for the third component of relative motion. The default value is **None**.

ur1ReferenceAngle

None or a Float specifying the reference angle in degrees associated with constitutive response for the fourth component of relative motion. The default value is **None**.

ur2ReferenceAngle

None or a Float specifying the reference angle in degrees associated with constitutive response for the fifth component of relative motion. The default value is **None**.

ur3ReferenceAngle

None or a Float specifying the reference angle in degrees associated with constitutive response for the sixth component of relative motion. The default value is **None**.

massPerLength

None or a Float specifying the mass per unit reference length of belt material. This argument is applicable only when *assembledType*=SLIPRING, and must be specified in that case. The default value is **None**.

contactAngle

None or a Float specifying the contact angle made by the belt wrapping around node b. This argument is applicable only to an Abaqus/Explicit analysis, and only when *assembledType*=SLIPRING. The default value is **None**.

materialFlowFactor

A Float specifying the scaling factor for material flow at node b. This argument is applicable only when *assembledType*=RETRACTOR or *rotationalType*=FLOW_CONVERTER. The default value is 1.0.

SECTION COMMANDS

regularize

A Boolean specifying whether or not all tabular data associated with the *behaviorOptions* will be regularized. This argument is applicable only for an Abaqus/Explicit analysis. The default value is ON.

defaultTolerance

A Boolean specifying whether or not the default regularization tolerance will be used for all tabular data associated with the *behaviorOptions*. This argument is applicable only for an Abaqus/Explicit analysis and only if *regularize*=ON. The default value is ON.

regularization

A Float specifying the regularization increment to be used for all tabular data associated with the *behaviorOptions*. This argument is applicable only for an Abaqus/Explicit analysis and only if *regularize*=ON and *defaultTolerance*=OFF. The default value is 0.03.

extrapolation

A SymbolicConstant specifying the extrapolation technique to be used for all tabular data associated with the *behaviorOptions*. Possible values are CONSTANT and LINEAR. The default value is CONSTANT.

behaviorOptions

A ConnectorBehaviorOptionArray object.

Return value

A ConnectorSection object.

Exceptions

InvalidNameError and RangeError.

46.8.2 setValues(...)

This method modifies the ConnectorSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.8.3 Members

The ConnectorSection object has members with the same names and descriptions as the arguments to the `ConnectorSection` method.

46.8.4 Corresponding analysis keywords

- *CONNECTOR SECTION
- *CONNECTOR BEHAVIOR
- *CONNECTOR CONSTITUTIVE REFERENCE

46.9 EulerianSection object

The EulerianSection object defines the properties of a Eulerian section.

The EulerianSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.9.1 EulerianSection(...)

This method creates a EulerianSection object.

Path

```
mdb.models[name].EulerianSection
session.odbs[name].EulerianSection
```

Required arguments

name

A String specifying the repository key.

SECTION COMMANDS

data

A String-to-String Dictionary specifying a dictionary mapping Material instance names to Material names. Internally the specified mapping gets sorted on Material instance name.

Optional arguments

None.

Return value

An EulerianSection object.

Exceptions

None.

46.9.2 setValues(...)

This method modifies the EulerianSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **EulerianSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.9.3 Members

The EulerianSection object has members with the same names and descriptions as the arguments to the **EulerianSection** method.

46.9.4 Corresponding analysis keywords

*EULERIAN SECTION

46.10 GasketSection object

The GasketSection object defines the properties of a gasket section.

The GasketSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.10.1 GasketSection(...)

This method creates a GasketSection object.

Path

```
mdb.models[name].GasketSection
session.odbs[name].GasketSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material of which the gasket is made or material that defines gasket behavior.

Optional arguments

crossSection

A Float specifying the cross-sectional area, width, or out-of-plane thickness, if applicable, depending on the gasket element type. The default value is 1.0.

initialGap

A Float specifying the initial gap. The default value is 0.0.

initialThickness

The SymbolicConstant DEFAULT or a Float specifying the initial gasket thickness. If DEFAULT is specified, the initial thickness is determined using nodal coordinates. The default value is DEFAULT.

initialVoid

A Float specifying the initial void. The default value is 0.0.

SECTION COMMANDS

stabilizationStiffness

The SymbolicConstant DEFAULT or a Float specifying the default stabilization stiffness used in all but link elements to stabilize gasket elements that are not supported at all nodes, such as those that extend outside neighboring components. If DEFAULT is specified, a value is used equal to 10^{-9} times the initial compressive stiffness in the thickness direction. The default value is DEFAULT.

Return value

A GasketSection object.

Exceptions

InvalidNameError and ValueError.

46.10.2 setValues(...)

This method modifies the GasketSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GasketSection** method, except for the *name* argument.

Return value

None

Exceptions

ValueError.

46.10.3 Members

The GasketSection object has members with the same names and descriptions as the arguments to the **GasketSection** method.

46.10.4 Corresponding analysis keywords

*GASKET SECTION

46.11 GeneralStiffnessSection object

The GeneralStiffnessSection object defines the properties of a shell section via the stiffness matrix.
The GeneralStiffnessSection object is derived from the ShellSection object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.11.1 GeneralStiffnessSection(...)

This method creates a GeneralStiffnessSection object.

Path

```
mdb.models[name].GeneralStiffnessSection
session.odbs[name].GeneralStiffnessSection
```

Required arguments

name

A String specifying the repository key.

stiffnessMatrix

A sequence of Floats specifying the stiffness matrix for the section in the order D11, D12, D22, D13, D23, D33, ..., D66. Twenty-one entries must be given.

Optional arguments

referenceTemperature

None or a Float specifying the reference temperature for thermal expansion. The default value is **None**.

applyThermalStress

A Boolean specifying whether or not the section stiffness varies with thermal stresses. The default value is OFF.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is OFF.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

SECTION COMMANDS

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- DEFAULT, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- VALUE, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is DEFAULT.

poisson

A Float specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=VALUE. The default value is 0.5.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is OFF.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

thermalStresses

A sequence of Floats specifying the generalized stress values caused by a unit temperature rise. Six entries must be given if the value of *applyThermalStress* is set to True. The default value is ("").

scalingData

A sequence of sequences of Floats specifying the scaling factors for given temperatures and/or field data. Each row should contain (Y, alpha, T, F1,...,Fn). The default value is an empty sequence.

Return value

A GeneralStiffnessSection object.

Exceptions

None.

46.11.2 setValues(...)

This method modifies the GeneralStiffnessSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GeneralStiffnessSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.11.3 Members

The GeneralStiffnessSection object has members with the same names and descriptions as the arguments to the **GeneralStiffnessSection** method.

In addition, the GeneralStiffnessSection object can have the following members:

rebarLayers

A RebarLayers object specifying reinforcement properties.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

46.11.4 Corresponding analysis keywords

*SHELL GENERAL SECTION

46.12 GeometryShellSection object

The GeometryShellSection object defines the properties of a geometry shell section. The GeometryShellSection object has no explicit constructor and no methods. The GeometryShellSection object is an abstract base type.

The GeometryShellSection object is derived from the ShellSection object.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].section
mdb.models[name].sections[name]
import odbSection
```

```
session.odbs[name].sections[name]
```

46.12.1 Section(...)

This method creates a GeometryShellSection object.

Path

```
mdb.models[name].parts[name].compositeLayups[name].Section
```

Required arguments

None.

Optional arguments

nodalThicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*=NODAL_ANALYTICAL_FIELD or *thicknessType*=NODAL_DISCRETE_FIELD. The default value is an empty string.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=ANALYTICAL_FIELD or *thicknessType*=DISCRETE_FIELD. The default value is an empty string.

thicknessType

A SymbolicConstant specifying the distribution used for defining the thickness of the elements. Possible values are UNIFORM, ANALYTICAL_FIELD, DISCRETE_FIELD, NODAL_ANALYTICAL_FIELD, and NODAL_DISCRETE_FIELD. The default value is UNIFORM.

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (ON) or integrated during the analysis (OFF). The default value is OFF.

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- DEFAULT, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- VALUE, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is DEFAULT.

poisson

A Float specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=VALUE. The default value is 0.5.

integrationRule

A SymbolicConstant specifying the shell section integration rule. Possible values are SIMPSON and GAUSS. The default value is SIMPSON.

temperature

A SymbolicConstant specifying the mode used for temperature and field variable input across the section thickness. Possible values are GRADIENT and POINTWISE. The default value is GRADIENT.

nTemp

None or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*=POINTWISE. The default value is **None**.

thicknessModulus

None or a Float specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is **None**.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is OFF.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

Return value

A GeometryShellSection object.

Exceptions

None.

46.12.2 Members

The GeometryShellSection object can have the following members:

name

A String specifying the repository key.

thicknessType

A SymbolicConstant specifying the distribution used for defining the thickness of the elements. Possible values are UNIFORM, ANALYTICAL_FIELD, DISCRETE_FIELD, NODAL_ANALYTICAL_FIELD, and NODAL_DISCRETE_FIELD. The default value is UNIFORM.

SECTION COMMANDS

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (ON) or integrated during the analysis (OFF). The default value is OFF.

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- **DEFAULT**, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- **VALUE**, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is **DEFAULT**.

poisson

A Float specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=**VALUE**. The default value is 0.5.

integrationRule

A SymbolicConstant specifying the shell section integration rule. Possible values are **SIMPSON** and **GAUSS**. The default value is **SIMPSON**.

temperature

A SymbolicConstant specifying the mode used for temperature and field variable input across the section thickness. Possible values are **GRADIENT** and **POINTWISE**. The default value is **GRADIENT**.

idealization

A SymbolicConstant specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to **ON**. Possible values are **NO_IDEALIZATION**, **SMEAR_ALL_LAYERS**, **MEMBRANE**, and **BENDING**. The default value is **NO_IDEALIZATION**.

nTemp

None or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*=**POINTWISE**. The default value is **None**.

thicknessModulus

None or a Float specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is **None**.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is OFF.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=ANALYTICAL_FIELD or *thicknessType*=DISCRETE_FIELD. The default value is an empty string.

rebarLayers

A RebarLayers object specifying reinforcement properties.

nodalThicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*=NODAL_ANALYTICAL_FIELD or *thicknessType*=NODAL_DISCRETE_FIELD. The default value is an empty string.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

46.13 HomogeneousShellSection object

The HomogeneousShellSection object defines the properties of a shell section.

The HomogeneousShellSection object is derived from the GeometryShellSection object.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.13.1 HomogeneousShellSection(...)

This method creates a HomogeneousShellSection object.

Path

```
mdb.models[name].parts[name].compositeLayups[i] \
.HomogeneousShellSection
mdb.models[name].HomogeneousShellSection
session.odbs[name].HomogeneousShellSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the section material.

Optional arguments

thickness

A Float specifying the thickness of the section. The *thickness* argument applies only when *thicknessType*=UNIFORM. The default value is 0.0.

numIntPts

An Int specifying the number of integration points to be used through the section. Possible values are *numIntPts* > 0. The default value is 5.

To use the default settings of the analysis products, set *numIntPts* to 5 if *integrationRule*=SIMPSON or set *numIntPts* to 7 if *integrationRule*=GAUSS.

thicknessType

A SymbolicConstant specifying the distribution used for defining the thickness of the elements. Possible values are UNIFORM, ANALYTICAL_FIELD, DISCRETE_FIELD, NODAL_ANALYTICAL_FIELD, and NODAL_DISCRETE_FIELD. The default value is UNIFORM.

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (ON) or integrated during the analysis (OFF). The default value is OFF.

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- DEFAULT, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- VALUE, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is DEFAULT.

poisson

A Float specifying the Poisson's ratio. Possible values are $-1.0 \leq \textit{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=VALUE. The default value is 0.5.

integrationRule

A SymbolicConstant specifying the shell section integration rule. Possible values are SIMPSON and GAUSS. The default value is SIMPSON.

temperature

A SymbolicConstant specifying the mode used for temperature and field variable input across the section thickness. Possible values are GRADIENT and POINTWISE. The default value is GRADIENT.

idealization

A SymbolicConstant specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to ON. Possible values are NO_IDEALIZATION, SMEAR_ALL_LAYERS, MEMBRANE, and BENDING. The default value is NO_IDEALIZATION.

nTemp

None or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*=POINTWISE. The default value is **None**.

thicknessModulus

None or a Float specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is **None**.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is OFF.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=ANALYTICAL_FIELD or *thicknessType*=DISCRETE_FIELD. The default value is an empty string.

nodalThicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*=NODAL_ANALYTICAL_FIELD or *thicknessType*=NODAL_DISCRETE_FIELD. The default value is an empty string.

Return value

A HomogeneousShellSection object.

Exceptions

None.

46.13.2 **setValues(...)**

This method modifies the `HomogeneousShellSection` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **HomogeneousShellSection** method, except for the *name* argument.

Return value

None

Exceptions

None.

46.13.3 **Members**

The `HomogeneousShellSection` object has members with the same names and descriptions as the arguments to the **HomogeneousShellSection** method.

In addition, the `HomogeneousShellSection` object can have the following members:

rebarLayers

A `RebarLayers` object specifying reinforcement properties.

transverseShear

A `TransverseShearShell` object specifying the transverse shear stiffness properties.

46.13.4 **Corresponding analysis keywords**

*SHELL SECTION

*SHELL GENERAL SECTION

46.14 **HomogeneousSolidSection object**

The `HomogeneousSolidSection` object defines the properties of a solid section.

The `HomogeneousSolidSection` object is derived from the `SolidSection` object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.14.1 HomogeneousSolidSection(...)

This method creates a HomogeneousSolidSection object.

Path

```
mdb.models[name].HomogeneousSolidSection
session.odbs[name].HomogeneousSolidSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material.

Optional argument

thickness

None or a Float specifying the thickness of the section. Possible values are None or a floating point value such that *thickness* > 0.0. The default value is None.

Return value

A HomogeneousSolidSection object.

Exceptions

InvalidNameError and RangeError.

46.14.2 setValues(...)

This method modifies the HomogeneousSolidSection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `HomogeneousSolidSection` method, except for the *name* argument.

Return value

None

Exceptions

`RangeError`.

46.14.3 Members

The `HomogeneousSolidSection` object has members with the same names and descriptions as the arguments to the `HomogeneousSolidSection` method.

46.14.4 Corresponding analysis keywords

*SOLID SECTION

46.15 LayerProperties object

The `LayerProperties` object defines the properties of a layer of reinforcement for membrane, shell, and surface sections.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].section.rebarLayers\
.layerTable[i]
mdb.models[name].sections[name].rebarLayers.layerTable[i]
import odbSection
session.odbs[name].sections[name].rebarLayers.layerTable[i]
```

46.15.1 LayerProperties(...)

This method creates a `LayerProperties` object.

Path

`section.LayerProperties`
`odbSection.LayerProperties`

Required arguments

barArea

A Float specifying the area per bar.

orientationAngle

A Float or a String specifying the orientation of the rebar. A Float specifies the angular orientation; a String specifies an orientation name.

layerName

A String specifying the name of the rebar layer.

material

A String specifying the name of the rebar material.

Optional arguments

barSpacing

A Float specifying the spacing of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to CONSTANT. The default value is 0.0.

layerPosition

A Float specifying the position of the rebar from the middle surface of the shell. *layerPosition* applies only for homogeneous shell sections and composite shell sections. The default value is 0.0.

spacingAngle

A Float specifying the spacing angle of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to ANGULAR. The default value is 0.0.

extensionRatio

A Float specifying the extension ratio for the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to LIFT_EQUATION. The default value is 0.0.

radius

A Float specifying the radius of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to LIFT_EQUATION. The default value is 0.0.

Return value

A LayerProperties object.

Exceptions

None.

46.15.2 Members

The `LayerProperties` object has members with the same names and descriptions as the arguments to the `LayerProperties` method.

46.15.3 Corresponding analysis keywords

*REBAR LAYER

46.16 MembraneSection object

The `MembraneSection` object defines the properties of a membrane section.

The `MembraneSection` object is derived from the `Section` object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.16.1 MembraneSection(...)

This method creates a `MembraneSection` object.

Path

```
mdb.models[name].MembraneSection
session.odbs[name].MembraneSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material.

Optional arguments*thickness*

A Float specifying the thickness for the section. Possible values are *thickness* > 0.0. The default value is 1.0.

thicknessType

A SymbolicConstant specifying the distribution used for defining the thickness of the elements. Possible values are UNIFORM, ANALYTICAL_FIELD, and DISCRETE_FIELD. The default value is UNIFORM.

poissonDefinition

A SymbolicConstant specifying whether to use the default value for the Poisson's ratio. Possible values are:

- DEFAULT, specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- VALUE, specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is DEFAULT.

poisson

A Float specifying the section Poisson's ratio. Possible values are $-1.0 \leq \textit{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*=VALUE. The default value is 0.5.

thicknessField

A String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*=ANALYTICAL_FIELD or *thicknessType*=DISCRETE_FIELD. The default value is an empty string.

Return value

A MembraneSection object.

Exceptions

RangeError and InvalidNameError.

46.16.2 setValues(...)

This method modifies the MembraneSection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MembraneSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.16.3 Members

The **MembraneSection** object has members with the same names and descriptions as the arguments to the **MembraneSection** method.

In addition, the **MembraneSection** object can have the following member:

rebarLayers

A **RebarLayers** object specifying reinforcement properties.

46.16.4 Corresponding analysis keywords

*MEMBRANE SECTION

46.17 MPCSection object

The **MPCSection** object defines the properties of a multi-point constraint section.

The **MPCSection** object is derived from the **Section** object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.17.1 MPCSection(...)

This method creates a **MPCSection** object.

Path

`mdb.models[name].MPCSection`
`session.odbs[name].MPCSection`

Required arguments

name

A String specifying the repository key.

mpcType

A SymbolicConstant specifying the MPC type of the section. Possible values are BEAM_MPC, ELBOW_MPC, PIN_MPC, LINK_MPC, TIE_MPC, and USER_DEFINED.

Optional arguments

userMode

A SymbolicConstant specifying the mode of the MPC when it is user-defined. Possible values are DOF_MODE and NODE_MODE. The default value is DOF_MODE.

The *userMode* argument applies only when *mpcType*=USER_DEFINED.

userType

An Int specifying to differentiate between different constraint types in a user-defined MPCSection. The default value is 0.

The *userType* argument applies only when *mpcType*=USER_DEFINED.

Return value

A MPCSection object.

Exceptions

RangeError and InvalidNameError.

46.17.2 Members

The MPCSection object has members with the same names and descriptions as the arguments to the `MPCSection` method.

46.17.3 Corresponding analysis keywords

*MPC

46.18 PEGSection object

The PEGSection object defines the properties of a solid section.

The PEGSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.18.1 PEGSection(...)

This method creates a PEGSection object.

Path

```
mdb.models[name].PEGSection
session.odbs[name].PEGSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material.

Optional arguments

thickness

A Float specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

wedgeAngle1

A Float specifying the value of the x component of the angle between the bounding planes, $\Delta\phi_x$. The default value is 0.0.

wedgeAngle2

A Float specifying the value of the y component of the angle between the bounding planes, $\Delta\phi_y$. The default value is 0.0.

Return value

A PEGSection object.

Exceptions

InvalidNameError and RangeError.

46.18.2 setValues(...)

This method modifies the PEGSection object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PEGSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.18.3 Members

The PEGSection object has members with the same names and descriptions as the arguments to the **PEGSection** method.

46.18.4 Corresponding analysis keywords

*SOLID SECTION

46.19 RebarLayers object

The RebarLayers object defines the rebar properties of a section.

Access

`import section`

SECTION COMMANDS

```
mdb.models[name].parts[name].compositeLayups[i].section.rebarLayers
mdb.models[name].sections[name].rebarLayers
import odbSection
session.odbs[name].sections[name].rebarLayers
```

46.19.1 RebarLayers(...)

This method creates a RebarLayers object.

Path

```
mdb.models[name].parts[name].compositeLayups[i].section.RebarLayers
mdb.models[name].sections[name].RebarLayers
session.odbs[name].sections[name].RebarLayers
```

Required arguments

rebarSpacing

A SymbolicConstant specifying the type of rebar geometry. Possible values are CONSTANT, ANGULAR, and LIFT_EQUATION.

layerTable

A LayerPropertiesArray object specifying the layers of reinforcement.

Optional arguments

None.

Return value

A RebarLayers object.

Exceptions

None.

46.19.2 setValues(...)

This method modifies the RebarLayers object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RebarLayers** method.

Return value

None

Exceptions

None.

46.19.3 Members

The `RebarLayers` object has members with the same names and descriptions as the arguments to the `RebarLayers` method.

46.19.4 Corresponding analysis keywords

*REBAR LAYER

46.20 SectionLayer object

The `SectionLayer` object defines the material layer in a composite shell.

Access

```
import section
mdb.models[name].parts[name].compositeLayups[i].section.layup[i]
mdb.models[name].sections[name].layup[i]
import odbSection
session.odbs[name].sections[name].layup[i]
```

46.20.1 SectionLayer(...)

This method creates a `SectionLayer` object.

Path

```
section.SectionLayer
odbSection.SectionLayer
```

SECTION COMMANDS

Required arguments

thickness

A Float specifying the thickness of the section layer.

material

A String specifying the name of the section layer material.

Optional arguments

orientAngle

A Float or a String specifying the relative orientation of the section layer. A Float specifies the angular orientation; a String specifies a user-subroutine orientation name. If a String is specified, a user-subroutine orientation is used, otherwise the Float value is used as an angular orientation. The default value is 0.0.

numIntPts

An Int specifying the number of integration points to be used through the section. This argument is valid only if the *preIntegrate* argument on the parent CompositeShellSection object is set to ON. The default value is 3.

axis

A SymbolicConstant specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. This only applies if a valid reference is provided for the orientation. Possible values are AXIS_1, AXIS_2, and AXIS_3. The default value is AXIS_3.

angle

A Float specifying the angle of the additional rotation. This only applies if a valid reference is provided for the orientation. The default value is 0.0.

additionalRotationType

A SymbolicConstant specifying the method used to describe the additional rotation when a valid orientation is specified. Possible values are ROTATION_NONE, ROTATION_ANGLE, and ROTATION_FIELD. The default value is ROTATION_NONE.

thicknessType

A SymbolicConstant specifying the method used to describe the thickness. Possible values are THICKNESS_MAGNITUDE and THICKNESS_DISCRETE_FIELD. The default value is THICKNESS_MAGNITUDE.

plyName

A String specifying the ply identifier for this section layer. The default value is "".

orientation

The SymbolicConstant None or a DatumCsys object specifying a coordinate system reference for the relative orientation of this layer. If this reference is valid it is used as the relative orientation of the layer, otherwise the *orientAngle* is used as described. The default value is None.

additionalRotationField

A String specifying the name of the field specifying the additional rotation. The default value is "".

thicknessField

A String specifying the name of the field specifying the thickness The default value is "".

Return value

A SectionLayer object.

Exceptions

None.

46.20.2 Members

The SectionLayer object has members with the same names and descriptions as the arguments to the `SectionLayer` method.

46.20.3 Corresponding analysis keywords

*SHELL SECTION
*SHELL GENERAL SECTION

46.21 ShellSection object

The ShellSection object defines the properties of a shell section. The ShellSection object is derived from the Section object. The ShellSection object has no explicit constructor and no methods or members.

The ShellSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.21.1 Members

The ShellSection object can have the following members:

SECTION COMMANDS

name

A String specifying the repository key.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

46.22 SolidSection object

The SolidSection object defines the properties of a solid section. The SolidSection object has no explicit constructor, no members, and no methods. The SolidSection object is an abstract base type.

The SolidSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.22.1 Members

The SolidSection object has the following member:

name

A String specifying the repository key.

46.23 SurfaceSection object

The SurfaceSection object defines the properties of a surface section.

The SurfaceSection object is derived from the Section object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.23.1 SurfaceSection(...)

This method creates a SurfaceSection object.

Path

`mdb.models[name].SurfaceSection`
`session.odbs[name].SurfaceSection`

Required argument

name

A String specifying the repository key.

Optional arguments

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is OFF.

density

A Float specifying the value of density to apply to this section. The default value is 0.0.

Return value

A SurfaceSection object.

Exceptions

RangeError and InvalidNameError.

46.23.2 Members

The SurfaceSection object has members with the same names and descriptions as the arguments to the **SurfaceSection** method.

In addition, the SurfaceSection object can have the following member:

rebarLayers

A RebarLayers object specifying reinforcement properties.

46.23.3 Corresponding analysis keywords

*SURFACE SECTION

46.24 TransverseShearBeam object

The TransverseShearBeam object defines the transverse shear stiffness properties of a beam section.

Access

```
import section
mdb.models[name].sections[name].beamTransverseShear
import odbSection
session.odbs[name].sections[name].beamTransverseShear
```

46.24.1 TransverseShearBeam(...)

This method creates a TransverseShearBeam object.

Path

```
mdb.models[name].sections[name].TransverseShearBeam
session.odbs[name].sections[name].TransverseShearBeam
```

Required argument

scfDefinition

A SymbolicConstant specifying how slenderness compensation factor of the section is given. Possible values are ANALYSIS_DEFAULT, COMPUTED, and VALUE.

Optional arguments

k23

None or a Float specifying the k23 shear stiffness of the section. The default value is **None**.

k13

None or a Float specifying the k13 shear stiffness of the section. The default value is **None**.

slendernessCompensation

The SymbolicConstant COMPUTED or a Float specifying the slenderness compensation factor of the section. The default value is 0.25.

Return value

A TransverseShearBeam object.

Exceptions

None.

46.24.2 setValues(...)

This method modifies the TransverseShearBeam object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `TransverseShearBeam` method.

Return value

None

Exceptions

None.

46.24.3 Members

The `TransverseShearBeam` object has members with the same names and descriptions as the arguments to the `TransverseShearBeam` method.

46.24.4 Corresponding analysis keywords

*TRANSVERSE SHEAR STIFFNESS

46.25 TransverseShearShell object

The `TransverseShearShell` object defines the transverse shear stiffness properties of a shell section.

Access

```
import section
mdb.models[name].sections[name].transverseShear
import odbSection
session.odbs[name].sections[name].transverseShear
```

46.25.1 TransverseShearShell(...)

This method creates a `TransverseShearShell` object.

Path

```
mdb.models[name].sections[name].TransverseShearShell  
session.odbs[name].sections[name].TransverseShearShell
```

Required arguments

k11

A Float specifying the shear stiffness of the section in the first direction.

k22

A Float specifying the shear stiffness of the section in the second direction.

k12

A Float specifying the coupling term in the shear stiffness of the section.

Optional arguments

None.

Return value

A TransverseShearShell object.

Exceptions

None.

46.25.2 **setValues(...)**

This method modifies the TransverseShearShell object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TransverseShearShell** method.

Return value

None

Exceptions

None.

46.25.3 Members

The `TransverseShearShell` object has members with the same names and descriptions as the arguments to the `TransverseShearShell` method.

46.25.4 Corresponding analysis keywords

*TRANSVERSE SHEAR STIFFNESS

46.26 TrussSection object

The `TrussSection` object defines the properties of a truss section.

The `TrussSection` object is derived from the `Section` object.

Access

```
import section
mdb.models[name].sections[name]
import odbSection
session.odbs[name].sections[name]
```

46.26.1 TrussSection(...)

This method creates a `TrussSection` object.

Path

```
mdb.models[name].TrussSection
session.odbs[name].TrussSection
```

Required arguments

name

A String specifying the repository key.

material

A String specifying the name of the material.

SECTION COMMANDS

Optional argument

area

A Float specifying the cross-sectional area for the section. Possible values are *area* > 0. The default value is 1.0.

Return value

A TrussSection object.

Exceptions

RangeError and InvalidNameError.

46.26.2 **setValues(...)**

This method modifies the TrussSection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TrussSection** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

46.26.3 **Members**

The TrussSection object has members with the same names and descriptions as the arguments to the **TrussSection** method.

46.26.4 **Corresponding analysis keywords**

*SOLID SECTION

47. Session commands

Session commands are used to create objects that are not stored with the model; for example, viewports and display groups. Abaqus/CAE retains Session objects only for the duration of the session; they are not saved when the model database is saved.

47.1 Session object

The Session object has no constructor. Abaqus creates the *session* member when a session is started.

Access

`session`

47.1.1 setValues(...)

This method modifies the Session object.

Arguments

Required arguments

None.

Optional argument

kernelMemoryLimit

A Float specifying the memory limit value for the Abaqus/CAE kernel process in megabytes. If the limit is exceeded, Abaqus/CAE displays an error message.

The default memory limit value for Windows 32-bit systems if not set is 1800 MB. Increasing the memory limit is not recommended unless you are using a Windows 32-bit system with the boot option `/3GB /userva=SizeInMBytes` to extend the amount of memory available for Abaqus/CAE. In this case the limit can be changed to 2800 MB.

If the kernel memory size reaches the `abq_ker_memory` value or the virtual memory limit of the machine, the following message will be displayed:

Operation did not complete due to a memory allocation failure.

For optimal performance, the memory limit should be set to a value less than the physical amount of memory on the machine.

The minimum setting allowed is 256 MB.

Return value

None

Exceptions

None.

47.1.2 enableCADConnection(...)

This method enables an associative import CAD connection for the specified *CADName*.

Arguments**Required argument**

CADName

A String specifying the CAD system. Available options are Pro/ENGINEER, CATIA V5, CATIA V6, NX and SolidWorks.

Optional argument

portNum

An Integer specifying the port number to be used by the CAD system to communicate with Abaqus/CAE. If unspecified, attempts will be made to identify an open port.

Return value

The connection port number used for the CAD connection.

Exceptions

None.

47.1.3 isCADConnectionEnabled()

This method checks the status of CAD Connection.

Arguments

None.

Return value

A Boolean value of True if the CAD connection enabled and False if the CAD connection disabled.

Exceptions

None.

47.1.4 disableCADConnection(...)

This method disables an associative import CAD connection that was enabled.

Arguments**Required argument**

CADName

A String specifying the CAD system for which associative import will be disabled. Available options are Pro/ENGINEER, CATIA V5, CATIA V6, NX and SolidWorks.

Optional arguments

None.

Return value

None

Exceptions

None.

47.1.5 enableParameterUpdate(...)

This method enables an associative import CAD connection with parameters for the specified *CADName*.

Arguments**Required arguments**

CADName

A String specifying the CAD system for which parameter update will be enabled. Available option is Pro/ENGINEER.

CADVersion

A String specifying the CAD Version. Allowable options are Wildfire2 and Wildfire3.

Optional argument

CADPort

An Integer specifying the port number to be used by Abaqus/CAE to communicate with the CAD system. If unspecified, attempts will be made to identify an open port. This port number is not the same as the *portNum* used by the associative import interface.

Return value

None

Exceptions

None.

47.1.6 setCADPortNumber(...)

For the given CAD system, this method saves the port number in Abaqus/CAE. This port number is used to send the parameter information to the CAD system.

Arguments

Required arguments

CADName

A String specifying the CAD system for which the port number will be saved. The available options are 'CATIA V5' and 'CATIA V6'.

Port

An integer specifying the port number to be used by Abaqus/CAE to send the modified parameters to the CAD system.

Optional arguments

None.

Return value

None

Exceptions

None.

47.1.7 updateCADParameters(...)

This method updates the parameters for the specified model using the specified parameter file.

Arguments

Required arguments

modelName

A String specifying the model name to update.

CADName

A String specifying the CAD system for which Abaqus updates the parameters. The available options are 'Pro/ENGINEER', 'CATIA V5' and 'CATIA V6'.

parameterFile

A parameter file containing the parameters that were exposed in the CAD system using the 'ABQ_' prefix.

CADPartFile

A file name specifying the CAD part file for which parameter update is triggered.

For *CADName*='CATIA V5' or 'CATIA V6', you can specify either products or parts using this argument. If you specify a product, Abaqus updates all of the parts associated with that product.

For *CADName*='Pro/ENGINEER', this argument is optional, and you can specify update for parts only. However, a single file can be associated with multiple parts in the case of family tables. In this case, Abaqus updates all listed parts.

Optional arguments*CADPartName*

An String specifying the part name to update. This part name should match the part name in the originating CAD system.

If you specify neither *CADPartFile* nor *CADPartName* during an update in which you specified *CADName*='Pro/ENGINEER', Abaqus updates all of the parts in the specified file.

Return value

None

Exceptions

None.

47.1.8 disableParameterUpdate(...)

This method disables an associative CAD connection using parameters.

Arguments**Required argument***CADName*

A String specifying the CAD system for which the parameter update will be disabled. Available option is Pro/ENGINEER.

Optional arguments

None.

Return value

None

Exceptions

None.

47.1.9 **printToFile(...)**

This method prints canvas objects to a file using the attributes stored in the `PrintOptions` object and the appropriate format options object.

Arguments

Required argument

fileName

A String specifying the file to which the image is to be written. If no file extension is supplied, an extension is added based on the selected image format (**.ps**, **.eps**, **.png**, **.tif**, **.svg**, or **.svgz**).

Optional arguments

format

A `SymbolicConstant` specifying the image format. Possible values are `PS`, `EPS`, `PNG`, `TIFF`, and `SVG`. The default value is `PS`.

canvasObjects

A sequence of canvas objects (viewports, text strings, or arrows) to print. The default is to print all canvas objects.

compression

A Boolean specifying the format for an SVG file. It is only valid to use this argument when *format* is `SVG`. Possible values are `False` (Uncompressed) and `True` (Compressed).

Return value

None

Exceptions

None.

47.1.10 **printToPrinter(...)**

This method prints canvas objects to a Windows printer or to a PostScript printer. The attributes used for printing to a Windows printer are stored in the `PrintOptions` object and the `PageSetupOptions` object; the

attributes used for printing to a PostScript printer are stored in the `PrintOptions` object and the `PsOptions` object.

Arguments

Required arguments

None.

Optional arguments

printCommand

A String specifying the operating system command or printer name to issue for printing to the printer. The default value is “lpr” or the value specified by the `printOptions` method. If you create a script to print directly to a Windows printer, the *printCommand* must take the following form:

```
session.printToPrinter.setValues(printCommand='PRINTER[
                                number of characters in name
                                ] :
                                printername
                                PROPERTIES [
                                number of characters in properties
                                ] :
                                document properties
                                ')
```

The **PROPERTIES** is a list of characters that represents the printing preferences for the selected Windows printer. The properties are not required in a script; the printed output will use the current settings for the selected printer. You can use '**PRINTER[7] : DEFAULT**' to specify the default Windows printer.

numCopies

An Int specifying the number of copies to print. Possible values are $1 \leq \text{numCopies} \leq 100$. The default value is 1.

canvasObjects

A sequence of canvas objects (viewports, text strings, or arrows) to print. The default is to print all canvas objects.

Return value

None

Exceptions

If *printCommand* is invalid:

SESSION COMMANDS

SystemError: invalid print command

If the print command fails:

SystemError: print command failed

If *numCopies* is out of range:

RangeError: numCopies must be in the range $1 \leq \text{value} \leq 100$

If *compression* is specified when *format* is not SVG :

TypeError: keyword error on compression

47.1.11 saveOptions(...)

This method saves your customized display settings.

Arguments

Required argument

directory

A SymbolicConstant specifying the directory in which Abaqus saves the file that will be used to restore your customized settings (**abaqus_2016.gpr**). Possible values are HOME and CURRENT.

Optional arguments

None.

Return value

None

Exceptions

None.

47.1.12 writeVrmlFile(...)

This method exports the current viewport objects to a file.

Arguments

Required argument

fileName

A String specifying the file to which the graphics data is to be written. If no file extension is supplied, an extension is added based on the selected format (**.wrl**, **.wrz**).

Optional arguments*format*

A Boolean specifying the format. Possible values are False (Uncompressed) and True (Compressed).

canvasObjects

A sequence of canvas objects (viewports, text strings, or arrows) to export.

Return value

None

Exceptions

None.

47.1.13 write3DXMLFile(...)

This method exports the current viewport objects to a file.

Arguments**Required argument***fileName*

A String specifying the file to which the graphics data is to be written. If no file extension is supplied, (.3dxml) will be added.

Optional arguments*format*

A Boolean specifying the format. Possible values are False (Uncompressed) and True (Compressed).

canvasObjects

A sequence of canvas objects to export.

Return value

None

Exceptions

None.

47.1.14 writeOBJFile(...)

This method exports the current viewport objects to a file.

Arguments

Required argument

fileName

A String specifying the file to which the graphics data is to be written. If no file extension is supplied, (.obj) will be added.

Optional argument

canvasObjects

A sequence of canvas objects to export.

Return value

None

Exceptions

None.

47.1.15 Members

The Session object can have the following members:

attachedToGui

A Boolean specifying whether an Abaqus interactive session is running.

replayInProgress

A Boolean specifying whether Abaqus is executing a replay file.

kernelMemoryFootprint

A Float specifying the memory usage value for the Abaqus/CAE kernel process in megabytes.

kernelMemoryMaxFootprint

A Float specifying the maximum value for the memory usage for the Abaqus/CAE kernel process in megabytes.

kernelMemoryLimit

A Float specifying the limit for the memory use for the Abaqus/CAE kernel process in megabytes.

colors

A repository of Color objects.

journalOptions

A JournalOptions object specifying how to record selection of geometry in the journal and replay files.

memoryReductionOptions

A MemoryReductionOptions object specifying options for running in reduced memory mode.

nodeQuery

A NodeQuery object specifying nodes and their coordinates in a path.

sketcherOptions

A ConstrainedSketcherOptions object specifying common options for all sketches.

viewerOptions

A ViewerOptions object.

animationController

An AnimationController object.

aviOptions

An AVIOptions object.

imageAnimationOptions

An ImageAnimationOptions object.

imageAnimation

An ImageAnimation object.

quickTimeOptions

A QuickTimeOptions object.

viewports

A repository of Viewport objects.

customData

A RepositorySupport object.

defaultFieldReportOptions

A FieldReportOptions object.

defaultFreeBodyReportOptions

A FreeBodyReportOptions object.

fieldReportOptions

A FieldReportOptions object.

freeBodyReportOptions

A FreeBodyReportOptions object.

odbs

A repository of Odb objects.

scratchOdb

A repository of ScratchOdb objects.

defaultOdbDisplay

A DefaultOdbDisplay object.

defaultPlot

A DefaultPlot object.

defaultChartOptions

A DefaultChartOptions object.

odbData

A repository of OdbData objects.

SESSION COMMANDS

mdbData

A repository of MdbData objects.

paths

A repository of Path objects.

freeBodies

A repository of FreeBody objects.

streams

A repository of Stream objects.

spectrums

A repository of Spectrum objects.

currentProbeValues

A CurrentProbeValues object.

defaultProbeOptions

A ProbeOptions object.

probeOptions

A ProbeOptions object.

probeReport

A ProbeReport object.

defaultProbeReport

A ProbeReport object.

selectedProbeValues

A SelectedProbeValues object.

printOptions

A PrintOptions object.

epsOptions

An EpsOptions object.

pageSetupOptions

A PageSetupOptions object.

pngOptions

A PngOptions object.

psOptions

A PsOptions object.

svgOptions

A SvgOptions object.

tiffOptions

A TiffOptions object.

autoColors

An AutoColors object specifying the color palette to be used for color coding.

xyColors

An AutoColors object specifying the color palette to be used for XYCurve objects.

xyDataObjects

A repository of XYData objects.

curves

A repository of XYCurve objects.

xyPlots

A repository of XYPlot objects.

charts

A repository of Chart objects.

defaultXYReportOptions

An XYReportOptions object.

xyReportOptions

An XYReportOptions object.

views

A repository of View objects.

networkDatabaseConnectors

A repository of NetworkDatabaseConnector objects.

displayGroups

A repository of DisplayGroup objects.

graphicsInfo

A GraphicsInfo object.

defaultGraphicsOptions

A GraphicsOptions object.

graphicsOptions

A GraphicsOptions object.

defaultViewportAnnotationOptions

A ViewportAnnotationOptions object.

queues

A repository of Queue objects.

currentViewportName

A String specifying the name of the current viewport.

sessionState

A Dictionary object specifying the viewports and their associated models. The Dictionary key specifies the viewport name. The Dictionary value is a Dictionary specifying the model name.

images

A repository of Image objects.

SESSION COMMANDS

movies

A repository of Movie objects.

defaultLightOptions

A LightOptions object.

drawingArea

A DrawingArea object.

defaultMesherOptions

A MesherOptions object specifying how to control default settings in the Mesh module.

drawings

A repository of Drawing objects.

47.2 AutoColors object

The AutoColors object defines a color palette.

Access

`session.autoColors`

47.2.1 setValues(...)

This method changes the color palette.

Arguments

Required argument

colors

A sequence of Strings specifying the colors of the palette. Strings must be named colors or represent red, green, blue components in hexadecimal form. For instance, '#FF0000' for red, '#00FF00' for green and '#0000FF' for blue

Optional arguments

None.

Return value

None

Exceptions

None.

47.2.2 Members

The AutoColors object has members with the same names and descriptions as the arguments to the `setValues` method.

47.3 Color object

The Color object contains the RGB definition of a system color.

Access

`session.colors[name]`

47.3.1 setByRGB(...)

This method changes the RGB value of a user-defined color. However, users cannot define colors, and this method does not modify system-defined colors.

Arguments

Required argument

rgb

A sequence of three Floats specifying the RGB value of the color. The Float values must be between 0.0 and 1.0.

Optional arguments

None.

Return value

None

Exceptions

None.

47.3.2 Members

The Color object has the following members:

name

A String specifying the name of the color.

rgb

A tuple of three Floats specifying the RGB value of the color. The Float values must be between 0.0 and 1.0.

47.4 Drawing object

A drawing is the container for a geometric object. The Drawing object stores the vertex data and various settings that determine how the drawing will be displayed.

Access

`session.drawings [name]`

47.4.1 Drawing(...)

This method creates an empty Drawing object.

Path

`session.Drawing`

Required argument

name

A String specifying the repository key.

Optional arguments

None.

Return value

A Drawing object.

Exceptions

ValueError.

If the user attempts to create a new drawing with the name of an existing drawing:

ValueError: There is already a drawing with this name

47.4.2 setVertices(...)

This method accepts the vertex data that defines the Drawing object. It defines in an array of vertices with a length equal to the length of the *vertexData* sequence divided by *vertexDimension*.

Arguments**Required arguments***vertexDimension*

An Integer in the range of 2 to 4 specifying how many Float values are needed to compose a single vertex.

vertexData

A sequence of Float values that will be used to compose the vertices. There must be *vertexDimension* values in the sequence for each vertex.

Optional arguments

None.

Return value

The number of vertices described.

Exceptions

RangeError.

ValueError.

If an invalid *vertexDimension* is specified:

RangeError: *vertexDimension* must be in the range $2 \leq \text{value} \leq 4$

If *vertexData* is an empty sequence:

ValueError: *vertexData* cannot be empty.

If **setVertices** has already been called and this call is sending fewer vertices:

ValueError: *vertexData* cannot be reduced

47.4.3 setNormals(...)

This method accepts the normal data for each vertex. It defines in an array of normal vectors with a length equal to the length of the *normalData* sequence divided by 3.

Arguments**Required argument***normalData*

A sequence of Float values that will be used to compose the normals. There must be 3 values in the sequence for each normal.

If only one normal is specified, all vertices will use the normal value.

Optional arguments

None.

Return value

The number of normals described.

Exceptions

RangeError.

ValueError.

If *normalData* is sequence with less than 3 values:

ValueError: *normalData* must have at least three values.

If **setNormals** has already been called and this call is sending fewer values:

ValueError: *normalData* cannot be reduced.

47.4.4 **setColors(...)**

This method accepts the color data for each vertex. It defines in an array of colors with a length equal to the length of the *colorData* sequence divided by *colorDimension*.

Arguments

Required arguments

colorDimension

An Integer in the range of 3 to 4 specifying how many Float values are needed to compose a single color.

colorData

A sequence of Float values in the range of 0.0 to 1.0 that will be used to compose the colors. There must be *colorDimension* values in the sequence for each color. The first color will be associated with the first vertex and so on.

The first Float will be the red value for the first color. The second Float will be the green value and the third will be the blue value. When *colorDimension* is 4 the 4th Float will be the alpha value for the first color but is ignored.

If only one color is specified, all vertices will use the color value.

Optional arguments

None.

Return value

The number of colors described.

Exceptions

RangeError.

ValueError.

If an invalid *colorDimension* is specified:

RangeError: *colorDimension* must be in the range $3 \leq \text{value} \leq 4$

If *colorData* is an empty sequence:

ValueError: *colorData* cannot be empty.

If **setColors** has already been called and this call is sending fewer colors:

ValueError: *colorData* cannot be reduced

47.4.5 setEdgeColor(...)

This method allows a separate, single color to be used when rendering the edges of the drawing. Once called, edges will be rendered using the specified color but facets will continue to use the colors specified in the **setColors** method. An empty sequence can be specified to resume using the colors arrays for edges.

Arguments

Required argument

edgeColor

A sequence of 0 or 3 Float values in the range of 0.0 to 1.0 that will be used to compose the edge color.

If the initial Float value is -1, the viewport background color will be used for the edge color.

Optional arguments

None.

Return value

None

Exceptions

ValueError.

If *edgeColor* is not a sequence of 0 or 3 Floats:

ValueError: *edgeColor* must be a tuple with 3 values

47.4.6 setPointColor(...)

This method allows a separate, single color to be used when rendering the points of the drawing. Once called, points will be rendered using the specified color but facets will continue to use the colors specified in the **setColors** method. An empty sequence can be specified to resume using the colors arrays for points.

Arguments

Required argument

pointColor

A sequence of 0 or 3 Float values in the range of 0.0 to 1.0 that will be used to compose the point color.

If the initial Float value is -1, the viewport background color will be used for the edge color.

Optional arguments

None.

Return value

None

Exceptions

ValueError.

If *pointColor* is a not sequence of 0 or 3 Floats:

ValueError: *pointColor* must be a tuple with 3 values

47.4.7 **addArrayDraw(...)**

This method adds a rendering command to the drawing and can be called multiple times to add additional rendering commands. When the drawing is referenced by a Viewport , the drawing commands are used the render the Drawing.

The rendering command constructs the specified type of geometric primitive using *numVertices* array elements starting at element index *startIndex*.

Arguments

Required arguments

type

A SymbolicConstant specifying the type of graphics primitive rendered by this command. Possible values are POINTS, LINES, LINE_LOOP, LINE_STRIP, TRIANGLES, TRIANGLE_STRIP, TRIANGLE_FAN, QUADS, and QUAD_STRIP.

startIndex

An Integer specifying the index of the first vertex to render.

numVertices

An Integer specifying the total number of vertices to render.

Optional argument*polygonMode*

A SymbolicConstant specifying how polygonal graphics primitives will be rendered by this command. Possible values are FILL, EDGES, and POINTS. The default value is FILL.

Return value

The total number of rendering commands that have been specified.

Exceptions

ValueError.

If $(startIndex + numVertices - 1)$ is larger than the length of the vertex array:

ValueError: Drawing request extends past array size of vertices.

If $(startIndex + numVertices - 1)$ is larger than the length of the normal array and normals are required for the graphics primitive:

Drawing request extends past array size of normals.

If $(startIndex + numVertices - 1)$ is larger than the length of the color array and vertex colors are required for the graphics primitive:

Drawing request extends past array size of colors.

47.4.8 addIndexDraw(...)

This method adds a rendering command to the drawing and can be called multiple times to add additional rendering commands. When the drawing is referenced by a Viewport, the drawing commands are used to render the Drawing.

The rendering command constructs the specified type of geometric primitive using *numVertices* array elements starting at element index *startIndex*.

Arguments**Required arguments***type*

A SymbolicConstant specifying the type of graphics primitive rendered by this command. Possible values are POINTS, LINES, LINE_LOOP, LINE_STRIP, TRIANGLES, TRIANGLE_STRIP, TRIANGLE_FAN, QUADS, and QUAD_STRIP.

indices

A sequence of Integer values specifying index of each vertex to render.

Optional argument*polygonMode*

A SymbolicConstant specifying how polygonal graphics primitives will be rendered by this command. Possible values are FILL, EDGES, and POINTS. The default value is FILL.

Return value

The total number of rendering commands that have been specified.

Exceptions

`ValueError`.

If any value in the *indices* sequence negative:

`ValueError`: Index values must be positive.

If any value in the *indices* sequence is larger than the length of the vertex array:

`ValueError`: Drawing request extends past array size of vertices.

If any value in the *indices* sequence is larger than the length of the normal array and normals are required for the graphics primitive:

Drawing request extends past array size of normals.

If any value in the *indices* sequence is larger than the length of the color array and vertex colors are required for the graphics primitive:

Drawing request extends past array size of colors.

47.4.9 **setValues(...)**

This method modifies the rendering of the Drawing object.

Arguments

Required arguments

None.

Optional arguments

show

A Boolean specifying whether the drawing object will be rendered when referenced. The default value is OFF.

cullBackfaces

A Boolean specifying whether polygonal graphics primitives facing away from the viewer should be culled (not rendered). The default value is OFF.

The winding order, and not the normal, of the graphics primitive is used to determine its facing.

frontFaceOrder

A SymbolicConstant specifying the winding order for polygonal graphics primitives that face the viewer. Possible values are:

- CCW, specifying front face winding order is counter-clockwise.
- CW, specifying front face winding order is clockwise.

The default value is CCW.

smoothShade

A Boolean specifying whether the lighting of polygonal graphics primitives is consistent for each facet or calculated for each displayed pixel. The default value is ON.

When False, only the last normal for each facet will be used in the lighting calculation.

edgesInShaded

A Boolean specifying whether edge and point drawing commands will be issued in a FILLED or SHADED display. The default value is ON.

If no edge or point drawing commands have been defined, the polygonal drawing commands will be issued in WIREFRAME and HIDDEN_LINE displays with the *polygonMode* set to EDGES. If only edge and point drawing commands have been defined, the Drawing will not be rendered in FILLED or SHADED displays.

translucency

A Float specifying the opacity for polygonal graphics primitives. Possible values are $0.0 \leq translucency \leq 1.0$ with 0.0 being completely transparent (invisible) and 1.0 being opaque. The default value is 1.0.

A value greater than 0.3 will cause the translucent facets to be sorted by depth before being rendered and has the side effect of disabling two-sided lighting for those facets.

lineSize

A Float specifying the width of the line, in millimeters, used to render edges. Possible values are $0.0 \leq lineSize \leq 5.0$ with 0.0 being interpreted as the thinnest possible line. The default value is 0.0.

A value of 0.0 will be one pixel on the output device. One pixel on the screen is generally visible but one pixel on a 1200 DPI printer may not be clear.

pointSize

A Float specifying the width of points, in millimeters, used to render points. Possible values are $0.0 \leq lineSize \leq 5.0$ with 0.0 being interpreted as the smallest possible point. The default value is 0.0.

A value of 0.0 will be one pixel on the output device. One pixel on the screen is generally visible but one pixel on a 1200 DPI printer may not be clear.

Return value

None

Exceptions

RangeError.

If an invalid *translucency* value is specified:

RangeError: *translucency* must be in the range $0.0 \leq value \leq 1.0$

If an invalid *lineSize* value is specified:

RangeError: *lineSize* must be in the range $0.0 \leq \text{value} \leq 5.0$

If an invalid *pointSize* value is specified:

RangeError: *pointSize* must be in the range $0.0 \leq \text{value} \leq 5.0$

47.4.10 Members

The Drawing object has members with the same names and descriptions as the arguments to the **Drawing** method.

In addition, the Drawing object has the following members:

vertexCount

An Int specifying the number of vertices in the vertex array after a call to the **setVertices** method. The default value is 0.

normalCount

An Int specifying the number of normal vectors in the normal array after a call to the **setNormals** method. The default value is 0.

colorCount

An Int specifying the number of colors in the color array after a call to the **setColors** method. The default value is 0.

show

A Boolean specifying whether the drawing object will be rendered when referenced. The default value is OFF.

cullBackfaces

A Boolean specifying whether polygonal graphics primitives facing away from the viewer should be culled (not rendered). The default value is OFF.

The winding order, and not the normal, of the graphics primitive is used to determine its facing.

frontFaceOrder

A SymbolicConstant specifying the winding order for polygonal graphics primitives that face the viewer. Possible values are:

- CCW, specifying front face winding order is counter-clockwise.
- CW, specifying front face winding order is clockwise.

The default value is CCW.

translucency

A Float specifying the opacity for polygonal graphics primitives. Possible values are $0.0 \leq \text{translucency} \leq 1.0$ with 0.0 being completely transparent (invisible) and 1.0 being opaque. The default value is 1.0.

A value greater than 0.3 will cause the translucent facets to be sorted by depth before being rendered and has the side effect of disabling two-sided lighting for those facets.

lineSize

A Float specifying the width of the line, in millimeters, used to render edges. Possible values are $0.0 \leq \text{lineSize} \leq 5.0$ with 0.0 being interpreted as the thinnest possible line. The default value is 0.0.

A value of 0.0 will be one pixel on the output device. One pixel on the screen is generally visible but one pixel on a 1200 DPI printer may not be clear.

pointSize

A Float specifying the width of points, in millimeters, used to render points. Possible values are $0.0 \leq \text{lineSize} \leq 5.0$ with 0.0 being interpreted as the smallest possible point. The default value is 0.0.

A value of 0.0 will be one pixel on the output device. One pixel on the screen is generally visible but one pixel on a 1200 DPI printer may not be clear.

smoothShade

A Boolean specifying whether the lighting of polygonal graphics primitives is consistent for each facet or calculated for each displayed pixel. The default value is ON.

When False, only the last normal for each facet will be used in the lighting calculation.

edgesInShaded

A Boolean specifying whether edge and point drawing commands will be issued in a FILLED or SHADED display. The default value is ON.

If no edge or point drawing commands have been defined, the polygonal drawing commands will be issued in WIREFRAME and HIDDEN_LINE displays with the *polygonMode* set to EDGES. If only edge and point drawing commands have been defined, the Drawing will not be rendered in FILLED or SHADED displays.

edgeColor

A tuple of three Floats specifying the *Red*, *Green*, and *Blue* component values for the edge color. Possible values for each component are between 0.0 and 1.0.

pointColor

A tuple of three Floats specifying the *Red*, *Green*, and *Blue* component values for the point color. Possible values for each component are between 0.0 and 1.0.

47.5 Image object

The Image object is used to store color values and attributes associated with a raster file. Upon creation, the Image object is added to the `session.images` repository.

Access

`session.images [name]`

47.5.1 Image(...)

This method creates an Image object from the contents of the specified file.

Path

`session.Image`

Required arguments

name

A String specifying the repository name for the image.

fileName

A String specifying the file from which the image is to be read. The file extension must be specified and indicate the image format (**.bmp**, **.gif**, **.jpg**, **.jpeg**, **.ico**, **.pcx**, **.png**, **.rgb**, **.tga**, **.tif**, or **.xpm**).

Optional arguments

None.

Return value

An Image object.

Exceptions

ValueError.

If *fileName* does not exist or can not be read:

ValueError: Unable to open image file

If *fileName* references an unsupported image file format:

ValueError: Unsupported image format

If the contents of *fileName* are corrupt or can not be decoded:

ValueError: Unable to decode image file

47.5.2 ImageFromMovie(...)

This method creates an Image object from a given frame of an existing Movie object.

Arguments

Required arguments

name

A String specifying the repository name for the image.

movieName

A String specifying the name of the movie from which the image is to be extracted. The movie must exist in the session.movies repository.

frame

An Int specifying the movie frame number defining the image to be extracted.

time

A Float specifying the time of the movie defining the image to be extracted.

Optional arguments

None.

Return value

An Image object.

Exceptions

ValueError.

TypeError.

If *movieName* does not exist:

ValueError: There is no movie object with this name: 'movieName'

If *frame* references an non existing frame:

ValueError: Could not load frame n from movie: 'movieName'

If *time* references an non existing frame:

ValueError: Could not load frame at time 't' from movie: 'movieName'

If *time* and *frame* are given in the same command:

TypeError: keyword error on time

47.5.3 Members

The Image object has members with the same names and descriptions as the arguments to the **Image** method.

47.6 JournalOptions object

A JournalOptions object specifies how to record selection of geometry in the journal and replay files. *journalOptions* can also be used to set the numeric formatting options for field report output, geometry commands output, and a default format for other numeric output. The JournalOptions object has no constructor. Abaqus creates the *journalOptions* member when a session is started.

Access

`session.journalOptions`

47.6.1 **setValues(...)**

This method modifies the JournalOptions object.

Arguments

Required arguments

None.

Optional arguments

replayGeometry

A SymbolicConstant specifying the format of the geometry in the replay file. Possible values are COORDINATE, INDEX, and COMPRESSEDINDEX. The default value is COMPRESSEDINDEX.

recoverGeometry

A SymbolicConstant specifying the format of the geometry in the recovery file. Possible values are COORDINATE, INDEX, and COMPRESSEDINDEX. The default value is COMPRESSEDINDEX.

defaultFormat

A NumberFormat object specifying the default format for numeric output. The default values are the same as the default values for the NumberFormat object.

fieldReportFormat

A NumberFormat object specifying the default format for numbers in a field report output. The default values are the same as the default values for the NumberFormat object.

geometryFormat

A NumberFormat object specifying the default format for numbers in geometry commands output. The default values are the same as the default values for the NumberFormat object.

Return value

None

Exceptions

None.

47.6.2 Members

The JournalOptions object has members with the same names and descriptions as the arguments to the `setValues` method.

47.7 MemoryReductionOptions object

The MemoryReductionOptions object controls the default settings that Abaqus/CAE uses for running in reduced memory mode. The MemoryReductionOptions object has no constructor. Abaqus creates the *MemoryReductionOptions* member when a session is started.

Access

`session.memoryReductionOptions`

47.7.1 setValues(...)

This method modifies the MemoryReductionOptions object.

Arguments

Required arguments

None.

Optional arguments

reducedMemoryMode

A Boolean specifying whether Abaqus/CAE should run in reduced memory mode. The default value is ON.

percentThreshold

A Float specifying the percent of *kernelMemoryLimit* at which the reduced memory mode starts. The default value is 75.0.

Return value

None

Exceptions

None.

47.7.2 Members

The `MemoryReductionOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

47.8 NetworkDatabaseConnector object

The `NetworkDatabaseConnector` object allows you to access an output database on a remote system.

Access

`session.networkDatabaseConnectors [name]`

47.8.1 NetworkDatabaseConnector(...)

This method creates a `NetworkDatabaseConnector` object that you can use to access a remote output database. You can create a network database connector from any platform: Windows or Linux. However, the network database connector server must reside on a Linux platform; you cannot access an output database that resides on a remote Windows system. You can access only a remote output database; you cannot access a remote model database.

Path

`session.NetworkDatabaseConnector`

Required arguments

name

A String specifying the repository key.

hostName

A String specifying the name of the remote computer.

directory

A String specifying the directory on the remote computer.

Optional arguments

remoteAbaqusDriverName

A String specifying the name of command to execute Abaqus/CAE on the remote computer.

remoteLoginMechanism

A `SymbolicConstant` specifying the remote shell command on the local system. Possible values are `RSH` and `SSH`. The default value is `SSH`.

sshPath

A String specifying the path to the **ssh** command on the local system. The default value is an empty string.

serverPort

An Int specifying the server port on the remote computer. If *serverPort* =0, the host and remote systems are allowed to establish their own port numbers. The default value is 0.

connectionPort

An Int specifying the connection port on the remote computer. The default value is 0.

serverTimeout

An Int specifying the timeout in seconds for the remote server. For example: 86400 corresponds to one day. The server exits if it does not receive any communication from the client during the time specified. The default value is 86400.

allowAutomaticStartup

A Boolean specifying whether to start the remote network database connector server. The default value is ON.

Return value

A NetworkDatabaseConnector object.

Exceptions

None.

47.8.2 start(...)

This method starts the remote network database connector server on the remote host.

Arguments**Required arguments**

None.

Optional arguments*serverPort*

An Int specifying the server port on the remote computer. If *serverPort* =0, the host and remote systems are allowed to establish their own port numbers. The default value is 0.

serverTimeout

An Int specifying the timeout in seconds for the remote server. For example: 86400 corresponds to one day. The server exits if it does not receive any communication from the client during the time specified. The default value is 86400.

Return value

None

Exceptions

None.

47.8.3 stop()

This method stops the remote network database connector server on the remote host.

Arguments

None.

Return value

None

Exceptions

None.

47.8.4 setValues(...)

This method modifies the `NetworkDatabaseConnector` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `NetworkDatabaseConnector` method, except for the *name* argument.

Return value

None

Exceptions

None.

47.8.5 Members

The `NetworkDatabaseConnector` object has members with the same names and descriptions as the arguments to the **`NetworkDatabaseConnector`** method.

In addition, the `NetworkDatabaseConnector` object has the following member:

connected

A Boolean specifying if the connection between the client and the server is established.

47.9 NumberFormat object

The `NumberFormat` object is a formatting template used to define formatting options for certain numeric output.

Access

```
import visualization
session.defaultFieldReportOptions.numberFormat
session.fieldReportOptions.numberFormat
session.journalOptions.defaultFormat
session.journalOptions.fieldReportFormat
session.journalOptions.geometryFormat
```

47.9.1 NumberFormat(...)

This method creates a `NumberFormat` object.

Path

```
session.defaultFieldReportOptions.NumberFormat
session.fieldReportOptions.NumberFormat
session.journalOptions.NumberFormat
```

Required arguments

None.

Optional arguments

blankPad

A Boolean specifying whether the printed digits should be padded with blank characters to ensure equal sized fields. The *blankPad* argument is useful when your printed output includes columns. The default value is ON.

SESSION COMMANDS

format

A SymbolicConstant specifying the formatting type. Possible values are ENGINEERING, SCIENTIFIC, and AUTOMATIC. The default value is ENGINEERING.

numDigits

An Int specifying the number of digits to be displayed in the result. *numDigits* > 0. The default value is 6.

precision

An Int specifying the number of decimal places to which the number is to be truncated for display. *precision* ≤ 0. If *precision* = 0, no truncation is applied. The default value is 0.

Return value

A NumberFormat object.

Exceptions

None.

47.9.2 Members

The NumberFormat object has members with the same names and descriptions as the arguments to the **NumberFormat** method.

48. Sketcher commands

48.1 ConstrainedSketch object

A `ConstrainedSketch` object contains the entities that are used to create a sketch. The objects include `ConstrainedSketchGeometry` objects contained in the Geometry Repository, such as `Line`, `Arc`, and `Spline`. `Vertex`, `Dimension`, `Constraint`, and `Parameter` objects are contained in their respective repositories.

Access

```
import sketch
mdb.models[name].sketches[name]
```

48.1.1 ConstrainedSketch(...)

This method creates a `ConstrainedSketch` object. If the sketch cannot be created, the method returns `None`.

Path

```
mdb.models[name].ConstrainedSketch
```

Required arguments

name

A String specifying the repository key.

sheetSize

A Float specifying the sheet size.

Optional arguments

gridSpacing

A Float specifying the spacing between gridlines. Possible values are Floats > 0 . The default value is approximately 2 percent of *sheetSize*.

transform

A sequence of sequences of Floats specifying the three-dimensional orientation of the sketch. The sequence is a 3×4 transformation matrix specifying the axis of rotation and the translation vector. Possible values are any Floats.

The default value for the axis of rotation is the identity matrix

```
(1.0, 0.0, 0.0), (0.0, 1.0, 0.0), (0.0, 0.0, 1.0)
```

SKETCHER COMMANDS

The default value for the translation vector is

`(0.0, 0.0, 0.0)`

The default values position the sketch on the *X–Y* plane centered at the origin.

Return value

A `ConstrainedSketch` object.

Exceptions

None.

48.1.2 `ConstrainedSketch(...)`

This method copies one `ConstrainedSketch` object to a new `ConstrainedSketch` object.

Note: If the name of the sketch to be copied to is `__edit__`, Abaqus creates an exact copy that contains both reference geometry and a non-identity transform matrix. Otherwise, the **`Sketch`** copy constructor strips the reference geometry from the copied sketch and sets the transform matrix to identity, creating a standalone copy.

Path

`mdb.models[name].ConstrainedSketch`

Required arguments

name

A String specifying the repository key.

objectToCopy

A `ConstrainedSketch` object to be copied.

Optional arguments

None.

Return value

A `ConstrainedSketch` object.

Exceptions

`InvalidNameError`.

48.1.3 ConstrainedSketchFromGeometryFile(...)

This method creates a `ConstrainedSketch` object and places it in the **sketches** repository.

Path

`mdb.models[name].ConstrainedSketchFromGeometryFile`

Required arguments

name

A String specifying the repository key.

geometryFile

An `AcisFile` object specifying a file containing geometry. The geometry in the file is converted to two-dimensional sketch geometry in the *X–Y* plane.

Optional arguments

None.

Return value

A `ConstrainedSketch` object.

Exceptions

`InvalidNameError`.

48.1.4 print()

This method prints the following statistics about a sketch:

- The sketch Id (a positive integer).
- The number of geometry curves (the number of `ConstrainedSketchGeometry` objects).
- The number of dimensions (the number of `ConstrainedSketchDimension` objects).
- The number of vertices (the number of `ConstrainedSketchVertex` objects).

Arguments

None.

Return value

None

Exceptions

None.

48.1.5 assignCenterline(...)

This method indicates the construction line that will be used as a centerline for revolved features.

Arguments

Required argument

line

A ConstrainedSketchGeometry object specifying a construction line that indicates the centerline of revolved features.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.6 assignCenterOfTwist(...)

This method indicates the isolated point that will be used as the center of twist when an extruded feature is created with twist.

Arguments

Required argument

point

A ConstrainedSketchVertex object specifying an isolated point that indicates the center of twist for extruded features that use a twist angle.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.7 autoDimension(...)

This method applies dimensions to the selected ConstrainedSketchGeometry objects in an effort to make the ConstrainedSketch well defined.

Arguments**Required argument**

objectList

A sequence specifying the ConstrainedSketchGeometry objects to dimension.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.8 autoTrimCurve(...)

This method automatically trims a selected ConstrainedSketchGeometry object at the specified location. If the object does not intersect other ConstrainedSketchGeometry objects, the entire selected object will be deleted.

Arguments**Required arguments**

curve1

The ConstrainedSketchGeometry object to be trimmed.

point1

A pair of Floats specifying the location on ConstrainedSketchGeometry where the trimming should be applied. *point1* and *parameter1* are mutually exclusive.

parameter1

A Float specifying the parameter location on the ConstrainedSketchGeometry where the trimming should be applied. *point1* and *parameter1* are mutually exclusive.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.9 breakCurve(...)

This method breaks a specified ConstrainedSketchGeometry object (*curve1*) using another specified ConstrainedSketchGeometry object (*curve2*). If the selected ConstrainedSketchGeometry objects intersect, then only *curve1* will be broken; *curve2* is not affected by the operation. The location for the break is determined by the specified point values.

Arguments

Required arguments

curve1

A ConstrainedSketchGeometry object specifying the object to be broken.

point1

A pair of Floats specifying the location on *curve1* near where the break should be applied.

curve2

A ConstrainedSketchGeometry object specifying where *curve1* should be broken.

point2

A pair of Floats specifying the location on *curve2* near where *curve1* should be broken.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.10 **copyMirror(...)**

This method creates copies of the given `ConstrainedSketchGeometry` objects, mirrors them about a selected line, and inserts them into the appropriate repositories of the `ConstrainedSketch` object.

Arguments

Required arguments

mirrorLine

A `ConstrainedSketchGeometry` object specifying the line about which Abaqus will mirror the sketch.

objectList

A sequence of `ConstrainedSketchGeometry` objects specifying the sketch to be copied and mirrored.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.11 **copyMove(...)**

This method creates copies of the given `ConstrainedSketchGeometry` objects, moves them from their original position, and inserts them into the appropriate repositories of the `ConstrainedSketch` object.

Arguments

Required arguments

vector

A sequence of two `Floats` specifying the translation vector.

objectList

A sequence of `ConstrainedSketchGeometry` objects to be copied and moved.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.12 **copyRotate(...)**

This method creates copies of the given `ConstrainedSketchGeometry` objects, rotates them, and inserts them into the appropriate repositories of the `ConstrainedSketch` object.

Arguments

Required arguments

centerPoint

A pair of Floats specifying the center of rotation.

angle

A Float specifying the angle of rotation in degrees.

objectList

A sequence of `ConstrainedSketchGeometry` objects to be copied and moved.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.13 **copyScale(...)**

This method creates copies of the given `ConstrainedSketchGeometry` objects, scales them by the specified value about a selected point, and inserts them into the appropriate repositories of the `ConstrainedSketch` object.

Arguments

Required arguments

scaleValue

A Float specifying the value for scaling.

scaleCenter

A pair of Floats specifying the center of scaling.

objectList

A sequence of ConstrainedSketchGeometry objects to be copied and scaled.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.14 delete(...)

This method deletes the given ConstrainedSketchGeometry , ConstrainedSketchDimension , or ConstrainedSketchConstraint objects.

Arguments

Required argument

objectList

A sequence of ConstrainedSketchGeometry , ConstrainedSketchDimension , or ConstrainedSketchConstraint objects to be deleted.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.15 deleteParameter(...)

The command deletes a specified parameter.

Arguments

Required argument

name

A String specifying the name of the parameter to delete.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.16 dragEntity(...)

This method drags a specified `ConstrainedSketchGeometry` or `ConstrainedSketchVertex` object to a specific location.

Arguments

Required arguments

entity

A `ConstrainedSketchGeometry` or `ConstrainedSketchVertex` object specifying the object to drag.

points

A sequence of sequences of three Floats specifying a sequence of points along which to drag the entity. The order of points in the sequence defines a path that determines the solution.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.17 linearPattern(...)

This method copies `ConstrainedSketchGeometry` objects in a linear pattern along one or two directions. This method also copies any associated dimension or constraint objects that exist between the given objects.

Arguments**Required arguments***number1*

An Integer specifying the total number of copies, including the original objects, that appear along the first direction in the pattern. Possible values are $1 \leq \textit{number1} \leq 1000$.

spacing1

A Float specifying the spacing between copies along the first direction in the pattern. Possible values are $0.0 \leq \textit{spacing1}$.

angle1

A Float specifying the angle in degrees of the first direction in the pattern. Possible values are $-360.0 \leq \textit{angle1} \leq 360.0$.

Optional arguments*vertexList*

A sequence of `ConstrainedSketchVertex` objects to copy.

geomList

A sequence of `ConstrainedSketchGeometry` objects to copy.

number2

An integer specifying the total number of copies, including the original objects, that appear along the second direction in the pattern. Possible values are $1 \leq \textit{number2} \leq 1000$. The default value is 1. The value of either *number1* or *number2* must be greater than one.

spacing2

A Float specifying the spacing between copies along the first direction in the pattern. Possible values are $0.0 \leq \textit{spacing2}$. The default value is *spacing1*.

angle2

A Float specifying the angle in degrees of the first direction in the pattern. Possible values are $-360.0 \leq \textit{angle2} \leq 360.0$. The default value is 90° beyond the value of *angle1*.

Return value**None****Exceptions**`AbaqusException`

Number must be greater than 1 for at least one direction

48.1.18 mergeVertices(...)

This method merges the `ConstrainedSketchVertex` objects that lie within the specified distance of each other. If only one `ConstrainedSketchVertex` object is selected, it will merge all `ConstrainedSketchVertex`

SKETCHER COMMANDS

objects that lie within the specified distance of that vertex. If more than one vertex is selected, the search will be restricted to only the selected `ConstrainedSketchVertex` objects.

Arguments

Required arguments

value

A Float specifying the search radius.

vertexList

A sequence of `ConstrainedSketchVertex` objects to be merged.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.19 **move(...)**

This method translates the given `ConstrainedSketchGeometry` objects by the given vector.

Arguments

Required arguments

vector

A sequence of two Floats specifying the translation vector.

objectList

A sequence of `ConstrainedSketchGeometry` objects specifying the objects to be translated.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.20 **offset(...)**

This method creates copies of the selected `ConstrainedSketchGeometry` objects, offsets them by the specified distance in the specified direction, and inserts them into the `ConstrainedSketch` object's appropriate repositories. If connected objects are selected, trim or extend is carried out to complete the offset.

Arguments

Required arguments

distance

A Float specifying the distance to be offset.

objectList

A sequence of `ConstrainedSketchGeometry` objects to be copied and offset.

side

A `SymbolicConstant` specifying which side the offset should occur. Possible values are `LEFT` and `RIGHT`.

Optional argument

filletCorners

A Boolean specifying whether the corners need to be rounded instead of being extended.

Return value

None

Exceptions

None.

48.1.21 **radialPattern(...)**

This method copies `ConstrainedSketchGeometry` objects in a radial pattern about a specified center point.

Arguments

Required arguments

number

An Int specifying the total number of copies, including the original objects, that appear in the radial pattern. Possible values are $2 \leq \text{number} \leq 1000$.

totalAngle

A Float specifying the total angle in degrees between the first and last instance in the pattern. A positive angle corresponds to a counter-clockwise direction. The values 360° and -360° represent

a special case where the pattern makes a full circle. In this case, because the copy would overlay the original, the copy is not placed at the last position. Possible values are $-360.0 \leq totalAngle \leq 360.0$.

centerPoint

A pair of Floats specifying the center of the radial pattern.

Optional arguments

vertexList

A sequence of ConstrainedSketchVertex objects to copy.

geomList

A sequence of ConstrainedSketchGeometry objects to copy.

Return value

None

Exceptions

None.

48.1.22 resetView()

This method resets the view to be perpendicular to the sketching plane.

Arguments

None.

Return value

None

Exceptions

None.

48.1.23 rectangle(...)

This method creates four lines that form a rectangle with diagonal corners defined by the given points and inserts them into the **geometry** repository of the ConstrainedSketch object.

Arguments**Required arguments***point1*

A pair of Floats specifying the first corner of the rectangle.

point2

A pair of Floats specifying the second corner of the rectangle.

Optional arguments

None.

Return value

An Int specifying the success or failure of the method. A value of 0 indicates failure.

Exceptions

None.

48.1.24 removeGapsAndOverlaps(...)

This method removes gaps and overlaps between sketch geometries specified by the user. This method is particularly useful when cleaning up imported sketches

Arguments**Required arguments***tolerance*

A float value which specifies the largest size of the gap or overlap between entities that is to be removed. Typically this value is small and is used to close gaps and overlaps which may not exist in the originating program but exist in the sketch because of mismatched tolerances between the two programs.

geomList

A sequence of ConstrainedSketchGeometry objects where the gaps and overlaps are to be removed.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.25 **repairShortEdges(...)**

This method deletes the short edges specified, optionally selecting only those short edges whose lengths are smaller than the specified tolerance and healing the resultant gap in the sketch. This method is particularly useful in conjunction with **removeGapsAndOverlap** when cleaning up imported sketches.

Arguments

Required argument

geomList

A sequence of ConstrainedSketchGeometry objects where the short edges are to be removed.

Optional argument

tolerance

A float value that is used to select and delete only those edges specified in *geomList* whose lengths are smaller than the given value. The default value is -1.0 . This value implies that all edges specified in *geomList* will be removed and the sketch healed to remove gaps left by their removal.

Return value

None

Exceptions

None.

48.1.26 **retrieveSketch(...)**

This method copies all ConstrainedSketchGeometry , ConstrainedSketchDimension , ConstrainedSketchConstraint , and ConstrainedSketchParameter objects from the specified ConstrainedSketch object. The new objects are added to the existing objects (if any). The objects in the specified ConstrainedSketch object are not modified by the retrieve operation.

Arguments

Required argument

sketch

A ConstrainedSketch object specifying the object from which to copy.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.27 rotate(...)

This method rotates the given `ConstrainedSketchGeometry` objects by the given angle and about the given point.

Arguments**Required arguments**

centerPoint

A pair of Floats specifying the center of rotation.

angle

A Float specifying the angle of rotation in degrees.

objectList

A sequence of `ConstrainedSketchGeometry` specifying the objects to be rotated.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.28 scale(...)

This method scales the given `ConstrainedSketchGeometry` objects by the given scale factor and about the given point.

Arguments**Required arguments**

scaleValue

A Float specifying the value of scale.

scaleCenter

A pair of Floats specifying the center of scale.

objectList

A sequence of ConstrainedSketchGeometry objects specifying the objects to be scaled.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.29 setPrimaryObject(...)

This method makes the ConstrainedSketch object the primary object in the current viewport. The sketch remains the primary object in the current viewport until an **unsetPrimaryObject** command is issued.

Arguments

Required argument

option

A SymbolicConstant specifying how the sketch is displayed. Possible values are:

- **STANDALONE**: Indicates a new stand-alone sketch. The current viewport is cleared and is replaced by the stand-alone sketch. The view direction is set to $-Z$.
- **SUPERIMPOSE**: Indicates that the sketch is superimposed on the current viewport. The view direction is changed to be perpendicular to the sketch plane. The change is effected smoothly as an animated sequence of many small viewing steps.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.30 trimExtendCurve(...)

This method trims or extends a specified ConstrainedSketchGeometry object (*curve1*) using another specified ConstrainedSketchGeometry object (*curve2*). *curve2* is not affected by the operation. The location for the trim or extend is determined by the specified point values.

Arguments

Required arguments

curve1

The ConstrainedSketchGeometry object specifying the object to be trimmed or extended.

point1

A pair of Floats specifying the location on *curve1* where trim or extend should be applied.

curve2

The ConstrainedSketchGeometry object specifying the object to which *curve1* is trimmed or extended. *curve2* is not trimmed or extended.

point2

A pair of Floats specifying the location on *curve2* near where *curve1* should be trimmed or extended.

Optional arguments

None.

Return value

None

Exceptions

None.

48.1.31 undo()

This method undoes the effects of the last ConstrainedSketch object method.

Arguments

None.

Return value

None

Exceptions

None.

48.1.32 **unsetPrimaryObject()**

This method removes the `ConstrainedSketch` object from the current viewport, reversing the effects of the **`setPrimaryObject`** command. If the *option* argument was set to `SUPERIMPOSE`, the viewport will be returned to the view orientation that was in place when the **`setPrimaryObject`** command was issued. If the *option* argument was set to `STANDALONE`, the viewport will be left empty.

Arguments

None.

Return value

None

Exceptions

None.

48.1.33 **writeAcisFile(...)**

This method exports the geometry of the sketch to a named file in ACIS format.

Arguments

Required argument

fileName

A String specifying the file name.

Optional argument

version

A Float specifying the ACIS version. For example, the Float “12.0” corresponds to ACIS Version 12.0. The default value is the current version of ACIS.

Return value

None

Exceptions

`InvalidNameError`.

48.1.34 **writelgesFile(...)**

This method exports the geometry of the sketch to a named file in IGES format.

Arguments

Required argument

filename

A String specifying the file name.

Optional argument

flavor

A SymbolicConstant specifying a particular flavor of IGES to export. Possible values are STANDARD, AUTOCAD, SOLIDWORKS, JAMA, and MSBO.

Return value

None

Exceptions

InvalidNameError.

48.1.35 **Members**

The ConstrainedSketch object can have the following members:

constraints

A repository of ConstrainedSketchConstraint objects.

dimensions

A repository of ConstrainedSketchDimension objects.

geometry

A ConstrainedSketchGeometryArray object specifying the sketch geometry, such as lines, arcs, circles, and splines.

parameters

A repository of ConstrainedSketchParameter objects specifying sketch parameters, which may be associated with dimensions.

sketchOptions

A ConstrainedSketchOptions object specifying the sketch option settings.

vertices

A ConstrainedSketchVertexArray object.

imageOptions

A ConstrainedSketchImageOptions object.

48.2 ConstrainedSketchConstraint object

The ConstrainedSketchConstraint object stores the constraints associated with a sketch.

Access

```
import sketch
mdb.models[name].sketches[name].constraints[i]
```

48.2.1 CoincidentConstraint(...)

This method creates a coincident constraint. This constraint applies to two vertices, to a vertex and a ConstrainedSketchGeometry object, or to two ConstrainedSketchGeometry objects of the same type and constrains them to be coincident.

Path

```
mdb.models[name].sketches[name].CoincidentConstraint
```

Required arguments

entity1

A ConstrainedSketchGeometry object or a Vertex object specifying the first object.

entity2

A ConstrainedSketchGeometry object or a Vertex object specifying the second object.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.2 ConcentricConstraint(...)

This method creates a concentric constraint. This constraint applies to any combination of circles, arcs, ellipses, and points and constrains them to be concentric. A concentric constraint implies that the center of ConstrainedSketchGeometry objects coincide.

Path

`mdb.models[name].sketches[name].ConcentricConstraint`

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first arc, circle, ellipse, or sketch vertex.

entity2

A ConstrainedSketchGeometry object specifying the second arc, circle, ellipse, or sketch vertex.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.3 EqualLengthConstraint(...)

This method creates an equal length constraint. This constraint applies to lines and constrains them such that their lengths are equal.

Path

`mdb.models[name].sketches[name].EqualLengthConstraint`

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first line.

entity2

A ConstrainedSketchGeometry object specifying the second line.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.4 EqualRadiusConstraint(...)

This method creates an equal radius constraint. This constraint applies to circles and arcs and constrains them such that their radii are equal.

Path

`mdb.models[name].sketches[name].EqualRadiusConstraint`

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first arc or circle.

entity2

A ConstrainedSketchGeometry specifying the second arc or circle.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.5 FixedConstraint(...)

This method creates a fixed constraint. This constraint applies to a ConstrainedSketchGeometry object or a Vertex object and constrains them to be fixed in space. Both the location and the shape of the sketch geometry is fixed.

Path

`mdb.models[name].sketches[name].FixedConstraint`

Required argument

entity

A ConstrainedSketchGeometry object or a Vertex object specifying the item to fix in space.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.6 HorizontalConstraint(...)

This method creates a horizontal constraint. This constraint applies to a line and constrains it to be horizontal.

Path

```
mdb.models[name].sketches[name].HorizontalConstraint
```

Required argument

entity

A ConstrainedSketchGeometry object specifying the line to constrain.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.7 VerticalConstraint(...)

This method creates a vertical constraint. This constraint applies to a line and constrains it to be vertical.

Path

```
mdb.models[name].sketches[name].VerticalConstraint
```

Required argument

entity

A ConstrainedSketchGeometry object specifying the line to constrain.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.8 ParallelConstraint(...)

This method creates a parallel constraint. This constraint applies to lines and constrains them to be parallel.

Path

`mdb.models[name].sketches[name].ParallelConstraint`

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first line.

entity2

A ConstrainedSketchGeometry object specifying the second line.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.9 PerpendicularConstraint(...)

This method creates a perpendicular constraint. This constraint applies to different types of ConstrainedSketchGeometry objects and constrains them to be perpendicular to each other.

Path

```
mdb.models[name].sketches[name].PerpendicularConstraint
```

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first object.

entity2

A ConstrainedSketchGeometry object specifying the second object.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.10 EqualDistanceConstraint(...)

This method creates an equal distance constraint. This constraint can be applied between a midpoint Vertex object and any other two Vertex objects or between a midpoint Vertex object and two ConstrainedSketchGeometry objects that are lines. The equal distance constraint forces the midpoint vertex to remain at an equal distance from the two other vertices or lines.

Path

```
mdb.models[name].sketches[name].EqualDistanceConstraint
```

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first line or Vertex object.

entity2

A ConstrainedSketchGeometry object specifying the second line or Vertex object.

midpoint

A Vertex object specifying the vertex that will be positioned an equal distance from *entity1* and *entity2*.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.11 TangentConstraint(...)

This method creates a tangent constraint. This constraint applies to different types of ConstrainedSketchGeometry objects and constrains them to remain tangential.

Path

`mdb.models[name].sketches[name].TangentConstraint`

Required arguments

entity1

A ConstrainedSketchGeometry object specifying the first object.

entity2

A ConstrainedSketchGeometry object specifying the second object.

Optional arguments

None.

Return value

A ConstrainedSketchConstraint object.

Exceptions

None.

48.2.12 Members

The ConstrainedSketchConstraint object has no members.

48.3 ConstrainedSketchDimension object

The ConstrainedSketchDimension object stores the dimensions associated with a sketch.

Access

```
import sketch
mdb.models[name].sketches[name].dimensions[i]
```

48.3.1 AngularDimension(...)

This method constructs a ConstrainedSketchDimension object between two ConstrainedSketchGeometry objects, with the given angle between them.

Path

```
mdb.models[name].sketches[name].AngularDimension
```

Required arguments

line1

A ConstrainedSketchGeometry object specifying the first line.

line2

A ConstrainedSketchGeometry object specifying the second line.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments

value

A Float specifying the angle between the two lines.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the angle between two lines.

Return value

A ConstrainedSketchDimension object (**None** if the dimension cannot be created).

Exceptions

None.

48.3.2 HorizontalDimension(...)

This method constructs a `ConstrainedSketchDimension` object between two vertices. A horizontal dimension indicates the horizontal distance along the *X*-axis between two vertices.

Path

`mdb.models[name].sketches[name].HorizontalDimension`

Required arguments

vertex1

A `ConstrainedSketchVertex` object specifying the first endpoint.

vertex2

A `ConstrainedSketchVertex` object specifying the second endpoint.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments

value

A Float distance between the two vertices.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the distance between the two vertices.

Return value

A `ConstrainedSketchDimension` object (**None** if the dimension cannot be created).

Exceptions

None.

48.3.3 ObliqueDimension(...)

This method constructs a `ConstrainedSketchDimension` object between two vertices. An oblique dimension indicates the distance between two vertices.

Path

`mdb.models[name].sketches[name].ObliqueDimension`

Required arguments*vertex1*

A ConstrainedSketchVertex object specifying the first endpoint.

vertex2

A ConstrainedSketchVertex object specifying the second endpoint.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments*value*

A Float specifying the distance between the two ConstrainedSketchVertex objects.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the distance between the two vertices.

Return valueA ConstrainedSketchDimension object (**None** if the dimension cannot be created).**Exceptions**

None.

48.3.4 RadialDimension(...)

This method constructs a ConstrainedSketchDimension object on a circular or elliptical arc. A radial dimension indicates the radius of an arc or circle or the major or minor radius of an ellipse.

Path`mdb.models[name].sketches[name].RadialDimension`**Required arguments***curve*

A ConstrainedSketchGeometry object specifying the circular or elliptical arc.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments*value*

A Float specifying the radius of the arc, circle or ellipse.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the angle between two lines.

majorRadius

A Float specifying the major Radius if *curve* is an ellipse. This is mutually exclusive with *value* and *minorRadius*.

minorRadius

A Float specifying the minor Radius if *curve* is an ellipse. This is mutually exclusive with *value* and *majorRadius*.

Return value

A ConstrainedSketchDimension object (**None** if the dimension cannot be created).

Exceptions

None.

48.3.5 VerticalDimension(...)

This method constructs a ConstrainedSketchDimension between two vertices. A vertical dimension controls the vertical distance along the Y-axis between two vertices.

Path

`mdb.models [name] . sketches [name] . VerticalDimension`

Required arguments

vertex1

A ConstrainedSketchVertex object specifying the first endpoint.

vertex2

A ConstrainedSketchVertex object specifying the second endpoint.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments

value

A Float specifying the angle between the two lines.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the angle between two lines.

Return value

A `ConstrainedSketchDimension` object (**None** if the dimension cannot be created).

Exceptions

None.

48.3.6 DistanceDimension(...)

This method constructs a `ConstrainedSketchDimension` object between two `ConstrainedSketchGeometry`, or a `ConstrainedSketchVertex` and `ConstrainedSketchGeometry` object. A distance dimension specifies the shortest distance between two entities.

Path

`mdb.models[name].sketches[name].DistanceDimension`

Required arguments

entity1

A `ConstrainedSketchVertex` object or `ConstrainedSketchGeometry` object.

vertex2

A `ConstrainedSketchVertex` object or `ConstrainedSketchGeometry` object.

textPoint

A pair of Floats specifying the location of the dimension text.

Optional arguments

value

A Float specifying the angle between the two lines.

reference

A Boolean specifying whether the created dimension enforces the above value or if it simply measures the angle between two lines.

Return value

A `ConstrainedSketchDimension` object (**None** if the dimension cannot be created).

Exceptions

None.

48.3.7 Members

The `ConstrainedSketchDimension` object has no members.

48.4 ConstrainedSketcherOptions object

The ConstrainedSketcherOptions object is used to store values and attributes which will be applied to all sketches used in the current session. The ConstrainedSketcherOptions object has no constructor.

Access

```
import sketch
session.sketcherOptions
```

48.4.1 setValues(...)

This method modifies the ConstrainedSketchOptions object.

Arguments

Required arguments

None.

Optional arguments

constructionGeometry

A Boolean specifying whether construction geometry is shown. The default value is ON.

gridSnap

A Boolean specifying whether the cursor snaps to the grid. The default value is ON.

preselection

A Boolean specifying whether geometry will be preselected. The default value is ON.

addImpliedConstraints

A Boolean specifying if implied constraints are added during sketching. The default value is ON.

maxCoplanarEntities

An Int specifying the maximum number of coplanar entities which should be automatically projected from the background, when a sketch based feature is created or edited. When this value is exceeded no entities are automatically projected and a warning issued. Possible values are *maxCoplanarEntities* > 0. The default value is 300.

autoConstrainAngularTolerance

A Float specifying the angular tolerance in degrees which is used to determine parallel and tangential conditions during the auto-constrain operation. For example any two lines which have an angle smaller than the given *autoConstrainAngularTolerance* will be assumed to be parallel, and a parallel constrain may be added during the auto-constrain operation. The default value is 0.01.

autoConstrainLinearTolerance

A Float specifying the linear tolerance which is used to determine when two points or geometries are coincident during the auto-constrain operation. The default value is 10^{-6} .

autoConstrainOptions

A sequence of SymbolicConstants specifying which type of constraints may be added by the auto-constraint tool. Possible values are PARALLEL, PERPENDICULAR, IDENTICAL, TANGENT, CONCENTRIC, and EQUALRADIUS. The default value is (PARALLEL,, PERPENDICULAR,, IDENTICAL,, TANGENT,, CONCENTRIC,, EQUALRADIUS).

dragMethod

A SymbolicConstant specifying the constraint solving mode used by the sketcher during drag operation. Possible values are MINIMUM_MOVE, STANDARD, WEIGHTED, and RELAXATION. The default value is MINIMUM_MOVE.

editMethod

A SymbolicConstant specifying the constraint solving mode used by the sketcher during regular sketch editing and adding new constraints and dimensions. Possible values are MINIMUM_MOVE, STANDARD, WEIGHTED, and RELAXATION. The default value is STANDARD.

Return value

None

Exceptions

RangeError.

48.4.2 Members

The ConstrainedSketcherOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

48.5 ConstrainedSketchGeometry object

The ConstrainedSketchGeometry object stores the geometry of a sketch, such as lines, circles, arcs, and construction lines.

Access

```
import sketch
mdb.models[name].sketches[name].geometry[i]
mdb.models[name].sketches[name].geometry[i][i]
```

48.5.1 Arc3Points(...)

This method constructs an arc using a two endpoints and an intermediate third point on the arc.

Path

`mdb.models[name].sketches[name].Arc3Points`

Required arguments

point1

A pair of Floats specifying the first endpoint of the arc.

point2

A pair of Floats specifying the second endpoint of the arc.

point3

A pair of Floats specifying the third point on the arc.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the arc cannot be created).

Exceptions

None.

48.5.2 ArcByCenterEnds(...)

This method constructs an arc using a center point and two vertices. The Arc object is added to the geometry repository of the ConstrainedSketch object. The arc is created in a clockwise fashion from *point1* to *point2*.

Path

`mdb.models[name].sketches[name].ArcByCenterEnds`

Required arguments

center

A pair of Floats specifying the center point of the arc.

point1

A pair of Floats specifying the first endpoint of the arc.

point2

A pair of Floats specifying the second endpoint of the arc.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the arc cannot be created).

Exceptions

If incompatible data are given, the second endpoint is ignored.

48.5.3 ArcByStartEndTangent(...)

This method constructs an arc using two vertices. The Arc object is added to the geometry repository of the ConstrainedSketch object.

Path

`mdb.models[name].sketches[name].ArcByStartEndTangent`

Required arguments

point1

A pair of Floats specifying the first endpoint of the arc.

point2

A pair of Floats specifying the second endpoint of the arc.

vector

A sequence of two Floats specifying the start direction for constructing the arc.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the arc cannot be created).

Exceptions

None.

48.5.4 CircleByCenterPerimeter(...)

This method constructs a circle using a center point and a point on the perimeter. The circle is added to the geometry repository of the ConstrainedSketch object.

Path

```
mdb.models[name].sketches[name].CircleByCenterPerimeter
```

Required arguments

center

A pair of Floats specifying the center point of the circle.

point1

A pair of Floats specifying a point on the perimeter of the circle.

Optional argument**Return value**

A ConstrainedSketchGeometry object (**None** if the circle cannot be created).

Exceptions

None.

48.5.5 ConstructionCircleByCenterPerimeter(...)

This method constructs a construction circle using a center point and a point on the perimeter. The circle is added to the geometry repository of the ConstrainedSketch object.

Path

```
mdb.models[name].sketches[name].ConstructionCircleByCenterPerimeter
```

Required arguments

center

A pair of Floats specifying the center point of the construction circle.

point1

A pair of Floats specifying a point on the perimeter of the construction circle.

Optional argument**Return value**

A ConstrainedSketchGeometry object (**None** if the circle cannot be created).

Exceptions

None.

48.5.6 EllipseByCenterPerimeter(...)

This method constructs an ellipse using a center point, a major axis point, and a minor axis point. The ellipse is added to the geometry repository of the ConstrainedSketch object.

Path

```
mdb.models[name].sketches[name].EllipseByCenterPerimeter
```

Required arguments

center

A pair of Floats specifying the center point of the ellipse.

axisPoint1

A pair of Floats specifying the major or minor axis point of the ellipse.

axisPoint2

A pair of Floats specifying the minor or major axis point of the ellipse.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the ellipse cannot be created).

Exceptions

None.

48.5.7 FilletByRadius(...)

This method constructs a fillet arc of a given radius between two curves. The fillet is added to the geometry repository of the ConstrainedSketch object.

Path

`mdb.models[name].sketches[name].FilletByRadius`

Required arguments

radius

A Float specifying the radius of the fillet arc. Possible values are Floats > 0 .

curve1

A ConstrainedSketchGeometry object specifying the first curve.

nearPoint1

A pair of Floats specifying a point on the sketch near where the user wishes the fillet to intersect with *curve1*. This point does not need to be on *curve1*; it is used as a hint to draw the fillet.

curve2

A ConstrainedSketchGeometry object specifying the second curve.

nearPoint2

A pair of Floats specifying a point on the sketch near where the user wishes the fillet to intersect with *curve2*. This point does not need to be on *curve2*; it is used as a hint to draw the fillet.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the fillet cannot be created).

Exceptions

If the radius given cannot be used to create a fillet between the two curves given:

Range Error: cannot construct the Fillet specified

48.5.8 Line(...)

This method creates a line between two given points.

Path

`mdb.models[name].sketches[name].Line`

Required arguments

point1

A pair of Floats specifying the first endpoint.

point2

A pair of Floats specifying the second endpoint.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the line cannot be created).

Exceptions

None.

48.5.9 ConstructionLine(...)

This method creates an oblique construction line that runs between two given points.

Path

`mdb.models[name].sketches[name].ConstructionLine`

Required arguments

point1

A pair of Floats specifying the first endpoint.

point2

A pair of Floats specifying the second endpoint.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the line cannot be created).

Exceptions

None.

48.5.10 Spline(...)

This method creates a spline curve running through a sequence of points.

Path

`mdb.models[name].sketches[name].Spline`

Required argument

points

A sequence of pairs of Floats specifying the points through which the spline passes.

Optional argument

constrainPoints

A Boolean that determines whether the points given are to constrained to always remain on the Spline. The default is True. For a large sequence of *points*, significant performance gains may be achieved by setting the value to False.

Return value

A ConstrainedSketchGeometry object (**None** if the spline cannot be created).

Exceptions

None.

48.5.11 Spot(...)

This method creates a spot construction point located at the specified coordinates. The spot is added to the vertex repository of the ConstrainedSketch object.

Path

```
mdb.models[name].sketches[name].Spot
```

Required argument

point

A pair of Floats specifying the coordinates of the spot construction point.

Optional arguments

None.

Return value

A ConstrainedSketchGeometry object (**None** if the spot cannot be created).

Exceptions

None.

48.5.12 **getVertices()**

This method returns an list of `ConstrainedSketchVertex` objects which are a part of the given `ConstrainedSketchGeometry` object.

Arguments

None.

Return value

A list of `ConstrainedSketchVertex` objects.

Exceptions

None.

48.5.13 **getSize()**

This method returns the length of the given `ConstrainedSketchGeometry` object.

Arguments

None.

Return value

The length of the given `ConstrainedSketchGeometry`.

Exceptions

None.

48.5.14 **getPointAtDistance(...)**

This method returns a point offset along the given `ConstrainedSketchGeometry` from the given end by a specified arc length distance or a percentage of the total length of the `ConstrainedSketchGeometry` object.

Arguments

Required arguments

point

A pair of Floats specifying the point from which the distance is to be measured.

SKETCHER COMMANDS

distance

A float specifying the arc length distance along the `ConstrainedSketchGeometry` from the *point* at which the required point is situated.

Optional argument

percentage

A Boolean that specifies if the *distance* is an absolute distance or is a fraction relative to the length of the `ConstrainedSketchGeometry` object.

Return value

A pair of floats representing the point along the edge.

Exceptions

None.

48.5.15 Members

The `ConstrainedSketchGeometry` object has the following members:

id

An Int specifying the index of the sketch entity in the `ConstrainedSketchGeometryArray`.

curveType

A `SymbolicConstant` specifying the geometry of the sketch entity. Possible values are `ARC`, `CIRCLE`, `ELLIPSE`, `LINE`, and `SPLINE`.

type

A `SymbolicConstant` specifying the type of sketch entity. Possible values are `REGULAR`, `REFERENCE`, and `CONSTRUCTION`.

pointOn

A tuple of Floats specifying the X- and Y-coordinates of a point located on the geometry.

48.6 `ConstrainedSketchGeometryArray` object

The `ConstrainedSketchGeometryArray` is a sequence of `ConstrainedSketchGeometry` objects.

Access

```
import sketch
mdb.models[name].sketches[name].geometry[i]
```


48.6.1 findAt(...)

This method returns the ConstrainedSketchGeometry object located at the given coordinates.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X- and Y-coordinates of the object to find.

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

A ConstrainedSketchGeometry object.

Exceptions

None.

48.6.2 Members

The ConstrainedSketchGeometryArray object has no members.

48.7 ConstrainedSketchImageOptions object

The ConstrainedSketchImageOptions object is used to store values and attributes associated with the background image for a particular sketch. The ConstrainedSketchImageOptions object has no constructor.

Access

```
import sketch
mdb.models[name].sketches[name].imageOptions
```

48.7.1 setValues(...)

This method modifies the ConstrainedSketchOptions object.

Arguments

Required arguments

None.

Optional arguments

imageName

A String specifying the name of the image. A list of valid image names is in the *images* repository in the *session* object.

showImage

A Boolean specifying whether an image should be displayed in the sketcher background. The default value is OFF.

origin

A pair of Floats specifying the X- and Y-offsets in millimeters from the lower-left corner of the viewport. The default value is (0, 0).

xScale

A Float specifying the scale applied to the image width. The default value is 1.0.

When *xScale* is negative, the image is mirrored about its y-axis but its position is not affected.

yScale

A Float specifying the scale applied to the image height. The default value is 1.0.

When *yScale* is negative, the image is mirrored about its x-axis but its position is not affected.

translucency

A Float specifying the translucency factor to use when displaying the image. Possible values are $0.0 \leq \text{translucency} \leq 1.0$ with 0.0 being invisible and 1.0 being opaque. The default value is 1.0.

Return value

None

Exceptions

RangeError.

48.7.2 Members

The `ConstrainedSketchImageOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

48.8 ConstrainedSketchOptions object

The ConstrainedSketchOptions object is used to store values and attributes associated with a particular sketch. The ConstrainedSketchOptions object has no constructor.

Access

```
import sketch
mdb.models[name].sketches[name].sketchOptions
```

48.8.1 setValues(...)

This method modifies the ConstrainedSketchOptions object.

Arguments

Required arguments

None.

Optional arguments

sheetSize

A Float specifying the sheet size. Possible values are Floats > 0 . The default value is the *sheetSize* specified with the **Sketch** method.

gridSpacing

A Float specifying the spacing between gridlines. Possible values are Floats > 0 . The default value is approximately 2.5% of *sheetSize*.

grid

A Boolean specifying whether the grid is shown. The default value is ON.

gridFrequency

An Int specifying how often gridlines are shown. Possible values are *gridFrequency* > 0 . The default value is 1.

dimensionTextHeight

A Float specifying the height of the dimension text in points. Possible values are Floats > 0 . The default value is 12.0.

decimalPlaces

An Int specifying how many decimal places are shown in dimensions. Possible values are $0 \leq \text{decimalPlaces} \leq 6$. The initial value depends on the value of *sheetSize*.

constructionGeometry

A Boolean specifying whether construction geometry is shown. The default value is ON.

SKETCHER COMMANDS

gridSnap

A Boolean specifying whether the cursor snaps to the grid. The default value is ON.

preselection

A Boolean specifying whether geometry will be preselected. The default value is ON.

sheetAuto

A Boolean specifying if the sheet size and the grid spacing are automatically computed. The default value is ON.

gridOrigin

A sequence of Floats specifying the X–Y coordinates for the origin of the grid. The default value is (0, 0).

gridAngle

A Float specifying the angle of the grid relative to the computer screen. The default value is 0.0.

viewStyle

A SymbolicConstant specifying the type of sketch displayed in the viewport. Possible values are REGULAR and AXISYM. The default value is REGULAR.

addImpliedConstraints

A Boolean specifying if implied constraints are added during sketching. The default value is ON.

maxCoplanarEntities

An Int specifying the maximum number of coplanar entities which should be automatically projected from the background, when a sketch based feature is created or edited. When this value is exceeded no entities are automatically projected and a warning issued. Possible values are *maxCoplanarEntities* > 0. The default value is 300.

autoConstrainAngularTolerance

A Float specifying the angular tolerance in degrees which is used to determine parallel and tangential conditions during the auto-constrain operation. For example any two lines which have an angle smaller than the given *autoConstrainAngularTolerance* will be assumed to be parallel, and a parallel constrain may be added during the auto-constrain operation. The default value is 0.01.

autoConstrainLinearTolerance

A Float specifying the linear tolerance which is used to determine when two points or geometries are coincident during the auto-constrain operation. The default value is 10^{-6} .

autoConstrainOptions

A sequence of SymbolicConstants specifying which type of constraints may be added by the auto-constraint tool. Possible values are PARALLEL, PERPENDICULAR, IDENTICAL, TANGENT, CONCENTRIC, and EQUALRADIUS. The default value is (PARALLEL,, PERPENDICULAR,, IDENTICAL,, TANGENT,, CONCENTRIC,, EQUALRADIUS).

dragMethod

A SymbolicConstant specifying the constraint solving mode used by the sketcher during drag operation. Possible values are MINIMUM_MOVE, STANDARD, WEIGHTED, and RELAXATION. The default value is MINIMUM_MOVE.

editMethod

A SymbolicConstant specifying the constraint solving mode used by the sketcher during regular sketch editing and adding new constraints and dimensions. Possible values are MINIMUM_MOVE, STANDARD, WEIGHTED, and RELAXATION. The default value is STANDARD.

Return value

None

Exceptions

RangeError.

48.8.2 Members

The ConstrainedSketchOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

48.9 ConstrainedSketchParameter object

The ConstrainedSketchParameter object stores the definition of a parameter in the sketch.

Access

```
import sketch
mdb.models[name].sketches[name].parameters[i]
```

48.9.1 Parameter(...)

This method creates a parameter and optionally associates a dimension with this parameter.

Path

```
mdb.models[name].sketches[name].Parameter
```

Required argument

name

A String specifying the name of the `ConstrainedSketchParameter` object. No two parameters in the same `ConstrainedSketch` can have the same name.

Optional arguments

path

A String specifying the `ConstrainedSketchDimension` object with which this parameter is associated.

expression

A String specifying the expression or value associated with the `ConstrainedSketchParameter`.

previousParameter

A String specifying the name of the previous `ConstrainedSketchParameter`, if it exists. The *previousParameter* argument implies an order among the parameters. No two parameters can reference the same parameter as the previous parameter.

Return value

A `ConstrainedSketchParameter` object.

Exceptions

None.

48.9.2 Members

The `ConstrainedSketchParameter` object has the following members:

name

A String specifying the name of the `ConstrainedSketchParameter` object.

path

A String specifying the path to the `ConstrainedSketchDimension` that depends on this `ConstrainedSketchParameter`.

expression

A String specifying an expression or value associated with this `ConstrainedSketchParameter`.

previousParameter

A String specifying the name of the `ConstrainedSketchParameter` that appears before this one in the ordered list.

48.10 ConstrainedSketchVertex object

The ConstrainedSketchVertex object stores the vertex position.

Access

```
import sketch
mdb.models[name].sketches[name].vertices[i]
mdb.models[name].sketches[name].vertices[i][i]
```

48.10.1 Spot(...)

This method creates a spot (construction point) located at the specified coordinates.

Path

```
mdb.models[name].sketches[name].Spot
```

Required argument

point

A pair of Floats specifying the coordinates of the construction point.

Optional arguments

None.

Return value

A ConstrainedSketchVertex object (**None** if the spot cannot be created).

Exceptions

None.

48.10.2 Members

The ConstrainedSketchVertex object has the following member:

coords

A tuple of Floats specifying the X-, Y-, and Z-coordinates of the sketch vertex.

48.11 ConstrainedSketchVertexArray object

The ConstrainedSketchVertexArray is a sequence of ConstrainedSketchVertex objects.

Access

```
import sketch
mdb.models[name].sketches[name].vertices[i]
```

48.11.1 findAt(...)

This method returns the ConstrainedSketchVertex located at the given coordinates.

Arguments

Required argument

coordinates

A sequence of Floats specifying the X- and Y-coordinates of the object to find.

Optional argument

printWarning

A Boolean specifying whether a message is to be printed to the CLI if no entity is found at the specified location. The default value is True.

Return value

A ConstrainedSketchVertex object.

Exceptions

None.

48.11.2 Members

The ConstrainedSketchVertexArray object has no members.

49. Step commands (step)

The Step commands described in this chapter are used to create and configure analysis steps. Chapter 51, “Step commands (output),” describes the commands used to create and configure output requests and integrated output sections and the commands to configure diagnostic printing, monitoring, and restart. Chapter 50, “Step commands (miscellaneous),” describes the commands used to configure controls, damping, and frequency tables.

49.1 Step object

The Step object stores the parameters that determine the context of the step. The Step object is the abstract base type for other Step objects. The Step object has no explicit constructor. The methods and members of the Step object are common to all objects derived from the Step.

Access

```
import step
mdb.models[name].steps[name]
```

49.1.1 resume()

This method resumes the step that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

None.

49.1.2 suppress()

This method suppresses the step.

Arguments

None.

STEP COMMANDS (STEP)

Return value

None

Exceptions

None.

49.1.3 Members

The Step object can have the following members:

name

A String specifying the repository key.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER

- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

STEP COMMANDS (STEP)

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.2 AnalysisStep object

The AnalysisStep object is the abstract base type for other Step objects, except the InitialStep object. It has no explicit constructor, no methods, and has only inherited members.

The AnalysisStep object is derived from the Step object.

Access

```
import step
mdb.models[name].steps[name]
```

49.2.1 Members

The AnalysisStep object can have the following members:

name

A String specifying the repository key.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC

- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

STEP COMMANDS (STEP)

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.3 AnnealStep object

The AnnealStep object anneals a structure by setting the velocities and all appropriate state variables to zero.

The AnnealStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.3.1 AnnealStep(...)

This method creates an AnnealStep object.

Path

```
mdb.models[name].AnnealStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

refTemp

A Float specifying the post-anneal reference temperature. The default value is the current temperature at all nodes in the model after the annealing has completed.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

An AnnealStep object.

Exceptions

RangeError.

49.3.2 setValues(...)

This method modifies the AnnealStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **AnnealStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.3.3 Members

The AnnealStep object can have the following members:

STEP COMMANDS (STEP)

name

A String specifying the repository key.

refTemp

A Float specifying the post-anneal reference temperature. The default value is the current temperature at all nodes in the model after the annealing has completed.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS

- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

STEP COMMANDS (STEP)

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.3.4 Corresponding analysis keywords

*ANNEAL

*STEP

49.4 BuckleStep object

The BuckleStep object controls eigenvalue buckling estimation.

The BuckleStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.4.1 BuckleStep(...)

This method creates a BuckleStep object.

Path

```
mdb.models[name].BuckleStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

numEigen

An Int specifying the number of eigenvalues to be estimated.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

eigensolver

A SymbolicConstant specifying the eigensolver. Possible values are SUBSPACE and LANCZOS. The default value is SUBSPACE.

minEigen

None or a Float specifying the minimum eigenvalue of interest. The default value is **None**.

maxEigen

None or a Float specifying the maximum eigenvalue of interest. The default value is **None**.

vectors

An Int specifying the number of vectors used in the iteration. The default value is the minimum of $(2n, n + 8)$, where n is the number of eigenvalues requested.

maxIterations

An Int specifying the maximum number of iterations. The default value is 30.

blockSize

The SymbolicConstant DEFAULT or an Int specifying the size of the Lanczos block steps. The default value is DEFAULT.

maxBlocks

The SymbolicConstant DEFAULT or an Int specifying the maximum number of Lanczos block steps within each Lanczos run. The default value is DEFAULT.

Note: *minEigen*, *blockSize*, and *maxBlocks* are ignored unless *eigensolver*=LANCZOS.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A BuckleStep object.

Exceptions

RangeError.

49.4.2 **setValues(...)**

This method modifies the `BuckleStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BuckleStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.4.3 **Members**

The `BuckleStep` object can have the following members:

name

A String specifying the repository key.

numEigen

An Int specifying the number of eigenvalues to be estimated.

eigensolver

A SymbolicConstant specifying the eigensolver. Possible values are `SUBSPACE` and `LANCZOS`. The default value is `SUBSPACE`.

minEigen

None or a Float specifying the minimum eigenvalue of interest. The default value is **None**.

maxEigen

None or a Float specifying the maximum eigenvalue of interest. The default value is **None**.

vectors

An Int specifying the number of vectors used in the iteration. The default value is the minimum of $(2n, n + 8)$, where n is the number of eigenvalues requested.

maxIterations

An Int specifying the maximum number of iterations. The default value is 30.

blockSize

The SymbolicConstant DEFAULT or an Int specifying the size of the Lanczos block steps. The default value is DEFAULT.

maxBlocks

The SymbolicConstant DEFAULT or an Int specifying the maximum number of Lanczos block steps within each Lanczos run. The default value is DEFAULT.

Note: *minEigen*, *blockSize*, and *maxBlocks* are ignored unless *eigsolver*=LANCZOS.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL

STEP COMMANDS (STEP)

- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.4.4 Corresponding analysis keywords

*BUCKLE

*STEP

49.5 ComplexFrequencyStep object

The ComplexFrequencyStep object is used to perform eigenvalue extraction to calculate the complex eigenvalues and corresponding complex mode shapes of a system.

The ComplexFrequencyStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.5.1 ComplexFrequencyStep(...)

This method creates a ComplexFrequencyStep object.

Path

```
mdb.models[name].ComplexFrequencyStep
```

Required arguments

name

A String specifying the repository key.

STEP COMMANDS (STEP)

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

numEigen

The SymbolicConstant ALL or an Int specifying the number of complex eigenmodes to be calculated or a SymbolicConstant ALL. The default value is ALL.

description

A String specifying a description of the new step. The default value is an empty string.

shift

None or a Float specifying the shift point in cycles per time. The default value is **None**.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

minEigen

None or a Float specifying the minimum frequency of interest in cycles per time. The default value is **None**.

maxEigen

None or a Float specifying the maximum frequency of interest in cycles per time. The default value is **None**.

propertyEvaluationFrequency

None or a Float specifying the frequency at which to evaluate frequency-dependent properties for viscoelasticity, springs, and dashpots during the eigenvalue extraction. If the value is **None**, the analysis product will evaluate the stiffness associated with frequency-dependent springs and dashpots at zero frequency and will not consider the stiffness contributions from frequency-domain viscoelasticity in the step. The default value is **None**.

Return value

A ComplexFrequencyStep object.

Exceptions

RangeError.

49.5.2 **setValues(...)**

This method modifies the `ComplexFrequencyStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ComplexFrequencyStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.5.3 **Members**

The `ComplexFrequencyStep` object can have the following members:

name

A String specifying the repository key.

numEigen

The SymbolicConstant `ALL` or an Int specifying the number of complex eigenmodes to be calculated or a SymbolicConstant `ALL`. The default value is `ALL`.

shift

None or a Float specifying the shift point in cycles per time. The default value is **None**.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is `OFF`.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are `SYMMETRIC`, `UNSYMMETRIC`, and `SOLVER_DEFAULT`. The default value is `SOLVER_DEFAULT`.

minEigen

None or a Float specifying the minimum frequency of interest in cycles per time. The default value is **None**.

STEP COMMANDS (STEP)

maxEigen

None or a Float specifying the maximum frequency of interest in cycles per time. The default value is **None**.

propertyEvaluationFrequency

None or a Float specifying the frequency at which to evaluate frequency-dependent properties for viscoelasticity, springs, and dashpots during the eigenvalue extraction. If the value is **None**, the analysis product will evaluate the stiffness associated with frequency-dependent springs and dashpots at zero frequency and will not consider the stiffness contributions from frequency-domain viscoelasticity in the step. The default value is **None**.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC

- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

STEP COMMANDS (STEP)

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.5.4 Corresponding analysis keywords

*COMPLEX FREQUENCY

*STEP

49.6 CoupledTempDisplacementStep object

The CoupledTempDisplacementStep object is used to analyze problems where the simultaneous solution of the temperature and stress/displacement fields is necessary.

The CoupledTempDisplacementStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.6.1 CoupledTempDisplacementStep(...)

This method creates a CoupledTempDisplacementStep object.

Path

```
mdb.models[name].CoupledTempDisplacementStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable and *stabilizationMethod*≠NONE. The default value is 2×10^{-4} .

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

STEP COMMANDS (STEP)

creepIntegration

A SymbolicConstant specifying the type of integration to be used for creep and swelling effects throughout the step. Possible values are IMPLICIT, EXPLICIT, and NONE. The default value is IMPLICIT.

solutionTechnique

A SymbolicConstant specifying the type of solution technique. Possible values are FULL_NEWTON and SEPARATED. The default value is FULL_NEWTON.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

Return value

A CoupledTempDisplacementStep object.

Exceptions

RangeError.

49.6.2 **setValues(...)**

This method modifies the `CoupledTempDisplacementStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the `CoupledTempDisplacementStep` method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.6.3 **Members**

The `CoupledTempDisplacementStep` object can have the following members:

name

A String specifying the repository key.

response

A `SymbolicConstant` specifying the analysis type. Possible values are `STEADY_STATE` and `TRANSIENT`. The default value is `TRANSIENT`.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is `OFF`.

stabilizationMethod

A `SymbolicConstant` specifying the stabilization type. Possible values are `NONE`, `DISSIPATED_ENERGY_FRACTION`, and `DAMPING_FACTOR`. The default value is `NONE`.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable and *stabilizationMethod*≠`NONE`. The default value is 2×10^{-4} .

STEP COMMANDS (STEP)

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

creepIntegration

A SymbolicConstant specifying the type of integration to be used for creep and swelling effects throughout the step. Possible values are IMPLICIT, EXPLICIT, and NONE. The default value is IMPLICIT.

solutionTechnique

A SymbolicConstant specifying the type of solution technique. Possible values are FULL_NEWTON and SEPARATED. The default value is FULL_NEWTON.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW

STEP COMMANDS (STEP)

- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.6.4 Corresponding analysis keywords

*COUPLED TEMPERATURE-DISPLACEMENT

*SOLUTION TECHNIQUE

*STEP

49.7 CoupledThermalElectricalStructuralStep object

The CoupledThermalElectricalStructuralStep object is used to analyze problems where the simultaneous solution of the temperature, stress/displacement and electrical fields is necessary.

The CoupledThermalElectricalStructuralStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.7.1 CoupledThermalElectricalStructuralStep(...)

This method creates a CoupledThermalElectricalStructuralStep object.

Path

```
mdb.models[name].CoupledThermalElectricalStructuralStep
```

Required arguments

name

A String specifying the repository key.

STEP COMMANDS (STEP)

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable and *stabilizationMethod*≠NONE. The default value is 2×10^{-4} .

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

creepIntegration

A SymbolicConstant specifying the type of integration to be used for creep and swelling effects throughout the step. Possible values are IMPLICIT, EXPLICIT, and NONE. The default value is IMPLICIT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

Return value

A CoupledThermalElectricalStructuralStep object.

Exceptions

RangeError.

49.7.2 **setValues(...)**

This method modifies the `CoupledThermalElectricalStructuralStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CoupledThermalElectricalStructuralStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.7.3 **Members**

The `CoupledThermalElectricalStructuralStep` object can have the following members:

name

A String specifying the repository key.

response

A `SymbolicConstant` specifying the analysis type. Possible values are `STEADY_STATE` and `TRANSIENT`. The default value is `TRANSIENT`.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is `OFF`.

stabilizationMethod

A `SymbolicConstant` specifying the stabilization type. Possible values are `NONE`, `DISSIPATED_ENERGY_FRACTION`, and `DAMPING_FACTOR`. The default value is `NONE`.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable and *stabilizationMethod*≠`NONE`. The default value is 2×10^{-4} .

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

creepIntegration

A SymbolicConstant specifying the type of integration to be used for creep and swelling effects throughout the step. Possible values are IMPLICIT, EXPLICIT, and NONE. The default value is IMPLICIT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

STEP COMMANDS (STEP)

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER

- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

STEP COMMANDS (STEP)

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.7.4 Corresponding analysis keywords

*COUPLED TEMPERATURE-DISPLACEMENT

*SOLUTION TECHNIQUE

*STEP

49.8 CoupledThermalElectricStep object

The CoupledThermalElectricStep object is used to analyze problems where the electrical potential and temperature fields must be solved simultaneously.

The CoupledThermalElectricStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.8.1 CoupledThermalElectricStep(...)

This method creates a CoupledThermalElectricStep object.

Path

```
mdb.models[name].CoupledThermalElectricStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

mxdem

A Float specifying the maximum allowable emissivity change with temperature and field variables during an increment. The default value is 0.1.

solutionTechnique

A SymbolicConstant specifying the type of solution technique. Possible values are FULL_NEWTON and SEPARATED. The default value is FULL_NEWTON.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

STEP COMMANDS (STEP)

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

Return value

A CoupledThermalElectricStep object.

Exceptions

RangeError.

49.8.2 setValues(...)

This method modifies the CoupledThermalElectricStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **CoupledThermalElectricStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.8.3 Members

The CoupledThermalElectricStep object can have the following members:

name

A String specifying the repository key.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment in a transient analysis. The default value is 0.0.

mxdem

A Float specifying the maximum allowable emissivity change with temperature and field variables during an increment. The default value is 0.1.

solutionTechnique

A SymbolicConstant specifying the type of solution technique. Possible values are FULL_NEWTON and SEPARATED. The default value is FULL_NEWTON.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

STEP COMMANDS (STEP)

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC

- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

STEP COMMANDS (STEP)

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.8.4 Corresponding analysis keywords

*COUPLED THERMAL-ELECTRICAL

*SOLUTION TECHNIQUE

*STEP

49.9 DirectCyclicStep object

The DirectCyclicStep object is used to provide a direct cyclic procedure for nonlinear, non-isothermal quasi-static analysis. It can also be used to predict progressive damage and failure for ductile bulk materials and/or to predict delamination/debonding growth at the interfaces in laminated composites in a low-cycle fatigue analysis.

The DirectCyclicStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.9.1 DirectCyclicStep(...)

This method creates a DirectCyclicStep object.

Path

```
mdb.models[name].DirectCyclicStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the time of single loading cycle. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

maxNumIterations

An Int specifying the maximum number of iterations in a step. The default value is 200.

initialTerms

An Int specifying the initial number of terms in the Fourier series. The default value is 11.

maxTerms

An Int specifying the maximum number of terms in the Fourier series. The default value is 25.

termsIncrement

An Int specifying the increment in number of terms in the Fourier series. The default value is 5.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

timePoints

None or a String specifying a String specifying the name of a time point object used to determine at which times the response of the structure will be evaluated. The default value is NONE.

STEP COMMANDS (STEP)

fatigue

A Boolean specifying whether to include low-cycle fatigue analysis. The default value is OFF.

continueAnalysis

A Boolean specifying whether the displacement solution in the Fourier series obtained in the previous direct cyclic step is used as the starting values for the current step. The default value is OFF.

minCycleInc

An Int specifying the minimum number of cycles over which the damage is extrapolated forward. The default value is 100.

maxCycleInc

An Int specifying the maximum number of cycles over which the damage is extrapolated forward. The default value is 1000.

maxNumCycles

The SymbolicConstant DEFAULT or an Int specifying the maximum number of cycles allowed in a step or DEFAULT. A value of 1 plus half of the maximum number of cycles will be used if DEFAULT is specified. The default value is DEFAULT.

damageExtrapolationTolerance

A Float specifying the maximum extrapolated damage increment. The default value is 1.0.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

Return value

A DirectCyclicStep object.

Exceptions

RangeError.

49.9.2 **setValues(...)**

This method modifies the `DirectCyclicStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DirectCyclicStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.9.3 **Members**

The `DirectCyclicStep` object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the time of single loading cycle. The default value is 1.0.

timeIncrementationMethod

A `SymbolicConstant` specifying the time incrementation method to be used. Possible values are `FIXED` and `AUTOMATIC`. The default value is `AUTOMATIC`.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

STEP COMMANDS (STEP)

maxNumIterations

An Int specifying the maximum number of iterations in a step. The default value is 200.

initialTerms

An Int specifying the initial number of terms in the Fourier series. The default value is 11.

maxTerms

An Int specifying the maximum number of terms in the Fourier series. The default value is 25.

termsIncrement

An Int specifying the increment in number of terms in the Fourier series. The default value is 5.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment. The default value is 0.0.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

fatigue

A Boolean specifying whether to include low-cycle fatigue analysis. The default value is OFF.

continueAnalysis

A Boolean specifying whether the displacement solution in the Fourier series obtained in the previous direct cyclic step is used as the starting values for the current step. The default value is OFF.

minCycleInc

An Int specifying the minimum number of cycles over which the damage is extrapolated forward. The default value is 100.

maxCycleInc

An Int specifying the maximum number of cycles over which the damage is extrapolated forward. The default value is 1000.

maxNumCycles

The SymbolicConstant DEFAULT or an Int specifying the maximum number of cycles allowed in a step or DEFAULT. A value of 1 plus half of the maximum number of cycles will be used if DEFAULT is specified. The default value is DEFAULT.

damageExtrapolationTolerance

A Float specifying the maximum extrapolated damage increment. The default value is 1.0.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

timePoints

None or a String specifying a String specifying the name of a time point object used to determine at which times the response of the structure will be evaluated. The default value is NONE.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER

STEP COMMANDS (STEP)

- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.9.4 Corresponding analysis keywords

*DIRECT CYCLIC

*STEP

49.10 EmagTimeHarmonicStep object

The EmagTimeHarmonicStep object is used to calculate the electromagnetic response of the system to harmonic excitation of the model.

The EmagTimeHarmonicStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.10.1 EmagTimeHarmonicStep(...)

This method creates a EmagTimeHarmonicStep object.

Path

```
mdb.models[name].EmagTimeHarmonicStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

STEP COMMANDS (STEP)

frequencyRange

An `EmagTimeHarmonicFrequencyArray` object.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

factorization

A `SymbolicConstant` specifying whether damping terms are to be ignored so that a real, rather than a complex, system matrix is factored. Possible values are `REAL_ONLY` and `COMPLEX`. The default value is `COMPLEX`.

Return value

An `EmagTimeHarmonicStep` object.

Exceptions

`RangeError`.

49.10.2 `setValues(...)`

This method modifies the `EmagTimeHarmonicStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `EmagTimeHarmonicStep` method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

49.10.3 Members

The `EmagTimeHarmonicStep` object has members with the same names and descriptions as the arguments to the `EmagTimeHarmonicStep` method.

In addition, the `EmagTimeHarmonicStep` object can have the following members:

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is OFF.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL

STEP COMMANDS (STEP)

- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.10.4 Corresponding analysis keywords

*ELECTROMAGNETIC TIME HARMONIC
*STEP

49.11 ExplicitDynamicsStep object

The ExplicitDynamicsStep object is used to perform a dynamic stress/displacement analysis using explicit integration in Abaqus/Explicit.

The ExplicitDynamicsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.11.1 ExplicitDynamicsStep(...)

This method creates an ExplicitDynamicsStep object.

Path

```
mdb.models[name].ExplicitDynamicsStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

STEP COMMANDS (STEP)

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is ON.

adiabatic

A Boolean specifying that an adiabatic stress analysis is to be performed. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are AUTOMATIC_GLOBAL, AUTOMATIC_EBE, FIXED_USER_DEFINED_INC, and FIXED_EBE. The default value is AUTOMATIC_GLOBAL.

maxIncrement

None or a Float specifying the maximum time increment. If there is no upper limit, *maxIncrement*=**None**. This argument is required only when *timeIncrementationMethod*=AUTOMATIC_GLOBAL or AUTOMATIC_EBE. The default value is **None**.

scaleFactor

A Float specifying the factor that is used to scale the time increment. This argument is required only when *timeIncrementationMethod*=AUTOMATIC_GLOBAL, AUTOMATIC_EBE, or FIXED_EBE. The default value is 1.0.

massScaling

A MassScalingArray object specifying mass scaling controls. The default value is PREVIOUS_STEP.

linearBulkViscosity

A Float specifying the linear bulk viscosity parameter, b_1 . The default value is 0.06.

quadBulkViscosity

A Float specifying the quadratic bulk viscosity parameter, b_2 . The default value is 1.2.

userDefinedInc

None or a Float specifying the user-defined time increment. This argument is required only when *timeIncrementationMethod*=FIXED_USER_DEFINED_INC. The default value is **None**.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

An ExplicitDynamicsStep object.

Exceptions

RangeError.

49.11.2 **setValues(...)**

This method modifies the `ExplicitDynamicsStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ExplicitDynamicsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.11.3 **Members**

The `ExplicitDynamicsStep` object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is ON.

adiabatic

A Boolean specifying that an adiabatic stress analysis is to be performed. The default value is OFF.

timeIncrementationMethod

A `SymbolicConstant` specifying the time incrementation method to be used. Possible values are `AUTOMATIC_GLOBAL`, `AUTOMATIC_EBE`, `FIXED_USER_DEFINED_INC`, and `FIXED_EBE`. The default value is `AUTOMATIC_GLOBAL`.

maxIncrement

None or a Float specifying the maximum time increment. If there is no upper limit, *maxIncrement*=**None**. This argument is required only when

STEP COMMANDS (STEP)

timeIncrementationMethod=AUTOMATIC_GLOBAL or AUTOMATIC_EBE. The default value is **None**.

scaleFactor

A Float specifying the factor that is used to scale the time increment. This argument is required only when *timeIncrementationMethod*=AUTOMATIC_GLOBAL, AUTOMATIC_EBE, or FIXED_EBE. The default value is 1.0.

linearBulkViscosity

A Float specifying the linear bulk viscosity parameter, b_1 . The default value is 0.06.

quadBulkViscosity

A Float specifying the quadratic bulk viscosity parameter, b_2 . The default value is 1.2.

userDefinedInc

None or a Float specifying the user-defined time increment. This argument is required only when *timeIncrementationMethod*=FIXED_USER_DEFINED_INC. The default value is **None**.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

massScaling

A `MassScalingArray` object specifying mass scaling controls. The default value is PREVIOUS_STEP.

explicit

A `SymbolicConstant` specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A `SymbolicConstant` specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT

- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

STEP COMMANDS (STEP)

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.11.4 Corresponding analysis keywords

- *BULK VISCOSITY
- *DYNAMIC
- *FIXED MASS SCALING
- *STEP
- *VARIABLE MASS SCALING

49.12 FlowStep object

The FlowStep object is used to create an analysis step for use in a CFD analysis.

The FlowStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.12.1 FlowStep(...)

This method creates a FlowStep object.

Path

`mdb.models[name].FlowStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period for the step. The default value is 1.0.

energyEquation

A SymbolicConstant specifying the energy equation method to be used. Possible values are ENERGY_NONE and TEMPERATURE. The default value is ENERGY_NONE.

incrementation

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED_TIME and FIXED_CFL. The default value is FIXED_TIME.

initialInc

A Float specifying the time increment for the step, or the initial time increment when *incrementation*=FIXED_CFL. The default value is 0.01.

divergenceTol

A Float specifying the divergence tolerance.. The default value is 10^{-10} .

maximumCFL

A Float specifying the maximum Courant-Friedrichs-Levy increment. This member is applicable only when *incrementation*=FIXED_CFL. The default value is 0.45.

incAdjustmentFreq

An Int specifying the increment adjustment frequency. This member is applicable only when *incrementation*=FIXED_CFL. The default value is 1.

stepGrowthFactor

A Float specifying the time step growth scale factor. This member is applicable only when *incrementation*=FIXED_CFL. The default value is 0.025.

viscousFactor

A SymbolicConstant specifying the viscous time integration parameter. Possible values are TRAPEZOID, GALERKIN, and BACKWARD_EULER. The default value is TRAPEZOID.

STEP COMMANDS (STEP)

boundaryFactor

A SymbolicConstant specifying the load and boundary condition time integration parameter. Possible values are TRAPEZOID, GALERKIN, and BACKWARD_EULER. The default value is TRAPEZOID.

advectionFactor

A SymbolicConstant specifying the advection parameter. Possible values are TRAPEZOID, GALERKIN, and BACKWARD_EULER. The default value is TRAPEZOID.

momOutputDiagnostics

A Boolean specifying whether or not to output diagnostics for the momentum equation solver. The default value is OFF.

momOutputConvergence

A Boolean specifying whether or not to output convergence data for the momentum equation solver. The default value is OFF.

momIterationLimit

An Int specifying the iteration limit for the momentum equation solver. The default value is 50.

momCheckingFreq

An Int specifying the momentum equation solver checking frequency. The default value is 2.

momConvergenceLimit

A Float specifying the linear convergence limit for the momentum equation solver. The default value is 10^{-5} .

pressureType

A SymbolicConstant specifying the pressure equation preconditioner type. Possible values are AMG and SSOR. The default value is AMG.

pressSolverType

A SymbolicConstant specifying the pressure equation solver type. This member cannot be changed from the default value when *pressureType*=SSOR. Possible values are CG, BCGS, and FGMRES. The default value is CG.

pressSmootherType

A SymbolicConstant specifying the pressure equation residual smoother type. This member is applicable only when *pressureType*=AMG. Possible values are ICC and CHEBYCHEV. The default value is ICC.

pressPreSweeps

An Int specifying the pre-sweeps used by the residual smoother. This member is applicable only when *pressureType*=AMG. The default value is 1.

pressPostSweeps

An Int specifying the post-sweeps used by the residual smoother. This member is applicable only when *pressureType*=AMG. The default value is 1.

pressOutputDiagnostics

A Boolean specifying whether or not to output diagnostics for the pressure equation solver. The default value is OFF.

pressOutputConvergence

A Boolean specifying whether or not to output convergence data for the pressure equation solver. The default value is OFF.

pressIterationLimit

An Int specifying the iteration limit for the pressure equation solver. This default value is 250 when *pressureType*=AMG and 1000 when *pressureType*=SSOR.

pressCheckingFreq

An Int specifying the pressure equation solver checking frequency. The default value is 2.

pressConvergenceLimit

A Float specifying the linear convergence limit for the pressure equation solver. The default value is 10^{-5} .

transOutputDiagnostics

A Boolean specifying whether or not to output diagnostics for the transport equation solver. The default value is OFF.

transOutputConvergence

A Boolean specifying whether or not to output convergence data for the transport equation solver. The default value is OFF.

transIterationLimit

An Int specifying the iteration limit for the transport equation solver. The default value is 50.

transCheckingFreq

An Int specifying the transport equation solver checking frequency. The default value is 2.

transConvergenceLimit

A Float specifying the linear convergence limit for the transport equation solver. The default value is 10^{-5} .

turbulenceModel

A SymbolicConstant specifying the turbulence model. Possible values are TURB_NONE, SPALART, and KEPS_RNG. The default value is TURB_NONE.

turbPrandtlNumber

A Float specifying the turbulent Prandtl number. This member is applicable only when *energyEquation*=TEMPERATURE and when *turbulenceModel*!=TURB_NONE. The default value is 0.8889.

turbCb1

A Float specifying the turbulence constant Cb1. This member is applicable only when *turbulenceModel*=SPALART. The default value is 0.1355.

STEP COMMANDS (STEP)

turbCb2

A Float specifying the turbulence constant Cb2. This member is applicable only when *turbulenceModel*=SPALART. The default value is 0.622.

turbCv1

A Float specifying the turbulence constant Cv1. This member is applicable only when *turbulenceModel*=SPALART. The default value is 7.1.

turbCv2

A Float specifying the turbulence constant Cv2. This member is applicable only when *turbulenceModel*=SPALART. The default value is 5.0.

turbCw1

A Float specifying the turbulence constant Cw1. This member is applicable only when *turbulenceModel*=SPALART. The default value is 3.2391.

turbCw2

A Float specifying the turbulence constant Cw2. This member is applicable only when *turbulenceModel*=SPALART. The default value is 0.3.

turbCw3

A Float specifying the turbulence constant Cw3. This member is applicable only when *turbulenceModel*=SPALART. The default value is 2.0.

turbSigma

A Float specifying the turbulence constant Sigma. This member is applicable only when *turbulenceModel*=SPALART. The default value is 0.6667.

turbKappa

A Float specifying the turbulence constant Kappa. This member is applicable only when *turbulenceModel*=SPALART. The default value is 0.41.

turbCmu

A Float specifying the turbulence constant Cmu. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 0.085.

turbCeps1

A Float specifying the turbulence constant cps1. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 1.42.

turbCeps2t

A Float specifying the turbulence constant cps2t. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 1.68.

turbSigma_k

A Float specifying the turbulence constant sigma_k. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 0.72.

turbSigma_eps

A Float specifying the turbulence constant sigma_eps. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 0.72.

turbBeta

A Float specifying the turbulence constant beta. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 0.012.

turbLambda0

A Float specifying the turbulence constant lambda0. This member is applicable only when *turbulenceModel*=KEPS_RNG. The default value is 4.38.

Return value

A FlowStep object.

Exceptions

RangeError.

49.12.2 setValues(...)

This method modifies the FlowStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FlowStep** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

49.12.3 Members

The FlowStep object has members with the same names and descriptions as the arguments to the **FlowStep** method.

In addition, the FlowStep object can have the following members:

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

STEP COMMANDS (STEP)

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.12.4 Corresponding analysis keywords

*CFD

*STEP

49.13 FrequencyStep object

The FrequencyStep object is used to perform eigenvalue extraction to calculate the natural frequencies and corresponding mode shapes of a system.

The FrequencyStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.13.1 FrequencyStep(...)

This method creates a FrequencyStep object.

Path

```
mdb.models[name].FrequencyStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

eigensolver

A SymbolicConstant specifying the eigensolver. Possible values are LANCZOS, SUBSPACE, and AMS.

The following optional arguments are ignored unless *eigensolver*=LANCZOS: *blockSize*, *maxBlocks*, *normalization*, *propertyEvaluationFrequency*.

The following optional arguments are ignored unless *eigensolver*=LANCZOS or AMS: *minEigen*, *maxEigen*, and *acousticCoupling*.

The following optional arguments are ignored unless *eigensolver*=AMS: *projectDamping*, *acousticRangeFactor*, *substructureCutoffMultiplier*, *firstCutoffMultiplier*, *secondCutoffMultiplier*, *residualModeRegion*, *regionalModeDof*, and *limitSavedEigenvectorRegion*.

Optional arguments*numEigen*

The SymbolicConstant ALL or an Int specifying the number of eigenvalues to be calculated or ALL. The default value is ALL.

description

A String specifying a description of the new step. The default value is an empty string.

shift

A Float specifying the shift point in cycles per time. The default value is 0.0.

minEigen

None or a Float specifying the minimum frequency of interest in cycles per time. The default value is **None**.

maxEigen

None or a Float specifying the maximum frequency of interest in cycles per time. The default value is **None**.

vectors

None or an Int specifying the number of vectors used in the iteration. The default is the minimum of $(2n, n + 8)$, where n is the number of eigenvalues requested. The default value is **None**.

maxIterations

An Int specifying the maximum number of iterations. The default value is 30.

blockSize

A SymbolicConstant specifying the size of the Lanczos block steps. The default value is DEFAULT.

maxBlocks

A SymbolicConstant specifying the maximum number of Lanczos block steps within each Lanczos run. The default value is DEFAULT.

normalization

A SymbolicConstant specifying the method for normalizing eigenvectors. Possible values are DISPLACEMENT and MASS. The default value is DISPLACEMENT.

A value of DISPLACEMENT indicates normalizing the eigenvectors so that the largest displacement entry in each vector is unity. A value of MASS indicates normalizing the eigenvectors with respect to the structure's mass matrix, which results in scaling the eigenvectors so that the generalized mass for each vector is unity.

propertyEvaluationFrequency

None or a Float specifying the frequency at which to evaluate frequency-dependent properties for viscoelasticity, springs, and dashpots during the eigenvalue extraction. If the value is **None**, the analysis product will evaluate the stiffness associated with frequency-dependent springs and dashpots at zero frequency and will not consider the stiffness contributions from frequency-domain viscoelasticity in the step. The default value is **None**.

STEP COMMANDS (STEP)

projectDamping

A Boolean specifying whether to include projection of viscous and structural damping operators during AMS eigenvalue extraction. Valid only when *eigenSolver*=AMS. The default value is ON.

acousticCoupling

A SymbolicConstant specifying the type of acoustic-structural coupling in models with acoustic and structural elements coupled using the *TIE option or in models with ASI-type elements. Possible values are AC_ON, AC_OFF, and AC_PROJECTION. The default value is AC_ON.

acousticRangeFactor

A Float specifying the ratio of the maximum acoustic frequency to the maximum structural frequency. The default value is 1.0.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

simLinearDynamics

A Boolean specifying whether to activate the SIM-based linear dynamics procedures. The default value is OFF.

residualModes

A Boolean specifying whether to include residual modes from an immediately preceding Static, Linear Perturbation step. The default value is OFF.

substructureCutoffMultiplier

A Float specifying the cutoff frequency for substructure eigenproblems, defined as a multiplier of the maximum frequency of interest. The default value is 5.0.

firstCutoffMultiplier

A Float specifying the first cutoff frequency for a reduced eigenproblem, defined as a multiplier of the maximum frequency of interest. The default value is 1.7.

secondCutoffMultiplier

A Float specifying the second cutoff frequency for a reduced eigenproblem defined as a multiplier of the maximum frequency of interest. The default value is 1.1.

residualModeRegion

None or a sequence of Strings specifying the name of a region for which residual modes are requested. The default value is **None**.

residualModeDof

None or a sequence of Ints specifying the degree of freedom for which residual modes are requested. The default value is **None**.

limitSavedEigenvectorRegion

None or a Region object specifying a region for which eigenvectors should be saved or the SymbolicConstant None representing the whole model. The default value is **None**.

Return value

A FrequencyStep object.

Exceptions

RangeError.

49.13.2 setValues(...)

This method modifies the FrequencyStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FrequencyStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.13.3 Members

The FrequencyStep object can have the following members:

name

A String specifying the repository key.

eigsolver

A SymbolicConstant specifying the eigsolver. Possible values are LANCZOS, SUBSPACE, and AMS.

STEP COMMANDS (STEP)

The following optional arguments are ignored unless *eigensolver*=LANCZOS: *blockSize*, *maxBlocks*, *normalization*, *propertyEvaluationFrequency*.

The following optional arguments are ignored unless *eigensolver*=LANCZOS or AMS: *minEigen*, *maxEigen*, and *acousticCoupling*.

The following optional arguments are ignored unless *eigensolver*=AMS: *projectDamping*, *acousticRangeFactor*, *substructureCutoffMultiplier*, *firstCutoffMultiplier*, *secondCutoffMultiplier*, *residualModeRegion*, *regionalModeDof*, and *limitSavedEigenvectorRegion*.

numEigen

The SymbolicConstant ALL or an Int specifying the number of eigenvalues to be calculated or ALL. The default value is ALL.

shift

A Float specifying the shift point in cycles per time. The default value is 0.0.

minEigen

None or a Float specifying the minimum frequency of interest in cycles per time. The default value is **None**.

maxEigen

None or a Float specifying the maximum frequency of interest in cycles per time. The default value is **None**.

vectors

None or an Int specifying the number of vectors used in the iteration. The default is the minimum of $(2n, n + 8)$, where n is the number of eigenvalues requested. The default value is **None**.

maxIterations

An Int specifying the maximum number of iterations. The default value is 30.

blockSize

A SymbolicConstant specifying the size of the Lanczos block steps. The default value is DEFAULT.

maxBlocks

A SymbolicConstant specifying the maximum number of Lanczos block steps within each Lanczos run. The default value is DEFAULT.

normalization

A SymbolicConstant specifying the method for normalizing eigenvectors. Possible values are DISPLACEMENT and MASS. The default value is DISPLACEMENT.

A value of DISPLACEMENT indicates normalizing the eigenvectors so that the largest displacement entry in each vector is unity. A value of MASS indicates normalizing the eigenvectors with respect to the structure's mass matrix, which results in scaling the eigenvectors so that the generalized mass for each vector is unity.

propertyEvaluationFrequency

None or a Float specifying the frequency at which to evaluate frequency-dependent properties for viscoelasticity, springs, and dashpots during the eigenvalue extraction. If the value is **None**,

the analysis product will evaluate the stiffness associated with frequency-dependent springs and dashpots at zero frequency and will not consider the stiffness contributions from frequency-domain viscoelasticity in the step. The default value is **None**.

projectDamping

A Boolean specifying whether to include projection of viscous and structural damping operators during AMS eigenvalue extraction. Valid only when *eigenSolver*=AMS. The default value is ON.

acousticCoupling

A SymbolicConstant specifying the type of acoustic-structural coupling in models with acoustic and structural elements coupled using the *TIE option or in models with ASI-type elements. Possible values are AC_ON, AC_OFF, and AC_PROJECTION. The default value is AC_ON.

acousticRangeFactor

A Float specifying the ratio of the maximum acoustic frequency to the maximum structural frequency. The default value is 1.0.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

simLinearDynamics

A Boolean specifying whether to activate the SIM-based linear dynamics procedures. The default value is OFF.

residualModes

A Boolean specifying whether to include residual modes from an immediately preceding Static, Linear Perturbation step. The default value is OFF.

substructureCutoffMultiplier

A Float specifying the cutoff frequency for substructure eigenproblems, defined as a multiplier of the maximum frequency of interest. The default value is 5.0.

firstCutoffMultiplier

A Float specifying the first cutoff frequency for a reduced eigenproblem, defined as a multiplier of the maximum frequency of interest. The default value is 1.7.

secondCutoffMultiplier

A Float specifying the second cutoff frequency for a reduced eigenproblem defined as a multiplier of the maximum frequency of interest. The default value is 1.1.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

STEP COMMANDS (STEP)

residualModeRegion

None or a tuple of Strings specifying the name of a region for which residual modes are requested. The default value is **None**.

residualModeDof

None or a tuple of Ints specifying the degree of freedom for which residual modes are requested. The default value is **None**.

limitSavedEigenvectorRegion

None or a Region object specifying a region for which eigenvectors should be saved or the SymbolicConstant None representing the whole model. The default value is **None**.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE

- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

STEP COMMANDS (STEP)

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.13.4 Corresponding analysis keywords

*FREQUENCY

*STEP

49.14 GeostaticStep object

The GeostaticStep object is used to verify that the geostatic stress field is in equilibrium with the applied loads and boundary conditions on the model and to iterate, if needed, to obtain equilibrium.

The GeostaticStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.14.1 GeostaticStep(...)

This method creates a GeostaticStep object.

Path

```
mdb.models[name].GeostaticStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

utol

None or a Float specifying the tolerance for maximum change of displacements. The default value is **None**.

timePeriod

A Float specifying the total time period. The default value is 1.0.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

STEP COMMANDS (STEP)

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

Return value

A GeostaticStep object.

Exceptions

RangeError.

49.14.2 setValues(...)

This method modifies the GeostaticStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **GeostaticStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.14.3 Members

The GeostaticStep object can have the following members:

name

A String specifying the repository key.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

utol

None or a Float specifying the tolerance for maximum change of displacements. The default value is **None**.

timePeriod

A Float specifying the total time period. The default value is 1.0.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

STEP COMMANDS (STEP)

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

Note: This parameter is ignored unless *timeIncrementationMethod*=AUTOMATIC.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL

- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

STEP COMMANDS (STEP)

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.14.4 Corresponding analysis keywords

*GEOSTATIC

*STEP

49.15 HeatTransferStep object

The HeatTransferStep object is used to control uncoupled heat transfer for either transient or steady-state response.

The HeatTransferStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.15.1 HeatTransferStep(...)

This method creates a HeatTransferStep object.

Path

```
mdb.models[name].HeatTransferStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of 0.8 times the initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

end

None or a Float specifying the temperature change rate (temperature per time) used to define steady state. When all nodal temperatures are changing at less than this rate, the solution terminates. The default value is **None**.

Note: This parameter is ignored unless *response*=STEADY_STATE.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment during a transient heat transfer analysis. The default value is 0.0.

mxdem

A Float specifying the maximum allowable emissivity change with temperature and field variables during an increment. The default value is 0.1.

STEP COMMANDS (STEP)

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

Return value

A HeatTransferStep object.

Exceptions

RangeError.

49.15.2 setValues(...)

This method modifies the HeatTransferStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **HeatTransferStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.15.3 Members

The HeatTransferStep object can have the following members:

name

A String specifying the repository key.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of 0.8 times the initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

STEP COMMANDS (STEP)

end

None or a Float specifying the temperature change rate (temperature per time) used to define steady state. When all nodal temperatures are changing at less than this rate, the solution terminates. The default value is **None**.

Note: This parameter is ignored unless *response*=STEADY_STATE.

deltmx

A Float specifying the maximum temperature change to be allowed in an increment during a transient heat transfer analysis. The default value is 0.0.

mxdem

A Float specifying the maximum allowable emissivity change with temperature and field variables during an increment. The default value is 0.1.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT

STEP COMMANDS (STEP)

- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.15.4 Corresponding analysis keywords

*HEAT TRANSFER

*STEP

49.16 ImplicitDynamicsStep object

The ImplicitDynamicsStep object is used to provide direct integration of a dynamic stress/displacement response in Abaqus/Standard analyses and is generally used for nonlinear cases.

The ImplicitDynamicsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.16.1 ImplicitDynamicsStep(...)

This method creates an ImplicitDynamicsStep object.

Path

```
mdb.models[name].ImplicitDynamicsStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period of the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is based on the previous step.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

STEP COMMANDS (STEP)

application

A SymbolicConstant specifying the application type of the step. Possible values are ANALYSIS_PRODUCT_DEFAULT, TRANSIENT_FIDELITY, MODERATE DISSIPATION, and QUASI_STATIC. The default value is ANALYSIS_PRODUCT_DEFAULT.

adiabatic

A Boolean specifying whether an adiabatic stress analysis is to be performed. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

The SymbolicConstant DEFAULT or a Float specifying the maximum time increment allowed.

hafTolMethod

A SymbolicConstant specifying the way of specifying half-increment residual tolerance with the automatic time incrementation scheme. Possible values are ANALYSIS_PRODUCT_DEFAULT, VALUE, and SCALE. The default value is VALUE.

haftol

None or a Float specifying the half-increment residual tolerance to be used with the automatic time incrementation scheme. The default value is **None**.

halfIncScaleFactor

None or a Float specifying the half-increment residual tolerance scale factor to be used with the automatic time incrementation scheme. The default value is **None**.

nohaf

A Boolean specifying whether to suppress calculation of the half-increment residual. The default value is OFF.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

alpha

The SymbolicConstant DEFAULT or a Float specifying the nondefault value of the numerical (artificial) damping control parameter, α , in the implicit operator. Possible values are $-.333 < \alpha < 0$. The default value is DEFAULT.

initialConditions

A SymbolicConstant specifying whether accelerations should be calculated or recalculated at the beginning of the step. Possible values are DEFAULT, BYPASS, and ALLOW. The default value is DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, PARABOLIC, VELOCITY_PARABOLIC, and ANALYSIS_PRODUCT_DEFAULT. The default value is ANALYSIS_PRODUCT_DEFAULT.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed have been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: You should set *noStop*=OFF only in special cases when you have a thorough understanding of how to interpret the results.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

Return value

An ImplicitDynamicsStep object.

Exceptions

RangeError.

49.16.2 **setValues(...)**

This method modifies the `ImplicitDynamicsStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ImplicitDynamicsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.16.3 **Members**

The `ImplicitDynamicsStep` object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the total time period of the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is based on the previous step.

matrixStorage

A `SymbolicConstant` specifying the type of matrix storage. Possible values are `SYMMETRIC`, `UNSMMETRIC`, and `SOLVER_DEFAULT`. The default value is `SOLVER_DEFAULT`.

application

A `SymbolicConstant` specifying the application type of the step. Possible values are `ANALYSIS_PRODUCT_DEFAULT`, `TRANSIENT_FIDELITY`, `MODERATE_DISSIPATION`, and `QUASI_STATIC`. The default value is `ANALYSIS_PRODUCT_DEFAULT`.

adiabatic

A Boolean specifying whether an adiabatic stress analysis is to be performed. The default value is `OFF`.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

The SymbolicConstant DEFAULT or a Float specifying the maximum time increment allowed.

hafTolMethod

A SymbolicConstant specifying the way of specifying half-increment residual tolerance with the automatic time incrementation scheme. Possible values are ANALYSIS_PRODUCT_DEFAULT, VALUE, and SCALE. The default value is VALUE.

haftol

None or a Float specifying the half-increment residual tolerance to be used with the automatic time incrementation scheme. The default value is **None**.

halfIncScaleFactor

None or a Float specifying the half-increment residual tolerance scale factor to be used with the automatic time incrementation scheme. The default value is **None**.

nohaf

A Boolean specifying whether to suppress calculation of the half-increment residual. The default value is OFF.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

alpha

The SymbolicConstant DEFAULT or a Float specifying the nondefault value of the numerical (artificial) damping control parameter, α , in the implicit operator. Possible values are $-0.333 < \alpha < 0$. The default value is DEFAULT.

initialConditions

A SymbolicConstant specifying whether accelerations should be calculated or recalculated at the beginning of the step. Possible values are DEFAULT, BYPASS, and ALLOW. The default value is DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, PARABOLIC,

STEP COMMANDS (STEP)

VELOCITY_PARABOLIC, and ANALYSIS_PRODUCT_DEFAULT. The default value is ANALYSIS_PRODUCT_DEFAULT.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed have been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: You should set *noStop*=OFF only in special cases when you have a thorough understanding of how to interpret the results.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY

- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

STEP COMMANDS (STEP)

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.16.4 Corresponding analysis keywords

*DYNAMIC

*STEP

49.17 InitialStep object

The InitialStep object is a placeholder that you cannot create, delete, or modify. The InitialStep object exists in every model by default and is used to allow the preexisting boundary conditions and interactions to be defined in the model.

The InitialStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.17.1 Members

The InitialStep object can have the following members:

name

A String specifying the repository key.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL

STEP COMMANDS (STEP)

- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.18 MassDiffusionStep object

The MassDiffusionStep object is used to control uncoupled transient or steady-state mass diffusion analysis.

The MassDiffusionStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.18.1 MassDiffusionStep(...)

This method creates a MassDiffusionStep object.

Path

```
mdb.models[name].MassDiffusionStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

STEP COMMANDS (STEP)

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of 0.8 times the initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

dcmax

A Float specifying the maximum normalized concentration change to be allowed in an increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

Return value

A MassDiffusionStep object.

Exceptions

RangeError.

49.18.2 setValues(...)

This method modifies the MassDiffusionStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **MassDiffusionStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.18.3 Members

The MassDiffusionStep object can have the following members:

name

A String specifying the repository key.

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period. The default value is 1.0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of 0.8 times the initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

STEP COMMANDS (STEP)

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

dcmax

A Float specifying the maximum normalized concentration change to be allowed in an increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC

- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

STEP COMMANDS (STEP)

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.18.4 Corresponding analysis keywords

*MASS DIFFUSION

*STEP

49.19 ModalDynamicsStep object

The ModalDynamicsStep object is used to provide dynamic time history response as a linear perturbation procedure using modal superposition.

The ModalDynamicsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.19.1 ModalDynamicsStep(...)

This method creates a ModalDynamicsStep object.

Path

`mdb.models[name].ModalDynamicsStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

continueAnalysis

A Boolean specifying that the step starts with zero initial conditions. The default value is OFF.

timePeriod

A Float specifying the total time period. The default value is 1.0.

incSize

A Float specifying the time increment to be used. The default value is 1.0.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

Return value

A ModalDynamicsStep object.

Exceptions

RangeError.

49.19.2 **setValues(...)**

This method modifies the `ModalDynamicsStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ModalDynamicsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.19.3 **Members**

The `ModalDynamicsStep` object can have the following members:

name

A String specifying the repository key.

continueAnalysis

A Boolean specifying that the step starts with zero initial conditions. The default value is OFF.

timePeriod

A Float specifying the total time period. The default value is 1.0.

incSize

A Float specifying the time increment to be used. The default value is 1.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER

STEP COMMANDS (STEP)

- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.19.4 Corresponding analysis keywords

*DAMPING

*MODAL DAMPING

*MODAL DYNAMIC

*STEP

49.20 RandomResponseStep object

The RandomResponseStep object is used to give the linearized response of a model to random excitation.

The RandomResponseStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.20.1 RandomResponseStep(...)

This method creates a RandomResponseStep object.

Path

```
mdb.models[name].RandomResponseStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

STEP COMMANDS (STEP)

freq

A RandomResponseFrequencyArray object specifying frequencies over ranges of modes.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

scale

A SymbolicConstant specifying the frequency scale. Possible values are LINEAR and LOG. The default value is LOG.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

structuralDamping

A StructuralDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

structuralDampingByFrequency

A StructuralDampingByFrequency object.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A RandomResponseStep object.

Exceptions

RangeError.

49.20.2 setValues(...)

This method modifies the RandomResponseStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **RandomResponseStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.20.3 Members

The RandomResponseStep object can have the following members:

name

A String specifying the repository key.

scale

A SymbolicConstant specifying the frequency scale. Possible values are LINEAR and LOG. The default value is LOG.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

freq

A RandomResponseFrequencyArray object specifying frequencies over ranges of modes.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

structuralDamping

A StructuralDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

STEP COMMANDS (STEP)

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

structuralDampingByFrequency

A StructuralDampingByFrequency object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION

- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.20.4 Corresponding analysis keywords

- *DAMPING
- *MODAL DAMPING
- *RANDOM RESPONSE
- *STEP

49.21 ResponseSpectrumStep object

The ResponseSpectrumStep object is used to calculate estimates of peak values of displacements and stresses based on user-supplied response spectra and on the natural modes of the system.

The ResponseSpectrumStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.21.1 ResponseSpectrumStep(...)

This method creates a ResponseSpectrumStep object.

Path

```
mdb.models[name].ResponseSpectrumStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

components

A ResponseSpectrumComponentArray object.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

comp

A SymbolicConstant specifying the order and method used to sum the components. Possible values are SINGLE_DIRECTION, MULTIPLE_DIRECTION_ABSOLUTE_SUM, MULTIPLE_DIRECTION_SRSS_SUM, MULTIPLE_DIRECTION_THIRTY_PERCENT_RULE, and MULTIPLE_DIRECTION_FORTY_PERCENT_RULE. The default value is SINGLE_DIRECTION.

sum

A SymbolicConstant specifying the method used to sum the components. Possible values are ABS, CQC, NRL, SRSS, TENP, DSC, and GRP. The default value is ABS.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A ResponseSpectrumStep object.

Exceptions

RangeError.

49.21.2 setValues(...)

This method modifies the ResponseSpectrumStep object.

Arguments**Required arguments**

None.

STEP COMMANDS (STEP)

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ResponseSpectrumStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.21.3 Members

The **ResponseSpectrumStep** object can have the following members:

name

A String specifying the repository key.

comp

A **SymbolicConstant** specifying the order and method used to sum the components. Possible values are **SINGLE_DIRECTION**, **MULTIPLE_DIRECTION_ABSOLUTE_SUM**, **MULTIPLE_DIRECTION_SRSS_SUM**, **MULTIPLE_DIRECTION_THIRTY_PERCENT_RULE**, and **MULTIPLE_DIRECTION_FORTY_PERCENT_RULE**. The default value is **SINGLE_DIRECTION**.

sum

A **SymbolicConstant** specifying the method used to sum the components. Possible values are **ABS**, **CQC**, **NRL**, **SRSS**, **TENP**, **DSC**, and **GRP**. The default value is **ABS**.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

components

A **ResponseSpectrumComponentArray** object.

directDamping

A **DirectDamping** object.

compositeDamping

A **CompositeDamping** object.

rayleighDamping

A **RayleighDamping** object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

structuralDamping

A StructuralDamping object.

structuralDampingByFrequency

A StructuralDampingByFrequency object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM

STEP COMMANDS (STEP)

- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.21.4 Corresponding analysis keywords

*RESPONSE SPECTRUM

*STEP

49.22 SoilsStep object

The SoilsStep object is used to specify transient (consolidation) or steady-state response analysis of partially or fully saturated fluid-filled porous media.

The SoilsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.22.1 SoilsStep(...)

This method creates a SoilsStep object.

Path

```
mdb.models[name].SoilsStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

STEP COMMANDS (STEP)

response

A SymbolicConstant specifying the analysis type. Possible values are STEADY_STATE and TRANSIENT. The default value is TRANSIENT.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not NONE. The default value is 2×10^{-4} .

creep

A Boolean specifying whether a creep response occurs during this step. The default value is ON.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

utol

None or a Float specifying the maximum pore pressure change permitted in any increment (in pressure units) in a transient consolidation analysis. The default value is **None**.

cetol

A Float specifying the maximum allowable difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

Return value

A SoilsStep object.

Exceptions

RangeError.

49.22.2 **setValues(...)**

This method modifies the `SoilsStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SoilsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

`RangeError`.

49.22.3 **Members**

The `SoilsStep` object can have the following members:

name

A String specifying the repository key.

response

A `SymbolicConstant` specifying the analysis type. Possible values are `STEADY_STATE` and `TRANSIENT`. The default value is `TRANSIENT`.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is `OFF`.

stabilizationMethod

A `SymbolicConstant` specifying the stabilization type. Possible values are `NONE`, `DISSIPATED_ENERGY_FRACTION`, and `DAMPING_FACTOR`. The default value is `NONE`.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not `NONE`. The default value is 2×10^{-4} .

creep

A Boolean specifying whether a creep response occurs during this step. The default value is `ON`.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

end

A SymbolicConstant specifying the time period to be analyzed in a transient analysis. Possible values are PERIOD and SS. The default value is PERIOD.

utol

None or a Float specifying the maximum pore pressure change permitted in any increment (in pressure units) in a transient consolidation analysis. The default value is **None**.

cetol

A Float specifying the maximum allowable difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. The default value is STEP. Possible values are STEP and RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

STEP COMMANDS (STEP)

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE

- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

STEP COMMANDS (STEP)

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.22.4 Corresponding analysis keywords

*SOILS

*STEP

49.23 StaticLinearPerturbationStep object

The StaticLinearPerturbationStep object is used to indicate that the static step should be analyzed as a linear perturbation load step.

The StaticLinearPerturbationStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.23.1 StaticLinearPerturbationStep(...)

This method creates a StaticLinearPerturbationStep object.

Path

```
mdb.models[name].StaticLinearPerturbationStep
```

Required arguments*name*

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments*description*

A String specifying a description of the new step. The default value is an empty string.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A StaticLinearPerturbationStep object.

Exceptions

RangeError.

49.23.2 setValues(...)

This method modifies the StaticLinearPerturbationStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StaticLinearPerturbationStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.23.3 Members

The StaticLinearPerturbationStep object can have the following members:

name

A String specifying the repository key.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC

- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

STEP COMMANDS (STEP)

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.23.4 Corresponding analysis keywords

*STATIC

*STEP

49.24 StaticRiksStep object

The StaticRiksStep object is used to indicate that the step should be analyzed as a static load step using the modified Riks method for proportional loading cases.

The StaticRiksStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.24.1 StaticRiksStep(...)

This method creates a StaticRiksStep object.

Path

`mdb.models [name] .StaticRiksStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

adiabatic

A Boolean specifying whether to perform an adiabatic stress analysis. The default value is OFF.

maxLPF

None or a Float specifying the maximum value of the load proportionality factor. The default value is **None**.

nodeOn

A Boolean specifying whether to monitor the finishing displacement value at a node. The default value is OFF.

maximumDisplacement

A Float specifying the value of the total displacement (or rotation) at the node and degree of freedom that, if crossed during an increment, ends the step at the current increment. This argument is required when *nodeOn*=ON. The default value is 0.0.

dof

An Int specifying the degree of freedom being monitored. This argument is required when *nodeOn*=ON. The default value is 0.

region

A Region object specifying the vertex at which the finishing displacement value is being monitored. This argument is required when *nodeOn*=ON.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

STEP COMMANDS (STEP)

totalArcLength

A Float specifying the total load proportionality factor associated with the load in this step. The default value is 1.0.

initialArcInc

A Float specifying the initial load proportionality factor. The default value is the total load proportionality factor for the step.

minArcInc

A Float specifying the minimum arc length increment allowed. The default value is the smaller of the suggested initial load proportionality factor or 10^{-5} times the total load proportionality factor for the step.

maxArcInc

A Float specifying the maximum arc length increment allowed. The default value is the total load proportionality factor for the step.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

fullyPlastic

A String specifying the name of the region being monitored for fully plastic behavior. The default value is an empty string.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed have been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: You should set *noStop*=ON only in special cases when you have a thorough understanding of how to interpret the results.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

useLongTermSolution

A Boolean specifying whether to obtain the fully relaxed long-term elastic solution with time-domain viscoelasticity or the long-term elastic-plastic solution for two-layer viscoplasticity. The default value is OFF.

convertSDI

A `SymbolicConstant` specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are `PROPAGATED`, `CONVERT_SDI_OFF`, and `CONVERT_SDI_ON`. The default value is `PROPAGATED`.

Return value

A `StaticRiksStep` object.

Exceptions

`RangeError`.

49.24.2 setValues(...)

This method modifies the `StaticRiksStep` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StaticRiksStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

`None`

Exceptions

`RangeError`.

49.24.3 Members

The `StaticRiksStep` object can have the following members:

name

A String specifying the repository key.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is `OFF`.

adiabatic

A Boolean specifying whether to perform an adiabatic stress analysis. The default value is `OFF`.

STEP COMMANDS (STEP)

maxLPF

None or a Float specifying the maximum value of the load proportionality factor. The default value is **None**.

nodeOn

A Boolean specifying whether to monitor the finishing displacement value at a node. The default value is OFF.

maximumDisplacement

A Float specifying the value of the total displacement (or rotation) at the node and degree of freedom that, if crossed during an increment, ends the step at the current increment. This argument is required when *nodeOn*=ON. The default value is 0.0.

dof

An Int specifying the degree of freedom being monitored. This argument is required when *nodeOn*=ON. The default value is 0.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

totalArcLength

A Float specifying the total load proportionality factor associated with the load in this step. The default value is 1.0.

initialArcInc

A Float specifying the initial load proportionality factor. The default value is the total load proportionality factor for the step.

minArcInc

A Float specifying the minimum arc length increment allowed. The default value is the smaller of the suggested initial load proportionality factor or 10^{-5} times the total load proportionality factor for the step.

maxArcInc

A Float specifying the maximum arc length increment allowed. The default value is the total load proportionality factor for the step.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed have been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: You should set *noStop*=ON only in special cases when you have a thorough understanding of how to interpret the results.

useLongTermSolution

A Boolean specifying whether to obtain the fully relaxed long-term elastic solution with time-domain viscoelasticity or the long-term elastic-plastic solution for two-layer viscoplasticity. The default value is OFF.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

fullyPlastic

A String specifying the name of the region being monitored for fully plastic behavior. The default value is an empty string.

region

A Region object specifying the vertex at which the finishing displacement value is being monitored. This argument is required when *nodeOn*=ON.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL

STEP COMMANDS (STEP)

- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.24.4 Corresponding analysis keywords

*STATIC

*STEP

49.25 StaticStep object

The StaticStep object is used to indicate that the step should be analyzed as a static load step.

The StaticStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.25.1 StaticStep(...)

This method creates a StaticStep object.

Path

`mdb.models[name].StaticStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not NONE. The default value is 2×10^{-4} .

adiabatic

A Boolean specifying whether to perform an adiabatic stress analysis. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

fullyPlastic

A String specifying the region being monitored for fully plastic behavior. The default value is an empty string.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed has been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: You should set *noStop*=ON only in special cases when you have a thorough understanding of how to interpret the results.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

useLongTermSolution

A Boolean specifying whether to obtain the fully relaxed long-term elastic solution with time-domain viscoelasticity or the long-term elastic-plastic solution for two-layer viscoplasticity. The default value is OFF.

STEP COMMANDS (STEP)

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

Return value

A StaticStep object.

Exceptions

RangeError.

49.25.2 setValues(...)

This method modifies the StaticStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **StaticStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.25.3 Members

The StaticStep object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not NONE. The default value is 2×10^{-4} .

adiabatic

A Boolean specifying whether to perform an adiabatic stress analysis. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default value is the total time period for the step.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

STEP COMMANDS (STEP)

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is RAMP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

noStop

A Boolean specifying whether to accept the solution to an increment after the maximum number of iterations allowed has been completed, even if the equilibrium tolerances are not satisfied. The default value is OFF.

WARNING: *You should set noStop=ON only in special cases when you have a thorough understanding of how to interpret the results.*

useLongTermSolution

A Boolean specifying whether to obtain the fully relaxed long-term elastic solution with time-domain viscoelasticity or the long-term elastic-plastic solution for two-layer viscoplasticity. The default value is OFF.

solutionTechnique

A SymbolicConstant specifying the technique used to solve nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

fullyPlastic

A String specifying the region being monitored for fully plastic behavior. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM

STEP COMMANDS (STEP)

- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.25.4 Corresponding analysis keywords

*STATIC

*STEP

49.26 SteadyStateDirectStep object

The SteadyStateDirectStep object is used to calculate the linearized steady-state response of the system to harmonic excitation in terms of the physical degrees of freedom of the model.

The SteadyStateDirectStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.26.1 SteadyStateDirectStep(...)

This method creates a SteadyStateDirectStep object.

Path

```
mdb.models[name].SteadyStateDirectStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

frequencyRange

A SteadyStateDirectFrequencyArray object.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

STEP COMMANDS (STEP)

factorization

A SymbolicConstant specifying whether damping terms are to be ignored so that a real, rather than a complex, system matrix is factored. Possible values are REAL_ONLY and COMPLEX. The default value is COMPLEX.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is OFF.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

Return value

A SteadyStateDirectStep object.

Exceptions

RangeError.

49.26.2 setValues(...)

This method modifies the SteadyStateDirectStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SteadyStateDirectStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.26.3 Members

The SteadyStateDirectStep object can have the following members:

name

A String specifying the repository key.

factorization

A SymbolicConstant specifying whether damping terms are to be ignored so that a real, rather than a complex, system matrix is factored. Possible values are REAL_ONLY and COMPLEX. The default value is COMPLEX.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is OFF.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

frequencyRange

A SteadyStateDirectFrequencyArray object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

STEP COMMANDS (STEP)

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.26.4 Corresponding analysis keywords

*STEADY STATE DYNAMICS

*STEP

49.27 SteadyStateModalStep object

The SteadyStateModalStep object is used to calculate the linearized steady-state response of the system to harmonic excitation.

The SteadyStateModalStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.27.1 SteadyStateModalStep(...)

This method creates a SteadyStateModalStep object.

Path

```
mdb.models[name].SteadyStateModalStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

frequencyRange

A SteadyStateModalFrequencyArray object.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

structuralDamping

A StructuralDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

structuralDampingByFrequency

A StructuralDampingByFrequency object.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is ON.

Return value

A SteadyStateModalStep object.

Exceptions

RangeError.

49.27.2 **setValues(...)**

This method modifies the SteadyStateModalStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SteadyStateModalStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.27.3 Members

The SteadyStateModalStep object can have the following members:

name

A String specifying the repository key.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is ON.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

frequencyRange

A SteadyStateModalFrequencyArray object.

directDamping

A DirectDamping object.

compositeDamping

A CompositeDamping object.

rayleighDamping

A RayleighDamping object.

structuralDamping

A StructuralDamping object.

directDampingByFrequency

A DirectDampingByFrequency object.

rayleighDampingByFrequency

A RayleighDampingByFrequency object.

structuralDampingByFrequency

A StructuralDampingByFrequency object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

STEP COMMANDS (STEP)

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.27.4 Corresponding analysis keywords

*DAMPING

*MODAL DAMPING

*STEADY STATE DYNAMICS

*STEP

49.28 **SteadyStateSubspaceStep object**

The SteadyStateSubspaceStep object is used to calculate the linearized steady-state response of the system to harmonic excitation on the basis of the subspace projection method.

The SteadyStateSubspaceStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.28.1 **SteadyStateSubspaceStep(...)**

This method creates a SteadyStateSubspaceStep object.

Path

```
mdb.models[name].SteadyStateSubspaceStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

frequencyRange

A SteadyStateSubspaceFrequencyArray object.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

factorization

A SymbolicConstant specifying whether damping terms are to be ignored so that a real, rather than a complex, system matrix is factored. Possible values are REAL_ONLY and COMPLEX. The default value is COMPLEX.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

STEP COMMANDS (STEP)

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is ON.

projection

A SymbolicConstant specifying how often to perform subspace projections onto the modal subspace. Possible values are ALL_FREQUENCIES, CONSTANT, EIGENFREQUENCY, PROPERTY_CHANGE, and RANGE. The default value is ALL_FREQUENCIES.

maxDampingChange

A Float specifying the maximum relative change in damping material properties before a new projection is to be performed. The default value is 0.1.

maxStiffnessChange

A Float specifying the maximum relative change in stiffness material properties before a new projection is to be performed. The default value is 0.1.

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

Return value

A SteadyStateSubspaceStep object.

Exceptions

RangeError.

49.28.2 setValues(...)

This method modifies the SteadyStateSubspaceStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SteadyStateSubspaceStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.28.3 Members

The SteadyStateSubspaceStep object can have the following members:

name

A String specifying the repository key.

factorization

A SymbolicConstant specifying whether damping terms are to be ignored so that a real, rather than a complex, system matrix is factored. Possible values are REAL_ONLY and COMPLEX. The default value is COMPLEX.

scale

A SymbolicConstant specifying whether a logarithmic or linear scale is used for output. Possible values are LOGARITHMIC and LINEAR. The default value is LOGARITHMIC.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

subdivideUsingEigenfrequencies

A Boolean specifying whether to subdivide each frequency range using the eigenfrequencies of the system. The default value is ON.

projection

A SymbolicConstant specifying how often to perform subspace projections onto the modal subspace. Possible values are ALL_FREQUENCIES, CONSTANT, EIGENFREQUENCY, PROPERTY_CHANGE, and RANGE. The default value is ALL_FREQUENCIES.

maxDampingChange

A Float specifying the maximum relative change in damping material properties before a new projection is to be performed. The default value is 0.1.

maxStiffnessChange

A Float specifying the maximum relative change in stiffness material properties before a new projection is to be performed. The default value is 0.1.

STEP COMMANDS (STEP)

frictionDamping

A Boolean specifying whether to add to the damping matrix contributions due to friction effects. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

frequencyRange

A SteadyStateSubspaceFrequencyArray object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS

- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

STEP COMMANDS (STEP)

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.28.4 Corresponding analysis keywords

*STEADY STATE DYNAMICS

*STEP

49.29 SubspaceDynamicsStep object

The SubspaceDynamicsStep object is used to calculate the linearized steady-state response of the system to harmonic excitation on the basis of the subspace projection method.

The SubspaceDynamicsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.29.1 SubspaceDynamicsStep(...)

This method creates a SubspaceDynamicsStep object.

Path

```
mdb.models[name].SubspaceDynamicsStep
```

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period of the step. The default value is 1.0.

vectors

The SymbolicConstant ALL or an Int specifying the number of modes to be used for subspace projection. The possible value for the SymbolicConstant is ALL. The default value is ALL.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

incSize

A Float specifying the suggested time increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A SubspaceDynamicsStep object.

Exceptions

RangeError.

49.29.2 setValues(...)

This method modifies the SubspaceDynamicsStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SubspaceDynamicsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.29.3 Members

The SubspaceDynamicsStep object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the total time period of the step. The default value is 1.0.

vectors

The SymbolicConstant ALL or an Int specifying the number of modes to be used for subspace projection. The possible value for the SymbolicConstant is ALL. The default value is ALL.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

incSize

A Float specifying the suggested time increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL

- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

STEP COMMANDS (STEP)

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.29.4 Corresponding analysis keywords

*DYNAMIC

*STEP

49.30 SubstructureGenerateStep object

TheSubstructureGenerateStep object is used to generate a substructure.

The SubstructureGenerateStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```


49.30.1 SubstructureGenerateStep(...)

This method creates a SubstructureGenerateStep object.

Path

`mdb.models [name] .SubstructureGenerateStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

substructureIdentifier

A String specifying a unique identifier for the substructure. The default value is an empty string.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

recoveryMatrix

A SymbolicConstant specifying the substructure recovery to be computed. Possible values are WHOLE_MODEL, REGION, and NONE. The default value is WHOLE_MODEL.

recoveryRegion

A Region object specifying the region for substructure recovery. This argument is required when *recoveryMatrix*=REGION.

computeGravityLoadVectors

A Boolean specifying whether to compute the gravity load vectors. The default value is False.

computeReducedMassMatrix

A Boolean specifying whether to compute the reduced mass matrix. The default value is False.

computeReducedStructuralDampingMatrix

A Boolean specifying whether to compute the reduced structural damping matrix. The default value is False.

computeReducedViscousDampingMatrix

A Boolean specifying whether to compute the reduced viscous damping matrix. The default value is False.

evaluateFrequencyDependentProperties

A Boolean specifying whether to evaluate the frequency dependent properties. The default value is False.

STEP COMMANDS (STEP)

frequency

A Float specifying the frequency at which to evaluate the frequency dependent properties. The default value is 0.0.

retainedEigenmodesMethod

A SymbolicConstant specifying the eigenmodes to be retained. Possible values are MODE_RANGE, FREQUENCY_RANGE, and NONE. The default value is NONE.

modeRange

A SubstructureGenerateModesArray object.

frequencyRange

A SubstructureGenerateFrequencyArray object.

globalDampingField

A SymbolicConstant specifying the field to which the global damping factors should be applied. Possible values are ALL, ACOUSTIC, MECHANICAL, and NONE. The default value is NONE.

alphaDampingRatio

A Float specifying the factor to create global Rayleigh mass proportional damping. The default value is 0.0.

betaDampingRatio

A Float specifying the factor to create global Rayleigh stiffness proportional damping. The default value is 0.0.

structuralDampingRatio

A Float specifying the factor to create frequency-independent stiffness proportional structural damping. The default value is 0.0.

viscousDampingControl

A SymbolicConstant specifying the damping control to include the viscous damping matrix. Possible values are ELEMENT, FACTOR, COMBINED, and NONE. The default value is NONE.

structuralDampingControl

A SymbolicConstant specifying the damping control to include the structural damping matrix. Possible values are ELEMENT, FACTOR, COMBINED, and NONE. The default value is NONE.

Return value

A SubstructureGenerateStep object.

Exceptions

RangeError.

49.30.2 setValues(...)

This method modifies the SubstructureGenerateStep object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SubstructureGenerateStep** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

49.30.3 Members

The SubstructureGenerateStep object can have the following members:

name

A String specifying the repository key.

recoveryMatrix

A SymbolicConstant specifying the substructure recovery to be computed. Possible values are WHOLE_MODEL, REGION, and NONE. The default value is WHOLE_MODEL.

frequency

A Float specifying the frequency at which to evaluate the frequency dependent properties. The default value is 0.0.

retainedEigenmodesMethod

A SymbolicConstant specifying the eigenmodes to be retained. Possible values are MODE_RANGE, FREQUENCY_RANGE, and NONE. The default value is NONE.

globalDampingField

A SymbolicConstant specifying the field to which the global damping factors should be applied. Possible values are ALL, ACOUSTIC, MECHANICAL, and NONE. The default value is NONE.

alphaDampingRatio

A Float specifying the factor to create global Rayleigh mass proportional damping. The default value is 0.0.

betaDampingRatio

A Float specifying the factor to create global Rayleigh stiffness proportional damping. The default value is 0.0.

STEP COMMANDS (STEP)

structuralDampingRatio

A Float specifying the factor to create frequency-independent stiffness proportional structural damping. The default value is 0.0.

viscousDampingControl

A SymbolicConstant specifying the damping control to include the viscous damping matrix. Possible values are ELEMENT, FACTOR, COMBINED, and NONE. The default value is NONE.

structuralDampingControl

A SymbolicConstant specifying the damping control to include the structural damping matrix. Possible values are ELEMENT, FACTOR, COMBINED, and NONE. The default value is NONE.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

substructureIdentifier

A String specifying a unique identifier for the substructure. The default value is an empty string.

recoveryRegion

A Region object specifying the region for substructure recovery. This argument is required when *recoveryMatrix*=REGION.

frequencyRange

A SubstructureGenerateFrequencyArray object.

modeRange

A SubstructureGenerateModesArray object.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY
- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC

- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

STEP COMMANDS (STEP)

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.30.4 Corresponding analysis keywords

*SUBSTRUCTURE GENERATE

*STEP

49.31 TempDisplacementDynamicsStep object

The TempDisplacementDynamicsStep object is used to perform a dynamic coupled thermal-stress analysis using explicit integration.

The TempDisplacementDynamicsStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.31.1 TempDisplacementDynamicsStep(...)

This method creates a TempDisplacementDynamicsStep object.

Path

`mdb.models[name].TempDisplacementDynamicsStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the time period of the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are AUTOMATIC_GLOBAL, AUTOMATIC_EBE, FIXED_USER_DEFINED_INC, and FIXED_EBE. The default value is AUTOMATIC_GLOBAL.

maxIncrement

None or a Float specifying the maximum time increment allowed. If there is no upper limit, *maxIncrement*=**None**. The default value is **None**.

scaleFactor

A Float specifying the factor that is used to scale the time increment. This argument is required only when *timeIncrementationMethod*=AUTOMATIC_GLOBAL, AUTOMATIC_EBE, or FIXED_EBE. The default value is 1.0.

userDefinedInc

None or a Float specifying the user-defined time increment. The default value is **None**.

massScaling

A MassScalingArray object specifying mass scaling controls. The default value is PREVIOUS_STEP.

linearBulkViscosity

A Float specifying the linear bulk viscosity parameter, b_1 . The default value is 0.06.

quadBulkViscosity

A Float specifying the quadratic bulk viscosity parameter, b_2 . The default value is 1.2.

STEP COMMANDS (STEP)

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

Return value

A TempDisplacementDynamicsStep object.

Exceptions

RangeError.

49.31.2 setValues(...)

This method modifies the TempDisplacementDynamicsStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TempDisplacementDynamicsStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.31.3 Members

The TempDisplacementDynamicsStep object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the time period of the step. The default value is 1.0.

nlgeom

A Boolean specifying whether geometric nonlinearities should be accounted for during the step. The default value is OFF.

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are AUTOMATIC_GLOBAL, AUTOMATIC_EBE, FIXED_USER_DEFINED_INC, and FIXED_EBE. The default value is AUTOMATIC_GLOBAL.

maxIncrement

None or a Float specifying the maximum time increment allowed. If there is no upper limit, *maxIncrement*=**None**. The default value is **None**.

scaleFactor

A Float specifying the factor that is used to scale the time increment. This argument is required only when *timeIncrementationMethod*=AUTOMATIC_GLOBAL, AUTOMATIC_EBE, or FIXED_EBE. The default value is 1.0.

userDefinedInc

None or a Float specifying the user-defined time increment. The default value is **None**.

linearBulkViscosity

A Float specifying the linear bulk viscosity parameter, b_1 . The default value is 0.06.

quadBulkViscosity

A Float specifying the quadratic bulk viscosity parameter, b_2 . The default value is 1.2.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

massScaling

A MassScalingArray object specifying mass scaling controls. The default value is PREVIOUS_STEP.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY

STEP COMMANDS (STEP)

- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.31.4 Corresponding analysis keywords

- *BULK VISCOSITY
- *DYNAMIC
- *FIXED MASS SCALING
- *STEP
- *VARIABLE MASS SCALING

49.32 ViscoStep object

The ViscoStep object is used to obtain a transient static response in an analysis with time-dependent material behavior (creep, swelling, and viscoelasticity).

The ViscoStep object is derived from the AnalysisStep object.

Access

```
import step
mdb.models[name].steps[name]
```

49.32.1 ViscoStep(...)

This method creates a ViscoStep object.

Path

`mdb.models[name].ViscoStep`

Required arguments

name

A String specifying the repository key.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

Optional arguments

description

A String specifying a description of the new step. The default value is an empty string.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not NONE. The default value is 2×10^{-4} .

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default is the total time period for the step. The default value is 1.0.

integration

A SymbolicConstant specifying which type of integration to use throughout the step. Possible values are IMPLICIT_EXPLICIT and EXPLICIT_ONLY. The default value is IMPLICIT_EXPLICIT.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

maintainAttributes

A Boolean specifying whether to retain attributes from an existing step with the same name. The default value is False.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

STEP COMMANDS (STEP)

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

Return value

A ViscoStep object.

Exceptions

RangeError.

49.32.2 setValues(...)

This method modifies the ViscoStep object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ViscoStep** method, except for the *name*, *previous*, and *maintainAttributes* arguments.

Return value

None

Exceptions

RangeError.

49.32.3 Members

The ViscoStep object can have the following members:

name

A String specifying the repository key.

timePeriod

A Float specifying the total time period. The default value is 1.0.

nlgeom

A Boolean specifying whether to allow for geometric nonlinearity. The default value is OFF.

stabilizationMethod

A SymbolicConstant specifying the stabilization type. Possible values are NONE, DISSIPATED_ENERGY_FRACTION, and DAMPING_FACTOR. The default value is NONE.

stabilizationMagnitude

A Float specifying the damping intensity of the automatic damping algorithm if the problem is expected to be unstable, and *stabilizationMethod* is not NONE. The default value is 2×10^{-4} .

timeIncrementationMethod

A SymbolicConstant specifying the time incrementation method to be used. Possible values are FIXED and AUTOMATIC. The default value is AUTOMATIC.

matrixSolver

A SymbolicConstant specifying the type of solver. Possible values are DIRECT and ITERATIVE. The default value is DIRECT.

matrixStorage

A SymbolicConstant specifying the type of matrix storage. Possible values are SYMMETRIC, UNSYMMETRIC, and SOLVER_DEFAULT. The default value is SOLVER_DEFAULT.

initialInc

A Float specifying the initial time increment. The default value is the total time period for the step.

maxNumInc

An Int specifying the maximum number of increments in a step. The default value is 100.

minInc

A Float specifying the minimum time increment allowed. The default value is the smaller of the suggested initial time increment or 10^{-5} times the total time period.

maxInc

A Float specifying the maximum time increment allowed. The default is the total time period for the step. The default value is 1.0.

integration

A SymbolicConstant specifying which type of integration to use throughout the step. Possible values are IMPLICIT_EXPLICIT and EXPLICIT_ONLY. The default value is IMPLICIT_EXPLICIT.

cetol

A Float specifying the maximum difference in the creep strain increment calculated from the creep strain rates at the beginning and end of the increment. The default value is 0.0.

amplitude

A SymbolicConstant specifying the amplitude variation for loading magnitudes during the step. Possible values are STEP and RAMP. The default value is STEP.

STEP COMMANDS (STEP)

extrapolation

A SymbolicConstant specifying the type of extrapolation to use in determining the incremental solution for a nonlinear analysis. Possible values are NONE, LINEAR, and PARABOLIC. The default value is LINEAR.

solutionTechnique

A SymbolicConstant specifying the technique used to for solving nonlinear equations. Possible values are FULL_NEWTON and QUASI_NEWTON. The default value is FULL_NEWTON.

reformKernel

An Int specifying the number of quasi-Newton iterations allowed before the kernel matrix is reformed.. The default value is 8.

convertSDI

A SymbolicConstant specifying whether to force a new iteration if severe discontinuities occur during an iteration. Possible values are PROPAGATED, CONVERT_SDI_OFF, and CONVERT_SDI_ON. The default value is PROPAGATED.

adaptiveDampingRatio

A Float specifying the maximum allowable ratio of the stabilization energy to the total strain energy and can be used only if *stabilizationMethod* is not NONE. The default value is 0.05.

continueDampingFactors

A Boolean specifying whether this step will carry over the damping factors from the results of the preceding general step. This parameter must be used in conjunction with the *adaptiveDampingRatio* parameter. The default value is OFF.

previous

A String specifying the name of the previous step. The new step appears after this step in the list of analysis steps.

description

A String specifying a description of the new step. The default value is an empty string.

explicit

A SymbolicConstant specifying whether the step has an explicit procedure type (*procedureType*=ANNEAL, DYNAMIC_EXPLICIT, or DYNAMIC_TEMP_DISPLACEMENT).

perturbation

A Boolean specifying whether the step has a perturbation procedure type.

nonmechanical

A Boolean specifying whether the step has a mechanical procedure type.

procedureType

A SymbolicConstant specifying the Abaqus procedure. Possible values are:

- ANNEAL
- BUCKLE
- COMPLEX_FREQUENCY

- COUPLED_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRIC
- DIRECT_CYCLIC
- DYNAMIC_IMPLICIT
- DYNAMIC_EXPLICIT
- DYNAMIC_SUBSPACE
- DYNAMIC_TEMP_DISPLACEMENT
- COUPLED_THERMAL_ELECTRICAL_STRUCTURAL
- FLOW
- FREQUENCY
- GEOSTATIC
- HEAT_TRANSFER
- MASS_DIFFUSION
- MODAL_DYNAMICS
- RANDOM_RESPONSE
- RESPONSE_SPECTRUM
- SOILS
- STATIC_GENERAL
- STATIC_LINEAR_PERTURBATION
- STATIC_RIKS
- STEADY_STATE_DIRECT
- STEADY_STATE_MODAL
- STEADY_STATE_SUBSPACE
- VISCO

suppressed

A Boolean specifying whether the step is suppressed or not. The default value is OFF.

fieldOutputRequestState

A repository of FieldOutputRequestState objects.

historyOutputRequestState

A repository of HistoryOutputRequestState objects.

diagnosticPrint

A DiagnosticPrint object.

monitor

A Monitor object.

restart

A Restart object.

STEP COMMANDS (STEP)

adaptiveMeshConstraintStates

A repository of AdaptiveMeshConstraintState objects.

adaptiveMeshDomains

A repository of AdaptiveMeshDomain objects.

control

A Control object.

solverControl

A SolverControl object.

boundaryConditionStates

A repository of BoundaryConditionState objects.

interactionStates

A repository of InteractionState objects.

loadStates

A repository of LoadState objects.

loadCases

A repository of LoadCase objects.

predefinedFieldStates

A repository of PredefinedFieldState objects.

49.32.4 Corresponding analysis keywords

*VISCO

*STEP

50. Step commands (miscellaneous)

Miscellaneous Step commands are used for configuring controls, damping, and frequency tables.

50.1 CompositeDamping object

A CompositeDamping object contains composite modal damping parameters.

Access

```
import step
mdb.models[name].steps[name].compositeDamping
```

50.1.1 Members

The CompositeDamping object can have the following member:

components

A CompositeDampingComponentArray object.

50.2 CompositeDampingComponent object

A CompositeDampingComponent object is used to define composite damping over a range of modes.

Access

```
import step
mdb.models[name].steps[name].compositeDamping.components[i]
```

50.2.1 Members

The CompositeDampingComponent object has the following members:

start

An Int specifying the mode number of the lowest mode of a range.

end

An Int specifying the mode number of the highest mode of a range.

50.3 Control object

The Control object is used to provide additional optional general solution controls.

Access

```
import step
mdb.models[name].steps[name].control
```

50.3.1 setValues(...)

This method modifies the Control object.

Arguments

Required arguments

None.

Optional arguments

allowPropagation

A Boolean specifying whether to allow all control values to propagate from a previous step. Setting this argument to ON automatically sets *resetDefaultValues* to OFF. The default value is ON.

resetDefaultValues

A Boolean specifying whether to use all default control values. Setting this argument to ON automatically sets *allowPropagation* to OFF. The default value is OFF.

discontinuous

A Boolean specifying whether to set *timeIncrementation* values that will usually improve efficiency for analyses with severely discontinuous behavior. The default value is OFF.

constraints

The SymbolicConstant DEFAULT or a sequence of Floats specifying tolerances on constraint equations. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

lineSearch

The SymbolicConstant DEFAULT or a sequence of Floats specifying line search control parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

timeIncrementation

The SymbolicConstant DEFAULT or a sequence of Floats specifying time incrementation control parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its

system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

directCyclic

The SymbolicConstant DEFAULT or a sequence of Floats specifying direct cyclic control parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

concentrationField

The SymbolicConstant DEFAULT or a sequence of Floats specifying mass concentration field equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

displacementField

The SymbolicConstant DEFAULT or a sequence of Floats specifying displacement field and warping degree of freedom field equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

electricalPotentialField

The SymbolicConstant DEFAULT or a sequence of Floats specifying electrical potential field equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

globalField

The SymbolicConstant DEFAULT or a sequence of Floats specifying parameters for all applicable field equilibrium equations. This argument overwrites all other field equilibrium equation control values. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

hydrostaticFluidPressureField

The SymbolicConstant DEFAULT or a sequence of Floats specifying hydrostatic fluid element volume constraint parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

poreFluidPressureField

The SymbolicConstant DEFAULT or a sequence of Floats specifying pore liquid volumetric continuity equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

STEP COMMANDS (MISCELLANEOUS)

rotationField

The SymbolicConstant DEFAULT or a sequence of Floats specifying rotation field equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

temperatureField

The SymbolicConstant DEFAULT or a sequence of Floats specifying temperature field equilibrium equation parameters. If a specified sequence contains a value of 0, that item in the sequence will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

vcctLinearScaling

The SymbolicConstant DEFAULT or a Float specifying linear scaling parameter for a VCCT debonding analysis. If a specified value is 0, it will be set to its system-defined value. The value can also be the SymbolicConstant DEFAULT. The default value is DEFAULT.

Return value

None

Exceptions

RangeError.

50.3.2 Members

The Control object has members with the same names and descriptions as the arguments to the `setValues` method.

50.4 DirectDamping object

A DirectDamping object contains direct modal damping parameters.

Access

```
import step
mdb.models[name].steps[name].directDamping
```

50.4.1 Members

The DirectDamping object can have the following member:

components

A DirectDampingComponentArray object.

50.5 DirectDampingByFrequency object

A DirectDampingByFrequency object contains direct damping parameters.

Access

```
import step
mdb.models[name].steps[name].directDampingByFrequency
```

50.5.1 Members

The DirectDampingByFrequency object can have the following member:

components

A DirectDampingByFrequencyComponentArray object.

50.6 DirectDampingByFrequencyComponent object

A DirectDampingByFrequencyComponent object is used to define direct damping over a range of frequencies.

Access

```
import step
mdb.models[name].steps[name].directDampingByFrequency.components[i]
```

50.6.1 Members

The DirectDampingByFrequencyComponent object has the following members:

frequency

A Float specifying the frequency value in cycles/time.

fraction

A Float specifying the fraction of critical damping.

50.7 DirectDampingComponent object

A DirectDampingComponent object is used to define direct damping over a range of modes.

Access

```
import step
mdb.models[name].steps[name].directDamping.components[i]
```

50.7.1 Members

The DirectDampingComponent object has the following members:

start

An Int specifying the mode number of the lowest mode of a range.

end

An Int specifying the mode number of the highest mode of a range.

fraction

A Float specifying the fraction of critical damping.

50.8 EmagTimeHarmonicFrequency object

A EmagTimeHarmonicFrequency is an object used to define frequency over range of modes.

Access

```
import step
mdb.models[name].steps[name].frequencyRange[i]
```

50.8.1 Members

The EmagTimeHarmonicFrequency object has the following members:

lower

A Float specifying the lower limit of frequency range or a single frequency, in cycles/time.

upper

A Float specifying the upper limit of frequency range, in cycles/time.

nPoints

An Int specifying the number of points in the frequency range at which results should be given.

50.9 MassScaling object

A MassScaling object defines the region and controls that govern mass scaling.

Access

```
import step
mdb.models[name].steps[name].massScaling[i]
```

50.9.1 Members

The MassScaling object can have the following members:

objective

A SymbolicConstant specifying the objective of the mass scaling definition. Possible values are SEMI_AUTOMATIC, AUTOMATIC, and REINITIALIZE. The default value is SEMI_AUTOMATIC.

occurs

A SymbolicConstant specifying whether mass scaling should be performed at the beginning of the step or throughout the step. Possible values are AT_BEGINNING and THROUGHOUT_STEP.

type

A SymbolicConstant specifying the type of scaling. Possible values are UNIFORM, BELOW_MIN, SET_EQUAL_DT, and ROLLING. The default value is BELOW_MIN.

factor

A Float specifying a scaling factor.

dt

A Float specifying a target time increment.

frequency

An Int specifying the frequency at which mass scaling calculations are performed.

numberInterval

An Int specifying the number of intervals at which mass scaling calculations are performed.

feedRate

A Float specifying the estimated average velocity of the workpiece in the rolling direction at steady-state conditions.

extrudedLength

A Float specifying the average element length in the extruded direction.

crossSection

An Int specifying the number of nodes in the cross-section of the workpiece.

direction

A SymbolicConstant specifying the rolling direction. Possible values are GLOBAL_X, GLOBAL_Y, GLOBAL_Z, and GLOBAL_NONE. The default value is GLOBAL_X.

region

The SymbolicConstant MODEL or a Region object specifying where the mass scaling is applied. The default value is MODEL.

50.10 RandomResponseFrequency object

A RandomResponseFrequency is an object used to define frequency over a range of modes.

Access

```
import step
mdb.models[name].steps[name].freq[i]
```

50.10.1 Members

The RandomResponseFrequency object has the following members:

lower

A Float specifying the lower limit of the frequency range in cycles per time.

upper

A Float specifying the upper limit of the frequency range in cycles per time.

nCalcs

An Int specifying the number of points between eigenfrequencies at which the response should be calculated.

bias

A Float specifying the bias parameter.

50.11 RayleighDamping object

A RayleighDamping object contains Rayleigh Damping parameters.

Access

```
import step
mdb.models[name].steps[name].rayleighDamping
```

50.11.1 Members

The RayleighDamping object can have the following member:

components

A RayleighDampingComponentArray object.

50.12 RayleighDampingByFrequency object

A RayleighDampingByFrequency object contains Rayleigh Damping parameters.

Access

```
import step
mdb.models[name].steps[name].rayleighDampingByFrequency
```

50.12.1 Members

The RayleighDampingByFrequency object can have the following member:

components

A RayleighDampingByFrequencyComponentArray object.

50.13 RayleighDampingByFrequencyComponent object

A RayleighDampingByFrequencyComponent object is used to define Rayleigh damping over a range of frequencies.

Access

```
import step
mdb.models[name].steps[name].rayleighDampingByFrequency.components[i]
```

50.13.1 Members

The RayleighDampingByFrequencyComponent object has the following members:

frequency

A Float specifying the frequency value in cycles/time.

alpha

A Float specifying the mass proportional damping, α_M .

beta

A Float specifying the stiffness proportional damping, β_M .

50.14 RayleighDampingComponent object

A RayleighDampingComponent object is used to define Rayleigh damping over a range of modes.

Access

```
import step
mdb.models[name].steps[name].rayleighDamping.components[i]
```

50.14.1 Members

The RayleighDampingComponent object has the following members:

start

An Int specifying the mode number of the lowest mode of a range.

end

An Int specifying the mode number of the highest mode of a range.

alpha

A Float specifying the mass proportional damping, α_M .

beta

A Float specifying the stiffness proportional damping, β_M .

50.15 ResponseSpectrumComponent object

A ResponseSpectrumComponent is an element of the ResponseSpectrumComponentArray.

Access

```
import step
mdb.models[name].steps[name].components[i]
```

50.15.1 Members

The ResponseSpectrumComponent object has the following members:

x

A Float specifying the X-direction cosine.

y

A Float specifying the Y-direction cosine.

z

A Float specifying the Z-direction cosine.

scale

A Float specifying the scale factor.

timeDuration

A Float specifying the time duration of the dynamic event, from which this spectrum was created.

Note: This parameter is ignored unless used with the DSC modal summation rule.

respSpectrum

A String specifying the name of the response spectrum specified with the keyword *SPECTRUM.

50.16 SolverControl object

The SolverControl object is used to provide additional optional solver controls.

Access

```
import step
mdb.models[name].steps[name].solverControl
```

50.16.1 setValues(...)

This method modifies the SolverControl object.

Arguments

Required arguments

None.

Optional arguments

allowPropagation

A Boolean specifying whether to allow all solver control values to propagate from a previous step. Setting this argument to ON automatically sets *resetDefaultValues* to OFF. The default value is ON.

resetDefaultValues

A Boolean specifying whether to use all default solver control values. Setting this argument to ON automatically sets *allowPropagation* to OFF. The default value is OFF.

relativeTolerance

The SymbolicConstant DEFAULT or a Float specifying the relative tolerance for convergence of the domain decomposition iterative solver. The default value is DEFAULT.

maxIterations

The SymbolicConstant DEFAULT or an Int specifying the maximum number of linear solver iterations. The default value is DEFAULT.

fillInLevel

The SymbolicConstant DEFAULT or an Int specifying the incomplete LU factorization fill-in level (for geostatic and soil analysis only). The default value is DEFAULT.

Return value

None

Exceptions

RangeError.

50.16.2 Members

The SolverControl object has members with the same names and descriptions as the arguments to the `setValues` method.

50.17 SteadyStateDirectFrequency object

A SteadyStateDirectFrequency is an object used to define frequency over range of modes.

Access

```
import step
mdb.models[name].steps[name].frequencyRange[i]
```

50.17.1 Members

The SteadyStateDirectFrequency object has the following members:

lower

A Float specifying the lower limit of frequency range or a single frequency, in cycles/time.

upper

A Float specifying the upper limit of frequency range, in cycles/time.

nPoints

An Int specifying the number of points in the frequency range at which results should be given.

bias

A Float specifying the Bias parameter. When results are requested at four or more frequency points, Abaqus biases the results toward the ends of the intervals to obtain better resolution. The default value is 3.0.

50.18 SteadyStateModalFrequency object

A SteadyStateModalFrequency is an object used to define frequency over a range of modes.

Access

```
import step
mdb.models[name].steps[name].frequencyRange[i]
```

50.18.1 Members

The SteadyStateModalFrequency object has the following members:

lower

A Float specifying the lower limit of frequency range or a single frequency, in cycles/time.

upper

A Float specifying the upper limit of frequency range, in cycles/time.

nPoints

An Int specifying the number of points in the frequency range at which results should be given.

bias

A Float specifying the bias parameter. When results are requested at four or more frequency points, Abaqus biases the results toward the ends of the intervals to obtain better resolution. The default value is 3.0.

50.19 SteadyStateSubspaceFrequency object

A SteadyStateSubspaceFrequency is an object used to define frequency over range of modes.

Access

```
import step
mdb.models[name].steps[name].frequencyRange[i]
```

50.19.1 Members

The SteadyStateSubspaceFrequency object has the following members:

lower

A Float specifying the lower limit of frequency range or a single frequency, in cycles/time.

upper

A Float specifying the upper limit of frequency range, in cycles/time.

nPoints

An Int specifying the number of points in the frequency range at which results should be given.

bias

A Float specifying the Bias parameter. When results are requested at four or more frequency points, Abaqus biases the results toward the ends of the intervals to obtain better resolution. The default value is 3.0.

50.20 StructuralDamping object

A StructuralDamping object contains structural damping parameters.

Access

```
import step
mdb.models[name].steps[name].structuralDamping
```

50.20.1 Members

The StructuralDamping object can have the following member:

components

A StructuralDampingComponentArray object.

50.21 StructuralDampingByFrequency object

A StructuralDampingByFrequency object contains structural damping parameters.

Access

```
import step
mdb.models[name].steps[name].structuralDampingByFrequency
```

50.21.1 Members

The StructuralDampingByFrequency object can have the following member:

components

A StructuralDampingByFrequencyComponentArray object.

50.22 StructuralDampingByFrequencyComponent object

A StructuralDampingByFrequencyComponent object is used to define structural damping over a range of frequencies.

Access

```
import step
mdb.models[name].steps[name].structuralDampingByFrequency\
.components[i]
```

50.22.1 Members

The StructuralDampingByFrequencyComponent object has the following members:

frequency

A Float specifying the frequency value in cycles/time.

factor

A Float specifying the damping factor, s .

50.23 StructuralDampingComponent object

A StructuralDampingComponent object is used to define structural damping over a range of modes.

Access

```
import step
mdb.models[name].steps[name].structuralDamping.components[i]
```

50.23.1 Members

The StructuralDampingComponent object has the following members:

start

An Int specifying the mode number of the lowest mode of a range.

end

An Int specifying the mode number of the highest mode of a range.

factor

A Float specifying the damping factor, s .

50.24 SubstructureGenerateFrequency object

A SubstructureGenerateFrequency object is used to define the modes to be used in a modal dynamic analysis. These modes are selected from the specified frequency range including the frequency boundary.

Access

```
import step
mdb.models[name].steps[name].frequencyRange[i]
```

50.24.1 Members

The SubstructureGenerateFrequency object has the following members:

lower

A Float specifying the lower limit of the frequency range, in cycles/time.

upper

A Float specifying the upper limit of the frequency range, in cycles/time.

50.25 SubstructureGenerateModes object

A SubstructureGenerateModes object is used to define the modes to be used in a modal dynamic analysis.

Access

```
import step
mdb.models[name].steps[name].modeRange[i]
```

50.25.1 Members

The SubstructureGenerateModes object has the following members:

start

An Int specifying the mode number of the lowest mode of a range.

end

An Int specifying the mode number of the highest mode of a range.

increment

An Int specifying the increment used to define the intermediate mode numbers beginning from the lowest mode to the highest mode.

51. Step commands (output)

Step output commands are used for configuring output requests, integrated output sections, diagnostic printing, monitoring, and restart.

51.1 DiagnosticPrint object

The DiagnosticPrint object is used to request detailed diagnostic output or to disable specific diagnostic checks.

Access

```
import step
mdb.models[name].steps[name].diagnosticPrint
```

51.1.1 DiagnosticPrint(...)

This method creates a DiagnosticPrint object.

Path

```
mdb.models[name].steps[name].DiagnosticPrint
```

Required arguments

None.

Optional arguments

allke

A Boolean specifying a request for a column containing the total kinetic energy. This argument is valid only for an Abaqus/Explicit analysis. The default value is ON.

criticalElement

A Boolean specifying a request for a column containing the element that has the smallest stable time increment and a column listing the value. This argument is valid only for an Abaqus/Explicit analysis. The default value is ON.

dmass

A Boolean specifying a request for a column containing the percent change in total mass of the model as a result of mass scaling. This argument is valid only for an Abaqus/Explicit analysis. The default value is OFF unless mass scaling is present in the model.

STEP COMMANDS (OUTPUT)

etotal

A Boolean specifying a request for a column containing the energy balance of the model. This argument is valid only for an Abaqus/Explicit analysis. The default value is OFF.

contact

A Boolean specifying a request for detailed output of points that are contacting or separating in interface and gap problems. This argument is valid only for an Abaqus/Standard analysis. The default value is ON.

modelChange

A Boolean specifying a request for detailed output of which elements are being removed or reactivated in the step. This argument is valid only for an Abaqus/Standard analysis. The default value is OFF.

plasticity

A Boolean specifying a request for detailed output of element and integration point numbers for which the plasticity algorithms have failed to converge in the material routines. This argument is valid only for an Abaqus/Standard analysis. The default value is OFF.

residual

A Boolean specifying a request for output of equilibrium residuals during the equilibrium iterations. This argument is valid only for an Abaqus/Standard analysis. The default value is ON.

frequency

An Int specifying the frequency of output, in increments. The default value is 1.

solve

A Boolean specifying a request for information regarding the actual number of equations and the wavefront in each iteration. This argument is valid only for an Abaqus/Standard analysis. The default value is ON.

mass

A Boolean specifying a request for a column containing the total mass of the model as a result of mass scaling. This argument is valid only for an Abaqus/Explicit analysis. The default value is OFF.

Return value

A DiagnosticPrint object.

Exceptions

None.

51.1.2 setValues(...)

This method modifies the DiagnosticPrint object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DiagnosticPrint** method.

Return value

None

Exceptions

None.

51.1.3 Members

The DiagnosticPrint object has members with the same names and descriptions as the arguments to the **DiagnosticPrint** method.

51.1.4 Corresponding analysis keywords

*DIAGNOSTICS

51.2 FieldOutputRequest object

The FieldOutputRequest object defines a field output request.

Access

```
import step
mdb.models[name].fieldOutputRequests[name]
```

51.2.1 FieldOutputRequest(...)

This method creates a FieldOutputRequest object.

STEP COMMANDS (OUTPUT)

Path

`mdb.models[name].FieldOutputRequest`

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the object is created.

Optional arguments

region

The SymbolicConstant MODEL or a Region object specifying the region from which output is requested. The SymbolicConstant MODEL represents the whole model. The default value is MODEL.

variables

A sequence of Strings specifying output request variable or component names, or the SymbolicConstant PRESELECT or ALL. PRESELECT represents all default output variables for the given step. ALL represents all valid output variables. The default value is PRESELECT.

frequency

The SymbolicConstant LAST_INCREMENT or an Int specifying the output frequency in increments. The default value is 1.

modes

The SymbolicConstant ALL or a sequence of Ints specifying a list of eigenmodes for which output is desired. The default value is ALL.

timeInterval

The SymbolicConstant EVERY_TIME_INCREMENT or a Float specifying the time interval at which the output states are to be written. The default value is EVERY_TIME_INCREMENT.

numIntervals

An Int specifying the number of intervals during the step at which output database states are to be written. The default value is 20.

timeMarks

A Boolean specifying when to write results to the output database. OFF indicates that output is written immediately after the time dictated by the specified number of intervals. ON indicates that output is written at the exact times dictated by the specified number of intervals. The default value is OFF.

boltLoad

A String specifying a bolt load from which output is requested.

sectionPoints

The SymbolicConstant DEFAULT or a sequence of Ints specifying the section points for which output requested. The default is DEFAULT.

interactions

None or a sequence of Strings specifying the interaction names. The default value is **None**.

The sequence can contain only one String.

rebar

A SymbolicConstant specifying whether output is requested for rebar. Possible values are EXCLUDE, INCLUDE, and ONLY. The default value is EXCLUDE.

filter

The SymbolicConstant ANTIALIASING or a String specifying the name of an output filter object. The default value is **None**.

directions

A Boolean specifying whether to output directions of the local material coordinate system. The default value is ON.

fasteners

A String specifying the fastener name. The default value is an empty string.

assembledFastener

A String specifying the assembled fastener name. The default value is an empty string.

assembledFastenerSet

A String specifying the set name from the model referenced by the assembled fastener, *assembledFastener*. The default value is an empty string.

exteriorOnly

A Boolean specifying whether the output domain is restricted to the exterior of the model. This argument is only valid if *region*=MODEL. The default value is OFF.

layupNames

A List of Composite Layer Names.

layupLocationMethod

A Symbolic constant specifying the method used to indicate the output locations for composite layups. Possible values are ALL_LOCATIONS, SPECIFIED and TYPED_IN. The default value is SPECIFIED.

outputAtPlyTop

A Boolean specifying whether to output at the ply top section point. The default value is False.

outputAtPlyMid

A Boolean specifying whether to output at the ply mid section point. The default value is True.

outputAtPlyBottom

A Boolean specifying whether to output at the ply bottom section point. The default value is False.

STEP COMMANDS (OUTPUT)

Return value

A FieldOutputRequest object.

Exceptions

None.

51.2.2 deactivate(...)

This method deactivates the field output request in the specified step and all its subsequent steps.

Arguments

Required argument

stepName

A String specifying the name of the step in which the field output request is deactivated.

Optional arguments

None.

Return value

None

Exceptions

TextError.

51.2.3 move(...)

This method moves the field output request state object from one step to a different step.

Arguments

Required arguments

fromStepName

A String specifying the name of the step from which the field output request state is moved.

toStepName

A String specifying the name of the step to which the field output request state is moved.

Optional arguments

None.

Return value

None

Exceptions

TextError.

51.2.4 reset(...)

This method resets the field output request state of the specified step to the state of the previous step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the field output request state is reset.

Optional arguments

None.

Return value

None

Exceptions

TextError.

51.2.5 resume()

This method resumes the field output request that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

TextError.

51.2.6 **suppress()**

This method suppresses the field output request.

Arguments

None.

Return value

None

Exceptions

TextError.

51.2.7 **setValues(...)**

This method modifies the data for an existing FieldOutputRequest object in the step where it is created.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **FieldOutputRequest** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

51.2.8 **setValuesInStep(...)**

This method modifies the propagating data for an existing FieldOutputRequest object in the specified step.

Arguments**Required argument***stepName*

A String specifying the name of the step in which the field output request is modified.

Optional arguments*variables*

A sequence of Strings specifying output request variable or component names, or the SymbolicConstant PRESELECT or ALL. PRESELECT represents all default output variables for the given step. ALL represents all valid output variables.

frequency

The SymbolicConstant LAST_INCREMENT or an Int specifying the output frequency in increments. The default value is 1.

modes

The SymbolicConstant ALL or a sequence of Ints specifying a list of eigenmodes for which output is desired. The default value is ALL.

timeInterval

The SymbolicConstant EVERY_TIME_INCREMENT or a Float specifying the time interval at which the output states are to be written. The default value is EVERY_TIME_INCREMENT.

numIntervals

An Int equal to the number of intervals during the step at which output database states are to be written. The default value is 20.

timePoints

A String specifying the name of a time point object. The default value is equal to the number of intervals during the step at which output database states are to be written. The default value is **None**.

timeMarks

A Boolean specifying when to write results to the output database. The default value is OFF.

Return value**None****Exceptions**

None.

51.2.9 Members

The FieldOutputRequest object can have the following members:

STEP COMMANDS (OUTPUT)

boltLoad

A String specifying a bolt load from which output is requested.

region

The SymbolicConstant MODEL or a Region object specifying the region from which output is requested. The SymbolicConstant MODEL represents the whole model. The default value is MODEL.

interactions

None or a tuple of Strings specifying the interaction names. The default value is **None**.
The sequence can contain only one String.

51.2.10 Corresponding analysis keywords

- *CONTACT OUTPUT
- *ELEMENT OUTPUT
- *ENERGY OUTPUT
- *INCREMENTATION OUTPUT
- *MODAL OUTPUT
- *NODE OUTPUT
- *OUTPUT
- *RADIATION OUTPUT

51.3 FieldOutputRequestState object

The FieldOutputRequestState object stores the propagating data of a field output request current in a step. One instance of this object is created internally by the FieldOutputRequest object for each step. The instance is also deleted internally by the FieldOutputRequest object.

The FieldOutputRequestState object has no constructor or methods.

Access

```
import step
mdb.models[name].steps[name].fieldOutputRequestState[name]
```

51.3.1 Members

The FieldOutputRequestState object can have the following members:

variablesState

A SymbolicConstant specifying the propagation state of the field output request variables. Possible values are UNSET, SET, and UNCHANGED.

frequency

The SymbolicConstant LAST_INCREMENT or an Int specifying the output frequency in increments. The default value is 1.

frequencyState

A SymbolicConstant specifying the propagation state of the field output request frequency. Possible values are UNSET, SET, and UNCHANGED.

modesState

A SymbolicConstant specifying the propagation state of the field output request modes. Possible values are UNSET, SET, and UNCHANGED.

timeInterval

The SymbolicConstant EVERY_TIME_INCREMENT or a Float specifying the time interval at which the output states are to be written. The default value is EVERY_TIME_INCREMENT.

timeIntervalState

A SymbolicConstant specifying the propagation state of the field output request time interval. Possible values are UNSET, SET, and UNCHANGED.

numIntervals

An Int specifying the number of intervals during the step at which output database states are to be written. The default value is 20.

numIntervalsState

A SymbolicConstant specifying the propagation state of the field output request. Possible values are UNSET, SET, and UNCHANGED.

timeMarks

A Boolean specifying when to write results to the output database. The default value is OFF.

timeMarksState

A SymbolicConstant specifying the propagation state of the field output request. Possible values are UNSET, SET, and UNCHANGED.

timePointState

A SymbolicConstant specifying the propagation state of the field output request. Possible values are UNSET, SET, and UNCHANGED.

status

A SymbolicConstant specifying the propagation state of the FieldOutputRequestState object. Possible values are NOT_YET_ACTIVE, CREATED, PROPAGATED, MODIFIED, DEACTIVATED, NO_LONGER_ACTIVE, TYPE_NOT_APPLICABLE, and INSTANCE_NOT_APPLICABLE.

STEP COMMANDS (OUTPUT)

variables

A tuple of Strings specifying output request variable or component names, or the SymbolicConstant PRESELECT or ALL. PRESELECT represents all default output variables for the given step. ALL represents all valid output variables.

modes

The SymbolicConstant ALL or a tuple of Ints specifying a list of eigenmodes for which output is desired. The default value is ALL.

timePoint

A String specifying the name of a time point object used to determine which output database states are to be written. The default value is an empty string.

frequencyType

A String specifying a read-only SymbolicConstant describing which type of frequency of output is used. Possible values are FREQUENCY, NUMBER_INTERVALS, TIME_INTERVAL, TIME_POINT and MODES. The default value depends on the procedure. The default value is an empty string.

51.4 HistoryOutputRequest object

The HistoryOutputRequest object defines a history output request.

Access

```
import step
mdb.models[name].historyOutputRequests[name]
```

51.4.1 HistoryOutputRequest(...)

This method creates a HistoryOutputRequest object.

Path

```
mdb.models[name].HistoryOutputRequest
```

Required arguments

name

A String specifying the repository key.

createStepName

A String specifying the name of the step in which the object is created.

Optional arguments*region*

The SymbolicConstant MODEL or a Region object specifying the region from which output is requested. The SymbolicConstant MODEL represents the whole model. The default value is MODEL.

If the region is a surface region, the surface must lie within the general contact surface domain.

variables

A sequence of Strings specifying output request variable or component names, or the SymbolicConstant PRESELECT or ALL. PRESELECT represents all default output variables for the given step. ALL represents all valid output variables. The default value is PRESELECT.

frequency

The SymbolicConstant LAST_INCREMENT or an Int specifying the output frequency in increments. The default value is 1.

modes

The SymbolicConstant ALL or a sequence of Ints specifying a list of eigenmodes for which output is desired. The default value is ALL.

timeInterval

The SymbolicConstant EVERY_TIME_INCREMENT or a Float specifying the time interval at which the output states are to be written. The default value is EVERY_TIME_INCREMENT.

numIntervals

An Int specifying the number of intervals during the step at which output database states are to be written. The default value is 20.

boltLoad

A String specifying a bolt load from which output is requested. The default value is an empty string.

sectionPoints

The SymbolicConstant DEFAULT or a sequence of Ints specifying the section points for which output is requested. The default value is DEFAULT.

stepName

A String specifying the name of the step. The default value is an empty string.

interactions

None or a sequence of Strings specifying the interaction names. The default value is **None**.
The sequence can contain only one String.

contourIntegral

A String specifying the contour integral name. The default value is **None**.

numberOfContours

An Int specifying the number of contour integrals to output for the contour integral object. The default value is 0.

STEP COMMANDS (OUTPUT)

stressInitializationStep

A String specifying the name of the stress initialization step. The default value is **None**.

contourType

A SymbolicConstant specifying the type of contour integral. Possible values are J_INTEGRAL, C_INTEGRAL, T_STRESS, and K_FACTORS. The default value is J_INTEGRAL.

kFactorDirection

A SymbolicConstant specifying the stress intensity factor direction. Possible values are MTS, MERR, and K110. The *kFactorDirection* argument is valid only if *contourType*=K_FACTORS. The default value is MTS.

rebar

A SymbolicConstant specifying whether output is requested for rebar. Possible values are EXCLUDE, INCLUDE, and ONLY. The default value is EXCLUDE.

integratedOutputSection

A String specifying the integrated output section. The default value is an empty string.

springs

A sequence of Strings specifying the springs/dashpots names. The default value is **None**. The sequence can contain only one String.

filter

The SymbolicConstant ANTIALIASING or a String specifying the name of an output filter object. The default value is **None**.

fasteners

A String specifying the fastener name. The default value is an empty string.

assembledFastener

A String specifying the assembled fastener name. The default value is an empty string.

assembledFastenerSet

A String specifying the set name from the model referenced by the assembled fastener, *assembledFastener*. The default value is an empty string.

sensor

A Boolean specifying whether to associate the output request with a sensor definition. The default value is OFF.

useGlobal

A Boolean specifying whether to output vector-valued nodal variables in the global directions. The default value is True.

Return value

A HistoryOutputRequest object.

Exceptions

None.

51.4.2 deactivate(...)

This method deactivates the history output request in the specified step and all subsequent steps.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the history output request is deactivated.

Optional arguments

None.

Return value

None

Exceptions

TextError.

51.4.3 move(...)

This method moves the history output request state object from one step to a different step.

Arguments**Required arguments**

fromStepName

A String specifying the name of the step from which the history output request state is moved.

toStepName

A String specifying the name of the step to which the history output request state is moved.

Optional arguments

None.

Return value

None

STEP COMMANDS (OUTPUT)

Exceptions

TextError.

51.4.4 **reset(...)**

This method resets the history output request state of the specified step to the state of the previous step.

Arguments

Required argument

stepName

A String specifying the name of the step in which the history output request state is reset.

Optional arguments

None.

Return value

None

Exceptions

TextError.

51.4.5 **resume()**

This method resumes the history output request that was previously suppressed.

Arguments

None.

Return value

None

Exceptions

TextError.

51.4.6 **suppress()**

This method suppresses the history output request.

Arguments

None.

Return value

None

Exceptions

TextError.

51.4.7 **setValues(...)**

This method modifies the data for an existing HistoryOutputRequest object in the step where it is created.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **HistoryOutputRequest** method, except for the *name* and *createStepName* arguments.

Return value

None

Exceptions

None.

51.4.8 **setValuesInStep(...)**

This method modifies the propagating data for an existing HistoryOutputRequest object in the specified step.

Arguments**Required argument**

stepName

A String specifying the name of the step in which the history output request is modified.

STEP COMMANDS (OUTPUT)

Optional arguments

variables

A sequence of Strings specifying output request variable or component names or the SymbolicConstant PRESELECT or ALL. PRESELECT represents all default output variables for the given step. ALL represents all valid output variables.

frequency

The SymbolicConstant LAST_INCREMENT or an Int specifying the output frequency in increments. The default value is 1.

modes

The SymbolicConstant ALL or a sequence of Ints specifying a list of eigenmodes for which output is desired. The default value is ALL.

timeInterval

The SymbolicConstant EVERY_TIME_INCREMENT or a Float specifying the time interval at which the output states are to be written. The default value is EVERY_TIME_INCREMENT.

numIntervals

An Int specifying the number of intervals during the step at which output database states are to be written. The default value is 20.

timePoints

A String specifying the name of a time point object. The default value is equal to the number of intervals during the step at which output database states are to be written. The default value is **None**.

Return value

None

Exceptions

None.

51.4.9 Members

The HistoryOutputRequest object can have the following members:

boltLoad

A String specifying a bolt load from which output is requested. The default value is an empty string.

region

The SymbolicConstant MODEL or a Region object specifying the region from which output is requested. The SymbolicConstant MODEL represents the whole model. The default value is MODEL.

If the region is a surface region, the surface must lie within the general contact surface domain.

sectionPoints

The SymbolicConstant DEFAULT or a tuple of Ints specifying the section points for which output is requested. The default value is DEFAULT.

interactions

None or a tuple of Strings specifying the interaction names. The default value is **None**.
The sequence can contain only one String.

51.4.10 Corresponding analysis keywords

- *CONTACT OUTPUT
- *ELEMENT OUTPUT
- *ENERGY OUTPUT
- *INCREMENTATION OUTPUT
- *MODAL OUTPUT
- *NODE OUTPUT
- *OUTPUT
- *RADIATION OUTPUT

51.5 HistoryOutputRequestState object

The HistoryOutputRequestState object stores the propagating data of a History output request current in a step. One instance of this object is created internally by the HistoryOutputRequest object for each step. The instance is also deleted internally by the HistoryOutputRequest object.

The HistoryOutputRequestState object has no constructor or methods.

Access

```
import step
mdb.models[name].steps[name].historyOutputRequestState[name]
```

51.5.1 Members

The HistoryOutputRequestState object can have the following members:

variablesState

A SymbolicConstant specifying the propagation state of the history output request variables. Possible values are UNSET, SET, and UNCHANGED.

STEP COMMANDS (OUTPUT)

frequency

The SymbolicConstant `LAST_INCREMENT` or an Int specifying the output frequency in increments. The default value is 1.

frequencyState

A SymbolicConstant specifying the propagation state of the history output request frequency. Possible values are `UNSET`, `SET`, and `UNCHANGED`.

modesState

A SymbolicConstant specifying the propagation state of the history output request modes. Possible values are `UNSET`, `SET`, and `UNCHANGED`.

timeInterval

The SymbolicConstant `EVERY_TIME_INCREMENT` or a Float specifying the time interval at which the output states are to be written. The default value is `EVERY_TIME_INCREMENT`.

timeIntervalState

A SymbolicConstant specifying the propagation state of the history output request time interval. Possible values are `UNSET`, `SET`, and `UNCHANGED`.

numIntervals

An Int specifying the number of intervals during the step at which output database states are to be written. The default value is 20.

timePointState

A SymbolicConstant specifying the propagation state of the history output request. Possible values are `UNSET`, `SET`, and `UNCHANGED`.

numIntervalsState

A SymbolicConstant specifying the propagation state of the history output request. Possible values are `UNSET`, `SET`, and `UNCHANGED`.

status

A SymbolicConstant specifying the propagation state of the `HistoryOutputRequestState` object. Possible values are `NOT_YET_ACTIVE`, `CREATED`, `PROPAGATED`, `MODIFIED`, `DEACTIVATED`, `NO_LONGER_ACTIVE`, `TYPE_NOT_APPLICABLE`, and `INSTANCE_NOT_APPLICABLE`.

variables

A tuple of Strings specifying output request variable or component names, or the SymbolicConstant `PRESELECT` or `ALL`. `PRESELECT` represents all default output variables for the given step. `ALL` represents all valid output variables.

modes

The SymbolicConstant `ALL` or a tuple of Ints specifying a list of eigenmodes for which output is desired. The default value is `ALL`.

timePoint

A String specifying the name of a time point object used to determine at which points in the time period data is written to the output database. The default value is an empty string.

frequencyType

A String specifying a read-only SymbolicConstant describing which type of frequency of output is used. Possible values are FREQUENCY, NUMBER_INTERVALS, TIME_INTERVAL, TIME_POINT and MODES. The default value depends on the procedure. The default value is an empty string.

51.6 IntegratedOutputSection object

The IntegratedOutputSection object specifies parameters used for integrated output.

Access

```
import step
```

```
mdb.models[name].integratedOutputSections[name]
```

51.6.1 IntegratedOutputSection(...)

This method creates an IntegratedOutputSection object.

Path

```
mdb.models[name].IntegratedOutputSection
```

Required argument

name

A String specifying the repository key.

Optional arguments

surface

A Region object specifying the surface over which the output is based.

refPoint

None or a Region object specifying the anchor point about which the integrated moment over the output region is computed or the SymbolicConstant None representing the global origin. The default value is **None**.

refPointAtCenter

A Boolean specifying that the *refPoint* be adjusted so that it coincides with the center of the output region in the initial configuration. This argument is valid only when you include the *refPoint* argument. The default value is OFF.

refPointMotion

A SymbolicConstant specifying how to relate the motion of *refPoint* to the average motion of the output region. A value of INDEPENDENT will allow the *refPoint* to move independent

STEP COMMANDS (OUTPUT)

of the output region. A value of `AVERAGE_TRANSLATION` will set the displacement of the *refPoint* equal to the average translation of the output region. A value of `AVERAGE` will set the displacement and rotation of the *refPoint* equal to the average translation of the output region. The default value is `INDEPENDENT`.

This argument is valid only when you include the *refPoint* argument.

localCsys

None or a `DatumCsys` object specifying the local coordinate system used to express vector output. If *localCsys*=**None**, the degrees of freedom are defined in the global coordinate system. The default value is **None**.

projectOrientation

A Boolean specifying that the coordinate system be projected onto the *surface* such that the 1-axis is normal to the *surface*. Projection onto a planar *surface* is such that the 1-axis is normal to the surface, and a projection onto a nonplanar *surface* is such that a least-squares fit surface will be used. The default value is `OFF`.

Return value

An `IntegratedOutputSection` object.

Exceptions

None.

51.6.2 **setValues(...)**

This method modifies the `IntegratedOutputSection` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the `IntegratedOutputSection` method, except for the *name* argument.

Return value

None

Exceptions

None.

51.6.3 Members

The `IntegratedOutputSection` object has members with the same names and descriptions as the arguments to the `IntegratedOutputSection` method.

51.7 Monitor object

The Monitor object defines a degree of freedom to monitor.

Access

```
import step
mdb.models[name].steps[name].monitor
```

51.7.1 Monitor(...)

This method creates a request for a degree of freedom to be monitored in a general or modal procedure.

Path

```
mdb.models[name].steps[name].Monitor
```

Required arguments

node

A String specifying the name of the region to be monitored.

dof

A SymbolicConstant specifying the degree of freedom to be monitored at the node. Possible values are:

- U1
- U2
- U3
- UR1
- UR2
- UR3
- WARP
- FLUID_PRESSURE
- ELECTRICAL_POTENTIAL
- NT11
- NT30

STEP COMMANDS (OUTPUT)

- NN11
- NN30

The NT identifiers are not available for mass diffusion. The NN identifiers are available only for mass diffusion.

frequency

An Int specifying the output frequency in increments. This argument is valid only for an Abaqus/Standard analysis.

Optional arguments

None.

Return value

A Monitor object.

Exceptions

RangeError.

51.7.2 **setValues(...)**

This method modifies the Monitor object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Monitor** method.

Return value

None

Exceptions

RangeError.

51.7.3 **Members**

The Monitor object has members with the same names and descriptions as the arguments to the **Monitor** method.

51.7.4 Corresponding analysis keywords

*MONITOR

51.8 Restart object

The Restart object defines a restart request.

Access

```
import step
mdb.models[name].steps[name].restart
```

51.8.1 Restart(...)

This method creates a restart request.

Path

```
mdb.models[name].steps[name].Restart
```

Required arguments

None.

Optional arguments

numberIntervals

An Int specifying the number of intervals during the step at which restart information will be written. The default value is 0. The default value is 1.

timeMarks

A Boolean specifying whether to use exact time marks for writing during an analysis. The default value is OFF. The default value is OFF.

overlay

A Boolean specifying that only one increment per step should be retained on the restart file, thus minimizing the size of the restart file. The default value is OFF. The default value is ON.

frequency

An Int specifying the increments at which restart information will be written. The default value is 0. The default value is 0.

This argument applies only to Abaqus/Standard analyses.

Return value

A Restart object.

Exceptions

RangeError.

51.8.2 setValues(...)

This method modifies the Restart object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Restart** method.

Return value

None

Exceptions

RangeError.

51.8.3 Members

The Restart object has members with the same names and descriptions as the arguments to the **Restart** method.

51.8.4 Corresponding analysis keywords

*RESTART

51.9 TimePoint object

The TimePoint object defines time points at which data are written to the output database or restart files.

Access

```
import step
mdb.models[name].timePoints[name]
```

51.9.1 TimePoint(...)

This method creates a TimePoint object.

Path

```
mdb.models[name].TimePoint
```

Required arguments

name

A String specifying the repository key.

points

A sequence of sequences of Floats specifying time points at which data are written to the output database or restart files.

Optional arguments

None.

Return value

A TimePoint object.

Exceptions

InvalidNameError and RangeError.

51.9.2 setValues(...)

This method modifies the TimePoint object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TimePoint** method, except for the *name* argument.

STEP COMMANDS (OUTPUT)

Return value

None

Exceptions

RangeError.

51.9.3 Members

The TimePoint object has members with the same names and descriptions as the arguments to the **TimePoint** method.

51.9.4 Corresponding analysis keywords

*TIME POINTS

52. Text Representation commands

52.1 redentABQ module

The Python module redentABQ is a wrapper around the Python redent module. The module makes the indentation in a Python text file consistent.

The module can be run as a script from a command prompt using the following statement:

abaqus Python -m redentABQ [options] pathToFile

where the following options are available to customize the redentABQ process:

- **-b**: Create a backup of the selected file if any modifications are made.
- **-i**: Specify a number of spaces to be used for indentation.
- **-t**: Test the newly indented file after any changes are made.

For more information on running the **abaqus Python** execution procedure, see “Python execution,” Section 3.2.10 of the Abaqus Analysis User’s Guide.

Access

```
import redentABQ
```

52.1.1 indentFile(...)

This method outputs the indented file to the terminal window or backs up the specified file and replaces it.

Arguments

Required argument

path

A String specifying the file to be processed.

Optional arguments

indent

A String specifying the amount of indentation to be used. The default value is four spaces of indentation. Custom indentation strings must be flanked by quotation marks (" ").

backup

A Boolean specifying whether to back up the file specified in the *path* argument if changes are made. The default value is False.

runTest

A Boolean specifying whether to test the newly indented file to ensure that it is semantically the same after any changes. The default value is False.

Return value

A Boolean. True, if the indentation is successful; otherwise, False.

Exceptions

None.

52.2 textRepr module

The Python module `textRepr` contains functions to print an Abaqus object and to convert the output from the `str()` function into a formatted string, indented for each object in the recursive listing of a Python object.

Access

```
import textRepr
```

52.2.1 getIndentedRepr(...)

This method returns a String with each level of parentheses indented.

Arguments

Required argument

object

A Python object to be processed. (The Python object can be an Abaqus object.) This argument can also be a String representation of a Python object obtained from `str(object)`.

Optional arguments

maxRecursionDepth

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. If *object* is not an Abaqus object, *maxRecursionDepth* has no effect. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

significantDigits

An Int specifying the number of significant digits for Floats in the output. Possible values are $0 < \text{significantDigits} \leq 15$. The default value is 6.

Return value

A String.

Exceptions

None.

52.2.2 getPaths(...)

This method processes the argument and interprets its structure. It then returns a String containing the object paths of all the child objects found.

Arguments**Required argument***object*

A Python object to be processed. (The Python object can be an Abaqus object.) This argument can also be a String representation of a Python object.

Optional arguments*maxRecursionDepth*

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. If *object* is not an Abaqus object, *maxRecursionDepth* has no effect. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

pathRoot

A String specifying the root of the paths. This String is prepended to each path found. The default value is **None**, implying that the path to *object* will be used unless *object* is a String.

Return value

A String.

Exceptions

None.

52.2.3 **getTypes(...)**

This method processes the argument, interprets its structure, and returns a String containing all the object types within the object in the form '*TypeName Path*'.

Arguments

Required argument

object

An Abaqus object.

Optional arguments

maxRecursionDepth

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

pathRoot

A String specifying the root of the paths. This String is prepended to each path found. The default value is **None**, implying that the path to *object* will be used unless *object* is a String.

Return value

A String.

Exceptions

None.

52.2.4 prettyPrint(...)

This method prints a formatted version of the object. The **prettyPrint** function uses **getIndentedRepr** to format the String representation, but does not save the full text output of **getIndentedRepr**. Therefore this function can be used in cases where **getIndentedRepr** would run out of memory.

Arguments**Required argument**

object

An Abaqus object.

Optional arguments

maxRecursionDepth

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

significantDigits

An Int specifying the number of significant digits for Floats in the output. Possible values are $0 < \text{significantDigits} \leq 15$. The default value is 6.

Return value

None

Exceptions

None.

52.2.5 **printPaths(...)**

This method prints the object path of the *object* argument and its members, depending on the *maxRecursionDepth* argument. The **printPaths** function uses **getPaths** to print a list of paths to each of the child objects in *object*.

Arguments

Required argument

object

An Abaqus object. This argument can also be a String representation of an Abaqus object obtained from `str(object)`.

Optional arguments

maxRecursionDepth

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

pathRoot

A String specifying the root of the paths to be printed. This String is prepended to each path found. The default value is **None**, implying that the path to *object* will be used.

Return value

None

Exceptions

None.

52.2.6 **printTypes(...)**

This method prints the object type. The **printTypes** function uses **getTypes** to print a list of all the object types in *object*.

Arguments**Required argument***object*

An Abaqus object.

Optional arguments*maxRecursionDepth*

An Int specifying the maximum depth to which the object will be printed, the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. The default value is **None**.

You should take care when setting *maxRecursionDepth*=UNLIMITED because the resulting output can be very large. To limit the output, you should set *maxElementsInSequence* to a small number.

maxElementsInSequence

An Int specifying the maximum number of elements of a sequence to return or the SymbolicConstant UNLIMITED. The initial value is 100. After the maximum number of elements have been printed, the remainder are indicated by the string '...'.

pathRoot

A String specifying the root of the paths to be printed. This String is prepended to each path found. The default value is **None**, implying that the path to *object* will be used.

Return value**None****Exceptions**

None.

52.3 TextReprOptions object

The TextReprOptions object is used to configure the output of the Python **str()** command within Abaqus/CAE. (The **str()** command is used by the Python print function.) The TextReprOptions object stores the various settings that determine how objects are printed on the Python command line.

The TextReprOptions object has no constructor. Abaqus creates the *textReprOptions* member when a session is started.

Access

```
import textRepr
textReprOptions
```

`session.textReprOptions`

52.3.1 setValues(...)

This method modifies the TextReprOptions object.

Arguments

Required arguments

None.

Optional arguments

style

A SymbolicConstant specifying the style of the text representation. Possible values are:

- RECURSIVE: return a recursive representation of the object.
- SIMPLE: return a String representing the object as either 'interface object', 'dictionary object', or 'sequence object'.

The initial value is RECURSIVE.

maxRecursionDepth

An Int, SymbolicConstant, or **None** specifying the maximum depth to which the object will be printed. Possible values for the depth are Ints ≥ 0 , the SymbolicConstant UNLIMITED, or **None**. A value of **None** implies that the current setting in the TextReprOptions object will be used. If *object* is not an Abaqus object, *maxRecursionDepth* has no effect. The default value is **None**.

maxRecursionString

A String specifying the string to be returned when the maximum depth of recursion is reached. The string can include a format specifier (%s), which will be substituted by the object type. The initial value is '%s object'.

maxElementsInSequence

An Int or the SymbolicConstant UNLIMITED specifying the maximum number of elements of a sequence to return. Possible values are UNLIMITED or Ints > 0 . The initial value is 100. After the maximum number of elements, the remainder are indicated by the string '...'.

Return value

None

Exceptions

None.

52.3.2 Members

The `TextReprOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

53. Utility commands

Utility commands are used to do the following:

- Create SymbolicConstant and Boolean objects.
- Change the name of a key to a Repository object.
- Control the use of deprecated commands.
- Request input from the user.
- Display status information.
- Execute Python commands or functions.
- Register a callback that will be invoked when certain Abaqus Scripting Interface commands are about to be executed.
- Register a callback that will be invoked when certain Abaqus Scripting Interface objects are about to be deleted.
- Upgrade Abaqus kernel or GUI scripts from one release of Abaqus to a later release.

53.1 SymbolicConstant object

The SymbolicConstant object represents a string in a way that can be stored in a replay file and used as an argument to Abaqus Scripting Interface methods and functions. By convention the string representation of the SymbolicConstant object is the same as its variable name. If you pass a SymbolicConstant object to the Python `repr()` function, the function returns the text without quotes. In effect, the text is the variable that, by convention, refers to the SymbolicConstant object.

Two SymbolicConstant objects with the same text are the same Python object, although you can assign them to different variables. All of the SymbolicConstant objects that are required in Abaqus Scripting Interface methods are defined in the `abaqusConstants` module. Some SymbolicConstant objects and the SymbolicConstant constructor are defined in the `abaqus` module. The SymbolicConstant constructor is also defined in the `symbolicConstants` module.

Access

```
from symbolicConstants import *
from abaqusConstants import *
```

53.1.1 SymbolicConstant(...)

The `SymbolicConstant` method creates a SymbolicConstant object.

Path**SymbolicConstant****Required argument***text*

A String specifying the text of the SymbolicConstant object. The String must contain only capital letters, digits, or underscores and must not start with a digit.

Optional arguments

None.

Return value

A SymbolicConstant object.

Exceptions

None.

53.2 AbaqusBoolean object

The AbaqusBoolean object is used in a similar way to the SymbolicConstant object. If you pass an AbaqusBoolean object to the Python **repr()** function, the function returns the text without quotes. In effect, the text is the variable that, by convention, refers to the AbaqusBoolean object.

An AbaqusBoolean object has a value of 0 or 1, and an AbaqusBoolean object can be tested in an **if** statement. You can use an AbaqusBoolean object as an argument to a method in place of 1 or 0. Conversely, you can pass a 0 or a 1 to an Abaqus Scripting Interface method that expects a Boolean argument, and the 0 or 1 will be coerced to the appropriate AbaqusBoolean value.

There are only two possible values for an AbaqusBoolean object: 1 and 0. You can import both values from the symbolicConstants module or from the abaqus module. Abaqus Scripting Interface commands that expect an AbaqusBoolean object will also accept a Python bool (True, False), or a Python int (1, 0).

Access

```
from symbolicConstants import *
from abaqusConstants import *
```

53.2.1 AbaqusBoolean(...)

The **AbaqusBoolean** method creates an AbaqusBoolean object.

Path**AbaqusBoolean****Required argument***value*

An Int specifying whether the **AbaqusBoolean** object will test true or false. Possible values are 0 and 1, which will create the **AbaqusBoolean** method OFF and ON, respectively.

Optional arguments

None.

Return valueAn **AbaqusBoolean** object.**Exceptions**

None.

53.3 Repository object

Repositories are containers that store a particular type of object; for example, the steps repository contains all the steps defined in the model. An Abaqus Scripting Interface Repository maps a key to a value. The key is usually a String, and the value is any Python object, usually an Abaqus object. A repository is similar to a Python dictionary; however, only a constructor can add an object to a repository. In addition, all of the objects in a repository are of the same base type. For more information, see “Repositories,” Section 5.3.3 of the Abaqus Scripting User’s Guide. A Repository has no constructor. Abaqus creates empty repositories when you import a module. For example, Abaqus creates an empty **parts** repository when you import the **part** module.

The following methods of the Repository object are standard Python dictionary methods and are not described here:

- **has_key**
- **items**
- **keys**
- **values**

53.3.1 changeKey(...)

This method changes the name of a key in a repository and the *name* member of the value object.

Arguments

Required arguments

fromName

A String specifying the old name of the repository key.

toName

A String specifying the new name of the repository key.

Optional arguments

None.

Return value

None

Exceptions

None.

53.4 BackwardCompatibility object

The BackwardCompatibility object enables the user to control access to deprecated commands in the Abaqus Scripting Interface and to collect data on which deprecated commands have been used. This enables the user to check that no deprecated methods, members, or arguments have been used. Data are recorded on any deprecated members, methods, or arguments that are used.

The BackwardCompatibility object has no constructor. The `abaqus` module and the `odbAccess` module each have a member named `backwardCompatibility`.

By default, the value of the *includeDeprecated* member is ON, and Abaqus will execute a script containing deprecated commands with no indication that you should update the script. You can do either of the following to change the value of the *includeDeprecated* member and to determine which commands have been deprecated:

- From the command line interface in Abaqus/CAE or from an Abaqus Scripting Interface script that is run from within Abaqus/CAE, use the following command:

```
backwardCompatibility.setValues(includeDeprecated=OFF)
```

- From an Abaqus Scripting Interface script that is run using **abaqus python** at the system prompt, use the following two commands:

```
from odbAccess import *  
backwardCompatibility.setValues(includeDeprecated=OFF)
```

In addition, the `BackwardCompatibility` object provides tools to assist you in determining the deprecated commands that have been used. For example, to determine the deprecated commands used in the script `createLug.py`, use the following commands:

```
backwardCompatibility.resetDeprecatedMethodsUsed()
backwardCompatibility.resetDeprecatedMembersUsed()
backwardCompatibility.resetDeprecatedArgsUsed()
execfile('createLug.py')
print backwardCompatibility.getDeprecatedMethodsUsed()
print backwardCompatibility.getDeprecatedMembersUsed()
print backwardCompatibility.getDeprecatedArgsUsed()
```

Access

`backwardCompatibility`

53.4.1 `getDeprecatedMethodsUsed()`

This method returns a list of deprecated methods used since the last call to `resetDeprecatedMethodsUsed`.

Arguments

None.

Return value

A list of Strings.

Exceptions

None.

53.4.2 `getDeprecatedMembersUsed()`

This method returns a list of deprecated members used since the last call to `resetDeprecatedMembersUsed`.

Arguments

None.

Return value

A list of Strings.

Exceptions

None.

53.4.3 `getDeprecatedArgsUsed()`

This method returns a list of deprecated arguments used since the last call to `resetDeprecatedArgsUsed`.

Arguments

None.

Return value

A list of Strings.

Exceptions

None.

53.4.4 `resetDeprecatedMethodsUsed()`

This method clears the list of deprecated methods used.

Arguments

None.

Return value

None

Exceptions

None.

53.4.5 `resetDeprecatedMembersUsed()`

This method clears the list of deprecated members used.

Arguments

None.

Return value

None

Exceptions

None.

53.4.6 resetDeprecatedArgsUsed()

This method clears the list of deprecated arguments used.

Arguments

None.

Return value

None

Exceptions

None.

53.4.7 setValues(...)

This method modifies the BackwardCompatibility object.

Arguments**Required arguments**

None.

Optional argument*includeDeprecated*

A Boolean specifying whether deprecated members, methods, and arguments can be seen and used. The default value is ON.

reportDeprecated

A Boolean specifying whether a warning is displayed after running a script that contains deprecated commands. The default value is True.

showKeysInReport

A Boolean specifying whether the keys and indices are included in the report that is displayed when *reportDeprecated* is True. The default value is False.

Return value

None

Exceptions

None.

53.4.8 Members

The BackwardCompatibility object has the following members:

includeDeprecated

A Boolean specifying whether deprecated members, methods, and arguments can be seen and used. Possible values are ON and OFF. The default value is ON.

reportDeprecated

A Boolean specifying whether a warning is displayed after running a script that contains deprecated commands. The default value is True.

showKeysInReport

A Boolean specifying whether the keys and indices are included in the report that is displayed when *reportDeprecated* is True. The default value is False.

53.5 User input commands

The following commands are used to request data entry from a user.

53.5.1 **getInput(...)**

This method is used to obtain a single input from a user from a dialog box. The method displays a modal dialog box with a text field prefaced by the specified label. The user enters data in the text field or accepts the optional default value that is displayed in the text field. When the user clicks the **OK** button, the **getInput** method reads the data from the dialog box and closes the dialog box. You can use a “\t” separator in the label argument to provide a tooltip from the dialog box; for example,

```
getInput('Enter a number:\tEnter the number of nodes to delete')
```

Note: You cannot use a script that contains **getInput** if you are running the script from the command line and passing the script name to the command line options **-start**, **-script**, **-replay**, or **-noGUI**.

Path

getInput

Arguments**Required argument***prompt*

A String specifying the text field in the dialog box.

Optional argument*default*

A String specifying a default value to be displayed in the text field in the dialog box.

Return value

A String or **None** if the user clicks **Cancel**.

Exceptions

None.

53.5.2 `getInputs(...)`

This method is used to obtain multiple inputs from a user from a dialog box. The method displays a modal dialog box with a column of text fields prefaced by the specified labels. The user enters data in the text fields or accepts the optional default values that are displayed in the text field. When the user clicks the **OK** button, the **getInputs** method reads the data from the dialog box and closes the dialog box.

Note: You cannot use a script that contains **getInputs** if you are running the script from the command line and passing the script name to the command line options **-start**, **-script**, **-replay**, or **-noGUI**.

Path**`getInputs`****Arguments****Required argument***fields*

A sequence of sequence of Strings specifying the text fields in the dialog box. Each inner sequence is a pair of Strings specifying a label that describes a text field along with a default value that appears in the text field. If the field has no default value, you should enter an empty string in the second string in the pair.

Optional arguments

label

A String specifying a label to be placed at the top of the dialog box. The default is an empty string, indicating that no label will be shown.

dialogTitle

A String specifying the text to be shown in the title bar of the dialog box. The default is “Get Inputs”.

Return value

A sequence of Strings representing the data in each of the text fields in the dialog box. If the user clicks the **Cancel** button, the method returns a sequence of **None** objects.

Exceptions

None.

53.5.3 **getWarningReply(...)**

This method is used to obtain a reply from a user from a warning dialog box. The method displays a modal warning dialog box with a message and standard buttons. The user clicks the one of the standard buttons, the **getWarningReply** returns the corresponding button value and closes the dialog box. You can use a “\n” separator in the message argument to provide a multi-line message in the warning dialog box; for example,

```
getWarningReply('Out of disk space!\nOkay to continue', (YES,NO))
```

Note: You cannot use a script that contains **getWarningReply** if you are running the script from the command line and passing the script name to the command line options **-start**, **-script**, **-replay**, or **-noGUI**.

Path

getWarningReply

Arguments

Required argument

message

A String specifying the message in the warning dialog box.

buttons

A Sequence of standard buttons to be displayed in the warning dialog box. Permissible values are YES, NO, YES_TO_ALL and CANCEL.

Optional arguments

None.

Return value

The button that the user clicks.

Exceptions

None.

53.6 **Status commands**

These functions display status information.

53.6.1 **milestone(...)**

This function displays a string in the prompt area.

Path

`milestone`

Arguments**Required argument**

message

A String specifying the text to display.

Optional arguments

None.

Return value

`None`

Exceptions

None.

53.6.2 **milestone(...)**

This function displays a percentage complete message in the prompt area.

UTILITY COMMANDS

Path

milestone

Arguments

Required arguments

message

A String specifying the text to display.

percent

An Int specifying the percentage complete.

Optional arguments

None.

Return value

None

Exceptions

None.

53.6.3 milestone(...)

This function displays a message in the prompt area indicating the number done out of a total. The form of the message is **operation: object nn out of nn**

Path

milestone

Arguments

Required arguments

message

A String specifying the operation.

object

A String specifying the object.

done

An Int specifying the number being processed.

total

An Int specifying the total number.

Optional arguments

None.

Return value

None

Exceptions

None.

53.7 **Callback commands**

These functions execute Python commands or functions.

53.7.1 **addImportCallback(...)**

This function defines a function to be called when a specified Abaqus/CAE module is imported. You cannot specify a custom module.

For more information, see “An example of a callback function,” Section 6.8.3 of the Abaqus Scripting User’s Guide.

Path

addImportCallback

Arguments**Required arguments**

moduleName

A String specifying the name of a specified Abaqus/CAE module.

callback

A Python function to be called. The interface definition of the callback function is :

```
def functionName (moduleName, userData)
```

- *moduleName* is a String.
- *userData* is the object passed as the *userData* argument to the **addImportCallback** method.

Optional argument

userData

Any Python object or **None**. This object is passed to the callback function.

Return value

None

Exceptions

None.

53.7.2 removeImportCallback(...)

This function removes a callback added in **addImportCallback**.

Path

removeImportCallback

Arguments

Required arguments

callback

A Python function to be called; it must be the same as the *callback* argument specified in the original call to **addImportCallback**.

userData

Any Python object or **None**; it must be the same as the *userData* argument specified in the original call to **addImportCallback**.

Optional arguments

None.

Return value

None

Exceptions

None.

53.8 Method callback command

The **methodCallback** module provides functions that allow you to register a callback that will be invoked when certain Abaqus Scripting Interface commands are about to be executed. This callback allows you to augment the standard behavior of Abaqus Scripting Interface commands.

For example, in the following script the **myCallback** function is executed when the **writeInput** method of an object whose type is **JobType** (in other words, any **Job** object) is about to be called:

```
import methodCallback
from job import JobType

def myCallback(callingObject, arguments, keywordArguments, userData):

    print 'An input file is about to be written.'

methodCallback.addCallback(JobType, 'writeInput', myCallback)
```

Access

```
import methodCallback
```

53.8.1 addCallback(...)

This method adds a callback function that will be invoked when certain Abaqus/CAE commands are about to be executed.

Path

```
methodCallback.addCallback
```

Arguments

Required arguments

caller

An object or type object specifying which object will trigger the callback function to be invoked or the Symbolic Constant **ALL_TYPES**.

methodName

A String specifying the name of the method on the caller that will trigger the callback function to be invoked or the Symbolic Constant **ALL_METHODS**.

callback

A Python function to be called when a command matching the specified caller and method name is about to be executed. The interface definition of the callback function is:

```
def functionName(callingMethod, args, keywordArgs, userData)
```

where:

- *callingMethod* is the method that called this function.
- *args* is the sequence of non-keyword arguments that was passed to the calling method.

- *kwargs* is the dictionary of keyword arguments that was passed to the calling method.
- *userData* is the object passed as the *userData* argument to the **addCallback** method.

Optional arguments

userData

Any type of data. This data will be passed to the callback function. The default value is **None**.

callAfter

A Boolean specifying that the callback should be called after the method has executed (instead of before the method is called). The default value is False, which indicates that the callback should be called before the method has executed.

If *callAfter*=True, you can also access the return value of the command from within the callback by including the following statement:

```
returnValue = getMethodReturnValue()
```

The **getMethodReturnValue** function is in the global namespace of the **callback** function.

Return value

None

Exceptions

None.

53.9 Delete object callback command

The **deleteObjectCallback** module provides methods that allow you to register a callback that will be invoked when specified Abaqus Scripting Interface objects are about to be deleted. This callback allows you to examine an object before it is deleted.

For example, in the following script the **myCallback** function is executed when any **Job** object is about to be deleted:

```
import deleteObjectCallback

def myCallback(path, userData):

    print 'About to delete', path

deleteObjectCallback.addCallback(path='mdb.jobs[*]')
```

Access

```
import deleteObjectCallback
```


53.9.1 deleteObjectCallback(...)

This method adds a callback function that will be invoked when the specified Abaqus Scripting Interface objects are about to be deleted. The callback is invoked only when the object is deleted using the Python statement `del object`. The callback is not invoked when the object is deleted using an Abaqus Scripting Interface command such as `mdb.models[name].parts[name].deleteFeature()`.

Path

`deleteObjectCallback.addCallback`

Arguments

Required arguments

callback

A Python function to be called when an object matching the specified path is about to be deleted. The interface definition of the callback function is:

```
def functionName(objectPath, userData)
```

where:

- *objectPath* is the path to the object about to be deleted.
- *userData* is the object passed as the *userData* argument to the `addCallback` method.

path

A String specifying the path to an object or the SymbolicConstant ANY. You can include wildcards in the path to specify a pattern to be matched. Examples of valid paths are:

```
path='mdb.models[*] '
path="mdb.models['Axle*'].parts[*]"
path='mdb.models[*].materials[*] '
```

Optional arguments

userData

Any type of data. This data will be passed to the callback function. The default value is **None**.

includeChildren

A Boolean specifying that the callback should be called if an object owned by the object specified by the *path* argument is about to be deleted. The default value is False.

Return value

None

Exceptions

None.

53.10 Upgrade script commands

The **upgradeScript** module allows you to upgrade scripts from one release of Abaqus to a later release. The **upgradeScript** module allows you to upgrade over several releases at one time. It also allows you to upgrade only kernel commands, only GUI commands, or both.

Access

```
import upgradeScript
```

53.10.1 upgradeScript(...)

This function can be used to upgrade a directory, a file, or a list of both directories and files. You can preview the changes before you choose to upgrade the file. A user interface to this function is available via the Plug-ins menu. For more information, see “Upgrading a script,” Section 82.5 of the Abaqus/CAE User’s Guide. You can also use a simpler interface from the command line to upgrade scripts. For more information, see the summary of changed commands at the end of this guide.

Path

```
upgradeScript.upgradeScript
```

Arguments**Required argument**

fileNames

A String or sequence of Strings specifying the files or directories to upgrade. If a directory path is found, it will be searched for files with the suffix **.py** or **.guiLog**. If *searchSubdirectories* is True, any directories found within a directory will also be searched.

Optional arguments

searchSubdirectories

A Boolean specifying whether to search any subdirectories (if the file name is a directory). If *searchSubdirectories* is True, the command will search the subdirectories of any directory in *fileNames*. If *searchSubdirectories* is False, the command will search the directory but not the subdirectories. The default value is True.

backup

A Boolean specifying whether to make backups of the files. The default value is True. This argument is ignored if *preview* is set.

preview

A Boolean specifying whether to preview the changes to the file instead of changing the file. If *preview* is True, display a preview of the file changes. The default value is True.

By default, the preview is displayed in a web browser. The *diffExecutable* argument allows you to specify a different application in which to preview the changes.

diffExecutable

A String specifying the application used to display the differences between the script and the upgraded script. The default value is an empty string, and the differences are displayed by a web browser. Examples of values for *diffExecutable* are winmerge and diff. This argument is ignored if *preview* is False.

logFileName

A String specifying the name of the log file where any warnings and changes found during the upgrade are written. The default value is **asiUpgrade.log**.

fromVersion

A String specifying the Abaqus release from which to upgrade. The default value is EARLIEST.

toVersion

A String specifying the Abaqus release to which to upgrade. The default value is LATEST.

scriptType

A SymbolicConstant specifying the type of scripting commands to be upgraded. Possible values are KERNEL, GUI, or BOTH. If *scriptType* is KERNEL only Abaqus Scripting Interface commands will be upgraded; if *scriptType* is GUI only Abaqus GUI Toolkit commands will be upgraded. The default value is BOTH.

Return value

The number of changes made or, if *preview* was used, the number of changes that would have been made.

Exceptions

None.

53.11 Customization commands

The following command is used to record a user-defined command in the Abaqus journal file.

53.11.1 journalMethodCall(...)

This function may be used by a user-defined command to record itself in the Abaqus journal file.

For example,

```
def setValues( self, **kargs ):
    for arg,value in kargs.items():
```

UTILITY COMMANDS

```
        setattr(self, arg, value)
    from abaqus import journalMethodCall
    objPath = '%s[%r]' % (self.reposPath, self.name)
    journalMethodCall(objPath, 'setValues', (), kargs)
```

Note: Your command should not call `journalMethodCall` if the command changes the mdb using built-in Abaqus Scripting Interface commands, because built-in commands are journaled by default. A command that changes the mdb customData is one example of a command that should call `journalMethodCall`.

Path

`journalMethodCall`

Arguments

Required arguments

objectPath

A String specifying the path to the object.

methodName

A String specifying the name of the method.

args

A sequence specifying the positional arguments to be written to the journal file.

kargs

A Python dict object specifying the keyword arguments to be written to the journal file.

Optional arguments

None.

Return value

None

Exceptions

None.

54. View commands

54.1 View object

The Session and Viewport View objects store view settings for custom (both predefined and user-defined) views. The paradigm used to define a view is based on a camera analogy. Similar to taking a photograph with a camera, features such as camera position, view direction, orientation, depth of field, and projection are specified to transform three-dimensional views to the screen.

The Layer View objects store a transformation matrix used to position the contents of the Layer within a viewport.

Access

```
session.viewports[name].layers[name].view
session.viewports[name].view
session.views[name]
```

54.1.1 View(...)

This method creates a View object.

Note: All dimensions and coordinates are specified in the model coordinate system.

Note: This method cannot be used to create a View for a Layer object.

Path

```
session.View
```

Required arguments

name

A String specifying the name of the view (also used as the repository key). Possible values are 'Front', 'Back', 'Top', 'Bottom', 'Left', 'Right', 'Iso', 'User-1', 'User-2', 'User-3', and 'User-4'. The object member associated with this argument is a SymbolicConstant. Possible values of the *name* member are: FRONT, BACK, TOP, BOTTOM, LEFT, RIGHT, ISO, USER1, USER2, USER3, and USER4.

fieldOfViewAngle

A Float specifying the viewing angle of the camera. Possible values are $0.0 < \text{fieldOfViewAngle} < 180.0$.

VIEW COMMANDS

nearPlane

A Float specifying the distance from the camera to the near clipping plane. Possible values are *nearPlane* > 0.0.

farPlaneMode

A SymbolicConstant specifying how the distance from the camera to the far clipping plane is set. Possible values are AUTO_COMPUTE and SPECIFY.

farPlane

A Float specifying the distance from the camera to the far clipping plane when *farPlaneMode* =SPECIFY. Possible values are *farPlane* > *nearPlane*.

width

A Float specifying the width of the front clipping plane. Possible values are *width* > 0.0.

height

A Float specifying the height of the front clipping plane. Possible values are *height* > 0.0.

projection

A SymbolicConstant specifying the projection mode. Possible values are PERSPECTIVE and PARALLEL.

cameraPosition

A sequence of three Floats specifying the camera position.

cameraUpVector

A sequence of three Floats specifying the camera's up vector (the screen's positive Y-axis). The initial value is (0, 0, 0).

cameraTarget

A sequence of three Floats specifying the center of the scene.

viewOffsetX

A Float specifying the amount to pan the model in the screen X-direction as a fraction of the viewport width. A positive value pans the model to the right. A negative value pans the model to the left.

The *viewOffsetX* and *viewOffsetY* arguments allow you to pan the view without changing the position of the camera or the target (*cameraPosition* and *cameraTarget* arguments to the **View** method). The resulting change in the view allows you to pan a perspective display without producing an apparent rotation of the model.

viewOffsetY

A Float specifying the amount to pan the model in the screen Y-direction as a fraction of the viewport height. A positive value pans the model upward. A negative value pans the model downward.

autoFit

A Boolean specifying whether the view is auto-fit when applied.

Optional argument*movieMode*

A Boolean specifying whether or not the camera is in movie mode. The default value is OFF.

Return value

A View object.

Exceptions

RangeError.

54.1.2 **fitView(...)**

This method scales the displayable object (such as a part, the assembly, or an X – Y plot) to fit the viewport.

Arguments**Required arguments**

None.

Optional argument*drawImmediately*

A Boolean specifying the viewport should refresh immediately after the command is processed. This is typically only used when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.3 **next(...)**

This method restores the view in the viewport to the next view setting in the list. (There is a list of eight views stored for each viewport.) If there is no next view, no action is taken.

Note: This method is not available for a Layer View.

VIEW COMMANDS

Arguments

Required arguments

None.

Optional argument

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This is typically only used when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.4 **pan(...)**

This method pans the view in the viewport using absolute, not relative, mode.

Arguments

Required arguments

None.

Optional arguments

xFraction

A Float specifying the amount to pan the model in the screen X-direction as a fraction of the viewport width. A positive value pans the model to the right. A negative value pans the model to the left. The default value is 0.0.

yFraction

A Float specifying the amount to pan the model in the screen Y-direction as a fraction of the viewport height. A positive value pans the model upward. A negative value pans the model downward. The default value is 0.0.

asMovie

A Boolean specifying the alternate mode of the **pan** view manipulation should be used. The default value is OFF. This argument is ignored for a Layer View.

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.5 **previous(...)**

This method restores the view in the viewport to the previous view setting in the list. (There is a list of eight views stored for each viewport.) If there is no previous view, no action is taken.

Note: This method is not available for a Layer View.

Arguments**Required arguments**

None.

Optional argument

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.6 **rotate(...)**

This method rotates the view in the viewport. If a center of rotation has been previously specified and *asMovie* is OFF then this method will honor that rotation center.

Arguments**Required arguments**

None.

VIEW COMMANDS

Optional arguments

xAngle

A Float specifying the degrees to rotate about the *X*-axis. The default value is 0.0.

yAngle

A Float specifying the degrees to rotate about the *Y*-axis. The default value is 0.0.

zAngle

A Float specifying the degrees to rotate about the *Z*-axis. The default value is 0.0.

mode

A SymbolicConstant specifying the rotation mode. Possible values are:

- TOTAL : Set the view to (0, 0, 1), then rotate about the screen's axes (an absolute rotation).
- SCREEN : Rotate incrementally about the screen's axes (a relative rotation).
- MODEL : Rotate incrementally about the model's axes (a relative rotation).

The default value is MODEL.

asMovie

A Boolean specifying the alternate mode of the **rotate** view manipulation should be used. The default value is OFF.

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.7 setLayerTransform(...)

This method modifies the transformation used to position a Layer .

Note: This method is not available for Session and Viewport Views.

Arguments

Required arguments

None.

Optional arguments*layerTransform*

A sequence of 16 Floats specifying the transformation matrix.

options

A View object from which the view settings are to be copied. If the *layerTransform* argument is also supplied to **setLayerTransform**, it will override the values in the View object specified by *view*.

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.8 setProjection(...)

This method modifies the appearance of three-dimensional models in the viewport. Choosing PERSPECTIVE makes a model appear more realistic by decreasing the apparent size of features that are farther away from the viewing point.

Note: This method is not available for a Layer View.

Arguments**Required argument***projection*

A SymbolicConstant specifying the projection mode. Possible values are PERSPECTIVE and PARALLEL.

Optional argument*drawImmediately*

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

RangeError.

54.1.9 setRotationCenter(...)

This method sets the center of rotation to the specified location.

Arguments

Required argument

rotationCenter

A sequence of a String and an Int specifying a part instance name and a node label or a sequence of 3 Floats specifying a point.

Optional arguments

None.

Return value

None

Exceptions

TypeError: rotationCenter cannot be set using a part instance and node label unless the displayed object is an ODB

54.1.10 setValues(...)

This method modifies the View object.

Note: This method is not available for a Layer View.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **View** method, except for the *name* and *autoFit* arguments. In addition, **setValues** has the following optional arguments:

options

A View object from which the view settings are to be copied. If other arguments are also supplied to **setValues**, they will override the values in the View object specified by *view*.

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

RangeError.

54.1.11 setViewpoint(...)

This method sets the camera's position in the viewport.

Note: This method is not available for a Layer View.

Arguments**Required argument***viewVector*

A sequence of three Floats specifying the viewing vector (from the camera to the origin of the model).

Optional arguments*cameraUpVector*

A sequence of three Floats specifying the camera's up vector (the screen's positive Y-axis). The initial value is (0, 0, 0).

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

54.1.12 **zoom(...)**

This method magnifies the view in the viewport.

Arguments

Required argument

zoomFactor

A Float specifying the amount to zoom. Possible values are $0.000001 \leq \text{zoomFactor} \leq 1000000$.

A *zoomFactor* less than one reduces the image. A *zoomFactor* greater than one enlarges the image.

Optional arguments

mode

A SymbolicConstant specifying the way the zoom is executed. Possible values are:

- ABSOLUTE : Execute **fitView**, then zoom.
- RELATIVE : Zoom from the current camera settings.

The default value is ABSOLUTE.

asMovie

A Boolean specifying the alternate mode of the **zoom** view manipulation should be used. The default value is OFF. This argument is ignored for a Layer View.

drawImmediately

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

RangeError.

54.1.13 **zoomRectangle(...)**

This method fills the viewport with the graphics located within the given rectangle.

Arguments**Required arguments***point1*

A pair of Floats specifying the X- and Y-coordinates of one corner of the rectangle in fractions of the viewport width and height.

point2

A pair of Floats specifying the X- and Y-coordinates of the other corner of the rectangle in fractions of the viewport width and height.

Optional argument*drawImmediately*

A Boolean specifying the viewport should refresh immediately after the command is processed. This argument is typically used only when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value**None****Exceptions**

None.

54.1.14 Members

The View object has members with the same names and descriptions as the arguments to the **View** method.

In addition, the View object has the following members:

displayedObjectScreenWidth

A Float specifying the width in viewport millimeters of the bounding rectangle around the viewport contents. This value does not include annotations or symbols and it is not clipped to the size of the viewport window.

displayedObjectScreenHeight

A Float specifying the height in viewport millimeters of the bounding rectangle around the viewport contents. This value does not include annotations or symbols and it is not clipped to the size of the viewport window.

layerTransform

A tuple of Floats specifying a transformation matrix used to position the contents of the Layer within a viewport.

55. XY commands

55.1 XYData object

The XYData object is used to store values and attributes associated with XYData type objects.

XYData objects can be created using the methods described below. The methods accessed via the Session object cause the XYData object to be added to the **session.xyData** repository.

Temporary XYData objects will be created if no name is supplied. Temporary XYData objects will be added to the **session.xyData** repository but automatically deleted when they are not used anymore. Temporary XYData objects are also created as a result of math operations found in the **abaqusMath** module.

Access

```
import visualization
session.charts[name].axes1[i].axisData.curves[i].data
session.charts[name].axes2[i].axisData.curves[i].data
session.charts[name].curves[name].data
session.curves[name].data
session.defaultChartOptions.defaultAxis1Options.axisData.curves[i] \
.data
session.defaultChartOptions.defaultAxis2Options.axisData.curves[i] \
.data
import odbAccess
session.odbs[name].userData.xyDataObjects[name]
session.xyDataObjects[name]
session.xyPlots[name].charts[name].axes1[i].axisData.curves[i].data
session.xyPlots[name].charts[name].axes2[i].axisData.curves[i].data
session.xyPlots[name].charts[name].curves[name].data
session.xyPlots[name].curves[name].data
```

55.1.1 XYData(...)

This method creates an XYData object from a sequence of X–Y data pairs.

Path

```
session.XYData
xyPlot.XYData
```

XY COMMANDS

Required argument

data

A sequence of pairs of Floats specifying the *X–Y* data pairs.

Optional arguments

name

The repository key. If the name is not supplied, a default name in the form `_temp#_` is generated and the XYData object is temporary.

sourceDescription

A String specifying the source of the *X–Y* data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

A String specifying the content of the *X–Y* data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

A String specifying additional information about the *X–Y* data (e.g., “for whole model”). The default value is an empty string.

legendLabel

A String specifying the label to be used in the legend. The default value is the name of the XYData object.

xValuesLabel

A String specifying the label for the X-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

yValuesLabel

A String specifying the label for the Y-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

axis1QuantityType

A QuantityType object specifying the QuantityType object associated to the X -axis1- values.

axis2QuantityType

A QuantityType object specifying the QuantityType object associated to the Y -axis2- values.

Return value

An XYData object.

Exceptions

InvalidNameError.

55.1.2 XYData(...)

This method creates an XYData object by copying an existing XYData object.

Path

```
session.odbs[name].userData.XYData
session.XYData
xyPlot.XYData
```

Required argument

objectToCopy

An XYData object to be copied.

Optional arguments

None.

The optional arguments are the same as the optional arguments to the **XYData** method.

Return value

An XYData object.

Exceptions

InvalidNameError.

55.1.3 XYDataFromFile(...)

This method creates an XYData object from data in an ASCII file.

Path

```
session.XYDataFromFile
xyPlot.XYDataFromFile
```

Required argument

fileName

A String specifying the name of the file from which the X–Y data will be read.

Optional arguments

name

The repository key. If the name is not supplied, a default name in the form `_temp#_` is generated and the XYData object is temporary.

XY COMMANDS

sourceDescription

A String specifying the source of the *X–Y* data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

A String specifying the content of the *X–Y* data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

A String specifying additional information about the *X–Y* data (e.g., “for whole model”). The default value is an empty string.

legendLabel

A String specifying the label to be used in the legend. The default value is the name of the XYData object.

xValuesLabel

A String specifying the label for the X-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

yValuesLabel

A String specifying the label for the Y-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

axis1QuantityType

A QuantityType object specifying the QuantityType object associated to the X -axis1- values.

axis2QuantityType

A QuantityType object specifying the QuantityType object associated to the Y -axis2- values.

xField

An Int specifying the field from which the X-data will be read. Fields are delimited by spaces, tabs, or commas. The default value is 1.

yField

An Int specifying the field from which the Y-data will be read. Fields are delimited by spaces, tabs, or commas. The default value is 2.

skipFrequency

An Int specifying how often data rows will be skipped. A *skipFrequency* of 1 means skip every other row. The first row is always read. Possible values are *skipFrequency* ≥ 0 . The default value is 0 (data are read from every row).

Return value

An XYData object.

Exceptions

InvalidNameError and RangeError.

55.1.4 XYDataFromHistory(...)

This method creates an XYData object by reading history data from an Odb object.

Path

`session.XYDataFromHistory`
`xyPlot.XYDataFromHistory`

Required arguments

odb

An Odb object specifying the output database from which data will be read.

outputVariableName

A String specifying the output variable from which the X–Y data will be read.

steps

A sequence of Strings specifying the names of the steps from which data will be extracted.

Optional arguments

name

The repository key. If the name is not supplied, a default name in the form `_temp#_` is generated and the XYData object is temporary (this argument is required if the method is accessed from the `session` object).

sourceDescription

A String specifying the source of the X–Y data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

A String specifying the content of the X–Y data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

A String specifying additional information about the X–Y data (e.g., “for whole model”). The default value is an empty string.

legendLabel

A String specifying the label to be used in the legend. The default value is the name of the XYData object.

skipFrequency

An Int specifying how often data frames will be skipped. If *skipFrequency*=1, Abaqus will skip every other frame. The first frame is always read. Possible values are *skipFrequency* ≥ 0. The default value is 0 (data are read from every frame).

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_VAL_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_VAL_AT_ANGLE. The default value is 0.

stepTuple

A tuple of Integers specifying the steps to include when extracting data.

Return value

An XYData object.

Exceptions

InvalidNameError and RangeError.

55.1.5 xyDataListFromField(...)

This method creates a list of XYData objects by reading field data from an Odb object.

Path

```
session.xyDataListFromField  
xyPlot.xyDataListFromField
```

Required arguments

odb

An Odb object specifying the output database from which data will be read.

outputPosition

A SymbolicConstant specifying the position from which output will be read. Possible values are ELEMENT_CENTROID, ELEMENT_NODAL, INTEGRATION_POINT, and NODAL.

variable

A tuple of tuples containing the descriptions of variables for which to extract data from the field. Each tuple specifies the following:

- Variable label: A String specifying the variable; for example, 'U'.
- Variable output position: A SymbolicConstant specifying the output position. Possible values are ELEMENT_CENTROID, ELEMENT_FACE, ELEMENT_NODAL, GENERAL_PARTICLE, INTEGRATION_POINT, NODAL, WHOLE_ELEMENT, WHOLE_MODEL, WHOLE_PART_INSTANCE, and WHOLE_REGION.

- **Refinement:** A tuple specifying the refinement. If the refinement tuple is omitted, data are written for all components and invariants (if applicable). This element is required if the location dictionary (the following element in the tuple) is included. The refinement tuple contains the following:
 - **Type:** A SymbolicConstant specifying the type of refinement. Possible values are INVARIANT and COMPONENT.
 - **Label:** A String specifying the invariant or the component; for example, 'Mises' or 'S22'.
- **Location:** An optional Dictionary specifying the location. The dictionary contains pairs of the following:
 - A String specifying the category selection label.
 - A String specifying the section point label.

For example,

```
variable= ('S',INTEGRATION_POINT, ( (COMPONENT, 'S22' ), ), )
variable= (('S',INTEGRATION_POINT, ((COMPONENT, 'S11' ), ), ),
          ('U',NODAL,((COMPONENT, 'U1'),)),)
variable= (('S', INTEGRATION_POINT, ((INVARIANT, 'Mises' ), ),
          {'shell < STEEL > < 3 section points >':'SNEG,
          (fraction = -1.0)', }, ), )
```

Optional arguments

At least one of the following arguments *elementSets*, *elementLabels*, *nodeSets*, and *nodeLabels* is required.

elementSets

A sequence of Strings specifying element sets or a String specifying a single element set.

elementLabels

A sequence of expressions specifying element labels per part instance in the model. Each part instance element expression is a sequence of a String specifying the part instance name and a sequence of element expressions; for example, (('partInstance1', (1, '7', '3:15;3'),), ('partInstance2', '8'),).

The element expressions can be any of the following:

- An Int specifying a single element label; for example, 1.
- A String specifying a single element label; for example, '7'.

XY COMMANDS

- A String specifying a sequence of element labels; for example, '3:5' and '3:15:3'.

nodeSets

A sequence of Strings specifying node sets or a String specifying a single node set.

nodeLabels

A sequence of expressions specifying node labels per part instance in the model. Each part instance node expression is a sequence of a String specifying the part instance name and a sequence of node expressions; for example, (('partInstance1', (1, '7', '3:15:3')), ('partInstance2', '8')),)). The node expressions can be any of the following:

- An Int specifying a single node label; for example, 1.
- A String specifying a single node label; for example, '7'.
- A String specifying a sequence of node labels; for example, '3:5' and '3:15:3'.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_VAL_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_VAL_AT_ANGLE. The default value is 0.

Return value

A list of XYData objects.

Exceptions

InvalidNameError and RangeError.

55.1.6 XYDataFromFreeBody(...)

This method creates a list of XYData objects by computing free body data from an Odb object.

Path

```
session.XYDataFromFreeBody  
xyPlot.XYDataFromFreeBody
```

Required arguments

odb

An Odb object specifying the output database from which data will be read.

Optional arguments*force*

A boolean indicating whether to compute the force. The default is ON.

moment

A boolean indicating whether to compute the moment. The default is OFF.

heatFlowRate

A boolean indicating whether to compute the heat flow rate resultant magnitude. It is extracted only for viewcut based freebodies. The default is OFF.

resultant

A boolean indicating whether to compute the resultant. It applies only to *force* and *moment*. The default is ON.

comp1

A boolean indicating whether to compute the first component. It applies only to *force* and *moment*. The default is OFF.

comp2

A boolean indicating whether to compute the second component. It applies only to *force* and *moment*. The default is OFF.

comp3

A boolean indicating whether to compute the third component. It applies only to *force* and *moment*. The default is OFF.

Return value

A list of XYData objects.

Exceptions

InvalidNameError and RangeError.

55.1.7 XYDataFromShellThickness(...)

This method creates a list of XYData objects by reading through the thickness field data from an Odb object.

Path

`xyPlot.XYDataFromShellThickness`

Required arguments*odb*

An Odb object specifying the output database from which data will be read.

outputPosition

A SymbolicConstant specifying the position from which output will be read. Possible values are ELEMENT_CENTROID, ELEMENT_NODAL, INTEGRATION_POINT, and NODAL.

variable

A tuple of tuples containing the descriptions of variables for which to extract data from the field. Each tuple specifies the following:

- Variable label: A String specifying the variable; for example, 'U'.
- Variable output position: A SymbolicConstant specifying the output position. Possible values are ELEMENT_CENTROID, ELEMENT_FACE, ELEMENT_NODAL, GENERAL_PARTICLE, INTEGRATION_POINT, NODAL, WHOLE_ELEMENT, WHOLE_MODEL, WHOLE_PART_INSTANCE, and WHOLE_REGION.
- Refinement: A tuple specifying the refinement. If the refinement tuple is omitted, data are written for all components and invariants (if applicable). This element is required if the location dictionary (the following element in the tuple) is included. The refinement tuple contains the following:
 - Type: A SymbolicConstant specifying the type of refinement. Possible values are INVARIANT and COMPONENT.
 - Label: A String specifying the invariant or the component; for example, 'Mises' or 'S22'.
- Location: An optional Dictionary specifying the location. The dictionary contains pairs of the following:
 - A String specifying the category selection label.
 - A String specifying the section point label.

For example,

```
variable= ('S',INTEGRATION_POINT, ( (COMPONENT, 'S22' ), ), )
variable= (('S',INTEGRATION_POINT, ((COMPONENT, 'S11' ), ), ),
          ('U',NODAL,((COMPONENT, 'U1'),)),)
variable= (('S', INTEGRATION_POINT, ((INVARIANT, 'Mises' ), ),
          {'shell < STEEL > < 3 section points >':'SNEG,
          (fraction = -1.0)', }, ), )
```

Optional arguments

At least one of the following arguments *elementSets*, *elementLabels*, *nodeSets*, and *nodeLabels* is required.

elementSets

A sequence of Strings specifying element sets or a String specifying a single element set.

elementLabels

A sequence of expressions specifying element labels per part instance in the model. Each part instance element expression is a sequence of a String specifying the part instance name and a sequence of element expressions; for example, `(('partInstance1', (1, '7', '3:15;3'), ), ('partInstance2', '8'), )`.

The element expressions can be any of the following:

- An Int specifying a single element label; for example, `1`.
- A String specifying a single element label; for example, `'7'`.
- A String specifying a sequence of element labels; for example, `'3:5'` and `'3:15:3'`.

nodeSets

A sequence of Strings specifying node sets or a String specifying a single node set.

nodeLabels

A sequence of expressions specifying node labels per part instance in the model. Each part instance node expression is a sequence of a String specifying the part instance name and a sequence of node expressions; for example, `(('partInstance1', (1, '7', '3:15;3'), ), ('partInstance2', '8'), )`. The node expressions can be any of the following:

- An Int specifying a single node label; for example, `1`.
- A String specifying a single node label; for example, `'7'`.
- A String specifying a sequence of node labels; for example, `'3:5'` and `'3:15:3'`.

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_VAL_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_VAL_AT_ANGLE. The default value is 0.

Return value

A list of XYData objects.

Exceptions

InvalidNameError and RangeError.

55.1.8 XYDataFromPath(...)

This method creates an XYData object from path information.

Path

`session.XYDataFromPath`

`xyPlot.XYDataFromPath`

Required arguments

path

A Path object to use in X - Y data generation.

name

A String specifying the repository key:

- for *session* 'name' is required argument and for *xyPlot* 'name' is optional argument.

includeIntersections

A Boolean specifying whether to include X - Y data for the intersections between the path and element faces or edges. The default value is False.

shape

A SymbolicConstant specifying the model shape to use. Possible values are UNDEFORMED and DEFORMED.

pathStyle

A SymbolicConstant specifying the path style. Possible values are PATH_POINTS and UNIFORM_SPACING.

numIntervals

An Int specifying the number of uniform-spacing intervals. The default value is 10.

labelType

A SymbolicConstant specifying the X -label type to use. Possible values are NORM_DISTANCE, SEQ_ID, TRUE_DISTANCE, TRUE_DISTANCE_X, TRUE_DISTANCE_Y, TRUE_DISTANCE_Z, X_COORDINATE, Y_COORDINATE and Z_COORDINATE.

Optional arguments

viewport

A String specifying the viewport name or an Int specifying the viewport id from which to obtain values. The default is the current viewport.

step

An Int identifying the step from which to obtain values. The default value is the current step.

frame

An Int identifying the frame from which to obtain values. The default value is the current frame.

variable

A tuple of tuples containing the descriptions of variables for which to extract data along the path. The default value is the current variable. Each tuple specifies the following:

- Variable label: A String specifying the variable; for example, 'U'.

- Variable output position: A SymbolicConstant specifying the output position. Possible values are ELEMENT_CENTROID, ELEMENT_FACE, ELEMENT_NODAL, GENERAL_PARTICLE, INTEGRATION_POINT, NODAL, WHOLE_ELEMENT, WHOLE_MODEL, WHOLE_PART_INSTANCE, and WHOLE_REGION.
- Refinement: A tuple specifying the refinement. If the refinement tuple is omitted, data are written for all components and invariants (if applicable). This element is required if the location dictionary (the following element in the tuple) is included. The refinement tuple contains the following:
 - Type: A SymbolicConstant specifying the type of refinement. Possible values are INVARIANT and COMPONENT.
 - Label: A String specifying the invariant or the component; for example, 'Mises' or 'S22'.
- Location: An optional Dictionary specifying the location. The dictionary contains pairs of the following:
 - A String specifying the category selection label.
 - A String specifying the section point label.

For example,

```
variable= ('S',INTEGRATION_POINT, ( (COMPONENT, 'S22' ), ), )
variable= (('S',INTEGRATION_POINT, ((COMPONENT, 'S11' ), ), ),
          ('U',NODAL,((COMPONENT, 'U1'),)),)
variable= (('S', INTEGRATION_POINT, ((INVARIANT, 'Mises' ), ),
          {'shell < STEEL > < 3 section points >':'SNEG,
          (fraction = -1.0)', }, ), )
```

deformedMag

A tuple of three Floats specifying the deformation magnitude in the X-, Y-, and Z-planes. The default value is (1, 1, 1).

numericForm

A SymbolicConstant specifying the numeric form in which to display results that contain complex numbers. Possible values are COMPLEX_MAGNITUDE, COMPLEX_PHASE, REAL, IMAGINARY, and COMPLEX_VAL_AT_ANGLE. The default value is REAL.

complexAngle

A Float specifying the angle (in degrees) at which to display results that contain complex numbers when *numericForm*=COMPLEX_VAL_AT_ANGLE. The default value is 0.

Return value

If *variable* specified has one fieldoutput: Returns an XYData object.

If *variable* specified has more than one fieldoutputs: Returns list of XYData objects.

Exceptions

If *path* is invalid:

ErrorPathNotFound: Path not found.

If *viewport* is invalid:

ErrorCurrentVPNotFound: Current viewport not found.

If *step* and/or *frame* are invalid:

ErrorInvalidUserStepAndFrame: The user step and frame specified have not been defined.

If the *variable* argument is empty:

ErrorNoVarInPathExtract: No variable selection for XY data extraction from path.

If the specified output variable is not available in the output database:

ErrorUnavailableSelectedVariable: The selected variable is not available for the current frame.

If the specified output variable cannot be used to obtain *X–Y* data:

ErrorUnusableVarInPathExtract: Specified variable cannot be used in XY data extraction from path.

If the SymbolicConstant specifying the refinement type is invalid:

ErrorUnsupportedRefinementType: Unsupported refinement type.

If the label specifying the refinement invariant or component is invalid:

ErrorInvalidRefinementSpecification: Invalid refinement specification.

If *deformedMag* does not contain three Floats:

ErrorDeformedMagTupleInPathExtract: Deformed magnification tuple must contain X, Y and Z values.

55.1.9 save()

This method saves a temporary XYData. The name of the XYData is changed to "XYData-#". If the XYData is already saved, nothing is done.

Arguments

None.

Return value

None

Exceptions

None.

55.1.10 setValues(...)

This method modifies the XYData object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **XYData** method, except for the *name* argument.

Return value

None

Exceptions

None.

55.1.11 Members

The XYData object has members with the same names and descriptions as the arguments to the **XYData** method.

In addition, the XYData object has the following members:

sourceType

A SymbolicConstant specifying the source type of the XYData object. Possible values are FROM_ODB, FROM_KEYBOARD, FROM_ASCII_FILE, FROM_OPERATION, and FROM_USER_DEFINED.

fileName

A String specifying the source file name of the XYData object.

description

A String specifying the complete description of the XYData object.

55.2 Area object

The Area object is used to display a rectangular area in an XYPlot. The Area object has no constructor. Area objects are automatically created whenever a XYPlot , Chart , PlotTitle, or Legend objects are created.

Access

```
import visualization
session.charts[name].area
session.charts[name].gridArea
session.charts[name].legend.area
session.defaultChartOptions.gridArea
session.defaultChartOptions.legend.area
session.defaultPlot.area
session.defaultPlot.title.area
session.xyPlots[name].area
session.xyPlots[name].charts[name].area
session.xyPlots[name].charts[name].gridArea
session.xyPlots[name].charts[name].legend.area
session.xyPlots[name].title.area
```

55.2.1 setValues(...)

This method modifies the Area object.

Arguments

Required arguments

None.

Optional arguments

area

An Area object from which attributes are to be copied.

style

An AreaStyle object.

border

A LineStyle object.

positionMethod

A SymbolicConstant specifying how the area is positioned. Possible values are AUTO_ALIGN and MANUAL. The default value is AUTO_ALIGN.

alignment

A SymbolicConstant specifying the relative position of the area in its parent when *positionMethod*=AUTO_ALIGN. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

sizeMethod

A SymbolicConstant specifying how the area size is defined. Possible values are AUTOMATIC and MANUAL. The default value is AUTOMATIC.

originOffset

A pair of Floats specifying the X- and Y-offsets of the origin as a fraction of the available area. The *originOffset* argument is ignored unless *positionMethod*=MANUAL. The default value is (-1, 0). The valid range for each float is (0, 1).

widthScale

A Float specifying the scale as a fraction of the width of the available area when the *sizeMethod*=MANUAL. The valid range is (0, 1). The default value is 1.0.

heightScale

A Float specifying the scale as a fraction of the height of the available area when the *sizeMethod*=MANUAL. The valid range is (0, 1). The default value is 1.0.

inset

A Boolean specifying whether the area is inset or occupies a reserved area. The default value is OFF.

pl

A Float specifying the left padding of the area in mm. The default value is 1.0.

pr

A Float specifying the right padding of the area in mm. The default value is 1.0.

pt

A Float specifying the top padding of the area in mm. The default value is 1.0.

pb

A Float specifying the bottom padding of the area in mm. The default value is 1.0.

Return value

None

Exceptions

RangeError.

55.2.2 Members

The Area object can have the following members:

inset

A Boolean specifying whether the area is inset or occupies a reserved area. The default value is OFF.

positionMethod

A SymbolicConstant specifying how the area is positioned. Possible values are AUTO_ALIGN and MANUAL. The default value is AUTO_ALIGN.

alignment

A SymbolicConstant specifying the relative position of the area in its parent when *positionMethod*=AUTO_ALIGN. Possible values are:

- BOTTOM_LEFT
- BOTTOM_CENTER
- BOTTOM_RIGHT
- CENTER_LEFT
- CENTER
- CENTER_RIGHT
- TOP_LEFT
- TOP_CENTER
- TOP_RIGHT

The default value is BOTTOM_LEFT.

sizeMethod

A SymbolicConstant specifying how the area size is defined. Possible values are AUTOMATIC and MANUAL. The default value is AUTOMATIC.

width

A Float specifying the width of the area in mm. The default value is 1.0.

height

A Float specifying the height of the area in mm. The default value is 1.0.

widthScale

A Float specifying the scale as a fraction of the width of the available area when the *sizeMethod*=MANUAL. The valid range is (0, 1). The default value is 1.0.

heightScale

A Float specifying the scale as a fraction of the height of the available area when the *sizeMethod*=MANUAL. The valid range is (0, 1). The default value is 1.0.

pl

A Float specifying the left padding of the area in mm. The default value is 1.0.

pr

A Float specifying the right padding of the area in mm. The default value is 1.0.

pt

A Float specifying the top padding of the area in mm. The default value is 1.0.

pb

A Float specifying the bottom padding of the area in mm. The default value is 1.0.

style

An *AreaStyle* object specifying whether and how to fill the area.

border

A *LineStyle* object specifying whether and how to draw the border of the area.

origin

A pair of Floats specifying the X- and Y-offsets in millimeters from the lower-left corner of the XYPlot.

originOffset

A pair of Floats specifying the X- and Y-offsets of the origin as a fraction of the available area. The *originOffset* argument is ignored unless *positionMethod*=MANUAL. The default value is (-1, 0). The valid range for each float is (0, 1).

55.3 **AreaStyle object**

The *AreaStyle* object is used to define how areas are to be filled when drawing XY-plot objects.

AreaStyle objects are automatically created whenever an *Area* object is created. *AreaStyle* objects can be created using the methods described below.

Access

```
import visualization
session.charts[name].area.style
session.charts[name].gridArea.style
session.charts[name].legend.area.style
session.charts[name].tagAreaStyle
```

```
session.defaultChartOptions.areaStyle
session.defaultChartOptions.gridArea.style
session.defaultChartOptions.legend.area.style
session.defaultChartOptions.tagAreaStyle
session.defaultPlot.area.style
session.defaultPlot.title.area.style
session.xyPlots [name] .area.style
session.xyPlots [name] .charts [name] .area.style
session.xyPlots [name] .charts [name] .gridArea.style
session.xyPlots [name] .charts [name] .legend.area.style
session.xyPlots [name] .charts [name] .tagAreaStyle
session.xyPlots [name] .title.area.style
```

55.3.1 AreaStyle(...)

This method creates an AreaStyle.

Path

```
session.AreaStyle
xyPlot.AreaStyle
```

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when filling an area with this AreaStyle object. The default value is "White".

fill

A Boolean specifying whether to fill the area when using this AreaStyle. The default value is ON.

style

A SymbolicConstant specifying the area pattern style to be used when filling an area using this AreaStyle. The default value is SOLID.

Return value

An AreaStyle object.

Exceptions

ColorError

55.3.2 **setValues(...)**

This method modifies the AreaStyle object.

Arguments

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when filling an area with this AreaStyle object. The default value is "White".

fill

A Boolean specifying whether to fill the area when using this AreaStyle. The default value is ON.

style

A SymbolicConstant specifying the area pattern style to be used when filling an area using this AreaStyle. The default value is SOLID.

Return value

None

Exceptions

None.

55.3.3 **Members**

The AreaStyle object has members with the same names and descriptions as the arguments to the **setValues** method.

55.4 **Axis object**

The Axis object is used to store the display attributes of axes. Axes objects are automatically created when adding XYCurve objects to a Chart object.

Access

```
import visualization
session.charts[name].axes1[i]
session.charts[name].axes2[i]
session.defaultChartOptions.defaultAxis1Options
```

```

session.defaultChartOptions.defaultAxis2Options
session.xyPlots [name] .charts [name] .axes1 [i]
session.xyPlots [name] .charts [name] .axes2 [i]

```

55.4.1 **setValues(...)**

This method modifies the Axis object.

Arguments

Required arguments

None.

Optional arguments

axis

An Axis object from which attributes are to be copied.

labelFrequency

An Int specifying the frequency of the labels with respect to the tick marks. The default value is 1.

labelPlacement

A SymbolicConstant specifying how labels are placed on the axis. Possible values are:

- NONE, specifying that no labels are displayed.
- INSIDE, specifying that the labels are placed on the inside of the axis.
- OUTSIDE, specifying that the labels are placed on the outside of the axis.

The default value is INSIDE.

labelStyle

A TextStyle object specifying the text properties to be used when displaying axis labels.

lineStyle

A LineStyle object specifying the line properties used to display the axis.

placement

A SymbolicConstant specifying the placement of the axis on the grid. Possible values are:

- MIN_EDGE, specifying that the axis is placed at the minimum edge - for an abscissa at the bottom, for an ordinate to the left.
- MAX_EDGE, specifying that the axis is placed at the maximum edge - for an abscissa at the top, for an ordinate at the right.
- MIN_MAX_EDGE, specifying that the axis is placed at the minimum edge - for an abscissa at the bottom, for an ordinate to the left - and repeated without labels and title at the maximum edge.
- CENTER, specifying that the axis is placed at the center of the grid.

The default value is MIN_MAX_EDGE.

tickLength

A Float specifying the length of the ticks in mm. The default value is 2.0.

tickPlacement

A SymbolicConstant specifying how tick marks are placed on the axis. Possible values are:

- NONE, specifying that no tick marks are displayed.
- INSIDE, specifying that the tick marks are placed on the inside of the axis.
- OUTSIDE, specifying that the tick marks are placed on the outside of the axis.
- ACROSS, specifying that the tick marks are placed across the axis.

The default value is INSIDE.

tickStyle

A LineStyle object specifying the line properties to be used when displaying axis ticks.

titleStyle

A TextStyle object specifying the text properties to be used when displaying the axis title.

Return value

None

Exceptions

None.

55.4.2 Members

The Axis object can have the following members:

labelFrequency

An Int specifying the frequency of the labels with respect to the tick marks. The default value is 1.

tickLength

A Float specifying the length of the ticks in mm. The default value is 2.0.

placement

A SymbolicConstant specifying the placement of the axis on the grid. Possible values are:

- MIN_EDGE, specifying that the axis is placed at the minimum edge - for an abscissa at the bottom, for an ordinate to the left.
- MAX_EDGE, specifying that the axis is placed at the maximum edge - for an abscissa at the top, for an ordinate at the right.
- MIN_MAX_EDGE, specifying that the axis is placed at the minimum edge - for an abscissa at the bottom, for an ordinate to the left - and repeated without labels and title at the maximum edge.
- CENTER, specifying that the axis is placed at the center of the grid.

The default value is MIN_MAX_EDGE.

tickPlacement

A SymbolicConstant specifying how tick marks are placed on the axis. Possible values are:

- NONE, specifying that no tick marks are displayed.
- INSIDE, specifying that the tick marks are placed on the inside of the axis.
- OUTSIDE, specifying that the tick marks are placed on the outside of the axis.
- ACROSS, specifying that the tick marks are placed across the axis.

The default value is INSIDE.

labelPlacement

A SymbolicConstant specifying how labels are placed on the axis. Possible values are:

- NONE, specifying that no labels are displayed.
- INSIDE, specifying that the labels are placed on the inside of the axis.
- OUTSIDE, specifying that the labels are placed on the outside of the axis.

The default value is INSIDE.

axisData

An AxisData object specifying the numerical data of the axis.

lineStyle

A LineStyle object specifying the line properties used to display the axis.

labelStyle

A TextStyle object specifying the text properties to be used when displaying axis labels.

titleStyle

A TextStyle object specifying the text properties to be used when displaying the axis title.

tickStyle

A LineStyle object specifying the line properties to be used when displaying axis ticks.

55.5 AxisData object

The AxisData object is used to store the data attributes of axes. An AxisData object is automatically created when creating an Axis object.

Access

```
import visualization
session.charts[name].axes1[i].axisData
session.charts[name].axes2[i].axisData
session.defaultChartOptions.defaultAxis1Options.axisData
session.defaultChartOptions.defaultAxis2Options.axisData
session.xyPlots[name].charts[name].axes1[i].axisData
```



```
session.xyPlots [name] .charts [name] .axes2 [i] .axisData
```

55.5.1 **setValues(...)**

This method modifies the AxisData object.

Arguments

Required arguments

None.

Optional arguments

axisData

An AxisData object from which attributes are to be copied.

labelFormat

A SymbolicConstant specifying how tick labels are formatted. Possible values are AUTOMATIC, DECIMAL, SCIENTIFIC, and ENGINEERING. The default value is AUTOMATIC.

labelNumDigits

An Int specifying the number of significant digits displayed for the labels. Possible values are 1 to 7. The default value is 2.

scale

A SymbolicConstant specifying the type of scale to use for the axis. Possible values are:

- LINEAR, specifying tickmarks and labels are linearly distributed.
- LOG, specifying tickmarks and labels are logarithmically distributed.
- DB, specifying tickmarks and labels are distributed on a decibel scale.
- DB2, specifying tickmarks and labels are distributed on a 2*decibel scale.

The default value is LINEAR.

dbReference

A Float specifying the reference value for decibel computation. The default value is 1.0.

minAutoCompute

A Boolean specifying whether or not to use the automatically computed minimum value for the axis. The default value is ON.

minValue

A Float specifying the minimum value when *minAutoCompute* is false. By default, *minValue* is set to *minAutoValue*.

maxAutoCompute

A Boolean specifying whether or not to use the automatically computed maximum value for the axis. The default value is ON.

maxValue

A Float specifying the maximum value when *maxAutoCompute* is false. By default, *maxValue* is set to *maxAutoValue*.

tickMode

A SymbolicConstant specifying the type of scale to use for the axis. Possible values are:

- AUTOCOMPUTE, specifying tickmarks and labels are automatically computed.
- INCREMENT, specifying tickmarks and labels are defined by a given increment.
- TOTAL_NUMBER, specifying tickmarks and labels are defined by the total number of ticks.

The default value is AUTOCOMPUTE.

tickIncrement

A Float specifying the increment of the major tick marks on the axis when *tickMode* = INCREMENT. Valid values are $0 < tickIncrement$. The default value is computed based on the results of the automatic method and the range being plotted. When the *scale* is set to LOG, the *tickIncrement* is interpreted as a value per decade and should be between 0.05 and 1.

tickCount

An Int specifying the number of major tick marks on the axis when *tickMode* = TOTAL_NUMBER. Possible values are $0 \leq tickCount \leq 30$. The default value is computed based on the range of the axis. When the *scale* is set to LOG, the *tickCount* is interpreted as the number of ticks per decade and acceptable values are 1, 4, 8, and 17.

minorTickCount

An Int specifying the number the number of minor tick marks between major ticks. Possible values are $0 \leq minorTickCount \leq 20$. When the *scale* is set to LOG, the *minorTickCount* is interpreted as the number of ticks per decade and limited to 0, 1, 4, 8, and 17. The default value is 1.

title

A String specifying the title of the axis. By default, the title is set to the *systemTitle*.

useSystemTitle

A Boolean specifying whether the title to use for the axis title is system defined or user defined. The default value is ON.

Return value

None

Exceptions

RangeError.

55.5.2 Members

The AxisData object can have the following members:

dbReference

A Float specifying the reference value for decibel computation. The default value is 1.0.

direction

A SymbolicConstant specifying the direction of the axis. Possible values are ABSCISSA and ORDINATE.

labelFormat

A SymbolicConstant specifying how tick labels are formatted. Possible values are AUTOMATIC, DECIMAL, SCIENTIFIC, and ENGINEERING. The default value is AUTOMATIC.

labelNumDigits

An Int specifying the number of significant digits displayed for the labels. Possible values are 1 to 7. The default value is 2.

maxAutoCompute

A Boolean specifying whether or not to use the automatically computed maximum value for the axis. The default value is ON.

maxAutoValue

A Float specifying the maximum value when *maxAutoCompute* is true.

maxValue

A Float specifying the maximum value when *maxAutoCompute* is false. By default, *maxValue* is set to *maxAutoValue*.

maxShownValue

A Float specifying the current maximum value displayed for this axis. This value is different from *maxAutoValue* or *maxValue* when the axis is being transformed (zoom or pan).

minAutoCompute

A Boolean specifying whether or not to use the automatically computed minimum value for the axis. The default value is ON.

minAutoValue

A Float specifying the minimum value when *minAutoCompute* is true.

minValue

A Float specifying the minimum value when *minAutoCompute* is false. By default, *minValue* is set to *minAutoValue*.

minShownValue

A Float specifying the current minimum value displayed for this axis. This value is different from *minAutoValue* or *minValue* when the axis is being transformed (zoom or pan).

minorTickCount

An Int specifying the number the number of minor tick marks between major ticks. Possible values are $0 \leq \text{minorTickCount} \leq 20$. When the *scale* is set to LOG, the *minorTickCount* is interpreted as the number of ticks per decade and limited to 0, 1, 4, 8, and 17. The default value is 1.

scale

A SymbolicConstant specifying the type of scale to use for the axis. Possible values are:

XY COMMANDS

- LINEAR, specifying tickmarks and labels are linearly distributed.
- LOG, specifying tickmarks and labels are logarithmically distributed.
- DB, specifying tickmarks and labels are distributed on a decibel scale.
- DB2, specifying tickmarks and labels are distributed on a 2*decibel scale.

The default value is LINEAR.

tickMode

A SymbolicConstant specifying the type of scale to use for the axis. Possible values are:

- AUTOCOMPUTE, specifying tickmarks and labels are automatically computed.
- INCREMENT, specifying tickmarks and labels are defined by a given increment.
- TOTAL_NUMBER, specifying tickmarks and labels are defined by the total number of ticks.

The default value is AUTOCOMPUTE.

tickCount

An Int specifying the number of major tick marks on the axis when *tickMode*=TOTAL_NUMBER. Possible values are $0 \leq \text{tickCount} \leq 30$. The default value is computed based on the range of the axis. When the *scale* is set to LOG, the tickCount is interpreted as the number of ticks per decade and acceptable values are 1, 4, 8, and 17.

tickCountShown

An Int specifying the number of major ticks effectively shown. This value takes zoom, pan and rounding into account.

tickIncrement

A Float specifying the increment of the major tick marks on the axis when *tickMode* = INCREMENT. Valid values are $0 < \text{tickIncrement}$. The default value is computed based on the results of the automatic method and the range being plotted. When the *scale* is set to LOG, the tickIncrement is interpreted as a value per decade and should be between 0.05 and 1.

tickIncrementShown

A Float specifying the shown tick increment of the major ticks. This value takes zoom/pan into account.

useSystemTitle

A Boolean specifying whether the title to use for the axis title is system defined or user defined. The default value is ON.

curves

An XYCurveArray object specifying a read-only sequence of Curve objects associated to this axis.

quantityType

A QuantityType object specifying the quantity type: i.e. the physical dimension and associated label of the data represented by this axis.

tickValues

A tuple of Floats specifying the read-only major tick values shown.

tickLabels

A tuple of Strings specifying the read-only major tick labels shown.

systemTitle

A String specifying the system title. The system title is based on the *quantityType* of the axis and associated curves.

title

A String specifying the title of the axis. By default, the title is set to the *systemTitle*.

55.6 Chart object

The Chart object is used to display XYCurve objects. A Chart object is automatically created when creating an XYPlot object

Access

```
import visualization
session.charts[name]
session.xyPlots[name].charts[name]
```

55.6.1 autoColor(...)

This method distributes the colors on all curves displayed in the chart using the color palette defined by the xyColors object.

Arguments

Required arguments

None.

Optional arguments

lines

A Boolean defining whether color distribution affects curve lines.

symbols

A Boolean defining whether color distribution affects curve symbols.

Return value

None

Exceptions

None.

55.6.2 autoSymbol()

This method distributes the symbols on all curves displayed in the chart.

Arguments

None.

Return value

None

Exceptions

None.

55.6.3 fitCurves()

This method resets the transform of the chart. It cancels any zoom or pan action.

Arguments

None.

Return value

None

Exceptions

None.

55.6.4 getAxis1(...)

This method returns the Axis object used for displaying the Axis1 of the XYCurve specified by name or object or used for the given QuantityType object.

Arguments

Required arguments

curve

The name or the XYCurve object associated to the Axis object.

quantityType

The QuantityType object associated to the Axis object.

Optional arguments

None.

Return value

An Axis object.

Exceptions

If the given XYCurve is not used in the Chart.

XypError: Curve not found:

If both arguments are specified.

TypeError: Specify curve or quantityType; too many arguments; expected 1, got 2.

If the given QuantityType is not used in the Chart.

ValueError: QuantityType not found

55.6.5 getAxis2(...)

This method returns the Axis object used for displaying the Axis2 of the XYCurve specified by name or object or used for the given QuantityType object.

Arguments**Required arguments**

curve

The name or the XYCurve object associated to the Axis object.

quantityType

The QuantityType object associated to the Axis object.

Optional arguments

None.

Return value

An Axis object.

Exceptions

If the given XYCurve is not used in the Chart.

XypError: Curve not found:

If both arguments are specified.

TypeError: Specify curve or quantityType; too many arguments; expected 1, got 2.

If the given QuantityType is not used in the Chart.

ValueError: QuantityType not found

55.6.6 **moveAxisUp(...)**

This method moves the relative position of the given Axis object up in the axis sequence of the Chart.

Arguments

Required argument

axis

The Axis object to be moved.

Optional arguments

None.

Return value

None

Exceptions

None.

55.6.7 **moveAxisDown(...)**

This method moves the relative position of the given Axis object down in the axis sequence of the Chart.

Arguments

Required argument

axis

The Axis object to be moved.

Optional arguments

None.

Return value

None

Exceptions

None.

55.6.8 **removeCurve(...)**

This method removes the given XYCurve from the Chart.

Arguments**Required argument***curve*

The XYCurve name or the XYCurve object or a sequence of XYCurve names or XYCurve objects to be removed from the Chart.

Optional arguments

None.

Return value

None

Exceptions

None.

55.6.9 setValues(...)

This method modifies the Chart object.

Arguments**Required arguments**

None.

Optional arguments*chart*

A Chart object from which attributes are to be copied.

curvesToPlot

A sequence of Strings specifying the names of the curves to plot. In addition to this type, the argument can also be one of the following:

- A String specifying the name of the curve to plot.
- An XYCurve object specifying the curve to plot.
- A sequence of XYCurve objects specifying the curves to plot (as returned by the **curveSet** method).

aspectRatio

A Float specifying the aspect ratio of the grid area. A value of -1 specifies that the gridArea will take up all available space. The default value is -1.

transform

A sequence of Floats specifying a transformation matrix used to scale or pan along the axes of the Chart.

XY COMMANDS

view

A View object.

useQuantityType

A Boolean specifying whether to use the QuantityType to associate curves with axes. The default value is ON.

Return value

None

Exceptions

RangeError.

55.6.10 Members

The Chart object can have the following members:

name

A String specifying the name of the Chart object.

useQuantityType

A Boolean specifying whether to use the QuantityType to associate curves with axes. The default value is ON.

aspectRatio

A Float specifying the aspect ratio of the grid area. A value of -1 specifies that the gridArea will take up all available space. The default value is -1.

curves

A repository of XYCurve objects specifying a repository of XYCurve objects to display in the Chart.

axes1

An AxisArray object specifying a read-only sequence of axis objects displayed as axes1 - the abscissa for a Cartesian chart.

axes2

An AxisArray object specifying a read-only sequence of axis objects displayed as axes2 - the ordinate for a Cartesian chart.

area

An Area object specifying position, padding, background and borders of the chart.

gridArea

An Area object specifying how to display the grid area.

legend

A Legend object specifying the attributes for the legend of the chart.

majorAxis1GridStyle

A *LineStyle* object specifying the line properties to be used when drawing major gridlines along axis 1.

majorAxis2GridStyle

A *LineStyle* object specifying the line properties to be used when drawing major gridlines along axis 2.

minorAxis1GridStyle

A *LineStyle* object specifying the line properties to be used when drawing minor gridlines along axis 1.

minorAxis2GridStyle

A *LineStyle* object specifying the line properties to be used when drawing minor gridlines along axis 2.

tagTextStyle

A *TextStyle* object specifying the text properties to be used when creating tags.

tagAreaStyle

An *AreaStyle* object specifying the area properties to be used when creating tags.

tagBorder

A *LineStyle* object specifying the tag area border properties to be used when creating tags.

transform

A tuple of Floats specifying a transformation matrix used to scale or pan along the axes of the Chart.

55.7 DefaultChartOptions object

The *DefaultChartOptions* object is used to hold on default chart and axis attributes. The *DefaultChartOptions* object attributes are used whenever *Chart* or *Axis* are created. A *DefaultChartOptions* object is automatically created when opening a session.

Access

```
import visualization
session.defaultChartOptions
```

55.7.1 setValues(...)

This method modifies the *DefaultChartOptions* object.

Arguments

Required arguments

None.

Optional arguments

areaStyle

An AreaStyle object specifying an AreaStyle used to hold on to the default display properties for the chart area.

aspectRatio

A Float specifying the default aspect ratio of the grid area. A value of -1 specifies that the gridArea will take up all available space. The default value is -1.

defaultAxis1Options

An Axis object specifying an Axis object used to hold on to the default properties for direction 1 axes—the abscissa for a Cartesian chart.

defaultAxis2Options

An Axis object specifying an Axis object used to hold on to the default properties for direction 2 axes—the ordinate for a Cartesian chart.

gridArea

An Area object specifying how to display the grid area by default.

legend

A Legend object specifying the default attributes for the legend of the chart.

majorAxis1GridStyle

A LineStyle object specifying the default line properties to be used when drawing major gridlines along axis 1.

majorAxis2GridStyle

A LineStyle object specifying the default line properties to be used when drawing major gridlines along axis 2.

minorAxis1GridStyle

A LineStyle object specifying the default line properties to be used when drawing minor gridlines along axis 1.

minorAxis2GridStyle

A LineStyle object specifying the default line properties to be used when drawing minor gridlines along axis 2.

tagAreaStyle

An AreaStyle object specifying the default area properties to be used when creating tags.

tagBorder

A LineStyle object specifying the default tag area border properties to be used when creating tags.

tagTextStyle

A TextStyle object specifying the default text properties to be used when creating tags.

useQuantityType

A Boolean specifying whether to use the `QuantityType` to associate curves with axes. The default value is ON.

Return value

None

Exceptions

None.

55.7.2 Members

The `DefaultChartOptions` object has members with the same names and descriptions as the arguments to the `setValues` method.

55.8 DefaultPlot object

The `DefaultPlot` object is used to hold on default plot attributes. The `DefaultPlot` object attributes are used whenever an `XYPlot` object is created. A `DefaultPlot` object is automatically created when opening a session.

Access

```
import visualization
session.defaultPlot
```

55.8.1 Members

The `DefaultPlot` object can have the following members:

area

An `Area` object specifying an `Area` used to hold on to the default display properties for the plot area.

title

A `Title` object specifying a `Title` object used to hold on to the default properties of the `XY-Plot` title.

55.9 Legend object

The Legend object is used to store the display attributes of the chart legend. A legend object is automatically created when creating a Chart object.

Access

```
import visualization
session.charts[name].legend
session.defaultChartOptions.legend
session.xyPlots[name].charts[name].legend
```

55.9.1 setValues(...)

This method modifies the Legend object.

Arguments

Required arguments

None.

Optional arguments

legend

A Legend object from which attributes are to be copied.

show

A Boolean specifying whether to show the legend. The default value is ON.

showMinMax

A Boolean specifying whether to display the minimum and maximum values. The default value is OFF.

title

A String specifying the title to appear on the legend. The default value is an empty string.

numberFormat

A SymbolicConstant specifying how the minimum and maximum values are formatted. Possible values are AUTOMATIC, DECIMAL, SCIENTIFIC, and ENGINEERING. The default value is AUTOMATIC.

numDigits

An Int specifying the number of significant digits displayed for the minimum and maximum values. Possible values are 1 to 7. The default value is 2.

textStyle

A TextStyle object specifying the text properties used to display the legend text.

titleStyle

A TextStyle object specifying the text properties used to display the legend title.

Return value

None

Exceptions

None.

55.9.2 Members

The Legend object can have the following members:

numberFormat

A SymbolicConstant specifying how the minimum and maximum values are formatted. Possible values are AUTOMATIC, DECIMAL, SCIENTIFIC, and ENGINEERING. The default value is AUTOMATIC.

numDigits

An Int specifying the number of significant digits displayed for the minimum and maximum values. Possible values are 1 to 7. The default value is 2.

show

A Boolean specifying whether to show the legend. The default value is ON.

showMinMax

A Boolean specifying whether to display the minimum and maximum values. The default value is OFF.

area

An Area object specifying the area of the legend.

textStyle

A TextStyle object specifying the text properties used to display the legend text.

title

A String specifying the title to appear on the legend. The default value is an empty string.

titleStyle

A TextStyle object specifying the text properties used to display the legend title.

55.10 LineStyle object

The LineStyle object is used to define the line style to be used for drawing XY-Plot objects.

LineStyle objects can be created using the methods described below.

Access

```

import visualization
session.charts[name].area.border
session.charts[name].axes1[i].axisData.curves[i].lineStyle
session.charts[name].axes1[i].lineStyle
session.charts[name].axes1[i].tickStyle
session.charts[name].axes2[i].axisData.curves[i].lineStyle
session.charts[name].axes2[i].lineStyle
session.charts[name].axes2[i].tickStyle
session.charts[name].curves[name].lineStyle
session.charts[name].gridArea.border
session.charts[name].legend.area.border
session.charts[name].majorAxis1GridStyle
session.charts[name].majorAxis2GridStyle
session.charts[name].minorAxis1GridStyle
session.charts[name].minorAxis2GridStyle
session.charts[name].tagBorder
session.curves[name].lineStyle
session.defaultChartOptions.defaultAxis1Options.axisData.curves[i] \
.lineStyle
session.defaultChartOptions.defaultAxis1Options.lineStyle
session.defaultChartOptions.defaultAxis1Options.tickStyle
session.defaultChartOptions.defaultAxis2Options.axisData.curves[i] \
.lineStyle
session.defaultChartOptions.defaultAxis2Options.lineStyle
session.defaultChartOptions.defaultAxis2Options.tickStyle
session.defaultChartOptions.gridArea.border
session.defaultChartOptions.legend.area.border
session.defaultChartOptions.majorAxis1GridStyle
session.defaultChartOptions.majorAxis2GridStyle
session.defaultChartOptions.minorAxis1GridStyle
session.defaultChartOptions.minorAxis2GridStyle
session.defaultChartOptions.tagBorder
session.defaultPlot.area.border
session.defaultPlot.title.area.border
session.xyPlots[name].area.border
session.xyPlots[name].charts[name].area.border
session.xyPlots[name].charts[name].axes1[i].axisData.curves[i] \
.lineStyle
session.xyPlots[name].charts[name].axes1[i].lineStyle

```



```

session.xyPlots[name].charts[name].axes1[i].tickStyle
session.xyPlots[name].charts[name].axes2[i].axisData.curves[i] \
.lineStyle
session.xyPlots[name].charts[name].axes2[i].lineStyle
session.xyPlots[name].charts[name].axes2[i].tickStyle
session.xyPlots[name].charts[name].curves[name].lineStyle
session.xyPlots[name].charts[name].gridArea.border
session.xyPlots[name].charts[name].legend.area.border
session.xyPlots[name].charts[name].majorAxis1GridStyle
session.xyPlots[name].charts[name].majorAxis2GridStyle
session.xyPlots[name].charts[name].minorAxis1GridStyle
session.xyPlots[name].charts[name].minorAxis2GridStyle
session.xyPlots[name].charts[name].tagBorder
session.xyPlots[name].curves[name].lineStyle
session.xyPlots[name].title.area.border

```

55.10.1 LineStyle(...)

This method creates a LineStyle.

Path

```

session.LineStyle
xyPlot.LineStyle

```

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing a line with this LineStyle object. The default value is "White".

show

A Boolean specifying whether to draw the line when using this LineStyle. The default value is ON.

style

A SymbolicConstant specifying the line style to be used when drawing lines using this LineStyle. Possible values are SOLID, DASHED, DOTTED, and DOT_DASH. The default value is SOLID.

thickness

A Float specifying the line thickness in mm to be used when drawing lines using this LineStyle. The default value is 0.2.

Return value

A `LineStyle` object.

Exceptions

`ColorError`

55.10.2 `setValues(...)`

This method modifies the `LineStyle` object.

Arguments

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing a line with this `LineStyle` object. The default value is "White".

show

A Boolean specifying whether to draw the line when using this `LineStyle`. The default value is ON.

style

A `SymbolicConstant` specifying the line style to be used when drawing lines using this `LineStyle`. Possible values are `SOLID`, `DASHED`, `DOTTED`, and `DOT_DASH`. The default value is `SOLID`.

thickness

A Float specifying the line thickness in mm to be used when drawing lines using this `LineStyle`. The default value is 0.2.

Return value

None

Exceptions

None.

55.10.3 **Members**

The `LineStyle` object has members with the same names and descriptions as the arguments to the `setValues` method.

55.11 QuantityType object

The QuantityType object is used to store attributes defining the physical dimension and label of the quantity type to be associated with an XYData or an AxisData object.

QuantityType objects are automatically created when creating XYData objects from the odb.

QuantityType objects can be created using the methods described below.

Access

```
import visualization
session.charts[name].axes1[i].axisData.curves[i].data\
.axis1QuantityType
session.charts[name].axes1[i].axisData.curves[i].data\
.axis2QuantityType
session.charts[name].axes1[i].axisData.quantityType
session.charts[name].axes2[i].axisData.curves[i].data\
.axis1QuantityType
session.charts[name].axes2[i].axisData.curves[i].data\
.axis2QuantityType
session.charts[name].axes2[i].axisData.quantityType
session.charts[name].curves[name].data.axis1QuantityType
session.charts[name].curves[name].data.axis2QuantityType
session.curves[name].data.axis1QuantityType
session.curves[name].data.axis2QuantityType
session.defaultChartOptions.defaultAxis1Options.axisData.curves[i] \
.data.axis1QuantityType
session.defaultChartOptions.defaultAxis1Options.axisData.curves[i] \
.data.axis2QuantityType
session.defaultChartOptions.defaultAxis1Options.axisData.quantityType
session.defaultChartOptions.defaultAxis2Options.axisData.curves[i] \
.data.axis1QuantityType
session.defaultChartOptions.defaultAxis2Options.axisData.curves[i] \
.data.axis2QuantityType
session.defaultChartOptions.defaultAxis2Options.axisData.quantityType
import odbAccess
session.odbs[name].userData.axis1QuantityType
session.odbs[name].userData.axis2QuantityType
import visualization
import xyPlot
session.odbs[name].userData.xyDataObjects[name].axis1QuantityType
```

```

session.odbs[name].userData.xyDataObjects[name].axis2QuantityType
session.xyDataObjects[name].axis1QuantityType
session.xyDataObjects[name].axis2QuantityType
session.xyPlots[name].charts[name].axes1[i].axisData.curves[i].data\
.axis1QuantityType
session.xyPlots[name].charts[name].axes1[i].axisData.curves[i].data\
.axis2QuantityType
session.xyPlots[name].charts[name].axes1[i].axisData.quantityType
session.xyPlots[name].charts[name].axes2[i].axisData.curves[i].data\
.axis1QuantityType
session.xyPlots[name].charts[name].axes2[i].axisData.curves[i].data\
.axis2QuantityType
session.xyPlots[name].charts[name].axes2[i].axisData.quantityType
session.xyPlots[name].charts[name].curves[name].data.axis1QuantityType
session.xyPlots[name].charts[name].curves[name].data.axis2QuantityType
session.xyPlots[name].curves[name].data.axis1QuantityType
session.xyPlots[name].curves[name].data.axis2QuantityType

```

55.11.1 QuantityType(...)

This method creates a QuantityType object.

Path

```

session.QuantityType
xyPlot.QuantityType

```

Required argument

type

A SymbolicConstant specifying the physical dimension of the axis. Possible values are:

- NONE.
- ACCELERATION.
- ACOUSTIC_INTENSITY.
- ANGLE.
- ANGULAR_MOMENTUM.
- ARC_LENGTH.
- AREA.
- AREA_VELOCITY_SQUARED, specifying "Velocity squared per area".
- BIMOMENT.
- CURVATURE.

- CORIOLIS_LOAD.
- DAMAGE.
- DAMAGE_CRITERION.
- DENSITY.
- DENSITY_ROTATIONAL_ACCELERATION, specifying "Density * Angular acceleration".
- DISPLACEMENT.
- ECURRENT_AREA_TIME, specifying "Time integrated electric current per area".
- ELECTRIC_CHARGE.
- ELECTRIC_CURRENT.
- ELECTRIC_CURRENT_AREA, specifying "Electric current per unit area".
- ELECTRIC_POTENTIAL.
- ENERGY.
- ENERGY_DENSITY.
- ENERGY_RELEASE_RATE.
- EPOTENTIAL_GRADIENT, specifying "Electric potential gradient".
- FREQUENCY.
- FORCE.
- FORCE_VOLUME, specifying "Force per volume".
- HEAT_FLUX.
- HEAT_FLUX_AREA, specifying "Heat flux per area".
- HEAT_FLUX_RATE.
- HEAT_FLUX_VOLUME, specifying "Heat flux per volume".
- LENGTH.
- LINEAR_PRESSURE.
- LUMIN, specifying "Luminous intensity".
- MASS.
- MASS_FLOW_AREA, specifying "Mass flow per area".
- MASS_FLOW_AREA_RATE, specifying "Mass flow rate per area".
- MASS_FLOW_RATE.
- MODE_NUMBER.
- MOMENT.
- NUMBER.
- PATH.
- PHASE.
- POSITION.

XY COMMANDS

- PRESSURE.
- PRESSURE_GRADIENT.
- RATE.
- ROTARY_INERTIA.
- ROTATIONAL_ACCELERATION.
- ROTATIONAL_VELOCITY.
- STATUS.
- STRAIN.
- STRAIN_RATE.
- STRESS.
- STRESS_INTENS_FACTOR, specifying "Stress intensity factor".
- SUBSTANCE, specifying "Amount of substance".
- TEMPERATURE.
- THICKNESS.
- TIME.
- TIME_INCREMENT.
- TIME_HEAT_FLUX, specifying "Time integrated heat flux".
- TIME_HEAT_FLUX_AREA, specifying "Time integrated heat flux per area".
- TIME_VOLUME, specifying "Time integrated volume".
- TIME_VOLUME_FLUX, specifying "Time integrated volume flux per area".
- TWIST.
- VELOCITY.
- VELOCITY_SQUARED.
- VOLUME.
- VOLUME_FLUX.
- VOLUME_FLUX_AREA, specifying "Volume flux per area".
- VOLUME_FRACTION.

Optional argument

label

A String specifying the label for this quantity type.

Return value

A QuantityType object.

Exceptions

None.

55.11.2 setValues(...)

This method modifies the QuantityType object.

Arguments**Required arguments**

None.

Optional arguments

type

A SymbolicConstant specifying the physical dimension of the axis. Possible values are:

- NONE.
- ACCELERATION.
- ACOUSTIC_INTENSITY.
- ANGLE.
- ANGULAR_MOMENTUM.
- ARC_LENGTH.
- AREA.
- AREA_VELOCITY_SQUARED, specifying "Velocity squared per area".
- BIMOMENT.
- CURVATURE.
- CORIOLIS_LOAD.
- DAMAGE.
- DAMAGE_CRITERION.
- DENSITY.
- DENSITY_ROTATIONAL_ACCELERATION, specifying "Density * Angular acceleration".
- DISPLACEMENT.
- ECURRENT_AREA_TIME, specifying "Time integrated electric current per area".
- ELECTRIC_CHARGE.
- ELECTRIC_CURRENT.
- ELECTRIC_CURRENT_AREA, specifying "Electric current per unit area".
- ELECTRIC_POTENTIAL.
- ENERGY.

XY COMMANDS

- ENERGY_DENSITY.
- ENERGY_RELEASE_RATE.
- EPOTENTIAL_GRADIENT, specifying "Electric potential gradient".
- FREQUENCY.
- FORCE.
- FORCE_VOLUME, specifying "Force per volume".
- HEAT_FLUX.
- HEAT_FLUX_AREA, specifying "Heat flux per area".
- HEAT_FLUX_RATE.
- HEAT_FLUX_VOLUME, specifying "Heat flux per volume".
- LENGTH.
- LINEAR_PRESSURE.
- LUMIN, specifying "Luminous intensity".
- MASS.
- MASS_FLOW_AREA, specifying "Mass flow per area".
- MASS_FLOW_AREA_RATE, specifying "Mass flow rate per area".
- MASS_FLOW_RATE.
- MODE_NUMBER.
- MOMENT.
- NUMBER.
- PATH.
- PHASE.
- POSITION.
- PRESSURE.
- PRESSURE_GRADIENT.
- RATE.
- ROTARY_INERTIA.
- ROTATIONAL_ACCELERATION.
- ROTATIONAL_VELOCITY.
- STATUS.
- STRAIN.
- STRAIN_RATE.
- STRESS.
- STRESS_INTENS_FACTOR, specifying "Stress intensity factor".
- SUBSTANCE, specifying "Amount of substance".

- TEMPERATURE.
- THICKNESS.
- TIME.
- TIME_INCREMENT.
- TIME_HEAT_FLUX, specifying "Time integrated heat flux".
- TIME_HEAT_FLUX_AREA, specifying "Time integrated heat flux per area".
- TIME_VOLUME, specifying "Time integrated volume".
- TIME_VOLUME_FLUX, specifying "Time integrated volume flux per area".
- TWIST.
- VELOCITY.
- VELOCITY_SQUARED.
- VOLUME.
- VOLUME_FLUX.
- VOLUME_FLUX_AREA, specifying "Volume flux per area".
- VOLUME_FRACTION.

label

A String specifying the label for this quantity type.

Return value

None

Exceptions

None.

55.11.3 Members

The QuantityType object has members with the same names and descriptions as the arguments to the `setValues` method.

55.12 SymbolStyle object

The SymbolStyle object is used to define the marker properties to be used when drawing curves. SymbolStyle objects can be created using the methods described below.

Access

```
import visualization
```

```

session.charts [name] .axes1 [i] .axisData.curves [i] .symbolStyle
session.charts [name] .axes2 [i] .axisData.curves [i] .symbolStyle
session.charts [name] .curves [name] .symbolStyle
session.curves [name] .symbolStyle
session.defaultChartOptions.defaultAxis1Options.axisData.curves [i] \
.symbolStyle
session.defaultChartOptions.defaultAxis2Options.axisData.curves [i] \
.symbolStyle
session.xyPlots [name] .charts [name] .axes1 [i] .axisData.curves [i] \
.symbolStyle
session.xyPlots [name] .charts [name] .axes2 [i] .axisData.curves [i] \
.symbolStyle
session.xyPlots [name] .charts [name] .curves [name] .symbolStyle
session.xyPlots [name] .curves [name] .symbolStyle

```

55.12.1 SymbolStyle(...)

This method creates a SymbolStyle object.

Path

```

session.SymbolStyle
xyPlot.SymbolStyle

```

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing a marker with this SymbolStyle object. The default value is "White".

show

A Boolean specifying whether to draw the marker when using this SymbolStyle object. The default value is ON.

marker

A SymbolicConstant specifying the marker type be used when drawing symbols using this SymbolStyle object. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE
- FILLED_DIAMOND
- FILLED_TRI

- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER
- POINT

The default value is FILLED_CIRCLE.

size

A Float specifying the marker size to be used when drawing markers using this SymbolStyle object. The default value is 2.0.

Return value

A SymbolStyle object.

Exceptions

ColorError

55.12.2 setValues(...)

This method modifies the SymbolStyle object.

Arguments

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing a marker with this SymbolStyle object. The default value is "White".

show

A Boolean specifying whether to draw the marker when using this SymbolStyle object. The default value is ON.

marker

A SymbolicConstant specifying the marker type be used when drawing symbols using this SymbolStyle object. Possible values are:

- FILLED_CIRCLE
- FILLED_SQUARE

XY COMMANDS

- FILLED_DIAMOND
- FILLED_TRI
- HOLLOW_CIRCLE
- HOLLOW_SQUARE
- HOLLOW_DIAMOND
- HOLLOW_TRI
- CROSS
- XMARKER
- POINT

The default value is FILLED_CIRCLE.

size

A Float specifying the marker size to be used when drawing markers using this SymbolStyle object.
The default value is 2.0.

Return value

None

Exceptions

None.

55.12.3 Members

The SymbolStyle object has members with the same names and descriptions as the arguments to the `setValues` method.

55.13 TextStyle object

The TextStyle object is used to store the text properties to be used for drawing XY-plot text objects.

TextStyle objects are automatically created when creating a chart or can be created with methods described below.

Access

```
import visualization
session.charts[name].axes1[i].labelStyle
session.charts[name].axes1[i].titleStyle
session.charts[name].axes2[i].labelStyle
session.charts[name].axes2[i].titleStyle
```

```

session.charts [name] .legend.textStyle
session.charts [name] .legend.titleStyle
session.charts [name] .tagTextStyle
session.defaultChartOptions.defaultAxis1Options.labelStyle
session.defaultChartOptions.defaultAxis1Options.titleStyle
session.defaultChartOptions.defaultAxis2Options.labelStyle
session.defaultChartOptions.defaultAxis2Options.titleStyle
session.defaultChartOptions.legend.textStyle
session.defaultChartOptions.legend.titleStyle
session.defaultChartOptions.tagTextStyle
session.defaultPlot.title.titleStyle
session.xyPlots [name] .charts [name] .axes1 [i] .labelStyle
session.xyPlots [name] .charts [name] .axes1 [i] .titleStyle
session.xyPlots [name] .charts [name] .axes2 [i] .labelStyle
session.xyPlots [name] .charts [name] .axes2 [i] .titleStyle
session.xyPlots [name] .charts [name] .legend.textStyle
session.xyPlots [name] .charts [name] .legend.titleStyle
session.xyPlots [name] .charts [name] .tagTextStyle
session.xyPlots [name] .title.titleStyle

```

55.13.1 TextStyle(...)

This method creates a TextStyle.

Path

```

session.TextStyle
xyPlot.TextStyle

```

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing text with this TextStyle object. The default value is "White".

show

A Boolean specifying whether to draw the text when using this TextStyle object. The default value is ON.

font

A String specifying the name of the font to be used when drawing text with this TextStyle object. The default value is "-*-verdana-medium-r-normal-*-*-120-*-*-p-*-*-*".

XY COMMANDS

rotationAngle

A Float specifying the angle in degrees used for displaying the text. The default value is 0.0.

Return value

A TextStyle object.

Exceptions

ColorError

55.13.2 setValues(...)

This method modifies the TextStyle object.

Arguments

Required arguments

None.

Optional arguments

color

A String specifying the color to be used when drawing text with this TextStyle object. The default value is "White".

show

A Boolean specifying whether to draw the text when using this TextStyle object. The default value is ON.

font

A String specifying the name of the font to be used when drawing text with this TextStyle object. The default value is "-*-verdana-medium-r-normal-*-*-120-*-*-p-*-*-".

rotationAngle

A Float specifying the angle in degrees used for displaying the text. The default value is 0.0.

Return value

None

Exceptions

None.

55.13.3 Members

The TextStyle object has members with the same names and descriptions as the arguments to the **setValues** method.

55.14 Title object

The Title object is used to store the display attributes of the XYPlot title. An Title object is automatically created when creating a XYPlot object.

Access

```
import visualization
session.defaultPlot.title
session.xyPlots[name].title
```

55.14.1 setValues(...)

This method modifies the Title object.

Arguments

Required arguments

None.

Optional arguments

title

A Title object from which attributes are to be copied.

text

A String specifying the text to appear as a title. By default the title is set to the XYPlot object name. The default value is an empty string.

area

An Area object specifying the area of the title.

useDefault

A Boolean specifying whether to show the default title. The default value is OFF.

titleStyle

A TextStyle object specifying the text properties used to display the legend title.

Return value

None

Exceptions

None.

55.14.2 Members

The Title object can have the following members:

useDefault

A Boolean specifying whether to show the default title. The default value is OFF.

area

An Area object specifying the area of the title.

text

A String specifying the text to appear as a title. By default the title is set to the XYPlot object name. The default value is an empty string.

titleStyle

A TextStyle object specifying the text properties used to display the legend title.

55.15 XYCurve object

The XYCurve object is used to plot X–Y data and to store its display attributes.

Access

```
import visualization
session.charts[name].axes1[i].axisData.curves[i]
session.charts[name].axes2[i].axisData.curves[i]
session.charts[name].curves[name]
session.curves[name]
session.defaultChartOptions.defaultAxis1Options.axisData.curves[i]
session.defaultChartOptions.defaultAxis2Options.axisData.curves[i]
session.xyPlots[name].charts[name].axes1[i].axisData.curves[i]
session.xyPlots[name].charts[name].axes2[i].axisData.curves[i]
session.xyPlots[name].charts[name].curves[name]
session.xyPlots[name].curves[name]
```

55.15.1 Curve(...)

This method creates an XYCurve object from an XYData object.

Path

```
session.Curve
```


Required arguments*name*

A String specifying the repository key.

data

An XYData object specifying the data for the curve.

Optional arguments

None.

Return value

An XYCurve object.

Exceptions

InvalidNameError.

55.15.2 setValues(...)

This method modifies the XYCurve object.

Arguments**Required arguments**

None.

Optional arguments*displayTypes*

A sequence of SymbolicConstants specifying that describe how curves are to be displayed. Possible values are LINE and SYMBOL. The default value is (LINE).

legendLabel

A String specifying the label to be displayed in the legend. By default, the label is system defined.

*symbolFrequency*An Int specifying the frequency of plotting the markers. Possible values are *symbolFrequency* > 0. If *symbolFrequency*=1, then markers are plotted at every point. The default value is 1.*useDefault*

A Boolean specifying whether to use the system supplied legend label. The default value is ON.

Return value

None

Exceptions

None.

55.15.3 Members

The XYCurve object can have the following members:

name

A String specifying the repository key.

symbolFrequency

An Int specifying the frequency of plotting the markers. Possible values are *symbolFrequency* > 0. If *symbolFrequency*=1, then markers are plotted at every point. The default value is 1.

useDefault

A Boolean specifying whether to use the system supplied legend label. The default value is ON.

legendSource

A SymbolicConstant specifying how the system supplied, default legend label is to be generated. Possible values are CURVE_LEGEND, CURVE_NAME, and CURVE_NAME_LEGEND. The default value is CURVE_LEGEND.

data

An XYData object specifying the data for the curve.

lineStyle

A LineStyle object specifying the line properties used to display the curve.

symbolStyle

A SymbolStyle object specifying the symbol properties used to display the curve.

legendLabel

A String specifying the label to be displayed in the legend. By default, the label is system defined.

displayTypes

A tuple of SymbolicConstants specifying that describe how curves are to be displayed. Possible values are LINE and SYMBOL. The default value is (LINE).

55.16 XYPlot object

The XYPlot object is used to display Chart objects.

Access

```
import visualization
session.xyPlots [name]
```

55.16.1 XYPlot(...)

This method creates an empty XYPlot object.

Path

`session.XYPlot`

Required argument

name

A String specifying the name of the XYPlot object.

Optional arguments

None.

Return value

An XYPlot object.

Exceptions

InvalidNameError.

55.16.2 autoColor(...)

This method distributes the colors on all curves displayed in the XYPlot using the color palette defined by the xyColors AutoColors object.

Arguments

Required arguments

None.

Optional arguments

lines

A Boolean defining whether color distribution affects curve lines.

symbols

A Boolean defining whether color distribution affects curve symbols.

Return value

None

Exceptions

None.

55.16.3 autoSymbol()

This method distributes the symbols on all curves displayed in the XYPlot.

Arguments

None.

Return value

None

Exceptions

None.

55.16.4 fitCurves()

This method resets the transform of all the charts of the XYPlot object. It cancels any zoom or pan action.

Arguments

None.

Return value

None

Exceptions

None.

55.16.5 next(...)

This method restores the *transform* member of the active Chart object to the next setting in the transform list. (There is a list of eight transforms stored for each chart.) If there is no next transform, no action is taken.

Arguments

Required arguments

None.

Optional argument*drawImmediately*

A Boolean specifying the viewport should refresh immediately after the command is processed. This is typically only used when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

55.16.6 previous(...)

This method restores the *transform* member of the active Chart object to the previous setting in the transform list. (There is a list of eight transforms stored for each chart.) If there is no next transform, no action is taken.

Arguments**Required arguments**

None.

Optional argument*drawImmediately*

A Boolean specifying the viewport should refresh immediately after the command is processed. This is typically only used when writing a script and it is desirable to show intermediate results before the script completes. The default value is False.

Return value

None

Exceptions

None.

55.16.7 setValues(...)

This method modifies the XYPlot object.

Arguments

Required arguments

None.

Optional arguments

title

A Title object specifying the title of the XYPlot object.

transform

A sequence of Floats specifying a transformation matrix used to scale or pan along the axes of the active Chart object of this XYPlot.

Return value

None

Exceptions

None.

55.16.8 Members

The XYPlot object has members with the same names and descriptions as the arguments to the **XYPlot** method.

In addition, the XYPlot object can have the following members:

area

An Area object specifying position, padding, background and borders of the XYPlot object.

title

A Title object specifying the title of the XYPlot object.

charts

A repository of Chart objects.

curves

A repository of XYCurve objects.

transform

A tuple of Floats specifying a transformation matrix used to scale or pan along the axes of the active Chart object of this XYPlot.

55.17 XYReportOptions object

The XYReportOptions object stores settings used by the **writeXYReport** method when you write an XYData object to an ASCII file. The XYReportOptions object has no constructor. Abaqus creates the *xyReportOptions* member when you import the Visualization module.

Access

```
import visualization
session.defaultXYReportOptions
session.xyReportOptions
```

55.17.1 setValues(...)

This method modifies the XYReportOptions object.

Arguments

Required arguments

None.

Optional arguments

pageWidth

An Int specifying the number of characters per line of the report file when *pageWidthLimited*=ON. Possible values are *pageWidth* > 0. The default value is 80.

numDigits

An Int specifying the number of significant digits to be included for each data value in the report file. Possible values are $0 \leq \text{numDigits} \leq 9$. The default value is 6.

interpolation

A Boolean specifying whether to perform linear interpolation for missing data values. The default value is OFF.

xyData

A Boolean specifying whether to print the X- and Y-values of the selected XYData objects. (If *xyData*=OFF, *totals* and *minMax* can still be printed.) The default value is ON.

totals

A Boolean specifying whether to print the sum of the Y-values of the selected XYData objects. The default value is OFF.

minMax

A Boolean specifying whether to print the minimum and maximum X- and Y-values of the selected XYData objects. The default value is OFF.

XY COMMANDS

pageWidthLimited

A Boolean specifying whether the page width is limited. The default value is OFF.

numberFormat

A SymbolicConstant specifying the number format to be used in reporting XYData objects. Possible values are AUTOMATIC, ENGINEERING, and SCIENTIFIC. The default value is ENGINEERING.

layout

A SymbolicConstant specifying the format used in reporting the XYData objects. Possible values are SINGLE_TABLE and SEPARATE_TABLES. The default value is SINGLE_TABLE.

Return value

None

Exceptions

RangeError.

If *xyData*, *total*, and *minMax* are all OFF:

At least one of the data print methods must be selected

55.17.2 Members

The XYReportOptions object has members with the same names and descriptions as the arguments to the **setValues** method.

55.18 Session object

The following commands operate on Session objects. For more information about the Session object, see “Session object,” Section 47.1.

Access

import visualization

55.18.1 linearizeStress(...)

This method is used to perform stress linearization along a defined stress line.

Arguments**Required arguments***name*

A String specifying the name of the stress line.

path

A Path object specifying the end points of the stress line.

Note: You must provide either the *path* argument or the *startPoint* and *endPoint* arguments.

startPoint

A String specifying a part instance and a node belonging to that part instance, or a sequence of three Floats specifying the coordinates of a point.

endPoint

A String specifying a part instance and a node belonging to that part instance, or a sequence of three Floats specifying the coordinates of a point.

modelShape

A SymbolicConstant specifying the model shape to be used for obtaining coordinates of the intermediate points along the stress line. Possible values are UNDEFORMED and DEFORMED. When DEFORMED is selected, the deformation scale factor is always set to a uniform value of 1.0.

components

A sequence of Strings specifying the linearized stress components to be displayed in the xyPlot object. Possible values are "S11", "S22", "S33", "S12", "S23", and "S13".

Note: You must provide at least one of the *components* , *xyMembraneComps* , and *xyBendingComps* arguments.

xyMembraneComps

A sequence of Strings specifying the linearized stress membrane components to be displayed in the xyPlot object. Possible values are "S11", "S22", "S33", "S12", "S23", and "S13".

xyBendingComps

A sequence of Strings specifying the linearized stress bending components to be displayed in the xyPlot object. Possible values are "S11", "S22", "S33", "S12", "S23", and "S13".

Optional arguments*invariantBendingComps*

A sequence of Strings specifying the linearized stress bending components to be included in the computation of the linearized stress invariants. Possible values are "S11", "S22", "S33", "S12", "S23", and "S13". The default value of the sequence includes all the possible values.

XY COMMANDS

intervals

An Int specifying the number of equally-spaced intervals into which the stress line is to be divided. The default value is 40.

radiusOfCurvature

A Float specifying the in-plane radius of curvature of the center section along the stress line for axi-symmetric models. The default value is ∞ .

oopRadiusOfCurvature

A Float specifying the out-of-plane radius of curvature of the center section along the Stress Line for axi-symmetric models. The default value forces Abaqus to compute the radius of curvature.

curvatureCorrection

A Boolean specifying whether the out-of-plane curvature correction should be applied for non-axisymmetric models when performing linearized stress computations. Curvature correction is always applied for axisymmetric models. The default value is OFF.

curvatureCsys

A String specifying the name of the user-defined local coordinate system to be used for computing the out-of-plane radius of curvature for non-axisymmetric models when *curvatureCorrection* =ON . The default value uses the global coordinate system to compute the out-of-plane radius of curvature.

useCurvatureCsysForOrient

A Boolean specifying whether to use the user-defined local coordinate system to compute the local stress line orientation for non-axisymmetric models when *curvatureCorrection* =ON and *curvatureCsys* is specified. The default value is OFF.

saveXy

A Boolean specifying whether to save the xyData objects created during the stress linearization operation to the session. The default value is OFF .

writeReport

A Boolean specifying whether to write the output from the stress linearization operation to a text file. The default value is ON .

reportFile

A String specifying the report file name. The default value is “linearStress.rpt”.

appendToFile

A Boolean specifying whether to append output from the stress linearization to the text file specified by the *reportFile* argument. The default value is ON .

saveToPath

A Boolean specifying whether to create a Path object containing all the points used when performing the linearized stress computation. The default value is OFF . When this argument is set to ON , a Path object is created with the same name as that of the stress line and placed in the **Path** repository.

Return value

A list of xyData objects.

Exceptions

None.

Text Error

The stress line passes through incompatible part instances. Results cannot be extracted.

Text Error

Specified variables cannot be used for extracting results along the stress line.

Text Error

Results for all the points along the stress line were not found. Selected end points and all intermediate sample points must lie within the current display group, and within solid continuum elements. Due to tolerance limitations, intermediate points lying on display group boundaries may fail to meet this criterion. Choose new end points, adjust the number of intervals along the line, or modify the current display group to obtain the stress linearization plot.

55.18.2 setPathTolerance(...)

This method is used to set the *tolerance* to be used when creating XYData objects by extracting results along Path objects. This command should be exercised with caution since setting a value too low or too high may result in getting no results or unpredictable results.

Arguments**Required arguments**

None.

Optional argument

tolerance

A Double specifying the tolerance. The default value is 0.00001.

Return value

None

Exceptions

None.

55.18.3 **getPathTolerance()**

This method is used to get the *tolerance* used when creating XYData objects by extracting results along Path objects.

Arguments

None.

Return value

A Double specifying the tolerance.

Exceptions

None.

55.18.4 **setLimitForXYDataPlots(...)**

This method is used to set the *limit* for number of XY data objects while creating XYData from field output.

Arguments

Required arguments

None.

Optional argument

limit

None or an Int specifying the limit for number of XY data objects. The default value is no limit

Return value

None

Exceptions

None.

55.18.5 **getLimitForXYDataPlots()**

This method is used to get the *limit* for number of XY data objects while creating XYData from field output.

Arguments

None.

Return value

Int specifying the limit for number of XY data objects.

Exceptions

None.

55.19 **writeXYReport**

This method writes an XYData object to a user-defined ASCII file.

55.19.1 **writeXYReport(...)**

This method writes an XYData object to a user-defined ASCII file.

Path

`session.writeXYReport`

Arguments**Required arguments**

fileName

A String specifying the name of the file to which X–Y data will be written.

xyData

A sequence of XYData objects to be written to the output file.

Optional argument

appendMode

A Boolean specifying whether to append the X–Y data to the existing file. The default value is ON.

Return value

None

Exceptions

None.

Part II: C++ commands

56. Amplitude commands

Amplitude commands are used to create arbitrary time or frequency variations of load, displacement, and some interaction attributes throughout a step using step time or throughout an analysis using total time.

56.1 Amplitude object

The Amplitude object is the abstract base type for other Amplitude objects. The Amplitude object has no explicit constructor. The methods and members of the Amplitude object are common to all objects derived from the Amplitude.

Access

`amplitudeApi.amplitudes()` [*name*]

56.1.1 Members

The Amplitude object has the following members:

Prototype

```
odb_String name() const;
odb_String timeSpan() const;
```

name

An odb_String specifying the repository key.

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

56.2 ActuatorAmplitude object

The ActuatorAmplitude object defines an actuator amplitude curve.

The ActuatorAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes()` [*name*]

56.2.1 **ActuatorAmplitude(...)**

This method creates a `ActuatorAmplitude` object.

Path

`amplitudeApi.ActuatorAmplitude`

Prototype

```
odb_ActuatorAmplitude&  
ActuatorAmplitude(const odb_String& name,  
                  const odb_String& timeSpan);
```

Required argument

name

An `odb_String` specifying the repository key.

Optional argument

timeSpan

An `odb_String` specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

An `ActuatorAmplitude` object.

Exceptions

`InvalidNameError` and `RangeError`.

56.2.2 **setValues(...)**

This method modifies the `ActuatorAmplitude` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ActuatorAmplitude` method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.2.3 Members

The `ActuatorAmplitude` object has members with the same names and descriptions as the arguments to the `ActuatorAmplitude` method.

56.2.4 Corresponding analysis keywords

*AMPLITUDE

56.3 BaselineCorrection object

The `BaselineCorrection` object modifies an acceleration history to minimize the overall drift of the displacement obtained from the time integration of the given acceleration.

Access

```
amplitudeApi.amplitudes() [name].baselineCorrection()
```

56.3.1 BaselineCorrection(...)

This method creates a `BaselineCorrection` object.

Path

```
amplitudeApi.amplitudes() [name].BaselineCorrection
```

Prototype

```
odb_BaselineCorrection&  
BaselineCorrection(const odb_SequenceDouble& intervals);
```

Required arguments

None.

Optional argument

intervals

An odb_SequenceDouble specifying the correction time interval end points. Possible values are positive and monotonically increasing Floats. The default value is an empty sequence.

Return value

A BaselineCorrection object.

Exceptions

RangeError.

56.3.2 **setValues(...)**

This method modifies the BaselineCorrection object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **BaselineCorrection** method.

Return value

None

Exceptions

RangeError.

56.3.3 **Members**

The BaselineCorrection object has members with the same names and descriptions as the arguments to the **BaselineCorrection** method.

56.3.4 **Corresponding analysis keywords**

*BASELINE CORRECTION

56.4 DecayAmplitude object

The DecayAmplitude object defines an amplitude curve using an exponential decay.

The DecayAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes()` [*name*]

56.4.1 DecayAmplitude(...)

This method creates a DecayAmplitude object.

Path

`amplitudeApi.DecayAmplitude`

Prototype

```
odb_DecayAmplitude&
DecayAmplitude(const odb_String& name,
               double initial,
               double maximum,
               double start,
               double decayTime,
               const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

initial

A Double specifying the constant A_0 .

maximum

A Double specifying the coefficient A .

start

A Double specifying the starting time t_0 . Possible values are non-negative numbers.

decayTime

A Double specifying the decay time t_d . Possible values are non-negative numbers.

Optional argument

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A DecayAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.4.2 **setValues(...)**

This method modifies the DecayAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **DecayAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.4.3 **Members**

The DecayAmplitude object has members with the same names and descriptions as the arguments to the **DecayAmplitude** method.

56.4.4 **Corresponding analysis keywords**

*AMPLITUDE

56.5 EquallySpacedAmplitude object

The EquallySpacedAmplitude object defines a list of amplitude values at fixed time intervals beginning at a specified value of time.

The EquallySpacedAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes()` [*name*]

56.5.1 EquallySpacedAmplitude(...)

This method creates an EquallySpacedAmplitude object.

Path

`amplitudeApi.EquallySpacedAmplitude`

Prototype

```
odb_EquallySpacedAmplitude&
EquallySpacedAmplitude(const odb_String& name,
                        double fixedInterval,
                        const odb_SequenceDouble& data,
                        double begin,
                        odb_Union smooth,
                        const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

fixedInterval

A Double specifying the fixed time interval at which the amplitude data are given. Possible values are positive numbers.

data

An odb_SequenceDouble specifying the amplitude values.

Optional arguments

begin

A Double specifying the time at which the first amplitude data are given. Possible values are non-negative numbers. The default value is 0.0.

AMPLITUDE COMMANDS

smooth

The string "SOLVER_DEFAULT" or a Double specifying the degree of smoothing. Possible float values are $0 \leq \textit{smoothing} \leq 0.5$. If *smooth*="SOLVER_DEFAULT", the default degree of smoothing will be determined by the solver. The default value is "SOLVER_DEFAULT".

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

An EquallySpacedAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.5.2 setValues(...)

This method modifies the EquallySpacedAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **EquallySpacedAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.5.3 Members

The EquallySpacedAmplitude object has members with the same names and descriptions as the arguments to the **EquallySpacedAmplitude** method.

In addition, the EquallySpacedAmplitude object can have the following member:

Prototype

```
odb_BaselineCorrection baselineCorrection() const;
```


baselineCorrection

A BaselineCorrection object.

56.5.4 Corresponding analysis keywords

*AMPLITUDE

56.6 ModulatedAmplitude object

The ModulatedAmplitude object defines a modulated amplitude curve.

The ModulatedAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes() [name]`

56.6.1 ModulatedAmplitude(...)

This method creates a ModulatedAmplitude object.

Path

`amplitudeApi.ModulatedAmplitude`

Prototype

```
odb_ModulatedAmplitude&
ModulatedAmplitude(const odb_String& name,
                    double initial,
                    double magnitude,
                    double start,
                    double frequency1,
                    double frequency2,
                    const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

AMPLITUDE COMMANDS

initial

A Double specifying the constant A_0 .

magnitude

A Double specifying the coefficient A .

start

A Double specifying the starting time t_0 . Possible values are non-negative numbers.

frequency1

A Double specifying the circular frequency 1 (ω_1). Possible values are positive numbers.

frequency2

A Double specifying the circular frequency 2 (ω_2). Possible values are positive numbers.

Optional argument

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A ModulatedAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.6.2 setValues(...)

This method modifies the ModulatedAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ModulatedAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.6.3 Members

The `ModulatedAmplitude` object has members with the same names and descriptions as the arguments to the `ModulatedAmplitude` method.

56.6.4 Corresponding analysis keywords

*AMPLITUDE

56.7 PeriodicAmplitude object

The `PeriodicAmplitude` object defines an amplitude curve using a Fourier series.

The `PeriodicAmplitude` object is derived from the `Amplitude` object.

Access

`amplitudeApi.amplitudes()` [*name*]

56.7.1 PeriodicAmplitude(...)

This method creates a `PeriodicAmplitude` object.

Path

`amplitudeApi.PeriodicAmplitude`

Prototype

```
odb_PeriodicAmplitude&
PeriodicAmplitude(const odb_String& name,
                  double frequency,
                  double start,
                  double a_0,
                  const odb_SequenceSequenceDouble& data,
                  const odb_String& timeSpan);
```

Required arguments

name

An `odb_String` specifying the repository key.

AMPLITUDE COMMANDS

frequency

A Double specifying the circular frequency ω . Possible values are positive numbers.

start

A Double specifying the starting time t_0 . Possible values are positive numbers.

a_0

A Double specifying the constant A_0 .

data

An odb_SequenceSequenceDouble specifying A_i and B_i pairs.

Optional argument

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A PeriodicAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.7.2 **setValues(...)**

This method modifies the PeriodicAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PeriodicAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.7.3 Members

The `PeriodicAmplitude` object has members with the same names and descriptions as the arguments to the `PeriodicAmplitude` method.

56.7.4 Corresponding analysis keywords

*AMPLITUDE

56.8 PsdDefinition object

The `PsdDefinition` object defines the cross-spectral density frequency function for random response loading.

The `PsdDefinition` object is derived from the `Amplitude` object.

Access

`Api.amplitudes()` [*name*]

56.8.1 PsdDefinition(...)

This method creates a `PsdDefinition` object.

Path

`Api.PsdDefinition`

Prototype

```
odb_PsdDefinition&
PsdDefinition(const odb_String& name,
               const odb_SequenceSequenceDouble& data,
               const odb_String& unitType,
               double referenceGravityAcceleration,
               double referencePower,
               bool user,
               const odb_String& timeSpan,
               const odb_String& amplitude);
```

Required arguments

name

An odb_String specifying the repository key.

data

An odb_SequenceSequenceDouble specifying the real part of the frequency function, the imaginary part of the frequency function, and the frequency or frequency band number values, depending on the value of *unitType*.

Optional arguments

unitType

An odb_String specifying the type of units for specifying the frequency function. "FORCE" implies power units. "BASE" implies gravity used to define base motion. "DB" implies decibel units. Possible values are "FORCE", "BASE", and "DB". The default value is "FORCE".

referenceGravityAcceleration

A Double specifying the reference gravity acceleration. This argument applies when *unitType* = "BASE". The default value is 1.0.

referencePower

A Double specifying the reference power value, in load units squared. This argument applies when *unitType* = "DB". The default value is 0.0.

user

A Boolean specifying whether the frequency function is defined in user subroutine UPSD. If specified, then *data* is not applicable, and the *unitType* value must not be "DB". The default value is false.

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

amplitude

An odb_String specifying the name of the amplitude that describes the dynamic event used to define the cross-spectral density frequency function. The default value is an empty string.

Return value

A PsdDefinition object.

Exceptions

InvalidNameError and RangeError.

56.8.2 **setValues(...)**

This method modifies the PsdDefinition object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **PsdDefinition** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.8.3 Members

The PsdDefinition object has members with the same names and descriptions as the arguments to the **PsdDefinition** method.

56.8.4 Corresponding analysis keywords

*PSD-DEFINITION

56.9 SmoothStepAmplitude object

The SmoothStepAmplitude object defines an amplitude that ramps up or down smoothly from one data point to another.

The SmoothStepAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes() [name]`

56.9.1 SmoothStepAmplitude(...)

This method creates a SmoothStepAmplitude object.

Path

`amplitudeApi.SmoothStepAmplitude`

Prototype

```
odb_SmoothStepAmplitude&  
SmoothStepAmplitude(const odb_String& name,  
                     const odb_SequenceSequenceDouble& data,  
                     const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

data

An odb_SequenceSequenceDouble specifying time/frequency and amplitude pairs. Possible values for time/frequency are positive numbers.

Optional argument

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A SmoothStepAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.9.2 **setValues(...)**

This method modifies the SmoothStepAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SmoothStepAmplitude** method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

56.9.3 Members

The `SmoothStepAmplitude` object has members with the same names and descriptions as the arguments to the `SmoothStepAmplitude` method.

56.9.4 Corresponding analysis keywords

*AMPLITUDE

56.10 SolutionDependentAmplitude object

The `SolutionDependentAmplitude` object defines a solution-dependent amplitude for superplastic forming analysis.

The `SolutionDependentAmplitude` object is derived from the `Amplitude` object.

Access

`amplitudeApi.amplitudes()` [*name*]

56.10.1 SolutionDependentAmplitude(...)

This method creates a `SolutionDependentAmplitude` object.

Path

`amplitudeApi.SolutionDependentAmplitude`

Prototype

```
odb_SolutionDependentAmplitude&
SolutionDependentAmplitude(const odb_String& name,
                           double initial,
```

AMPLITUDE COMMANDS

```
double minimum,  
double maximum,  
const odb_String& timeSpan);
```

Required argument

name

An odb_String specifying the repository key.

Optional arguments

initial

A Double specifying the initial amplitude value. Possible values are those between *minimum* and *maximum*. The default value is 1.0.

minimum

A Double specifying the minimum amplitude value. Possible values are those smaller than *maximum* and *initial*. The default value is 0.1.

maximum

A Double specifying the maximum amplitude value. Possible values are those larger than *minimum* and *initial*. The default value is 1000.0.

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A SolutionDependentAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.10.2 setValues(...)

This method modifies the SolutionDependentAmplitude object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SolutionDependentAmplitude** method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

56.10.3 Members

The `SolutionDependentAmplitude` object has members with the same names and descriptions as the arguments to the `SolutionDependentAmplitude` method.

56.10.4 Corresponding analysis keywords

*AMPLITUDE

56.11 SpectrumAmplitude object

The `SpectrumAmplitude` object defines the spectrum of responses for displacement, velocity, or acceleration to be used in a response spectrum analysis.

The `SpectrumAmplitude` object is derived from the `Amplitude` object.

Access

`Api.amplitudes() [name]`

56.11.1 SpectrumAmplitude(...)

This method creates a `SpectrumAmplitude` object.

Path

`Api.SpectrumAmplitude`

Prototype

```
odb_SpectrumAmplitude&
SpectrumAmplitude(const odb_String& name,
                  const odb_String& method,
```

```
const odb_SequenceSequenceDouble& data,
const odb_String& specificationUnits,
const odb_String& eventUnits,
const odb_String& solution,
double timeIncrement,
double gravity,
bool criticalDamping,
const odb_String& timeSpan,
const odb_String& amplitude);
```

Required arguments

name

An odb_String specifying the repository key.

method

An odb_String specifying the method for specifying the spectrum. Possible values are "DEFINE" and "CALCULATE".

data

An odb_SequenceSequenceDouble specifying the magnitude, frequency, and damping values.

Optional arguments

specificationUnits

An odb_String specifying the units used for specifying the spectrum. Possible values are "DISPLACEMENT", "VELOCITY", "ACCELERATION", and "GRAVITY". The default value is "ACCELERATION".

eventUnits

An odb_String specifying the units used to describe the dynamic event in the amplitude used for the calculation. Possible values are "EVENT_DISPLACEMENT", "EVENT_VELOCITY", "EVENT_ACCELERATION", and "EVENT_GRAVITY". The default value is "EVENT_ACCELERATION".

solution

An odb_String specifying the solution method for the dynamic equations. Possible values are "ABSOLUTE_VALUE" and "RELATIVE_VALUE". The default value is "ABSOLUTE_VALUE".

timeIncrement

A Double specifying the implicit time increment used to calculate the spectrum. This argument is required when the *method* = "CALCULATE". The default value is 0.0.

gravity

A Double specifying the acceleration due to gravity. This argument applies only when *specificationUnits* = "GRAVITY" or *eventUnits* = "GRAVITY". The default value is 1.0.

criticalDamping

A Boolean specifying whether to calculate the spectrum for only the specified range of critical damping values or a list of values. If *criticalDamping* = true, the spectrum is calculated only for the specified range of critical damping values. If *criticalDamping* = false, the spectrum is calculated for a list of damping values. The default value is false.

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

amplitude

An odb_String specifying the name of the amplitude that describes the dynamic event used to calculate the spectrum. The default value is an empty string.

Return value

A SpectrumAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.11.2 setValues(...)

This method modifies the SpectrumAmplitude object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **SpectrumAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.11.3 Members

The SpectrumAmplitude object has members with the same names and descriptions as the arguments to the **SpectrumAmplitude** method.

56.11.4 Corresponding analysis keywords

*SPECTRUM

56.12 TabularAmplitude object

The TabularAmplitude object defines an amplitude curve as a table of values at convenient points on the time scale.

The TabularAmplitude object is derived from the Amplitude object.

Access

`amplitudeApi.amplitudes() [name]`

56.12.1 TabularAmplitude(...)

This method creates a TabularAmplitude object.

Path

`amplitudeApi.TabularAmplitude`

Prototype

```
odb_TabularAmplitude&
TabularAmplitude(const odb_String& name,
                  const odb_SequenceSequenceDouble& data,
                  odb_Union smooth,
                  const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

data

An odb_SequenceSequenceDouble specifying time/frequency and amplitude pairs. Possible values for time/frequency are positive numbers.

Optional arguments*smooth*

The string "SOLVER_DEFAULT" or a Double specifying the degree of smoothing. Possible float values are between 0 and 0.5. If *smooth*="SOLVER_DEFAULT", the default degree of smoothing will be determined by the solver. The default value is "SOLVER_DEFAULT".

timeSpan

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A TabularAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.12.2 setValues(...)

This method modifies the TabularAmplitude object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TabularAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.12.3 Members

The TabularAmplitude object has members with the same names and descriptions as the arguments to the **TabularAmplitude** method.

In addition, the TabularAmplitude object can have the following member:

Prototype

```
odb_BaselineCorrection baselineCorrection() const;
```

baselineCorrection

A BaselineCorrection object.

56.12.4 Corresponding analysis keywords

*AMPLITUDE

56.13 UserAmplitude object

The UserAmplitude object defines an amplitude curve using the **UAMP** or **VUAMP** user subroutine.

The UserAmplitude object is derived from the Amplitude object.

Access

```
amplitudeApi.amplitudes() [name]
```

56.13.1 UserAmplitude(...)

This method creates a UserAmplitude object.

Path

```
amplitudeApi.UserAmplitude
```

Prototype

```
odb_UserAmplitude&  
UserAmplitude(const odb_String& name,  
               int numVariables,  
               const odb_String& timeSpan);
```

Required arguments

name

An odb_String specifying the repository key.

numVariables

An Int specifying the number of variables for the **UAMP** or **VUAMP** user subroutine.

Optional argument*timeSpan*

An odb_String specifying the time span of the amplitude. Possible values are "STEP" and "TOTAL". The default value is "STEP".

Return value

A UserAmplitude object.

Exceptions

InvalidNameError and RangeError.

56.13.2 setValues(...)

This method modifies the UserAmplitude object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **UserAmplitude** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

56.13.3 Members

The UserAmplitude object has members with the same names and descriptions as the arguments to the **UserAmplitude** method.

56.13.4 Corresponding analysis keywords

*AMPLITUDE

57. Beam Section profile commands

The Beam Section profile commands are used to create profiles. A profile definition specifies the properties of a beam that are related to its cross-section geometry. When you define a beam section, you must include a reference to a profile in the section definition.

57.1 Profile object

The Profile object defines the geometrical properties of a beam cross-section. Profile is an abstract base type.

Access

`sectionApi.profiles()` [*name*]

57.1.1 Members

The Profile object has the following member:

Prototype

`odb_String name() const;`

name

An odb_String specifying the repository key.

57.2 ArbitraryProfile object

The ArbitraryProfile object defines the properties of an arbitrary profile.

The ArbitraryProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.2.1 ArbitraryProfile(...)

This method creates a ArbitraryProfile object.

Path

`sectionApi.ArbitraryProfile`

Prototype

```
odb_ArbitraryProfile&  
ArbitraryProfile(const odb_String& name,  
                 const odb_SequenceSequenceDouble& table);
```

Required arguments

name

An odb_String specifying the repository key.

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

The first sequence in the table specifies the following:

- 1-coordinate of the first point defining the profile.
- 2-coordinate of the first point defining the profile.

All other sequences in the table specify the following:

- 1-coordinate of the next point defining the profile.
- 2-coordinate of the next point defining the profile.
- The thickness of the segment ending at that point.

Return value

An ArbitraryProfile object.

Exceptions

RangeError.

57.2.2 Members

The ArbitraryProfile object has members with the same names and descriptions as the arguments to the `ArbitraryProfile` method.

57.2.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=ARBITRARY

57.3 BoxProfile object

The BoxProfile object defines the properties of a box profile.

The BoxProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.3.1 BoxProfile(...)

This method creates a BoxProfile object.

Path

`sectionApi.BoxProfile`

Prototype

```
odb_BoxProfile&
BoxProfile(const odb_String& name,
           double a,
           double b,
           bool uniformThickness,
           double t1,
           double t2,
           double t3,
           double t4);
```

Required arguments

name

An odb_String specifying the repository key.

a

A Double specifying the *a* dimension of the box profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

BEAM SECTION PROFILE COMMANDS

b

A Double specifying the *b* dimension of the box profile.

uniformThickness

A Boolean specifying whether the thickness is uniform.

t1

A Double specifying the uniform wall thickness if *uniformThickness*=true, and the wall thickness of the first segment if *uniformThickness*=false.

Optional arguments

t2

A Double specifying the wall thickness of the second segment. *t2* is required only when *uniformThickness*=false. The default value is 0.0.

t3

A Double specifying the wall thickness of the third segment. *t3* is required only when *uniformThickness*=false. The default value is 0.0.

t4

A Double specifying the wall thickness of the fourth segment. *t4* is required only when *uniformThickness*=false. The default value is 0.0.

Return value

A BoxProfile object.

Exceptions

RangeError.

57.3.2 Members

The BoxProfile object has members with the same names and descriptions as the arguments to the **BoxProfile** method.

57.3.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=BOX

57.4 CircularProfile object

The CircularProfile object defines the properties of a solid circular profile.

The CircularProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.4.1 CircularProfile(...)

This method creates a CircularProfile object.

Path

`sectionApi.CircularProfile`

Prototype

```
odb_CircularProfile&
CircularProfile(const odb_String& name,
               double r);
```

Required arguments

name

An odb_String specifying the repository key.

r

A positive Double specifying the *r* dimension (outer radius) of the circular profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

Optional arguments

None.

Return value

A CircularProfile object.

Exceptions

RangeError.

57.4.2 Members

The CircularProfile object has members with the same names and descriptions as the arguments to the `CircularProfile` method.

57.4.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=CIRC

57.5 GeneralizedProfile object

The GeneralizedProfile object defines the properties of a profile via its area, moment of inertia, etc.
The GeneralizedProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.5.1 GeneralizedProfile(...)

This method creates a GeneralizedProfile object.

Path

`sectionApi.GeneralizedProfile`

Prototype

```
odb_GeneralizedProfile&
GeneralizedProfile(const odb_String& name,
                   double area,
                   double i11,
                   double i12,
                   double i22,
                   double j,
                   double gamma0,
                   double gammaW);
```

Required arguments

name

An odb_String specifying the repository key.

area

A Double specifying the cross-sectional area for the profile.

*i11*A Double specifying the moment of inertia for bending about the 1-axis, I_{11} .*i12*A Double specifying the moment of inertia for cross bending, I_{12} .*i22*A Double specifying the moment of inertia for bending about the 2-axis, I_{22} .*j*A Double specifying the torsional constant, J .*gammaO*A Double specifying the sectorial moment, Γ_0 .*gammaW*A Double specifying the warping constant, Γ_W .**Optional arguments**

None.

Return value

A GeneralizedProfile object.

Exceptions

RangeError.

57.5.2 Members

The GeneralizedProfile object has members with the same names and descriptions as the arguments to the **GeneralizedProfile** method.

57.5.3 Corresponding analysis keywords

*BEAM GENERAL SECTION, SECTION=GENERAL or NONLINEAR GENERAL

57.6 HexagonalProfile object

The HexagonalProfile object defines the properties of a hexagonal profile.

The HexagonalProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.6.1 HexagonalProfile(...)

This method creates a HexagonalProfile object.

Path

`sectionApi.HexagonalProfile`

Prototype

```
odb_HexagonalProfile&
HexagonalProfile(const odb_String& name,
                 double r,
                 double t);
```

Required arguments

name

An odb_String specifying the repository key.

r

A positive Double specifying the *r* dimension (outer radius) of the hexagonal profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

t

A positive Double specifying the *t* dimension (wall thickness) of the hexagonal profile, $t < (\sqrt{3}/2)r$.

Optional arguments

None.

Return value

A HexagonalProfile object.

Exceptions

RangeError.

57.6.2 Members

The HexagonalProfile object has members with the same names and descriptions as the arguments to the `HexagonalProfile` method.

57.6.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=HEX

57.7 IProfile object

The IProfile object defines the properties of an I profile.

The IProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.7.1 IProfile(...)

This method creates an IProfile object.

Path

`sectionApi.IProfile`

Prototype

```
odb_IProfile&
IProfile(const odb_String& name,
         double l,
         double h,
         double b1,
         double b2,
         double t1,
         double t2,
         double t3);
```

Required arguments

name

An odb_String specifying the repository key.

l

A Double specifying the *l* dimension (offset of 1-axis from the bottom flange surface) of the I profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

h

A Double specifying the *h* dimension (height) of the I profile.

b1

A Double specifying the *b1* dimension (bottom flange width) of the I profile.

b2

A Double specifying the *b2* dimension (top flange width) of the I profile.

t1

A Double specifying the *t1* dimension (bottom flange thickness) of the I profile.

t2

A Double specifying the *t2* dimension (top flange thickness) of the I profile.

t3

A Double specifying the *t3* dimension (web thickness) of the I profile.

Optional arguments

None.

Return value

An IProfile object.

Exceptions

RangeError.

57.7.2 Members

The IProfile object has members with the same names and descriptions as the arguments to the **IProfile** method.

57.7.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=I

57.8 LProfile object

The LProfile object defines the properties of a L profile.

The LProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.8.1 LProfile(...)

This method creates a LProfile object.

Path

`sectionApi.LProfile`

Prototype

```
odb_LProfile&
LProfile(const odb_String& name,
         double a,
         double b,
         double t1,
         double t2);
```

Required arguments

name

An odb_String specifying the repository key.

a

A positive Double specifying the *a* dimension (flange length) of the L profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A positive Double specifying the *b* dimension (flange length) of the L profile.

t1

A positive Double specifying the *t1* dimension (flange thickness) of the L profile ($t1 < b$).

t2

A positive Double specifying the *t2* dimension (flange thickness) of the L profile ($t2 < a$).

Optional arguments

None.

Return value

A LProfile object.

Exceptions

RangeError.

57.8.2 Members

The LProfile object has members with the same names and descriptions as the arguments to the `LProfile` method.

57.8.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=L

57.9 PipeProfile object

The PipeProfile object defines the properties of a circular pipe profile.

The PipeProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.9.1 PipeProfile(...)

This method creates a PipeProfile object.

Path

`sectionApi.PipeProfile`

Prototype

```
odb_PipeProfile&  
PipeProfile(const odb_String& name,
```

```
double r,
double t,
const odb_String& formulation);
```

Required arguments*name*

An odb_String specifying the repository key.

r

A Double specifying the outer radius of the pipe. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

t

A Double specifying the wall thickness of the pipe.

Optional argument*formulation*

An odb_String specifying the formulation. Possible values are "THIN_WALL" and "THICK_WALL". The default value is "THIN_WALL".

Return value

A PipeProfile object.

Exceptions

RangeError.

57.9.2 Members

The PipeProfile object has members with the same names and descriptions as the arguments to the **PipeProfile** method.

57.9.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=PIPE

*BEAM SECTION, SECTION=THICK PIPE

57.10 RectangularProfile object

The RectangularProfile object defines the properties of a solid rectangular profile.

The RectangularProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.10.1 RectangularProfile(...)

This method creates a RectangularProfile object.

Path

`sectionApi.RectangularProfile`

Prototype

```
odb_RectangularProfile&
RectangularProfile(const odb_String& name,
                  double a,
                  double b);
```

Required arguments

name

An odb_String specifying the repository key.

a

A positive Double specifying the *a* dimension of the rectangular profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A positive Double specifying the *b* dimension of the rectangular profile.

Optional arguments

None.

Return value

A RectangularProfile object.

Exceptions

RangeError.

57.10.2 Members

The RectangularProfile object has members with the same names and descriptions as the arguments to the `RectangularProfile` method.

57.10.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=RECT

57.11 TProfile object

The TProfile object defines the properties of a T profile.

The TProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.11.1 TProfile(...)

This method creates a TProfile object.

Path

`sectionApi.TProfile`

Prototype

```
odb_TProfile&
TProfile(const odb_String& name,
         double b,
         double h,
         double l,
         double tf,
         double tw);
```

Required arguments

name

An odb_String specifying the repository key.

b

A positive Double specifying the *b* dimension (flange width) of the T profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

h

A positive Double specifying the *h* dimension (height) of the T profile.

l

A positive Double specifying the *l* dimension (offset of 1-axis from the edge of web) of the T profile.

tf

A positive Double specifying the *tf* dimension (flange thickness) of the T profile ($tf < h$).

tw

A positive Double specifying the *tw* dimension (web thickness) of the T profile ($tw < b$).

Optional arguments

None.

Return value

A TProfile object.

Exceptions

RangeError.

57.11.2 Members

The TProfile object has members with the same names and descriptions as the arguments to the **TProfile** method.

57.11.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=I

57.12 TrapezoidalProfile object

The TrapezoidalProfile object defines the properties of a trapezoidal profile.

The TrapezoidalProfile object is derived from the Profile object.

Access

`sectionApi.profiles()` [*name*]

57.12.1 TrapezoidalProfile(...)

This method creates a TrapezoidalProfile object.

Path

`sectionApi.TrapezoidalProfile`

Prototype

```
odb_TrapezoidalProfile&
TrapezoidalProfile(const odb_String& name,
                   double a,
                   double b,
                   double c,
                   double d);
```

Required arguments

name

An odb_String specifying the repository key.

a

A positive Double specifying the *a* dimension of the Trapezoidal profile. For more information, see “Beam cross-section library,” Section 29.3.9 of the Abaqus Analysis User’s Guide.

b

A positive Double specifying the *b* dimension of the Trapezoidal profile.

c

A positive Double specifying the *c* dimension of the Trapezoidal profile.

d

A Double specifying the *d* dimension of the Trapezoidal profile.

Optional arguments

None.

Return value

A TrapezoidalProfile object.

Exceptions

RangeError.

57.12.2 Members

The TrapezoidalProfile object has members with the same names and descriptions as the arguments to the **TrapezoidalProfile** method.

57.12.3 Corresponding analysis keywords

*BEAM SECTION, SECTION=TRAPEZOID

58. Connector commands

A connector describes the relative motions between two points. A connector also describes the behavior associated with the relative motion.

58.1 ConnectorBehaviorOption object

The ConnectorBehaviorOption object is the abstract base type for other ConnectorBehaviorOption objects. The ConnectorBehaviorOption object has no explicit constructor. The members of the ConnectorBehaviorOption object are common to all objects derived from the ConnectorBehaviorOption.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.1.1 Members

The ConnectorBehaviorOption object has no members.

58.2 CDCTerm object

The CDCTerm object is used to create contributing terms for a DerivedComponent object.

Access

```
sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.derivedComponent().cdcTerms(i)
sectionApi.sections() [name].behaviorOptions(i).derivedComponent() \
.cdcTerms(i)
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.derivedComponent().cdcTerms(i)
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.derivedComponent().cdcTerms(i)
```

58.2.1 CDCTerm(...)

This method creates a CDCTerm object.

Path

```

sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.derivedComponent().CDCTerm
sectionApi.sections() [name].behaviorOptions(i).derivedComponent() \
.CDCTerm
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.derivedComponent().CDCTerm
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.derivedComponent().CDCTerm

```

Prototype

```

odb_CDCTerm&
CDCTerm(const odb_SequenceInt& intrinsicComponents,
         const odb_SequenceSequenceDouble& table,
         const odb_String& termOperator,
         const odb_String& termSign,
         bool localDependency,
         const odb_String& indepCompType,
         const odb_SequenceInt& indepComponents,
         bool tempDependency,
         int fieldDependencies);

```

Required arguments*intrinsicComponents*

An odb_SequenceInt specifying the components of relative motion for which the contributing term is defined. Possible values are $1 \leq \text{intrinsicComponents} \leq 6$. Only available components can be specified if the DerivedComponent object is being referenced by a Potential object. This is not the case if the DerivedComponent object is referenced by a ConnectorFriction object directly. The default value is an empty sequence.

table

An odb_SequenceSequenceDouble specifying components numbers and temperature and field values. Each sequence of the table data specifies:

- The first intrinsic component number.
- If applicable, the second intrinsic component number.
- Etc.
- If applicable, the first independent component number.
- If applicable, the second independent component number.
- Etc.

- If applicable, the temperature value.
- If applicable, the value of the first field variable.
- If applicable, the value of the second field variable.
- Etc.

The default value is an empty sequence.

Optional arguments

termOperator

An odb_String specifying the method for combining contributing terms: square root of a sum of the squares, direct sum, or Macauley sum. Possible values are "RSS", "SUM", and "MACAULEY". The default value is "RSS".

termSign

An odb_String specifying the overall sign for the contributing term. Possible values are "POSITIVE" and "NEGATIVE". The default value is "POSITIVE".

localDependency

A Boolean specifying whether the table data depend on either components of relative position or components of constitutive relative motion. The default value is false.

indepCompType

An odb_String specifying whether localDependency refers to components of relative position or components of constitutive relative motion. Possible values are "POSITION" and "MOTION". The default value is "POSITION".

The *indepCompType* argument applies only if *localDependency*=true.

indepComponents

An odb_SequenceInt specifying the independent components included in the derived component definition. Possible values are $1 \leq indepComponents \leq 6$. Only available components can be specified. The *indepComponents* argument applies only if *localDependency*=true. The default value is an empty sequence.

tempDependency

A Boolean specifying whether the table data depend on temperature. The default value is false.

fieldDependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Return value

A CDCTerm object.

Exceptions

ValueError and TextError.

58.2.2 **setValues(...)**

This method modifies the `CDCTerm` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `CDCTerm` method.

Return value

None

Exceptions

`ValueError`.

58.2.3 **Members**

The `CDCTerm` object has members with the same names and descriptions as the arguments to the `CDCTerm` method.

In addition, the `CDCTerm` object can have the following member:

Prototype

```
odb_ConnectorOptions options() const;
```

options

A `ConnectorOptions` object specifying the `ConnectorOptions` used to define tabular options for this `ConnectorBehaviorOption`.

58.3 **ConnectorDamage object**

The `ConnectorDamage` object defines damage behavior for one or more components of a connector's relative motion.

The `ConnectorDamage` object is derived from the `ConnectorBehaviorOption` object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```


58.3.1 ConnectorDamage(...)

This method creates a connector damage behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorDamage
```

Prototype

```
odb_ConnectorDamage&
ConnectorDamage(const odb_String& coupling,
    const odb_String& criterion,
    bool initiationTemperature,
    const odb_String& initiationPotentialOperator,
    double initiationPotentialExponent,
    int initiationDependencies,
    bool evolution,
    const odb_String& evolutionType,
    const odb_String& softening,
    bool useAffected,
    const odb_String& degradation,
    bool evolutionTemperature,
    int evolutionDependencies,
    const odb_String& evolutionPotentialOperator,
    double evolutionPotentialExponent,
    const odb_SequenceConnectorPotential& initiationPotentials,
    const odb_SequenceConnectorPotential& evolutionPotentials,
    const odb_SequenceSequenceDouble& initiationTable,
    const odb_SequenceSequenceDouble& evolutionTable,
    const odb_SequenceInt& affectedComponents,
    const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments

coupling

An odb_String specifying whether or not the behavior is coupled. Possible values are "UNCOUPLED" and "COUPLED". The default value is "UNCOUPLED".

CONNECTOR COMMANDS

criterion

An odb_String specifying the damage initiation criterion to be used. Possible values are "FORCE", "MOTION", and "PLASTIC_MOTION". The default value is "FORCE".

initiationTemperature

A Boolean specifying whether the initiation data depend on temperature. The default value is false.

initiationPotentialOperator

An odb_String specifying the contribution operator for the initiation potential contributions. Possible values are "SUM" and "MAXIMUM". The default value is "SUM".

This argument is only if *coupling*="COUPLED" and if *criterion*="FORCE" or "MOTION".

initiationPotentialExponent

A Double specifying the number equal to the inverse of the overall exponent in the initiation potential definition. The default value is 2.0.

This argument is applicable only if *coupling*="COUPLED", when *initiationPotentialOperator*="SUM", and when *criterion*="FORCE" or "MOTION".

initiationDependencies

An Int specifying the number of field variable dependencies for the initiation data. The default value is 0.

evolution

A Boolean specifying whether damage evolution data will be used. The default value is true.

evolutionType

An odb_String specifying the type of damage evolution to be specified. Possible values are "MOTION_TYPE" and "ENERGY_TYPE". The default value is "MOTION_TYPE".

This argument is applicable only if *evolution*=true.

softening

An odb_String specifying the damage evolution law to be specified. Possible values are "LINEAR", "EXPONENTIAL", and "TABULAR". The default value is "LINEAR".

This argument is applicable only if *evolution*=true and when *evolutionType*="MOTION_TYPE".

useAffected

A Boolean specifying whether or not *affectedComponents* will be specified. If *useAffected*=false, then only the components of relative motion specified by *components* will undergo damage. The default value is false.

This argument is applicable only if *evolution*=true.

degradation

An odb_String specifying the contribution of each damage mechanism when more than one damage mechanism is defined. Possible values are "MAXIMUM" and "MULTIPLICATIVE". The default value is "MAXIMUM".

This argument is applicable if *evolution*=true.

evolutionTemperature

A Boolean specifying whether the evolution data depend on temperature. The default value is false.

This argument is applicable only if *evolution*=true.

evolutionDependencies

An Int specifying the number of field variable dependencies for the evolution data. The default value is 0.

This argument is applicable only if *evolution*=true.

evolutionPotentialOperator

An odb_String specifying the contribution operator for the evolution potential contributions. Possible values are "SUM" and "MAXIMUM". The default value is "SUM".

This argument is applicable only if *coupling*="COUPLED", when *evolution*=true, when *evolutionType*="MOTION_TYPE", and when *criterion*="FORCE" or "MOTION".

evolutionPotentialExponent

A Double specifying the number equal to the inverse of the overall exponent in the evolution potential definition. The default value is 2.0.

This argument is applicable only if *coupling*="COUPLED", when *evolution*=true, when *evolutionPotentialOperator*="SUM", when *evolutionType*="MOTION", and when *criterion*="FORCE" or "MOTION".

initiationPotentials

A sequence of ConnectorPotential objects specifying one ConnectorPotential object for each initiation potential contribution. This member can be specified only if *coupling*="COUPLED" and if *criterion*="FORCE" or "MOTION".

evolutionPotentials

A sequence of ConnectorPotential objects specifying one ConnectorPotential object for each evolution potential contribution). This member can be specified only if *coupling*="COUPLED", if *evolution*=true, if *evolutionType*="MOTION", and if *criterion*="FORCE" or "MOTION".

initiationTable

An odb_SequenceSequenceDouble specifying the initiation properties. The default value is an empty sequence.

Items in the *initiationTable* data are described below.

evolutionTable

An odb_SequenceSequenceDouble specifying the evolution properties. The default value is an empty sequence.

Items in the *evolutionTable* data are described below. This argument is only applicable if *evolution*=true.

affectedComponents

An odb_SequenceInt specifying the components of relative motion that will be damaged. Possible values are $1 \leq components \leq 6$. Only available components can be specified. This argument is applicable only if *evolution*=true and *useAffected*=true. The default value is an empty sequence.

CONNECTOR COMMANDS

components

An `odb_SequenceInt` specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. This argument can be specified only if *coupling*="UNCOUPLED". The default value is an empty sequence.

Table data

Table data for *initiationTable*:

If *criterion*=FORCE, then each sequence of the table data specifies the following:

- Lower (compression) limiting force or moment. Use -1.0E+36 to indicate an unspecified lower limit.
- Upper (tension) limiting force or moment. Use 1.0E+36 to indicate an unspecified upper limit. At least one limit, lower or upper, must be specified.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *criterion*=MOTION, then each sequence of the table data specifies the following:

- Lower (compression) limiting connector constitutive relative displacement or rotation. Use -1.0E+36 to indicate an unspecified lower limit.
- Upper (tension) limiting connector constitutive relative displacement or rotation. Use 1.0E+36 to indicate an unspecified upper limit. At least one limit, lower or upper, must be specified.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *criterion*=PLASTIC_MOTION, then each sequence of the table data specifies the following:

- Relative equivalent plastic displacement/rotation at which damage will be initiated.
- Mode-mix ratio (only if *coupling*=COUPLED).
- Relative equivalent plastic displacement/rotation rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Table data for *evolutionTable*:

If *evolutionType*=MOTION and *softening*=LINEAR, then each sequence of the table data specifies the following:

- Post-initiation equivalent relative plastic motion at ultimate failure if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation) at ultimate failure.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=MOTION and *softening*=EXPONENTIAL, then each sequence of the table data specifies the following:

- Post-initiation equivalent relative plastic motion at ultimate failure if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation) at ultimate failure.
- Exponential law parameter.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=MOTION and *softening*=TABULAR, then each sequence of the table data specifies the following:

- Damage variable (cannot be less than 0 or greater than 1).
- Post-initiation equivalent relative plastic motion if *criterion*=PLASTIC_MOTION. Otherwise, post-initiation constitutive relative motion (displacement/rotation).
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *evolutionType*=ENERGY, then each sequence of the table data specifies the following:

- Total energy dissipated by damage at ultimate failure.
- Mode-mix ratio (only if *coupling*=COUPLED and *criterion*=PLASTIC_MOTION).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `ConnectorDamage` object.

Exceptions

`ValueError` and `TextError`.

58.3.2 `setValues(...)`

This method modifies the `ConnectorDamage` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ConnectorDamage` method.

Return value

`None`

Exceptions

`ValueError`.

58.3.3 Members

The `ConnectorDamage` object has members with the same names and descriptions as the arguments to the `ConnectorDamage` method.

In addition, the `ConnectorDamage` object can have the following members:

Prototype

```
odb_ConnectorOptions initiationOptions() const;  
odb_ConnectorOptions evolutionOptions() const;
```

initiationOptions

A `ConnectorOptions` object specifying the `ConnectorOptions` used to define tabular options for the damage initiation table.

evolutionOptions

A `ConnectorOptions` object specifying the `ConnectorOptions` used to define tabular options for the damage evolution table.

58.3.4 Corresponding analysis keywords

*CONNECTOR DAMAGE INITIATION, *CONNECTOR DAMAGE EVOLUTION,
*CONNECTOR POTENTIAL

58.4 ConnectorDamping object

The ConnectorDamping object defines damping behavior for one or more components of a connector's relative motion.

The ConnectorDamping object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.4.1 ConnectorDamping(...)

This method creates a connector damping behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorDamping
```

Prototype

```
odb_ConnectorDamping&  
ConnectorDamping(const odb_String& behavior,  
                 const odb_String& coupling,  
                 int dependencies,  
                 bool temperatureDependency,  
                 bool frequencyDependency,  
                 const odb_SequenceSequenceDouble& table,  
                 const odb_SequenceInt& independentComponents,  
                 const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments*behavior*

An odb_String specifying if the damping behavior is linear or nonlinear. Possible values are "LINEAR" and "NONLINEAR". The default value is "LINEAR".

coupling

An odb_String specifying whether the damping behavior is coupled between the connector's components of relative motion. If *behavior*="LINEAR", then possible values are "UNCOUPLED" and "COUPLED". If *behavior*="NONLINEAR", then possible values are "UNCOUPLED", "COUPLED_POSITION", and "COUPLED_MOTION". Possible values are "UNCOUPLED", "COUPLED", "COUPLED_POSITION", and "COUPLED_MOTION". The default value is "UNCOUPLED".

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

temperatureDependency

A Boolean specifying whether the behavior data depend on temperature. The default value is false.

frequencyDependency

A Boolean specifying whether the behavior data depend on frequency. This value is applicable only if *behavior*= "LINEAR" and *coupling*="UNCOUPLED". The default value is false.

table

An odb_SequenceSequenceDouble specifying damping properties. Items in the table data are described below. The default value is an empty sequence.

independentComponents

An odb_SequenceInt specifying the list of independent components that are included in the definition of the connector damping data. This argument is applicable only if *behavior*="NONLINEAR" and *coupling*="COUPLED_POSITION" or "COUPLED_MOTION". When this argument is applicable, at least one value must be specified. Only available components can be specified. The default value is an empty sequence.

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Table data

If *behavior*=LINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Damping coefficient (force or moment per relative velocity).
- Frequency (cycles/time), if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

- Etc.

If *behavior*=NONLINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Force or moment.
- Relative displacement or rotation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=LINEAR and *coupling*=COUPLED, the table data specify the symmetric portion of the damping matrix for the specified components followed by any temperature data and then any field data. For example, if components 2, 3, and 5 are specified, the table portion of the command is as follows:

table=((C₂₂, C₂₃, C₂₅, C₃₃, C₃₅, C₅₅),)

The following items should then be specified as comma-separated data:

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION, each sequence of the table data specifies the following:

- Force or moment for the directions in the *components* list.
- Relative velocity for the directions in the *components* list.
- Relative position or angle for the first *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the first *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Relative position or angle for the second *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the second *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Etc. up to the Nth *independentComponents* direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorDamping object.

Exceptions

ValueError and TextError.

58.4.2 **setValues(...)**

This method modifies the ConnectorDamping object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorDamping** method.

Return value

None

Exceptions

ValueError.

58.4.3 **Members**

The ConnectorDamping object has members with the same names and descriptions as the arguments to the **ConnectorDamping** method.

In addition, the ConnectorDamping object can have the following member:

Prototype

```
odb_ConnectorOptions options() const;
```

options

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for this ConnectorBehaviorOption .

58.4.4 **Corresponding analysis keywords**

*CONNECTOR DAMPING

58.5 ConnectorElasticity object

The ConnectorElasticity object defines elastic behavior for one or more components of a connector's relative motion.

The ConnectorElasticity object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.5.1 ConnectorElasticity(...)

This method creates a connector elasticity behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorElasticity
```

Prototype

```
odb_ConnectorElasticity&
ConnectorElasticity(const odb_String& behavior,
                    const odb_String& coupling,
                    int dependencies,
                    bool temperatureDependency,
                    bool frequencyDependency,
                    const odb_SequenceSequenceDouble& table,
                    const odb_SequenceInt& independentComponents,
                    const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments

behavior

An odb_String specifying whether the elastic behavior is linear, nonlinear, or rigid. Possible values are "LINEAR", "NONLINEAR", and "RIGID". The default value is "LINEAR".

CONNECTOR COMMANDS

coupling

An odb_String specifying whether the elastic behavior is coupled between the connector's components of relative motion. If *behavior*="LINEAR", then possible values are "UNCOUPLED" and "COUPLED". If *behavior*="NONLINEAR", then possible values are "UNCOUPLED", "COUPLED_POSITION", and "COUPLED_MOTION". Possible values are "UNCOUPLED", "COUPLED", "COUPLED_POSITION", and "COUPLED_MOTION". The default value is "UNCOUPLED".

This argument is not applicable if *behavior*="RIGID".

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

This argument is not applicable if *behavior*="RIGID".

temperatureDependency

A Boolean specifying whether the behavior data depend on temperature. The default value is false.

This argument is not applicable if *behavior*="RIGID".

frequencyDependency

A Boolean specifying whether the behavior data depend on frequency. This value is applicable only if *behavior*="LINEAR" and *coupling*="UNCOUPLED". The default value is false.

This argument is not applicable if *behavior*="RIGID".

table

An odb_SequenceSequenceDouble specifying elasticity properties. Items in the table data are described below. This argument is not applicable if *behavior*="RIGID". The default value is an empty sequence.

independentComponents

An odb_SequenceInt specifying the list of independent components that are included in the definition of the connector elasticity data. This argument is applicable only if *behavior*="NONLINEAR" and *coupling*="COUPLED_POSITION" or "COUPLED_MOTION". If this argument is applicable, at least one value must be specified. Only available components can be specified. The default value is an empty sequence.

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Table data

If *behavior*=LINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Elastic stiffness (per each specified force/moment component).
- Frequency (cycles/time), if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=UNCOUPLED, then each sequence of the table data specifies the following:

- Force or moment.
- Relative displacement or rotation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=LINEAR and *coupling*=COUPLED, the table data specify the symmetric portion of the elastic stiffness matrix for the specified components followed by any temperature data and then any field data. For example, if components 2, 3, and 5 are specified, the table portion of the command is as follows:

```
table=( (D22, D23, D25, D33, D35, D55),
)
```

The following items should then be specified as comma-separated data:

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *behavior*=NONLINEAR and *coupling*=COUPLED_POSITION or COUPLED_MOTION, each sequence of the table data specifies the following:

- Force or moment for the directions in the *components* list.
- Relative position or angle for the first *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the first *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Relative position or angle for the second *independentComponents* direction if *coupling*=COUPLED_POSITION. Relative displacement or rotation for the second *independentComponents* direction if *coupling*=COUPLED_MOTION.
- Etc. up to the Nth *independentComponents* direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorElasticity object.

Exceptions

ValueError and TextError.

58.5.2 **setValues(...)**

This method modifies the ConnectorElasticity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorElasticity** method.

Return value

None

Exceptions

ValueError.

58.5.3 **Members**

The ConnectorElasticity object has members with the same names and descriptions as the arguments to the **ConnectorElasticity** method.

In addition, the ConnectorElasticity object can have the following member:

Prototype

```
odb_ConnectorOptions options() const;
```

options

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for this ConnectorBehaviorOption .

58.5.4 **Corresponding analysis keywords**

*CONNECTOR ELASTICITY

58.6 ConnectorFailure object

The ConnectorFailure object defines failure criteria for one or more components of a connector's relative motion.

The ConnectorFailure object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.6.1 ConnectorFailure(...)

This method creates a connector failure behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorFailure
```

Prototype

```
odb_ConnectorFailure&
ConnectorFailure(odb_Union releaseComponent,
                 odb_Union minMotion,
                 odb_Union maxMotion,
                 odb_Union minForce,
                 odb_Union maxForce,
                 const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments

releaseComponent

The string "ALL" or an Int specifying the motion components that fail. If an Int is specified, only that motion component fails when the failure criteria are satisfied. If *releaseComponent*="ALL", all motion components fail. The default value is "ALL".

CONNECTOR COMMANDS

minMotion

The string "NONE" or a Double specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is "NONE".

maxMotion

The string "NONE" or a Double specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is "NONE".

minForce

The string "NONE" or a Double specifying the lower bound of the force or moment in the directions of the specified components at which locking occurs, or no lower bound. The default value is "NONE".

maxForce

The string "NONE" or a Double specifying the upper bound of the force or moment in the directions of the specified components at which locking occurs, or no upper bound. The default value is "NONE".

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq components \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorFailure object.

Exceptions

ValueError and TextError.

58.6.2 setValues(...)

This method modifies the ConnectorFailure object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorFailure** method.

Return value

None

Exceptions

ValueError.

58.6.3 Members

The ConnectorFailure object has members with the same names and descriptions as the arguments to the `ConnectorFailure` method.

58.6.4 Corresponding analysis keywords

*CONNECTOR FAILURE

58.7 ConnectorFriction object

The ConnectorFriction object defines Coulomb-like or hysteretic friction behavior for one or more components of a connector's relative motion.

The ConnectorFriction object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.7.1 ConnectorFriction(...)

This method creates a connector friction behavior option for a ConnectorSection object. Depending upon the arguments provided, the friction behavior can be Coulomb-like or hysteretic in nature.

Path

```
sectionApi.sections() [name].ConnectorFriction
```

Prototype

```
odb_ConnectorFriction&
ConnectorFriction(const odb_String& frictionModel,
    const odb_String& slipStyle,
    odb_Union tangentDirection,
    odb_Union stickStiffness,
    const odb_String& componentType,
```

CONNECTOR COMMANDS

```
bool slipDependency,  
bool temperatureDependency,  
int dependencies,  
bool useContactForceComponent,  
const odb_String& contactForceStyle,  
int contactForceComponent,  
const odb_String& forcePotentialOperator,  
double forcePotentialExponent,  
const odb_SequenceConnectorPotential& connectorPotentials,  
const odb_SequenceSequenceDouble& table,  
const odb_SequenceInt& independentComponents);
```

Required arguments

None.

Optional arguments

frictionModel

An odb_String specifying the desired frictional response model. Possible values are "PREDEFINED" and "USER_CUSTOMIZED". The default value is "PREDEFINED".

slipStyle

An odb_String specifying the method of indicating the slip direction: either specified or computed based upon the force potential data. Possible values are "SPECIFY" and "COMPUTE". The default value is "SPECIFY".

This argument is applicable only if *frictionModel*="USER_CUSTOMIZED".

tangentDirection

The string "NONE" or an Int specifying the direction for which the frictional behavior is specified. Possible values are $1 \leq \textit{tangentDirection} \leq 6$, indicating an available component of relative motion. This argument applies only if *frictionModel*="USER_CUSTOMIZED" and if *slipStyle*="SPECIFY". The default value is "NONE".

stickStiffness

The string "NONE" or a Double specifying the stick stiffness associated with the frictional behavior in the direction specified by *tangentDirection*. If this argument is omitted, Abaqus computes an appropriate number for the stick stiffness. The default value is "NONE".

componentType

An odb_String specifying the type of the *independentComponents*. Possible values are "POSITION", "MOTION", and "NO_INDEPENDENT_COMPONENTS". The default value is "NO_INDEPENDENT_COMPONENTS".

slipDependency

A Boolean specifying whether the table data depend on accumulated slip. The default value is false.

This argument applies only if *frictionModel*="USER_CUSTOMIZED".

temperatureDependency

A Boolean specifying whether the table data depend on temperature. The default value is false.

This argument applies only if *frictionModel*="USER_CUSTOMIZED".

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

This argument applies only if *frictionModel*="USER_CUSTOMIZED".

useContactForceComponent

A Boolean specifying whether the contact force component will be defined. The default value is false.

This argument applies only if *frictionModel*="USER_CUSTOMIZED".

contactForceStyle

An odb_String specifying the method of indicating the contact force component direction: either specified or computed based on upon a DerivedComponent. Possible values are "COMPONENT_NUMBER" and "DERIVED_COMPONENT". The default value is "COMPONENT_NUMBER".

This argument is applicable only if *frictionModel*="USER_CUSTOMIZED" and if *useContactForceComponent*=true.

contactForceComponent

An Int specifying the contact force component direction. This argument applies only if *frictionModel*="USER_CUSTOMIZED", if *useContactForceComponent*=true, and if *contactForceStyle*="COMPONENT_NUMBER". The default value is 0.

forcePotentialOperator

An odb_String specifying the contribution operator for the force potential contributions. Possible values are "SUM" and "MAXIMUM". The default value is "SUM".

This argument is applicable only if *frictionModel*="USER_CUSTOMIZED" and if *slipStyle*="COMPUTE".

forcePotentialExponent

A Double specifying the number equal to the inverse of the overall exponent in the force potential definition. The default value is 2.0.

This argument is applicable only if *frictionModel*="USER_CUSTOMIZED", if *slipStyle*="COMPUTE", and if *forcePotentialOperator*="SUM".

connectorPotentials

A sequence of ConnectorPotential objects specifying one ConnectorPotential object for each force potential contribution. This member can be specified only if *frictionModel*="USER_CUSTOMIZED", and if *slipStyle*="COMPUTE".

table

An odb_SequenceSequenceDouble specifying friction properties. The default value is an empty sequence.

If *frictionModel*="PREDEFINED", each sequence of the table data specifies:

- If applicable, the first geometric scaling constant relevant to frictional interactions.

CONNECTOR COMMANDS

- Etc., up to as many geometric scaling constants as are associated with this connection type.
- Internal contact force/moment generating friction in the first predefined slip direction.
- If applicable, internal contact force/moment generating friction in the second predefined slip direction.
- Connector constitutive relative motion in the direction specified by *independentComponent*.
- Accumulated slip in the first predefined slip direction, if the data depend on accumulated slip.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *frictionModel*="USER_CUSTOMIZED", each sequence of the table data specifies:

- Effective radius of the cylindrical or spherical surface over which frictional slip occurs in the connector associated with frictional effects in the direction specified by *tangentDirection*. This radius is relevant only if the connection type includes an available rotational component of relative motion and *tangentDirection*="SLIP_DIRECTION".
- Internal contact force/moment generating friction in the direction specified by *tangentDirection*.
- Connector constitutive relative motion in the direction specified by *independentComponent*.
- Accumulated slip in the direction specified by *tangentDirection*, if the data depend on accumulated slip.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

independentComponents

An `odb_SequenceInt` specifying the independent components. Possible values are $1 \leq \text{independentComponents} \leq 6$. In addition, each independent component value must be unique. The *independentComponents* argument applies only if *frictionModel*="USER_CUSTOMIZED". Only available components can be specified. The default value is an empty sequence.

Return value

A `ConnectorFriction` object.

Exceptions

`ValueError` and `TextError`.

58.7.2 **setValues(...)**

This method modifies the `ConnectorFriction` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorFriction** method.

Return value

None

Exceptions

`ValueError`.

58.7.3 **Members**

The `ConnectorFriction` object has members with the same names and descriptions as the arguments to the **ConnectorFriction** method.

In addition, the `ConnectorFriction` object can have the following members:

Prototype

```
odb_TangentialBehavior tangentialBehavior() const;
odb_DerivedComponent derivedComponent() const;
odb_ConnectorOptions options() const;
```

tangentialBehavior

A `TangentialBehavior` object.

derivedComponent

A `DerivedComponent` object specifying the `DerivedComponent` used to compute the contact force component direction. This argument applies only if *frictionModel*="USER_CUSTOMIZED", if *useContactForceComponent*=true, and if *contactForceStyle*="DERIVED_COMPONENT".

options

A `ConnectorOptions` object specifying the `ConnectorOptions` used to define tabular options for this `ConnectorBehaviorOption`.

58.7.4 Corresponding analysis keywords

*CONNECTOR FRICTION, *FRICTION, *CONNECTOR POTENTIAL, *CONNECTOR DERIVED COMPONENT

58.8 ConnectorLock object

The ConnectorLock object defines locking criteria for one or more available components of a connector's relative motion.

The ConnectorLock object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.8.1 ConnectorLock(...)

This method creates a connector lock behavior option for a ConnectorSection .

Path

```
sectionApi.sections() [name].ConnectorLock
```

Prototype

```
odb_ConnectorLock&  
ConnectorLock(odb_Union lockingComponent,  
              odb_Union minMotion,  
              odb_Union maxMotion,  
              odb_Union minForce,  
              odb_Union maxForce,  
              const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments*lockingComponent*

The string "ALL" or an Int specifying the motion components that are locked. If an Int is specified, only that motion component is locked when the locking criteria are satisfied. If *lockingComponent*=ALL, all motion components are locked. The default value is "ALL".

minMotion

The string "NONE" or a Double specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is "NONE".

maxMotion

The string "NONE" or a Double specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is "NONE".

minForce

The string "NONE" or a Double specifying the lower bound of the force or moment in the directions of the specified components at which locking occurs, or no lower bound. The default value is "NONE".

maxForce

The string "NONE" or a Double specifying the upper bound of the force or moment in the directions of the specified components at which locking occurs, or no upper bound. The default value is "NONE".

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq \text{components} \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorLock object.

Exceptions

ValueError and TextError.

58.8.2 setValues(...)

This method modifies the ConnectorLock object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorLock** method.

Return value

None

Exceptions

ValueError.

58.8.3 Members

The ConnectorLock object has members with the same names and descriptions as the arguments to the `ConnectorLock` method.

58.8.4 Corresponding analysis keywords

*CONNECTOR LOCK

58.9 ConnectorOptions object

The ConnectorOptions object is used to define various options for connector behaviors. It can be used only in conjunction with CDCTerm , ConnectorDamage , ConnectorDamping , ConnectorElasticity , ConnectorFriction , and ConnectorPlasticity objects. Because the ConnectorDamage object contains two separate ConnectorOptions repositories (one for damage initiation and one for damage evolution), there are two ConnectorOptions constructors associated with that behavior—initiationOptions and evolutionOptions. The ConnectorPlasticity object also contains two separate ConnectorOptions repositories (one for isotropic hardening and one for kinematic hardening), so there are two ConnectorOptions constructors associated with that behavior—`isotropicOptions` and `kinematicOptions`.

Access

```

sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.derivedComponent().cdcTerms(i).options()
sectionApi.sections() [name].behaviorOptions(i).derivedComponent() \
.cdcTerms(i).options()
sectionApi.sections() [name].behaviorOptions(i).evolutionOptions()
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.derivedComponent().cdcTerms(i).options()
sectionApi.sections() [name].behaviorOptions(i).initiationOptions()
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.derivedComponent().cdcTerms(i).options()
sectionApi.sections() [name].behaviorOptions(i).isotropicOptions()
sectionApi.sections() [name].behaviorOptions(i).kinematicOptions()

```



```
sectionApi.sections() [name].behaviorOptions(i).options()
```

58.9.1 ConnectorOptions(...)

This method creates a connector options object to be used in conjunction with an allowable connector behavior option, derived component term, or connector section.

Path

```
sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.derivedComponent().cdcTerms(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).derivedComponent() \
.cdcTerms(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.derivedComponent().cdcTerms(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.derivedComponent().cdcTerms(i).ConnectorOptions
sectionApi.sections() [name].behaviorOptions(i).ConnectorOptions
```

Prototype

```
odb_ConnectorOptions&
ConnectorOptions(bool useBehRegSettings,
                 bool regularize,
                 bool defaultTolerance,
                 double regularization,
                 bool defaultRateFactor,
                 double rateFactor,
                 const odb_String& interpolation,
                 bool useBehExtSettings,
                 const odb_String& extrapolation);
```

Required arguments

None.

Optional arguments

useBehRegSettings

A Boolean specifying whether or not to use the behavior-level settings for regularization options. This argument is applicable only for an Abaqus/Explicit analysis. The default value is true.

CONNECTOR COMMANDS

regularize

A Boolean specifying whether or not the tabular data will be regularized. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=false. The default value is true.

defaultTolerance

A Boolean specifying whether or not the analysis default regularization tolerance will be used. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=false and *regularize*=true. The default value is true.

regularization

A Double specifying the regularization increment to be used. This argument is applicable only for an Abaqus/Explicit analysis and only if *useBehRegSettings*=false, *regularize*=true, and *defaultTolerance*=false. The default value is 0.03.

defaultRateFactor

A Boolean specifying whether or not the analysis default rate filter factor will be used. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. The default value is true.

rateFactor

A Double specifying the rate filter factor to be used. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. This argument is also applicable only if *defaultRateFactor*=false. The default value is 0.9.

interpolation

An odb_String specifying the type of interpolation increment to be used on rate-dependent tabular data. This argument is applicable only for an Abaqus/Explicit analysis that includes isotropic hardening with tabular definition or damage initiation with plastic motion criteria. Possible values are "LINEAR" and "LOGARITHMIC". The default value is "LINEAR".

useBehExtSettings

A Boolean specifying whether or not to use the behavior-level settings for extrapolation options. The default value is true.

extrapolation

An odb_String specifying the extrapolation technique to be used. This argument is applicable only if *useBehExtSettings*=false. Possible values are "CONSTANT" and "LINEAR". The default value is "CONSTANT".

Return value

A ConnectorOptions object.

Exceptions

ValueError and TextError.

58.9.2 **setValues(...)**

This method modifies the `ConnectorOptions` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorOptions** method.

Return value

None

Exceptions

`ValueError`.

58.9.3 **Members**

The `ConnectorOptions` object has members with the same names and descriptions as the arguments to the **ConnectorOptions** method.

58.9.4 **Corresponding analysis keywords**

*CONNECTOR BEHAVIOR, *CONNECTOR DAMAGE INITIATION, *CONNECTOR DAMAGE EVOLUTION, *CONNECTOR DAMPING, *CONNECTOR DERIVED COMPONENT, *CONNECTOR ELASTICITY, *CONNECTOR FRICTION, and *CONNECTOR PLASTICITY

58.10 **ConnectorPlasticity object**

The `ConnectorPlasticity` object defines plastic behavior for one or more components of a connector's relative motion.

The `ConnectorPlasticity` object is derived from the `ConnectorBehaviorOption` object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.10.1 ConnectorPlasticity(...)

This method creates a connector plasticity behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorPlasticity
```

Prototype

```
odb_ConnectorPlasticity&
ConnectorPlasticity(const odb_String& coupling,
    bool isotropic,
    const odb_String& isotropicType,
    bool isotropicTemperature,
    int isotropicDependencies,
    bool kinematic,
    const odb_String& kinematicType,
    bool kinematicTemperature,
    int kinematicDependencies,
    const odb_String& forcePotentialOperator,
    double forcePotentialExponent,
    const odb_SequenceConnectorPotential& connectorPotentials,
    const odb_SequenceSequenceDouble& isotropicTable,
    const odb_SequenceSequenceDouble& kinematicTable,
    const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments*coupling*

An odb_String specifying whether or not the behavior is coupled. Possible values are "UNCOUPLED" and "COUPLED". The default value is "UNCOUPLED".

isotropic

A Boolean specifying whether isotropic hardening data will be used. The default value is true. If *isotropic*=false, then *kinematic* must be specified as true.

isotropicType

An odb_String specifying the type of isotropic hardening to be specified. Possible values are "TABULAR" and "EXPONENTIAL_LAW". The default value is "TABULAR".

This argument is applicable only if *isotropic*=true.

isotropicTemperature

A Boolean specifying whether the isotropic data depend on temperature. The default value is false.

This argument is applicable only if *isotropic*=true.

isotropicDependencies

An Int specifying the number of field variable dependencies for the isotropic data. The default value is 0.

This argument is applicable only if *isotropic*=true.

kinematic

A Boolean specifying whether kinematic hardening data will be used. The default value is false.

If *kinematic*=false, then *isotropic* must be specified as true.

kinematicType

An odb_String specifying the type of kinematic hardening to be specified. Possible values are "HALF_CYCLE", "STABILIZED", and "PARAMETERS". The default value is "HALF_CYCLE".

This argument is applicable only if *kinematic*=true.

kinematicTemperature

A Boolean specifying whether the kinematic data depend on temperature. The default value is false.

This argument is applicable only if *kinematic*=true.

kinematicDependencies

An Int specifying the number of field variable dependencies for the kinematic data. The default value is 0.

This argument is applicable only if *kinematic*=true.

forcePotentialOperator

An odb_String specifying the contribution operator for the force potential contributions. Possible values are "SUM" and "MAXIMUM". The default value is "SUM".

This argument is applicable only if *coupling*="COUPLED".

forcePotentialExponent

A Double specifying the number equal to the inverse of the overall exponent in the force potential definition. The default value is 2.0.

This argument is applicable only if *coupling*="COUPLED" and if *forcePotentialOperator*="SUM".

connectorPotentials

A sequence of ConnectorPotential objects specifying one ConnectorPotential object for each force potential contribution. This member can be specified only if *coupling*="COUPLED".

CONNECTOR COMMANDS

isotropicTable

An odb_SequenceSequenceDouble specifying isotropic plasticity properties. Items in the *isotropicTable* data are described below. This argument is applicable only if *isotropic*=true. The default value is an empty sequence.

kinematicTable

An odb_SequenceSequenceDouble specifying kinematic plasticity properties. Items in the *kinematicTable* data are described below. This argument is applicable only if *kinematic*=true. The default value is an empty sequence.

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq components \leq 6$. Only available components can be specified. This argument can be specified only if *coupling*="UNCOUPLED". The default value is an empty sequence.

Table data

Table data for *isotropicTable*:

If *isotropicType*=TABULAR, then each sequence of the table data specifies the following:

- Equivalent yield force or moment defining the size of the elastic range.
- Equivalent relative plastic motion.
- Equivalent relative plastic motion rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *isotropicType*=EXPONENTIAL_LAW, then each sequence of the table data specifies the following:

- Equivalent force or moment defining the size of the elastic range at zero plastic motion.
- Isotropic hardening parameter Q_{inf} .
- Isotropic hardening parameter b .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Table data for *kinematicTable*:

If *kinematicType*=HALF_CYCLE, then each sequence of the table data specifies the following:

- Yield force or moment.
- Connector relative plastic motion.
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *kinematicType*=STABILIZED, then each sequence of the table data specifies the following:

- Yield force or moment.
- Connector relative plastic motion.
- Connector relative constitutive motion range.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *kinematicType*=PARAMETERS, then each sequence of the table data specifies the following:

- Yield force or moment at zero relative plastic motion.
- Kinematic hardening parameter C .
- Kinematic hardening parameter γ . Set $\gamma=0$ to specify linear Ziegler kinematic hardening.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConnectorPlasticity object.

Exceptions

ValueError and TextError.

58.10.2 setValues(...)

This method modifies the ConnectorPlasticity object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorPlasticity** method.

Return value

None

Exceptions

ValueError.

58.10.3 Members

The ConnectorPlasticity object has members with the same names and descriptions as the arguments to the `ConnectorPlasticity` method.

In addition, the ConnectorPlasticity object can have the following members:

Prototype

```
odb_ConnectorOptions isotropicOptions() const;  
odb_ConnectorOptions kinematicOptions() const;
```

isotropicOptions

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for the isotropic hardening table.

kinematicOptions

A ConnectorOptions object specifying the ConnectorOptions used to define tabular options for the kinematic hardening table.

58.10.4 Corresponding analysis keywords

*CONNECTOR PLASTICITY, *CONNECTOR HARDENING, *CONNECTOR POTENTIAL

58.11 ConnectorPotential object

The ConnectorPotential object is used to define a restricted set of mathematical functions to represent yield or limiting surfaces in the space spanned by connector available components. It can be used only in conjunction with ConnectorDamage , ConnectorFriction , and ConnectorPlasticity objects. Because the ConnectorDamage object contains two separate ConnectorPotential repositories (one for damage initiation and one for damage evolution), there are two ConnectorPotential constructors associated with that behavior—IniPotential and EvoPotential.

Access

```

sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i)
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i)
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i)

```

58.11.1 ConnectorPotential(...)

This method creates a connector potential object to be used in conjunction with an allowable connector behavior option.

Path

```

sectionApi.sections() [name].behaviorOptions(i).ConnectorPotential

```

Prototype

```

odb_ConnectorPotential&
ConnectorPotential(const odb_String& componentStyle,
                  int componentNumber,
                  const odb_String& sign,
                  double scaleFactor,
                  double positiveExponent,
                  double shiftFactor,
                  const odb_String& hFunction);

```

Required arguments

None.

Optional arguments*componentStyle*

An odb_String specifying whether a component number or the name of the DerivedComponent object will be used in the contribution. Possible values are "COMPONENT_NUMBER" and "DERIVED_COMPONENT". The default value is "COMPONENT_NUMBER".

componentNumber

An Int specifying the component number used in the contribution. This argument is applicable only if *componentStyle*="COMPONENT_NUMBER". Possible values are $1 \leq \text{componentNumber} \leq 6$. Only available components can be specified. The default value is 0.

sign

An odb_String specifying the sign of the contribution. Possible values are "POSITIVE" and "NEGATIVE". The default value is "POSITIVE".

CONNECTOR COMMANDS

scaleFactor

A Double specifying the scaling factor for the contribution. The default value is 1.0.

positiveExponent

A Double specifying the positive exponent for the contribution. The default value is 2.0.

This argument is ignored if the potential operator of the invoking behavior option is set to "MAXIMUM".

shiftFactor

A Double specifying the shift factor for the contribution. The default value is 0.0.

hFunction

An odb_String specifying the H function of the contribution: either absolute value, Macauley bracket, or the identity function. Possible values are "ABS", "MACAULEY", and "IDENTITY". The default value is "ABS".

The value of "IDENTITY" can be used only if *positiveExponent*=1.0 and the potential exponent of the invoking behavior option is also 1.0 (i.e., the potential operator of the invoking behavior option must be "SUM").

Return value

A ConnectorPotential object.

Exceptions

ValueError and TextError.

58.11.2 setValues(...)

This method modifies the ConnectorPotential object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorPotential** method.

Return value

None

Exceptions

ValueError.

58.11.3 Members

The ConnectorPotential object has members with the same names and descriptions as the arguments to the **ConnectorPotential** method.

In addition, the ConnectorPotential object can have the following member:

Prototype

```
odb_DerivedComponent derivedComponent() const;
```

derivedComponent

A DerivedComponent object specifying the DerivedComponent used in the contribution. This argument is applicable only if *componentStyle*="DERIVED_COMPONENT".

58.11.4 Corresponding analysis keywords

*CONNECTOR POTENTIAL

58.12 ConnectorStop object

The ConnectorStop object defines connector stops for one or more components of a connector's relative motion.

The ConnectorStop object is derived from the ConnectorBehaviorOption object.

Access

```
sectionApi.sections() [name].behaviorOptions(i)
```

58.12.1 ConnectorStop(...)

This method creates a connector stop behavior option for a ConnectorSection object.

Path

```
sectionApi.sections() [name].ConnectorStop
```

Prototype

```
odb_ConnectorStop&  
ConnectorStop(odb_Union minMotion,  
              odb_Union maxMotion,
```

CONNECTOR COMMANDS

```
const odb_SequenceInt& components);
```

Required arguments

None.

Optional arguments

minMotion

The string "NONE" or a Double specifying the lower bound for the connector's relative position for all specified components, or no lower bound. The default value is "NONE".

maxMotion

The string "NONE" or a Double specifying the upper bound for the connector's relative position for all specified components, or no upper bound. The default value is "NONE".

components

An odb_SequenceInt specifying the components of relative motion for which the behavior is defined. Possible values are $1 \leq components \leq 6$. Only available components can be specified. The default value is an empty sequence.

Return value

A ConnectorStop object.

Exceptions

ValueError and TextError.

58.12.2 setValues(...)

This method modifies the ConnectorStop object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **ConnectorStop** method.

Return value

None

Exceptions

ValueError.

58.12.3 Members

The ConnectorStop object has members with the same names and descriptions as the arguments to the `ConnectorStop` method.

58.12.4 Corresponding analysis keywords

*CONNECTOR STOP

58.13 DerivedComponent object

A `DerivedComponent` object describes user-customized components for use in defining `ConnectorFriction` and `Potential` objects.

Access

```
sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.derivedComponent()
sectionApi.sections() [name].behaviorOptions(i).derivedComponent()
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.derivedComponent()
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.derivedComponent()
```

58.13.1 DerivedComponent()

This method creates a `DerivedComponent` object.

Path

```
sectionApi.sections() [name].behaviorOptions(i).connectorPotentials(i) \
.DerivedComponent
sectionApi.sections() [name].behaviorOptions(i).DerivedComponent
sectionApi.sections() [name].behaviorOptions(i).evolutionPotentials(i) \
.DerivedComponent
sectionApi.sections() [name].behaviorOptions(i).initiationPotentials(i) \
.DerivedComponent
```

Prototype

```
odb_DerivedComponent&  
DerivedComponent();
```

Arguments

None.

Return value

A `DerivedComponent` object.

Exceptions

`ValueError` and `TextError`.

58.13.2 `setValues(...)`

This method modifies the `DerivedComponent` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `DerivedComponent` method.

Return value

`None`

Exceptions

`ValueError`.

58.13.3 Members

The `DerivedComponent` object can have the following member:

Prototype

```
odb_SequenceCDCTerm cdcTerms() const;  
odb_CDCTerm cdcTerms(int index) const;
```

cdcTerms

A sequence of CDCTerm objects.

58.13.4 Corresponding analysis keywords

*CONNECTOR DERIVED COMPONENT

58.14 TangentialBehavior object

The TangentialBehavior object specifies tangential behavior for a connector friction behavior option.

Access

```
sectionApi.sections() [name].behaviorOptions(i).tangentialBehavior()
```

58.14.1 TangentialBehavior(...)

This method creates a TangentialBehavior object.

Path

```
sectionApi.sections() [name].behaviorOptions(i).TangentialBehavior
```

Prototype

```
odb_TangentialBehavior&
TangentialBehavior(const odb_String& formulation,
                    bool slipRateDependency,
                    bool pressureDependency,
                    bool temperatureDependency,
                    int dependencies,
                    const odb_String& exponentialDecayDefinition,
                    odb_Union shearStressLimit,
                    const odb_String& maximumElasticSlip,
                    double fraction,
                    odb_Union absoluteDistance,
                    const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments

formulation

An odb_String specifying the friction coefficient formulation. Possible values are "PENALTY" and "EXPONENTIAL_DECAY". The default value is "PENALTY".

slipRateDependency

A Boolean specifying whether the data depend on slip rate. The default value is false.

pressureDependency

A Boolean specifying whether the data depend on contact pressure. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variables for the data. The default value is 0.

exponentialDecayDefinition

An odb_String specifying the exponential decay definition for the data. Possible values are "COEFFICIENTS" and "TEST_DATA". The default value is "COEFFICIENTS".

shearStressLimit

The string "NONE" or a Double specifying no upper limit or the friction coefficient shear stress limit. The default value is "NONE".

maximumElasticSlip

An odb_String specifying the method for modifying the allowable elastic slip. Possible values are "FRACTION" and "ABSOLUTE_DISTANCE". The default value is "FRACTION".

This argument applies only to Abaqus/Standard analyses.

fraction

A Double specifying the ratio of the allowable maximum elastic slip to a characteristic model dimension. The default value is 10^{-4} .

This argument applies only to Abaqus/Standard analyses.

absoluteDistance

The string "NONE" or a Double specifying the absolute magnitude of the allowable elastic slip. The default value is "NONE".

This argument applies only to Abaqus/Standard analyses.

table

An odb_SequenceSequenceDouble specifying the tangential properties. Items in the table data are described below. The default value is an empty sequence.

Table data

If *formulation*=PENALTY, the table data specify the following:

- Friction coefficient in the slip direction, μ .

- Slip rate, if the data depend on slip rate.
- Contact pressure, if the data depend on contact pressure.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=COEFFICIENTS, the table data specify the following:

- Static friction coefficient, μ_s .
- Kinetic friction coefficient, μ_k .
- Decay coefficient, d_c .

If *formulation*=EXPONENTIAL_DECAY and *exponentialDecayDefinition*=TEST_DATA, the table data specify the following:

- Static coefficient of friction.
- Dynamic friction coefficient measured at the reference slip rate, $\dot{\gamma}_2$.
- Reference slip rate, $\dot{\gamma}_2$, used to measure the dynamic friction coefficient.
- Kinetic friction coefficient, μ_∞ . This value corresponds to the asymptotic value of the friction coefficient at infinite slip rate, $\dot{\gamma}_\infty$.

Return value

A TangentialBehavior object.

Exceptions

None.

58.14.2 setValues(...)

This method modifies the TangentialBehavior object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **TangentialBehavior** method.

Return value

None

Exceptions

None.

58.14.3 Members

The `TangentialBehavior` object has members with the same names and descriptions as the arguments to the `TangentialBehavior` method.

58.14.4 Corresponding analysis keywords

*FRICTION

59. Filter commands

Filter commands are used to create real-time filters of output request data.

59.1 Filter object

The Filter object is the abstract base type for other Filter objects. The Filter object has no explicit constructor. The methods and members of the Filter object are common to all objects derived from the Filter.

Access

`filterApi.filters()` [*name*]

59.1.1 Members

The Filter object has the following members:

Prototype

```
odb_String name() const;
double cutoffFrequency() const;
int order() const;
odb_String operation() const;
bool halt() const;
odb_Union limit() const;
odb_String invariant() const;
```

name

An odb_String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Double specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

An odb_String specifying the filter operator. Possible values are "NONE", "MIN", "MAX", and "ABS". The default value is "NONE".

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is false.

limit

The string "NONE" or a Double specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is "NONE".

invariant

An odb_String specifying the invariant to which filtering is applied. Possible values are "NONE", "FIRST", and "SECOND". The default value is "NONE".

59.2 ButterworthFilter object

The ButterworthFilter object defines a Butterworth type filter.

The ButterworthFilter object is derived from the Filter object.

Access

`filterApi.filters()` [*name*]

59.2.1 ButterworthFilter(...)

This method creates a ButterworthFilter object.

Path

`filterApi.ButterworthFilter`

Prototype

```
odb_ButterworthFilter&
ButterworthFilter(const odb_String& name,
                  double cutoffFrequency,
                  int order,
                  const odb_String& operation,
                  bool halt,
                  odb_Union limit,
                  const odb_String& invariant);
```

Required arguments*name*

An odb_String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Double specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments*order*

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

An odb_String specifying the filter operator. Possible values are "NONE", "MIN", "MAX", and "ABS". The default value is "NONE".

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is false.

limit

The string "NONE" or a Double specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is "NONE".

invariant

An odb_String specifying the invariant to which filtering is applied. Possible values are "NONE", "FIRST", and "SECOND". The default value is "NONE".

Return value

A ButterworthFilter object.

Exceptions

InvalidNameError and RangeError.

59.2.2 setValues(...)

This method modifies the ButterworthFilter object.

Arguments**Required arguments**

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `ButterworthFilter` method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

59.2.3 Members

The `ButterworthFilter` object has members with the same names and descriptions as the arguments to the `ButterworthFilter` method.

59.2.4 Corresponding analysis keywords

*FILTER

59.3 Chebyshev1Filter object

The `Chebyshev1Filter` object defines a Chebyshev type 1 filter.

The `Chebyshev1Filter` object is derived from the `Filter` object.

Access

`filterApi.filters()` [*name*]

59.3.1 Chebyshev1Filter(...)

This method creates a `Chebyshev1Filter` object.

Path

`filterApi.Chebyshev1Filter`

Prototype

`odb_Chebyshev1Filter&`

```
Chebyshev1Filter(const odb_String& name,
                 double cutoffFrequency,
                 double rippleFactor,
                 int order,
                 const odb_String& operation,
                 bool halt,
                 odb_Union limit,
                 const odb_String& invariant);
```

Required arguments

name

An odb_String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Double specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments

rippleFactor

A Double specifying the amount of allowable ripple in the filter. Possible values are non-negative numbers. The default value is 0.225.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

An odb_String specifying the filter operator. Possible values are "NONE", "MIN", "MAX", and "ABS". The default value is "NONE".

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is false.

limit

The string "NONE" or a Double specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is "NONE".

invariant

An odb_String specifying the invariant to which filtering is applied. Possible values are "NONE", "FIRST", and "SECOND". The default value is "NONE".

Return value

A Chebyshev1Filter object.

Exceptions

`InvalidNameError` and `RangeError`.

59.3.2 `setValues(...)`

This method modifies the `Chebyshev1Filter` object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to `setValues` are the same as the arguments to the `Chebyshev1Filter` method, except for the *name* argument.

Return value

`None`

Exceptions

`RangeError`.

59.3.3 Members

The `Chebyshev1Filter` object has members with the same names and descriptions as the arguments to the `Chebyshev1Filter` method.

59.3.4 Corresponding analysis keywords

*FILTER

59.4 `Chebyshev2Filter` object

The `Chebyshev2Filter` object defines a Chebyshev type 2 filter.

The `Chebyshev2Filter` object is derived from the `Filter` object.

Access

```
filterApi.filters() [name]
```

59.4.1 Chebyshev2Filter(...)

This method creates a Chebyshev2Filter object.

Path

```
filterApi.Chebyshev2Filter
```

Prototype

```
odb_Chebyshev2Filter&
Chebyshev2Filter(const odb_String& name,
                 double cutoffFrequency,
                 double rippleFactor,
                 int order,
                 const odb_String& operation,
                 bool halt,
                 odb_Union limit,
                 const odb_String& invariant);
```

Required arguments

name

An odb_String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Double specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

Optional arguments

rippleFactor

A Double specifying the amount of allowable ripple in the filter. Possible values are non-negative numbers less than 1. The default value is 0.025.

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

An odb_String specifying the filter operator. Possible values are "NONE", "MIN", "MAX", and "ABS". The default value is "NONE".

FILTER COMMANDS

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is false.

limit

The string "NONE" or a Double specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is "NONE".

invariant

An odb_String specifying the invariant to which filtering is applied. Possible values are "NONE", "FIRST", and "SECOND". The default value is "NONE".

Return value

A Chebyshev2Filter object.

Exceptions

InvalidNameError and RangeError.

59.4.2 setValues(...)

This method modifies the Chebyshev2Filter object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **Chebyshev2Filter** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

59.4.3 Members

The Chebyshev2Filter object has members with the same names and descriptions as the arguments to the **Chebyshev2Filter** method.

59.4.4 Corresponding analysis keywords

*FILTER

59.5 OperatorFilter object

The OperatorFilter object defines an operator filter.

The OperatorFilter object is derived from the Filter object.

Access

`filterApi.filters()` [*name*]

59.5.1 OperatorFilter(...)

This method creates an OperatorFilter object.

Path

`filterApi.OperatorFilter`

Prototype

```
odb_OperatorFilter&
OperatorFilter(const odb_String& name,
               double cutoffFrequency,
               int order,
               const odb_String& operation,
               bool halt,
               odb_Union limit,
               const odb_String& invariant);
```

Required arguments

name

An odb_String specifying the repository key. This name ANTIALIASING is reserved for filters generated internally by the program.

cutoffFrequency

A Double specifying the attenuation point of the filter. Possible values are non-negative numbers. Order is not available for OperatorFilter.

FILTER COMMANDS

Optional arguments

order

An Int specifying the highest power of the filter transfer function. Possible values are non-negative numbers less than or equal to 20. Order is not available for OperatorFilter. The default value is 2.

operation

An odb_String specifying the filter operator. Possible values are "NONE", "MIN", "MAX", and "ABS". The default value is "NONE".

halt

A Boolean specifying whether to stop the analysis if the specified limit is reached. The default value is false.

limit

The string "NONE" or a Double specifying the threshold limit, an upper or lower bound for output values depending on the operation, or a bound for stopping the analysis when Halt is used. The default value is "NONE".

invariant

An odb_String specifying the invariant to which filtering is applied. Possible values are "NONE", "FIRST", and "SECOND". The default value is "NONE".

Return value

An OperatorFilter object.

Exceptions

InvalidNameError and RangeError.

59.5.2 **setValues(...)**

This method modifies the OperatorFilter object.

Arguments

Required arguments

None.

Optional arguments

The optional arguments to **setValues** are the same as the arguments to the **OperatorFilter** method, except for the *name* argument.

Return value

None

Exceptions

RangeError.

59.5.3 Members

The `OperatorFilter` object has members with the same names and descriptions as the arguments to the `OperatorFilter` method.

59.5.4 Corresponding analysis keywords

*FILTER

60. Material commands

The Material commands are used to define the materials in an ODB.

60.1 Material object

A Material object is the object used to specify a material. The Material object stores the various settings that determine how a material behaves.

A material is created by combining one or more individual material options and sub options. A particular material option is associated with the Material object through a member. For example: the *acousticMedium* member may contain an *AcousticMedium* object. The alternative of having a *MaterialOption* abstract base class and a container of *MaterialOptions* was rejected because it would make it more difficult to enforce the fact that one Material object cannot contain two *AcousticMedium* objects, for example.

Access

```
materialApi.materials() [name]
```

60.1.1 Material(...)

This method creates a Material object.

Path

```
materialApi.Material
```

Prototype

```
odb_Material&
Material(const odb_String& name,
         const odb_String& description,
         const odb_String& materialIdentifier);
```

Required argument

name

An odb_String specifying the name of the new material.

Optional arguments

description

An odb_String specifying user description of the material. The default value is an empty string.

MATERIAL COMMANDS

materialIdentifier

An odb_String specifying material identifier for customer use. The default value is an empty string.

Return value

A Material object.

Exceptions

InvalidNameError.

60.1.2 Members

The Material object has members with the same names and descriptions as the arguments to the **Material** method.

In addition, the Material object can have the following members:

Prototype

```
odb_AcousticMedium acousticMedium() const;
odb_BrittleCracking brittleCracking() const;
odb_CapPlasticity capPlasticity() const;
odb_CastIronPlasticity castIronPlasticity() const;
odb_ClayPlasticity clayPlasticity() const;
odb_Concrete concrete() const;
odb_ConcreteDamagedPlasticity concreteDamagedPlasticity() const;
odb_Conductivity conductivity() const;
odb_Creep creep() const;
odb_CrushableFoam crushableFoam() const;
odb_DamageInitiation ductileDamageInitiation() const;
odb_DamageInitiation fldDamageInitiation() const;
odb_DamageInitiation flsdDamageInitiation() const;
odb_DamageInitiation johnsonCookDamageInitiation() const;
odb_DamageInitiation maxeDamageInitiation() const;
odb_DamageInitiation maxsDamageInitiation() const;
odb_DamageInitiation maxpeDamageInitiation() const;
odb_DamageInitiation maxpsDamageInitiation() const;
odb_DamageInitiation mkDamageInitiation() const;
odb_DamageInitiation msfldDamageInitiation() const;
odb_DamageInitiation quadeDamageInitiation() const;
odb_DamageInitiation quadsDamageInitiation() const;
odb_DamageInitiation shearDamageInitiation() const;
odb_DamageInitiation hashinDamageInitiation() const;
```



```

odb_Damping damping() const;
odb_DeformationPlasticity deformationPlasticity() const;
odb_Density density() const;
odb_Depvar depvar() const;
odb_Dielectric dielectric() const;
odb_Diffusivity diffusivity() const;
odb_DruckerPrager druckerPrager() const;
odb_Elastic elastic() const;
odb_ElectricalConductivity electricalConductivity() const;
odb_Eos eos() const;
odb_Expansion expansion() const;
odb_FluidLeakoff fluidLeakoff() const;
odb_GapFlow gapFlow() const;
odb_GasketThicknessBehavior gasketThicknessBehavior() const;
odb_GasketTransverseShearElastic gasketTransverseShearElastic() const;
odb_GasketMembraneElastic gasketMembraneElastic() const;
odb_Gel gel() const;
odb_HeatGeneration heatGeneration() const;
odb_Hyperelastic hyperelastic() const;
odb_Hyperfoam hyperfoam() const;
odb_Hypoelastic hypoelastic() const;
odb_InelasticHeatFraction inelasticHeatFraction() const;
odb_JouleHeatFraction jouleHeatFraction() const;
odb_LatentHeat latentHeat() const;
odb_LowDensityFoam lowDensityFoam() const;
odb_MagneticPermeability magneticPermeability() const;
odb_MohrCoulombPlasticity mohrCoulombPlasticity() const;
odb_MoistureSwelling moistureSwelling() const;
odb_MullinsEffect mullinsEffect() const;
odb_Permeability permeability() const;
odb_Piezoelectric piezoelectric() const;
odb_Plastic plastic() const;
odb_PoreFluidExpansion poreFluidExpansion() const;
odb_PorousBulkModuli porousBulkModuli() const;
odb_PorousElastic porousElastic() const;
odb_PorousMetalPlasticity porousMetalPlasticity() const;
odb-Regularization regularization() const;
odb_Solubility solubility() const;
odb_Sorption sorption() const;
odb_SpecificHeat specificHeat() const;
odb_Swelling swelling() const;

```

MATERIAL COMMANDS

```
odb_UserDefinedField userDefinedField() const;  
odb_UserMaterial userMaterial() const;  
odb_UserOutputVariables userOutputVariables() const;  
odb_Viscoelastic viscoelastic() const;  
odb_Viscosity viscosity() const;  
odb_Viscous viscous() const;
```

acousticMedium

An AcousticMedium object.

brittleCracking

A BrittleCracking object.

capPlasticity

A CapPlasticity object.

castIronPlasticity

A CastIronPlasticity object.

clayPlasticity

A ClayPlasticity object.

concrete

A Concrete object.

concreteDamagedPlasticity

A ConcreteDamagedPlasticity object.

conductivity

A Conductivity object.

creep

A Creep object.

crushableFoam

A CrushableFoam object.

ductileDamageInitiation

A DamageInitiation object.

fldDamageInitiation

A DamageInitiation object.

flsdDamageInitiation

A DamageInitiation object.

johnsonCookDamageInitiation

A DamageInitiation object.

maxeDamageInitiation

A DamageInitiation object.

maxsDamageInitiation

A DamageInitiation object.

maxpeDamageInitiation
A DamageInitiation object.

maxpsDamageInitiation
A DamageInitiation object.

mkDamageInitiation
A DamageInitiation object.

msfldDamageInitiation
A DamageInitiation object.

quadeDamageInitiation
A DamageInitiation object.

quadsDamageInitiation
A DamageInitiation object.

shearDamageInitiation
A DamageInitiation object.

hashinDamageInitiation
A DamageInitiation object.

damping
A Damping object.

deformationPlasticity
A DeformationPlasticity object.

density
A Density object.

depvar
A Depvar object.

dielectric
A Dielectric object.

diffusivity
A Diffusivity object.

druckerPrager
A DruckerPrager object.

elastic
An Elastic object.

electricalConductivity
An ElectricalConductivity object.

eos
An Eos object.

expansion
An Expansion object.

MATERIAL COMMANDS

fluidLeakoff

A FluidLeakoff object.

gapFlow

A GapFlow object.

gasketThicknessBehavior

A GasketThicknessBehavior object.

gasketTransverseShearElastic

A GasketTransverseShearElastic object.

gasketMembraneElastic

A GasketMembraneElastic object.

gel

A Gel object.

heatGeneration

A HeatGeneration object.

hyperelastic

A Hyperelastic object.

hyperfoam

A Hyperfoam object.

hypoelastic

A Hypoelastic object.

inelasticHeatFraction

An InelasticHeatFraction object.

jouleHeatFraction

A JouleHeatFraction object.

latentHeat

A LatentHeat object.

lowDensityFoam

A LowDensityFoam object.

magneticPermeability

A MagneticPermeability object.

mohrCoulombPlasticity

A MohrCoulombPlasticity object.

moistureSwelling

A MoistureSwelling object.

mullinsEffect

A MullinsEffect object.

permeability

A Permeability object.

piezoelectric
A Piezoelectric object.

plastic
A Plastic object.

poreFluidExpansion
A PoreFluidExpansion object.

porousBulkModuli
A PorousBulkModuli object.

porousElastic
A PorousElastic object.

porousMetalPlasticity
A PorousMetalPlasticity object.

regularization
A Regularization object.

solubility
A Solubility object.

sorption
A Sorption object.

specificHeat
A SpecificHeat object.

swelling
A Swelling object.

userDefinedField
A UserDefinedField object.

userMaterial
A UserMaterial object.

userOutputVariables
A UserOutputVariables object.

viscoelastic
A Viscoelastic object.

viscosity
A Viscosity object.

viscous
A Viscous object.

60.1.3 Corresponding analysis keywords

*MATERIAL

60.2 AcousticMedium object

The AcousticMedium object specifies the acoustic properties of a material.

Access

```
materialApi.materials() [name].acousticMedium()
```

60.2.1 AcousticMedium(...)

This method creates an AcousticMedium object.

Path

```
materialApi.materials() [name].AcousticMedium
```

Prototype

```
odb_AcousticMedium&
AcousticMedium(bool acousticVolumetricDrag,
                bool temperatureDependencyB,
                bool temperatureDependencyV,
                int dependenciesB,
                int dependenciesV,
                const odb_SequenceSequenceDouble& bulkTable,
                const odb_SequenceSequenceDouble& volumetricTable);
```

Required arguments

None.

Optional arguments

acousticVolumetricDrag

A Boolean specifying whether the *volumetricTable* data is specified. The default value is false.

temperatureDependencyB

A Boolean specifying whether the data in *bulkTable* depend on temperature. The default value is false.

temperatureDependencyV

A Boolean specifying whether the data in *volumetricTable* depend on temperature. The default value is false.

dependenciesB

An Int specifying the number of field variable dependencies for the data in *bulkTable*. The default value is 0.

dependenciesV

An Int specifying the number of field variable dependencies for the data in *volumetricTable*. The default value is 0.

bulkTable

An odb_SequenceSequenceDouble specifying the following:

- Bulk modulus.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The default value is an empty sequence.

volumetricTable

An odb_SequenceSequenceDouble specifying the following:

- Volumetric drag.
- Frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

The default value is an empty sequence.

Return value

An AcousticMedium object.

Exceptions

RangeError.

60.2.2 Members

The AcousticMedium object has members with the same names and descriptions as the arguments to the **AcousticMedium** method.

60.2.3 Corresponding analysis keywords

*ACOUSTIC MEDIUM

60.3 AnnealTemperature object

The AnnealTemperature object specifies the material annealing temperature.

Access

```
materialApi.materials() [name].plastic().annealTemperature()
```

60.3.1 AnnealTemperature(...)

This method creates an AnnealTemperature object.

Path

```
materialApi.materials() [name].plastic().AnnealTemperature
```

Prototype

```
odb_AnnealTemperature&  
AnnealTemperature(const odb_SequenceSequenceDouble& table,  
                  int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional argument

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The annealing temperature, θ .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

An `AnnealTemperature` object.

Exceptions

`RangeError`.

60.3.2 Members

The `AnnealTemperature` object has members with the same names and descriptions as the arguments to the `AnnealTemperature` method.

60.3.3 Corresponding analysis keywords

*ANNEAL TEMPERATURE

60.4 BiaxialTestData object

The `BiaxialTestData` object provides equibiaxial test data (compression and/or tension).

Access

```
materialApi.materials() [name].hyperelastic().biaxialTestData()
materialApi.materials() [name].hyperfoam().biaxialTestData()
materialApi.materials() [name].mullinsEffect().biaxialTests(i)
```

60.4.1 BiaxialTestData(...)

This method creates a `BiaxialTestData` object.

Path

```
materialApi.materials() [name].hyperelastic().BiaxialTestData
materialApi.materials() [name].hyperfoam().BiaxialTestData
materialApi.materials() [name].mullinsEffect().BiaxialTestData
```

Prototype

```
odb_BiaxialTestData&
```

MATERIAL COMMANDS

```
BiaxialTestData(const odb_SequenceSequenceDouble& table,  
                odb_Union smoothing,  
                bool lateralNominalStrain,  
                bool temperatureDependency,  
                int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the following:

- Nominal stress, T_B .
- Nominal strain, ϵ_B .

Optional arguments

smoothing

The string "NONE" or an Int specifying the value for smoothing. If *smoothing*="NONE", no smoothing is employed. The default value is "NONE".

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Return value

A BiaxialTestData object.

Exceptions

None.

60.4.2 Members

The BiaxialTestData object has members with the same names and descriptions as the arguments to the **BiaxialTestData** method.

60.4.3 Corresponding analysis keywords

*BIAXIAL TEST DATA

60.5 BrittleCracking object

The BrittleCracking object specifies cracking and postcracking properties for the brittle cracking material model.

Access

```
materialApi.materials() [name].brittleCracking()
```

60.5.1 BrittleCracking(...)

This method creates a BrittleCracking object.

Path

```
materialApi.materials() [name].BrittleCracking
```

Prototype

```
odb_BrittleCracking&
BrittleCracking(const odb_SequenceSequenceDouble& table,
                bool temperatureDependency,
                int dependencies,
                const odb_String& type);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

An odb_String specifying the type of postcracking behavior. Possible values are "STRAIN", "DISPLACEMENT", and "GFI". The default value is "STRAIN".

Table data

If *type*=STRAIN the table data specify the following:

- Remaining direct stress after cracking.

MATERIAL COMMANDS

- Direct cracking strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT the table data specify the following:

- Remaining direct stress after cracking.
- Direct cracking displacement.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=GFI the table data specify the following:

- Failure stress.
- Mode I fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleCracking object.

Exceptions

None.

60.5.2 Members

The BrittleCracking object has members with the same names and descriptions as the arguments to the **BrittleCracking** method.

In addition, the BrittleCracking object can have the following members:

Prototype

```
odb_BrittleShear brittleShear() const;  
odb_BrittleFailure brittleFailure() const;
```

brittleShear

A BrittleShear object.

brittleFailure

A BrittleFailure object.

60.5.3 Corresponding analysis keywords

*BRITTLE CRACKING

60.6 BrittleFailure object

The BrittleFailure object specifies the brittle failure of the material.

Access

```
materialApi.materials() [name].brittleCracking().brittleFailure()
```

60.6.1 BrittleFailure(...)

This method creates a BrittleFailure object.

Path

```
materialApi.materials() [name].brittleCracking().BrittleFailure
```

Prototype

```
odb_BrittleFailure&
BrittleFailure(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies,
               const odb_String& failureCriteria);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

failureCriteria

An odb_String specifying the failure criteria. Possible values are "UNIDIRECTIONAL", "BIDIRECTIONAL", and "TRIDIRECTIONAL". The default value is "UNIDIRECTIONAL".

Table data

If parent BrittleCracking member *type*=STRAIN the table data specify the following:

- Direct cracking failure strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If parent BrittleCracking member *type*=DISPLACEMENT or *type*=GFI the table data specify the following:

- Direct cracking failure displacement.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleFailure object.

Exceptions

RangeError.

60.6.2 Members

The BrittleFailure object has members with the same names and descriptions as the arguments to the **BrittleFailure** method.

60.6.3 Corresponding analysis keywords

*BRITTLE FAILURE

60.7 BrittleShear object

The BrittleShear object specifies the postcracking shear behavior of a material used in a brittle cracking model.

Access

```
materialApi.materials() [name].brittleCracking().brittleShear()
```

60.7.1 BrittleShear(...)

This method creates a BrittleShear object.

Path

```
materialApi.materials() [name].brittleCracking().BrittleShear
```

Prototype

```
odb_BrittleShear&
BrittleShear(const odb_SequenceSequenceDouble& table,
             bool temperatureDependency,
             int dependencies,
             const odb_String& type);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

An odb_String specifying the type of postcracking shear behavior. Possible values are "RETENTION_FACTOR" and "POWER_LAW". The default value is "RETENTION_FACTOR".

Table data

If *type*=RETENTION_FACTOR the table data specify the following:

MATERIAL COMMANDS

- Shear retention factor.
- Crack opening strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=POWER_LAW the table data specify the following:

- e.
- p.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BrittleShear object.

Exceptions

RangeError.

60.7.2 Members

The BrittleShear object has members with the same names and descriptions as the arguments to the **BrittleShear** method.

60.7.3 Corresponding analysis keywords

*BRITTLE SHEAR

60.8 CapCreepCohesion object

The CapCreepCohesion object specifies a cap creep model and material properties.

Access

```
materialApi.materials() [name].capPlasticity().capCreepCohesion()
```

60.8.1 CapCreepCohesion(...)

This method creates a CapCreepCohesion object.

Path

```
materialApi.materials() [name].capPlasticity().CapCreepCohesion
```

Prototype

```
odb_CapCreepCohesion&
CapCreepCohesion(const odb_SequenceSequenceDouble& table,
                  const odb_String& law,
                  bool temperatureDependency,
                  int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

law

An odb_String specifying the cap creep law. Possible values are "STRAIN", "TIME", "SINGHM", and "USER". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

MATERIAL COMMANDS

If $law=SINGHM$, the table data specify the following:

- A .
- α .
- m .
- t_1 .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `CapCreepCohesion` object.

Exceptions

None.

60.8.2 Members

The `CapCreepCohesion` object has members with the same names and descriptions as the arguments to the `CapCreepCohesion` method.

60.8.3 Corresponding analysis keywords

*CAP CREEP

60.9 CapCreepConsolidation object

The `CapCreepConsolidation` object specifies a cap creep model and material properties.

Access

```
materialApi.materials() [name].capPlasticity().capCreepConsolidation()
```

60.9.1 CapCreepConsolidation(...)

This method creates a `CapCreepConsolidation` object.

Path

```
materialApi.materials() [name].capPlasticity().CapCreepConsolidation
```

Prototype

```
odb_CapCreepConsolidation&
CapCreepConsolidation(const odb_SequenceSequenceDouble& table,
                      const odb_String& law,
                      bool temperatureDependency,
                      int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*law*

An odb_String specifying the cap creep law. Possible values are "STRAIN", "TIME", "SINGHM", and "USER". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=SINGHM, the table data specify the following:

- *A*.
- α .
- *m*.
- t_1 .
- Temperature, if the data depend on temperature.

MATERIAL COMMANDS

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapCreepConsolidation object.

Exceptions

None.

60.9.2 Members

The CapCreepConsolidation object has members with the same names and descriptions as the arguments to the **CapCreepConsolidation** method.

60.9.3 Corresponding analysis keywords

*CAP CREEP

60.10 CapHardening object

The CapHardening object specifies Drucker-Prager/Cap plasticity hardening.

Access

```
materialApi.materials() [name].capPlasticity().capHardening()
```

60.10.1 CapHardening(...)

This method creates a CapHardening object.

Path

```
materialApi.materials() [name].capPlasticity().CapHardening
```

Prototype

```
odb_CapHardening&
```

```
CapHardening(const odb_SequenceSequenceDouble& table,
              bool temperatureDependency,
              int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Hydrostatic pressure yield stress.
- Absolute value of the corresponding volumetric inelastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapHardening object.

Exceptions

RangeError.

60.10.2 Members

The CapHardening object has members with the same names and descriptions as the arguments to the **CapHardening** method.

60.10.3 Corresponding analysis keywords

*CAP HARDENING

60.11 CapPlasticity object

The CapPlasticity object specifies the modified Drucker-Prager/Cap plasticity model.

Access

```
materialApi.materials() [name].capPlasticity()
```

60.11.1 CapPlasticity(...)

This method creates a CapPlasticity object.

Path

```
materialApi.materials() [name].CapPlasticity
```

Prototype

```
odb_CapPlasticity&
CapPlasticity(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Material cohesion, d , in the p - t plane (Abaqus/Standard) or in the p - q plane (Abaqus/Explicit).
- Material angle of friction, β , in the p - t plane (Abaqus/Standard) or in the p - q plane (Abaqus/Explicit). Give the value in degrees.
- Cap eccentricity parameter, R . Its value must be greater than zero (typically $0.0 < R < 1.0$).
- Initial cap yield surface position, $\varepsilon_{vol}^{pl}|_0$.
- Transition surface radius parameter, α . The default value is 0.0 (i.e., no transition surface).

- (Not used in Abaqus/Explicit) K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. Possible values are $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, Abaqus/Standard assumes $K = 1.0$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CapPlasticity object.

Exceptions

RangeError.

60.11.2 Members

The CapPlasticity object has members with the same names and descriptions as the arguments to the **CapPlasticity** method.

In addition, the CapPlasticity object can have the following members:

Prototype

```
odb_CapCreepCohesion capCreepCohesion() const;
odb_CapCreepConsolidation capCreepConsolidation() const;
odb_CapHardening capHardening() const;
```

capCreepCohesion

A CapCreepCohesion object.

capCreepConsolidation

A CapCreepConsolidation object.

capHardening

A CapHardening object.

60.11.3 Corresponding analysis keywords

*CAP PLASTICITY

60.12 CastIronCompressionHardening object

The CastIronCompressionHardening object specifies hardening for the Cast- Iron plasticity model.

Access

```
materialApi.materials() [name].castIronPlasticity()\
.castIronCompressionHardening()
```

60.12.1 CastIronCompressionHardening(...)

This method creates a CastIronCompressionHardening object.

Path

```
materialApi.materials() [name].castIronPlasticity()\
.castIronCompressionHardening
```

Prototype

```
odb_CastIronCompressionHardening&
CastIronCompressionHardening(const odb_SequenceSequenceDouble& table,
                               bool temperatureDependency,
                               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in compression, σ_c .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronCompressionHardening object.

Exceptions

RangeError.

60.12.2 Members

The CastIronCompressionHardening object has members with the same names and descriptions as the arguments to the **CastIronCompressionHardening** method.

60.12.3 Corresponding analysis keywords

*CAST IRON COMPRESSION HARDENING

60.13 CastIronPlasticity object

The CastIronPlasticity object specifies the Cast Iron plasticity model.

Access

```
materialApi.materials() [name].castIronPlasticity()
```

60.13.1 CastIronPlasticity(...)

This method creates a CastIronPlasticity object.

Path

```
materialApi.materials() [name].CastIronPlasticity
```

Prototype

```
odb_CastIronPlasticity&
```

```
CastIronPlasticity(const odb_SequenceSequenceDouble& table,
                  bool temperatureDependency,
                  int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

The table data specify the following:

- Plastic Poisson's ratio, ν_{pl} (dimensionless).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronPlasticity object.

Exceptions

RangeError.

60.13.2 Members

The CastIronPlasticity object has members with the same names and descriptions as the arguments to the **CastIronPlasticity** method.

In addition, the CastIronPlasticity object can have the following members:

Prototype

```
odb_CastIronTensionHardening castIronTensionHardening() const;
odb_CastIronCompressionHardening castIronCompressionHardening() const;
```

castIronTensionHardening

A CastIronTensionHardening object.

castIronCompressionHardening

A CastIronCompressionHardening object.

60.13.3 Corresponding analysis keywords

*CAST IRON PLASTICITY

60.14 CastIronTensionHardening object

The CastIronTensionHardening object specifies hardening for the Cast- Iron plasticity model.

Access

```
materialApi.materials() [name].castIronPlasticity() \
.castIronTensionHardening()
```

60.14.1 CastIronTensionHardening(...)

This method creates a CastIronTensionHardening object.

Path

```
materialApi.materials() [name].castIronPlasticity() \
.CastIronTensionHardening
```

Prototype

```
odb_CastIronTensionHardening&
CastIronTensionHardening(const odb_SequenceSequenceDouble& table,
                          bool temperatureDependency,
                          int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in uniaxial tension, σ_t .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CastIronTensionHardening object.

Exceptions

RangeError.

60.14.2 Members

The CastIronTensionHardening object has members with the same names and descriptions as the arguments to the **CastIronTensionHardening** method.

60.14.3 Corresponding analysis keywords

*CAST IRON TENSION HARDENING

60.15 ClayHardening object

The ClayHardening object specifies hardening for the clay plasticity model.

Access

```
materialApi.materials() [name].clayPlasticity().clayHardening()
```

60.15.1 ClayHardening(...)

This method creates a ClayHardening object.

Path

```
materialApi.materials()[name].clayPlasticity().ClayHardening
```

Prototype

```
odb_ClayHardening&
ClayHardening(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The hydrostatic pressure stress at yield, p_c .
- The absolute value of the corresponding volumetric plastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ClayHardening object.

Exceptions

RangeError.

60.15.2 Members

The ClayHardening object has members with the same names and descriptions as the arguments to the `ClayHardening` method.

60.15.3 Corresponding analysis keywords

*CLAY HARDENING

60.16 ClayPlasticity object

The ClayPlasticity object specifies the extended Cam-clay plasticity model.

Access

```
materialApi.materials() [name].clayPlasticity()
```

60.16.1 ClayPlasticity(...)

This method creates a ClayPlasticity object.

Path

```
materialApi.materials() [name].ClayPlasticity
```

Prototype

```
odb_ClayPlasticity&  
ClayPlasticity(const odb_SequenceSequenceDouble& table,  
               odb_Union intercept,  
               const odb_String& hardening,  
               bool temperatureDependency,  
               int dependencies);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional arguments*intercept*

The string "NONE" or a Double specifying e_1 , the intercept of the virgin consolidation line with the void ratio axis in a plot of void ratio versus the logarithm of pressure stress. The default value is "NONE".

This argument is valid only if *hardening*="EXPONENTIAL".

hardening

An odb_String specifying the type of hardening/softening definition. Possible values are "EXPONENTIAL" and "TABULAR". The default value is "EXPONENTIAL".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=EXPONENTIAL, the table data specify the following:

- Logarithmic plastic bulk modulus, λ (dimensionless).
- Stress ratio at critical state, M .
- The initial yield surface size, a_0 .
- β , the parameter defining the size of the yield surface on the “wet” side of critical state.
- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, a value of 1.0 is assumed.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=TABULAR, the table data specify the following:

- Stress ratio at critical state, M .
- The initial volumetric plastic strain, $\varepsilon_{vol}^{pl}|_0$, corresponding to $p_c|_0$ according to the ClayHardening definition.
- β , the parameter defining the size of the yield surface on the “wet” side of critical state.
- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ClayPlasticity object.

Exceptions

RangeError.

60.16.2 Members

The ClayPlasticity object has members with the same names and descriptions as the arguments to the `ClayPlasticity` method.

In addition, the ClayPlasticity object can have the following member:

Prototype

```
odb_ClayHardening clayHardening() const;
```

clayHardening

A ClayHardening object.

60.16.3 Corresponding analysis keywords

*CLAY PLASTICITY

60.17 CombinedTestData object

The CombinedTestData object specifies simultaneously the normalized shear and bulk compliances or relaxation moduli as functions of time.

Access

```
materialApi.materials() [name].viscoelastic().combinedTestData()
```

60.17.1 CombinedTestData(...)

This method creates a CombinedTestData object.

Path

```
materialApi.materials() [name].viscoelastic().CombinedTestData
```


Prototype

```
odb_CombinedTestData&
CombinedTestData(const odb_SequenceSequenceDouble& table,
                 odb_Union volinf,
                 odb_Union shrinkf);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below. The values of the table data depend on the value of the *time* member of the Viscoelastic object.

Optional arguments*volinf*

The string "NONE" or a Double specifying a normalized volume. The value of *volinf* depends on the value of the *time* member of the Viscoelastic object. The default value is "NONE".

If *time*="RELAXATION_TEST_DATA", *volinf* specifies the value of the long-term normalized volumetric modulus, $k_R(\infty)$.

If *time*="CREEP_TEST_DATA", *volinf* specifies the value of the long-term normalized volumetric compliance, $j_K(\infty)$.

shrinkf

The string "NONE" or a Double specifying a normalized shear. The value of *shrinkf* depends on the value of the *time* member of the Viscoelastic object. The default value is "NONE".

If *time*="RELAXATION_TEST_DATA", *shrinkf* specifies the value of the long-term normalized shear modulus, $g_R(\infty)$.

If *time*="CREEP_TEST_DATA", *shrinkf* specifies the value of the long-term normalized shear compliance, $j_S(\infty)$.

Table data

If *time*=RELAXATION_TEST_DATA, the table data specify the following:

- Normalized shear modulus, $g_R(t)$ ($0 \leq g_R(t) \leq 1$).
- Normalized volumetric (bulk) modulus, $k_R(t)$ ($0 \leq k_R(t) \leq 1$).
- Time t ($t > 0$).

If *time*=CREEP_TEST_DATA, the table data specify the following:

- Normalized shear compliance, $j_S(t)$ ($j_S(t) \geq 1$).
- Normalized volumetric (bulk) compliance, $j_K(t)$ ($j_K(t) \geq 1$).
- Time t ($t > 0$).

Return value

A CombinedTestData object.

Exceptions

None.

60.17.2 Members

The `CombinedTestData` object has members with the same names and descriptions as the arguments to the `CombinedTestData` method.

60.17.3 Corresponding analysis keywords

*COMBINED TEST DATA

60.18 Concrete object

The Concrete object defines concrete properties beyond the elastic range.

Access

```
materialApi.materials() [name].concrete()
```

60.18.1 Concrete(...)

This method creates a Concrete object.

Path

```
materialApi.materials() [name].Concrete
```

Prototype

```
odb_Concrete&  
Concrete(const odb_SequenceSequenceDouble& table,  
         bool temperatureDependency,  
         int dependencies);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Absolute value of compressive stress.
- Absolute value of plastic strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Concrete object.

Exceptions

RangeError.

60.18.2 Members

The Concrete object has members with the same names and descriptions as the arguments to the **Concrete** method.

In addition, the Concrete object can have the following members:

Prototype

```
odb_FailureRatios failureRatios() const;
odb_ShearRetention shearRetention() const;
odb_TensionStiffening tensionStiffening() const;
```

failureRatios

A FailureRatios object.

shearRetention

A ShearRetention object.

tensionStiffening

A TensionStiffening object.

60.18.3 Corresponding analysis keywords

*CONCRETE

60.19 ConcreteCompressionDamage object

The ConcreteCompressionDamage object specifies hardening for the concrete damaged plasticity model.

Access

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.concreteCompressionDamage()
```

60.19.1 ConcreteCompressionDamage(...)

This method creates a ConcreteCompressionDamage object.

Path

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.ConcreteCompressionDamage
```

Prototype

```
odb_ConcreteCompressionDamage&
ConcreteCompressionDamage(const odb_SequenceSequenceDouble& table,
                           double tensionRecovery,
                           bool temperatureDependency,
                           int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

tensionRecovery

A Double specifying the value of the stiffness recovery factor, w_t , that determines the amount of tension stiffness that is recovered as loading changes from compression to tension. The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Compressive damage variable, d_c .
- Inelastic (crushing) strain, ϵ_c^{in} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteCompressionDamage object.

Exceptions

RangeError.

60.19.2 Members

The ConcreteCompressionDamage object has members with the same names and descriptions as the arguments to the **ConcreteCompressionDamage** method.

60.19.3 Corresponding analysis keywords

*CONCRETE COMPRESSION DAMAGE

60.20 ConcreteCompressionHardening object

The ConcreteCompressionHardening object specifies hardening for the concrete damaged plasticity model.

Access

```
materialApi.materials() [name].concreteDamagedPlasticity() \
```

```
.concreteCompressionHardening()
```

60.20.1 ConcreteCompressionHardening(...)

This method creates a ConcreteCompressionHardening object.

Path

```
materialApi.materials() [name].concreteDamagedPlasticity() \
    .ConcreteCompressionHardening
```

Prototype

```
odb_ConcreteCompressionHardening&
ConcreteCompressionHardening(const odb_SequenceSequenceDouble& table,
                              bool rate,
                              bool temperatureDependency,
                              int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

rate

A Boolean specifying whether the data depend on rate. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress in compression, σ_c .
- Inelastic (crushing) strain, ϵ_c^{in} .
- Inelastic (crushing) strain rate, $\dot{\epsilon}_c^{in}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteCompressionHardening object.

Exceptions

RangeError.

60.20.2 Members

The ConcreteCompressionHardening object has members with the same names and descriptions as the arguments to the **ConcreteCompressionHardening** method.

60.20.3 Corresponding analysis keywords

*CONCRETE COMPRESSION HARDENING

60.21 ConcreteDamagedPlasticity object

The ConcreteDamagedPlasticity object specifies the concrete damaged plasticity model.

Access

```
materialApi.materials() [name].concreteDamagedPlasticity()
```

60.21.1 ConcreteDamagedPlasticity(...)

This method creates a ConcreteDamagedPlasticity object.

Path

```
materialApi.materials() [name].ConcreteDamagedPlasticity
```

Prototype

```
odb_ConcreteDamagedPlasticity&
ConcreteDamagedPlasticity(const odb_SequenceSequenceDouble& table,
                           bool temperatureDependency,
                           int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

The table data specify the following:

- Dilation angle, ψ (in degrees) in the p - q plane.
- Flow potential eccentricity, ϵ . The default value is 0.1.
- σ_{b0}/σ_{t0} , the ratio of initial equibiaxial compressive yield stress to initial uniaxial compressive yield stress. The default value is 1.16.
- K_c , the ratio of the second stress invariant on the tensile meridian, to that on the compressive meridian, at initial yield for any given value of the pressure invariant p such that the maximum principal stress is negative. The default value is 2/3.
- Viscosity parameter, μ , used for the viscoplastic regularization of the concrete constitutive equations in an Abaqus/Standard analysis. This parameter is ignored in an Abaqus/Explicit analysis. The default value is 0.0.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteDamagedPlasticity object.

Exceptions

RangeError.

60.21.2 Members

The ConcreteDamagedPlasticity object has members with the same names and descriptions as the arguments to the **ConcreteDamagedPlasticity** method.

In addition, the ConcreteDamagedPlasticity object can have the following members:

Prototype

```
odb_ConcreteCompressionHardening concreteCompressionHardening() const;
odb_ConcreteTensionStiffening concreteTensionStiffening() const;
odb_ConcreteCompressionDamage concreteCompressionDamage() const;
odb_ConcreteTensionDamage concreteTensionDamage() const;
```

concreteCompressionHardening

A ConcreteCompressionHardening object.

concreteTensionStiffening

A ConcreteTensionStiffening object.

concreteCompressionDamage

A ConcreteCompressionDamage object.

concreteTensionDamage

A ConcreteTensionDamage object.

60.21.3 Corresponding analysis keywords

*CONCRETE DAMAGED PLASTICITY

60.22 ConcreteTensionDamage object

The ConcreteTensionDamage object specifies hardening for the concrete damaged plasticity model.

Access

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.concreteTensionDamage()
```

60.22.1 ConcreteTensionDamage(...)

This method creates a ConcreteTensionDamage object.

Path

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.ConcreteTensionDamage
```

Prototype

```
odb_ConcreteTensionDamage&
ConcreteTensionDamage(const odb_SequenceSequenceDouble& table,
                      double compressionRecovery,
                      const odb_String& type,
                      bool temperatureDependency,
                      int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

compressionRecovery

A Double specifying the value of the stiffness recovery factor, w_c , that determines the amount of compression stiffness that is recovered as loading changes from tension to compression. The default value is 1.0.

type

An odb_String specifying the type of tensile damage data. Set *type*="STRAIN" to specify the tensile damage variable as a function of cracking strain. Set *type*="DISPLACEMENT" to specify the tensile damage variable as a function of cracking displacement. Possible values are "STRAIN" and "DISPLACEMENT". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Tensile damage variable, d_t .
- Direct cracking strain, ϵ_t^{ck} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Tensile damage variable, d_t .
- Direct cracking displacement, u_t^{ck} .
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteTensionDamage object.

Exceptions

RangeError.

60.22.2 Members

The ConcreteTensionDamage object has members with the same names and descriptions as the arguments to the `ConcreteTensionDamage` method.

60.22.3 Corresponding analysis keywords

*CONCRETE TENSION DAMAGE

60.23 ConcreteTensionStiffening object

The ConcreteTensionStiffening object specifies hardening for the concrete damaged plasticity model.

Access

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.concreteTensionStiffening()
```

60.23.1 ConcreteTensionStiffening(...)

This method creates a ConcreteTensionStiffening object.

Path

```
materialApi.materials() [name].concreteDamagedPlasticity() \
.ConcreteTensionStiffening
```

Prototype

```
odb_ConcreteTensionStiffening&
ConcreteTensionStiffening(const odb_SequenceSequenceDouble& table,
                          bool rate,
                          const odb_String& type,
                          bool temperatureDependency,
                          int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

rate

A Boolean specifying whether the data depend on rate. The default value is false.

type

An odb_String specifying the type of postcracking behavior data. Possible values are:

- "STRAIN", specifying postfailure stress as a function of cracking strain.
- "DISPLACEMENT", specifying postfailure stress as a function of cracking displacement.
- "GFI", specifying failure stress as a function of the fracture energy.

The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Remaining direct stress after cracking, σ_t .
- Direct cracking strain, ϵ_t^{ck} .
- Direct cracking strain rate, $\dot{\epsilon}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Remaining direct stress after cracking, σ_t .
- Direct cracking displacement, u_t^{ck} .

- Direct cracking displacement rate, $\dot{u}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=GFI, the table data specify the following:

- Failure stress, σ_{t0} .
- Fracture energy, G_f .
- Direct cracking displacement rate, $\dot{u}_t^{ck}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ConcreteTensionStiffening object.

Exceptions

RangeError.

60.23.2 Members

The ConcreteTensionStiffening object has members with the same names and descriptions as the arguments to the **ConcreteTensionStiffening** method.

60.23.3 Corresponding analysis keywords

*CONCRETE TENSION STIFFENING

60.24 Conductivity object

The Conductivity object specifies thermal conductivity.

Access

```
materialApi.materials() [name].conductivity()
```

60.24.1 Conductivity(...)

This method creates a Conductivity object.

Path

```
materialApi.materials() [name].Conductivity
```

Prototype

```
odb_Conductivity&
Conductivity(const odb_SequenceSequenceDouble& table,
              const odb_String& type,
              bool temperatureDependency,
              int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of conductivity. Possible values are "ISOTROPIC", "ORTHOTROPIC", and "ANISOTROPIC". The default value is "ISOTROPIC".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Conductivity, k .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- k_{11} .
- k_{22} .
- k_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- k_{11} .
- k_{12} .
- k_{22} .
- k_{13} .
- k_{23} .
- k_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Conductivity object.

Exceptions

RangeError.

60.24.2 Members

The Conductivity object has members with the same names and descriptions as the arguments to the **Conductivity** method.

60.24.3 Corresponding analysis keywords

*CONDUCTIVITY

60.25 ContactArea object

A ContactArea object specifies a suboption of gasket thickness behavior when *variableUnits*=FORCE on the GasketThicknessBehavior object. The ContactArea object defines the contact area or contact width versus closure curves to output an average pressure through variable CS11.

Access

```
materialApi.materials() [name].gasketThicknessBehavior().contactArea()
```

60.25.1 ContactArea(...)

This method creates a ContactArea object.

Path

```
materialApi.materials() [name].gasketThicknessBehavior().ContactArea
```

Prototype

```
odb_ContactArea&
ContactArea(const odb_SequenceSequenceDouble& table,
            bool temperatureDependency,
            int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether contact area data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies included in the definition of the contact area data, in addition to temperature. The default value is 0.

Table data

- Contact area or width; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ContactArea object.

Exceptions

None.

60.25.2 Members

The ContactArea object has members with the same names and descriptions as the arguments to the `ContactArea` method.

60.25.3 Corresponding analysis keywords

*GASKET CONTACT AREA

60.26 Creep object

The Creep object defines a creep law.

Access

```
materialApi.materials() [name].creep()
```

60.26.1 Creep(...)

This method creates a Creep object.

Path

```
materialApi.materials() [name].Creep
```

Prototype

```
odb_Creep&
```

MATERIAL COMMANDS

```
Creep(const odb_SequenceSequenceDouble& table,  
      const odb_String& law,  
      bool temperatureDependency,  
      int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

law

An odb_String specifying the strain-hardening law. Possible values are "STRAIN", "TIME", "HYPERBOLIC_SINE", and "USER". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *law*=HYPERBOLIC_SINE, the table data specify the following:

- *A*.
- *B*.
- *n*.
- ΔH , if the data depend on temperature.
- *R*.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Creep object.

Exceptions

RangeError.

60.26.2 Members

The Creep object has members with the same names and descriptions as the arguments to the **Creep** method.

In addition, the Creep object can have the following members:

Prototype

```
odb_Ornl ornl() const;
odb_Potential potential() const;
```

ornl

An Ornl object.

potential

A Potential object.

60.26.3 Corresponding analysis keywords

*CREEP

60.27 CrushableFoam object

The CrushableFoam object specifies the crushable foam plasticity model.

Access

```
materialApi.materials() [name].crushableFoam()
```

60.27.1 CrushableFoam(...)

This method creates a CrushableFoam object.

Path

`materialApi.materials()[name].CrushableFoam`

Prototype

```
odb_CrushableFoam&
CrushableFoam(const odb_SequenceSequenceDouble& table,
               const odb_String& hardening,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

hardening

An odb_String specifying the type of hardening/softening definition. Possible values are "VOLUMETRIC" and "ISOTROPIC". The default value is "VOLUMETRIC".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=VOLUMETRIC, the table data specify the following:

- Ratio, k , of initial yield stress in uniaxial compression, σ_c^0 , to initial yield stress in hydrostatic compression, p_c^0 ; $0.0 < k < 3.0$.
- Ratio, k_t , of yield stress in hydrostatic tension, p_t , to initial yield stress in hydrostatic compression, p_c^0 . The default value is 1.0.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=ISOTROPIC, the table data specify the following:

- Ratio, k , of initial yield stress in uniaxial compression, σ_c^0 , to initial yield stress in hydrostatic compression, p_c^0 ; $0.0 \leq k \leq 3.0$.
- Plastic Poisson's ratio ν_p ; $-1 \leq \nu_p \leq 0.5$.
- Temperature, if the data depend on temperature.

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `CrushableFoam` object.

Exceptions

`RangeError`.

60.27.2 Members

The `CrushableFoam` object has members with the same names and descriptions as the arguments to the `CrushableFoam` method.

In addition, the `CrushableFoam` object can have the following members:

Prototype

```
odb_CrushableFoamHardening crushableFoamHardening() const;
odb_RateDependent rateDependent() const;
```

crushableFoamHardening

A `CrushableFoamHardening` object.

rateDependent

A `RateDependent` object.

60.27.3 Corresponding analysis keywords

*CRUSHABLE FOAM

60.28 CrushableFoamHardening object

The `CrushableFoamHardening` object specifies hardening for the crushable foam plasticity model.

Access

```
materialApi.materials() [name].crushableFoam().crushableFoamHardening()
```

60.28.1 CrushableFoamHardening(...)

This method creates a CrushableFoamHardening object.

Path

```
materialApi.materials() [name].crushableFoam().CrushableFoamHardening
```

Prototype

```
odb_CrushableFoamHardening&
CrushableFoamHardening(const odb_SequenceSequenceDouble& table,
                        bool temperatureDependency,
                        int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- The yield stress in uniaxial compression, σ_c .
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CrushableFoamHardening object.

Exceptions

RangeError.

60.28.2 Members

The `CrushableFoamHardening` object has members with the same names and descriptions as the arguments to the `CrushableFoamHardening` method.

60.28.3 Corresponding analysis keywords

*CRUSHABLE FOAM HARDENING

60.29 CycledPlastic object

The `CycledPlastic` object specifies cycled yield stress data for the ORNL constitutive model.

Access

```
materialApi.materials() [name].plastic().cycledPlastic()
```

60.29.1 CycledPlastic(...)

This method creates a `CycledPlastic` object.

Path

```
materialApi.materials() [name].plastic().CycledPlastic
```

Prototype

```
odb_CycledPlastic&
CycledPlastic(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional argument

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

Table data

MATERIAL COMMANDS

- Yield stress.
- Plastic strain.
- Temperature, if the data depend on temperature.

Return value

A CycledPlastic object.

Exceptions

None.

60.29.2 Members

The CycledPlastic object has members with the same names and descriptions as the arguments to the `CycledPlastic` method.

60.29.3 Corresponding analysis keywords

*CYCLED PLASTIC

60.30 CyclicHardening object

The CyclicHardening object defines the evolution of the elastic domain for the nonlinear isotropic/kinematic hardening model.

Access

```
materialApi.materials() [name].plastic().cyclicHardening()
```

60.30.1 CyclicHardening(...)

This method creates a CyclicHardening object.

Path

```
materialApi.materials() [name].plastic().CyclicHardening
```


Prototype

```
odb_CyclicHardening&
CyclicHardening(const odb_SequenceSequenceDouble& table,
                 bool temperatureDependency,
                 int dependencies,
                 bool parameters);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

parameters

A Boolean specifying whether material parameters are to be input directly. The default value is false.

Table data

- Equivalent stress.
- Q_{∞} (only if *parameters*=ON).
- Hardening parameter (only if *parameters*=ON).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A CyclicHardening object.

Exceptions

None.

60.30.2 Members

The CyclicHardening object has members with the same names and descriptions as the arguments to the `CyclicHardening` method.

60.30.3 Corresponding analysis keywords

*CYCLIC HARDENING

60.31 DamageEvolution object

The DamageEvolution object specifies material properties to define the evolution of damage.

Access

```
materialApi.materials() [name].ductileDamageInitiation() \
    .damageEvolution()
materialApi.materials() [name].fldDamageInitiation().damageEvolution()
materialApi.materials() [name].flsdDamageInitiation().damageEvolution()
materialApi.materials() [name].hashinDamageInitiation().damageEvolution()
materialApi.materials() [name].johnsonCookDamageInitiation() \
    .damageEvolution()
materialApi.materials() [name].maxeDamageInitiation().damageEvolution()
materialApi.materials() [name].maxpeDamageInitiation().damageEvolution()
materialApi.materials() [name].maxpsDamageInitiation().damageEvolution()
materialApi.materials() [name].maxsDamageInitiation().damageEvolution()
materialApi.materials() [name].mkDamageInitiation().damageEvolution()
materialApi.materials() [name].msfldDamageInitiation().damageEvolution()
materialApi.materials() [name].quadeDamageInitiation().damageEvolution()
materialApi.materials() [name].quadsDamageInitiation().damageEvolution()
materialApi.materials() [name].shearDamageInitiation().damageEvolution()
```

60.31.1 DamageEvolution(...)

This method creates a DamageEvolution object.

Path

```
materialApi.materials() [name].ductileDamageInitiation().DamageEvolution
```

```

materialApi.materials() [name].fldDamageInitiation().DamageEvolution
materialApi.materials() [name].flsdDamageInitiation().DamageEvolution
materialApi.materials() [name].hashinDamageInitiation().DamageEvolution
materialApi.materials() [name].johnsonCookDamageInitiation()\
.DamageEvolution
materialApi.materials() [name].maxeDamageInitiation().DamageEvolution
materialApi.materials() [name].maxpeDamageInitiation().DamageEvolution
materialApi.materials() [name].maxpsDamageInitiation().DamageEvolution
materialApi.materials() [name].maxsDamageInitiation().DamageEvolution
materialApi.materials() [name].mkDamageInitiation().DamageEvolution
materialApi.materials() [name].msfldDamageInitiation().DamageEvolution
materialApi.materials() [name].quadeDamageInitiation().DamageEvolution
materialApi.materials() [name].quadsDamageInitiation().DamageEvolution
materialApi.materials() [name].shearDamageInitiation().DamageEvolution

```

Prototype

```

odb_DamageEvolution&
DamageEvolution(const odb_String& type,
                 const odb_SequenceSequenceDouble& table,
                 const odb_String& degradation,
                 bool temperatureDependency,
                 int dependencies,
                 const odb_String& mixedModeBehavior,
                 const odb_String& modeMixRatio,
                 odb_Union power,
                 const odb_String& softening);

```

Required arguments

type

An odb_String specifying the type of damage evolution. Possible values are "DISPLACEMENT" and "ENERGY".

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

degradation

An odb_String specifying the degradation. Possible values are "MAXIMUM" and "MULTIPLICATIVE". The default value is "MAXIMUM".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

mixedModeBehavior

An odb_String specifying the mixed mode behavior. Possible values are "MODE_INDEPENDENT", "TABULAR", "POWER_LAW", and "BK". The default value is "MODE_INDEPENDENT".

modeMixRatio

An odb_String specifying the mode mix ratio. Possible values are "ENERGY" and "TRACTION". The default value is "ENERGY".

power

The string "NONE" or a Double specifying the exponent in the power law or the Benzeggagh-Kenane criterion that defines the variation of fracture energy with mode mix for cohesive elements. The default value is "NONE".

softening

An odb_String specifying the softening. Possible values are "LINEAR", "EXPONENTIAL", and "TABULAR". The default value is "LINEAR".

Table data

If *type*=DISPLACEMENT, and *softening*=LINEAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Equivalent total or plastic displacement at failure, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=LINEAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Fracture energy.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Equivalent total or plastic displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Total displacement at failure, measured from the time of damage initiation.
- Exponential law parameter.
- Appropriate mode mix ratio.

MATERIAL COMMANDS

- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=EXPONENTIAL, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Fracture energy.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=TABULAR, and *mixedModeBehavior*=MODE_INDEPENDENT, the table data specify the following:

- Damage variable.
- Equivalent total or plastic displacement, measured from the time of damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, and *softening*=TABULAR, and *mixedModeBehavior*=TABULAR, the table data specify the following:

- Damage variable.
- Equivalent total or plastic displacement, measured from the time of damage initiation.
- Appropriate mode mix ratio.
- Appropriate mode mix ratio (if relevant, for three-dimensional problems with anisotropic shear behavior).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, and *softening*=LINEAR or EXPONENTIAL, and *mixedModeBehavior*=POWER_LAW or BK, the table data specify the following:

- Normal mode fracture energy.
- Shear mode fracture energy for failure in the first shear direction.
- Shear mode fracture energy for failure in the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENERGY, *softening*=LINEAR and constructor for *DamageInitiation* =**HashinDamageInitiation** the table data specify the following:

- Fiber tensile fracture energy.
- Fiber compressive fracture energy.
- Matrix tensile fracture energy.
- Matrix compressive fracture energy.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A *DamageEvolution* object.

Exceptions

RangeError.

60.31.2 Members

The *DamageEvolution* object has members with the same names and descriptions as the arguments to the **DamageEvolution** method.

60.31.3 Corresponding analysis keywords

*DAMAGE EVOLUTION

60.32 DamageInitiation object

The DamageInitiation object specifies material properties to define the initiation of damage.

Access

```
materialApi.materials() [name].ductileDamageInitiation()
materialApi.materials() [name].fldDamageInitiation()
materialApi.materials() [name].flsdDamageInitiation()
materialApi.materials() [name].hashinDamageInitiation()
materialApi.materials() [name].johnsonCookDamageInitiation()
materialApi.materials() [name].maxeDamageInitiation()
materialApi.materials() [name].maxpeDamageInitiation()
materialApi.materials() [name].maxpsDamageInitiation()
materialApi.materials() [name].maxsDamageInitiation()
materialApi.materials() [name].mkDamageInitiation()
materialApi.materials() [name].msfldDamageInitiation()
materialApi.materials() [name].quadeDamageInitiation()
materialApi.materials() [name].quadsDamageInitiation()
materialApi.materials() [name].shearDamageInitiation()
```

60.32.1 DuctileDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials() [name].DuctileDamageInitiation
```

Prototype

```
odb_DamageInitiation&
DuctileDamageInitiation(const odb_SequenceSequenceDouble& table,
                        const odb_String& definition,
                        double feq,
                        double fnn,
                        double fnt,
                        int frequency,
                        double ks,
                        int numberImperfections,
                        bool temperatureDependency,
```



```

    int dependencies,
    double alpha,
    double omega,
    double tolerance,
    const odb_String& direction);

```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

MATERIAL COMMANDS

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Table data

If constructor is **DuctileDamageInitiation**, the table data specify the following:

- Equivalent fracture strain at damage initiation.
- Stress triaxiality.
- Strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **FldDamageInitiation**, the table data specify the following:

- Major principal strain at damage initiation.
- Minor principal strain.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **FlsdDamageInitiation**, the table data specify the following:

- Major principal stress at damage initiation.
- Minor principal stress.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **JohnsonCookDamageInitiation**, the table data specify the following:

- Johnson-Cook failure parameter D1.
- Johnson-Cook failure parameter D2.
- Johnson-Cook failure parameter D3.

- Johnson-Cook failure parameter D4.
- Johnson-Cook failure parameter D5.
- Melting temperature.
- Transition temperature.
- Reference strain rate.

If constructor **MkDamageInitiation**, the table data specify the following:

- Flaw size relative to nominal thickness of the section.
- Angle (in degrees) with respect to the 1-direction of the local material orientation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MsfldDamageInitiation** and *definition*=MSFLD, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.
- Equivalent plastic strain at initiation of localized necking.
- Ratio of minor to major principal strains.
- Equivalent plastic strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MsfldDamageInitiation** and *definition*=FLD, the table data specify the following:

- Major principal strain at initiation of localized necking.
- Equivalent plastic strain at initiation of localized necking.
- Ratio of minor to major principal strains.
- Equivalent plastic strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **QuadeDamageInitiation** or **MaxeDamageInitiation**, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.

MATERIAL COMMANDS

- Nominal strain at damage initiation in a shear-only mode that involves separation only along the first shear direction.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **QuadsDamageInitiation** or **MaxsDamageInitiation**, the table data specify the following:

- Nominal strain at damage initiation in a normal-only mode.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the first shear direction.
- Nominal strain at damage initiation in a shear-only mode that involves separation only along the second shear direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MaxpeDamageInitiation**, the table data specify the following:

- Maximum principal strain at damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **MaxpsDamageInitiation**, the table data specify the following:

- Maximum principal stress at damage initiation.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **ShearDamageInitiation**, the table data specify the following:

- Equivalent fracture strain at damage initiation.
- Shear stress ratio.
- Strain rate.

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If constructor is **HashinDamageInitiation**, the table data specify the following:

- Fiber tensile strength.
- Fiber compressive strength.
- Matrix tensile strength.
- Matrix compressive strength.
- Longitudinal shear strength.
- Transverse shear strength.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.2 FldDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials() [name].FldDamageInitiation
```

Prototype

```
odb_DamageInitiation&
FldDamageInitiation(const odb_SequenceSequenceDouble& table,
                    const odb_String& definition,
                    double feq,
                    double fnn,
                    double fnt,
                    int frequency,
```

MATERIAL COMMANDS

```
double ks,  
int numberImperfections,  
bool temperatureDependency,  
int dependencies,  
double alpha,  
double omega,  
double tolerance,  
const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

freq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.3 FlsdDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].FlsdDamageInitiation
```

Prototype

```
odb_DamageInitiation&
FlsdDamageInitiation(const odb_SequenceSequenceDouble& table,
                     const odb_String& definition,
                     double feq,
                     double fnn,
                     double fnt,
                     int frequency,
                     double ks,
                     int numberImperfections,
                     bool temperatureDependency,
                     int dependencies,
                     double alpha,
```

MATERIAL COMMANDS

```
double omega,  
double tolerance,  
const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.4 JohnsonCookDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].JohnsonCookDamageInitiation
```

Prototype

```
odb_DamageInitiation&
JohnsonCookDamageInitiation(const odb_SequenceSequenceDouble& table,
                             const odb_String& definition,
                             double feq,
                             double fnn,
                             double fnt,
                             int frequency,
                             double ks,
                             int numberImperfections,
                             bool temperatureDependency,
                             int dependencies,
                             double alpha,
                             double omega,
                             double tolerance,
                             const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.5 MaxeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`materialApi.materials()[name].MaxeDamageInitiation`

Prototype

```
odb_DamageInitiation&
MaxeDamageInitiation(const odb_SequenceSequenceDouble& table,
                     const odb_String& definition,
                     double feq,
                     double fnn,
                     double fnt,
                     int frequency,
                     double ks,
                     int numberImperfections,
                     bool temperatureDependency,
                     int dependencies,
                     double alpha,
                     double omega,
                     double tolerance,
                     const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.6 MaxsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`materialApi.materials() [name].MaxsDamageInitiation`

Prototype

```
odb_DamageInitiation&
MaxsDamageInitiation(const odb_SequenceSequenceDouble& table,
                     const odb_String& definition,
                     double feq,
                     double fnn,
                     double fnt,
                     int frequency,
                     double ks,
                     int numberImperfections,
                     bool temperatureDependency,
                     int dependencies,
                     double alpha,
                     double omega,
                     double tolerance,
                     const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

MATERIAL COMMANDS

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.7 MkDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].MkDamageInitiation
```

Prototype

```
odb_DamageInitiation&
MkDamageInitiation(const odb_SequenceSequenceDouble& table,
                    const odb_String& definition,
                    double feq,
                    double fnn,
                    double fnt,
                    int frequency,
                    double ks,
                    int numberImperfections,
                    bool temperatureDependency,
                    int dependencies,
                    double alpha,
                    double omega,
                    double tolerance,
                    const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

MATERIAL COMMANDS

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.8 MsfldDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials() [name].MsfldDamageInitiation
```

Prototype

```
odb_DamageInitiation&
MsfldDamageInitiation(const odb_SequenceSequenceDouble& table,
                      const odb_String& definition,
                      double feq,
                      double fnn,
                      double fnt,
                      int frequency,
                      double ks,
                      int numberImperfections,
                      bool temperatureDependency,
                      int dependencies,
                      double alpha,
                      double omega,
                      double tolerance,
                      const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

MATERIAL COMMANDS

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.9 QuadeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials() [name].QuadeDamageInitiation
```

Prototype

```
odb_DamageInitiation&
QuadeDamageInitiation(const odb_SequenceSequenceDouble& table,
                      const odb_String& definition,
                      double feq,
                      double fnn,
                      double fnt,
                      int frequency,
                      double ks,
                      int numberImperfections,
                      bool temperatureDependency,
                      int dependencies,
                      double alpha,
                      double omega,
                      double tolerance,
                      const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

MATERIAL COMMANDS

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.10 QuadsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`materialApi.materials()[name].QuadsDamageInitiation`

Prototype

```

odb_DamageInitiation&
QuadsDamageInitiation(const odb_SequenceSequenceDouble& table,
                      const odb_String& definition,
                      double feq,
                      double fnn,
                      double fnt,
                      int frequency,
                      double ks,
                      int numberImperfections,
                      bool temperatureDependency,
                      int dependencies,
                      double alpha,
                      double omega,
                      double tolerance,
                      const odb_String& direction);

```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments*definition*

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

MATERIAL COMMANDS

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.11 MaxpeDamageInitiation(...)

This method creates a DamageInitiation object.

Path

`materialApi.materials()[name].MaxpeDamageInitiation`

Prototype

```
odb_DamageInitiation&  
MaxpeDamageInitiation(const odb_SequenceSequenceDouble& table,  
                       const odb_String& definition,
```

```

double feq,
double fnn,
double fnt,
int frequency,
double ks,
int numberImperfections,
bool temperatureDependency,
int dependencies,
double alpha,
double omega,
double tolerance,
const odb_String& direction);

```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

MATERIAL COMMANDS

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.12 MaxpsDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].MaxpsDamageInitiation
```

Prototype

```
odb_DamageInitiation&  
MaxpsDamageInitiation(const odb_SequenceSequenceDouble& table,  
                       const odb_String& definition,  
                       double feq,  
                       double fnn,  
                       double fnt,  
                       int frequency,  
                       double ks,
```



```

    int numberImperfections,
    bool temperatureDependency,
    int dependencies,
    double alpha,
    double omega,
    double tolerance,
    const odb_String& direction);

```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments*definition*

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

MATERIAL COMMANDS

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.13 ShearDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].ShearDamageInitiation
```

Prototype

```
odb_DamageInitiation&  
ShearDamageInitiation(const odb_SequenceSequenceDouble& table,  
                      const odb_String& definition,  
                      double feq,  
                      double fnn,  
                      double fnt,  
                      int frequency,  
                      double ks,  
                      int numberImperfections,  
                      bool temperatureDependency,  
                      int dependencies,  
                      double alpha,
```

```
double omega,
double tolerance,
const odb_String& direction);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

MATERIAL COMMANDS

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.14 HashinDamageInitiation(...)

This method creates a DamageInitiation object.

Path

```
materialApi.materials()[name].HashinDamageInitiation
```

Prototype

```
odb_DamageInitiation&  
HashinDamageInitiation(const odb_SequenceSequenceDouble& table,  
                        const odb_String& definition,  
                        double feq,  
                        double fnn,  
                        double fnt,  
                        int frequency,  
                        double ks,  
                        int numberImperfections,  
                        bool temperatureDependency,  
                        int dependencies,  
                        double alpha,  
                        double omega,  
                        double tolerance,  
                        const odb_String& direction);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

Optional arguments*definition*

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

MATERIAL COMMANDS

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

Return value

A DamageInitiation object.

Exceptions

RangeError.

60.32.15 Members

The DamageInitiation object can have the following members:

Prototype

```
odb_String definition() const;
double feq() const;
double fnn() const;
double fnt() const;
int frequency() const;
double ks() const;
int numberImperfections() const;
bool temperatureDependency() const;
int dependencies() const;
double alpha() const;
double omega() const;
double tolerance() const;
odb_String direction() const;
odb_SequenceSequenceDouble table() const;
odb_DamageEvolution damageEvolution() const;
odb_DamageStabilization damageStabilization() const;
odb_DamageStabilizationCohesive damageStabilizationCohesive() const;
```

definition

An odb_String specifying the damage initiation definition. Possible values are "FLD" and "MSFLD". The default value is "MSFLD".

feq

A Double specifying the critical value of the deformation severity index for equivalent plastic strains. The default value is 10.0.

fnn

A Double specifying the critical value of the deformation severity index for strains normal to the groove direction. The default value is 10.0.

fnt

A Double specifying the critical value of the deformation severity index for shear strains. The default value is 10.0.

frequency

An Int specifying the frequency, in increments, at which the Marciniak-Kuczynski analysis is going to be performed. The default value is 1.

ks

A Double specifying the value of Ks. The default value is 0.0.

numberImperfections

An Int specifying the number of imperfections to be considered for the evaluation of the Marciniak-Kuczynski analysis. These imperfections are assumed to be equally spaced in the angular direction. The default value is 4.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

alpha

A Double specifying the value of the coefficient that will multiply the shear contribution to the Hashin's fiber initiation criterion. The default value is 0.0.

omega

A Double specifying the factor used for filtering the ratio of principal strain rates used for the evaluation of the MSFLD damage initiation criterion. The default value is 1.0.

tolerance

A Double specifying the tolerance within which the damage initiation criterion must be satisfied. The default value is 0.05.

direction

An odb_String specifying the damage initiation direction. Possible values are "NMORI" and "TMORI". The default value is "NMORI".

table

An odb_SequenceSequenceDouble specifying the items described in the "Table data" section.

damageEvolution

A DamageEvolution object.

damageStabilization

A DamageStabilization object.

damageStabilizationCohesive

A DamageStabilizationCohesive object.

60.32.16 Corresponding analysis keywords

*DAMAGE INITIATION

60.33 DamageStabilization object

The DamageStabilization object specifies the viscosity coefficients for the damage model for fiber-reinforced materials.

Access

```
materialApi.materials() [name].ductileDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].fldDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].flsdDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].hashinDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].johnsonCookDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].maxeDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].maxpeDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].maxpsDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].maxsDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].mkDamageInitiation().damageStabilization()
materialApi.materials() [name].msfldDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].quadeDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].quadsDamageInitiation()\
.damageStabilization()
materialApi.materials() [name].shearDamageInitiation()\
.damageStabilization()
```


60.33.1 DamageStabilization(...)

This method creates a DamageStabilization object.

Path

```
materialApi.materials() [name].ductileDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].fldDamageInitiation().DamageStabilization
materialApi.materials() [name].flsdDamageInitiation().DamageStabilization
materialApi.materials() [name].hashinDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].johnsonCookDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].maxeDamageInitiation().DamageStabilization
materialApi.materials() [name].maxpeDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].maxpsDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].maxsDamageInitiation().DamageStabilization
materialApi.materials() [name].mkDamageInitiation().DamageStabilization
materialApi.materials() [name].msfldDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].quadeDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].quadsDamageInitiation()\
.DamageStabilization
materialApi.materials() [name].shearDamageInitiation()\
.DamageStabilization
```

Prototype

```
odb_DamageStabilization&
DamageStabilization(double fiberTensileCoeff,
                    double fiberCompressiveCoeff,
                    double matrixTensileCoeff,
                    double matrixCompressiveCoeff);
```

Required arguments

fiberTensileCoeff

A Double specifying the viscosity coefficient for the fiber tensile failure mode.

MATERIAL COMMANDS

fiberCompressiveCoeff

A Double specifying the viscosity coefficient for the fiber compressive failure mode.

matrixTensileCoeff

A Double specifying the viscosity coefficient for the matrix tensile failure mode.

matrixCompressiveCoeff

A Double specifying the viscosity coefficient for the matrix compressive failure mode.

Optional arguments

None.

Return value

A DamageStabilization object.

Exceptions

RangeError.

60.33.2 Members

The DamageStabilization object has members with the same names and descriptions as the arguments to the **DamageStabilization** method.

60.33.3 Corresponding analysis keywords

*DAMAGE STABILIZATION

60.34 DamageStabilizationCohesive object

The DamageStabilizationCohesive object specifies the viscosity coefficients for the damage model for surface-based cohesive behavior or enriched cohesive behavior.

Access

```
materialApi.materials() [name].ductileDamageInitiation()\n.damageStabilizationCohesive()\nmaterialApi.materials() [name].fldDamageInitiation()\n.damageStabilizationCohesive()\nmaterialApi.materials() [name].flsdDamageInitiation()\n.damageStabilizationCohesive()
```

```

materialApi.materials() [name].hashinDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].johnsonCookDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].maxeDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].maxpeDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].maxpsDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].maxsDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].mkDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].msfldDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].quadeDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].quadsDamageInitiation()\
.damageStabilizationCohesive()
materialApi.materials() [name].shearDamageInitiation()\
.damageStabilizationCohesive()

```

60.34.1 DamageStabilizationCohesive(...)

This method creates a DamageStabilizationCohesive object.

Path

```

materialApi.materials() [name].ductileDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].fldDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].flsdDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].hashinDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].johnsonCookDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].maxeDamageInitiation()\
.DamageStabilizationCohesive
materialApi.materials() [name].maxpeDamageInitiation()\

```

```
.DamageStabilizationCohesive  
materialApi.materials() [name] .maxpsDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .maxsDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .mkDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .msfldDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .quadeDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .quadsDamageInitiation() \  
.DamageStabilizationCohesive  
materialApi.materials() [name] .shearDamageInitiation() \  
.DamageStabilizationCohesive
```

Prototype

```
odb_DamageStabilizationCohesive &  
DamageStabilizationCohesive(odb_Union cohesiveCoeff);
```

Required arguments

None.

Optional argument

cohesiveCoeff

The string "NONE" or a Double specifying the viscosity coefficient. The default value is "NONE".

Return value

A DamageStabilizationCohesive object.

Exceptions

RangeError.

60.34.2 Members

The DamageStabilizationCohesive object has members with the same names and descriptions as the arguments to the **DamageStabilizationCohesive** method.

60.34.3 Corresponding analysis keywords

*DAMAGE STABILIZATION

60.35 Damping object

The Damping object specifies material damping.

Access

```
materialApi.materials() [name].damping()
```

60.35.1 Damping(...)

This method creates a Damping object.

Path

```
materialApi.materials() [name].Damping
```

Prototype

```
odb_Damping&
Damping(double alpha,
         double beta,
         double composite,
         double structural);
```

Required arguments

None.

Optional arguments

alpha

A Double specifying the α_R factor to create mass proportional damping in direct-integration and explicit dynamics. The default value is 0.0.

beta

A Double specifying the β_R factor to create stiffness proportional damping in direct-integration and explicit dynamics. The default value is 0.0.

composite

A Double specifying the fraction of critical damping to be used with this material in calculating composite damping factors for the modes (for use in modal dynamics). The default value is 0.0.

This argument applies only to Abaqus/Standard analyses.

structural

A Double specifying the structural factor to create material damping in direct-integration and explicit dynamics. The default value is 0.0.

Return value

A Damping object.

Exceptions

RangeError.

60.35.2 Members

The Damping object has members with the same names and descriptions as the arguments to the **Damping** method.

60.35.3 Corresponding analysis keywords

*DAMPING

60.36 DeformationPlasticity object

The DeformationPlasticity object specifies the deformation plasticity model.

Access

```
materialApi.materials() [name].deformationPlasticity()
```

60.36.1 DeformationPlasticity(...)

This method creates a DeformationPlasticity object.

Path

```
materialApi.materials() [name].DeformationPlasticity
```

Prototype

```
odb_DeformationPlasticity&  
DeformationPlasticity(const odb_SequenceSequenceDouble& table,
```

```
bool temperatureDependency);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional argument*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is false.

Table data

- Young's modulus, E .
- Poisson's ratio, ν .
- Yield stress, σ^0 .
- Exponent, n .
- Yield offset, α .
- Temperature, if the data depend on temperature.

Return value

A DeformationPlasticity object.

Exceptions

RangeError.

60.36.2 Members

The DeformationPlasticity object has members with the same names and descriptions as the arguments to the **DeformationPlasticity** method.

60.36.3 Corresponding analysis keywords

*DEFORMATION PLASTICITY

60.37 Density object

The Density object specifies the material density.

Access

```
materialApi.materials() [name].density()
```

60.37.1 Density(...)

This method creates a Density object.

Path

```
materialApi.materials() [name].Density
```

Prototype

```
odb_Density&
Density(const odb_SequenceSequenceDouble& table,
        bool temperatureDependency,
        int dependencies,
        const odb_String& distributionType,
        const odb_String& fieldName);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

distributionType

An odb_String specifying how the density is distributed spatially. Possible values are "UNIFORM", "ANALYTICAL_FIELD", and "DISCRETE_FIELD". The default value is "UNIFORM".

fieldName

An odb_String specifying the name of the AnalyticalField or DiscreteField object associated with this material option. The *fieldName* argument applies only when *distributionType*="ANALYTICAL_FIELD" or *distributionType*="DISCRETE_FIELD". The default value is an empty string.

Table data

- The mass density.

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Density object.

Exceptions

RangeError.

60.37.2 Members

The Density object has members with the same names and descriptions as the arguments to the **Density** method.

60.37.3 Corresponding analysis keywords

*DENSITY

60.38 Depvar object

The Depvar object specifies solution-dependent state variables.

Access

```
materialApi.materials() [name].depvar()
```

60.38.1 Depvar(...)

This method creates a Depvar object.

Path

```
materialApi.materials() [name].Depvar
```

Prototype

```
odb_Depvar&  
Depvar(int deleteVar,  
        int n);
```

Required arguments

None.

Optional arguments

deleteVar

An Int specifying the state variable number controlling the element deletion flag. The default value is 0.

This argument applies only to Abaqus/Explicit analyses.

n

An Int specifying the number of solution-dependent state variables required at each integration point. The default value is 0.

Return value

A Depvar object.

Exceptions

RangeError.

60.38.2 Members

The Depvar object has members with the same names and descriptions as the arguments to the **Depvar** method.

60.38.3 Corresponding analysis keywords

*DEPVAR

60.39 DetonationPoint object

A DetonationPoint object specifies a suboption of the Eos object. The DetonationPoint object defines either isotropic linear elastic shear or linear viscous shear behavior for a hydrodynamic material.

Access

```
materialApi.materials() [name].eos().detonationPoint()
```

60.39.1 DetonationPoint(...)

This method creates a DetonationPoint object.

Path

```
materialApi.materials() [name].eos().DetonationPoint
```

Prototype

```
odb_DetonationPoint&  
DetonationPoint(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- X value for coordinate of detonation point.
- Y value for coordinate of detonation point.
- Z value for coordinate of detonation point.
- Detonation delay time.

Return value

A DetonationPoint object.

Exceptions

None.

60.39.2 Members

The DetonationPoint object has members with the same names and descriptions as the arguments to the **DetonationPoint** method.

60.39.3 Corresponding analysis keywords

*DETONATION POINT

60.40 Dielectric object

The Dielectric object specifies dielectric material properties.

Access

`materialApi.materials() [name].dielectric()`

60.40.1 Dielectric(...)

This method creates a Dielectric object.

Path

`materialApi.materials() [name].Dielectric`

Prototype

```
odb_Dielectric&
Dielectric(const odb_SequenceSequenceDouble& table,
           const odb_String& type,
           bool frequencyDependency,
           bool temperatureDependency,
           int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the dielectric behavior. Possible values are "ISOTROPIC", "ORTHOTROPIC", and "ANISOTROPIC". The default value is "ISOTROPIC".

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Dielectric constant.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{11}^{φ} .
- D_{22}^{φ} .
- D_{33}^{φ} .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{11}^{φ} .
- D_{12}^{φ} .
- D_{22}^{φ} .
- D_{13}^{φ} .
- D_{23}^{φ} .
- D_{33}^{φ} .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Dielectric object.

Exceptions

None.

60.40.2 Members

The Dielectric object has members with the same names and descriptions as the arguments to the `Dielectric` method.

60.40.3 Corresponding analysis keywords

*DIELECTRIC

60.41 Diffusivity object

The Diffusivity object specifies mass diffusivity.

Access

```
materialApi.materials() [name].diffusivity()
```

60.41.1 Diffusivity(...)

This method creates a Diffusivity object.

Path

```
materialApi.materials() [name].Diffusivity
```

Prototype

```
odb_Diffusivity&  
Diffusivity(const odb_SequenceSequenceDouble& table,  
            const odb_String& type,  
            const odb_String& law,  
            bool temperatureDependency,
```

```
int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of diffusivity. Possible values are "ISOTROPIC", "ORTHOTROPIC", and "ANISOTROPIC". The default value is "ISOTROPIC".

law

An odb_String specifying the diffusion behavior. Possible values are "GENERAL" and "FICK". The default value is "GENERAL".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Diffusivity, D .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{11} .
- D_{22} .
- D_{33} .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{11} .
- D_{12} .

MATERIAL COMMANDS

- D_{22} .
- D_{13} .
- D_{23} .
- D_{33} .
- Concentration, c .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Diffusivity object.

Exceptions

RangeError.

60.41.2 Members

The Diffusivity object has members with the same names and descriptions as the arguments to the **Diffusivity** method.

In addition, the Diffusivity object can have the following members:

Prototype

```
odb_PressureEffect pressureEffect() const;  
odb_SoretEffect soretEffect() const;
```

pressureEffect

A PressureEffect object.

soretEffect

A SoretEffect object.

60.41.3 Corresponding analysis keywords

*DIFFUSIVITY

60.42 DruckerPrager object

The DruckerPrager object specifies the extended Drucker-Prager plasticity model.

Access

```
materialApi.materials() [name].druckerPrager()
```

60.42.1 DruckerPrager(...)

This method creates a DruckerPrager object.

Path

```
materialApi.materials() [name].DruckerPrager
```

Prototype

```
odb_DruckerPrager&
DruckerPrager(const odb_SequenceSequenceDouble& table,
               const odb_String& shearCriterion,
               double eccentricity,
               bool testData,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

shearCriterion

An odb_String specifying the yield criterion. Possible values are "LINEAR", "HYPERBOLIC", and "EXPONENTIAL". The default value is "LINEAR".

This argument applies only to Abaqus/Standard analyses. Only the linear Drucker-Prager model is available in Abaqus/Explicit analyses.

eccentricity

A Double specifying the flow potential eccentricity, ϵ , a small positive number that defines the rate at which the hyperbolic flow potential approaches its asymptote. The default value is 0.1.

This argument applies only to Abaqus/Standard analyses.

MATERIAL COMMANDS

testData

A Boolean specifying whether the material constants for the exponent model are to be computed by Abaqus/Standard from triaxial test data at different levels of confining pressure. The default value is false.

This argument is valid only if *shearCriterion*="EXPONENTIAL".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *shearCriterion*=LINEAR (the only option allowed in an Abaqus/Explicit analysis), the table data specify the following:

- Material angle of friction, β , in the p - t plane. Give the value in degrees.
- K , the ratio of the flow stress in triaxial tension to the flow stress in triaxial compression. $0.778 \leq K \leq 1.0$. If the default value of 0.0 is accepted, a value of 1.0 is assumed.
- Dilation angle, ψ , in the p - t plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shearCriterion*=HYPERBOLIC, the table data specify the following:

- Material angle of friction, β , at high confining pressure in the p - q plane. Give the value in degrees.
- Initial hydrostatic tension strength, $p_t|_0$.
- Dilation angle, ψ , at high confining pressure in the p - q plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shearCriterion*=EXPONENTIAL, the table data specify the following:

- Dilation angle, ψ , at high confining pressure in the p - q plane. Give the value in degrees.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPrager object.

Exceptions

RangeError.

60.42.2 Members

The DruckerPrager object has members with the same names and descriptions as the arguments to the **DruckerPrager** method.

In addition, the DruckerPrager object can have the following members:

Prototype

```
odb_DruckerPragerCreep druckerPragerCreep() const;
odb_DruckerPragerHardening druckerPragerHardening() const;
odb_RateDependent rateDependent() const;
odb_TriaxialTestData triaxialTestData() const;
```

druckerPragerCreep

A DruckerPragerCreep object.

druckerPragerHardening

A DruckerPragerHardening object.

rateDependent

A RateDependent object.

triaxialTestData

A TriaxialTestData object.

60.42.3 Corresponding analysis keywords

*DRUCKER PRAGER

60.43 DruckerPragerCreep object

The DruckerPragerCreep object specifies creep for Drucker-Prager plasticity models.

Access

`materialApi.materials() [name].druckerPrager().druckerPragerCreep()`

60.43.1 DruckerPragerCreep(...)

This method creates a DruckerPragerCreep object.

Path

`materialApi.materials() [name].druckerPrager().DruckerPragerCreep`

Prototype

```
odb_DruckerPragerCreep&
DruckerPragerCreep(const odb_SequenceSequenceDouble& table,
                   const odb_String& law,
                   bool temperatureDependency,
                   int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

law

An odb_String specifying the type of data defining the creep law. Possible values are:

- "STRAIN", specifying a strain-hardening power law.
- "TIME", specifying a time-hardening power law.
- "SINGHM", specifying a Singh-Mitchell type law.
- "USER", specifying the creep law is input from user subroutine **CREEP**.

The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=TIME or *law*=STRAIN, the table data specify the following:

- A . (Units of $F^{-n} L^{2n} T^{-1-m}$.)
- n .

- m .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $law=SINGHM$, the table data specify the following:

- A . (Units of T^{-1} .)
- α . (Units of $F^{-1}L^2$.)
- m .
- t_1 . (Units of T .)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPragerCreep object.

Exceptions

RangeError.

60.43.2 Members

The DruckerPragerCreep object has members with the same names and descriptions as the arguments to the **DruckerPragerCreep** method.

60.43.3 Corresponding analysis keywords

*DRUCKER PRAGER CREEP

60.44 DruckerPragerHardening object

The DruckerPragerHardening object specifies hardening for Drucker-Prager plasticity models.

Access

`materialApi.materials() [name].druckerPrager().druckerPragerHardening()`

60.44.1 DruckerPragerHardening(...)

This method creates a DruckerPragerHardening object.

Path

`materialApi.materials() [name].druckerPrager().DruckerPragerHardening`

Prototype

```
odb_DruckerPragerHardening&
DruckerPragerHardening(const odb_SequenceSequenceDouble& table,
                        const odb_String& type,
                        bool rate,
                        bool temperatureDependency,
                        int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of data defining the hardening behavior. Possible values are "COMPRESSION", "TENSION", and "SHEAR". The default value is "COMPRESSION".

rate

A Boolean specifying whether the data depend on rate. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Yield stress.
- Absolute value of the corresponding plastic strain. (The first tabular value entered must always be zero.)
- Equivalent plastic strain rate, $\dot{\epsilon}^{pl}$, for which this hardening curve applies.

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A DruckerPragerHardening object.

Exceptions

RangeError.

60.44.2 Members

The DruckerPragerHardening object has members with the same names and descriptions as the arguments to the **DruckerPragerHardening** method.

60.44.3 Corresponding analysis keywords

*DRUCKER PRAGER HARDENING

60.45 Elastic object

The Elastic object specifies elastic material properties.

Access

```
materialApi.materials() [name].elastic()
```

60.45.1 Elastic(...)

This method creates an Elastic object.

Path

```
materialApi.materials() [name].Elastic
```

Prototype

```
odb_Elastic&  
Elastic(const odb_SequenceSequenceDouble& table,  
        const odb_String& type,  
        bool noCompression,  
        bool noTension,  
        bool temperatureDependency,  
        int dependencies,  
        const odb_String& moduli);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of elasticity data provided. Possible values are:

- "ISOTROPIC"
- "ORTHOTROPIC"
- "ANISOTROPIC"
- "ENGINEERING_CONSTANTS"
- "LAMINA"
- "TRACTION"
- "COUPLED_TRACTION"
- "SHORT_FIBER"
- "SHEAR"

The default value is "ISOTROPIC".

noCompression

A Boolean specifying whether compressive stress is allowed. The default value is false.

noTension

A Boolean specifying whether tensile stress is allowed. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

moduli

An odb_String specifying the time-dependence of the elastic material constants. Possible values are "INSTANTANEOUS" and "LONG_TERM". The default value is "LONG_TERM".

Table data

If *type*=ISOTROPIC, the table data specify the following:

- The Young's modulus, E .
- The Poisson's ratio, ν .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=SHEAR, the table data specify the following:

- The shear modulus, G .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ENGINEERING_CONSTANTS, the table data specify the following:

- E_1 .
- E_2 .
- E_3 .
- ν_{12} .
- ν_{13} .
- ν_{23} .
- G_{12} .
- G_{13} .
- G_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=LAMINA, the table data specify the following:

- E_1 .
- E_2 .
- ν_{12} .
- G_{12} .
- G_{13} . This shear modulus is needed to define transverse shear behavior in shells.
- G_{23} . This shear modulus is needed to define transverse shear behavior in shells.

MATERIAL COMMANDS

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- D_{1111} .
- D_{1122} .
- D_{2222} .
- D_{1133} .
- D_{2233} .
- D_{3333} .
- D_{1212} .
- D_{1313} .
- D_{2323} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- D_{1111} .
- D_{1122} .
- D_{2222} .
- D_{1133} .
- D_{2233} .
- D_{3333} .
- D_{1112} .
- D_{2212} .
- D_{3312} .
- D_{1212} .
- D_{1113} .
- D_{2213} .
- D_{3313} .
- D_{1213} .
- D_{1313} .
- D_{1123} .

- D_{2223} .
- D_{3323} .
- D_{1223} .
- D_{1323} .
- D_{2323} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $type=TRACTION$, the table data specify the following:

- E for warping elements; E_{nn} for cohesive elements.
- G_1 for warping elements; E_{ss} for cohesive elements.
- G_2 for warping elements; E_{tt} for cohesive elements.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $type=COUPLED_TRACTION$, the table data specify the following:

- E_{nn} .
- E_{ss} .
- E_{tt} .
- E_{ns} .
- E_{nt} .
- E_{st} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If $type=SHORT_FIBER$, there is no table data.

Return value

An Elastic object.

Exceptions

RangeError.

60.45.2 Members

The Elastic object has members with the same names and descriptions as the arguments to the **Elastic** method.

In addition, the Elastic object can have the following members:

Prototype

```
odb_FailStress failStress() const;
odb_FailStrain failStrain() const;
```

failStress

A FailStress object.

failStrain

A FailStrain object.

60.45.3 Corresponding analysis keywords

*ELASTIC

60.46 ElectricalConductivity object

The ElectricalConductivity object specifies electrical conductivity.

Access

```
materialApi.materials() [name].electricalConductivity()
```

60.46.1 ElectricalConductivity(...)

This method creates an ElectricalConductivity object.

Path

```
materialApi.materials() [name].ElectricalConductivity
```

Prototype

```
odb_ElectricalConductivity&
ElectricalConductivity(const odb_SequenceSequenceDouble& table,
                       const odb_String& type,
```

```
bool frequencyDependency,
bool temperatureDependency,
int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*type*

An odb_String specifying the type of electrical conductivity. Possible values are "ISOTROPIC", "ORTHOTROPIC", and "ANISOTROPIC". The default value is "ISOTROPIC".

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Electrical conductivity.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- σ_{11}^E .
- σ_{22}^E .
- σ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- σ_{11}^E .

MATERIAL COMMANDS

- σ_{12}^E .
- σ_{22}^E .
- σ_{13}^E .
- σ_{23}^E .
- σ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

An ElectricalConductivity object.

Exceptions

RangeError.

60.46.2 Members

The ElectricalConductivity object has members with the same names and descriptions as the arguments to the **ElectricalConductivity** method.

60.46.3 Corresponding analysis keywords

*ELECTRICAL CONDUCTIVITY

60.47 Eos object

The Eos object specifies an equation of state model.

Access

```
materialApi.materials() [name].eos()
```

60.47.1 Eos(...)

This method creates an Eos object.

Path

`materialApi.materials() [name].Eos`

Prototype

```
odb_Eos&
Eos(const odb_String& type,
    bool temperatureDependency,
    int dependencies,
    double detonationEnergy,
    const odb_SequenceSequenceDouble& solidTable,
    const odb_SequenceSequenceDouble& gasTable,
    const odb_SequenceSequenceDouble& reactionTable,
    const odb_SequenceSequenceDouble& gasSpecificTable,
    const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments

type

An odb_String specifying the equation of state. Possible values are "USUP", "JWL", "IDEALGAS", "TABULAR", and "IGNITIONANDGROWTH". The default value is "IDEALGAS".

temperatureDependency

A Boolean specifying whether the data in *gasSpecificTable* depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies for the data in *gasSpecificTable*. The default value is 0.

detonationEnergy

A Double specifying the detonation energy text field. The default value is 0.0.

solidTable

An odb_SequenceSequenceDouble specifying the following:

- $\$A_{\{s\}}\$$.
- $\$B_{\{s\}}\$$.

MATERIAL COMMANDS

- ω_s .
- R_{1s} .
- R_{2s} .

The default value is an empty sequence.

gasTable

An odb_SequenceSequenceDouble specifying the following:

- A_g .
- B_g .
- ω_g .
- R_{1g} .
- R_{2g} .

The default value is an empty sequence.

reactionTable

An odb_SequenceSequenceDouble specifying the following:

- Initial Pressure, P .
- Product co-volume, V_p .
- Exponent on the unreacted fraction (ignition term), α .
- First burn rate coefficient, G_1 .
- Exponent on the unreacted fraction (growth term), β .
- Exponent on the reacted fraction (growth term), γ .
- Pressure exponent (growth term), δ .
- Second burn rate coefficient, G_2 .
- Exponent on the unreacted fraction (completion term), ϵ .
- Exponent on the reacted fraction (completion term), ζ .
- Pressure exponent (completion term), η .
- Initial reacted fraction, $F_{ig}^{\max}$.
- Maximum reacted fraction for the growth term, $F_{G1}^{\max}$.
- Minimum reacted fraction, $F_{G2}^{\min}$.

The default value is an empty sequence.

gasSpecificTable

An odb_SequenceSequenceDouble specifying the following:

- Specific Heat per unit mass.
- Temperature dependent data.
- Value of first field variable.
- Value of second field variable.

- Etc.

The default value is an empty sequence.

table

An `odb_SequenceSequenceDouble` specifying the items described below. The default value is an empty sequence.

Table data

If *type*=IDEALGAS, the table data represents the following:

- Gas constant, R .
- The ambient pressure, p_A . If this field is left blank, a default of 0.0 is used.

If *type*=JWL, the table data represents the following:

- Detonation wave speed, C_d .
- A .
- B .
- ω . (Dimensionless.)
- R_1 . (Dimensionless.)
- R_2 . (Dimensionless.)
- Pre-detonation bulk modulus, K_{pd} .
- Detonation energy density, E_0 .

If *type*=USUP, the table data represents the following:

- c_0 .
- s . (Dimensionless.)
- Γ_0 . (Dimensionless.)

If *type*=TABULAR, the table data represents the following:

- F_1 .
- F_2 .
- ε_{vol}^c . (Dimensionless.)

Return value

An Eos object.

Exceptions

None.

60.47.2 Members

The Eos object has members with the same names and descriptions as the arguments to the **Eos** method.

In addition, the Eos object can have the following members:

Prototype

```
odb_DetonationPoint detonationPoint() const;  
odb_EosCompaction eosCompaction() const;
```

detonationPoint

A DetonationPoint object.

eosCompaction

An EosCompaction object.

60.47.3 Corresponding analysis keywords

*EOS

60.48 EosCompaction object

The EosCompaction object specifies material eos compaction.

Access

```
materialApi.materials() [name].eos().eosCompaction()
```

60.48.1 EosCompaction(...)

This method creates a EosCompaction object.

Path

```
materialApi.materials() [name].eos().EosCompaction
```

Prototype

```
odb_EosCompaction&  
EosCompaction(double soundSpeed,  
               double porosity,  
               double pressure,  
               double compactionPressure);
```

Required arguments*soundSpeed*

A Double specifying reference sound speed in the porous material.

porosity

A Double specifying value of the porosity of the unloaded material.

pressure

A Double specifying pressure required to initialize plastic behavior.

compactionPressure

A Double specifying compaction pressure at which all pores are crushed.

Optional arguments

None.

Return value

An EosCompaction object.

Exceptions

RangeError.

60.48.2 Members

The EosCompaction object has members with the same names and descriptions as the arguments to the **EosCompaction** method.

60.48.3 Corresponding analysis keywords

*EOS COMPACTION

60.49 Expansion object

The Expansion object specifies thermal expansion.

Access

`materialApi.materials() [name].expansion()`

60.49.1 Expansion(...)

This method creates an Expansion object.

Path

```
materialApi.materials()[name].Expansion
```

Prototype

```
odb_Expansion&
Expansion(const odb_String& type,
          bool userSubroutine,
          double zero,
          bool temperatureDependency,
          int dependencies,
          const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments

type

An odb_String specifying the type of expansion. Possible values are "ISOTROPIC", "ORTHOTROPIC", "ANISOTROPIC", and "SHORT_FIBER". The default value is "ISOTROPIC".

userSubroutine

A Boolean specifying whether a user subroutine is used to define the increments of thermal strain. The default value is false.

zero

A Double specifying the value of θ^0 if the thermal expansion is temperature-dependent or field-variable-dependent. The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

table

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

This argument is required only if *type* is not "USER".

Table data

If *type*=ISOTROPIC, the table data specify the following:

- α in Abaqus/Standard or Abaqus/Explicit analyses or β in an Abaqus/CFD analysis.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- α_{11} .
- α_{22} .
- α_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- α_{11} .
- α_{22} .
- α_{33} . (Not used for plane stress case.)
- α_{12} .
- α_{13} .
- α_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=SHORT_FIBER, there is no table data.

Return value

An Expansion object.

Exceptions

RangeError.

60.49.2 Members

The Expansion object has members with the same names and descriptions as the arguments to the **Expansion** method.

60.49.3 Corresponding analysis keywords

*EXPANSION

60.50 FailStrain object

The FailStrain object defines parameters for strain-based failure measures.

Access

```
materialApi.materials() [name].elastic().failStrain()
```

60.50.1 FailStrain(...)

This method creates a FailStrain object.

Path

```
materialApi.materials() [name].elastic().FailStrain
```

Prototype

```
odb_FailStrain&  
FailStrain(const odb_SequenceSequenceDouble& table,  
            bool temperatureDependency,  
            int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tensile strain limit in fiber direction, $X_{\varepsilon t}$.
- Compressive strain limit in fiber direction, $X_{\varepsilon c}$.
- Tensile strain limit in transverse direction, $Y_{\varepsilon t}$.
- Compressive strain limit in transverse direction, $Y_{\varepsilon c}$.
- Shear strain limit in the X – Y plane, S_{ε} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FailStrain object.

Exceptions

RangeError.

60.50.2 Members

The FailStrain object has members with the same names and descriptions as the arguments to the **FailStrain** method.

60.50.3 Corresponding analysis keywords

*FAIL STRAIN

60.51 FailStress object

The FailStress object defines parameters for stress-based failure measures.

Access

```
materialApi.materials() [name].elastic().failStress()
```

60.51.1 FailStress(...)

This method creates a FailStress object.

Path

```
materialApi.materials() [name].elastic().FailStress
```

Prototype

```
odb_FailStress&
FailStress(const odb_SequenceSequenceDouble& table,
            bool temperatureDependency,
            int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tensile stress limit in fiber direction, X_t .
- Compressive stress limit in fiber direction, X_c .
- Tensile stress limit in transverse direction, Y_t .
- Compressive stress limit in transverse direction, Y_c .
- Shear strength in the X – Y plane, S .
- Cross product term coefficient, f^* ($-1.0 \leq f^* \leq 1.0$). The default value is zero.
- Biaxial stress limit, σ_{biax} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FailStress object.

Exceptions

RangeError.

60.51.2 Members

The FailStress object has members with the same names and descriptions as the arguments to the `FailStress` method.

60.51.3 Corresponding analysis keywords

*FAIL STRESS

60.52 FailureRatios object

The FailureRatios object specifies the shape of the failure surface for a Concrete model.

Access

```
materialApi.materials() [name].concrete().failureRatios()
```

60.52.1 FailureRatios(...)

This method creates a FailureRatios object.

Path

```
materialApi.materials() [name].concrete().FailureRatios
```

Prototype

```
odb_FailureRatios&
FailureRatios(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Ratio of the ultimate biaxial compressive stress to the uniaxial compressive ultimate stress. The default value is 1.16.
- Absolute value of the ratio of the uniaxial tensile stress at failure to the uniaxial compressive stress at failure. The default value is 0.09.
- Ratio of the magnitude of a principal component of plastic strain at ultimate stress in biaxial compression to the plastic strain at ultimate stress in uniaxial compression. The default value is 1.28.
- Ratio of the tensile principal stress value at shear in plane stress, when the other nonzero principal stress component is at the ultimate compressive stress value, to the tensile cracking stress under uniaxial tension. The default value is 1/3.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `FailureRatios` object.

Exceptions

`RangeError`.

60.52.2 Members

The `FailureRatios` object has members with the same names and descriptions as the arguments to the `FailureRatios` method.

60.52.3 Corresponding analysis keywords

*FAILURE RATIOS

60.53 FluidLeakoff object

The FluidLeakoff object specifies leak-off coefficients for pore pressure cohesive elements.

Access

```
materialApi.materials() [name].fluidLeakoff()
```

60.53.1 FluidLeakoff(...)

This method creates a FluidLeakoff object.

Path

```
materialApi.materials() [name].FluidLeakoff
```

Prototype

```
odb_FluidLeakoff&
FluidLeakoff(bool temperatureDependency,
              int dependencies,
              const odb_String& type,
              const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

An odb_String specifying the type of fluid leak-off. Possible values are "COEFFICIENTS" and "USER". The default value is "COEFFICIENTS".

MATERIAL COMMANDS

table

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

Table data

The table data specify the following:

- Fluid leak-off coefficient at top element surface.
- Fluid leak-off coefficient at bottom element surface.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A FluidLeakoff object.

Exceptions

None.

60.53.2 Members

The FluidLeakoff object has members with the same names and descriptions as the arguments to the **FluidLeakoff** method.

60.53.3 Corresponding analysis keywords

*FLUID LEAKOFF

60.54 GapFlow object

The GapFlow object specifies tangential flow constitutive parameters for pore pressure cohesive elements.

Access

`materialApi.materials() [name].gapFlow()`

60.54.1 GapFlow(...)

This method creates a GapFlow object.

Path

```
materialApi.materials() [name].GapFlow
```

Prototype

```
odb_GapFlow&
GapFlow(const odb_SequenceSequenceDouble& table,
         odb_Union kmax,
         bool temperatureDependency,
         int dependencies,
         const odb_String& type);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

kmax

The string "NONE" or a Double specifying the maximum permeability value that should be used. If *kmax*="NONE", Abaqus assumes that the permeability is not bounded. This value is meaningful only when *type*="NEWTONIAN". The default value is "NONE".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

type

An odb_String specifying the type of gap flow. Possible values are "NEWTONIAN" and "POWER_LAW". The default value is "NEWTONIAN".

Table data

If *type*=NEWTONIAN the table data specify the following:

- Pore viscosity.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

MATERIAL COMMANDS

If *type*=POWER_LAW the table data specify the following:

- Consistency.
- Exponent.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GapFlow object.

Exceptions

None.

60.54.2 Members

The GapFlow object has members with the same names and descriptions as the arguments to the **GapFlow** method.

60.54.3 Corresponding analysis keywords

*GAP FLOW

60.55 GasketMembraneElastic object

The GasketMembraneElastic object defines the elastic parameters for the membrane shear behavior of a gasket.

Access

```
materialApi.materials() [name].gasketMembraneElastic()
```

60.55.1 GasketMembraneElastic(...)

This method creates a GasketMembraneElastic object.

Path

```
materialApi.materials()[name].GasketMembraneElastic
```

Prototype

```
odb_GasketMembraneElastic&
GasketMembraneElastic(const odb_SequenceSequenceDouble& table,
                       bool temperatureDependency,
                       int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Young's modulus, E .
- Poisson's ratio, ν .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GasketMembraneElastic object.

Exceptions

RangeError.

60.55.2 Members

The GasketMembraneElastic object has members with the same names and descriptions as the arguments to the **GasketMembraneElastic** method.

60.55.3 Corresponding analysis keywords

*GASKET ELASTICITY

60.56 GasketThicknessBehavior object

The GasketThicknessBehavior object defines the behavior in the thickness direction for a gasket.

Access

```
materialApi.materials() [name].gasketThicknessBehavior()
```

60.56.1 GasketThicknessBehavior(...)

This method creates a GasketThicknessBehavior object.

Path

```
materialApi.materials() [name].GasketThicknessBehavior
```

Prototype

```
odb_GasketThicknessBehavior&
GasketThicknessBehavior(const odb_SequenceSequenceDouble& table,
    bool temperatureDependency,
    int dependencies,
    double tensileStiffnessFactor,
    const odb_String& type,
    int unloadingDependencies,
    bool unloadingTemperatureDependency,
    const odb_String& variableUnits,
    double yieldOnset,
    const odb_String& yieldOnsetMethod,
    const odb_SequenceSequenceDouble& unloadingTable);
```


Required argument*table*

An odb_SequenceSequenceDouble specifying loading data. The first sequence must contain only 0. At least two sequences must be specified if *type*="DAMAGE", and at least 3 sequences must be specified if *type*="ELASTIC_PLASTIC". The items in the table data are described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the loading data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies included in the definition of the loading data, in addition to temperature. The default value is 0.

tensileStiffnessFactor

A Double specifying the fraction of the initial compressive stiffness that defines the stiffness in tension. The default value is 10^{-3} .

type

An odb_String specifying a damage elasticity model or an elastic-plastic model for gasket thickness-direction behavior. Possible values are "ELASTIC_PLASTIC" and "DAMAGE". The default value is "ELASTIC_PLASTIC".

unloadingDependencies

An Int specifying the number of field variable dependencies included in the definition of the unloading data, in addition to temperature. The default value is 0.

unloadingTemperatureDependency

A Boolean specifying whether unloading data depends on temperature. The default value is false.

variableUnits

An odb_String specifying the behavior in terms of units of force (or force in unit length) versus closure or pressure versus closure. Possible values are "STRESS" and "FORCE". The default value is "STRESS".

yieldOnset

A Double specifying the closure value at which the onset of yield occurs or the relative drop in slope on the loading curve that defines the onset of plastic deformation (depending on the value of *yieldOnsetMethod*). The default value is 0.1.

yieldOnsetMethod

An odb_String specifying the method used to determine yield onset. Possible values are "RELATIVE_SLOPE_DROP" and "CLOSURE_VALUE". The default value is "RELATIVE_SLOPE_DROP".

unloadingTable

An odb_SequenceSequenceDouble specifying unloading data. The items in the table data are described below. The default value is an empty sequence.

Table data

If *variableUnits*=STRESS, the loading table data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE, the loading table data specify the following:

- Force or force per unit length; this value must be positive.
- Closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=STRESS and *type*=ELASTIC_PLASTIC, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Plastic closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE and *type*=ELASTIC_PLASTIC, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.
- Closure; this value must be positive.
- Plastic closure; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=STRESS and *type*=DAMAGE, the *unloadingTable* data specify the following:

- Pressure; this value must be positive.

- Closure; this value must be positive.
- Maximum closure reached while loading the gasket; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *variableUnits*=FORCE and *type*=DAMAGE, the *unloadingTable* data specify the following:

- Force or force per unit length; this value must be positive.
- Closure; this value must be positive.
- Maximum closure reached while loading the gasket; this value must be positive.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A GasketThicknessBehavior object.

Exceptions

RangeError.

60.56.2 Members

The GasketThicknessBehavior object has members with the same names and descriptions as the arguments to the **GasketThicknessBehavior** method.

In addition, the GasketThicknessBehavior object can have the following member:

Prototype

```
odb_ContactArea contactArea() const;
```

contactArea

A ContactArea object.

60.56.3 Corresponding analysis keywords

*GASKET THICKNESS BEHAVIOR

60.57 GasketTransverseShearElastic object

The GasketTransverseShearElastic object defines the elastic parameters for the transverse shear behavior of a gasket.

Access

```
materialApi.materials() [name].gasketTransverseShearElastic()
```

60.57.1 GasketTransverseShearElastic(...)

This method creates a GasketTransverseShearElastic object.

Path

```
materialApi.materials() [name].GasketTransverseShearElastic
```

Prototype

```
odb_GasketTransverseShearElastic&
GasketTransverseShearElastic(const odb_SequenceSequenceDouble& table,
                             const odb_String& variableUnits,
                             bool temperatureDependency,
                             int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

variableUnits

An odb_String specifying the unit system in which the transverse shear behavior will be defined. Possible values are "STRESS" and "FORCE". The default value is "STRESS".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Shear stiffness. (This value cannot be negative.)

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A `GasketTransverseShearElastic` object.

Exceptions

`RangeError`.

60.57.2 Members

The `GasketTransverseShearElastic` object has members with the same names and descriptions as the arguments to the `GasketTransverseShearElastic` method.

60.57.3 Corresponding analysis keywords

*GASKET ELASTICITY

60.58 Gel object

The Gel object defines a swelling gel.

Access

```
materialApi.materials() [name].gel()
```

60.58.1 Gel(...)

This method creates a Gel object.

Path

```
materialApi.materials() [name].Gel
```

Prototype

```
odb_Gel&  
Gel(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- Radius of gel particles when completely dry, r_a^{dry} .
- Fully swollen radius of gel particles, r_a^f .
- Number of gel particles per unit volume, k_a .
- Relaxation time constant for long-term swelling of gel particles, τ_1 .

Return value

A Gel object.

Exceptions

None.

60.58.2 Members

The Gel object has members with the same names and descriptions as the arguments to the **Gel** method.

60.58.3 Corresponding analysis keywords

*GEL

60.59 HeatGeneration object

The HeatGeneration object includes volumetric heat generation in heat transfer analyses.

Access

```
materialApi.materials() [name].heatGeneration()
```

60.59.1 HeatGeneration()

This method creates a HeatGeneration object.

Path

```
materialApi.materials() [name].HeatGeneration
```

Prototype

```
odb_HeatGeneration&  
HeatGeneration();
```

Arguments

None.

Return value

A HeatGeneration object.

Exceptions

None.

60.59.2 Members

The HeatGeneration object has no members.

60.59.3 Corresponding analysis keywords

*HEAT GENERATION

60.60 Hyperelastic object

The Hyperelastic object specifies elastic properties for approximately incompressible elastomers.

Access

```
materialApi.materials() [name].hyperelastic()
```

60.60.1 Hyperelastic(...)

This method creates a Hyperelastic object.

Path

```
materialApi.materials() [name].Hyperelastic
```

Prototype

```
odb_Hyperelastic&
Hyperelastic(const odb_SequenceSequenceDouble& table,
             const odb_String& type,
             const odb_String& moduliTimeScale,
             bool temperatureDependency,
             int n,
             odb_Union beta,
             bool testData,
             bool compressible,
             int properties,
             const odb_String& deviatoricResponse,
             const odb_String& volumetricResponse,
             double poissonRatio,
             const odb_String& materialType,
             const odb_String& anisotropicType,
             const odb_String& formulation,
             const odb_String& behaviorType,
             int dependencies,
             int localDirections);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below. This argument is valid only if *testData*=OFF.

Optional arguments

type

An odb_String specifying the type of strain energy potential. Possible values are:

- "ARRUDA_BOYCE"
- "MARLOW"
- "MOONEY_RIVLIN"
- "NEO_HOOKE"
- "OGDEN"
- "POLYNOMIAL"
- "REDUCED_POLYNOMIAL"
- "USER"
- "VAN_DER_WAALS"
- "YEOH"
- "UNKNOWN"

The default value is "UNKNOWN".

moduliTimeScale

An odb_String specifying the nature of the time response. Possible values are "INSTANTANEOUS" and "LONG_TERM". The default value is "LONG_TERM".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

n

An Int specifying the order of the strain energy potential. The default value is 1.

If *testData*=ON and *type*="POLYNOMIAL", *n* can take only the values 1 or 2.

If *testData*=ON and *type*="OGDEN" or if *testData*=OFF for either type, $1 \leq n \leq 6$.

If *type*="USER", this argument cannot be used.

beta

The string "FITTED_VALUE" or a Double specifying the invariant mixture parameter. This argument is valid only if *testData*=ON and *type*="VAN_DER_WAALS". The default value is "FITTED_VALUE".

testData

A Boolean specifying whether test data are supplied. The default value is true.

compressible

A Boolean specifying whether the hyperelastic material is compressible. This parameter is applicable only to user-defined hyperelastic materials. The default value is false.

properties

An Int specifying the number of property values needed as data for the user-defined hyperelastic material. The default value is 0.

deviatoricResponse

An odb_String specifying which test data to use. Possible values are "UNIAXIAL", "BIAXIAL", and "PLANAR". The default value is "UNIAXIAL".

MATERIAL COMMANDS

volumetricResponse

An odb_String specifying the volumetric response. Possible values are "DEFAULT", "VOLUMETRIC_DATA", "POISSON_RATIO", and "LATERAL_NOMINAL_STRAIN". The default value is "DEFAULT".

poissonRatio

A Double specifying the poisson ratio. This argument is valid only if *volumetricResponse*="POISSON_RATIO". The default value is 0.0.

materialType

An odb_String specifying the type of material. Possible values are "ISOTROPIC" and "ANISOTROPIC". The default value is "ISOTROPIC".

anisotropicType

An odb_String specifying the type of strain energy potential. Possible values are "FUNG_ANISOTROPIC", "FUNG_ORTHOTROPIC", "HOLZAPFEL", and "USER_DEFINED". The default value is "FUNG_ANISOTROPIC".

formulation

An odb_String specifying the type of formulation. Possible values are "STRAIN" and "INVARIANT". The default value is "STRAIN".

behaviorType

An odb_String specifying the type of anisotropic hyperelastic material behavior. Possible values are "INCOMPRESSIBLE" and "COMPRESSIBLE". The default value is "INCOMPRESSIBLE".

dependencies

An Int specifying the number of field variable dependencies for the data in *volumetricTable*. The default value is 0.

localDirections

An Int specifying the number of local directions for the data in *volumetricTable*. The default value is 0.

Table data

If *type*=ARRUDA_BOYCE, the table data specify the following:

- μ .
- λ_m .
- D .
- Temperature, if the data depend on temperature.

If *type*=MOONEY_RIVLIN, the table data specify the following:

- C_{10} .
- C_{01} .
- D_1 .
- Temperature, if the data depend on temperature.

If *type*=NEO_HOOKE, the table data specify the following:

- C_{10} .
- D_1 .
- Temperature, if the data depend on temperature.

If *type*=OGDEN, the table data specify the following for values of *n*:

- μ_i and α_i for *i* from 1 to *n*.
- *n* coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=POLYNOMIAL, the table data specify the following for values of *n*:

- C_{ij} for each value of $(i + j)$ from 1 to *n* with *i* decreasing from $(i + j)$ to zero and *j* increasing from zero to $(i + j)$.
- *n* coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $3 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=REDUCED_POLYNOMIAL, the table data specify the following for values of *n*:

- C_{i0} for *i* from 1 to *n*.
- *n* coefficients D_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

If *type*=VAN_DER_WAALS, the table data specify the following:

- μ .
- λ_m .
- a .
- β .
- D .
- Temperature, if the data depend on temperature.

If *type*=YEOH, the table data specify the following:

- C_{10} .
- C_{20} .
- C_{30} .
- D_1 .
- D_2 .
- D_3 .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed in an Abaqus/Explicit analysis.

MATERIAL COMMANDS

The **None** object is the default value if *testData*=ON.

Return value

A Hyperelastic object.

Exceptions

InvalidNameError and RangeError.

60.60.2 Members

The Hyperelastic object has members with the same names and descriptions as the arguments to the **Hyperelastic** method.

In addition, the Hyperelastic object can have the following members:

Prototype

```
odb_BiaxialTestData biaxialTestData() const;  
odb_PlanarTestData planarTestData() const;  
odb_UniaxialTestData uniaxialTestData() const;  
odb_VolumetricTestData volumetricTestData() const;  
odb_Hysteresis hysteresis() const;
```

biaxialTestData

A BiaxialTestData object.

planarTestData

A PlanarTestData object.

uniaxialTestData

A UniaxialTestData object.

volumetricTestData

A VolumetricTestData object.

hysteresis

A Hysteresis object.

60.60.3 Corresponding analysis keywords

*HYPERELASTIC

60.61 Hyperfoam object

The Hyperfoam object specifies elastic properties for a hyperelastic foam.

Access

```
materialApi.materials() [name].hyperfoam()
```

60.61.1 Hyperfoam(...)

This method creates a Hyperfoam object.

Path

```
materialApi.materials() [name].Hyperfoam
```

Prototype

```
odb_Hyperfoam&
Hyperfoam(bool testData,
           odb_Union poisson,
           int n,
           bool temperatureDependency,
           const odb_String& moduli,
           const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments

testData

A Boolean specifying whether test data are supplied. The default value is false.

poisson

The string "NONE" or a Double specifying the effective Poisson's ratio, ν , of the material. This argument is valid only when *testData*=true. The default value is "NONE".

n

An Int specifying the order of the strain energy potential. Possible values are $1 \leq n \leq 6$. The default value is 1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

moduli

An odb_String specifying the time-dependence of the material constants. Possible values are "INSTANTANEOUS" and "LONG_TERM". The default value is "LONG_TERM".

table

An odb_SequenceSequenceDouble specifying the items described below. This argument is valid only when *testData*=false. The default value is an empty sequence.

Table data

The items in the table data specify the following for values of n :

- μ_i and α_i for i from 1 to n .
- ν_i .
- Temperature, if the data depend on temperature. Temperature dependence is not allowed for $4 \leq n \leq 6$ in an Abaqus/Explicit analysis.

Return value

A Hyperfoam object.

Exceptions

RangeError.

60.61.2 Members

The Hyperfoam object has members with the same names and descriptions as the arguments to the **Hyperfoam** method.

In addition, the Hyperfoam object can have the following members:

Prototype

```
odb_BiaxialTestData biaxialTestData() const;  
odb_VolumetricTestData volumetricTestData() const;  
odb_PlanarTestData planarTestData() const;  
odb_SimpleShearTestData simpleShearTestData() const;  
odb_UniaxialTestData uniaxialTestData() const;
```

biaxialTestData

A BiaxialTestData object.

volumetricTestData

A VolumetricTestData object.

planarTestData

A PlanarTestData object.

simpleShearTestData

A SimpleShearTestData object.

uniaxialTestData

A UniaxialTestData object.

60.61.3 Corresponding analysis keywords

*HYPERFOAM

60.62 Hypoelastic object

The Hypoelastic object specifies hypoelastic material properties.

Access

`materialApi.materials() [name].hypoelastic()`

60.62.1 Hypoelastic(...)

This method creates a Hypoelastic object.

Path

`materialApi.materials() [name].Hypoelastic`

Prototype

```
odb_Hypoelastic&
Hypoelastic(const odb_SequenceSequenceDouble& table,
             bool user);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional argument

user

A Boolean specifying that hypoelasticity is defined by user subroutine **UHYP**EL. The default value is false.

Table data

- Instantaneous Young's modulus, E .
- Instantaneous Poisson's ratio, ν .
- First strain invariant, I_1 .
- Second strain invariant, I_2 .
- Third strain invariant, I_3 .

Return value

A Hypoelastic object.

Exceptions

None.

60.62.2 Members

The Hypoelastic object has members with the same names and descriptions as the arguments to the **Hypoelastic** method.

60.62.3 Corresponding analysis keywords

*HYPOELASTIC

60.63 Hysteresis object

The Hysteresis object specifies the creep part of the material model for the hysteretic behavior of elastomers.

Access

```
materialApi.materials() [name].hyperelastic().hysteresis()
```

60.63.1 Hysteresis(...)

This method creates a Hysteresis object.

Path

```
materialApi.materials() [name].hyperelastic().Hysteresis
```

Prototype

```
odb_Hysteresis&  
Hysteresis(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- Stress scaling factor.
- Creep parameter.
- Effective stress exponent.
- Creep strain exponent.

Return value

A Hysteresis object.

Exceptions

RangeError.

60.63.2 Members

The Hysteresis object has members with the same names and descriptions as the arguments to the **Hysteresis** method.

60.63.3 Corresponding analysis keywords

*HYSTERESIS

60.64 InelasticHeatFraction object

The InelasticHeatFraction object defines the fraction of the rate of inelastic dissipation that appears as a heat source.

Access

```
materialApi.materials() [name].inelasticHeatFraction()
```

60.64.1 InelasticHeatFraction(...)

This method creates an InelasticHeatFraction object.

Path

```
materialApi.materials() [name].InelasticHeatFraction
```

Prototype

```
odb_InelasticHeatFraction&  
InelasticHeatFraction(double fraction);
```

Required arguments

None.

Optional argument

fraction

A Double specifying the fraction of inelastic dissipation rate that appears as a heat flux per unit volume. The fraction may include a unit conversion factor if required. Possible values are $0.0 \leq \text{fraction} \leq 1.0$. The default value is 0.9.

Return value

An InelasticHeatFraction object.

Exceptions

RangeError.

60.64.2 Members

The InelasticHeatFraction object has members with the same names and descriptions as the arguments to the **InelasticHeatFraction** method.

60.64.3 Corresponding analysis keywords

*INELASTIC HEAT FRACTION

60.65 JouleHeatFraction object

The JouleHeatFraction object defines the fraction of electric energy released as heat.

Access

```
materialApi.materials() [name].jouleHeatFraction()
```

60.65.1 JouleHeatFraction(...)

This method creates a JouleHeatFraction object.

Path

```
materialApi.materials() [name].JouleHeatFraction
```

Prototype

```
odb_JouleHeatFraction&  
JouleHeatFraction(double fraction);
```

Required arguments

None.

Optional argument

fraction

A Double specifying the fraction of electrical energy released as heat, including any unit conversion factor. Possible values are $0.0 \leq \textit{fraction} \leq 1.0$. The default value is 1.0.

Return value

A JouleHeatFraction object.

Exceptions

RangeError.

60.65.2 Members

The `JouleHeatFraction` object has members with the same names and descriptions as the arguments to the `JouleHeatFraction` method.

60.65.3 Corresponding analysis keywords

*JOULE HEAT FRACTION

60.66 LatentHeat object

The `LatentHeat` object specifies a material's latent heat.

Access

```
materialApi.materials() [name].latentHeat()
```

60.66.1 LatentHeat(...)

This method creates a `LatentHeat` object.

Path

```
materialApi.materials() [name].LatentHeat
```

Prototype

```
odb_LatentHeat&  
LatentHeat(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional arguments

None.

Table data

- Latent heat per unit mass.

- Solidus temperature.
- Liquidus temperature.

Return value

A LatentHeat object.

Exceptions

RangeError.

60.66.2 Members

The LatentHeat object has members with the same names and descriptions as the arguments to the `LatentHeat` method.

60.66.3 Corresponding analysis keywords

*LATENT HEAT

60.67 LowDensityFoam object

The LowDensityFoam object specifies properties for low-density foam.

Access

```
materialApi.materials() [name].lowDensityFoam()
```

60.67.1 LowDensityFoam(...)

This method creates a LowDensityFoam object.

Path

```
materialApi.materials() [name].LowDensityFoam
```

Prototype

```
odb_LowDensityFoam&
LowDensityFoam(bool elementRemoval,
                odb_Union maxAllowablePrincipalStress,
```

```
bool extrapolateStressStrainCurve,
const odb_String& strainRateType,
double mu0,
double mu1,
double alpha);
```

Required arguments

None.

Optional arguments

elementRemoval

A Boolean specifying whether elements are removed if exceeding maximum principal tensile stress. This argument is valid only when *maxAllowablePrincipalStress* is defined. The default value is false.

maxAllowablePrincipalStress

The string "NONE" or a Double specifying the maximum allowable principal tensile stress. The default value is "NONE".

extrapolateStressStrainCurve

A Boolean specifying whether the stress-strain curve is extrapolated if exceeding maximum strain rate. The default value is false.

strainRateType

An odb_String specifying strain rate measure used for constitutive calculations. Possible values are "PRINCIPAL" and "VOLUMETRIC". The default value is "VOLUMETRIC".

mu0

A Double specifying the relaxation coefficient μ_0 . The default value is 10^{-4} .

mu1

A Double specifying the relaxation coefficient μ_1 . The default value is 0.5×10^{-2} .

alpha

A Double specifying the relaxation coefficient α . The default value is 2.0.

Return value

A LowDensityFoam object.

Exceptions

RangeError.

60.67.2 Members

The LowDensityFoam object has members with the same names and descriptions as the arguments to the **LowDensityFoam** method.

In addition, the LowDensityFoam object can have the following members:

Prototype

```
odb_UniaxialTestData uniaxialTensionTestData() const;
odb_UniaxialTestData uniaxialCompressionTestData() const;
uniaxialTensionTestData
    A UniaxialTestData object.
uniaxialCompressionTestData
    A UniaxialTestData object.
```

60.67.3 Corresponding analysis keywords

*LOW DENSITY FOAM

60.68 MagneticPermeability object

The MagneticPermeability object specifies magnetic permeability.

Access

```
materialApi.materials() [name].magneticPermeability()
```

60.68.1 MagneticPermeability(...)

This method creates a MagneticPermeability object.

Path

```
materialApi.materials() [name].MagneticPermeability
```

Prototype

```
odb_MagneticPermeability&
MagneticPermeability(const odb_SequenceSequenceDouble& table,
                     const odb_SequenceSequenceDouble& table2,
                     const odb_SequenceSequenceDouble& table3,
                     const odb_String& type,
                     bool frequencyDependency,
                     bool temperatureDependency,
```

MATERIAL COMMANDS

```
int dependencies,  
bool nonlinearBH);
```

Required arguments

table

An odb_SequenceSequenceDouble specifying the items described below in “Table data.” If *type*=ORTHOTROPIC and nonlinearBH=ON, the data specified in the *table* is for the first direction and *table2* and *table3* must be specified.

table2

An odb_SequenceSequenceDouble specifying the items described below in “Table data” in the second direction. *table2* must be specified only if *type*=ORTHOTROPIC and nonlinearBH=ON.

table3

An odb_SequenceSequenceDouble specifying the items described below in “Table data” in the third direction. *table3* must be specified only if *type*=ORTHOTROPIC and nonlinearBH=ON.

Optional arguments

type

An odb_String specifying the type of magnetic permeability. Possible values are "ISOTROPIC", "ORTHOTROPIC", and "ANISOTROPIC". The default value is "ISOTROPIC".

frequencyDependency

A Boolean specifying whether the data depend on frequency. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

nonlinearBH

A Boolean specifying whether the magnetic behavior is nonlinear and available in tabular form of magnetic flux density versus magnetic field values. The default value is false.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- Magnetic permeability.
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ISOTROPIC, and *nonlinearBH*=TRUE, the table data specify the following:

- Magnitude of the magnetic flux density vector.
- Magnitude of the magnetic field vector.

- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, the table data specify the following:

- μ_{11}^E .
- μ_{22}^E .
- μ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ORTHOTROPIC, and *nonlinearBH*=TRUE, the table data specify the following:

- Magnitude of the magnetic flux density vector in the first direction.
- Magnitude of the magnetic field vector in the second direction.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=ANISOTROPIC, the table data specify the following:

- μ_{11}^E .
- μ_{12}^E .
- μ_{22}^E .
- μ_{13}^E .
- μ_{23}^E .
- μ_{33}^E .
- Frequency, if the data depend on frequency.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MagneticPermeability object.

Exceptions

RangeError.

60.68.2 Members

The MagneticPermeability object has members with the same names and descriptions as the arguments to the **MagneticPermeability** method.

60.68.3 Corresponding analysis keywords

*MAGNETIC PERMEABILITY

60.69 MohrCoulombHardening object

The MohrCoulombHardening object specifies hardening for the Mohr-Coulomb plasticity model.

Access

```
materialApi.materials() [name].mohrCoulombPlasticity() \
.mohrCoulombHardening()
```

60.69.1 MohrCoulombHardening(...)

This method creates a MohrCoulombHardening object.

Path

```
materialApi.materials() [name].mohrCoulombPlasticity() \
.MohrCoulombHardening
```

Prototype

```
odb_MohrCoulombHardening&
MohrCoulombHardening(const odb_SequenceSequenceDouble& table,
                     bool temperatureDependency,
```

```
int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Cohesion yield stress.
- The absolute value of the corresponding plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MohrCoulombHardening object.

Exceptions

RangeError.

60.69.2 Members

The MohrCoulombHardening object has members with the same names and descriptions as the arguments to the **MohrCoulombHardening** method.

60.69.3 Corresponding analysis keywords

*MOHR COULOMB HARDENING

60.70 MohrCoulombPlasticity object

The MohrCoulombPlasticity object specifies the extended Mohr-Coulomb plasticity model.

Access

```
materialApi.materials() [name].mohrCoulombPlasticity()
```

60.70.1 MohrCoulombPlasticity(...)

This method creates a MohrCoulombPlasticity object.

Path

```
materialApi.materials() [name].MohrCoulombPlasticity
```

Prototype

```
odb_MohrCoulombPlasticity&
MohrCoulombPlasticity(const odb_SequenceSequenceDouble& table,
                      odb_Union deviatoricEccentricity,
                      double meridionalEccentricity,
                      bool temperatureDependency,
                      int dependencies,
                      bool useTensionCutoff);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

deviatoricEccentricity

The string "NONE" or a Double specifying the flow potential eccentricity in the deviatoric plane, e ; $1/2 \leq e \leq 1.0$. If *deviatoricEccentricity*="NONE", Abaqus calculates the value using the specified Mohr-Coulomb angle of friction. The default value is "NONE".

meridionalEccentricity

A Double specifying the flow potential eccentricity in the meridional plane, ϵ . The default value is 0.1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

useTensionCutoff

A Boolean specifying whether tension cutoff specification is needed. The default value is false.

Table data

The table data specify the following:

- Friction angle (given in degrees), ϕ , at high confining pressure in the p - $R_{mc}q$ plane.
- Dilation angle, ψ , at high confining pressure in the p - $R_{mu}q$ plane.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A MohrCoulombPlasticity object.

Exceptions

RangeError.

60.70.2 Members

The MohrCoulombPlasticity object has members with the same names and descriptions as the arguments to the **MohrCoulombPlasticity** method.

In addition, the MohrCoulombPlasticity object can have the following members:

Prototype

```
odb_MohrCoulombHardening mohrCoulombHardening() const;
```

```
odb_TensionCutOff tensionCutOff() const;
```

mohrCoulombHardening

A MohrCoulombHardening object.

tensionCutOff

A TensionCutOff object.

60.70.3 Corresponding analysis keywords

*MOHR COULOMB

60.71 MoistureSwelling object

The MoistureSwelling object defines moisture-driven swelling.

Access

```
materialApi.materials() [name].moistureSwelling()
```

60.71.1 MoistureSwelling(...)

This method creates a MoistureSwelling object.

Path

```
materialApi.materials() [name].MoistureSwelling
```

Prototype

```
odb_MoistureSwelling&  
MoistureSwelling(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- Volumetric moisture swelling strain, ε^{ms} .
- Saturation, s . This value must lie in the range $0.0 \leq s \leq 1.0$.

Return value

A MoistureSwelling object.

Exceptions

None.

60.71.2 Members

The MoistureSwelling object has members with the same names and descriptions as the arguments to the **MoistureSwelling** method.

In addition, the MoistureSwelling object can have the following member:

Prototype

```
odb_Ratios ratios() const;
```

ratios

A Ratios object.

60.71.3 Corresponding analysis keywords

*MOISTURE SWELLING

60.72 MullinsEffect object

The MullinsEffect specifies properties for mullins data.

Access

```
materialApi.materials() [name].mullinsEffect()
```

60.72.1 Members

The MullinsEffect object can have the following members:

Prototype

```
odb_String definition() const;
bool temperatureDependency() const;
int dependencies() const;
int properties() const;
odb_SequenceSequenceDouble table() const;
odb_SequenceUniaxialTestData uniaxialTests() const;
odb_UniaxialTestData uniaxialTests(int index) const;
odb_SequenceBiaxialTestData biaxialTests() const;
odb_BiaxialTestData biaxialTests(int index) const;
```

MATERIAL COMMANDS

```
odb_SequencePlanarTestData planarTests() const;  
odb_PlanarTestData planarTests(int index) const;
```

definition

An odb_String specifying the method of specifying the data. Possible values are "USER", "CONSTANTS", and "TEST_DATA". The default value is "CONSTANTS".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

properties

An Int specifying the number of property values needed as data for the user-defined hyperelastic material. The default value is 0.

table

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

uniaxialTests

A sequence of UniaxialTestData objects.

biaxialTests

A sequence of BiaxialTestData objects.

planarTests

A sequence of PlanarTestData objects.

60.73 Ornl object

The Ornl object specifies the constitutive model developed by Oak Ridge National Laboratory.

Access

```
materialApi.materials() [name].creep().ornl()  
materialApi.materials() [name].plastic().ornl()
```

60.73.1 Ornl(...)

This method creates an Ornl object.

Path

```
materialApi.materials() [name].creep().Ornl  
materialApi.materials() [name].plastic().Ornl
```


Prototype

```
odb_Ornl&
Ornl(double a,
      odb_Union h,
      bool reset);
```

Required arguments

None.

Optional arguments

a

A Double specifying the saturation rates for kinematic shift caused by creep strain, as defined by Equation (15) of Section 4.3.3–3 of the Nuclear Standard. The default value corresponds to that section of the Standard. Set $a=0.0$ to use the 1986 revision of the Standard. The default value is 0.3.

h

The string "NONE" or a Double specifying the rate of kinematic shift with respect to creep strain [Equation (7) of Section 4.3.2–1 of the Nuclear Standard]. If $h="NONE"$, the value of h is determined according to Section 4.3.3–3 of the 1981 revision of the Standard. Set $h=0.0$ to use the 1986 revision of the Standard. The default value is "NONE".

reset

A Boolean specifying whether to invoke the optional α reset procedure described in Section 4.3.5 of the Nuclear Standard. The default value is false.

Return value

An Ornl object.

Exceptions

RangeError.

60.73.2 Members

The Ornl object has members with the same names and descriptions as the arguments to the **Ornl** method.

60.73.3 Corresponding analysis keywords

*ORNL

60.74 Permeability object

The Permeability object defines permeability for pore fluid flow.

Access

```
materialApi.materials() [name].permeability()
```

60.74.1 Permeability(...)

This method creates a Permeability object.

Path

```
materialApi.materials() [name].Permeability
```

Prototype

```
odb_Permeability&
Permeability(double specificWeight,
              double inertialDragCoefficient,
              const odb_SequenceSequenceDouble& table,
              const odb_String& type,
              bool temperatureDependency,
              int dependencies);
```

Required arguments

specificWeight

A Double specifying the specific weight of the wetting liquid, γ_w .

inertialDragCoefficient

A Double specifying The inertial drag coefficient of the wetting liquid, γ_w .

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of permeability. Possible values are "ISOTROPIC", "ORTHOTROPIC", "ANISOTROPIC", "ISOTROPIC-CFD", and "CARMAN_KOZENY". The default value is "ISOTROPIC".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=ISOTROPIC, the table data specify the following:

- k .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If *type*=ORTHOTROPIC, the table data specify the following:

- k_{11} .
- k_{22} .
- k_{33} .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If *type*=ANISOTROPIC, the table data specify the following:

- k_{11} .
- k_{12} .
- k_{22} .
- k_{13} .
- k_{23} .
- k_{33} .
- Void ratio, e .
- Temperature, if the data depend on temperature.

If *type*=ISOTROPIC_CFD, the table data specify the following:

- k .
- Porosity, e .

If *type*=CARMAN_KOZENY, the table data specify the following:

- Kozeny constant e .
- Pore particle radius, e .

Return value

A Permeability object.

Exceptions

RangeError.

60.74.2 Members

The Permeability object has members with the same names and descriptions as the arguments to the **Permeability** method.

In addition, the Permeability object can have the following members:

Prototype

```
odb_SaturationDependence saturationDependence() const;  
odb_VelocityDependence velocityDependence() const;
```

saturationDependence

A SaturationDependence object specifying the dependence of the permeability of a material on the saturation of the wetting liquid.

velocityDependence

A VelocityDependence object specifying the dependence of the permeability of a material on the velocity of fluid flow.

60.74.3 Corresponding analysis keywords

*PERMEABILITY

60.75 Piezoelectric object

The Piezoelectric object specifies piezoelectric material properties.

Access

```
materialApi.materials() [name].piezoelectric()
```

60.75.1 Piezoelectric(...)

This method creates a Piezoelectric object.

Path

```
materialApi.materials() [name].Piezoelectric
```

Prototype

```
odb_Piezoelectric&
```

```
Piezoelectric(const odb_SequenceSequenceDouble& table,
              const odb_String& type,
              bool temperatureDependency,
              int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of material coefficients for the piezoelectric property. Possible values are "STRAIN" and "STRESS". The default value is "STRESS".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRESS, the table data specify the following:

- e_{111}^{φ} .
- e_{122}^{φ} .
- e_{133}^{φ} .
- e_{112}^{φ} .
- e_{113}^{φ} .
- e_{123}^{φ} .
- e_{211}^{φ} .
- e_{222}^{φ} .
- e_{233}^{φ} .
- e_{212}^{φ} .
- e_{213}^{φ} .
- e_{223}^{φ} .
- e_{311}^{φ} .
- e_{322}^{φ} .
- e_{333}^{φ} .
- e_{312}^{φ} .
- e_{313}^{φ} .
- e_{323}^{φ} .
- Temperature, if the data depend on temperature.

MATERIAL COMMANDS

- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If `type=STRAIN`, the table data specify the following:

- d_{111}^{φ} .
- d_{122}^{φ} .
- d_{133}^{φ} .
- d_{112}^{φ} .
- d_{113}^{φ} .
- d_{123}^{φ} .
- d_{211}^{φ} .
- d_{222}^{φ} .
- d_{233}^{φ} .
- d_{212}^{φ} .
- d_{213}^{φ} .
- d_{223}^{φ} .
- d_{311}^{φ} .
- d_{322}^{φ} .
- d_{333}^{φ} .
- d_{313}^{φ} .
- d_{323}^{φ} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Piezoelectric object.

Exceptions

None.

60.75.2 Members

The Piezoelectric object has members with the same names and descriptions as the arguments to the `Piezoelectric` method.

60.75.3 Corresponding analysis keywords

*PIEZOELECTRIC

60.76 PlanarTestData object

The PlanarTestData object specifies planar test (or pure shear) data (compression and/or tension).

Access

```
materialApi.materials() [name].hyperelastic().planarTestData()
materialApi.materials() [name].hyperfoam().planarTestData()
materialApi.materials() [name].mullinsEffect().planarTests(i)
```

60.76.1 PlanarTestData(...)

This method creates a PlanarTestData object.

Path

```
materialApi.materials() [name].hyperelastic().PlanarTestData
materialApi.materials() [name].hyperfoam().PlanarTestData
materialApi.materials() [name].mullinsEffect().PlanarTestData
```

Prototype

```
odb_PlanarTestData&
PlanarTestData(const odb_SequenceSequenceDouble& table,
               odb_Union smoothing,
               bool lateralNominalStrain,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

MATERIAL COMMANDS

Optional arguments

smoothing

The string "NONE" or an Int specifying the value for smoothing. If *smoothing*="NONE", no smoothing is employed. The default value is "NONE".

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic material model, the table data specify the following:

- Nominal stress, T_S .
- Nominal strain in the direction of loading, ϵ_S .

For a hyperfoam material model, the table data specify the following:

- Nominal stress, T_L .
- Nominal strain in the direction of loading, ϵ_p .
- Nominal transverse strain, ϵ_3 . The default value is 0.

Return value

A PlanarTestData object.

Exceptions

None.

60.76.2 Members

The PlanarTestData object has members with the same names and descriptions as the arguments to the **PlanarTestData** method.

60.76.3 Corresponding analysis keywords

*PLANAR TEST DATA

60.77 Plastic object

The Plastic object specifies a metal plasticity model.

Access

```
materialApi.materials() [name].plastic()
```

60.77.1 Plastic(...)

This method creates a Plastic object.

Path

```
materialApi.materials() [name].Plastic
```

Prototype

```
odb_Plastic&
Plastic(const odb_SequenceSequenceDouble& table,
        const odb_String& hardening,
        bool rate,
        const odb_String& dataType,
        bool strainRangeDependency,
        int numBackstresses,
        bool temperatureDependency,
        int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

hardening

An odb_String specifying the type of hardening. Possible values are "ISOTROPIC", "KINEMATIC", "COMBINED", "JOHNSON_COOK", and "USER". The default value is "ISOTROPIC".

rate

A Boolean specifying whether the data depend on rate. The default value is false.

MATERIAL COMMANDS

dataType

An odb_String specifying the type of combined hardening. This argument is only valid if *hardening*="COMBINED". Possible values are "HALF_CYCLE", "PARAMETERS", and "STABILIZED". The default value is "HALF_CYCLE".

strainRangeDependency

A Boolean specifying whether the data depend on strain range. This argument is only valid if *hardening*="COMBINED" and *dataType*="STABILIZED". The default value is false.

numBackstresses

An Int specifying the number of backstresses. This argument is only valid if *hardening*="COMBINED". The default value is 1.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *hardening*=ISOTROPIC, or if *hardening*=COMBINED and *dataType*=HALF_CYCLE, the table data specify the following:

- Yield stress.
- Plastic strain.
- Equivalent plastic strain rate, $\dot{\epsilon}^{pl}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=COMBINED and *dataType*=STABILIZED, the table data specify the following:

- Yield stress.
- Plastic strain.
- Strain range, if the data depend on strain range.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=COMBINED and *dataType*=PARAMETERS, the table data specify the following:

- Yield stress at zero plastic strain.
- The first kinematic hardening parameter, C_1 .
- The first kinematic hardening parameter, γ_1 .

- If applicable, the second kinematic hardening parameter, C_2 .
- If applicable, the second kinematic hardening parameter, γ_2 .
- Etc.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *hardening*=KINEMATIC, the table data specify the following:

- Yield stress.
- Plastic strain.
- Temperature, if the data depend on temperature.

If *hardening*=JOHNSON_COOK, the table data specify the following:

- A.
- B.
- n.
- m.
- Melting temperature.
- Transition temperature.

If *hardening*=USER, the table data specify the following:

- Hardening properties.

Return value

A Plastic object.

Exceptions

RangeError.

60.77.2 Members

The Plastic object has members with the same names and descriptions as the arguments to the **Plastic** method.

In addition, the Plastic object can have the following members:

Prototype

```
odb_RateDependent rateDependent() const;
odb_Potential potential() const;
```

MATERIAL COMMANDS

```
odb_CyclicHardening cyclicHardening() const;  
odb_Ornl ornl() const;  
odb_CycledPlastic cycledPlastic() const;  
odb_AnnealTemperature annealTemperature() const;
```

rateDependent

A RateDependent object.

potential

A Potential object.

cyclicHardening

A CyclicHardening object.

ornl

An Ornl object.

cycledPlastic

A CycledPlastic object.

annealTemperature

An AnnealTemperature object.

60.77.3 Corresponding analysis keywords

*PLASTIC

60.78 PoreFluidExpansion object

The PoreFluidExpansion object specifies the thermal expansion coefficient for a hydraulic fluid.

Access

```
materialApi.materials() [name].poreFluidExpansion()
```

60.78.1 PoreFluidExpansion(...)

This method creates a PoreFluidExpansion object.

Path

```
materialApi.materials() [name].PoreFluidExpansion
```

Prototype

```
odb_PoreFluidExpansion&
PoreFluidExpansion(const odb_SequenceSequenceDouble& table,
                    double zero,
                    bool temperatureDependency,
                    int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*zero*

A Double specifying the value of θ^0 . The default value is 0.0.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Mean coefficient of thermal expansion, α .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PoreFluidExpansion object.

Exceptions

RangeError.

60.78.2 Members

The PoreFluidExpansion object has members with the same names and descriptions as the arguments to the **PoreFluidExpansion** method.

60.78.3 Corresponding analysis keywords

*EXPANSION

60.79 PorousBulkModuli object

The PorousBulkModuli object defines bulk moduli for soils and rocks.

Access

```
materialApi.materials() [name].porousBulkModuli()
```

60.79.1 PorousBulkModuli(...)

This method creates a PorousBulkModuli object.

Path

```
materialApi.materials() [name].PorousBulkModuli
```

Prototype

```
odb_PorousBulkModuli&
PorousBulkModuli(const odb_SequenceSequenceDouble& table,
                  bool temperatureDependency);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional argument

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

Table data

- Bulk modulus of solid grains.
- Bulk modulus of permeating fluid.
- Temperature, if the data depend on temperature.

Return value

A PorousBulkModuli object.

Exceptions

None.

60.79.2 Members

The PorousBulkModuli object has members with the same names and descriptions as the arguments to the `PorousBulkModuli` method.

60.79.3 Corresponding analysis keywords

*POROUS BULK MODULI

60.80 PorousElastic object

The PorousElastic object specifies elastic material properties for porous materials.

Access

```
materialApi.materials() [name].porousElastic()
```

60.80.1 PorousElastic(...)

This method creates a PorousElastic object.

Path

```
materialApi.materials() [name].PorousElastic
```

Prototype

```
odb_PorousElastic&
PorousElastic(const odb_SequenceSequenceDouble& table,
               const odb_String& shear,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

shear

An odb_String specifying the shear definition form. Possible values are "G" and "POISSON". The default value is "POISSON".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *shear*=G, the table data specify the following:

- The logarithmic bulk modulus, κ . (Dimensionless.)
- The shear modulus, G .
- The elastic tensile limit, p_t^{el} . (This value cannot be negative.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *shear*=POISSON, the table data specify the following:

- The logarithmic bulk modulus, κ . (Dimensionless.)
- The Poisson's ratio, ν .
- The elastic tensile limit, p_t^{el} . (This value cannot be negative.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PorousElastic object.

Exceptions

RangeError.

60.80.2 Members

The PorousElastic object has members with the same names and descriptions as the arguments to the `PorousElastic` method.

60.80.3 Corresponding analysis keywords

*POROUS ELASTIC

60.81 PorousFailureCriteria object

The PorousFailureCriteria object specifies the material failure criteria for a porous metal.

Access

```
materialApi.materials() [name].porousMetalPlasticity() \
.porousFailureCriteria()
```

60.81.1 PorousFailureCriteria(...)

This method creates a PorousFailureCriteria object.

Path

```
materialApi.materials() [name].porousMetalPlasticity() \
.PorousFailureCriteria
```

Prototype

```
odb_PorousFailureCriteria&
PorousFailureCriteria(double fraction,
                      double criticalFraction);
```

Required arguments

None.

Optional arguments

fraction

A Double specifying the void volume fraction at total failure, $f_F > 0$. The default value is 1.0.

criticalFraction

A Double specifying the critical void volume fraction, $f_c \geq 0$. The default value is 1.0.

Return value

A PorousFailureCriteria object.

Exceptions

RangeError.

60.81.2 Members

The PorousFailureCriteria object has members with the same names and descriptions as the arguments to the **PorousFailureCriteria** method.

60.81.3 Corresponding analysis keywords

*POROUS FAILURE CRITERIA

60.82 PorousMetalPlasticity object

The PorousMetalPlasticity object specifies a porous metal plasticity model.

Access

`materialApi.materials() [name].porousMetalPlasticity()`

60.82.1 PorousMetalPlasticity(...)

This method creates a PorousMetalPlasticity object.

Path

`materialApi.materials() [name].PorousMetalPlasticity`

Prototype

```
odb_PorousMetalPlasticity&
PorousMetalPlasticity(const odb_SequenceSequenceDouble& table,
                      odb_Union relativeDensity,
```

```
bool temperatureDependency,
int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*relativeDensity*

The string "NONE" or a Double specifying the initial relative density of the material, r_0 . The default value is "NONE".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- q_1 .
- q_2 .
- q_3 .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PorousMetalPlasticity object.

Exceptions

RangeError.

60.82.2 Members

The PorousMetalPlasticity object has members with the same names and descriptions as the arguments to the **PorousMetalPlasticity** method.

In addition, the PorousMetalPlasticity object can have the following members:

Prototype

```
odb_PorousFailureCriteria porousFailureCriteria() const;
```

```
odb_VoidNucleation voidNucleation() const;
```

porousFailureCriteria

A PorousFailureCriteria object.

voidNucleation

A VoidNucleation object.

60.82.3 Corresponding analysis keywords

*POROUS METAL PLASTICITY

60.83 Potential object

The Potential object defines an anisotropic yield/creep model.

Access

```
materialApi.materials() [name].creep().potential()  
materialApi.materials() [name].plastic().potential()  
materialApi.materials() [name].viscous().potential()
```

60.83.1 Potential(...)

This method creates a Potential object.

Path

```
materialApi.materials() [name].creep().Potential  
materialApi.materials() [name].plastic().Potential  
materialApi.materials() [name].viscous().Potential
```

Prototype

```
odb_Potential&  
Potential(const odb_SequenceSequenceDouble& table,  
          bool temperatureDependency,  
          int dependencies);
```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*temperatureDependency*

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- R_{11} .
- R_{22} .
- R_{33} .
- R_{12} .
- R_{13} .
- R_{23} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Potential object.

Exceptions

RangeError.

60.83.2 Members

The Potential object has members with the same names and descriptions as the arguments to the **Potential** method.

60.83.3 Corresponding analysis keywords

*POTENTIAL

60.84 PressureEffect object

The PressureEffect object defines equivalent pressure stress driven mass diffusion.

Access

```
materialApi.materials() [name].diffusivity().pressureEffect()
```

60.84.1 PressureEffect(...)

This method creates a PressureEffect object.

Path

```
materialApi.materials() [name].diffusivity().PressureEffect
```

Prototype

```
odb_PressureEffect&
PressureEffect(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Pressure stress factor, κ_p .
- Concentration.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A PressureEffect object.

Exceptions

RangeError.

60.84.2 Members

The PressureEffect object has members with the same names and descriptions as the arguments to the `PressureEffect` method.

60.84.3 Corresponding analysis keywords

*KAPPA

60.85 RateDependent object

The RateDependent object defines a rate-dependent viscoplastic model.

Access

```
materialApi.materials() [name].crushableFoam().rateDependent()
materialApi.materials() [name].druckerPrager().rateDependent()
materialApi.materials() [name].plastic().rateDependent()
```

60.85.1 RateDependent(...)

This method creates a RateDependent object.

Path

```
materialApi.materials() [name].crushableFoam().RateDependent
materialApi.materials() [name].druckerPrager().RateDependent
materialApi.materials() [name].plastic().RateDependent
```

Prototype

```
odb_RateDependent&
```

MATERIAL COMMANDS

```
RateDependent(const odb_SequenceSequenceDouble& table,  
              const odb_String& type,  
              bool temperatureDependency,  
              int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of rate-dependent data. Possible values are "POWER_LAW", "YIELD_RATIO", and "JOHNSON_COOK". The default value is "POWER_LAW".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=POWER_LAW, the table data specify the following:

- D .
- n .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=YIELD_RATIO, the table data specify the following:

- Yield stress ratio, $R = \bar{\sigma}/\sigma^0$.
- Equivalent plastic strain rate, $\dot{\epsilon}^{pl}$.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=JOHNSON_COOK, the table data specify the following:

- C .
- $\dot{\epsilon}_0$.

Return value

A RateDependent object.

Exceptions

RangeError.

60.85.2 Members

The RateDependent object has members with the same names and descriptions as the arguments to the `RateDependent` method.

60.85.3 Corresponding analysis keywords

*RATE DEPENDENT

60.86 Ratios object

The Ratios object specifies ratios that define anisotropic swelling.

Access

```
materialApi.materials() [name].moistureSwelling().ratios()
materialApi.materials() [name].swelling().ratios()
```

60.86.1 Ratios(...)

This method creates a Ratios object.

Path

```
materialApi.materials() [name].moistureSwelling().Ratios
materialApi.materials() [name].swelling().Ratios
```

Prototype

```
odb_Ratios&
Ratios(const odb_SequenceSequenceDouble& table,
        bool temperatureDependency,
```

MATERIAL COMMANDS

```
int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- r_{11} .
- r_{22} .
- r_{33} .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Ratios object.

Exceptions

RangeError.

60.86.2 Members

The Ratios object has members with the same names and descriptions as the arguments to the **Ratios** method.

60.86.3 Corresponding analysis keywords

*RATIOS

60.87 Regularization object

The Regularization object defines the tolerance to be used for regularizing material data.

Access

```
materialApi.materials() [name].regularization()
```

60.87.1 Regularization(...)

This method creates a Regularization object.

Path

```
materialApi.materials() [name].Regularization
```

Prototype

```
odb_Regularization&  
Regularization(double rtol,  
               const odb_String& strainRateRegularization);
```

Required arguments

None.

Optional arguments

rtol

A Double specifying the tolerance to be used for regularizing material data. The default value is 0.03.

strainRateRegularization

An odb_String specifying the form of regularization of strain-rate-dependent material data. Possible values are "LOGARITHMIC" and "LINEAR". The default value is "LOGARITHMIC".

Return value

A Regularization object.

Exceptions

RangeError.

60.87.2 Members

The Regularization object has members with the same names and descriptions as the arguments to the **Regularization** method.

60.87.3 Corresponding analysis keywords

*DASHPOT

60.88 SaturationDependence object

The SaturationDependence object specifies the dependence of the permeability of a material on the saturation of the wetting liquid.

Access

`materialApi.materials() [name].permeability().saturationDependence()`

60.88.1 SaturationDependence(...)

This method creates a SaturationDependence object.

Path

`materialApi.materials() [name].permeability().SaturationDependence`

Prototype

```
odb_SaturationDependence&
SaturationDependence(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- k_s . (Dimensionless.)

- Saturation, s . (Dimensionless.)

Return value

A SaturationDependence object.

Exceptions

RangeError.

60.88.2 Members

The SaturationDependence object has members with the same names and descriptions as the arguments to the `SaturationDependence` method.

60.88.3 Corresponding analysis keywords

*PERMEABILITY

60.89 ShearRetention object

The ShearRetention object defines the reduction of the shear modulus associated with crack surfaces in a Concrete model as a function of the tensile strain across the crack.

Access

```
materialApi.materials() [name].concrete().shearRetention()
```

60.89.1 ShearRetention(...)

This method creates a ShearRetention object.

Path

```
materialApi.materials() [name].concrete().ShearRetention
```

Prototype

```
odb_ShearRetention&
ShearRetention(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
```

```
int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- ρ^{close} for dry concrete. The default value is 1.0.
- ϵ^{max} for dry concrete. The default value is a very large number (full shear retention).
- ρ^{close} for wet concrete. The default value is 1.0.
- ϵ^{max} for wet concrete. The default value is a very large number (full shear retention).
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A ShearRetention object.

Exceptions

RangeError.

60.89.2 Members

The ShearRetention object has members with the same names and descriptions as the arguments to the **ShearRetention** method.

60.89.3 Corresponding analysis keywords

*SHEAR RETENTION

60.90 ShearTestData object

The ShearTestData object specifies the normalized shear creep compliance or relaxation modulus as a function of time.

Access

```
materialApi.materials() [name].viscoelastic().shearTestData()
```

60.90.1 ShearTestData(...)

This method creates a ShearTestData object.

Path

```
materialApi.materials() [name].viscoelastic().ShearTestData
```

Prototype

```
odb_ShearTestData&
ShearTestData(const odb_SequenceSequenceDouble& table,
               odb_Union shrink);
```

Required argument

table

An odb_SequenceSequenceDouble specifying values that depend on the *time* member of the Viscoelastic object.

If *time*="RELAXATION_TEST_DATA", the table data specify the following:

- Normalized shear relaxation modulus $g_R(t)$. ($0 \leq g_R(t) \leq 1$).
- Time t . ($t > 0$).

If *time*="CREEP_TEST_DATA", the table data specify the following:

- Normalized shear compliance $j_S(t)$. ($j_S(t) \geq 1$).
- Time t . ($t > 0$).

Optional argument

shrink

The string "NONE" or a Double specifying a normalized shear. The value of *shrink* depends on the value of the *time* member of the Viscoelastic object. The default value is "NONE".

If *time*="RELAXATION_TEST_DATA", *shrinf* specifies the value of the long-term, normalized shear modulus $g_R(\infty)$.

If *time*="CREEP_TEST_DATA", *shrinf* specifies the value of the long-term, normalized shear compliance $j_S(\infty)$.

Return value

A ShearTestData object.

Exceptions

None.

60.90.2 Members

The ShearTestData object has members with the same names and descriptions as the arguments to the **ShearTestData** method.

60.90.3 Corresponding analysis keywords

*SHEAR TEST DATA

60.91 SimpleShearTestData object

The SimpleShearTestData object provides simple shear test data.

Access

```
materialApi.materials() [name].hyperfoam().simpleShearTestData()
```

60.91.1 SimpleShearTestData(...)

This method creates a SimpleShearTestData object.

Path

```
materialApi.materials() [name].hyperfoam().SimpleShearTestData
```

Prototype

```
odb_SimpleShearTestData&
```



```
SimpleShearTestData(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- Nominal shear stress, T_S .
- Nominal shear strain, γ .
- Nominal transverse stress, T_T (normal to edge with shear stress). This stress value is optional.

Return value

A SimpleShearTestData object.

Exceptions

None.

60.91.2 Members

The SimpleShearTestData object has members with the same names and descriptions as the arguments to the **SimpleShearTestData** method.

60.91.3 Corresponding analysis keywords

*SIMPLE SHEAR TEST DATA

60.92 Solubility object

The Solubility object specifies solubility.

Access

```
materialApi.materials() [name].solubility()
```

60.92.1 Solubility(...)

This method creates a Solubility object.

Path

```
materialApi.materials() [name].Solubility
```

Prototype

```
odb_Solubility&  
Solubility(const odb_SequenceSequenceDouble& table,  
            bool temperatureDependency,  
            int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Solubility.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Solubility object.

Exceptions

RangeError.

60.92.2 Members

The Solubility object has members with the same names and descriptions as the arguments to the `Solubility` method.

60.92.3 Corresponding analysis keywords

*SOLUBILITY

60.93 SoretEffect object

The SoretEffect object defines temperature gradient driven mass diffusion.

Access

```
materialApi.materials() [name].diffusivity().soretEffect()
```

60.93.1 SoretEffect(...)

This method creates a SoretEffect object.

Path

```
materialApi.materials() [name].diffusivity().SoretEffect
```

Prototype

```
odb_SoretEffect&
SoretEffect(const odb_SequenceSequenceDouble& table,
             bool temperatureDependency,
             int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Soret effect factor, κ_S .
- Concentration.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A SoretEffect object.

Exceptions

RangeError.

60.93.2 Members

The SoretEffect object has members with the same names and descriptions as the arguments to the `SoretEffect` method.

60.93.3 Corresponding analysis keywords

*KAPPA

60.94 Sorption object

The Sorption object defines absorption and exsorption behaviors of a partially saturated porous medium in the analysis of coupled wetting liquid flow and porous medium stress.

Access

`materialApi.materials() [name].sorption()`

60.94.1 Sorption(...)

This method creates a Sorption object.

Path

`materialApi.materials() [name].Sorption`

Prototype

```
odb_Sorption&
Sorption(const odb_SequenceSequenceDouble& absorptionTable,
         const odb_String& lawAbsorption,
         bool exsorption,
         const odb_String& lawExsorption,
         double scanning,
         const odb_SequenceSequenceDouble& exsorptionTable);
```

Required argument

absorptionTable

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

lawAbsorption

An odb_String specifying absorption behavior. Possible values are "LOG" and "TABULAR". The default value is "TABULAR".

exsorption

A Boolean specifying whether the exsorption data is specified. The default value is false.

lawExsorption

An odb_String specifying exsorption behavior. Possible values are "LOG" and "TABULAR". The default value is "TABULAR".

scanning

A Double specifying the slope of the scanning line, $(du_w/ds)|_s$. This slope must be positive and larger than the slope of the absorption or exsorption behaviors. The default value is 0.0.

exsorptionTable

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

Table data

If *lawAbsorption*=TABULAR or *lawExsorption*=TABULAR, the *absorptionTable* and *exsorptionTable* data respectively specify the following:

- Pore pressure, u_w .

MATERIAL COMMANDS

- Saturation, s .

If *lawAbsorption*=LOG or *lawExsorption*=LOG, the *absorptionTable* and *exsorptionTable* data respectively specify the following:

- A.
- B.
- s_0 .
- s_1 .

Return value

A Sorption object.

Exceptions

RangeError.

60.94.2 Members

The Sorption object has members with the same names and descriptions as the arguments to the **Sorption** method.

60.94.3 Corresponding analysis keywords

*SORPTION

60.95 SpecificHeat object

The SpecificHeat object specifies a material's specific heat.

Access

```
materialApi.materials() [name].specificHeat()
```

60.95.1 SpecificHeat(...)

This method creates a SpecificHeat object.

Path

```
materialApi.materials()[name].SpecificHeat
```

Prototype

```
odb_SpecificHeat&
SpecificHeat(const odb_SequenceSequenceDouble& table,
             const odb_String& law,
             bool temperatureDependency,
             int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

law

An odb_String specifying the specific heat behavior. Possible values are "CONSTANTVOLUME" and "CONSTANTPRESSURE". The default value is "CONSTANTVOLUME".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Specific heat per unit mass.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A SpecificHeat object.

Exceptions

RangeError.

60.95.2 Members

The `SpecificHeat` object has members with the same names and descriptions as the arguments to the `SpecificHeat` method.

60.95.3 Corresponding analysis keywords

*SPECIFIC HEAT

60.96 Swelling object

The Swelling object specifies time-dependent volumetric swelling for a material.

Access

```
materialApi.materials() [name].swelling()
```

60.96.1 Swelling(...)

This method creates a Swelling object.

Path

```
materialApi.materials() [name].Swelling
```

Prototype

```
odb_Swelling&  
Swelling(const odb_SequenceSequenceDouble& table,  
         const odb_String& law,  
         bool temperatureDependency,  
         int dependencies);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.
This argument is valid only when *law*="INPUT".

Optional arguments*law*

An odb_String specifying the type of data defining the swelling behavior. Possible values are "INPUT" and "USER". The default value is "INPUT".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Volumetric swelling strain rate.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Swelling object.

Exceptions

RangeError.

60.96.2 Members

The Swelling object has members with the same names and descriptions as the arguments to the **Swelling** method.

In addition, the Swelling object can have the following member:

Prototype

```
odb_Ratios ratios() const;
```

ratios

A Ratios object.

60.96.3 Corresponding analysis keywords

*SWELLING

60.97 **TensionCutoff object**

The TensionCutoff object specifies tension cutoff for different material models for example the Mohr-Coulomb plasticity model.

Access

```
materialApi.materials() [name].mohrCoulombPlasticity().tensionCutoff()
```

60.97.1 **TensionCutoff(...)**

This method creates a TensionCutoff object.

Path

```
materialApi.materials() [name].mohrCoulombPlasticity().TensionCutoff
```

Prototype

```
odb_TensionCutoff&
TensionCutoff(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- Tension cutoff stress.
- The value of the corresponding tensile plastic strain.(The first tabular value entered must always be zero.)
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.

- Value of the second field variable.
- Etc.

Return value

A TensionCutOff object.

Exceptions

RangeError.

60.97.2 Members

The TensionCutOff object has members with the same names and descriptions as the arguments to the `TensionCutOff` method.

60.97.3 Corresponding analysis keywords

*TENSION CUTOFF

60.98 TensionStiffening object

The TensionStiffening object defines the retained tensile stress normal to a crack in a Concrete model.

Access

```
materialApi.materials() [name].concrete().tensionStiffening()
```

60.98.1 TensionStiffening(...)

This method creates a TensionStiffening object.

Path

```
materialApi.materials() [name].concrete().TensionStiffening
```

Prototype

```
odb_TensionStiffening&
TensionStiffening(const odb_SequenceSequenceDouble& table,
                  const odb_String& type,
```

MATERIAL COMMANDS

```
bool temperatureDependency,  
int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying how the postcracking behavior is defined. Possible values are "DISPLACEMENT" and "STRAIN". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=STRAIN, the table data specify the following:

- Fraction of remaining stress to stress at cracking.
- Absolute value of the direct strain minus the direct strain at cracking.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

If *type*=DISPLACEMENT, the table data specify the following:

- Displacement, u_0 , at which a linear loss of strength after cracking gives zero stress.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A TensionStiffening object.

Exceptions

RangeError.

60.98.2 Members

The `TensionStiffening` object has members with the same names and descriptions as the arguments to the `TensionStiffening` method.

60.98.3 Corresponding analysis keywords

*TENSION STIFFENING

60.99 TriaxialTestData object

The `TriaxialTestData` object provides triaxial test data.

Access

```
materialApi.materials() [name].druckerPrager().triaxialTestData()
```

60.99.1 TriaxialTestData(...)

This method creates a `TriaxialTestData` object.

Path

```
materialApi.materials() [name].druckerPrager().TriaxialTestData
```

Prototype

```
odb_TriaxialTestData&
TriaxialTestData(const odb_SequenceSequenceDouble& table,
                  odb_Union a,
                  odb_Union b,
                  odb_Union pt);
```

Required argument

table

An `odb_SequenceSequenceDouble` specifying the items described below.

Optional arguments

a

The string "NONE" or a Double specifying the value of the material constant *a*. "NONE" is used when the value is unknown or it is not held fixed at the input value. The default value is "NONE".

b

The string "NONE" or a Double specifying the value of the material constant *b*. "NONE" is used when the value is unknown or it is not held fixed at the input value. The default value is "NONE".

p_t

The string "NONE" or a Double specifying the value of the material constant *p_t*. "NONE" is used when the value is unknown or it is not held fixed at the input value. The default value is "NONE".

Table data

- Sign and magnitude of confining stress, $\sigma_1 = \sigma_2$.
- Sign and magnitude of the stress in loading direction, σ_3 .

Return value

A TriaxialTestData object.

Exceptions

RangeError.

60.99.2 Members

The TriaxialTestData object has members with the same names and descriptions as the arguments to the **TriaxialTestData** method.

60.99.3 Corresponding analysis keywords

*TRIAXIAL TEST DATA

60.100 Trs object

The Trs object defines the temperature-time shift for time history viscoelastic analysis.

Access

```
materialApi.materials() [name].viscoelastic().trs()
materialApi.materials() [name].viscosity().trs()
```

60.100.1 Trs(...)

This method creates a Trs object.

Path

```
materialApi.materials() [name].viscoelastic().Trs
materialApi.materials() [name].viscosity().Trs
```

Prototype

```
odb_Tr&
Trs(const odb_String& definition,
    const odb_SequenceSequenceDouble& table);
```

Required arguments

None.

Optional arguments*definition*

An odb_String specifying the definition of the shift function. Possible values are "WLF", "ARRHENIUS", and "USER". The default value is "WLF".

table

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

This argument is valid only when *definition*="WLF".

Table data

- Reference temperature, θ_0 .
- Calibration constant, C_1 .
- Calibration constant, C_2 .

Return value

A Trs object.

Exceptions

None.

60.100.2 Members

The Trs object has members with the same names and descriptions as the arguments to the **Trs** method.

60.100.3 Corresponding analysis keywords

*TRS

60.101 UniaxialTestData object

The UniaxialTestData object provides uniaxial test data (compression and/or tension).

Access

```
materialApi.materials() [name].hyperelastic().uniaxialTestData()  
materialApi.materials() [name].hyperfoam().uniaxialTestData()  
materialApi.materials() [name].lowDensityFoam() \  
.uniaxialCompressionTestData()  
materialApi.materials() [name].lowDensityFoam().uniaxialTensionTestData()  
materialApi.materials() [name].mullinsEffect().uniaxialTests(i)
```

60.101.1 UniaxialTestData(...)

This method creates a UniaxialTestData object.

Path

```
materialApi.materials() [name].hyperelastic().UniaxialTestData  
materialApi.materials() [name].hyperfoam().UniaxialTestData  
materialApi.materials() [name].lowDensityFoam().UniaxialTestData  
materialApi.materials() [name].mullinsEffect().UniaxialTestData
```

Prototype

```
odb_UniaxialTestData&  
UniaxialTestData(const odb_SequenceSequenceDouble& table,
```



```

    odb_Union smoothing,
    bool lateralNominalStrain,
    bool temperatureDependency,
    int dependencies);

```

Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*smoothing*

The string "NONE" or an Int specifying the value for smoothing. If *smoothing*="NONE", no smoothing is employed. The default value is "NONE".

lateralNominalStrain

A Boolean specifying whether to include lateral nominal strain. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic material model, the table data specify the following:

- Nominal stress, T_U .
- Nominal strain, ϵ_U .

For a hyperfoam material model, the table data specify the following:

- Nominal stress, T_L .
- Nominal strain, ϵ_U .
- Nominal lateral strain, $\epsilon_2 = \epsilon_3$. The default value is 0.

For a low-density foam material model, the table data specify the following:

- Nominal stress, T_U .
- Nominal strain, ϵ_U .
- Nominal strain rate, $\dot{\epsilon}_U$.

Return value

A UniaxialTestData object.

Exceptions

None.

60.101.2 Members

The `UniaxialTestData` object has members with the same names and descriptions as the arguments to the `UniaxialTestData` method.

60.101.3 Corresponding analysis keywords

*UNIAXIAL TEST DATA

60.102 UserDefinedField object

The `UserDefinedField` object redefines field variables at a material point.

Access

```
materialApi.materials() [name].userDefinedField()
```

60.102.1 UserDefinedField()

This method defines a `UserDefinedField` object.

Path

```
materialApi.materials() [name].UserDefinedField
```

Prototype

```
odb_UserDefinedField&  
UserDefinedField();
```

Arguments

None.

Return value

A `UserDefinedField` object.

Exceptions

None.

60.102.2 Members

The UserDefinedField object has no members.

60.102.3 Corresponding analysis keywords

*USER DEFINED FIELD

60.103 UserMaterial object

The UserMaterial object defines material constants for use in subroutines **UMAT**, **UMATHT**, or **VUMAT**.

Access

```
materialApi.materials() [name].userMaterial()
```

60.103.1 UserMaterial(...)

This method creates a UserMaterial object.

Path

```
materialApi.materials() [name].UserMaterial
```

Prototype

```
odb_UserMaterial&
UserMaterial(const odb_String& type,
             bool unsymm,
             const odb_SequenceDouble& mechanicalConstants,
             const odb_SequenceDouble& thermalConstants);
```

Required arguments

None.

Optional arguments

type

An odb_String specifying the type of material behavior defined by the command. Possible values are "MECHANICAL", "THERMAL", and "THERMOMECHANICAL". The default value is "MECHANICAL".

MATERIAL COMMANDS

unsymm

A Boolean specifying if the material stiffness matrix, $\partial\Delta\sigma/\partial\Delta\varepsilon$, is not symmetric or, when a thermal constitutive model is used, if $\partial f/\partial(\partial\theta/\partial\mathbf{x})$ is not symmetric. The default value is false.

This argument is valid only for an Abaqus/Standard analysis.

mechanicalConstants

An odb_SequenceDouble specifying the mechanical constants of the material. This argument is valid only when *type*="MECHANICAL" or "THERMOMECHANICAL". The default value is an empty sequence.

thermalConstants

An odb_SequenceDouble specifying the thermal constants of the material. This argument is valid only when *type*="THERMAL" or "THERMOMECHANICAL". The default value is an empty sequence.

Return value

A UserMaterial object.

Exceptions

RangeError.

60.103.2 Members

The UserMaterial object has members with the same names and descriptions as the arguments to the **UserMaterial** method.

60.103.3 Corresponding analysis keywords

*USER MATERIAL

60.104 UserOutputVariables object

The UserOutputVariables object specifies the number of user-defined output variables.

Access

`materialApi.materials()[name].userOutputVariables()`

60.104.1 UserOutputVariables(...)

This method creates a UserOutputVariables object.

Path

```
materialApi.materials() [name].UserOutputVariables
```

Prototype

```
odb_UserOutputVariables&  
UserOutputVariables(int n);
```

Required arguments

None.

Optional argument

n

An Int specifying the number of user-defined variables required at each material point. The default value is 0.

Return value

A UserOutputVariables object.

Exceptions

RangeError.

60.104.2 Members

The UserOutputVariables object has members with the same names and descriptions as the arguments to the **UserOutputVariables** method.

60.104.3 Corresponding analysis keywords

*USER OUTPUT VARIABLES

60.105 VelocityDependence object

The VelocityDependence object specifies the dependence of the permeability of a material on the velocity of fluid flow.

Access

```
materialApi.materials() [name].permeability().velocityDependence()
```

60.105.1 VelocityDependence(...)

This method creates a VelocityDependence object.

Path

```
materialApi.materials() [name].permeability().VelocityDependence
```

Prototype

```
odb_VelocityDependence&  
VelocityDependence(const odb_SequenceSequenceDouble& table);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

None.

Table data

- β . Only $\beta > 0.0$ is allowed.
- Void ratio, e .

Return value

A VelocityDependence object.

Exceptions

RangeError.

60.105.2 Members

The VelocityDependence object has members with the same names and descriptions as the arguments to the `VelocityDependence` method.

60.105.3 Corresponding analysis keywords

*PERMEABILITY

60.106 Viscoelastic object

The Viscoelastic object specifies dissipative behavior for use with elasticity.

Access

```
materialApi.materials() [name].viscoelastic()
```

60.106.1 Viscoelastic(...)

This method creates a Viscoelastic object.

Path

```
materialApi.materials() [name].Viscoelastic
```

Prototype

```
odb_Viscoelastic&
Viscoelastic(const odb_String& domain,
             const odb_SequenceSequenceDouble& table,
             const odb_String& frequency,
             const odb_String& type,
             const odb_String& preload,
             const odb_String& time,
             double errrtol,
             int nmax,
             const odb_SequenceSequenceDouble& volumetricTable);
```

Required arguments

domain

An odb_String specifying the domain definition. Possible values are:

- "FREQUENCY", specifying a frequency domain. This domain is only available for an Abaqus/Standard analysis.
- "TIME", specifying a time domain.

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

frequency

An odb_String specifying the frequency domain definition. This argument is required only when *domain*="FREQUENCY". Possible values are "FORMULA", "TABULAR", "PRONY", "CREEP_TEST_DATA", and "RELAXATION_TEST_DATA". The default value is "FORMULA".

type

An odb_String specifying the type. This argument is required only when *domain*="FREQUENCY" and *frequency*="TABULAR". Possible values are "ISOTROPIC" and "TRACTION". The default value is "ISOTROPIC".

preload

An odb_String specifying the preload. This argument is required only when *domain*="FREQUENCY" and *frequency*="TABULAR". Possible values are "NONE", "UNIAXIAL", "VOLUMETRIC", and "UNIAXIAL_VOLUMETRIC". The default value is "NONE".

time

An odb_String specifying the time domain definition. This argument is required only when *domain*="TIME". Possible values are "PRONY", "CREEP_TEST_DATA", "RELAXATION_TEST_DATA", and "FREQUENCY_DATA". The default value is "PRONY".

errtol

A Double specifying the allowable average root-mean-square error of the data points in the least-squares fit. The Float values correspond to percentages; for example, 0.01 is 1%. The default value is 0.01.

This argument is valid only when *time*="CREEP_TEST_DATA", "RELAXATION_TEST_DATA" or "FREQUENCY_DATA"; or only when *frequency*="CREEP_TEST_DATA" or "RELAXATION_TEST_DATA".

nmax

An Int specifying the maximum number of terms *N* in the Prony series. The maximum value is 13. The default value is 13.

This argument is valid only when *time*="CREEP_TEST_DATA", "RELAXATION_TEST_DATA" or "FREQUENCY_DATA"; or only when *frequency*="CREEP_TEST_DATA" or "RELAXATION_TEST_DATA".

volumetricTable

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

Table data

If *frequency*=FORMULA, the table data for *table* specify the following:

- Real part of g_1^* ($g^*(\omega) = g_1^* f^{-a}$).
- Imaginary part of g_1^* .
- Value of a .
- Real part of k_1^* ($k^*(\omega) = k_1^* f^{-b}$). If the material is incompressible, this value is ignored.
- Imaginary part of k_1^* . If the material is incompressible, this value is ignored.
- Value of b . If the material is incompressible, this value is ignored.

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=NONE, or *time*=FREQUENCY_DATA the table data for *table* specify the following:

- Real part of ωg^* ($\omega \Re(g^*) = G_\ell / G_\infty$).
- Imaginary part of ωg^* ($\omega \Im(g^*) = 1 - G_s / G_\infty$).
- Real part of ωk^* ($\omega \Re(k^*) = K_\ell / K_\infty$). If the material is incompressible, this value is ignored.
- Imaginary part of ωk^* ($\omega \Im(k^*) = 1 - K_s / K_\infty$). If the material is incompressible, this value is ignored.
- Frequency f in cycles per time.

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=UNIAXIAL the table data for *table* specify the following:

- Loss modulus.
- Storage modulus.
- Frequency.
- Uniaxial strain.

If *frequency*=TABULAR and *type*=TRACTION and *preload*=NONE the table data for *table* specify the following:

- Normalized loss modulus.
- Normalized shear modulus.
- Frequency.

If *frequency*=TABULAR and *type*=TRACTION and *preload*=UNIAXIAL or *preload*=UNIAXIAL_VOLUMETRIC the table data for *table* specify the following:

- Loss modulus.

MATERIAL COMMANDS

- Storage modulus.
- Frequency.
- Closure.

If *time*=PRONY or *frequency*=PRONY, the table data for *table* specify the following:

- $\bar{g}_1^P$, the modulus ratio in the first term in the Prony series expansion of the shear relaxation modulus.
- $\bar{k}_1^P$, the modulus ratio in the first term in the Prony series expansion of the bulk relaxation modulus.
- τ_1 , the relaxation time for the first term in the Prony series expansion.

If *frequency*=TABULAR and *type*=ISOTROPIC and *preload*=VOLUMETRIC or *preload*=UNIAXIAL_VOLUMETRIC the table data for *volumetricTable* specify the following:

- Loss modulus.
- Storage modulus.
- Frequency.
- Volume ratio.

Return value

A Viscoelastic object.

Exceptions

RangeError.

60.106.2 Members

The Viscoelastic object has members with the same names and descriptions as the arguments to the **Viscoelastic** method.

In addition, the Viscoelastic object can have the following members:

Prototype

```
odb_CombinedTestData combinedTestData() const;  
odb_ShearTestData shearTestData() const;  
odb_TrS trs() const;  
odb_VolumetricTestData volumetricTestData() const;
```

combinedTestData

A CombinedTestData object.

shearTestData

A ShearTestData object.

trs

A Trs object.

volumetricTestData

A VolumetricTestData object.

60.106.3 Corresponding analysis keywords

*VISCOELASTIC

60.107 Viscosity object

The Viscosity object specifies mechanical viscosity.

Access

`materialApi.materials() [name].viscosity()`

60.107.1 Viscosity(...)

This method creates a Viscosity object.

Path

`materialApi.materials() [name].Viscosity`

Prototype

```
odb_Viscosity&
Viscosity(const odb_SequenceSequenceDouble& table,
          const odb_String& type,
          bool temperatureDependency,
          int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

type

An odb_String specifying the type of viscosity. The default value is "NEWTONIAN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

MATERIAL COMMANDS

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *type*=NEWTONIAN, the table data specify the following:

- Viscosity, k .
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Viscosity object.

Exceptions

RangeError.

60.107.2 Members

The Viscosity object has members with the same names and descriptions as the arguments to the **Viscosity** method.

In addition, the Viscosity object can have the following member:

Prototype

```
odb_Trs trs() const;
```

trs

A Trs object.

60.107.3 Corresponding analysis keywords

*VISCOSITY

60.108 Viscous object

The Viscous object specifies the viscous properties for a two-layer viscoplastic material model.

Access

```
materialApi.materials() [name].viscous()
```

60.108.1 Viscous(...)

This method creates a Viscous object.

Path

```
materialApi.materials() [name].Viscous
```

Prototype

```
odb_Viscous&
Viscous(const odb_SequenceSequenceDouble& table,
        const odb_String& law,
        bool temperatureDependency,
        int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

law

An odb_String specifying the creep law. Possible values are "STRAIN", "TIME", and "USER". The default value is "STRAIN".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

If *law*=STRAIN or *law*=TIME, the table data specify the following:

- *A*.
- *n*.
- *m*.
- *f*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.

MATERIAL COMMANDS

- Etc.

If *law*=USER, the table data specify the following:

- *f*.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A Viscous object.

Exceptions

None.

60.108.2 Members

The Viscous object has members with the same names and descriptions as the arguments to the **Viscous** method.

In addition, the Viscous object can have the following member:

Prototype

```
odb_Potential potential() const;
```

potential

A Potential object.

60.108.3 Corresponding analysis keywords

*VISCOUS

60.109 VoidNucleation object

The VoidNucleation object defines the nucleation of voids in a porous material.

Access

```
materialApi.materials() [name].porousMetalPlasticity().voidNucleation()
```

60.109.1 VoidNucleation(...)

This method creates a VoidNucleation object.

Path

```
materialApi.materials()[name].porousMetalPlasticity().VoidNucleation
```

Prototype

```
odb_VoidNucleation&
VoidNucleation(const odb_SequenceSequenceDouble& table,
               bool temperatureDependency,
               int dependencies);
```

Required argument

table

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

- ε_N , the mean value of the nucleation-strain normal distribution.
- s_N , the standard deviation of the nucleation-strain normal distribution.
- f_N , the volume fraction of nucleating voids.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A VoidNucleation object.

Exceptions

RangeError.

60.109.2 Members

The VoidNucleation object has members with the same names and descriptions as the arguments to the `VoidNucleation` method.

60.109.3 Corresponding analysis keywords

*VOID NUCLEATION

60.110 VolumetricTestData object

The VolumetricTestData object provides volumetric test data.

Access

```
materialApi.materials() [name].hyperelastic().volumetricTestData()  
materialApi.materials() [name].hyperfoam().volumetricTestData()  
materialApi.materials() [name].viscoelastic().volumetricTestData()
```

60.110.1 VolumetricTestData(...)

This method creates a VolumetricTestData object.

Path

```
materialApi.materials() [name].hyperelastic().VolumetricTestData  
materialApi.materials() [name].hyperfoam().VolumetricTestData  
materialApi.materials() [name].viscoelastic().VolumetricTestData
```

Prototype

```
odb_VolumetricTestData&  
VolumetricTestData(const odb_SequenceSequenceDouble& table,  
                   odb_Union volinf,  
                   odb_Union smoothing,  
                   bool temperatureDependency,  
                   int dependencies);
```


Required argument*table*

An odb_SequenceSequenceDouble specifying the items described below.

Optional arguments*volinf*

The string "NONE" or a Double specifying a normalized volumetric value that depends on the value of the *time* member of the Viscoelastic object. The default value is "NONE".

If *time*="RELAXATION_TEST_DATA", *volinf* specifies the value of the long-term, normalized volumetric modulus, $k_R(\infty)$.

If *time*="CREEP_TEST_DATA", *volinf* specifies the value of the long-term, normalized volumetric compliance, $j_K(\infty)$.

This argument is valid only for a viscoelastic material model.

smoothing

The string "NONE" or an Int specifying the value for smoothing. If *smoothing*="NONE", no smoothing is employed. The default value is "NONE".

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

Table data

For a hyperelastic or hyperfoam material model, the table data specify the following:

- Pressure, p .
- Volume ratio, J (current volume/original volume).

For a viscoelastic material model, the values depend on the value of the *time* member of the Viscoelastic object.

If *time*=RELAXATION_TEST_DATA, the table data specify the following:

- Normalized volumetric (bulk) modulus $k_R(t)$, ($0 \leq k_R(t) \leq 1$).
- Time t ($t > 0$).

If *time*=CREEP_TEST_DATA, the table data specify the following:

- Normalized volumetric (bulk) compliance $j_K(t)$, ($j_K(t) \geq 1$).
- Time t ($t > 0$).

Return value

A VolumetricTestData object.

Exceptions

None.

60.110.2 Members

The VolumetricTestData object has members with the same names and descriptions as the arguments to the **VolumetricTestData** method.

60.110.3 Corresponding analysis keywords

*VOLUMETRIC TEST DATA

61. Odb commands

The C++ ODB API commands are used to read and write data from an output database (**.odb**) file. In this chapter the **Access** and **Path** statements refer to a variable called **odb** that represents an existing Odb object.

61.1 Odb object

The Odb object is the in-memory representation of an output database (ODB) file.

Access

odb

61.1.1 Odb(...)

This method creates a new Odb object.

Path

Odb

Prototype

```
odb_Odb&
Odb(const odb_String& name,
     const odb_String& analysisTitle,
     const odb_String& description,
     const odb_String& path);
```

Required argument

name

An odb_String specifying the repository key.

Optional arguments

analysisTitle

An odb_String specifying the title of the output database. The default value is an empty string.

description

An odb_String specifying the description of the output database. The default value is an empty string.

path

An odb_String specifying the path to the file where the new output database (**.odb**) file will be written. The default value is an empty string.

Return value

An Odb object.

Exceptions

None.

61.1.2 close()

This method closes an output database.

Prototype

```
void  
close();
```

Arguments

None.

Return value

None

Exceptions

None.

61.1.3 getFrame(...)

This method returns the frame at the specified time, frequency, or mode. It will not interpolate values between frames. The method is not applicable to an Odb object containing steps with different domains or to an Odb object containing a step with load case specific data.

Prototype

```
odb_Frame  
getFrame(double frameValue,  
          odb_Enum::odb_MatchEnum match);
```

Arguments

Required argument

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the total time or frequency.

Optional argument

match

An odb_Enum::odb_MatchEnum specifying which frame to return if there is no frame at the exact frame value. Possible values are odb_Enum::CLOSEST, odb_Enum::BEFORE, odb_Enum::AFTER, and odb_Enum::EXACT. The default value is odb_Enum::CLOSEST.

When *match*=odb_Enum::CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=odb_Enum::EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the exact frame is not found:

OdbError: Frame not found.

61.1.4 save()

This method saves output to an output database (**.odb**) file.

Prototype

```
void
save();
```

Arguments

None.

Return value

None

Exceptions

OdbError

Database save failed. The database was opened as read-only. Modification of data is not permitted.

61.1.5 update()

This method is used to update an Odb object in memory while an Abaqus analysis writes data to the associated output database. **update** checks if additional steps have been written to the output database since it was opened or last updated. If additional steps have been written to the output database, **update** adds them to the Odb object.

Prototype

```
bool  
update();
```

Arguments

None.

Return value

A Boolean specifying whether additional steps or frames were added to the Odb object.

Exceptions

None.

61.1.6 hasSectorDefinition()

This method checks whether or not a valid SectorDefinition object, indicating a cyclic symmetry model, is present.

Prototype

```
bool  
hasSectorDefinition();
```

Arguments

None.

Return value

A Boolean specifying whether a valid sector definition is available.

Exceptions

None.

61.1.7 Members

The Odb object has members with the same names and descriptions as the arguments to the **Odb** method.

In addition, the Odb object can have the following members:

Prototype

```
odb_String name() const;
    odb_String analysisTitle() const;
    odb_String description() const;
    odb_String path() const;
    bool isReadOnly() const;
    odb_Assembly& rootAssembly();
    odb_JobData jobData() const;
    odb_PartRepository& parts();
    odb_StepRepository& steps();
    odb_SectionCategoryRepository& sectionCategories();
    odb_SectorDefinition& sectorDefinition();
    odb_InteractionRepository& interactions();
    odb_InteractionPropertyRepository& interactionProperties();
    odb_ConstraintRepository& constraints();
```

isReadOnly

A Boolean specifying whether the output database was opened with read-only access.

rootAssembly

An OdbAssembly object.

jobData

A JobData object.

parts

A repository of OdbPart objects.

steps

A repository of OdbStep objects.

sectionCategories

A repository of SectionCategory objects.

sectorDefinition

A SectorDefinition object.

userData

A UserData object.

61.2 AnalyticSurface object

The AnalyticSurface object is a geometric surface that can be described with straight and/or curved line segments.

Access

```
odb.parts() [name].analyticSurface()
odb.rootAssembly().instances() [name].analyticSurface()
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.analyticSurface()
```

61.2.1 Members

The AnalyticSurface object has the following members:

Prototype

```
odb_String name() const;
odb_String type() const;
double filletRadius() const;
odb_SequenceSequenceFloat localCoordData() const;
odb_SequenceAnalyticSurfaceSegment segments() const;
```

name

An odb_String specifying the name of the analytic surface.

type

An odb_Enum::odb_typeEnum specifying the type of AnalyticSurface object. Possible values are odb_Enum::SEGMENTS, odb_Enum::CYLINDER, and odb_Enum::REVOLUTION.

filletRadius

A Double specifying radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

segments

An OdbSequenceAnalyticSurfaceSegment object specifying the profile associated with the surface.

localCoordData

An odb_SequenceSequenceFloat specifying the global coordinates of points representing the local coordinate system, if used.

61.3 AnalyticSurfaceSegment object

An individual segment of the analytic surface.

Access

```
odb.parts()[name].analyticSurface().segments(i)
odb.rootAssembly().instances()[name].analyticSurface().segments(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.analyticSurface().segments(i)
```

61.3.1 Members

The AnalyticSurfaceSegment object has the following members:

Prototype

```
odb_String type() const;
odb_SequenceSequenceFloat data() const;
```

type

An odb_Enum::odb_typeEnum specifying the type of AnalyticSurfaceSegment. Possible values are odb_Enum::START, odb_Enum::LINE, odb_Enum::CIRCLE, and odb_Enum::PARABOLA.

data

An odb_SequenceSequenceFloat specifying the coordinates of point/s representing the segment of the AnalyticSurface object. If *type*=odb_Enum::CIRCLE, the first row contains coordinates of the end point and the second row contains coordinates of the center point. If *type*=odb_Enum::PARABOLA, the first row contains coordinates of the middle point and the second row contains coordinates of the end point. If *type*=odb_Enum::START or *type*=odb_Enum::LINE, a single row contains coordinates of the start/end point.

61.4 BeamOrientation object

The BeamOrientation object represents the direction of the first beam section axis $\mathbf{n}_1$. Specifying the beam orientation using an additional node in the element connectivity list is not supported.

Access

```
odb.parts() [name].beamOrientations(i)
odb.rootAssembly().instances() [name].beamOrientations(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.beamOrientations(i)
```

61.4.1 Members

The BeamOrientation object can have the following members:

Prototype

```
odb_Set region() const;
odb_Enum::odb_OrientationMethodEnum method() const;
odb_SequenceFloat vector() const;
```

method

An odb_Enum::odb_OrientationMethodEnum specifying the orientation assignment method. Possible values are odb_Enum::N1_COSINES, odb_Enum::CSYS, and odb_Enum::VECT.

region

An OdbSet object specifying a region for which the beam orientation is defined.

vector

An odb_SequenceFloat specifying direction cosines of the $\mathbf{n}_1$ -direction of the beam cross-section.

61.5 FieldBulkData object

The FieldBulkData object represents the entire field data for a class of elements or nodes. All elements in a class correspond to the same element type and material.

Access

```
odb.steps() [name].frames(i).fieldOutputs() [name].bulkDataBlocks(i)
```

61.5.1 Members

The FieldBulkData object can have the following members:

Prototype

```
odb_Enum::odb_ResultPositionEnum position() const;
        odb_Enum::odb_PrecisionEnum precision() const;
int* elementLabels() const;
```

```

int* nodeLabels() const;
int* integrationPoints() const;
odb_String baseElementType() const;
odb_Enum::odb_ElementFaceEnum* faces() const;
odb_Enum::odb_DataTypeEnum type() const;
float* data() const;
double* dataDouble() const;
float* conjugateData() const;
double* conjugateDataDouble() const;
float* mises() const;
float* localCoordSystem() const;
double* localCoordSystemDouble() const;
int orientationWidth() const;
int numberOfElements() const;
int length() const;
int valuesPerElement() const;
int width() const;
const odb_Instance& instance() const;
const odb_SectionPoint& sectionPoint() const;
odb_SequenceString componentLabels() const;

```

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::ELEMENT_FACE, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

precision

An odb_Enum::odb_PrecisionEnum specifying the precision of the output. Possible values are:

- odb_Enum::SINGLE_PRECISION, specifying that the output is in single precision.
- odb_Enum::DOUBLE_PRECISION, specifying that the output is in double precision.

type

An odb_Enum::odb_DataTypeEnum specifying the output type. Possible values are odb_Enum::SCALAR, odb_Enum::VECTOR, odb_Enum::TENSOR_3D_FULL,

ODB COMMANDS

odb_Enum::TENSOR_3D_PLANAR, odb_Enum::TENSOR_3D_SURFACE,
odb_Enum::TENSOR_2D_PLANAR, and odb_Enum::TENSOR_2D_SURFACE.

orientationWidth

An Int specifying the number of direction cosines necessary to specify the local coordinate system at each output location. You use *orientationWidth* to read the orientation data from the *localCoordSystem*.

numberOfElements

An Int specifying the number of elements in the current block of data.

length

An Int specifying the number of output locations in the current block of data.

valuesPerElement

An Int specifying the number of values per element in the current block of data. If *position*=odb_Enum::ELEMENT_NODAL, *valuesPerElement* is the number of nodes per element for all elements in the current block of data.

width

An Int specifying the number of components at each output location.

baseElementType

An odb_String specifying the element type corresponding to the current block of data.

instance

An OdbInstance object specifying the part to which the labels belong.

sectionPoint

A SectionPoint object specifying the section point number of the current block of data.

elementLabels

A pointer to an array of Ints specifying the element labels of the elements in the block. *elementLabels* is valid only if *position* =odb_Enum::INTEGRATION_POINT , odb_Enum::CENTROID , odb_Enum::ELEMENT_NODAL , or odb_Enum::ELEMENT_FACE. If *position* =odb_Enum::NODAL , *elementLabels* returns a NULL pointer.

nodeLabels

A pointer to an array of Ints specifying the node labels of the nodes in the block. *nodeLabels* is valid only if *position* =odb_Enum::NODAL or odb_Enum::ELEMENT_NODAL. If *position* =odb_Enum::INTEGRATION_POINT , odb_Enum::CENTROID , odb_Enum::ELEMENT_NODAL , or odb_Enum::ELEMENT_FACE , *nodeLabels* returns a NULL pointer.

integrationPoints

A pointer to an array of Ints specifying the integration points in the elements in the block. *integrationPoint* is available only if *position* =odb_Enum::INTEGRATION_POINT.

faces

A pointer to an array of odb_Enum::odb_ElementFaceEnum enumerations specifying the faces of the elements in the block. *faces* is available only if *position* =odb_Enum::ELEMENT_FACE.

data

A pointer to an array of Floats specifying the field's data in the order described by *type*. If *type* = odb_Enum::TENSOR or odb_Enum::VECTOR, *data* is an array containing the components for each element or node in the block. If the underlying data are in double precision, an exception will be thrown.

dataDouble

A pointer to an array of Doubles specifying the field's data in the order described by *type*. If *type* = odb_Enum::TENSOR or odb_Enum::VECTOR, *data* is an array containing the components for each element or node in the block. If the underlying data are in single precision, an exception will be thrown.

conjugateData

A pointer to an array of Floats specifying the imaginary portion of a complex result. The order of the Floats is described by *type*. If *type* = odb_Enum::TENSOR or odb_Enum::VECTOR, *conjugateData* is an array containing the imaginary part of the components for each element or node in the block. If the underlying data are in double precision, an exception will be thrown.

conjugateDataDouble

A pointer to an array of Doubles specifying the imaginary portion of a complex result. The order of the Doubles is described by *type*. If *type* = odb_Enum::TENSOR or odb_Enum::VECTOR, *conjugateData* is an array containing the imaginary part of the components for each element or node in the block. If the underlying data are in single precision, an exception will be thrown.

mises

A pointer to an array of Floats specifying the calculated von Mises stress at each output location in the block of element data, or NULL. If *validInvariants* includes odb_Enum::MISES, *mises* returns an array pointer. If *validInvariants* does not include odb_Enum::MISES, *mises* returns a NULL pointer. Conjugate data will be ignored in invariant calculation.

localCoordSystem

A pointer to an array of Floats specifying the quaternion representing the local coordinate system at each output location. The quaternion is returned in the form

$$\mathbf{q} = (\mathbf{q}, q_0)$$

, which is the reverse of that shown in "Rotation variables," Section 1.3.1 of the Abaqus Theory Guide. *localCoordSystem* is available only for odb_Enum::TENSOR data written in a local coordinate system. If the underlying data are in double precision, an exception will be thrown.

localCoordSystemDouble

A pointer to an array of Doubles specifying the quaternion representing the local coordinate system at each output location. The quaternion is returned in the form

$$\mathbf{q} = (\mathbf{q}, q_0)$$

, which is the reverse of that shown in "Rotation variables," Section 1.3.1 of the Abaqus Theory Guide. *localCoordSystemDouble* is available only for odb_Enum::TENSOR data written in a local coordinate system. If the underlying data are in single precision, an exception will be thrown.

componentLabels

An odb_SequenceString specifying the component labels.

61.6 FieldLocation object

The FieldLocation object specifies locations for which data are available in the field. For example, a displacement field will have a FieldLocation object with a *position* member value of NODAL. The FieldLocation object has no constructor; it is created automatically as an element of the *location* member of a FieldOutput object by the **addData** method of a FieldOutput object.

Access

`odb.steps()[name].frames(i).fieldOutputs()[name].locations(i)`

61.6.1 Members

The FieldLocation object can have the following members:

Prototype

```
odb_Enum::odb_ResultPositionEnum position() const;
const odb_SequenceSectionPoint& sectionPoints() const;
const odb_SectionPoint& sectionPoints(int index) const;
```

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output in the element. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::ELEMENT_FACE.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

sectionPoints

A sequence of SectionPoint objects.

61.7 FieldOutput object

A FieldOutput object contains field data for a specific output variable.

Access

```
odb.steps()[name].frames(i).fieldOutputs()[name]
```

61.7.1 FieldOutput(...)

This method creates a FieldOutput object.

Path

```
odb.steps()[name].frames(i).FieldOutput
```

Prototype

```
odb_FieldOutput&
FieldOutput(const odb_String& name,
            const odb_String& description,
            odb_Enum::odb_DataTypeEnum type,
            const odb_SequenceString& componentLabels,
            const odb_SequenceInvariant& validInvariants,
            bool isEngineeringTensor);
```

Required arguments

name

An odb_String specifying the output variable name.

description

An odb_String specifying the output variable. Colon (:) should not be used as a part of the field output description.

type

An odb_Enum::odb_DataTypeEnum specifying the output type. Possible values are odb_Enum::SCALAR, odb_Enum::VECTOR, odb_Enum::TENSOR_3D_FULL, odb_Enum::TENSOR_3D_PLANAR, odb_Enum::TENSOR_3D_SURFACE, odb_Enum::TENSOR_2D_PLANAR, and odb_Enum::TENSOR_2D_SURFACE.

Optional arguments

componentLabels

An odb_SequenceString specifying the labels for each component of the value. The length of the sequence must match the type. If *type*=odb_Enum::TENSOR, the default value is *name* with the suffixes ('11', '22', '33', '12', '13', '23'). If *type*=odb_Enum::VECTOR, the default value is *name* with the suffixes ('1', '2', '3'). If *type*=odb_Enum::SCALAR, the default value is an empty sequence.

validInvariants

An odb_SequenceInvariant specifying which invariants should be calculated for this field. An empty sequence indicates that no invariants are valid for this field. Possible values are:

- odb_Enum::MAGNITUDE
- odb_Enum::MISES
- odb_Enum::TRESCA
- odb_Enum::PRESS
- odb_Enum::INV3
- odb_Enum::MAX_PRINCIPAL
- odb_Enum::MID_PRINCIPAL
- odb_Enum::MIN_PRINCIPAL
- odb_Enum::MAX_INPLANE_PRINCIPAL
- odb_Enum::MIN_INPLANE_PRINCIPAL
- odb_Enum::OUTOFPLANE_PRINCIPAL

The default value is an empty sequence.

isEngineeringTensor

A Boolean specifying whether the field is an engineering tensor or not. Setting *isEngineeringTensor* to true makes a tensor field behave as a strain like quantity where the off-diagonal components of tensor are halved for invariants computation. This parameter applies only to tensor field outputs. The default value is false.

Return value

A FieldOutput object.

Exceptions

None.

61.7.2 FieldOutput(...)

This method creates a FieldOutput object from an existing FieldOutput object.

Path

```
odb.steps() [name].frames(i).FieldOutput
```

Prototype

```
odb_FieldOutput&
FieldOutput(const odb_FieldOutput& field,
            const odb_String& name,
            const odb_String& description);
```

Required argument

field

A FieldOutput object.

Optional arguments

name

A String specifying the name of the FieldOutput object.

description

An odb_String specifying the output variable. Colon (:) should not be used as a part of the field output description.

Return value

A FieldOutput object.

Exceptions

None.

61.7.3 VectorOutput(...)

This method creates a FieldOutput object.

Path

```
odb.steps() [ name ].frames( i ).VectorOutput
```

Prototype

```
odb_FieldOutput&
VectorOutput(const odb_String& name,
            const odb_String& description,
            int width);
```

Required arguments

name

An odb_String specifying the output variable name.

description

An odb_String specifying the output variable. Colon (:) should not be used as a part of the field output description.

width

An Int specifying the width of the vector.

Optional arguments

None.

Return value

A FieldOutput object.

Exceptions

None.

61.7.4 MatrixOutput(...)

This method creates a FieldOutput object.

Path

```
odb.steps() [ name ].frames( i ).MatrixOutput
```

Prototype

```
odb_FieldOutput&
MatrixOutput(const odb_String& name,
             const odb_String& description,
             int rows,
             int cols);
```

Required arguments

name

An odb_String specifying the output variable name.

description

An odb_String specifying the output variable. Colon (:) should not be used as a part of the field output description.

rows

An Int specifying the number of rows in the matrix.

cols

An Int specifying the number of columns in the matrix.

Optional arguments

None.

Return value

A FieldOutput object.

Exceptions

None.

61.7.5 SymmetricMatrixOutput(...)

This method creates a FieldOutput object.

Path

```
odb.steps()[ name ].frames( i ).SymmetricMatrixOutput
```

Prototype

```
odb_FieldOutput&
SymmetricMatrixOutput(const odb_String& name,
                      const odb_String& description,
                      int dim);
```

Required arguments

name

An odb_String specifying the output variable name.

description

An odb_String specifying the output variable. Colon (:) should not be used as a part of the field output description.

dim

An Int specifying the dimension of the symmetric matrix.

Optional arguments

None.

Return value

A FieldOutput object.

Exceptions

None.

61.7.6 **addData(...)**

This method adds data to a FieldOutput object.

Prototype

```
void
addData(odb_Enum::odb_ResultPositionEnum position,
        const odb_Instance& instance,
        const odb_SequenceInt& labels,
        const odb_SequenceSequenceFloat& data,
        const odb_SequencePoint& sectionPoint,
        const odb_SequenceSequenceFloat& localCoordSystem,
        const odb_SequenceSequenceFloat& conjugateData);
```

Arguments

Required arguments

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

instance

An OdbInstance object specifying the namespace for labels.

labels

An odb_SequenceInt specifying the labels of the nodes or elements where the values in *data* are located. The node or element labels must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

An odb_SequenceSequenceFloat specifying the data values for the specified *position*, *instance*, and *labels*. The values must be given in the correct order. Element nodal data follow the order of nodal connectivity defined in the Abaqus documentation. Integration point data follow the order defined in the Abaqus documentation. Section point data for beams and shells follow the convention given in the Abaqus documentation. For more information, see Part VI, “Elements,” of the Abaqus Analysis User’s Guide. These data create FieldValue objects internally.

Optional arguments

sectionPoint

A SectionPoint object specifying the location in the section. Although *sectionPoint* is an optional argument to the **addData** method, omitting the argument does have consequences for visualization. If you omit the argument when you are writing field output data for a shell or a beam, you cannot subsequently select the section point to display when you are displaying the field output data using the Visualization module.

localCoordSystem

An odb_SequenceSequenceFloat specifying the 3×3 matrix of direction cosines of the local coordinate system. This argument is available only for fields with type=odb_Enum::TENSOR or odb_Enum::VECTOR.

User-supplied values of localCoordSystem are transposed before storing in the database.

conjugateData

An odb_SequenceSequenceFloat specifying the imaginary data values for the specified position, instance, and labels. You must provide these data when you add complex fields to the output database. The order of the values follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.

Return value

None

Exceptions

The **addData** method throws many exceptions of type odbException. For example, if the local coordinate system is specified for scalar data:

odbException: Transformation not allowed for scalar data.

61.7.7 **addData(...)**

This method adds the data from a field created using the **getSubset** method and mathematical operators to the database. The user must create a field to contain the new data and then use the **addData** method to assign the data from the fields.

Prototype

```
void
addData(const odb_FieldOutput& field);
```

Arguments

Required argument

field
A FieldOutput object specifying the data to add.

Optional arguments

None.

Return value

None

Exceptions

The **addData** method throws many exceptions of type `odbException`. For example, if the local coordinate system is specified for scalar data:

`odbException`: Transformation not allowed for scalar data.

61.7.8 **addData(...)**

This method adds data to a FieldOutput object.

Prototype

```
void
addData(odb_Enum::odb_ResultPositionEnum position,
        const odb_Set& set,
        const odb_SequenceSequenceFloat& data,
        const odb_SectionPoint& sectionPoint,
        const odb_SequenceSequenceFloat& conjugateData);
```

Arguments

Required arguments

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.
- odb_Enum::ELEMENT_FACE_INTEGRATION_POINT, specifying the values calculated at the element face integration points.
- odb_Enum::SURFACE_INTEGRATION_POINT, specifying the values calculated at the surface integration points. Selecting this value prompts the Visualization module to calculate the sum of the values at the ELEMENT_FACE_INTEGRATION_POINT position from multiple surfaces.

set

An OdbSet object specifying the instance-level set defining the region for **addData**. The set must be defined in the same output database as the output database into which the new field output data is being written. The node or element labels in the set must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

An odb_SequenceSequenceFloat specifying the data values for the specified position and labels in the set. Each row of data provides the value at one unique position. The width of each row should match the number of required components for the data. The values must be given in the order that matches the ordering of labels in the set.

The order of the element nodal data, integration point data, and section point data for beams and shells follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.”

Optional arguments

sectionPoint

A SectionPoint object specifying the location in the section. Although *sectionPoint* is an optional argument to the **addData** method, omitting the argument does have consequences for visualization. If you omit the argument when you are writing field output data for a shell or a beam, you cannot subsequently select the section point to display when you are displaying the field output data using the Visualization module.

conjugateData

An odb_SequenceSequenceFloat specifying the imaginary data values for the specified position, instance, and labels. You must provide this data when you add complex fields to the output database. The order of the values follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.

Return value

None

Exceptions

If you specify an odbSet containing entities from multiple instances:

odbException: Entities from multiple instances present in set.

The **addData** method throws many exceptions of type odbException. For example, if the local coordinate system is specified for scalar data:

odbException: Transformation not allowed for scalar data.

61.7.9 addData(...)

This method adds data to a FieldOutput object.

Prototype

```
void
addData(odb_Enum::odb_ResultPositionEnum position,
const odb_Instance& instance,
const odb_SequenceInt& labels,
const odb_SequenceSequenceFloat& data,
const odb_SectionPoint& sectionPoint,
const odb_SequenceSequenceSequenceFloat& localCoordSystem,
const odb_SequenceSequenceFloat& conjugateData);
```

Arguments

Required arguments

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.

- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

instance

An OdbInstance object specifying the namespace for labels.

labels

An odb_SequenceInt specifying the labels of the nodes or elements where the values in *data* are located. The node or element labels must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

An odb_SequenceSequenceFloat specifying the data values for the specified *position*, *instance*, and *labels*. The values must be given in the correct order. Element nodal data follow the order of nodal connectivity defined in the Abaqus documentation. Integration point data follow the order defined in the Abaqus documentation. Section point data for beams and shells follow the convention given in the Abaqus documentation. For more information, see Part VI, “Elements,” of the Abaqus Analysis User’s Guide. These data create FieldValue objects internally.

Optional arguments

sectionPoint

A SectionPoint object specifying the location in the section. Although *sectionPoint* is an optional argument to the **addData** method, omitting the argument does have consequences for visualization. If you omit the argument when you are writing field output data for a shell or a beam, you cannot subsequently select the section point to display when you are displaying the field output data using the Visualization module.

localCoordSystem

An odb_SequenceSequenceSequenceFloat specifying the direction cosines of the local coordinates systems, where the sequence is the same length as *data*. When specified this way a different local coordinate system applies to each data value.

User-supplied values of localCoordSystem are transposed before storing in the database.

conjugateData

An odb_SequenceSequenceFloat specifying the imaginary data values for the specified *position*, *instance*, and *labels*. You must provide this data when you add complex fields to the output database. The order of the values follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.

Return value

None

Exceptions

The **addData** method throws many exceptions of type `odbException`. For example, if the local coordinate system is specified for scalar data:

`odbException`: Transformation not allowed for scalar data.

61.7.10 addData(...)

This method adds double precision data to a `FieldOutput` object.

Prototype

```
void
addData(odb_Enum::odb_ResultPositionEnum position,
        const odb_Instance& instance,
        const odb_SequenceInt& labels,
        const odb_SequenceSequenceDouble& data,
        const odb_SequenceSequenceSequenceDouble& localCoordSystem,
        const odb_SequenceSequenceDouble& conjugateData);
```

Arguments

Required arguments

position

An `odb_Enum::odb_ResultPositionEnum` specifying the position of the output. Only `odb_Enum::NODAL`, specifying the values calculated at the nodes, is supported.

instance

An `OdbInstance` object specifying the namespace for labels.

labels

An `odb_SequenceInt` specifying the labels of the nodes or elements where the values in *data* are located. The node or element labels must be sorted in ascending order and must be specified in the same order as the values provided for the *data* argument.

data

An `odb_SequenceSequenceDouble` specifying the data values for the specified *position*, *instance*, and *labels*. The values must be given in the correct order. These data create `FieldValue` objects internally.

Optional arguments

localCoordSystem

An `odb_SequenceSequenceSequenceDouble` specifying the direction cosines of the local coordinates systems, where the sequence is the same length as *data*. When specified this way a different local coordinate system applies to each data value.

User-supplied values of `localCoordSystem` are transposed before storing in the database.

conjugateData

An `odb_SequenceSequenceDouble` specifying the imaginary data values for the specified *position*, *instance*, and *labels*. You must provide this data when you add complex fields to the output database. The order of the values follows the conventions defined in Part VI, “Elements,” of the Abaqus Analysis User’s Guide.

Return value

None

Exceptions

The **addData** method throws many exceptions of type `odbException`. For example, if the local coordinate system is specified for scalar data:

`odbException: Transformation not allowed for scalar data.`

61.7.11 getScalarField(...)

This method generates a scalar field containing the extracted component or calculated invariant values. The new field will hold values for the same nodes or elements as the parent field. Abaqus will perform this operation on only the real part of the `FieldOutput` object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Prototype

```
odb_FieldOutput
getScalarField(odb_Enum::odb_InvariantEnum invariant);
```

Arguments

Required argument

invariant

An `odb_Enum::odb_InvariantEnum` specifying the invariant. Possible values are `odb_Enum::MAGNITUDE`, `odb_Enum::MISES`, `odb_Enum::TRESCA`, `odb_Enum::PRESS`, `odb_Enum::INV3`, `odb_Enum::MAX_PRINCIPAL`, `odb_Enum::MID_PRINCIPAL`, `odb_Enum::MIN_PRINCIPAL`, `odb_Enum::MAX_INPLANE_PRINCIPAL`, `odb_Enum::MIN_INPLANE_PRINCIPAL`, and `odb_Enum::OUTOFPLANE_PRINCIPAL`.

Optional arguments

None.

Return value

A `FieldOutput` object.

Exceptions

None.

61.7.12 getScalarField(...)

This method generates a scalar field containing the extracted component or calculated invariant values. The new field will hold values for the same nodes or elements as the parent field. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Prototype

```
odb_FieldOutput
getScalarField(const odb_String& componentLabel);
```

Arguments

Required argument

componentLabel

A String specifying the component label, such as “S11”.

Optional arguments

None.

Return value

A FieldOutput object.

Exceptions

None.

61.7.13 getSubset(...)

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput
getSubset(odb_Enum::odb_ResultPositionEnum position,
          bool readOnly);
```

Arguments

Required arguments

None.

Optional arguments

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output in the element. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

If the requested field values are not found in the output database at the specified odb_Enum::ELEMENT_NODAL or odb_Enum::CENTROID positions, they are extrapolated from the field data at the odb_Enum::INTEGRATION_POINT position.

readOnly

A Boolean specifying whether the extrapolated data returned by this call is written to the output database. The default value is false.

Return value

A FieldOutput object.

Exceptions

None.

61.7.14 getSubset(...)

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput
getSubset(const odb_Set& region);
```

Arguments

Required arguments

None.

Optional argument

region

An OdbSet specifying the region for which to extract values. Nodes or elements in the sets must be sorted and arranged in ascending order. Use of unsorted sets is not supported. This is especially true for sets in Abaqus output databases obtained through running the analysis in multiple domains. In such cases, a new sorted set must be created before using it in the getSubset method.

Return value

A FieldOutput object.

Exceptions

None.

61.7.15 getSubset(...)

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput
getSubset(const odb_SequenceSequenceFloat& localCoordSystem);
```

Arguments

Required arguments

None.

Optional argument

localCoordSystem

An odb_SequenceSequenceFloat specifying the 3×3 matrix of direction cosines. Field values associated with the supplied coordinate system will be extracted.

Return value

A FieldOutput object.

Exceptions

None.

61.7.16 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput  
getSubset(const odb_SectionPoint& sectionPoint);
```

Arguments

Required arguments

None.

Optional argument

sectionPoint

A SectionPoint object.

Return value

A FieldOutput object.

Exceptions

None.

61.7.17 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput  
getSubset(const odb_FieldLocation& location);
```

Arguments

Required arguments

None.

Optional argument

location

A FieldLocation object.

Return value

A FieldOutput object.

Exceptions

None.

61.7.18 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput  
getSubset(const odb_Element& region);
```

Arguments

Required arguments

None.

Optional argument

region

An OdbMeshElement specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

61.7.19 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput  
getSubset(const odb_Node& region);
```


Arguments

Required arguments

None.

Optional argument

region

An OdbMeshNode specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

61.7.20 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput
getSubset(const odb_Instance& region);
```

Arguments

Required arguments

None.

Optional argument

region

An OdbInstance specifying the region for which to extract values.

Return value

A FieldOutput object.

Exceptions

None.

61.7.21 **getSubset(...)**

A FieldOutput object with a subset of the field values.

Prototype

```
odb_FieldOutput  
getSubset(const odb_String& elementType);
```

Arguments

Required arguments

None.

Optional argument

elementType

A String specifying the element type for which to extract values. The string must correspond to a valid Abaqus element type.

Return value

A FieldOutput object.

Exceptions

None.

61.7.22 getTransformedField(...)

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Prototype

```
odb_FieldOutput  
getTransformedField(const odb_DatumCsys& datumCsys,  
                    int projected22Axis,  
                    double projectionTol);
```

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

61.7.23 getTransformedField(...)

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Prototype

```
odb_FieldOutput
getTransformedField(const odb_DatumCsys& datumCsys,
                    const odb_FieldOutput& deformationField,
                    int projected22Axis,
                    double projectionTol);
```

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

61.7.24 getTransformedField(...)

This method generates a new vector or tensor field containing the transformed component values of the parent field. The new field will hold values for the same nodes or elements as the parent field. Results will be transformed based on the orientations specified by the input DatumCsys object. Abaqus will perform this operation on only the real part of the FieldOutput object. The operation is not performed on the conjugate data (the imaginary portion of a complex result).

Prototype

```
odb_FieldOutput
getTransformedField(const odb_DatumCsys& datumCsys,
                   const odb_FieldOutput& deformationField,
                   const odb_FieldOutput& rotationField,
                   int projected22Axis,
                   double projectionTol);
```

Arguments

Required argument

datumCsys

A valid DatumCsys object designating the coordinate system. Valid systems can be fixed or positioned with respect to nodes on the model and can be cartesian, cylindrical, or spherical.

Optional arguments

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

rotationField

A FieldOutput object specifying the nodal rotational displacement vectors required by moving coordinate systems that follow a 6-dof node, to determine instantaneous configurations.

projected22Axis

An Int specifying which axis of the coordinate system will be projected as the second component for local result orientations. Valid values are 1, 2, or 3; the default value is 2.

projectionTol

A Double specifying the minimum allowable angle (radians) between the specified projection axis and the element normal. The next axis will be used for projection if this tolerance test fails.

Return value

A FieldOutput object.

Exceptions

The **getTransformedField** method throws an exception if the field contains any assembly level nodes.

odbException: Cannot apply transformation to field containing assembly level nodes.

61.7.25 getConnectorFieldXformedToNodeA(...)

This method generates a new vector field containing the transformed component values of the parent connector field to the node A coordinate system. The new field will hold values for the same connector elements as the parent field. Some connection types such as Axial, Link, Slip Ring, and Radial Thrust require that the deformationField be specified.

Prototype

`odb_FieldOutput`

`getConnectorFieldXformedToNodeA(const odb_FieldOutput& deformationField) const`

Arguments

Required arguments

None.

Optional arguments

deformationField

A FieldOutput object specifying the nodal displacement vectors required by moving coordinate systems to determine instantaneous configurations.

Return value

A FieldOutput object.

Exceptions

The `getConnectorFieldXformedToNodeA` method throws an exception if the field requires a *deformationField* and the argument is not supplied.

odbException: Deformation field is required for transforming this connector field.

61.7.26 setComponentLabels(...)

This method sets the component labels for the FieldOutput object.

Prototype

```
void
setComponentLabels(const odb_SequenceString& componentLabels);
```

Arguments

Required argument

componentLabels

An odb_SequenceString specifying the labels for each component of the value. The length of the sequence must match the type. If *type*=odb_Enum::TENSOR, the default value is *name* with the suffixes ('11', '22', '33', '12', '13', '23'). If *type*=odb_Enum::VECTOR, the default value is *name* with the suffixes ('1', '2', '3'). If *type*=odb_Enum::SCALAR, the default value is an empty sequence.

Return value

None

Exceptions

None.

61.7.27 setDataType(...)

This method sets the data type of a FieldOutput object.

Prototype

```
void
setDataType(odb_Enum::odb_DataTypeEnum type);
```

Arguments

Required argument

type

An odb_Enum::odb_DataTypeEnum specifying the output type. Possible values are odb_Enum::SCALAR, odb_Enum::VECTOR, odb_Enum::TENSOR_3D_FULL, odb_Enum::TENSOR_3D_PLANAR, odb_Enum::TENSOR_3D_SURFACE, odb_Enum::TENSOR_2D_PLANAR, and odb_Enum::TENSOR_2D_SURFACE.

Return value

None

Exceptions

None.

61.7.28 setValidInvariants(...)

This method sets the invariants valid for the FieldOutput object.

Prototype

```
void
setValidInvariants(const odb_SequenceInvariant& validInvariants);
```

Arguments

Required argument

validInvariants

An odb_SequenceInvariant specifying which invariants should be calculated for this field. An empty sequence indicates that no invariants are valid for this field. Possible values are:

- odb_Enum::MAGNITUDE
- odb_Enum::MISES
- odb_Enum::TRESCA

- odb_Enum::PRESS
- odb_Enum::INV3
- odb_Enum::MAX_PRINCIPAL
- odb_Enum::MID_PRINCIPAL
- odb_Enum::MIN_PRINCIPAL
- odb_Enum::MAX_INPLANE_PRINCIPAL
- odb_Enum::MIN_INPLANE_PRINCIPAL
- odb_Enum::OUTOFPLANE_PRINCIPAL

The default value is an empty sequence.

Return value

None

Exceptions

None.

61.7.29 Members

The FieldOutput object has members with the same names and descriptions as the arguments to the **FieldOutput** method.

In addition, the FieldOutput object can have the following members:

Prototype

```
odb_String name() const;
    odb_String description() const;
    odb_Enum::odb_DataTypeEnum type() const;
    odb_Enum::odb_DataSubTypeEnum subtype() const;
    int dim() const;
    int dim2() const;
    bool isComplex() const;
    odb_SequenceInvariant validInvariants() const;
    odb_SequenceString componentLabels() const;
    odb_SequenceString baseElementTypes() const;
    const odb_SequenceFieldLocation& locations() const;
    odb_FieldLocation locations(int index) const;
    const odb_FieldValue values(int i);
    odb_SequenceFieldValue values();
    const odb_FieldBulkData& bulkDataBlocks(int i);
    odb_SequenceFieldBulkData& bulkDataBlocks();
```


dim

An Int specifying the dimension of vector or the first dimension (number of rows) of matrix.

dim2

An Int specifying the second dimension (number of columns) of matrix.

isComplex

A Boolean specifying whether the data are complex.

locations

A sequence of FieldLocation objects.

values

A sequence of FieldValue objects specifying the order of the objects in the array is determined by the Abaqus Scripting Interface; see the *data* argument to the **addData** method for a description of the order.

bulkDataBlocks

A sequence of FieldBulkData objects.

61.8 FieldValue object

The FieldValue object represents the field data at a point. The FieldValue object has no constructor; it is created by the Odb object when data are added to the FieldOutput object using the **addData** method. For faster, bulk-data access, see “Using bulk data access to an output database,” Section 10.10.7 of the Abaqus Scripting User’s Guide.

Access

```
odb.steps()[name].frames(i).fieldOutputs()[name].values(i)
```

61.8.1 Members

The FieldValue object has the following members:

Prototype

```
odb_Enum::odb_ResultPositionEnum position() const;
const odb_Instance& instance() const;
int elementLabel() const;
int nodeLabel() const;
int integrationPoint() const;
odb_Enum::odb_ElementFaceEnum face() const;
odb_SectionPoint sectionPoint() const;
odb_Enum::odb_PrecisionEnum precision() const;
odb_Enum::odb_DataTypeEnum type() const;
```

ODB COMMANDS

```
const float* data(int& numVal) const;
odb_SequenceFloat data() const;
const float* conjugateData(int& numVal) const;
odb_SequenceFloat conjugateData() const;
const double* dataDouble(int& numVal) const;
odb_SequenceDouble dataDouble() const;
const double* conjugateDataDouble(int& numVal) const;
odb_SequenceDouble conjugateDataDouble() const;
odb_SequenceSequenceFloat localCoordSystem() const;
odb_SequenceSequenceDouble localCoordSystemDouble() const;
float magnitude() const;
float mises() const;
float tresca() const;
float press() const;
float inv3() const;
float maxPrincipal() const;
float midPrincipal() const;
float minPrincipal() const;
float maxInPlanePrincipal() const;
float minInPlanePrincipal() const;
float outOfPlanePrincipal() const;
```

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the output in the element. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::ELEMENT_FACE, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element.
- odb_Enum::CENTROID, specifying the value at the centroid obtained by extrapolating results calculated at the integration points.

precision

An odb_Enum::odb_PrecisionEnum specifying the precision of the output in the element. Possible values are:

- odb_Enum::SINGLE_PRECISION, specifying that the output values are in single precision.
- odb_Enum::DOUBLE_PRECISION, specifying that the output values are in double precision.

elementLabel

An Int specifying the element label of the element containing the location. *elementLabel* is available only if *position*=odb_Enum::INTEGRATION_POINT, odb_Enum::CENTROID, odb_Enum::ELEMENT_NODAL, or odb_Enum::ELEMENT_FACE.

nodeLabel

An Int specifying the node label of the node containing the location. *nodeLabel* is available only if *position*=odb_Enum::ELEMENT_NODAL or odb_Enum::NODAL.

integrationPoint

An Int specifying the integration point in the element. *integrationPoint* is available only if *position*=odb_Enum::INTEGRATION_POINT.

face

An odb_Enum::odb_ElementFaceEnum specifying the face of the element. *face* is available only if *position*=odb_Enum::ELEMENT_FACE.

type

An odb_Enum::odb_DataTypeEnum specifying the output type. Possible values are odb_Enum::SCALAR, odb_Enum::VECTOR, odb_Enum::TENSOR_3D_FULL, odb_Enum::TENSOR_3D_PLANAR, odb_Enum::TENSOR_3D_SURFACE, odb_Enum::TENSOR_2D_PLANAR, and odb_Enum::TENSOR_2D_SURFACE.

magnitude

A Float specifying the length or magnitude of the vector. *magnitude* is valid only when *type*=odb_Enum::VECTOR.

mises

A Float specifying the calculated von Mises stress. The value is valid only when the *validInvariants* member includes odb_Enum::MISES; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

tresca

A Float specifying the calculated Tresca stress. The value is valid only when the *validInvariants* member includes odb_Enum::TRESCA; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

press

A Float specifying the calculated pressure stress. The value is valid only when the *validInvariants* member includes odb_Enum::PRESS; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

inv3

A Float specifying the calculated third stress invariant. The value is valid only when the *validInvariants* member includes odb_Enum::INV3; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

maxPrincipal

A Float specifying the calculated maximum principal stress. The value is valid only when the *validInvariants* member includes `odb_Enum::MAX_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

midPrincipal

A Float specifying the calculated intermediate principal stress. The value is valid only when the *validInvariants* member includes `odb_Enum::MID_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

minPrincipal

A Float specifying the minimum principal stress. The value is valid only when the *validInvariants* member includes `odb_Enum::MIN_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

maxInPlanePrincipal

A Float specifying the maximum principal in-plane stress. The value is valid only when the *validInvariants* member includes `odb_Enum::MAX_INPLANE_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

minInPlanePrincipal

A Float specifying the calculated minimum principal in-plane stress. The value is valid only when the *validInvariants* member includes `odb_Enum::MIN_INPLANE_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

outOfPlanePrincipal

A Float specifying the calculated principal out-of-plane stress. The value is valid only when the *validInvariants* member includes `odb_Enum::OUTOFPLANE_PRINCIPAL`; otherwise, the value is indeterminate. Conjugate data will be ignored in invariant calculation.

instance

An `OdbInstance` object specifying the part to which the labels belong.

sectionPoint

A `SectionPoint` object.

localCoordSystem

An `odb_SequenceSequenceFloat` specifying the 3 x 3 matrix of Floats specifying the direction cosines of the local coordinate system. Each sequence represents a row in the direction cosine matrix. *localCoordSystem* is available only for `odb_Enum::TENSOR` data written in a local coordinate system. If the underlying data are in double precision, an exception will be thrown.

localCoordSystemDouble

An `odb_SequenceSequenceDouble` specifying the 3 x 3 matrix of Doubles specifying the direction cosines of the local coordinate system. Each sequence represents a row in the direction cosine matrix. *localCoordSystemDouble* is available only for `odb_Enum::TENSOR` data written in a local coordinate system. If the underlying data are in single precision, an exception will be thrown.

data

A pointer to an array of Floats specifying data in the form described by *type*. If *type*=odb_Enum::TENSOR or odb_Enum::VECTOR, *data* is a sequence containing the components. If the underlying data are in double precision an exception will be thrown.

dataDouble

A pointer to an array of Doubles specifying data in the form described by *type*. If *type*=odb_Enum::TENSOR or odb_Enum::VECTOR, *data* is a sequence containing the components. If the underlying data are in single precision, an exception will be thrown.

conjugateData

A pointer to an array of Floats specifying data in the form described by *type*. If *type*=odb_Enum::TENSOR or odb_Enum::VECTOR, *conjugateData* is a sequence containing the components. If the underlying data are in double precision, an exception will be thrown.

conjugateDataDouble

A pointer to an array of Doubles specifying data in the form described by *type*. If *type*=odb_Enum::TENSOR or odb_Enum::VECTOR, *conjugateData* is a sequence containing the components. If the underlying data are in single precision, an exception will be thrown.

61.9 HistoryOutput object

The HistoryOutput object contains the history output at a point for the specified variable.

Access

`odb.steps() [name].historyRegions() [name].historyOutputs() [name]`

61.9.1 HistoryOutput(...)

This method creates a HistoryOutput object.

Path

`odb.steps() [name].historyRegions() [name].HistoryOutput`

Prototype

```
odb_HistoryOutput&
HistoryOutput(const odb_String& name,
              const odb_String& description,
              odb_Enum::odb_DataTypeEnum type,
              const odb_SequenceInvariant& validInvariants);
```

Required arguments

name

An odb_String specifying the output variable name.

description

An odb_String specifying the output variable.

type

An odb_Enum::odb_DataTypeEnum specifying the output type. Only odb_Enum::SCALAR is currently supported.

Optional argument

validInvariants

An odb_SequenceInvariant specifying which invariants should be calculated for this field. Possible values are odb_Enum::MAGNITUDE, odb_Enum::MISES, odb_Enum::TRESCA, odb_Enum::PRESS, odb_Enum::INV3, odb_Enum::MAX_PRINCIPAL, odb_Enum::MID_PRINCIPAL, and odb_Enum::MIN_PRINCIPAL. The default value is an empty sequence.

Return value

A HistoryOutput object.

Exceptions

None.

61.9.2 addData(...)

This method adds data to the *data* member of the HistoryOutput object.

Prototype

```
void
addData(double frame,
        double value);
```

Arguments

Required arguments

frame

A Double specifying the frame value. *frame* can be specified in step time, frequency, or mode number.

value

A Double specifying the value of the variable at the frame value specified in *frame*.

Optional arguments

None.

Return value

None

Exceptions

None.

61.9.3 **addData(...)**

This method adds data to the *data* member of the HistoryOutput object.

Prototype

```
void
addData(const odb_SequenceFloat& frame,
        const odb_SequenceFloat& value);
```

Arguments

Required arguments

frame

An odb_SequenceFloat specifying the frame values. *frame* can be specified in step time, frequency, or mode number.

value

An odb_SequenceFloat specifying the value of the variable at the frame values specified in *frame*.

Optional arguments

None.

Return value

None

Exceptions

If the length of *frame* is not the same as the length of *value* a ValueError is raised.

61.9.4 **addData(...)**

This method adds data to the *data* member of the HistoryOutput object.

Prototype

```
void  
addData(const odb_SequenceSequenceFloat& data);
```

Arguments

Required argument

data

An odb_SequenceSequenceFloat specifying the pairs (*frameValue*, *value*) where *frameValue* is either time, frequency, or mode and *value* is the value of the specified variable at *frameValue*. (This value depends on the type of the variable.)

Optional arguments

None.

Return value

None

Exceptions

None.

61.9.5 Members

The HistoryOutput object has members with the same names and descriptions as the arguments to the **HistoryOutput** method.

In addition, the HistoryOutput object has the following members:

Prototype

```
odb_String name() const;  
odb_String description() const;  
odb_SequenceSequenceFloat data() const;  
odb_SequenceSequenceFloat conjugateData() const;
```

data

An odb_SequenceSequenceFloat specifying the pairs (*frameValue*, *value*) where *frameValue* is either time, frequency, or mode and *value* is the value of the specified variable at *frameValue*. (This value depends on the type of the variable.)

conjugateData

An odb_SequenceSequenceFloat specifying the imaginary portion of a specified complex variable at each frame value (time, frequency, or mode). The pairs have the form (*frameValue*, *value*).

61.10 HistoryPoint object

The HistoryPoint object specifies the point at which history data will be collected. The HistoryPoint object is a temporary object used as an argument to the **HistoryRegion** method.

Access

```
odb.steps() [name].historyRegions() [name].point()
```

61.10.1 HistoryPoint(...)

This method creates a HistoryPoint object for a node.

Path

HistoryPoint

Prototype

```
odb_HistoryPoint&  
HistoryPoint(const odb_Node& node);
```

Required argument

node

An OdbMeshNode object specifying the node for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

61.10.2 HistoryPoint(...)

This method creates a HistoryPoint object for an element.

Path

`odb.steps()[name].historyRegions()[name].HistoryPoint`

Prototype

```
odb_HistoryPoint&
HistoryPoint(const odb_Element& element,
             int ipNumber,
             const odb_SectionPoint& sectionPoint,
             odb_Enum::odb_ElementFaceEnum face,
             const odb_Node& node);
```

Required argument

element

An OdbMeshElement object specifying the element for which the data are to be collected.

Optional arguments

ipNumber

An Int specifying the integration point. This argument is used to define a history output position of `odb_Enum::INTEGRATION_POINT` or `odb_Enum::ELEMENT_FACE_INTEGRATION_POINT`. The default value is 0.

sectionPoint

A SectionPoint object.

face

An `odb_Enum::odb_ElementFaceEnum` specifying the element face. This argument is used to define a history output position of `odb_Enum::ELEMENT_FACE` or `odb_Enum::ELEMENT_FACE_INTEGRATION_POINT`. Possible values are:

- `odb_Enum::FACE_UNKOWN`, specifying this value indicates that no value has been specified.
- `odb_Enum::FACE1`, specifying this value indicates that element face 1 has been specified.
- `odb_Enum::FACE2`, specifying this value indicates that element face 2 has been specified.
- `odb_Enum::FACE3`, specifying this value indicates that element face 3 has been specified.
- `odb_Enum::FACE4`, specifying this value indicates that element face 4 has been specified.
- `odb_Enum::FACE5`, specifying this value indicates that element face 5 has been specified.
- `odb_Enum::FACE6`, specifying this value indicates that element face 6 has been specified.
- `odb_Enum::SIDE1`, specifying this value indicates that element side 1 has been specified.
- `odb_Enum::SIDE2`, specifying this value indicates element side 2 has been specified.
- `odb_Enum::END1`, specifying this value indicates that element end 1 has been specified.
- `odb_Enum::END2`, specifying this value indicates that element end 2 has been specified.

- odb_Enum::END3, specifying this value indicates that element end 3 has been specified.

The default value is odb_Enum::FACE_UNKNOWN.

node

An OdbMeshNode object specifying the node for which the data are to be collected.

Return value

A HistoryPoint object.

Exceptions

None.

61.10.3 HistoryPoint(...)

This method creates a HistoryPoint object for a region.

Path

`odb.steps()[name].historyRegions()[name].HistoryPoint`

Prototype

```
odb_HistoryPoint&
HistoryPoint(const odb_Set& region);
```

Required argument

region

An OdbSet object specifying the region for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

61.10.4 HistoryPoint(...)

This method creates a HistoryPoint object for the OdbAssembly object.

Path

`odb.steps() [name].historyRegions() [name].HistoryPoint`

Prototype

```
odb_HistoryPoint&  
HistoryPoint(const odb_Assembly& assembly);
```

Required argument

assembly

An OdbAssembly object specifying the assembly for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

61.10.5 HistoryPoint(...)

This method creates a HistoryPoint object for the OdbInstance object.

Path

`odb.steps() [name].historyRegions() [name].HistoryPoint`

Prototype

```
odb_HistoryPoint&  
HistoryPoint(const odb_Instance& instance);
```

Required argument

instance

An OdbInstance object specifying the instance for which the data are to be collected.

Optional arguments

None.

Return value

A HistoryPoint object.

Exceptions

None.

61.10.6 Members

The HistoryPoint object has members with the same names and descriptions as the arguments to the **HistoryPoint** method.

In addition, the HistoryPoint object can have the following members:

Prototype

```
odb_Node node() const;
odb_Element element() const;
int ipNumber() const;
odb_Enum::odb_ElementFaceEnum face() const;
odb_Enum::odb_ResultPositionEnum position() const;
odb_SectionPoint sectionPoint() const;
const odb_Set& region() const;
const odb_Instance& instance() const;
const odb_Assembly& assembly() const;
```

ipNumber

An Int specifying the integration point. This argument is used to define a history output position of odb_Enum::INTEGRATION_POINT or odb_Enum::ELEMENT_FACE_INTEGRATION_POINT. The default value is 0.

face

An odb_Enum::odb_ElementFaceEnum specifying the element face. This argument is used to define a history output position of odb_Enum::ELEMENT_FACE or odb_Enum::ELEMENT_FACE_INTEGRATION_POINT. Possible values are:

- odb_Enum::FACE_UNKOWN, specifying this value indicates that no value has been specified.
- odb_Enum::FACE1, specifying this value indicates that element face 1 has been specified.
- odb_Enum::FACE2, specifying this value indicates that element face 2 has been specified.
- odb_Enum::FACE3, specifying this value indicates that element face 3 has been specified.
- odb_Enum::FACE4, specifying this value indicates that element face 4 has been specified.
- odb_Enum::FACE5, specifying this value indicates that element face 5 has been specified.
- odb_Enum::FACE6, specifying this value indicates that element face 6 has been specified.

ODB COMMANDS

- odb_Enum::SIDE1, specifying this value indicates that element side 1 has been specified.
- odb_Enum::SIDE2, specifying this value indicates element side 2 has been specified.
- odb_Enum::END1, specifying this value indicates that element end 1 has been specified.
- odb_Enum::END2, specifying this value indicates that element end 2 has been specified.
- odb_Enum::END3, specifying this value indicates that element end 3 has been specified.

The default value is odb_Enum::FACE_UNKNOWN.

position

An odb_Enum::odb_ResultPositionEnum specifying the result position of the history point. Possible values are:

- odb_Enum::NODAL, specifying the values calculated at the nodes.
- odb_Enum::ELEMENT_NODAL, specifying the values obtained by extrapolating results calculated at the integration points.
- odb_Enum::INTEGRATION_POINT, specifying the values calculated at the integration points.
- odb_Enum::ELEMENT_FACE, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element.
- odb_Enum::ELEMENT_FACE_INTEGRATION_POINT, specifying the results obtained for surface variables such as cavity radiation that are defined for the surface facets of an element when the surface facets have integration points.
- odb_Enum::WHOLE_ELEMENT, specifying the results obtained for whole element variables.
- odb_Enum::WHOLE_REGION, specifying the results for an entire region of the model.
- odb_Enum::WHOLE_PART_INSTANCE, specifying the results for an entire part instance of the model.
- odb_Enum::WHOLE_MODEL, specifying the results for the entire model.

element

An OdbMeshElement object specifying the element for which the data are to be collected.

sectionPoint

A SectionPoint object.

region

An OdbSet object specifying the region for which the data are to be collected.

assembly

An OdbAssembly object specifying the assembly for which the data are to be collected.

instance

An OdbInstance object specifying the instance for which the data are to be collected.

61.11 HistoryRegion object

The HistoryRegion object contains history data for a single location in the model.

Access

`odb.steps() [name].historyRegions() [name]`

61.11.1 HistoryRegion(...)

This method creates a HistoryRegion object.

Path

`odb.steps() [name].HistoryRegion`

Prototype

```
odb_HistoryRegion&
HistoryRegion(const odb_String& name,
              const odb_String& description,
              const odb_HistoryPoint& point,
              const odb_LoadCase& loadCase);
```

Required arguments

name

An odb_String specifying the name of the HistoryRegion object.

description

An odb_String specifying the description of the HistoryRegion object.

point

A HistoryPoint object specifying the point to which the history data refer.

Optional argument

loadCase

An OdbLoadCase object specifying the load case associated with the HistoryRegion object.

Return value

A HistoryRegion object.

Exceptions

None.

61.11.2 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Prototype

```
odb_HistoryRegion  
getSubset(const odb_String& variableName);
```

Arguments

Required argument

variableName

A String specifying the name of the output variable to return.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

61.11.3 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Prototype

```
odb_HistoryRegion  
getSubset(float start);
```

Arguments

Required argument

start

A Float specifying the start of the subset. This is the same as the first item in the data array member of the HistoryOutput object.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

61.11.4 **getSubset(...)**

This method returns a subset of the data in the HistoryRegion object.

Prototype

```
odb_HistoryRegion
getSubset(float start,
          float end);
```

Arguments

Required arguments

start

A Float specifying the start of the subset. This is the same as the first item in the data array member of the HistoryOutput object.

end

A Float specifying the end of the subset.

Optional arguments

None.

Return value

A HistoryRegion object.

Exceptions

None.

61.11.5 **Members**

The HistoryRegion object has members with the same names and descriptions as the arguments to the **HistoryRegion** method.

In addition, the HistoryRegion object can have the following members:

Prototype

```
odb_String description() const;
const odb_LoadCase& loadCase() const;
odb_String name() const;
odb_HistoryPoint point() const;
odb_Enum::odb_ResultPositionEnum position() const;
const odb_HistoryOutputRepository& historyOutputs();
```

position

An odb_Enum::odb_ResultPositionEnum specifying the position of the history output. Possible values are odb_Enum::NODAL, odb_Enum::INTEGRATION_POINT, odb_Enum::WHOLE_ELEMENT, odb_Enum::WHOLE_REGION, and odb_Enum::WHOLE_MODEL.

historyOutputs

A repository of HistoryOutput objects.

61.12 JobData object

The JobData object describes the context in which the analysis was run.

Access

```
odb.jobData()
```

61.12.1 Members

The JobData object has the following members:

Prototype

```
odb_String name() const;
    odb_String analysisCode() const;
    odb_String version() const;
    odb_String dateRun() const;
    odb_String machineName() const;
    odb_String precision() const;
```

name

An odb_String specifying the name of the job.

analysisCode

An odb_Enum::odb_AnalysisCodeEnum specifying the analysis code. Possible values are odb_Enum::ABAQUS_STANDARD, odb_Enum::ABAQUS_EXPLICIT, and odb_Enum::UNKNOWN_ANALYSIS_CODE.

precision

An odb_Enum::odb_PrecisionEnum specifying the precision. Only odb_Enum::SINGLE_PRECISION is currently supported. Possible values are odb_Enum::DOUBLE_PRECISION and odb_Enum::SINGLE_PRECISION.

version

An odb_String specifying the release of the analysis code.

creationTime

An odb_String specifying the date and time at which the analysis was run.

modificationTime

An odb_String specifying the date and time at which the database was last modified.

machineName

An odb_String specifying the name of the machine on which the analysis was run.

productAddOns

An odb_String specifying an odb_Sequence of productAddOns. Possible values are: Possible values are odb_Enum::AQUA, odb_Enum::DESIGN, odb_Enum::BIORID, odb_Enum::CEL, odb_Enum::SOLITER, and odb_Enum::CAVPARALLEL.

61.13 OdbAssembly object

The OdbAssembly object has no constructor; it is created automatically when an Odb object is created. Abaqus creates the *rootAssembly* member when an Odb object is created.

Access

`odb.rootAssembly()`

61.13.1 ConnectorOrientation(...)

This method assigns a connector orientation to a connector region.

Prototype

```
void
ConnectorOrientation(const odb_Set& region,
                    const odb_DatumCsys& csys1,
                    odb_Enum::odb_AxisEnum axis1,
```

```
float angle1,
const odb_DatumCsys& csys2,
odb_Enum::odb_AxisEnum axis2,
float angle2);
```

Arguments

Required arguments

region

An OdbSet specifying a region.

csys1

An OdbDatumCsys object specifying the first connector node local coordinate system.

axis1

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation of the first connector node is applied. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3.

angle1

A Float specifying the angle of the additional rotation about the first connector node axis. The default value is 0.0.

csys2

An OdbDatumCsys object specifying the second connector node local coordinate system.

axis2

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation of the second connector node is applied. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3.

angle2

A Float specifying the angle of the additional rotation about the second connector node axis.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Connector orientation assignment requires element set.

61.13.2 ConnectorOrientation(...)

This method assigns a connector orientation to a connector region.

Prototype

```
void
ConnectorOrientation(const odb_Set& region,
                    const odb_DatumCsys& csys1,
                    odb_Enum::odb_AxisEnum axis1,
                    float angle1,
                    bool orient2sameAs1);
```

Arguments

Required arguments

region

An OdbSet specifying a region.

csys1

An OdbDatumCsys object specifying the first connector node local coordinate system.

axis1

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation of the first connector node is applied. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3.

angle1

A Float specifying the angle of the additional rotation about the first connector node axis.

Optional arguments

orient2sameAs1

A Boolean specifying whether the same orientation settings should be used for the second node of the connector. The default value is false.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Connector orientation assignment requires element set.

61.13.3 SectionAssignment(...)

This method is used to assign a section to a region on an instance. Only connector sections can be assigned at the assembly level.

Prototype

```
void
SectionAssignment(const odb_Set& region,
                  const odb_section& section);
```

Arguments

Required arguments

region

An OdbSet specifying a region.

section

A Section object.

Optional arguments

None.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Section assignment requires element set.

61.13.4 addElements(...)

This method is used to define elements using nodes defined at the OdbAssembly and/or OdbInstance level. For connector elements connected to ground, specify the lone node in the connectivity. The position of the ground node cannot be specified. This is a limitation.

WARNING: Adding elements not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Prototype

```
void
addElements(const odb_SequenceInt& labels,
             const odb_SequenceSequenceInt& connectivity,
             const odb_SequenceString& instanceNames,
             const odb_String& type,
             const odb_String& elementSetName,
```

```
const odb_SectionCategory& sectionCategory);
```

Arguments

Required arguments

labels

An odb_SequenceInt specifying the element labels.

connectivity

An odb_SequenceSequenceInt specifying the nodal connectivity.

instanceNames

An odb_SequenceString specifying the instanceNames of each node in the nodal connectivity array. If the node is defined at the assembly level, the instance name should be an empty string

type

A String specifying the element type.

Optional arguments

elementSetName

A String specifying a name for this element set. The default value is the empty string.

sectionCategory

A SectionCategory object for this element set.

Return value

None

Exceptions

Only certain element types are permitted at the assembly level. e.g., connector elements.

OdbError: Addition of this element type is not permitted at the assembly level

If length of label array does not match connectivity data length:

OdbError: Connectivity array must be provided for all element

61.13.5 addNodes(...)

This method adds nodes to the OdbAssembly object using node labels and coordinates.

WARNING: Adding nodes not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Prototype

void

addNodes(const odb_SequenceInt& labels,

```
const odb_SequenceSequenceFloat& coordinates,  
const odb_String& nodeSetName);
```

Arguments

Required arguments

labels

An odb_SequenceInt specifying the node labels.

coordinates

An odb_SequenceSequenceFloat specifying the nodal coordinates.

Optional argument

nodeSetName

A String specifying a name for this node set. The default value is None.

Return value

None

Exceptions

If length of labels does not match length of coordinates:

OdbError: Number of node labels and coordinates does not match

If width of coordinate array does not match assembly dimension:

OdbError: Node location specification does not correspond to part dimensions

61.13.6 sectionAssignments(...)

This method is used to retrieve a section assignment.

Prototype

```
odb_SectionAssignment  
sectionAssignments(int index);
```

Arguments

Required argument

index

An Int specifying the section assignment.

Optional arguments

None.

Return value

A SectionAssignment object.

Exceptions

None.

61.13.7 RigidBody(...)

This method defines an OdbRigidBody on the assembly.

Prototype

```
void
RigidBody(const odb_Set& referenceNode,
          odb_Enum::odb_PositionEnum position,
          bool isothermal,
          const odb_Set& elset,
          const odb_Set& pinNodes,
          const odb_Set& tieNodes);
```

Arguments

Required argument

referenceNode

An OdbSet specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are odb_Enum::INPUT and odb_Enum::CENTER_OF_MASS. The default value is odb_Enum::INPUT.

isothermal

A Boolean specifying an isothermal rigid body. The default value is false. This parameter is used only for a fully coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

61.13.8 Members

The OdbAssembly object can have the following members:

Prototype

```
odb_InstanceRepository& instances();  
    odb_SetRepository& nodeSets();  
    odb_SetRepository& elementSets();  
    odb_SetRepository& surfaces();  
    odb_SequenceNode& nodes() const;  
    odb_SequenceElement& elements() const;  
    odb_DatumCsysRepository& datumCsyses();  
    odb_SequenceSectionAssignment sectionAssignments();  
    odb_SequenceConnectorOrientation connectorOrientations();  
    odb_SequenceRigidBody rigidBodies();  
    odb_SequencePretensionSection pretensionSections();
```

instances

A repository of OdbInstance objects.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

nodes

A sequence of OdbMeshNode objects.

elements

A sequence of OdbMeshElement objects.

datumCsyses

A repository of OdbDatumCsys objects.

sectionAssignments

A sequence of SectionAssignment objects.

rigidBodies

A sequence of OdbRigidBody objects.

pretensionSections

A sequence of OdbPretensionSection objects.

connectorOrientations

A sequence of ConnectorOrientation objects.

61.14 OdbDatumCsys object

The OdbDatumCsys object contains a coordinate system that can be stored in an output database. You can create the datum coordinate system in the Visualization module during an Abaqus/CAE session and save the datum coordinate system to the output database before you exit Abaqus/CAE. Alternatively, the analysis code can write the datum coordinate system to the output database.

Access

```
odb.rootAssembly().datumCsyses() [name]
```

61.14.1 DatumCsysByThreePoints(...)

This method creates an OdbDatumCsys object using three points. A datum coordinate system created with this method results in a fixed system.

Path

```
odb.rootAssembly().DatumCsysByThreePoints
```

Prototype

```
odb_DatumCsys&
DatumCsysByThreePoints(const odb_String& name,
                        odb_Enum::odb_DatumCsysTypeEnum type,
                        const odb_SequenceFloat& origin,
                        const odb_SequenceFloat& point1,
                        const odb_SequenceFloat& point2);
```

Required arguments

name

An odb_String specifying the repository key.

type

An odb_Enum::odb_DatumCsysTypeEnum specifying the type of coordinate system. Possible values are odb_Enum::CARTESIAN, odb_Enum::CYLINDRICAL, and odb_Enum::SPHERICAL.

origin

An odb_SequenceFloat specifying the coordinates of the origin of the datum coordinate system.

point1

An odb_SequenceFloat specifying the coordinates of a point on the local 1- or r -axis.

point2

An odb_SequenceFloat specifying the coordinates of a point in the 1–2 or r – θ plane.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

61.14.2 DatumCsysByThreeNodes(...)

This method creates an OdbDatumCsys object using the coordinates of three OdbMeshNode objects. A datum coordinate system created with this method results in a system that follows the position of the three nodes. Results, such as those for displacement, are resolved into the orientation of the datum coordinate system without regard to the position of its origin. The last three arguments are given in the form of an OdbMeshNode object.

Path

`odb.rootAssembly().DatumCsysByThreeNodes`

Prototype

```
odb_DatumCsys&
DatumCsysByThreeNodes(const odb_String& name,
                      odb_Enum::odb_DatumCsysTypeEnum type,
                      const odb_Node& origin,
                      const odb_Node& point1,
                      const odb_Node& point2);
```

Required arguments

name

An odb_String specifying the repository key.

type

An odb_Enum::odb_DatumCsysTypeEnum specifying the type of coordinate system. Possible values are odb_Enum::CARTESIAN, odb_Enum::CYLINDRICAL, and odb_Enum::SPHERICAL.

origin

An OdbMeshNode object specifying a node at the origin of the datum coordinate system.

point1

An OdbMeshNode object specifying a node on the local 1- or r -axis.

point2

An OdbMeshNode object specifying a node in the 1–2 or r – θ plane.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

61.14.3 DatumCsys(...)

This method copies oneOdbDatumCsys object to a new OdbDatumCsys object.

Path

`odb.rootAssembly().DatumCsys`

Prototype

```
odb_DatumCsys&
DatumCsys(const odb_String& name,
          const odb_DatumCsys& datumCsys);
```

Required arguments

name

An odb_String specifying the repository key.

datumCsys

An OdbDatumCsys object specifying the object to be copied.

Optional arguments

None.

Return value

An OdbDatumCsys object.

Exceptions

None.

61.14.4 Members

The OdbDatumCsys object has the following members:

Prototype

```
odb_String name() const;  
odb_Enum::odb_DatumCsysTypeEnum type() const;  
const float* origin() const;  
float origin(int index) const;  
const float* xAxis() const;  
float xAxis(int index) const;  
const float* yAxis() const;  
float yAxis(int index) const;  
const float* zAxis() const;  
float zAxis(int index) const;
```

name

An odb_String specifying the repository key.

type

An odb_Enum::odb_DatumCsysTypeEnum specifying the type of coordinate system. Possible values are odb_Enum::CARTESIAN, odb_Enum::CYLINDRICAL, and odb_Enum::SPHERICAL.

origin

An odb_SequenceFloat specifying the coordinates of the origin of the datum coordinate system.

xAxis

An odb_SequenceFloat specifying a point on the X-axis.

yAxis

An odb_SequenceFloat specifying a point on the Y-axis.

zAxis

An odb_SequenceFloat specifying a point on the Z-axis.

61.15 OdbFrame object

The domain of the OdbFrame object is taken from the parent step.

Access

`odb.steps() [name].frames(i)`

61.15.1 Frame(...)

This method creates an OdbFrame object and appends it to the frame sequence.

Path

`odb.steps() [name].Frame`

Prototype

```
odb_Frame
    Frame(int incrementNumber,
          float frameValue,
          const odb_String& description);
```

Required arguments

incrementNumber

An Int specifying the frame increment number within the step. The base frame has normally increment number 0, and the results run from 1. In case of multiple load cases, the same increment number is duplicated for each loadcase.

frameValue

A Float specifying the value in units determined by the *domain* member of the Step object. The equivalent in the time domain is *stepTime*; in the frequency domain the equivalent is *frequency*; and in the modal domain the equivalent is *mode*.

Optional argument

description

An odb_String specifying the contents of the frame. The default value is an empty string.

Return value

An OdbFrame object.

Exceptions

None.

61.15.2 Frame(...)

This constructor creates an OdbFrame object in the frequency domain and appends it to the frame sequence. The arguments to the constructor are valid only when *domain*=FREQUENCY or *domain*=MODAL.

Path

`odb.steps()[name].Frame`

Prototype

```
odb_Frame
    Frame(int mode,
          float frequency,
          const odb_String& description);
```

Required arguments

mode

An Int specifying the eigenmode. This member is valid only if *domain*=odb_Enum::MODAL.

frequency

A Float specifying the frequency. This member is valid only if *domain*=odb_Enum::FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

Optional argument

description

An odb_String specifying the contents of the frame. The default value is an empty string.

Return value

An OdbFrame object.

Exceptions

None.

61.15.3 Frame(...)

This constructor creates an OdbFrame object for a specific load case and appends it to the frame sequence.

Path

`odb.steps() [name].Frame`

Prototype

```
odb_Frame
    Frame(const odb_LoadCase& loadCase,
          const odb_String& description,
          float frequency);
```

Required argument

loadCase

An OdbLoadCase object specifying the load case for the frame.

Optional arguments

description

An odb_String specifying the contents of the frame. The default value is an empty string.

frequency

A Float specifying the frequency. This member is valid only if *domain*=odb_Enum::FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

Return value

An OdbFrame object.

Exceptions

None.

61.15.4 Members

The OdbFrame object has members with the same names and descriptions as the arguments to the **Frame** method.

In addition, the OdbFrame object can have the following members:

Prototype

```
odb_Frame associatedFrame();
int cyclicModeNumber();
odb_String description() const;
odb_Enum::odb_DomainEnum domain() const;
odb_FieldOutputRepository& fieldOutputs();
int incrementNumber() const;
float frameValue() const;
float frequency() const;
const odb_LoadCase& loadCase() const;
int mode() const;
```

cyclicModeNumber

An Int specifying the cyclic mode number associated with the data stored on this frame. Only frequency analyses of cyclic symmetry models possess cyclic mode numbers.

domain

An odb_Enum::odb_DomainEnum specifying the domain of the step of which the frame is a member. Possible values are odb_Enum::TIME, odb_Enum::FREQUENCY, and odb_Enum::MODAL.

frequency

A Float specifying the frequency. This member is valid only if *domain*=odb_Enum::FREQUENCY or if the *procedureType* member of the Step object="FREQUENCY". The default value is 0.0.

mode

An Int specifying the eigenmode. This member is valid only if *domain*=odb_Enum::MODAL.

associatedFrame

An OdbFrame object specifying the real or imaginary portion of the data corresponding to this cyclic symmetry mode.

fieldOutputs

A repository of FieldOutput objects specifying the key to the *fieldOutputs* repository is a String representing an output variable.

loadCase

An OdbLoadCase object specifying the load case for the frame.

61.16 OdbInstance object

A part instance is the usage of a part within an assembly.

Access

```
odb.rootAssembly().instances() [name]
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance()
```

61.16.1 Instance(...)

This method creates an OdbInstance object from an OdbPart object.

Path

```
odb.rootAssembly().Instance
```

Prototype

```
odb_Instance&
Instance(const odb_String& name,
         const odb_Part& object,
         const odb_SequenceSequenceFloat& localCoordSystem);
```

Required arguments

name

An odb_String specifying the instance name.

object

An OdbPart object.

Optional argument

localCoordSystem

An odb_SequenceSequenceFloat specifying the rotation and translation of the part instance in the global Cartesian coordinate system. The first three sequences specify the new local coordinate system with its center at the origin.

- The first sequence specifies a point on the 1-axis.
- The second sequence specifies a point on the 2-axis.
- The third sequence specifies a point on the 3-axis.

The fourth sequence specifies the translation of the local coordinate system from the origin to its intended location.

For example, the following sequence moves a part 10 units in the X-direction with no rotation:

```
localCoordSystem = ((1, 0, 0), (0, 1, 0),
                    (0, 0, 1), (10, 0, 0))
```

The following sequence moves a part 5 units in the X-direction with rotation:

ODB COMMANDS

```
localCoordSystem = ((0, 1, 0), (1, 0, 0),  
                    (0, 0, 1), (5, 0, 0))
```

transforms a part containing the two points

```
Pt1= (1,0,0)  
Pt2= (2,0,0)
```

to

```
Pt1 = (0, 6, 0)  
Pt2 = (0, 7, 0)
```

Return value

An OdbInstance object.

Exceptions

InvalidNameError.

61.16.2 assignBeamOrientation(...)

This method assigns a beam section orientation to a region of a part instance.

Prototype

```
void  
assignBeamOrientation(const odb_Set& region,  
                    odb_Enum::odb_OrientationMethodEnum method,  
                    const odb_SequenceFloat& vector);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

method

An odb_Enum::odb_OrientationMethodEnum specifying the assignment method. Only a value of odb_Enum::N1_COSINES is currently supported.

vector

An odb_SequenceFloat specifying the approximate local $\mathbf{n}_1$ -direction of the beam cross-section.

Optional arguments

None.

Return value

None

Exceptions

None.

61.16.3 assignMaterialOrientation(...)

This method assigns a material orientation to a region of a part instance.

Prototype

```
void
assignMaterialOrientation(const odb_Set& region,
                          const odb_DatumCsys& localCsys,
                          odb_Enum::odb_AxisEnum axis,
                          float angle,
                          odb_Enum::odb_StackDirectionEnum stackDirection);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3. The default value is odb_Enum::AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

stackDirection

An odb_Enum::odb_StackDirectionEnum specifying the stack or thickness direction of the material. Possible values are odb_Enum::STACK_1, odb_Enum::STACK_2, odb_Enum::STACK_3, and odb_Enum::STACK_ORIENTATION. The default value is odb_Enum::STACK_3.

Return value

None

Exceptions

None.

61.16.4 assignRebarOrientation(...)

This method assigns a rebar reference orientation to a region of a part instance.

Prototype

```
void
assignRebarOrientation(const odb_Set& region,
                      const odb_DatumCsys& localCsys,
                      odb_Enum::odb_AxisEnum axis,
                      float angle);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3. The default value is odb_Enum::AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

61.16.5 getElementFromLabel(...)

This method is used to retrieved an element with a specific label from an instance object.

Prototype

```
odb_Element
getElementFromLabel(int label);
```

Arguments

Required argument

label

An Int specifying the element label.

Optional arguments

None.

Return value

An OdbMeshElement object.

Exceptions

If no element with the specified label exists:

OdbError: Invalid element label

61.16.6 getNodeFromLabel(...)

This method is used to retrieved a node with a specific label from an instance object.

Prototype

```
odb_Node
getNodeFromLabel(int label);
```

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

61.16.7 assignSection(...)

This method is used to assign a section to a region on an instance.

Prototype

```
void  
assignSection(const odb_Set& region,  
              const odb_section& section);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

section

A Section object.

Optional arguments

None.

Return value

None

Exceptions

If *region* is not an element set:

OdbError: Section assignment requires element set.

If the element set is not from the current instance:

OdbError: Section assignment requires element set from this part instance.

61.16.8 AnalyticRigidSurf2DPlanar(...)

This method is used to define a two-dimensional AnalyticSurface object on the instance.

Prototype

```
void
AnalyticRigidSurf2DPlanar(const odb_String& name,
                           const odb_SequenceAnalyticSurfaceSegment& profile,
                           double filletRadius);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If OdbPart associated with the part instance is of type THREE_D:

OdbError: 2D-Planar Analytic Rigid Surface can be defined only if the instance is of type TWO_D_PLANAR or AXISYMMETRIC.

61.16.9 AnalyticRigidSurfExtrude(...)

This method is used to define a three-dimensional cylindrical AnalyticSurface on the instance.

Prototype

```
void
AnalyticRigidSurfExtrude(const odb_String& name,
                        const odb_SequenceAnalyticSurfaceSegment& profile,
                        double filletRadius,
                        const odb_SequenceSequenceFloat& localCoordData);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional arguments

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

localCoordData

An odb_SequenceSequenceFloat specifying the global coordinates of points used to define the local coordinate system.

Return value

None

Exceptions

If OdbPart associated with the part instance is not of type THREE_D:

OdbError: Analytic Rigid Surface of type CYLINDER can be defined only if the instance is of type THREE_D.

61.16.10 AnalyticRigidSurfRevolve(...)

This method is used to define a three-dimensional AnalyticSurface of revolution on the instance.

Prototype

```
void
AnalyticRigidSurfRevolve(const odb_String& name,
                        const odb_SequenceAnalyticSurfaceSegment& profile,
```

```
double filletRadius,
const odb_SequenceSequenceFloat& localCoordData);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional arguments

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

localCoordData

An odb_SequenceSequenceFloat specifying the global coordinates of points used to define the local coordinate system.

Return value

None

Exceptions

If OdbPart associated with the part instance is not of type THREE_D:

OdbError: Analytic Rigid Surface of type REVOLUTION can be defined only if the instance is of type THREE_D.

61.16.11 RigidBody(...)

This method defines an OdbRigidBody on the instance.

Prototype

void

```
RigidBody(const odb_Set& referenceNode,
          odb_Enum::odb_PositionEnum position,
          bool isothermal,
          const odb_Set& elset,
          const odb_Set& pinNodes,
          const odb_Set& tieNodes);
```

Arguments

Required argument

referenceNode

An OdbSet specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are odb_Enum::INPUT, and odb_Enum::CENTER_OF_MASS. The default value is odb_Enum::INPUT.

isothermal

A Boolean specifying an isothermal rigid body. The default value is false. This parameter is used only for a fully-coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

61.16.12 getNodeFromLabel(...)

This method is used to retrieved a node with a specific label from an instance object.

Prototype

```
odb_Node  
getNodeFromLabel(int label);
```

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:
OdbError: Invalid node label

61.16.13 getNodeFromLabel(...)

This method is used to retrieve a node with a specific label from an instance object.

Prototype

```
odb_Node  
getNodeFromLabel(int label);
```

Arguments

Required argument

label
An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:
OdbError: Invalid node label

61.16.14 nodesLabelsFromNodeTypeFace(...)

Given a sequence of integers defining an element connectivity, this method is used to retrieve the sequence of nodes attached to a specified face of the element based on element type.

Prototype

```
odb_SequenceInt
nodesLabelsFromNodeTypeFace(const odb_SequenceInt& elementNodeLabels,
                             const odb_String& elementType,
                             odb_Enum::odb_ElementFaceEnumEnum elementFace);
```

Arguments

Required arguments

elementNodeLabels

A sequence of Ints specifying the node labels defining the connectivity for the entire element.

elementType

A string specifying the type of element whose connectivity is provided.

elementFace

A SymbolicConstant specifying the face of the element for which the node labels are desired.

Optional arguments

None.

Return value

A sequence of Ints specifying the node labels corresponding to the desired element face.

Exceptions

If the specified face is invalid for the element type:

OdbError: Invalid face

61.16.15 Members

The OdbInstance object can have the following members:

Prototype

```
odb_String name() const;
odb_Enum::odb_DimensionEnum embeddedSpace() const;
odb_Enum::odb_PartTypeEnum type() const;
odb_Node nodes(int i) const;
odb_SequenceNode& nodes() const;
odb_Element elements(int i) const;
odb_SequenceElement& elements() const;
odb_SetRepository& nodeSets() const;
odb_SetRepository& elementSets() const;
```

```

odb_SetRepository& surfaces() const;
odb_SequenceSectionAssignment sectionAssignments() \
    const;
odb_SequenceBeamOrientation beamOrientations() \
    const;
odb_SequenceMaterialOrientation materialOrientations() \
    const;
odb_SequenceRebarOrientation rebarOrientations() \
    const;
odb_SequenceRigidBody rigidBodies() const;
bool hasAnalyticSurface() const;
odb_AnalyticSurface analyticSurface() const;

```

name

An odb_String specifying the instance name.

type

An odb_Enum::odb_PartTypeEnum specifying the type of the OdbPart object. Only a value of odb_Enum::DEFORMABLE_BODY is currently supported.

embeddedSpace

An odb_Enum::odb_DimensionEnum specifying the dimensionality of the OdbPart object. Possible values are odb_Enum::THREE_D, odb_Enum::TWO_D_PLANAR, odb_Enum::AXISYMMETRIC, and odb_Enum::UNKNOWN_DIMENSION.

resultState

An odb_Enum::odb_ResultSetEnum specifying the state of the Instance as modified by the analysis. This member is only present if the Instance is part of the RootAssemblyState tree. Possible values are:

- odb_Enum::PROPAGATED, specifying that the value is the same as the previous frame or the original rootAssembly.
- odb_Enum::MODIFIED, specifying that the geometry of the instance has been changed at this frame.

The default value is odb_Enum::PROPAGATED.

nodes

A sequence of OdbMeshNode objects.

elements

A sequence of OdbMeshElement objects.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

sectionAssignments

A sequence of SectionAssignment objects.

rigidBodies

A sequence of OdbRigidBody objects.

beamOrientations

A sequence of BeamOrientation objects.

materialOrientations

A sequence of MaterialOrientation objects.

rebarOrientations

A sequence of RebarOrientation objects.

analyticSurface

An AnalyticSurface object specifying analytic Surface defined on the instance.

61.17 OdbLoadCase object

The OdbLoadCase object describes a load case.

Access

```
odb.steps() [name].frames(i).loadCase()  
odb.steps() [name].historyRegions() [name].loadCase()  
odb.steps() [name].loadCases() [name]
```

61.17.1 LoadCase(...)

This method creates an OdbLoadCase object.

Path

```
odb.steps() [name].LoadCase
```

Prototype

```
odb_LoadCase&  
LoadCase(const odb_String& name);
```


Required argument

name

An odb_String specifying the name of the OdbLoadCase object.

Optional arguments

None.

Return value

An OdbLoadCase object.

Exceptions

None.

61.17.2 Members

The OdbLoadCase object has members with the same names and descriptions as the arguments to the **LoadCase** method.

61.18 OdbMeshElement object

OdbMeshElement objects are created with the **part.addElements** or **rootAssembly.addElements** methods.

Access

```

odb.parts() [name].elements(i)
odb.parts() [name].elementSets() [name].elements(i)
odb.parts() [name].nodeSets() [name].elements(i)
odb.parts() [name].surfaces() [name].elements(i)
odb.rootAssembly().elements(i)
odb.rootAssembly().elementSets() [name].elements(i)
odb.rootAssembly().instances() [name].elements(i)
odb.rootAssembly().instances() [name].elementSets() [name].elements(i)
odb.rootAssembly().instances() [name].nodeSets() [name].elements(i)
odb.rootAssembly().instances() [name].surfaces() [name].elements(i)
odb.rootAssembly().nodeSets() [name].elements(i)
odb.rootAssembly().surfaces() [name].elements(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance()\
.elements(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance()\

```

```
.elementSets() [name].elements(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.nodeSets() [name].elements(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.surfaces() [name].elements(i)
```

61.18.1 Members

The OdbMeshElement object can have the following members:

Prototype

```
int label() const;
        const int* connectivity(int& numNodes) const ;
        odb_SequenceInt connectivity() const;
        odb_SectionCategory sectionCategory() const;
        odb_String type() const;
        odb_String instanceName() const;
        odb_SequenceString instanceNames() const;
```

label

An Int specifying the element label.

type

An odb_String specifying the element type.

sectionCategory

A SectionCategory object specifying the element section properties.

connectivity

A pointer to an array of Ints specifying the element connectivity. For connector elements connected to ground, the other node is repeated in the connectivity data. The position of the ground node cannot be ascertained. This is a limitation. It is important to note the difference with MeshElement object of MDB where the connectivity is node indices instead of node labels.

instanceNames

A pointer to an array of Strings specifying the instance names for nodes in the element connectivity.

instanceName

An odb_String specifying the instance name.

61.19 OdbMeshNode object

OdbMeshNode objects are created with the **part.addNodes** method.

Access

```
odb.parts() [name].elementSets() [name].nodes(i)
odb.parts() [name].nodes(i)
odb.parts() [name].nodeSets() [name].nodes(i)
odb.parts() [name].surfaces() [name].nodes(i)
odb.rootAssembly().elementSets() [name].nodes(i)
odb.rootAssembly().instances() [name].elementSets() [name].nodes(i)
odb.rootAssembly().instances() [name].nodes(i)
odb.rootAssembly().instances() [name].nodeSets() [name].nodes(i)
odb.rootAssembly().instances() [name].surfaces() [name].nodes(i)
odb.rootAssembly().nodes(i)
odb.rootAssembly().nodeSets() [name].nodes(i)
odb.rootAssembly().surfaces() [name].nodes(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.elementSets() [name].nodes(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.nodes(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.nodeSets() [name].nodes(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.surfaces() [name].nodes(i)
```

61.19.1 Members

The OdbMeshNode object has the following members:

Prototype

```
int label() const;
const float* coordinates() const;
```

label

An Int specifying the node label.

coordinates

A pointer to an array of Floats specifying the nodal coordinates in the global Cartesian coordinate system.

61.20 OdbPart object

The OdbPart object is similar to the kernel Part object and contains nodes and elements, but not geometry.

Access

`odb.parts()` [*name*]

61.20.1 Part(...)

This method creates an OdbPart object. Nodes and elements are added to this object at a later stage.

Path

`odb.Part`

Prototype

```
odb_Part&
Part(const odb_String& name,
      odb_Enum::odb_DimensionEnum embeddedSpace,
      odb_Enum::odb_PartTypeEnum type);
```

Required arguments

name

An odb_String specifying the part name.

embeddedSpace

An odb_Enum::odb_DimensionEnum specifying the dimensionality of the OdbPart object. Possible values are odb_Enum::THREE_D, odb_Enum::TWO_D_PLANAR, and odb_Enum::AXISYMMETRIC.

type

An odb_Enum::odb_PartTypeEnum specifying the type of the OdbPart object. Possible values are odb_Enum::DEFORMABLE_BODY and odb_Enum::ANALYTIC_RIGID_SURFACE.

Optional arguments

None.

Return value

An OdbPart object.

Exceptions

None.

61.20.2 addElements(...)

This method adds elements to an OdbPart object using element labels and nodal connectivity.

WARNING: Adding elements not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.

Prototype

```
void
addElements(const odb_SequenceInt& labels,
            const odb_SequenceSequenceInt& connectivity,
            const odb_String& type,
            const odb_String& elementSetName,
            const odb_SectionCategory& sectionCategory);
```

Arguments

Required arguments

labels

An odb_SequenceInt specifying the element labels.

connectivity

An odb_SequenceSequenceInt specifying the nodal connectivity.

type

A String specifying the element type.

Optional arguments

elementSetName

A String specifying a name for this element set. The default value is the empty string.

sectionCategory

A SectionCategory object for this element set.

Return value

None

Exceptions

None.

61.20.3 addNodes(...)

This method adds nodes to an OdbPart object using node labels and coordinates.

WARNING: *Adding nodes not in ascending order of their labels may cause Abaqus/Viewer to plot contours incorrectly.*

Prototype

```
void
addNodes(const odb_SequenceInt& labels,
         const odb_SequenceSequenceFloat& coordinates,
         const odb_String& nodeSetName);
```

Arguments

Required arguments

labels

An odb_SequenceInt specifying the node labels.

coordinates

An odb_SequenceSequenceFloat specifying the nodal coordinates.

Optional argument

nodeSetName

A String specifying a name for this node set. The default value is None.

Return value

None

Exceptions

None.

61.20.4 assignBeamOrientation(...)

This method assigns a beam section orientation to a region of a part instance.

Prototype

```
void
assignBeamOrientation(const odb_Set& region,
                    odb_Enum::odb_OrientationMethodEnum method,
```

```
const odb_SequenceFloat& vector);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

method

An odb_Enum::odb_OrientationMethodEnum specifying the assignment method. Only a value of odb_Enum::N1_COSINES is currently supported.

vector

An odb_SequenceFloat specifying the approximate local $\mathbf{n}_1$ -direction of the beam cross-section.

Optional arguments

None.

Return value

None

Exceptions

None.

61.20.5 assignMaterialOrientation(...)

This method assigns a material orientation to a region of a part instance.

Prototype

```
void
assignMaterialOrientation(const odb_Set& region,
                          const odb_DatumCsys& localCsys,
                          odb_Enum::odb_AxisEnum axis,
                          float angle,
                          odb_Enum::odb_StackDirectionEnum stackDirection);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3. The default value is odb_Enum::AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

stackDirection

An odb_Enum::odb_StackDirectionEnum specifying the stack or thickness direction of the material. Possible values are odb_Enum::STACK_1, odb_Enum::STACK_2, odb_Enum::STACK_3, and odb_Enum::STACK_ORIENTATION. The default value is odb_Enum::STACK_3.

Return value

None

Exceptions

None.

61.20.6 assignRebarOrientation(...)

This method assigns a rebar reference orientation to a region of a part instance.

Prototype

```
void
assignRebarOrientation(const odb_Set& region,
                      const odb_DatumCsys& localCsys,
                      odb_Enum::odb_AxisEnum axis,
                      float angle);
```

Arguments

Required arguments

region

An OdbSet specifying a region on an instance.

localCsys

An OdbDatumCsys object specifying the local coordinate system or **None**, indicating the global coordinate system.

Optional arguments

axis

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3. The default value is odb_Enum::AXIS_1.

angle

A Float specifying the angle of the additional rotation. The default value is 0.0.

Return value

None

Exceptions

None.

61.20.7 getElementFromLabel(...)

This method is used to retrieve an element with a specific label from a part object.

Prototype

```
odb_Element
getElementFromLabel(int label);
```

Arguments

Required argument

label

An Int specifying the element label.

Optional arguments

None.

Return value

An OdbMeshElement object.

Exceptions

If no element with the specified label exists:

OdbError: Invalid element label

61.20.8 getNodeFromLabel(...)

This method is used to retrieve a node with a specific label from a part object.

Prototype

```
odb_Node  
getNodeFromLabel(int label);
```

Arguments

Required argument

label

An Int specifying the node label.

Optional arguments

None.

Return value

An OdbMeshNode object.

Exceptions

If no node with the specified label exists:

OdbError: Invalid node label

61.20.9 AnalyticRigidSurf2DPlanar(...)

This method is used to define a two-dimensional AnalyticSurface object on the part object.

Prototype

```
void  
AnalyticRigidSurf2DPlanar(const odb_String& name,  
                           const odb_SequenceAnalyticSurfaceSegment& profile,  
                           double filletRadius);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of `AnalyticSurfaceSegment` objects or an `OdbSequenceAnalyticSurfaceSegment` object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If `OdbPart` is of type `THREE_D`:

`OdbError`: 2D-Planar Analytic Rigid Surface can be defined only if the part is of type `TWO_D_PLANAR` or `AXISYMMETRIC`.

61.20.10 AnalyticRigidSurfExtrude(...)

This method is used to define a three-dimensional cylindrical `AnalyticSurface` on the part object.

Prototype

```
void
AnalyticRigidSurfExtrude(const odb_String& name,
                        const odb_SequenceAnalyticSurfaceSegment& profile,
                        double filletRadius);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of `AnalyticSurfaceSegment` objects or an `OdbSequenceAnalyticSurfaceSegment` object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If OdbPart is not of type THREE_D:

OdbError: Analytic Rigid Surface of type CYLINDER can be defined only if the part is of type THREE_D.

61.20.11 AnalyticRigidSurfRevolve(...)

This method is used to define a three-dimensional AnalyticSurface of revolution on the part object.

Prototype

```
void
AnalyticRigidSurfRevolve(const odb_String& name,
                        const odb_SequenceAnalyticSurfaceSegment& profile,
                        double filletRadius);
```

Arguments

Required arguments

name

The name of the analytic surface.

profile

A sequence of AnalyticSurfaceSegment objects or an OdbSequenceAnalyticSurfaceSegment object.

Optional argument

filletRadius

A Double specifying the radius of curvature to smooth discontinuities between adjoining segments. The default value is 0.0.

Return value

None

Exceptions

If OdbPart is not of type THREE_D:

OdbError: Analytic Rigid Surface of type REVOLUTION can be defined only if the part is of type THREE_D.

61.20.12 RigidBody(...)

This method defines an OdbRigidBody on the part object.

Prototype

```
void
RigidBody(const odb_Set& referenceNode,
          odb_Enum::odb_PositionEnum position,
          bool isothermal,
          const odb_Set& elset,
          const odb_Set& pinNodes,
          const odb_Set& tieNodes);
```

Arguments

Required argument

referenceNode

An OdbSet specifying the reference node assigned to the rigid body.

Optional arguments

position

A symbolic constant specify if the location of the reference node is to be defined by the user. Possible values are odb_Enum::INPUT and odb_Enum::CENTER_OF_MASS. The default value is odb_Enum::INPUT.

isothermal

A Boolean specifying an isothermal rigid body. The default value is false. This parameter is used only for a fully-coupled thermal stress analysis.

elset

An OdbSet specifying an element set assigned to the rigid body.

pinNodes

An OdbSet specifying pin-type nodes assigned to the rigid body.

tieNodes

An OdbSet specifying tie-type nodes assigned to the rigid body.

Return value

None

Exceptions

If *referenceNode* is not a node set:

OdbError: Rigid body definition requires a node set.

61.20.13 Members

The OdbPart object has members with the same names and descriptions as the arguments to the **Part** method.

In addition, the OdbPart object can have the following members:

Prototype

```
odb_String name() const;
    odb_Enum::odb_DimensionEnum embeddedSpace() const;
    odb_Enum::odb_PartTypeEnum type() const;
    odb_Node& nodes(int index) const;
    odb_SequenceNode& nodes() const;
    odb_Element& elements(int index) const;
    odb_SequenceElement& elements() const;
    odb_SetRepository& nodeSets() const;
    odb_SetRepository& elementSets() const;
    odb_SetRepository& surfaces() const;
    odb_SequenceSectionAssignment sectionAssignments() const;
    odb_SequenceBeamOrientation beamOrientations() const;
    odb_SequenceMaterialOrientation materialOrientations() const;
    odb_SequenceRebarOrientation rebarOrientations() const;
    odb_SequenceRigidBody rigidBodies() const;
    bool hasAnalyticSurface() const;
    odb_AnalyticSurface analyticSurface() const;
```

nodes

A sequence of OdbMeshNode objects.

elements

A sequence of OdbMeshElement objects.

nodeSets

A repository of OdbSet objects specifying node sets.

elementSets

A repository of OdbSet objects specifying element sets.

surfaces

A repository of OdbSet objects specifying surfaces.

sectionAssignments

A sequence of SectionAssignment objects.

beamOrientations

A sequence of BeamOrientation objects.

materialOrientations

A sequence of MaterialOrientation objects.

rebarOrientations

A sequence of RebarOrientation objects.

rigidBodies

A sequence of OdbRigidBody objects.

analyticSurface

An AnalyticSurface object specifying analytic Surface defined on the instance.

61.21 OdbPretensionSection object

The pretension section object is used to define an assembly load. It associates a pretension node with a pretension section.

Access

```
odb.rootAssembly().pretensionSections(i)
```

61.21.1 Members

The OdbPretensionSection object can have the following members:

Prototype

```
odb_Set node() const;
odb_Set element() const;
odb_Set surface() const;
odb_SequenceFloat normal() const;
float normal(int index) const;
```

node

An OdbSet object specifying the node set containing the pretension node.

element

An OdbSet object specifying the element set that defines the pretension section.

surface

An OdbSet object specifying the surface set that defines the pretension section.

normal

An odb_SequenceFloat specifying the components of the normal to the pretension section.

61.22 OdbRigidBody object

The Rigid body object is used to bind a set of elements and/or a set of nodes and/or an analytical surface with a reference node.

Access

```
odb.parts() [name].rigidBodies(i)
odb.rootAssembly().instances() [name].rigidBodies(i)
odb.rootAssembly().rigidBodies(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.rigidBodies(i)
```

61.22.1 Members

The OdbRigidBody object can have the following members:

Prototype

```
odb_Enum::odb_PositionEnum position() const;
bool isothermal() const;
odb_Set referenceNode() const;
odb_Set elements() const;
odb_Set tieNodes() const;
odb_Set pinNodes() const;
odb_AnalyticSurface analyticSurface() const;
```

position

An odb_Enum::odb_PositionEnum specifying the specific location of the OdbRigidBody reference node relative to the rest of the rigid body. Possible values are odb_Enum::INPUT and odb_Enum::CENTER_OF_MASS. The default value is odb_Enum::INPUT.

isothermal

A Boolean specifying whether the OdbRigidBody can have temperature gradients or be isothermal. This is used only for fully coupled thermal-stress analysis. The default value is true.

referenceNode

An OdbSet object specifying the reference node set associated with the rigid body.

elements

An OdbSet object specifying the element set whose motion is governed by the motion of rigid body reference node.

tieNodes

An OdbSet object specifying the node set which have both translational and rotational degrees of freedom associated with the rigid body.

pinNodes

An OdbSet object specifying the node set which have only translational degrees of freedom associated with the rigid body.

analyticSurface

An AnalyticSurface object specifying the analytic surface whose motion is governed by the motion of rigid body reference node.

61.23 OdbSequenceAnalyticSurfaceSegment object

A sequence of AnalyticSurfaceSegment describing an analytic surface profile.

Access

```
odb.parts()[name].analyticSurface().segments()
odb.rootAssembly().instances()[name].analyticSurface().segments()
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.analyticSurface().segments()
```

61.23.1 Start(...)

This method adds a AnalyticSurfaceSegment describing the first segment of the surface profile.

Prototype

```
void
Start(const odb_SequenceFloat& origin);
```

Arguments

Required argument

origin

An odb_SequenceFloat specifying the coordinates of start point.

Optional arguments

None.

Return value

None

Exceptions

None.

61.23.2 Line(...)

This method adds a `AnalyticSurfaceSegment` describing the line segment of the surface profile.

Prototype

```
void  
Line(const odb_SequenceFloat& endPoint);
```

Arguments

Required argument

endPoint

An `odb_SequenceFloat` specifying the coordinates of end point.

Optional arguments

None.

Return value

None

Exceptions

None.

61.23.3 Circle(...)

This method adds a `AnalyticSurfaceSegment` describing a circular segment of the surface profile.

Prototype

```
void  
Circle(const odb_SequenceFloat& center,  
        const odb_SequenceFloat& endPoint);
```

Arguments

Required arguments

center

An `odb_SequenceFloat` specifying the coordinates of center of the circular segment.

endPoint

An odb_SequenceFloat specifying the coordinates of end point of the circular segment.

Optional arguments

None.

Return value

None

Exceptions

None.

61.23.4 Parabola(...)

This method adds a AnalyticSurfaceSegment describing a parabolic segment of the surface profile.

Prototype

```
void
Parabola(const odb_SequenceFloat& middlePoint,
         const odb_SequenceFloat& endPoint);
```

Arguments

Required arguments

middlePoint

An odb_SequenceFloat specifying the coordinates of middle point of the parabolic segment.

endPoint

An odb_SequenceFloat specifying the coordinates of end point of the parabolic segment.

Optional arguments

None.

Return value

None

Exceptions

None.

61.23.5 Members

The OdbSequenceAnalyticSurfaceSegment object has no members.

61.24 OdbSet object

The set objects are used to identify regions of a model.

Access

```
odb.parts() [name].elementSets() [name]
odb.parts() [name].nodeSets() [name]
odb.parts() [name].surfaces() [name]
odb.rootAssembly().elementSets() [name]
odb.rootAssembly().instances() [name].elementSets() [name]
odb.rootAssembly().instances() [name].nodeSets() [name]
odb.rootAssembly().instances() [name].surfaces() [name]
odb.rootAssembly().nodeSets() [name]
odb.rootAssembly().surfaces() [name]
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.elementSets() [name]
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.nodeSets() [name]
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.surfaces() [name]
```

61.24.1 NodeSet(...)

This method creates a node set from an array of OdbMeshNode objects (for part instance-level sets)

Path

```
odb.parts() [name].NodeSet
odb.rootAssembly().instances() [name].NodeSet
```

Prototype

```
odb_Set&
NodeSet(const odb_String& name,
        const odb_SequenceNode& nodes);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

nodes

A sequence of OdbMeshNode objects. For example, for a part:

```
nodes=part1.nodes[1:5]
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

61.24.2 ElementSet(...)

This method creates an element set from an array of OdbMeshElement objects (for part instance-level sets).

Path

```
odb.parts()[name].ElementSet  
odb.rootAssembly().instances()[name].ElementSet
```

Prototype

```
odb_Set&  
ElementSet(const odb_String& name,  
            const odb_SequenceElement& elements);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

elements

A sequence of OdbMeshElement objects. For example, for a part:

```
elements=instance1.elements[1:5]
```

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

InvalidNameError.

61.24.3 NodeSet(...)

This method creates a node set.

Path

```
odb.parts() [name].NodeSet  
odb.rootAssembly().instances() [name].NodeSet
```

Prototype

```
odb_Set&  
NodeSet(const odb_String& name,  
         const odb_SequenceInt& nodeLabels);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

nodeLabels

An odb_SequenceInt specifying the node labels.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.4 NodeSet(...)

This method creates a node set.

Path

`odb.rootAssembly().NodeSet`

Prototype

```
odb_Set&
NodeSet(const odb_String& name,
        const odb_SequenceString& instanceNames,
        const odb_SequenceSequenceInt& nodeLabels);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

instanceNames

An odb_SequenceString specifying the namespaces for *nodeLabels*. The string could be an empty string to denote the nodeLabels at the assembly level.

nodeLabels

An odb_SequenceSequenceInt specifying the node labels.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.5 ElementSet(...)

This method creates an element set.

Path

```
odb.parts()[name].ElementSet
odb.rootAssembly().instances()[name].ElementSet
```

Prototype

```
odb_Set&
ElementSet(const odb_String& name,
```

```
const odb_SequenceInt& elementLabels);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

elementLabels

An odb_SequenceInt specifying the element labels.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.6 ElementSet(...)

This method creates an element set.

Path

```
odb.rootAssembly().ElementSet
```

Prototype

```
odb_Set&
ElementSet(const odb_String& name,
            const odb_SequenceString& instanceNames,
            const odb_SequenceSequenceInt& elementLabels);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

instanceNames

An odb_SequenceString specifying the namespaces for *elementLabels*. The string could be an empty string to denote the elementLabels at the assembly level.

elementLabels

An odb_SequenceSequenceInt specifying the element labels.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.7 Surface(...)

This method creates a surface set.

Path

```
odb.parts() [name].Surface
odb.rootAssembly().instances() [name].Surface
```

Prototype

```
odb_Set&
Surface(const odb_String& name,
        const odb_SequenceElement& elements,
        const odb_SequenceElementFace& faces);
```

Required arguments

name
An odb_String specifying the name of the set and the repository key.

elements
An odb_SequenceElement specifying the elements in the surface.

faces
An odb_SequenceElementFace specifying the element faces.
The possible face enumerations depend on the type of element, as described in the following table:

Element type	Face enumerations
Solid elements	FACE1, FACE2, FACE3, FACE4, FACE5, FACE6
Three-dimensional shell elements	SIDE1, SIDE2

Two-dimensional elements	FACE1, FACE2, FACE3, FACE4
Wire elements	END, END2

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.8 Surface(...)

This method creates a surface set.

Path

```
odb.parts() [name].Surface
odb.rootAssembly().instances() [name].Surface
```

Prototype

```
odb_Set&
Surface(const odb_String& name,
        const odb_SequenceInt& elementLabels,
        const odb_SequenceElementFace& faces);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

elementLabels

An odb_SequenceInt specifying the element labels.

faces

An odb_SequenceElementFace specifying the element faces. An odb_SequenceSequenceElementFace is required for an assembly set.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.9 Surface(...)

This method creates a surface set.

Path

`odb.rootAssembly().Surface`

Prototype

```
odb_Set&
Surface(const odb_String& name,
        const odb_SequenceString& instanceNames,
        const odb_SequenceSequenceInt& elementLabels,
        const odb_SequenceSequenceElementFace& faces);
```

Required arguments

name

An odb_String specifying the name of the set and the repository key.

instanceNames

An odb_SequenceString specifying the namespaces for *elementLabels*. This member is only valid for assembly sets. The string could be an empty string to denote the elementLabels at the assembly level.

elementLabels

An odb_SequenceSequenceInt specifying the element labels.

faces

An odb_SequenceSequenceElementFace specifying the element faces.

Optional arguments

None.

Return value

An OdbSet object.

Exceptions

None.

61.24.10 Members

The OdbSet object can have the following members:

Prototype

```
odb_String name() const;
        odb_SequenceString instanceNames();
        odb_Node nodes(int index);
        const odb_SequenceNode& nodes();
        const odb_SequenceNode& nodes(const odb_String& \
            instanceName); // For an assembly level set
        odb_Element elements(int index);
        const odb_SequenceElement& elements();
        const odb_SequenceElement& elements(const odb_String& \
            instanceName); // For an assembly level set
        odb_Enum::odb_ElementFaceEnum faces(int index);
        const odb_SequenceElementFace& faces() ;
        const odb_SequenceElementFace& faces(const odb_String& \
            instanceName); // For an assembly level set
        odb_Enum::odb_SetTypeEnum type() const;
```

name

An odb_String specifying the name of the set and the repository key.

instanceNames

An odb_SequenceString specifying the namespaces for the nodes, elements, and faces constituting the set.

nodes

A sequence of OdbMeshNode objects.

elements

A sequence of OdbMeshElement objects.

faces

An odb_SequenceElementFace specifying the element face.

61.25 OdbStep object

An output database contains the same steps of the model database that originated it.

Access

`odb.steps()` [*name*]

61.25.1 Step(...)

This method creates an OdbStep object.

Path

`odb.Step`

Prototype

```
odb_Step&
Step(const odb_String& name,
      const odb_String& description,
      odb_Enum::odb_DomainEnum domain,
      double timePeriod,
      const odb_String& previousStepName,
      const odb_String& procedure,
      double totalTime);
```

Required arguments

name

An odb_String specifying the repository key.

description

An odb_String specifying the step description.

domain

An odb_Enum::odb_DomainEnum specifying the domain of the step. Possible values are odb_Enum::TIME, odb_Enum::FREQUENCY, odb_Enum::ARC_LENGTH, and odb_Enum::MODAL.

The type of OdbFrame object that can be created for this step is based on the value of the *domain* argument.

Optional arguments

timePeriod

A Double specifying the time period of the step. *timePeriod* is required if *domain*=odb_Enum::TIME; otherwise, this argument is not applicable. The default value is 0.0.

previousStepName

An odb_String specifying the preceding step. If *previousStepName* is the empty string, the last step in the repository is used. If *previousStepName* is not the last step, this will result in a change

to the *previousStepName* member of the step that was in that position. A special value 'Initial' refers to the internal initial model step and may be used exclusively for inserting a new step at the first position before any other existing steps. The default value is an empty string.

procedure

An odb_String specifying the step procedure. The default value is an empty string.

totalTime

A Double specifying the analysis time spend in all the steps previous to this step. The default value is -1.0.

Return value

An OdbStep object.

Exceptions

If *previousStepName* is invalid:

ValueError: previousStepName is invalid

61.25.2 getFrame(...)

This method retrieves an OdbFrame object associated with a given frame value.

Prototype

```
odb_Frame
getFrame(double frameValue,
         odb_Enum::odb_MatchEnum match);
```

Arguments

Required argument

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the step time or frequency.

Optional argument

match

An odb_Enum::odb_MatchEnum specifying which frame to return if there is no frame at the exact frame value. Possible values are odb_Enum::CLOSEST, odb_Enum::BEFORE, odb_Enum::AFTER, and odb_Enum::EXACT. The default value is odb_Enum::CLOSEST.

When *match*=odb_Enum::CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=odb_Enum::EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:
OdbError: Frame not found.

61.25.3 **getFrame(...)**

This method retrieves an OdbFrame object associated with a given load case.

Prototype

```
odb_Frame  
getFrame(const odb_LoadCase& loadCase);
```

Arguments

Required argument

loadCase

An OdbLoadCase object specifying a load case in the step.

Optional arguments

None.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:
OdbError: Frame not found.

61.25.4 **getFrame(...)**

This method retrieves an OdbFrame object associated with a given load case and frame value.

Prototype

```
odb_Frame  
getFrame(const odb_LoadCase& loadCase,  
         double frameValue,
```

```
odb_Enum::odb_MatchEnum match);
```

Arguments

Required arguments

loadCase

An OdbLoadCase object specifying a load case in the step.

frameValue

A Double specifying the value at which the frame is required. *frameValue* can be the step time or frequency.

Optional argument

match

An odb_Enum::odb_MatchEnum specifying which frame to return if there is no frame at the exact frame value. Possible values are odb_Enum::CLOSEST, odb_Enum::BEFORE, odb_Enum::AFTER, and odb_Enum::EXACT. The default value is odb_Enum::CLOSEST.

When *match*=odb_Enum::CLOSEST, Abaqus returns the closest frame. If the frame value requested is exactly halfway between two frames, Abaqus returns the frame after the value.

When *match*=odb_Enum::EXACT, Abaqus raises an exception if the exact frame value does not exist.

Return value

An OdbFrame object.

Exceptions

If the OdbFrame object is not found:

OdbError: Frame not found.

61.25.5 getHistoryRegion(...)

This method retrieves a HistoryRegion object associated with a HistoryPoint in the model.

Prototype

```
odb_HistoryRegion  
getHistoryRegion(const odb_HistoryPoint& point,  
                 const odb_LoadCase& loadCase);
```

Arguments

Required argument

point

A HistoryPoint object specifying the point in the model.

Optional argument

loadCase

An OdbLoadCase object specifying a load case in the step.

Return value

A HistoryRegion object.

Exceptions

If a HistoryRegion object is not found:

OdbError: HistoryRegion not found.

61.25.6 **setDefaultDeformedField(...)**

This method sets the default deformed field variable in a step.

Prototype

```
void
setDefaultDeformedField(const odb_FieldOutput& field);
```

Arguments

Required argument

field

A FieldOutput object specifying the default deformed field variable for visualization.

Optional arguments

None.

Return value

None

Exceptions

None.

61.25.7 **setDefaultField(...)**

This method sets the default field variable in a step.

Prototype

```
void
```

```
setDefaultField(const odb_FieldOutput& field);
```

Arguments

Required argument

field

A FieldOutput object specifying the default field variable for visualization.

Optional arguments

None.

Return value

None

Exceptions

None.

61.25.8 Members

The OdbStep object has members with the same names and descriptions as the arguments to the **Step** method.

In addition, the OdbStep object can have the following members:

Prototype

```
odb_String name() const;
odb_String description() const;
odb_Enum::odb_DomainEnum domain() const;
const odb_LoadCaseRepository& loadCases();
float timePeriod() const;
float totalTime() const;
odb_String previousStepName() const;
odb_String procedure() const;
int number() const;
bool nlgeom() const;
odb_Frame frames(int frameNo);
odb_SequenceFrame& frames();
odb_HistoryRegionRepository& historyRegions();
double mass() const;
odb_SequenceDouble massCenter() const;
odb_SequenceDouble inertiaAboutCenter() const;
odb_SequenceDouble inertiaAboutOrigin() const;
double acousticMass() const;
```

```
odb_SequenceDouble acousticMassCenter() const;
odb_SequenceNodalDofs eliminatedNodalDofs() const;
```

number

An Int specifying the step number.

nlgeom

A Boolean specifying whether geometric nonlinearity can occur in this step.

mass

A Double specifying the current value of the mass of the model. This does not include the mass of the acoustic media if any present.

acousticMass

A Double specifying the current value of the mass of the acoustic media of the model.

frames

A sequence of OdbFrame objects.

historyRegions

A repository of HistoryRegion objects.

loadCases

A repository of OdbLoadCase objects.

massCenter

An odb_SequenceDouble specifying the coordinates of the center of mass.

inertiaAboutCenter

An odb_SequenceDouble specifying the moments and products of inertia about the center of mass. For 3-D models inertia quantities are written in the following order: I(XX), I(YY), I(ZZ), I(XY), I(XZ), and I(YZ). For 2-D models only I(ZZ) and I(XY) are outputted.

inertiaAboutOrigin

An odb_SequenceDouble specifying the moments and products of inertia about the origin of the global coordinate system. For 3-D models inertia quantities are written in the following order: I(XX), I(YY), I(ZZ), I(XY), I(XZ), and I(YZ). For 2-D models only I(ZZ) and I(XY) are outputted.

acousticMassCenter

An odb_SequenceDouble specifying the coordinates of the center of mass of the acoustic media.

61.26 RebarOrientation object

The RebarOrientation object represents the orientation of the rebar reference.

Access

```
odb.parts()[name].rebarOrientations(i)
odb.rootAssembly().instances()[name].rebarOrientations(i)
```

```
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.rebarOrientations(i)
```

61.26.1 Members

The RebarOrientation object can have the following members:

Prototype

```
odb_Set region() const;
odb_DatumCsys csys() const;
odb_Enum::odb_AxisEnum axis() const;
float angle() const;
```

axis

An odb_Enum::odb_AxisEnum specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. Possible values are odb_Enum::AXIS_1, odb_Enum::AXIS_2, and odb_Enum::AXIS_3.

angle

A Float specifying the angle of the additional rotation.

region

An OdbSet object specifying a region for which the rebar orientation is defined.

csys

An OdbDatumCsys object specifying a datum coordinates system.

61.27 SectionCategory object

The SectionCategory object is used to group regions of the model with like sections. Section definitions that contain the same number of section points or integration points are grouped together.

To access data for a particular section definition, use the individual Section objects in the output database. For more information, see Chapter 8, “Beam Section profile commands,” and Chapter 46, “Section commands.”

Access

```
odb.parts() [name].elements(i).sectionCategory()
odb.parts() [name].elementSets() [name].elements(i).sectionCategory()
odb.parts() [name].nodeSets() [name].elements(i).sectionCategory()
odb.parts() [name].surfaces() [name].elements(i).sectionCategory()
odb.rootAssembly().elements(i).sectionCategory()
odb.rootAssembly().elementSets() [name].elements(i).sectionCategory()
```

```

odb.rootAssembly().instances()[name].elements(i).sectionCategory()
odb.rootAssembly().instances()[name].elementSets()[name].elements(i)\
.sectionCategory()
odb.rootAssembly().instances()[name].nodeSets()[name].elements(i)\
.sectionCategory()
odb.rootAssembly().instances()[name].surfaces()[name].elements(i)\
.sectionCategory()
odb.rootAssembly().nodeSets()[name].elements(i).sectionCategory()
odb.rootAssembly().surfaces()[name].elements(i).sectionCategory()
odb.sectionCategories()[name]
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.elements(i).sectionCategory()
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.elementSets()[name].elements(i).sectionCategory()
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.nodeSets()[name].elements(i).sectionCategory()
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.surfaces()[name].elements(i).sectionCategory()

```

61.27.1 SectionCategory(...)

This method creates a SectionCategory object.

Path

```
odb.SectionCategory
```

Prototype

```

odb.SectionCategory&
SectionCategory(const odb_String& name,
               const odb_String& description);

```

Required arguments

name

An odb_String specifying the name of the category.

description

An odb_String specifying the description of the category.

Optional arguments

None.

Return value

A `SectionCategory` object.

Exceptions

None.

61.27.2 Members

The `SectionCategory` object has members with the same names and descriptions as the arguments to the `SectionCategory` method.

In addition, the `SectionCategory` object can have the following member:

Prototype

```
odb_String name() const;
odb_String description() const;
const odb_SectionPoint& sectionPoints(int index) const;
const odb_SequenceSectionPoint& sectionPoints() const;
```

sectionPoints

A sequence of `SectionPoint` objects.

61.28 SectionPoint object

The `SectionPoint` object describes the location of a section point within a section category.

Access

```
odb.parts()[name].elements(i).sectionCategory().sectionPoints(i)
odb.parts()[name].elementSets()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.parts()[name].nodeSets()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.parts()[name].surfaces()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.rootAssembly().elements(i).sectionCategory().sectionPoints(i)
odb.rootAssembly().elementSets()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.rootAssembly().instances()[name].elements(i).sectionCategory()\
.sectionPoints(i)
```

```

odb.rootAssembly().instances()[name].elementSets()[name].elements(i)\
.sectionCategory().sectionPoints(i)
odb.rootAssembly().instances()[name].nodeSets()[name].elements(i)\
.sectionCategory().sectionPoints(i)
odb.rootAssembly().instances()[name].surfaces()[name].elements(i)\
.sectionCategory().sectionPoints(i)
odb.rootAssembly().nodeSets()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.rootAssembly().surfaces()[name].elements(i).sectionCategory()\
.sectionPoints(i)
odb.sectionCategories()[name].sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].locations(i)\
.sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.elements(i).sectionCategory().sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.elementSets()[name].elements(i).sectionCategory().sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.nodeSets()[name].elements(i).sectionCategory().sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).instance()\
.surfaces()[name].elements(i).sectionCategory().sectionPoints(i)
odb.steps()[name].frames(i).fieldOutputs()[name].values(i).sectionPoint()

```

61.28.1 SectionPoint(...)

This method creates a SectionPoint object.

Path

```
odb.sectionCategories()[name].SectionPoint
```

Prototype

```

odb_SectionPoint&
SectionPoint(int number,
             const odb_String& description);

```

Required arguments

number

An Int specifying the number of the section point. See “Beam elements,” Section 29.3 of the Abaqus Analysis User’s Guide, and “Shell elements,” Section 29.6 of the Abaqus Analysis User’s Guide, for the numbering convention.

description

An odb_String specifying the description of the section point.

Optional arguments

None.

Return value

A SectionPoint object.

Exceptions

None.

61.28.2 Members

The SectionPoint object has members with the same names and descriptions as the arguments to the **SectionPoint** method.

Prototype

```
int number() const;  
const odb_String description() const;
```

61.29 SectorDefinition object

The SectorDefinition object describes the number of symmetry sectors and axis of symmetry for a cyclic symmetry model.

Access

```
odb.sectorDefinition()
```

61.29.1 Members

The SectorDefinition object has the following members:

Prototype

```
int numSectors();  
const odb_SequenceSequenceFloat symmetryAxis();
```

numSectors

An Int specifying the number of sectors in the cyclic symmetry model.

symmetryAxis

An odb_SequenceSequenceFloat specifying the coordinates of two points on the axis of symmetry.

61.30 **UserData object**

The UserData object contains user-defined XY data. The UserData object has no constructor; it is created automatically when an Odb object is created.

Access

`odb.userData()`

61.30.1 **XYData(...)**

This method creates an XYData object from a sequence of *X–Y* data pairs.

Path

`odb.userData().XYData`

Prototype

```
odb_UserData&
XYData(const odb_String& name,
        const odb_SequenceSequenceDouble& data,
        const odb_String& sourceDescription,
        const odb_String& contentDescription,
        const odb_String& positionDescription,
        const odb_String& legendLabel,
        const odb_String& xValuesLabel,
        const odb_String& yValuesLabel,
        const odb_QuantityType& axis1QuantityType,
        const odb_QuantityType& axis2QuantityType);
```

Required arguments

name

An odb_String specifying the repository key.

data

An odb_SequenceSequenceDouble specifying the *X–Y* data pairs.

Optional arguments

sourceDescription

An odb_String specifying the source of the X–Y data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

An odb_String specifying the content of the X–Y data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

An odb_String specifying additional information about the X–Y data (e.g., “for whole model”). The default value is an empty string.

legendLabel

An odb_String specifying the label to be used in the legend. The default value is the name of the XYData object.

xValuesLabel

An odb_String specifying the label for the X-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

yValuesLabel

An odb_String specifying the label for the Y-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

axis1QuantityType

A QuantityType object specifying the QuantityType object associated to the X -axis1- values.

axis2QuantityType

A QuantityType object specifying the QuantityType object associated to the Y -axis2- values.

Return value

An XYData object.

Exceptions

InvalidNameError.

61.30.2 Members

The UserData object can have the following members:

Prototype

```
odb_UserXYDataRepository& xyDataObjects();  
odb_UserXYData& xyDataObjects(const odb_String& xyName);
```

name

An odb_String specifying the repository key.

sourceDescription

An odb_String specifying the source of the *X–Y* data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

An odb_String specifying the content of the *X–Y* data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

An odb_String specifying additional information about the *X–Y* data (e.g., “for whole model”). The default value is an empty string.

xValuesLabel

An odb_String specifying the label for the *X*-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

yValuesLabel

An odb_String specifying the label for the *Y*-values. This value may be overridden if the *X–Y* data are combined with other *X–Y* data. The default value is an empty string.

axis1QuantityType

A QuantityType object specifying the QuantityType object associated to the *X* -axis1- values.

axis2QuantityType

A QuantityType object specifying the QuantityType object associated to the *Y* -axis2- values.

legendLabel

An odb_String specifying the label to be used in the legend. The default value is the name of the XYData object.

xyDataObjects

A repository of UserXYData objects.

data

An odb_SequenceSequenceDouble specifying the *X–Y* data pairs.

61.31 UserXYData object

The UserXYData object stores a sequence of pairs and information about the axes.

Access

`Api.userData().xyDataObjects()[name]`

61.31.1 **addData(...)**

This method replaces the contents of the *data* member of the XYData object.

Prototype

```
void  
addData(const odb_SequenceSequenceDouble& data);
```

Arguments

Required argument

data

An odb_SequenceSequenceDouble specifying the X–Y data pairs.

Optional arguments

None.

Return value

None

Exceptions

None.

61.31.2 **getData()**

Returns the contents of the *data* member of the XYData object.

Prototype

```
odb_SequenceSequenceFloat  
getData();
```

Arguments

None.

Return value

An odb_SequenceSequenceFloat.

Exceptions

None.

61.31.3 Members

The UserXYData object can have the following members:

Prototype

```
odb_String name() const;
odb_String sourceDescription() const;
odb_String contentDescription() const;
odb_String positionDescription() const;
odb_String xAxisLabel() const;
odb_String yAxisLabel() const;
odb_String legendLabel() const;
odb_String description() const;
odb_SequenceSequenceDouble data() const;
```

name

An odb_String specifying the repository key.

sourceDescription

An odb_String specifying the source of the X–Y data (e.g., “Entered from keyboard”, “Taken from ASCII file”, “Read from an ODB”, etc.). The default value is an empty string.

contentDescription

An odb_String specifying the content of the X–Y data (e.g., “field 1 vs. field 2”). The default value is an empty string.

positionDescription

An odb_String specifying additional information about the X–Y data (e.g., “for whole model”). The default value is an empty string.

xAxisLabel

An odb_String specifying the label for the X-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

yAxisLabel

An odb_String specifying the label for the Y-values. This value may be overridden if the X–Y data are combined with other X–Y data. The default value is an empty string.

legendLabel

An odb_String specifying the label to be used in the legend. The default value is the name of the XYData object.

description

An odb_String specifying the complete description of the XYData object.

data

An odb_SequenceSequenceDouble specifying the X–Y data pairs.

61.32 Top level commands

The Odb commands do the following:

- Determine if an output database (`.odb`) file needs to be upgraded to the current release.
- Determine the extreme value for an output variable over a number of fields; for example, over a number of load cases.
- Open an existing output database file and create a new Odb object.
- Upgrade an output database file to the current release and write the upgraded output database to a new file.
- Fetch the active output database from inside a C++ user subroutine during an analysis run.
- Determine the actual subtype of a base Section, Profile, or Amplitude object.
- Cast a base Section, Profile, or Amplitude object to the proper subtype.

61.32.1 `isUpgradeRequiredForOdb(...)`

This method determines if an output database file needs to be upgraded to the current release.

Prototype

```
bool isUpgradeRequiredForOdb(const odb_String& upgradeRequiredOdbPath);
```

Arguments

Required argument

upgradeRequiredOdbPath

An `odb_String` specifying the path to an output database file to test. The test determines if the output database needs to be upgraded to the current release.

Optional arguments

None.

Return value

A Boolean indicating the result of the test. A value of `True` indicates that the output database needs to be upgraded to the current release.

Exceptions

None.

61.32.2 maxEnvelope(...)

Retrieve the maximum value of an output variable over a number of fields.

Prototype

```
odb_SequenceFieldOutput maxEnvelope(
    const odb_SequenceFieldOutput & inputFields);
odb_SequenceFieldOutput maxEnvelope(
    const odb_SequenceFieldOutput & inputFields,
    odb_Enum::odb_InvariantEnum invariant);
odb_SequenceFieldOutput maxEnvelope(
    const odb_SequenceFieldOutput & inputFields,
    const odb_String& componentLabel);
```

Arguments

Required arguments

Argument
An odb_SequenceFieldOutput object containing all the fieldOutput objects from which the maximum value will be computed. A SymbolicConstant specifying the invariant or component label to be used when comparing vectors or tensors. Possible values are • odb_Enum::MAGNITUDE • odb_Enum::MISES • odb_Enum::TRESCA • odb_Enum::PRESS • odb_Enum::INV3 • odb_Enum::MAX_PRINCIPAL • odb_Enum::MID_PRINCIPAL • odb_Enum::MIN_PRINCIPAL You must provide either this argument or the following argument if the field is a vector or tensor. An odb_String specifying the component of the tensor or vector to be used for selecting the maximum value.

Optional arguments

None.

Return value

An odb_SequenceFieldOutput object. The first fieldOutput object contains the maximum value. The second fieldOutput object contains the index of the field containing the maximum value. The index follows the order in which fields are positioned in the list of fieldOutput objects provided as the argument to the function.

Exceptions

OdbError

TypeError

This function takes no keyword arguments.

61.32.3 minEnvelope(...)

Retrieve the minimum value of an output variable over a number of fields.

Prototype

```

odb_SequenceFieldOutput minEnvelope(
    const odb_SequenceFieldOutput & inputFields);
odb_SequenceFieldOutput minEnvelope(
    const odb_SequenceFieldOutput & inputFields,
    odb_Enum::odb_InvariantEnum invariant);
odb_SequenceFieldOutput minEnvelope(
    const odb_SequenceFieldOutput & inputFields,
    const odb_String& componentLabel);
    
```

Arguments

Required arguments

Argument
An odb_SequenceFieldoutput object containing all the fieldOutput objects from which the maximum value will be computed.

Argument

A SymbolicConstant specifying the invariant or component label to be used when comparing vectors or tensors. Possible values are

- odb_Enum::MAGNITUDE
- odb_Enum::MISES
- odb_Enum::TRESCA
- odb_Enum::PRESS
- odb_Enum::INV3
- odb_Enum::MAX_PRINCIPAL
- odb_Enum::MID_PRINCIPAL
- odb_Enum::MIN_PRINCIPAL

You must provide either this argument or the following argument if the field is a vector or tensor.

An odb_String specifying the component of the tensor or vector to be used for selecting the minimum value.

Optional arguments

None.

Return value

An odb_SequenceFieldOutput object. The first fieldOutput object contains the minimum value. The second fieldOutput object contains the index of the field containing the minimum value. The index follows the order in which fields are positioned in the list of fieldOutput objects provided as the argument to the function.

Exceptions

OdbError

TypeError

This function takes no keyword arguments.

61.32.4 openOdb(...)

This method opens an existing output database (.odb) file and creates a new Odb object.

Prototype

```
odb_Odb& openOdb(const odb_String& path);
```

Arguments

Required argument

path

A odb_String specifying the path to an existing output database (**.odb**) file.

Optional argument

readOnly

A Boolean specifying whether the file will permit only read access or both read and write access. The initial value is False, indicating that both read and write access will be permitted.

readInternalSets

A Boolean specifying whether the file will permit access to sets specified as Internal on the database. The initial value is False, indicating that internal sets will not be read.

Return value

An Odb object.

Exceptions

If the output database was generated by a previous release of Abaqus and needs upgrading:

OdbError: The database is from a previous release of Abaqus. Run **abaqus upgrade -job <newFilename> -odb <oldFileName>** to upgrade it.

If the output database was generated by a newer release of Abaqus, and the installation of Abaqus needs upgrading:

OdbError: Abaqus installation must be upgraded before this output database can be opened.

61.32.5 openOdb(...)

This method opens an existing output database (**.odb**) file and creates a new Odb object.

Prototype

```
odb_Odb& openOdb(const odb_String& name,  
const odb_String& path, bool readOnly);
```

Arguments

Required argument

name

A odb_String specifying the repository key.

Optional arguments

path

A odb_String specifying the path to an existing output database (**.odb**) file.

readOnly

A Boolean specifying whether the file will permit only read access or both read and write access. The initial value is False, indicating that both read and write access will be permitted.

Return value

An Odb object.

Exceptions

If the output database was generated by a previous release of Abaqus and needs upgrading:

OdbError: The database is from a previous release of Abaqus. Run **abaqus upgrade -job <newFileName> -odb <oldFileName>** to upgrade it.

If the output database was generated by a newer release of Abaqus, and the installation of Abaqus needs upgrading:

OdbError: Abaqus installation must be upgraded before this output database can be opened.

If the file is not a valid database:

AbaqusException: Cannot open file <filename>.

61.32.6 upgradeOdb(...)

This method upgrades an existing Odb object to the current release and writes the upgraded version of the Odb object to a file. In addition, Abaqus/CAE writes information about the status of the upgrade to a log (***.log**) file.

Prototype

```
void upgradeOdb(const odb_String& existingOdbPath,
const odb_String& upgradedOdbPath);
```

Arguments

Required arguments

existingOdbPath

An odb_String specifying the path to the file containing the output database to be upgraded.

upgradedOdbPath

An odb_String specifying the path to the file that will contain the upgraded output database.

Optional arguments

None.

Return value

None

Exceptions

If the output database upgrade fails:

OdbError: cannot convert database

61.32.7 **getActiveOdb(...)**

This method is intended to be used inside an Abaqus/Standard or Abaqus/Explicit user subroutine written in C++. It fetches the object that is in use by the analysis product and enables a user to modify it inside the user subroutine. This method will throw an exception if more than one output database is currently open.

Prototype

```
odb_Odb& getActiveOdb();
```

Arguments

None.

Return value

An object.

Exceptions

None.

If more than one output database file is open:

OdbError: The getActiveOdb method can only be called if a single odb file is open.

If no output database files are open:

OdbError: No odb files are open.

61.32.8 **odb_IsA(...)**

This method takes a type and a base object and returns a boolean indicating whether the subtype of the supplied base object matches the type argument. This method is useful when fetching base Section, Profile, or Amplitude objects from their containers. If, for example, the user wants to determine if the base odb_Section object returned by the odb_SectionContainer is actually of type odb_BeamSection, he would call this method with the odb_BeamSection as the type and the odb_Section object as the base object.

Prototype

```
bool odb_IsA(TYPE, const BASETYPE& obj);
```

Arguments

Required arguments

TYPE

The potential type of the base class object.

obj

The base object being tested for the TYPE.

Optional arguments

None.

Return value

A Boolean.

Exceptions

None.

61.32.9 odb_dynamicCast(...)

This method casts a supplied Section, Profile, or Amplitude to the specified subtype. This method is useful for retrieving the base objects out of the appropriate containers and casting them to the subtype to access the data members for that particular subtype. This method will throw an odb_BaseException if the cast is not possible.

Prototype

```
const TYPE& returnObj = odb_dynamicCast(TYPE, const BASETYPE& obj);
```

Arguments

Required arguments

TYPE

The potential type of the base class object.

obj

The base object being case to TYPE.

Optional arguments

None.

Return value

An object of type TYPE.

Exceptions

If the cast is not possible:

Bad cast: From BASETYPE To TYPE.

62. Property commands

The Property commands are used to create and manage reinforcements and to assign properties to a part or part instance.

62.1 SectionAssignment object

The SectionAssignment object is used to specify a section assignment on an assembly, part or part instance. Section assignments on the assembly are limited to connector elements only.

Access

```
odb.parts() [name].sectionAssignments(i)
odb.rootAssembly().instances() [name].sectionAssignments(i)
odb.rootAssembly().sectionAssignments(i)
odb.steps() [name].frames(i).fieldOutputs() [name].values(i).instance() \
.sectionAssignments(i)
```

62.1.1 Members

The SectionAssignment object has members with the same names and descriptions as the arguments to the **SectionAssignment** method.

In addition, the SectionAssignment object has the following member:

Prototype

```
odb_Set region() const;
      odb_String sectionName() const;
      double offset() const;
```

suppressed

A Boolean specifying whether the section assignment is suppressed or not. The default value is false.

63. Section commands

The Section commands are used to create sections and profiles with their associated properties and behavior. The various section objects are all derived from the Section object. The various profile objects are all derived from the Profile object. See Chapter 44, “Property commands,” for the property assignment commands.

63.1 Section object

The Section object defines the properties of a section. The Section object is the abstract base type for other Section objects. The Section object has no explicit constructor. The methods and members of the Section object are common to all objects derived from the Section.

Access

`sectionApi.sections()` [*name*]

63.1.1 Members

The Section object has the following member:

Prototype

`odb_String name() const;`

name

An `odb_String` specifying the repository key.

63.2 AcousticInfiniteSection object

The `AcousticInfiniteSection` object defines the properties of an acoustic section.

The `AcousticInfiniteSection` object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.2.1 AcousticInfiniteSection(...)

This method creates an `AcousticInfiniteSection` object.

Path

`sectionApi.AcousticInfiniteSection`

Prototype

```
odb_AcousticInfiniteSection&
AcousticInfiniteSection(const odb_String& name,
                        const odb_String& material,
                        double thickness,
                        int order);
```

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material.

Optional arguments

thickness

A Double specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

order

An Int specifying the number of ninth-order polynomials that will be used to resolve the variation of the acoustic field in the infinite direction. Possible values are $0 < order \leq 10$. The default value is 10.

Return value

An AcousticInfiniteSection object.

Exceptions

InvalidNameError and RangeError.

63.2.2 Members

The AcousticInfiniteSection object has members with the same names and descriptions as the arguments to the **AcousticInfiniteSection** method.

63.2.3 Corresponding analysis keywords

*SOLID SECTION

63.3 AcousticInterfaceSection object

The AcousticInterfaceSection object defines the properties of an acoustic section.

The AcousticInterfaceSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.3.1 AcousticInterfaceSection(...)

This method creates an AcousticInterfaceSection object.

Path

`sectionApi.AcousticInterfaceSection`

Prototype

```
odb_AcousticInterfaceSection&
AcousticInterfaceSection(const odb_String& name,
                        double thickness);
```

Required argument

name

An odb_String specifying the repository key.

Optional argument

thickness

A Double specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

Return value

An AcousticInterfaceSection object.

Exceptions

InvalidNameError and RangeError.

63.3.2 Members

The `AcousticInterfaceSection` object has members with the same names and descriptions as the arguments to the `AcousticInterfaceSection` method.

63.3.3 Corresponding analysis keywords

*INTERFACE

63.4 BeamSection object

The `BeamSection` object defines the properties of a beam section.

The `BeamSection` object is derived from the `Section` object.

Access

`sectionApi.sections()` [*name*]

63.4.1 BeamSection(...)

This method creates a `BeamSection` object.

Path

`sectionApi.BeamSection`

Prototype

```
odb_BeamSection&
BeamSection(const odb_String& name,
            const odb_String& integration,
            const odb_String& profile,
            double poissonRatio,
            bool thermalExpansion,
            bool temperatureDependency,
            int dependencies,
            odb_Union density,
            odb_Union referenceTemperature,
            const odb_String& temperatureVar,
```

```

double alphaDamping,
double betaDamping,
double compositeDamping,
bool useFluidInertia,
const odb_String& submerged,
odb_Union fluidMassDensity,
odb_Union crossSectionRadius,
double lateralMassCoef,
double axialMassCoef,
double massOffsetX,
double massOffsetY,
const odb_String& material,
const odb_SequenceSequenceDouble& table,
const odb_SequenceSequenceDouble& outputPts,
const odb_SequenceDouble& centroid,
const odb_SequenceDouble& shearCenter);

```

Required arguments

name

An odb_String specifying the repository key.

integration

An odb_String specifying the integration method for the section. Possible values are "BEFORE_ANALYSIS" and "DURING_ANALYSIS".

profile

An odb_String specifying the name of the profile.

Optional arguments

poissonRatio

A Double specifying the Poisson's ratio of the section. The default value is 0.0.

thermalExpansion

A Boolean specifying whether to use thermal expansion data. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

density

The string "NONE" or a Double specifying the density of the section. The default value is "NONE".

SECTION COMMANDS

referenceTemperature

The string "NONE" or a Double specifying the reference temperature of the section. The default value is "NONE".

temperatureVar

An odb_String specifying the temperature variation for the section. Possible values are "LINEAR" and "INTERPOLATED". The default value is "LINEAR".

alphaDamping

A Double specifying the α_R factor to create mass proportional damping in direct-integration dynamics. The default value is 0.0.

betaDamping

A Double specifying the β_R factor to create stiffness proportional damping in direct-integration dynamics. The default value is 0.0.

compositeDamping

A Double specifying the fraction of critical damping to be used in calculating composite damping factors for the modes (for use in modal dynamics). The default value is 0.0.

useFluidInertia

A Boolean specifying whether added mass effects will be simulated. The default value is false.

submerged

An odb_String specifying whether the section is either full submerged or half submerged. This argument applies only when *useFluidInertia* = True. Possible values are "FULLY" and "HALF". The default value is "FULLY".

fluidMassDensity

The string "NONE" or a Double specifying the mass density of the fluid. This argument applies only when *useFluidInertia* = True and must be specified in that case. The default value is "NONE".

crossSectionRadius

The string "NONE" or a Double specifying the radius of the cylindrical cross-section. This argument applies only when *useFluidInertia* = True and must be specified in that case. The default value is "NONE".

lateralMassCoef

A Double specifying the added mass coefficient, C_A , for lateral motions of the beam. This argument applies only when *useFluidInertia* = True. The default value is 1.0.

axialMassCoef

A Double specifying the added mass coefficient, $C(A - E)$, for motions along the axis of the beam. This argument affects only the term added to the free end(s) of the beam, and applies only when *useFluidInertia* = True. The default value is 0.0.

massOffsetX

A Double specifying the local 1-coordinate of the center of the cylindrical cross-section with respect to the beam cross-section. This argument applies only when *useFluidInertia* = True. The default value is 0.0.

massOffsetY

A Double specifying the local 2-coordinate of the center of the cylindrical cross-section with respect to the beam cross-section. This argument applies only when *useFluidInertia* = True. The default value is 0.0.

material

An odb_String specifying the name of the material. The default value is an empty string. The material is required when *integration* is "DURING_ANALYSIS".

table

An odb_SequenceSequenceDouble specifying the items described below. The default value is an empty sequence.

outputPts

An odb_SequenceSequenceDouble specifying the positions at which output is requested. The default value is an empty sequence.

centroid

An odb_SequenceDouble specifying the X–Y coordinates of the centroid. The default value is (0.0, 0.0).

shearCenter

An odb_SequenceDouble specifying the X–Y coordinates of the shear center. The default value is (0.0, 0.0).

Table data

The table data specify the following:

- E, the Young's modulus of the section.
- G, the torsional shear modulus of the section.
- Thermal expansion coefficient, if using thermal expansion.
- Temperature, if the data depend on temperature.
- Value of the first field variable, if the data depend on field variables.
- Value of the second field variable.
- Etc.

Return value

A BeamSection object.

Exceptions

None.

63.4.2 Members

The BeamSection object has members with the same names and descriptions as the arguments to the **BeamSection** method.

In addition, the BeamSection object can have the following member:

Prototype

```
odb_TransverseShearBeam beamTransverseShear() const;
```

beamTransverseShear

A TransverseShearBeam object specifying the transverse shear stiffness properties.

63.4.3 Corresponding analysis keywords

- *BEAM GENERAL SECTION
- *BEAM SECTION
- *BEAM FLUID INERTIA
- *CENTROID
- *DAMPING
- *SHEAR CENTER
- *SECTION POINTS

63.5 CohesiveSection object

The CohesiveSection object defines the properties of a cohesive section.

The CohesiveSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.5.1 CohesiveSection(...)

This method creates a CohesiveSection object.

Path

```
sectionApi.CohesiveSection
```


Prototype

```
odb_CohesiveSection&
CohesiveSection(const odb_String& name,
                 const odb_String& response,
                 const odb_String& material,
                 const odb_String& initialThicknessType,
                 double initialThickness,
                 odb_Union outOfPlaneThickness);
```

Required arguments*name*

An odb_String specifying the repository key.

response

An odb_String specifying the geometric assumption that defines the constitutive behavior of the cohesive elements. Possible values are "TRACTION_SEPARATION", "CONTINUUM", and "GASKET".

material

An odb_String specifying the name of the material.

Optional arguments*initialThicknessType*

An odb_String specifying the method used to compute the initial thickness. Possible values are:

- "SOLVER_DEFAULT", specifying that Abaqus will use the analysis product default
- "GEOMETRY", specifying that Abaqus will compute the thickness from the nodal coordinates of the elements.
- "SPECIFY", specifying that Abaqus will use the value given for *initialThickness*

The default value is "SOLVER_DEFAULT".

initialThickness

A Double specifying the initial thickness for the section. The *initialThickness* argument applies only when *initialThicknessType*="SPECIFY". The default value is 1.0.

outOfPlaneThickness

The string "NONE" or a Double specifying the out-of-plane thickness for the section. The default value is "NONE".

Return value

A CohesiveSection object.

Exceptions

RangeError and InvalidNameError.

63.5.2 Members

The CohesiveSection object has members with the same names and descriptions as the arguments to the `CohesiveSection` method.

63.5.3 Corresponding analysis keywords

*COHESIVE SECTION

63.6 CompositeShellSection object

The CompositeShellSection object defines the properties of a composite shell section.

The CompositeShellSection object is derived from the GeometryShellSection object.

Access

`sectionApi.sections()` [*name*]

63.6.1 CompositeShellSection(...)

This method creates a CompositeShellSection object.

Path

`sectionApi.CompositeShellSection`

Prototype

```
odb_CompositeShellSection& CompositeShellSection(const odb_String& name,  
    const kseO_SequenceSectionLayer& layup,  
    bool symmetric,  
    odb_String layupName,  
    odb_String thicknessType,  
    bool preIntegrate,  
    odb_String poissonDefinition,  
    double poisson,
```

```

    odb_String integrationRule,
    odb_String temperature,
    odb_String simplification,
    odb_Union nTemp,
    odb_Union thicknessModulus,
    bool useDensity,
    double density,
    odb_String thicknessField,
    odb_String nodalThicknessField);

```

Required arguments

name

An odb_String specifying the repository key.

layup

A sequence of SectionLayer objects specifying the shell cross-section.

Optional arguments

symmetric

A Boolean specifying whether or not the layup should be made symmetric by the analysis. The default value is false.

layupName

An odb_String specifying the layup name for this section. The default value is an empty string.

thicknessType

An odb_String specifying the distribution used for defining the thickness of the elements. Possible values are "UNIFORM", "ANALYTICAL_FIELD", "DISCRETE_FIELD", "NODAL_ANALYTICAL_FIELD", and "NODAL_DISCRETE_FIELD". The default value is "UNIFORM".

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (true) or integrated during the analysis (false). The default value is false.

poissonDefinition

An odb_String specifying whether to use the default value for the Poisson's ratio. Possible values are:

- "DEFAULT", specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- "VALUE", specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is "DEFAULT".

SECTION COMMANDS

poisson

A Double specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*="VALUE". The default value is 0.5.

integrationRule

An odb_String specifying the shell section integration rule. Possible values are "SIMPSON" and "GAUSS". The default value is "SIMPSON".

temperature

An odb_String specifying the mode used for temperature and field variable input across the section thickness. Possible values are "GRADIENT" and "POINTWISE". The default value is "GRADIENT".

idealization

An odb_String specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to true. Possible values are "NO_IDEALIZATION", "SMEAR_ALL_LAYERS", "MEMBRANE", and "BENDING". The default value is "NO_IDEALIZATION".

nTemp

The string "NONE" or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*="POINTWISE". The default value is "NONE".

thicknessModulus

The string "NONE" or a Double specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is "NONE".

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is false.

density

A Double specifying the value of density to apply to this section. The default value is 0.0.

thicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*="ANALYTICAL_FIELD" or *thicknessType*="DISCRETE_FIELD". The default value is an empty string.

nodalThicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*="NODAL_ANALYTICAL_FIELD" or *thicknessType*="NODAL_DISCRETE_FIELD". The default value is an empty string.

Return value

A CompositeShellSection object.

Exceptions

None.

63.6.2 Members

The CompositeShellSection object has members with the same names and descriptions as the arguments to the **CompositeShellSection** method.

In addition, the CompositeShellSection object can have the following members:

Prototype

```
odb_RebarLayers rebarLayers() const;
odb_TransverseShearShell transverseShear() const;
```

rebarLayers

A RebarLayers object specifying reinforcement properties.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

63.6.3 Corresponding analysis keywords

*SHELL SECTION

*SHELL GENERAL SECTION

63.7 CompositeSolidSection object

The CompositeSolidSection object defines the properties of a composite solid section.

The CompositeSolidSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.7.1 CompositeSolidSection(...)

This method creates a CompositeSolidSection object.

Path

`sectionApi.CompositeSolidSection`

Prototype

```
odb_CompositeSolidSection&  
CompositeSolidSection(const odb_String& name,  
                      const odb_SequenceSectionLayer& layup,  
                      bool symmetric,  
                      const odb_String& layupName);
```

Required arguments

name

An odb_String specifying the repository key.

layup

A sequence of SectionLayer objects specifying the solid cross-section.

Optional arguments

symmetric

A Boolean specifying whether or not the layup should be made symmetric by the analysis. The default value is false.

layupName

An odb_String specifying the layup name for this section. The default value is an empty string.

Return value

A CompositeSolidSection object.

Exceptions

None.

63.7.2 Members

The CompositeSolidSection object has members with the same names and descriptions as the arguments to the `CompositeSolidSection` method.

63.7.3 Corresponding analysis keywords

*SOLID SECTION

63.8 ConnectorSection object

A ConnectorSection object describes the connection type and the behavior of a connector.
The ConnectorSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.8.1 ConnectorSection(...)

This method creates a ConnectorSection object.

Path

`sectionApi.ConnectorSection`

Prototype

```
odb_ConnectorSection&
ConnectorSection(const odb_String& name,
    const odb_String& assembledType,
    const odb_String& rotationalType,
    const odb_String& translationalType,
    const odb_String& integration,
    odb_Union u1ReferenceLength,
    odb_Union u2ReferenceLength,
    odb_Union u3ReferenceLength,
    odb_Union ur1ReferenceAngle,
    odb_Union ur2ReferenceAngle,
    odb_Union ur3ReferenceAngle,
    odb_Union massPerLength,
    odb_Union contactAngle,
    double materialFlowFactor,
    bool regularize,
    bool defaultTolerance,
    double regularization,
    const odb_String& extrapolation,
    const odb_SequenceConnectorBehaviorOption& behaviorOptions);
```

SECTION COMMANDS

Required argument

name

An odb_String specifying the repository key.

Optional arguments

assembledType

An odb_String specifying the assembled connection type. Possible values are:

- "NONE"
- "BEAM"
- "BUSHING"
- "CVJOINT"
- "CYLINDRICAL"
- "HINGE"
- "PLANAR"
- "RETRACTOR"
- "SLIPRING"
- "TRANSLATOR"
- "UJOINT"
- "WELD"

The default value is "NONE".

You cannot include the *assembledType* argument if *translationalType* or *rotationalType* are given a value other than "NONE". At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given a value other than "NONE".

rotationalType

An odb_String specifying the basic rotational connection type. Possible values are:

- "NONE"
- "ALIGN"
- "CARDAN"
- "CONSTANT_VELOCITY"
- "EULER"
- "FLEXION_TORSION"
- "FLOW_CONVERTER"
- "PROJECTION_FLEXION_TORSION"
- "REVOLUTE"
- "ROTATION"
- "ROTATION_ACCELEROMETER"

- "UNIVERSAL"

The default value is "NONE".

You cannot include the *rotationalType* argument if *assembledType* is given a value other than "NONE". At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given an value other than "NONE".

translationalType

An odb_String specifying the basic translational connection type. Possible values are:

- "NONE"
- "ACCELEROMETER"
- "AXIAL"
- "CARTESIAN"
- "JOIN"
- "LINK"
- "PROJECTION_CARTESIAN"
- "RADIAL_THRUST"
- "SLIDE_PLANE"
- "SLOT"

The default value is "NONE".

You cannot include the *translationalType* argument if *assembledType* is given a value other than "NONE". At least one of the arguments *assembledType*, *translationalType*, or *rotationalType* must be given an value other than "NONE".

integration

An odb_String specifying the time integration scheme to use for analysis. This argument is applicable only to an Abaqus/Explicit analysis. Possible values are "UNSPECIFIED", "IMPLICIT", and "EXPLICIT". The default value is "UNSPECIFIED".

u1ReferenceLength

The string "NONE" or a Double specifying the reference length associated with constitutive response for the first component of relative motion. The default value is "NONE".

u2ReferenceLength

The string "NONE" or a Double specifying the reference length associated with constitutive response for the second component of relative motion. The default value is "NONE".

u3ReferenceLength

The string "NONE" or a Double specifying the reference length associated with constitutive response for the third component of relative motion. The default value is "NONE".

ur1ReferenceAngle

The string "NONE" or a Double specifying the reference angle in degrees associated with constitutive response for the fourth component of relative motion. The default value is "NONE".

SECTION COMMANDS

ur2ReferenceAngle

The string "NONE" or a Double specifying the reference angle in degrees associated with constitutive response for the fifth component of relative motion. The default value is "NONE".

ur3ReferenceAngle

The string "NONE" or a Double specifying the reference angle in degrees associated with constitutive response for the sixth component of relative motion. The default value is "NONE".

massPerLength

The string "NONE" or a Double specifying the mass per unit reference length of belt material. This argument is applicable only when *assembledType*="SLIPRING", and must be specified in that case. The default value is "NONE".

contactAngle

The string "NONE" or a Double specifying the contact angle made by the belt wrapping around node b. This argument is applicable only to an Abaqus/Explicit analysis, and only when *assembledType*="SLIPRING". The default value is "NONE".

materialFlowFactor

A Double specifying the scaling factor for material flow at node b. This argument is applicable only when *assembledType*="RETRACTOR" or *rotationalType*="FLOW_CONVERTER". The default value is 1.0.

regularize

A Boolean specifying whether or not all tabular data associated with the *behaviorOptions* will be regularized. This argument is applicable only for an Abaqus/Explicit analysis. The default value is true.

defaultTolerance

A Boolean specifying whether or not the default regularization tolerance will be used for all tabular data associated with the *behaviorOptions*. This argument is applicable only for an Abaqus/Explicit analysis and only if *regularize*=true. The default value is true.

regularization

A Double specifying the regularization increment to be used for all tabular data associated with the *behaviorOptions*. This argument is applicable only for an Abaqus/Explicit analysis and only if *regularize*=true and *defaultTolerance*=false. The default value is 0.03.

extrapolation

An odb_String specifying the extrapolation technique to be used for all tabular data associated with the *behaviorOptions*. Possible values are "CONSTANT" and "LINEAR". The default value is "CONSTANT".

behaviorOptions

A sequence of ConnectorBehaviorOption objects.

Return value

A ConnectorSection object.

Exceptions

InvalidNameError and RangeError.

63.8.2 getVertices()

This method returns a sequence consisting of tuples of coordinates of the connector's endpoints.

Path

`sectionApi.getVertices`

Prototype

```
odb_ConnectorSection&
getVertices();
```

Arguments

None.

Return value

A sequence of tuples of floats.

Exceptions

None.

63.8.3 Members

The ConnectorSection object has members with the same names and descriptions as the arguments to the `ConnectorSection` method.

63.8.4 Corresponding analysis keywords

- *CONNECTOR SECTION
- *CONNECTOR BEHAVIOR
- *CONNECTOR CONSTITUTIVE REFERENCE

63.9 EulerianSection object

The EulerianSection object defines the properties of a Eulerian section.

The EulerianSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.9.1 EulerianSection(...)

This method creates a EulerianSection object.

Path

`sectionApi.EulerianSection`

Prototype

```
odb_EulerianSection&
EulerianSection(const odb_String& name,
                 const odb_StringRepository& data);
```

Required arguments

name

An odb_String specifying the repository key.

data

A repository of String objects specifying a dictionary mapping Material instance names to Material names. Internally the specified mapping gets sorted on Material instance name.

Optional arguments

None.

Return value

An EulerianSection object.

Exceptions

None.

63.9.2 Members

The EulerianSection object has members with the same names and descriptions as the arguments to the `EulerianSection` method.

63.9.3 Corresponding analysis keywords

*EULERIAN SECTION

63.10 GasketSection object

The GasketSection object defines the properties of a gasket section.

The GasketSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.10.1 GasketSection(...)

This method creates a GasketSection object.

Path

```
sectionApi.GasketSection
```

Prototype

```
odb_GasketSection&
GasketSection(const odb_String& name,
               const odb_String& material,
               double crossSection,
               double initialGap,
               odb_Union initialThickness,
               double initialVoid,
               odb_Union stabilizationStiffness);
```

SECTION COMMANDS

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material of which the gasket is made or material that defines gasket behavior.

Optional arguments

crossSection

A Double specifying the cross-sectional area, width, or out-of-plane thickness, if applicable, depending on the gasket element type. The default value is 1.0.

initialGap

A Double specifying the initial gap. The default value is 0.0.

initialThickness

The string "DEFAULT" or a Double specifying the initial gasket thickness. If "DEFAULT" is specified, the initial thickness is determined using nodal coordinates. The default value is "DEFAULT".

initialVoid

A Double specifying the initial void. The default value is 0.0.

stabilizationStiffness

The string "DEFAULT" or a Double specifying the default stabilization stiffness used in all but link elements to stabilize gasket elements that are not supported at all nodes, such as those that extend outside neighboring components. If "DEFAULT" is specified, a value is used equal to 10^{-9} times the initial compressive stiffness in the thickness direction. The default value is "DEFAULT".

Return value

A GasketSection object.

Exceptions

InvalidNameError and ValueError.

63.10.2 Members

The GasketSection object has members with the same names and descriptions as the arguments to the **GasketSection** method.

63.10.3 Corresponding analysis keywords

*GASKET SECTION

63.11 GeneralStiffnessSection object

The GeneralStiffnessSection object defines the properties of a shell section via the stiffness matrix.
The GeneralStiffnessSection object is derived from the ShellSection object.

Access

`sectionApi.sections() [name]`

63.11.1 GeneralStiffnessSection(...)

This method creates a GeneralStiffnessSection object.

Path

`sectionApi.GeneralStiffnessSection`

Prototype

```
odb_GeneralStiffnessSection&
GeneralStiffnessSection(const odb_String& name,
                        const odb_SequenceDouble& stiffnessMatrix,
                        odb_Union referenceTemperature,
                        bool applyThermalStress,
                        bool temperatureDependency,
                        int dependencies,
                        const odb_String& poissonDefinition,
                        double poisson,
                        bool useDensity,
                        double density,
                        const odb_SequenceDouble& thermalStresses,
                        const odb_SequenceSequenceDouble& scalingData);
```

Required arguments

name

An odb_String specifying the repository key.

stiffnessMatrix

An odb_SequenceDouble specifying the stiffness matrix for the section in the order D11, D12, D22, D13, D23, D33, ..., D66. Twenty-one entries must be given.

Optional arguments

referenceTemperature

The string "NONE" or a Float specifying the reference temperature for thermal expansion. The default value is "NONE".

applyThermalStress

A Boolean specifying whether or not the section stiffness varies with thermal stresses. The default value is false.

temperatureDependency

A Boolean specifying whether the data depend on temperature. The default value is false.

dependencies

An Int specifying the number of field variable dependencies. The default value is 0.

poissonDefinition

An odb_String specifying whether to use the default value for the Poisson's ratio. Possible values are:

- "DEFAULT", specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- "VALUE", specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is "DEFAULT".

poisson

A Double specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*="VALUE". The default value is 0.5.

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is false.

density

A Double specifying the value of density to apply to this section. The default value is 0.0.

thermalStresses

An odb_SequenceDouble specifying the generalized stress values caused by a unit temperature rise. Six entries must be given if the value of *applyThermalStress* is set to True. The default value is ("").

scalingData

An odb_SequenceSequenceDouble specifying the scaling factors for given temperatures and/or field data. Each row should contain (Y, alpha, T, F1,...,Fn). The default value is an empty sequence.

Return value

A GeneralStiffnessSection object.

Exceptions

None.

63.11.2 Members

The GeneralStiffnessSection object has members with the same names and descriptions as the arguments to the **GeneralStiffnessSection** method.

In addition, the GeneralStiffnessSection object can have the following members:

Prototype

```
odb_RebarLayers rebarLayers() const;
odb_TransverseShearShell transverseShear() const;
```

rebarLayers

A RebarLayers object specifying reinforcement properties.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

63.11.3 Corresponding analysis keywords

*SHELL GENERAL SECTION

63.12 GeometryShellSection object

The GeometryShellSection object defines the properties of a geometry shell section. The GeometryShellSection object has no explicit constructor and no methods. The GeometryShellSection object is an abstract base type.

The GeometryShellSection object is derived from the ShellSection object.

Access

```
sectionApi.sections() [name]
```

63.12.1 Members

The GeometryShellSection object can have the following members:

Prototype

```

odb_String name() const;
odb_String thicknessType() const;
bool preIntegrate() const;
odb_String poissonDefinition() const;
double poisson() const;
odb_String integrationRule() const;
odb_String temperature() const;
odb_String idealization() const;
odb_Union nTemp() const;
odb_Union thicknessModulus() const;
bool useDensity() const;
double density() const;
odb_String thicknessField() const;
odb_RebarLayers rebarLayers() const;
odb_String nodalThicknessField() const;
odb_TransverseShearShell transverseShear() const;

```

name

An odb_String specifying the repository key.

thicknessType

An odb_String specifying the distribution used for defining the thickness of the elements. Possible values are "UNIFORM", "ANALYTICAL_FIELD", "DISCRETE_FIELD", "NODAL_ANALYTICAL_FIELD", and "NODAL_DISCRETE_FIELD". The default value is "UNIFORM".

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (true) or integrated during the analysis (false). The default value is false.

poissonDefinition

An odb_String specifying whether to use the default value for the Poisson's ratio. Possible values are:

- "DEFAULT", specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- "VALUE", specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is "DEFAULT".

poisson

A Double specifying the Poisson's ratio. Possible values are $-1.0 \leq \text{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*="VALUE". The default value is 0.5.

integrationRule

An odb_String specifying the shell section integration rule. Possible values are "SIMPSON" and "GAUSS". The default value is "SIMPSON".

temperature

An odb_String specifying the mode used for temperature and field variable input across the section thickness. Possible values are "GRADIENT" and "POINTWISE". The default value is "GRADIENT".

idealization

An odb_String specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to true. Possible values are "NO_IDEALIZATION", "SMEAR_ALL_LAYERS", "MEMBRANE", and "BENDING". The default value is "NO_IDEALIZATION".

nTemp

The string "NONE" or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*="POINTWISE". The default value is "NONE".

thicknessModulus

The string "NONE" or a Double specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is "NONE".

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is false.

density

A Double specifying the value of density to apply to this section. The default value is 0.0.

thicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*="ANALYTICAL_FIELD" or *thicknessType*="DISCRETE_FIELD". The default value is an empty string.

rebarLayers

A RebarLayers object specifying reinforcement properties.

nodalThicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*="NODAL_ANALYTICAL_FIELD" or *thicknessType*="NODAL_DISCRETE_FIELD". The default value is an empty string.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

63.13 HomogeneousShellSection object

The HomogeneousShellSection object defines the properties of a shell section.

The HomogeneousShellSection object is derived from the GeometryShellSection object.

Access

`sectionApi.sections() [name]`

63.13.1 HomogeneousShellSection(...)

This method creates a HomogeneousShellSection object.

Path

`sectionApi.HomogeneousShellSection`

Prototype

```
odb_HomogeneousShellSection&
    HomogeneousShellSection(const odb_String& name,
        double thickness,
        const odb_String& material,
        int numIntPts,
        odb_String thicknessType,
        bool preIntegrate,
        odb_String poissonDefinition,
        double poisson,
        odb_String integrationRule,
        odb_String temperature,
        odb_String simplification,
        odb_Union nTemp,
        odb_Union thicknessModulus,
        bool useDensity,
        double density,
        odb_String thicknessField,
        odb_String nodalThicknessField);
```

Required arguments*name*

An odb_String specifying the repository key.

thickness

A Double specifying the thickness of the section. The *thickness* argument applies only when *thicknessType*="UNIFORM".

material

An odb_String specifying the name of the section material.

Optional arguments*numIntPts*

An Int specifying the number of integration points to be used through the section. Possible values are *numIntPts* > 0. The default value is 5.

To use the default settings of the analysis products, set *numIntPts* to 5 if *integrationRule*="SIMPSON" or set *numIntPts* to 7 if *integrationRule*="GAUSS".

thicknessType

An odb_String specifying the distribution used for defining the thickness of the elements. Possible values are "UNIFORM", "ANALYTICAL_FIELD", "DISCRETE_FIELD", "NODAL_ANALYTICAL_FIELD", and "NODAL_DISCRETE_FIELD". The default value is "UNIFORM".

preIntegrate

A Boolean specifying whether the shell section properties are specified by the user prior to the analysis (true) or integrated during the analysis (false). The default value is false.

poissonDefinition

An odb_String specifying whether to use the default value for the Poisson's ratio. Possible values are:

- "DEFAULT", specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- "VALUE", specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is "DEFAULT".

poisson

A Double specifying the Poisson's ratio. Possible values are $-1.0 \leq \textit{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*="VALUE". The default value is 0.5.

integrationRule

An odb_String specifying the shell section integration rule. Possible values are "SIMPSON" and "GAUSS". The default value is "SIMPSON".

SECTION COMMANDS

temperature

An odb_String specifying the mode used for temperature and field variable input across the section thickness. Possible values are "GRADIENT" and "POINTWISE". The default value is "GRADIENT".

idealization

An odb_String specifying the mechanical idealization used for the section calculations. This member is only applicable when *preIntegrate* is set to true. Possible values are "NO_IDEALIZATION", "SMEAR_ALL_LAYERS", "MEMBRANE", and "BENDING". The default value is "NO_IDEALIZATION".

nTemp

The string "NONE" or an Int specifying the number of temperature points to be input. This argument is valid only when *temperature*="POINTWISE". The default value is "NONE".

thicknessModulus

The string "NONE" or a Double specifying the effective thickness modulus. This argument is relevant only for continuum shells and must be used in conjunction with the argument *poisson*. The default value is "NONE".

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is false.

density

A Double specifying the value of density to apply to this section. The default value is 0.0.

thicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*="ANALYTICAL_FIELD" or *thicknessType*="DISCRETE_FIELD". The default value is an empty string.

nodalThicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements at each node. The *nodalThicknessField* argument applies only when *thicknessType*="NODAL_ANALYTICAL_FIELD" or *thicknessType*="NODAL_DISCRETE_FIELD". The default value is an empty string.

Return value

A HomogeneousShellSection object.

Exceptions

None.

63.13.2 Members

The HomogeneousShellSection object has members with the same names and descriptions as the arguments to the **HomogeneousShellSection** method.

In addition, the HomogeneousShellSection object can have the following members:

Prototype

```
odb_RebarLayers rebarLayers() const;
odb_TransverseShearShell transverseShear() const;
```

rebarLayers

A RebarLayers object specifying reinforcement properties.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

63.13.3 Corresponding analysis keywords

*SHELL SECTION

*SHELL GENERAL SECTION

63.14 HomogeneousSolidSection object

The HomogeneousSolidSection object defines the properties of a solid section.

The HomogeneousSolidSection object is derived from the SolidSection object.

Access

```
sectionApi.sections() [name]
```

63.14.1 HomogeneousSolidSection(...)

This method creates a HomogeneousSolidSection object.

Path

```
sectionApi.HomogeneousSolidSection
```

Prototype

```
odb_HomogeneousSolidSection&  
HomogeneousSolidSection(const odb_String& name,  
                        const odb_String& material,  
                        odb_Union thickness);
```

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material.

Optional argument

thickness

The string "NONE" or a Double specifying the thickness of the section. Possible values are None or a floating point value such that $thickness > 0.0$. The default value is "None".

Return value

A HomogeneousSolidSection object.

Exceptions

InvalidNameError and RangeError.

63.14.2 Members

The HomogeneousSolidSection object has members with the same names and descriptions as the arguments to the **HomogeneousSolidSection** method.

63.14.3 Corresponding analysis keywords

*SOLID SECTION

63.15 LayerProperties object

The LayerProperties object defines the properties of a layer of reinforcement for membrane, shell, and surface sections.

Access

```
sectionApi.sections() [name].rebarLayers().layerTable(i)
```

63.15.1 LayerProperties(...)

This method creates a LayerProperties object.

Path

```
LayerProperties
```

Prototype

```
odb_LayerProperties&
LayerProperties(double barArea,
                odb_Union orientationAngle,
                const odb_String& layerName,
                const odb_String& material,
                double barSpacing,
                double layerPosition,
                double spacingAngle,
                double extensionRatio,
                double radius);
```

Required arguments

barArea

A Double specifying the area per bar.

orientationAngle

An odb_Union specifying the orientation of the rebar. A Float specifies the angular orientation; a String specifies an orientation name.

layerName

An odb_String specifying the name of the rebar layer.

material

An odb_String specifying the name of the rebar material.

Optional arguments

barSpacing

A Double specifying the spacing of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to "CONSTANT". The default value is 0.0.

SECTION COMMANDS

layerPosition

A Double specifying the position of the rebar from the middle surface of the shell. *layerPosition* applies only for homogeneous shell sections and composite shell sections. The default value is 0.0.

spacingAngle

A Double specifying the spacing angle of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to "ANGULAR". The default value is 0.0.

extensionRatio

A Double specifying the extension ratio for the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to "LIFT_EQUATION". The default value is 0.0.

radius

A Double specifying the radius of the rebar. This argument is only valid if the *rebarSpacing* argument on the parent RebarLayers object is set to "LIFT_EQUATION". The default value is 0.0.

Return value

A LayerProperties object.

Exceptions

None.

63.15.2 Members

The LayerProperties object has members with the same names and descriptions as the arguments to the **LayerProperties** method.

63.15.3 Corresponding analysis keywords

*REBAR LAYER

63.16 MembraneSection object

The MembraneSection object defines the properties of a membrane section.

The MembraneSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.16.1 MembraneSection(...)

This method creates a MembraneSection object.

Path

```
sectionApi.MembraneSection
```

Prototype

```
odb_MembraneSection&
MembraneSection(const odb_String& name,
                 const odb_String& material,
                 double thickness,
                 const odb_String& thicknessType,
                 const odb_String& poissonDefinition,
                 double poisson,
                 const odb_String& thicknessField);
```

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material.

Optional arguments

thickness

A Double specifying the thickness for the section. Possible values are *thickness* > 0.0. The default value is 1.0.

thicknessType

An odb_String specifying the distribution used for defining the thickness of the elements. Possible values are "UNIFORM", "ANALYTICAL_FIELD", and "DISCRETE_FIELD". The default value is "UNIFORM".

poissonDefinition

An odb_String specifying whether to use the default value for the Poisson's ratio. Possible values are:

SECTION COMMANDS

- "DEFAULT", specifying that the default value for the Poisson's ratio is 0.5 in an Abaqus/Standard analysis and is obtained from the material definition in an Abaqus/Explicit analysis.
- "VALUE", specifying that the Poisson's ratio used in the analysis is the value provided in *poisson*.

The default value is "DEFAULT".

poisson

A Double specifying the section Poisson's ratio. Possible values are $-1.0 \leq \textit{poisson} \leq 0.5$. This argument is valid only when *poissonDefinition*="VALUE". The default value is 0.5.

thicknessField

An odb_String specifying the name of the AnalyticalField or DiscreteField object used to define the thickness of the shell elements. The *thicknessField* argument applies only when *thicknessType*="ANALYTICAL_FIELD" or *thicknessType*="DISCRETE_FIELD". The default value is an empty string.

Return value

A MembraneSection object.

Exceptions

RangeError and InvalidNameError.

63.16.2 Members

The MembraneSection object has members with the same names and descriptions as the arguments to the **MembraneSection** method.

In addition, the MembraneSection object can have the following member:

Prototype

```
odb_RebarLayers rebarLayers() const;
```

rebarLayers

A RebarLayers object specifying reinforcement properties.

63.16.3 Corresponding analysis keywords

*MEMBRANE SECTION

63.17 MPCSection object

The MPCSection object defines the properties of a multi-point constraint section.

The MPCSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.17.1 MPCSection(...)

This method creates a MPCSection object.

Path

`sectionApi.MPCSection`

Prototype

```
odb_MPCSection&
MPCSection(const odb_String& name,
            const odb_String& mpcType,
            const odb_String& userMode,
            int userType);
```

Required arguments

name

An odb_String specifying the repository key.

mpcType

An odb_String specifying the MPC type of the section. Possible values are "BEAM_MPC", "ELBOW_MPC", "PIN_MPC", "LINK_MPC", "TIE_MPC", and "USER_DEFINED".

Optional arguments

userMode

An odb_String specifying the mode of the MPC when it is user-defined. Possible values are "DOF_MODE" and "NODE_MODE". The default value is "DOF_MODE".

The *userMode* argument applies only when *mpcType*="USER_DEFINED".

userType

An Int specifying to differentiate between different constraint types in a user-defined MPCSection. The default value is 0.

The *userType* argument applies only when *mpcType*="USER_DEFINED".

SECTION COMMANDS

Return value

A MPCSection object.

Exceptions

RangeError and InvalidNameError.

63.17.2 Members

The MPCSection object has members with the same names and descriptions as the arguments to the MPCSection method.

63.17.3 Corresponding analysis keywords

*MPC

63.18 PEGSection object

The PEGSection object defines the properties of a solid section.

The PEGSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.18.1 PEGSection(...)

This method creates a PEGSection object.

Path

`sectionApi.PEGSection`

Prototype

```
odb_PEGSection&
PEGSection(const odb_String& name,
           const odb_String& material,
           double thickness,
```

```
double wedgeAngle1,
double wedgeAngle2);
```

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material.

Optional arguments

thickness

A Double specifying the thickness of the section. Possible values are *thickness* > 0.0. The default value is 1.0.

wedgeAngle1

A Double specifying the value of the x component of the angle between the bounding planes, $\Delta\phi_x$. The default value is 0.0.

wedgeAngle2

A Double specifying the value of the y component of the angle between the bounding planes, $\Delta\phi_y$. The default value is 0.0.

Return value

A PEGSection object.

Exceptions

InvalidNameError and RangeError.

63.18.2 Members

The PEGSection object has members with the same names and descriptions as the arguments to the **PEGSection** method.

63.18.3 Corresponding analysis keywords

*SOLID SECTION

63.19 RebarLayers object

The RebarLayers object defines the rebar properties of a section.

Access

```
sectionApi.sections() [name].rebarLayers()
```

63.19.1 RebarLayers(...)

This method creates a RebarLayers object.

Path

```
sectionApi.sections() [name].RebarLayers
```

Prototype

```
odb_RebarLayers&
RebarLayers(const odb_String& rebarSpacing,
             const odb_SequenceLayerProperties& layerTable);
```

Required arguments

rebarSpacing

An odb_String specifying the type of rebar geometry. Possible values are "CONSTANT", "ANGULAR", and "LIFT_EQUATION".

layerTable

A sequence of LayerProperties objects specifying the layers of reinforcement.

Optional arguments

None.

Return value

A RebarLayers object.

Exceptions

None.

63.19.2 Members

The `RebarLayers` object has members with the same names and descriptions as the arguments to the `RebarLayers` method.

63.19.3 Corresponding analysis keywords

*REBAR LAYER

63.20 SectionLayer object

The `SectionLayer` object defines the material layer in a composite shell.

Access

`sectionApi.sections() [name].layup(i)`

63.20.1 SectionLayer(...)

This method creates a `SectionLayer` object.

Path

`SectionLayer`

Prototype

```
odb_SectionLayer&
SectionLayer(double thickness,
             const odb_String& material,
             odb_Union orientAngle,
             int numIntPts,
             const odb_String& axis,
             float angle,
             const odb_String& additionalRotationType,
             const odb_String& thicknessType,
             const odb_String& plyName,
             const odb_DatumCsys& orientation,
             const odb_String& additionalRotationField,
```

SECTION COMMANDS

```
const odb_String& thicknessField);
```

Required arguments

thickness

A Double specifying the thickness of the section layer.

material

An odb_String specifying the name of the section layer material.

Optional arguments

orientAngle

An odb_Union specifying the relative orientation of the section layer. A Float specifies the angular orientation; a String specifies a user-subroutine orientation name. If a String is specified, a user-subroutine orientation is used, otherwise the Float value is used as an angular orientation. The default value is 0.0.

numIntPts

An Int specifying the number of integration points to be used through the section. This argument is valid only if the *preIntegrate* argument on the parent CompositeShellSection object is set to true. The default value is 3.

axis

An odb_String specifying the axis of a cylindrical or spherical datum coordinate system about which an additional rotation is applied. For shells this axis is also the shell normal. This only applies if a valid reference is provided for the orientation. Possible values are "AXIS_1", "AXIS_2", and "AXIS_3". The default value is "AXIS_3".

angle

A Float specifying the angle of the additional rotation. This only applies if a valid reference is provided for the orientation. The default value is 0.0.

additionalRotationType

An odb_String specifying the method used to describe the additional rotation when a valid orientation is specified. Possible values are "ROTATION_NONE", "ROTATION_ANGLE", and "ROTATION_FIELD". The default value is "ROTATION_NONE".

thicknessType

An odb_String specifying the method used to describe the thickness. Possible values are "THICKNESS_MAGNITUDE" and "THICKNESS_DISCRETE_FIELD". The default value is "THICKNESS_MAGNITUDE".

plyName

An odb_String specifying the ply identifier for this section layer. The default value is "".

orientation

A DatumCsys object specifying a coordinate system reference for the relative orientation of this layer. If this reference is valid it is used as the relative orientation of the layer, otherwise the *orientAngle* is used as described.

additionalRotationField

An odb_String specifying the name of the field specifying the additional rotation. The default value is "".

thicknessField

An odb_String specifying the name of the field specifying the thickness The default value is "".

Return value

A SectionLayer object.

Exceptions

None.

63.20.2 Members

The SectionLayer object has members with the same names and descriptions as the arguments to the **SectionLayer** method.

63.20.3 Corresponding analysis keywords

*SHELL SECTION

*SHELL GENERAL SECTION

63.21 ShellSection object

The ShellSection object defines the properties of a shell section. The ShellSection object is derived from the Section object. The ShellSection object has no explicit constructor and no methods or members.

The ShellSection object is derived from the Section object.

Access

`sectionApi.sections() [name]`

63.21.1 Members

The ShellSection object can have the following members:

Prototype

`odb_String name() const;`

SECTION COMMANDS

```
odb_TransverseShearShell transverseShear() const;
```

name

An odb_String specifying the repository key.

transverseShear

A TransverseShearShell object specifying the transverse shear stiffness properties.

63.22 SolidSection object

The SolidSection object defines the properties of a solid section. The SolidSection object has no explicit constructor, no members, and no methods. The SolidSection object is an abstract base type.

The SolidSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.22.1 Members

The SolidSection object has the following member:

Prototype

```
odb_String name() const;
```

name

An odb_String specifying the repository key.

63.23 SurfaceSection object

The SurfaceSection object defines the properties of a surface section.

The SurfaceSection object is derived from the Section object.

Access

```
sectionApi.sections() [name]
```

63.23.1 SurfaceSection(...)

This method creates a SurfaceSection object.

Path

`sectionApi.SurfaceSection`

Prototype

```
odb_SurfaceSection&
SurfaceSection(const odb_String& name,
               bool useDensity,
               double density);
```

Required argument

name

An odb_String specifying the repository key.

Optional arguments

useDensity

A Boolean specifying whether or not to use the value of *density*. The default value is false.

density

A Double specifying the value of density to apply to this section. The default value is 0.0.

Return value

A SurfaceSection object.

Exceptions

RangeError and InvalidNameError.

63.23.2 Members

The SurfaceSection object has members with the same names and descriptions as the arguments to the **SurfaceSection** method.

In addition, the SurfaceSection object can have the following member:

Prototype

```
odb_RebarLayers rebarLayers() const;
```

rebarLayers

A RebarLayers object specifying reinforcement properties.

63.23.3 Corresponding analysis keywords

*SURFACE SECTION

63.24 TransverseShearBeam object

The TransverseShearBeam object defines the transverse shear stiffness properties of a beam section.

Access

```
sectionApi.sections() [name].beamTransverseShear()
```

63.24.1 TransverseShearBeam(...)

This method creates a TransverseShearBeam object.

Path

```
sectionApi.sections() [name].TransverseShearBeam
```

Prototype

```
odb_TransverseShearBeam&
TransverseShearBeam(const odb_String& scfDefinition,
                    odb_Union k23,
                    odb_Union k13,
                    odb_Union slendernessCompensation);
```

Required argument

scfDefinition

An odb_String specifying how slenderness compensation factor of the section is given. Possible values are "ANALYSIS_DEFAULT", "COMPUTED", and "VALUE".

Optional arguments

k23

The string "NONE" or a Double specifying the k23 shear stiffness of the section. The default value is "NONE".

k13

The string "NONE" or a Double specifying the k13 shear stiffness of the section. The default value is "NONE".

slendernessCompensation

The string "COMPUTED" or a Double specifying the slenderness compensation factor of the section. The default value is 0.25.

Return value

A TransverseShearBeam object.

Exceptions

None.

63.24.2 Members

The TransverseShearBeam object has members with the same names and descriptions as the arguments to the **TransverseShearBeam** method.

63.24.3 Corresponding analysis keywords

*TRANSVERSE SHEAR STIFFNESS

63.25 TransverseShearShell object

The TransverseShearShell object defines the transverse shear stiffness properties of a shell section.

Access

```
sectionApi.sections() [name].transverseShear()
```

63.25.1 TransverseShearShell(...)

This method creates a TransverseShearShell object.

Path

```
sectionApi.sections() [name].TransverseShearShell
```

Prototype

```
odb_TransverseShearShell&  
TransverseShearShell(double k11,
```

SECTION COMMANDS

```
double k22,  
double k12);
```

Required arguments

k11

A Double specifying the shear stiffness of the section in the first direction.

k22

A Double specifying the shear stiffness of the section in the second direction.

k12

A Double specifying the coupling term in the shear stiffness of the section.

Optional arguments

None.

Return value

A TransverseShearShell object.

Exceptions

None.

63.25.2 Members

The TransverseShearShell object has members with the same names and descriptions as the arguments to the **TransverseShearShell** method.

63.25.3 Corresponding analysis keywords

*TRANSVERSE SHEAR STIFFNESS

63.26 TrussSection object

The TrussSection object defines the properties of a truss section.

The TrussSection object is derived from the Section object.

Access

`sectionApi.sections()` [*name*]

63.26.1 TrussSection(...)

This method creates a TrussSection object.

Path

`sectionApi.TrussSection`

Prototype

```
odb_TrussSection&
TrussSection(const odb_String& name,
              const odb_String& material,
              double area);
```

Required arguments

name

An odb_String specifying the repository key.

material

An odb_String specifying the name of the material.

Optional argument

area

A Double specifying the cross-sectional area for the section. Possible values are $area > 0$. The default value is 1.0.

Return value

A TrussSection object.

Exceptions

RangeError and InvalidNameError.

63.26.2 Members

The TrussSection object has members with the same names and descriptions as the arguments to the `TrussSection` method.

63.26.3 Corresponding analysis keywords

*SOLID SECTION

64. Infrastructure commands

The generic objects described in this chapter are used in the construction of specific Abaqus objects.

64.1 BaseException object

The odb_BaseException object catches all exceptions thrown in the output database

Access

```
catch(odb_BaseException& exc)
```

64.1.1 UserReport()

This method returns a description of the error condition that generated the exception.

Prototype

```
odb_String UserReport() const;
```

Arguments

None.

Return value

None

Exceptions

None.

64.2 Repository object

Repositories are provided to store objects retrieved by name. Both the repositories and the content of the repositories are created by the API; the user can only retrieve objects from repositories. Iterators are provided to navigate the repositories.

64.2.1 operator[](...)

This method accesses an item in an odb_Repository object. This method should be used with caution because it does not perform any checking of the index range.

Prototype

```
type [odb_string name] const;
```

Arguments**Required argument**

name

A char* or an odb_String specifying the object to be retrieved.

Optional arguments

None.

Return value

The object of type *type* saved in the repository under the given name.

Exceptions

None.

64.2.2 get/constGet(...)

This method accesses an item in an odb_Repository object.

Prototype

```
type get(const char* name);  
const type& constGet(const char* name) const;
```

Arguments**Required argument**

name

A char* or an odb_String specifying the object to be retrieved.

Optional arguments

None.

Return value

The object of type *type* saved in the repository under the given name.

Exceptions

None.

64.2.3 isMember(...)

This method checks whether an object with a given name is stored in the repository.

Prototype

```
bool isMember(const odb_String& name) const;  
bool isMember(const char* name) const;
```

Arguments**Required argument**

name

A char* or an odb_String specifying the name under which the object is stored.

Optional arguments

None.

Return value

A Boolean specifying whether the object is stored in the repository.

Exceptions

None.

64.2.4 size(...)

This method returns the number of items in an **odb_Repository** object.

Prototype

```
int size() const;
```

Arguments

None.

Return value

An Int indicating the number of items in the repository.

Exceptions

None.

64.3 **RepositoryIterator object**

Iterators are provided to navigate the repositories. Examples of using an iterator can be found in the example problem section.

64.3.1 **odb_TypeRepositoryIT(...)**

This method constructs an iterator for a repository.

Path

`odb_TypeRepositoryIT`

Prototype

`odb_TypeRepositoryIT(const odb_TypeRepository& repository);`

Required argument

repository

The repository over which to iterate.

Optional arguments

None.

Return value

An Iterator object.

Exceptions

None.

64.3.2 **first(...)**

This method sets the iterator to the first item in the repository.

Prototype

`first();`

Arguments

None.

Return value

None

Exceptions

None.

64.3.3 next(...)

This method sets the iterator to the next item in the repository.

Prototype

```
next () ;
```

Arguments

None.

Return value

None

Exceptions

None.

64.3.4 isDone(...)

This method checks whether the iterator is at the end of the repository.

Prototype

```
bool isDone () ;
```

Arguments

None.

Return value

A Boolean specifying whether the end of the repository has been reached.

Exceptions

None.

64.3.5 **currentKey(...)**

This method returns the name of the item to which the iterator is currently pointing.

Prototype

```
odb_String currentKey();
```

Arguments

None.

Return value

An `odb_String` specifying the name of the item that will be returned by the **currentValue** method.

Exceptions

None.

64.3.6 **currentValue(...)**

This method returns the item in the repository to which the iterator is currently pointing.

Prototype

```
const type& currentValue();
```

Arguments

None.

Return value

An item of the class *type* to which the iterator is currently pointing.

Exceptions

None.

64.4 Sequence object

An `odb_Sequence` object is a repository used to hold an ordered list of objects of a specific type. Data are added using the **append** method. The `odb_Sequence` object will automatically allocate memory to contain the appended object; the user may, however, control the efficiency of the memory allocation process through the constructor.

64.4.1 `odb_SequenceType(...)`

This method creates an empty `odb_Sequence` object of objects of type *Type*.

Path

`odb_SequenceType`

Prototype

`odb_SequenceType() ;`

Required arguments

None.

Optional arguments

initialSize

An Int specifying the amount of memory to be allocated when the object is created. The default value is 0.

Return value

An empty `odb_Sequence` object.

Exceptions

None.

64.4.2 `append(...)`

This method adds items to an `odb_Sequence` object.

Prototype

`void append(Type item);`

Arguments

Required argument

item

An item of type *Type* to be appended to the odb_Sequence object, where *Type* must be one of the following:

- Float
- Int
- String
- SectionLayer
- LayerProperties
- FieldOutput

For example, an odb_SequenceInt object accepts an argument of type Int.

Optional arguments

None.

Return value

None

Exceptions

None.

64.4.3 **constGet/get(...)**

This method accesses an item in an odb_Sequence object.

Prototype

```
Type get(int index);  
const Type& constGet(int index) const;
```

Arguments

Required argument

index

An Int specifying the index of the item to be accessed from the odb_Sequence object.

Optional arguments

None.

Return value

An item of the *Type* stored in the odb_Sequence object.

Exceptions

None.

64.4.4 isMember(...)

This method checks whether an object is stored in the sequence.

Prototype

```
bool isMember(const Type& item) const;
```

Arguments**Required argument**

item

An object of type *Type*.

Optional arguments

None.

Return value

A Boolean specifying whether the object is stored in the sequence.

Exceptions

None.

64.4.5 operator[](...)

This method accesses an item in an odb_Sequence object.

Prototype

```
const Type& [int index] const;
```

Arguments**Required argument**

index

An Int specifying the index of the item to be accessed from the odb_Sequence object.

Optional arguments

None.

Return value

An item of the *Type* stored in the odb_Sequence object.

Exceptions

None.

64.4.6 size(...)

This method returns the number of items in an odb_Sequence object.

Prototype

```
int size() const;
```

Arguments

None.

Return value

An Int indicating the number of items in the sequence.

Exceptions

None.

64.5 String object

An odb_String object is used to specify a character string. The odb_String object provides a convenient means of storing and passing strings, along with a simple interface to append and modify the data stored in the string.

64.5.1 odb_String(...)

This constructor creates an odb_String object from a char*. Any method that accepts an odb_String argument will also accept a char*.

Prototype

```
odb_String(const char* string)
odb_String()
```

Required argument

A `char*` to initialize the `odb_String`.

Optional arguments

None.

Return value

An `odb_String` object.

Exceptions

None.

64.5.2 **append(...)**

This method adds characters to an `odb_String` object.

Prototype

```
void append(const odb_String& string);
```

Arguments**Required arguments**

string

An `odb_String` to be appended to the contents of `odb_String`.

Optional arguments

None.

Return value

None

Exceptions

None.

64.5.3 cStr(...)

This method returns a `char*` to the string held by an `odb_String` object.

Prototype

```
char* cStr();
```

Arguments

None.

Return value

A `char*` to the string held by the `odb_String` object.

Exceptions

None.

64.5.4 length(...)

This method returns the number of characters of an `odb_String` object.

Prototype

```
int length();
```

Arguments

None.

Return value

An `Int` denoting the number of characters.

Exceptions

None.

64.6 Union object

An `odb_Union` object is used to determine the type of variable used when an object allows multiple variable types. The `odb_Union` object can have a value that is either a `SymbolicConstant`, `Int`, `Float`, `Double`, `Boolean`, or `String`.

64.6.1 `type(...)`

This method returns the current type of the `odb_Union` object.

Prototype

```
odb_UnionType type();
```

Arguments

None.

Return value

The type of the `odb_Union` object. Possible values are `odb_UNION_INT`, `odb_UNION_FLOAT`, `odb_UNION_DOUBLE`, `odb_UNION_BOOL`, and `odb_UNION_STRING`.

Exceptions

None.

`odbException`

Summary of Abaqus Scripting Interface changes between Abaqus 6.14 and Abaqus 2016

This section summarizes the changes and the additions that have been made to the Abaqus Scripting Interface between Abaqus 6.14 and Abaqus 2016. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

To help you determine the status of your scripts, Abaqus displays a dialog box listing the deprecated commands when you execute the script from the main menu using **File→Run Script**. You are advised to upgrade scripts that contain deprecated commands as soon as possible. To run the upgrade utility, type the following command at the system prompt:

```
abaqus python -m upgradeScript [options] scriptName [scriptName ...]
```

scriptName can be either a file or a directory. You can also enter a wildcard (*) rather than a directory to upgrade all scripts within the current working directory. The options for the upgrade utility are:

-fileNames <i>fileName</i>	Read a list of file names from the file specified.
-logfileName <i>logFileName</i>	Specify the name of the log file.
-backup	Whether to make backup versions of the upgraded files. By default, no backup versions are created.
-preview	Preview the changes to the file instead of changing the file.
-diffExecutable <i>application</i>	The application used to display the differences between the script and the upgraded script. By default, the differences are displayed by a web browser.

The upgrade utility upgrades your scripts to the most recent version of Abaqus, regardless of when they were created. The upgrade utility upgrades both kernel commands and GUI commands. If *scriptName* is a directory, the upgrade utility upgrades each script in the directory. For example,

```
abaqus python -m upgradeScript -preview tireSlip.py tireWear
```

would preview all of the upgrades to the **tireSlip.py** script and to all of the scripts in the **tireWear** directory.

More options are available if you use the **upgradeScript** method in an Abaqus Scripting Interface script to upgrade your scripts. For more information, see “Upgrade script commands,” Section 53.10. In addition, an Abaqus plug-in is available that provides a graphical interface to the **upgradeScript** method. For more information, see “Upgrading a script,” Section 82.5 of the Abaqus/CAE User’s Guide.

Alternatively, you can run the following upgrade utilities from the command line interface. The same options are available as for the **upgradeScript** function.

- The **upgradeScriptM_N** utility allows you to upgrade a kernel script from one version to another (where *M* and *N* are general release numbers without any punctuation); for example,

INFRASTRUCTURE COMMANDS

abaqus python -m upgradeScript614_2016 *filenames*

- The **upgradeGuiScript M_N** utility allows you to upgrade a GUI script from one version to another (where M and N are general release numbers without any punctuation); for example,

abaqus python -m upgradeGuiScript614_2016 *filenames*

In the following list of modified commands, objects and methods are listed along with the required and optional arguments. Arguments enclosed within angle brackets, “< >”, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on an entry to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.14 for a description of items that have been removed.

Abaqus/CAE Display Preferences commands

No changes.

Adaptivity commands

No changes.

Amplitude commands

No changes.

Animation commands

No changes.

Annotation commands

Text(<*font*>)

Assembly commands

Assembly.getMassProperties(<*regions*>)

Basic geometry commands

CellArray(cells)

Edge.isTangentFlipped

EdgeArray(edges)

FaceArray(faces)

VertexArray(vertices)

Beam section profile commands

No changes.

Boundary condition commands

No changes.

Calibration commands

Behavior.compute_E(dataSet)

Behavior.compute_elasticModulus(yieldPoint)

Behavior.compute_nu(dataSet)

Behavior.compute_plasticPoints(dataSet, slider_val, start_index, end_index<, yp>)

Behavior.compute_ultimatePoint(dataSet)

Behavior.mapToMaterial(materialName)

Behavior.setValues(<E, nu, ds1Name, ds2Name, materialName, yieldPoint, ultimatePoint, plasticPoints, PoissonsRatio, elasticModulus, plasticPointsRange, name, uniaxialName, biaxialName, interpolation, uniWeight, biWeight, uMullinsReload, uMullinsUnload, uPYieldPoint, uPermSet, uPrimary, bMullinsReload, bMullinsUnload, bPYieldPoint, bPermSet, bPrimary>)

Behavior.xyDataDissect(dsName, modelName, calibrationName<, biaxial>)

DataSet.setValues(<data>)

ElasIsoBehavior(name, typeName)

ElasIsoBehavior.setValues(<E, nu, ds1Name, ds2Name, materialName>)

ElasPlasIsoBehavior(name, typeName)

ElasPlasIsoBehavior.setValues(<yieldPoint, ultimatePoint, plasticPoints, PoissonsRatio, elasticModulus, ds1Name, ds2Name, materialName, plasticPointsRange, name>)

FeFpBehavior.FeFpBehavior(name, typeName)

FeFpBehavior.setValues(<uniaxialName, biaxialName, interpolation, uniWeight, biWeight>)

Canvas commands

No changes.

Connector commands

No changes.

Constraint commands

No changes.

INFRASTRUCTURE COMMANDS

customKernel module

No changes.

Datum commands

No changes.

Display Group commands

LeafFromOdbEdgePick(edgePick)

Display Options commands

ViewportAnnotationOptions.setValues(<*triadFont*>)

Edit Mesh commands

No changes.

Engineering feature commands

No changes.

Feature commands

No changes.

Field commands

OdbMeshRegionData(<transformationType, sectionPoint, refinementType,
refinementLabel, displayOutputPosition>)

Filter commands

No changes.

History and Field Report commands

No changes.

Input File Reader commands

No changes.

Interaction commands

ContactPropertyAssignment.delete(surfPair)

**Model.getSurfaceSeparation(<name, searchDomain, separationTolerance, extendByAngle,
mergeWithinAngle, searchSingleInstances, createUnionOfMasterSurfaces,
createUnionOfSlaveSurfaces, createUnionOfMasterSlaveSurfaces, includePlanar,
includeCylindricalSphericalToric, includeSplineBased, includeMeshSolid,**

includeMeshShell, includeMeshMembrane, includeOverclosed, includeNonOverlapping, meshedGeometrySearchTechnique, useShellThickness, surfaceSmoothing>)

Job commands

OptimizationProcess.extract(<**instances**>)

Queue(<*localPlatform*, **remotePlatform**, *filesToCopy*, **deleteAfterCopy**>)

Load commands

No changes.

Load Case commands

No changes.

Material commands

MagneticPermeability(*table*, *table2*, *table3*)

Mdb commands

No changes.

Mesh commands

MeshEdge.getElementsViaTopology(<**domain**>)

MeshEdge.getNodesViaTopology(<domain>)

MeshFace.getElemFacesByLayer

MeshFace.getNormal

Messaging commands

No changes.

Miscellaneous commands

No changes.

Model commands

Model(<**copyConstraints**, **copyConnectors**, **copyInteractions**>)

Model.setValues(<**copyConstraints**, **copyConnectors**, **copyInteractions**>)

Odb commands

FieldOutput.getConnectorFieldXformedToNodeA(<deformationField>)

FieldOutput.setComponentLabels(componentLabels)

FieldOutput.setData Type(type)

FieldOutput.setValidInvariants(validInvariants)

Odb Display commands

OrientationOptions.setValues(<*symbolSize*>)

Optimization commands

BeadFixedRegion(name, region<, csys, u1, u2, u3>)

BeadFixedRegion.setValues(region<, csys, u1, u2, u3>)

BeadGrowth(name, region<, beadGrowth, shrink>)

BeadGrowth.setValues(region<, beadGrowth, shrink>)

BeadPenetrationCheck(name, beadPenetrationCheckRegion, region)

BeadPenetrationCheck.setValues(beadPenetrationCheckRegion, region)

BeadPlanarSymmetry(name, region<, axis, csys>)

BeadPlanarSymmetry.setValues(region<, axis, csys>)

BeadPointSymmetry(name, region<, csys>)

BeadPointSymmetry.setValues(region<, csys>)

BeadRotationalSymmetry(name, angle, region<, axis, csys>)

BeadRotationalSymmetry.setValues(angle, region<, axis, csys>)

BeadTask(name<, algorithm, areBCRegionsFrozen, beadIter, beadMaxMembraneStress, beadMinStress, beadPerturbation, beadWidth, curveSmooth, filterRadius, filterRadiusBy, flipNormalDir, frozenBoundaryConditionRegion, isSensCalcOnlyOnDesignNodes, modeTrackingRegion, nodalMoveLimit, nodeSmooth, nodeUpdateStrategy, numTrackedModes, updateShapeBasisVectors>)

BeadTask.setValues(<algorithm, areBCRegionsFrozen, beadIter, beadMaxMembraneStress, beadMinStress, beadPerturbation, beadWidth, curveSmooth, filterRadius, filterRadiusBy, flipNormalDir, frozenBoundaryConditionRegion, isSensCalcOnlyOnDesignNodes, modeTrackingRegion, nodalMoveLimit, nodeSmooth, nodeUpdateStrategy, numTrackedModes, updateShapeBasisVectors>)

ShapeTask(<activateDurability, additionalDurabilityFiles, durabilityInputfile, durabilitySolver, updateShapeBasisVectors>)

SizingTask(<*modeTrackingRegion*>)

TopologyTask(<*modeTrackingRegion*>)

Part commands

Feature.WirePolyLine(<*meshable*>)

Partition commands

No changes.

Path and Probe commands

No changes.

Plot Options commands

CouplingConstraint.constraintData

FreeBodyOptions(<*comp1ColorM*, *comp2ColorM*, *comp3ColorM*, *resultantColorM*>)

MpcConstraint.constraintData

OdbAnalysisError

OdbAnalysisWarning

OdbAuxiliaryData

OdbContactDiagnostics

OdbDiagnosticAttempt

OdbDiagnosticData

OdbDiagnosticIncrement

OdbDiagnosticStep.extractData(incrementStatistics)

OdbJobTime

OdbNumericalProblemSummary

RigidBodyConstraint.constraintData

TieConstraint.constraintData

Plug-in commands

No changes.

Predefined Field commands

No changes.

Print commands

No changes.

Property commands

CompositePly(**plyName**, **orientationType**, **thicknessType**<, *orientationType*,
thicknessType, **thicknessField**, *plyName*>)

MaterialOrientation.ReferenceOrientation(<***normalAxisDefinition***, ***primaryAxisDefinition***>)

Region commands

Set(name, objectToCopy)

Surface(name, objectToCopy)

INFRASTRUCTURE COMMANDS

Section commands

No changes.

Session commands

No changes.

Sketcher commands

No changes.

Step commands (step)

No changes.

Step commands (miscellaneous)

No changes.

Step (output) commands

No changes.

Text Representation commands

No changes.

Utility commands

No changes.

View commands

No changes.

XY commands

`XYData.XYDataFromFreeBody(force, moment, resultant, comp1, comp2, comp3<, force, moment, heatFlowRate, resultant, comp1, comp2, comp3>)`

`XYData.XYDataFromPath(labelType<, viewport>)`

Summary of C++ ODB API changes between Abaqus 6.14 and Abaqus 2016

This section summarizes the changes and the additions that have been made to the C++ ODB API between Abaqus 6.14 and Abaqus 2016. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

Objects and methods are listed along with the required and optional arguments. Arguments enclosed within angle brackets, “< >”, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on an item to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.14 for a description of items that have been removed.

Amplitude commands

No changes.

Beam section profile commands

No changes.

Connector commands

No changes.

Filter commands

No changes.

Material commands

MagneticPermeability(*table*, *table2*, *table3*)

Odb commands

FieldOutput.getConnectorFieldXformedToNodeA(<deformationField>)

FieldOutput.setComponentLabels(componentLabels)

FieldOutput.setDataTypes(type)

FieldOutput.setValidInvariants(validInvariants)

INFRASTRUCTURE COMMANDS

Property commands

No changes.

Section commands

No changes.

Infrastructure commands

No changes.

Summary of Abaqus Scripting Interface changes between Abaqus 6.13 and Abaqus 6.14

This section summarizes the changes and the additions that have been made to the Abaqus Scripting Interface between Abaqus 6.13 and Abaqus 6.14. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

To help you determine the status of your scripts, Abaqus displays a dialog box listing the deprecated commands when you execute the script from the main menu using **File→Run Script**. You are advised to upgrade scripts that contain deprecated commands as soon as possible. To run the upgrade utility, type the following command at the system prompt:

```
abaqus python -m upgradeScript [options] scriptName [scriptName ...]
```

scriptName can be either a file or a directory. You can also enter a wildcard (*) rather than a directory to upgrade all scripts within the current working directory. The options for the upgrade utility are:

-fileNames <i>fileName</i>	Read a list of file names from the file specified.
-logfileName <i>logFileName</i>	Specify the name of the log file.
-backup	Whether to make backup versions of the upgraded files. By default, no backup versions are created.
-preview	Preview the changes to the file instead of changing the file.
-diffExecutable <i>application</i>	The application used to display the differences between the script and the upgraded script. By default, the differences are displayed by a web browser.

The upgrade utility upgrades your scripts to the most recent version of Abaqus, regardless of when they were created. The upgrade utility upgrades both kernel commands and GUI commands. If *scriptName* is a directory, the upgrade utility upgrades each script in the directory. For example,

```
abaqus python -m upgradeScript -preview tireSlip.py tireWear
```

would preview all of the upgrades to the **tireSlip.py** script and to all of the scripts in the **tireWear** directory.

More options are available if you use the **upgradeScript** method in an Abaqus Scripting Interface script to upgrade your scripts. For more information, see “Upgrade script commands,” Section 53.10. In addition, an Abaqus plug-in is available that provides a graphical interface to the **upgradeScript** method. For more information, see “Upgrading a script,” Section 82.5 of the Abaqus/CAE User’s Guide.

Alternatively, you can run the following upgrade utilities from the command line interface. The same options are available as for the **upgradeScript** function.

- The **upgradeScript6M_6N** utility allow you to upgrade a kernel script from one version to another (where *M* and *N* are two-digit minor release numbers); for example,

INFRASTRUCTURE COMMANDS

abaqus python -m upgradeScript613_614 filenames

- The **upgradeGuiScript6M_6N** utility allows you to upgrade a GUI script from one version to another; for example,

abaqus python -m upgradeGuiScript613_614 filenames

In the following list of modified commands, the full path to the method is listed along with the required and optional arguments. Arguments enclosed within angle brackets, “< >”, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on a method to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.13 for a description of methods that have been removed.

Abaqus/CAE Display Preferences commands

No changes.

Adaptivity commands

No changes.

Amplitude commands

No changes.

Animation commands

No changes.

Annotation commands

No changes.

Assembly commands

No changes.

Basic geometry commands

No changes.

Beam section profile commands

No changes.

Boundary condition commands

No changes.

Calibration commands

No changes.

Canvas commands

No changes.

Connector commands

No changes.

Constraint commands

No changes.

customKernel module

No changes.

Datum commands

No changes.

Display Group commands

No changes.

Display Options commands

No changes.

Edit Mesh commands

No changes.

Engineering feature commands

No changes.

Feature commands

No changes.

INFRASTRUCTURE COMMANDS

Field commands

`writeFreeBodyReport(fileName, append<, step, frame, stepFrame, odb>)`

Filter commands

No changes.

History and Field Report commands

No changes.

Input File Reader commands

No changes.

Interaction commands

No changes.

Job commands

`OptimizationProcess(name, model, task, prototypeJob<, description, maxDesignCycle,
dataSaveFrequency, saveInitial, saveFirst, saveLast, saveEvery>)`

`Job.writeParAndInputFiles()`

Load commands

No changes.

Load Case commands

No changes.

Material commands

`Shear(table<, temperatureDependency, dependencies>)`

`MaxeDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections,
temperatureDependency, dependencies, alpha, omega, tolerance, direction, position>)`

`MaxpeDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections,
temperatureDependency, dependencies, alpha, omega, tolerance, direction, position>)`

`MaxpsDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections,
temperatureDependency, dependencies, alpha, omega, tolerance, direction, position>)`

`MaxsDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections,
temperatureDependency, dependencies, alpha, omega, tolerance, direction, position>)`

`QuadeDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections,
temperatureDependency, dependencies, alpha, omega, tolerance, direction, position>)`

QuadsDamageInitiation(table<, definition, feq, fnn, fnt, frequency, ks, numberImperfections, temperatureDependency, dependencies, alpha, omega, tolerance, direction, **position**>)

Mdb commands

Mdb.CombineOptResults(optResultLocation<, optIter, nValues, models, steps, analysisFieldVariables, includeResultsFrom, originalModel>)

Mesh commands

Mesh.getElementsByFeatureEdge(**angle**<, *angle*>)

Mesh.getNodesByFeatureEdge(**angle**<, *angle*>)

Mesh.getElemFacesByFaceAngle(**angle**<, *angle*>)

Mesh.getElemEdgesByFaceAngle(**angle**<, *angle*>)

Mesh.getElementsByFaceAngle(**angle**<, *angle*>)

Mesh.getElemFacesByLimitingAngle(**angle**<, *angle*>)

Mesh.getElemFacesByLayer(**numLayer**)

Messaging commands

No changes.

Miscellaneous commands

No changes.

Model commands

No changes.

Odb commands

No changes.

Odb Display commands

No changes.

Optimization commands

SizingClusterAreas(name, regions)

SizingCyclicSymmetry(name, region, translation<, axis, csys, ignoreFrozenArea>)

SizingFrozenArea(name, region)

SizingMemberSize(name, region, minWidth)

SizingPlanarSymmetry(name, region<, axis, csys, ignoreFrozenArea>)

SizingPointSymmetry(name, region<, csys, ignoreFrozenArea>)

SizingRotationalSymmetry(name, angle, region<, axis, csys, ignoreFrozenArea>)

INFRASTRUCTURE COMMANDS

SizingTask(name<, elementthicknessDeltaStopCriteria, freezeBoundaryConditionRegions, freezeLoadRegions, modeTrackingRegion, numFulfilledStopCriteria, numTrackedModes, objectiveFunctionDeltaStopCriteria, stopCriteriaDesignCycle, thicknessMoveLimit, thicknessUpdateStrategy>)

SizingThicknessControl(name, region, thicknessRestrictionBy<, maxThickness, minThickness>)

Part commands

Feature.FaceFromElementFaces(elementFaces<, stitch, stitchTolerance, analyticFitTolerance, **associateFace**>)

Feature.Mirror(mirrorPlane, keepOriginal<, keepInternalBoundaries>)

Feature.WireSpline(points<, **mergeType**, mergeWire, smoothClosedSpline>)

Feature.WirePolyLine(points<, mergeType>)

Partition commands

No changes.

Path and Probe commands

No changes.

Plot Options commands

No changes.

Plug-in commands

No changes.

Predefined Field commands

No changes.

Print commands

No changes.

Property commands

SectionAssignment.getVertices()

Region commands

No changes.

Section commands

SectionAssignment.getVertices()

Session commands

Session.updateCADParameters(modelName, CADName, parameterFile, **CADPartFile**<, CADPartName, *CADPartFile*>)

Sketcher commands

No changes.

Step commands (step)

No changes.

Step commands (miscellaneous)

No changes.

Step (output) commands

No changes.

Text Representation commands

No changes.

Utility commands

No changes.

View commands

No changes.

XY commands

No changes.

Summary of C++ ODB API changes between Abaqus 6.13 and Abaqus 6.14

This section summarizes the changes and the additions that have been made to the C++ ODB API between Abaqus 6.13 and Abaqus 6.14. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

The full path to the method is listed along with the required and optional arguments. Arguments enclosed within angle brackets, “< >”, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on a method to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.13 for a description of methods that have been removed.

Amplitude commands

No changes.

Beam section profile commands

No changes.

Connector commands

No changes.

Filter commands

No changes.

Material commands

No changes.

Odb commands

FieldOutput.addData(position, instance, labels, data<, sectionPoint, localCoordSystem, **conjugateData**>)

**FieldOutput.addData(position, instance, labels, data<, sectionPoint,
localCoordSystem, conjugateData>)**

OdbAssembly.ConnectorOrientation(region, **csys1, axis1, angle1, csys2, axis2, angle2**<,
csys1, axis1, angle1, csys2, axis2, angle2>)

INFRASTRUCTURE COMMANDS

OdbAssembly.ConnectorOrientation(region, csys1, axis1, angle1<, orient2sameAs1>)

Property commands

No changes.

Section commands

CompositeShellSection(name, layup<, symmetric, **layupName**, thicknessType, preIntegrate, poissonDefinition, poisson, integrationRule, temperature, idealization, nTemp, thicknessModulus, useDensity, density, *layupName*, thicknessField, nodalThicknessField>)

HomogeneousShellSection(name, **thickness**, material<, *thickness*, numIntPts, thicknessType, preIntegrate, poissonDefinition, poisson, integrationRule, temperature, idealization, nTemp, thicknessModulus, useDensity, density, thicknessField, nodalThicknessField, >)

Infrastructure commands

Sequence.odt_SequenceType(<initialSize, *growSize*>)

Summary of Abaqus Scripting Interface changes between Abaqus 6.12 and Abaqus 6.13

This section summarizes the changes and the additions that have been made to the Abaqus Scripting Interface between Abaqus 6.12 and Abaqus 6.13. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

To help you determine the status of your scripts, Abaqus displays a dialog box listing the deprecated commands when you execute the script from the main menu using **File→Run Script**. You are advised to upgrade scripts that contain deprecated commands as soon as possible. To run the upgrade utility, type the following command at the system prompt:

```
abaqus python -m upgradeScript [options] scriptName [scriptName ...]
```

scriptName can be either a file or a directory. You can also enter a wildcard (*) rather than a directory to upgrade all scripts within the current working directory. The options for the upgrade utility are:

-fileNames <i>fileName</i>	Read a list of file names from the file specified.
-logfileName <i>logFileName</i>	Specify the name of the log file.
-backup	Whether to make backup versions of the upgraded files. By default, no backup versions are created.
-preview	Preview the changes to the file instead of changing the file.
-diffExecutable <i>application</i>	The application used to display the differences between the script and the upgraded script. By default, the differences are displayed by a web browser.

The upgrade utility upgrades your scripts to the most recent version of Abaqus, regardless of when they were created. The upgrade utility upgrades both kernel commands and GUI commands. If *scriptName* is a directory, the upgrade utility upgrades each script in the directory. For example,

```
abaqus python -m upgradeScript -preview tireSlip.py tireWear
```

would preview all of the upgrades to the **tireSlip.py** script and to all of the scripts in the **tireWear** directory.

More options are available if you use the **upgradeScript** method in an Abaqus Scripting Interface script to upgrade your scripts. For more information, see “Upgrade script commands,” Section 53.10. In addition, an Abaqus plug-in is available that provides a graphical interface to the **upgradeScript** method. For more information, see “Upgrading a script,” Section 82.5 of the Abaqus/CAE User’s Guide.

Alternatively, you can run the following upgrade utilities from the command line interface. The same options are available as for the **upgradeScript** function.

- The **upgradeScript6M_6N** utility allow you to upgrade a kernel script from one version to another (where *M* and *N* are two-digit minor release numbers); for example,

INFRASTRUCTURE COMMANDS

abaqus python -m upgradeScript612_613 filenames

- The **upgradeGuiScript6M_6N** utility allows you to upgrade a GUI script from one version to another; for example,

abaqus python -m upgradeGuiScript612_613 filenames

In the following list of modified commands, the full path to the method is listed along with the required and optional arguments. Arguments enclosed within angle brackets, “<>”, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on a method to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.12 for a description of methods that have been removed.

Abaqus/CAE Display Preferences commands

No changes.

Adaptivity commands

No changes.

Amplitude commands

No changes.

Animation commands

No changes.

Annotation commands

No changes.

Assembly commands

Assembly.copyMeshPattern(<elements, faces, elemFaces, targetFace, nodes, coordinates>)

Assembly.getAngle(plane1, plane2, line1, line2<, commonVertex>)

Assembly.smoothNodes(<name, coordinates>)

ModelInstance.Instance(name, model)

ModelInstance.ConvertConstraints()

ModelInstance.translate(vector)

Basic geometry commands

`CellArray.getMask()`
`Edge.getEdgesByEdgeAngle(angle)`
`EdgeArray.getMask()`
`Face.getFacesByCurvature()`
`FaceArray.getMask()`
`IgnoredEdgeArray.getMask()`
`IgnoredVertexArray.getMask()`
`VertexArray.getMask()`

Beam section profile commands

No changes.

Boundary condition commands

No changes.

Calibration commands

No changes.

Canvas commands

No changes.

Connector commands

No changes.

Constraint commands

No changes.

customKernel module

No changes.

Datum commands

No changes.

Display Group commands

No changes.

Display Options commands

No changes.

INFRASTRUCTURE COMMANDS

Edit Mesh commands

Assembly.generateMesh(*regions*<, **regions**, seedConstraintOverride, meshTechniqueOverride, boundaryPreview, boundaryMeshOverride>)

Part.copyMeshPattern(<elements, faces, elemFaces, targetFace, nodes, coordinates>)

Part.generateMesh(*regions*<, **regions**, seedConstraintOverride, meshTechniqueOverride, boundaryPreview, boundaryMeshOverride>)

Engineering feature commands

EngineeringFeature.DebondVCCT(name, initiationStep, surfToSurfInteraction <, debondingForceAmplitude, printToDATFrequency>)

Feature commands

No changes.

Field commands

No changes.

Filter commands

No changes.

History and Field Report commands

No changes.

Input File Reader commands

No changes.

Interaction commands

ContactProperty.FractureCriterion(initTable<, type, mixedModeBehavior, temperatureDependency, dependencies, tolerance, **specifyUnstableCrackProp, unstableTolerance**>)

Job commands

No changes.

Load commands

No changes.

Load Case commands

No changes.

Material commands

Material.MagneticPermeability(table, **table2**, **table3**<, type, frequencyDependency, temperatureDependency, dependencies, **nonlinearBH**>)

Mdb commands

No changes.

Mesh commands

MeshEdge.getElementsViaTopology()
MeshEdgeArray.getMask()
MeshElement.getElementsByFeatureEdge(<angle>)
MeshElementArray.getMask()
MeshFace.getElemEdgesByFaceAngle(<angle>)
MeshFace.getElemFacesByLayer()
MeshFace.getElemFacesByLimitingAngle(<angle>)
MeshFace.getElementsViaTopology()
MeshFaceArray.getMask()
MeshNode.getNodesByFeatureEdge(<angle>)
MeshNodeArray.getMask()

Messaging commands

No changes.

Miscellaneous commands

No changes.

Model commands

No changes.

Odb commands

No changes.

Odb Display commands

OrientationOptions.setValues(<options, axis1Color, showAxis1, axis2Color, showAxis2, axis3Color, showAxis3, symbolSize, lineThickness, **orientation**, arrowheadStyle, scaleMode>)

Optimization commands

No changes.

INFRASTRUCTURE COMMANDS

Part commands

Model.PartFromGeometryFile(name, geometryFile, dimensionality, type <, bodyNum, combine, retainBoundary, **usePartNameFromFile**, stitchTolerance, twist, scale, convertToAnalytical, convertToPrecise>)

Part.getAngle(plane1, plane2, line1, line2<, **commonVertex**>)

Part.FaceFromElementFaces(elementFaces<, **stitch**, **stitchTolerance**, **analyticFitTolerance**>)

Part.getFeatureCells(name)

Part.getFeatureEdges(name)

Part.getFeatureFaces(name)

Part.getFeatureVertices(name)

Part.seedPart(*regions*, size<, deviationFactor, minSizeFactor, constraint>)

Part.smoothNodes(<nodes, coordinates>)

Partition commands

No changes.

Path and Probe commands

No changes.

Plot Options commands

BasicOptions.setValues(<options, cameraCsysName, cameraMovesWithCsys, cameraFollowsRotation, averagingThreshold, quantityToPlot, curveRefinementLevel, noResultsColor, featureAngle, otherSymbolSize, renderBeamProfiles, beamScaleFactor, renderShellThickness, shellScaleFactor, accountForDeactivatedElems, bcDisplay, connectorDisplay, highlightConnectorPts, showConnectorAxes, showConnectorType, pointElements, referencePoints, massElements, springElements, spotWelds, tracerParticles, **pc3dElements**, **pd3dElements**, sweepArs, sweepElem, sweepStartAngleArs, sweepStartAngleElem, sweepEndAngleArs, sweepEndAngleElem, numSweepSegmentsArs, numSweepSegmentsElem, numericForm, complexAngle, sectionResults, envelopeCriteria, envelopeDataPosition, plyResultLocation, sectionPointScheme, sweepSectors, sectorSelectionType, selectedSectorNumbers, sweepSectorStartAngle, sweepSectorEndAngle, extrudeArs, extrudeArsDepthAutoCompute, extrudeElem, extrudeArsDepth, extrudeElemDepth, mirrorPatternOrder, mirrorCsysName, mirrorAboutXyPlane, mirrorAboutXzPlane, mirrorAboutYzPlane, mirrorDisplayBodies, patternCsysName, patternNumX, patternNumY, patternNumZ, patternOffsetX, patternOffsetY, patternOffsetZ, patternRotationAxis, patternTotalAngle, patternNumCircular, couplingDisplay, coordSystemDisplay, scratchCoordSystemDisplay, transformationType, datumCsys, rigidTransformPrimary, rigidTransformDeformed, transformOnDeformed, averageElementOutput, averageOnlyDisplayed, computeOutput, regionBoundaries, useRegionBoundaries, userRegions, includeFeatureBoundaries>)

Plug-in commands

No changes.

Predefined Field commands

No changes.

Print commands

No changes.

Property commands

No changes.

Region commands

No changes.

Section commands

BeamSection.TransverseShearBeam(**scfDefinition**<, k23, k13, slendernessCompensation>)

Session commands

No changes.

Sketcher commands

No changes.

Step commands (step)

No changes.

Step commands (miscellaneous)

Control.setValues(<allowPropagation, resetDefaultValues, discontinuous, constraints, lineSearch, timeIncrementation, directCyclic, concentrationField, displacementField, electricalPotentialField, globalField, hydrostaticFluidPressureField, poreFluidPressureField, rotationField, temperatureField, **vcctLinearScaling**>)

Step (output) commands

No changes.

Text Representation commands

No changes.

INFRASTRUCTURE COMMANDS

Utility commands

No changes.

View commands

No changes.

XY commands

Session.XYDataFromPath(path, **name**, includeIntersections, shape, pathStyle, numIntervals, labelType
<, *name*, viewport, step, frame, variable, deformedMag, numericForm, complexAngle>)

Summary of C++ ODB API changes between Abaqus 6.12 and Abaqus 6.13

This section summarizes the changes and the additions that have been made to the C++ ODB API between Abaqus 6.12 and Abaqus 6.13. SIMULIA makes every attempt to have the Abaqus Scripting Interface be backward compatible, and Abaqus can execute most Abaqus Scripting Interface scripts from previous versions of Abaqus. However, backward compatibility is not guaranteed beyond two versions of Abaqus, and it is recommended that you upgrade your commands to the most recent version.

The full path to the method is listed along with the required and optional arguments. Arguments enclosed within angle brackets, “<””, are optional arguments. The syntax of the commands is as follows:

(Bold text)	New method or argument.
<i>(Italic text)</i>	Removed method or argument.
<i>(Bold italic text)</i>	Changed argument; for example, the argument type has changed or the possible values of the SymbolicConstant have changed.

You can click on a method to view its description in the command reference. Refer to the Abaqus Scripting Reference Guide for Abaqus 6.12 for a description of methods that have been removed.

Amplitude commands

No changes.

Beam section profile commands

No changes.

Connector commands

No changes.

Filter commands

No changes.

Material commands

Material.MagneticPermeability(table, **table2**, **table3**<, type, frequencyDependency, temperatureDependency, dependencies, **nonlinearBH**>)

Odb commands

No changes.

Property commands

No changes.

INFRASTRUCTURE COMMANDS

Section commands

BeamSection.TransverseShearBeam(**scfDefinition**<, k23, k13, slendernessCompensation>)

Infrastructure commands

No changes.

About SIMULIA

Dassault Systèmes SIMULIA applications, including Abaqus, Isight, Tosca, and Simulation Lifecycle Management, enable users to leverage physics-based simulation and high-performance computing to explore real-world behavior of products, nature, and life. As an integral part of Dassault Systèmes' **3DEXPERIENCE** platform, SIMULIA applications accelerate the process of making highly informed, mission-critical design and engineering decisions before committing to costly and time-consuming physical prototypes. www.3ds.com/simulia

Our **3DEXPERIENCE** Platform powers our brand applications, serving 12 industries, and provides a rich portfolio of industry solution experiences.

Dassault Systèmes, the **3DEXPERIENCE** Company, provides business and people with virtual universes to imagine sustainable innovations. Its world-leading solutions transform the way products are designed, produced, and supported. Dassault Systèmes' collaborative solutions foster social innovation, expanding possibilities for the virtual world to improve the real world. The group brings value to over 170,000 customers of all sizes in all industries in more than 140 countries. For more information, visit www.3ds.com.

